

AI-инфраструктура изнутри:

количественный анализ и
проектирование систем



Боцзе Ли

Русский перевод · community edition
Полная книга

Предисловие

Моя предыдущая книга «Глубокое понимание AI-агента» посвящена архитектуре агентов и практике их разработки. Общаясь с читателями и отвечая на их вопросы, я всё яснее осознавал: чтобы хорошо разрабатывать приложения на основе моделей, необходимо также понимать инфраструктуру, на которой они работают. С одной стороны, это сами модели: как они обрабатывают входные данные, генерируют выходные и взаимодействуют с контекстом и окружением. С другой — системы выполнения моделей: где хранятся параметры и состояние контекста, как выполняются вычисления, как взаимодействуют несколько ускорителей и как вызовы моделей сопрягаются с программными инструментами. Так появилась эта книга, посвящённая инфраструктуре, которая обеспечивает обучение и инференс моделей, — AI Infra.

Чтобы понять, зачем разработчикам приложений нужны такие низкоуровневые знания, рассмотрим знакомую аналогию. Большинству инженеров-программистов не приходится самостоятельно разрабатывать операционные системы, компиляторы и микросхемы, однако им всё равно необходимо изучать операционные системы, принципы построения компиляторов и архитектуру компьютеров. Эти знания помогают понять абстракции, от которых зависят программы, и скрытые за ними реализации. Выделение блока памяти, чтение файла и вызов функции кажутся простыми операциями, но каждая из них требует ресурсов и времени. Только понимая эти издержки, можно объяснить, почему программа работает медленно и как её улучшить. Для разработки приложений на основе моделей необходима такая же база. Размер выбранной модели, длина сохраняемого контекста, число одновременно выполняемых запросов и размещение задачи локально или в облаке — всё это меняет объём работы, которую должна выполнить система.

Повышение уровня абстракции программирования: от операционной системы к контексту модели

Во время обучения в докторантуре по совместной программе Microsoft Research Asia и Научно-технического университета Китая я занимался исследованиями компьютерных систем, поэтому вполне естественно привык рассматривать приложения через распределение задач между операционной системой, компилятором и аппаратным обеспечением. В названиях двух ведущих конференций в этой области — SOSPP (Symposium on Operating Systems Principles) и OSDI (Operating Systems Design and Implementation) — присутствует «OS», то есть операционная система. Традиционная операционная система должна поддерживать самые разнообразные приложения, а компилятор и аппаратное обеспечение — предоставлять универсальные возможности для заранее неизвестных программ. Раньше при оптимизации систем нам всегда приходилось учиты-

вать компромисс между программируемостью и производительностью: жертвовать программируемостью ради предельной производительности зачастую было не лучшим решением.

Но сегодня важнейшим приложением стала LLM, и в системах инференса и обучения AI всё — от выполнения операторов до распределённого планирования — можно оптимизировать под конкретную модель и архитектуру ускорителя, что открывает новые возможности для совместной межуровневой оптимизации. Я считаю, что **перенос абстракции программирования с операционной системы на контекст модели — одно из важнейших изменений в области компьютерных систем за последние десятилетия**. В некотором смысле модель стала операционной системой эпохи LLM, а AI-инфраструктура — компьютерной архитектурой эпохи LLM. Книга «Компьютерная архитектура: количественный подход» стала для меня введением в компьютерную архитектуру, однако в области AI-инфраструктуры пока нет подобной книги, в которой проектирование систем количественно выводилось бы из аппаратных ограничений и архитектуры моделей. Именно это побудило меня написать эту книгу.

От оценки порядка величин к проектированию систем

Раньше, занимаясь ускорением центров обработки данных, независимо от того, шла ли речь об инференсе моделей, обработке сетевого трафика или доступе к хранилищу, я всегда начинал с оценки порядка величин: сколько вычислений требуется для выполнения задачи и какой объём данных нужно прочитать и записать? Помещаются ли данные, которые должны постоянно находиться в памяти, в видеопамять? Достаточна ли пропускная способность хранилища и межсоединений для подачи данных? Какие нижние границы времени выполнения задают соответственно вычисления, чтение и запись, а также этапы, которые необходимо выполнять последовательно? Именно эту привычку отражает практика back-of-the-envelope estimation, за которую выступает Jeff Dean, — грубая оценка с помощью бумаги и ручки. Зачастую всего несколько вычислений помогают понять, стоит ли продолжать развивать выбранное направление и какую проблему следует решать в первую очередь.

Есть и более непосредственная причина, побудившая меня взяться за эту книгу. Общаясь со специалистами по AI-инфраструктуре, я обнаружил, что многие хорошо знакомы с моделями и фреймворками, но ещё не развили интуитивного понимания порядка величин ёмкости, пропускной способности, вычислительной мощности и задержки. Уметь перечислить использованные в проекте технологии и уметь оценить его осуществимость и примерный уровень производительности — две разные компетенции. Я надеюсь, что эта книга поможет читателям приобрести вторую из них.

Ошибки в оценке порядка величин свойственны не только людям, но и AI. Недавно я пробовал поручать таким моделям, как GPT-6 Astra и Claude Fable 5.1, оценку производительности систем инференса или обучения моделей, а также новых архитектур чипов. Эти модели способны привести формулы и получить на вид точные числа, но всё равно упускают фундаментальные ограничения, определяющие результат: иногда учитывают только чтение весов, забывая, что механизму внимания также требуется читать KV-кэш; иногда оценивают время чтения, но не проверяют, помещаются ли в видеопамять в совокупности веса, KV-кэш и рабочая область среды выполнения; иногда вычисляют скорость исходя из пиковой вычислительной мощности, но не проверяют, способна ли

пропускная способность хранилища непрерывно обеспечивать подачу данных; иногда равномерно распределяют объём работы между несколькими ускорителями, но упускают обмен данными между ними; а иногда получают очень высокую пропускную способность, не учитывая задержку, обусловленную последовательными зависимостями и обменом данными туда и обратно. Если упустить хотя бы одно из этих ограничений, вывод может отличаться от действительности в несколько раз или даже на несколько порядков величины.

Поэтому я хочу, чтобы через всю книгу проходил следующий метод: **выводить проектные решения из ограничений**. Сначала необходимо чётко определить задачу и требования к качеству, затем перечислить вычисления, хранение данных, обмен данными и зависимости, проверить, достаточно ли ёмкости видеопамати и других ресурсов хранения, удовлетворяет ли пропускная способность хранилища и межсоединений потребностям вычислений в данных и какие задержки, вызванные зависимостями, невозможно устранить. После этого можно обсуждать, как распределить работу между моделями, где хранить состояние и как организовать выполнение. Только последовательно учтя все известные фундаментальные ограничения, можно исключить заведомо невозможные варианты; накладные расходы и вариативность реальных систем следует и дальше уточнять с помощью измерений. Оценка не обязана быть точной с самого начала, но необходимо понимать, что в неё включено, а что осталось за её пределами.

От ускорения операторов до объединения десяти тысяч ускорителей

Ограничения — источник проектных решений. В 2016 году, проходя стажировку в Microsoft Research Asia, я вместе с командой исследовал ускорение на FPGA глубоких нейронных сетей, применявшихся для ранжирования результатов поиска Microsoft Bing. Веса модели постоянно переносились во внутрикристальную память из внешней, что ограничивало скорость вычислений. Я подумал: нельзя ли распределить модель между несколькими FPGA, чтобы каждая её часть оставалась в локальной внутрикристальной памяти, а через высокоскоростную сеть передавались лишь промежуточные результаты? Я с воодушевлением рассказал об этой идее наставнику Сюй Нини, и он ответил, что это называется «параллелизмом модели». Где выполнять вычисления, как должны перемещаться данные и что позволяет сэкономить специализация — уже тогда я постоянно размышлял над этими вопросами. В те годы мой научный руководитель, доктор Чжан Линьтао, постоянно напоминал: оптимизацию необходимо доводить до предела, допускаемого физикой. Позже в статье о KV-Direct я написал: «близко к физическим пределам базового оборудования». Тогда у меня и сформировалась привычка: сначала на основе первых принципов вычислить верхнюю границу, допускаемую оборудованием, а затем определить, насколько система от неё далека. Эта привычка проходит через всю мою последующую работу и через всю эту книгу.

Присоединившись к Huawei в 2019 году, я участвовал в проекте AKG по автоматической генерации операторов для фреймворка глубокого обучения MindSpore. Эта работа продолжала прежний подход совместного проектирования программного и аппаратного обеспечения: вычисления модели преобразовывались в программы, подходящие для выполнения на NPU Ascend, а компилятор определял разбиение данных на блоки, их раз-

мещение и порядок выполнения. Я до сих пор помню, как повсюду искал подходящий алгоритм для слияния оператора softmax. Прочитав опубликованную NVIDIA в 2018 году работу о потоковом softmax, я наконец нашёл способ реализовать такое слияние. Лишь позднее я осознал, что тогда фактически нащупал одну из ключевых идей будущего алгоритма внимания FlashAttention: сочетать потоковую редукцию с разбиением на блоки и слиянием, сокращая объём хранения, чтения и записи промежуточных результатов.

В 2020 году я присоединился к проекту Unified Bus (UB, унифицированное соединение) и начал исследовать совместную работу ускорителей в ещё большем масштабе: как обеспечить эффективное совместное выполнение одной вычислительной задачи десятками тысяч процессоров?

Когда мы только начинали продвигать эту идею внутри Huawei, то столкнулись с многочисленными сомнениями со стороны других подразделений. Тогда кто-то спросил: «Сейчас модели обучают максимум на восьми картах. Ты создаёшь систему на десяти тысяч карт — но когда вообще появятся десять тысяч карт?» В то время во всей компании не было и десяти тысяч карт. Для людей, привыкших обучать модели на нескольких картах в пределах одной машины, проектирование соединения для десяти тысяч карт действительно казалось далёким от насущных потребностей.

Мы пришли к такому выводу, поскольку увидели другую совокупность свидетельств. В 2019 году доктор Тань Кунь представил тенденцию развития потребностей AI в вычислительных ресурсах в виде диаграммы рассеяния и заметил, что потребности растут гораздо быстрее, чем производительность одного чипа. Это побудило его продвигать исследования высокопроизводительных соединений для десятков тысяч карт. В 2020 году я присоединился к проекту, а публикация статьи о GPT-3 предоставила новые аргументы в пользу этого направления. Если объём вычислений, необходимый для обучения моделей, продолжит расти такими темпами, придётся организовать совместную работу большего числа ускорителей. Исследование и реализация архитектуры соединения требуют времени, поэтому начинать лишь после того, как потребность проявится в полной мере, скорее всего, будет уже слишком поздно.

Сегодня UB применяется в системах Ascend 910C и 950. На момент написания этой книги архитектура кластера для обучения AI на NPU, основанная на UB, является единственной в стране архитектурой, поддерживающей масштаб свыше десяти тысяч карт. Теперь, оглядываясь назад, необходимость обучения на десяти тысячах карт уже нетрудно понять. При проектировании системы необходимо одновременно ясно видеть как текущую нагрузку, так и изменяющиеся условия. То, что восьми карт было достаточно для выполнения привычных тогда задач, — практический опыт; то, что рост потребностей в вычислениях приведёт к более масштабной совместной работе, — прогноз на будущее. Чтобы убедить других, необходимо чётко объяснить тенденции, ресурсы и издержки, на которых основан этот прогноз.

От ускорения инференса моделей через AKG к UB — эти работы охватывали разные масштабы, но я вновь и вновь сталкивался с одними и теми же вопросами: где находятся данные, необходимые для вычислений, и не превосходят ли затраты на их перемещение стоимость собственно необходимых вычислений? Можно ли снизить затраты на перемещение данных, изменив абстракции системы и приложения так, чтобы система располагала дополнительной информацией от приложения? Все эти работы начинались с изменения некоторого соотношения: скорость чтения из внешней памяти перестала удовлетворять вычислительным потребностям, что побудило нас ускорить ранжирование Bing с помощью FPGA; потребности в вычислениях начали расти быстрее произво-

дительности одного чипа, что побудило нас продвигать соединение десяти тысяч карт.

После ухода из Huawei и основания собственной компании в 2023 году эти вопросы вновь предстали передо мной, но уже в другой форме. Разрабатываемому нами AI-агенту требовалось голосовое взаимодействие с человеком в реальном времени. Сколько придётся ждать пользователю после окончания фразы, прежде чем он услышит ответ? И сколько будет стоить обслуживание получасового телефонного разговора?

В 2024 году, ещё до выпуска GPT-4o, мы запустили голосовое взаимодействие в реальном времени. Позднее, сравнив его с API голосового взаимодействия в реальном времени на базе GPT-4o, мы обнаружили, что наши эксплуатационные расходы составляют примерно одну сотую от стоимости использования этого API. Я до сих пор помню, что в конце 2023 года задержка в первой демонстрации голосового телефонного разговора достигала пяти секунд. Мы разделили весь конвейер взаимодействия на части, сначала оценили, сколько вычислений требуется для одного инференса модели и сколько данных нужно прочитать, а затем начали последовательно всё улучшать; после этого продолжили оптимизировать передачу данных по сети, обращения к базе данных и другие операции. Так задержка сократилась с пяти секунд до 2,5 секунды, затем до одной секунды, а позднее — примерно до 500–600 миллисекунд.

Этот опыт помог мне осознать, насколько непосредственно разработка приложений связана с инфраструктурой. Даже для одного и того же голосового взаимодействия в реальном времени многократное различие в задержке способно полностью изменить впечатление от продукта, а различие в стоимости на порядок — изменить набор поддерживаемых бизнес-моделей.

Что я хочу объяснить в этой книге

Такая связь между приложениями и инфраструктурой существует и внутри процесса разработки моделей. В сильных командах, создающих фундаментальные модели, я всё чаще наблюдаю подход, выходящий за рамки традиционного разделения труда: **тот, кто лучше всех понимает Infra, занимается алгоритмами; тот, кто лучше всех понимает алгоритмы, занимается данными; тот, кто лучше всех понимает данные, занимается Infra.** Эта фраза отражает глубину взаимопонимания. Разработчик алгоритмов должен знать ограничения оборудования по ёмкости, пропускной способности и обмену данными, чтобы учитывать их в архитектуре модели; специалист по данным должен понимать, как обучается модель, чтобы определять наиболее ценные примеры и задачи обучения; инженер Infra должен знать, как организованы и используются данные и какова их длина, чтобы действительно направлять системные ресурсы на эффективное обучение и инференс.

В последние годы такое межуровневое взаимодействие уже проявляется в проектировании моделей. Хорошие примеры — DeepSeek V4 и V4.1. В архитектуре V4 изначально в полной мере учитывается эффективность Infra: сочетание локального окна, сжатия контекста и разреженного выбора уменьшает объём состояния, которое необходимо хранить для длинного контекста, и количество повторно считываемых данных. Ограничения, связанные с объёмом видеопамати, пропускной способностью хранилища и вычислениями, напрямую влияют на то, как модель представляет и использует контекст.

V4.1 Flash идёт ещё дальше: асимметричная архитектура каузального кодировщика-декодировщика (Causal Encoder-Decoder, CED) по-новому распределяет задачи понима-

ния и генерации, перераспределяя вычислительные ресурсы с учётом нагрузки, при которой AI-агент получает много входных данных, но генерирует сравнительно мало выходных. Это показывает, что требования к эффективности системы способны стимулировать инновации в самой архитектуре модели.

Я хочу, чтобы читатели также сформировали такое межуровневое понимание: видя архитектуру модели, могли представить, какую работу она требует от ускорителя; видя возможности оборудования, могли определить, как изменить модель и способ её выполнения, чтобы действительно воспользоваться этими возможностями.

Сквозная тема этого анализа — **перемещение данных**. Вычислительным блокам необходимо получать веса и промежуточные результаты, нескольким ускорителям — обмениваться вычисленными данными, а удалённым сервисам — принимать входные данные и возвращать результаты. Мы видим, как модель генерирует ответ, но на нижнем уровне раз за разом происходят чтение, вычисление, сохранение и передача. На эффективность выполнения влияют место хранения данных, количество повторных использований, интерфейсы, через которые они должны пройти, и необходимость ожидания со стороны последующих операций.

Поэтому в этой книге постоянно задаются пять вопросов: **что перемещать, сколько перемещать, сколько раз перемещать, через что перемещать и кто должен этого ждать**. От иерархии памяти на кристалле через суперузлы — группы ускорителей, тесно взаимодействующих посредством высокоскоростных соединений, — и сети центров обработки данных до распределения задач между оконечными устройствами, периферией и облаком эти пять вопросов помогают определить величины, которые необходимо рассчитать. Сохранение данных позволяет сократить повторные вычисления, но занимает доступную ёмкость; увеличение масштаба параллелизма позволяет распределить работу, но усиливает обмен данными; сокращение передачи может потребовать дополнительных локальных вычислений, а также способно изменить качество результата.

Чтобы ответить на эти пять вопросов, необходимо рассчитать каждую величину — именно поэтому подзаголовок этой книги звучит как «Количественный анализ и проектирование систем». Книга «Архитектура компьютера: количественный подход» служит прекрасным примером такого метода анализа. Настоящая книга также призвана применять упомянутую выше привычку к оценке на каждом уровне проектирования: исходя из рабочей нагрузки модели и сопоставляя её с ограничениями ресурсов, объяснять выбор способа выполнения и то, как его следует пересмотреть при изменении условий.

Структура книги

Двенадцать глав выстроены в последовательности «понять требования рабочей нагрузки — изучить ресурсы выполнения — организовать целостную систему», как показано на рис. 0-1. Первая часть посвящена моделям и рабочим нагрузкам (главы 1–3): в ней формируется методика анализа и объясняется происхождение объёма вычислений, объёма данных и зависимостей между задачами. Вторая часть посвящена чипам и системам (главы 4–7): от выполнения на одном ускорителе мы переходим к совместной работе нескольких ускорителей и рассматриваем, как ресурсы справляются с этой нагрузкой. Третья часть посвящена системам инференса и обучения (главы 8–12): в ней исследуется организация запросов, состояния модели и процесса обучения, а также рассматриваются среда выполнения задач и развёртывание на конечных устройствах, периферии и в

облаке.

Ниже перечислены основные вопросы, на которые отвечает каждая глава. Сначала вы можете найти здесь интересующий вас вопрос, а затем посмотреть, какие основы для его понимания заложены в предыдущих главах.

Часть I. Модели и рабочие нагрузки

- **Глава 1. Первое знакомство с AI Infra:** как увидеть систему целиком и с помощью нескольких ключевых чисел оценить один запуск модели?
- **Глава 2. Архитектура модели:** как вычисления, параметры и состояние контекста модели формируют требования к ресурсам?
- **Глава 3. Нагрузки инференса и обучения:** как поступление запросов, многоаундовые вызовы, мультимодальный ввод и процесс обучения изменяют требования к ресурсам и время ожидания?

Часть II. Чипы и системы

- **Глава 4. Архитектура ускорителей:** как взаимодействуют вычислительные блоки, память и тракты передачи данных чипа и для каких нагрузок они подходят?
- **Глава 5. Операторы и среда выполнения:** как организовать операторы и процесс выполнения, чтобы сократить повторные операции чтения и записи, отправку заданий и ожидание?
- **Глава 6. Суперузлы:** как распределить работу модели между несколькими ускорителями и каким должен быть масштаб их совместной работы?
- **Глава 7. Сеть центра обработки данных:** как данные передаются между ускорителями и как правила передачи работы, перегрузка и сбои влияют на вычисления?

Часть III. Системы инференса и обучения

- **Глава 8. Оптимизация инференса:** как организовать пакетную обработку, запросы и кэширование, чтобы повысить эффективность обслуживания при заданной конфигурации ускорителей?
- **Глава 9. Распределённый инференс:** где следует размещать вычисления и состояние и как организовать распределение работы, совместное использование ресурсов, увеличение и уменьшение масштаба?
- **Глава 10. Системы обучения:** как организовать состояние обучения, коммуникацию и восстановление, чтобы добиться результативного прогресса обучения в установленный срок?
- **Глава 11. Планирование ресурсов и среда выполнения:** как объединить сервисы моделей, среды инструментов и общие ресурсы в целостную систему выполнения задач?
- **Глава 12. Взаимодействие конечных, периферийных и облачных устройств:** как с учётом фактических требований к передаче данных и взаимодействию выбрать, где развёртывать задачу — на конечном устройстве, периферии или в облаке?

По мере чтения в анализ одного и того же вопроса будут постепенно добавляться новые условия: сначала учитывается только объём весов, а затем потребуется вычислять и состояние контекста; сначала сравнивается однократное выполнение, а затем придётся учитывать конкурентное выполнение, передачу работы и восстановление после сбоев. Я надеюсь, что вы будете следить за этими изменениями и возвращаться к своим прежним выводам, чтобы проверять их заново. Именно поэтому модели, чипы, сети и системы обслуживания рассматриваются в одной книге: все они совместно определяют, как будет выполнена задача.

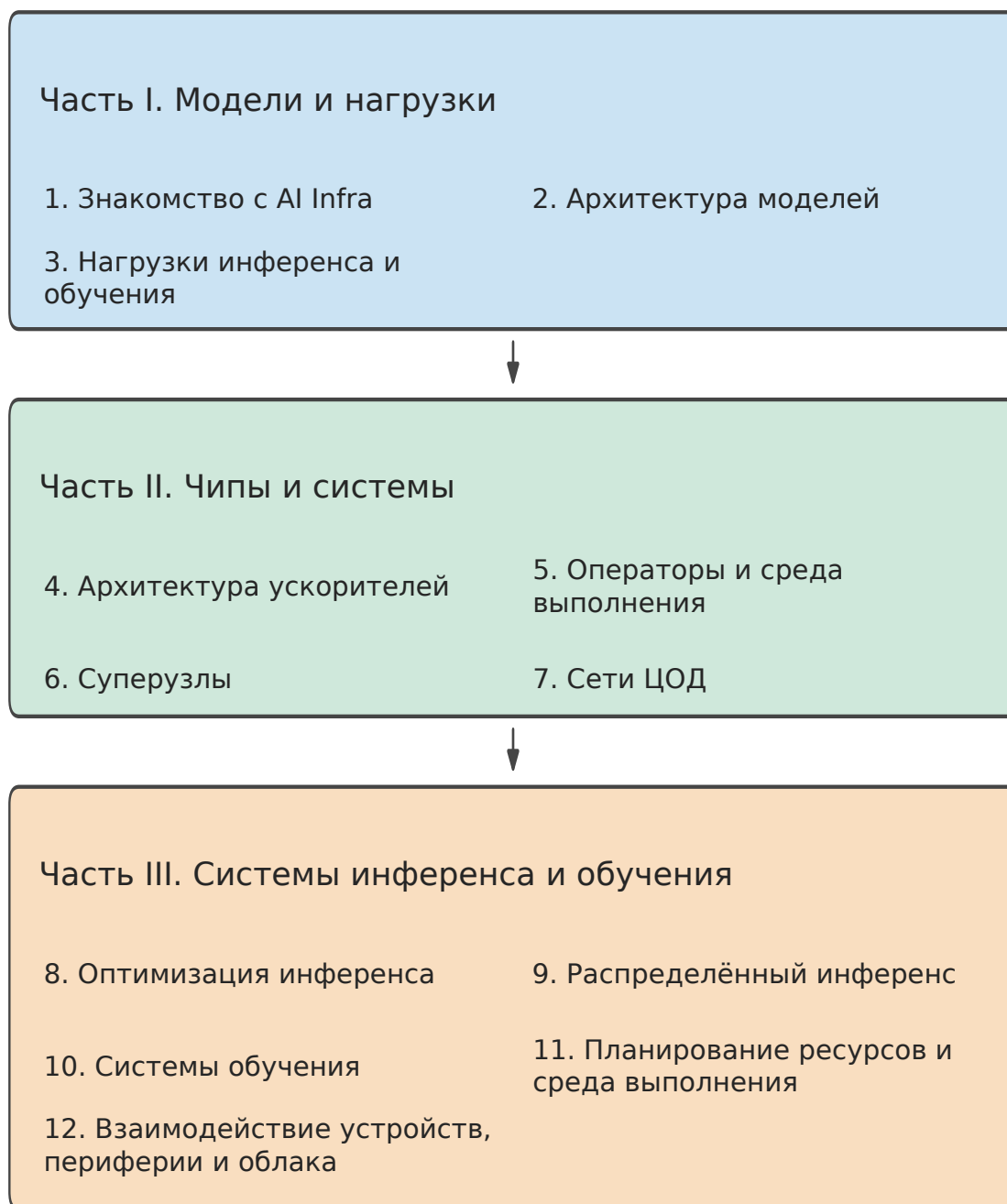


Рис. 0-1. Структура книги и последовательность чтения. Каждая предыдущая часть закладывает аналитическую основу для следующей; при переходе к обслуживанию и развёртыванию необходимо заново проверить выбор модели и ресурсов с учётом качества выполнения задачи, времени её завершения и стоимости.

Как читать эту книгу

Если вы хотите с нуля сформировать системное понимание, я рекомендую сначала прочитать главы 1–3: научиться оценивать одно выполнение и разобраться, какие требования предъявляют модель и нагрузка, а затем последовательно перейти к двум следующим частям. Опираясь на эту общую основу, читатели с разным опытом могут выбрать собственные приоритеты.

- **Разработчикам моделей и приложений** стоит уделить особое внимание сервисам инференса в главах 8 и 9, а также средам выполнения задач и развёртыванию в главах 11 и 12. Столкнувшись с проблемами ёмкости, вычислительных операторов или передачи данных, вернитесь к главам 4–7, чтобы проследить их причины.
- **Системным и сетевым инженерам** стоит сосредоточиться на выполнении и взаимодействии с ускорителями в главах 5–7, а затем перейти к главам 9 и 10, чтобы проверить, как эти механизмы влияют на распределённый инференс и обучение.
- **Инженерам по микросхемам и архитектуре вычислительных систем** стоит уделить особое внимание главам 4–7 и сопоставить их с примерами задач из последующих глав, чтобы проверить, как знакомые им аппаратные показатели преобразуются в реальную производительность сервиса.

Какой бы путь чтения вы ни выбрали, я особенно рекомендую сначала сделать собственную оценку и лишь затем смотреть ответ. Возьмите лист бумаги, запишите объём данных, производительность обработки и этапы, завершения которых необходимо дожидаться, — обычно этого достаточно, чтобы обнаружить вопросы, требующие дальнейшего изучения. Если ваш результат отличается от приведённого в книге, сначала проверьте, использовались ли в обоих случаях одинаковые условия. Если условия совпадают, продолжайте искать неучтённую или посчитанную дважды работу. Такая последовательная перепроверка зачастую приносит больше пользы, чем простое запоминание готового вывода.

Упражнения в книге делятся на основные и дополнительные. Основные упражнения помогут вам выполнить ключевые для главы выкладки, а дополнительные можно выбирать в зависимости от интересов и потребностей проекта. Сопутствующие эксперименты, инструменты расчёта и справочные материалы собраны в открытом репозитории книги:

Сопутствующий открытый репозиторий: <https://github.com/bojieli/ai-infra-book>

Репозиторий содержит материалы трёх типов, которые рекомендуется использовать вместе с основным текстом:

- **Эксперименты и расчёты.** Для экспериментов, организованных по главам, приведены инструкции по запуску, входные условия и зафиксированные результаты. Сопутствующие инструменты расчёта помогут вам перепроверить числа из книги или задать другой набор условий и посмотреть, как изменятся выводы.
- **Научные статьи и технические материалы.** Указатели и сведения об источниках для связанных научных статей, открытого программного обеспечения и официальной технической документации позволяют продолжить изучение рассмотренных в книге вопросов и проверить конкретные аргументы.
- **Таблицы параметров микросхем и моделей.** В них собраны аппаратные характеристики и конфигурации моделей, систематизированные в ходе исследования: вычислительная мощность, объём и пропускная способность памяти, возможности межсоединений, а также число слоёв и размерности моделей, конфигурации экс-

пертов в моделях типа *mixture-of-experts*, состояние контекста и другие сведения, необходимые для оценок. Используйте таблицы параметров вместе с источниками и инструментами расчёта: сначала проверьте условия применимости чисел, а затем подставьте их в собственный анализ.

С главной страницы репозитория можно перейти в каталоги экспериментов, количественных расчётов и справочных материалов и запустить каждый проект по соответствующей инструкции. Для экспериментов, требующих определённых ускорителей, указаны требования к окружению и ускорителям. Если нужного ускорителя нет, можно сначала проанализировать уже имеющиеся записи, а затем изменить входные данные и повторить расчёты. Модели и аппаратное обеспечение постоянно обновляются, поэтому эти материалы также будут дополняться и исправляться по мере работы над рукописью.

Читая эту книгу, вы также можете рассматривать собственные модели, машины и бизнес-задачи. Замените исходные данные из книги своими, посмотрите, изменятся ли выводы, и затем решите, что стоит измерить на следующем этапе. Надеюсь, эти примеры станут отправной точкой для анализа ваших собственных задач.

Предварительные знания

Эта книга предназначена для читателей с опытом программирования, которые хотят разобраться в выполнении моделей и проектировании систем. Перечисленные ниже базовые знания помогут вам следить за рассуждениями и экспериментами в книге.

- **Программирование и инструменты.** Умение читать и изменять простые программы на Python, работать с командной строкой и устанавливать основные зависимости. Базовые упражнения начинаются с ручных расчётов, небольших скриптов и готовых записей экспериментов; условия для экспериментов с конкретными ускорителями будут оговариваться отдельно.
- **Математика.** Понимание векторов, умножения матриц и основ алгебры, умение переводить единицы измерения. Базовые знания о средних величинах и вероятностях помогут анализировать нагрузку и ожидание; интуитивное понимание производных и градиентов упростит чтение глав об обучении.
- **Компьютерные системы.** Понимание основного назначения процессов, памяти, файлов и сетевого взаимодействия. Устройство моделей, способы распараллеливания и механизмы специализированного оборудования будут постепенно объясняться по мере рассмотрения соответствующих задач.

Если вы уже обращались к API модели или запускали модель на своём компьютере, этот опыт можно применить к примерам расчётов из книги: почему длинный контекст замедляет работу, почему при повышении уровня конкурентности каждому приходится ждать дольше, почему замена ускорителя не дала ожидаемого ускорения? Всё это — отличные отправные точки для чтения.

Необязательно ждать, пока вы освоите все уровни, чтобы начать читать. В ходе системных исследований и инженерной практики мне постоянно приходится отталкиваться от конкретной задачи и осваивать знания на смежных уровнях. Столкнувшись с незнакомым понятием, можно сначала понять, какую проблему оно решает, а затем вернуться к деталям реализации.

Я надеюсь, что после прочтения книги, столкнувшись с новой моделью, новым ускорителем или новым требованием к развёртыванию, вы сможете изобразить процесс вы-

полнения, рассчитать несколько ключевых показателей, указать гипотезы, которые важнее всего проверить, и скорректировать свой выбор на основе наблюдений. Ваши выводы могут отличаться от изложенных в книге; способность объяснить причину этих различий сама по себе свидетельствует о понимании системы.

Благодарности

Прежде всего я хочу поблагодарить модель GPT-6 Astra. Я давно хотел написать такую книгу, систематизировав накопившиеся за эти годы размышления о компьютерных системах и AI Infra, но мне всё не удавалось найти достаточно времени. Недавно я наконец свёл свои мысли в план. После выхода GPT-6 Astra я поручил AI-агенту непрерывно работать в течение недели: помогать мне изучать научные статьи о моделях, систематизировать сведения о развитии программного обеспечения с открытым исходным кодом, проводить эксперименты, а затем постепенно превращать всё это в книгу. Версия, которую вы сейчас видите, пока лишь черновик. Я продолжаю осмысливать свои идеи и редактировать содержание книги.

Я благодарен всем соавторам, вместе с которыми в последние годы занимался исследованиями и публиковал научные работы в областях сетей, систем и AI Infra. Я также благодарен руководителям, экспертам и коллегам, которые наставляли меня и работали со мной бок о бок над такими проектами, как ускорение с помощью программируемых сетевых адаптеров, генерация операторов и крупномасштабное сетевое взаимодействие Unified Bus, когда я работал в Microsoft и Huawei. Здесь невозможно перечислить всех поимённо, но я очень многому у них научился: от выдвижения гипотез, теоретического анализа и экспериментальной проверки до создания полноценных систем промышленного уровня. Многие идеи этой книги были бы невозможны без тех дней, когда мы вместе вели обсуждения, проводили выкладки и эксперименты и решали проблемы.

Наконец, я хочу поблагодарить свою жену Мэн Цзяин. Как и во время работы над книгой «Глубокое понимание AI-агента», она неизменно поддерживала моё стремление довести начатое до конца. В последние дни она также уступила мне свою квоту токенов Codex, чтобы у меня было достаточно токенов для продолжения работы над этой книгой.

Содержание

Предисловие	i
Повышение уровня абстракции программирования: от операционной системы к контексту модели	i
От оценки порядка величин к проектированию систем	ii
От ускорения операторов до объединения десяти тысяч ускорителей	iii
Что я хочу объяснить в этой книге	v
Структура книги	vi
Как читать эту книгу	ix
Предварительные знания	x
Благодарности	xi
1 Первое знакомство с AI Infra	1
1.1 Почему важна инфраструктура ИИ	1
1.2 Ключевые показатели проектирования систем	13
1.3 Оценка одного выполнения модели с помощью нескольких чисел	20
1.4 Как требования определяют проектирование архитектуры	30
1.5 Заблуждения и ловушки	36
1.6 Краткий справочник чисел для этой главы	36
1.7 Итоги главы	38
2 Архитектура модели	39
2.1 Вычислительный граф и прямой проход	40
2.2 Расчёт потребления ресурсов при авторегрессионной генерации	52
2.3 Представление состояния контекста и механизм доступа	58
2.4 Условные вычисления и повторное использование весов экспертов	80
2.5 Форма сети и организация вычислений	85
2.6 Требования к ресурсам всей модели и сравнение	90
2.7 Заблуждения и ловушки	114
2.8 Упражнения и воспроизведение результатов главы	114
2.9 Справочные таблицы матриц моделей	115
2.10 Итоги главы	125
3 Нагрузки инференса и обучения	127
3.1 От единичного запроса к непрерывно поступающей нагрузке	127
3.2 Многораундовое взаимодействие и задачи AI-агента	138
3.3 Мультимодальность и взаимодействие в реальном времени	149
3.4 Требования обучения к вычислениям и состоянию	158

3.5	Обучение с подкреплением (RL)	166
3.6	От требований нагрузки к бюджету обучения и обслуживания	172
3.7	Заблуждения и ловушки	183
3.8	Траектория выполнения задачи с кодом	183
3.9	Итоги главы	183
4	Архитектура ускорителей	185
4.1	Знакомство с ускорителем через вычисления модели	185
4.2	Вычислительные блоки	204
4.3	Иерархия памяти	214
4.4	Перемещение данных	222
4.5	Многочиповая компоновка и межсоединения	235
4.6	Как требования моделей стимулируют эволюцию архитектур	242
4.7	Специализированные архитектуры	257
4.8	Анализ производительности и выбор ускорителя на основе архитектуры	261
4.9	Итоги главы	276
5	Операторы и среда выполнения	277
5.1	Отправка, перемещение и синхронизация при выполнении на ускорителе	278
5.2	Разбиение отдельного оператора на блоки, размещение данных и фактический доступ к памяти	291
5.3	Слияние соседних операторов, буферизация и конвейерное выполнение	305
5.4	Компилятор: представление, преобразование и выбор	318
5.5	Среда выполнения: отправка, повторное воспроизведение и динамические формы	332
5.6	От локальной оптимизации к полному запросу	347
5.7	Распространённые заблуждения	354
5.8	Упражнения и сопутствующие эксперименты	354
5.9	История и материалы для дальнейшего чтения	357
5.10	Итоги главы	357
6	Суперузлы	359
6.1	От одной карты к нескольким	360
6.2	Шесть способов распараллеливания	369
6.3	Комбинирование видов параллелизма и масштаб модели	388
6.4	Реализация коллективных коммуникаций и стоимость их выполнения .	397
6.5	Физическая организация суперузлов	410
6.6	Пул памяти	433
6.7	От параллельных решений к масштабу суперузлов	439
6.8	Итоги главы	457
7	Сети центров обработки данных	459
7.1	От суперузла к кластеру	460
7.2	Межузловой трафик и физические маршруты	471
7.3	Пути удалённого доступа и параллелизм	493
7.4	Передача данных, порядок и управление состоянием	513
7.5	Перегрузка и надёжность в общей сети	527
7.6	От совершенствования коммуникации к выполнению задачи	545

7.7	Итоги главы	565
8	Оптимизация инференса	567
8.1	Процесс выполнения запросов на инференс и требования к ресурсам . .	568
8.2	Пакетная обработка и планирование запросов	573
8.3	Распределение, повторное использование и освобождение KV-кэша . . .	584
8.4	Сжатие и выгрузка	603
8.5	Спекулятивное декодирование	613
8.6	Производительность и выбор конфигурации при потоке запросов	621
8.7	История и дальнейшее чтение	631
8.8	Итоги главы	631
9	Распределённый инференс	633
9.1	Способы распределения вычислений и состояния	634
9.2	Разделение Prefill–Decode и распределение ресурсов	640
9.3	Разделение механизма внимания и FFN и гетерогенное выполнение . . .	652
9.4	Размещение и репликация экспертов, балансировка нагрузки	664
9.5	Распределение и совместное использование KV, маршрутизация запросов	680
9.6	Запуск сервиса, масштабирование и восстановление после сбоев	698
9.7	Комплексное сравнение схем развёртывания	705
9.8	Упражнения	712
9.9	Итоги главы	715
10	Системы обучения	716
10.1	Оценка требований к ресурсам и времени на основе задачи обучения . .	717
10.2	Шардирование, повторное вычисление и выгрузка состояния обучения .	728
10.3	Конвейерное планирование шага обучения и перекрытие коммуникаций	744
10.4	Ввод данных, checkpoint и восстановление после сбоев	763
10.5	Обучение с подкреплением	780
10.6	От системного решения к сроку завершения и выбору оборудования . . .	793
10.7	Распространённые заблуждения	800
10.8	Упражнения и эксперименты	801
10.9	Итоги главы	804
11	Планирование ресурсов и среда выполнения	806
11.1	Расчёт потребности в вычислительных ресурсах на основе процесса вы- полнения задачи	808
11.2	Создание, сохранение и освобождение среды выполнения	814
11.3	Распределение и планирование общего пула ресурсов	834
11.4	Выбор сервиса модели и стоимость вызовов	852
11.5	Восстановление полной задачи, расчёт затрат и масштабирование	862
11.6	Упражнения и эксперименты	878
11.7	Итоги главы	881
12	Совместная работа устройств, периферии и облака	883
12.1	От времени работы модели до полного времени взаимодействия	884
12.2	Распределение работы между устройствами	900

12.3	Дополнительные накладные расходы и ожидание при передаче по гло- бальной сети	911
12.4	Взаимодействие и восстановление в условиях изменчивой беспроводной связи	927
12.5	От распределения задач между устройством, периферией и облаком к полноценному развёртыванию	933
12.6	Упражнения	946
12.7	Итоги главы	951

Глава 1

Первое знакомство с AI Infra

В предисловии рассказано, что побудило написать эту книгу, как организованы её главы и как с ней работать. Начиная с этой главы мы проследим процесс выполнения модели и познакомимся с системами, которые его обеспечивают. **AI Infra** — это инфраструктура, обеспечивающая обучение и инференс AI, включая вычислительные устройства, хранилища, межсоединения и программное обеспечение, которое организует работу этих ресурсов. В ходе обучения параметры модели корректируются на основе примеров; при инференсе полученные параметры используются для обработки входных данных и формирования результата.

Сначала в этой главе объясняется, как приложение выражает задачу посредством кода, модели и контекста. Затем мы проследим обработку одного запроса и познакомимся с системой, используя ёмкость, вычислительную мощность и пропускную способность для ответа на конкретные вопросы проектирования: поместятся ли данные, сколько придётся ждать ответа, какой выигрыш даст добавление ускорителей или изменение представления данных. В завершение мы рассмотрим процесс формирования нескольких реальных архитектур и покажем, как требования определяют проектирование системы.

1.1 Почему важна инфраструктура ИИ

1.1.1 Повышение уровня абстракции программирования: от операционной системы к контексту модели

Рассмотрим сервис ответов на вопросы по документам. Сначала разработчик задаёт программный процесс чтения документов, поиска и отображения результатов, а затем передаёт языковой модели вопрос пользователя и найденные материалы для генерации ответа. **Контекст** (context) — это входные данные, доступные при текущем вызове модели, включая инструкции к задаче, справочные материалы и результаты предыдущих взаимодействий. Не изменяя параметры модели, но меняя инструкции, примеры или справочные материалы, можно изменить решаемую моделью задачу и способ формирования ответа.

Если поручить такому сервису проверку кода, модели можно предоставить код, требования к тестированию и доступные инструменты, чтобы она предложила изменения или инициировала вызов инструмента; приложение выполнит тесты, добавит результаты в контекст следующего вызова, после чего модель продолжит анализ. Инструменты — это программные функции, доступные приложению, например чтение файлов или запуск тестов. Модель выбирает, что делать дальше, а хост-программа проверяет раз-

решения, выполняет операции и сохраняет результаты. Для выполнения одной задачи может потребоваться несколько вызовов модели и запусков инструментов.

Абстракция скрывает сложную реализацию за чётко определённым интерфейсом, позволяя разработчику организовать программу с помощью меньшего числа понятий. **Программируемость** — это способность разработчика изменять поведение системы через определённый интерфейс. В традиционных приложениях поведение в основном выражается программным кодом, компилятор преобразует код в исполняемые инструкции, а операционная система управляет ресурсами, необходимыми для работы программы. В приложениях на основе моделей появляется ещё одна важная точка управления: задача выражается через выбор модели, организацию контекста и предоставление инструментов. Часть поведения, для реализации которого раньше требовалось последовательно описывать правила, теперь определяется моделью на основе контекста; код по-прежнему организует процесс вызовов, а программы-инструменты по-прежнему обращаются к файлам, сети и устройствам через операционную систему.¹

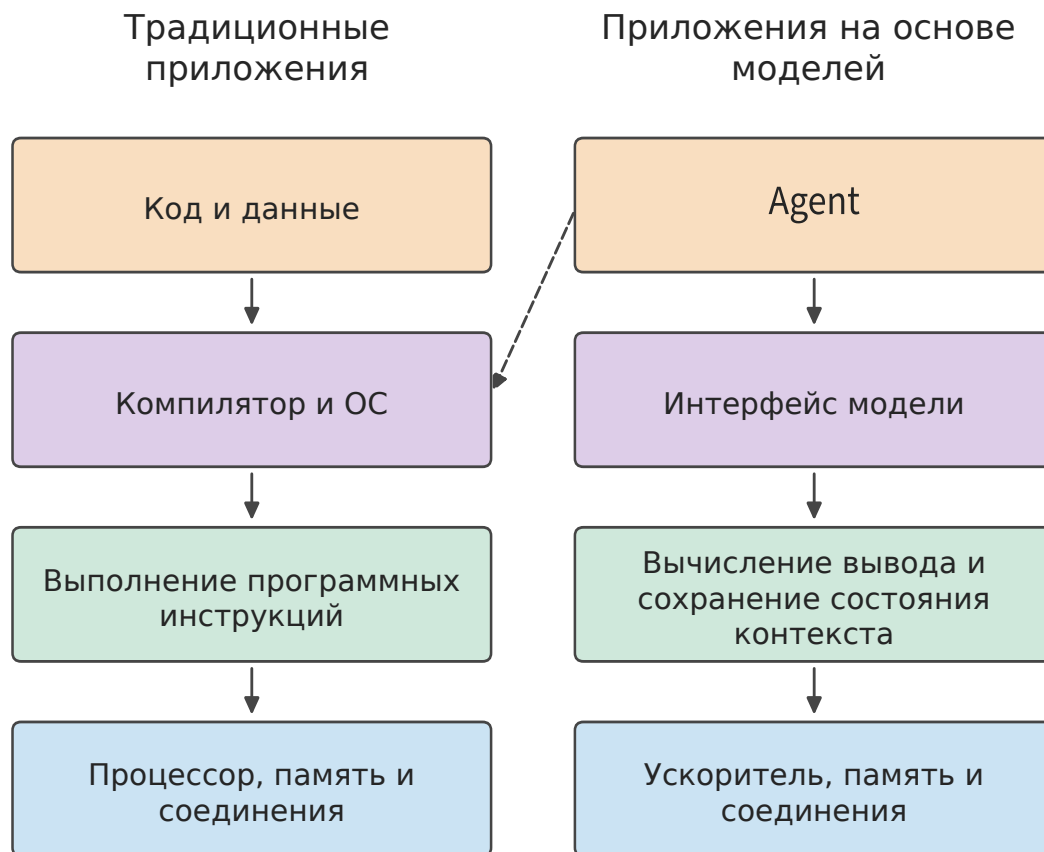
На рисунке 1-1 сопоставлены эти два способа организации. AI-агент (Agent) справа организует контекст, вызывает модель и выполняет выбранные ею инструменты. При этом **интерфейс модели** определяет, как приложение передаёт входные данные и получает выходные. Исполняющая система за интерфейсом вычисляет выход модели и сохраняет состояние контекста, необходимое для повторного использования на последующих шагах; ускоритель — это процессор, приспособленный к параллельному выполнению большого числа операций модели, память хранит параметры и состояние выполнения, а интерконнект передаёт данные между устройствами. Разработчик изменяет задачу на верхнем уровне, а нижние уровни должны воплотить эти изменения в конкретных вычислениях и обращениях к данным.

Изменение программируемости повлияло и на работу сервисов моделей. Когда в системе ИИ одна модель обрабатывает запросы разных приложений, она использует один и тот же набор параметров. Система может объединить несколько запросов для совместного вычисления и использовать однократно считанные параметры, повышая утилизацию ускорителя. Такой способ называется пакетной обработкой, а число совместно выполняемых запросов в одном пакете — batch size (размер пакета). В то же время длинные документы, длинные ответы и многоэтапные вызовы инструментов изменяют объём вычислений, занимаемое состоянием место и время ожидания. Производительность одной и той же модели на одном и том же наборе ускорителей при обработке разных задач может значительно различаться.

За этой разницей в производительности стоит взаимное влияние модели и аппаратного обеспечения. С точки зрения проектирования модели сжатие сохраняемого состояния, изменение механизма внимания или использование лишь части параметров модели может изменить работу, которую должно выполнить аппаратное обеспечение. С точки зрения аппаратных условий объём памяти, скорость передачи данных и способ соединения устройств, в свою очередь, влияют на то, какие структуры моделей и методы выполнения подходят лучше. Только понимая эту двустороннюю связь, можно одновременно обсуждать качество ответов, время отклика и стоимость, а также объяснять, почему решение, подходящее для одного приложения, требует повторной оценки при использовании в другом.

В системах, основными задачами которых являются обучение и инференс моделей,

¹Связанные исследования приведены в обзоре границ абстракции; обсуждение применения базовых моделей для множества последующих задач см. в Stanford CRFM.



Инструменты по-прежнему выполняет ОС

Рисунок 1-1. Способы выражения поведения приложения и пути выполнения. Слева программа организуется посредством кода, справа модель вызывается посредством контекста; сплошные линии обозначают направленные вниз зависимости выполнения, пунктирная линия показывает, что для выполнения инструментов по-прежнему требуется традиционная операционная система. Ускоритель, память и интерконнект отвечают соответственно за вычисления модели, хранение данных и передачу между устройствами.

разные уровни можно совместно проектировать вокруг единого полного процесса выполнения. Структура модели задаёт зависимости вычислений и данных, среда выполнения знает, когда состояние создаётся и когда используется, а аппаратное обеспечение выполняет соответствующие вычисления и передачу данных. Реорганизация выполнения с учётом этой информации позволяет устранить часть накладных расходов универсальной реализации. Одна и та же модель способна обрабатывать разные задачи в зависимости от контекста, одновременно поддерживая разнообразное поведение приложений и позволяя нижним уровням выполнять специализированную оптимизацию исполнения модели. **Изменение программируемости верхнего уровня открывает новые возможности для совместной межуровневой оптимизации.** В этой книге такие возможности оптимизации исследуются с помощью количественного анализа: объясняется, как меняются прежние проектные компромиссы и как новые реализации повышают производительность обучения и инференса.

1.1.2 Разделение обязанностей между шестью уровнями: от задачи до аппаратного обеспечения

От постановки до завершения запрос последовательно проходит через несколько уровней системы. Пользователь отправляет фрагмент кода и просит модель найти ошибку. Приложение передаёт код и вопрос сервису модели, сервис выбирает ускоритель для выполнения запроса, ускоритель считывает веса и входные данные, выполняет вычисления, а затем возвращает сгенерированный ответ приложению. Если выполнение совместно осуществляют несколько ускорителей, между ними также необходимо передавать промежуточные результаты. Следуя по этому пути, можно последовательно определить, где возникает ожидание и какие данные требуется перемещать.

Главный научный сотрудник Huawei Semiconductor доктор Ляо Хэн предложил модель «восемнадцатилетней пагоды», описывающую множество уровней: от приложений, моделей и программных систем до микросхем, технологических процессов производства и фундаментальной физики. Он отметил, что большинство людей знакомы лишь с одним или несколькими из этих уровней и потому упускают множество возможностей совместной межуровневой оптимизации.² На рисунке 1-2 этот подход используется для разделения материала книги на шесть уровней:

На самом верхнем уровне находятся **приложения и задачи**. Диалог, генерация кода, голосовое взаимодействие, а также агенты (Agent), способные продвигаться к выполнению задачи посредством вызова инструментов, формулируют на этом уровне конкретные требования: какую задачу выполнить, насколько быстро вернуть результат и каков предельный бюджет. У обучения также есть собственные цели, например выполнить одно обновление модели или одну эпоху предварительного обучения за заданное время.

Следующий уровень — **модели и нагрузка**. Структура модели определяет, какие вычисления необходимо выполнить и какие данные сохранить, а фактическая нагрузка — когда эта работа поступит и как долго будет продолжаться. Краткий вопрос и длинный документ создают разные требования к вычислениям и памяти, даже если передаются одной и той же модели. Структура модели рассматривается в главе 2, а нагрузка обучения и инференса — в главе 3.

²Метафору «восемнадцатилетней пагоды» предложил главный научный сотрудник Huawei в области полупроводников доктор Ляо Хэн в продолжительном публичном интервью в июле 2026 года, чтобы описать многоуровневые зависимости — от приложений до технологических процессов производства.



Рисунок 1-2. Разделение обязанностей между шестью уровнями: от приложения до аппаратного обеспечения.

Системы обучения и инференса организуют эти требования в исполняемую работу. Этот уровень принимает запросы или задания, формирует batch, управляет состоянием выполнения и определяет, как использовать один или несколько ускорителей. При инференсе он должен координировать запросы, для которых уже идёт генерация, и вновь поступающие запросы; при обучении — дополнительно организовывать обновление параметров, обмен данными и восстановление. Этот уровень рассматривается в главах 8—10.

Ещё ниже находятся **операторы, компилятор и среда выполнения**. Такие операции модели, как умножение матриц и нормализация (изменение масштаба значений на основе статистик), необходимо преобразовать в программы, которые способен выполнить процессор. В этой книге базовые вычисления, такие как умножение матриц, называются операторами; библиотека операторов предоставляет реализации этих вычислений, компилятор преобразует программу в форму, исполняемую ускорителем, а среда выполнения отвечает за отправку программы и управление ресурсами во время исполнения. Вместе они определяют, как разбивать работу на блоки, использовать временное хранилище и выбирать моменты отправки заданий. Одно и то же математическое выражение может иметь разные способы доступа к данным и выполнения; эти различия подробно анализируются в главе 5. Здесь же содержится ключ к пониманию распределённого параллелизма: матрицу можно обрабатывать блоками в пределах одной карты, а модель на нескольких картах можно разделить по образцам, последовательностям, признакам, слоям или экспертам — подсетям в модели смеси экспертов, которые выбираются отдельно для каждого входа. В обоих случаях сначала нужно определить, кто вычисляет каждый блок, а затем организовать поступление входных данных и объединение результатов. Разница заключается в том, пересекают ли блоки границы внутрикристалльной памяти, видеопамяти или сети. В главе 6 различные стратегии параллелизма объясняются на основе этой общей идеи, а не как не связанные друг с другом приёмы.

Процессоры и память выполняют вычисления и хранят данные. CPU — центральный процессор, выполняющий программы общего назначения; GPU — графический процессор, хорошо приспособленный для параллельной обработки множества однотипных операций и теперь широко используемый для вычислений моделей; NPU — процессор, разработанный для операций нейронных сетей. Основная память хранит данные, используемые CPU, видеопамять — данные, используемые GPU, а внутрикристалльная память находится внутри процессорного кристалла и позволяет вычислительным блокам считывать данные поблизости. Вычислительный блок сначала считывает входные данные, затем выполняет операцию и в конце записывает результат. В главе 4 устройство ускорителей объясняется через соотношение вычислительной мощности и скорости чтения данных.

На самом нижнем уровне находятся **интерконнекты и центры обработки данных**. Устройства обмениваются данными через интерконнекты, серверы и более крупные совместно работающие группы соединяются сетями, а электропитание и охлаждение определяют возможные способы размещения ресурсов. Суперузлы и сети центров обработки данных рассматриваются в главах 6 и 7, а в главе 12 возможные места выполнения расширяются до конечных устройств, периферии и облака.

Шесть уровней описывают основное разделение обязанностей, но некоторые функции охватывают несколько уровней. Планирование ресурсов, среда эксплуатации, наблюдаемость и тарификация должны организовываться с учётом информации с нескольких уровней; технологический процесс, корпусирование, электропитание и

охлаждение определяют способы изготовления и размещения устройств. Сеть также играет роль не только на самом нижнем уровне: конкретные интерконнекты требуются моделям, операторам и сервисам, предполагающим совместную работу нескольких устройств.

При движении вниз по этим шести уровням задача постепенно превращается в конкретные вычисления и обращения к данным; при движении вверх доступный объём памяти, вычислительная мощность и пропускная способность ускорителя определяют, модель какого размера сможет выполнять система, сколько запросов она сможет обрабатывать одновременно и насколько быстро отвечать. Интерфейсы между этими уровнями служат точками приложения совместной оптимизации. Изменение представления состояния в модели меняет требования к памяти и передаче данных; слияние соседних операторов компилятором меняет место хранения промежуточных результатов; прямая передача данных между ускорителями, организованная средой выполнения, меняет частоту участия хоста.

1.1.3 Обработка одного запроса на генерацию

Экземпляр инференса — это набор исполнительных ресурсов, способный независимо обрабатывать запросы модели и состоящий из одного или нескольких ускорителей. Каждый запрос передаётся одному из таких экземпляров. На рисунке 1-3 показано разделение обязанностей между распределением запросов и выполнением внутри экземпляра: маршрутизатор сервиса — это программа, выбирающая место выполнения запроса; после выбора экземпляра маршрутизатором выполнение модели на ускорителях планируется внутри экземпляра. Сначала необходимо определить, кто отвечает за выполнение, и лишь затем можно понять, где следует разместить веса и кому передавать промежуточные результаты.

Контекст включает пользовательский ввод, историю диалога, результаты поиска и ранее сгенерированное содержимое. Приложение формирует из этой информации запрос и передаёт его во входную точку сервиса модели. После проверки прав доступа, квоты и содержимого запроса маршрутизатор выбирает экземпляр с учётом целевой модели, нагрузки экземпляров и других условий; экземпляр может работать на одной карте или быть распределён по нескольким картам. После выбора экземпляра совместная работа этих карт организуется уже внутри него.

Затем текст преобразуется в последовательность токенов. Токен (token) — базовая единица обработки текста моделью; он может соответствовать одному символу, части слова или другому фрагменту, а число токенов в конкретном тексте определяется результатом токенизации. Преобразование текста может выполняться экземпляром или общим фронтом. Планировщик экземпляра помещает запрос в очередь, затем включает его в batch и выделяет место для его состояния. После этого среда выполнения на CPU отправляет вычислительные задания на GPU, а GPU выполняет соответствующие операторы.³

Сначала модель обрабатывает входные данные — этот этап называется предзаполнением (prefill); затем она пошагово генерирует новые токены — этот этап называется

³Схема запроса представляет собой обобщённую иллюстрацию зон ответственности; разделение ответственности за планирование и управление состоянием см. в документации vLLM Scheduler версии v0.26.0; в измерениях раздела 1.3.3 использовалась vLLM 0.23.0. Стратегия маршрутизации, расположение tokenizer и разделение prefill и decode зависят от выбранной конфигурации развёртывания.

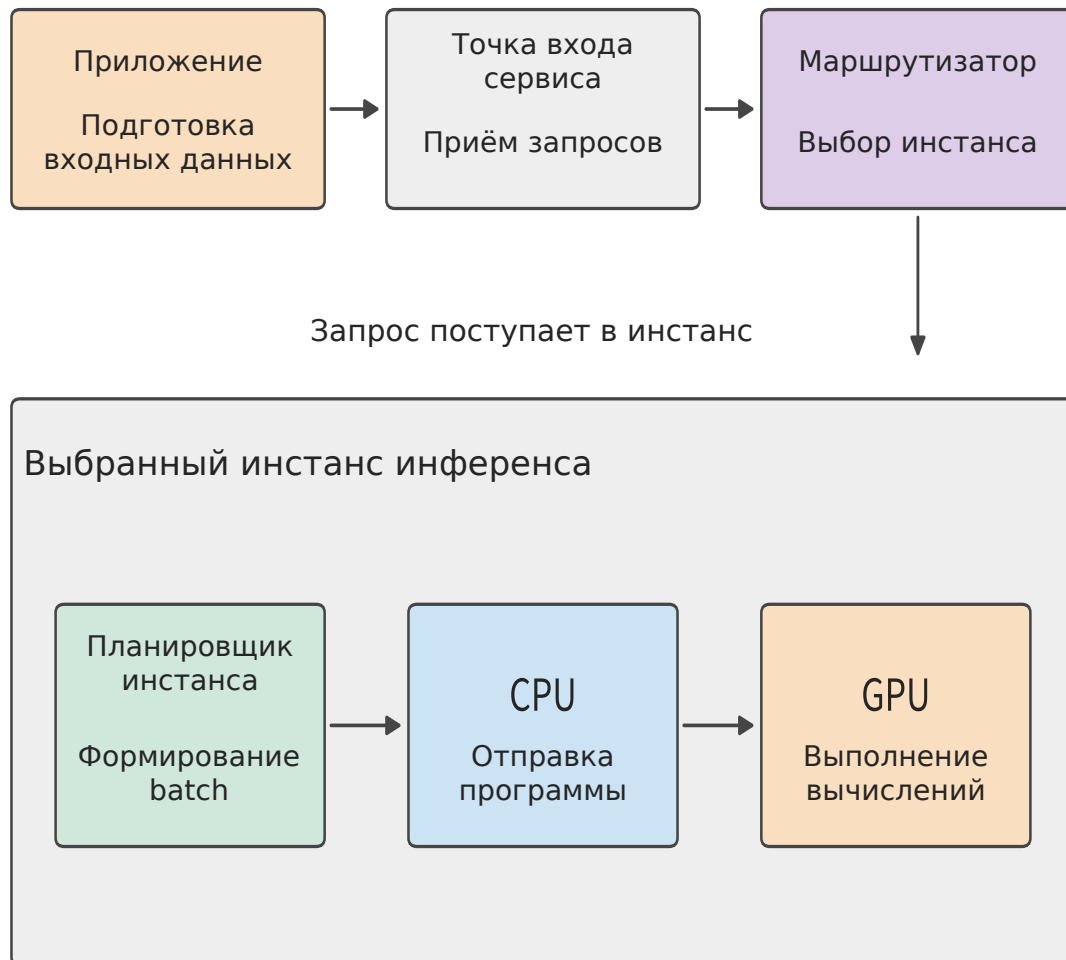


Рисунок 1-3. Маршрутизатор сначала выбирает экземпляр инференса для запроса, после чего внутренний планировщик экземпляра организует выполнение. Серая внешняя рамка охватывает один экземпляр, сплошные линии показывают направление передачи запросов и заданий.

декодированием (decode). На последующих шагах повторно используется ранее сохранённое состояние. Входящий в него KV-кэш (KV cache) хранит два вида векторов — ключи и значения, созданные токенами контекста, — для вычисления внимания на последующих позициях. Ключи и значения представляют собой используемые механизмом внимания данные; в главе 2 на примере конкретных операций объясняется, как они создаются. Планировщик внутри экземпляра непрерывно продвигает цикл генерации, и выход предыдущего шага становится входом следующего. Созданные токены преобразуются выходной обработкой в текст и возвращаются по соединению в потоковом режиме; если выход требует вызова инструмента, приложение запускает инструмент, добавляет результат в контекст и выполняет следующий вызов.

Прежде всего на каждом шаге вычислений повторно используются веса модели. Веса модели — полученные при обучении числовые параметры, используемые для преобразования входных данных в выходные. При запуске экземпляра или переключении модели веса загружаются из хранилища, а затем постоянно находятся в видеопамяти — это называется резидентным размещением — и используются для обработки множества запросов. Во время генерации вычислительные блоки GPU считывают из видеопамяти веса текущего слоя и состояние контекста, выполняют вычисления, а затем передают результат следующему слою. Таким образом, одни и те же данные перемещаются с двумя разными частотами: при загрузке модель передаётся на ускоритель, а при выполнении необходимые данные многократно передаются вычислительным блокам. В экземпляре с несколькими картами промежуточные результаты, вычисленные одной картой, также должны передаваться другим картам, чтобы последующие вычисления могли продолжиться. Промежуточный тензор — это созданный одной операцией и переданный последующим операциям многомерный числовой массив; матрица является двумерным тензором. **Внешний запрос может состоять всего из короткого фрагмента текста, но внутри перемещаются гораздо более объёмные веса, состояния и промежуточные тензоры.**

На рисунке 1-5 показано аппаратное обеспечение, соответствующее этим программным функциям. Центр обработки данных включает входную точку и сервисы на CPU, общее хранилище, а также пул ресурсов ускорителей, состоящий из нескольких суперузлов. Суперузел — это группа ускорителей, тесно взаимодействующих через высокоскоростной интерконнект; они могут быть распределены по нескольким серверам или вычислительным лоткам — устанавливаемым в стойку сменным блокам с CPU и ускорителями.

Внутри сервера или вычислительного лотка CPU соединён с основной памятью и отвечает за хост-программы и управление выполнением; каждый GPU соединён со своей видеопамятью, выполняет вычисления модели и хранит необходимые для работы данные. HBM (память с высокой пропускной способностью) — распространённый тип видеопамяти ускорителей, непрерывно предоставляющий данные через широкий интерфейс. CPU и GPU могут соединяться через PCIe (высокоскоростной интерконнект периферийных устройств), а GPU могут напрямую обмениваться данными через отдельный высокоскоростной интерконнект. Сетевая карта подключает локальный узел к сети. В системах с поддержкой прямого доступа к памяти передающее оборудование может напрямую записывать данные в заданную область памяти, а CPU отвечает за запуск передачи и управление ею; подробнее этот механизм передачи рассматривается в главе 7.

Интерконнект внутри суперузла обычно называется **scale-up** (вертикальное масштабирование); его основная задача — обеспечить совместные вычисления группы

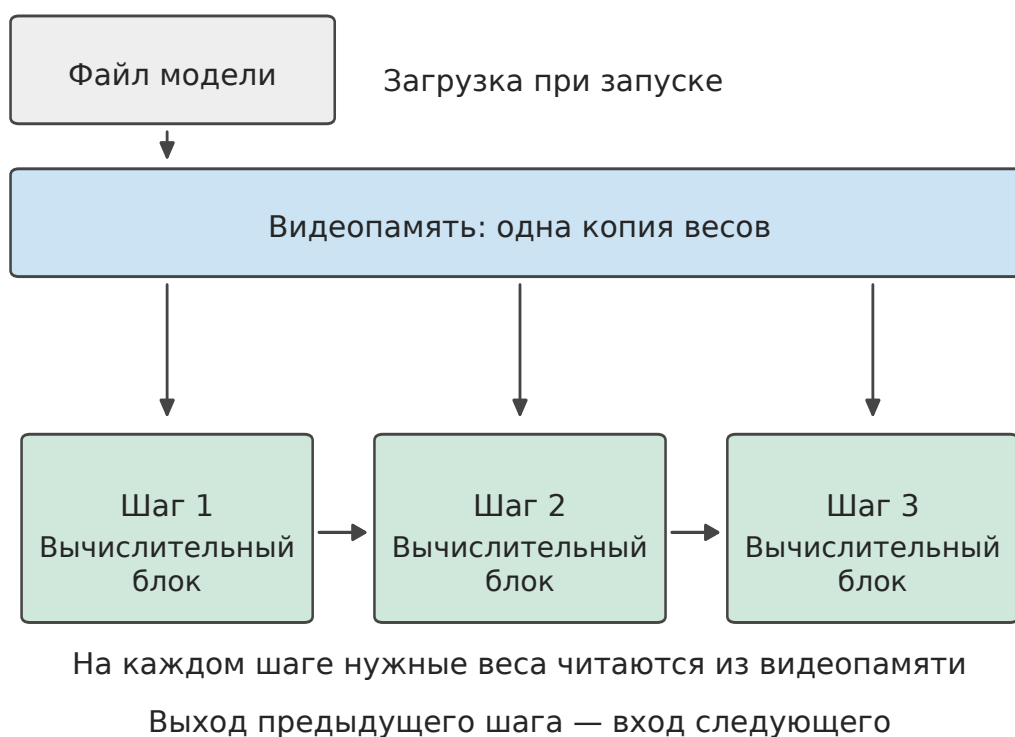


Рисунок 1-4. Веса загружаются один раз, а во время генерации считываются пошагово. Синяя рамка обозначает один и тот же набор весов, постоянно хранящийся в видеопамяти, три зелёные рамки — последовательные вычисления; стрелки обозначают чтение или зависимость результата, расстояние между шагами не отражает длительность.

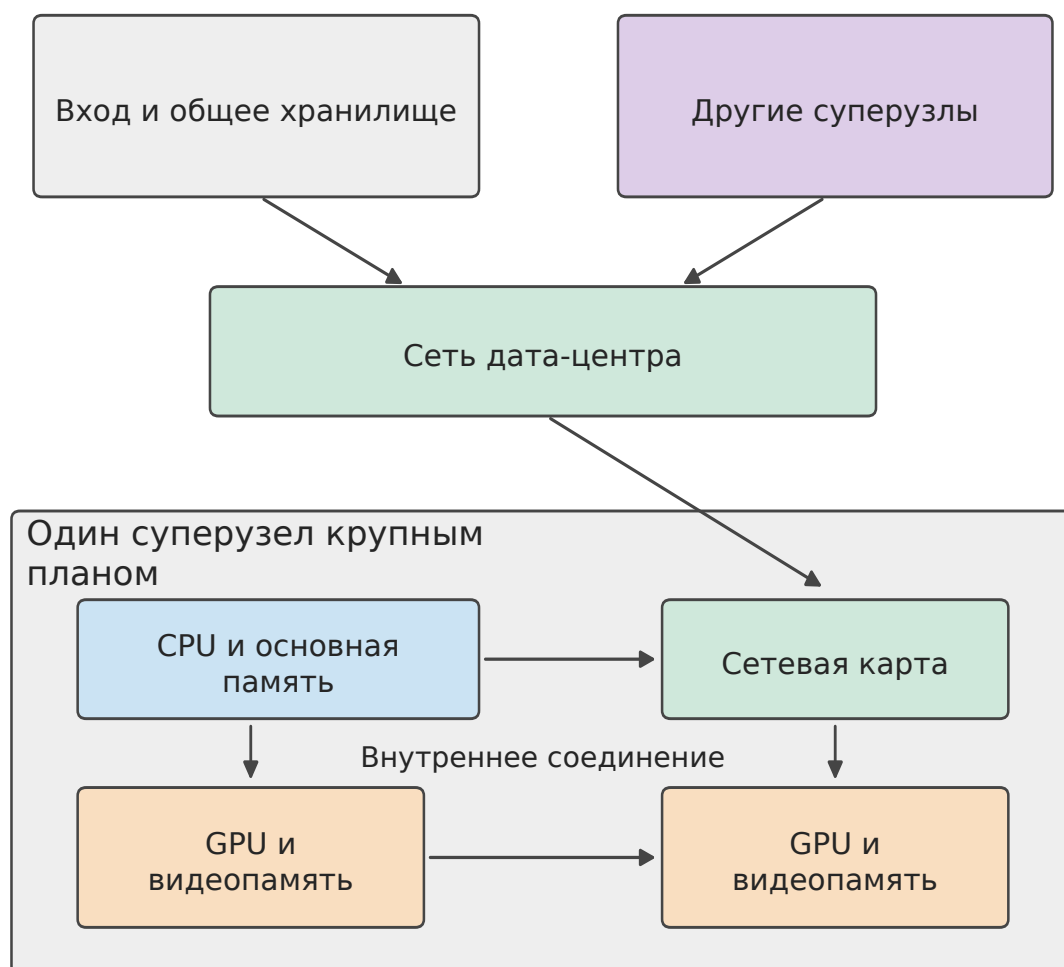


Рисунок 1-5. Двухуровневое представление физических соединений. В верхней части сервисы и суперузлы соединены сетью центра обработки данных; в нижней увеличен один суперузел с хостами, сетевыми картами, GPU и видеопамятью. Линии показывают пути данных, количество устройств приведено для иллюстрации.

ускорителей с небольшими коммуникационными издержками. Несколько суперузлов, в свою очередь, соединяются через сетевые карты и коммутируемую сеть — это называется **scale-out** (горизонтальное масштабирование) и служит для расширения кластера. В обоих случаях необходимо учитывать пропускную способность, задержку, перегрузки и отказы, но масштаб совместной работы и её стоимость различаются. Например, в официальной конструкции NVIDIA GB200 NVL72 36 CPU Grace и 72 GPU Blackwell объединены в одной стойке, а 72 GPU напрямую взаимодействуют через высокоскоростной GPU-интерконнект NVIDIA NVLink, образуя единый домен интерконнекта; за внешнее подключение отвечает кластерная сеть. Такая организация расширяет область тесного взаимодействия от одного сервера до целой стойки: веса и состояние модели можно распределить между большим числом ускорителей, а частая передача данных выполняется по внутristоечному интерконнекту.⁴

Таким образом, сети ИИ и традиционные сети центров обработки данных разделяют обязанности. Входная точка сервиса передаёт запросы, общее хранилище предоставляет файлы моделей, а интерконнект ускорителей передаёт промежуточные результаты вычислений. Эти три вида трафика различаются частотой передачи, объёмом данных и зависимостями: файлы модели можно загрузить заранее, тогда как часть результатов текущего слоя должна поступить своевременно, иначе следующий слой не сможет продолжить работу. Поэтому физические соединения непосредственно влияют на время выполнения модели.

Упражнение 1-1 (углублённое): чтение весов, рост состояния и передача данных в одном запросе

На основе рисунков 1-3—1-5 дорисуйте процесс роста состояния KV по мере увеличения выходных данных, а затем отдельно обозначьте загрузку файла модели, чтение весов за один шаг, добавление в KV и передачу результатов между картами. Если длина ответа увеличится со 100 до 1000 токенов, какие данные загрузятся только один раз, а число каких обращений возрастет? Если создать второй экземпляр как копию первого, как изменятся объём весов и маршрутизация запросов?

На выполнение модели влияют как возможности одной карты, так и способ соединения ускорителей. Вычислительная мощность и пропускная способность памяти одной карты определяют время локальных вычислений, чтения и записи, а совместная работа нескольких карт добавляет обмен данными и ожидание. В этой главе сначала оценивается выполнение на одной карте, а в последующих главах тот же метод применяется к суперузлам и сетям.

Сначала представим набор реальных моделей, проходящих через всю книгу. DeepSeek V4 и V4.1 Flash образуют сквозной пример, за развитием которого можно последовательно проследить. В V4 контекстное сжатие и разреженный доступ уже изменили требования к хранению и чтению данных, а V4.1 Flash дополнительно перераспределяет обязанности внутри модели. В традиционной модели decoder-only (только с декодером) входные токены обычно проходят через всю основную часть модели, послойно формируя необходимое для генерации состояние; V4.1 Flash использует архитектуру **причинного кодировщика-декодера (Causal Encoder-Decoder, CED)**, в которой кодировщик формирует представление контекста, а декодер получает из него глобальную информацию. Поэтому множеству входных токенов не требуется

⁴Конкретные примеры продуктов см. в материалах NVIDIA GB200 NVL72, руководстве по аппаратному обеспечению и руководстве по сети. На рисунке 1-5 показана обобщённая схема соединений.

проходить через вычисления основной части декодера; при генерации нового токена модель по-прежнему последовательно проходит через кодировщик и декодер. Пути выполнения для входных и выходных данных несимметричны, что позволяет модели перераспределить вычислительные затраты с учётом характерной для агентов нагрузки: большого объёма входных данных и сравнительно малого объёма выходных.⁵

Это архитектурное решение также меняет то, что система должна хранить и передавать. В книге рассматриваются однотипные сеансы: агент ожидает возврата результата инструмента, восстанавливает предыдущий контекст, обрабатывает новые входные данные и продолжает генерацию. В главе 2 объясняются структура и состояние CED, в главе 3 разбираются входная и генерационная нагрузки, в главах 4—7 отслеживаются выполнение на ускорителях и пути данных, в главах 8 и 9 рассматриваются кэширование состояния и распределение запросов, а в завершение сравниваются результаты выполнения задач целиком. Модель с открытым исходным кодом Qwen3 служит базовым примером для построения метода расчёта; V4/V4.1 используются для проверки того, как модель и система изменяются совместно.

1.2 Ключевые показатели проектирования систем

1.2.1 Оценка решений по порядку величины

Прежде чем оценивать выполнение на одной карте, необходимо знать примерную продолжительность основных типов операций. В 2009 году Джефф Дин представил на одном из своих выступлений ставшую впоследствии широко известной таблицу под названием «Numbers Everyone Should Know». В ней приведено время доступа к кэшу и памяти, а также выполнения дисковых и сетевых операций. Эта таблица отражает практический подход: прежде чем писать программу или строить систему, сначала оценить, сколько времени и ресурсов потребуют основные операции.⁶

Ниже полностью приведены 12 операций и соответствующие значения с исходного слайда. Всё время указано в ns (наносекундах, миллиардных долях секунды); 1 μ s (микросекунда) равна 1000 ns, а 1 ms (миллисекунда) — 1000 μ s. Среди единиц данных 1 byte (байт) равен 8 bit (битам); bit/s обозначает число передаваемых битов в секунду, а G в Gbit/s означает миллиард. Единицы ёмкости KB, MB, GB и TB обозначают соответственно 10^3 , 10^6 , 10^9 и 10^{12} байт; KiB, MiB и GiB — соответственно 2^{10} , 2^{20} и 2^{30} байт. Эти числа соответствуют оборудованию 2009 года и нужны для понимания соотношения порядков величин различных операций.

В кэше хранятся данные, которые могут вскоре понадобиться снова; кэш первого уровня обычно меньше кэша второго уровня и расположен ближе к вычислительному ядру. Предсказание ветвлений — это прогноз процессора относительно того, по какому пути инструкций продолжится выполнение; при ошибочном прогнозе выполнение приходится перестраивать. Поток — независимо планируемая единица выполнения инструкций внутри программы; мьютекс ограничивает одновременный доступ нескольких потоков к общим данным. Позиционирование диска — процесс перемещения го-

⁵Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и пересчёт сквозного примера из нескольких глав.

⁶Jeff Dean, выступление на LADIS 2009, локальный PDF, раздел «Numbers Everyone Should Know» и часть об оценках на бумаге. В примере с позиционированием головки диска предполагается последовательное выполнение 20 случайных чтений.

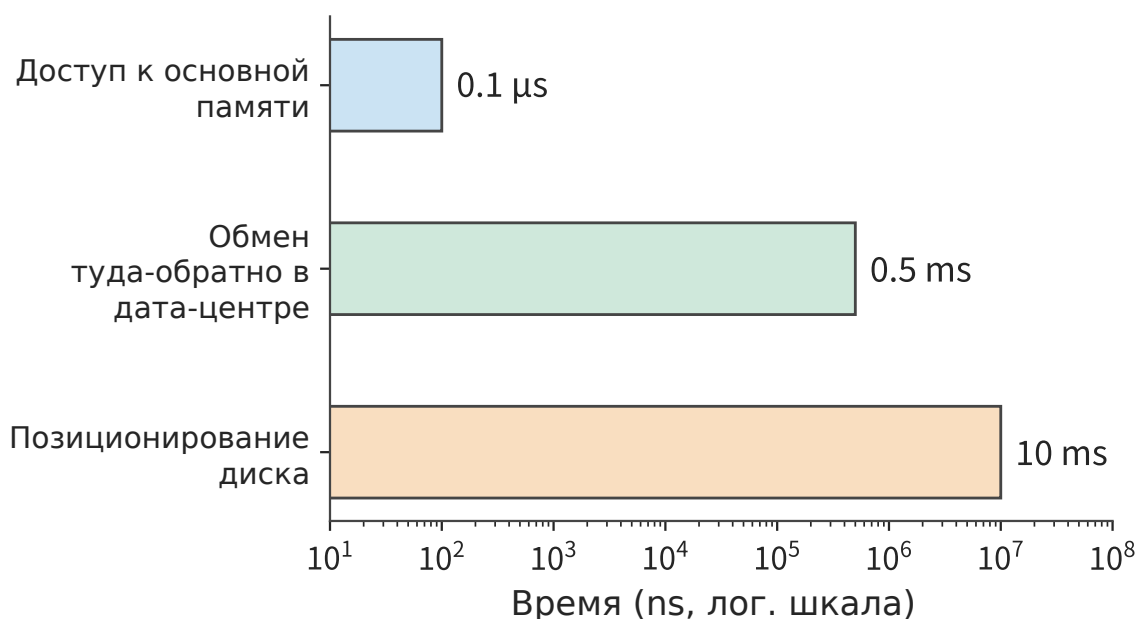


Рис. 1-6. Задержка трёх операций из выступления Джеффа Дина 2009 года. Горизонтальная ось имеет логарифмический масштаб, соседние порядки величины различаются в десять раз; сначала определите операцию, а затем сравните её стоимость при последовательном ожидании.

ловки механического диска к нужной дорожке. Zipпу — библиотека сжатия, которую в то время использовала Google. Эти операции относятся соответственно к вычислениям, синхронизации, хранению данных и сети, а продолжительность их выполнения может различаться на несколько порядков.

Операция	Время (ns)
Доступ к кэшу первого уровня	0.5
Восстановление после ошибочного предсказания ветвления	5
Доступ к кэшу второго уровня	7
Блокировка или разблокировка мьютекса	25
Доступ к основной памяти	100
Сжатие 1K байт с помощью Zipпу	3,000
Передача 2K байт по сети 1 Gbit/s	20,000
Последовательное чтение 1 MB из основной памяти	250,000
Передача туда и обратно внутри одного центра обработки данных	500,000
Позиционирование диска	10,000,000
Последовательное чтение 1 MB с диска	20,000,000
Передача пакета из Калифорнии в Нидерланды и обратно	150,000,000

Возьмём три исторических показателя: доступ к основной памяти занимает около 100 ns, одна передача туда и обратно внутри центра обработки данных — около 500,000 ns, а позиционирование диска — около 10,000,000 ns. После преобразования единиц получаем соответственно 0.1 мкс, 0.5 мс и 10 мс, как показано на рис. 1-6.

Согласно этим показателям, одно позиционирование диска занимает примерно в сто тысяч раз больше времени, чем один доступ к основной памяти. Если запрос должен последовательно ожидать нескольких операций доступа, их задержки складываются и увеличивают время выполнения запроса. Если для определения места следующего обра-

щения запрос должен сначала найти некоторый блок данных, то даже при минимальном объёме вычислений между двумя обращениями процессору придётся ждать завершения всей операции доступа к хранилищу.

Например, предположим, что запрос последовательно выполняет 20 позиционирований по 10 ms каждое, а все вычисления CPU занимают 1 ms. Общее время составит 201 ms. Двукратное ускорение вычислений сэкономит лишь 0.5 ms; если же разместить связанные данные последовательно и сократить число позиционирований с 20 до 10, экономия составит 100 ms. Эти две оптимизации воздействуют соответственно на вычисления и ожидание, а разница в выигрыше определяется долей каждого из них в исходном времени выполнения.

Разложив время на составляющие, можно определить направление оптимизации: чем большую долю исходного общего времени занимает некоторая часть, тем сильнее сокращение этой части улучшает общий результат. В предыдущем примере с позиционированием вычисления занимают лишь $1/201$ общего времени; даже если полностью устранить эту часть, общее время сократится всего примерно на 0.5%.

Обобщив это соотношение, получаем формулу общего ускорения. Если долю f исходного времени выполнения можно ускорить в s раз, а остальная работа и порядок выполнения остаются неизменными, то общее ускорение равно:

$$\text{Speedup} = \frac{1}{(1 - f) + f/s}.$$

Это закон Амдала. Даже если ускорить эту часть настолько, что её временем можно будет пренебречь, общее ускорение не превысит $1/(1 - f)$. При сокращении числа позиционирований уменьшается самая большая составляющая — время ожидания, поэтому такая оптимизация влияет на общее время гораздо сильнее, чем ускорение CPU.

Пример: почему при использовании одного и того же инструмента AI-программирования разные команды получают разное общее ускорение? Предположим, что выполнение задачи разработки состоит из последовательных этапов коммуникации, написания кода, проверки и согласования, а инструмент AI-программирования ускоряет написание кода в 5 раз, не изменяя продолжительность остальных этапов. В команде крупной компании со множеством уровней координации написание кода может занимать лишь 20–30% исходного общего времени, а остальное уходит на совещания, межкомандную коммуникацию и согласования. После подстановки в формулу общее ускорение составит лишь около 1.19–1.32 раза. Например, если из исходных 100 часов 20 часов уходило на написание кода, после сокращения этого этапа до 4 часов остальные этапы по-прежнему займут 80 часов, а общее время составит 84 часа.

В стартапе с меньшими затратами на коммуникацию написание кода может занимать 70–80% исходного общего времени, и такое же ускорение этого этапа позволит получить общее ускорение примерно в 2.27–2.78 раза. Например, при доле написания кода 80% исходные 100 часов превратятся в $20 + 80/5 = 36$ часов.

Закон Амдала применим к любому полному процессу, в котором ускоряется лишь часть этапов. CPU, инференс модели и разработку программного обеспечения можно анализировать с помощью одного и того же вопроса: какую долю времени первоначально занимала ускоряемая часть и на что уходит оставшееся время? Для дальнейшего сокращения срока поставки необходимо улучшать также коммуникацию, проверку

и согласование. Повышение локальной скорости в конечном счёте следует оценивать в рамках всей задачи.

Метод, представленный в этой таблице, можно напрямую применять к выполнению моделей: сначала определить, какие ресурсы использует каждая операция, а затем сравнить требуемое время.

1.2.2 Какие ключевые показатели необходимы для анализа AI-систем

Предположим, что некоторой задаче требуется одновременно хранить M байт, выполнить F операций с плавающей точкой и прочитать или записать R байт через некоторый интерфейс хранилища. Ускоритель предоставляет ёмкость M_{cap} , вычислительную пропускную способность Π и пропускную способность интерфейса β . Эти три вида требований необходимо по отдельности сопоставить с ресурсами ускорителя:

$$M \leq M_{\text{cap}}, \quad T_{\text{compute}} = \frac{F}{\Pi}, \quad T_{\text{memory}} = \frac{R}{\beta}.$$

Ёмкость определяет, можно ли одновременно хранить все необходимые данные; два других выражения преобразуют объём работы во время. Данные можно сохранить один раз, а прочитать многократно, поэтому M и R необходимо вычислять отдельно. Если для Π и β взять пиковые значения, результатом будет нижняя граница времени, необходимого для выполнения заданных вычислений и операций чтения и записи.

Пиковые значения определяются числом вычислительных блоков чипа, рабочей частотой и интерфейсом памяти, поэтому эта нижняя граница является физическим пределом оборудования: программное обеспечение при любой организации не сможет работать быстрее. Отношение нижней границы к фактическому времени T показывает, какая доля возможностей оборудования была использована. Для вычислений отношение $F/(\Pi T)$ называется **MFU** (model FLOPs utilization, коэффициент использования FLOPs модели), а для пропускной способности отношение $R/(\beta T)$ — **MBU** (memory bandwidth utilization, коэффициент использования пропускной способности памяти); оба значения не превышают 1. Эти два отношения неоднократно используются в книге для ответа на один и тот же вопрос: насколько проект далёк от физического предела оборудования.

Помимо ёмкости, вычислительной пропускной способности и пропускной способности памяти необходимо учитывать задержку операций. Ниже описано, как эти четыре типа показателей используются в оценках.

Ёмкость отвечает на вопрос, сколько данных можно разместить одновременно. Например, объём видеопамати ускорителя определяет, поместятся ли в неё веса модели, состояние контекста и временная рабочая область среды выполнения. Ёмкость измеряется в byte (байтах).

Вычислительная пропускная способность отвечает на вопрос, сколько операций можно выполнить за единицу времени. Обычно FLOP обозначает одну операцию с плавающей точкой, FLOPs — общее число операций, а FLOP/s — число операций с плавающей точкой в секунду. GFLOPs и TFLOPs обозначают соответственно миллиард и триллион операций с плавающей точкой, а GFLOP/s и TFLOP/s — соответствующую скорость в секунду. В матричных вычислениях одно умножение и одно сложение обычно считаются двумя операциями. Пиковая матричная производительность соответствует определённым условиям точности входных данных, точности накопления и разреженности. Плотные вычисления выполняются над полной матрицей; разреженные вычисления ис-

пользуют нулевые элементы или заданную структуру разреженности, чтобы пропускать часть операций.

Пропускная способность отвечает на вопрос, сколько данных можно передать за единицу времени. Пропускная способность видеопамати характеризует возможности передачи данных между видеопаматью и чипом, а пропускная способность межкарточных соединений и сети — возможности других путей. Для оценки определённого участка передачи используется пропускная способность интерфейса этого участка.

Задержка отвечает на вопрос, сколько времени приходится ждать после запуска одной операции. Даже интерфейс с высокой пропускной способностью может иметь существенное время запуска или передачи туда и обратно. При передаче больших блоков данных время главным образом определяется отношением объёма данных к пропускной способности; для небольших запросов, которые должны ожидаться последовательно, основную часть могут занимать время запуска и передачи туда и обратно.

Рассмотрим реальный ускоритель. H100 — одно из поколений GPU NVIDIA, а SXM обозначает форм-фактор модуля, используемый в этом примере. BF16 — формат с плавающей точкой, в котором каждое число занимает 16 бит (2 байта), а FP32 — формат, в котором каждое число занимает 32 бита (4 байта); матричное умножение может считывать входные данные BF16 и хранить результаты накопления в FP32. H100 SXM имеет 80 GB видеопамати, пропускную способность HBM 3.35 TB/s и пиковую производительность плотных матричных вычислений около 989.4 TFLOP/s при входных данных BF16 и накоплении FP32.⁷

Если добавить к H100 модели NVIDIA A100 80GB SXM и GeForce RTX 4090, получится краткая справочная таблица ресурсов GPU. Они используют архитектуры Hopper, Ampere и Ada соответственно. Ёмкость в таблице указана в номинальных GB производителя, пропускная способность — в десятичных TB/s; для матричной производительности во всех случаях приведено пиковое значение плотных вычислений с входными данными BF16 и накоплением FP32. Последняя строка получена делением 1 GB на пропускную способность видеопамати.⁸

Ресурс или операция	RTX 4090	A100 80GB SXM	H100 SXM
Ёмкость видеопамати (GB)	24	80	80
Пропускная способность видеопамати (TB/s)	1.008	2.039	3.35
Пиковая производительность матриц BF16 (TFLOP/s)	165.2	312	989.4
Чтение 1 GB при пиковой пропускной способности (ms)	0.99	0.49	0.30

Эта таблица содержит показатели производительности оборудования, необходимые для преобразования объёма работы во время. Например, если объём чтения и записи остаётся неизменным, а пропускная способность удваивается, R/β уменьшается вдвое; если объём вычислений остаётся неизменным, а вычислительная производительность удваивается, F/Π также уменьшается вдвое.

⁷NVIDIA, техническое описание архитектуры H100 и снимок страницы спецификаций. В этой главе используются характеристики форм-фактора SXM и плотных матричных операций BF16; исходные версии входных данных и контрольные значения приведены в перечне источников иллюстраций.

⁸Числа взяты из фиксированных спецификаций GPU с отдельными источниками для каждого показателя и соответствуют rtx4090, a100-80gb-sxm и h100-sxm; для каждого значения отдельно проверены точность, способ накопления и условие плотности. В этой таблице GB и TB являются десятичными единицами.

1.2.3 Пересчёт объёма хранения параметров и пропускной способности соединения

Чтобы использовать эти четыре типа показателей для оценки, их необходимо сопоставить с конкретной работой. Сначала параметры модели преобразуются в число байт, чтобы определить, поместятся ли они на ускорителе; затем число байт делится на пропускную способность, чтобы оценить время перемещения этих данных.

Рассмотрим реальную модель **DeepSeek-R1-Distill-Llama-70B**. Она основана на архитектуре Llama и является плотной моделью — для вычисления каждого токена используются все параметры. Опубликованные веса содержат около 70.554 миллиарда параметров. Буква B в названии означает миллиард, а 70B — приблизительное число параметров; в последующих формулах переменная B обозначает число запросов в batch. Обозначим число параметров через N , а число байт для хранения одного веса — через b_W . В BF16 каждый вес занимает 2 байта, поэтому общий объём опубликованных файлов весов равен⁹

$$\begin{aligned} M_W &= N b_W \\ &= 70,553,706,496 \times 2 \text{ bytes} \\ &\approx 141.11 \text{ GB.} \end{aligned}$$

В этом разделе везде используются десятичные GB, то есть $1 \text{ GB} = 10^9 \text{ bytes}$; двоичная единица GiB равна 2^{30} bytes . Таким образом, 141.11 GB — это примерно 131.42 GiB. Вычитать размер весов из доступной ёмкости ускорителя можно только после приведения обоих значений к одинаковым единицам.

Весам объёмом 141.11 GB недостаточно номинальных 80 GB видеопамати одной H100 SXM. Если равномерно разделить веса между двумя картами, на каждой они займут около 70.55 GB и останется примерно по 9.45 GB. **Разбиение** означает, что разные ускорители хранят и вычисляют отдельные части модели. Двум картам потребуется обмениваться промежуточными результатами; конкретные методы разбиения и коммуникации рассматриваются в главе 6.

Можно также изменить точность хранения весов. **Квантизация (quantization)** представляет значения меньшим числом битов: например, исходные 16-битные веса с плавающей точкой приближённо представляются 8-битными целыми числами, а scale (масштабирующий коэффициент) используется для восстановления из целого числа приблизительного значения в соответствующем диапазоне. После квантизации каждый вес занимает не 2, а 1 байт, поэтому потребление памяти заметно снижается.

В используемой в этой книге схеме групповой квантизации один scale является общим для каждых 128 весов, а часть параметров продолжает использовать BF16. Согласно расчёту для реальной конфигурации модели, все веса и дополнительные данные квантизации в 8-битной схеме занимают около 73.73 GB, поэтому помещаются на одной H100 SXM, оставляя примерно 6.27 GB. На рис. 1-7 оба варианта показаны в одном масштабе ёмкости: веса BF16 распределяются по двум картам либо после квантизации помещаются на одну карту.

⁹Количество параметров и число байтов BF16 получены из публичного индекса весов DeepSeek-R1-Distill-Llama-70B и конфигурации фиксированной версии. Общий объём весов при групповом квантовании взят из сводных данных по всей модели в записи поэлементного расчёта: 128 параметров на группу и 2 байта на scale. Номинальная спецификация RTX 4090 на 24 GB приведена в архивной копии официальной страницы. Для единой оценки ёмкости в этом разделе используются указанные в спецификациях значения GB.

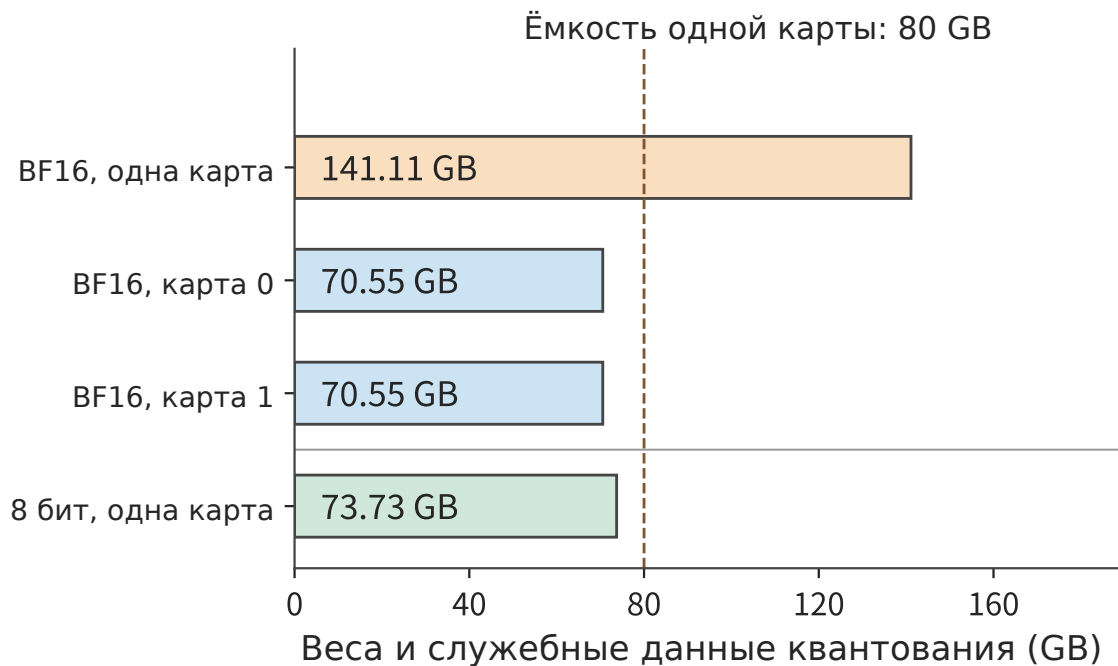


Рис. 1-7. DeepSeek-R1-Distill-Llama-70B: веса BF16 в сумме занимают 141.11 GB, а после равномерного распределения по двум картам — около 70.55 GB на каждой; после групповой 8-битной квантизации общий объём составляет 73.73 GB. Все полосы показаны в одинаковом масштабе, пунктирная линия обозначает номинальную ёмкость одной H100 SXM — 80 GB. Длина полос учитывает только веса и соответствующие дополнительные данные квантизации.

Как KV-кэш увеличивает потребность одного запроса в видеопамяти. При генерации текста в видеопамяти также необходимо хранить KV-кэш для непосредственного повторного использования в последующих вычислениях; кроме того, операторам нужна временная рабочая область. Поэтому проверку ёмкости следует записывать так:

$$M_W + M_{\text{state}} + M_{\text{work}} \leq M_{\text{cap}}.$$

Здесь M_W — веса и дополнительные данные квантизации, M_{state} — состояние контекста запроса, M_{work} — рабочая область, а M_{cap} — доступная ёмкость памяти ускорителя. Например, для одного запроса длиной 8192 токена KV-кэш BF16 этой модели занимает 2.5 GiB, то есть около 2.68 GB. Если дополнительно зарезервировать 2 GiB, или около 2.15 GB, для рабочей области, общий объём 8-битной схемы составит примерно $73.73 + 2.68 + 2.15 = 78.56$ GB, что соответствует данному бюджету ёмкости. В главе 2 размер KV-кэша будет выведен из числа слоёв модели и структуры внимания, а также будут рассчитаны требования к памяти для более длинного контекста и большего числа одновременных запросов.

Если вместо этого использовать RTX 4090 с номинальными 24 GB видеопамяти, те же 73.73 GB весов превысят суммарные 72 GB трёх карт. При таком же расчёте по номинальной ёмкости даже для хранения одних только весов потребуется не менее четырёх карт; кроме того, для каждой карты необходимо проверить объём KV-кэша и рабочей области. При одинаковых модели и точности разная ёмкость одной карты изменяет требуемое число ускорителей.

Для сетевой передачи необходимо выполнять такое же преобразование единиц. Пропускная способность сети часто измеряется в bit/s; поскольку 8 bit равны 1 byte, линей-

ная скорость сетевой карты ConnectX-7 с пропускной способностью 400 Gbit/s — то есть номинальная скорость передачи соединения — равна 50 GB/s. Передача 1 GB данных с такой линейной скоростью занимает 20 ms. Каждая передача также требует времени запуска, а накладные расходы протокола и другой трафик в общем соединении влияют на пропускную способность, фактически доступную приложению. Учёт этих факторов на всём пути передачи позволяет точнее оценить время коммуникации.

Упражнение 1-2 (основное): требования к видеопамяти и распределение по нескольким картам при разной точности весов

Веса BF16 модели DeepSeek-R1-Distill-Llama-70B занимают 141.11 GB, а 8-битные квантизованные веса — 73.73 GB. Модель развёртывается на картах H100 SXM, каждая из которых имеет 80 GB видеопамяти. Зарезервируйте на каждой карте 5 GB для состояния и рабочей области и определите, помещается ли модель при развёртывании на одной карте и при равномерном распределении весов между двумя картами. Затем рассмотрите две карты с разным объёмом видеопамяти: RTX A6000 на 48 GB и H100 SXM на 80 GB. Для обоих вариантов точности определите, возможно ли развёртывание, и для каждого допустимого варианта предложите один способ распределения весов. Наконец, преобразуйте 400 Gbit/s в GB/s и вычислите идеальное время передачи 1 GB.

1.3 Оценка одного выполнения модели с помощью нескольких чисел

1.3.1 Что проверить в первую очередь при генерации одного токена

Пример 1-1: как приблизить время одного шага генерации модели 70B к целевым 10 мс? Можно ли на одной H100 SXM заставить DeepSeek-R1-Distill-Llama-70B из предыдущего раздела генерировать новый токен не более чем за 10 мс? Сначала примем, что каждый параметр занимает один байт, а затем сравним удвоение вычислительной мощности, удвоение пропускной способности и повторное использование весов внутри batch.

Решение: сначала проверим объём видеопамяти, затем вычислим объём вычислений и чтения.

Чтобы упростить ручные расчёты, приблизительно примем число параметров модели равным $N = 70 \times 10^9$. Сначала оценим объём данных основных весов из расчёта один байт на параметр; после чтения они преобразуются в BF16 и участвуют в матричных операциях. Для одного запроса на каждом шаге обрабатывается один новый токен, а эти веса считываются из видеопамяти один раз за шаг.

Условия оценки: чтение весов и матричные вычисления полностью перекрываются. Для необходимого объёма памяти используем полученное в предыдущем разделе значение 73,73 ГБ, учитывающее дополнительные данные квантования; в этом разделе объём чтения основных весов сначала приблизительно примем равным 70 ГБ. В модели времени учитываются только умножение со сложением для матриц основных весов и одно полное чтение весов; используются пиковая производительность матричных вычислений BF16 и пропускная способность HBM из таблицы ресурсов. Чтение и вычисления оцениваются в предположении их полного перекрытия.



Путь чтения: 3350 GB/s

$$70 \text{ GB} \div 3350 \text{ GB/s} \approx 20.90 \text{ ms}$$

Байт на параметр, полное чтение на каждом шаге

Рис. 1-8. В этом примере из расчёта один байт на параметр объём чтения основных весов приблизительно принимается равным 70 ГБ. Веса постоянно находятся в видеопамяти, а вычислительные блоки на каждом шаге считывают их через один и тот же интерфейс; деление объёма чтения на пропускную способность интерфейса даёт нижнюю границу времени чтения 20,90 мс.

Бюджет памяти для одного запроса уже проверен в предыдущем разделе. Что касается скорости генерации, после загрузки весов в видеопамять на каждом шаге всё равно требуется передавать участвующие в вычислениях веса вычислительным блокам; согласно приведённому приближению, на каждом шаге считывается около 70 ГБ.

Затраты 20,90 мс на чтение повторяются на каждом шаге генерации. Для одного запроса вход следующего шага определяется текущим выходом, поэтому последующие шаги должны выполняться последовательно по этой цепочке зависимостей.

Объём вычислений также можно сначала оценить приближённо. При умножении матрицы на вектор каждый вес обычно участвует в одном умножении, после чего полученное произведение прибавляется к результату; если считать умножение и сложение отдельно, это соответствует примерно двум операциям с плавающей точкой. Поэтому объём основных матричных вычислений в этом примере приблизительно равен:

$$\begin{aligned} \text{Объём матричных вычислений на шаг} &\approx 2 \times \text{число параметров} \\ &\approx 2 \times 70 \times 10^9 \text{ FLOPs} \\ &= 140 \text{ GFLOPs.} \end{aligned}$$

Множитель $2N$ возникает потому, что при проекции — линейном преобразовании вектора признаков с помощью матрицы весов — каждый вес умножается на соответствующую компоненту вектора признаков токена, после чего произведение прибавляется к выходному значению. Чем больше весов в матрице параметров, тем больше объём этой работы; когда число участвующих в вычислении токенов в batch увеличивается, одна и та же матрица весов обрабатывает больше векторов признаков токенов, поэтому объём вычислений также возрастает. В главе 2 будет подробно рассмотрена реальная модель, где помимо этой линейной работы присутствуют взаимодействия механизма внимания, зависящие от длины контекста.

Обозначим размер batch через B и позволим этим B запросам совместно использовать однократно считанные веса. Тогда получим три величины этой учебной модели:

$$M_W = b_W N, \quad R_W = b_W N, \quad F \approx 2BN.$$

Первые два выражения на этом шаге совпадают, поскольку веса хранятся в одном экземпляре и считываются один раз. При выполнении следующего шага генерации резидентный объём по-прежнему равен M_W , но дополнительно выполняется ещё одно чтение объёмом R_W . Увеличение B повышает объём вычислений на текущем шаге: с одной и той же матрицей весов необходимо выполнить умножение со сложением для векторов текущих входных токенов большего числа запросов. В главе 2 дополнительно будет учтено независимое состояние контекста каждого запроса.

1.3.2 Сколько времени занимают вычисления и чтение по отдельности

Разделив $F = 140$ GFLOPs и $R_W = 70$ GB на соответствующие характеристики ресурсов, получим две нижние границы времени для одного запроса.

$$\text{Время только на чтение весов} \geq \frac{70 \text{ GB}}{3350 \text{ GB/s}} \approx 20,90 \text{ ms},$$

$$\text{Время матричных вычислений} \geq \frac{140 \text{ GFLOPs}}{989400 \text{ GFLOP/s}} \approx 0,1415 \text{ ms}.$$

Нижняя граница времени чтения весов примерно в 148 раз больше времени матричных вычислений. При номинальной вычислительной мощности матричные операции выполняются очень быстро, однако передача весов из видеопамати вычислительным блокам занимает гораздо больше времени. До завершения текущего шага все эти данные должны быть считаны.¹⁰

Причина в том, что в этом примере на каждый считанный вес выполняется лишь одна операция умножения со сложением. Быстро обработав уже считанные данные, вычислительные блоки вынуждены ждать поступления следующих. Повышение вычислительной мощности может сократить только время умножения со сложением, но не ускоряет чтение из видеопамати.

Следует ли складывать эти два времени или выбирать большее из них, зависит от возможности перекрытия чтения и вычислений. Если данные поступают блоками, то во время вычислений над текущим блоком можно продолжать чтение следующего. В упрощённой модели с полным перекрытием нижняя граница времени равна:

$$T_{\text{step}} \geq T_{\text{lower}} = \max\left(\frac{F}{\Pi}, \frac{R_W}{\beta}\right).$$

После выхода на установившийся режим каждый блок данных должен пройти этапы чтения и вычислений, поэтому скорость обработки определяется более медленным из них. В этом примере чтение весов значительно медленнее матричных вычислений, следовательно, оптимизацию прежде всего следует направить на сокращение этих 20,90 мс чтения.

¹⁰Пересчёт приближённого бюджета для 70B и сопутствующие сценарии; пересчёт единиц и ёмкости отдельных ускорителей. График оценки времени в разделе 1.3 использует этот фиксированный приближённый результат для 70B; график ёмкости в разделе 1.2.3 использует реальный индекс весов и результаты группового квантования.

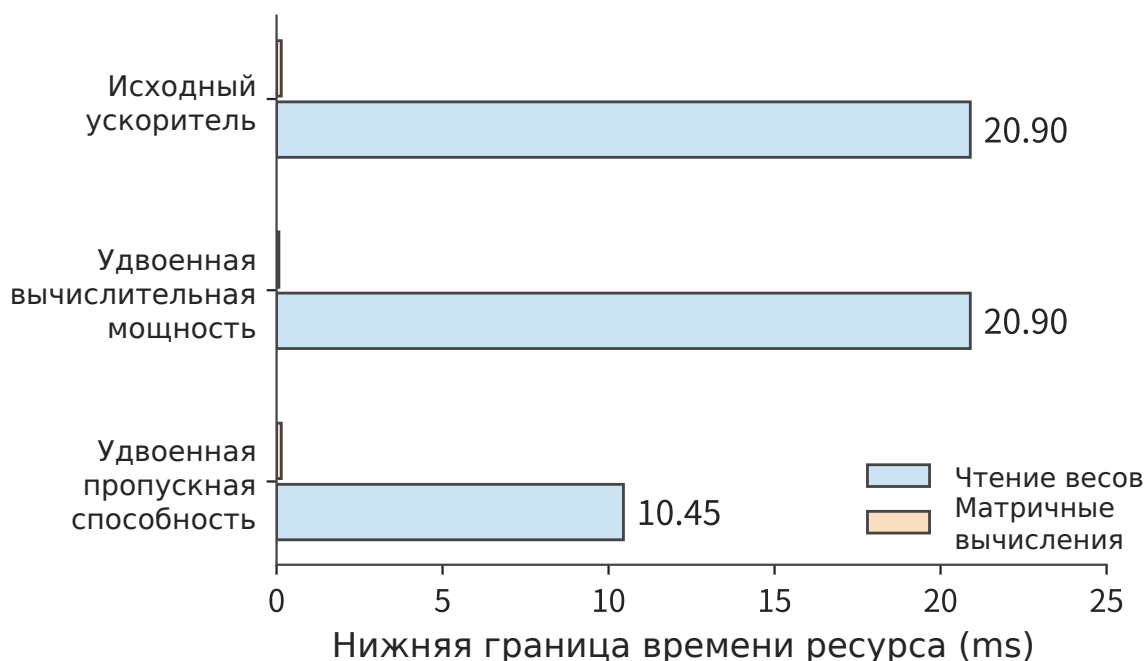


Рис. 1-9. При неизменных объемах вычислений и чтения по отдельности удваиваются вычислительная мощность и пропускная способность. Синяя полоса показывает нижнюю границу чтения весов, оранжевая — нижнюю границу матричных вычислений; более продолжительное чтение определяет направление оптимизации при этих условиях. Каждый столбец показывает нижнюю границу времени вычислений или чтения данных, однако полное выполнение всё равно должно учитывать зависимости.

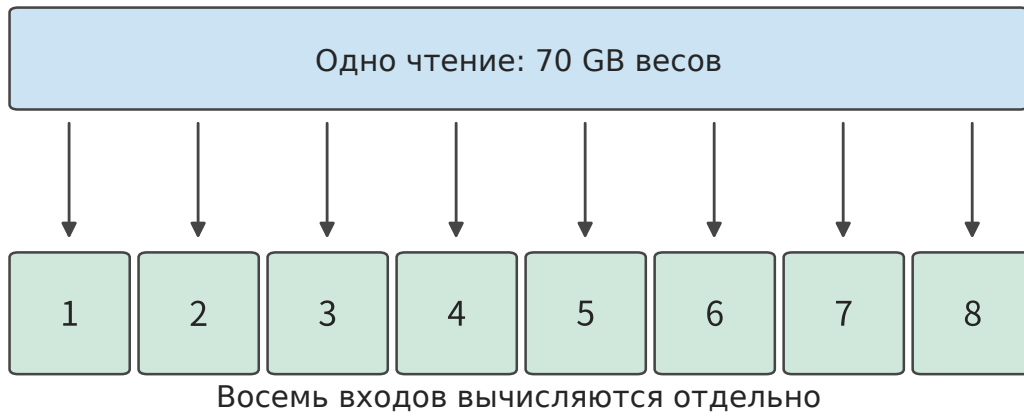
Обсуждение: насколько вычислительная мощность, пропускная способность и повторное использование внутри batch могут сократить время генерации? Одно только чтение весов занимает не менее 20,90 мс, что уже превышает целевые 10 мс. Хотя эти данные помещаются в видеопамять, скорость их чтения всё равно недостаточна. Сначала по отдельности сравним эффект повышения вычислительной мощности и пропускной способности.

При удвоении вычислительной мощности время матричных вычислений уменьшается примерно с 0,1415 до 0,0707 мс, но чтение весов по-прежнему занимает около 20,90 мс, поэтому определяющая время выполнения нижняя граница чтения не меняется. Только при удвоении пропускной способности НВМ время чтения снижается примерно до 10,45 мс, вслед за ним уменьшается и текущая нижняя граница. Обе модификации улучшают одну из характеристик оборудования, но по-разному влияют на время выполнения задачи.

Другой способ — уменьшить объём считываемых данных. Если при неизменном объёме вычислений сократить объём весов вдвое, нижняя граница чтения также снизится примерно до 10,45 мс. Удвоение пропускной способности вдвое повышает скорость чтения, а сокращение весов вдвое уменьшает объём считываемых данных; в этой формуле оба способа дают одинаковый результат. В главах 2 и 5 будут рассмотрены форматы представления с низкой разрядностью и процесс преобразования.

Теперь изменим организацию запросов. Предположим, что восемь запросов обрабатываются одновременно и совместно используют одно чтение весов. Объём чтения ве-

Одни веса обслуживают восемь запросов



Весь пакет ≈ 20.90 ms; на выходной токен ≈ 2.61 ms

Рис. 1-10. Восемь запросов в одном batch совместно используют одно чтение весов и создают по одному выходу. Чтение для всего batch по-прежнему занимает около 20,90 мс; деление на восемь даёт приходящееся на один выход время обслуживания, при этом каждый запрос проходит через выполнение всего batch. «Время на один выходной токен» — это время выполнения всего batch, делённое на число выходных токенов; оно используется для пересчёта пропускной способности и не является задержкой ответа отдельного запроса.

сов для всего batch остаётся равным 70 ГБ, а объём матричных вычислений становится примерно в восемь раз больше, чем для одного запроса. В этом случае нижняя граница времени матричных вычислений составляет около 1,13 мс, а нижняя граница чтения по-прежнему равна 20,90 мс. Если этот batch создаёт восемь выходов, то приходящееся на каждый выход время чтения весов составляет около 2,61 мс.

Выполнение всего batch создаёт восемь выходов, поэтому пропускная способность повышается примерно с 47,9 токена/с в модели одного запроса до 383 токенов/с, но каждый запрос по-прежнему должен ожидать завершения всего batch. **Повторное использование внутри batch увеличивает число выходов, генерируемых за единицу времени, тогда как задержка одного запроса зависит от выполнения и ожидания, через которые он проходит.**

Когда размер batch достигает определённой величины, время вычислений сравнивается со временем чтения весов. Положив $2BN/\Pi = b_W N/\beta$, получим точку перехода:

$$B_* = \frac{b_W \Pi}{2\beta}.$$

В этом примере при $b_W = 1$, $\Pi = 989,4 \times 10^{12}$ FLOP/s и $\beta = 3,35 \times 10^{12}$ bytes/s получаем $B_* \approx 147,7$. В модели, учитывающей только основные матричные вычисления и чтение весов, при небольшом batch увеличение числа запросов позволяет распределить затраты на чтение весов. После примерно 148 запросов время вычислений превышает время чтения весов, поэтому дальнейшее увеличение batch почти пропорционально увеличивает время выполнения всего batch. Эта точка перехода учитывает только мат-

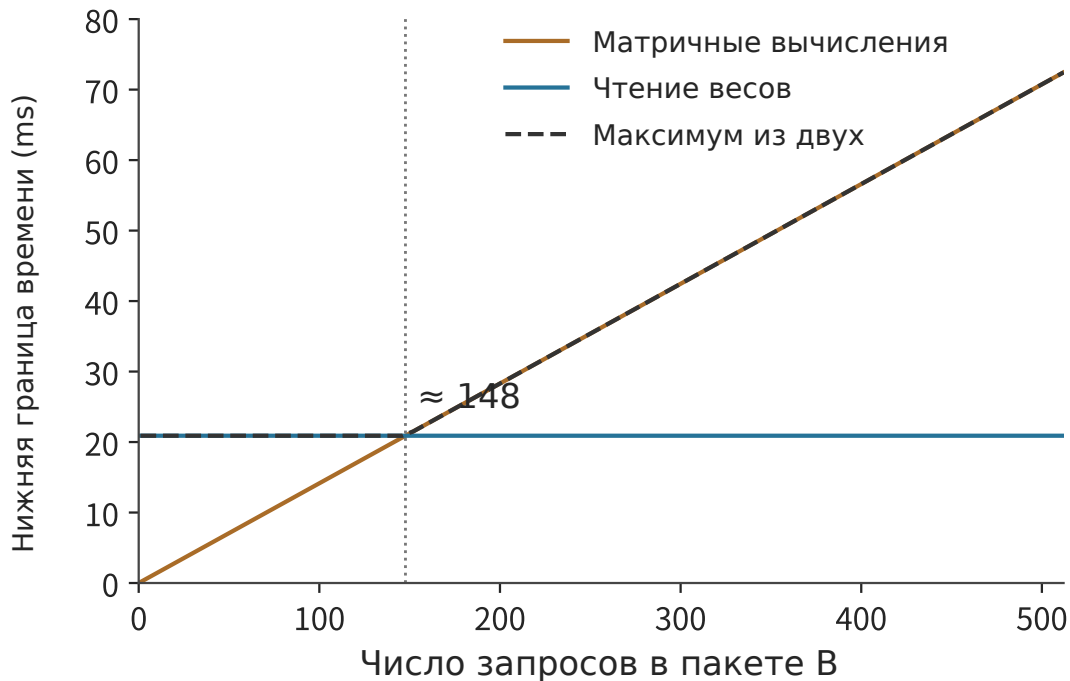


Рис. 1-11. С увеличением числа запросов в batch объём матричных вычислений растёт как $2BN$, а объём чтения весов остаётся равным 70 ГБ на batch. Примерно при 148 запросах две нижние границы сравниваются, после чего общая скорость определяется временем вычислений.

ричные вычисления и чтение весов, но не состояние контекста каждого запроса: согласно бюджету памяти из раздела 1.2.3, после размещения 73,73 ГБ весов на одной H100 SXM остаётся лишь около 6,27 ГБ, чего недостаточно для размещения KV-кэша 148 запросов.

Если разделить число выходов в batch B на показанную на предыдущем рисунке нижнюю границу времени, получим верхнюю границу пропускной способности для заданных условий. До точки перехода совместное чтение распределяется между большим числом выходов; после неё время вычислений растёт вместе с числом выходов, и кривая постепенно выходит на плато.

Ту же точку перехода можно описать через арифметическую интенсивность $I = F/R_W = 2B/b_W$, то есть число операций, выполненных на каждый считанный байт данных. Когда I меньше отношения вычислительной мощности ускорителя к его пропускной способности Π/β , больше времени занимает чтение; после превышения этого отношения больше времени занимают вычисления. Это исходная идея модели Roofline («линия крыши»), рассматриваемой в главе 4.¹¹

Упражнение 1-3 [основное]: как пропускная способность и размер batch влияют на предельное время одного шага генерации

Используя модель и ускоритель из примера 1-1, примите эффективную пропускную способность равной 70% номинальной, а эффективную вычислительную производительность — 50% пиковой. Для $B = 1, 16, 64$ вычислите нижнюю границу времени выполнения всего batch, среднее приходящееся на один выходной токен время выполнения и идеальную пропускную способность. Цель — не более 10 мс на один шаг каждого запроса: для каких

¹¹Williams, Waterman, Patterson, статья о Roofline.

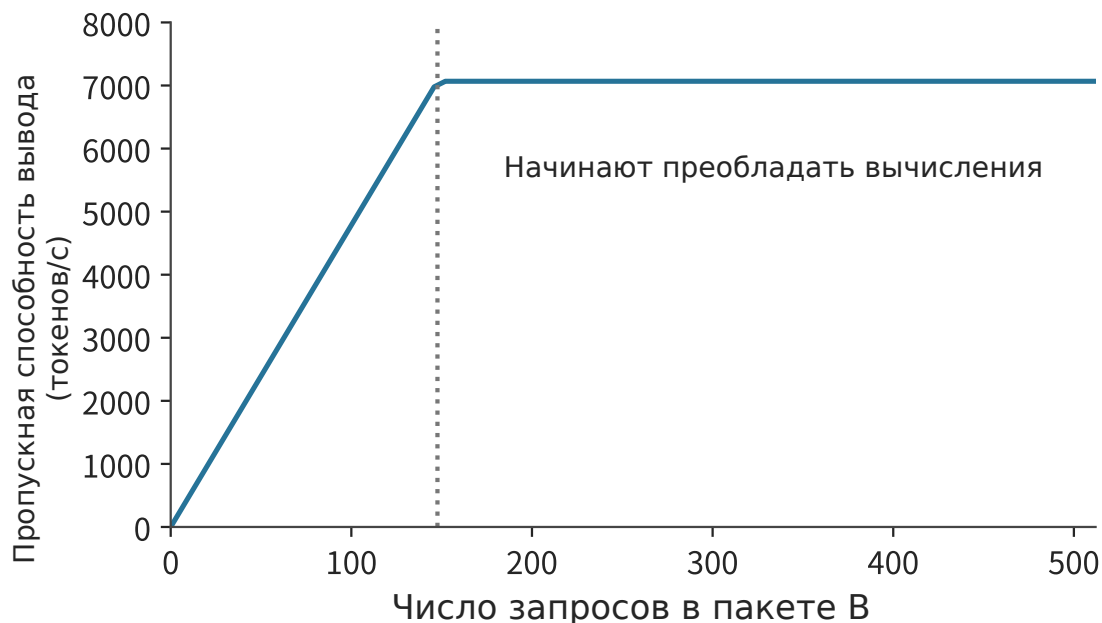


Рис. 1-12. Идеальная пропускная способность вывода описанной выше модели пакетной обработки. Кривая получается делением числа выходов в batch на нижнюю границу времени; вертикальная пунктирная линия соответствует той же точке перехода примерно при 148 запросах, что и на предыдущем рисунке.

размеров batch уже по нижней границе времени можно определить, что цель недостижима? Затем вдвое уменьшите объём чтения весов, оставив объём вычислений неизменным, и повторите оценку. Найдите размер batch B_* , при котором время вычислений равно времени чтения, и объясните, почему уменьшение приходящегося на один выход времени выполнения не означает сокращения задержки одного шага отдельного запроса.

1.3.3 Проверка оценки с помощью измерений

Предыдущая модель предсказывает две тенденции: при небольшом batch пропускная способность растёт с увеличением batch, а время выполнения всего batch ограничивается однократным чтением весов. В реальной программе каждый новый запрос дополнительно увеличивает объём доступа к контексту и вычислений. На рис. 1-13 и 1-14 приведены результаты измерений Qwen3-8B с весами BF16 на RTX PRO 6000 Blackwell Workstation Edition. Эта карта оснащена 96 ГБ видеопамяти с пропускной способностью 1,792 ТБ/с, а её пиковая производительность плотных матричных вычислений при входных данных BF16 и накоплении FP32 составляет 503,8 TFLOP/s. vLLM — предложенная в 2023 году исследователями Калифорнийского университета в Беркли и других организаций система с открытым исходным кодом для обслуживания инференса, которая организует запросы к модели и выполняет инференс. Изначально она была сосредоточена на проблеме неэффективного использования видеопамяти KV-кэшем, ограничивавшей размер batch. В этих измерениях использовалась версия 0.23 в режиме eager, то есть задания передавались ускорителю по очереди в порядке выполнения программы. Каждый запрос содержал 2048 входных токенов и генерировал

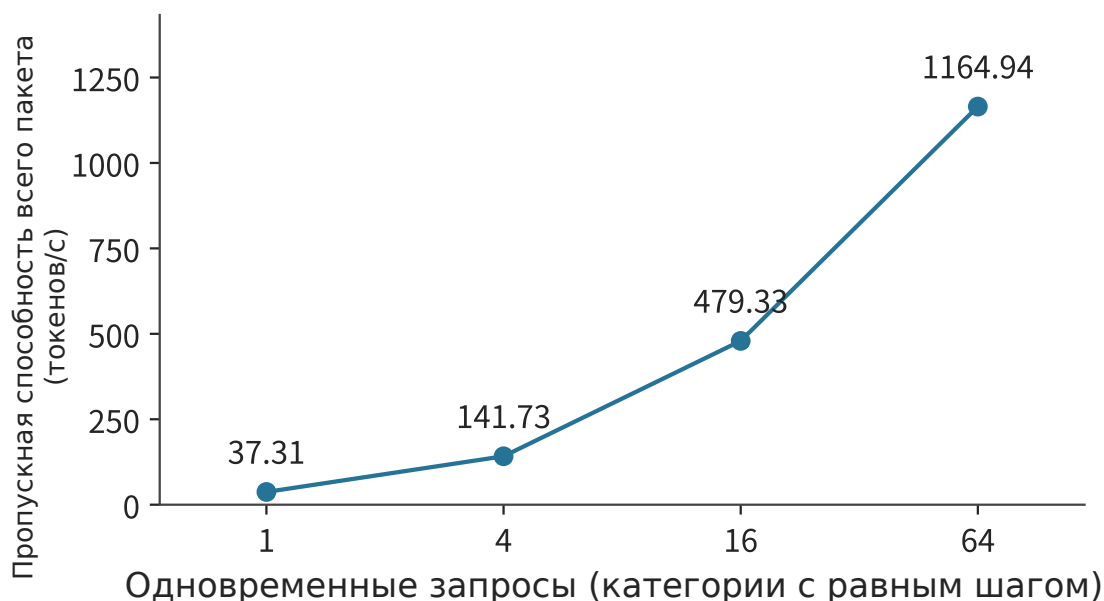


Рис. 1-13. Измеренная совокупная пропускная способность вывода Qwen3-8B. Четыре уровня числа запросов расположены с равными интервалами, вертикальная ось начинается с нуля; модель, точность, длины входа и выхода и интервал измерения остаются неизменными.

256 токенов; для каждого размера batch выполнялось по три измерения.¹² Пропускная способность означает число выходов, создаваемых за единицу времени; **время на выходной токен** (time per output token, ТРОТ) здесь означает наблюдаемый клиентом средний интервал между выходами.

При увеличении числа запросов с 1 до 64 совокупная пропускная способность возрастает примерно в 31 раз, а интервал между выходами отдельного запроса увеличивается примерно с 26,5 до 40,5 мс. Вместе два рисунка описывают компромисс пакетной обработки: ускоритель за тот же промежуток времени выполняет работу для большего числа запросов, но каждому запросу требуется больше времени для завершения одного шага генерации.

Пропускная способность на рис. 1-13 вычисляется делением совокупного числа выходов batch на общее время обработки этого batch, включая время обработки входных данных; из трёх измерений берётся медиана. Для ТРОТ сначала для каждого запроса вычисляется время между событиями первого и последнего выхода, затем оно делится на 255 интервалов между выходами, после чего берётся медиана по запросам в одном batch и, наконец, медиана по трём измерениям. Первая метрика характеризует скорость вывода всего экземпляра, вторая — воспринимаемый отдельным пользователем темп генерации. В главе 3 будет подробнее рассмотрено влияние поступления запросов, очередей и небольшого числа медленных запросов на эти метрики.

Эту тенденцию можно объяснить различием между весами и состоянием контекста. Несколько запросов совместно используют веса модели, но каждый из них хранит собственное состояние контекста. При увеличении параллелизма одна матричная операция одновременно обрабатывает векторы текущих входных токенов большего числа запро-

¹²Описание записей упражнения 1-4, результаты структурированного анализа и последующие дополнительные измерения. В основном тексте и на рисунках 1-13 и 1-14 используется первоначально записанная группа коротких входных данных.

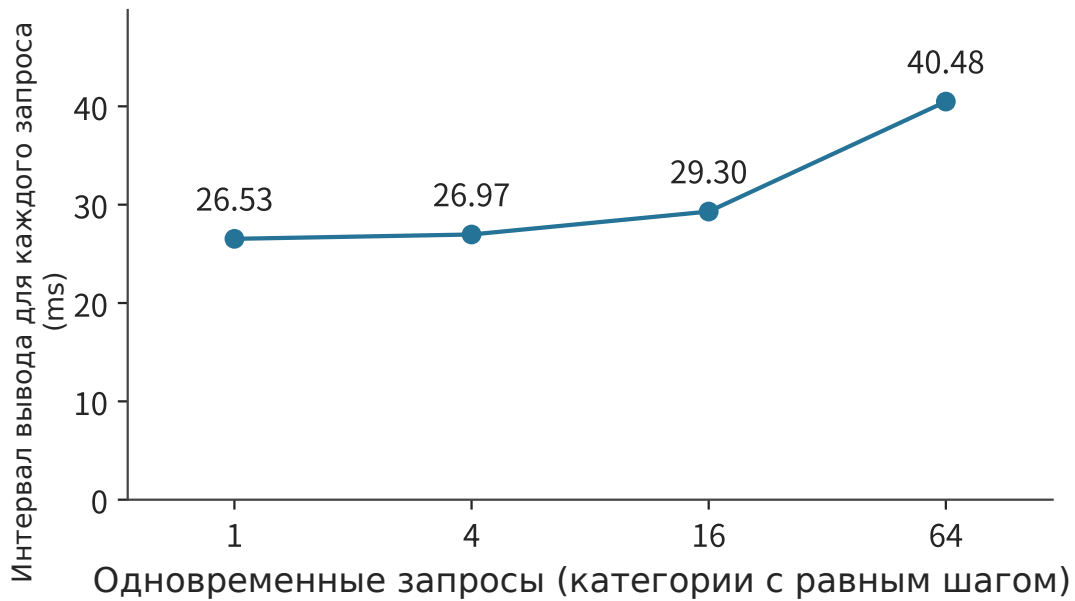


Рис. 1-14. Интервал между выходами отдельного запроса в той же серии измерений. Одновременно с ростом пропускной способности увеличивается и средний интервал для отдельного запроса; два рисунка соответственно показывают скорость вывода ускорителя и время ожидания пользователя.

сов, благодаря чему веса используются повторно чаще; в то же время объём вычислений внимания и доступа к контексту также растёт с числом запросов.

Если дополнить базовую модель этой растущей с числом запросов работой, время одного шага можно сначала записать как:

$$T_{\text{step}} \approx \max\left(\frac{BF_1}{\Pi}, \frac{R_W + BR_{\text{state},1}}{\beta}\right),$$

где F_1 — объём вычислений на запрос, а $R_{\text{state},1}$ — объём чтения и записи состояния на запрос. При увеличении batch затраты на чтение весов распределяются между большим числом выходов, но одновременно возрастает и суммарный объём чтения и записи состояния всех запросов. В следующей главе эти объёмы работы будут вычислены для реальной модели, а также будет предсказано, куда сместится точка равенства времени вычислений и чтения при увеличении длины контекста.

Упражнение 1-4 [основное]: проверка модели пакетной обработки по измеренным пропускной способности и интервалу между выходами

По значениям измерений для четырёх уровней параллелизма, указанным на рис. 1-13 и 1-14, вычислите для каждого уровня отношение пропускной способности к пропускной способности одного запроса, а также относительный рост ТРОТ. Предположите, что после увеличения параллелизма ограничением остаётся только совместное чтение весов, а вычисления и доступ к контексту не увеличивают время выполнения. Как в таком случае должна изменяться пропускная способность? Сопоставьте это с фактическими результатами, предложите два объяснения, которые можно различить измерениями, и спроектируйте измерение, в котором изменяется только длина контекста, чтобы проверить их. Укажите, какие модель, длину выхода, а также началь-

ную и конечную точки измерения необходимо оставить неизменными.

Вся эта оценка отвечала на пять вопросов: **что перемещается, в каком объёме, сколько раз, по какому пути и кто ожидает**. Структура модели определяет необходимые веса, разрядность и число параметров — объём хранения, повторное использование внутри batch изменяет число чтений, интерфейс видеопамяти задаёт пропускную способность, а зависимости выполнения определяют ожидание. Эти пять вопросов связывают структуру модели, путь данных и порядок выполнения в единый бюджет, каждый элемент которого можно вычислить отдельно.

1.3.4 Почему измерения не достигают предела: неполнота модели или системные накладные расходы

При оптимизации систем часто сравнивают новую реализацию со старой: если новый оператор на 30% быстрее старого, результат считается успешным. Но без знания предела невозможно определить, означают ли эти 30%, что реализация уже приблизилась к верхней границе, или до неё всё ещё остаётся целый порядок. Правильная отправная точка — сначала вывести из первых принципов допустимую оборудованной верхней границу, а затем определить, насколько измеренный результат от неё отстанет. Рассмотрим измерения из этой главы: при пропускной способности видеопамяти 1,792 ТБ/с чтение весов и старого KV на одном шаге decode должно занимать не менее 8,62 мс, тогда как измеренный интервал между выходами при batch, равном 1, составляет 26,5 мс, а MBU — всего 33%. У такого расхождения есть две причины, требующие совершенно разных действий. Первая — неполнота теоретической модели: приведённая выше модель учитывает только чтение весов и KV, но не учитывает векторные операции помимо внимания и семплирование. После добавления этих составляющих нижняя граница повысится, а расхождение уменьшится; в этом случае исправлять нужно модель, а не систему. Вторая причина — накладные расходы самой реализации: в данном эксперименте каждый kernel отправлялся отдельно в режиме eager, и каждая отправка проходила через Python-код на хосте, драйвер и PCIe, из-за чего ускоритель простаивал между двумя kernel; пошаговая запись журналов клиентом также занимала время хоста. Эти расходы не связаны с аппаратным пределом, и их можно устранить. Различить две причины можно только измерениями, в каждом из которых изменяется одно условие. При дополнительном измерении на той же карте с отключённым пошаговым журналированием интервал между выходами снизился до 12,1 мс; при измерении времени только по событиям ускорителя один шаг выполнения модели занял 10,7 мс, что лишь на 20% выше нижней границы 8,62 мс.¹³ Следовательно, большая часть трёхкратного расхождения обусловлена системными накладными расходами на стороне хоста. Во всех последующих главах несоответствие измерений нижней границе будет проверяться в том же порядке: сначала выясняется, не пропущены ли составляющие в модели, а затем — нет ли накладных расходов в реализации.

¹³Запись дополнительных измерений и хронометраж событий ускорителя. Во время дополнительных измерений одновременно был отключён кэш префиксов и заново сформированы входные данные, поэтому влияние пошагового журналирования невозможно полностью отделить от этих условий; события ускорителя охватывают только выполнение модели, без сэмплирования и вывода.

1.4 Как требования определяют проектирование архитектуры

В предыдущих оценках аппаратное обеспечение рассматривалось как заданное условие; однако и само это оборудование проектируется для удовлетворения конкретных требований. Изменения архитектуры в основном обусловлены двумя типами факторов: изменением требований, например ростом масштаба бизнеса или появлением новых нагрузок, и изменением аппаратного обеспечения, например более быстрым ростом производительности одного типа устройств по сравнению с другими. Оба типа изменений в конечном счёте проявляются как изменение соотношений. Если меняется соотношение между объёмом вычислений и объёмом чтения данных, пропускной способностью и вычислительной мощностью, ростом потребностей и возможностями одного чипа, то прежнее разумное разделение задач может перестать быть выгодным. Сначала в этом разделе рассматриваются три примера: требования к инференсу речи, с которыми Google столкнулась в 2013 году, требования к масштабированию облачной сети Azure в Microsoft в 2015–2016 годах и требования Huawei к организации вычислений и хранения данных на нескольких устройствах по мере развития больших моделей. Затем объясняется, как такие изменения последовательно распространяются по уровням AI-системы.

1.4.1 TPU

TPU — разработанный Google тензорный процессор для вычислений нейронных сетей; его первое поколение обозначается как TPU v1. В статье, представляющей это поколение, описывается одна из оценок потребностей. Google давно рассматривала применение в дата-центрах GPU, FPGA с аппаратной логикой, настраиваемой после развёртывания, и специализированных чипов, однако для некоторых ранних приложений было достаточно свободных ресурсов существующих дата-центров. В 2013 году прогноз использования голосового поиска изменил ход обсуждения: если каждый пользователь будет пользоваться голосовым поиском по три минуты в день, то для удовлетворения вычислительных потребностей может понадобиться удвоить масштаб дата-центров. Поэтому Google приступила к разработке специализированного чипа для инференса. TPU v1 начали развёртывать в дата-центрах в 2015 году, а статья об архитектуре была опубликована в 2017 году.¹⁴

Эту оценку можно выразить простой формулой: обозначим суточное число запросов через Q , а требуемый объём вычислений на один запрос — через F_1 ; тогда совокупная суточная потребность составит QF_1 . По мере роста числа запросов свободные мощности имеющегося оборудования в конце концов исчерпываются; тогда можно либо добавить серверы общего назначения, либо разработать специализированный процессор. Второй вариант требует первоначальных затрат на разработку, но может сократить время использования оборудования и энергопотребление на запрос; чем больше масштаб сервиса, тем легче накопленной экономии компенсировать первоначальные вложения.

В специализированном процессоре можно выделить больше вычислительных блоков для постоянно повторяющихся матричных операций, а затем подавать им входные данные через внутрикристальные буферы и тракты данных. Требования к времени отклика и энергопотреблению определяют распределение ресурсов вычислений, хранения и

¹⁴Jouppi и др., статья о TPU v1, раздел 2 с описанием происхождения, архитектуры и реализации. Голосовой поиск продолжительностью три минуты — исторический прогноз требований, приведённый в статье.

Больше ресурсов для специализированных вычислений и переноса данных

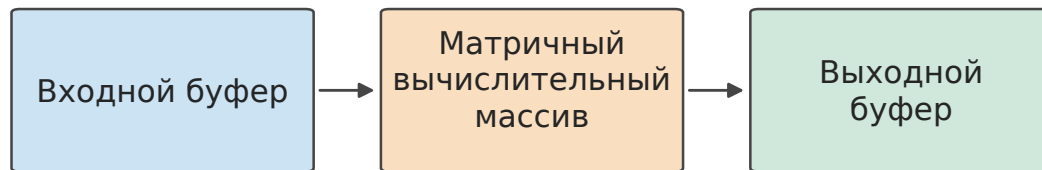


Рис. 1-15. Специализированный процессор организует вычислительный массив и буферы ввода-вывода вокруг повторяющихся матричных операций. Буфер — это область памяти для временного хранения данных, ожидающих вычисления или уже обработанных; три прямоугольника и соединяющие их стрелки показывают направления перемещения данных.

связи. Так изменение масштаба приложения воплощается в конструкции чипа.

В главе 4 будут представлены матричный массив, буферы и тракты данных TPU. Когда суточная вычислительная потребность QF_1 приближается к свободной вычислительной мощности дата-центра, специализированный процессор может оказаться выгоднее процессора общего назначения.

1.4.2 SmartNIC

При проектировании в 2015 году нового поколения облачных серверов с пропускной способностью 40 Gbit/s Microsoft Azure столкнулась с другим типом роста: пропускная способность сети увеличивалась, а политики виртуальной сети становились всё сложнее. **SmartNIC (интеллектуальная сетевая карта)** — это устройство, способное выполнять часть обработки данных непосредственно на сетевой карте. С конца 2015 года Microsoft устанавливала SmartNIC на базе FPGA в новые серверы Azure, а в 2016 году предоставила клиентам сервис Accelerated Networking (ускоренная сеть). Аппаратную логику FPGA можно перенастраивать, поэтому обработку сетевого трафика можно изменять вслед за обновлением политик.¹⁵

Проект должен был одновременно удовлетворять двум требованиям: сохранять больше ядер CPU для клиентских приложений и обеспечивать быстрое обновление правил сетевой изоляции, пересылки и контроля доступа. Обработка сетевого трафика на облачном сервере использует именно эти ядра CPU: помимо выполнения приложений, CPU должен обрабатывать отправляемые и принимаемые пакеты. Виртуальная машина — это изолированная вычислительная среда, выделенная на физическом хосте. Когда несколько виртуальных машин совместно используют физическую сеть, платформа должна определять принадлежность данных, находить правила пересылки, выполнять инкапсуляцию и поддерживать изоляцию.

¹⁵Firestone и др., Microsoft, Azure Accelerated Networking: SmartNICs in the Public Cloud, NSDI 2018; текст исходной статьи. В аннотации указано развёртывание в конце 2015 года и предоставление сервиса клиентам в 2016 году; в разделе 3 описаны цели разработки: снижение загрузки CPU, сохранение программируемости и поддержка более высокой пропускной способности.

Обработка пакетов по пути данных

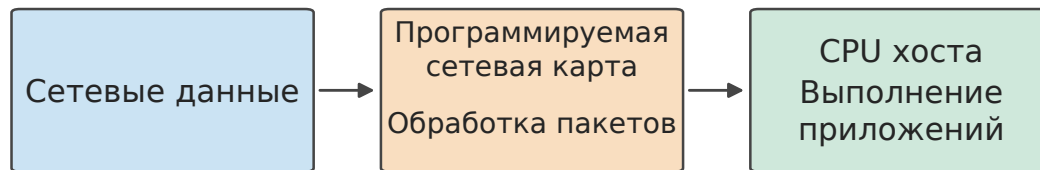


Рис. 1-16. Программируемая сетевая карта выполняет заданную обработку пакетов до поступления данных на хост. Сплошная линия показывает движение данных; обработка на сетевой карте сокращает объём вспомогательной работы CPU хоста.

Сначала оценим, сколько ядер CPU занимает эта работа. При минимальном размере кадра через канал 40 Gbit/s необходимо обрабатывать около шестидесяти миллионов пакетов в секунду. Если одно ядро обрабатывает от десяти до двадцати миллионов пакетов в секунду, то для непрерывной пересылки потребуется примерно

$$n_{\text{core}} = \frac{\lambda_{\text{packet}}}{\mu_{\text{core}}} \approx 3-6.$$

Здесь λ_{packet} — частота поступления пакетов, а μ_{core} — скорость их обработки одним ядром. При увеличении пропускной способности канала λ_{packet} растёт, тогда как μ_{core} не увеличивается синхронно, поэтому требуемое число ядер растёт пропорционально отношению этих величин. Эти ядра снова и снова выполняют один и тот же процесс обработки пакетов; перенеся соответствующую работу на сетевую карту, можно освободить время CPU для приложений.¹⁶

Подход Azure заключается в том, что программное обеспечение хоста управляет сложными политиками, а правила обработки пакетов, подходящие для многократного исполнения, передаются FPGA. Обработка выполняется непосредственно при прохождении данных через сетевую карту, и CPU может уделять больше времени клиентским приложениям. Ключевой задачей проектирования становится сочетание возможности программного обновления со скоростью аппаратной обработки.

После переноса обработки пакетов на сетевую карту часть операций можно выполнять непосредственно по мере поступления данных, что уменьшает нагрузку на CPU. Если сетевой карте также нужны данные из памяти хоста, она должна читать их через PCIe; в этом случае скорость чтения определяется задержкой ответа и числом запросов, которые можно отправить одновременно. В главе 7 такой доступ будет проанализирован на примере KV-Direct — системы, позволяющей программируемой сетевой карте непосредственно обрабатывать запросы к хранилищу ключей и значений. Такое хранилище

¹⁶ Докторская диссертация автора, базовый вариант простой пересылки в разделе 4.2.1; пересчёт бюджета ядер. Около шестидесяти миллионов пакетов в секунду получено расчётом для 40 Gbit/s при занятии линии каждым кадром в течение передачи 84 байтов; 3–6 ядер соответствуют базовому варианту простой пересылки.

находит, читает или обновляет соответствующее значение по уникальному ключу.

1.4.3 Unified Bus

SmartNIC меняет разделение задач внутри одного сервера. Если же проблема заключается в недостаточном совокупном объёме ресурсов отдельного ускорителя, необходимо дополнительно рассмотреть организацию совместной работы нескольких ускорителей. Когда модель и состояние её выполнения не помещаются на одном ускорителе, один из наиболее очевидных вариантов — добавить ускорители. Однако между «увеличением общей ёмкости» и «эффективным выполнением задачи» остаются вопросы разделения работы, обмена данными и синхронизации.

Например, суммарного объёма видеопамяти двух карт может быть достаточно для хранения весов, но выполняющий вычисления ускоритель должен иметь доступ к необходимым данным. Если один шаг вычислений зависит от результата другой карты, придётся ждать его передачи; если несколько карт совместно выполняют одну операцию, потребуется организовать их взаимодействие. Увеличение числа ускорителей добавляет ресурсы, но одновременно увеличивает объём работы по их соединению.

Развиваемая Huawei для Ascend и других гетерогенных вычислительных устройств технология **UB (Unified Bus, единое межсоединение)** расширяет эту задачу до совместной работы множества устройств. Автор участвовал в этом проекте в 2020–2023 годах, а соответствующие исследования начались ещё в 2019 году — до выпуска компанией OpenAI авторегрессионной языковой модели GPT-3 в 2020 году. После того как GPT-3 продемонстрировала возможности больших моделей, отрасль шире осознала потребность во взаимодействии множества устройств, вследствие чего инвестиции в проект увеличились. Требуемая для обучения моделей вычислительная мощность растёт на много быстрее производительности отдельного ускорителя; отношение этих величин постоянно увеличивается, а значит, совместно работающих ускорителей должно становиться всё больше.

Основной задачей в этот период было обеспечить использование нескольких ускорительных карт постоянно растущими моделями и упростить доступ вычислительных устройств к памяти и данным на других устройствах. Как только данные пересекают границу хоста, программному обеспечению приходится переходить на интерфейс передачи сообщений, заново организовывать буферы, а затем проходить через драйвер сетевой карты и стек протоколов; каждый дополнительный уровень абстракции добавляет задержку. UB позволяет устройствам напрямую обращаться к памяти других устройств, устраняя эти уровни абстракции и приближая время удалённого доступа к нижней границе, определяемой задержкой линии связи, а верхним уровням предоставляет возможность более гибко организовывать ресурсы. Единый механизм доступа позволяет устройствам напрямую использовать ресурсы в более широких пределах, а топология определяет расстояние до этих ресурсов и пропускную способность доступа. Поэтому разделение модели и проектирование межсоединения оказываются тесно связанными. В разделе 6.5.5 организация UB рассматривается с точки зрения масштаба суперузла, а в разделах 7.3 и 7.4 по пути одного удалённого обращения выводятся его задержка, частота запросов и состояние соединения, а также рассчитывается, сколько времени занимал каждый из устранённых уровней абстракции.

Рассмотрим в качестве примера развёртывание модели на двух картах из этой главы: равномерное распределение весов позволяет выполнить ограничение на ёмкость каж-

Прямой обмен нужными данными между устройствами

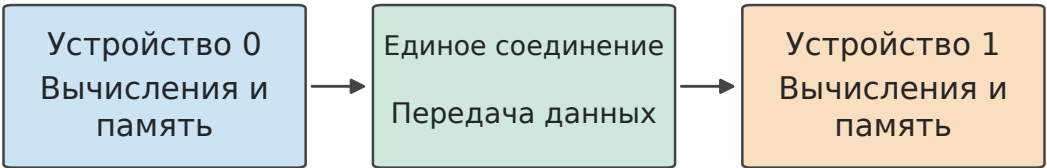


Рис. 1-17. Единое межсоединение объединяет вычислительные ресурсы и ресурсы хранения различных устройств. Разделение модели определяет, какими данными необходимо обмениваться, а межсоединение доставляет их устройствам, которые будут использовать эти данные далее.

дой карты, но на каждом этапе всё равно приходится ждать входных данных и результатов предыдущего этапа. Добавление ускорителей изменяет M_{cap} и доступную вычислительную мощность, но также создаёт новый объём обмена данными и новые зависимости. Поэтому на первом шаге нужно проверить, может ли каждый ускоритель вместить необходимые данные, на втором — рассчитать объём вычислений, чтения и записи для каждого ускорителя, а на третьем — вычислить общее время с учётом зависимостей выполнения. Как бы ни росли модель и число ускорителей, этот порядок анализа остаётся неизменным.

1.4.4 Последовательное распространение изменений

Решения из трёх рассмотренных примеров после развёртывания сами создают новые ограничения. После установки FPGA на серверы Azure облачная платформа должна обеспечивать обновление логики FPGA и обработку сбоев¹⁷; после объединения множества ускорителей посредством UB при каждой синхронизации приходится ждать больше устройств, а число компонентов, способных отказать, также увеличивается. В AI-системах такое последовательное распространение продолжается; в следующей таблице основные этапы перечислены в порядке глав книги.

Изменение	Предположение, которое перестаёт выполняться	Главы
Время выполнения нейронной сети сосредоточено в матричных операциях	Процессор общего назначения способен выполнять основные вычисления при приемлемой площади и энергопотреблении	Главы 4 и 5
Веса модели превышают объём видеопамяти одной карты	Одну модель можно разместить на одной карте	Глава 6

¹⁷Firestone и др., Microsoft, Azure Accelerated Networking: SmartNICs in the Public Cloud, NSDI 2018; текст исходной статьи. В аннотации указано развёртывание в конце 2015 года и предоставление сервиса клиентам в 2016 году; в разделе 3 описаны цели разработки: снижение загрузки CPU, сохранение программируемости и поддержка более высокой пропускной способности.

Изменение	Предположение, которое перестаёт выполняться	Главы
Обучение и инференс синхронно выполняются на нескольких серверах	Сетевой трафик состоит из независимых потоков данных, допускающих статистическое мультиплексирование	Глава 7
При декодировании каждое чтение весов сопровождается лишь небольшим объёмом вычислений	Скорость выполнения определяется пиковой вычислительной мощностью	Глава 8
Контекст становится длиннее, а объём KV приближается к объёму весов или даже превышает его	Состояние контекста невелико, и его всегда можно вычислить заново	Глава 9
Масштаб обучения достигает тысяч карт	Сбои происходят лишь изредка	Глава 10
Нагрузки AI-агентов и RL пакетно создают окружения инструментов	Программы в контейнерах независимы друг от друга и каждая определяет собственный темп выполнения	Глава 11
Периферийные устройства способны выполнять модель	Сосредоточение вычислений в облаке всегда сокращает время выполнения	Глава 12

По сравнению с операционными системами общего назначения и облачными платформами AI-системы обладают одним преимуществом: типов нагрузки немного, и они известны заранее. Платформы общего назначения должны выполнять заранее неизвестные программы, поэтому могут полагаться только на унифицированные абстракции, строгую изоляцию и статистическое мультиплексирование. AI-системы проектируются вокруг небольшого числа моделей и детерминированных процессов выполнения, поэтому можно заранее определить описанные в разделе 1.1.1 вычислительные зависимости и зависимости по данным, а затем на их основе нарушить прежние границы абстракций. Например, платформа песочниц RL из главы 11 получает от фреймворка обучения информацию о вытеснении и заблаговременно приостанавливает соответствующие песочницы.

Многие отдельные технологии в AI-системах не являются новыми изобретениями: постраничное управление KV в главе 8 заимствует механизм страниц у операционных систем, размещение песочниц в главе 11 использует предложенный в 2001 году рандомизированный алгоритм power-of-k-choices, а освобождение памяти виртуальных машин основано на предложенном в 2002 году механизме balloon.¹⁸ После изменения соотношений ранее невыгодные или не востребуемые методы снова становятся целесообразными; применимость каждой технологии по-прежнему необходимо заново рассчитывать для текущих соотношений.

От инференса речи к облачной сети, а затем к выполнению больших моделей на множестве устройств — изменения требований и аппаратного обеспечения определяют, какое узкое место необходимо устранить первым, а также заново распределяют работу между аппаратным обеспечением, программным обеспечением и соединениями. Если сопоставить эти примеры с приведённой в начале схемой из шести уровней, все эти решения по-новому организуют вычисления и перемещение данных. Через последующие главы проходят три сквозные темы: изменение соотношений объясняет, почему требу-

¹⁸Описание power-of-k-choices см. в M. Mitzenmacher, The Power of Two Choices in Randomized Load Balancing, IEEE TPDS, 2001; механизм balloon — в C. A. Waldspurger, Memory Resource Management in VMware ESX Server, OSDI 2002. Обе работы цитируются в статье DSec, а соответствующие механизмы рассматриваются в разделах 11.2.2 и 11.3.2.

ется новое решение; перемещение данных показывает, что именно оно изменило; физические пределы из раздела 1.3.4 позволяют оценить достигнутый результат. Отсюда следует главная тема книги: **перемещение данных формирует архитектуру AI Infra**.

1.5 Заблуждения и ловушки

Заблуждение: если удвоить пиковую вычислительную мощность, скорость выполнения также удвоится. Сначала проверьте, ограничено ли текущее выполнение вычислительной мощностью. В задаче 1-1 нижняя граница времени чтения весов намного больше нижней границы времени матричных вычислений, поэтому повышение пиковой производительности матричных вычислений почти не влияет на нижнюю границу времени выполнения. Если оптимизируется лишь часть всей задачи, верхнюю границу выигрыша также следует проверить по закону Амдала.

Заблуждение: если видеопамяти хватает для весов, значит, она поддержит целевое число параллельных запросов. Веса — лишь часть постоянно размещённых данных. Состояние контекста, рабочая область и пространство, зарезервированное средой выполнения, вместе занимают видеопамять; при использовании нескольких ускорителей расчёт необходимо выполнять отдельно для каждого из них.

Заблуждение: если реализация стала на 30% быстрее предыдущей, оптимизацию можно считать успешной. Не зная предела, допускаемого аппаратным обеспечением, невозможно определить, означают ли эти 30% приближение к верхней границе или до неё по-прежнему не хватает целого порядка величины. Следует сначала вычислить предел, а затем в порядке, указанном в разделе 1.3.4, выяснить, вызван ли разрыв неучтёнными составляющими модели или системными накладными расходами.

Заблуждение: систему, хорошо зарекомендовавшую себя в сценариях предыдущего поколения, можно напрямую использовать для новой нагрузки. Любая архитектура системы опирается на соотношения, принятые при её проектировании. Универсальные контейнерные платформы предполагают, что запросы поступают от независимых друг от друга арендаторов, а программы сами определяют темп выполнения. Но при RL-обучении песочницы создаются пакетами, а темп задаётся со стороны GPU; прямое заимствование прежней архитектуры заставит дорогостоящие ускорители простаивать в ожидании (глава 11). Прежде чем повторно использовать архитектуру, следует проверить, сохраняются ли соотношения, на которых она основана.

Заблуждение: если пропускная способность повысится, время ожидания каждого пользователя сократится. Повторное использование данных внутри пакета снижает приходящиеся на каждый результат затраты на чтение, однако каждый запрос всё равно должен пройти через очередь и выполнение всего пакета. Пропускную способность и время отклика следует указывать вместе.

1.6 Краткий справочник чисел для этой главы

В таблице сведены ключевые показатели модели Qwen3-8B, использованной в измерениях раздела 1.3.3. На этапе prefill уже имеющиеся входные данные обрабатываются за один раз и получается первый результат; при каждом шаге decode в модель передаётся один новый токен, после чего генерация продолжается с использованием сохранённого KV. В таблице зафиксированы веса, активации и KV в формате BF16 при числе одновре-

менных запросов 1; на вход prefill подаются 2048 токенов, а decode выполняет один шаг после контекста из 2048 токенов.¹⁹

Требования модели	Значение	Как использовать
Полные веса BF16	16.38 GB	Определить объём видеопамати, необходимый для загрузки модели
Полный KV модели на каждый token контекста	144 KiB	Умножить длину контекста на это число, чтобы получить объём KV для одного запроса
KV контекста из 2048 токенов	288 MiB	Около 0.302 GB; увеличивается с числом независимых запросов
Матричные операции prefill для 2048 токенов	29.69 TFLOPs	Разделить на соответствующую производительность матричных вычислений, чтобы получить нижнюю границу времени вычисления
Матричные операции одного шага decode	16.34 GFLOPs	На вход подаётся один token, также выполняется обращение к существующему контексту
Основной объём чтения весов за один шаг decode	15.14 GB	Весы считываются один раз; из слоя эмбедингов — таблицы поиска, преобразующей номер токена в вектор, — считывается только строка текущего токена
Чтение прежнего KV за один шаг decode	0.302 GB	Число байтов при однократном чтении KV каждого контекста

Оценка нижней границы времени выполнения по объёму вычислений и чтения. Здесь используется RTX PRO 6000 Blackwell Workstation Edition, на которой проводились измерения для рисунков 1-13 и 1-14; сначала отдельно рассчитываются матричные операции и чтение данных. При пиковой производительности матричных вычислений 503.8 TFLOP/s матричные операции prefill занимают около 58.9 ms, а decode — около 0.0324 ms; при пропускной способности видеопамати 1.792 TB/s чтение основных весов и прежнего KV за один шаг decode занимает около 8.62 ms. Измеренный интервал вывода при batch, равном 1, составляет около 26.5 ms — примерно втрое больше этой нижней границы чтения; причины расхождения рассматриваются в разделе 1.3.4. Здесь по отдельности оценивается минимальное время, необходимое для вычислений и чтения; при полном выполнении к нему также добавляются другие операции, фактические обращения к памяти и время запуска в соответствии с порядком операторов. В главах 2–5 эти расчёты будут последовательно построены по отдельным составляющим.

Эти числа объясняют различия между двумя этапами: prefill обрабатывает большой объём входных данных за один вызов, поэтому объём матричных вычислений велик; decode для одного запроса каждый раз обрабатывает лишь один token, но при этом всё равно должен считывать большой объём весов и контекста. Повышение вычислительной производительности, увеличение пропускной способности и расширение повторного использования данных внутри batch влияют на разные составляющие времени.

¹⁹Исходный код пересчёта справочных чисел вычисляет матричные операции prefill и decode на основе фиксированной конфигурации Qwen3-8B; объём чтения взят из расчёта повторного использования весов внутри batch. Объём весов включает полную таблицу эмбедингов, однако при пошаговой генерации считывается только строка эмбединга, соответствующая текущему токenu; время чтения и записи рассчитано для идеального случая, когда к каждому основному набору данных обращаются один раз.

1.7 Итоги главы

Приложения выражают поведение с помощью моделей и контекста, поэтому многие задачи используют схожие базовые вычисления. Понимание вычислительных зависимостей в модели и времени нахождения состояния в памяти позволяет комплексно рассматривать проектные решения, прежде относившиеся к разным уровням: изменение представления данных, объединение этапов выполнения, повторное использование результатов подготовительной работы или сокращение времени передачи данных и синхронизации между ускорителями. В следующих главах будет количественно оценён выигрыш от этих изменений, показано, какие проектные компромиссы вследствие этого меняются, а затем с учётом новых ограничений будут выбраны модель и системное решение.

При анализе системы сначала проверьте, помещаются ли данные в память, затем оцените время выполнения по объёму вычислений и операций чтения и записи, а в конце проанализируйте перекрытие операций и ожидание с учётом порядка выполнения. Нижняя граница, рассчитанная по пиковым показателям, представляет собой физический предел оборудования, а отношение нижней границы к фактически измеренному значению — коэффициент утилизации; расхождение между ними означает, что либо в модели не учтён какой-то фактор, либо в системе есть устранимые накладные расходы. В примере с моделью 70B чтение весов занимает примерно в 148 раз больше времени, чем матричные вычисления, поэтому увеличить пропускную способность, сократить объём чтения и повысить повторное использование данных полезнее для уменьшения времени выполнения. Совместное использование данных внутри батча может повысить пропускную способность, но время ответа на каждый запрос также включает выполнение всего батча и ожидание в очереди.

Ключевые упражнения этой главы — 1-2, 1-3 и 1-4: они посвящены соответственно преобразованию единиц и оценке осуществимости, оценке ресурсов после изменения условий, а также сравнению прогнозируемых и фактически измеренных результатов. В следующей главе на основе реальной конфигурации модели будут уточнены оценки объёма вычислений $2N$ и объёма чтения весов, а для этих расчётов будут приведены конкретные размеры матриц и состояния.

Глава 2

Архитектура модели

В главе 1 выполнение модели анализировалось с трёх точек зрения: объёма памяти, количества вычислений и объёма считываемых данных. Нижняя граница времени оценивалась с помощью ключевых показателей аппаратного обеспечения. Такая оценка позволяет быстро исключить некоторые варианты, однако оставляет открытым вопрос: откуда берутся подставляемые значения количества вычислений и объёма данных? Формулировка «модель 8B» (суффикс B в количестве параметров означает миллиард, то есть 8B — примерно 8 миллиардов параметров) указывает лишь приблизительное количество параметров, но не объясняет, какие параметры используются для каждого токена, где хранится контекст и в каком порядке выполняются различные операции.

В этой главе мы начнём с числового представления одного токена и последовательно разберём, как после прохождения через слои сети вычисляются оценки следующего токена, а затем рассчитаем необходимые для этого объём памяти M , количество вычислений F и объём чтения и записи R . В графе вычислений узлы обозначают операции, а рёбра — данные, передаваемые между ними. Только разобравшись с операциями и данными графа, можно понять происхождение каждого числа в бюджете ресурсов.

Этот граф вычислений также содержит информацию, необходимую для оптимизации реализации. Из структуры модели можно определить, какие веса остаются неизменными между запросами, какие состояния многократно используются при последующей генерации, а какие результаты нужны только непосредственно следующей операции. При проектировании хранения, компиляции и планирования в последующих главах эти различия в назначении данных помогут решить, какие данные можно совместно использовать, какие необходимо хранить длительное время, а какие достаточно сохранять временно.

В этой главе рассматриваются пять типичных моделей: DeepSeek V4.1 Flash, Qwen3-8B, Qwen3.6-35B-A3B (35B в названии означает около 35 миллиардов параметров всего, а A3B — около 3 миллиардов параметров, фактически активируемых для каждого токена; далее — Qwen3.6), DeepSeek V4-Flash и Kimi K3. Выпущенная в сентябре 2026 года V4.1 Flash использует архитектуру CED для разделения обработки входных данных и генерации, а также сочетает межслойное совместное использование KV, иерархическую индексацию и Engram — модуль, который по n -грамме, то есть фрагменту из последовательных n токенов, получает дополнительное представление из таблицы. Эта модель служит новым примером того, как архитектура модели изменяет системные требования.

Вывод формул начинается с Qwen3-8B: она содержит около 8,19 миллиарда параметров и обрабатывает текст сетью из 36 слоёв. Разобрав выполнение этой плотной модели, мы затем последовательно изменим способ хранения контекста, набор параметров,

используемых для каждого токена, и связи между слоями, после чего сравним пять моделей при одинаковых входных условиях.

Перед началом вывода договоримся о принятых в этой главе правилах подсчёта. При расчёте количества вычислений одна операция умножения и одна операция сложения считаются за 2 FLOPs. Для объёма памяти используются GB и GiB: $1 \text{ GB} = 10^9 \text{ bytes}$, $1 \text{ GiB} = 2^{30} \text{ bytes}$. Каждый элемент BF16 занимает 2 байта, поэтому размер тензора в памяти получается умножением количества его элементов на 2.

2.1 Вычислительный граф и прямой проход

Тензор — это массив чисел, упорядоченных по одному или нескольким измерениям: вектор имеет одно измерение, а матрица — два. Модель представляет характеристики каждого токена вектором, а векторы нескольких токенов располагает по строкам матрицы. Чтобы понять требования модели к ресурсам, прежде всего необходимо разделить вычислительные узлы и зависимости данных. Узел выполняет матричное умножение или другое преобразование, а ребро показывает, что последующему узлу необходим предыдущий результат. Объём операций узла определяет вычислительные требования, тензор на ребре — объём передаваемых данных, а направление ребра указывает, какие операции должны выполняться последовательно. Для понимания процесса выполнения модели необходимо учитывать все три аспекта.

Вычислительный граф в этой главе прежде всего описывает математические операции и зависимости данных. При фактическом выполнении фреймворк передаёт эти операции программам ускорителя; одна операция может быть разбита на несколько программ, а несколько операций могут быть объединены для совместного выполнения. Ребро, представляющее тензор в вычислительном графе, означает лишь, что последующей операции нужны эти данные, но не означает, что они обязательно должны копироваться между CPU и GPU. В главе 4 объясняется, где размещаются данные, а в главе 5 — как отправляются программы и как передаются входные и выходные данные.

2.1.1 Последовательные зависимости и причинный вычислительный граф

Чтобы рассчитать время выполнения, сначала нужно определить, какие операции могут начинаться одновременно. Рассмотрим четыре токена и трёхслойную сеть: по горизонтальной оси отложен индекс позиции токена в последовательности, по вертикальной — номер слоя. В этой главе термин «лексема» обозначает один элемент словаря, а термин «токен» — одно её вхождение в последовательность; если одна и та же лексема встречается в последовательности несколько раз, они различаются по индексам позиций. Рекуррентная нейронная сеть (RNN) передаёт состояние предыдущей позиции токена следующей, а состояние токена удовлетворяет равенству $h_t = f(x_t, h_{t-1})$. Здесь x_t — текущие входные данные, h_t — состояние, сохранённое после обработки текущей позиции токена, а f — преобразование с использованием весов модели. Текущий результат зависит от предыдущей позиции токена, поэтому позиции внутри одного слоя обрабатываются последовательно. Даже если все четыре входных токена уже поступили, токен с номером t должен ждать токена с номером $t - 1$.

Transformer использует механизм внимания, позволяющий одному токenu считывать информацию из других позиций; «причинность» означает, что текущей позиции токена доступна информация только о ней самой и о предшествующих позициях. Зависимости

На том же слое сначала завершается предыдущий токен

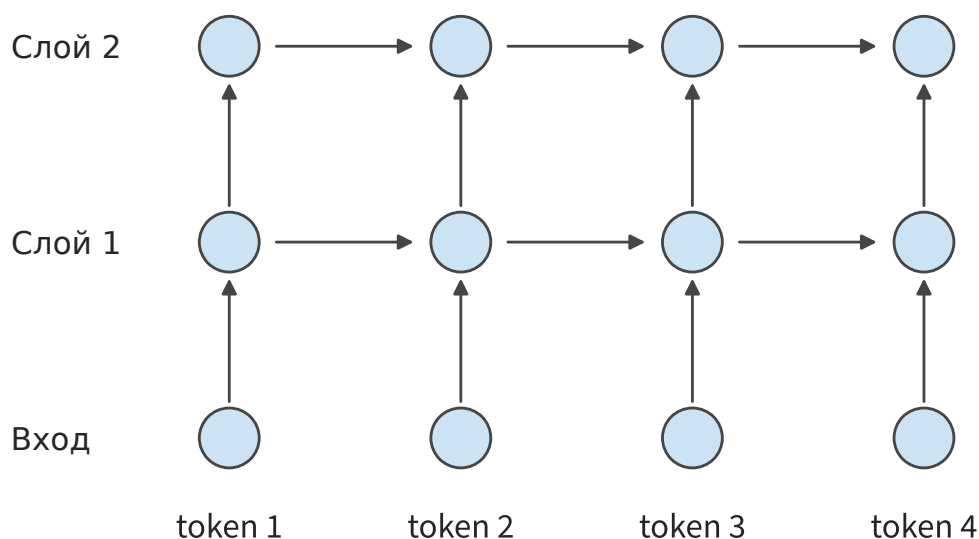


Рисунок 2-1. Зависимости в рекуррентной нейронной сети. Точки одинакового цвета обозначают вычисления для соответствующих позиций токенов, стрелки указывают на узлы, которым нужен предыдущий результат; позиции токенов в одном слое последовательно обрабатываются по горизонтали.

причинного Transformer устроены иначе. При вычислении некоторой позиции в слое l механизм внимания считывает представления этой и всех предшествующих позиций токенов из предыдущего слоя. Если входные токены известны и соответствующие представления предыдущего слоя уже вычислены, позиции одного слоя можно вычислять, не дожидаясь выхода предыдущего токена в этом же слое. Причинная маска определяет, к какой информации разрешён доступ каждой позиции токена, и скрывает информацию после текущей позиции. Если расположить запросы и опрашиваемые позиции токенов по осям, маска имеет треугольную форму; она ограничивает источники информации, но не требует обрабатывать все известные токены по одному. Зависимость по глубине сети сохраняется: следующий слой использует результаты текущего.

Проследив путь по стрелкам на рисунке, можно различить два вида ожидания: RNN в текущем слое ожидает предыдущую позицию токена, а причинный Transformer в следующем слое ожидает необходимые результаты предыдущего слоя. Возможность параллельно обрабатывать известные входные данные определяется этими зависимостями, а не самим понятием «последовательность». Генерация нового токена добавляет ещё одну зависимость: входные данные следующего вызова определяются выходом текущего.

Обработка известных входных данных и пошаговая генерация — два разных вида работы. Кодировщик преобразует известные входные данные в представления, а декодировщик пошагово создаёт выходные данные на основе доступной ему информации. Современные причинные языковые модели часто используют архитектуру decoder-only. В этой главе сначала закладываются основы на примере такой архитектуры, а затем рассматривается, как V4.1 Flash перераспределяет работу по обработке входных данных и генерации с помощью архитектуры CED. При обработке известных входных данных по-

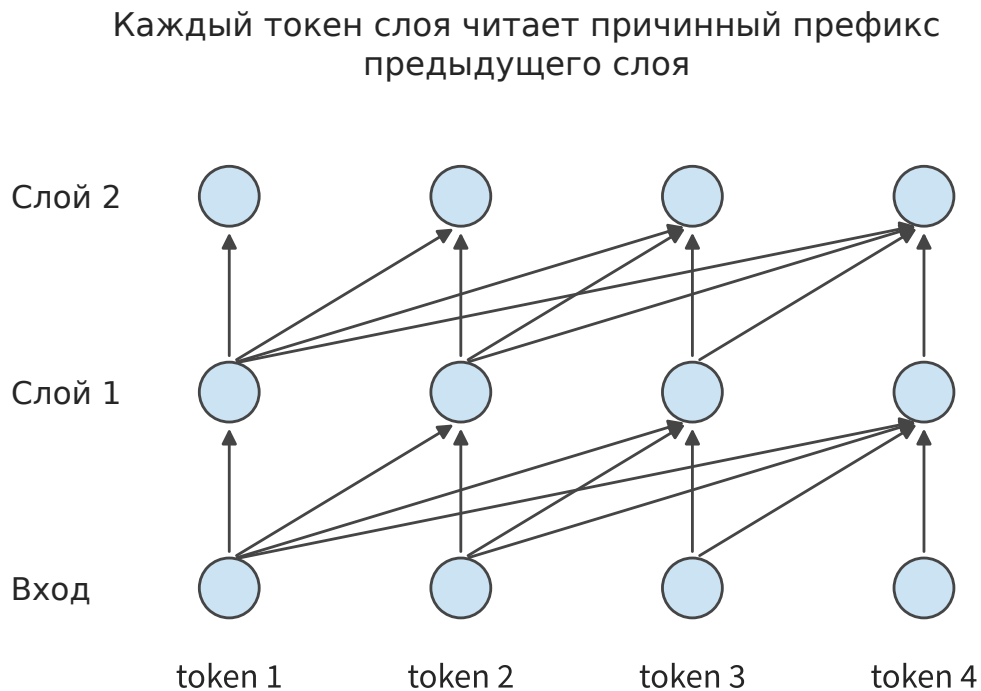


Рисунок 2-2. Межслойные зависимости причинного Transformer. Каждый запрос считывает разрешённые позиции предыдущего слоя; после завершения предыдущего слоя известные входные токены одного слоя можно вычислять параллельно.

зиции токенов в одном слое могут вычисляться параллельно; при генерации следующий вход можно сформировать лишь после определения первого выхода. Поэтому обработка входных данных продвигается по глубине сети, а генерация дополнительно создаёт последовательную зависимость между вызовами.¹

Упражнение 2-1 (дополнительное): как зависимости RNN и Transformer ограничивают параллельные вычисления

Для последовательности из четырёх известных входных токенов нарисуйте вычислительные графы зависимостей трёхслойной RNN и трёхслойного причинного Transformer. Затем добавьте одну позицию генерируемого выхода и укажите, завершения какого узла необходимо дождаться перед вычислением этой позиции. Считайте, что вычисление одного токена в одном слое занимает одну единицу времени, а параллельных ресурсов достаточно. Для каждого случая определите длину наибольшей цепочки зависимостей при обработке всех известных входных данных; объясните, какие ожидания можно сократить добавлением вычислительных блоков, а какие нельзя.

2.1.2 Конфигурация модели и структура тензоров

Сначала подробно рассмотрим один вычислительный узел. Пусть в этом вызове проекция выполняется для m входных токенов, каждый из которых представлен вектором из k признаков. Расположив эти векторы по строкам входной матрицы X , получим матрицу из m строк и k столбцов. Пусть ширина выхода равна n , тогда:

¹Журнал расчётов зависимостей последовательности.

$$X \in \mathbb{R}^{m \times k}, \quad W \in \mathbb{R}^{k \times n}, \quad Y = XW \in \mathbb{R}^{m \times n}.$$

Каждый элемент выхода представляет собой скалярное произведение длины k , а всего выход содержит mn элементов, поэтому

$$N_W = kn, \quad F = 2mkn, \quad M_W = b_W kn.$$

N_W — количество параметров весов, а b_W — число байтов на один параметр. При удвоении числа строк m объём вычислений удваивается, а размер весов остаётся неизменным; при удвоении ширины выхода n удваиваются и количество весов, и объём вычислений. Эти два изменения соответствуют случаям, когда «одна и та же модель обрабатывает больше входных данных» и когда «слой модели становится шире». Каждую строку приведённых ниже таблиц моделей можно рассчитать по этому правилу.

Сначала рассмотрим только один путь данных Qwen3-8B. Один **прямой проход** — это процесс вычисления выхода из входных данных с использованием текущих весов; скрытый вектор представляет собой внутреннее представление признаков, передаваемое по сети, а его размерность показывает, сколько чисел содержит вектор каждого токена. **Сеть прямого распространения** (feed-forward network, FFN) отдельно преобразует признаки каждого токена; **внимание** выбирает и агрегирует информацию между токенами. Чередование этих двух типов операций образует слой сети.

Qwen3-8B содержит $L = 36$ слоёв, имеет скрытую размерность $d = 4096$ и промежуточную размерность FFN $f = 12288$; внимание разделяет признаки на несколько групп, каждая из которых отдельно устанавливает связи между токенами. Каждая такая группа называется головой. Здесь используются 32 головы запросов и 8 доступных им KV-голов размерностью 128 каждая; в разделе 2.1.3 они будут рассмотрены по путям запросов, ключей и значений. Словарь содержит 151936 токенов. Эмбединг токенов — это таблица поиска, сопоставляющая идентификаторы токенов с векторами, а выходная голова преобразует вектор последнего слоя в оценки всех токенов-кандидатов. Каждая из этих двух частей хранит собственный набор весов. В сумме параметры всех слоёв и двух словарных матриц дают модели около 8,19 миллиарда параметров.²

Прямой проход начинается с token ID. Эмбединг токенов извлекает для каждого ID вектор размерностью 4096, затем векторы последовательно проходят через внимание и FFN 36 слоёв и, наконец, через нормализацию и словарную проекцию, формируя logits — ещё не преобразованные в вероятности оценки токенов-кандидатов. Нормализация удерживает значения при послойных вычислениях в подходящем диапазоне. Эмбединг токенов выбирает строки по ID: для одного токена извлекается один вектор, а весь словарь служит постоянно размещёнными данными, к которым обращаются разные токены. Выходная словарная голова умножает скрытый вектор на словарную матрицу; при генерации только следующего токена обычно требуются лишь logits последнего входного токена.

Пусть одновременно обрабатываются B запросов одинаковой длины, каждый из которых в текущем вызове содержит P известных входных токенов. Вход проекций и FFN можно представить как $X \in \mathbb{R}^{BP \times 4096}$, где каждая строка является представлением одного токена в текущем слое, а всего имеется $m = BP$ строк. Ключи и значения уже имеющихся токенов контекста считываются из кэша и здесь в m не входят. Если в каждый

²Конфигурация и расчёт параметров Qwen3-8B, общее число параметров составляет 8,190,735,360.

запрос добавляется лишь один токен, то $m = B$. В первом случае легко получить матрицу с большим числом строк, а второй при небольшом batch приближается к умножению матрицы на вектор. Веса одинаковы, но формы входных данных заметно различаются.

Ниже сравниваются две формы входных данных: один запрос, обрабатывающий сразу 8192 входных токена, и запрос с 8192 уже имеющимися токенами контекста, обрабатывающий ещё один новый токен. Обозначим через B количество запросов, через S — длину уже имеющегося перед вызовом контекста, через P — длину входа текущего вызова, а через d — скрытую размерность. В первом случае $B = 1, S = 0, P = 8192$, во втором — $B = 1, S = 8192, P = 1$.

Параметр конфигурации	Параметр Qwen3-8B	Непосредственное влияние на требования к ресурсам
Число слоёв L	36	Повторяющиеся вычисления, послонные состояния и зависимости по глубине
Скрытая размерность d	4096	Размеры входов, выходов и проекций подслоёв
Промежуточная размерность FFN f	12288	Веса и вычисления трёх матриц FFN
Число голов Q/KV	32/8	Ширина векторов запросов и представлений контекста
Размерность головы d_h	128	Скалярные произведения и ширина состояния каждой головы
Размер словаря V	151936	Размер эмбединга и выходной головы

Пример 2-1: расчёт числа параметров и объёма вычислений одного слоя по размерам матриц. В Qwen3-8B ширина проекций запросов и выхода равна 4096, ширина проекций ключей и значений — 1024, а промежуточная размерность FFN — 12288. Найдите количество параметров четырёх проекций внимания и трёх матриц FFN, а также объём вычислений, необходимый для выполнения этих проекций и FFN для m токенов текущего входа. В этом слое каждому токеноу соответствует вектор размерностью 4096, который образует одну строку входной матрицы; m — фактическое число участвующих в текущем вычислении токенов. Если одна и та же лексема встречается дважды, она учитывается как два токена. Если имеется B запросов одинаковой длины, в каждом из которых в текущем вызове обрабатывается P токенов, то $m = BP$. Например, для одного запроса со 128 входными токенами $m = 128$, а для восьми запросов, каждый из которых продвигается на один токен, $m = 8$.

Решение: запрос и выход используют по одной матрице 4096×4096 , ключ и значение — по одной матрице 4096×1024 , а каждая из трёх матриц FFN содержит 4096×12288 параметров. Следовательно,

$$N_{\text{proj}} = 2 \times 4096^2 + 2 \times 4096 \times 1024 = 41,943,040,$$

$$N_{\text{FFN}} = 3 \times 4096 \times 12288 = 150,994,944,$$

$$F_{\text{linear}} = 2m(N_{\text{proj}} + N_{\text{FFN}}) = 385,875,968m.$$

Длина контекста здесь ещё не использовалась: эти проекции выполняют одинаковое преобразование вектора признаков каждого токена. В следующем разделе рассматриваются вычисления внимания между запросами и контекстом, объём которых зависит от

количества пар «запрос — ключ». Разделение этих двух видов работы позволяет понять, почему «увеличение числа токенов, участвующих в текущем вычислении» и «увеличение длины контекста, считываемого каждым токеном» имеют разную стоимость.

Символ	Значение и область применения
B, P, S	Число запросов, число входных токенов на запрос в текущем вызове и длина уже имеющегося контекста
$m = BP$	Общее число токенов, участвующих в проекциях текущего вызова, то есть число строк входной матрицы; при обычном decode $P = 1$, поэтому $m = B$
d, f	Основная скрытая ширина и промежуточная ширина сети прямого распространения
n_Q, n_{KV}, d_h	Число голов запросов, число KV-голов и размерность каждой головы
N_{pair}	Суммарное для всех запросов число допустимых причинных пар «запрос — ключ» в каждом слое, без учёта числа голов
t_e	Число токенов, направленных эксперту e (см. раздел 2.4); вектор признаков каждого токена занимает одну строку входной матрицы этого эксперта; k_{top} — число экспертов, выбранных для каждого токена; если в процессе маршрутизации направленные экспертам токены не отбрасываются, то $\sum_e t_e = mk_{\text{top}}$
m_{out}	Число токенов, для которых фактически выполняется словарная проекция; если для предсказания следующего токена используется только последний токен каждого запроса, оно равно B и не совпадает с полной длиной генерируемого выхода

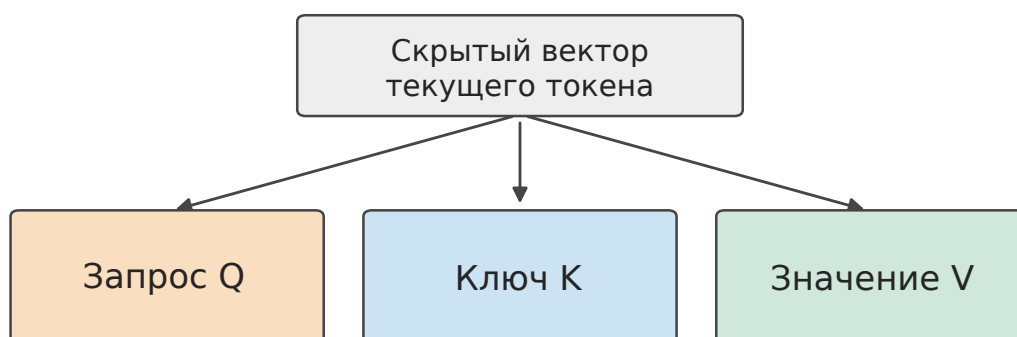
При расчёте потребности всей модели в ресурсах сначала по размерам матриц каждой операции в таблице определяется объём вычислений одного вызова, а затем он умножается на число слоёв соответствующего типа. Словарная голова рассчитывается отдельно по фактическому числу выходных токенов m_{out} ; при генерации следующего токена для каждого запроса нужны оценки только последнего токена. Нормализация, активация и табличный поиск выполняются каждый по своим правилам; далее они рассматриваются отдельно по ходу движения данных.

2.1.3 Вычисление внимания и позиционное кодирование

Сначала проследим вычисления для одного токена. Из скрытого вектора этого токена модель создаёт три представления: **запрос** Q формулирует, «какая информация сейчас нужна», **ключ** K используется для расчёта оценки соответствия запросу, а **значение** V содержит информацию, которая после выбора агрегируется в выходе. Все три представления создаются из входных данных одного токена с помощью разных матриц весов. Линейная проекция и есть такое матричное умножение: оно преобразует входные признаки в другую систему координат.

При математическом обозначении с умножением справа вход X умножается на три матрицы весов с получением Q, K и V . Общая ширина Q равна $32 \times 128 = 4096$, а общая

Три представления из одного входа



Сопоставление запроса и ключей даёт коэффициенты токенов

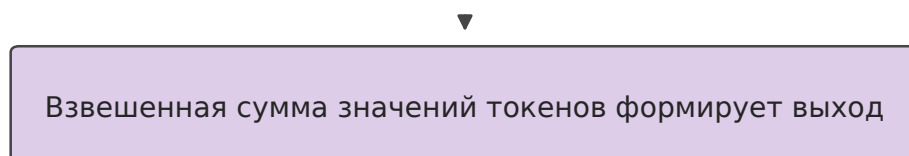


Рисунок 2-3. Входные данные одного токена посредством разных проекций преобразуются в запрос, ключ и значение. Внимание сначала использует запросы и ключи для определения связей между токенами, а затем по этим связям агрегирует значения.

ширина K и V — по $8 \times 128 = 1024$. Следовательно, размер W_q равен $[4096, 4096]$, а размеры W_k и W_v — по $[4096, 1024]$. В коде веса линейного слоя часто хранятся в форме $(d_{\text{out}}, d_{\text{in}})$; после перехода к математическому обозначению с умножением справа входная ширина указывается первой, а выходная — второй, что позволяет построчно сопоставить их с приведённой ниже таблицей.

Описанные проекции создали Q , K и V , но текущая позиция токена ещё не обменивалась информацией с контекстом. Далее Q разделяется на 32 головы, а K и V — на 8 голов. Каждая группа из четырёх голов Q совместно использует одну KV-голову. Для одной головы Q вычисляется скалярное произведение вектора запроса с K разрешённых для доступа позиций. Полученное произведение делится на квадратный корень из ширины головы, чтобы численный масштаб оценок не рос чрезмерно с увеличением ширины головы, после чего позиции будущих токенов маскируются. Softmax преобразует оценки s_j разрешённых позиций в положительные числа и нормализует их, формируя коэффициенты $a_j = e^{s_j} / \sum_i e^{s_i}$; сумма этих коэффициентов равна единице, и они определяют вклад каждого токена контекста. Под «весами внимания» здесь понимаются вычисленные для текущего входа коэффициенты, а не параметры модели, сохранённые после обучения. Затем с этими весами рассчитывается взвешенная сумма V . Результаты всех голов Q объединяются и с помощью выходной проекции W_o размером $[4096, 4096]$ снова преобразуются в скрытый вектор.

В матричной форме вычисление одной головы записывается так:

$$A = \text{Softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + \mathcal{M} \right), \quad O = AV.$$

A — веса контекстных токенов для каждого запроса, $\mathcal{M}$ — причинная маска, а d_h — размерность головы. Скалярное произведение определяет, какие позиции связаны с текущим запросом, Softmax преобразует оценки в весовые коэффициенты, после чего с помощью этих коэффициентов агрегируются векторы значений. В Qwen3 перед вычислением скалярного произведения также выполняются **нормализация запросов и ключей (QK Norm)** и **вращательное позиционное кодирование (Rotary Position Embedding, RoPE)**: первое регулирует численный масштаб запросов и ключей каждой головы, а второе с помощью зависящего от позиции вращения включает в скалярное произведение информацию об относительных позициях.

Теперь рассчитаем количество скалярных произведений. При наличии P новых входных токенов и S уже имеющихся токенов контекста i -му новому токеному доступны $S + i$ токенов, где $i = 1, \dots, P$. Суммирование числа токенов контекста, доступных каждому запросу, даёт прямоугольник и треугольник:

$$N_{\text{pair}} = B \left(PS + \frac{P(P+1)}{2} \right)$$

Для каждой пары запроса и токена контекста скалярное произведение $QK^\top$ одной головы Q требует примерно $2d_h$ FLOPs, а последующий вклад AV — ещё примерно $2d_h$ FLOPs. Суммарный объём полезных матричных вычислений $QK^\top$ и AV для всех голов Q равен:

$$F_{\text{attn}} = 4n_Q d_h N_{\text{pair}} = 16384 N_{\text{pair}}$$

Есть 2 токена, вводятся ещё 3

	Имеющийся контекст		Текущий ввод	
Новый токен 1	✓	✓	✓	
Новый токен 2	✓	✓	✓	✓
Новый токен 3	✓	✓	✓	✓

6 пар со старым контекстом + 6 причинных пар внутри блока

Рисунок 2-4. Два сегмента контекста образуют прямоугольник и треугольник. Синим показано, как три новых токена считывают два старых токена, зелёным — причинный доступ внутри нового входа, а пустыми остаются замаскированные позиции будущих токенов. Каждая цветная ячейка обозначает одну пару «запрос — ключ»; по горизонтальной оси расположены считываемые токены, по вертикальной — токены-запросы нового входа.

При prefill без предварительно закэшированного контекста удвоение длины последовательности удваивает число входных строк проекций, а число пар «запрос — ключ» для полного внимания увеличивается примерно в четыре раза, поскольку новые токены также должны взаимодействовать с предыдущим контекстом. Эти пары «запрос — ключ» можно вычислять двумя распространёнными способами: полное прямоугольное умножение сначала создаёт все оценки, после чего маскируется верхний треугольник, а причинное блочное вычисление пропускает верхний треугольник. Блочная обработка также позволяет объединить вычисление оценок, нормализацию и агрегацию V , чтобы оценки освобождались сразу после использования во внутрикристальной памяти. В главе 5 этот поток данных рассматривается с точки зрения сокращения чтения и записи промежуточных результатов в видеопамять.

Таким образом, сохранение постоянного значения BP сохраняет лишь число строк проекций, но не объём взаимодействия с контекстом. Например, если один длинный вход разделить на два независимых коротких, исчезнут пары «запрос — ключ», ранее пересекавшие границу разделения. И наоборот, добавление контекста перед текущим входом одного запроса увеличивает число скалярных произведений для каждого нового токена, но не меняет число строк входных проекций.

2.1.4 Сеть прямого распространения и объём вычислений прямого прохода

Внимание обеспечивает обмен информацией между разными токенами, а FFN выполняет нелинейное преобразование признаков каждого токена. SwiGLU — это вентильная

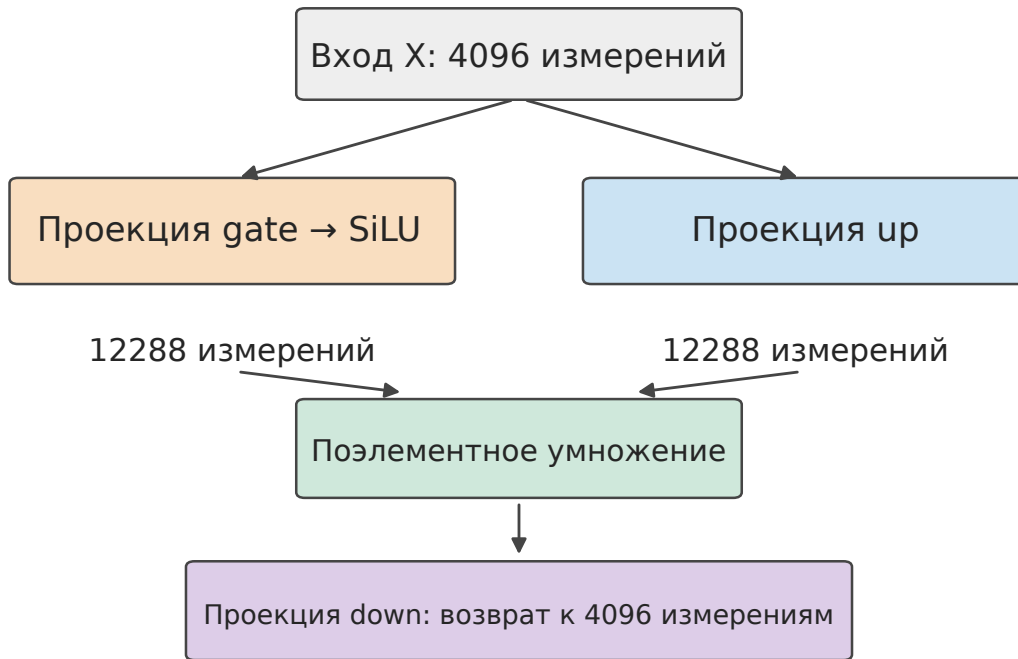


Рисунок 2-5. Две повышающие размерность ветви SwiGLU. После SiLU ветвь gate регулирует соответствующие элементы up, затем проекция down уменьшает размерность произведения до основной.

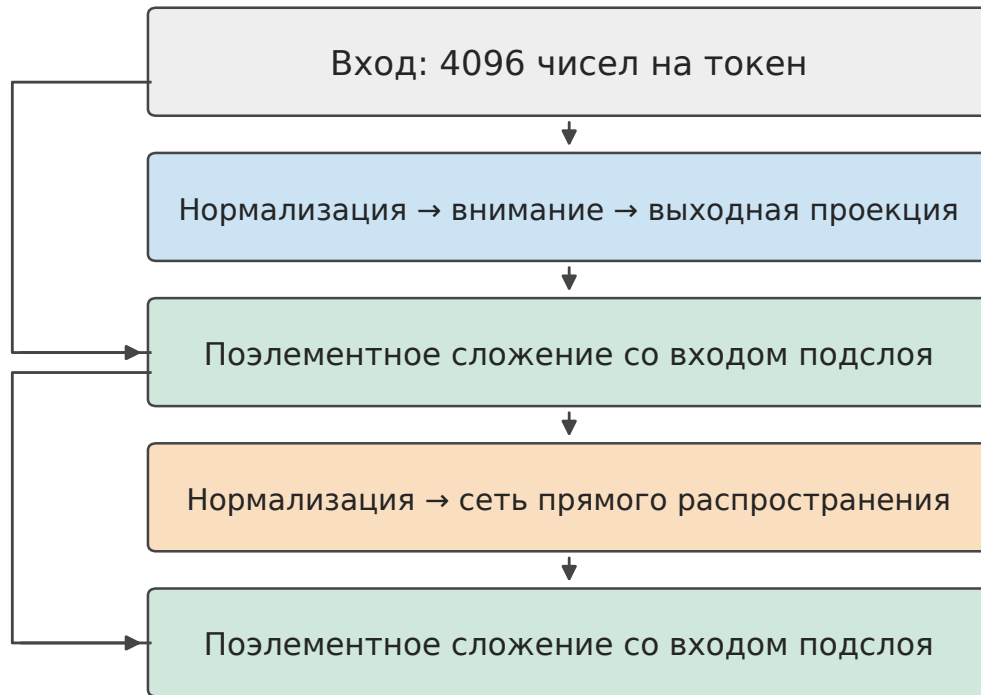
структура сети прямого распространения: одна ветвь преобразует входные признаки, а другая создаёт для неё поэлементные регулирующие значения. В SwiGLU используется функция активации SiLU $\text{SiLU}(x) = x / (1 + e^{-x})$, которая посредством нелинейного преобразования изменяет способ комбинирования признаков. Qwen3-8B использует эту структуру: вход отдельно проходит через две повышающие размерность проекции gate и up, затем ветвь gate после SiLU поэлементно умножается на ветвь up, после чего проекция down снижает размерность обратно до скрытой:

$$\text{FFN}(X) = [\text{SiLU}(XW_{\text{gate}}) \odot (XW_{\text{up}})] W_{\text{down}}$$

Здесь $\odot$ обозначает поэлементное умножение; gate — вентильная ветвь, up — ветвь повышения размерности, а down — проекция понижения размерности. Создаваемые ими промежуточные векторы называются активациями. gate и up увеличивают размерность каждой строки с d до f , а down снова уменьшает её до d . Поэтому три матрицы содержат в сумме $3df$ параметров, а обработка векторов признаков m токенов требует $6mdf$ FLOPs. Для Qwen3-8B $d = 4096$, $f = 12288$, поэтому FFN содержит около 151 миллиона параметров, а её веса BF16 занимают 288 MiB. Более широким активациям после повышения размерности также требуется временное пространство, но хранить их нужно только во время соответствующих вычислений.

Сравним два типа проекций: проекции QKV и выходная проекция внимания содержат в сумме около 41,94 миллиона параметров. После добавления примерно 151 миллиона параметров FFN доля FFN составляет около 78% параметров этих основных проекций, поэтому при коротком контексте значительная часть вычислений приходится на

Слой сначала обменивается данными, затем преобразует признаки



Левая обходная ветвь сохраняет вход подслюя для остаточного сложения

Рисунок 2-6. Структура одного слоя Qwen3-8B. Сначала выполняется подслой внимания, затем подслой сети прямого распространения; остаточные связи слева сохраняют входы подслоев и затем складывают их с результатами преобразований.

три матрицы FFN. С увеличением входа объём вычислений FFN растёт линейно с числом токенов, а объём вычислений полного внимания — пропорционально $P(P + 1)/2$. Скорости роста различаются, поэтому доли вычислений отдельных частей меняются с длиной контекста.

Помимо матричных преобразований, внутри слоя имеются связи между подслоями. Остаточная связь сохраняет вход подслюя и после завершения его преобразования поэлементно складывает его с выходом. Таким образом, один путь выполняет новое преобразование, а другой напрямую передаёт уже имеющееся представление. Ниже внимание, сеть прямого распространения и две остаточные связи объединены в полный слой.

Таблица 2-1. Модули одного слоя Qwen3-8B и выходной интерфейс

Строки внимания и сети прямого распространения суммируются по 36 основным слоям; эмбединг токенов считывает из словаря строку, соответствующую идентификатору токена, и не рассчитывается как одно умножение словарной матрицы (GEMM, общее умножение матриц). K и V внимания имеют размерность лишь 1024, тогда как размерность запроса остаётся равной 4096.

Вход модели

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (входная ширина $\times$ выходная шири- на)	Матричные FLOPs	Число слоёв/ вызовов
Получение строки эмбединга токена	$m \text{ ID} \rightarrow m \times 4096$	Таблица поиска 151936×4096	0; доступ к стро- кам учитывается отдельно	1

Внимание

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (входная ширина $\times$ выходная шири- на)	Матричные FLOPs	Число слоёв/ вызовов
Проекция Q : форми- рование запросов	$m \times 4096 \rightarrow$ $m \times 4096$	4096×4096	$2 \times m \times 4096 \times$ 4096	36
Проекция K, V : фор- мирование повторно используемого кон- текста	$m \times 4096 \rightarrow$ $m \times 1024$	4096×1024 , всего 2	$2 \times 2 \times m \times 4096 \times$ 1024	36
QK^T и AV : агрега- ция контекста по со- держимому	32 головы запро- сов размерностью 128 каждая	Нет дополнитель- ных весов проек- ций	$4 \times 32 \times 128 \times N_{\text{pair}}$	36
Выходная проекция: объединение голов	$m \times 4096 \rightarrow$ $m \times 4096$	4096×4096	$2 \times m \times 4096 \times$ 4096	36

Сеть прямого распространения и связи

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (входная ширина $\times$ выходная шири- на)	Матричные FLOPs	Число слоёв/ вызовов
SwiGLU: преобразова- ние признаков каждо- го токена	$m \times 4096 \rightarrow$ $m \times 12288 \rightarrow$ $m \times 4096$	Две матрицы 4096×12288 ; одна матрица 12288×4096	$6 \times m \times 4096 \times$ 12288	36
Нормализация, RoPE, активация и остаточ- ная связь	Сохранение или поэлементное преобразование указанных тензо- ров	Векторы нормали- зации и прочее	Поэлементные вычисления	36

Выходной интерфейс

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (входная ширина $\times$ выходная шири- на)	Матричные FLOPs	Число слоёв/ вызовов
Словарная голова: преобразование в оценки токенов (один раз в конце всей модели)	$m_{\text{out}} \times 4096 \rightarrow$ $m_{\text{out}} \times 151936$	4096×151936	$2 \times m_{\text{out}} \times 4096 \times$ 151936	1

В таблице показаны два вида роста: работа проекций и FFN растёт с m , а взаимодействие с контекстом — с N_{pair} . Три крупные проекции FFN объясняют, почему при коротком контексте на неё приходится значительная доля вычислений; по мере удлинения контекста доля взаимодействия с ним постепенно увеличивается.

Сложив линейные преобразования и взаимодействие с контекстом в каждом слое, а затем добавив словарную голову, получим основной объём матричных вычислений Qwen3-8B:

$$F_{\text{matrix}} = 36 (385,875,968m + 16,384N_{\text{pair}}) + 2m_{\text{out}} \times 4096 \times 151936.$$

Первая часть выражения растёт с числом входных строк, вторая — с числом пар «запрос — ключ», а последняя — с числом токенов, для которых фактически выполняется словарная проекция. Для входа из 8192 токенов при расчёте внимания по допустимым причинным парам «запрос — ключ» и обработке словарной головой только последнего токена требуется в сумме 133,6 TFLOPs; если уже имеется 8192 токена контекста и добавляется ещё один токен, требуется в сумме 20,0 GFLOPs.³

При prefill одновременно обрабатываются 8192 входных токена, каждый из которых должен пройти через проекции и FFN. При decode эти операции выполняются только для нового токена, а K и V старых токенов считываются из кэша. Таким образом, кэш устраняет повторное вычисление проекций и FFN старых токенов, однако внимание между текущим запросом и контекстом всё равно должно вычисляться. В следующем разделе рассчитываются ёмкость этого кэша и объём доступа к нему.

Упражнение 2-2 (основное): почему при одинаковом числе входных строк объём вычислений внимания различается

Используя таблицу 2-1, для $B = 4$, $S = 4096$, $P = 1024$ отдельно рассчитайте FLOPs, необходимые для проекций и FFN, допустимого причинного внимания и словарной проекции последнего токена каждой последовательности. Сохранив $BP = 4096$, измените параметры на $B = 1$, $S = 4096$, $P = 4096$: сначала предскажите, какие величины останутся одинаковыми, а затем вычислите различия. Наконец, на примере одного запроса с входом длиной 8К сравните полную обработку входа и повторное использование префикса. При повторном использовании сохраните KV первых 6144 токенов и обрабатывайте только 2048 новых токенов. Определите, какую долю матричных вычислений экономит второй способ относительно полной обработки входа.

2.2 Расчёт потребления ресурсов при авторегрессионной генерации

Чтобы рассчитать ресурсы, необходимые для генерации полного ответа, помимо числа операций за один прямой проход нужно знать, сколько раз вызывается модель и как долго хранится состояние. Один и тот же набор весов может использоваться сотни раз в рамках одного запроса, а KV-кэш при этом считывается и растёт. Поэтому итоговое число сохраняемых байтов и совокупное число байтов, к которым выполнялся доступ, могут различаться на несколько порядков.

³Послойный журнал.

2.2.1 Prefill, decode и повторное использование состояния

На этапе prefill модель получает известную входную последовательность и может одновременно вычислить представления нескольких токенов. После выбора первого выходного токена по logits последнего входного токена начинается этап decode: при следующем вызове этот токен передаётся как новый вход, модель выдаёт следующий выходной токен, и последовательность постепенно расширяется. На обоих этапах используется одна и та же модель, но на первом известно несколько входных токенов, а на втором при обычной последовательной генерации каждый запрос пополняется лишь одним токеном за вызов.

Если при генерации каждого токена заново передавать модели весь префикс, проекции и FFN для прежних токенов будут вычисляться повторно. В причинной модели прежние токены не могут видеть добавленные позднее; если веса, индексы позиций токенов и связанные условия вычислений не изменились, полученные для прежних токенов K и V можно сохранить. Последующие запросы внимания считывают эти контекстные K и V и дополняют их новыми K и V для позиции текущего токена, что позволяет избежать значительной части повторных вычислений для прежних токенов.

Кэширование заменяет «повторное вычисление представлений прежних токенов» на «сохранение и обращение к состоянию прежних токенов». Для будущей позиции токена будет вычислен собственный Q , который используется для оценки K прежних токенов, после чего V агрегируются с полученными весами. Поэтому длительно сохраняются именно K и V . Прежний запрос внимания уже выполнил свою задачу, а на следующем шаге используется новый запрос. В главе 8 подробнее рассматривается совместное использование этих состояний разными запросами.

Для Qwen3-8B объём BF16 KV-кэша на каждый токен контекста составляет:

$$\begin{aligned} c_{KV} &= 2Ln_{KV}d_hb_{KV} \\ &= 2 \times 36 \times 8 \times 128 \times 2 \text{ bytes} \\ &= 144 \text{ KiB}. \end{aligned}$$

Первый множитель 2 соответствует ключам и значениям; на каждом из L слоёв сохраняются n_{KV} групп векторов шириной d_h , а каждый элемент занимает b_{KV} байт. Следовательно, для хранения H токенов контекста требуется $M_{KV} = c_{KV}H$. Для Qwen3-8B при $H = 8192$ это соответствует 1.125 GiB, а обработка ещё одного нового входного токена добавляет 144 KiB. Если число слоёв или KV-голов уменьшить вдвое, скорость роста также уменьшится вдвое; при увеличении батча для каждого независимого запроса добавляется собственный контекст.⁴

2.2.2 Потребление памяти и объём доступа к данным

KV-кэш хранит уже вычисленные результаты и избавляет от повторных вычислений при последующей генерации. Чтобы проанализировать сэкономленные вычисления и дополнительные затраты на хранение, разделим данные по назначению и жизненному циклу на три категории: веса, состояние контекста и временные данные.

⁴Расчёт накопления кэша.

Категория данных	Хранение и повторное использование	Основные рассчитываемые в этой главе показатели
Веса	Обычно остаются неизменными между вызовами	Резидентный объём в байтах, набор используемых весов, повторное использование внутри батча
Состояние контекста	Сохраняется для запроса и непрерывно дополняется или обновляется	Фактически занимаемый объём, объём чтения и записи на каждом шаге, вычисления для обновления
Временные данные	Создаются соответствующим оператором или этапом и освобождаются после использования	Размеры тензоров, объём данных, прошедших через соответствующий интерфейс хранения, одновременно сохраняемые части

Сначала рассмотрим веса. Все веса Qwen3-8B в BF16 занимают приблизительно 16.38 GB, то есть 15.26 GiB. После загрузки модели эти данные многократно используются разными запросами. Во время одного прямого прохода строки эмбеддингов выбираются по ID токенов, проекции внутри слоёв и FFN используют соответствующие матрицы, а выходная голова словаря вычисляет итоговые оценки. Поэтому набор используемых весов определяется конкретными операциями вычислительного графа.

При умножении матриц $[m, d] \times [d, f]$ каждая строка входных данных использует одну и ту же матрицу весов. После загрузки блока весов во внутрикристальную память его можно последовательно применять к нескольким строкам входных данных, благодаря чему однократно считанные данные участвуют в большем числе операций умножения и сложения. При увеличении числа обрабатываемых строк m объём вычислений растёт, а размер всей матрицы остаётся неизменным. В главе 5 подробнее объясняется, как объём внутрикристальной памяти и разбиение на блоки определяют число повторных использований одного набора весов.

Состояние контекста уже рассчитано в разделе 2.2.1; объём памяти для временных данных зависит от того, какие тензоры существуют одновременно. После освобождения буфера предыдущего слоя следующий слой может повторно использовать ту же область памяти; слияние операторов, то есть объединение нескольких соседних операций в одну программу, также позволяет хранить часть промежуточных значений только в регистрах или внутрикристальной памяти. Поэтому при расчёте объёма чтения и записи необходимо учитывать каждое обращение, а при расчёте пикового потребления памяти — определить тензоры, которые всё ещё должны храниться одновременно. В главе 5 это будет разобрано на конкретных программах.

Объём хранения зависит от того, какие данные необходимо сохранять в определённый момент, а объём доступа — от того, сколько раз данные были прочитаны и записаны в процессе выполнения. Например, если за один шаг decode весь прежний контекст считывается один раз, логический объём чтения равен $c_{KV}H$; при последовательном выполнении нескольких шагов эти значения нужно суммировать. Только определив, через какой уровень хранения фактически проходят эти обращения, можно разделить соответствующее R на пропускную способность данного интерфейса.

Пример 2-2. Когда после увеличения батча объём чтения контекста превышает объём чтения весов? Qwen3-8B один раз за батч считывает необходимые общие веса объёмом около 15.14 GB; строки эмбеддингов, выбираемые отдельно для каждого запро-

са, учитываются дополнительно. Контекст каждого запроса длиной 8K в BF16 занимает 1.125 GiB. Предположим, что запросы независимы и прежний контекст каждого из них считывается один раз.

Решение: обозначим объём чтения общих весов на батч как R_W . Поскольку контексты запросов независимы, их совокупный объём чтения равен $Bc_{KV}H$. Эти величины равны при

$$B_* = \frac{R_W}{c_{KV}H}.$$

Подставив $R_W = 15,136,811,008$ байт и $c_{KV} = 147456$ байт, получим, что при $H = 8192$ минимальный целочисленный размер батча равен 13, а при $H = 2048$ — 51. Если сократить длину контекста до четверти исходной, число запросов, необходимое для превышения объёмом чтения контекста объёма чтения весов, увеличится приблизительно в четыре раза. Чем длиннее контекст, тем раньше ограничивается выигрыш от повторного использования весов внутри батча, поскольку каждый новый запрос добавляет чтение отдельного контекста.⁵

Если проследить путь данных из главы 1, при загрузке веса переносятся из хранилища в видеопамять GPU, а во время выполнения необходимые веса передаются вычислительным блокам. KV-кэш и межслойные активации также могут храниться и использоваться внутри ускорителя. Поэтому один и тот же логический доступ может остаться внутри кристалла, пройти через НВМ либо потребовать передачи между ускорителями; в главе 4 отдельно рассматриваются ёмкость хранилищ и пропускная способность этих путей передачи.

2.2.3 Объём вычислений и чтения-записи состояния при генерации полного ответа

Для получения полного ответа выполняется множество последовательных шагов, во время которых один и тот же контекст многократно считывается. Рассмотрим один обычный запрос с последовательной генерацией: $B = 1$, длина восстановленного префикса равна S , текущий вход содержит $P \geq 1$ токенов, а итоговый ответ — $G \geq 1$ токенов. Пусть $H = S + P$. Этап prefill обрабатывает новый вход и создаёт первый выходной токен, после чего выполняется $n_d = G - 1$ шагов decode. Последний возвращённый токен ещё не был повторно передан модели, поэтому к концу запроса KV для него автоматически не сохраняется.

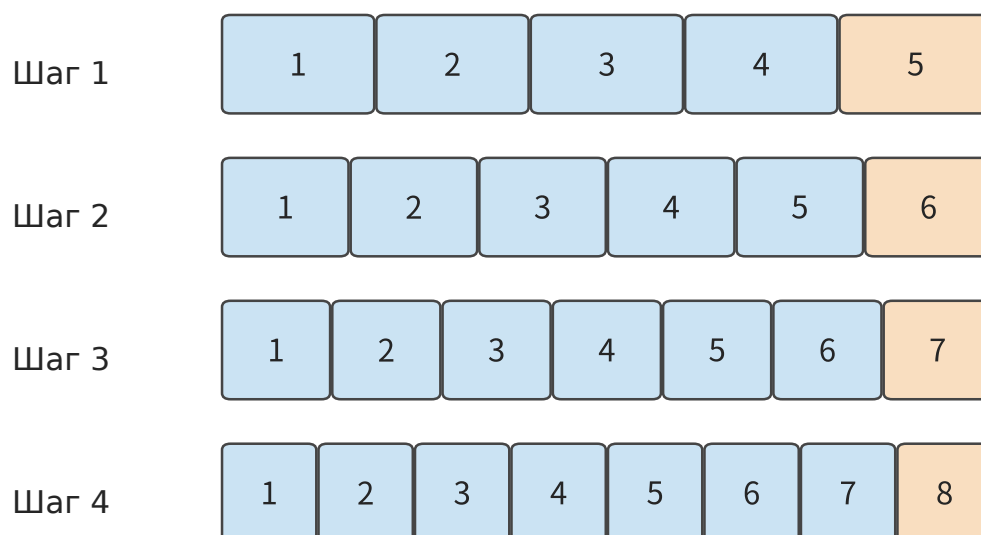
Каждая строка на рисунке 2-7 содержит на один закрашенный квадрат больше предыдущей: добавленная ранее позиция теперь также входит в контекст. Перед выполнением j -го шага decode, где нумерация j начинается с 0, длина контекста равна $H + j$. Поэтому, если на каждом шаге decode прежний контекст считывается один раз, совокупный объём чтения равен:

$$R_{\text{old}} = c_{KV} \sum_{j=0}^{n_d-1} (H + j) = c_{KV} \left[n_d H + \frac{n_d(n_d - 1)}{2} \right]$$

Размер состояния, добавленного на этапе decode, равен $c_{KV}n_d$; если сохраняется весь контекст, итоговый объём действительного KV-кэша составляет $c_{KV}(H + n_d)$. Во весь

⁵Межглавная проверка.

На каждом шаге добавляется токен и
пересчитывается контекст



Синий: прочитанный контекст; оранжевый: добавленный токен

Рисунок 2-7. Чтение и запись контекста при четырёх шагах генерации. На каждом шаге считываются отмеченные синим существующие позиции, после чего добавляется один новый токен, отмеченный оранжевым; исходный контекст на схеме состоит из 4 токенов, а после четырёх шагов сохраняется всего 8.

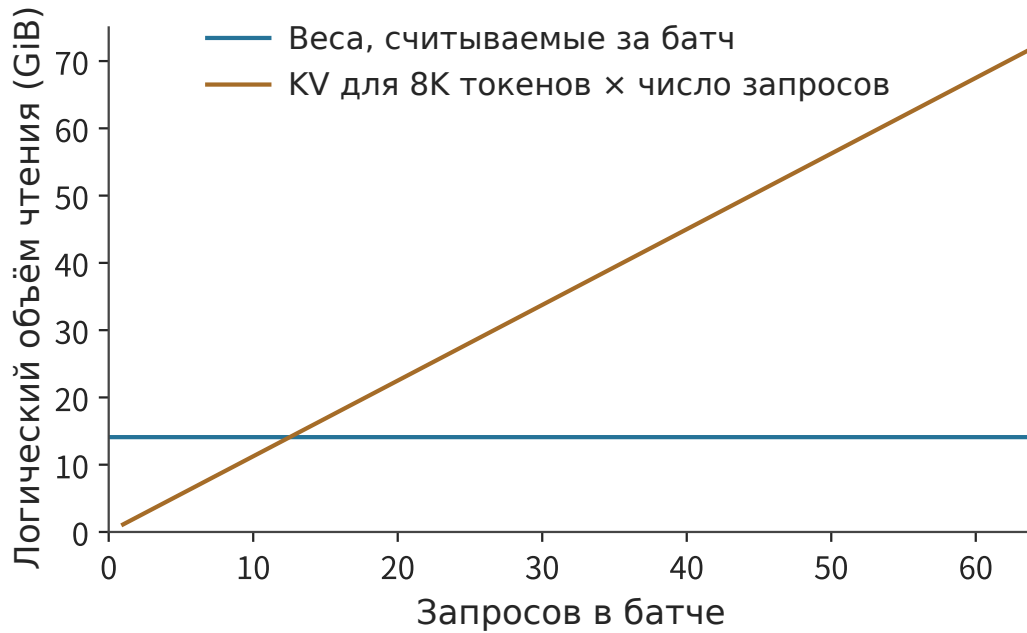


Рисунок 2-8. Основные веса проекций совместно считываются запросами одного батча, но каждый запрос имеет собственный контекст. На схеме длина контекста каждого запроса зафиксирована на уровне 8K; отдельно суммируются логические объёмы чтения общих весов и независимых KV-кэшей. Веса учитываются как однократно считанные для всего батча decode; для KV суммируется состояние контекста, считываемое на этом шаге каждым независимым запросом.

добавленный в рамках текущего запроса KV-кэш также следует включить объём $c_{KV}P$, записанный на этапе prefill. Восстановление и передача существующего префикса учитываются отдельно. Каждый член суммы соответствует контексту, существующему в начале очередного шага decode: появившиеся раньше позиции участвуют в большем числе последующих запросов внимания, а появившиеся позже — в меньшем.

Пример 2-3. Почему при пошаговой генерации совокупный объём чтения KV-кэша значительно превышает объём его хранения? Найдите размер KV-кэша, сохраняемого к концу запроса, и общий объём чтения существующего KV-кэша во время генерации.

Решение: примем $S = 0$, $P = 8192$, $G = 1025$, тогда $n_d = 1024$, а совокупное число чтений KV для токенов контекста за весь процесс составит 8,912,384. KV-кэш каждого токена занимает 144 KiB, поэтому совокупный объём чтения равен приблизительно 1224 GiB. Во время генерации добавляется лишь 144 MiB KV-кэша, а к завершению сохраняется в общей сложности около 1.27 GiB. Совокупный объём чтения почти в тысячу раз превышает итоговый объём хранения, поскольку каждый токен, уже вошедший в контекст, повторно считывается при каждом последующем шаге decode; в этом примере большую часть контекста составляют 8192 входных токена, записанных на этапе prefill.

Эта последовательность вызовов также определяет общий объём вычислений. Линейные проекции и FFN на каждом шаге decode обрабатывают одну строку, а объём взаимодействия с контекстом растёт вместе с $H + j$. Если фиксированный объём матричных вычислений на каждом шаге равен F_0 , а объём вычислений для взаимодействия с каждым токеном контекста — a , то

$$F_{\text{decode,total}} = n_d F_0 + a \left[n_d H + \frac{n_d(n_d - 1)}{2} \right].$$

При увеличении длины ответа совокупный объём фиксированной части работы каждого шага линейно растёт с n_d , а пополняющийся во время генерации контекст приводит к квадратичному росту части совокупного объёма вычислений относительно числа шагов генерации. При возврате одного токена $G = 1$ и $n_d = 0$, поэтому выполняется только prefill; этот частный случай также показывает, почему полный ответ включает $G - 1$ последующих шагов decode.

На рисунке 2-7 объём чтения суммируется по шагам генерации, а на рисунке 2-8 — по числу запросов. Ёмкости состояний и объёмы доступа к контексту нескольких независимых запросов складываются, тогда как веса внутри батча используются совместно. Если длины различаются, для каждого запроса следует отдельно подставить его значения H и n_d : чем длиннее выход, тем больше раз считывается существующий контекст; чем длиннее вход, тем больший контекст необходимо прочитать при первом шаге decode. Для реального диалога можно сначала восстановить эту цепочку вызовов по длине входа на каждом ходе, попаданием в кэш и числу возвращённых токенов; конкретный пример подстановки приведён в разделе Расчёт длины сохранённого чата.

Упражнение 2-3 (расширенное). Как растут ёмкость KV-кэша и совокупный объём чтения при увеличении длины ответа

Для Qwen3-8B при $B = 1$, $S = 0$, $P = 8192$ рассмотрите возврат соответственно $G = 513$ и $G = 1025$ токенов. Найдите число последующих шагов decode, итоговый размер KV-кэша и совокупный объём чтения прежнего контекста. Объясните, почему при почти двукратном увеличении числа выходных токенов объём KV-кэша, добавленного на этапе генерации, итоговый общий объём KV-кэша и совокупный объём чтения растут в разных пропорциях. Затем уменьшите разрядность KV-кэша вдвое и укажите, какие из этих трёх величин также уменьшатся вдвое.

2.3 Представление состояния контекста и механизм доступа

Продолжая генерацию, модель должна использовать уже прочитанное содержимое. Сначала сравним два способа ведения записей: при первом для каждой страницы сохраняется отдельная запись, которую можно получить при необходимости; при втором сохраняется только одна сводка фиксированной длины, переписываемая с поступлением нового содержимого. В первом случае объём записей растёт вместе с историей, во втором остаётся постоянным, однако нельзя гарантировать, что удастся извлечь все подробности каждого токена.

В применении к моделям один класс механизмов сохраняет ключ и значение для каждой позиции токена, а рекуррентное состояние конечного размера сводит вклад истории в состояние фиксированного размера. Как записи для каждого токена, так и рекуррентные сводки хранятся в виде числовых векторов или матриц. Такие представления, сохраняемые после обработки контекста моделью для повторного использования в последующих вычислениях, называются состоянием контекста. Длина контекста измеряется числом токенов, а ёмкость состояния — фактическим числом сохранённых байтов.

При сравнении этих механизмов сначала следует определить, что сохранилось от истории, а затем — как на новом шаге извлекается необходимая информация. Можно представлять каждый токен меньшим числом измерений, напрямую обращаться только к недавним позициям, объединять несколько позиций в меньшее число записей либо сначала отбирать записи, а затем считывать их. Другой класс механизмов непрерывно обновляет состояние фиксированного размера, из которого запрос получает результат. Эти подходы по-разному изменяют ширину представления, число записей и способ доступа; экономию на хранении и чтении необходимо рассчитывать вместе с дополнительными затратами на отбор, сжатие и обновление.

2.3.1 Многоголовое внимание и совместное использование KV

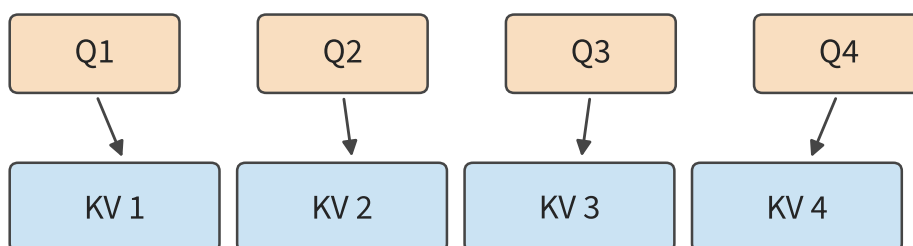
Самый прямой способ сократить объём данных, сохраняемых для каждого токена, — уменьшить количество сохраняемых для него наборов K , V . Многоголовое внимание позволяет разным головам запросов независимо изучать связи в контексте. В многоголовом внимании (Multi-Head Attention, MHA) каждой голове Q соответствуют собственные головы K , V ; во внимании с несколькими запросами (Multi-Query Attention, MQA) все головы Q используют один общий набор K , V ; во внимании с группированными запросами (Grouped-Query Attention, GQA) головы Q разделяются на группы, каждая из которых использует общий набор K , V . При совместном использовании уменьшается количество наборов K , V , которые требуется хранить для каждого токена контекста, но число токенов контекста остаётся неизменным.

Рассмотрим в качестве примера один слой всего с четырьмя головами запросов. В MHA для каждой из четырёх голов запросов хранится отдельный набор K , V ; в GQA первые две головы запросов могут совместно использовать первый набор K , V , а последние две — второй; в MQA все четыре головы запросов используют один общий набор. При одинаковой длине контекста и ширине головы количество сохраняемых наборов KV составляет соответственно четыре, два и один. Таким образом, в этом примере GQA вдвое сокращает объём памяти для контекста. Четыре головы запросов по-прежнему могут вычислять четыре разных набора оценок для одного и того же фрагмента контекста — именно этим «совместное использование представления контекста» отличается от «объединения запросов».

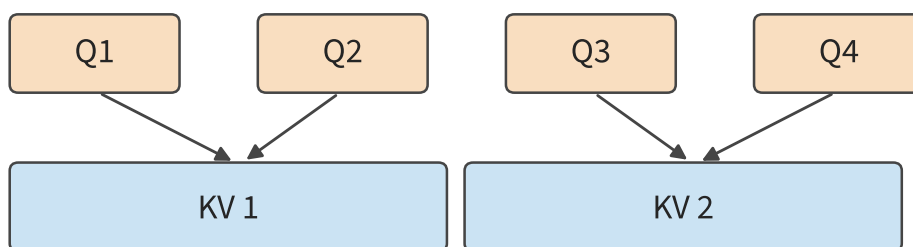
Qwen3-8B использует этот способ совместного доступа: каждые четыре головы запросов используют одну общую группу KV; всего предусмотрено 32 головы запросов и 8 голов KV. Если сохранять KV отдельно для каждой головы запросов, требуемая ёмкость была бы в четыре раза больше, чем при текущей конфигурации; если бы все запросы использовали одну общую группу, она составляла бы одну восьмую от текущей. В фактической конфигурации GQA для хранения 8192 токенов требуется 1.125 GiB. На диаграмме ёмкости в конце этого раздела зафиксированы длина контекста и ширина головы, а изменяется только количество групп KV, поэтому соотношение высот столбцов непосредственно отражает степень совместного использования.

Совместное использование K , V изменяет запрашиваемые данные, однако 32 головы Q по-прежнему отдельно вычисляют оценки и взвешенные выходные данные. Поэтому объём проекций и хранения KV сокращается пропорционально количеству групп, тогда как взаимодействия QK^T и AV по-прежнему суммируются по числу голов запросов. Однократно считанные K , V также могут использоваться несколькими головами запросов одной группы, благодаря чему при фактических вычислениях одни и те же дан-

МНА: отдельная группа KV для каждого запроса



GQA: одна группа KV на два запроса



MQA: одна группа KV на четыре запроса

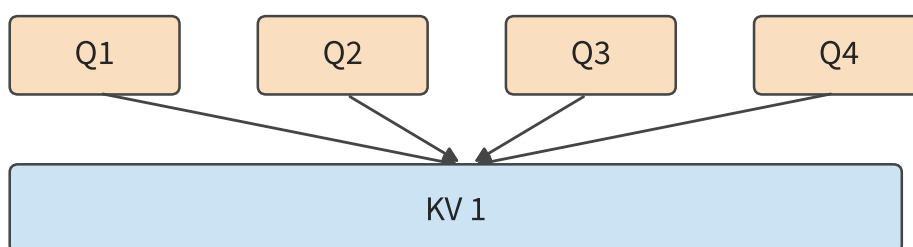


Рисунок 2-9. При фиксированных четырёх головах запросов KV сохраняется отдельно для каждой головы, каждой группы либо для всех запросов. Линии показывают отношения использования; после совместного использования контекста каждый запрос по-прежнему формирует собственные оценки и выходные данные.

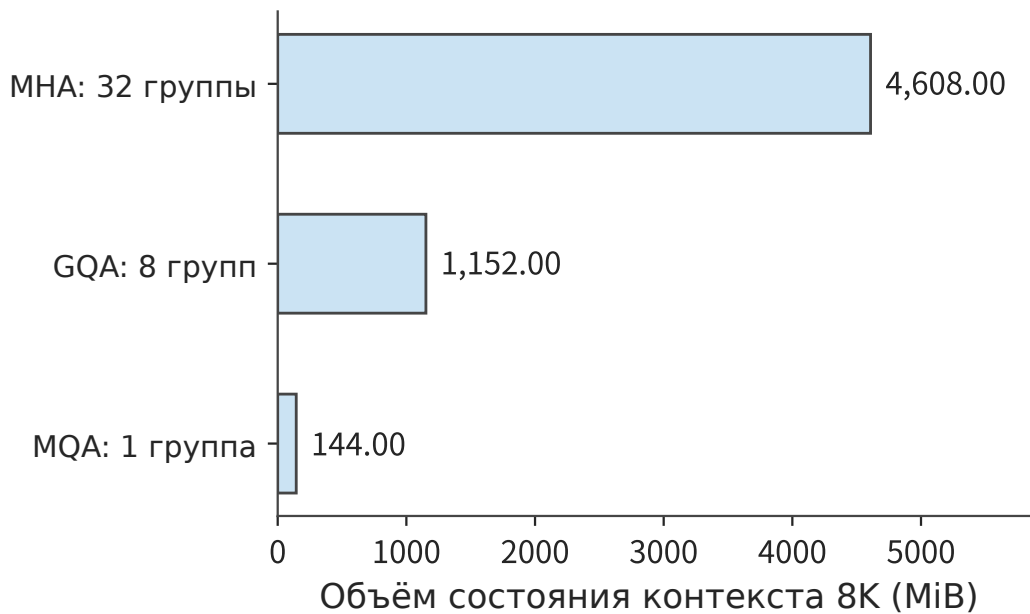


Рисунок 2-10. Число слоёв Qwen, ширина головы, 8192 токена контекста и BF16 зафиксированы; изменяется только количество групп KV. Фактическая модель использует GQA; два остальных столбца представляют структурные варианты для анализа степени совместного использования.

ные используются повторно.

Изменение потребления ресурсов можно непосредственно предсказать по количеству групп. Если вдвое уменьшить n_{KV} , вдвое сократятся как число параметров проекций ключей и значений, так и ёмкость состояния контекста; число голов запросов n_Q останется неизменным, поэтому объём вычислений при взаимодействии запросов с контекстом по-прежнему будет суммироваться по исходному числу голов.

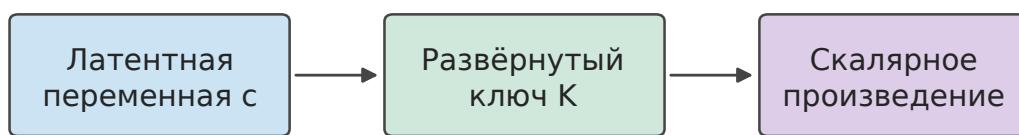
Совместное использование KV также показывает, что инфраструктурные ограничения влияют на архитектуру модели. В исходной статье о MQA непосредственной мотивацией названы затраты пропускной способности памяти при инкрементальном инференсе; GQA, в свою очередь, ищет компромисс между качеством МНА и скоростью MQA.⁶ Уменьшение числа голов KV не только изменяет отношения внимания, которые способна выражать модель, но и сокращает объём состояния, которое сервер должен хранить и считывать.

2.3.2 Сжатие скрытых переменных и пути выполнения MLA

GQA и MQA экономят память за счёт уменьшения числа групп KV, а многоголовое внимание со скрытыми переменными (Multi-head Latent Attention, MLA) — за счёт уменьшения размерности представления контекста. Сначала входные данные можно спроецировать в низкоразмерную скрытую переменную c , а затем с помощью повышающей проекции получить K и V , используемые для вычисления внимания. В процессе обучения модель осваивает эти низкоразмерные представления и повышающую проекцию, а при генерации сохраняет контекст в форме c .

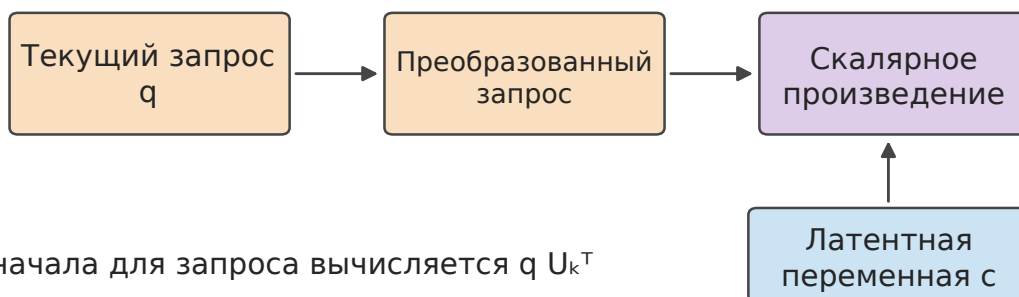
⁶Аннотация оригинальной статьи о MQA; аннотация оригинальной статьи о GQA; раздел 6.1 отчёта Llama 3. Пример ёмкости выведен из заданного в условии состояния сервиса.

Путь 1: сначала развернуть каждый токен контекста



Сначала для контекста вычисляется $K = c U_k$

Путь 2: сначала преобразовать текущий запрос



Сначала для запроса вычисляется $q U_k^T$

Рис. 2-11. Два пути вычисления различаются расположением повышающей проекции. Развёрнутый путь сначала восстанавливает ключи контекста, а компактный сначала преобразует текущий запрос, после чего напрямую считывает скрытые переменные; благодаря ассоциативности соответствующее скалярное произведение можно вычислять в обоих порядках.

Основание для сохранения только скрытых переменных выглядит следующим образом. Пусть повышающая проекция ключей имеет вид U_k , скрытые переменные контекста образуют по строкам матрицу c , а развёрнутые ключи имеют вид $K = cU_k$. Ассоциативность умножения матриц даёт

$$qK^\top = q(cU_k)^\top = (qU_k^\top)c^\top.$$

В левой части ключ сначала восстанавливается для каждого токена контекста, а в правой сначала преобразуется текущий запрос. Для одного нового запроса в правой части достаточно однократно преобразовать запрос, а затем напрямую вычислить его скалярное произведение со скрытыми переменными. Векторы значений также можно подвергнуть повышающей проекции после взвешенного суммирования: сначала суммировать низкоразмерные векторы, а затем восстановить размерность выхода. Таким образом, повышение размерности, ранее выполнявшееся для каждого токена контекста, теперь выполняется для текущего запроса и результата суммирования.

В этой главе в качестве примера используется **Gated MLA (многоголовое внимание со скрытыми переменными и гейтами)** из Kimi K3; гейты регулируют выход внимания с помощью обучаемых коэффициентов. Компактный путь сохраняет 512-мерную скрытую переменную и дополнительную 64-мерную ветвь, участвующую в скалярном произведении $QK^\top$. Все слои MLA в Kimi K3 используют NoPE (No Position Encoding): к запросам и ключам не применяется RoPE, а позиционную информацию обеспечивают рекуррентные гейты и затухание в слоях KDA (раздел 2.3.4); эталонная реализация всё равно проецирует и кэширует эту 64-мерную ветвь, но не применяет к ней вращение, поэтому для каждого токена сохраняется $512 + 64$ измерений. Для 24 слоёв MLA, контекста из 8192 токенов и BF16 объём хранения составляет $24 \times 8192 \times (512 + 64) \times 2$, то есть 216 MiB.

В развёрнутом кэше скрытые переменные сначала восстанавливаются в K и V каждой головы, после чего эти векторы сохраняются; для тех же 8192 токенов требуется около 11,25 GiB. В компактном кэше сохраняются скрытые переменные до повышающей проекции. Таким образом, различие в объёме обусловлено тем, с какой стороны линейного преобразования расположен кэш: компактный кэш на каждом шаге преобразует текущий запрос, а развёрнутый кэш напрямую считывает векторы контекста каждой головы.⁷

Это преобразование уменьшает ширину представления каждого токена контекста, однако для каждого токена по-прежнему необходимо сохранять один вектор скрытой переменной. Поэтому объём хранения и число операций скалярного произведения продолжают расти вместе с длиной контекста. Чем длиннее контекст, тем существеннее экономия объёма чтения благодаря компактному представлению; чем короче контекст, тем выше доля затрат на само преобразование запроса.

2.3.3 Локальное окно, сжатие контекста и разреженный индекс

Предположим, модель уже прочитала длинный текст и теперь должна обработать новый токен. Модель может напрямую прочитать ближайший фрагмент позиций, прочитать элементы, объединяющие несколько старых токенов, либо сначала найти элементы, релевантные текущему запросу, а затем прочитать их основные состояния внимания. Эти

⁷Развёрнутый расчёт MLA для Kimi K3.

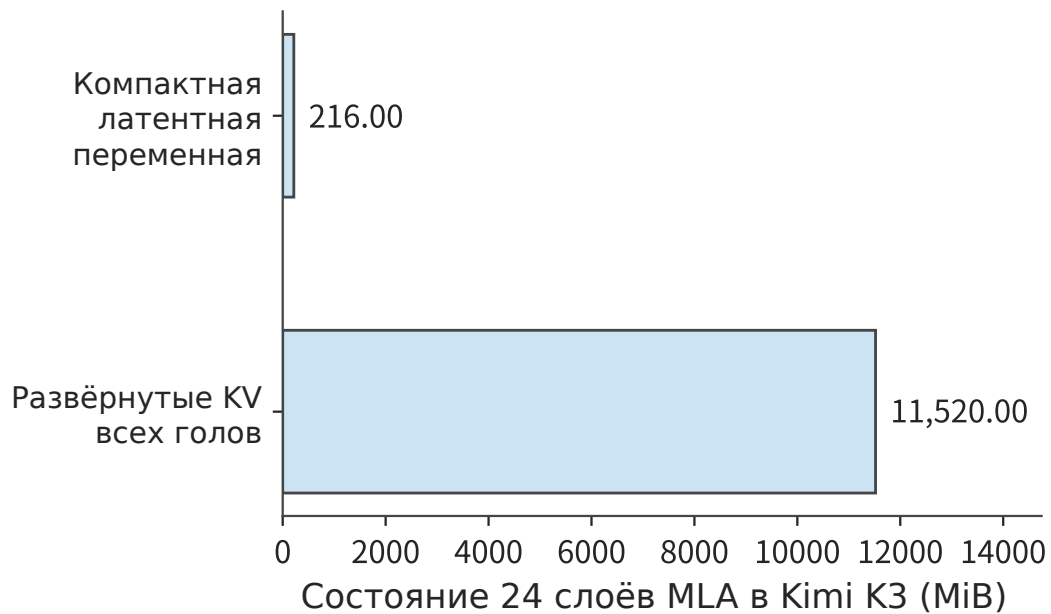


Рис. 2-12. Два объёма хранения для 24 слоёв MLA в Kimi K3 при одинаковом контексте 8K и BF16. Компактный путь сохраняет представления до повышающей проекции, а развёрнутый — ключи и значения каждой головы.

три действия приводят соответственно к локальному окну, сжатию контекста и разреженному выбору. Их можно использовать совместно, но учитывать следует отдельно: окно ограничивает ближайший диапазон, сжатие сокращает количество долгосрочных элементов, а индекс определяет, какие элементы читать сейчас. Сама фильтрация также требует чтения представлений индекса и вычисления оценок: нельзя учитывать только основные состояния внимания выбранных в итоге элементов.

Сначала разграничим три объекта. **Локальное внимание со скользящим окном (Sliding-Window Attention, SWA)** напрямую читает ближайший фрагмент позиций; при длине окна 128 более ранние позиции покидают это локальное окно. **Глобальные основные KV** — это долгосрочные представления, доступные основному механизму внимания для чтения; в них уже могут быть объединены несколько позиций. «Глобальные» означает, что они охватывают более длинную историю, но не то, что при каждом запросе читаются все элементы. **Индексные K** — это отдельные, более компактные представления для поиска: сначала индексатор использует их для оценки элементов, а затем основной механизм внимания читает основные KV выбранных элементов. Индексные оценки служат для выбора элементов кэша, тогда как основной механизм внимания использует собственный запрос для вычисления итоговых весов агрегации.

В ходе обучения модель учится сводить фрагмент входных данных в общее представление. Одна 512-мерная запись основных KV одновременно служит ключом и значением в механизме внимания, совместно используется несколькими головами запросов, а её ёмкость рассчитывается как для одной записи. SWA сохраняет недавние детали, глобальная ветвь предоставляет более раннюю информацию, и обе используются текущим запросом.

DeepSeek V4-Flash объединяет эти механизмы в одной модели с 43 основными слоями, скрытой размерностью 4096, 64 головами внимания и размерностью каждой головы

512. Сначала Q проецируется из 4096 измерений в 1024, а затем преобразуется в запрос размерностью 64×512 ; общие KV проецируются из 4096 измерений в 512; на выходе применяется низкоранговая проекция с разбиением на 8 групп: сначала в более узкое промежуточное пространство, а затем в целевое. Размерности входа и выхода каждой проекции определяют форму соответствующей матрицы.

DeepSeek V4-Flash использует два вида сжатого внимания: **сжатое разреженное внимание (Compressed Sparse Attention, CSA)** сначала сжимает контекст, а затем с помощью индекса выбирает релевантные элементы; **сильно сжатое внимание (Heavily Compressed Attention, HCA)** объединяет больше позиций в меньшее количество крупнозернистых элементов, доступных запросу. Оба механизма сокращают объём хранения и чтения длинного контекста, но различаются степенью сжатия и наличием дополнительного отбора элементов. Основная часть модели содержит 2 чисто оконных слоя, 21 слой CSA и 20 слоёв HCA; длина окна равна 128. CSA сохраняет контекст с коэффициентом сжатия 4 и поддерживает индекс; HCA сохраняет более грубый контекст с коэффициентом сжатия 128. Если использовать BF16 как эталонный формат хранения, каждый завершённый сжатый элемент CSA содержит 512-мерное состояние основного механизма внимания и отдельный 128-мерный индексный элемент. При наличии S токенов количество уже созданных элементов вычисляется как $\lfloor S/4 \rfloor$; для ещё не завершённого блока сжатия требуется отдельный буфер.

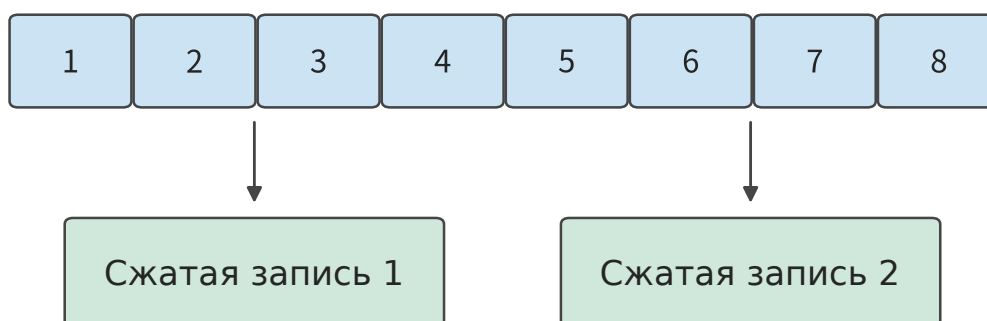
Коэффициент сжатия описывает количество выходных элементов, но не обязательно полный диапазон входных данных одного элемента. В CSA модели V4 одна запись создаётся на каждые четыре новых токена, однако соседние блоки перекрываются: кроме особой обработки границы первого блока, один элемент с помощью двух разных проекций агрегирует информацию из двух соседних блоков — всего из восьми токенов; для каждого токена также применяются обучаемые поканальные веса. HCA создаёт одну запись на каждые 128 токенов, не использует такое перекрытие соседних блоков и не применяет индекс top-k, то есть выбор только k элементов с наивысшими индексными оценками, а читает все завершённые крупнозернистые элементы. Перекрывающееся сжатие из CSA модели V4 было устранено лишь в CSA2, второй версии CSA из V4.1.

Основной механизм внимания одного запроса CSA читает не более окна плюс 512 сжатых элементов, однако при индексной оценке всё равно сканируются все сохранённые сжатые индексные элементы. При $S = 8192$ один слой CSA хранит 2048 сжатых элементов: состояния основного механизма внимания занимают около 2 MiB, индекс — около 0,5 MiB, а окно — около 0,125 MiB; выбранные основным механизмом внимания сжатые элементы в сумме занимают не более 0,5 MiB, но при сканировании индекса всё равно обрабатываются все 2048 индексных элементов. В HCA уже создано 64 элемента, состояния основного механизма внимания занимают около 0,0625 MiB, а читаются состояния окна и эта группа крупнозернистых сжатых элементов.

Если просуммировать данные по 43 слоям, то при указанном выше эталонном формате BF16 и контексте из 8192 токенов окно, сжатый контекст и индекс вместе занимают 59,125 MiB; ещё около 11,641 MiB занимает буфер компрессора в FP32, итого — около 70,8 MiB. Суммарный объём данных, прочитанных основным механизмом внимания последнего запроса, включая окно, и индексом составляет 27,625 MiB. В разбивке состояний эти данные перечислены по отдельности, поэтому из неё видно, что именно обозначают количество выбранных элементов, размер сохранённых состояний и объём чтения на каждом шаге.

Сжатие также вводит ритм обновления, отличный от ритма запросов. При поступле-

Сначала создаются сжатые записи, затем запрос выбирает их



Пример: 4 токена объединяются в 1 запись

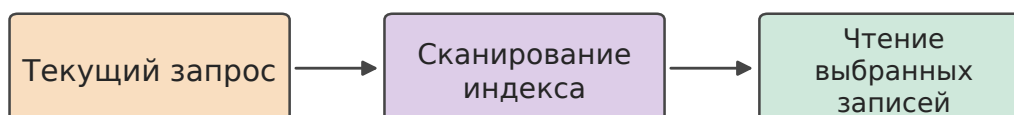


Рис. 2-13. Последовательность сжатия контекста и индексного выбора. В верхней части показано только количественное соотношение, при котором восемь токенов образуют две записи; перекрывающиеся проекции и взвешенная агрегация V4 CSA опущены. В нижней части показан процесс одного запроса: сначала сканируется индекс, а затем читаются состояния основного механизма внимания.

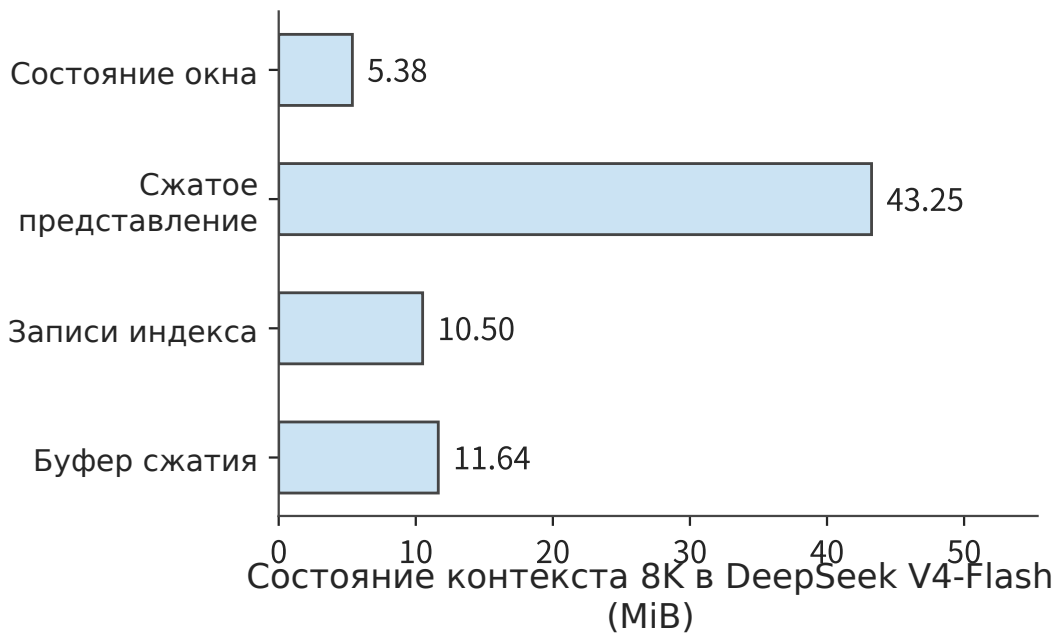


Рис. 2-14. Четыре вида состояний DeepSeek V4-Flash при контексте 8K. Окно, сжатый контекст и индекс хранят информацию об уже обработанном контексте; буфер сжатия хранит данные, которые ещё агрегируются или обновляются, и учитывается в ёмкости отдельно.

нии нового токена сначала обновляется текущий блок сжатия; когда совокупное количество обработанных токенов достигает значения, требуемого коэффициентом сжатия, создаётся долгосрочный элемент. CSA завершает блок каждые четыре токена, а HSA — каждые 128 токенов. Поэтому при обработке одного и того же входного токена необходимо отдельно рассчитывать, сколько элементов читает основной механизм внимания, сколько элементов сканирует индекс и завершается ли блок компрессора. При обработке 128-го токена оба вида блоков сжатия могут завершиться одновременно.⁸

Упражнение 2-4 (углублённое): как совместное использование голов KV и сжатие контекста изменяют объёмы хранения и чтения

Сначала уменьшите в Qwen3-8B количество голов KV с 8 до 4, сохранив 32 головы запросов, и отдельно рассчитайте изменения ёмкости KV, количества параметров проекции KV и объёма вычислений, необходимого для взаимодействия запроса с контекстом. Затем для слоя CSA модели DeepSeek V4-Flash увеличьте контекст с 8192 до 16384 и при коэффициенте сжатия 4 и выборе не более 512 сжатых элементов за один запрос рассчитайте количество сохранённых элементов, количество элементов, сканируемых индексом, и верхний предел количества выбранных элементов. Объясните, почему эти три величины растут не в одной и той же пропорции.

2.3.4 Линейное внимание и рекуррентное обновление конечного состояния

Механизмы из разделов 2.3.1–2.3.3 всё ещё позволяют различать отдельные сохранённые элементы контекста. При рекуррентном обновлении конечного состояния используется другой процесс: с поступлением каждого нового токена его вклад объединяется с

⁸Журнал продолжения вычислений кэшированного суффикса.

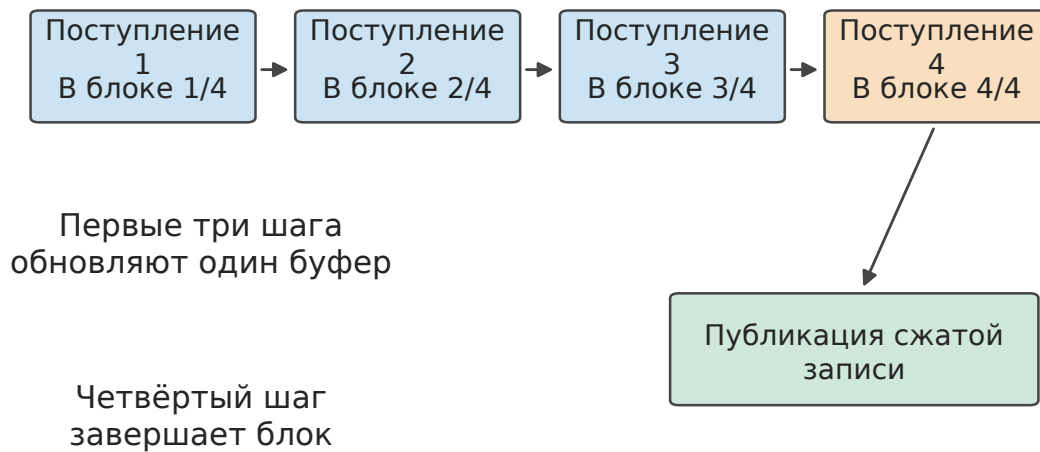


Рис. 2-15. Создание одного блока при коэффициенте сжатия четыре. Первые несколько входных токенов обновляют буфер одного и того же блока; после поступления четвёртого токена формируется сжатый элемент, доступный последующим запросам.

существующим состоянием, текущий запрос напрямую использует обновлённое состояние, а на следующем шаге это состояние снова перезаписывается. Это соответствует описанной в начале раздела 2.3 картине с непрерывным перезаписыванием сводки фиксированной длины. Такое линейное внимание меняет порядок вычисления запросов, ключей и значений, больше не вычисляя явно оценки внимания для всех пар «запрос — ключ». При фиксированной размерности признаков размер матрицы состояния остаётся неизменным, а объём вычислений для обработки всей последовательности линейно растёт с её длиной.

Основная операция при построении состояния — внешнее произведение: каждый элемент вектора ключа умножается на каждый элемент вектора значения, образуя двумерную таблицу связей. Рассмотрим матрицу $\mathcal{S}_t$, полученную суммированием внешних произведений ключей и значений. При поступлении каждого нового токена сначала считывается прежнее состояние, затем в него записывается вклад текущей позиции, после чего текущий запрос напрямую считывает результат из матрицы состояния:

$$\mathcal{S}_t = \mathcal{S}_{t-1} + k_t v_t^\top, \quad o_t = q_t^\top \mathcal{S}_t.$$

Если ширина ключа равна d_k , а ширина значения — d_v , состояние всегда содержит $d_k d_v$ элементов. Новый токен изменяет содержимое матрицы, но не её размер. Таким образом, список контекста по токенам превращается в набор накопленных связей между ключами и значениями, из которого запрос получает выходные данные.

В реальном рекуррентном внимании помимо простого суммирования применяются гейтирование и коррекция. **Kimi Delta Attention (KDA)** — используемый в Kimi механизм гейтированного рекуррентного внимания, который хранит информацию контекста в матрице связей фиксированного размера. KDA использует гейтирование и delta-обновление; здесь delta означает запись поправки на основе текущей ошибки предска-

Контекст растёт, размер матрицы состояния неизменен

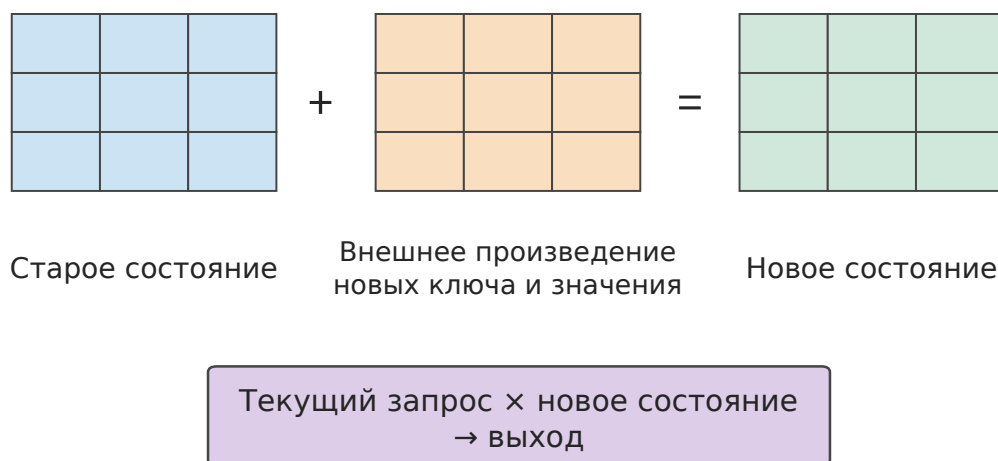


Рис. 2-16. Рекуррентное обновление матрицы фиксированного размера. Внешнее произведение нового ключа и значения добавляется к прежнему состоянию, при этом форма матрицы остаётся неизменной; запрос использует обновлённое состояние для вычисления выходных данных. На схеме показано простое рекуррентное суммирование.

зания. При каждом обновлении не только записывается новая информация: текущий ключ также используется для проверки предсказания существующего состояния, после чего на основе результата записывается поправка. Прежняя информация хранится в виде агрегированных связей, текущий ключ определяет, куда вносится поправка, а гейт затухания — сколько прежней информации следует сохранить. KDA обновляет матрицу связей фиксированного размера, а не добавляет элементы в список, где данные хранятся по каждому токenu.

Каждый слой KDA в Kimi K3 содержит 96 голов, а рекуррентная матрица каждой головы имеет размер 128×128 . При хранении в FP32 один слой занимает 6 MiB, а 69 слоёв — 414 MiB. За один шаг decode прежняя матрица считывается, с ней объединяется новая информация, после чего результат записывается обратно. В идеальном случае при однократном чтении и записи общий объём данных составляет 828 MiB. Этот объём чтения и записи не увеличивается с длиной контекста: при коротком контексте он представляет собой значительные фиксированные накладные расходы, а при длинном позволяет избежать непрерывного роста объёма сканирования вместе с количеством токенов контекста.

Реализации prefill и decode также различаются. Для decode используется одношаговое рекуррентное обновление; фрагмент заранее известного ввода можно параллельно обработать блочным алгоритмом, передавая состояние между блоками. Промежуточные данные внутри блоков и состояния между блоками занимают временное пространство, поэтому по одному лишь размеру итогового рекуррентного состояния невозможно предсказать пиковое потребление памяти во время prefill. В главе 5 этот вопрос анализиру-

ется на основе жизненного цикла буферов — от выделения до освобождения.⁹

2.3.5 Состав состояния при гибридном внимании

В реальных моделях слои с конечным состоянием часто сочетаются со слоями глобального внимания. Это позволяет одновременно использовать сжатие контекста за счёт рекуррентной структуры и частично сохранять возможность прямого доступа к глобальному контексту. Состояние всей модели состоит из нескольких частей: одни имеют фиксированный размер, другие растут вместе с длиной контекста.

Kimi K3 содержит 93 слоя: 69 слоёв KDA и 24 слоя MLA. При контексте в 8192 токена, компактном MLA в формате BF16, рекуррентном состоянии в FP32 и слотах короткой свёртки в BF16 (короткая свёртка смешивает вдоль последовательности входы нескольких соседних позиций, а слоты сохраняют недавние входы, необходимые для обновления на следующем шаге) эти три компонента занимают соответственно 216 MiB, 414 MiB и 19,4 MiB — всего около 649,4 MiB. При использовании эталонного развёрнутого кэша один только компонент MLA увеличивается до 11,25 GiB, соответственно изменяется и общий объём.¹⁰

Для Qwen3.6-35B-A3B также необходимо отдельно рассчитывать объём состояния слоёв линейного и полного внимания. Модель содержит 30 слоёв линейного внимания и 10 слоёв полного внимания; глобальный KV на каждый токен контекста одного запроса занимает 20 KiB, фиксированное рекуррентное состояние в FP32 — ещё 60 MiB, а слоты короткой свёртки в BF16 — 1,875 MiB. При контексте в 8192 токена эти три компонента в сумме занимают 221,9 MiB. Это состояния ветви внимания. Следующая за ней ветвь смеси экспертов (Mixture of Experts, MoE) предоставляет несколько подсетей прямого распространения, из которых маршрутизатор для каждого токена выбирает часть для выполнения, определяя, какие веса фактически используются в общем каркасе модели. Эти доступные для выбора подсети называются маршрутизируемыми экспертами, а ветвь, выполняемая для всех токенов, — общим экспертом. Механизмы внимания и MoE вместе образуют полную модель.

Форма этих пяти кривых состояния определяется различными представлениями. В Qwen3-8B при добавлении каждого токена все слои GQA дописывают K и V ; в Kimi K3 контекст дописывается только в части MLA, тогда как часть KDA сохраняет фиксированную матрицу; в DeepSeek V4-Flash записи добавляются после завершения блока сжатия. В V4.1 Flash несколько слоёв дополнительно читают один общий глобальный кэш, а Qwen3.6 сочетает рекуррентные слои со слоями полного внимания. Для длинного контекста скорость роста определяет дополнительный объём; для короткого контекста начальный объём определяется фиксированными компонентами, такими как рекуррентная матрица.

Сложение фиксированного рекуррентного состояния и контекста, растущего на каждый токен, даёт модель объёма гибридной структуры:

$$M_{\text{state}}(H) = M_{\text{fixed}} + c_{\text{state}}H, \quad H_* = \frac{M_{\text{fixed}}}{c_{\text{state}}}.$$

c_{state} — количество байтов состояния, добавляемых для каждого токена контекста, а H_* — длина контекста, при которой объёмы двух частей равны. Фиксированная часть

⁹Журнал блочного состояния и буферов.

¹⁰Расчёт смешанного состояния.

Тип внимания задаётся конфигурацией модели

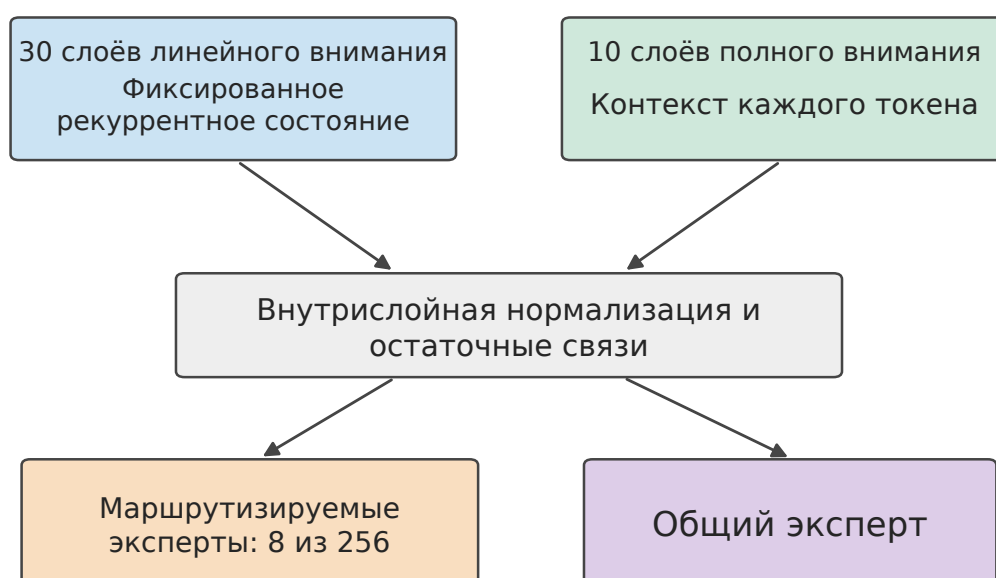


Рисунок 2-17. Два типа слоёв Qwen3.6. Конфигурация определяет, использует ли каждый слой линейное или полное внимание, после чего в обоих случаях выполняются маршрутизируемые и общие эксперты; две верхние ветви соответствуют линейному и полному вниманию, а модель содержит соответственно 30 и 10 слоёв этих типов.

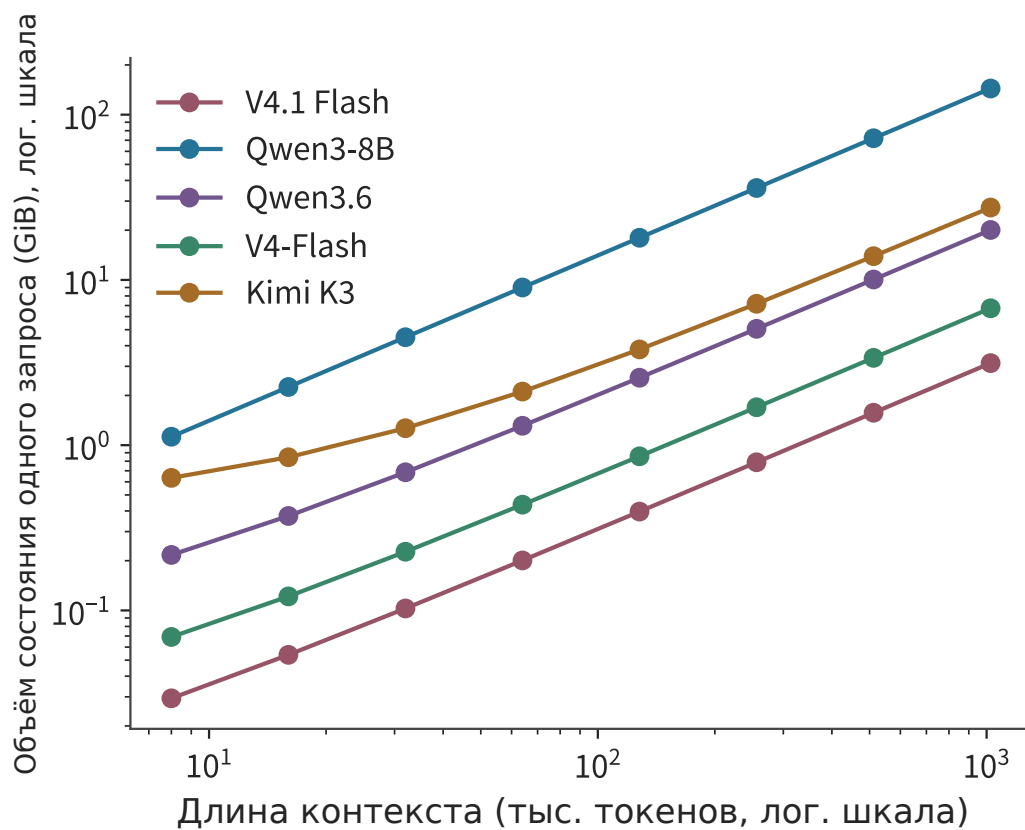


Рисунок 2-18. Объем состояния при росте контекста. Обе оси представлены в логарифмическом масштабе; пять моделей организуют состояние соответственно посредством добавления по одному токenu, рекуррентного обновления, сжатия и совместного использования между слоями. По вертикальной оси отложен объем состояния одного запроса, по горизонтальной — количество токенов контекста.

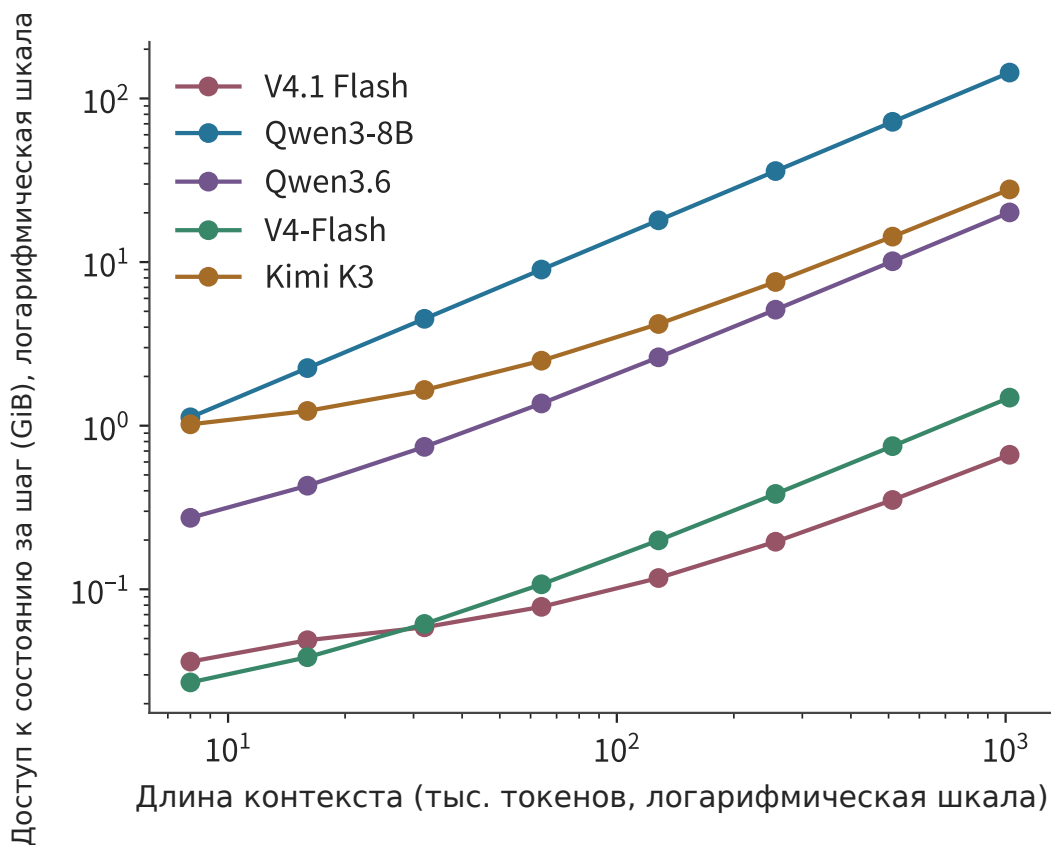


Рисунок 2-19. Учитываемый объем доступа к состоянию на каждом шаге для той же группы моделей. Для контекста, накапливаемого по одному токenu, учитывается чтение, а для рекуррентной матрицы — одно чтение и одна запись; это логический объем данных на заданном пути, используемый для сравнения разных закономерностей роста объема и доступа. Под шагом понимается один этап decode, генерирующий следующий токен для одного запроса; по вертикальной оси не учитывается чтение весов модели.

Qwen3.6 занимает 61.875 MiB, а каждый токен добавляет 20 KiB, поэтому $H_* = 3168$; в компактном представлении Kimi K3 каждый токен добавляет 27 KiB, фиксированная часть занимает около 433,4 MiB, а точка пересечения находится примерно на 16,4K токенов контекста. Ниже точки пересечения сокращение контекста позволяет сэкономить лишь небольшую часть пространства; значительно выше неё уменьшение ширины представления каждого токена экономит объём эффективнее. В DeepSeek V4-Flash записи добавляются по мере завершения блоков сжатия, поэтому кривая растёт ступенчато.

Эти состояния поддерживают разные способы доступа к информации: глобальный KV позволяет запросу напрямую выбирать старые токены, рекуррентная матрица сохраняет связи контекста посредством пошагового обновления, а сжатый индекс ищет релевантное содержимое среди меньшего числа записей. В разделе 2.6.3 требования к ресурсам будут сопоставлены на конкретных задачах поиска.¹¹

Упражнение 2-5 [основное]: при какой длине контекста объём KV превысит объём фиксированного состояния?

Для Qwen3.6 примите объём фиксированного состояния равным 61,875 MiB, а объём на каждый токен контекста — 20 KiB; для компактного представления Kimi K3 рекуррентное состояние занимает 414 MiB, состояние свёртки — 20 348 928 байт, а каждый токен контекста дополнительно занимает 27 KiB. Для каждой модели найдите длину контекста, при которой растущее вместе с контекстом состояние сравнивается по объёму с фиксированным состоянием, а также общий объём состояния при контексте 32K и 128K. Если бюджет состояния составляет всего 256 MiB, сколько токенов контекста сможет сохранить каждая модель? Объясните, почему при бюджете объёма меньше размера фиксированного состояния сокращение контекста всё равно не позволяет вместить состояние модели.

2.3.6 Сколько хранится на каждый токен и сколько считывается при каждом decode

Для сравнения стоимости контекста моделей необходимо одновременно привести три величины: средний прирост хранилища на каждый дополнительный токен глобальной истории, фактически сохраняемое состояние при длине N и объём истории, который требуется считать для одного запроса decode. Глобальный KV растёт вместе с контекстом; скользящее окно сохраняет только последний фрагмент; размер рекуррентного состояния фиксирован, но на каждом шаге его всё равно требуется читать и записывать. Деление всех трёх величин на длину контекста скроет различия в закономерностях их роста.

Рассмотрим запрос, которому в текущей позиции доступен контекст из $N = 8192$ токенов, включая сам токен запроса. В таблице с указанными для каждой строки точностью и путём кэша рассчитаны ёмкость глобальной истории и логический объём считывания для одного запроса: выбранные на каждом слое K/V или общая скрытая переменная считываются один раз, а затем повторно используются головами запросов для расчёта оценок QK и вычисления PV (то есть описанного выше AV — агрегирования значений с помощью весов внимания); дополнительно учитываются локальное окно SWA и сканирование индекса. Обновления состояния рекуррентной матрицы, свёртки и компрессора рассматриваются отдельно после таблицы.

¹¹Эксперимент с полной моделью.

Модель и путь кэша	Средний прирост глобальной истории (B/token)	Глобальная история 8K (MiB)	Считывание KV вни- мания и индекса при каждом decode (MiB)
Qwen3-8B, BF16 GQA	147,456	1,152	1,152
Qwen3-32B, BF16 GQA	262,144	2,048	2,048
Qwen3-30B-A3B, BF16 GQA	98,304	768	768
Qwen3-235B-A22B, BF16 GQA	192,512	1,504	1,504
Qwen3.5-397B-A17B, часть с полным вни- манием BF16	30,720	240	240
Qwen3.6, часть с пол- ным вниманием BF16	20,480	160	160
Kimi K3, компактный путь MLA BF16	27,648	216	216
DeepSeek V4-Flash, производственный смешанный формат	3,514.25	27.455	12.556
DeepSeek V4.1 Flash, производственный смешанный формат	890	6.953	11.375

Два представления MLA в Kimi K3 показывают, как путь кэша влияет на ёмкость: компактный путь из таблицы добавляет 27,648 B на каждый токен, тогда как используемая в этой книге эталонная реализация Hugging Face разворачивает K/V и добавляет 1,474,560 B на каждый токен. В разделе «Расчёты кэша для разных моделей» приведено точное количество байтов для обоих путей.

В строках двух поколений Flash используется производственный смешанный формат: основная запись V4-Flash имеет смешанный формат FP8/BF16 и занимает 584 B, а индекс использует MXFP4 — формат, в котором каждые 32 четырёхбитных значения совместно используют один scale; формат V4.1 Flash описан ниже. Числа для V4-Flash в разделе 2.3.3, напротив, рассчитаны для эталонного формата BF16: при том же контексте 8K глобальная история занимает 53.75 MiB, дополнительно окно — 5.375 MiB, а один запрос считывает 27.625 MiB. Указанные в таблице 27.455 MiB для V4-Flash — это резидентный объём глобальной истории в производственном формате, лишь численно близкий к объёму считывания 27.625 MiB в эталонном формате BF16.

Для смешанного внимания необходимо также учитывать фиксированное состояние. При используемых в этой главе рекуррентном состоянии FP32 и короткой свёртке BF16 рекуррентные матрицы и слоты свёртки суммарно занимают 184.219 MiB у Qwen3.5, 61.875 MiB у Qwen3.6 и 433.406 MiB у Kimi K3. Текущее потребление памяти совместно определяется этими фиксированными состояниями и контекстным кэшем из таблицы; при каждом decode также требуется читать и записывать рекуррентную матрицу и обновлять слоты свёртки.

Помимо отдельного сохранения на каждом слое, состояние можно совместно использовать между слоями. Долгое время decoder-only оставалась основной архитектурой универсальных генеративных языковых моделей: для обработки входных данных и поточной генерации токенов применяется один и тот же набор основных слоёв, причём каждый слой обычно создаёт KV из собственного скрытого состояния. V4.1 Flash существенно изменила такое разделение обязанностей: построение представления контекста от-

делено от последующих запросов, благодаря чему обработка входных данных требует меньше вычислений основных слоёв. В техническом отчёте эта архитектура называется CED; там же указано, что она вдохновлена идеей YoCo о повторном использовании KV между слоями.¹²

Формулировка «кодировщик отвечает за понимание, декодировщик — за генерацию» здесь конкретно означает различие обязанностей этих двух частей при формировании представления контекста и вычислениях генерации. Кодировщик соблюдает причинное ограничение и не является кодировщиком традиционной модели *sequence-to-sequence*, способным двунаправленно обращаться ко всему входу; при генерации нового токена участие кодировщика также по-прежнему требуется. Эта асимметрия выражается в том, где создаётся информация и какие слои выполняются на каждом этапе, а не в передаче входа и выхода двум моделям, каждая из которых не участвует в вычислениях другой.

Для понимания этой архитектуры сначала можно разделить «создание глобального представления» и «запрос к глобальному представлению». В CED модели V4.1 Flash 40 слоёв текстовой основы последовательно разделены на первые 20 слоёв причинного кодировщика и последующие 20 слоёв декодировщика. Обе части принадлежат одной авторегрессионной модели, и каждый токен в кодировщике имеет доступ только к информации в текущей позиции токена и предшествующих позициях.

Сначала кодировщик обрабатывает вход. Требуемый декодировщику глобальный KV создаётся проекцией представления последнего слоя кодировщика, поэтому нет необходимости предварительно пропускать каждый входной токен через все слои декодировщика. Однако локальный SWA каждого слоя декодировщика по-прежнему зависит от собственного входного представления этого слоя, поэтому в декодировщик необходимо передать выходы кодировщика максимум для последних 128 токенов промпта и построить локальное состояние; далее это называется воспроизведением конечного окна. После начала поточной генерации токенов каждый новый токен последовательно проходит все 40 слоёв кодировщика и декодировщика. В главе 3 будет рассчитано, какие вычисления над входом благодаря этому исключаются, а в главе 8 будут разграничены два вида воспроизведения для кодировщика и декодировщика.

Стоимость длинного контекста зависит также от количества слоёв, в которых повторно сохраняется один и тот же фрагмент информации. В техническом отчёте V4.1 ограничения по ёмкости HBM, SSD и памяти хоста, а также по пропускной способности переноса KV прямо указаны как проектные ограничения. В V4.1 степень сжатия последовательности по сравнению с V4 не увеличена: V4 использует CSA со сжатием 4:1 и HCA со сжатием 128:1, тогда как в V4.1 HCA устранён, глобальные записи кодировщика сжимаются в отношении 2:1, а записи декодировщика сохраняются в отношении 1:1. Уменьшение кэша в V4.1 обеспечивается главным образом совместным использованием между слоями и более низкой точностью при сохранении более мелкой гранулярности исторических токенов.¹³

Совместное использование также изменяет информацию, которую каждый слой способен выбирать независимо. Слои, использующие данные повторно, больше не создают собственные глобальные K/V, однако каждый слой по-прежнему сохраняет собственный запрос Q и локальный SWA, поэтому может вычислять для общих записей другие ве-

¹²Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и пересчёт для межглавной сессии.

¹³Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и пересчёт для межглавной сессии.

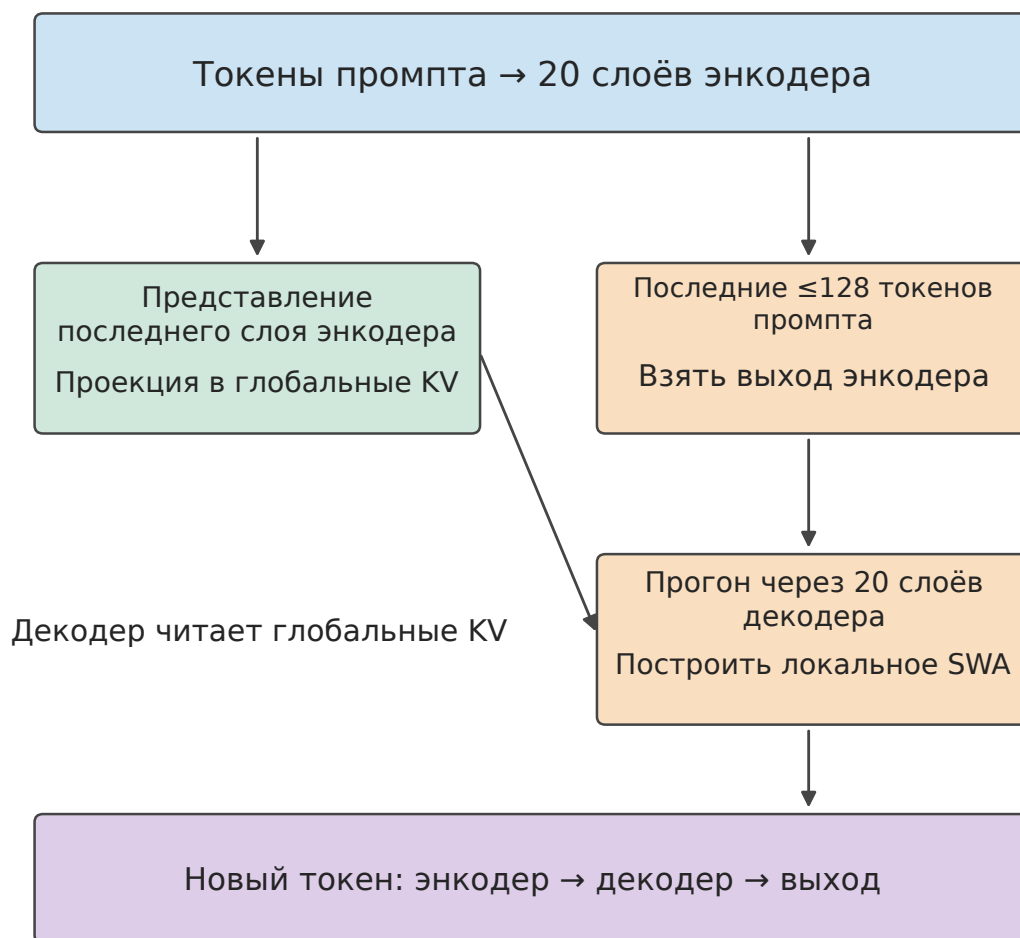


Рисунок 2-20. CED разделяет создание глобального KV и построение локального состояния декодировщика. Большинство токенов промпта проходят только основную часть кодировщика и проекцию глобального KV; выходы кодировщика для конечного окна дополнительно проходят через декодировщик, приближённо формируя локальный SWA. Генерируемые новые токены по-прежнему проходят все 40 слоёв.

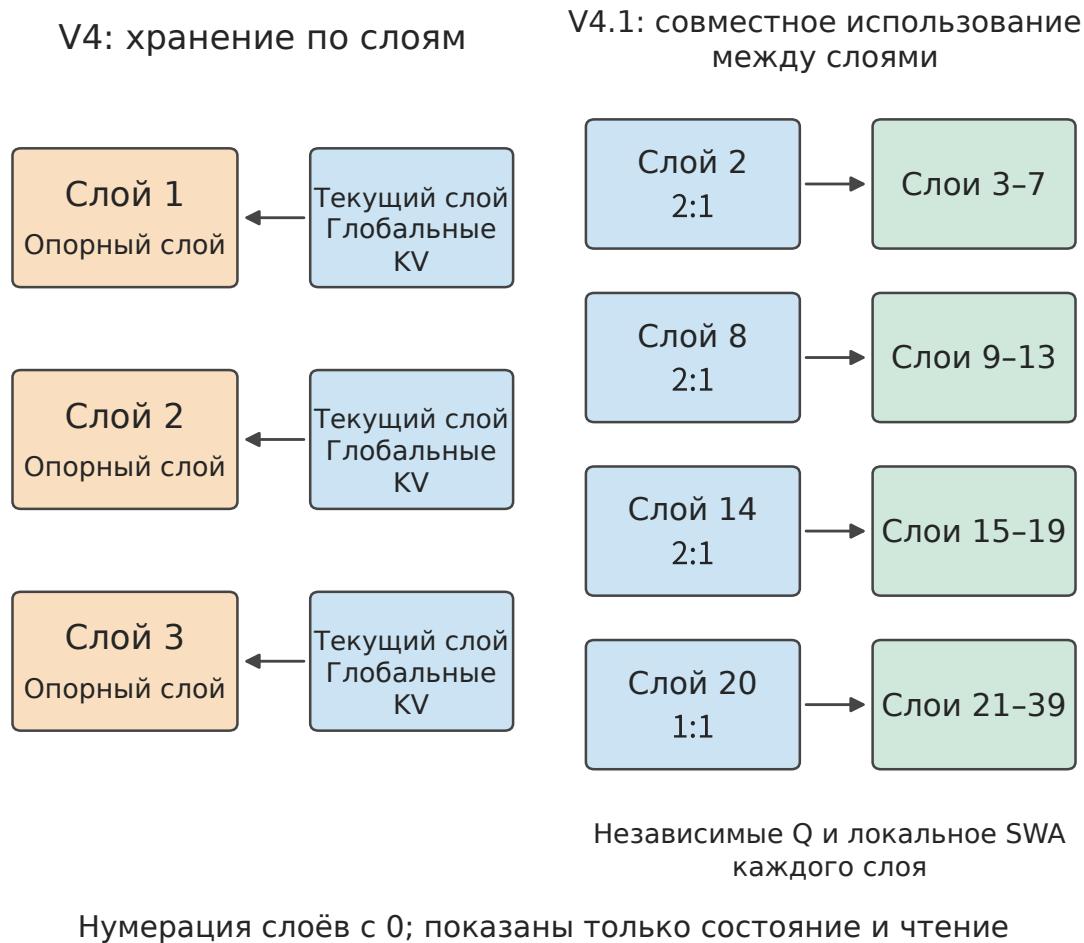


Рисунок 2-21. Способы хранения глобального KV в V4 и V4.1. Слева послойное хранение показано на примере трёх репрезентативных слоёв; справа показаны четыре слоя V4.1, независимо сохраняющие глобальный KV, и отношения совместного использования. Сплошные линии обозначают считывание данных; каждый использующий их слой по-прежнему имеет собственные Q и локальный SWA, остальные вычисления на схеме опущены.

са внимания и формировать новый выход. Совместное использование кэша устраняет один из видов независимых послойных представлений, но не превращает вычисления всех слоёв в одинаковые операции.

Подставим описанные выше отношения совместного использования в расчёт ёмкости. Выпущенная в сентябре 2026 года DeepSeek V4.1 Flash имеет 40 слоёв текстовой основы, но независимый глобальный KV есть только у четырёх слоёв: при нумерации с нуля слои 2, 8 и 14 создают одну сжатую запись на каждые два токена, а слой 20 создаёт одну запись на каждый токен; остальные слои глобального внимания совместно используют эти кэши. Основная запись содержит 512 значений в формате FP4 — формате с плавающей точкой, где каждое значение занимает 4 бита; кроме того, на каждые 16 значений приходится один байт scale, поэтому каждая запись занимает $512/2 + 512/16 = 288$ байт. Индексная запись занимает $128/2 + 128/32 = 68$ байт. Средний прирост хранилища глобального KV равен

$$\left(\frac{3}{2} + 1\right) (288 + 68) = 890 \text{ B/token.}$$

Для сравнения, прежняя V4-Flash имеет 21 слой со сжатием 4:1 и 20 слоёв со сжатием 128:1. В формате официальной FlashMLA — библиотеки DeepSeek с открытым исходным кодом для вычисления внимания MLA, — где основная запись занимает 584 В, а индексная — 68 В, средний прирост хранилища глобального KV равен

$$21 \frac{584 + 68}{4} + 20 \frac{584}{128} = 3514.25 \text{ B/token.}$$

Средние приросты глобального KV различаются примерно в 3.95 раза, что соответствует округлённым значениям 3,514 и 890 B/token на официальной схеме. Помимо кэша глобального контекста, локальное окно V4.1 сохраняет на каждом слое не более 128 записей; каждая запись FP8 — формата с плавающей точкой, где каждое значение занимает 1 байт, — вместе со scale занимает 528 В, что суммарно даёт 2.578 MiB для 40 слоёв. Глобальная история растёт вместе с контекстом, тогда как локальное окно после заполнения сохраняет фиксированную ёмкость.

При пошаговом рассмотрении процесса генерации виден также ритм роста глобального кэша. В V4.1 при переходе от чётной длины глобального кэша к следующей нечётной длине добавляется только 356 В — запись слоя 20, создаваемая для каждого токена. При переходе к следующей чётной длине каждый из трёх компрессоров со сжатием 2:1 на слоях 2, 8 и 14 завершает ещё по одной записи, поэтому на этом шаге суммарно добавляется $4 \times 356 = 1,424$ В. После заполнения окна последующие записи перезаписывают старые слоты, и ёмкость больше не растёт, однако запись данных продолжается: 40 слоёв SWA на каждом шаге суммарно записывают $40 \times 528 = 21,120$ В, не считая обновлений компрессоров.

Слоям, совместно использующим один глобальный кэш, всё равно требуется считывать данные, а некоторые слои дополнительно выполняют повторную индексацию. В сценарии 8K при указанном в таблице производственном смешанном формате объём логического считывания KV внимания, включая локальный SWA, и индекса составляет 12.556 MiB для V4-Flash и 11.375 MiB для V4.1 Flash; совместное использование между слоями сокращает главным образом пространство, занимаемое повторно сохранёнными данными.

Повторное использование кэша и выбор записей также можно организовать отдельно. CSA2 в V4.1 разделяет слои на три режима. Слои **Full** (полные) целиком выполняют процесс «создание глобального KV, индексирование, выбор и внимание». Слои **Reindex** (повторная индексация) совместно используют KV и заново выбирают top-k с помощью собственного индексного запроса; слои **Reuse** (повторное использование) повторно используют также уже выбранные записи кэша. Среди 38 слоёв глобального внимания V4.1 есть 4 слоя Full, 4 слоя Reindex и 30 слоёв Reuse; ещё два слоя используют только SWA. Первый слой Full декодировщика сканирует глобальное пространство и выбирает не более $2048 \times 8 = 16,384$ записей-кандидатов; в разделе 2.3.2 технического отчёта этот набор называется **пулом кандидатов (candidate pool)**, а его кандидатами являются записи кэша, доступные для извлечения механизмом внимания. Последующие слои Reindex независимо выбирают не более 512 записей только внутри этого пула.¹⁴ Поэтому объём

¹⁴В разделе 2.3.2 технического отчёта DeepSeek V4.1 Hierarchical Sparse Indexer использует candidate pool и candidate positions; Figure 5 демонстрирует различие между общим пулом кандидатов и последующим top-k.

поиска на последующих слоях ограничен сверху, но первоначальное глобальное сканирование по-прежнему растёт вместе с контекстом.

Например, если запросу доступны 131,072 записи глобального кэша, они разделяются на 16,384 блока по 8 записей. Первый слой Full декодировщика сначала оценивает все записи кэша, принимает максимальную оценку внутри блока за оценку блока, а затем выбирает 2,048 блоков, формируя 16,384 записи-кандидата. Сам этот слой по-прежнему выбирает top-512 из глобальных оценок; последующие слои Reindex с помощью новых индексных запросов пересчитывают оценки среди кандидатов и независимо выбирают по 512 записей кэша. Пул кандидатов ограничивает область поиска последующих слоёв, а top-512 каждого слоя определяет, какие записи в итоге считает этот слой.

Пул кандидатов не только устанавливает верхнюю границу объёма поиска для последующих слоёв, но и делает их выбор зависимым от первого этапа фильтрации: если запись кэша оказалась за пределами пула кандидатов, последующие слои Reindex уже не смогут выбрать её для этого запроса. Обучение модели должно учитывать такую область поиска; в главе 10 будет объяснено, как V4.1 включает ограничение по кандидатам в постобучение.

2.4 Условные вычисления и повторное использование весов экспертов

В разделе 2.3 объём состояния контекста уменьшался за счёт совместного использования, сжатия и рекуррентных вычислений. Другой основной источник данных — веса модели: обязательно ли для каждого токена использовать все параметры модели? В этом разделе данный вопрос рассматривается на примере FFN.

Плотная FFN использует для каждого токена один и тот же набор матриц; MoE распределяет параметры между несколькими экспертами, а маршрутизатор выбирает для каждого токена небольшое число экспертов. Благодаря этому объём хранимых моделью параметров и объём параметров, активируемых одним токеном, можно регулировать независимо, а выбор экспертов токенами внутри пакета также влияет на фактически используемый набор весов.

2.4.1 Плотная сеть прямого распространения и маршрутизация экспертов

Типичная ветвь MoE сначала вычисляет по входным данным оценки маршрутизации, выбирает по top-k экспертов с k наивысшими оценками, передаёт токен выбранным экспертам, а затем объединяет результаты с учётом весов маршрутизации. При наличии общего эксперта также существует ветвь, через которую проходят все токены. Сам эксперт обычно по-прежнему состоит из трёх матриц: gate, up и down, поэтому вычисления можно пошагово развернуть по схеме SwiGLU из раздела 2.1.4.

Рассмотрим Qwen3.6-35B-A3B. Её текстовая основа содержит 40 слоёв с размерностью скрытого состояния 2048; в каждом слое имеется 256 маршрутизируемых экспертов, из которых для каждого токена выбираются 8, промежуточная размерность эксперта равна 512, а также присутствует общий эксперт. Один маршрутизируемый эксперт содержит $3 \times 2048 \times 512 = 3,145,728$ параметров, которые при хранении в BF16 занимают 6 MiB. Все маршрутизируемые эксперты одного слоя занимают 1,5 GiB, а восемь экспертов, выбранных для одного токена, — 48 MiB.

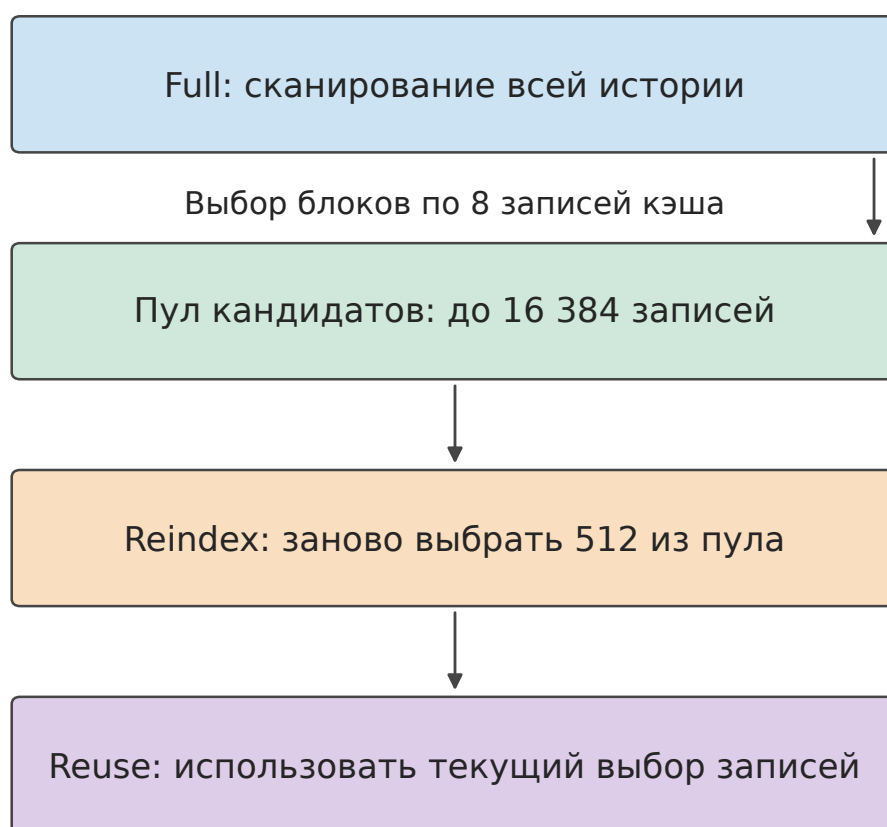


Рисунок 2-22. Иерархический выбор записей кэша декодировщиком V4.1. Первый слой Full сначала сканирует глобальное пространство и выбирает не более 16,384 записей-кандидатов; последующие слои Reindex повторно выбирают среди кандидатов 512 записей, а Reuse используют уже имеющийся выбор. Количество записей на схеме показано не в масштабе; независимые Q и локальный SWA каждого слоя не изображены.

Пусть эксперт e фактически получает t_e токенов. Тогда его gate/up имеют вид $[t_e, 2048] \times [2048, 512]$, а down — $[t_e, 512] \times [512, 2048]$; основной объём матричных вычислений равен $6t_e \times 2048 \times 512$. Сложив объёмы вычислений всех экспертов, получим общий объём матричных вычислений всех маршрутизируемых экспертов. Сначала маршрутизатор определяет значения t_e , выбранные эксперты выполняют три проекции, после чего результаты собираются и объединяются с учётом весов; общая ветвь обрабатывает все входные строки.

Обозначим через E общее число маршрутизируемых экспертов, через k_{top} — число экспертов, выбираемых для каждого токена, а через N_e — число параметров одного эксперта. Тогда в каждом слое хранится EN_e параметров маршрутизируемых экспертов, а объём выполняемых для одного токена матричных вычислений экспертов составляет приблизительно $2k_{\text{top}}N_e$. Увеличение E расширяет доступный для выбора набор параметров; непосредственно увеличить эту часть вычислений на один токен может только рост фактически выбираемого числа экспертов или их размера. Обозначение A3B в названии Qwen3.6 отражает объём активируемых параметров; всего текстовая модель содержит около 34,66 млрд параметров, а её веса в BF16 занимают примерно 69,321 GB.

2.4.2 Фактический объём вычислений и чтение весов внутри пакета

Величина $2k_{\text{top}}N_e$ из раздела 2.4.1 учитывает только работу экспертов для одного токена; при одновременной маршрутизации пакета токенов необходимо также учитывать, к скольким различным экспертам они попадут.

Пример 2-4. Почему при одинаковом объёме вычислений экспертов чтение весов различается в 32 раза? Сравним для одного слоя Qwen3.6 равномерное и концентрированное распределение.

Решение: даже при одинаковом числе активируемых параметров объём чтения весов внутри пакета может различаться. Предположим, что одновременно выполняется один шаг decode для $B = 64$ запросов, причём каждый запрос на этом шаге передаёт на вход один токен, поэтому $m = B = 64$. Каждый токен выбирает 8 экспертов, что в сумме даёт 512 назначений токенов экспертам. При равномерном распределении между 256 экспертами каждый эксперт в среднем получает 2 токена, и текущий пакет обращается ко всем маршрутизируемым экспертам. Если же все токены направляются одной и той же группе из 8 экспертов, каждый эксперт получает 64 токена, а текущий пакет обращается только к 8 наборам весов экспертов.

Пусть U — число различных экспертов, к которым фактически обращается данный пакет. Если в процессе маршрутизации не отбрасывается ни один назначенный экспертам токен, то $\sum_e t_e = Bk_{\text{top}}$, поэтому

$$F_{\text{expert}} = 2N_e \sum_e t_e = 2N_e Bk_{\text{top}}, \quad R_{\text{expert}} = b_W N_e U.$$

В первой формуле суммируется общее число назначенных строк, а во второй — фактически загружаемые различные веса. В обоих вариантах маршрутизации из примера выполняется 512 назначений токенов экспертам; один токен направляется восьми экспертам и занимает по одной строке во входной матрице каждого из них. Эти 512 строк образованы 64 исходными токенами, поэтому объём матричных вычислений одинаков; однако U принимает значения 256 и 8, а объём чтения на слой составляет соответственно 1,5 GiB и 48 MiB, то есть различается в 32 раза. Маршрутизируемые эксперты всех 40

Число назначений одинаково, но
эксперты могут различаться

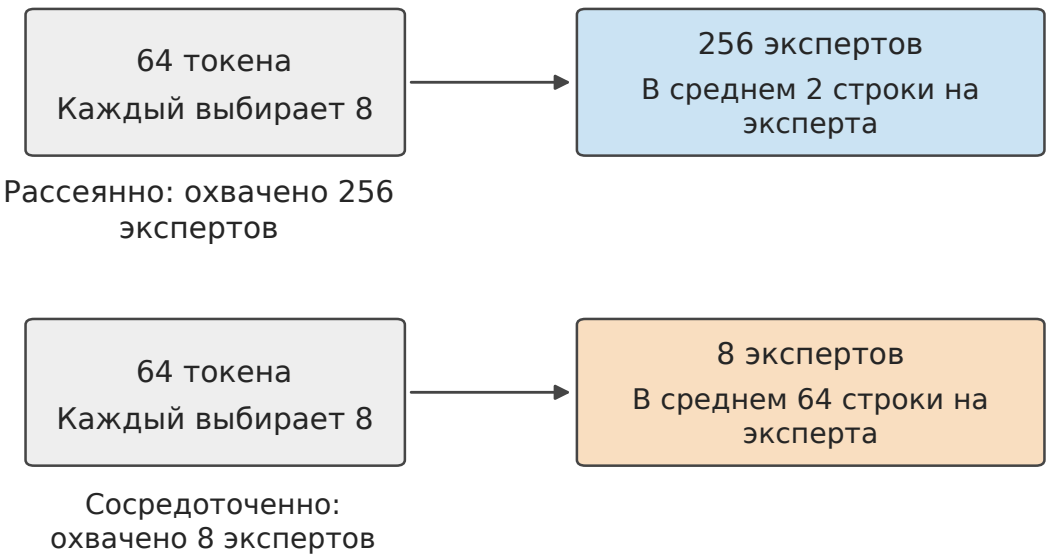


Рисунок 2-23. Каждый из 64 токенов выбирает восемь экспертов, что в сумме даёт 512 назначений. При рассеянном распределении они могут охватывать 256 экспертов, а при концентрированном используются только восемь; соответственно изменяется и число строк, обрабатываемых каждым экспертом.

слоёв вместе занимают 60 GiB, тогда как при концентрированной маршрутизации выбранные для одного пакета веса в сумме занимают лишь 1,875 GiB.

Вариант маршрутизации при $B = 64$	Различных экспертов, используемых в слое	Токенов на эксперта	Идеальный объём чтения маршрутизируемых экспертов на слой
Равномерное распределение	256	2	1,5 GiB
Концентрированное распределение в одну и ту же группу из 8 экспертов	8	64	48 MiB

Различие в объёме чтения обусловлено числом входных строк, для которых используется каждый набор весов: при равномерном распределении каждый эксперт обрабатывает два токена, а при концентрированном — 64 токена. Объём вычислений с участием одного и того же набора весов возрастает в 32 раза, а идеальный объём обращения к весам сокращается до одной тридцать второй исходного значения. Одновременно вместо множества матричных умножений с входами всего из двух строк теперь требуется выполнить лишь небольшое число матричных умножений с входами из 64 строк; это влияет как на повторное использование весов, так и на эффективность вычислений.¹⁵

¹⁵Равномерная маршрутизация; концентрированная маршрутизация.

Теперь сравним модели целиком, суммировав показатели по всем слоям: для Qwen3.6 при контексте из 8192 уже имеющихся токенов и decode с $B = 1$ требуется около 7,33 GFLOPs, что меньше примерно 20,0 GFLOPs у Qwen3-8B. Различие в объёме вычислений всей модели обусловлено несколькими архитектурными изменениями: основа размерностью 2048 уменьшает проекции, 30 линейных слоёв используют рекуррентное состояние, а в MoE каждый раз выполняются вычисления только выбранных экспертов и общей ветви. Увеличение числа экспертов повышает общее число параметров, тогда как объём работы при каждом вызове зависит от того, какие вычисления фактически выполнялись.

Разреженная активация позволяет независимо регулировать общее число параметров и число параметров, фактически используемых каждым токеном, благодаря чему модель может обращаться к более крупному набору параметров при меньшем объёме вычислений. При обслуживании результаты маршрутизации определяют, для скольких токенов повторно используются веса каждого эксперта; пример 2-4 как раз демонстрирует различие между этими двумя способами доступа.

2.4.3 Общие эксперты и вычисления в латентном пространстве

Описанное выше различие определяется числом входных строк, получаемых экспертами. Необходимо также учитывать размер самих экспертов: общий эксперт обрабатывает все входные данные, а эксперты в латентном пространстве предварительно уменьшают размерность входа. Оба подхода изменяют как необходимый одному эксперту объём вёсов, так и объём выполняемых им вычислений.

В каждом слое DeepSeek V4-Flash имеется 256 маршрутизируемых экспертов, из которых для каждого токена выбираются 6, а также 1 общий эксперт; промежуточная размерность маршрутизируемого эксперта равна 2048. Матрицы gate/up одного эксперта имеют вид $[t_e, 4096] \times [4096, 2048]$, а down преобразует ширину с 2048 обратно в 4096; три проекции вместе содержат 25 165 824 параметра. В первых трёх слоях используется хеш-маршрутизация, а в последующих — маршрутизация по оценкам: хеш-маршрутизация определяет эксперта по токenu, тогда как маршрутизация по оценкам выбирает эксперта на основании текущего представления.

Общий эксперт обрабатывает векторы признаков всех m токенов текущего входа, а маршрутизируемый эксперт e — только назначенные ему t_e токенов. Если общий эксперт содержит N_s параметров, объём матричных вычислений экспертов одного слоя равен $2mN_s + 2 \sum_e t_e N_e$. При обычном одношаговом decode каждый запрос передаёт один новый токен, поэтому $m = B$; при prefill необходимо суммировать все токены, обрабатываемые на этом шаге для каждого запроса. В DeepSeek V4-Flash для каждого токена выбираются шесть маршрутизируемых экспертов и выполняется один общий эксперт, поэтому объём вычислений представляет собой сумму этих двух составляющих; при этом для хранения необходимо сохранять все 256 маршрутизируемых экспертов и общую ветвь.

В каждом слое MoE модели Kimi K3 имеется 896 маршрутизируемых экспертов, из которых для каждого токена выбираются 16. Вход сначала проецируется из основного пространства размерностью 7168 в латентное пространство размерностью 3584, затем проходит преобразование эксперта с промежуточной размерностью 3072, а после объединения проецируется обратно в основное пространство. Общий эксперт непосредственно обрабатывает основное представление. Уменьшение размерности сокращает размеры

матриц маршрутизируемых экспертов, но добавляет две проекции — при входе в латентное пространство и выходе из него.¹⁶

Если эксперты размещены на разных картах, входные токены необходимо отправить на карты, где расположены соответствующие эксперты, а после вычислений объединить результаты. Поэтому объём коммуникации зависит от размещения экспертов и места хранения входных данных. В главе 6 рассматривается распределение по нескольким картам, а в главе 9 — размещение на CPU/GPU и разделение AF, при котором Attention и FFN передаются разным ресурсам.

Упражнение 2-6 (расширенное). Как число экспертов и способ распределения влияют на вычисления и чтение весов

Для одного слоя Qwen3.6 возьмите $B = 64, 256$ маршрутизируемых экспертов и выбор 8 экспертов для каждого токена. Отдельно рассмотрите равномерное распределение и концентрированное распределение в одну и ту же группу из 8 экспертов, запишите число токенов t_e , получаемых каждым экспертом, а затем вычислите FLOPs, число используемых экспертов и объём чтения весов в BF16. Затем вдвое уменьшите общее число экспертов, вдвое увеличьте промежуточную размерность и вдвое уменьшите число экспертов, выбираемых для каждого токена, после чего определите, изменятся ли общее число параметров экспертов и фактический объём выполняемых матричных вычислений экспертов. Наконец, объясните, как добавить к этим расчётам число параметров и объём вычислений общего эксперта; для модели с экспертами в латентном пространстве также объясните, как учесть проекции при входе в латентное пространство и выходе из него.

2.5 Форма сети и организация вычислений

Состояние контекста и выбор экспертов изменяют основные наборы данных, а глубина, ширина и способ соединения сети дополнительно определяют размер используемых матриц и порядок выполнения операций. Модели с близким числом параметров могут иметь разные вычислительные графы; даже у моделей с близким числом матричных FLOPs время хранения буферов и количество последовательных шагов могут различаться.

2.5.1 Глубина, ширина и гранулярность экспертов

Сначала не будем учитывать словарь и параметры малых векторов. Если ширина FFN сохраняет постоянное отношение к размерности скрытого состояния, число параметров плотной основной сети приблизительно пропорционально Ld^2 . Уменьшив размерность скрытого состояния вдвое и увеличив число слоёв в четыре раза, можно сохранить примерно то же число параметров этой части, однако длина цепочки послойных зависимостей также возрастёт в четыре раза. Каждая матрица станет меньше, но вычисления должны будут пройти через большее число слоёв; кроме того, для каждого слоя потребуется сохранять соответствующее состояние контекста. Таким образом, при близком бюджете параметров число слоёв и их ширина изменяют гранулярность вычислений и глубину зависимостей.

¹⁶Расчёт экспертов Kimi K3.

Структуру экспертов также можно изменить подобным образом, сохранив число параметров и объём вычислений. Общее число параметров экспертов с маршрутизацией в одном слое равно $3Edf$, а объём вычислений экспертов для одного токена — $6k_{\text{top}}df$. Если вдвое уменьшить число экспертов E , вдвое увеличить промежуточную размерность f и вдвое уменьшить число выбранных экспертов k_{top} , обе величины останутся неизменными. В качестве такого сопоставления можно сравнить 128 экспертов шириной 1536 в Qwen3-235B-A22B с выбором 8 экспертов для каждого токена и 64 эксперта шириной 3072 с выбором 4 экспертов для каждого токена. При этом размерность выхода маршрутизатора и число строк, поступающих каждому эксперту, всё равно изменятся, а значит, изменится и форма группового матричного умножения, при котором матричные умножения нескольких экспертов организуются как одна операция.

Проверим базовую конфигурацию и два варианта на числовом примере. В примере с 16 токенами объём матричных вычислений экспертов остаётся одинаковым для трёх вариантов гранулярности экспертов; как для базовой конфигурации, так и для варианта с 64 экспертами он составляет около 454 GFLOPs. При уменьшении числа экспертов вдвое и удвоении ширины каждого эксперта то же количество весов распределяется между меньшим числом крупных матриц; при удвоении числа экспертов и уменьшении ширины вдвое получаем большее число малых матриц. Маршрутизатор должен вычислять оценку маршрутизации для каждого эксперта, поэтому число его параметров также изменяется вместе с числом экспертов. В расчёте гранулярности экспертов вычисления маршрутизатора и матриц экспертов суммируются отдельно, показывая, почему при одинаковом общем числе параметров экспертов формы матриц всё же различаются.

Формы матриц и состояний также определяют минимальную единицу распределения работы между несколькими ускорителями. 128 экспертов можно разделить на восемь групп по 16 экспертов; четыре KV-головы при распределении только целыми головами образуют лишь четыре части, поэтому для выполнения на восьми ускорителях потребуется копировать часть KV или изменить группировку. В главе 5 эффективность выполнения рассматривается на основе разбиения матриц, а в главе 6 эти матрицы и состояния будут распределены между отдельными ускорителями.

2.5.2 Остаточные соединения и межслойные состояния

Помимо числа слоёв и их ширины, необходимо учитывать способ соединения слоёв. Остаточное соединение напрямую прибавляет вход подслоя к результату преобразования, создавая для исходной информации и градиента прямой путь через слои. Обычное остаточное соединение вычисляется как $x_{\ell+1} = x_{\ell} + f_{\ell}(x_{\ell})$, где f_{ℓ} — преобразование ℓ -го подслоя. Поэтому во время вычислений подслоя необходимо сохранять вход x_{ℓ} , чтобы прибавить его после получения результата преобразования. Если одно и то же представление используется ещё в нескольких последующих слоях, его необходимо хранить до завершения последнего использования. Таким образом, способ соединения непосредственно определяет, какие тензоры требуется сохранять одновременно.

Обычное остаточное соединение непрерывно складывает информацию с разных глубин в одном пути. Чтобы предоставить больше способов комбинирования межслойной информации и одновременно сохранить устойчивое распространение сигнала в глубокой сети, DeepSeek V4-Flash использует **mHC (Manifold-Constrained Hyper-Connections, гиперсоединения с ограничением на многообразии)**. mHC расширяет один остаточный путь до нескольких и накладывает ограничения на коэффициенты

смешивания между ними; в данном случае «ограничение на многообразии» означает, что элементы матрицы остаточного смешивания ограничиваются неотрицательными значениями, а суммы в каждой строке и каждом столбце должны быть близки к 1.¹⁷

Модель сохраняет четыре остаточных состояния, каждое из которых представляет собой 4096-мерное промежуточное представление текущего токена. Четыре представления сохраняются, чтобы последующие слои могли обучиться комбинировать информацию из разных путей. Каждый подслой сначала смешивает четыре представления в 4096-мерный вход подслоя, выполняет вычисление внимания или FFN, а затем записывает результат обратно в остаточные пути. Поэтому вход основных матриц остаётся 4096-мерным, а дополнительная работа сосредоточена на смешивании и объединении при входе в подслой и выходе из него. Четыре состояния позволяют передавать между слоями больше наборов представлений, а операции смешивания связывают эти представления с существующими соединениями подслоёв.

При обработке 8192 входных токенов проекции смешивания mHC требуют около 555 GFLOPs, а нематричные операции — около 149 GFLOPs. Матрица остаточного смешивания сначала нормализуется с помощью **итераций Sinkhorn**: каждая строка и каждый столбец поочерёдно делятся на соответствующую сумму, благодаря чему суммы строк и столбцов постепенно приближаются к 1. Это делает численный масштаб более устойчивым при смешивании во множестве слоёв. Затем нормализованные коэффициенты используются для комбинирования остаточных путей; в конце эти пути объединяются, образуя представление, которое передаётся в словарную голову. Таким образом, многопутевое остаточное соединение не только увеличивает число сохраняемых представлений, но и добавляет вычисления смешивания и нормализации в каждом слое.¹⁸

В Kimi K3 ставится задача дать текущему слою возможность при необходимости выбирать информацию с разных глубин, а не только получать единый смешанный результат последовательного послойного сложения. Для этого Kimi K3 использует **Attention Residuals (AttnRes, остаточные соединения с attention)**: представлениям более ранних слоёв назначаются веса, после чего их взвешенная сумма образует вход текущего слоя. Этот механизм внимания выбирает информацию вдоль глубины сети, тогда как предшествующее внимание к последовательности выбирает информацию между различными токенами. Чтобы сократить затраты на хранение и чтение выходов всех слоёв, блочная реализация объединяет несколько слоёв в блок и сохраняет представления, доступные для выбора последующими блоками.¹⁹

Поскольку эти представления блоков потребуются последующим слоям, их необходимо продолжать хранить во время текущего прямого прохода, но после его завершения их можно освободить. Контекстные KV, напротив, необходимо сохранять для последующих шагов генерации. Сроки хранения этих данных различаются: AttnRes увеличивает объём временного хранилища в рамках одного прямого прохода, а KV сохраняются между несколькими вызовами в рамках запроса.²⁰

¹⁷В разделе 2.2 технического отчёта DeepSeek V4 описаны многоканальные представления mHC и стабильное распространение, а в разделе 2.3 — CSA/HCA; технический отчёт Kimi K3 объясняет мотивацию выбора информации по глубине с помощью Attention Residuals и блочную реализацию.

¹⁸Объём вычислений mHC.

¹⁹В разделе 2.2 технического отчёта DeepSeek V4 описаны многоканальные представления mHC и стабильное распространение, а в разделе 2.3 — CSA/HCA; технический отчёт Kimi K3 объясняет мотивацию выбора информации по глубине с помощью Attention Residuals и блочную реализацию.

²⁰Журнал послойного планирования.

Обычная остаточная связь: одна обходная ветвь



Исходный вход складывается с выходом подслоя

mНС: четыре ветви с последующим смешением



Четыре ветви объединяются для подслоя, затем смешиваются с его выходом

Рисунок 2-24. Область соединений обычного остаточного соединения и четырёхпутевого mНС. Обычное остаточное соединение сохраняет один обходной путь для входа подслоя; mНС объединяет несколько состояний для вычислений подслоя, а затем смешивает выход. Вход подслоя в обеих схемах имеет размерность 4096.

2.5.3 Предсказание нескольких токенов и вспомогательные вычисления

В двух предыдущих разделах изменялись формы и соединения внутри основной сети; другой класс архитектур добавляет к её выходу модули предсказания, изменяя число токенов, которые можно получить за один прямой проход. Обычная авторегрессионная основная сеть предсказывает следующий токен на основе текущего контекста. **Предсказание нескольких токенов (Multi-Token Prediction, MTP)** добавляет цели предсказания нескольких последующих токенов или вспомогательные модули предсказания. Во время обучения эти дополнительные цели побуждают текущее представление включать информацию, полезную для более отдалённого продолжения; во время инференса вспомогательный модуль может сначала предложить несколько токенов-кандидатов, а затем основная модель проверит их, сократив число последовательных раундов, необходимых для генерации того же количества токенов.

Спекулятивное декодирование (speculative decoding) сначала генерирует последовательность кандидатов с меньшими затратами, после чего целевая модель проверяет эти кандидаты. При использовании MTP для спекулятивного декодирования последующие токены предварительно генерируются на основе текущих признаков основной сети, а затем передаются целевой модели для проверки. Если проверку проходят несколько токенов, за один раунд можно получить несколько действительных выходов; в противном случае генерация продолжается с позиции после последнего прошедшего проверку токена. Следовательно, ускорение зависит от времени генерации черновика, времени проверки целевой моделью и числа токенов, проходящих проверку в каждом раунде. В главе 8 будет рассчитана взаимосвязь между этими тремя величинами.

Рассчитаем одно вспомогательное предсказание на примере DeepSeek V4-Flash. Его вспомогательный блок MTP получает признаки основной сети и token ID, после чего предсказывает последующую позицию. Для одного токена, то есть при $B = 1, P = 1$, одно вспомогательное вычисление требует около 1,80 GFLOPs, из которых около 1,06 GFLOPs приходится на общую словарную голову. Общая словарная голова составляет более половины объёма этого вычисления: хотя она не добавляет отдельных весов, при каждом вызове всё равно необходимо выполнять словарную проекцию.²¹

Благодаря вспомогательному предсказанию один прямой проход может предоставить несколько кандидатов, из которых после проверки и сэмплирования формируется действительный выход. В главе 3 кодирование, генерация, итерации и вывод в реальном времени будут объединены в полную нагрузку.

Пусть один обычный шаг decode занимает T_d , а суммарное время подготовки черновика и проверки в одном раунде спекулятивного декодирования составляет $T_s + T_v$; при этом в среднем получается $\bar{g}$ действительных выходов. Тогда время на один выход равно $(T_s + T_v)/\bar{g}$, а условие получения выигрыша имеет вид

$$\bar{g} > \frac{T_s + T_v}{T_d}.$$

Например, если обычный шаг decode занимает 20 ms, подготовка черновика и проверка вместе занимают 30 ms, а за один раунд в среднем получаются два действительных выхода, то на каждый выход приходится 15 ms; если же получается только один выход, потребуется 30 ms. Предсказание большего числа позиций увеличивает объём работы в одном раунде, поэтому дополнительные вычислительные затраты компенсируются

²¹Расчёт MTP для DeepSeek V4-Flash.

лишь тогда, когда принимается достаточно много выходов.

2.6 Требования к ресурсам всей модели и сравнение

В разделах 2.3–2.5 были представлены три типа структурных изменений: представление контекста определяет размер состояния и способ доступа к нему, выбор экспертов определяет, какие веса используются при каждом вычислении, а глубина сети и способ соединения слоёв определяют порядок вычислений. В этом разделе мы объединим эти части, сравним объём вычислений и требования к памяти полных моделей при одинаковых входных данных, а затем проверим выходные данные моделей на конкретных задачах.

2.6.1 Требования компонентов к ресурсам и сводные показатели моделей

Структуры внимания и конфигурации экспертов пяти моделей. В этой главе выбраны одна плотная базовая модель и четыре модели МоЕ с различными архитектурами, требования к ресурсам которых сравниваются по единым правилам расчёта матриц. Суффикс Т у количества параметров в таблице 2-А означает триллионы.

Таблица 2-А. Общая архитектура пяти моделей (текстовая часть)

Параметр архитектуры	DeepSeek V4.1 Flash	Qwen3-8B	Qwen3.6-35B-A3B	DeepSeek V4-Flash	Kimi K3
Общее число логических параметров, приблизительно	551.57B + 196.93B	8.19B	34.66B	284.33B	2.78T
Число основных слоёв L	Engram 40 (20 + 20)	36	40	43	93
Размерность скрытого пространства основной части d	5120	4096	2048	4096	7168
Состав внимания	2 SWA + 38 CSA2	36 GQA	30 линейных + 10 с полным вниманием	2 оконных + 21 CSA + 20 HCA	69 KDA + 24 MLA
Состав прямого распространения	40 слоёв MoE	Плотный SwiGLU	40 слоёв MoE	43 слоя MoE	Первый слой плотный + 92 слоя MoE
Число маршрутизируемых экспертов E в каждом слое MoE	384	—	256	256	896
Число выбираемых экспертов k на токен	6	—	8	6	16
Вычислительная ширина маршрутизируемого эксперта d_e	5120	—	2048	4096	3584
Промежуточная ширина плотного FFN / маршрутизируемого эксперта f	2304	12288	512	2048	3072
Промежуточная ширина общего эксперта	2304	—	512	2048	Всего 6144
Межслойные соединения	Single-Pass mHC, 4 пути	Обычные остаточные соединения	Обычные остаточные соединения	mHC, 4 пути	AttnRes

В таблице приведены текстовые части пяти моделей. Приблизительно 551.57B параметров основной части V4.1 Flash и 196.93B параметров Engram указаны отдельно; последние включают около 196.61B параметров таблицы поиска, а также параметры её проекций и вентиляей. Оба компонента входят в полный объём текстовых весов, рассматриваемый далее. Single-Pass mHC в V4.1 представляет собой усовершенствованную версию mHC: каждый слой использует коэффициенты смешивания входов, вычисленные предыдущим слоем, благодаря чему остаточное состояние требуется прочитать лишь один раз. В Kimi K3 на разных слоях применяются разные структуры прямого распространения: первый слой представляет собой плотный FFN с промежуточной размерностью 33792, а последующие слои используют маршрутизируемых экспертов с промежуточной размерностью 3072. В плотном слое каждый токен проходит через один и тот же набор широких матриц, тогда как в слое MoE для каждого токена используется несколько наборов более узких экспертных матриц.

Этапы выполнения V4.1 Flash устроены следующим образом: 40 слоёв разделены на 20 слоёв причинного кодировщика и 20 слоёв декодера. Независимый глобальный KV сохраняется только на слоях 2, 8, 14 и 20, а индексатор выполняется на слоях 2, 8, 14, 20, 24, 28, 32 и 36; нумерация слоёв начинается с нуля. Слои Full создают кэш и индекс, слои Reindex пересчитывают индекс, а слои Reuse повторно используют уже сделанный выбор; при этом каждый слой по-прежнему сохраняет собственное локальное окно. Поэтому число слоёв больше не равно числу независимых глобальных кэшей, а длина входа больше не совпадает с числом токенов запросов, фактически обрабатываемых каждым слоем.

Из таблицы также следуют три соотношения. Общее число параметров Qwen3.6 больше, чем у Qwen3-8B, хотя ширина её основной части вдвое меньше: значительная часть параметров размещена в выбираемых экспертах, из которых для каждого токена вызываются только восемь. Основные части Qwen3-8B и DeepSeek V4-Flash имеют одинаковую размерность 4096, однако DeepSeek V4-Flash формирует более крупный набор параметров за счёт 256 экспертов на каждом слое. Kimi K3 дополнительно переносит вычисления экспертов в латентное пространство размерности 3584, благодаря чему ширина входа экспертов может не совпадать с шириной основной части. Эти различия показывают, что «какой объём модели необходимо хранить» и «какой объём работы выполняется для одного токена» следует рассчитывать отдельно. В таблицах 2-1, 2-2, 2-4, 2-5 и 2-6 эти пять моделей рассматриваются по отдельности, а таблица 2-3 для DeepSeek-V3 служит историческим ориентиром для MLA.

Зная параметры одного типа слоя, вычисления одного вызова и состояние одного запроса, показатели всей модели можно получить взвешенным суммированием этих величин по фактическому числу слоёв каждого типа:

$$M_W = b_W \sum_g L_g N_g + M_{\text{embedding}} + M_{\text{head}}, \quad F_{\text{model}} = \sum_g L_g F_g + F_{\text{head}}.$$

g обозначает тип слоя модели; слои одного типа используют одинаковые структуры внимания, сети прямого распространения и соединений. L_g — число слоёв типа g . Для MoE величина N_g включает всех экспертов, а в F_g используется фактическое число строк, получаемых каждым экспертом. В таблице 2-1 уже построена плотная базовая модель Qwen3-8B; справочные таблицы в конце главы аналогичным образом подробно рассматривают Qwen3.6, DeepSeek-V3, DeepSeek V4-Flash, Kimi K3 и DeepSeek V4.1

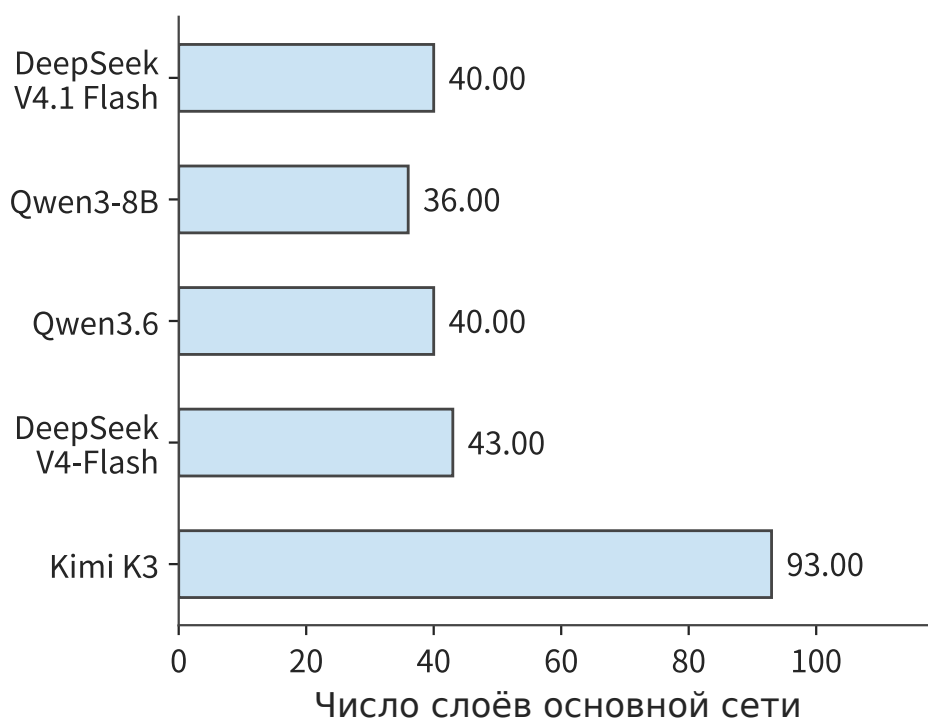


Рисунок 2-25. Число основных слоёв пяти моделей в едином линейном масштабе. Число слоёв определяет, сколько раз однотипная работа повторяется по глубине сети.

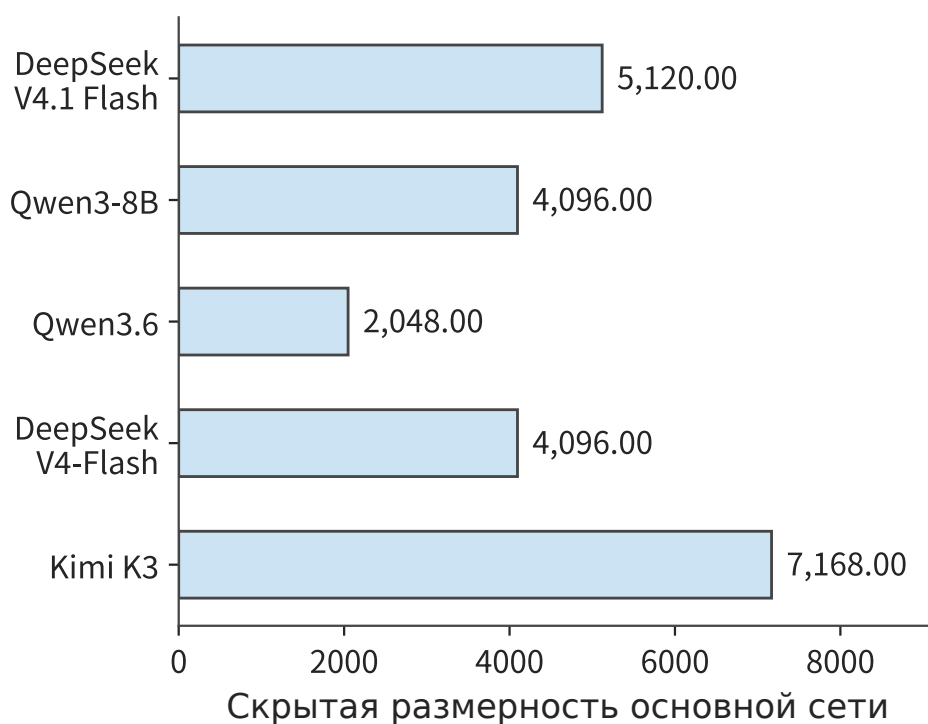


Рисунок 2-26. Размерность скрытого пространства основной части в едином линейном масштабе. Ширина признаков каждого токена определяет размер входа или выхода проекции.

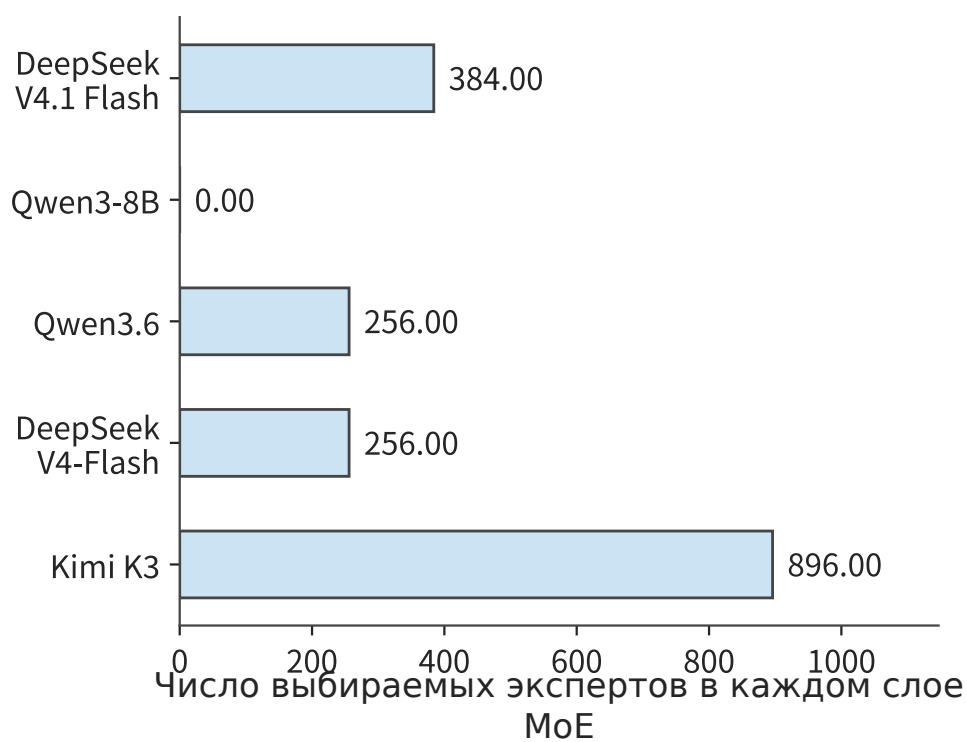


Рисунок 2-27. Число маршрутизируемых экспертов, хранящихся в каждом слое MoE. Qwen3-8B использует плотные слои прямого распространения, поэтому число маршрутизируемых экспертов равно нулю; фактическое число выбираемых экспертов на токен приведено в таблице 2-А.

Flash, позволяя читателю определить, из какой матрицы происходит каждая составляющая итогового значения.

При чтении можно проследить путь данных: в слой внимания какого типа поступает вход, через сеть прямого распространения какого типа он затем проходит и каким образом в итоге получаются оценки словаря. При суммировании также необходимо учитывать выборку строк эмбеддингов, конечную нормализацию и словарную голову. Каждая таблица разделена на группы внимания, прямого распространения и соединений, а также выходного интерфейса; столбец «Число слоёв / вызовов» указывает суммарный множитель для всей модели, тогда как «Матричные FLOPs» по-прежнему обозначают число операций с плавающей точкой при однократном выполнении соответствующей операции строки.

Применяя те же правила суммирования к другим моделям, можно поэлементно перечислить вычислительные ветви каждой из них. В следующей таблице для каждой модели указаны состояния контекста, веса и дополнительные вычисления, которые необходимо суммировать, а также компоненты, чаще всего упускаемые при подсчёте итогового значения. MTP, DSpark (модуль спекулятивного декодирования V4.1) и модуль обработки изображений добавляются отдельно в зависимости от области рассматриваемого выполнения.

Ветвь модели	Состояние контекста	Веса и дополнительные вычисления	Компоненты, чаще всего упускаемые при подсчёте
DeepSeek V4.1 Flash	4 общих глобальных KV / индекса + SWA каждого слоя + буфер сжатия и история токенов	CED, CSA2, Engram, эксперты, Single-Pass mHC	Разное число позиций для полного входа кодировщика, воспроизведения конца в декодере и глобальных проекций
Qwen3-8B	GQA KV	Плотный SwiGLU, входная голова	Нематричная арифметика, диапазон позиций выходной головы
Qwen3.6	Глобальный KV + рекуррентное состояние и состояние свёртки	Маршрутизируемые / общие эксперты	Линейные слои сохраняют рекуррентное состояние фиксированного размера
DeepSeek-V3	Латентные переменные MLA или развёрнутый KV в зависимости от пути выполнения	Плотный FFN на первых трёх слоях, последующие MoE и общие эксперты	Позиции выполнения восходящей проекции, различия двух путей кэширования
DeepSeek V4-Flash	Окно + сжатие + индекс + буфер	Эксперты, компрессор, mHC	Сканирование индекса, обновление границ блоков, общая ветвь
Kimi K3	MLA + KDA + свёртка	Эксперты в латентном пространстве, AttnRes	Путь развёртывания, временные межслойные состояния, отличие первого слоя

Сначала последовательно сравним полные веса, объём вычислений за один шаг и размер состояния контекста, а затем с помощью детализированных таблиц моделей покажем, какие матрицы и состояния создают эти различия.

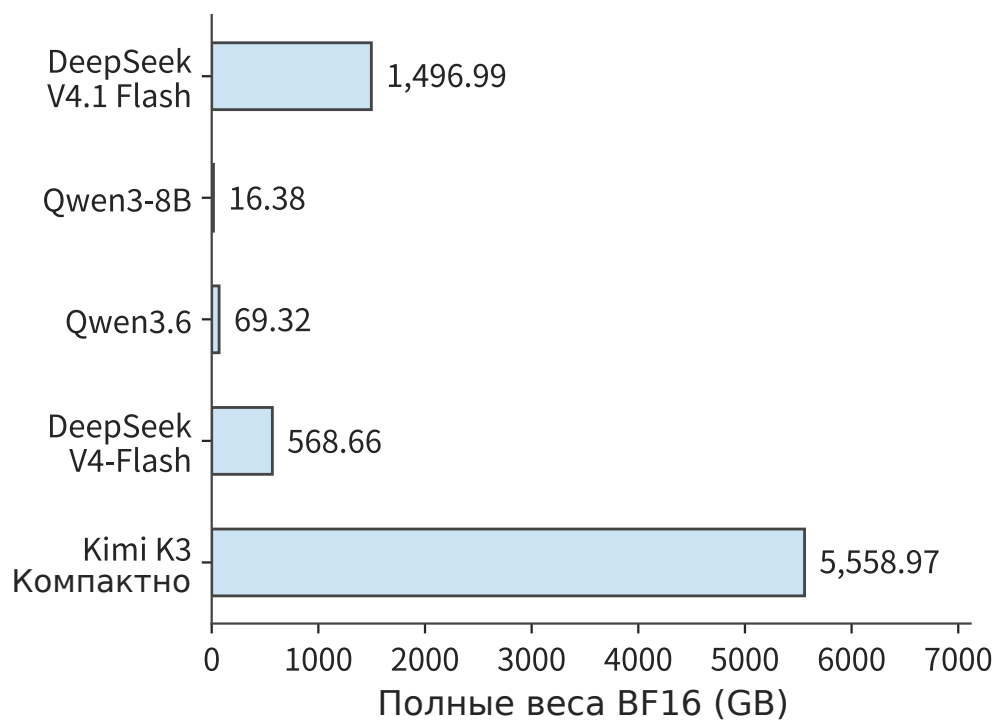


Рисунок 2-28. Объём полных весов пяти моделей при едином представлении BF16 в одинаковом линейном масштабе. Здесь учитываются все параметры, включая маршрутизируемых экспертов, из которых при каждом вызове выбирается лишь часть.

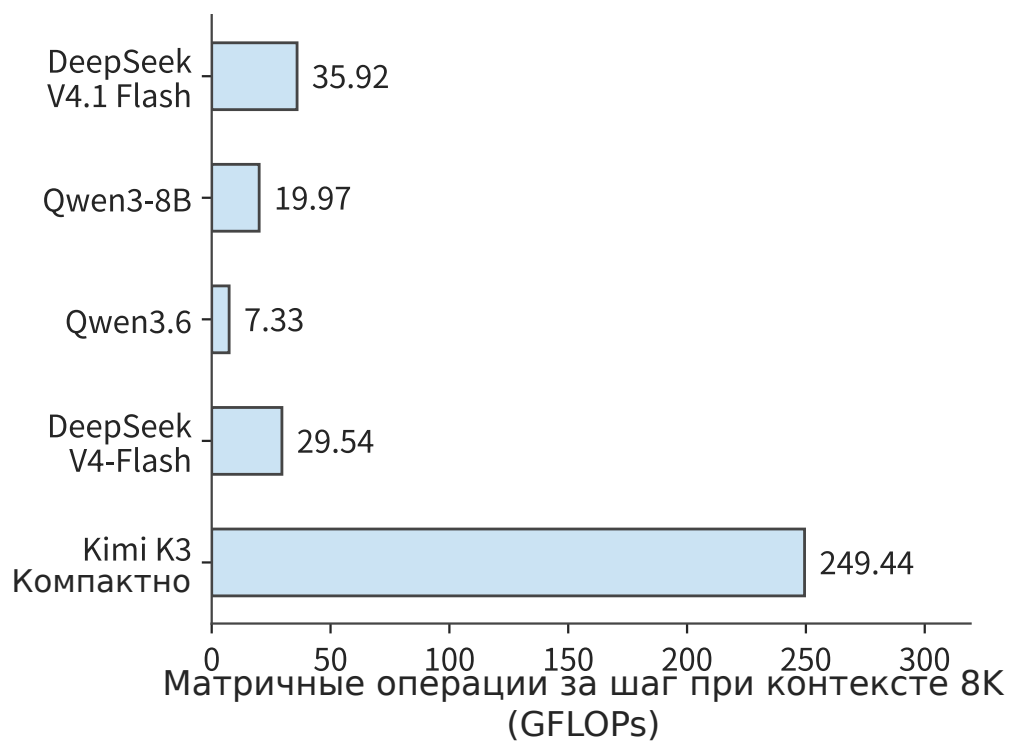


Рисунок 2-29. Объём матричных вычислений за один шаг для одного и того же одиночного запроса при уже имеющемся контексте 8K. Суммирование выполнено по путям выполнения моделей, описанным в таблице 2-С; для Kimi K3 используется компактный MLA.

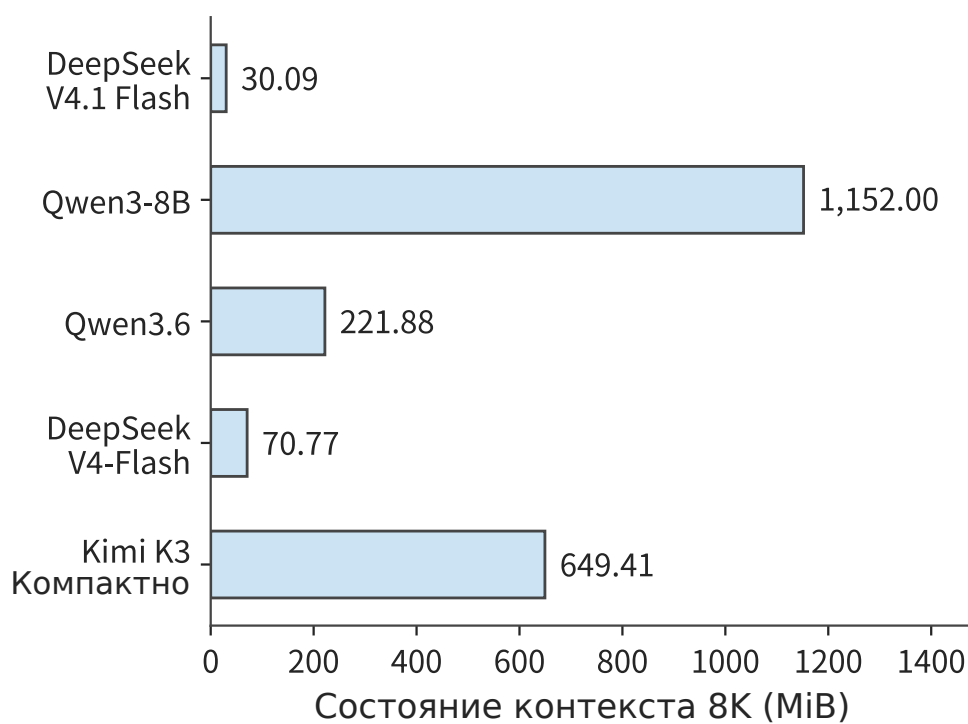


Рисунок 2-30. Состояние на запрос при одинаковом контексте 8K. Представления контекста используют BF16, а рекуррентные матрицы и буферы сжатия — точность соответствующей реализации; итоговая величина представляет собой объём, необходимый для хранения 8192 токенов контекста одного запроса.

Подробное разложение пяти моделей по матрицам приведено в справочных таблицах матриц моделей в конце главы. Далее различия в ресурсах объясняются с помощью промежуточных итогов.

Если перемножить размеры матриц и просуммировать их по числу слоёв, основная часть DeepSeek V4-Flash содержит около 284.3 миллиарда логических параметров, что при едином представлении BF16 составляет примерно 568.7 GB.

Текстовая основная часть Kimi K3 содержит около 2.78 триллиона параметров, подавляющее большинство которых находится в маршрутизируемых экспертах 92 слоёв MoE. На каждом слое для каждого токена выбираются только 16 экспертов, поэтому объём хранения полной модели и объём вычислений на один токен зависят от разных факторов: первый определяется всеми экспертами, а второй — выбранными экспертами, а также совместно выполняемыми вниманием, общими экспертами и проекциями.²²

Далее параметры каждой модели разделяются на шесть непересекающихся категорий: эмбединги и выход, проекции внимания, плотные или общие FFN, маршрутизируемые эксперты, Engram и остальные компоненты. Сумма параметров всех категорий даёт общее число параметров модели, а пересчёт по 2 байта на параметр — объём весов на рисунке 2-28. Такое разложение наглядно показывает распределение параметров и служит основой для объяснения следующей таблицы.

²²Описание конфигурации модели Kimi K3; вычисления прямого прохода Kimi K3.

Таблица 2-В. Составляющие параметров (единица измерения: миллиарды логических параметров)

Модель	Эмбединги и выход- ная голова	Проекции внима- ния	Плотные / общие FFN	Маршрутизируемые экспер- ты	Engram	Прочее
DeepSeek V4.1 Flash	1.324	5.126	1.416	543.582	196.929	0.118
Qwen3-8B	1.245	1.510	5.436	0.000	0.000	< 0.001
Qwen3.6	1.017	1.283	0.126	32.212	0.000	0.022
DeepSeek V4-Flash	1.059	5.085	1.082	277.025	0.000	0.081
Kimi K3 (компактный MLA)	2.349	36.180	12.882	2722.741	0.000	5.333

Категория «Прочее» включает маршрутизаторы, входные и выходные проекции латентного пространства, нормализацию, свёртки, соединения и другие параметры; проекции внимания включают проекции сжатия и индексирования DeepSeek V4-Flash. Основные параметры Qwen3-8B сосредоточены в плотном FFN, выполняемом при обработке каждого токена, тогда как Qwen3.6, V4-Flash и Kimi K3 размещают большую часть параметров в маршрутизируемых экспертах. V4.1 Flash дополнительно хранит крупную таблицу Engram; параметры её таблицы поиска, проекций и вентилей выделены в таблице отдельно. Параметры таблицы поиска увеличивают объём полной модели, но это не означает, что для каждого токена требуется обходить всю таблицу. На маршрутизируемых экспертах приходится около 93% параметров Qwen3.6 и примерно 97–98% параметров DeepSeek V4-Flash и Kimi K3. Поэтому общий объём этих трёх моделей в основном определяется совокупностью экспертов, тогда как объём экспертных вычислений на один токен зависит от экспертов, выбранных на каждом слое.

Таблица 2-С. Вычисления prefill и веса выбранных экспертов при одинаковых входных условиях

Модель	Prefill 8K (TFLOPs)	BF16-веса маршрутизируемых экспертов, выбранных для одного токена (GiB)
DeepSeek V4.1 Flash	143.95	15.820
Qwen3-8B	133.59	—
Qwen3.6	52.09	1.875
DeepSeek V4-Flash	231.13	12.094
Kimi K3 (компактный MLA)	1863.46	90.562

Для обоих типов вызова принято $B = 1$: для prefill используются $S = 0$, $P = 8192$, а для decode — $S = 8192$, $P = 1$; во всех случаях словарная голова обрабатывает только последний токен. Объём матричных вычислений за один шаг на рисунке 2-29 суммируется при тех же условиях. Состояние на рисунке 2-30 представляет собой пространство, необходимое для хранения 8192 токенов контекста: представления контекста используют BF16, а рекуррентные состояния и буферы сжатия — FP32 в соответствии с реализацией. Для V4.1 Flash также учтён буфер истории токенов int64 (64-битных целых чисел) из эталонной реализации, который Engram использует для построения межвызовных n-gram; объём производственного глобального кэша FP4 и окна FP8 приведён в разделе 2.3.6. В последнем столбце таблицы веса маршрутизируемых экспертов, выбранных для одного токена на всех слоях, суммируются и пересчитываются в байты для BF16. Сопоставление этой таблицы с рисунками 2-28–2-30 позволяет отдельно сравнить пространство, занимаемое полной моделью, объём вычислений одного вызова и пространство, занимаемое контекстом запроса.

Для каждой модели суммирование выполняется по её собственному пути выполнения: V4.1 Flash использует CED из статьи, воспроизведение конечного окна и индексирование внутри набора кандидатов; Qwen3-8B — эффективное причинное внимание; Qwen3.6 — последовательную реализацию прямоугольного внимания (eager) и блочную линейную ветвь; DeepSeek V4-Flash — эффективное основное внимание и эталонный индекс; Kimi K3 — компактный MLA и блочный KDA. Сопоставление рисунков 2-28–2-30 показывает: Qwen3.6 хранит примерно в четыре раза больше весов, чем Qwen3-8B, но выполняет меньше матричных вычислений за шаг; веса DeepSeek V4-Flash ещё больше, однако её состояние 8K благодаря сжатию и выборке, наоборот, меньше; Kimi K3 содер-

жит больше экспертов, имеет большую скрытую размерность и больше слоёв, поэтому ей требуется хранить больше весов и выполнять больше вычислений. Эти различия показывают, что общее число параметров, объём вычислений одного вызова и размер состояния контекста необходимо рассчитывать отдельно, исходя из соответствующих им структур.

Оценка объёма чтения за шаг по размеру состояния контекста. На каждом шаге Qwen3-8B использует 1152 MiB старого KV. Для DeepSeek V4-Flash в эталонном формате BF16, принятом на рисунке 2-30, суммарный объём чтения основного внимания и сканирования индекса составляет 27.625 MiB; кроме того, обновляется компрессор. При переходе на производственный смешанный формат из раздела 2.3.6 объём составляет 12.556 MiB. Компактный MLA Kimi K3 использует 216 MiB состояния контекста, а рекуррентную матрицу размером 414 MiB необходимо прочитать и записать, поэтому суммарный объём чтения и записи составляет 828 MiB. Для Qwen3.6 соответствующие значения равны 160 MiB для KV-кэша глобального внимания и 120 MiB для чтения и записи рекуррентных матриц. Рекуррентное состояние необходимо считывать и обновлять на каждом шаге, а представление контекста должно быть доступно текущему запросу. Поэтому даже при фиксированном размере состояния каждый шаг всё равно требует соответствующего объёма чтения и записи.

Кодирование при квантовании и метаданные совместно определяют объём весов. При едином расчёте в BF16 каждый параметр занимает 2 байта, поэтому размер весов непосредственно отражает различия в числе параметров. Фактически опубликованные checkpoint моделей (контрольные точки, то есть наборы файлов с сохранёнными параметрами) используют собственные форматы хранения: основная часть DeepSeek V4-Flash занимает около 156.02 GB, а текстовая часть Kimi K3 — около 1559.97 GB. Размер файлов совместно определяется матрицами с низкой разрядностью, параметрами в более высокой точности и метаданными формата; единая точность отражает масштаб параметров, а размер файлов — число байтов, фактически занимаемое этими параметрами. После загрузки модели также требуется пространство для преобразования формата и вычислений. В главе 5 будет рассмотрено, как формат хранения и реализация среды выполнения влияют на использование видеопамати.

При сравнении двух таблиц также необходимо учитывать распределение задач между этапами V4.1 Flash. Полные текстовые веса V4.1 включают Engram, а все матричные вычисления prefill совместно выполняются кодировщиком, глобальными KV-проекциями декодера, воспроизведением конечного окна и выходной головой. При генерации нового токена должны выполняться обе части сети, поэтому долю экономии на prefill нельзя напрямую применять к decode.

Состав параметров в таблице 2-B объясняет объём весов, а таблица 2-C показывает, как эти параметры участвуют в одном вызове. В совокупности они позволяют разделить вопросы «сколько пространства требуется для хранения всей модели» и «сколько ресурсов требуется для обработки одного токена».²³

Для prefill 8K V4.1 Flash в таблице 2-C требуется лишь 143.95 TFLOPs. В таблице 2-D эти вычисления распределены по этапам выполнения и сопоставлены с эталонным прямым проходом по всем слоям.

Таблица 2-D. Распределение входа 8K V4.1 Flash по этапам выполнения (TFLOPs)

²³ Данные сравнения пяти моделей; программа генерации таблиц; унифицированные вычисления для пяти моделей; вычисление выходной головы последнего токена Qwen3.6.

Этап	Эталонный прямой проход по всем слоям	CED + воспроизведение конечного окна
Кодировщик из 20 слоёв	142.139	141.727
Общий глобальный KV декодера и проекция ключей индекса	0.044	0.044
Вычисления запросов, прямого распространения и других компонентов 20 слоёв декодера	140.202	2.175
Словарная голова последнего токена	0.001	0.001
Итого	282.386	143.947

Эталонный путь выполняет 40 слоёв для всех 8192 токенов и, следуя открытой реализации, сначала вычисляет прямоугольные оценки индекса, а затем применяет маскирование. В пути CED кодировщик обрабатывает весь вход, глобальные KV и ключи индекса декодера по-прежнему создаются для всего входа, основная часть декодера обрабатывает только последние 128 токенов, а последующее индексирование ограничивается набором кандидатов. Таблица учитывает всю работу текстовых матриц каждого пути; различия обусловлены одновременно числом позиций выполнения и алгоритмом индексирования.

Оба пути включают внимание, оценку маршрутизации, маршрутизируемых и общих экспертов, проекции Engram, проекции Single-Pass mHC и словарную голову. Разделение глобальных проекций декодера и вычислений запросов позволяет увидеть, как CED сокращает вычислительную цепочку, через которую проходит длинный промпт.²⁴

Рост объёма вычислений и состояния при увеличении контекста с 8К до 1М. Вопросы и ответы по длинным документам и многошаговые задачи увеличивают стоимость доступа к контексту. Далее сохраняются условия одиночного запроса, добавления одного нового токена за вызов и обработки словарной головой только последнего токена. В сценарии 8К имеется 8192 исторических токена; в сценарии 1М — 1 048 575 исторических токенов, что вместе с текущим запросом даёт ровно 1 048 576 видимых позиций.

В таблице 2-Е отдельно суммируются вычисления оценок внимания и агрегации значений для текущего запроса; для V4-Flash и V4.1 Flash дополнительно учитываются скалярные произведения индекса. В столбцах состояния указаны контекст и фиксированное состояние перед вызовом с той же точностью, что и на рисунке 2-30. Опубликованные конфигурации Qwen3-8B и Qwen3.6 поддерживают соответственно 40 960 и 262 144 позиции; строки 1М для этих двух моделей экстраполированы при неизменной структуре для сравнения закономерностей роста.²⁵

²⁴Реализация полного расчёта текстовых матриц V4.1 Flash; ввод CED 8К, эталонный полносвязный ввод, decode для 8К, decode для 1М. Журнал вычислений и проверки.

²⁵Данные сравнения 8К/200К/1М; реализация унифицированных вычислений. При проверке вычислений подтверждено соответствие значений для 8К рисункам 2-29 и 2-30, а также проверены границы блоков сжатия, предел числа кандидатов и послойные вычисления.

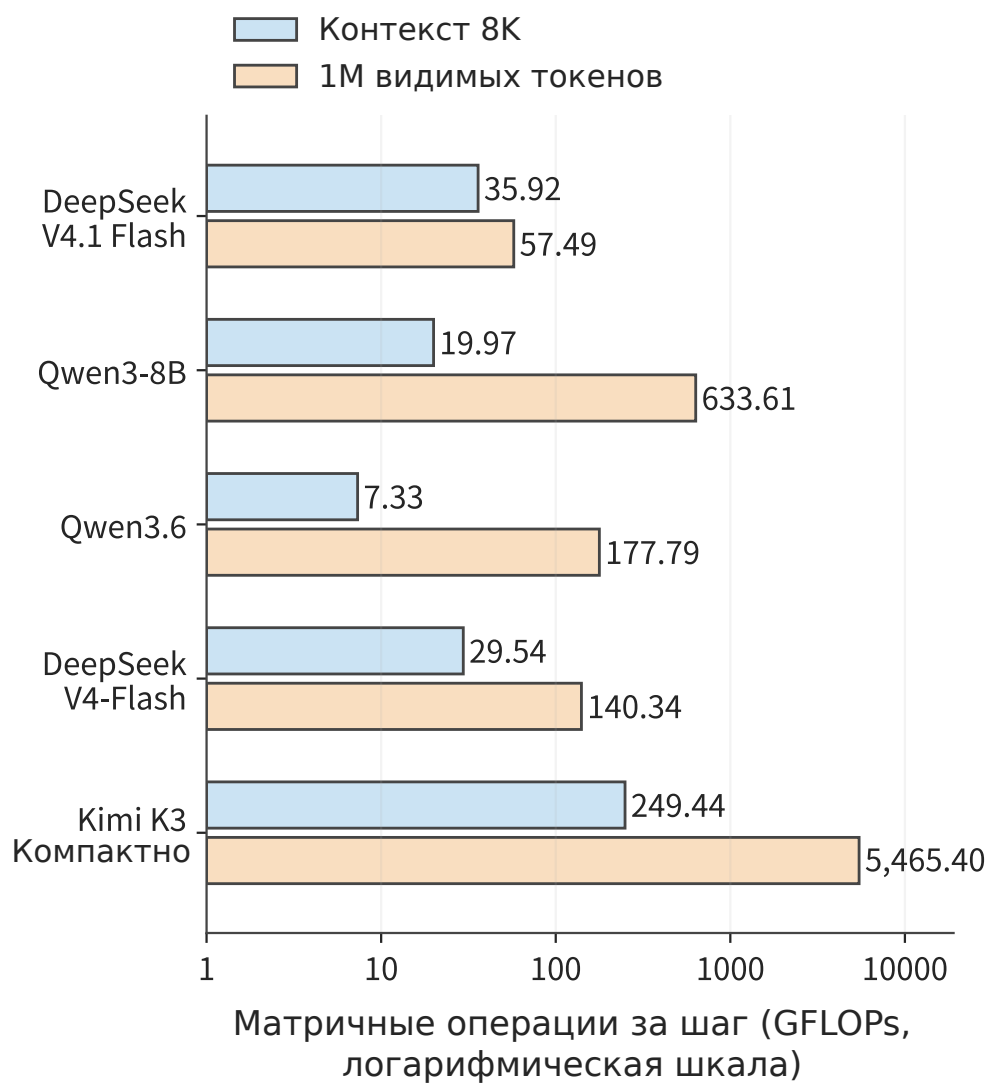


Рисунок 2-31. Объём матричных вычислений при обработке ещё одного токена в контексте 8K / 1М. Используются пути выполнения из таблицы 2-С, ось абсцисс имеет логарифмический масштаб; 1М включает текущий запрос.

Таблица 2-Е. Вычисления взаимодействия и объём состояния при длинном контексте

Модель	Взаимодействие с контекстом 8K (GFLOPs)	Взаимодействие с контекстом 1M (GFLOPs)	Состояние 8K (GiB)	Состояние 1M (GiB)
DeepSeek V4.1 Flash	3.66	25.23	0.029	3.138
Qwen3-8B	4.83	618.48	1.125	144.000
Qwen3.6	1.34	171.80	0.217	20.060
DeepSeek V4-Flash	3.00	113.80	0.069	6.735
Kimi K3 (компактный MLA)	41.08	5257.04	0.634	27.423

Глобальный KV V4.1 Flash имеет только четыре независимых источника, поэтому при увеличении контекста не требуется добавлять отдельный экземпляр для каждого из 40 слоёв. Основное внимание каждого слоя использует не более 128 локальных токенов и 512 глобальных элементов; первый слой Full декодера создаёт набор кандидатов, после чего каждый из четырёх слоёв Reindex сканирует не более 16 384 элементов-кандидатов. Три индексатора кодировщика и первый индексатор декодера всё равно должны сканировать растущую глобальную историю, поэтому объём вычислений продолжает увеличиваться; межслойное совместное использование и иерархическое индексирование изменяют темп роста. При 1M объём взаимодействия с контекстом составляет 25.23 GFLOPs, а полный объём матричных вычислений за один шаг — 57.49 GFLOPs.

V4-Flash продолжает использовать две гранулярности сжатия CSA и HCA: верхняя граница выборки для основного внимания CSA остаётся неизменной, но сканирование индекса растёт вместе с историей; HCA считывает все завершённые крупнозернистые элементы. При 1M объём взаимодействия с контекстом составляет 113.80 GFLOPs, а полный объём матричных вычислений за один шаг — 140.34 GFLOPs. При более коротком контексте более крупная основная часть V4.1 Flash увеличивает объём вычислений проекций и экспертов; лишь при достаточно длинной истории меньшее число индексаторов и иерархический выбор кандидатов компенсируют эту дополнительную работу.

В 69 слоях KDA модели Kimi K3 сохраняется фиксированное рекуррентное состояние, тогда как 24 слоя MLA продолжают обращаться к растущей истории контекста. Компактный MLA уменьшает объём кэша, но не устраняет вычисления между запросом и историческими токенами; при 1M это взаимодействие достигает 5257.04 GFLOPs, а вся модель — 5465.40 GFLOPs. При оценке архитектуры для длинного контекста следует отдельно проверять, какой объём состояния хранится, к каким позициям выполняется доступ и сколько дополнительной работы требуется для индексирования и обновления состояния.

В унифицированном расчёте также сохранён сценарий 200K: полный объём матричных вычислений за один шаг составляет 40.21 GFLOPs для V4.1 Flash и 50.48 GFLOPs для V4-Flash. При 1M различие в объёме вычислений между ними становится ещё больше, поэтому для демонстрации ценности архитектуры при длинном контексте в основном тексте используется 1M.

2.6.2 Число параметров, числовая точность и ограничения по ёмкости

Если свести рассчитанные по отдельности веса модели и состояние каждого запроса в одну таблицу использования видеопамати, можно определить, сколько запросов одновременно помещается на одной карте и сколько места высвобождает квантование весов.

Для одной карты сначала из общей ёмкости вычитаются веса, рабочая область и прочие фиксированные резервы; лишь оставшееся пространство можно использовать для состояний запросов. В упрощённом случае, когда запросы имеют одинаковую длину и KV каждого из них хранится независимо, получаем:

$$B_{\max} = \left\lfloor \frac{M_{\text{device}} - M_{\text{weight}} - M_{\text{workspace}}}{M_{\text{state/request}}} \right\rfloor$$

В числителе находится пространство, доступное запросам после вычета постоянно размещённых данных, а в знаменателе — размер состояния одного запроса. Для каждого дополнительного запроса необходимо выделить соответствующее пространство под состояние; если остатка недостаточно для полного состояния, добавить запрос нельзя, поэтому результат округляется вниз. Рекуррентные состояния и состояния свёртки также относятся к данным каждого запроса и учитываются в знаменателе вместе с KV.

Например, при запуске Qwen3-8B BF16 на RTX 4090 с 24 GB видеопамати веса занимают 16,381,470,720 байт, под рабочую область резервируется 2 GiB, а контекст 8K каждого запроса занимает 1.125 GiB. Число помещающихся запросов равно

$$B_{\max} = \left\lfloor \frac{24 \times 10^9 - 16,381,470,720 - 2 \times 2^{30}}{1.125 \times 2^{30}} \right\rfloor = 4.$$

При увеличении контекста до 16K каждый запрос занимает 2.25 GiB, поэтому в том же пространстве помещаются лишь два запроса; при уменьшении до 4K помещаются девять. Число запросов изменяется целыми скачками, поскольку оставшееся пространство должно вмещать полное состояние запроса, а размещение лишь его части не имеет смысла.

Таким образом, объём сохраняемой истории становится решением, влияющим как на приложение, так и на систему. Если приложение сохранит дополнительный фрагмент материала, модель сможет и дальше обращаться к нему, однако сервису придётся выделить больше пространства под состояние каждого запроса. Рассмотренные в разделе 2.3 GQA, MLA и рекуррентное представление дополнительно меняют способ хранения этой информации. Архитектуру модели, организацию контекста и допустимый параллелизм можно сопоставить в рамках единого бюджета, а затем с помощью тестов на реальных задачах определить, какие варианты обеспечивают требуемое качество ответов.

Чтобы оценить, сколько пространства высвобождает квантование, рассмотрим более крупную модель DeepSeek-R1-Distill-Llama-70B. В разделе 1.2.3 уже были приведены следующие значения: 141.107 GB для весов BF16, 73.726 GB для весов с групповым 8-bit-квантованием и 2.5 GiB для BF16 KV одного запроса с 8192 токенами. Если при той же схеме группировки сохранить более высокую точность для эмбеддингов, выходной головы, нормализации и других компонентов, а также учесть пространство для scale и заполнения в конце при упаковке, размер 4-bit-варианта составит около 39.500 GB.

В 39.500 GB для 4-bit-варианта входят низкоразрядные матрицы, параметры, по-прежнему использующие высокую точность, а также затраты на scale и упаковку,

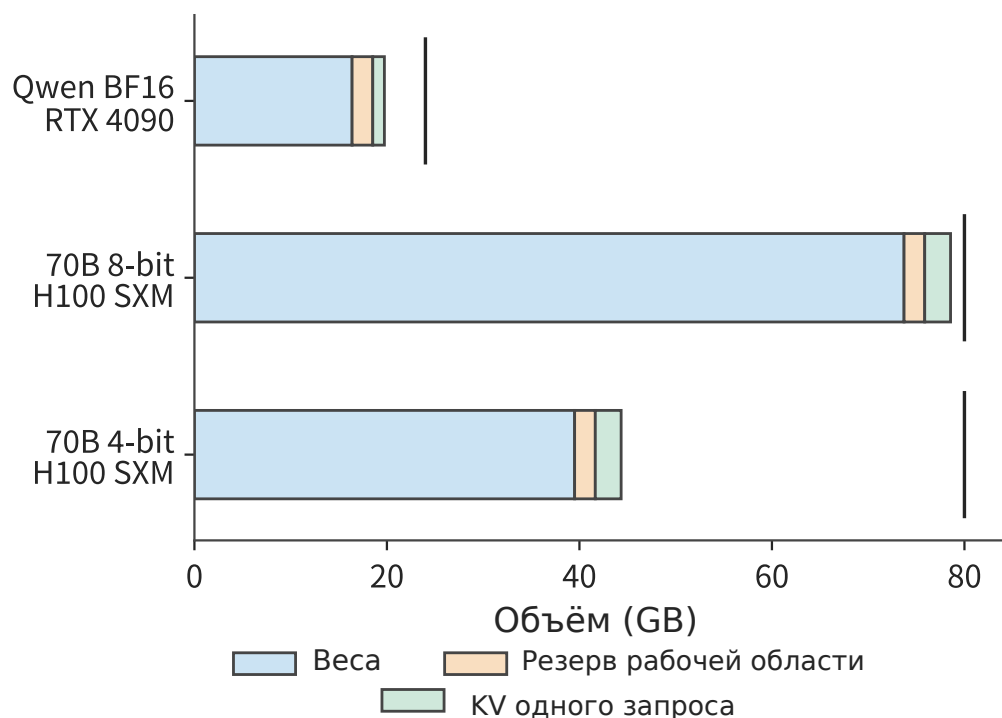


Рис. 2-32. Ёмкость весов, фиксированного резерва и KV одного запроса по отдельности. Короткими вертикальными линиями отмечены ёмкости RTX 4090 в 24 GB и H100 SXM в 80 GB; KV использует BF16 и длину контекста 8192, фиксированный резерв составляет 2 GiB. «Резерв рабочей области» означает временное пространство, необходимое для выполнения, а «KV одного запроса» — кэш контекста одного запроса с 8192 токенами.

поэтому этот объём больше результата, полученного простым пересчётом всех параметров по половине байта. На H100 SXM с 80 GB видеопамати в десятичном представлении и фиксированным резервом 2 GiB 8-bit-вариант позволяет разместить один запрос с контекстом 8K, а 4-bit-вариант — 14 запросов. При увеличении контекста с 8K до 32K объём KV каждого запроса возрастает в четыре раза, и 4-bit-вариант может одновременно вместить лишь три запроса. Пространство, высвобождённое благодаря сжатию весов, позволяет хранить контексты большего числа запросов, но чем длиннее каждый контекст, тем меньше запросов помещается одновременно.²⁶

Тем же способом можно анализировать и более крупные конфигурации. Веса модели 235B в формате BF16 занимают около 470 GB, что превышает суммарные 192 GB восьми RTX 4090, тогда как восемь H100 SXM в одном сервере HGX суммарно предоставляют 640 GB. После вычета весов у последнего остаётся около 170 GB для состояний запросов, рабочей области и данных, которые после распределения по нескольким картам приходится дублировать на нескольких из них. Например, на каждой карте отдельно хранятся параметры нормализации и маршрутизатор, а при распределении четырёх KV-голов между восемью картами KV каждой головы хранится на двух картах. В главе 6 эти статьи бюджета будут рассчитаны для каждой карты с учётом распределения функций модели между картами.

Ёмкость определяет минимальное число карт для одного экземпляра, а длина и па-

²⁶

Таблица ёмкости для 8K; сопоставление с 32K.

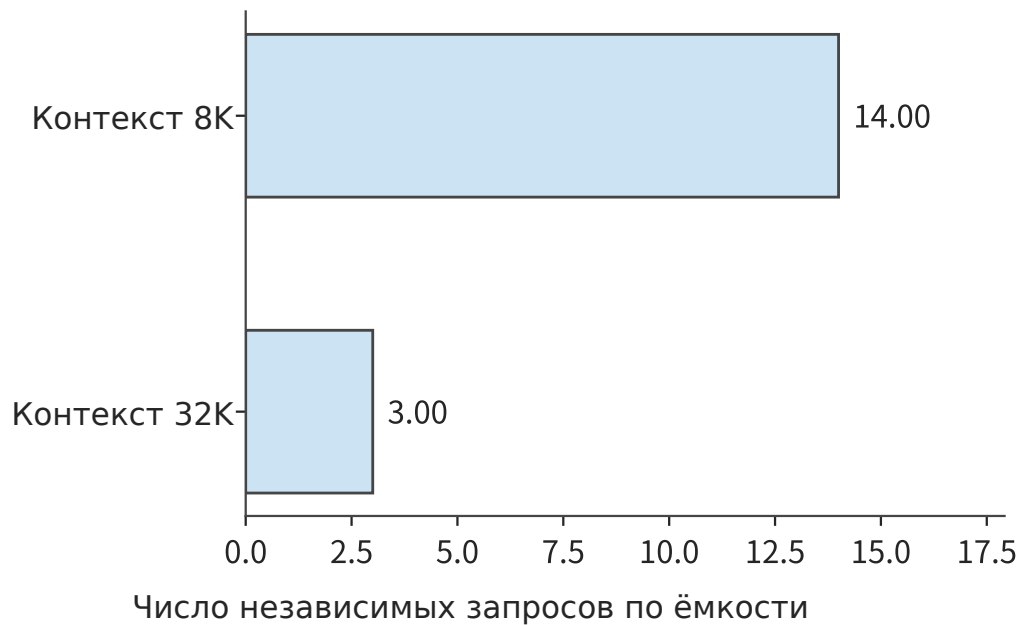


Рис. 2-33. Число независимых запросов, которые 4-bit-вариант 70B может одновременно разместить на одной H100 SXM с 80 GB при увеличении длины контекста с 8K до 32K. Состояние каждого запроса увеличивается, поэтому в оставшемся пространстве помещается меньше запросов.

параллелизм запросов — объём работы, который должен выполнять экземпляр. Более низкая разрядность высвобождает пространство, более короткий контекст позволяет одновременно держать в памяти больше запросов, а повторное использование данных внутри батча меняет стоимость их выполнения. Поэтому при выборе конфигурации развёртывания необходимо последовательно учитывать ёмкость, объём обращений и объём вычислений; в главе 3 дополнительно будут рассмотрены поступление запросов и ограничения по времени.

Упражнение 2-7 (основное): определение числа помещающихся запросов по ёмкости видеопамяти

Возьмите RTX 4090 с 24 GB видеопамяти, зарезервируйте 2 GiB под рабочую область и используйте для весов Qwen3-8B BF16 точное значение из таблицы. Определите максимальное число одновременно помещающихся запросов для контекстов 4K, 8K и 16K; затем для случая 8K увеличьте резерв рабочей области до 3 GiB и повторите расчёт. После этого возьмите из таблицы значение 39.500 GB для 4-bit-весов модели 70B, замените карту на RTX 6000 Ada с 48 GB видеопамяти, оставив резерв равным 2 GiB, и определите число запросов для контекстов 8K и 32K. Сначала сопоставьте требования к ёмкости для весов, рабочей области и KV, а затем по оставшейся видеопамяти вычислите число помещающихся запросов.

Можно решить и обратную задачу: по заданному ускорителю и целевым характеристикам сервиса определить пространство, остающееся для модели. Пусть доступная ёмкость каждой карты равна C , фиксированный резерв рабочей области — M_0 , KV каждого запроса — K , параллелизм — B , а каждый параметр занимает b_W байт. Если пренебречь прочими постоянно размещёнными объектами, верхняя граница числа параметров для одной карты равна

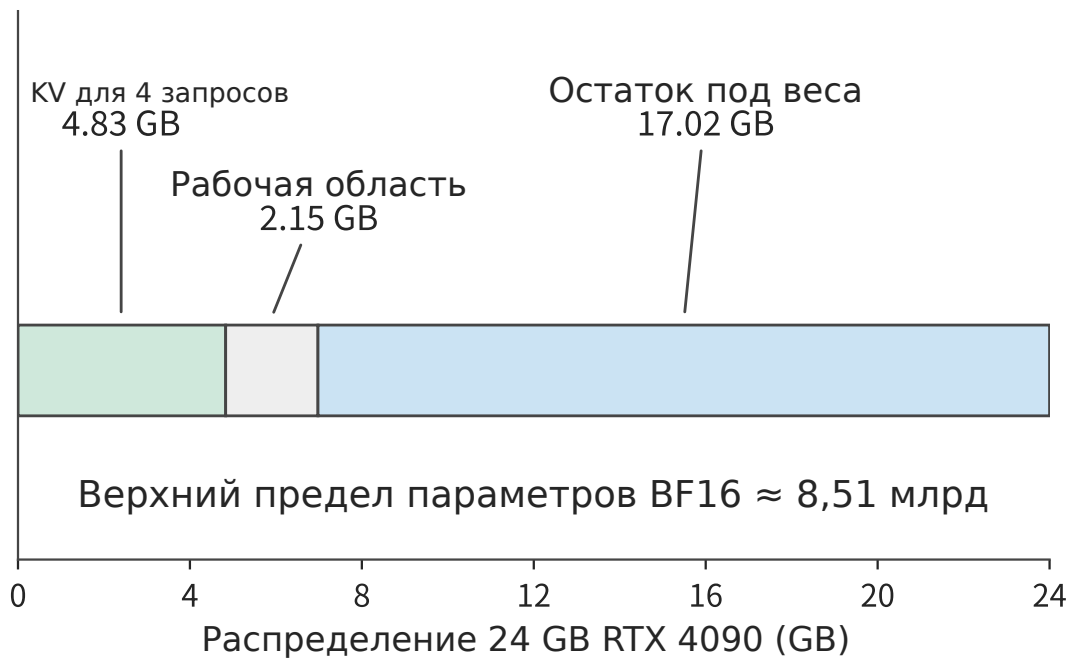


Рис. 2-34. Обратный расчёт бюджета весов по ускорителю. Из 24 GB RTX 4090 сначала резервируется пространство под KV четырёх запросов с контекстом 8K и рабочую область размером 2 GiB; остаток задаёт верхнюю границу числа параметров BF16. Все значения ёмкости на схеме приведены в десятичных GB.

$$N_{\max} = \frac{C - M_0 - BK}{b_W}.$$

Сначала определяется размер состояния, необходимый сервису, а затем вычисляется пространство, остающееся для весов. Возьмём RTX 4090 с ёмкостью 24 GB, рабочую область размером 2 GiB и четыре запроса по 8192 токена, для каждого из которых KV, как и у Qwen3-8B, занимает 1.125 GiB. При 2 байтах на параметр в формате BF16 оставшееся пространство вмещает не более чем около 8,51 млрд параметров.²⁷ На рис. 2-34 сначала вычитается состояние сервиса, после чего выделяется бюджет для весов.

При изменении архитектуры модели одновременно меняются бюджеты весов и состояний. Увеличение числа слоёв или KV-голов повышает K каждого запроса, сокращая пространство, остающееся для весов; развёртывание на нескольких картах увеличивает общую ёмкость, но одновременно повышает объём коммуникаций. Описанный в отчёте Llama 3 инференс модели 405B в формате BF16 выполнялся на двух серверах с 16 картами H100, поскольку требуемая ёмкость превышала объём видеопамяти одного сервера.²⁸ Поэтому число параметров модели, структуру состояния и число карт необходимо выбирать совместно.

²⁷Поэлементный пересчёт сравнения при условиях этой главы и программа вычислений.

²⁸Аннотация оригинальной статьи о MQA; аннотация оригинальной статьи о GQA; раздел 6.1 отчёта Llama 3. Пример ёмкости выведен из заданного в условии состояния сервиса.

2.6.3 Сравнение моделей при разных параметрах запроса

Наконец, объединим ёмкость, цепочку вызовов и качество выполнения задачи. Сначала исключим конфигурации с недостаточной ёмкостью с учётом ёмкости каждой карты, затем просуммируем работу по обработке запроса в соответствии с фактическим способом вычислений каждой модели и, наконец, проверим, способна ли модель выполнить целевую задачу.

При сравнении полных моделей можно исходить из двух вариантов входных данных. Если зафиксировать S , P , G , B , получим требования к ресурсам при одинаковой длине вызовов, что позволяет увидеть структурные различия. Если зафиксировать текст и задачу, то сначала получим длину входа с помощью токенизатора каждой модели, а затем проверим выход. Эти два вида сравнения соответственно отвечают на вопросы: «Как выполняются вычисления при одинаковом количестве позиций?» и «Как выполняется одна и та же задача?».

Сначала сравним требования к ресурсам при одинаковой длине входа и выхода. Пусть $S = 0$, $P = 128$, $G = 4$, $B = 1$: модель выполняет один prefill и три decode, а в конце сохраняет 131 обработанную позицию. Суммирование объёма матричных вычислений этих вызовов даёт следующую таблицу.²⁹

Модель	Матричные вычисления полного запроса (TFLOPs)	Использованный в расчёте способ реализации
DeepSeek V4.1 Flash	4.14	CED + повторное воспроизведение конечного окна, индексирование внутри множества кандидатов
Qwen3-8B	1.83	эффективное причинное внимание
Qwen3.6	0.66	прямоугольное полное внимание и блочная линейная ветвь
DeepSeek V4-Flash	3.40	эффективное основное внимание и эталонный индекс
Kimi K3 (компактный MLA)	27.16	компактный MLA и блочный KDA; для decode используется рекуррентный KDA

Эта таблица использует тот же путь выполнения, что и таблица 2-С: для Kimi K3 везде применяется компактный MLA, а восходящая проекция переносится на сторону запросов и агрегации. Вход V4.1 Flash содержит ровно 128 токенов, и все они попадают в окно повторного воспроизведения декодера, поэтому в этом коротком запросе отсутствует характерная для длинного промпта выгода от пропуска большого числа позиций декодера. Для полного запроса по одному и тому же выбранному пути суммируются prefill и три decode, а контекст увеличивается со 128 до 131 позиции; объём каждого вида вычислений определяется как формами матриц, так и представлением состояния на этом пути.

После перехода от одного вызова к полному запросу роль входа и выхода становится понятнее. Увеличение P в основном повышает объём однократной обработки входа, а

²⁹Расчёты полного запроса и отдельных вызовов для пяти моделей; запуск: `python3 calculations/reproduce_ch02.py`. Все пять моделей используют путь из таблицы 2-С этой главы; Kimi K3 использует компактный MLA, а прежний развёрнутый путь и сопоставление с V4-Pro сохранены в журнале исторических запросов.

также длину контекста в начале последующих decode. Увеличение G приводит к многократному выполнению проекций, экспертов и выходной головы, а также к многократному чтению контекста. Поэтому длинный вход с коротким выходом и короткий вход с длинным выходом предъявляют разные требования к различным ресурсам. В главе 3 будет рассмотрена системная нагрузка при непрерывном поступлении запросов этих двух типов.

Упражнение 2-8 (расширенное): как повторное использование префикса и изменение входа влияют на объём вычислений модели

Общая длина входа в одном раунде составляет 1443 токена, из которых первые 1392 найдены в кэше; модель возвращает 128 токенов. Найдите значения S , P , G , n_d , длину контекста после завершения запроса и размер добавленного в этом раунде KV. Насколько повторное использование кэша сокращает число строк проекций и число эффективных пар запрос—ключ по сравнению с полным повторным вычислением? Если изменить содержимое 501-го входного токена, заново определите максимальную длину повторно используемого префикса и длину нового входа.

Теперь рассмотрим второй вид сравнения — при фиксированном тексте. В этом случае разные токенизаторы создают входы разной длины. Например, в восьми задачах на поиск словосочетаний Qwen3-8B и DeepSeek V4-Flash правильно нашли целевые словосочетания, но короткий текст в Qwen занимал 523 или 524 токена, а в DeepSeek V4-Flash — 501 токен; более длинный текст занимал соответственно 2121 или 2122 токена и 2036 токенов. Использование одного и того же текста сохраняет неизменным содержание задачи, а количество токенов каждой модели определяет фактическую форму входа.

Таким образом, при сравнении моделей нужно одновременно ответить на два вопроса: способны ли они выполнить одну и ту же задачу и сколько ресурсов для этого требуется. Только если сначала оценить ответы по одинаковым критериям, а затем подставить соответствующую длину входа каждой модели в таблицу моделей, можно связать структурные различия с практическим применением.³⁰

Упражнение 2-9 (расширенное): выбор модели с учётом ёмкости, вычислительных требований и качества выполнения задачи

Используя рисунки 2-28–2-30 и таблицу 2-С, отдельно укажите для каждой модели требования к полным весам, состоянию 8K, одному шагу decode и prefill для 8K. Модель развёртывается на одной H100 SXM с 80 GB видеопамяти, из которых 2 GiB зарезервированы под рабочую область; веса представлены в BF16. Сначала определите, какие модели полностью помещаются на одной карте. Затем рассмотрите многокарточное развёртывание и укажите, какие параметры времени и обмена данными потребуется получить из следующих глав. Измените число выходных токенов G на 1 и заново вычислите количество вызовов prefill и decode. Наконец, сформулируйте критерий качества для задачи поиска словосочетания и объясните, на какие вопросы отвечают сравнение при одинаковом числе токенов и сравнение при одинаковом тексте.

В этой главе, начиная с вычислительного графа, мы последовательно рассчитали потребность полного запроса в ресурсах. Внимание определяет способ хранения контекста и доступа к нему, МоЕ — выбор весов при каждом вычислении, а глубина сети и способ

³⁰Парный эксперимент по поиску. Qwen использует BF16/vLLM; DeepSeek V4-Flash использует экспертов MXFP4, FP8 KV/SGLang и размещает часть весов в памяти CPU.

Вход: 128 токенов, выход: 4 токена

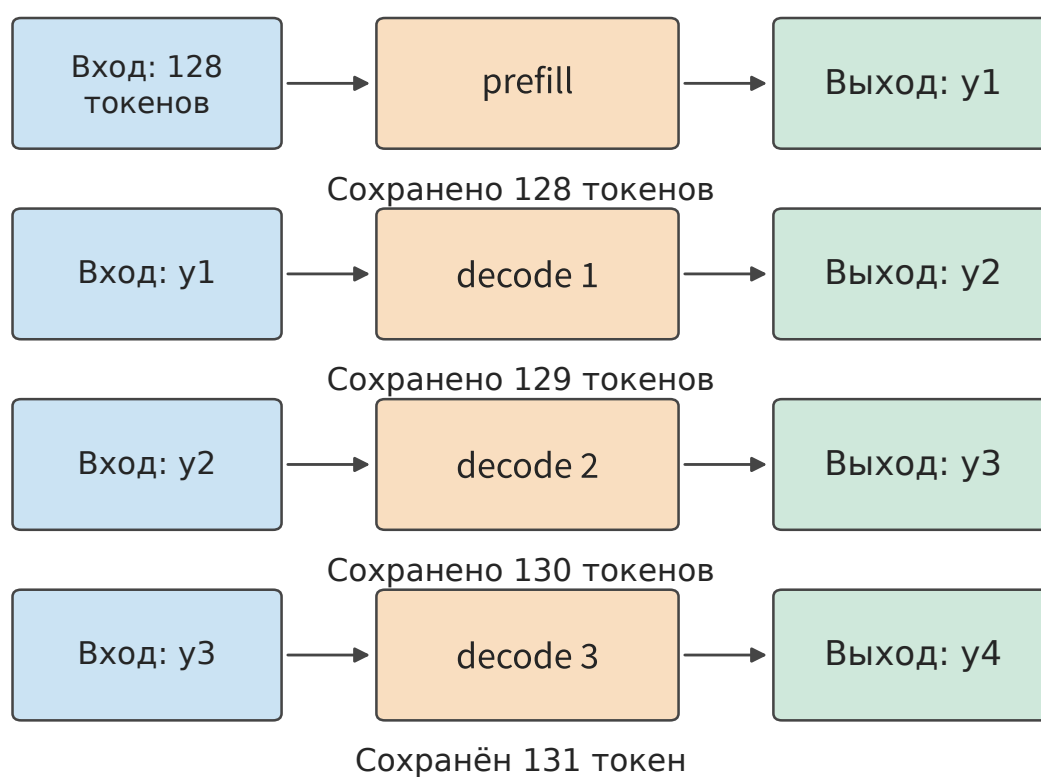


Рисунок 2-35. Последовательность вызовов при генерации четырёх выходных токенов. prefill обрабатывает 128 входных токенов и создаёт первый выходной токен, после чего каждый из трёх decode передаёт предыдущий выход обратно в модель; в итоге сохраняется 131 токен.

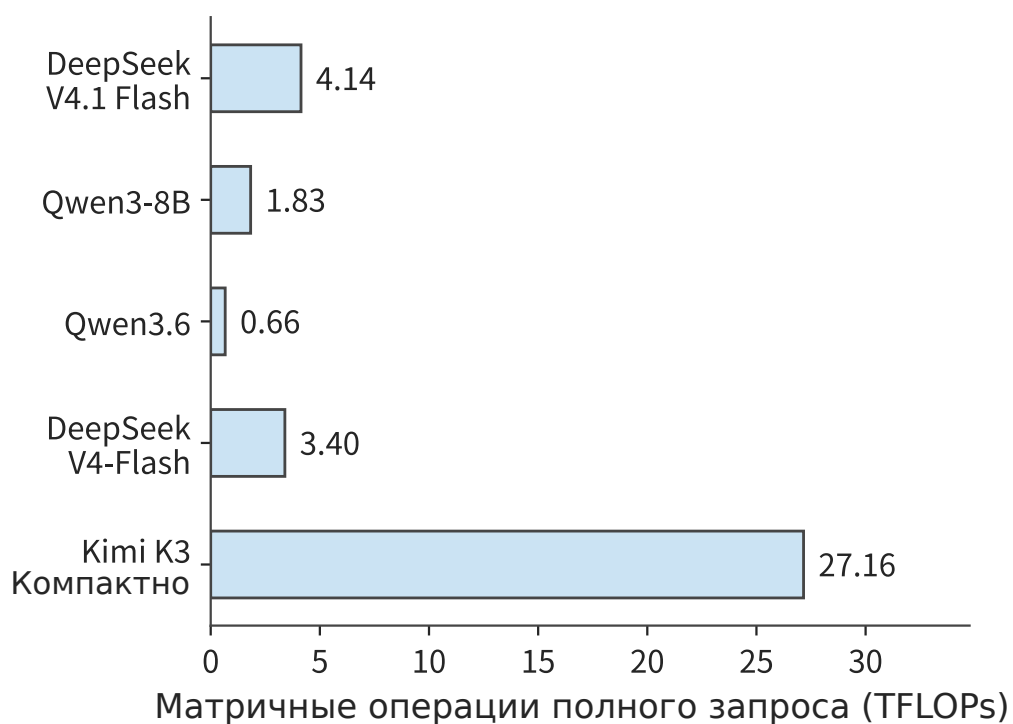


Рисунок 2-36. Объём матричных вычислений, необходимый каждой модели для обработки одного и того же запроса с входом из 128 токенов и выходом из четырёх токенов. Область выполнения моделей и пути кэша заданы в условиях этого раздела; показан объём вычислений, накопленный по всем вызовам.

соединения — порядок вычислений и потребность во временном хранилище. В следующей главе будут добавлены распределение длин запросов, время их поступления, вызовы инструментов, обучение и обучение с подкреплением (RL), чтобы проанализировать, как эти вычисления со временем формируют системную нагрузку.

2.7 Заблуждения и ловушки

Заблуждение: больше параметров — значит, объём вычислений на каждый токен обязательно выше. Общее число параметров определяет ёмкость полной модели; выбранные эксперты MoE, тип механизма внимания и форма вызова определяют текущий объём работы. Сопоставьте таблицу 2-А, таблицу 2-В и рисунок 2-29 и отдельно объясните, от каких факторов зависят число параметров и объём вычислений.

Заблуждение: если объём KV-кэша сократился до четверти исходного, объём вычислений механизма внимания также уменьшится пропорционально. GQA использует совместное представление контекста, но каждая голова запросов по-прежнему вычисляет оценки отдельно; MLA изменяет ширину контекста, а также место выполнения восходящей проекции. Сначала определите изменённые матрицы и состояния, а затем рассчитайте остальной объём работы.

Заблуждение: если размер состояния фиксирован, затратами на доступ можно пренебречь. Фиксированную рекуррентную матрицу всё равно необходимо обновлять; кроме того, на этапе prefill могут возникать временные блочные состояния. Ёмкость, совокупный объём обращений и пиковый размер рабочей области отвечают на разные вопросы.

Заблуждение: одинаковое число входных токенов позволяет объективно сравнить возможности моделей. Числа входных и выходных токенов подходят для сравнения структуры потребления ресурсов; при выполнении одной и той же текстовой задачи также необходимо зафиксировать оценивание, качество и условия выполнения, допуская при этом, что разные токенизаторы создают последовательности разной длины.

2.8 Упражнения и воспроизведение результатов главы

Упражнения 2-2, 2-5 и 2-7 — основные; остальные предназначены для анализа особых случаев и применения методов к другим моделям. Сначала самостоятельно сделайте прогнозы и выполните выкладки, а затем сверьте их с сопутствующими вычислениями. Команды для вычислений запускаются из корня репозитория; полные варианты приведены в расширенных материалах к главе 2.

Соответствующая тема	Точка входа для локального воспроизведения вычислений
Зависимости в последовательности и эквивалентность кэширования	<code>python3 calculations/calc.py sequence-dependencies --format md</code>
Прямой проход и эксперты Qwen3.6	<code>python3 calculations/calc.py qwen36-forward --format md</code>
Полное матричное вычисление DeepSeek V4.1 Flash	<code>python3 calculations/calc.py v41-forward --tokens 8192 --execution ced --format md</code>
Эталонный путь через все слои DeepSeek V4.1 Flash	<code>python3 calculations/calc.py v41-forward --tokens 8192 --execution reference --format md</code>

Соответствующая тема	Точка входа для локального воспроизведения вычислений
Сравнение пяти моделей и полный запрос Декомпозиция состояния DeepSeek V4-Flash	<code>python3 calculations/reproduce_ch02.py python3 calculations/calc.py state --model deepseek-v4-flash --length 8192</code>
Состояние и блочное выполнение Kimi K3	<code>python3 calculations/calc.py k3-kda-chunk --tokens 8192</code>
Гранулярность экспертов	<code>python3 calculations/calc.py qwen235- expert-granularity --format md</code>
Однократный МТР	<code>python3 calculations/calc.py v4-mtp- forward --format md</code>
Подстановка полей известной трассы	<code>python3 calculations/calc.py trace- resource-bridge --format md</code>
Сравнение запросов для пяти моделей	<code>python3 calculations/calc.py request- model-comparison --format md</code>

Иллюстрации к этой главе, данные вычислений и методы реконструкции приведены в каталоге иллюстраций. В материалах в конце главы описаны архитектура и устройство каждой модели.

2.9 Справочные таблицы матриц моделей

Эти таблицы позволяют перейти от суммарных показателей всей модели к конкретным матрицам. Сначала определите модель и ветвь выполнения, затем умножьте FLOPs одной матричной операции на число слоёв; для маршрутизируемых экспертов также необходимо суммировать фактическое число полученных ими входных строк. Базовые показатели Qwen3-8B в таблице 2-1 приведены в разделе 2.1.4.

Таблица 2-2. Qwen3.6-35B-A3B: расчёт потребления ресурсов модулями гибридного внимания и МоЕ

Основная часть состоит из 40 слоёв: 10 слоёв выбирают ветвь полного внимания, а 30 — ветвь линейного внимания; после этого слои обоих типов выполняют МоЕ. Для каждого токена выбираются 8 из 256 маршрутизируемых экспертов, а также один общий эксперт. Выход QKV линейной ветви разделён на размерности 2048, 2048 и 4096; её Q и K переиспользуются в соответствии с группировкой голов значений.

Вход модели

Модуль и назначение	Вход → выход (число строк × ширина)	Матрица весов (ширина входа × ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выбор строк эмбед- дингов токенов	$m \text{ ID} \rightarrow m \times 2048$	Таблица 248320×2048	0; обращения для выбора строк учи- тываются отдель- но	1

Ветвь полного внимания

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Полное внимание: Q и выходной гейтинг	$m \times 2048 \rightarrow$ $m \times 8192$	2048×8192	$2 \times m \times 2048 \times$ 8192	10
Полное внимание: K, V	$m \times 2048 \rightarrow$ $m \times 512$	2048×512 , всего 2	$2 \times 2 \times m \times 2048 \times$ 512	10
Полное внимание: взаимодействие с контекстом	16 голов Q , 2 го- ловы KV , ширина головы 256	Без дополнитель- ных весов	$4 \times 16 \times 256 \times N_{\text{pair}}$	10
Полное внимание: выходная проекция	$m \times 4096 \rightarrow$ $m \times 2048$	4096×2048	$2 \times m \times 4096 \times$ 2048	10

Ветвь линейного внимания

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Линейное внимание: совместная проекция QKV	$m \times 2048 \rightarrow$ $m \times 8192$	2048×8192	$2 \times m \times 2048 \times$ 8192	30
Линейное внимание: выходной гейтинг z	$m \times 2048 \rightarrow$ $m \times 4096$	2048×4096	$2 \times m \times 2048 \times$ 4096	30
Линейное внимание: управляющие величи- ны a, b	$m \times 2048 \rightarrow$ $m \times 32$	2048×32 , всего 2	$2 \times 2 \times m \times 2048 \times$ 32	30
Линейное внимание: свёртка и рекуррент- ное обновление delta	Q, K : по 16×128 ; V : 32×128	Свёртка: 8192 канала, ширина ядра 4	Рекуррентное обновление: $O(m \times 32 \times 128^2)$; блочный алго- ритм учитывается отдельно	30
Линейное внимание: выходная проекция	$m \times 4096 \rightarrow$ $m \times 2048$	4096×2048	$2 \times m \times 4096 \times$ 2048	30

Общая для всех слоёв ветвь экспертов

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
На каждом слое: оценка маршрути- зации	$m \times 2048 \rightarrow$ $m \times 256$	2048×256	$2 \times m \times 2048 \times 256$	40
Маршрутизируемый эксперт e : преобра- зование в выбранной ветви	$t_e \times 2048 \rightarrow t_e \times$ $512 \rightarrow t_e \times 2048$	Две матрицы 2048×512 ; од- на 512×2048	$6 \times t_e \times 2048 \times 512$	40
Общий эксперт: об- щее преобразование всех токенов	$m \times 2048 \rightarrow m \times$ $512 \rightarrow m \times 2048$	Две матрицы 2048×512 ; од- на 512×2048	$6 \times m \times 2048 \times 512$	40
Гейтинг общего экс- перта	$m \times 2048 \rightarrow m \times 1$	2048×1	$2 \times m \times 2048 \times 1$	40

Выходной интерфейс

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выходная голова слова (один раз в конце)	$m_{\text{out}} \times 2048 \rightarrow$ $m_{\text{out}} \times 248320$	2048×248320	$2 \times m_{\text{out}} \times 2048 \times$ 248320	1

При суммировании умножайте формулу каждой строки на число слоёв в последнем столбце; для маршрутизируемых экспертов также необходимо просуммировать фактическое число полученных каждым экспертом строк t_e . Финальная нормализация выполняется один раз, а внутрислойная нормализация, активация, обработка позиций и обновление состояния выполняются в соответствующих ветвях.

При чтении таблицы сначала определите, использует ли слой линейное или полное внимание, выберите соответствующую группу строк, а затем добавьте общую часть МоЕ. Линейное внимание определяет способ хранения контекста, а МоЕ — какие веса используются для текущего токена; оба механизма могут применяться одновременно. АЗВ обозначает номинальный объём активированных параметров, однако все эксперты в совокупности по-прежнему входят в постоянно размещённую модель.³¹

Таблица 2-3. DeepSeek-V3: MLA и плотные/экспертные сети прямого распространения

DeepSeek-V3 состоит из 61 слоя: первые три используют плотную FFN, а в каждом из последующих 58 слоёв для каждого токена выбираются 8 маршрутизируемых экспертов и выполняется один общий эксперт. Ниже FLOPs MLA рассчитываются для развёрнутого пути: скрытая переменная каждого токена посредством восходящей проекции преобразуется в K и V каждой головы, а 64-мерное позиционное представление конкатенируется с K . Запросы и ключи совместно участвуют в вычислении зависящих от позиции оценок, а V используется для агрегации содержимого.

Вход модели

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выбор строк эмбеддингов токенов	$m \text{ ID} \rightarrow m \times 7168$	Таблица 129280×7168	0; обращения для выбора строк учитываются отдельно	1

Внимание MLA

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Нисходящая проекция запросов: формирование низкоранговых запросов	$m \times 7168 \rightarrow$ $m \times 1536$	7168×1536	$2 \times m \times 7168 \times$ 1536	61
Восходящая проекция запросов: развёртывание 128 голов	$m \times 1536 \rightarrow$ $m \times 24576$	1536×24576	$2 \times m \times 1536 \times$ 24576	61

³¹Журнал прямого прохода Qwen3.6.

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Нисходящая проекция KV: 512-мерная скрытая переменная и 64-мерная позиционная ветвь	$m \times 7168 \rightarrow m \times 576$	7168×576	$2 \times m \times 7168 \times 576$	61
Восходящая проекция KV: развёртывание K , V каждой головы	$m \times 512 \rightarrow m \times 32768$	512×32768	$2 \times m \times 512 \times 32768$	61
Развёрнутое MLA: вычисление оценок и агрегация значений	128 голов; ширина $Q/K - 192$, ширина $V - 128$	Без дополнительных весов	$2 \times 128 \times (192 + 128)N_{\text{pair}}$	61
Выходная проекция внимания	$m \times 16384 \rightarrow m \times 7168$	16384×7168	$2 \times m \times 16384 \times 7168$	61

Ветвь прямого распространения

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Первые три слоя: плотная сеть прямого распространения	$m \times 7168 \rightarrow m \times 18432 \rightarrow m \times 7168$	Две матрицы 7168×18432 ; одна 18432×7168	$6 \times m \times 7168 \times 18432$	3
Последующие 58 слоёв: оценка маршрутизации	$m \times 7168 \rightarrow m \times 256$	7168×256	$2 \times m \times 7168 \times 256$	58
Маршрутизируемый эксперт e	$t_e \times 7168 \rightarrow t_e \times 2048 \rightarrow t_e \times 7168$	Две матрицы 7168×2048 ; одна 2048×7168	$6 \times t_e \times 7168 \times 2048$	58
Общий эксперт	$m \times 7168 \rightarrow m \times 2048 \rightarrow m \times 7168$	Две матрицы 7168×2048 ; одна 2048×7168	$6 \times m \times 7168 \times 2048$	58

Выходной интерфейс

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выходная голова словаря (один раз в конце)	$m_{\text{out}} \times 7168 \rightarrow m_{\text{out}} \times 129280$	7168×129280	$2 \times m_{\text{out}} \times 7168 \times 129280$	1

При суммировании умножайте формулу каждой строки на число слоёв в последнем столбце; для маршрутизируемых экспертов также необходимо просуммировать фактическое число полученных каждым экспертом строк t_e . Финальная нормализация выполняется один раз, а внутрислойная нормализация, активация, обработка позиций и обновление состояния выполняются в соответствующих ветвях.

V3 объединяет два способа экономии ресурсов: MLA представляет контекст с помощью низкоранговых скрытых переменных, а MoE позволяет токenu выполнять лишь часть экспертов. DeepSeek V4-Flash дополнительно изменяет организацию

контекста: недавние позиции сохраняются в окне, а долговременный контекст — с разными коэффициентами сжатия. В следующей таблице модулей эти типы контекста представлены отдельно.³²

Таблица 2-4. DeepSeek V4-Flash: избирательный доступ к контексту и экспертные вычисления

43 слоя включают 2 слоя только с оконным вниманием, 21 слой CSA и 20 слоёв HCA; для каждого токена выбираются 6 маршрутизируемых экспертов, а также один общий эксперт. A — эффективное число пар «запрос — ключ» основного внимания для всех запросов этого слоя, а I — число пар «запрос — ключ», фактически просканированных индексом; число голов не включено ни в одну из этих величин. Восемь групп сгруппированной выходной проекции обрабатывают восемь разных срезов. Проекция сжатия CSA сначала создаёт представления размерностью 1024/256, содержащие перекрывающиеся ветви, а затем отдельно формирует 512-мерное представление контекста основного внимания и 128-мерную индексную запись.

Вход модели

Модуль и назначение	Вход → выход (число строк × ширина)	Матрица весов (ширина входа × ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выбор строк эмбедингов токенов	$m \text{ ID} \rightarrow m \times 4096$	Таблица 129280×4096	0; обращения для выбора строк учитываются отдельно	1

Проекции внимания на всех слоях

Модуль и назначение	Вход → выход (число строк × ширина)	Матрица весов (ширина входа × ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Все слои: нисходящая проекция запросов	$m \times 4096 \rightarrow m \times 1024$	4096×1024	$2 \times m \times 4096 \times 1024$	43
Все слои: восходящая проекция запросов	$m \times 1024 \rightarrow m \times 32768$	1024×32768	$2 \times m \times 1024 \times 32768$	43
Все слои: общие KV	$m \times 4096 \rightarrow m \times 512$	4096×512	$2 \times m \times 4096 \times 512$	43
Основное внимание: окно и выбранный сжатый контекст	64 головы, по 512 измерений в каждой	Без дополнительных весов	$4 \times 64 \times 512 \times A$	43
Все слои: сгруппированная низкоранговая выходная проекция	$m \times 4096 \rightarrow m \times 1024$	4096×1024 , всего 8	$8 \times 2 \times m \times 4096 \times 1024$	43
Все слои: выходная проекция после конкатенации	$m \times 8192 \rightarrow m \times 4096$	8192×4096	$2 \times m \times 8192 \times 4096$	43

Операции, специфичные для CSA

³²Вычисления прямого прохода V3.

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
CSA: сжатие контекста и гейтинг основного внимания	$m \times 4096 \rightarrow m \times 1024$	4096×1024 , всего 2	$2 \times 2 \times m \times 4096 \times 1024$	21
CSA: индексные запросы	$m \times 1024 \rightarrow m \times 8192$	1024×8192	$2 \times m \times 1024 \times 8192$	21
CSA: веса индексных голов	$m \times 4096 \rightarrow m \times 64$	4096×64	$2 \times m \times 4096 \times 64$	21
CSA: сжатое содержимое индекса и гейтинг	$m \times 4096 \rightarrow m \times 256$	4096×256 , всего 2	$2 \times 2 \times m \times 4096 \times 256$	21
CSA: сканирование и выбор по индексу	64 индексные головы, по 128 измерений в каждой	Без дополнительных весов	Скалярные произведения: $2 \times 64 \times 128 \times I$; взвешивание и top-k учитываются отдельно	21

Операции, специфичные для HCA

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
HCA: сжатие контекста и гейтинг основного внимания	$m \times 4096 \rightarrow m \times 512$	4096×512 , всего 2	$2 \times 2 \times m \times 4096 \times 512$	20

Эксперты и соединения на каждом слое

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Оценка маршрутизации	$m \times 4096 \rightarrow m \times 256$	4096×256	$2 \times m \times 4096 \times 256$	43
Маршрутизируемый эксперт e	$t_e \times 4096 \rightarrow t_e \times 2048 \rightarrow t_e \times 4096$	Две матрицы 4096×2048 ; одна 2048×4096	$6 \times t_e \times 4096 \times 2048$	43
Общий эксперт	$m \times 4096 \rightarrow m \times 2048 \rightarrow m \times 4096$	Две матрицы 4096×2048 ; одна 2048×4096	$6 \times m \times 4096 \times 2048$	43
mHC: объединение и смешивание остаточных связей	$m \times 4 \times 4096 \leftrightarrow m \times 4096$	Параметры проекции смешивания и нормализации	Матрицы смешивания, скаляры и специальные функции суммируются отдельно	43

Выходной интерфейс

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выходная голова слова (один раз в конце)	$m_{\text{out}} \times 4096 \rightarrow$ $m_{\text{out}} \times 129280$	4096×129280	$2 \times m_{\text{out}} \times 4096 \times$ 129280	1

При суммировании умножайте формулу каждой строки на число слоёв в последнем столбце; для маршрутизируемых экспертов также необходимо просуммировать фактическое число полученных каждым экспертом строк t_e . Финальная нормализация выполняется один раз, а внутрислойная нормализация, активация, обработка позиций и обновление состояния выполняются в соответствующих ветвях.

Эта таблица показывает, что «выбор лишь небольшого объёма контекста на каждом шаге» не означает фиксированный общий объём работы: основное внимание использует A , сканирование индекса — I , а проекции сжатия непрерывно выполняются для новых входных данных. Эти три вида нагрузки растут по-разному.³³

Таблица 2-5. Kimi K3: KDA, компактное MLA и эксперты в латентном пространстве

93 слоя включают 69 слоёв KDA и 24 слоя MLA; первый слой использует плотную FFN, а остальные выбирают 16 из 896 маршрутизируемых экспертов и выполняют общую ветвь с суммарной шириной $2 \times 3072 = 6144$. В таблице для MLA используется компактный путь, при котором внимание вычисляется непосредственно над скрытыми переменными, а каждая из 96 небольших проекций использует параметры соответствующей головы. Общий эксперт непосредственно обрабатывает 7168-мерный вектор основной части модели; в 3584-мерное латентное пространство переходят только маршрутизируемые эксперты.

Вход модели

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
Выбор строк эмбеддингов токенов	$m \text{ ID} \rightarrow m \times 7168$	Таблица $163840 \times$ 7168	0; обращения для выбора строк учи- тываются отдель- но	1

Ветвь KDA

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
KDA: Q, K, V	$m \times 7168 \rightarrow$ $m \times 12288$	7168×12288 , всего 3	$3 \times 2 \times m \times 7168 \times$ 12288	69
KDA: низкоранговая проекция затухания a	$m \times 7168 \rightarrow$ $m \times 128$	7168×128	$2 \times m \times 7168 \times 128$	69
KDA: низкоранговая проекция затухания b	$m \times 128 \rightarrow m \times$ 12288	128×12288	$2 \times m \times 128 \times$ 12288	69

³³Полный журнал прямого прохода DeepSeek V4-Flash.

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
KDA: гейтинг обновления	$m \times 7168 \rightarrow m \times 96$	7168×96	$2 \times m \times 7168 \times 96$	69
KDA: выходной гейтинг	$m \times 7168 \rightarrow m \times 12288$	7168×12288	$2 \times m \times 7168 \times 12288$	69
KDA: обновление конечного состояния	96 матриц состояния 128×128	Короткая свёртка и параметры затухания	Рекуррентное обновление: $O(m \times 96 \times 128^2)$; блочная реализация учитывается отдельно	69
KDA: выходная проекция	$m \times 12288 \rightarrow m \times 7168$	12288×7168	$2 \times m \times 12288 \times 7168$	69

Ветвь MLA

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	FLOPs матрицы	Число слоёв/ вызовов
MLA: нисходящая проекция запросов	$m \times 7168 \rightarrow m \times 1536$	7168×1536	$2 \times m \times 7168 \times 1536$	24
MLA: восходящая проекция запросов	$m \times 1536 \rightarrow m \times 18432$	1536×18432	$2 \times m \times 1536 \times 18432$	24
MLA: скрытая переменная и дополнительная ветвь	$m \times 7168 \rightarrow m \times 576$	7168×576	$2 \times m \times 7168 \times 576$	24
MLA: объединение восходящей проекции ключей с преобразованием запросов	$m \times 128 \rightarrow m \times 512$	128×512 , всего 96	$96 \times 2 \times m \times 128 \times 512$	24
Компактное MLA: вычисление оценок и агрегация в латентном пространстве	96 голов; ширина вычисления оценок — 576, ширина агрегации — 512	Используется компактное представление контекста	$2 \times 96 \times (576 + 512)N_{\text{pair}}$	24
MLA: восстановление представления значений после агрегации	$m \times 512 \rightarrow m \times 128$	512×128 , всего 96	$96 \times 2 \times m \times 512 \times 128$	24
MLA: выходной гейтинг	$m \times 7168 \rightarrow m \times 12288$	7168×12288	$2 \times m \times 7168 \times 12288$	24
MLA: выходная проекция	$m \times 12288 \rightarrow m \times 7168$	12288×7168	$2 \times m \times 12288 \times 7168$	24

Ветви прямого распространения и экспертов

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	Матричные FLOPs	Число слоёв/ вызовов
Первый слой: плотная сеть прямого распространения	$m \times 7168 \rightarrow m \times 33792 \rightarrow m \times 7168$	две 7168×33792 ; одна 33792×7168	$6 \times m \times 7168 \times 33792$	1

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	Матричные FLOPs	Число слоёв/ вызовов
Остальные слои: оценка маршрути- зации	$m \times 7168 \rightarrow$ $m \times 896$	7168×896	$2 \times m \times 7168 \times 896$	92
Вход в латентное про- странство экспертов	$m \times 7168 \rightarrow$ $m \times 3584$	7168×3584	$2 \times m \times 7168 \times$ 3584	92
Маршрутизируемый эксперт e : преобра- зование латентного пространства	$t_e \times 3584 \rightarrow t_e \times$ $3072 \rightarrow t_e \times 3584$	две 3584×3072 ; одна 3072×3584	$6 \times t_e \times 3584 \times$ 3072	92
Выход из латентного пространства после объединения	$m \times 3584 \rightarrow$ $m \times 7168$	3584×7168	$2 \times m \times 3584 \times$ 7168	92
Общий эксперт: объ- единённая ширина двух экспертов	$m \times 7168 \rightarrow m \times$ $6144 \rightarrow m \times 7168$	две 7168×6144 ; одна 6144×7168	$6 \times m \times 7168 \times$ 6144	92

Связи по глубине

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	Матричные FLOPs	Число слоёв/ вызовов
AttnRes: смешивание по глубине	Доступные для выбора представ- ления блоков $\rightarrow$ текущее представ- ление основной ветви	Параметры сме- шивания по глу- бине	Рассчитывается по числу блоков, участвующих в смешивании на каждом слое; не учитывается как контекстный KV	Сумми- руется по числу бло- ков, участ- вующих в смешива- нии

Выходной интерфейс

Модуль и назначение	Вход $\rightarrow$ выход (число строк $\times$ ширина)	Матрица весов (ширина входа $\times$ ширина выхода)	Матричные FLOPs	Число слоёв/ вызовов
Словарная голова (однократно в конце)	$m_{\text{out}} \times 7168 \rightarrow$ $m_{\text{out}} \times 163840$	7168×163840	$2 \times m_{\text{out}} \times 7168 \times$ 163840	1

При суммировании формулу каждой строки умножают на число слоёв в последнем столбце; для маршрутизируемых экспертов также необходимо просуммировать фактическое число полученных каждым экспертом строк t_e . Конечная нормализация выполняется один раз, а внутрислойная нормализация, активация, позиционная обработка и обновление состояния выполняются вдоль соответствующих ветвей.

При чтении таблицы Kimi K3 необходимо отдельно учитывать число слоёв внимания и сети прямого распространения: 69 слоёв используют KDA, а 24 слоя — MLA; первый слой использует плотную FFN, последующие 92 слоя — МоЕ с латентным пространством. Общие эксперты сохраняют вход основной ветви размерности 7168, а маршрутизируемые эксперты сначала проходят через понижающую размерность проекцию и обрабатывают вход размерности 3584. Таким образом, число слоёв, ширина и t_e в таблице

определяют соответственно число повторений, размер одного выполнения и фактическое число вызовов.³⁴

Таблица 2-6. DeepSeek V4.1 Flash: CED, общий контекст и Engram

Эта таблица охватывает все матричные проекции при обычной генерации текста. Число входных строк m_l обозначает количество токенов запроса, обрабатываемых в текущем вызове на слое l ; u_l обозначает число позиций, для которых необходимо сгенерировать глобальные KV, а $n_{\text{cmp},l}$ — число новых записей сжатия, завершённых в текущем вызове. В эталонном prefill со всеми слоями $m_l = P$; в CED для кодировщика $m_l = P$, а для декодировщика $m_l = \min(P, 128)$, однако на слое — источнике глобальных KV декодировщика по-прежнему выполняется $u_{20} = P$. Число всех строк также необходимо умножить на число запросов B .

Модуль и назначение	Матрица весов (ширина входа × ширина выхода)	Число обрабатываемых строк	Число слоёв/вызовов
Однопроходная mHC-проекция внимания	20480×24	m_l	40
Однопроходная mHC-проекция FFN	20480×24	m_l	40
Понижающая проекция запросов	5120×1280	m_l	40
Повышающая проекция запросов	1280×32768	m_l	40
Проекция локальных SWA KV	5120×512	m_l	40
Групповая выходная проекция	4096×1024 , всего 8 групп	m_l	40
Проекция конкатенированного выхода	8192×5120	m_l	40
Оценка маршрутизации	5120×384	m_l	40
gate общего эксперта	5120×2304	m_l	40
up общего эксперта	5120×2304	m_l	40
down общего эксперта	2304×5120	m_l	40
gate маршрутизируемого эксперта	5120×2304	$\sum_e t_e = 6m_l$	40
up маршрутизируемого эксперта	5120×2304	$\sum_e t_e = 6m_l$	40
down маршрутизируемого эксперта	2304×5120	$\sum_e t_e = 6m_l$	40
KV-проекция результата запроса Engram	6144×25600	m_l	2
Проекция глобальных KV	5120×512	u_l	4
Вентиль сжатия 2:1	5120×512	u_l	3
Проекция ключей глобального индекса	512×128	$n_{\text{cmp},l}$	4
Проекция индексных запросов	1280×4096	m_l	8
Проекция весов индексных голов	5120×32	m_l	8
Словарная голова последнего токена	5120×129280	Однократно для последнего токена	1

³⁴Полный журнал прямого прохода.

Объём матричных вычислений для каждой двумерной проекции равен произведению числа входных строк, ширины входа и ширины выхода, умноженному на 2; групповая выходная проекция выполняет вычисления только внутри каждой группы, поэтому нельзя учитывать отсутствующие связи между группами. Три строки маршрутизируемых экспертов суммируются по числу строк, полученных каждым экспертом, а общий эксперт обрабатывает все позиции токенов запроса.

Взаимодействие внимания не добавляет новых весов: каждый запрос имеет 64 головы, каждая размерностью 512, а QK и PV совместно суммируются по фактически доступным позициям. Индексатор имеет 32 головы размерностью 128 каждая; режимы Full, Reindex и Reuse различаются диапазоном сканирования и тем, выполняется ли индексирование. Компрессор создаёт индексный ключ только после завершения соответствующего блока, поэтому число строк проекции ключей нельзя принимать равным числу позиций каждого входа.

Сначала Engram извлекает по одному 256-мерному вектору из каждого из 24 бакетов, соответствующих n-gram, объединяет их во вход размерностью 6144, а затем посредством указанной в таблице проекции формирует четыре набора ключей и одно общее значение; нормализованное скалярное произведение и вентилирование записывают их в четыре остаточные ветви. Поиск по таблице не считается матричным умножением; чтение полезной нагрузки, вентилирование и взвешенная редукция отдельно приводятся в журнале вычислений. Адрес поиска по таблице определяется только последовательностью токенов и не зависит от активаций, поэтому данные можно предварительно загрузить до выполнения этого слоя; в разделе 6.7.4 сравнивается стоимость размещения таблицы в памяти хоста, HBM или ROM. Конечное взвешенное объединение mHC и RMSNorm (нормализация по среднеквадратичному значению) не добавляют ещё один набор словарных весов.³⁵

2.10 Итоги главы

При проектировании модели можно сначала вычесть из ёмкости ускорителя объём памяти, необходимый для состояния и рабочей области сервиса, а затем сравнить варианты размера и структуры модели, которые могут разместиться в оставшемся объёме памяти. MQA, GQA и разреженная активация показывают, что повышение эффективности системы также служит важным критерием выбора архитектуры модели. В пределах доступной ёмкости необходимо дополнительно сопоставить качество, задержку и стоимость, чтобы окончательно определить проектное решение.

В этой главе три вида ресурсов из главы 1 представлены в виде вычисляемых величин. Размеры матриц определяют объём весов и количество линейных операций, число пар «запрос — ключ» определяет количество операций взаимодействия в механизме внимания, время нахождения кэша в памяти и число вызовов модели определяют объём хранения и чтения состояния, а маршрутизация экспертов — сколько наборов весов используется в каждом пакете. При появлении новых условий сначала следует определить, на какой объект они влияют, а затем вновь подставить значения в соответствующую формулу: увеличение batch, удлинение контекста, расширение набора экспертов и увеличение числа выбранных экспертов воздействуют на разные слагаемые.

³⁵Реализация полного расчёта текстовых матриц V4.1 Flash; ввод CED 8K, эталонный полносвязный ввод, decode для 8K, decode для 1M. Журнал вычислений и проверки.

После прочтения этой главы вы должны уметь для новой конфигурации модели перечислить требования к ресурсам отдельно для одного прямого прохода, одного шага decode и полной генерации, а также объяснить, какие слагаемые растут при изменении длины или batch. В главе 3 дополнительно будут рассмотрены процесс поступления запросов и зависимости между вызовами инструментов, благодаря чему эти требования будут представлены как изменяющаяся во времени нагрузка.

Глава 3

Нагрузки инференса и обучения

Предположим, что сервис получает в среднем четыре запроса в секунду, каждый из которых содержит 4 608 входных токенов и 1 152 выходных токена. В главе 2 уже было показано, как рассчитать объём работы модели по длине входных и выходных данных. Можно ли определить необходимый объём ресурсов, умножив средний объём работы на количество запросов в секунду?

У двух сервисов могут быть совершенно одинаковые средняя частота запросов и средняя длина входных и выходных данных: один всегда получает запросы равномерно, а другой в течение одной минуты обрабатывает преимущественно длинные документы, а в течение следующей — генерирует длинные ответы. Потребность первого в ресурсах на каждом этапе относительно стабильна, тогда как второму в разные периоды требуется больше ресурсов либо для обработки входных данных, либо для генерации. Помимо порядка поступления запросов, на нагрузку также влияют внутренние циклы взаимодействия и ожидания. AI-агент многократно переключается между моделью и инструментами; пока он ожидает результат инструмента, модель приостанавливает генерацию, однако состояние контекста необходимо сохранять для последующих вызовов. На объём вычислений и время удержания состояния при обучении также влияют длина последовательности, количество эффективных меток (токенов, участвующих в вычислении функции потерь) и количество обновлений.

В главе 2 были рассмотрены ёмкость, объём вычислений и объём чтения для одного вызова. В этой главе добавляются ещё три вопроса: когда поступает работа, какие этапы требуют ожидания и какой результат считается завершением задачи. На этой основе последовательно строятся модели очередей, времени удержания состояния, критического пути и стоимости успешно выполненной задачи, после чего выполняется сводный расчёт стоимости обучения и долгосрочной эксплуатации сервиса.

3.1 От единичного запроса к непрерывно поступающей нагрузке

3.1.1 Prefill и Decode

Нагрузка — это совокупность работы, которая за определённое время поступает в систему, ожидает и выполняется. Сначала разберём в хронологическом порядке один запрос. Восстанавливаются S токенов контекста, обрабатываются P новых входных токенов и возвращаются G токенов; цепочка вызовов состоит из одного prefill и $n_d = G - 1$ последующих decode. Пронумеруем последующие decode начиная с $j = 0$: к началу j -го вызова доступный контекст составляет $S + P + j$. Поэтому потребность каждого вызова в ресурсах можно рассчитать методом из главы 2, а порядок вызовов определяется

Повторно используются 6144 токена,
обрабатываются 2048 новых

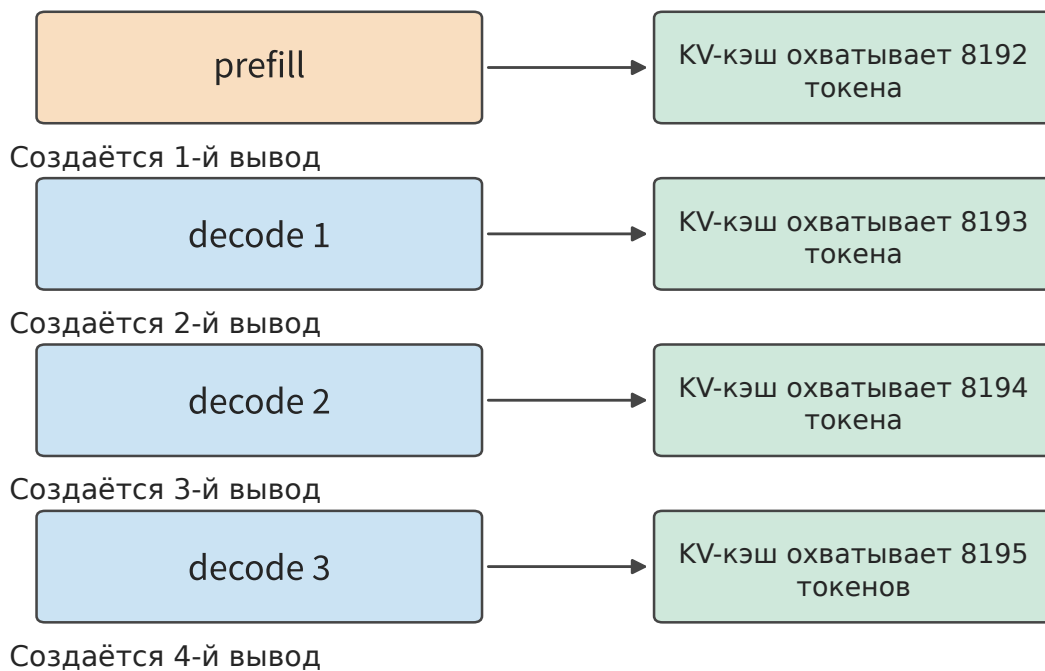


Рисунок 3-1. Восстановление 6144 токенов контекста, обработка 2048 новых входных токенов и генерация 4 выходных токенов. Первый выходной токен создаётся на этапе prefill, каждый из последующих трёх decode добавляет ещё один токен; вертикальное направление показывает порядок вызовов, а интервалы приведены для наглядности.

зависимостями генерации.¹

Оба этапа используют один и тот же набор весов, но предъявляют разные требования к ресурсам. Рассмотрим набор матриц весов: вход prefill содержит BP строк, тогда как на одном шаге decode их всего B . При большом числе входных токенов однократно загруженные веса участвуют в большем количестве операций умножения и сложения; при decode с низким параллелизмом весь набор весов может считываться ради всего нескольких токенов. Отсюда и происходит утверждение, что «prefill обычно ограничен вычислительной мощностью, а decode — пропускной способностью памяти».

Возможности повторного использования возникают не только внутри одного запроса, но и между разными запросами. При одновременном выполнении нескольких decode они совместно используют веса, но каждый считывает собственный контекст; когда поступает длинный вход, prefill может одновременно обработать много заранее известных токенов, векторы признаков которых образуют входную матрицу с большим числом строк. Система обслуживания должна распределять время между заданиями такой разной формы. Простое суммирование общего числа токенов не учитывает, к какому этапу относится каждый токен и когда его можно обработать.

Упражнение 3-1 (углублённое): как число запросов изменяет количество

¹Архитектура модели в главе 2, соглашение о вызове запросов из §2.2 и подсчёт KV для Qwen3-8B; расчёт запросов для четырёх моделей.

вызовов модели при одинаковом суммарном объёме вывода

Рассмотрите два случая: один запрос выводит 1024 токена и восемь запросов выводят по 128 токенов каждый. Вход каждого запроса содержит 1024 токена. Найдите для обоих случаев число prefill, общее количество последующих шагов decode и длину контекста к моменту завершения каждого запроса. Предполагая, что восемь запросов генерируются синхронно, сравните число циклов чтения весов, если в каждом пакете веса считываются один раз совместно для всех запросов, а затем объясните, почему одинакового общего объёма вывода недостаточно для оценки времени завершения.

3.1.2 Цели задач и метрики оценки

Запросы к модели и пользовательские задачи обычно не соответствуют друг другу один к одному. Для ответа на вопрос может хватить одной генерации, тогда как исправление функции может потребовать чтения файла, изменения кода, запуска тестов и повторного изменения после ошибки. При оценке системы сначала определите критерий завершения, а затем решите, что именно измерять.

Сначала отметим на временной шкале три события: поступление запроса t_a , возврат первого токена t_1 и возврат последнего токена t_G . Тогда

$$T_{\text{TTFT}} = t_1 - t_a, \quad T_{\text{request}} = t_G - t_a, \quad \bar{T}_{\text{token}} = \frac{t_G - t_1}{G - 1} \quad (G > 1).$$

TTFT (time to first token) — время от поступления запроса до возврата первого токена; в этой книге оно также называется временем первого ответа. Время первого ответа показывает, через сколько времени начинается вывод, полная задержка запроса — через сколько времени завершается генерация, а средний интервал между токенами характеризует темп вывода. Сокращение некоторого временного отрезка непосредственно изменяет только те метрики, которые включают этот отрезок. Например, сокращение ожидания в очереди перед началом выполнения запроса улучшает TTFT, но не обязательно сокращает интервалы между токенами после начала генерации.

До поступления первого токена клиенту запрос последовательно проходит этапы ожидания в очереди, подготовки входных данных, вычислений на ускорителе и возврата результата. Исходное размещение входных данных — на хосте или ускорителе — определяет, где происходит их передача; после завершения вычислений результат ещё должен пройти обработку на хосте и отправку по соединению. Если записывать время начала и окончания каждого этапа одного запроса, можно определить, на каких этапах в основном расходуется TTFT. В главе 5 подробно рассматривается последовательность отправки задач ускорителю, передачи данных и завершения.

Когда клиент получает несколько токенов блоками, можно непосредственно записывать время поступления и число токенов в каждом блоке, а затем рассчитывать темп вывода. У reasoning-моделей появляется дополнительная точка наблюдения: прежде чем выдать видимый ответ, модель генерирует промежуточное содержимое рассуждений, называемое токенами рассуждения, поэтому при появлении первого внутреннего токена пользователь ещё не обязательно видит ответ. Следовательно, внутренней генерации, видимому пользователю выводу и завершению задачи соответствуют разные моменты времени. Для первого воспроизведения речи дополнительно требуются декодирование и буферизация устройства; это рассматривается в разделе 3.3.3.

Три момента наблюдения одного запроса

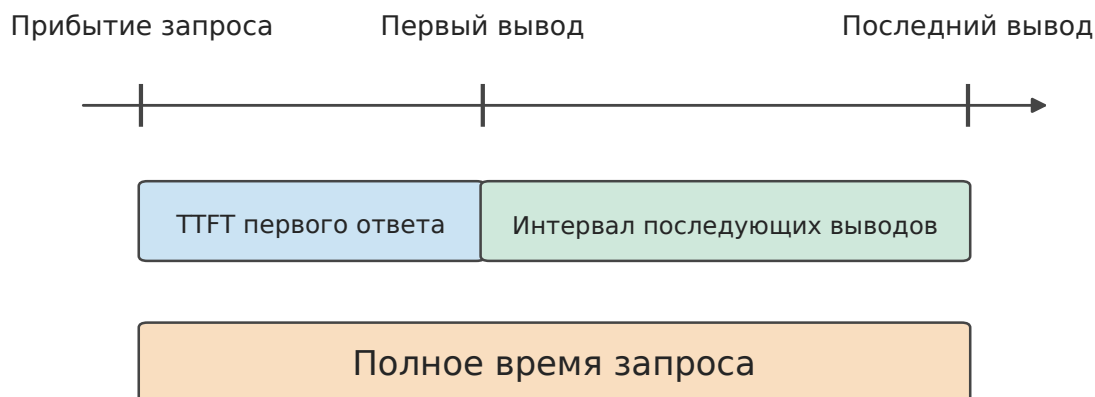


Рисунок 3-2. Поступление запроса, первый и последний выходные токены определяют три временных интервала. Время первого ответа включает ожидание перед началом генерации, интервал вывода описывает процесс генерации, а полное время запроса отсчитывается от поступления до завершения.

Вопрос, на который нужно ответить	Необходимая метрика или запись
Через сколько времени начинается ответ?	TTFT; для reasoning-моделей отдельно фиксируется первый полезный вывод
Непрерывен ли вывод?	Интервалы между токенами/блоками и их распределение
Через сколько времени задача будет завершена?	Полное время запроса, полное время задачи
Сколько соответствующих требованиям задач завершается за секунду?	Эффективная пропускная способность с учётом требований к качеству и срокам
Сколько стоит получить один успешный результат?	Суммарные затраты на модель, инструменты и окружение во всех попытках / число успешных задач
Что продолжает занимать ресурсы во время ожидания?	Запрос, которому принадлежит состояние, версия и размер модели, место хранения и время пребывания

Учёт только среднего значения скрывает долю более медленных запросов. p_{95} — это 95-й перцентиль, описывающий более медленную часть наблюдений выбранной метрики. В этой главе применяется метод ближайшего ранга: n наблюдений упорядочиваются по возрастанию, после чего выбирается наблюдение с номером $\lceil 0.95n \rceil$. Для четырёх запросов p_{95} равен максимальному значению, а для 480 запросов — 456-му значению. Если отдельно упорядочить длительность завершения запросов, ожидания в очереди и первого ответа, можно увидеть, на каком этапе возникает длительное ожидание; p_{95} полного времени запроса вычисляется непосредственно путём упорядочивания полной длительности каждого запроса.

Помимо времени ответа необходимо измерять ресурсы, занимаемые состоянием во время ожидания. Чем дольше состояние остаётся резидентным, тем меньше задач может



Рисунок 3-3. Состояние размером 1 GiB занимает пространство в течение 10 секунд, что соответствует площади 10 GiB·s. По горизонтальной оси отложено время резидентности состояния, по вертикальной — занятое пространство; площадь описывает совокупный объём хранения, занятый за этот период ожидания.

одновременно обслужить одна и та же ёмкость. Пусть размер состояния задачи равен $M(t)$. Тогда интеграл занятого состоянием пространства по времени равен

$$A_M = \int_{t_{\text{start}}}^{t_{\text{end}}} M(t) dt.$$

Если для каждого временного отрезка умножить занятое пространство на его длительность, а затем сложить результаты, получится площадь под кривой изменения занятого пространства во времени. Её единица измерения — byte·s: состояние размером 1 GiB, занимающее пространство в течение 10 секунд, соответствует 10 GiB·s. Размер состояния не увеличился, но всё это время данное пространство было недоступно другим запросам. Если задачи непрерывно поступают со средней скоростью λ задач в секунду, каждая занимает M_0 байт и в среднем ожидает τ секунд, то в стабильном состоянии средний объём пространства, занимаемый только ожидающими задачами, приблизительно равен $\lambda M_0 \tau$. Поэтому ожидание инструмента также становится проблемой ёмкости.

Помимо времени и состояния необходимо учитывать стоимость завершения задачи. Стоимость успешной задачи рассчитывается как сумма стоимости всех попыток, делённая на число успешных задач. Таким образом, неудачные генерации, проверки инструментами и работа окружения входят в числитель, а знаменатель соответствует числу полученных пользователями результатов требуемого качества. Если в пакете нет ни одной успешной задачи, фиксируются его суммарные затраты и причины неудач; после получения успешных результатов суммарная стоимость всех попыток делится на число успешных задач.²

²Исследование стоимости токенов за 2023—2026 годы, \$2, \$4 и \$11; цены API, производственные затраты и стоимость успешно выполненной задачи рассматриваются отдельно.

3.1.3 Распределение запросов

Если в первую минуту ускоритель простаивал, а во вторую внезапно поступило много запросов, требующих длительной генерации, поздние запросы всё равно будут ждать в очереди: вычислительное время предыдущего простоя невозможно сохранить и использовать сейчас. TTFT и интервал вывода описывают работу с отдельным запросом; когда несколько запросов поступают последовательно, эти метрики также зависят от того, конкурируют ли запросы за вычислительную мощность в один и тот же период.

Описание нагрузки должно сохранять совместное распределение времени поступления, длины входа и длины вывода. Длинный запрос не только увеличивает собственное время обслуживания, но и может продлить резидентность состояния; раннее завершение короткого запроса, напротив, может позволить следующему запросу начаться раньше. Поэтому даже при одинаковом среднем объёме работы время ожидания, время завершения и пиковый объём состояния могут различаться. Ниже мы сохраним неизменными общее число запросов и среднюю длину двухминутной нагрузки из начала главы, сначала рассчитаем влияние порядка поступления на накопление очереди, а затем проведём наблюдение на реальной системе.

Реальные запросы часто поступают от разных клиентов. Один клиент преимущественно отправляет длинные документы, другой в основном генерирует длинные ответы; даже если форма запросов каждого клиента стабильна, изменение частоты поступления запросов от разных клиентов изменяет общую нагрузку. Исследование производственных сервисов ServeGen выявило это явление и воспроизвело распределение запросов с учётом состава клиентов и его изменения во времени.³

Пример 3-1. Почему возникает очередь, если средней производительности достаточно? Decode выполняется на двух RTX PRO 6000 Blackwell Workstation Edition. Определите, способны ли они непрерывно обрабатывать нагрузку из начала главы.

Решение. Определим два класса запросов: класс с длинным входом содержит 8192 входных и 256 выходных токенов, а класс с длинным выводом — 1024 входных и 2048 выходных токенов. За две минуты поступает по 240 запросов каждого класса, то есть в среднем четыре запроса в секунду. В одном варианте нагрузки доли обоих классов всегда равны; в другом варианте в первую минуту соотношение классов равно 9 : 1, а во вторую — 1 : 9. Первый выходной token каждого запроса создаётся на этапе prefill, поэтому число последующих шагов decode на единицу меньше числа выходных токенов. Получаем:

Временное окно	Новые входные токены/с	Последующие шаги decode/с
Равномерная смесь	18,432	4,604
Изменяющаяся во времени смесь, первая минута	29,900.8	1,736.8
Изменяющаяся во времени смесь, вторая минута	6,963.2	7,471.2

Предположения модели очереди: равномерное поступление внутри окна и постоянная скорость обработки. Используем жидкостную модель очереди, аппроксимируя дискретную работу генерации непрерывным потоком; внутри каждого окна запросы поступают равномерно, скорость обработки постоянна, а исходная очередь пуста. Еди-

³ServeGen, NSDI 2026, §2–7; заметки по изучению материалов и исходного кода.

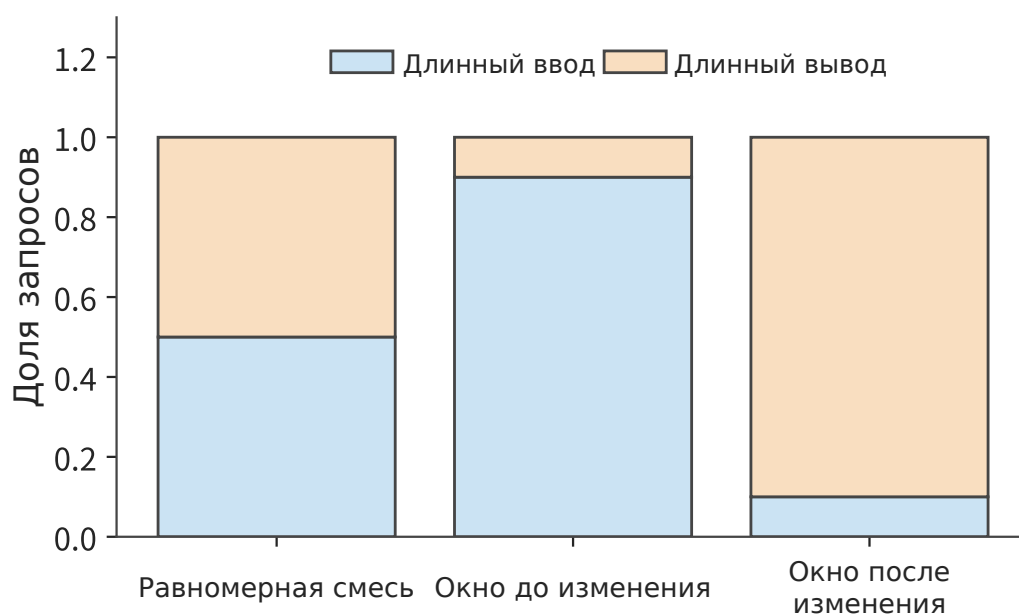


Рисунок 3-4. Состав запросов в трёх временных окнах с одинаковым суммарным объёмом за две минуты. Класс с длинным входом содержит 8192 входных и 256 выходных токенов, класс с длинным выводом — 1024 входных и 2048 выходных токенов; изменение соотношения классов изменяет требования этапов. Длина входа и вывода каждого класса измеряется в токенах.

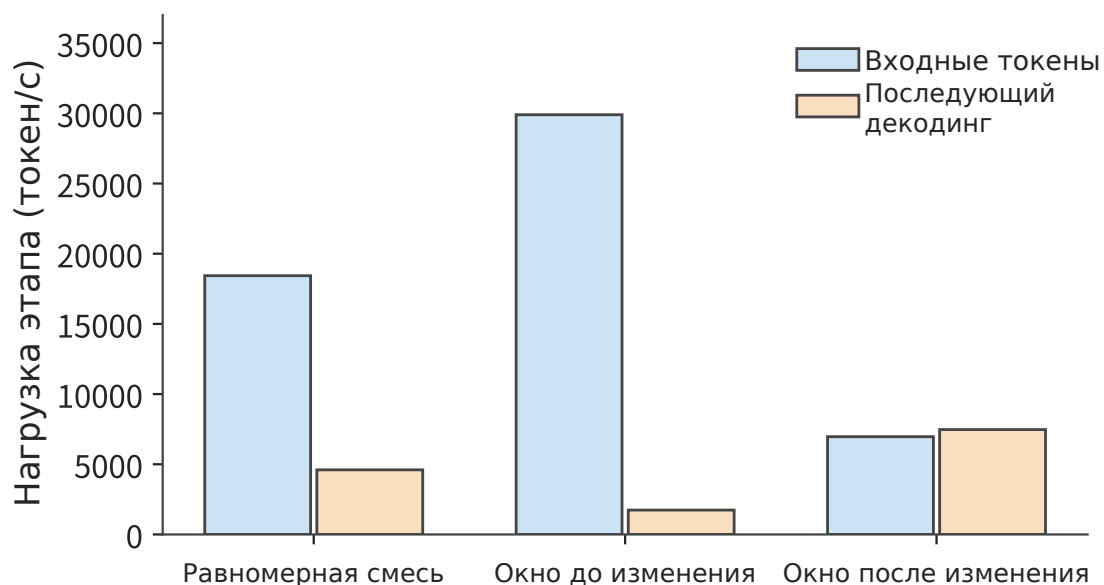
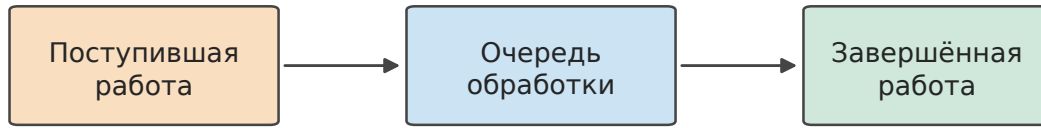


Рисунок 3-5. Требования к обработке входа и последующей генерации, соответствующие составу запросов в трёх временных окнах. Объём входа измеряется в токенах, последующая генерация — в шагах decode на запрос; первый выходной токен каждого запроса уже учтён в prefill. Входная составляющая показывает число новых входных токенов, обрабатываемых за секунду; составляющая decode показывает число последующих шагов decode, которые требуется выполнять за секунду для всех запросов, причём каждый шаг продвигает генерацию на один токен.



Если поступление быстрее обработки, очередь растёт

Рисунок 3-6. Работа сначала поступает в очередь, а затем выполняется вычислительными ресурсами. Если работа поступает быстрее, чем обрабатывается, разница остаётся в очереди; если обработка идёт быстрее поступления, ресурсы постепенно устраняют накопившийся объём работы.

ницей работы служит один последующий шаг decode одного запроса, производительность prefill учитывается отдельно.

Используя временное окно как шаг, накопившийся объём работы можно последовательно вычислять для каждого окна по формуле:

$$Q_{k+1} = \max(0, Q_k + A_k - \mu_k \Delta t).$$

Здесь Q_k — объём необработанной работы в начале окна, A_k — новая работа, поступившая в течение окна, а $\mu_k \Delta t$ — объём работы, который можно выполнить. Когда скорость поступления ниже скорости обработки, накопившийся объём постепенно уменьшается, пока очередь не опустеет; когда скорость поступления превышает скорость обработки, незавершённая работа переносится в следующее окно.

Скорость обработки μ_k в рекуррентной формуле необходимо вывести из параметров оборудования. Первые выходные токены четырёх запросов в секунду создаются на этапе prefill, поэтому из 4,608 выходных токенов 4,604 относятся к последующим шагам decode. Скорость обработки decode определяется пропускной способностью видеопамяти. RTX PRO 6000 Blackwell Workstation Edition имеет 96 GB видеопамяти и пропускную способность 1792 GB/s; на каждой карте размещена полная копия весов Qwen3-8B BF16, и одновременно может выполняться не более 64 последовательностей. На каждом шаге decode необходимо как минимум один раз прочитать 15.14 GB общих весов, а затем считать весь предыдущий KV каждой последовательности; каждый токен контекста занимает 144 KiB. Во время decode средняя длина контекста класса с длинным входом составляет $8192 + 127 = 8319$ токенов, а класса с длинным выводом — $1024 + 1023 = 2047$ токенов; для равномерной смеси средняя длина контекста, взвешенная по числу шагов decode, равна $(255 \times 8319 + 2047 \times 2047) / 2302 \approx 2742$ токенов. При одновременном выполнении 64 последовательностей за один шаг необходимо прочитать и записать не менее 41.02 GB, что занимает 22.89 ms; следовательно, одна карта может выполнять не более приблизительно 2,795.66 шага в секунду, а две карты — около 5,591.33 шага. Эта скорость обработки является физическим пределом, допускаемым пропускной способ-



Рисунок 3-7. Жидкостная модель, в которой две RTX PRO 6000 могут выполнять не более приблизительно 5,591 шага decode в секунду. Во вторую минуту очередь возрастает до 112,792 шага; после прекращения поступления запросов на 120-й секунде она опустошается приблизительно за 20.17 секунды.

ностью видеопамяти; фактическая скорость может быть только ниже. Для всех трёх временных окон используется одна и та же скорость обработки.

Пусть очередь decode изначально пуста, а работа генерации запросов равномерно поступает в каждом окне. Для равномерной смеси и первой минуты производительности достаточно; во вторую минуту каждую секунду сверх доступной производительности поступает $7471.2 - 5591.33 \approx 1879.87$ шага, поэтому за 60 секунд накапливается около 112,792 шага. После прекращения поступления запросов для опустошения очереди требуется ещё приблизительно 20.17 секунды согласно расчёту $112792/5591.33$. Неиспользованную в первую минуту производительность невозможно сохранить до второй минуты — именно поэтому глобальное среднее значение скрывает накопление очереди.⁴

Как ограничение ёмкости KV влияет на очередь и время первого ответа. Запуск этих двух последовательностей поступления запросов на Qwen3-8B/vLLM 0.23 позволяет наблюдать, как описанные различия влияют на ожидание пользователей. В качестве ускорителя используется одна RTX PRO 6000 Blackwell Workstation Edition с 96 GB видеопамяти. Пул KV — это область видеопамяти, специально выделенная для контекста ключей и значений запросов; в данном эксперименте его ёмкость составляет 24 GiB, а одновременно может выполняться не более 64 последовательностей. Кэширование префиксов означает повторное использование между запросами уже существующего KV для одинакового начала; в данном эксперименте оно отключено, и каждый запрос генерирует указанное число токенов. При нехватке пространства KV планировщик может при-

⁴Распределение запросов и конфигурация ресурсов, требования этапов после фиксации двухминутного входного потока и коррекции первого токена; нижняя граница пропускной способности decode для 64 последовательностей на RTX PRO 6000, средний контекст принят равным 2742 токенам.

остановить уже выполняющийся запрос, освободить занятое им пространство KV, а затем возобновить его или выполнить вычисления заново; такое поведение называется вытеснением. В обеих группах нагрузки за 120 секунд поступает 480 запросов. После завершения всех запросов получены следующие результаты.⁵

⁵Упражнение 3-2: реальное двухминутное воспроизведение, полный журнал входных данных, отправки и завершения, выборки KV и диапазон выполнения.

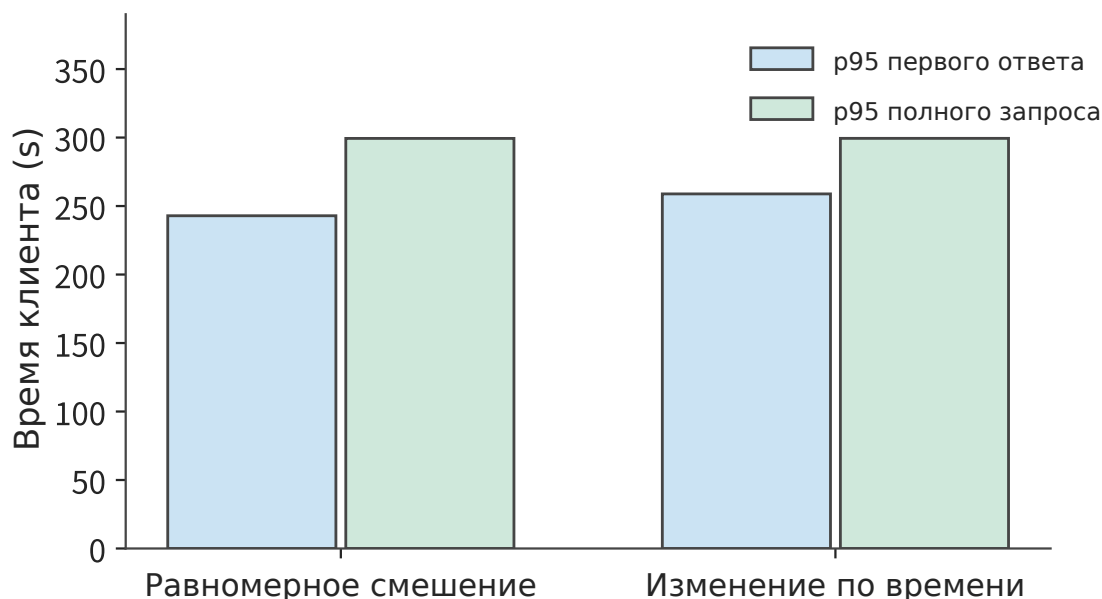


Рисунок 3-8. Результаты воспроизведения двух последовательностей поступления запросов на одном и том же реальном экземпляре. Значения p95 полного времени запроса близки, тогда как p95 времени первого ответа различается приблизительно на 16 секунд; модель, ускоритель, пул KV и предел параллелизма зафиксированы в соответствии с условиями эксперимента из основного текста.

Фактическое наблюдение	Равномерная смесь	Сначала 9:1, затем 1:9
p95 полного времени запроса	около 299.4 s	около 299.4 s
p95 TTFT	242.9 s	258.9 s
p95 ожидания первого назначения на выполнение	242.4 s	258.4 s
Время от начала до завершения последнего запроса	419.8 s	418.7 s
События вытеснения	62	24

В обоих воспроизведениях накопился значительный объём работы, что уже можно было предвидеть из нижней оценки, основанной на пропускной способности в примере 3-1: одна карта может выполнять не более приблизительно 2,796 шага decode в секунду, что ниже средней потребности равномерной смеси в 4,604 шага; кроме того, на той же карте необходимо выполнять prefill. Для 480 запросов требуется в общей сложности 552,480 последующих шагов decode, и даже только эта часть работы занимает не менее 197.6 секунды, тогда как все запросы поступают в течение 120 секунд.

Значения p95 полного времени запроса и время последнего завершения близки в обоих воспроизведениях, но TTFT и ожидание первого назначения на выполнение различаются приблизительно на 16 секунд. Записи выборки показывают, что в обоих экспериментах заполнение пула KV достигало 100%, а запросы долго ожидали перед началом выполнения. Распределение входов и выходов по времени изменяет моменты освобождения состояния и приёма новых запросов, поэтому даже при близком времени завершения всей группы пользователи могут начинать получать ответы в разное время. Вытеснение при заполнении пула KV также изменяет порядок ожидания и последующего

выполнения запросов.

Таким образом, очередь может возникать как из-за недостаточной скорости обработки, так и потому, что состояние уже выполняющихся запросов ещё не освобождено: после заполнения пула состояний новые запросы должны ждать освобождения пространства существующими запросами, даже если часть вычислительных блоков простаивает.⁶

Рекуррентная формула накопления очереди из примера 3-1 позволяет прогнозировать её размер и при других условиях всплеска. Если скорость поступления работы λ_w непрерывно превышает скорость обработки μ в течение τ секунд, а очередь изначально пуста, то накопленный объём равен $(\lambda_w - \mu)\tau$; после прекращения поступления время опустошения очереди равно $(\lambda_w - \mu)\tau/\mu$. Если длительность всплеска удваивается, накопленный объём и время опустошения также удваиваются.

Упражнение 3-2 (основное): как всплеск запросов и ожидание инструментов увеличивают очередь и объём состояния

Возьмите интенсивность нагрузки второй минуты из примера 3-1, то есть 7,471.2 нового шага decode в секунду. Измените длительность этой всплесковой нагрузки на 30 секунд и рассмотрите ресурсы decode из двух и трёх RTX PRO 6000 Blackwell Workstation Edition; для каждой карты используйте скорость обработки около 2,796 шага/с, рассчитанную в примере 3-1. Изначально очередь пуста. Найдите изменение накопленного объёма работы во времени за эти 30 секунд и время, необходимое для опустошения очереди после прекращения поступления запросов. Затем рассмотрите другую нагрузку: каждую секунду два задания переходят в стадию ожидания инструмента, каждое ожидает 10 секунд и в это время занимает 1 GiB пространства состояния. Найдите среднее число ожидающих заданий и объём занятого состояния в стабильном режиме; повторите расчёт после увеличения времени ожидания до 30 секунд.

3.2 Многораундовое взаимодействие и задачи AI-агента

3.2.1 Многораундовый диалог и рост префикса

В разделе 3.1 каждый запрос рассматривался как отдельный вызов; в диалогах и задачах агента несколько вызовов одной задачи связаны последовательно. Следующий запрос в многораундовом диалоге обычно содержит контекст. В первом раунде передаются системный промпт и вопрос пользователя; во втором помимо нового вопроса может присутствовать ответ из предыдущего раунда; затем содержимое продолжает накапливаться в каждом раунде. Если учитывать только объём нового текста пользователя, длина входных данных, передаваемых модели, будет недооценена.

Пусть длина входных данных в раунде i равна I_i , а длина повторно используемого префикса — K_i . Тогда объём входных данных, который требуется обработать заново в этом раунде, равен

$$P_i = I_i - K_i, \quad S_i = K_i.$$

Подставив P_i и S_i в формулы из главы 2, получим, что проекции и FFN обрабатывают только новые токены, однако механизм внимания по-прежнему должен обеспечивать

⁶Фактическое воспроизведение окна ServeGen.

новым запросам доступ ко всему префиксу. Таким образом, повторное использование кэша сокращает повторные вычисления, но не исключает старый контекст из последующих вычислений.

Рассмотрим выгоду от кэширования на примере задачи исправления кода с четырьмя вызовами модели. Совокупная длина входных данных четырёх раундов составляет 5 297 токенов, из которых 4 480 попали в кэш, а 817 — нет. Во втором раунде входные данные содержат 1 443 токена, из которых 1 392 попали в кэш, поэтому требуется обработать лишь 51 некешированный токен; однако последующий decode всё равно должен обращаться к состоянию контекста. Если в каждом раунде заново вычислять все входные данные, матричные операции *prefill* потребуют в сумме 76,196 TFLOPs; при повторном использовании найденного в кэше префикса потребуется в сумме 11,976 TFLOPs. Следовательно, попадание в кэш не только сокращает повторные вычисления для старых входных данных, но и требует от системы продолжать хранить старое состояние.⁷

Изменение контекста влияет на длину повторно используемого префикса. При добавлении нового содержимого в конец исходный префикс остаётся неизменным; если фрагмент старого контекста заменяется сводкой, представления после места замены необходимо заново вычислить от нового префикса; изменение определений инструментов или шаблонов может изменить входные данные ещё раньше. Эти операции определяют, сколько токенов в следующем раунде попадёт в кэш, а сколько снова поступит в *prefill*.

Стратегия работы с контекстом влияет и на будущую нагрузку. Постоянное добавление нового содержимого в конец контекста способствует повторному использованию исходного префикса, но устаревшее содержимое продолжает занимать место в контексте. Замена содержимого в начале позволяет сократить контекст, однако кэш для последующей части может потребовать перестроения. Стоимость сохранения, вытеснения и повторных вычислений зависит от объёма вычислений при перестроении, размера состояния и интервала повторного использования (глава 8).⁸

С помощью описанной в этом разделе зависимости повторного использования префикса сравним два способа редактирования: добавление 100 токенов в конец добавляет лишь 100 новых строк входных данных; если же изменить 501-й токен среди уже существующих 2 000, все последующие представления изменятся, а максимальная длина непосредственно повторно используемого префикса составит лишь 500 токенов. Выгода от кэширования зависит не только от объёма изменений, но и от их расположения.

С точки зрения приложения контекст служит интерфейсом оркестрации задачи; с точки зрения системы организация контекста определяет объём вычислений и время нахождения состояния в памяти. Добавление результатов инструментов, замена раннего содержимого и создание нескольких ветвей-кандидатов по-разному изменяют объём новых вычислений, повторно используемый префикс и приватное состояние. Зная эти зависимости, обслуживающая система может размещать кэш с учётом повторного использования, совместно использовать историю между ветвями и выгружать состояние на время ожидания инструментов. Поэтому журнал нагрузки должен также содержать стабильный префикс, позиции изменений, зависимости между ветвями и интервалы повторного использования. В главе 8 эти сведения будут использоваться для выбора стратегии контекста и кэширования, а в главе 9 к ним добавятся расположение состояния и

⁷Повторный расчёт реальной траектории с включённым режимом рассуждений.

⁸Организация контекста и пример AI-агента, связывающий главу 2 авторской книги об AI-агентах с журналом эксперимента.

время передачи.

3.2.2 Рассуждение и проверка: сколько стоит выполнение одной задачи

Кэширование префикса сокращает повторные вычисления для уже имеющихся входных данных. Другой класс методов, напротив, намеренно увеличивает объём вычислений при генерации в надежде повысить правильность ответа. Увеличить вычисления на этапе рассуждения можно тремя основными способами: удлинить процесс рассуждения в одном ответе, одновременно сгенерировать несколько ответов и выбрать один из них либо распределять число генераций и их длину в зависимости от сложности задачи. Удлинение рассуждения увеличивает время последовательной генерации и объём состояния контекста; генерация нескольких ответов увеличивает объём вычислений и количество проверок; распределение по сложности требует определить момент остановки для каждой задачи.

Чтобы понять, оправдано ли увеличение объёма вычислений при генерации, необходимо сравнивать стоимость одной успешно выполненной задачи. Пусть средняя стоимость одной попытки равна c , а вероятность успеха — p . Тогда ожидаемое число повторных попыток при неизменной стратегии равно $1/p$, а значит,

$$C_{\text{success}} = \frac{c}{p}.$$

Пусть стоимость и вероятность успеха нового метода равны соответственно c_2, p_2 , а старого — c_1, p_1 . Для снижения стоимости необходимо, чтобы выполнялось $p_2/p_1 > c_2/c_1$. Следовательно, удлинение рассуждения или увеличение числа выборок оправдано лишь тогда, когда относительный прирост правильности превышает относительный рост стоимости. В c учитывается стоимость всех генераций, проверок и неудачных попыток.

Пример: увеличение длины генерации не повысило вероятность успешного решения математических задач. Скорость генерации отражает полезность системы только при совместном рассмотрении с правильностью выполнения задачи. В одном эксперименте с математическими задачами для четырёх задач генерировалось по несколько ответов, а максимальная длина вывода была увеличена с 1 024 до 4 096 токенов, однако за два раунда попыток не удалось получить итоговый ответ, прошедший оценивание. Ресурсы на генерацию были израсходованы, но задача осталась невыполненной; увеличение длины вывода также не привело автоматически к получению корректного решения.⁹

При проверке ответа следует различать формат интерфейса и правильность содержимого. Ответ в правильном формате может содержать ошибку в вычислениях, а численно верный ответ может не включать поля, требуемые задачей. Только разделив эти два типа ошибок, можно определить, что именно следует изменить: способ генерации, ограничения на вывод или процесс решения задачи.¹⁰

При расчёте стоимости необходимо учитывать и изменение самой цены. Даже после снижения цены токена следует вычислять полную стоимость выполнения задачи. Рассмотрим простой пример: цена токена в новой системе снизилась до 1/10 от прежней,

⁹Упражнение 3-3: бюджет 1024 и бюджет 4096; теоретические основы см. в Test-Time Compute.

¹⁰Диагностика при отключённом режиме рассуждений, отдельно фиксирующая результат оценки по заранее определённым критериям, результат последующего извлечения ответа из вывода и данные наблюдения за параллельными процессами.

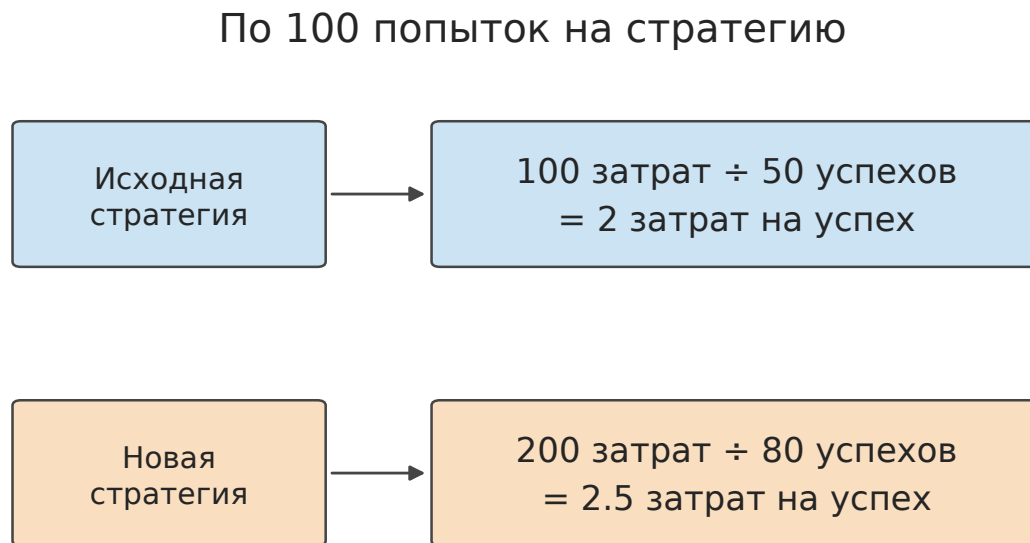


Рисунок 3-9. Учебное сравнение двух стратегий по 100 попыток. Стоимость всех попыток входит в числитель, а число успешных задач, прошедших проверку, — в знаменатель; стоимость успешной задачи для двух стратегий составляет соответственно 2 и 2,5 единицы.

расход токенов на задачу вырос в 20 раз, а вероятность успеха увеличилась с 50% до 80%. Если учитывать только стоимость модели и считать распределение повторных попыток стабильным, отношение стоимости одной успешной задачи составит $2 \times \frac{0.5}{0.8} = 1.25$. Снижение цены, повышение качества и рост стоимости успешной задачи могут происходить одновременно. Для агента с контекстом можно суммировать стоимость всех попыток по фактической траектории, а в качестве знаменателя использовать число задач, прошедших проверку.¹¹

Упражнение 3-3 (углублённое): может ли повышение вероятности успеха компенсировать рост стоимости одной попытки

Для старого метода стоимость одной попытки составляет 1 единицу, а вероятность успеха — 50%; для нового метода стоимость равна 2 единицам, а вероятность успеха — 80%. Рассчитайте среднюю стоимость выполнения одной успешной задачи и определите, какой метод экономичнее. Если стоимость нового метода снизится до 1,5 единицы, какой должна быть минимальная вероятность успеха, чтобы средняя стоимость успешной задачи не превышала стоимость старого метода? Для задач, требующих многократной выборки, объясните, как расходы на генерацию, проверку и неудачные попытки должны учитываться в общей стоимости.

Используйте пример запроса принятия решения из конца раздела 3.2.4: обработка одного запроса моделью принятия решений стоит 0,000081 доллара, а передача одного запроса LLM — 0,013880 доллара. Пусть вероятность успеха до и после передачи равна соответственно p_1, p_2 . Найдите среднюю

¹¹Исследование стоимости токенов за 2023—2026 годы, \$2, \$4 и \$11; цены API, производственные затраты и стоимость успешно выполненной задачи рассматриваются отдельно.

стоимость одного запроса при доле передачи 1% и 5%; какого минимального значения должно достичь p_2/p_1 , чтобы передача снижала стоимость одной успешной задачи?

3.2.3 Нагрузка агента

Обратная связь также может изменять последующий процесс вычислений. Агент для работы с кодом сначала создаёт план изменений, инструмент записывает изменения в код и запускает тесты, после чего модель читает результаты тестирования и определяет следующий шаг. Каждый цикл включает как вычисления модели, так и выполнение инструментов; возвращаемые инструментами журналы становятся входными данными следующего раунда. Поэтому удаление нерелевантных записей из журналов снижает объём вычислений prefill и занимаемого состоянием контекста в следующем раунде, а сокращение числа бесполезных изменений непосредственно уменьшает количество циклов.

Практическая задача этой главы состоит в исправлении функции слияния интервалов и проверке правильности возвращаемого результата и неизменности входных данных. Модель принимает решения об изменениях и вызывает инструменты, а инструменты записывают изменения в файл и запускают тесты. Ниже отдельно учитываются время модели, время инструментов и состояние, сохраняемое во время ожидания, для траектории, на которой исходные проверки были успешно выполнены за четыре раунда. Подробности вызовов инструментов приведены в конце главы в разделе Траектория выполнения задачи с кодом.¹²

Вопросно-ответные системы с дополненной поиском генерацией также включают похожий процесс: средство поиска находит документы, после чего они передаются языковой модели для генерации ответа. При одинаковом качестве ответа меньший объём найденного содержимого сокращает последующие расходы на prefill, хранение KV и чтение контекста. Поэтому содержимое, возвращаемое средством поиска, непосредственно влияет на нагрузку модели. В главе 8 различные схемы поиска будут сравнены на конкретных задачах.¹³

3.2.4 Ветвление, ожидание инструментов и хранение состояния

Последовательность вызовов модели и выполнения инструментов можно представить в виде графа зависимостей. Вдоль последовательного пути длительности этапов складываются; независимые ветви могут запускаться одновременно, а точка их слияния ожидает более медленную ветвь. Путь с наибольшей суммарной длительностью от начала до конца в графе зависимостей называется критическим путём; именно он определяет минимально возможное время завершения задачи. Сначала на реальной задаче с кодом посмотрим, какая доля времени приходится на модель и инструменты, а затем изменим зависимости и оценим возможный выигрыш от параллельного выполнения. В таблице ниже сравниваются два запуска одной задачи с выключенным и включённым режимом рассуждения; этот режим определяет, генерирует ли модель сначала токены рассуждения, описанные в разделе 3.1.2.

¹²Упражнение 3-4: две траектории агента для работы с кодом, включая независимую проверку и дополнительное условие для псевдонима.

¹³Пример расчёта для поиска и генерации.

Наблюдение	Режим рассуждения выключен	Режим рассуждения включён
Число раундов вызова модели	12	4
Общее число выходных токенов	765	2 733
Суммарное фактическое время вызовов модели	13,156 s	76,294 s
Суммарное фактическое время выполнения инструментов	0,476 s	0,078 s
Проверки правильности возвращаемого значения и неизменности входных данных	315/1013	1013/1013
Дополнительная проверка: отсутствие алиасов у возвращаемых подписков	1/1013	710/1013

При выключенном режиме рассуждения доля попаданий в кэш составляла около 83,4%, однако код так и не был исправлен. После включения режима время работы модели увеличилось, но обе исходные проверки — правильности возвращаемого значения и неизменности входных данных после вызова — прошли полностью. Дополнительная проверка отсутствия алиасов требует, чтобы возвращаемые подписки были независимы от исходных входных данных. При включённом режиме рассуждения программа прошла 710 таких проверок, однако в остальных 303 случаях подписки по-прежнему были общими.¹⁴ Две группы проверок соответствуют различным свойствам программы, что показывает необходимость формулировать критерии выполнения задачи в виде конкретных исполняемых проверок.

При включённом режиме рассуждения задача вывела в общей сложности 2 733 токена: 2 553 до маркера окончания рассуждения, обозначающего завершение его содержимого, 3 токена самого маркера и 177 токенов после него. На рассуждение приходится основная часть работы по генерации, а вызовы инструментов преобразуют решения модели в операции изменения файлов и запуска тестов. Если сложить фактическое время вызовов модели и выполнения инструментов в порядке исполнения, а затем добавить время управления и передачи, получится полное время ожидания пользователя до завершения задачи.

Пример 3-2: насколько ускорение вычислений модели в первом раунде сокращает время задачи с кодом? Выполните анализ с помощью закона Амдала из главы 1, предполагая, что последующее поведение и качество не изменяются.

Решение: возьмём за основу ту же траекторию выполнения и сократим вдвое вычисления модели в первом раунде, занимавшие 36,375 секунды, оставив без изменений остальные этапы, вывод, поведение инструментов и качество. Тогда общее время снизится с 76,510 до 58,323 секунды. В этом случае $f = 36.375/76.510$, $s = 2$, а общее ускорение составит приблизительно 1,31. Первый раунд занимает около 47,5% общего времени, поэтому его двукратное сокращение экономит около 23,8% общего времени; остальные этапы по-прежнему занимают в сумме около 40,1 секунды. Выгода от оптимизации зависит от первоначальной доли сокращаемого этапа.¹⁵

¹⁴Упражнение 3-4: две траектории агента для работы с кодом, включая независимую проверку и дополнительное условие для псевдонима.

¹⁵Двукратное ускорение только участка работы модели в первом раунде.

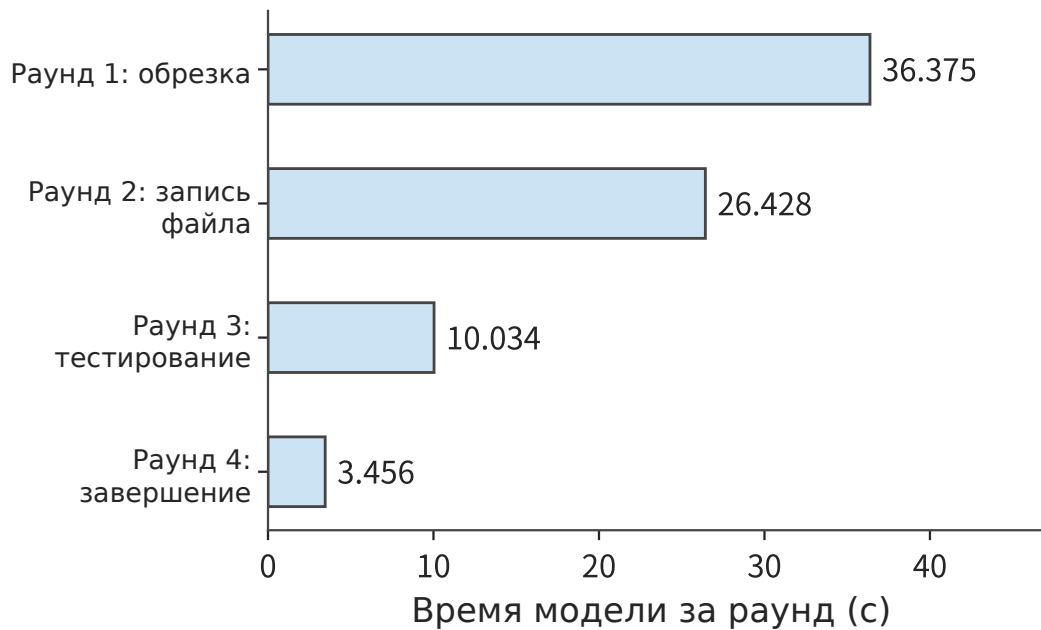


Рисунок 3-10. Фактическое время четырёх раундов вызова модели в задаче с кодом; время каждого раунда отсчитывается от его собственной точки начала. Суммарное время модели составляет 76,294 секунды, инструментов — около 0,078 секунды; полное время задачи также включает управление и передачу.

Зависимости между инструментами также влияют на общее время. Пусть модель сначала выполняет вычисления в течение 2 секунд, затем вызывает два инструмента длительностью 6 и 10 секунд соответственно, а в конце вычисляет ещё 3 секунды. Если второй инструмент зависит от результата первого, общее время составит $2 + 6 + 10 + 3 = 21$ секунд; если инструменты независимы, общее время составит $2 + \max(6, 10) + 3 = 15$ секунд. Параллельное выполнение сокращает время на 6 секунд, однако суммарный объём работы двух инструментов по-прежнему равен 16 секундам.

Параллельное выполнение и ветвление изменяют не только время, но и объём занимаемого состоянием пространства. Если каждая ветвь также должна отдельно генерировать текст, общий префикс можно совместно использовать, тогда как окончания ветвей растут независимо. Пусть длина общего префикса равна H_0 , ветвь i добавляет h_i токенов, а состояние одного токена занимает c_{KV} . Тогда общий объём состояния равен $c_{KV}(H_0 + \sum_i h_i)$. Параллельное выполнение сокращает критический путь, но одновременно приводит к тому, что состояния большего числа ветвей занимают память.

В результатах измерений задачи исправления кода из раздела 3.2.3 вызовы инструментов занимают крайне мало времени, а основная его часть расходуется на вычисления модели. Если вместо этого инструмент выполняет компиляцию длительностью 10 секунд, то во время паузы модели контекст, необходимый следующему раунду, продолжает занимать ёмкость; если одновременно ожидают несколько задач, объём хранимого состояния суммируется по всем задачам. Чем дольше выполняются инструменты, тем дольше сохраняется состояние и тем меньше запросов система может обрабатывать одновременно. В главах 8 и 9 на этой временной шкале будут размещены операции кэширования и передачи.

Упражнение 3-4 (основное): сколько времени задачи экономят парал-

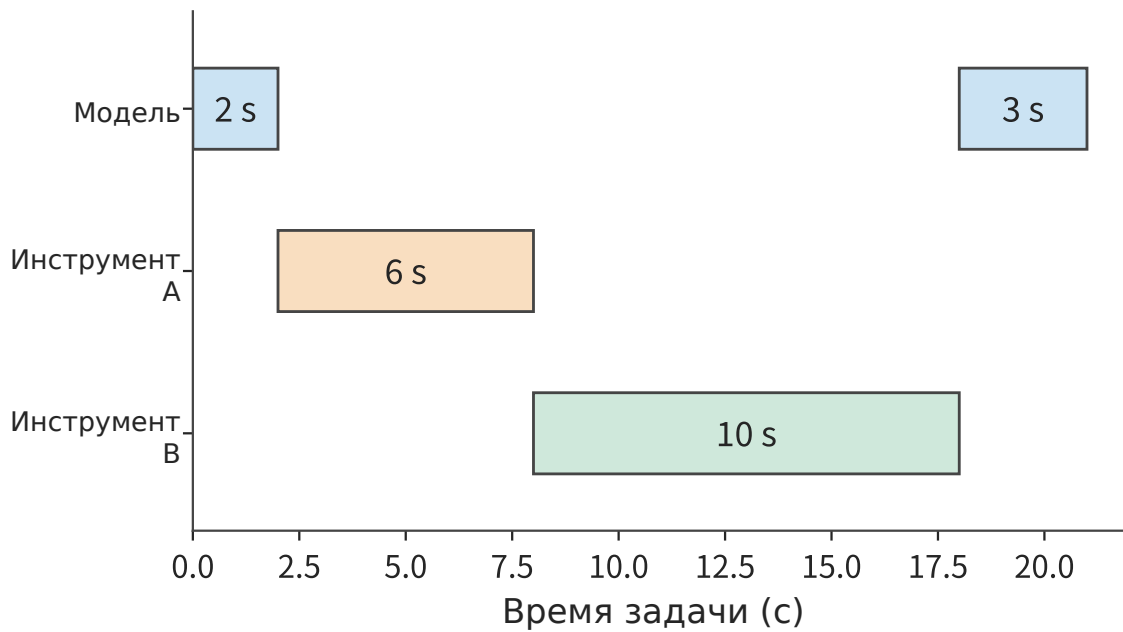


Рисунок 3-11. Временная шкала задачи с последовательной зависимостью между двумя инструментами. Модель сначала работает 2 секунды, инструмент А — 6 секунд, инструмент В — 10 секунд, после чего модель работает ещё 3 секунды; итого 21 секунда.

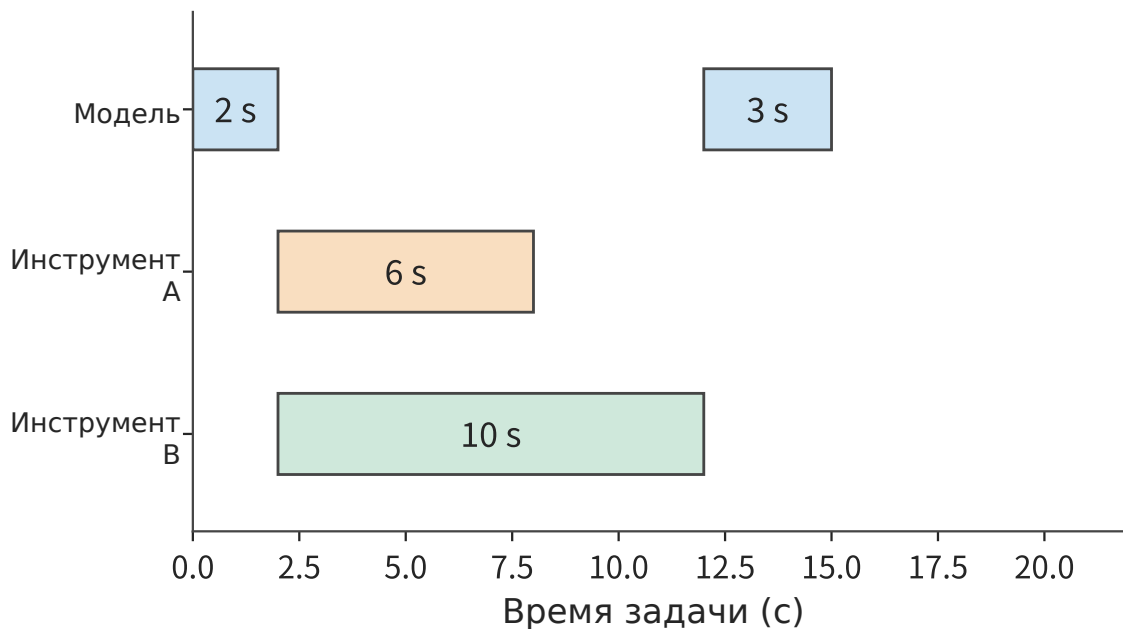


Рисунок 3-12. Если два инструмента независимы, их можно запустить одновременно. Модель продолжает работу после завершения более медленного инструмента В, и задача занимает в общей сложности 15 секунд. Масштаб времени совпадает с предыдущим рисунком; суммарный объём работы инструментов по-прежнему равен 16 секундам.

Общий префикс хранится один раз, ветви дополняются отдельно

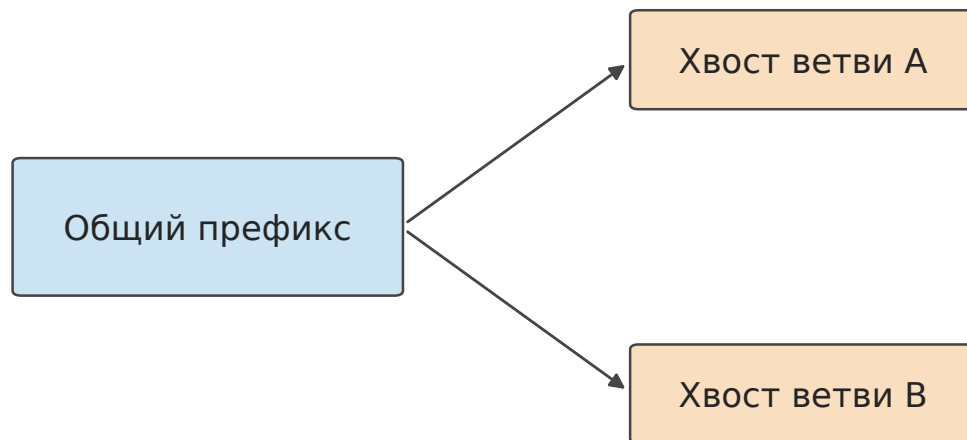


Рисунок 3-13. Две ветви генерации ссылаются на один общий префикс и отдельно сохраняют добавленные окончания. Стрелками обозначены связи ссылок; ёмкость общего префикса учитывается только один раз.

лельное выполнение инструментов и локальное ускорение

Сначала модель выполняет вычисления в течение 2 секунд, затем вызывает два независимых инструмента длительностью 6 и 10 секунд соответственно. После завершения обоих инструментов модель вычисляет ещё 3 секунды, и задача завершается. Рассчитайте общее время задачи при последовательном и одновременном выполнении инструментов. Предполагая, что состояние размером 1 GiB занимает пространство только во время ожидания результатов инструментов, найдите для обоих способов выполнения произведение объёма состояния на длительность его хранения в GiB·s.

Результаты измерений задачи с кодом из этого раздела показывают, что общее время задачи составляет 76,510 секунды, из которых 36,375 секунды занимает вычисление модели в первом раунде, а суммарное время выполнения всех инструментов равно 0,078 секунды. Оставив прочие этапы неизменными, рассчитайте, сколько времени задачи экономит каждая из двух оптимизаций: четырёхкратное ускорение вычислений модели в первом раунде и двукратное ускорение выполнения всех инструментов. На основе результатов объясните, почему одного коэффициента локального ускорения недостаточно для определения выгоды оптимизации для всей задачи.

Требования нагрузки, в свою очередь, могут стимулировать изменения в архитектуре модели. Конкретный пример можно увидеть, если перенести тот же сеанс на DeepSeek V4.1 Flash. Предположим, у агента уже имеется общий контекст, после чего инструмент возвращает новые материалы. Если префикс не попал в кэш, сервису потребуется заново обработать большой объём входных данных, хотя вывод в этом раунде может быть коротким. Для подобных сценариев с преобладанием входных данных снижение стоимости prefill полезнее, чем оптимизация только пошаговой генерации. Поэтому V4.1

использует описанную в главе 2 архитектуру CED с 20 слоями причинного кодировщика и 20 слоями декодировщика: основная часть входных данных проходит только через основную часть кодировщика, а глобальный KV декодировщика проецируется из представления последнего слоя кодировщика. Для построения состояния SWA декодировщика в него повторно передаются выходы кодировщика максимум для последних 128 токенов промпта, что приближённо восстанавливает локальное состояние этого фрагмента. Выходные токены продолжают проходить через все 40 слоёв.¹⁶

Сначала сравним объём работы, связанный с вычислениями экспертных матриц, измеряя его суммой числа экспертных слоёв, через которые проходят все токены. Пусть входные данные, обрабатываемые из пустого состояния, содержат P токенов. Тогда обычный путь через 40 слоёв даёт $40P$, а схема CED из отчёта — $20P + 20 \min(P, 128)$. При $P = 8192$ получаются соответственно 327 680 и 166 400, причём второе значение составляет около 50,8% первого; при $P \leq 128$ эта составляющая не сокращается.

При наличии готового префикса объём работы по обработке входных данных совместно определяется длиной добавления и способом восстановления. Если SWA кодировщика найден в кэше, основная часть кодировщика может продолжить обработку добавленных входных данных; если SWA кодировщика отсутствует, сначала повторно производится окно в конце префикса. После этого оба пути должны построить SWA декодировщика для текущего раунда. В главе 8 на учебных примерах контекстов 8K и 128K будут сравнены стоимости сохранения состояния и повторного выполнения.

Пример расчёта: почему Jev выдаёт ответ в мгновение ока. В сентябре 2026 года компания TypeSafe AI, основанная бывшим исследователем OpenAI Diogo Almeida, выпустила модель Jev. По словам основателя в статье о выпуске, в OpenAI он участвовал в разработке методов, обучавших языковые модели следовать инструкциям и вести диалог с людьми; впоследствии эти работы стали частью исследований, лежащих в основе ChatGPT.¹⁷ Официальная демонстрация Jev выглядит впечатляюще: для одного и того же запроса службы поддержки GPT-5.6 Terra потребовалось 8,566 секунды, а Jev — всего 0,114 секунды, при этом стоимость Jev составила лишь 1/171 от стоимости первой модели. У многих сразу возникает вопрос: действительно ли это всё ещё Transformer? Не используется ли какая-то выдающаяся infra? После прочтения этой книги станет понятно, что никакого секрета здесь нет и читатель также сможет без труда это реализовать.

Jev не генерирует текст. Вызывающая сторона передаёт состояние, например заявку службы поддержки, и несколько вопросов с заранее заданными вариантами ответов, а модель возвращает вероятности этих вариантов. Если вернуться к цепочке вызовов из раздела 3.1.1, такой запрос содержит только prefill и не содержит decode. Состояние и все вопросы объединяются и обрабатываются за один прямой проход, а распределение на выходе в последней позиции каждого вопроса и является ответом. Это похоже на несколько ветвей на рисунке 3-13, расходящихся из одного префикса: каждая делает только один шаг и останавливается. Вопросы не зависят друг от друга, а после завершения запроса хранить KV не требуется. Возможность непосредственно интерпретировать эти вероятности как уровень уверенности обеспечивается обучением: TypeSafe называет свой метод RLCD. Его идея совпадает с академическим методом RLCR — оба добавляют

¹⁶Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт для сквозного примера из нескольких глав.

¹⁷Цены и параллельная демонстрация на главной странице TypeSafe AI; блог о выпуске содержит рассказ основателя, а также пояснения по параллельной выборке, RLCD и методике оценки; документация интерфейса поясняет, что варианты параллельно оцениваются в одном и том же состоянии.

Обработка входа 8K с нуля

Обычные полные слои: 8,192 токена × 40 слоёв

Энкодер CED
8,192 токена × 20 слоёв

Локальный
повтор
128 × 20

327,680 → 166,400 токен-слоёв

Отдельно: глобальная KV-проекция, внимание и прочее

При генерации выполняются все 40 слоёв основной сети

Рисунок 3-14. Сумма числа экспертных слоёв, через которые проходят все токены в одном пакете входных данных размером 8K. Обычный путь через все слои выполняет 40 слоёв; путь CED выполняет 20 слоёв кодировщика и повторно воспроизводит 20 слоёв декодировщика для последних 128 токенов. Компоненты, обозначенные серым, по-прежнему требуется вычислять отдельно; на этапе генерации выполняется вся основная сеть.

к вознаграждению оценку Брайера, чтобы выдаваемые моделью вероятности соответствовали фактической доле правильных ответов.¹⁸

Проверим это в условиях, используемых в книге. Jev взимает плату только за входные данные — 0,042 доллара за миллион токенов; в демонстрации Jev стоил 0,000081 доллара, что соответствует примерно 1 929 входным токенам. Если выполнить такой запрос на Qwen3-8B, один прямой проход потребует около 28 TFLOPs, что при 40-процентной вычислительной эффективности H100 SXM займёт примерно 70 ms — сопоставимо с 0,114 секунды в демонстрации. При замене на DeepSeek V4.1 Flash и использовании пути CED объём вычислений увеличится лишь примерно на четверть. Если передать запрос обычной LLM, после prefill потребуется последовательно генерировать токены, причём каждый шаг ограничен пропускной способностью памяти и занимает около 9 ms. При генерации 64 токенов задержка возрастет до 0,65 секунды, что в 9 раз больше задержки запроса принятия решения; однако благодаря пакетной обработке decode время GPU увеличится лишь на 28%. На рисунке 3-15 оба способа выполнения показаны на одной временной шкале: мгновенный ответ достигается за счёт исключения последовательной генерации.¹⁹

Цена также не содержит никакой загадки. Если в соответствии с отчётом о системе инференса DeepSeek принять стоимость аренды GPU равной 2 долларам в час, то вычислительная стоимость обработки миллиона входных токенов плотной моделью 8B или V4.1 Flash составит около 0,02–0,025 доллара, а цена Jev превышает её в 1,7–2 раза. Иными словами, модель такого класса может обеспечить эту цену только за счёт обычного prefill; с другой стороны, если цена должна покрывать расходы, объём вычислений Jev на один токен вряд ли более чем вдвое превышает объём вычислений плотной модели 8B. Разница в стоимости в 171 раз из демонстрации лишь в малой степени обусловлена отсутствием генерации: на одной и той же модели исключение 64–512 выходных токенов сокращает время GPU только в 1,3–3,4 раза. Остальная разница связана с ценой самой передовой модели и большим количеством токенов, генерируемых во время инференса.

При практическом использовании запросы, в которых модель принятия решений недостаточно уверена, приходится передавать LLM для повторной обработки. Пусть доля передаваемых запросов равна e , тогда средняя стоимость одного запроса составляет $c_{\text{dec}} + e c_{\text{LLM}}$. При двух значениях стоимости из демонстрации достаточно, чтобы e превысило примерно 0,58%, и стоимость передачи LLM станет выше стоимости самой модели принятия решений. Поэтому стоимость s успешной задачи из раздела 3.2.2 также должна включать стоимость передачи. Экономичность модели принятия решений зависит от того, сколько запросов требуется передавать и насколько дорога принимающая их LLM.

3.3 Мультимодальность и взаимодействие в реальном времени

Результаты работы инструментов AI-агента обычно передаются в следующий раунд ввода в виде текста. Если на вход поступают изображения или непрерывный аудиопоток, перед вызовом модели необходимо дополнительно закодировать входные данные, а на

¹⁸Beyond Binary Rewards: Training LMs to Reason About Their Uncertainty, вознаграждение RLRCR из \$3.

¹⁹Сравнение путей выполнения и цен для запроса на принятие решения, включая полный послойный путь V4.1 Flash, путь LLM с 512 выходными токенами и сопоставление с тарифами DeepSeek; расчёт можно воспроизвести командой `python3 calculations/calc.py decision-request --format md`; обзор системы инференса DeepSeek с оценкой 2 доллара за GPU-час; модели и цены DeepSeek.

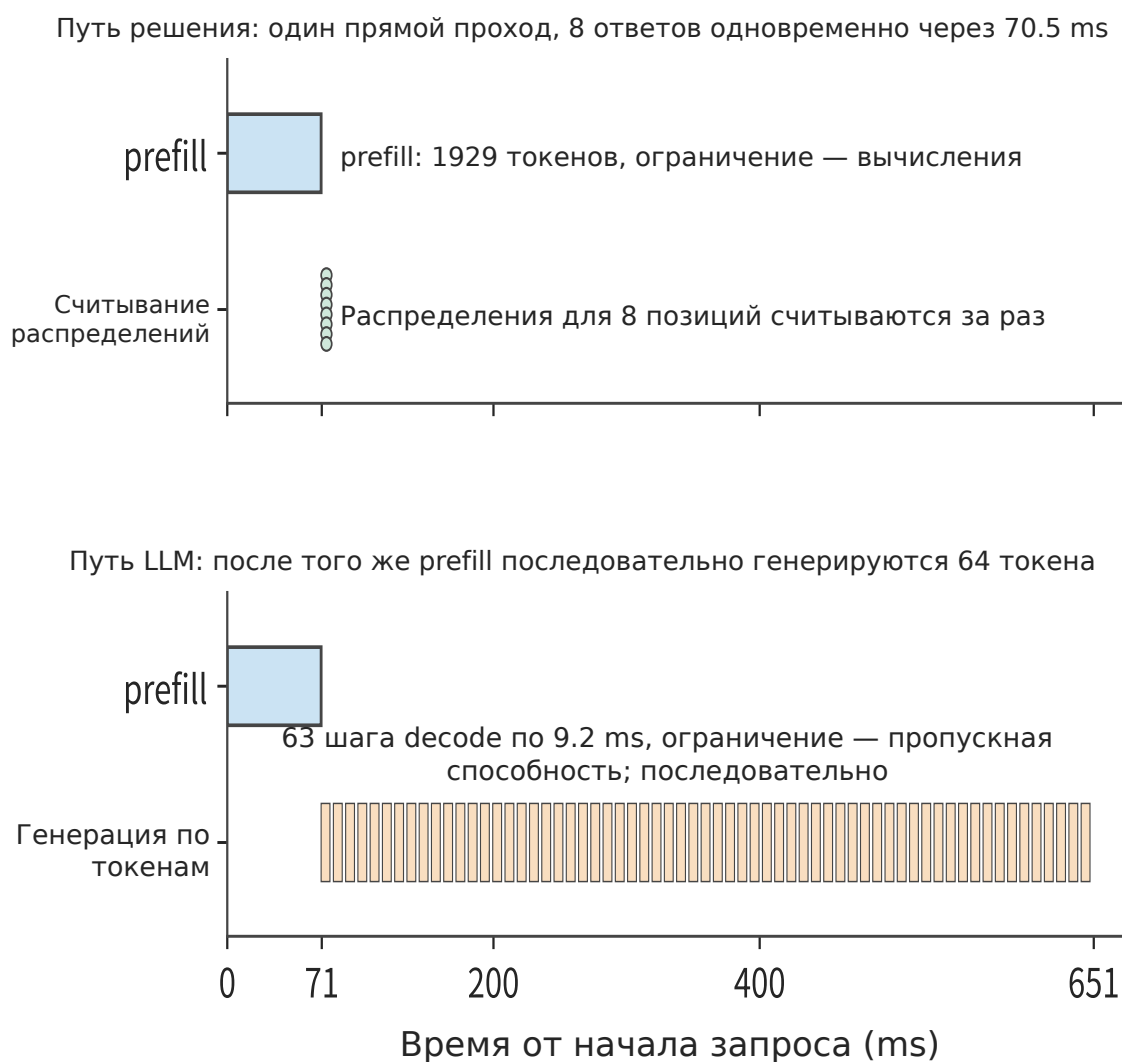


Рисунок 3-15. Два способа выполнения одного запроса из 1 929 токенов на H100 SXM. Вверху показан запрос принятия решения, где после одного прямого прохода одновременно считываются ответы на 8 вопросов; внизу — обычная LLM, последовательно генерирующая 64 токена после prefill. Используется модель Qwen3-8B с вычислительной эффективностью 40% и эффективностью использования пропускной способности 50%.

выходе может потребоваться непрерывное воспроизведение. В этом разделе сначала рассматривается, как меняется форма данных между этапами, а затем — к какому моменту должен завершиться каждый этап.

3.3.1 Визуальное кодирование, генерация текста и состояние

Прежде чем изображение поступит в языковую модель, оно проходит предварительную обработку, визуальное кодирование и проекцию. Обозначим визуальное кодирование через E , языковой prefill — через P , а последующий decode — через D . Тогда один сеанс визуальных вопросов и ответов имеет базовую зависимость $E \rightarrow P \rightarrow D$. Размер изображения влияет на объём работы E , а количество визуальных токенов после кодирования — на объём работы языковой части; эти две составляющие следует рассчитывать отдельно.

Сначала подсчитаем визуальные токены, затем — размерность признаков каждого вектора. Пусть высота и ширина изображения после предварительной обработки равны $H_{\text{img}}, W_{\text{img}}$. Разобьём изображение на квадратные фрагменты (patch) со стороной p_{patch} , а затем объединим по r фрагментов вдоль каждой стороны. Тогда количество визуальных токенов равно

$$n_v = \frac{H_{\text{img}} W_{\text{img}}}{p_{\text{patch}}^2 r^2}.$$

Рассмотрим конкретную конфигурацию. Изображение размером 640×640 в визуально-языковой модели Qwen3-VL-4B при разбиении на фрагменты 16×16 даёт 1600 фрагментов, а после объединения 2×2 — 400 визуальных токенов. Итоговая ширина признаков равна 2560, поэтому BF16-тензор $[400, 2560]$ с двумя байтами на элемент занимает $400 \times 2560 \times 2 = 2,048,000$ байт.

Однако результат кодирования этим не ограничивается. Чтобы предоставить языковой модели визуальную информацию с разных уровней глубины, DeepStack передаёт признаки промежуточных слоёв визуального кодировщика в соответствующие языковые слои. Модель также берёт признаки трёх промежуточных визуальных слоёв этого типа и объединяет их с признаками последнего слоя. Полный тензор имеет форму $[400, 10240]$ и занимает 8 192 000 байт, то есть 7,8125 МиБ. Четыре группы признаков соответствуют одним и тем же 400 визуальным токенам: ширина признаков увеличивается в четыре раза, но языковая последовательность по-прежнему пополняется 400 визуальными токенами. Объём матричных вычислений при визуальном кодировании составляет около 1,310 TFLOPs; декодирование и масштабирование изображения выполняются до кодирования, а языковые вычисления — после него.²⁰

На рис. 3-16 и 3-17 визуальное кодирование изменяет количество визуальных токенов и ширину признаков; после поступления в языковую модель создаётся ещё одно представление. Кэш кодировщика (encoder cache , EC) хранит выходные данные визуального кодировщика, а языковая магистраль создаёт KV-кэш. В этой конфигурации языковой KV-кэш занимает 144 КиБ на токен, поэтому 400 визуальным токенам соответствует 56,25 МиБ. Таким образом, один и тот же учебный снимок экрана на трёх этапах обработки представлен разными объёмами данных: исходный сжатый файл — 0,8 МБ, полный BF16 EC — 7,8125 МиБ, логический KV-кэш визуальных токенов — 56,25 МиБ.²¹

²⁰Матрица визуального кодирования и подсчёт тензоров.

²¹Байты, состояния и распределение по этапам для мультимодального ввода, фиксированная конфигурация Qwen3-VL-4B, предварительная обработка и полный DeepStack EC.

Изображение кодируется в визуальные токены и подаётся
в языковую модель

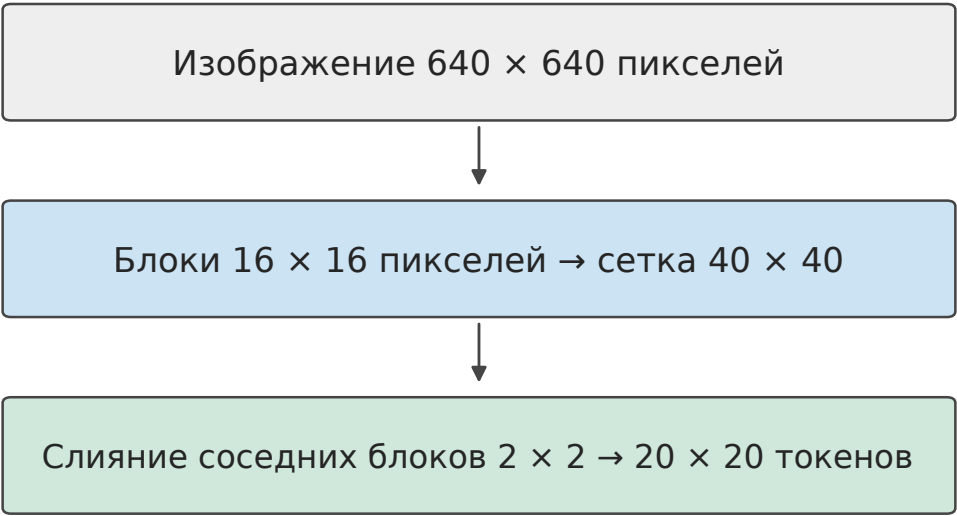


Рис. 3-16. Преобразование изображения из пиксельной сетки в визуальные токены. Квадратное изображение 640×640 разбивается на сетку из 40×40 фрагментов, соседние фрагменты 2×2 объединяются в один токен, образуя сетку 20×20, всего 400 визуальных токенов.

Объект	Размер в этом примере	Условие возможности повторного использования
Сжатый снимок экрана	0,8 МБ, заданный вход	Одинаковое содержимое файла
Полный результат кодирования ЕС	7,8125 МиБ	Совпадают изображение, предварительная обработка, кодировщик и формат вывода
Языковой KV-кэш визуальных токенов	56,25 МиБ	Дополнительно зависит от предшествующего контекста, порядка, позиций и состояния модели

Если изображение и конфигурация кодирования совпадают, ЕС можно использовать повторно. Языковой KV-кэш также зависит от контекста перед изображением: если вопрос расположен перед изображением, замена вопроса изменяет KV-кэш визуальных токенов; если вопрос расположен после изображения, то благодаря причинному вниманию KV-кэш предшествующих визуальных токенов остаётся неизменным.

Оценка ёмкости состояния кодирования и KV-кэша для мультимодального ввода. Пусть вход содержит четыре изображения и 400 текстовых токенов. Рассчитаем ЕС и логический KV-кэш этих входных токенов. ЕС четырёх изображений одинаковой спецификации занимает 31,25 МиБ, а KV-кэш визуальных токенов — 225 МиБ; с добавлением 400 текстовых токенов логический KV-кэш входа составляет 281,25 МиБ.

Исходное изображение, ЕС и KV-кэш — это три представления данных, соответству-

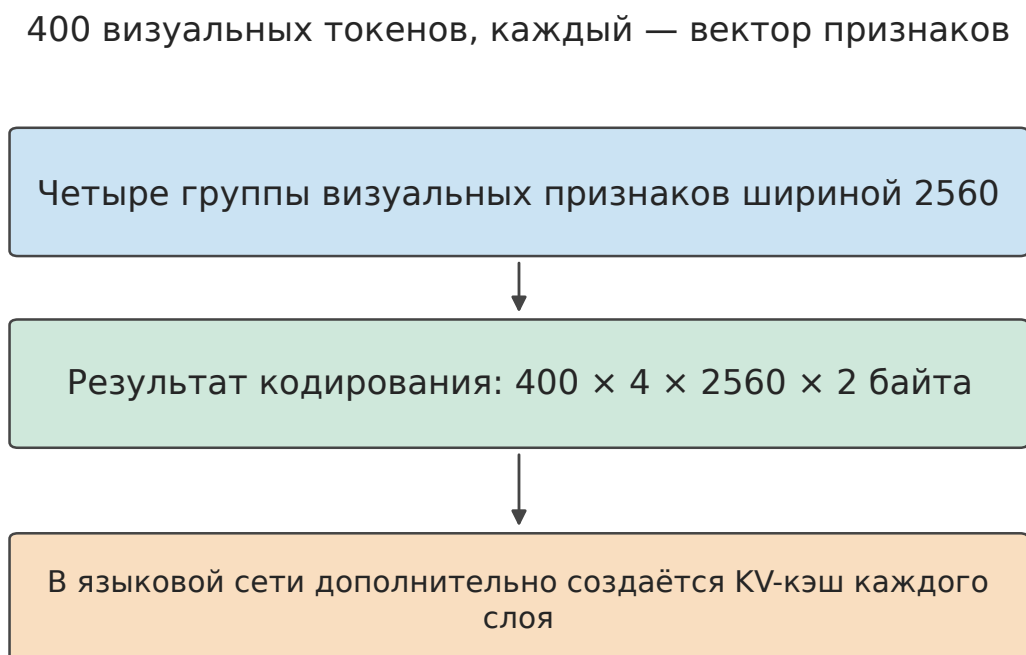


Рис. 3-17. Количество визуальных токенов и ширина признаков каждого токена измеряются отдельно. Четыре группы кодированных признаков BF16 размерностью 2560 занимают 7,8125 МиБ; после поступления этих визуальных токенов в языковую модель дополнительно создаётся KV-состояние каждого слоя.

ющие трём различным точкам передачи. При передаче исходного изображения принимающая сторона должна ещё выполнить визуальное кодирование; при передаче ЕС она начинает с языкового prefill; при передаче KV-кэша передаётся уже сформированный языковой контекст. В главе 8 рассматривается выполнение на одном ускорителе, в главе 9 сравнивается передача между этапами, а в главе 12 тот же поток данных размещается на канале «устройство — периферия — облако».

Если оставить размер фрагментов и способ объединения неизменными, но удвоить высоту и ширину изображения, количество визуальных токенов увеличится в четыре раза. Поэтому ЕС и языковой KV-кэш визуальных токенов также увеличатся в четыре раза; взаимодействие токенов внутри визуального кодировщика необходимо дополнительно рассчитывать с учётом структуры внимания. Объединение признаков увеличивает ширину, а увеличение изображения — количество визуальных токенов; в языковой магистрали эти изменения приводят к разным последствиям.

3.3.2 Процессы вычисления при генерации аудио и изображений

При визуальных вопросах и ответах языковая модель по-прежнему генерирует текст токен за токеном. Если же выходными данными являются звук или изображение, изменяются как объём данных, обрабатываемых за одну итерацию, так и количество итераций.

При генерации аудио существует также внутрикадровый цикл. Рассмотрим модель генерации речи S2 Pro от Fish Audio. Медленный путь (slow) на каждом шаге продвигается на один аудиокادر, а быстрый путь (fast) внутри кадра дополняет индексы нескольких

кодовых книг. Кодовая книга — это набор векторов-кандидатов дискретного представления звука, а кодек (codec) преобразует эти индексы обратно в звуковую волну. Для каждого кадра состояние сначала формируется одним прямым проходом по быстрому пути, после чего ещё девять предсказаний дополняют индексы кодовых книг. Таким образом, для генерации одного кадра требуется десять прямых проходов по быстрому пути. Следовательно, языковые токены в секунду, акустические кадры в секунду и продолжительность генерируемого аудио в секунду характеризуют разные скорости генерации.^{22,23}

При генерации изображений используется цикл другого типа. Латентное представление (latent) — это сжатое непрерывное представление изображения; в процессе удаления шума зашумлённое представление пошагово преобразуется в целевое изображение. Диффузионный Transformer (DiT) выполняет преобразование на каждом шаге, classifier-free guidance (CFG) объединяет условное и безусловное предсказания, а вариационный автокодировщик (VAE) преобразует пиксели в сжатое представление и обратно. Например, в модели генерации изображений Qwen-Image-2512 изображение размером 1024×1024 образует 4096 латентных позиций и обрабатывается за 50 шагов удаления шума. В режиме true CFG на каждом шаге выполняются две ветви прямого прохода DiT, то есть всего 100 прямых проходов DiT. Объём матричных вычислений при удалении шума составляет около $7,830 \times 10^{15}$ FLOPs; текстовый кодировщик и VAE выполняют собственную работу. Набор латентных позиций остаётся неизменным на протяжении удаления шума, а их содержимое обновляется на каждом шаге, поэтому суммарный объём работы растёт с количеством шагов удаления шума.²⁴

Обозначим объём вычислений на одном шаге удаления шума через $F_{\text{step}}(n_v)$, количество шагов — через n_s , а количество ветвей guidance на каждом шаге — через n_b . Тогда общий объём вычислений равен $n_s n_b F_{\text{step}}(n_v)$. При удвоении количества шагов объём вычислений также удваивается; повышение разрешения увеличивает количество пространственных позиций в латентном представлении изображения, а вместе с ним — число строк матриц и объём взаимодействий между позициями внутри одного шага. При генерации текста контекст постоянно дополняется, поэтому объём обращений к нему растёт с каждым шагом; при удалении шума изображения многократно обновляется один и тот же набор латентных позиций, и при фиксированном разрешении суммарный объём вычислений линейно растёт с количеством шагов удаления шума.

3.3.3 Непрерывное восприятие, начало ответа и прерывание

Общий объём работы определяет среднюю потребность в обработке, но воспроизведение в реальном времени также требует своевременного поступления каждого блока данных. Пусть $A(t)$ обозначает доступную для воспроизведения продолжительность аудио, поступившего к моменту t , а $P(t)$ — уже воспроизведённую продолжительность аудио. Тогда остаток аудио в буфере равен

$$B_{\text{audio}}(t) = A(t) - P(t).$$

Во время непрерывного воспроизведения за каждую прошедшую секунду воспроизведённая продолжительность $P(t)$ увеличивается на одну секунду, а $A(t)$ скачкообразно

²²Материалы по мультимодальным вычислениям.

²³Подбор материалов и путь выполнения генеративной модели; аудио Omni, аудио Fish, изображения FLUX, видео H3, видео Wan.

²⁴Подсчёт запросов для генерации изображений Qwen.

но растёт по мере поступления блоков данных. Расстояние между двумя кривыми — это запас в буфере; если он опускается до нуля, а следующий блок ещё не поступил, звук прерывается. Предварительная буферизация увеличивает начальный запас, но одновременно откладывает начало воспроизведения.

Временные требования действуют как на входе, так и на выходе. Если агент, работающий со снимками экрана, наблюдает за экраном только после завершения действия, он может пропустить всплывающее окно, которое появилось и исчезло во время выполнения. Исследование непрерывного наблюдения и взаимодействия АОI посвящено именно таким сценариям: оно отделяет наблюдение за экраном от выполнения действий, непрерывно собирает изображения, аудио и события, а затем отбирает записи для использования моделью. Повышение частоты наблюдений позволяет обнаружить больше кратковременных событий, но сохранение большего числа ключевых кадров также расходует контекст; избыток ключевых кадров может вытеснить важную информацию и снизить качество выполнения задачи.²⁵ Поэтому непрерывное наблюдение необходимо сочетать с фильтрацией, определяя, какие данные следует сохранить и передать модели для обработки.

Для речи временные ограничения ещё строже. После того как пользователь закончил говорить, система должна получить или подтвердить ввод, выполнить распознавание или кодирование, инференс, генерацию речи и воспроизведение. После начала воспроизведения последующие аудиоданные также должны поступать вовремя, чтобы избежать прерывания.

После поступления в приложение аудиоблоки должны пройти декодирование, постановку в очередь и воспроизведение. В двух измерениях речевого взаимодействия время от окончания речи пользователя до поступления первого аудиоблока составляло около 400 мс и 370 мс соответственно, а медианный интервал поступления последующих аудиоблоков в обоих случаях был около 94 мс.²⁶ Время поступления показывает, когда данные становятся доступны, а часы воспроизведения определяют, когда именно прозвучит аудио.

Рассмотрим временную шкалу с приведёнными ниже параметрами, чтобы показать, как буфер между этими двумя процессами предотвращает прерывание звука. На входе каждые 20 мс формируется один блок; при частоте 24 кГц, одном канале и двух байтах на отсчёт размер каждого блока равен:

$$24,000 \times 0.020 \times 1 \times 2 = 960 \text{ bytes.}$$

После завершения сбора каждого блока обработка моделью занимает 12 мс, отправка — 1 мс, а распространение по сети обычно требует ещё 5 мс. Первый блок поступает на 38-й мс, ожидает 40 мс в джиттер-буфере — буфере, который временно хранит поступившее аудио и сглаживает колебания времени поступления, — и начинает воспроизводиться на 78-й мс после начала сбора. Задержка распространения третьего блока задана равной 50 мс, поэтому он поступает только на 123-й мс, пропускает запланированный момент воспроизведения на 118-й мс и вызывает дополнительную паузу длительностью 5 мс. Хотя последующие блоки уже поступили, они должны ожидать воспроизведения в порядке звучания.

Если увеличить буфер до 60 мс, пауза исчезнет, но первое воспроизведение начнётся

²⁵Статья об АОI и пример с ключевыми кадрами.

²⁶Две исторические записи речи.

От входного представления до слышимого звука

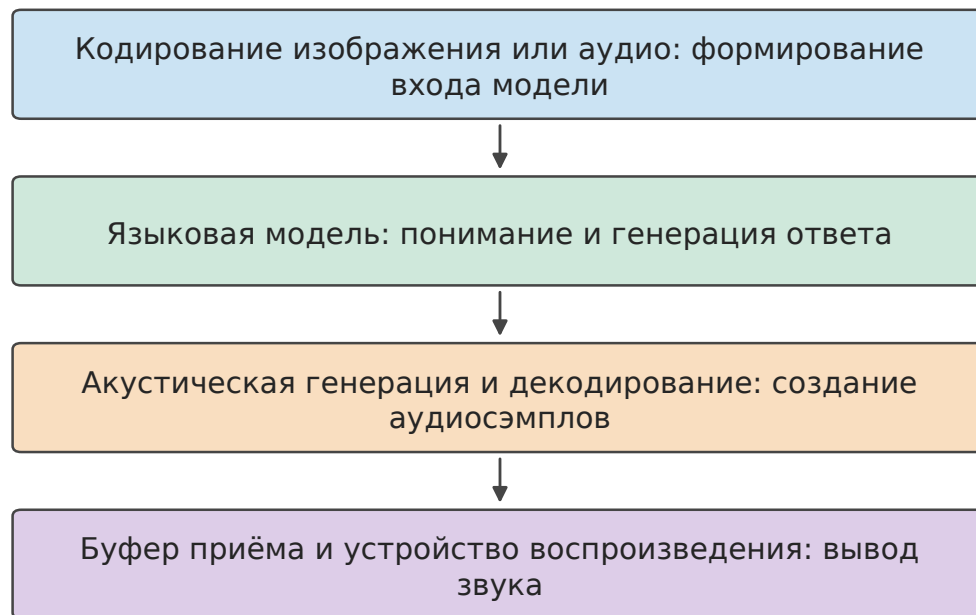


Рис. 3-18. Комбинируемые мультимодальные этапы. Кодирование формирует вход модели, языковая модель генерирует ответ, акустический модуль преобразует ответ в аудио, а буфер и устройство воспроизведения на принимающей стороне определяют момент фактического появления звука.

лишь на 98-й мс. Увеличение буфера позволяет избежать прерывания воспроизведения из-за данного колебания времени поступления, но ценой более позднего начала; если средняя скорость поступления аудио в течение длительного времени ниже скорости воспроизведения, конечный буфер всё равно рано или поздно опустеет.²⁷

Помимо непрерывности воспроизведения, необходимо отдельно проверять реакцию на прерывание. На рис. 3-20 отсчёт начинается с момента подачи пользователем команды прерывания и продолжается до фактической остановки локального воспроизведения; остановку удалённой генерации необходимо определять по другому управляющему пути.

Прерывание включает несколько операций: приложение отправляет команду отмены, устройство воспроизведения отключает звук, бэкенд прекращает генерацию, а буфер и состояние освобождаются. На учебной временной шкале команда прерывания подаётся на 123-й мс, задержка управления составляет 5 мс, а устройство проверяет и выполняет управляющие команды каждые 10 мс. Поэтому звук отключается на 130-й мс, и воспринимаемая пользователем задержка прерывания равна 7 мс. Отключение звука устройством, прекращение генерации на бэкенде и освобождение буфера и состояния соответствуют трём отдельным моментам: остановке воспроизведения, остановке вычислений и освобождению хранилища. Прекращение генерации на бэкенде определяет, когда завершатся последующие вычисления, а освобождение буфера — когда его ёмкость

²⁷ Учебная временная диаграмма с буфером 40 ms и с буфером 60 ms.

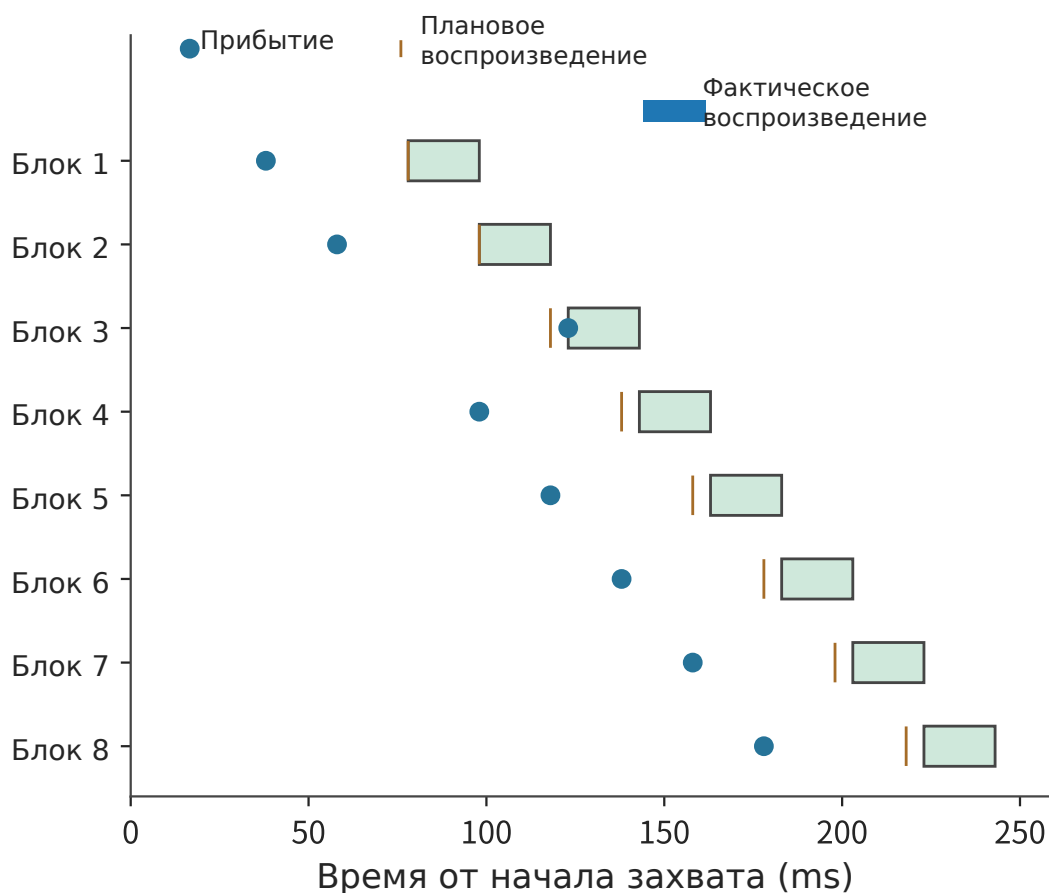


Рис. 3-19. Учебная временная шкала воспроизведения восьми аудиоблоков. Длительность каждого блока — 20 мс, точки обозначают поступление, короткие вертикальные линии — запланированные моменты воспроизведения, а цветные полосы — фактическое воспроизведение; третий блок опаздывает на 5 мс, из-за чего последующее воспроизведение сдвигается.

Прерывание и остановка речи — разные события



Интервал 7 ms; прежнее аудио продолжает звучать

Рис. 3-20. Локальное прерывание в том же учебном сценарии. На 123-й мс подаётся команда, на 130-й мс устройство отключает звук; остановка удалённых вычислений относится к другому управляющему пути.

станет доступна другим задачам.²⁸

Непрерывный вывод требуется не для всех задач. RAW — это представление изображения, сохраняющее исходные данные, полученные сенсором камеры. При высококачественной обработке RAW-изображений важно, когда будет получен полностью готовый результат, но непрерывный вывод, как для речи, не требуется. Такая обработка также требует сохранения исходной информации изображения, поэтому ЕС, используемый для визуального понимания, не может заменить оригинал. Следовательно, разные задачи предъявляют разные временные требования: в одних результат должен появиться как можно раньше, в других необходима непрерывная плавная выдача, а в третьих важно как можно скорее завершить обработку целиком.

Упражнение 3-5 (расширенное): как разрешение изображения и аудио-буфер влияют на потребность в ресурсах

Измените разрешение входного изображения Qwen3-VL-4B с 640×640 на 1280×1280 , оставив неизменными размер фрагмента, способ объединения и точность признаков. Найдите количество визуальных токенов, размер полного кэша кодировщика ЕС и размер KV-кэша, соответствующего этим визуальным токенам. Затем на основе временной шкалы аудио из этого раздела сравните момент первого воспроизведения и наличие пауз при начальном буфере 40 мс и 60 мс. Если каждую секунду поступает объём аудио, достаточный лишь для 0,9 секунды воспроизведения, а к началу воспроизведения накоплено 0,3 секунды аудио, определите, через какое время непрерывного воспроизведения буфер опустеет, и объясните, какую проблему позволяет решить увеличение конечного буфера.

3.4 Требования обучения к вычислениям и состоянию

3.4.1 Прямые вычисления, обратное распространение и обновление параметров

При инференсе веса используются для получения выходных данных, а при обучении веса также корректируются на основе расхождения между выходными данными и целевыми значениями. Рассмотрим простейшее умножение: входное значение x умножается на вес w , в результате чего получается y . После того как следующий слой передаст чувствительность функции потерь к y , для вычисления чувствительности функции потерь к w потребуется исходное значение x , а для передачи чувствительности на предыдущий шаг — значение w . Поэтому после передачи результата следующему слою некоторые входные данные необходимо сохранять до тех пор, пока они не будут использованы при обратном распространении.

Для линейного слоя $Y = XW$ вход X имеет форму $[m, k]$, а веса W — $[k, n]$. Функция потерь $\mathcal{L}$ — это числовая мера различия между выходом модели и целью обучения; задача обучения состоит в уменьшении потерь. Градиент описывает чувствительность потерь к небольшому изменению некоторого входа или параметра. При обратном распространении эти чувствительности передаются от выхода в обратном направлении по зависимостям. Пусть следующий слой передаёт градиент $\partial \mathcal{L} / \partial Y$. Текущий слой должен вычислить два вида градиентов:

²⁸Учебный пример расчёта для прерывания и отключения звука.

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} W^T, \quad \frac{\partial \mathcal{L}}{\partial W} = X^T \frac{\partial \mathcal{L}}{\partial Y}.$$

При прямом проходе вычисляется выход, а при обратном — отдельно градиенты входа и весов. Объём вычислений для всех трёх матричных умножений одинаков, поэтому объём матричных вычислений при обучении этого линейного слоя примерно втрое больше, чем при прямом проходе. В следующей таблице одна операция умножения со сложением считается за два FLOPs.

Операция	Решаемая задача	Формы левого и правого входов	Форма выхода	Матричные FLOPs
Прямой проход $Y = XW$	Получить выход по входу	$[m, k]$ и $[k, n]$	$[m, n]$	$2mkn$
Градиент входа $dX = dY W^T$	Передать ошибку предыдущему слою	$[m, n]$ и $[n, k]$	$[m, k]$	$2mkn$
Градиент весов $dW = X^T dY$	Определить на- правление кор- ректировки ве- сов слоя	$[k, m]$ и $[m, n]$	$[k, n]$	$2mkn$

Градиент входа передаёт сигнал ошибки предыдущему слою, а градиент весов определяет корректировку параметров текущего слоя. Рассмотрим размеры трёх групп матриц: хотя порядок умножения различается, каждая из трёх операций содержит одинаковое число mkn умножений со сложением, поэтому суммарно получается около $6mkn$ FLOPs. Если все обучающие токены проходят через один и тот же набор основных матриц параметров, суммирование даёт распространённую оценку $F_{\text{train}} \approx 6ND$, где N — число параметров, а D — общее число обучающих токенов.²⁹

Изменения архитектуры также можно анализировать с помощью этого вывода. После замораживания матрицы весов вычисление её градиента можно исключить; если градиенты всё ещё нужны более ранним слоям, градиент входа необходимо вычислять по-прежнему. В МоЕ оба вида градиентов суммируются по выбранным экспертам, а в механизме внимания — по парам «запрос — ключ». Последовательно суммируя операции вдоль обратных зависимостей, можно получить объём вычислений, необходимый каждому методу обучения.

Помимо вычислений, обучение требует хранения новых состояний. Кроме X , необходимого для вычисления dW , для получения градиентов нелинейных операций нужны соответствующие промежуточные результаты. Градиенты параметров создаются при обратном распространении, после чего оптимизатор использует их для вычисления обновлений параметров. Состояние оптимизатора хранит информацию, накопленную за несколько обновлений, поэтому его необходимо сохранять до следующего обновления. Срок хранения каждого вида данных определяется последней использующей его операцией: активации слоя обычно освобождаются после завершения обратных вычислений этого слоя, а импульс оптимизатора необходимо сохранять между множеством обновлений.

Пример 3-3. Почему градиенты и состояние оптимизатора многократно увеличивают требования обучения к памяти?

²⁹Заметки об объёме вычислений при обучении, прямой и обратный проходы линейного слоя, распределение последовательностей и отдельный расчёт экспертов DeepSeek V4-Flash.

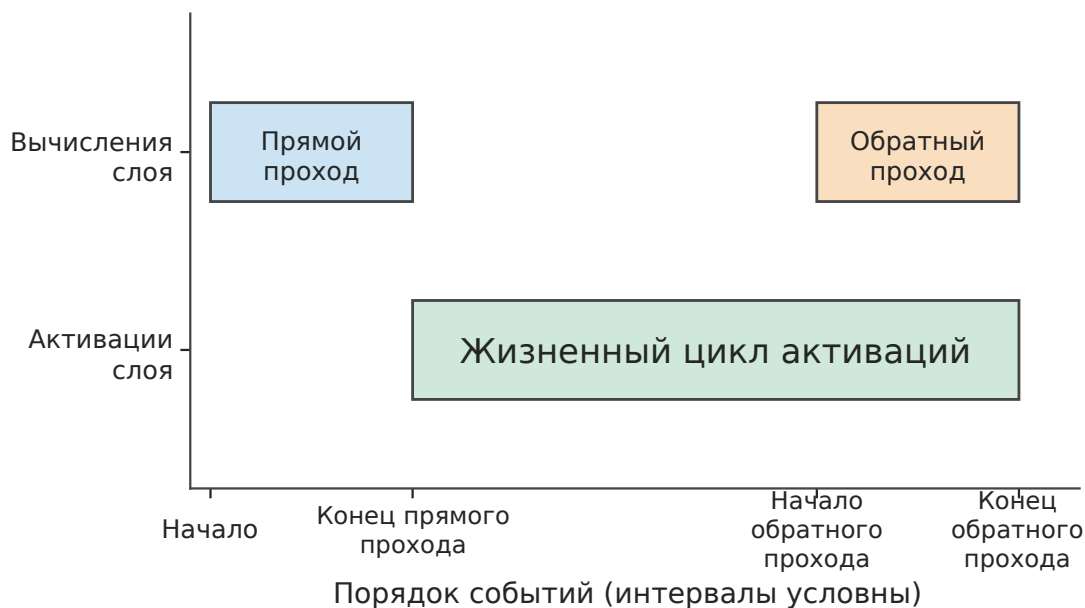


Рисунок 3-21. Жизненный цикл активаций: активации слоя сохраняются после завершения прямого прохода до окончания соответствующего обратного прохода. По горизонтальной оси расположены события, а расстояния обозначают лишь порядок процесса; после использования этих активаций соответствующими обратными вычислениями занимаемое ими пространство можно освободить.

Решение: рассмотрим 8 190 735 360 параметров Qwen3-8B. Необходимо хранить веса BF16, градиенты FP32, отдельную копию основных весов FP32 (master weight), а также две оценки моментов FP32 для Adam. Adam — это оптимизатор, регулирующий величину обновления на основе скользящих средних градиента и квадрата градиента; две оценки моментов хранят соответственно эти два средних значения. Без шардирования, то есть разделения этих состояний для хранения на нескольких картах, на каждый параметр требуется $2 + 4 + 4 + 4 + 4 = 18$ байт, всего 147,433 GB, или около 137,308 GiB. Даже без учёта активаций эти состояния уже превышают заявленный объём видеопамати одной H100 SXM (80 GB), RTX PRO 6000 Blackwell Workstation Edition (96 GB) и даже H200 (141 GB). Веса BF16 используются при прямых и обратных вычислениях, градиенты — для вычисления обновлений, основные веса FP32 — для высокоточного обновления, а две оценки моментов Adam накапливают статистику градиентов и их квадратов. В совокупности эти данные занимают в девять раз больше памяти, чем веса BF16 при инференсе.³⁰

После определения требований к состоянию можно проанализировать, как данные поступают на обучение пакетами. micro-batch (микробатч) — это подмножество данных, фактически обрабатываемое за один прямой и обратный проход; полный обучающий batch может состоять из нескольких micro-batch. Следует также различать micro-batch и обновление параметров. Если видеопамать не вмещает полный обучающий batch, градиенты можно накапливать в течение нескольких micro-batch, а затем один раз обновить параметры. Если цель, эффективные метки и нормализация остаются неизменными, разбиение одного обучающего batch на большее число micro-batch изменяет порядок выполнения и жизненный цикл активаций, но не общий объём обучающих данных.

³⁰Матрица обучения Qwen3-8B с контекстом 8K.

Сначала вычисляется вывод, затем градиент идёт назад по зависимостям

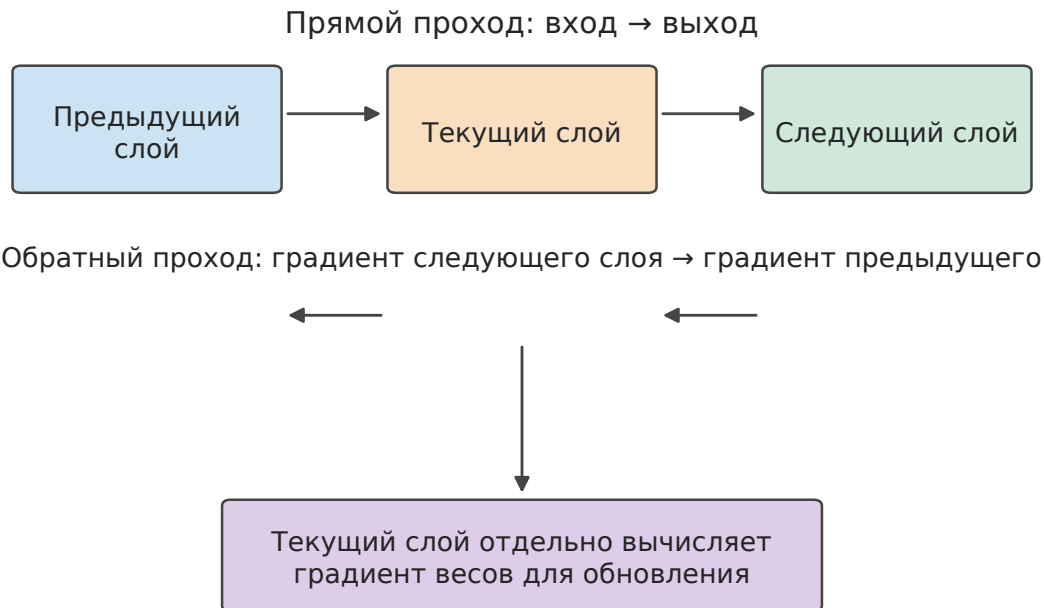


Рисунок 3-22. При обучении выход вычисляется по зависимостям прямого прохода, а затем градиенты передаются по обратным зависимостям. Текущий слой передаёт предыдущему слою градиент входа и одновременно вычисляет собственный градиент весов для обновления оптимизатором.

В главе 10 на этой основе будут рассмотрены шардирование и конвейерное выполнение.

Во время обучения также необходимо непрерывно подготавливать следующий пакет входных данных. CPU считывает, декодирует и токенизирует данные, формирует из них batch, а затем отправляет тензоры на ускоритель. Двойная буферизация позволяет во время вычислений для текущего batch подготавливать следующий: один буфер используется для чтения при вычислениях, а другой принимает новые данные, после чего они меняются ролями. Если вычисления завершились, а новые данные ещё не подготовлены, выполнение вынуждено ждать входных данных; при получении результатов с ускорителя оно также ждёт завершения соответствующих вычислений. В главе 5 на основе порядка выполнения прямого прохода, обратного распространения и обновления параметров будет рассмотрено, как подготовка данных совмещается с вычислениями.

3.4.2 Предобучение, промежуточное обучение и SFT

Независимо от названия этапа обучения система должна отвечать на одни и те же вопросы: сколько токенов поступает на вход, какие токены участвуют в вычислении потерь и через сколько micro-batch выполняется обновление параметров. При предобучении обычно выполняется предсказание следующего токена в непрерывном тексте; при промежуточном обучении корректируется состав данных или длина контекста; при дообучении с учителем (SFT) модель продолжает обучаться на заданных входах и целевых ответах, причём инструкции и ответы часто подаются вместе, а потери вычисляются толь-

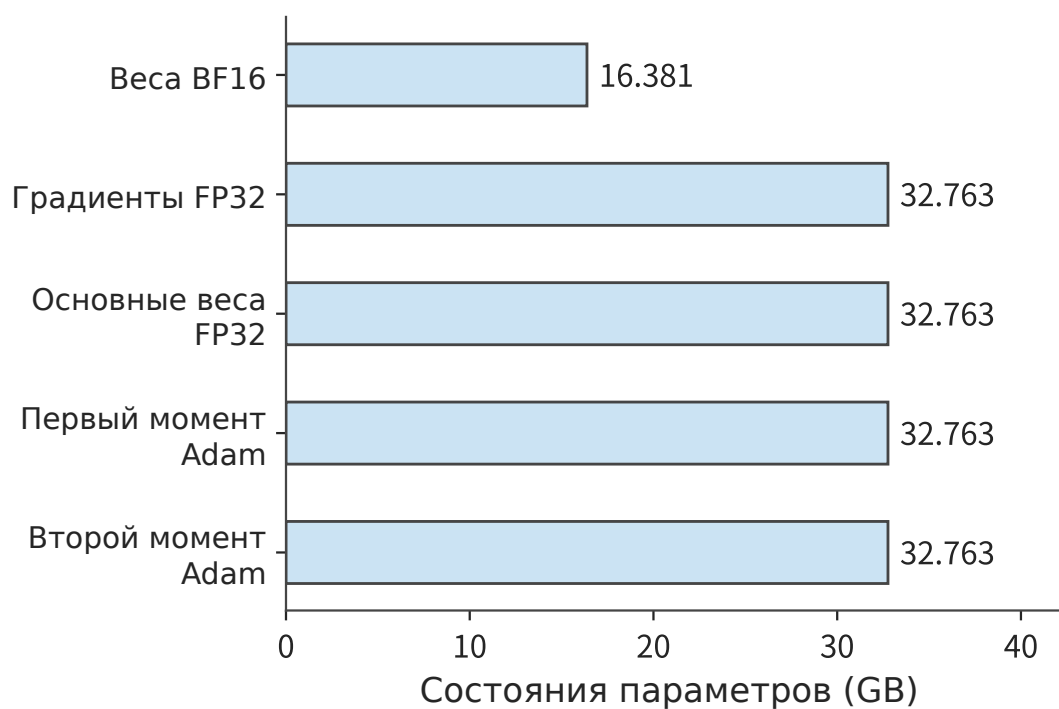


Рисунок 3-23. Связанные с параметрами состояния при полнопараметрическом обучении Qwen3-8B. На каждый параметр приходится 2 байта вычислительных весов и четыре состояния по 4 байта, всего 18 байт; требования к объёму памяти для активаций и рабочих областей рассчитываются отдельно с учётом их жизненных циклов.

ко для ответов. При оценке ресурсов эти различия необходимо выразить через форму входа, число токенов, участвующих в обучении с учителем, и частоту обновлений.

Процесс обучения Qwen3 демонстрирует эти изменения: на общем этапе модель обучалась на последовательностях длиной 4096 и более чем 30 трлн токенов, затем в данные добавили материалы по естественным наукам, технологиям, инженерии и математике (STEM), код, рассуждения и синтетические данные и обучили модель ещё примерно на 5 трлн токенов; на этапе длинного контекста длина была увеличена до 32768.³¹ Состав данных изменяет изучаемое содержимое, а длина последовательности непосредственно определяет объём предшествующего контекста, доступного каждому токеноу, поэтому эти факторы необходимо рассматривать отдельно.

Системные требования меняются вместе с длиной. Предположим, что две группы независимых документов содержат по 8192 токена: первая — $4096 + 4096$, вторая — $7168 + 1024$. Число строк при проекциях параметров одинаково, но число эффективных пар «запрос — ключ» в причинном внимании составляет соответственно 16 781 312 и 26 218 496. Для одного слоя Qwen3-8B объём прямых вычислений QK^T и AV возрастает примерно с 0,275 до 0,430 TFLOPs, то есть приблизительно на 56%. Токенам в более длинных документах доступно больше предшествующего контекста, поэтому при неизменном общем числе строк объём взаимодействий в контексте увеличивается; проекции и FFN по-прежнему обрабатывают те же 8192 представления токенов. Увеличение объёма вычислений всего слоя зависит от доли этих дополнительных взаимодействий в исходном объёме работы.³²

При SFT число токенов, считываемых моделью, и число токенов, участвующих в вычислении потерь, часто различаются. Один пример может содержать системный промпт, вопрос пользователя, результаты инструментов и ответ, но потери при обучении вычисляются только для части с ответом. Остальные токены всё равно проходят через основную сеть, предоставляя контекст для ответа. Реализация также может вычислять проекцию на словарь только для токенов, участвующих в обучении с учителем; если токены, не участвующие в обучении с учителем, маскируются лишь при вычислении потерь, но их проекция на словарь не пропускается, соответствующее матричное умножение всё равно выполняется.

Помимо вычисления потерь лишь для части токенов, при дообучении можно обучать только часть параметров; тогда число параметров, для которых требуется хранить градиенты и состояние оптимизатора, также уменьшается. Веса замороженных модулей остаются неизменными, но градиенты всё равно должны проходить через них к более ранним обучаемым модулям, поэтому соответствующие градиенты входа необходимо вычислять. Последовательно определяя для каждого слоя обратного графа, требуется ли вычислять градиент входа и градиент весов, можно по отдельности рассчитать состояние обучения и объём вычислений.

Число micro-batch и обновлений определяет количество повторений работы. Пусть каждый micro-batch содержит B_μ последовательностей длиной P , а одно обновление выполняется после накопления a micro-batch. Тогда при каждом обновлении считывается $aB_\mu P$ входных токенов. Если в каждой последовательности в обучении с учителем участвует только P_{label} токенов, то число эффективных токенов, используемых при нормализации потерь, равно $aB_\mu P_{\text{label}}$. Увеличение числа шагов накопления позволя-

³¹Технический отчёт Qwen3, трёхэтапное предобучение из §3.2 и ветви постобучения из таблицы 21.

³²Заметки об объёме вычислений при обучении, прямой и обратный проходы линейного слоя, распределение последовательностей и отдельный расчёт экспертов DeepSeek V4-Flash.

ет увеличить batch одного обновления, разделив его прямые и обратные вычисления на несколько проходов; состояние параметров сохраняется постоянно, а активации каждого micro-batch можно освободить после завершения обратного прохода.

3.4.3 От оценки $6ND$ к поэлементному расчёту затрат на обучение

После определения объёма обучающих данных и числа эффективных токенов можно рассчитать общий объём вычислений. Сначала можно выполнить грубую оценку на основе соотношения «обучение линейного слоя примерно вдвое дороже прямого прохода», а затем поэлементно добавить внимание, словарную голову и обновление состояния, чтобы увидеть расхождение между грубой оценкой и полной моделью.

Рассмотрим один полный прямой и обратный проход при обучении: Qwen3-8B, $B = 1$, $P = 8192$, полнопараметрическое обучение, словарная голова выполняется для всех входных токенов, повторных вычислений нет (то есть при обратном проходе не повторяется часть прямых вычислений ради сокращения объёма сохраняемых активаций). Поэлементное суммирование проекций, FFN, QK^T , AV и выходной головы по всем слоям даёт следующий объём матричных вычислений.³³

Матричные вычисления	TFLOPs
Матрицы прямого прохода	143,789
Матрицы обратного прохода	287,579
Прямой и обратный проходы	431,368
В том числе прямые и обратные вычисления причинного внимания QK^T и AV	59,381
Оценка $6ND$ по общему числу параметров	402,591

Чтобы скорректировать грубую оценку 402,591 TFLOPs до 431,368 TFLOPs, сначала необходимо определить, какие операции были пропущены, а какие учтены избыточно. Матричные умножения между запросами и контекстом не соответствуют отдельному набору параметров модели, однако при прямом и обратном проходах дают 59,381 TFLOPs. С другой стороны, такие модули, как встраивания токенов, хотя и содержат параметры, не выполняют полное матричное умножение для каждого токена, поэтому их нельзя безусловно учитывать по формуле $6ND$. После взаимной компенсации двух поправок общий объём матричных вычислений оказывается примерно на 7,15% выше грубой оценки. Это первый тип расхождений, описанный в разделе 1.3.4: грубая оценка пропускает квадратичный член внимания и одновременно избыточно учитывает параметры, не участвующие в полном матричном умножении. Корректировать здесь нужно саму модель оценки, а не реализацию. Только скорректированную нижнюю границу можно использовать для оценки эффективности системы.

Если уменьшить число эффективных меток до 4096, но по-прежнему вычислять проекцию на словарь для всех токенов, объём матричных вычислений не изменится. Если сначала выбрать скрытые векторы токенов, участвующих в обучении с учителем, а затем выполнить проекцию на словарь только для них, общий объём снизится до 416,074 TFLOPs. Основная сеть по-прежнему обрабатывает весь вход, а экономия достигается за счёт исключения проекции на словарь для токенов, не участвующих в обучении с учителем; при обратном проходе градиенты затем возвращаются в соответствующие пози-

³³Матрица обучения Qwen3-8B с контекстом 8K.

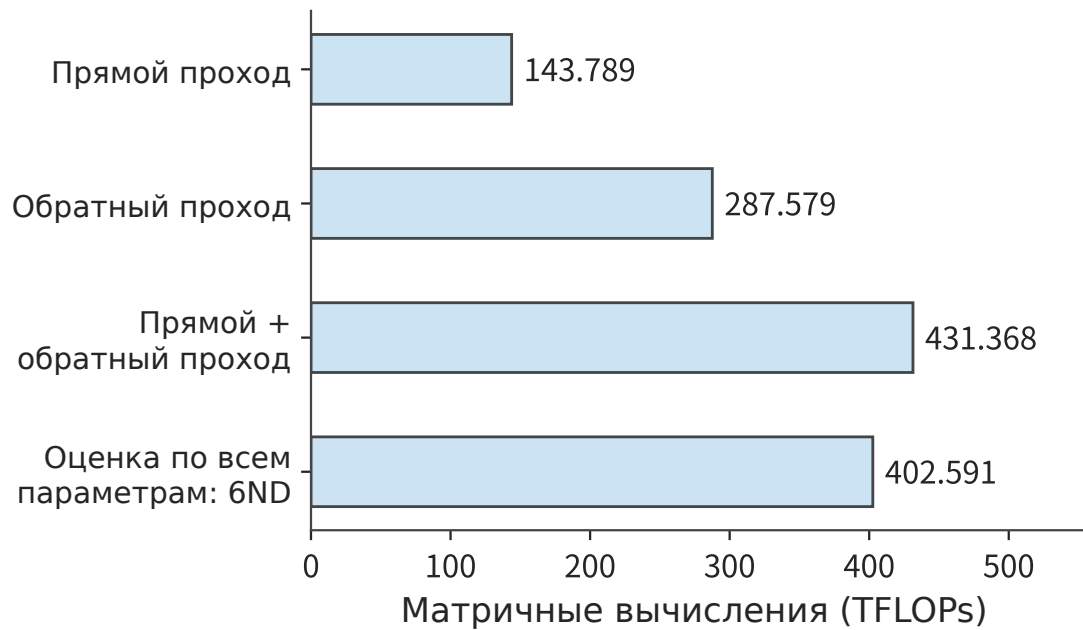


Рисунок 3-24. Полнопараметрическое обучение Qwen3-8B на входе из 8192 токенов. Словарная голова выполняется для всех входных токенов, повторных вычислений нет; поэлементное суммирование матричных операций даёт 431,368 TFLOPs для прямого и обратного проходов.

ции.³⁴

Нормализация, активации, функция потерь и оптимизатор также требуют вычислений, чтения и записи. Например, при обновлении параметров даже для короткой последовательности всё равно необходимо читать и записывать веса, градиенты и состояние оптимизатора; чем короче последовательность, тем выше обычно доля этих затрат в общем объёме. Поэтому при расчёте бюджета обучения необходимо учитывать как матричные вычисления, растущие вместе с числом токенов, так и затраты на обновление, определяемые числом параметров.³⁵

Новые операции прямого прохода модели также добавляют соответствующие обратные вычисления. При сжатии контекста необходимо вычислять градиенты для проекций, пулинга и обновления состояния, в mHC — распространять градиенты по всем остальным путям, а в MoE — вычислять градиенты для фактически выбранных экспертов.³⁶ Поэтому вычислительный граф из главы 2 также позволяет определить, какие промежуточные результаты требуется сохранять при обучении и какие операции необходимо выполнять при обратном проходе.

Таким образом, при изменении длины последовательности сначала можно определить тенденцию изменения каждой статьи затрат: проекции параметров растут с числом входных строк, внимание — с числом пар «запрос — ключ», а обновления оптимизатора — с числом параметров и обновлений. При использовании повторных вычислений требования к объёму памяти снижаются, но повторно выполненные операции прямого прохода необходимо добавить обратно к F_{train} . Все эти изменения можно поэлементно

³⁴ Сокращение числа меток вдвое и явное сжатие выходного слоя словаря.

³⁵ Дополнительный расчёт нематричных операций при обучении и отдельный расчёт для варианта 8K.

³⁶ Онлайн-компрессор DeepSeek V4-Flash, обратный проход механизма внимания, обучение одного слоя MoE, обратный проход оболочки mHC, группировка оптимизатора.

объяснить с помощью одного и того же графа прямого и обратного проходов.

Упражнение 3-6 (расширенное). Сколько вычислений, состояния обучения и токенов с разметкой требуется для одного обновления параметров

Выведите из $Y = XW$ размеры градиента входа и градиента весов и объясните, почему объём трёх матричных умножений одинаков. Для Qwen3-8B рассчитайте общий объём долговременного состояния обучения из расчёта 18 байт на параметр, затем замените градиенты на BF16 и определите сэкономленный объём памяти. Пусть каждый micro-batch содержит 2 последовательности, каждая из которых включает 4096 входных токенов, причём 1024 токена участвуют в обучении с учителем. Одно обновление параметров выполняется после накопления 8 micro-batch. Определите общее число входных токенов и эффективных токенов с разметкой, обрабатываемых при каждом обновлении. Объясните разницу в объёме вычислений между простым маскированием потерь для токенов, не участвующих в обучении с учителем, и выполнением проекции на словарь только для токенов, участвующих в обучении с учителем.

3.5 Обучение с подкреплением (RL)

3.5.1 Rollout, вознаграждение и проверка среды

Инференс начинается с внешнего запроса, а предварительное обучение — с уже имеющегося текста. Обучение с подкреплением (RL) использует обратную связь, полученную в результате действий, для улучшения последующих действий. В языковой модели стратегия представляет собой распределение вероятностей выбора следующего токена при заданном контексте. RL соединяет два процесса: текущая стратегия сначала генерирует ответ, затем обратная связь определяет, как обновить параметры, после чего обновлённая стратегия создаёт следующую порцию данных. Процесс генерации обычно называют rollout.

Этот подход к обучению уже применялся на практике публично: DeepSeek-R1 продемонстрировал метод, основанный на задачах на рассуждение и проверяемых вознаграждениях. Для задач, результаты которых легко проверить, например задач по программированию и математике, обратная связь может поступать от правил или тестов; для задач, требующих реальных действий, необходимо запускать инструменты и среду. Вознаграждение — это числовая оценка результата ответа или действия. Если вознаграждение генерирует другая модель, это также требует дополнительного инференса. Поэтому одна «итерация обучения» может включать несколько моделей, тесты на CPU, ожидание песочницы и обновление градиентов.³⁷

В более новых моделях RL сочетается с другими этапами обучения. В DeepSeek V4-Flash сначала посредством SFT и RL обучаются эксперты в отдельных областях, а затем способности нескольких экспертов объединяются в одной модели с помощью дистилляции нескольких учителей на основе текущей стратегии (on-policy distillation, OPD). При дистилляции модель-ученик обучается на выходной информации, предоставляемой моделями-учителями; при OPD текущий ученик сам генерирует траектории, а учи-

³⁷ Отчёт DeepSeek-R1.

теля предоставляют распределения вероятностей возможных выходов в соответствующих позициях токенов, на которых обучается ученик. Поэтому прямой проход учителя становится отдельной модельной операцией между генерацией и обновлением; проверка по правилам предоставляет обратную связь посредством тестов или выполнения программ и требует отдельного набора вычислительных ресурсов.³⁸

При анализе ресурсов в этом цикле сначала следует определить производителей и потребителей данных: стратегия генерирует токены, функция вознаграждения или учитель читает ответы, обучающий модуль читает обучающие примеры и создаёт новые веса, а отвечающие за генерацию экземпляры модели затем используют эти новые веса. Если одна из ролей замедляется, последующие роли вынуждены ждать; добавление моделей-учителей требует дополнительных прямых проходов и места для хранения весов. Все последующие схемы распараллеливания должны строиться на этой диаграмме зависимостей.

3.5.2 Эффективные примеры, обновление стратегии и синхронизация весов

После генерации ответов и вычисления вознаграждений необходимо решить, какие примеры использовать для обучения. Важно различать уже сгенерированные примеры и примеры, которые в итоге попадут в обновление: первые определяют затраты на генерацию, вторые — обучающий batch, и их количество необязательно совпадает.

Пусть за один раунд необходимо сохранить n_{keep} примеров, а средняя доля сохраняемых сгенерированных примеров равна a . Тогда ожидаемое требуемое количество сгенерированных примеров составляет

$$n_{\text{generate}} \approx \frac{n_{\text{keep}}}{a}.$$

Если нужно сохранить 16 примеров, то при доле в одну вторую в среднем потребуется сгенерировать 32 примера, а при снижении доли до одной четверти — 64. Генерация и оценка обрабатывают все ответы, тогда как обновление параметров — только сохранённые примеры; снижение доли сохраняемых примеров увеличивает объём работы на первых двух этапах, но не приводит к пропорциональному увеличению batch обновления. В разделе 3.5.3 будут использоваться примеры одинаковой длины и заданные количества генерируемых примеров для сравнения этих двух случаев.

Решение о сохранении примера зависит от метода обучения и правил фильтрации. Некоторые методы используют неудачные ответы как отрицательную обратную связь, другие исключают обрезанные ответы или ответы с недействительным вознаграждением. Преимущество показывает, насколько некоторый ответ лучше или хуже принятой базы сравнения. При использовании относительного преимущества внутри группы необходимо также сравнивать вознаграждения нескольких ответов на одну и ту же задачу. Таким образом, количество сохранённых примеров определяет объём данных, обрабатываемых при обновлении, а вознаграждение и функция потерь — то, какие градиенты будут получены из этих данных.

Пример: как одинаковые вознаграждения устраняют обучающий сигнал внутри группы. При обновлении стратегии с использованием относительных вознаграждений внутри группы, если ответы одной группы получают одинаковые вознагражде-

³⁸Отчёт DeepSeek V4, предметные эксперты и OPD с несколькими учителями из §5.1, планирование учителей, rollout и песочница из §5.2.

ния, то после вычитания среднего вознаграждения группы преимущество каждого ответа становится нулевым, как и соответствующий градиент стратегии. Такая ситуация может возникнуть, если все ответы правильны. Модель выполняет весь процесс генерации, оценки и обновления, но не получает от этой группы ответов сигнала, позволяющего различить лучшие и худшие варианты. Только различия в вознаграждениях дают ненулевое преимущество, благодаря которому функция потерь стратегии направляет изменение параметров в соответствующую сторону.³⁹

На обучение также влияет способ усреднения функции потерь. Если целевая функция усредняется по всем эффективным токенам batch, то при накоплении градиентов отдельных micro-batch в качестве знаменателя необходимо использовать общее количество эффективных токенов во всём batch. Если сначала отдельно вычислить среднее для каждого micro-batch, а затем усреднить полученные значения, токены в micro-batch с меньшим количеством эффективных токенов могут получить больший вес. В главе 10 это различие будет рассмотрено на примере конкретных методов обучения.⁴⁰

После обновления веса стратегии необходимо передать на сторону генерации. Если генерация продолжит использовать старую версию, следующая порция данных по-прежнему будет создана старой стратегией. Синхронный цикл может ждать готовности весов перед началом генерации; асинхронная схема позволяет частично перекрывать этапы, но требует учитывать версии и устаревание примеров. В этой главе сначала фиксируется данная зависимость, а в главе 10 будут рассмотрены передача управления, вытеснение, восстановление и распределение ресурсов.

3.5.3 Объём вычислений на каждом этапе при фиксированном количестве эффективных примеров

Это различие можно количественно оценить с помощью фиксированной цели: в каждом раунде для обновления необходимо сохранять 16 примеров. Если ответам сложнее пройти фильтрацию, сначала потребуется сгенерировать больше ответов.

Пример 3-4: насколько требуется увеличить объём вычислений при снижении доли сохраняемых примеров, чтобы сохранить прежнее количество примеров для обновления?

Решение: рассмотрим Qwen3-8B. Пусть дано восемь задач, для каждой генерируется по четыре ответа, $P = 1024$, $G = 256$, а для одного обновления стратегии сохраняется в общей сложности 16 примеров. Эталонная модель (reference model — модель с фиксированными весами, используемая для ограничения степени отклонения стратегии) имеет ту же архитектуру, что и модель стратегии; веса обеих моделей хранятся отдельно. Эталонная модель выполняет по одному прямому проходу для каждого из 32 ответов; модель-учитель в этом примере не вызывается. Все примеры имеют одинаковую длину, генерация не завершается досрочно, а префиксы не используются совместно.⁴¹

При генерации для каждого ответа сначала однократно выполняется prefill, а затем 255 шагов decode. Во время обучения и оценки траектория уже определена, поэтому прямой проход можно выполнять в режиме teacher forcing: на каждом шаге предсказания

³⁹Эксперимент 10-8: фиксированный минимальный процесс обучения verl. В этой главе эксперимент используется для разграничения качества задачи и обновления параметров; организацию полной системы см. в главе 10.

⁴⁰Нормализация функции потерь и процесс обновления параметров в фиксированной конфигурации обучения verl, привязанные к исходной конфигурации, исходному коду и экспортированным тензорам.

⁴¹Учебный RL batch для Qwen.

используется уже определённый предыдущий токен траектории, а не ожидается его повторная генерация моделью. Поэтому эти известные токены можно вычислять совместно. Длина входа составляет $P + G - 1 = 1279$, а 256 выходных меток располагаются в соответствующих позициях. Для 16 сохранённых примеров общее количество входных токенов за одно обновление составляет 20,464, а контролируемых выходов — 4096. Входные токены определяют объём вычислений контекста, а токены, участвующие в контролируемом обучении, определяют, к каким выходам применяется функция потерь; в бюджет обучения эти величины входят отдельно.

Этап	Обрабатываемые примеры	Объём матричных вычислений (TFLOPs)
Rollout prefill	32	465.143
Rollout decode, 255 шагов на пример	32	129.056
Эталонная модель, один прямой проход	32	634.944
Оценка учителем, задано ноль запусков	0	0
Обновление стратегии, один проход	16	952.416
Итого	Сохранено 16 примеров	2,181.559

Обозначим суммарный объём вычислений для генерации одного ответа и прямого прохода эталонной модели через $f_g + f_r$, а объём вычислений одного фиксированного batch обновления — через F_u . Тогда

$$F_{\text{cycle}}(n) = n(f_g + f_r) + F_u.$$

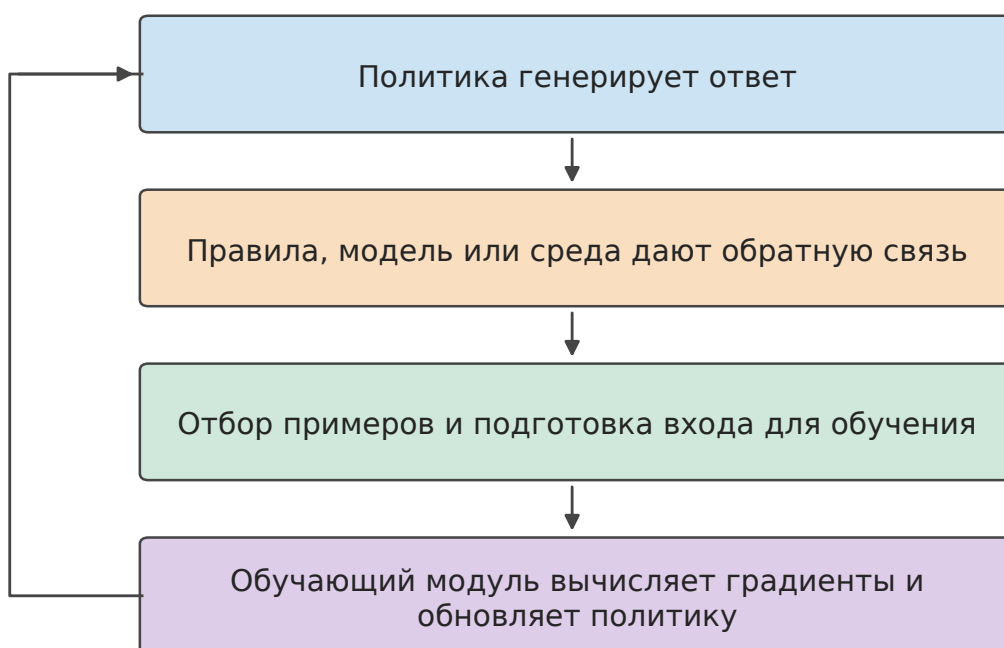
Эталонная модель читает полный ответ и может одновременно обрабатывать несколько известных токенов; генерация же продвигается последовательно, токен за токеном. Хотя в приведённой формуле объёмы вычислений складываются, формы матриц на каждом этапе всё равно необходимо учитывать отдельно, чтобы оценить время с учётом соответствующей эффективности выполнения.

Если доля сохраняемых примеров снижается с 1/2 до 1/4, но по-прежнему требуется получить 16 сохранённых примеров, то в учебном сравнении необходимо сгенерировать уже 64 ответа. Объём вычислений для генерации и эталонной модели удваивается, объём обновления остаётся неизменным, а общий объём матричных вычислений возрастает до 3,410.701 TFLOPs — примерно на 56.3%. Объём матричных вычислений в пересчёте на один сохранённый пример возрастает со 136.347 до 213.169 TFLOPs. Все дополнительные 1229.142 TFLOPs приходятся на генерацию дополнительных ответов и вычисления эталонной модели; количество попадающих в обновление примеров остаётся равным 16.⁴²

После завершения обновления экземпляры для генерации должны получить новые веса. Пусть имеется r экземпляров, каждому из которых независимо передаётся полный набор весов размером M_W . Тогда объём передачи составляет rM_W . Один набор весов Qwen3-8B в формате BF16 занимает примерно 16.381 GB, поэтому для четырёх экземпляров потребуется передать в общей сложности около 65.526 GB. Состояние оптимизатора

⁴²Снижение доли сохраняемых образцов при неизменном числе сохранённых образцов.

Генерация, обратная связь и обучение используют одну партию траекторий



Левый обратный путь: новые веса передаются следующей партии генерации

Рисунок 3-25. Роли и потоки данных в обучении с подкреплением. Сторона генерации создаёт ответы, этап обратной связи оценивает результаты, после фильтрации данные передаются обучающему модулю для обновления; новые веса затем используются при генерации следующей порции данных.

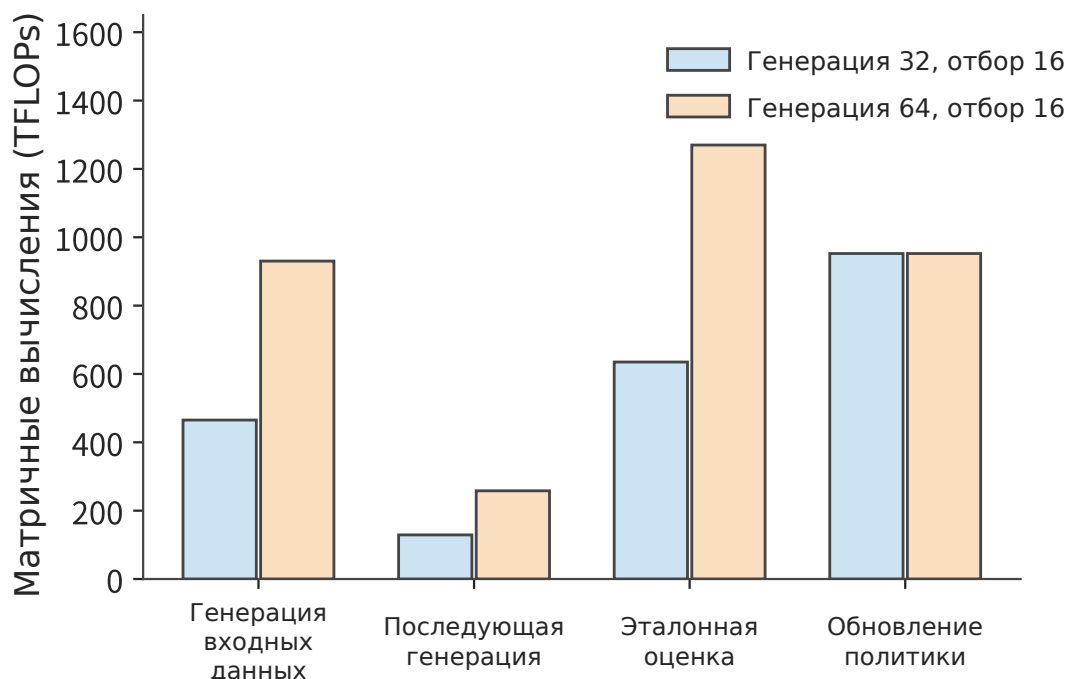


Рисунок 3-26. При сохранении целевого количества в 16 эффективных примеров число генерируемых ответов возрастает с 32 до 64. Обработка входных данных при генерации, последующий decode и оценка эталонной моделью растут вместе с общим количеством ответов, тогда как количество примеров, обрабатываемых при обновлении стратегии, остаётся неизменным.

остаётся в обучающем модуле и используется при следующем обновлении; экземплярам для генерации нужны только веса, необходимые для выполнения прямого прохода. Широковещательная или иерархическая рассылка позволяет нескольким экземплярам совместно использовать часть пути передачи, сокращая повторную отправку данных.

При выделении ресурсов каждому этапу необходимо учитывать как объём вычислений, так и способ их выполнения. На этапе генерации каждый шаг decode обычно обрабатывает только один новый токен каждого запроса, поэтому при небольшом batch одновременно обрабатывается мало токенов; при оценке и обучении можно совместно обрабатывать несколько токенов уже известных ответов, а для проверки инструментами дополнительно требуются CPU и среда выполнения. Полный цикл можно получить, только отдельно оценив длительность каждого этапа и расположив их в порядке генерации, обратной связи, обновления и синхронизации весов. В главах 4–10 будут последовательно рассмотрены соответствующие аппаратные средства и методы планирования.

Упражнение 3-7 (основное): насколько увеличиваются затраты на RL при генерации дополнительных ответов, если количество примеров для обновления не меняется

Одно обновление по-прежнему использует 16 примеров одинаковой длины. Используя общий объём вычислений цикла из примера 3-4 для случаев генерации 32 и 64 ответов, найдите дополнительный объём вычислений для генерации и эталонной модели в расчёте на каждый дополнительный ответ, а также фиксированный объём вычислений, необходимый для одного обновления. Не изменяя количество примеров для обновления, на основе этих данных

спрогнозируйте общий объём вычислений цикла при генерации 48 ответов. Затем добавьте модель-учителя с той же архитектурой, что и у генерирующей модели, и выполните для каждого из 48 ответов по одному прямому проходу; найдите дополнительный объём вычислений. Если обновлённые веса BF16 передаются четырём экземплярам и каждый экземпляр независимо получает полную копию, найдите общий объём передачи. Наконец, изобразите зависимости между генерацией, обратной связью, обновлением и синхронизацией и укажите, какие этапы могут перекрываться между разными batch.

3.6 От требований нагрузки к бюджету обучения и обслуживания

3.6.1 Scaling Law: взаимосвязь размера модели и объёма данных

Вычисления для одного обновления можно определить по отдельным составляющим, однако количество обучаемых параметров и объём используемых данных необходимо выбирать отдельно. Пусть общий вычислительный бюджет обучения равен C , число параметров — N , а число обучающих токенов — D ; тогда основные вычисления приблизительно удовлетворяют равенству $C = 6ND$. При увеличении N приходится уменьшать D . Чтобы выбрать соотношение между ними, нужно понимать, как параметры и данные по отдельности влияют на качество. Scaling Law (закон масштабирования) экспериментально аппроксимирует взаимосвязь между размером модели, обучающими данными и потерями. Ниже используется степенная модель, в которой слагаемые параметров и данных положительны, а конфигурация обучения и способ оценки потерь зафиксированы:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}.$$

L — потери на валидации, E — асимптотические потери, A/N^α описывает влияние недостаточного размера модели, а B/D^β — влияние недостаточного объёма данных. Здесь A, B, α, β — коэффициенты аппроксимации. Отдельное увеличение числа параметров или объёма данных уменьшает соответствующее слагаемое, но при фиксированном вычислительном бюджете эти два слагаемых изменяются в противоположных направлениях.

Исключим объём данных с помощью $D = C/(6N)$, а затем возьмём производную по N . В оптимальной точке выполняется

$$\alpha AN^{-\alpha-1} = \beta B \left(\frac{6}{C}\right)^\beta N^{\beta-1}.$$

Левая часть — снижение потерь за счёт увеличения числа параметров, правая — рост потерь из-за сокращения объёма данных. Когда они равны, небольшое дальнейшее изменение числа параметров уже не даёт чистого выигрыша. После преобразований получаем:

$$N_{\text{opt}} = \left(\frac{\alpha A}{\beta B}\right)^{1/(\alpha+\beta)} \left(\frac{C}{6}\right)^{\beta/(\alpha+\beta)}.$$

Полученное значение N_{opt} является непрерывным; при практическом проектировании можно выбрать вблизи него подходящие число слоёв и ширину. Это оптимальное решение минимизирует потери на валидации только при фиксированном вычислительном бюджете обучения, не учитывая ни стоимость инференса после развёртывания, ни достаточность доступных данных. Если после запуска модель должна обслуживать большое число запросов, оптимальный выбор может сместиться в сторону меньшей, но более полно обученной конфигурации. В разделе 3.6.2 на основе записей об обучении открытых моделей будет проверено, насколько фактические затраты отличаются от этого прогноза.

Показатели распределения, полученные в разных исследованиях, различаются. В раннем исследовании Kaplan и соавторов было получено приблизительное соотношение $N \propto C^{0.73}$, $D \propto C^{0.27}$; исследование Chinchilla на основе других экспериментов и аппроксимации показало, что размер модели и объём данных должны расти с бюджетом ближе к одинаковым пропорциям. При первом варианте быстрее увеличивается число параметров модели, при втором больше дополнительного бюджета направляется на обучающие данные. Поэтому кривые и показатели их аппроксимации непосредственно влияют на выбор между параметрами и данными.⁴³

Пример 3-5: позволяет ли аппроксимация масштаба обучения предсказать потери более крупных моделей? Выберите из открытых записей об обучении наблюдения для шести небольших моделей и используйте их для аппроксимации, а наблюдения для двух более крупных моделей отложите. Сначала выберите метод аппроксимации, затем сравните прогнозы для отложенных моделей с фактическими наблюдениями.

Решение: datablations — это открытые экспериментальные данные об изменении размера моделей и объёма данных, а C4 — использованный в этих экспериментах текстовый корпус. Возьмём восемь моделей: по шести меньшим аппроксимируем кривую, а две модели с $N \geq 2 \times 10^9$ оставим для проверки. Фактические потери двух крупных моделей составляют приблизительно 2,574 и 2,337, а прогноз аппроксимированной кривой — 2,583 и 2,363. Прогноз немного выше фактических значений, а среднеквадратичная ошибка составляет приблизительно 0,0193 нат/токен. Нат — единица количества информации при использовании натурального логарифма; нат/токен обозначает средние потери на токен. На рис. 3-27 точки аппроксимации и проверки обозначены разными символами.⁴⁴

Проверка на отложенных данных необходима потому, что аппроксимация гарантирует лишь максимальную близость кривой к данным, по которым вычислялись коэффициенты. Только исключив крупные модели из аппроксимации и затем сравнив прогноз с фактическими потерями, можно проверить, способна ли эта зависимость предсказывать результаты для нового масштаба. Здесь две ошибки составляют приблизительно 0.009 и 0.026; если возвести их в квадрат, найти среднее и извлечь квадратный корень, получится среднеквадратичная ошибка.

После прогнозирования потерь на валидации необходимо также определить, способна ли модель выполнять практические задачи. В отчёте Llama 3 сначала прогнозируются потери по объёму вычислений обучения, а затем устанавливается взаимосвязь между

⁴³Kaplan Scaling Laws, §6; Chinchilla, оптимальное по вычислительным затратам распределение и метод аппроксимации.

⁴⁴Аппроксимация по восьми открытым точкам C4, вместе с отчётом сохранены исходные журналы, исключённые элементы и анализ чувствительности четырёх слагаемых.

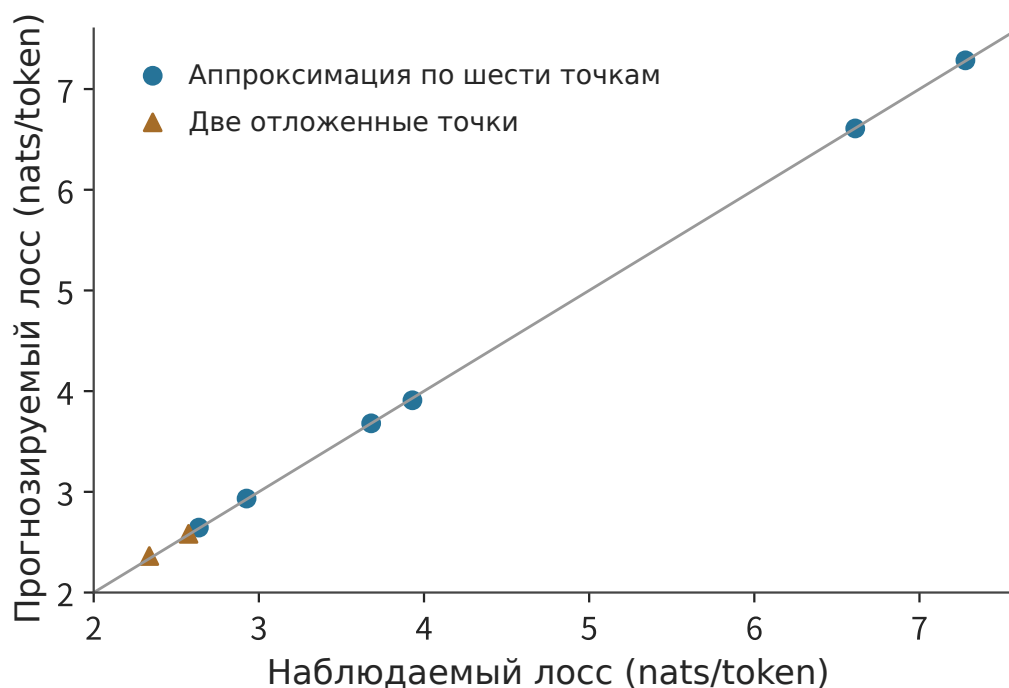


Рис. 3-27. Прогнозируемые и фактические потери для восьми открытых наблюдений С4. Шесть точек использованы для аппроксимации, две заранее отложены для проверки; диагональ соответствует равенству прогноза и наблюдения, а отклонение точки от линии отражает ошибку.

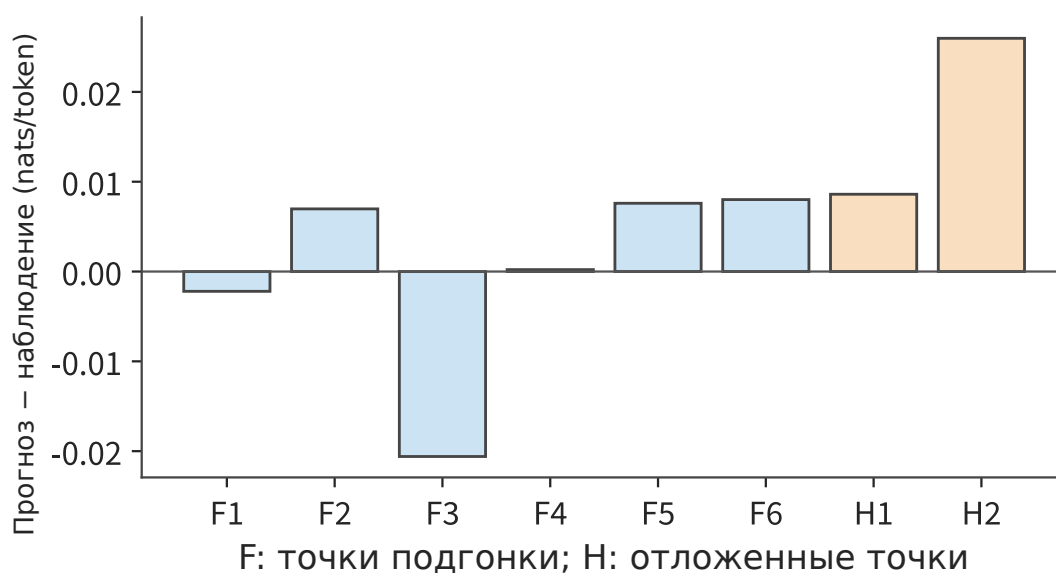


Рис. 3-28. Увеличенное представление ошибок прогноза для того же набора. F1-F6 — точки аппроксимации, H1-H2 — отложенные точки; по вертикальной оси показана разность между прогнозом и наблюдением с сохранением знака.

потерями и результатами решения задач.⁴⁵ Только после этого требования к качеству в задачах программирования, поиска и использования инструментов можно преобразовать в выбор масштаба обучения и объёма вычислений.

И аппроксимация, и прогноз основаны на наблюдениях одного запуска обучения, тогда как само обучение также подвержено колебаниям. При одинаковых данных и размере модели изменение случайной инициализации может привести к другим кривым обучения; более крупная модель, пока она недостаточно обучена, также не обязательно превосходит меньшую. Повторное обучение с несколькими случайными seed позволяет отделить тенденцию, вызванную изменением масштаба, от случайных колебаний отдельного запуска.⁴⁶

При длительном обслуживании стоимость каждого вызова накапливается поверх бюджета обучения. Меньшая модель может достичь целевого качества за счёт более продолжительного обучения: первоначальные затраты будут выше, зато каждый последующий вызов потребует чтения меньшего объёма весов и выполнения меньшего числа операций. Работа Beyond Chinchilla-Optimal включает такую потребность в инференсе в анализ 47 моделей размером от 150М до 6В; модель 150М обучалась максимум на 10 000 токенов на параметр, а более крупные модели — максимум на 1 000.⁴⁷ Бюджет обучения и число обслуживаемых запросов совместно определяют, когда меньшая модель окупит дополнительные затраты на обучение.

Затраты на обучение и совокупная стоимость обслуживания при фиксированных целевых потерях. Зафиксируем целевые потери аппроксимации на уровне 2,9, оценим затраты на обучение и обслуживание по числу матричных операций, а затем приведём их к GPU-времени H100 SXM — произведению числа GPU на время использования, измеряемому в GPU-секундах или GPU-часах. Для запроса примем $P = 512$, $G = 128$; объём вычислений обучения равен $6ND$, а объём вычислений одного полного запроса — $2N[P + (G - 1)]$. Пиковая производительность H100 SXM для плотных вычислений BF16 составляет 989,4 TFLOP/s. В отчёте Llama 3 для предобучения на H100 указана BF16 MFU (определение см. в разделе 1.2.2) в диапазоне 38–43%; для обучения и обслуживания примем 40%, что соответствует 395,76 TFLOP на одну GPU-секунду.⁴⁸ Для модели 0,1В требуется приблизительно 298,6В обучающих токенов, что превышает верхнюю границу объёма данных, использованного для аппроксимации, поэтому на графике затрат эта экстраполированная кривая показана пунктиром. Согласно этой экстраполяции, обучение модели 0,1В требует приблизительно 125,7 GPU-часа H100 — на 73,5 GPU-часа больше, чем для модели 0,5В; зато каждый вызов расходует приблизительно на 0,00129 GPU-секунды меньше. Когда число вызовов достигнет приблизительно 204,8 миллиона, их совокупные затраты сравняются. До этой точки дополнительные затраты на обучение модели 0,1В ещё не окупятся; после неё накопленная экономия на каждом обслуживании превысит первоначальные вложения. Обучение и обслуживание приводятся с одним коэффициентом, поэтому точка пересечения зависит только от объёма

⁴⁵ Отчёт Llama 3, бюджет обучения, прогноз функции потерь и результаты в последующих задачах; §3.3.2 и таблица 4 приводят BF16 MFU на H100 в диапазоне 38%–43%.

⁴⁶ Упражнение 3-8: шесть практических запусков обучения малой модели на фиксированном тексте, сохранены checkpoint и полные результаты для двух значений seed.

⁴⁷ Beyond Chinchilla-Optimal, официальная версия ICML 2024, требования к инференсу, диапазон экспериментов и экстраполяция на длительное обучение.

⁴⁸ Отчёт Llama 3, бюджет обучения, прогноз функции потерь и результаты в последующих задачах; §3.3.2 и таблица 4 приводят BF16 MFU на H100 в диапазоне 38%–43%.

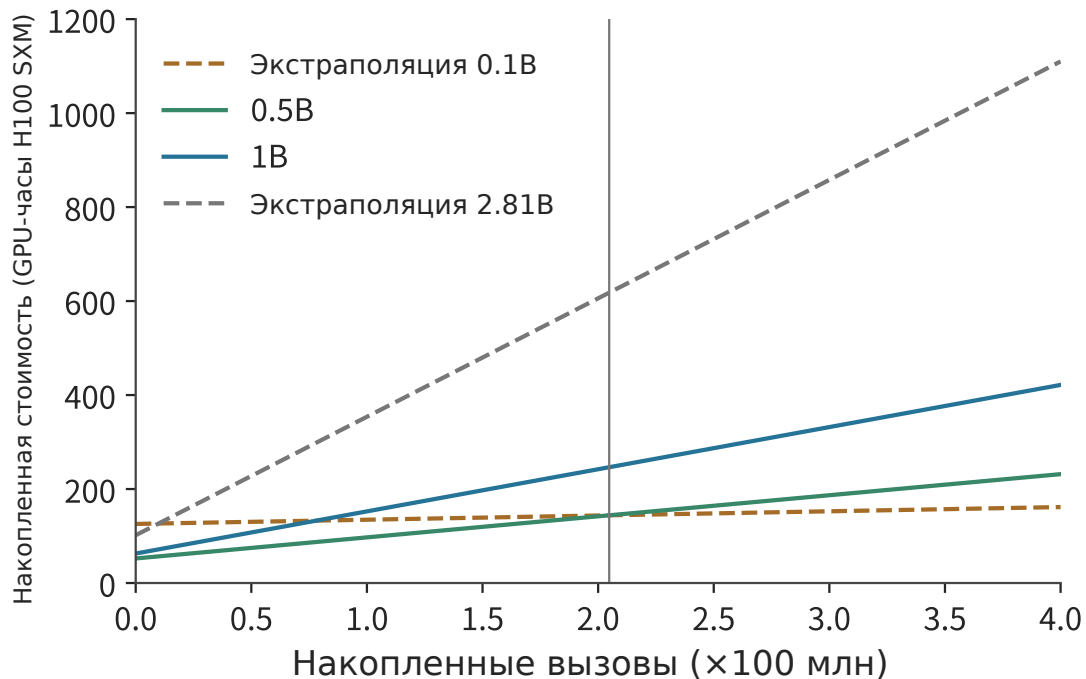


Рис. 3-29. Сравнение совокупных затрат на обучение и обслуживание в условиях задачи. Пересечение с вертикальной осью соответствует затратам на обучение, наклон — стоимости одного вызова; пунктиром показаны варианты, выходящие за диапазон параметров или данных аппроксимации, вертикальной линией — точка равенства затрат приблизительно на 204,8 миллиона вызовов. В легенде обозначает миллиард параметров модели; по вертикальной оси отложены GPU-часы H100 SXM, пересчитанные при MFU 40%.

вычислений, а не от принятого значения MFU.⁴⁹

Вернувшись к распределению параметров и данных, можно непосредственно прогнозировать результат увеличения бюджета. Из выражения для оптимального решения следует $N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}$; подставив его обратно в ограничение вычислений, получаем $D_{\text{opt}} \propto C^{\alpha/(\alpha+\beta)}$. Если оба показателя равны, при четырёхкратном увеличении бюджета число параметров и объём обучающих данных удвоятся. Тем самым смысл показателей аппроксимации становится конкретным: они определяют, как дополнительные вычисления следует распределять между моделью и данными.

Упражнение 3-8 (расширенное): как распределить вычислительный бюджет между параметрами модели и обучающими данными

Используя модель потерь $L = E + A/N^\alpha + B/D^\beta$ и ограничение вычислительного бюджета $C = 6ND$, выведите зависимости оптимальных числа параметров N и объёма данных D от вычислительного бюджета C . Примите $\alpha = \beta$ и определите, во сколько раз должны увеличиться оптимальные число параметров и объём данных при увеличении вычислительного бюджета соответственно в четыре и девять раз. Затем возьмите восемь наблюдений C4 из примера 3-5, используйте шесть для аппроксимации, а два — для проверки, вычислите ошибки прогноза для двух проверочных точек и объясните, почему ошибки на точках аппроксимации не могут заменить ошибки проверки на

⁴⁹ Оценка стоимости жизненного цикла по результатам аппроксимации C4, затраты рассчитаны по секундам работы GPU H100 SXM.

отложенных данных.

3.6.2 Затраты на обучение от Llama до Qwen

Аппроксимация из раздела 3.6.1 показала, как выбирать соотношение между размером модели и объёмом обучающих данных. В этом разделе по открытым записям об обучении моделей анализируются фактические затраты разработчиков: сначала сравнивается число обучающих токенов на параметр, затем алгоритмический объём вычислений отделяется от времени обучения на конкретном оборудовании.

Из опубликованных в отчётах этапов и масштабов видна следующая тенденция: при близком размере в 7–8B параметров число обучающих токенов продолжает расти.⁵⁰

Модель и охват отчёта	Обучающие токены	Приблизительная оценка D/N	Приблизительная оценка $6ND$
Llama 1, 6,7B	1T	149	4.02×10^{22} FLOPs
Llama 2, около 7B	2T	286	8.40×10^{22} FLOPs
Llama 3.1, около 8B	около 15T	около 1 875	около 7.20×10^{23} FLOPs
Qwen2.5, около 7B, данные по серии моделей	около 18T	около 2 571	около 7.56×10^{23} FLOPs
Qwen3, около 8B, данные по серии моделей	около 36T	около 4 500	около 1.728×10^{24} FLOPs

D/N выражает затраты данных как «число обучающих токенов на параметр». Для плотных моделей размером около 7–8B в таблице D/N выросло приблизительно со 149 до 4 500. Это показывает, что при близком числе параметров объёмы обучающих данных могут различаться на порядки. Первоначальная стоимость обучения растёт вместе с объёмом данных, но объём весов при каждом вызове после развёртывания по-прежнему определяется главным образом числом параметров. Поэтому более полное обучение небольшой модели может повысить единовременные затраты, одновременно снизив стоимость длительного обслуживания.

Эти значения также поясняют взаимосвязь между результатами аппроксимации из раздела 3.6.1 и фактическими затратами на обучение. При пропорциональном распределении Chinchilla вычислительно оптимальный объём данных для модели около 8B составляет приблизительно 20 токенов на параметр, то есть около 160B токенов. Фактические объёмы в таблице выросли с 1T примерно до 36T, превышая этот ориентир на один-два порядка. Они отвечают на разные вопросы: аппроксимация минимизирует потери на валидации при фиксированном вычислительном бюджете обучения, тогда как разработчики включают в целевую функцию стоимость обслуживания после запуска и при достаточном объёме данных и большом ожидаемом числе вызовов выбирают меньшую, но более полно обученную модель. Количественные условия такого выбора приведены в работе Beyond Chinchilla-Optimal, упомянутой в разделе 3.6.1, и в формуле точки равенства затрат из раздела 3.6.3. Оптимум бюджета обучения и оптимум полного жизненного цикла — разные цели; фактические затраты сверх первого не означают, что он был рассчитан неверно.

⁵⁰Исторические заметки о затратах на обучение и Scaling Law, а также повторный расчёт по зафиксированным общедоступным полям.

Более продолжительное обучение, фильтрация данных, дистилляция и постобучение позволяют за счёт дополнительных первоначальных работ повысить способность модели заданного размера решать задачи. При оценке базовых моделей в карточке Llama 3 модели Llama 3 8B и Llama 2 70B получили соответственно 66,6 и 69,7 балла в бенчмарке MMLU, охватывающем вопросы с выбором ответа из разных дисциплин, и 78,5 и 87,5 балла в бенчмарке TriviaQA-Wiki для ответов на вопросы по свидетельствам из Википедии.⁵¹ Различие между разрывами в этих двух оценках показывает, что сторона, выполняющая развёртывание, должна выбирать требования к качеству в соответствии со своими задачами. Только небольшая модель, достигшая порога качества, может преобразовать меньший объём весов в экономию ёмкости и чтения при обслуживании.

При анализе крупных моделей и МоЕ необходимо отдельно указывать общее и активное число параметров. Для DeepSeek-V3 сообщается 671B общих параметров, около 37B активных параметров на токен и 14,8T токенов предобучения; для DeepSeek V4-Flash — 284B / около 13B и 32T, а для DeepSeek V4-Pro — 1,6T / около 49B и 33T. Общий объём весов влияет на ёмкость, а активные параметры дают лишь грубую оценку объёма вычислений. Три проекции основных экспертов DeepSeek V4-Flash можно рассчитать отдельно:

$$C_{\text{expert/token}} = 18n_L h f(k_{\text{routed}} + k_{\text{shared}}) = 18n_L h f(6 + 1).$$

Здесь n_L — число основных слоёв, h — размерность скрытого состояния, f — промежуточная размерность эксперта, а k_{routed} и k_{shared} — число маршрутизируемых и общих экспертов, выбираемых для каждого токена; коэффициент $18 = 3 \times 2 \times 3$ последовательно соответствует трём проекциям, 2 FLOPs на каждое умножение со сложением и трёхкратному относительно прямого прохода суммарному объёму прямого и обратного проходов. Вычисления основных экспертов DeepSeek V4-Flash составляют приблизительно 45,4495 GFLOPs/токен, что при 32T токенов даёт около 1.45438×10^{24} FLOPs; для DeepSeek V4-Pro — около 169,2465 GFLOPs/токен и при 33T токенов около 5.58513×10^{24} FLOPs. Полный объём вычислений одного шага обучения складывается из этих матриц экспертов, внимания, маршрутизации, МТР, оптимизатора и повторных вычислений. Указанные в отчёте веса потерь также не являются долями выполнения: например, вес МТР 0,3 не означает, что выполняется лишь 30% вспомогательных вычислений.⁵²

Упражнение 3-9 (расширенное): какие требования к ресурсам изменяются при увеличении объёма обучающих данных или числа экспертов
Пересчитайте D/N и $6ND$ по таблице Llama / Qwen. Если число параметров останется неизменным, а объём обучающих данных вырастет в четыре раза, спрогнозируйте изменение объёма вычислений обучения и чистой ёмкости весов после запуска. Для МоЕ объясните, можно ли считать, что при удвоении общего числа экспертов и неизменном числе экспертов, выбираемых на токен, ёмкость параметров и объём вычислений обучения экспертов увеличатся вдвое.

Помимо объёма вычислений затраты на обучение часто измеряют в GPU-часах. Если непрерывно используются n_{GPU} ускорителей, а общее число GPU-часов равно H_{GPU} , фактическая продолжительность составит

⁵¹Карточка модели Meta Llama 3, столбцы Llama 3 8B и Llama2 70B в таблице Base pretrained models.

⁵²Заметки об объёме вычислений при обучении, прямой и обратный проходы линейного слоя, распределение последовательностей и раздельный расчёт экспертов DeepSeek V4-Flash.

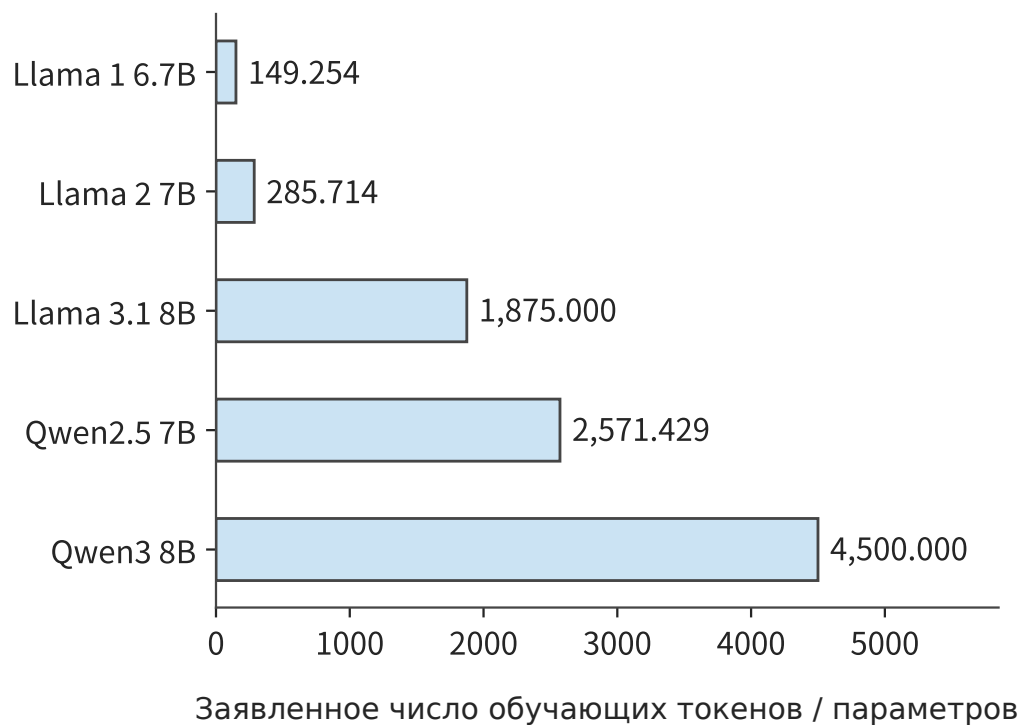


Рис. 3-30. Модели близкого размера обучаются на разном числе токенов. Значение столбца равно указанному объёму обучающих данных, делённому на число параметров; для Qwen используется опубликованный бюджет данных всей серии моделей.

$$T_{\text{calendar}} = \frac{H_{\text{GPU}}}{n_{\text{GPU}}}.$$

Обучение Llama 1 65B потребовало 1 022 362 GPU-часа A100. При непрерывном использовании 2048 ускорителей это соответствует приблизительно 499,2 часа, или 20,80 дня; при той же эффективности ускорителей, но вдвое меньшем их количестве потребуется около 41,60 дня. GPU-часы отражают общие затраты, а число ускорителей определяет фактическое время выполнения этой работы.

В отчёте DeepSeek-V3 указано 2,664M GPU-часа H800 на предобучение; 14,8T токенов относятся только к этапу предобучения. При непрерывном использовании 2048 ускорителей это соответствует приблизительно 54,20 дня. Для последующего расширения контекста и постобучения приведены отдельные объёмы, но статистика разных этапов имеет разный охват, поэтому их нельзя вместе делить на число токенов предобучения.⁵³

Чтобы преобразовать GPU-часы в фактическую продолжительность, их нужно разделить на число одновременно используемых ускорителей. Для Llama 3.1 405B возьмём опубликованные 30,84M GPU-часа H100 и предположим, что вся работа выполнялась на максимальном кластере из 16 384 ускорителей. Тогда фактическая продолжительность составит около $30.84 \times 10^6 / (16384 \times 24) \approx 78.43$ дня. Если на определённом этапе используется меньше ускорителей, то то же число GPU-часов потребует больше фактического времени; следовательно, график распределения обучающих ресурсов определяет

⁵³ Исторические заметки о затратах на обучение и Scaling Law, а также повторный расчёт по зафиксированным общедоступным полям.

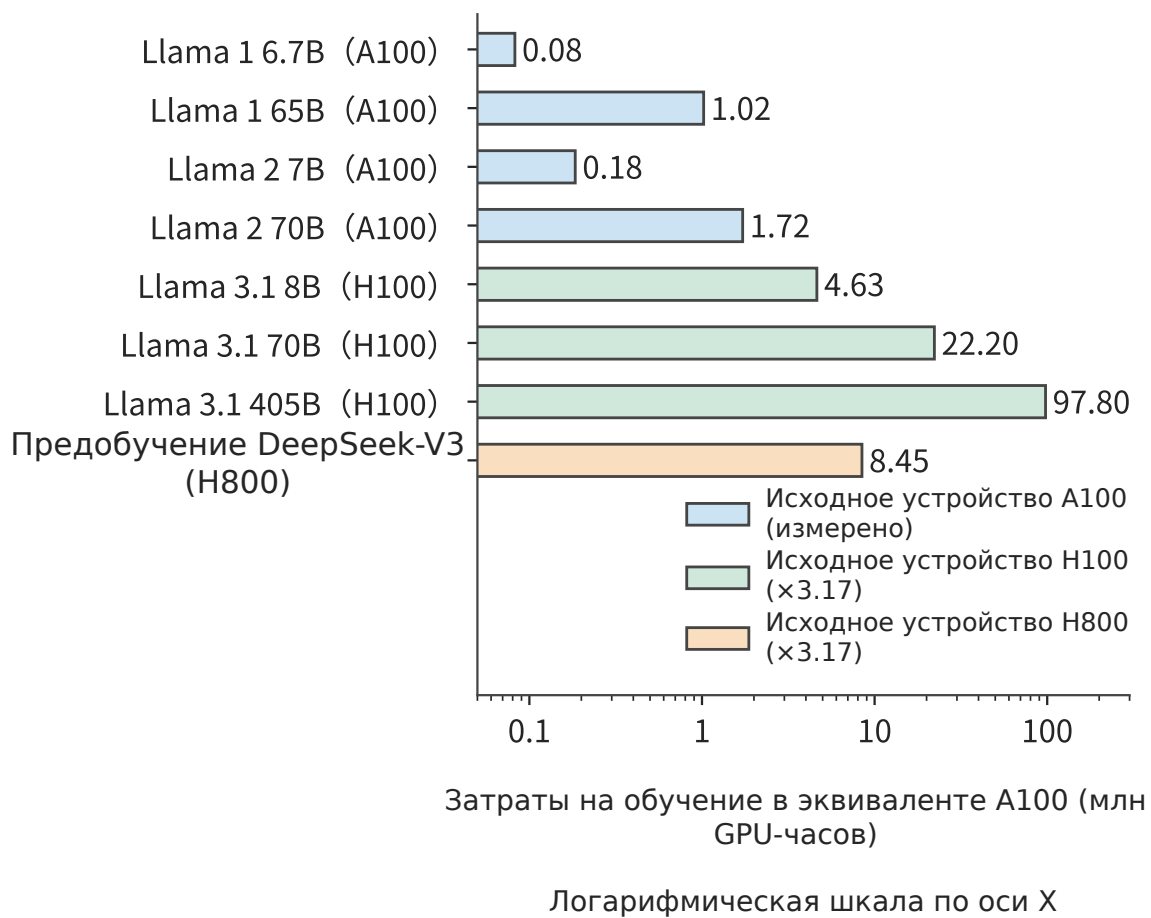


Рис. 3-31. Опубликованные затраты на обучение Llama и DeepSeek-V3, приведённые к эквивалентным GPU-часам A100 80GB. Для Llama 1 / Llama 2 приведены фактические часы A100; часы H100 и H800 пересчитаны по отношению пиковой производительности плотных вычислений BF16 $989.4/312 \approx 3.17$, а для DeepSeek-V3 учтён только этап предобучения. Горизонтальная ось имеет логарифмический масштаб; пересчёт предполагает близкую фактическую утилизацию всех устройств и не отражает различий в эффективности или стоимости.

срок завершения обучения.

Указанные объёмы получены на трёх типах ускорителей, поэтому GPU-часы нельзя напрямую сопоставлять между устройствами. Пересчитаем их по пиковой производительности плотных вычислений BF16: для A100 80GB она составляет 312 TFLOP/s, для H100 и H800 — 989,4 TFLOP/s (H800 отличается лишь меньшей пропускной способностью межсоединений). Один GPU-час H100 или H800 приблизительно эквивалентен $989.4/312 \approx 3.17$ GPU-часа A100. На рис. 3-31 три опубликованных значения приведены к эквивалентным часам A100 80GB. Пересчёт предполагает близкую фактическую утилизацию всех устройств и служит для сравнения масштаба затрат, а не различий в эффективности или стоимости.

Все приведённые выше значения относятся к предобучению, однако затраты на постобучение также можно сравнивать в GPU-часах. В таблице 21 технического отчёта Qwen3 сопоставляются два варианта постобучения одной модели 8B — RL и OPD, требующие соответственно 17 920 и 1 800 GPU-часов. Эти варианты являются альтернативными, и каждый включает генерацию, обратную связь и обучение. Если

соотнести GPU-часы с конкретными этапами и ускорителями, а затем учесть процесс выполнения каждого этапа, можно объяснить, на что в основном расходуются ресурсы обучения.

3.6.3 Стоимость полного жизненного цикла при соблюдении требований к качеству

Теперь объединим анализ из главы 2 и этой главы. Структура модели определяет необходимую для каждого вызова ёмкость хранения, объём вычислений и чтения; нагрузка определяет число вызовов и ожидание; обучение — затраты до запуска. Пусть первоначальные затраты модели m равны $C_0(m)$, в будущем предстоит выполнить Q задач, доля задач класса j равна w_j , а средняя стоимость выполнения одной задачи — $c_j(m)$. Тогда

$$C_{\text{life}}(m) = C_0(m) + Q \sum_j w_j c_j(m).$$

Перед сравнением все модели должны удовлетворять одинаковым требованиям к качеству, срокам и условиям завершения. Если вероятность успешного выполнения задач различается, c_j должна также учитывать неудачи и повторные попытки. При составлении бюджета необходимо указать и такие предположения, как изменение цен, срок службы модели, обслуживание и дисконтирование. Если для специализированного обслуживания требуется длительный период развёртывания, условием выбора становится также срок использования версии модели.

Рассмотрим две модели, каждая из которых удовлетворяет требованиям к качеству и срокам. Если меньшая модель требует дополнительных затрат на обучение ΔC_0 , но экономит $\Delta c > 0$ на каждой успешно выполненной задаче, число вызовов, при котором совокупные затраты сравниваются, равно

$$Q_* = \frac{\Delta C_0}{\Delta c}.$$

Если дополнительное обучение меньшей модели требует 100 тысяч GPU-часов H100, то есть 3.6×10^8 GPU-секунд, а каждая задача экономит 0,72 GPU-секунды H100, тогда $Q_* = 3.6 \times 10^8 / 0.72 = 5 \times 10^8$, то есть 500 миллионов задач. Если ожидаемый спрос ниже этого значения, дополнительные затраты на обучение ещё не окупятся; если выше — меньшая модель обеспечит большую совокупную экономию. Выбор совместно определяется ёмкостью, задержкой, качеством и числом вызовов; одного размера модели или цены токена для такого решения недостаточно.

Упражнение 3-10 (расширенное): продолжительность обучения и совокупная стоимость обслуживания

Предположим, что для завершения обучения требуется в общей сложности 1 022 362 GPU-часа A100 — опубликованное значение для Llama 1 65B. Рассчитайте число дней обучения при непрерывном использовании соответственно 2048 и 1024 ускорителей A100, предполагая, что общее число GPU-часов при изменении количества ускорителей остаётся неизменным. Затем сравните две модели, удовлетворяющие одинаковым требованиям к качеству и срокам: на обучение меньшей модели дополнительно затрачено 100 тысяч

GPU-часов H100, но каждая успешно выполненная задача экономит 0,72 GPU-секунды H100. Какую модель следует выбрать при ожидаемом выполнении 200 миллионов и одного миллиарда успешных задач? Рассчитайте разность совокупных затрат и укажите, при каком фактическом числе задач выбор изменится на противоположный.

В последующих главах будут использоваться три типа информации о нагрузке, систематизированные в этой главе. В таблице ниже перечислены сведения, которые необходимо сохранять для каждого типа нагрузки, а также задачи проектирования системы, в которых они будут использоваться.

Нагрузка	Обязательные поля	Последующее использование
Chat / AI-агент	Проверка задачи, модель и версия, вход / попадание / выход каждого раунда, поступление, зависимости от инструментов, ветвление и интервалы повторного использования	В главе 8 — планирование пакетной обработки и состояния; в главе 9 — передача и маршрутизация; в главе 11 — управление средой инструментов
Зрение / реальное время	Размер исходного изображения в байтах, E/P/D, EC/KV, поступление кадров или блоков, требования к первому ответу / непрерывному воспроизведению, вычисления, более не нужные после прерывания	В главах 8 и 9 — планирование кодирования и передачи; в главе 12 — добавление канала связи и места выполнения
Обучение / RL	Данные и качество, длина и метки, micro-batch / обновление, число сгенерированных / сохранённых образцов, вычисление вознаграждения и вызовы модели, версия весов, срок завершения	В главах 4–7 — анализ требований к вычислениям, хранению и обмену данными; в главе 10 — организация обучения; в главе 11 — настройка среды и пула ресурсов

Все три примера показывают, что для описания нагрузки необходимо фиксировать конкретный процесс выполнения. В двухминутном запросе соотношение ввода и вывода в разные периоды определяет момент возникновения очереди; в агенте для работы с кодом зависимости от инструментов определяют время ожидания и продолжительность хранения состояния; в RL число образцов до и после фильтрации определяет объём данных, обрабатываемых на этапах генерации и обновления. Только располагая этой информацией, можно распределить ресурсы между этапами.

В главе 4 вычислительная производительность, ёмкость хранения и пропускная способность микросхем будут использованы для анализа выполнения этих нагрузок на ускорителях. Рассчитанные в этой главе размеры матриц, объёмы состояния и число обращений позволят определить продолжительность вычислений, возможность размещения данных и своевременность чтения и записи. Зависимости между этапами определяют, какие работы можно выполнять с перекрытием, а какие должны ожидать.

3.7 Заблуждения и ловушки

Заблуждение: если средняя частота запросов и их длина одинаковы, то и нагрузка одинакова. Небольшое число сверхдлинных запросов, сочетание длин входных и выходных данных, а также порядок поступления запросов влияют на кратковременную потребность в ресурсах. В задаче 3-1 вычислительной мощности достаточно для обработки среднего входящего потока, однако в следующую минуту всё равно образуется очередь.

Заблуждение: если цена токена снижается, то снижается и стоимость успешно выполненной задачи. Дополнительные рассуждения, ответы, не прошедшие проверку, инструменты и окружение должны учитываться в числителе, а число успешно выполненных задач — в знаменателе. Для batch с нулевым числом успешных задач можно сообщить только о затратах и неудачах, но нельзя оценить стоимость успешного выполнения.

Заблуждение: если общее число обучающих токенов одинаково, то и объём работы системы одинаков. Изменение распределения последовательностей меняет число пар «запрос — ключ» в механизме внимания, стратегии обработки меток и выходного слоя словаря меняют объём вычислений на выходе, а число обновлений влияет на затраты оптимизатора; при обучении с подкреплением (RL) также приходится нести затраты на генерацию и обратную связь для неиспользованных ответов.

Заблуждение: вычислительно оптимальный масштаб, полученный из аппроксимации закона масштабирования, и есть масштаб обучения, который следует использовать. Вычислительный оптимум справедлив только при фиксированном бюджете обучения и без учёта стоимости инференса. В открытых моделях число обучающих токенов на параметр превышает этот ориентир на один-два порядка — это выбор, сделанный после включения в целевую функцию долгосрочной стоимости обслуживания; различие целевых функций не означает, что результаты аппроксимации опровергнуты.

3.8 Траектория выполнения задачи с кодом

Задача состоит в исправлении функции объединения интервалов, чтобы она правильно объединяла вложенные интервалы и интервалы с соприкасающимися границами, не изменяя входные данные и содержащиеся в них списки. Модель может прочитать указанный файл, перезаписать его, запустить фиксированные тесты и завершить задачу; контроллер не изменяет код вместо модели. При отключённом режиме рассуждений модель за 12 раундов один раз прочитала файл, пять раз перезаписала его и шесть раз запустила тесты, но так и не завершила задачу. При включённом режиме четыре раунда соответственно включали усечённый вывод, запись файла, запуск тестов и завершение. Ожидание, вызванное усечением в первом раунде, также учитывается в общем времени выполнения задачи.⁵⁴

3.9 Итоги главы

Нагрузка непрерывно работающего сервиса зависит как от объёма работы для каждого запроса, так и от времени поступления запросов и зависимостей между отдельными

⁵⁴Упражнение 3-4: две траектории агента для работы с кодом, включая независимую проверку и дополнительное условие для псевдонима.

этапами. Средняя частота позволяет приблизительно оценить необходимые ресурсы, а анализ образовавшейся за короткое время очереди и времени пребывания в состояниях — выявить нагрузку на ресурсы, не отражённую средними значениями. Многораундовые задачи также включают ожидание инструментов и неудачные попытки; в задачах реального времени необходимо обеспечить своевременное поступление данных, а обучение дополнительно требует обратного прохода, обновления параметров и синхронизации весов.

Сравнивая ресурсы и стоимость разных решений, следует применять одинаковые требования к качеству и критерии завершения задачи. *Scaling Law* предоставляет проверяемую статистическую модель для распределения ресурсов на обучение, тогда как долгосрочные потребности сервиса могут повлиять на выбор первоначальных вложений. В основных упражнениях 3-2, 3-4 и 3-7 эти методы применяются соответственно к непрерывному потоку запросов, интерактивным задачам и циклу RL.

Глава 4

Архитектура ускорителей

Вычисление занимает 100 мкс, из которых 60 мкс приходится на умножение матриц, а остальные 40 мкс — на прочую работу. Если удвоить скорость матричного блока, общее время сократится до 70 мкс, а ускорение составит примерно 1,4 раза. Если же умножение матриц занимает лишь 10 мкс, то такое же изменение сократит общее время только до 95 мкс. Прирост вычислительной мощности одинаков, но выигрыш разительно отличается. Процессоры общего назначения распределяют площадь кристалла между блоками для выполнения различных типов инструкций. Когда основная часть времени выполнения нейронных сетей стала приходиться на матричные операции, такое распределение перестало быть оправданным, что привело к появлению специализированных матричных блоков. Задача проектирования архитектуры как раз и состоит в том, чтобы при ограничениях на площадь кристалла, энергопотребление и объём памяти выбрать решение, которое максимально сокращает время выполнения.

В главах 2 и 3 были рассмотрены вычислительный граф модели и вычислительные потребности запроса на разных этапах. В этой главе на примере проекции запросов Qwen3-8B (Q-проекции), то есть матричного умножения, преобразующего скрытый вектор текущего токена в запрос механизма внимания, объясняется, как ускоритель выполняет такие вычисления. Сначала подсчитаем количество операций и объём считываемых и записываемых данных, затем последовательно рассмотрим разбиение матриц на блоки, иерархию памяти и зависимости данных, чтобы объяснить, почему фактическое время выполнения отличается от идеальной оценки. После этого добавим вычисления механизма внимания и экспертов и разберём, как более сложные операторы используют тот же набор аппаратных средств. Затем подробно сравним эволюцию поколений NVIDIA, Ascend и Apple и покажем, как требования моделей стимулируют совместное развитие этих компонентов.

Глава посвящена трём вопросам: какие объёмы вычислений, памяти и передачи данных требуются для задачи; почему вычислительные блоки ожидают данные или результат предыдущего шага; какая часть станет новым узким местом после повышения производительности одного из компонентов. Понимание этих трёх вопросов позволяет применять единый метод для анализа GPU, чипов Ascend и Apple, а также более специализированных архитектур.

4.1 Знакомство с ускорителем через вычисления модели

При анализе оператора прежде всего необходимо определить, какие операции он выполняет и какие данные читает и записывает. В этом разделе на основе этих двух требо-

ваний рассматриваются компоненты ускорителя и предварительно анализируются компромиссы между вычислительными блоками, памятью и интерфейсами.

4.1.1 Требования матричного умножения к вычислениям и данным

Скрытая размерность Qwen3-8B равна 4 096, каждая из 32 голов запросов содержит 128 компонентов, а ширина выхода Q-проекции также равна 4 096. При одновременной обработке M позиций токенов проекция имеет вид

$$Y = XW_Q, \quad X \in \mathbb{R}^{M \times 4096}, \quad W_Q \in \mathbb{R}^{4096 \times 4096}, \quad Y \in \mathbb{R}^{M \times 4096}.$$

В проекции $Y = XW$ величина M — общее число обрабатываемых в этот раз токенов, причём вектор признаков каждого токена занимает одну строку X ; K — число признаков каждого входного вектора и одновременно число строк матрицы весов W ; N — число признаков каждого выходного вектора и одновременно число столбцов W ; в данном примере $K = N = 4096$. Эти прописные буквы описывают форму матрицы и используются в другом контексте, нежели обозначение ёмкости M в главе 1. Если одновременно обрабатываются B запросов одинаковой длины, каждый из которых содержит P новых входных токенов, то для prefill $M = BP$; при пошаговом decode для каждого запроса обрабатывается один токен, поэтому $M = B$. Оба этапа используют одни и те же веса и выполняют одно и то же линейное преобразование; различается лишь число строк входных данных, совместно использующих эти веса при каждом вызове.¹

Для вычисления каждого выходного элемента требуется скалярное произведение длины 4 096. Если считать одно умножение и одно сложение двумя операциями с плавающей точкой, объём вычислений матричного умножения равен

$$F = 2MKN = 2M \times 4096^2 = 33\,554\,432M \text{ FLOPs}.$$

Теперь рассмотрим данные. Входные данные, веса и выходные данные представлены в BF16, по два байта на элемент; аккумулятор начинает с нуля и хранит частичные суммы в FP32, по четыре байта на элемент. Если входные данные и веса по одному разу считываются из внешней памяти, а итоговый результат один раз записывается обратно, общий объём чтения и записи равен

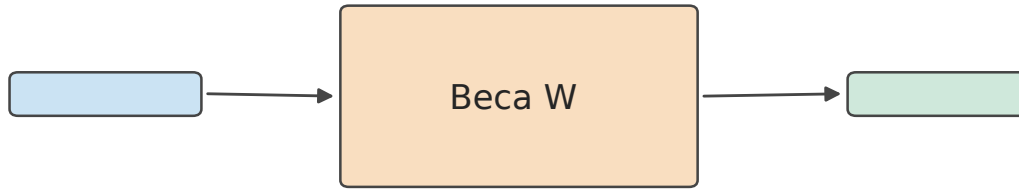
$$V = 2MK + 2KN + 2MN = 33\,554\,432 + 16\,384M \text{ bytes}.$$

Первое слагаемое — фиксированные 32 MiB весов, второе — объём входных и выходных данных, растущий с числом строк. Здесь MiB означает 2^{20} байт, а в характеристиках пропускной способности GB/s означает 10^9 байт/с. Разделив объём вычислений на объём обращений к памяти, получим арифметическую интенсивность $I = F/V$.

Одна Q-проекция	$M = 1$	$M = 256$
Объём вычислений матричного умножения	около 33,6 MFLOPs	около 8,59 GFLOPs
Чтение весов	32 MiB	32 MiB
Суммарный объём чтения входных данных и записи выходных	16 KiB	4 MiB

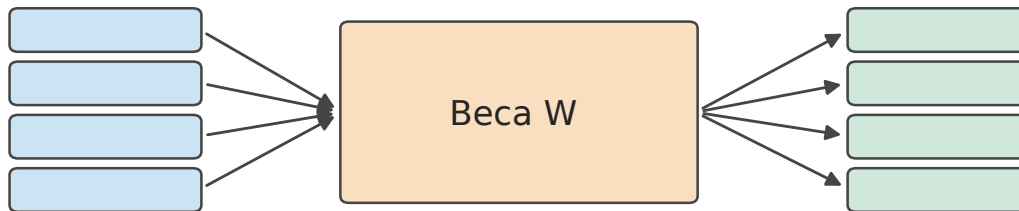
¹Фиксированную конфигурацию Qwen3-8B, индексы тензоров и реализацию модели см. в конфигурации модели и результатах расчёта проекции.

1 строка использует общие
веса 32 MiB



На строку: 32 MiB

256 строк используют общие
веса 32 MiB



На строку: 128 KiB

Рис. 4-1. Оба вызова используют одни и те же 32 MiB весов. Сверху показана одна входная строка, снизу — 256 строк, часть которых изображена полосами; приходящийся на одну строку объём чтения весов уменьшается с 32 MiB до 128 KiB.

Одна Q-проекция	$M = 1$	$M = 256$
Общий объём обращений	около 32 MiB	36 MiB
Арифметическая интенсивность	около 1 FLOP/байт	около 228 FLOPs/байт

На рис. 4-1 объём чтения весов распределён между входными строками: одна строка самостоятельно несёт нагрузку в 32 MiB, а при 256 строках на каждую приходится 128 KiB.

При увеличении числа строк в 256 раз число операций также возрастает в 256 раз, но объём обращений увеличивается лишь примерно на одну восьмую. Повторное использование весов приводит к тому, что объём вычислений растёт быстрее объёма обращений, поэтому арифметическая интенсивность повышается.²

Разделив числитель и знаменатель арифметической интенсивности на M , можно определить предел повторного использования:

$$I(M) = \frac{2KN}{2KN/M + 2(K + N)}.$$

²Проекция Q, RTX 4090, M=1; M=256.

По мере роста M приходящийся на одну строку весовой член $2KN/M$ постоянно уменьшается, тогда как член входных и выходных данных остаётся неизменным. В данном примере $I(M)$ в пределе приближается к 2 048 FLOPs/байт. Поэтому польза от дальнейшего увеличения batch постепенно уменьшается: хотя приходящийся на строку объём чтения весов становится всё меньше, входные и выходные данные новых токенов по-прежнему необходимо читать и записывать.

В приведённых вычислениях предполагается, что веса считываются только один раз; чтобы обеспечить это на аппаратном уровне, операнды и промежуточные результаты необходимо сохранять на всё время их использования.

4.1.2 Состав ускорителя и распределение функций

Хостом (host) обычно называется сторона CPU, на которой выполняется фреймворк, подготавливаются входные данные и запускаются задачи, а ускоритель отвечает за вычисления модели; в программном интерфейсе «хост — устройство» (host-device) сторона ускорителя обычно называется устройством. У GPU с собственной видеопамью память хоста и видеопамья представляют собой два отдельных хранилища. Перед выполнением проекции входные данные и веса размещаются в доступном ускорителю месте; затем CPU отправляет команду, указывая оператор, адреса данных и параметры выполнения. Если данные уже размещены в памяти, последующие вызовы могут многократно обращаться к одному и тому же адресу.

На рис. 4-2 показано, как внутренние компоненты ускорителя совместно выполняют одно матричное умножение. Во внешней памяти хранятся крупные входные данные и веса; общий кэш — это память, доступная нескольким группам вычислительных блоков и автоматически сохраняющая недавно использованные данные; локальный буфер — память, выделенная текущей вычислительной группе для временного хранения входных данных; матричный блок многократно обновляет частичные суммы; память аккумуляторов сохраняет частичные суммы до завершения вычисления текущего блока. Векторные и универсальные вычислительные блоки отвечают за преобразование результатов, вычисление адресов, управление потоком выполнения и другие задачи.

При вычислении одного выходного блока необходимо последовательно считывать несколько блоков входных данных и весов вдоль измерения K . После каждой операции умножения со сложением частичная сумма остаётся в памяти аккумуляторов, и следующий блок продолжает её обновлять. Если после каждого обновления записывать её во внешнюю память, промежуточный результат придётся многократно записывать и считывать; если же частичная сумма остаётся на чипе, из этих обращений сохраняется лишь финальная запись. Чем чаще данные используются повторно, тем выгоднее хранить их на чипе.

NVIDIA объединяет группу потоков, универсальные вычислительные блоки, матричные блоки и локальную память, способные совместно выполнять работу, в потоковый мультипроцессор (Streaming Multiprocessor, SM), а матричный блок умножения со сложением называет Tensor Core. Управляемый программно буфер внутри SM, совместно используемый потоками, называется общей памятью (shared memory). Поток — единица работы программы, отдельно исполняющая инструкции; запускаемая на ускорителе функция называется kernel (ядром), и один kernel может совместно исполняться множеством потоков. В архитектуре Ascend DaVinci блоки Scalar, Vector, Cube и MTE обозначают соответственно блок управления, векторный вычислительный блок, матричный вы-

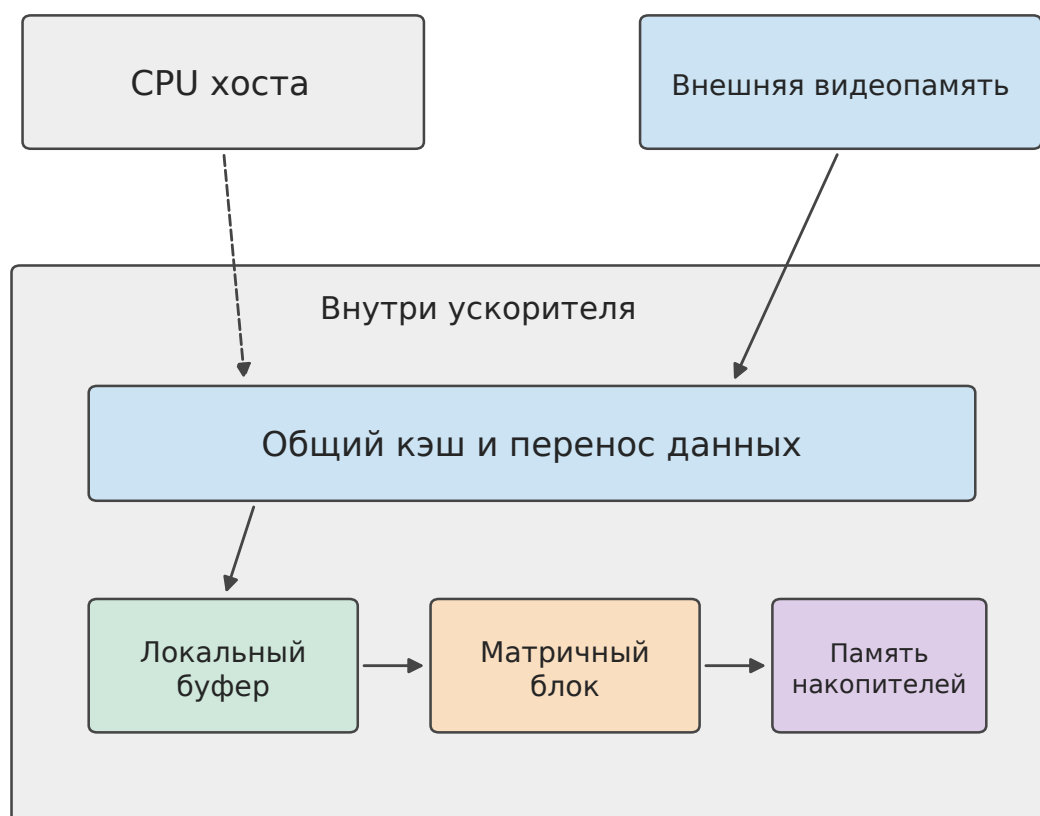


Рис. 4-2. Связь между хостом, видеопамятью и внутренними компонентами чипа. Сплошными линиями показано прохождение данных через кэш, локальный буфер, матричный блок и память аккумуляторов, пунктирной линией — отправка работы хостом. Компоненты на схеме сгруппированы по аппаратным функциям.

числительный блок и блок перемещения данных. Apple Metal — программный интерфейс отправки работы на GPU, использующий группы потоков для организации потоков, способных совместно использовать локальную память и синхронизироваться. Несмотря на различия в названиях, все эти компоненты должны решать несколько общих задач: как доставить операнды в исполнительный блок, как сохранить результат и когда можно начинать следующую работу.³

По стрелкам на рис. 4-2 можно различить два вида ожидания. Первый возникает, когда данные ещё не поступили, например если входной блок всё ещё передаётся. Второй — когда предыдущее вычисление ещё не завершилось, например если Softmax ожидает завершения скалярного произведения запросов и ключей (QK). Увеличение пропускной способности сокращает ожидание первого вида, а изменение разбиения на блоки или порядка выполнения влияет на ожидание второго вида. В следующих разделах эти два механизма будут рассмотрены отдельно.

Объём ресурсов, выделяемый каждому из этих компонентов, не определяется раз и навсегда. Модели и чипы взаимно влияют друг на друга от поколения к поколению. Возможности существующих ускорителей влияют на размер состояния моделей, форму матриц и способы коммуникации, а программное обеспечение повышает эффективность выполнения посредством сжатия, слияния и параллелизма. Если определённый вид нагрузки долгое время занимает значительную долю работы, разработчики чипов рассматривают возможность добавить специализированные средства для его обработки. После ввода в эксплуатацию нового поколения чипов изменяется и стоимость выполнения различных моделей. Проектирование и производство требуют времени, поэтому аппаратное обеспечение отвечает на уже наблюдавшиеся потребности, которые, как ожидается, сохранятся в будущем. В разделе 4.6 каждая связь на рис. 4-3 будет подробно рассмотрена на примере смены поколений трёх архитектур.

У такой обратной связи уже есть публично описанные примеры. В отчёте DeepSeek V3 на основе узких мест выполнения на H800 сформулированы требования к аппаратной поддержке: разгрузка коммуникаций, согласованные операции через межсоединение, низкоточное накопление и поддержка квантования; необходимость разгрузки коммуникаций обусловлена тем, что часть SM занята коммуникационными операциями.⁴

4.1.2.1 GPU и Ascend: кто организует перемещение данных и синхронизацию?

Обе архитектуры перемещают операнды во внутрикристальную память, выполняют матричные инструкции и записывают результаты обратно. Главное различие заключается в том, какие уровни управляются автоматически, а какие явно организуются ядром или компилятором.

Аспект	NVIDIA GPU (типичный высокопроизводительный путь CUDA)	Ascend DaVinci (типичный высокопроизводительный путь)
Матричные вычисления	Tensor Core внутри SM обрабатывают тайлы; варпы обеспечивают совместную работу потоков	Cube обрабатывает матричные тайлы; Vector/Scalar выполняют поэлементные операции, редукции и управление

³Сравнение архитектур ускорителей и выполнения.

⁴Технический отчёт DeepSeek V3, раздел 3.5, Suggestions on Hardware Design.

Аспект	NVIDIA GPU (типичный высокопроизводительный путь CUDA)	Ascend DaVinci (типичный высокопроизводительный путь)
Перемещение из глобальной памяти на кристалл	Кэши L2/L1 обычно управляются аппаратно; для предсказуемого повторного использования можно явно задействовать shared memory, асинхронное копирование или TMA	МТЕ/NDDMA по дескрипторам перемещают данные между Global Memory, L1/UB и L0; путь обычно планирует оператор или компилятор
Частичные суммы	Регистры, shared memory или специализированная матричная память новых поколений	L0C хранит частичные суммы Cube; последующие этапы записывают их обратно или передают в Vector
Перекрытие перемещений и вычислений	Передача между этапами выражается через memcpy_async, TMA, барьеры и конвейер	МТЕ и Cube/Vector взаимодействуют через флаги, события или синхронизацию, сгенерированную компилятором
Синхронизация	Раздельная синхронизация на уровнях warp, block, cluster и device; видимость памяти имеет собственную семантику	Этапы внутри ядра, передача между Cube/Vector и обмен между ядрами используют соответствующие флаги и протоколы синхронизации
Ответственность программиста	Обычные загрузки можно оставить кэшам; высокопроизводительный путь всё равно требует организации тайлов, shared memory, раскладки и асинхронных зависимостей	Раньше приходится учитывать ёмкость L1/UB/L0, раскладки, этапы перемещения и передачу данных; режим Developer может автоматически вставлять часть синхронизации

Кэш GPU обычно организует путь от глобальной памяти автоматически, а производительное CUDA-ядро явно задаёт shared memory, асинхронные копирования и барьеры. Компилятор AKG для Ascend также может автоматически сгенерировать перемещения МТЕ, внутрикристалльные буферы и синхронизацию. В главе 5 эта граница ответственности будет показана через циклы, адреса и события.

4.1.3 Ограничения площади чипа, энергопотребления и корпуса

Вычислительные блоки, память и интерфейсы занимают площадь чипа и потребляют электроэнергию. Чем больше ресурсов выделено одной части, тем меньший бюджет остаётся другим. Поэтому при оценке локального улучшения необходимо учитывать, насколько оно сокращает время выполнения всей задачи.

Обобщим пример из начала главы: пусть f обозначает долю времени последовательной задачи, приходящуюся на ускоряемую часть, а s — кратность её ускорения. Согласно закону Амдала из раздела 1.2.1, общее ускорение равно:

$$S = \frac{1}{(1 - f) + f/s} = \frac{1}{0.4 + 0.6/2} \approx 1.43.$$

Если дополнительно повысить скорость матричных операций до четырёхкратной относительно исходной, общее время уменьшится до $60/4 + 40 = 55 \mu\text{s}$. Первое удвоение

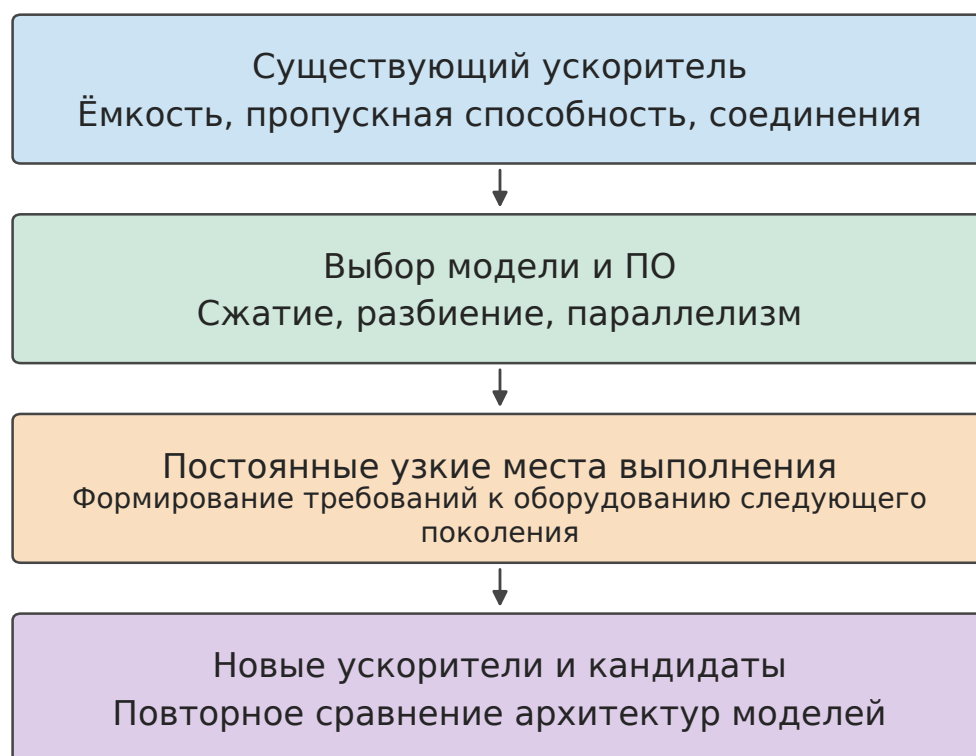


Рис. 4-3. Совместная эволюция моделей и аппаратного обеспечения между поколениями. Сплошные линии направлены вниз по оси времени: существующие ускорители влияют на выбор моделей, выполнение программ выявляет долговременные узкие места, проектирование аппаратного обеспечения отвечает на эти потребности, а новые ускорители делают возможным больше вариантов моделей.

экономит 30 μ s, а второе — лишь 15 μ s; даже если время матричных вычислений приблизится к нулю, остальная работа всё равно потребует 40 μ s. Предельное общее ускорение равно 2,5 раза. Доля оставшейся работы в новом общем времени становится всё больше, и вместе с ней изменяются приоритеты распределения ресурсов.

Распределение площади ранних TPU отражает реальные компромиссы между вычислениями и памятью. TPU — тензорный процессор, разработанный Google. TPU v1 первого поколения содержит 256×256 8-битных блоков умножения со сложением, 24 MiB унифицированного буфера данных (Unified Buffer) и 4 MiB памяти аккумуляторов. Согласно распределению площади чипа, приведённому в статье о TPU v1, буфер данных занимает около 37%, вычислительные блоки — около 30%, а схемы управления — около 2%. Буфер занимает даже больше площади, чем вычислительные блоки, поскольку массив должен многократно использовать входные данные, веса и частичные суммы. Если каждый раз считывать их из внешней памяти, блоки умножения со сложением будут долго ожидать данные.⁵

Предположим, что ради увеличения числа блоков умножения со сложением разработчик уменьшил ёмкость буфера. Блок входных данных, который раньше оставался в буфере, будет удалён раньше времени, и при следующем использовании его придётся загружать снова; дополнительное время повторного чтения может свести на нет выигрыш от новых блоков умножения со сложением. И наоборот, чрезмерно большой буфер для данных, которые почти не используются повторно, займёт площадь, которую можно было бы отдать вычислениям. При сравнении вариантов проектирования необходимо рассчитывать полное время одной и той же задачи до и после изменения, одновременно учитывая ускорение вычислений и увеличение числа повторных чтений.

Вопрос для размышления: продолжать наращивать производительность матричных вычислений или ускорить остальную работу? В описанной выше задаче, состоящей из 60 μ s и 40 μ s, что экономит больше времени: повышение ускорения матричных операций с двух до четырёх раз относительно исходной скорости или сокращение времени остальной работы с 40 до 25 μ s? Как выбирать, если эти изменения требуют разной площади?

Помимо площади, ещё одним бюджетом является энергопотребление; чем крупнее чип и чем больше карт, тем вероятнее, что энергопотребление станет определяющим ограничением. Далее с помощью четырёх выводов энергопотребление превращается в величину, которую можно рассчитать: почему перемещение данных расходует энергию, сколько энергии требуется для перемещения одного байта на каждом уровне памяти, насколько большим может быть один чип и сколько памяти к нему можно подключить, а также как ограничение мощности снижает фактическую рабочую частоту.

Почему перемещение расходует энергию. Энергия на перемещение данных расходуется при зарядке и разрядке соединений. Энергопотребление цифровой схемы состоит из статической и динамической частей: статическое энергопотребление обусловлено токами утечки и не зависит от переключений, а динамическое возникает при переключении сигналов между 0 и 1. Согласно статье Хэ Тинбо о LogicFolding, в типичной нагрузке смартфона около 90% энергопотребления приходится на динамическую часть.⁶

⁵Jouppi и др., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017, архивная версия статьи.

⁶Хэ Тинбо, *Huawei's τ Chip Was Supposed to Melt?*, ChinaXiv:202609.00031v1, 2026-09-04, архивная версия статьи, §II—V и рисунки 1—2: доля динамической мощности, ёмкость межсоединений, сравнение NPU при одинаковой производительности (29 TOPS, 0.85→0.55 V, снижение частоты на 63%, снижение энергопотребления на 66%), снижение энергопотребления DSP на 25% и проекционной площади на 40%.

Сигнальную линию вместе с управляемыми ею вентилями можно представить как конденсатор C : чтобы зарядить его от 0 до напряжения V , источник питания должен предоставить энергию CV^2 . Пусть α — доля сигналов, переключающихся в каждом такте, а f — тактовая частота (здесь f означает частоту, а не долю из приведённого выше закона Амдала). Тогда динамическая мощность равна

$$P_{\text{dyn}} \approx \alpha CV^2 f.$$

В передовых технологических процессах основная часть C приходится не на затворы транзисторов, а на соединения: ёмкость металлической линии распределена по её длине, и чем дальше проходит сигнал, тем большую ёмкость приходится заряжать и разряжать. В верхней части рис. 4-4 показан этот механизм: один и тот же вентиль управляет горизонтальной линией длиной в несколько сотен микрометров, поэтому требуется зарядить всю распределённую вдоль неё ёмкость; после складывания схемы в два расположенных друг над другом слоя прежний горизонтальный путь заменяется вертикальным соединением длиной в несколько микрометров. Согласно статье, после складывания длина соединений типичного ядра уменьшилась на 20%, длина некоторых критических путей — на 70%, а число тактовых буферов в одном вычислительном модуле сократилось с 43 600 до 19 000.

Два примера из статьи можно непосредственно рассчитать по этой формуле. Первый — сравнение NPU при одинаковой производительности: при тех же 29 TOPS (29 триллионов операций в секунду) напряжение сложенного NPU снижено с 0,85 V до 0,55 V. Если учитывать только напряжение,

$$\frac{P_{\text{after}}}{P_{\text{before}}} = \left(\frac{0.55}{0.85} \right)^2 = \frac{121}{289} \approx 0.42.$$

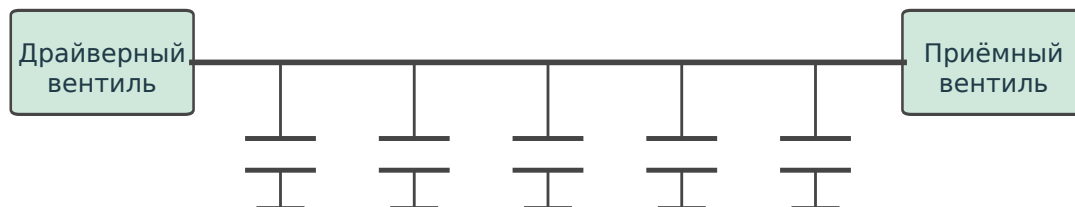
Если учитывать только частоту, её снижение на 63% означает, что f составляет 0,37 от исходного значения. Перемножение коэффициентов напряжения и частоты даёт $0.42 \times 0.37 \approx 0.16$, что значительно ниже приведённого в статье отношения общей мощности 0,34 (снижение на 66%). Разница обусловлена параллелизмом: NPU предыдущего поколения содержал одно большое и два малых ядра, а после складывания — четыре больших ядра (раздел V статьи), что позволяет за счёт большего числа параллельных ядер достигать тех же 29 TOPS при низкой частоте. С увеличением числа ядер возрастает переключаемая за такт ёмкость αC , компенсируя уменьшение C благодаря более коротким соединениям. Только совместный учёт напряжения, частоты и параллелизма даёт 0,34, причём наибольший вклад вносит напряжение; поэтому авторы статьи считают вклад снижения напряжения более значительным, чем вклад сокращения соединений.

Второй пример — DSP (цифровой сигнальный процессор): складывание первого поколения снизило мощность до 0,75 от исходной, а площадь проекции (площадь, занимаемая чипом на плоскости) — до 0,60. Поэтому мощность на единицу площади, то есть плотность мощности, стала равна

$$\frac{0.75}{0.60} = 1.25,$$

что на 25% выше исходного значения и согласуется с приведёнными в статье 24%. Тепло необходимо отводить через площадь чипа, поэтому снижение общей мощности

Горизонтальная линия: сотни μm ,
ёмкость растёт с длиной



Каждая зарядка 0→1 забирает от источника энергию $C \cdot V^2$

После складывания: вертикальная связь между слоями — лишь
несколько μm

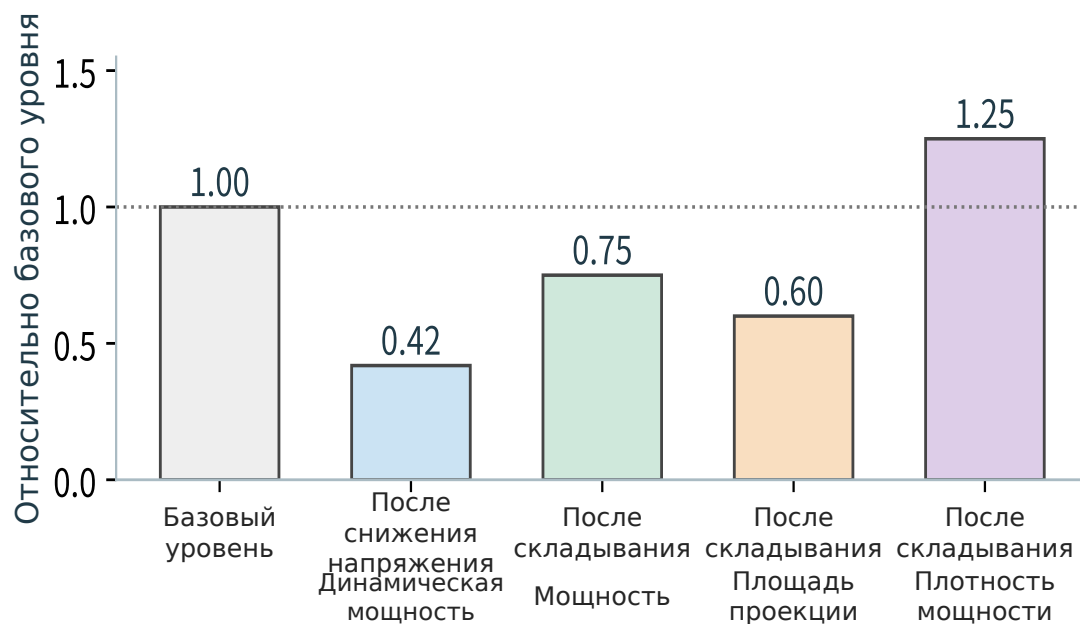


Рис. 4-4. Сверху: один и тот же управляющий вентиль передаёт сигнал по горизонтальному соединению длиной в несколько сотен микрометров, поэтому приходится заряжать распределённую вдоль линии ёмкость; после складывания оно заменяется вертикальным соединением длиной в несколько микрометров. Снизу: снижение напряжения с 0,85 V до 0,55 V уменьшает динамическую мощность до 0,42; мощность снижается до 0,75, а площадь проекции — до 0,60, поэтому плотность мощности, наоборот, возрастает до 1,25.

не обязательно уменьшает требования к охлаждению. В нижней части рис. 4-4 рядом приведены числа для этих двух примеров.

Условие обращения вывода: плотность мощности не возрастает, только если отношение площадей не меньше отношения мощностей; в данном примере отношение площадей 0,60 меньше отношения мощностей 0,75, поэтому плотность мощности увеличивается.

Иерархия энергозатрат: сколько энергии требуется для перемещения одного байта. Энергозатраты зависят от ёмкости, которую необходимо заряжать и разряжать, а ёмкость растёт с расстоянием, поэтому энергия получения одного и того же байта из разных уровней памяти сильно различается. В следующей таблице приведены затраты энергии на один байт для разных уровней памяти и каналов связи: SRAM (статическая память с произвольным доступом) располагается на чипе, HBM и LPDDR (энергоэффективная DRAM) представляют собой внешнюю DRAM (динамическую память с произвольным доступом), а NVLink-C2C — межчиповый канал, соединяющий CPU и GPU в суперчипе Grace Hopper. Единица pJ означает пикоджоуль (10^{-12} J). Значения взяты из четырёх источников; в таблице также указаны соответствующие технологические процессы и годы.⁷

Уровень	Исходное значение	Энергия на байт	Технологический процесс и год
Локальная SRAM объёмом порядка KB	5 pJ/32-битное слово	1,25 pJ	45 nm, Horowitz ISSCC 2014, цитируется по Dally Hot Chips 2023
Внутричиповая SRAM объёмом порядка MB	50 pJ/32-битное слово	12,5 pJ	Dally Hot Chips 2023, стр. 52, технологический процесс не указан
Канал NVLink-C2C	1.3 pJ/bit	10,4 pJ	Суперчип Grace Hopper, технический блог NVIDIA, ноябрь 2022 года
HBM2	3.97 pJ/bit	31,76 pJ	Модель энергопотребления DRAM 28 nm, MICRO 2017
LPDDR DRAM	640 pJ/32-битное слово	160 pJ	45 nm, Horowitz ISSCC 2014
16-битное умножение со сложением (2 FLOPs)	1,5 pJ/операцию	0.75 pJ/FLOP	45 nm, Dally Hot Chips 2023

Большая часть энергии HBM2 на один бит расходуется внутри DRAM, а не в соединении с GPU. Die (кристалл) — фрагмент кремниевой пластины, вырезанный из полупроводниковой пластины. Согласно разбиению из статьи MICRO 2017, тракт данных внутри DRAM die от массива памяти до базового die (нижнего die стека HBM) потребляет 2.24 pJ/bit, активация строки (открытие строки в массиве памяти) — 1.21 pJ/bit, а на I/O для

⁷Dally, Hardware for Deep Learning, программный доклад на Hot Chips 2023, страницы 12 и 51–52: на странице 12 — HFMA 1,5 pJ, HMMA 110 pJ, накладные расходы на инструкции около 30 pJ и доля накладных расходов 2000%/22% (45 nm); на странице 51 указано, что таблица энергопотребления взята из доклада Horowitz на ISSCC 2014, техпроцесс — 45 nm; на странице 52 для трёх уровней памяти приведены значения 5/50/640 pJ на каждое 32-битное слово, техпроцесс не указан; Fine-Grained DRAM, MICRO 2017, §1–2: 3,97 pJ/bit для HBM2 и разложение этого значения, модель энергопотребления DRAM рассчитана для 28 nm; NVIDIA Grace Hopper Superchip Architecture In-Depth, 2022-11-10: 1,3 pJ/bit для NVLink-C2C.

передачи данных к GPU через интерпозер (interposer, кремниевая пластина с соединениями между чипами) приходится лишь 0.3 pJ/bit. Суммарно эти три компонента дают 3.75 pJ/bit, а с учётом ECC (кода коррекции ошибок) и других накладных расходов — 3.92 pJ/bit; в основном тексте статьи приведено итоговое значение 3.97 pJ/bit, указанное в таблице. Значение 1.3 pJ/bit для NVLink-C2C учитывает лишь сам канал; передаваемые по нему данные сначала необходимо считать из памяти на противоположной стороне.

Применим эту таблицу к задаче генерации из раздела 4.8.3: Qwen3-8B, BF16, один запрос, контекст 8K. Один шаг decode считывает 15.137 GB весов и 1.208 GB KV, всего 16.345 GB, а объём матричных вычислений составляет 19.97 GFLOPs. Если использовать строки HBM и умножения со сложением из таблицы, получим⁸

$$E_{\text{weights}} = 15.137 \times 10^9 \times 31.76 \text{ pJ} \approx 0.481 \text{ J},$$

$$E_{\text{KV}} \approx 0.038 \text{ J},$$

$$E_{\text{compute}} = 19.97 \times 10^9 \times 0.75 \text{ pJ} \approx 0.015 \text{ J},$$

всего 0.534 J, из которых 90% приходится на чтение весов: при decode одного запроса почти вся энергия расходуется на перемещение данных. Строка HBM основана на модели энергопотребления HBM2 для технологического процесса 28 nm, а строка вычислений — на энергозатратах умножения со сложением для логического технологического процесса 45 nm. Поскольку технологические процессы различаются, этот расчёт показывает лишь порядок величин энергии перемещения данных и вычислений. Время этого шага ограничено пропускной способностью: при 3.35 TB/s чтение 16.345 GB занимает 4,9 ms, а распределение 0.534 J на этот промежуток времени даёт лишь около 110 W. В нижней части рис. 4-5 предполагается, что те же 16.345 GB полностью поступают с одного уровня: для SRAM объёмом порядка KB требуется лишь 0.020 J, для SRAM объёмом порядка MB — 0.204 J, для канала NVLink-C2C — 0.170 J, для HBM — 0.519 J, а для LPDDR — 2.62 J.

Пропускная способность определяет только скорость одного шага, но не его энергозатраты: если удвоить пропускную способность HBM, время этого шага сократится вдвое, а 0.534 J останутся неизменными. Снизить энергозатраты можно лишь двумя способами: получать байты с более близкого уровня или использовать одни и те же байты для большего числа токенов. При batch, равном B , веса считываются один раз и обслуживают B токенов, поэтому энергия на токен равна $0.481/B + 0.038 + 0.015$ J; при $B = 32$ она составляет около 0.068 J, то есть одну восьмую от значения для одного запроса. Таким образом, повторное использование данных — прежде всего вопрос энергопотребления и лишь затем пропускной способности. Порог из раздела 4.8.1, рассчитанный по времени, определяется отношением пропускной способности к вычислительной мощности, а рассматриваемый здесь порог — отношением энергозатрат на байт к энергозатратам на FLOP; эти пороги не совпадают.

Условие обращения вывода: $0.481/B < 0.015$, то есть энергозатраты на перемещение весов в расчёте на токен становятся ниже энергозатрат вычислений только при batch больше 32.

Фотошаблон и корпусирование: насколько большим может быть один чип и сколько памяти к нему можно подключить. Литографическая машина за одну экспозицию может покрыть лишь область рисунка одного фотошаблона (reticle, маска для

⁸Результаты раздельного учёта энергопотребления: количество байтов и FLOPs считывается из результатов decode одного запроса с контекстом 8K, после чего складывается энергопотребление на каждый байт и каждый FLOP согласно приведённой выше таблице. Расчёт можно повторить командой `python3 calculations/calc.py energy-ledger --format md`.

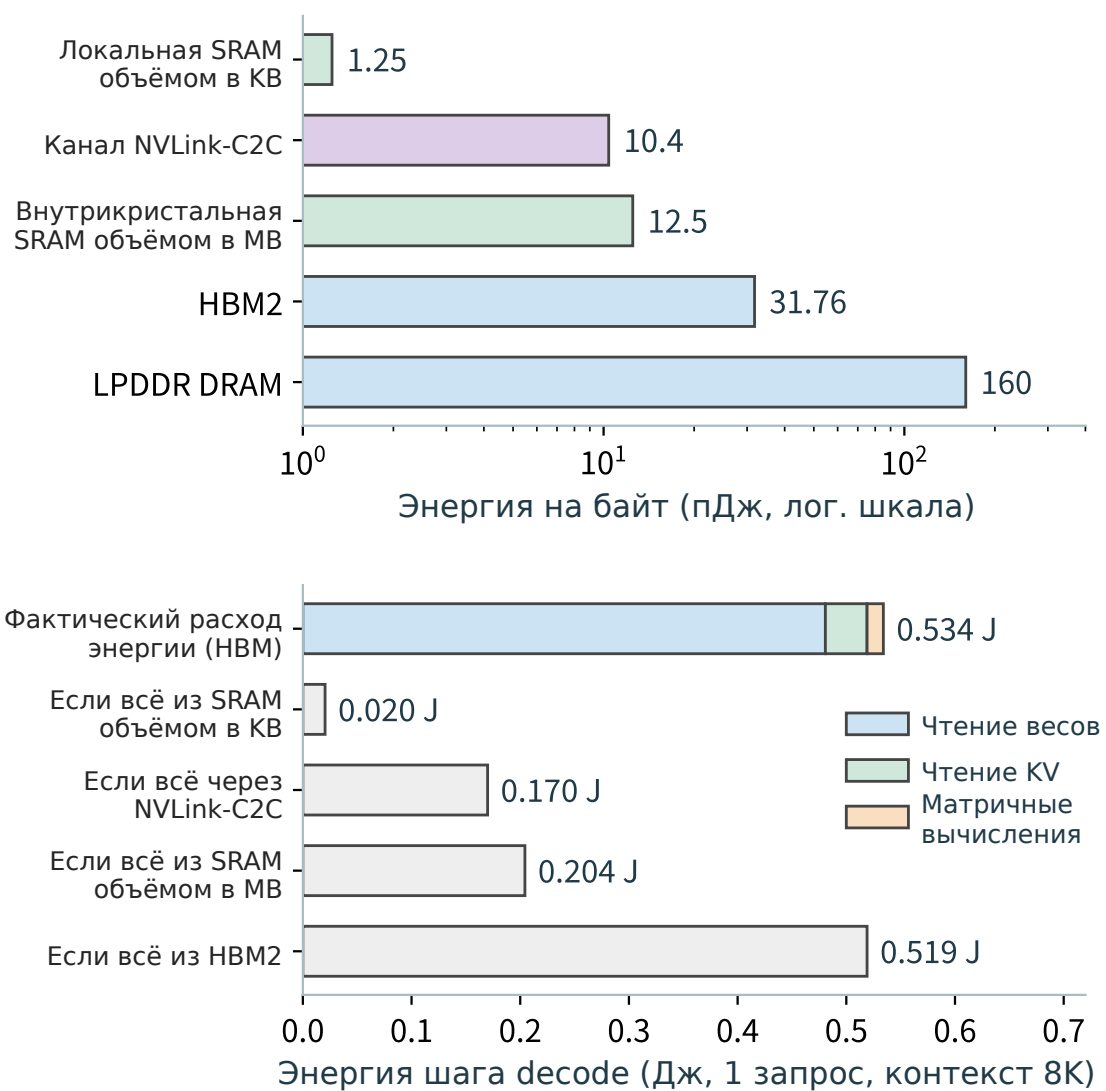


Рис. 4-5. Сверху: энергия на один байт для различных уровней памяти и каналов связи, логарифмическая шкала. Снизу: энергетический баланс одного шага decode Qwen3-8B для одного запроса с контекстом 8K (веса — 0.481 J, KV — 0.038 J, вычисления — 0.015 J), а также энергия в случае, когда те же 16.345 GB полностью поступают с одного уровня.

экспонирования); эта площадь называется пределом фотошаблона, и один die не может его превышать. Площадь die H100 составляет 814 mm², а A100 — 826 mm²; оба поколения близки к этому пределу. GPU Blackwell для центров обработки данных содержит два die, каждый из которых достигает предела фотошаблона; они объединены в один чип каналом NV-HBI с пропускной способностью 10 TB/s.⁹ После разделения чипа на два die расположение вычислений определяет, какая доля обращений должна проходить через этот канал; конкретный расчёт приведён в разделе 4.5.1.

HBM укладывает несколько слоёв DRAM die на один базовый die, соединяет слои сквозными кремниевыми переходами (TSV, вертикальными проводниками, проходящими через кремний), а весь стек подключается к GPU линиями на интерпозере. Каждый стек имеет 1 024 контакта данных. При скорости контакта r пропускная способность и ёмкость одного стека равны

$$R_{\text{stack}} = \frac{1024 r}{8}, \quad C_{\text{stack}} = \text{число слоёв} \times \text{ёмкость одного слоя}.$$

Рассмотрим HBM3E: восьмислойный стек Micron имеет ёмкость 24 GB, а двенадцатислойный — 36 GB; если подставить указанную SK hynix скорость контакта 9.6 Gbit/s в приведённую формулу, получится $1024 \times 9.6/8 \approx 1229$ GB/s, что соответствует значению 1.23 TB/s на странице продукта. На рис. 4-6 показана такая компоновка: die расположен в центре, стеки HBM размещены вдоль его краёв, а интерпозер несёт соединения между ними. По той же формуле можно восстановить конфигурации HBM трёх поколений GPU:

9

Таблица 4 технического описания архитектуры H100: пять стеков HBM3, 5120-битный интерфейс, DDR с частотой 2619 MHz, 3352 GB/s (в основном тексте этой главы единообразно используется округлённое значение 3,35 TB/s, то есть 3 350 GB/s, из технического паспорта), 814 mm², техпроцесс TSMC 4N; площадь A100 — 826 mm²; технический обзор Blackwell: на странице 7 указаны два кристалла предельного для фотошаблона размера и NV-HBI с пропускной способностью 10 TB/s, а в таблице 3 для B200 указаны 192 GB/7,7 TB/s; таблица 1 описания компонентов платформы HGX: каждый GPU в HGX B200 имеет 180 GB и пропускную способность до 8 TB/s; технический паспорт H200: 141 GB/4,8 TB/s; страница продукта Micron HBM3E: 1024 вывода, 24 GB в восьми слоях и 36 GB в двенадцати слоях; страница продуктов SK hynix HBM: 9,6 Gbit/s и 1,23 TB/s.

GPU	Ёмкость	Пропускная способность	Число стеков	Ёмкость одного стека	Пропускная способность одного стека	Скорость контакта
H100 SXM5	80 GB	3,350 GB/s	5, согласно техническому описанию	16 GB	670 GB/s	5.24 Gbit/s, в техническом описании указано 2619 MHz DDR (двойная скорость передачи данных, две передачи за такт)
H200 SXM	141 GB	4,800 GB/s	6 стеков общей ёмкостью 144 GB, в продукте доступен 141 GB	24 GB	800 GB/s	6.25 Gbit/s, расчётное значение
B200 (HGX)	180 GB	8,000 GB/s	8 стеков общей ёмкостью 192 GB, в продукте доступно 180 GB	24 GB	1,000 GB/s	7.81 Gbit/s, расчётное значение

Рост каждого поколения в таблице можно разложить на число стеков, ёмкость одного стека и скорость контакта. Пять стеков H100 образуют 5 120-битный интерфейс; умножение 5 120 бит на 5.24 Gbit/s и деление на 8 даёт около 3,350 GB/s. От H100 к H200 ёмкость выросла на 76% благодаря увеличению ёмкости одного стека с 16 до 24 GB и числа стеков с 5 до 6; пропускная способность выросла на 43% за счёт дополнительного стека и повышения скорости контакта с 5.24 до 6.25 Gbit/s. В B200 по сравнению с H200 добавлено ещё два стека: H100 и H200 содержат по одному die, к которым подключено соответственно 5 и 6 стеков; в B200 восемь стеков размещены вдоль внешних сторон двух die предельного для фотошаблона размера (рис. 4-6). Число стеков получено из указанной в техническом обзоре Blackwell конфигурации 192 GB и 7.7 TB/s при ёмкости одного стека 24 GB; в остальных разделах книги используются характеристики платформы HGX B200: 180 GB и 8 TB/s на один GPU.

Условие обращения вывода: если требуемое число стеков (целевая ёмкость, делённая на ёмкость одного стека) превышает число стеков, подключаемых к одному die, необходимо либо увеличить число слоёв в каждом стеке, либо добавить die; при переходе на двенадцатислойные стеки ёмкостью 36 GB шесть стеков дают 216 GB, поэтому увеличить их число не требуется.

Ограничение мощности: почему пиковая вычислительная производительность не может поддерживаться постоянно. Расчётная тепловая мощность (thermal design power, TDP) — максимальная длительная мощность модуля; для H100 SXM она равна 700 W, а для B200 — 1 000 W.¹⁰ Этот бюджет мощности совместно используется памятью и вычислительными блоками. На примере H100 оценим мощность HBM при полной пропускной способности по строке HBM2 из приведённой выше таблицы: $3.35 \text{ TB/s} \times 31.76 \text{ pJ/byte} \approx 106 \text{ W}$. После вычитания для вычислений остаётся 594 W; разделив это значение на пиковую производительность плотных вычислений BF16, равную 989.4 TFLOP/s, получим энергетический бюджет одного FLOP при пиковой скорости:

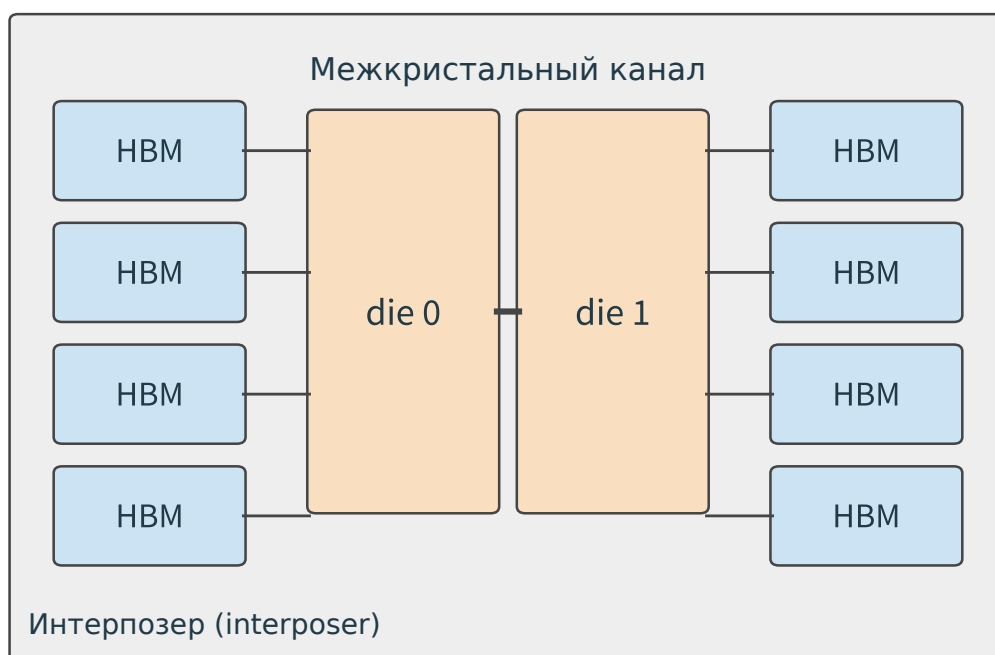
$$b = \frac{700 \text{ W} - 106 \text{ W}}{989.4 \text{ TFLOP/s}} \approx 0.60 \text{ pJ/FLOP}.$$

Уложиться в бюджет 0.60 pJ/FLOP позволяют только матричные инструкции и совершенствование технологического процесса; эталонное значение для 45 nm объясняет причину. Одна скалярная инструкция полуточного умножения со сложением (HFMA) выполняет 2 FLOPs и потребляет 1.5 pJ, то есть 0.75 pJ/FLOP, что уже превышает бюджет; выборка и декодирование инструкции, а также получение операндов требуют ещё около 30 pJ — в 20 раз больше энергии самой операции. Одна матричная инструкция умножения со сложением HMMA потребляет 110 pJ, но выполняет значительно больше 2 FLOPs, поэтому доля тех же накладных расходов на инструкцию после распределения составляет лишь 22%.¹¹ H100 производится по технологическому процессу TSMC 4N;

¹⁰Технический паспорт H100: максимальная расчётная тепловая мощность SXM составляет 700 W; таблица 4 технического описания архитектуры H100: 989,4 TFLOPS для плотных вычислений BF16 и TDP 700 W; таблица 3 технического обзора Blackwell: B200 — 1000 W.

¹¹Dally, Hardware for Deep Learning, программный доклад на Hot Chips 2023, страницы 12 и 51—52: на странице 12 — HFMA 1,5 pJ, HMMA 110 pJ, накладные расходы на инструкции около 30 pJ и доля накладных расходов 2000%/22% (45 nm); на странице 51 указано, что таблица энергопотребления взята из доклада Horowitz на ISSCC 2014, техпроцесс — 45 nm; на странице 52 для трёх уровней памяти приведены значения 5/50/640 pJ на каждое 32-битное слово, техпроцесс не указан; Fine-Grained DRAM, MICRO 2017, §1—2: 3,97 pJ/bit для HBM2 и разложение этого значения, модель энергопотребления DRAM рассчитана для 28 nm; NVIDIA Grace Hopper Superchip Architecture In-Depth, 2022-11-10: 1,3 pJ/bit для NVLink-C2C.

Площадь каждого die не больше поля экспонирования ретикла
(H100: 814 mm²)



Пропускная способность стека = 1024 вывода × скорость вывода;
ёмкость стека = число слоёв × ёмкость слоя; стеки вдоль края die

Рис. 4-6. Схема корпуса, вид сверху: два die предельного для фотошаблона размера расположены в центре, восемь стеков HBM — вдоль краёв с двух сторон, а интерпозер несёт соединения между die и HBM, а также между двумя die. Пропускная способность каждого стека определяется 1 024 контактами и скоростью контакта, а ёмкость каждого стека — числом слоёв и ёмкостью одного слоя.

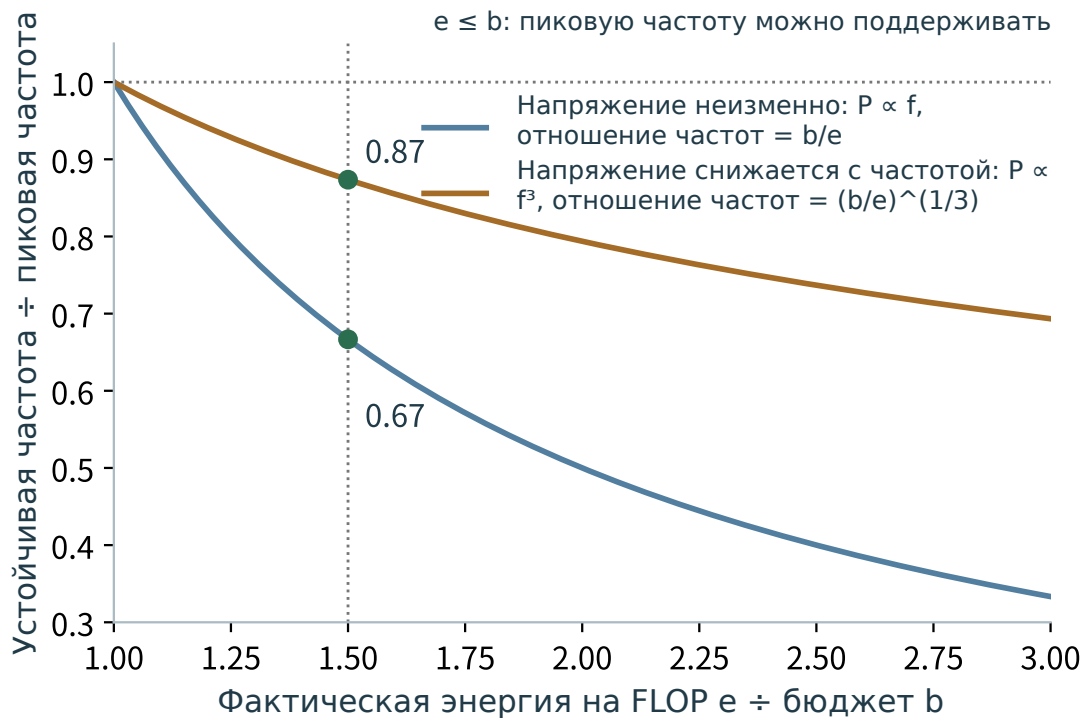


Рис. 4-7. Отношение длительно поддерживаемой частоты к пиковой в зависимости от отношения энергозатрат e на один FLOP к бюджету b : при неизменном напряжении оно равно b/e , а при снижении напряжения вместе с частотой — $(b/e)^{1/3}$. При e/b не больше 1 пиковая частота может поддерживаться постоянно; при e/b , равном 1,5, две кривые дают соответственно 0,67 и 0,87.

при переходе от 45 nm к 4N ёмкость и напряжение каждого переключения снижались от поколения к поколению.

Пусть фактические энергозатраты на FLOP равны e . При $e \leq b$ пиковая частота может поддерживаться постоянно; при $e > b$ контроллер мощности снижает частоту, а степень снижения определяется зависимостью $P \propto fV^2$: при неизменном напряжении мощность линейно зависит от частоты, поэтому длительно поддерживаемая частота составляет b/e от пиковой; если напряжение снижается вместе с частотой, то $P \propto f^3$, а длительно поддерживаемая частота составляет $(b/e)^{1/3}$ от пиковой. На рис. 4-7 показаны эти две кривые: когда e в 1,5 раза превышает бюджет, первая снижается до 0,67, а вторая — лишь до 0,87. Длительно поддерживаемая вычислительная производительность снижается пропорционально частоте, поэтому расчёт времени вычислений по пиковой производительности даёт значение меньше фактического. Пиковая вычислительная производительность — физический предел аппаратного обеспечения, но он достижим не при любых условиях: при ограничении мощности сам предел снижается вместе с длительно поддерживаемой частотой, поэтому при оценке утилизации знаменателем должна служить длительно поддерживаемая вычислительная производительность.

Условие обращения вывода: если энергозатраты на FLOP ниже $(TDP - P_{\text{mem}})/F_{\text{peak}}$, пиковая частота может поддерживаться постоянно; здесь P_{mem} — мощность памяти, F_{peak} — пиковая вычислительная производительность, а в данном примере порог равен 0,60 pJ/FLOP. С каждым новым поколением технологического процесса энергозатраты на FLOP снижаются, а TDP от поколения к поколению повышается; эти два фактора совместно определяют, когда выполняется данное условие.

4.2 Вычислительные блоки

В разделе 4.1 производительность матричных блоков выражалась через скорость вычислений. В этом разделе объясняется, откуда берётся эта скорость: матрицу какого размера обрабатывает одна инструкция, почему небольшие матрицы не позволяют полностью задействовать вычислительные блоки и какие векторные и управляющие операции дополнительно выполняются над результатами матричных вычислений.

4.2.1 Почему матричные блоки подходят для нейронных сетей

Большую матрицу приходится разбивать на множество небольших блоков, а матричные блоки — это именно то оборудование, которое выполняет вычисления над такими блоками. Они поддерживают ограниченный набор форм матричных инструкций, поэтому программа должна организовать большую задачу в последовательность соответствующих небольших шагов. Несколько вычислительных блоков одной карты-ускорителя могут одновременно обрабатывать разные блоки.

При вычислении выходного блока размером $m \times n$ одна и та же строка входных данных участвует в вычислении n выходных элементов, а один и тот же столбец весов — в вычислении m выходных элементов. Матричный блок позволяет нескольким блокам умножения-сложения совместно использовать эти операнды: данные передаются между соседними вычислительными блоками, а частичные суммы сохраняются и обновляются локально. Одна инструкция описывает работу над целым блоком, поэтому затраты на управление также распределяются между большим числом операций. В главе 5 будет подробнее показано, как программное обеспечение выбирает размеры блоков и порядок их обхода, чтобы эти небольшие шаги непрерывно получали необходимые данные.

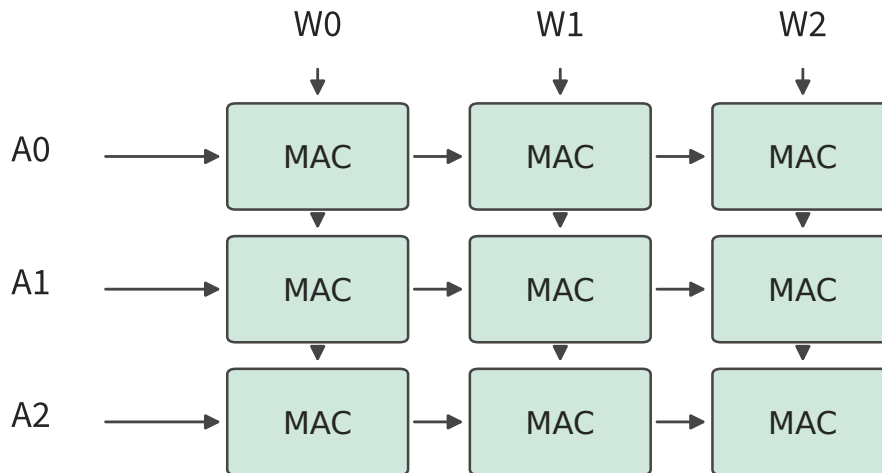
Рассмотрим матричную операцию размером $16 \times 16 \times 16$. Каждый из двух входных блоков содержит 256 элементов, а для выполнения операции требуется $2 \times 16^3 = 8192$ операций с плавающей точкой. Каждый входной элемент участвует внутри блока в 16 умножениях. По сравнению с отдельной загрузкой всех операндов для каждого выходного значения такой подход уменьшает объём повторного перемещения данных и накладные расходы на инструкции.

Матричные инструкции обычно выполняются над блоками фиксированного размера. Пусть минимальный вычислительный блок некоторой реализации имеет размер $m_t \times n_t \times k_t$, а части матрицы, не заполняющие целый блок, дополняются нулями. Доля полезных операций среди фактически выполненных равна

$$\eta_{\text{shape}} = \frac{MNK}{\lceil M/m_t \rceil m_t \lceil N/n_t \rceil n_t \lceil K/k_t \rceil k_t}.$$

Если минимальный размер по строкам равен 16, а реальная матрица содержит только одну строку и выровнена по остальным измерениям, то полезные входные данные находятся лишь в одной из 16 строк. Удвоение числа матричных блоков одинаково ускорит как полезные вычисления, так и вычисления над дополнением нулями. Применение реализации матрично-векторного умножения (GEMV), подходящей для одной строки, позволяет уменьшить объём бесполезных вычислений, вызванных дополнением. Практические измерения того, как библиотеки выбирают реализацию, приведены в разделе 4.8.

Малый массив многократно использует входы и веса



Каждый MAC хранит частичную сумму и добавляет следующую пару входов

Рисунок 4-8. Схема массива умножения-сложения из трёх строк и трёх столбцов. Входные данные передаются вдоль строк, веса — вдоль столбцов, а каждый блок умножения-сложения хранит собственную частичную сумму. Небольшой массив здесь используется для объяснения повторного использования операндов.

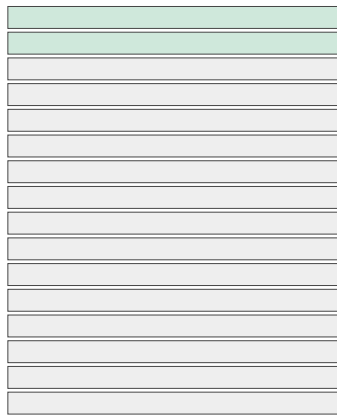
В моделях с экспертами проблема числа строк проявляется ещё сильнее. Пусть каждый из 64 токенов выбирает восемь экспертов, что даёт 512 назначений, а размеры матриц всех экспертов одинаковы. Если назначения распределены точно поровну между 256 экспертами, каждый эксперт обрабатывает векторы признаков лишь двух токенов. Если все токены выбирают одних и тех же восьмерых экспертов, каждый эксперт обрабатывает векторы признаков 64 токенов. В обоих случаях 64 исходных токена создают 512 назначений экспертам и в сумме занимают 512 полезных строк во входных матрицах всех экспертов.

Продолжим использовать реализацию с дополнением числа строк до кратного 16. В первом случае выполняются вычисления для $256 \times 16 = 4096$ строк, из которых полезны 512. Во втором случае обрабатываются $8 \times 64 = 512$ строк, и все они полезны. Объём полезных вычислений в обоих случаях одинаков, но после дополнения объём фактически выполненных вычислений в первом случае в восемь раз больше, чем во втором. То, каких экспертов выбрала группа токенов, определяет не только то, какие веса потребуется прочитать, но и то, сколько строк будет обрабатывать каждый эксперт, а значит, влияет на утилизацию матричных блоков.

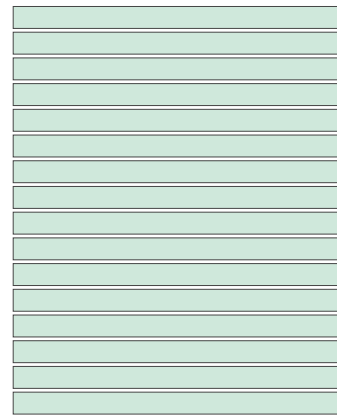
Светло-серые области на рисунках 4-9 и 4-10 обозначают вычисления, вызванные дополнением нулями. Увеличение числа блоков умножения-сложения ускоряет и эти бесполезные операции. Чтобы уменьшить светло-серую область, необходимо изменить число входных строк у каждого эксперта или выбрать меньший вычислительный блок.

Это объясняет, почему библиотеки предоставляют несколько kernel для разных форм: большие матрицы используют повторное использование данных в массиве, малым мат-

В блоке ровно 16 строк, серые тоже
выполняются



2 строки на эксперта



64 строки на эксперта
Состоит из четырёх блоков

Рисунок 4-9. 16-строчный вычислительный блок одного эксперта. Если на каждого эксперта приходится только две строки, остальные четырнадцать заполняются нулями. Если на эксперта приходится 64 строки, из них можно сформировать четыре полных блока; на рисунке показан один из них.

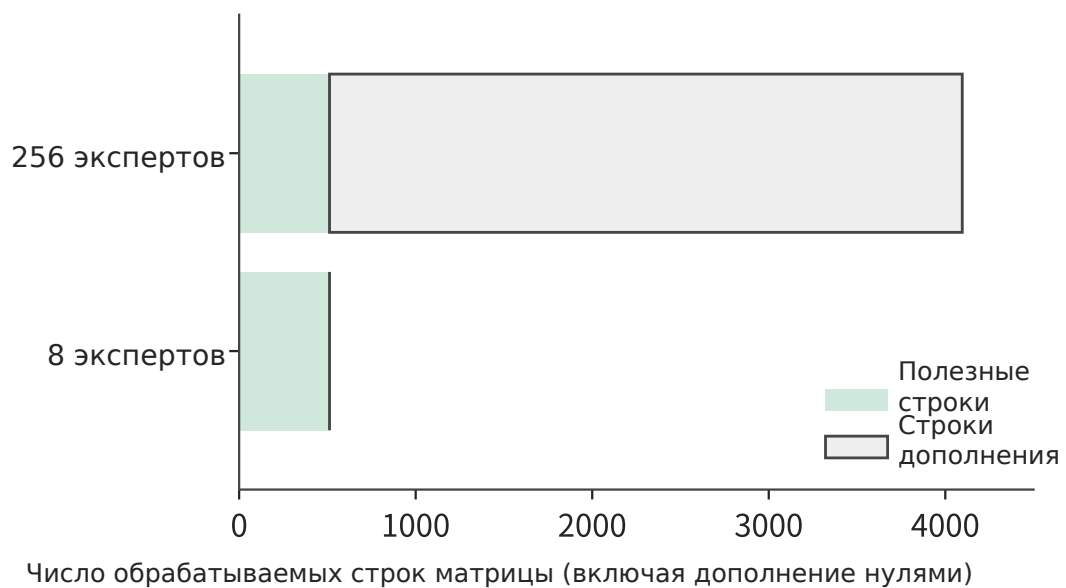


Рисунок 4-10. Объём вычислений для одинаковых 512 полезных входных строк у всех экспертов. После распределения между 256 экспертами в общей сложности обрабатываются 4096 строк, а при концентрации у восьми экспертов — только 512 строк.

рицам нужна более подходящая гранулярность выполнения, а несколько небольших задач можно группировать для сокращения простоев при диспетчеризации. Tensor Core и Ascend Cube предназначены для выполнения регулярных матричных операций, тогда как универсальные вычислительные блоки сохраняют возможность обрабатывать другие формы. Вместе они обрабатывают матрицы разных размеров, встречающиеся в реальной нагрузке.¹²

4.2.2 За какую работу отвечают векторные и управляющие блоки

Помимо матричного умножения, слой Transformer содержит нормализацию, позиционное кодирование и функции активации, а модели с экспертами также выполняют маршрутизацию. Эти этапы включают как поэлементные операции, так и зависимости между несколькими элементами.

Рассмотрим Softmax для строки из n оценок. Для обеспечения численной устойчивости сначала находится максимальное значение, затем вычисляются экспоненты смещённых оценок, после чего выполняются суммирование и нормализация:

$$a = \max_j z_j, \quad e_j = \exp(z_j - a), \quad p_j = \frac{e_j}{\sum_k e_k}.$$

При $n = 128$ необходимо вычислить 128 значений экспоненты и нормализовать 128 результатов, а для нахождения максимума и суммы требуется отдельно объединить 128 входных значений. Поэлементные операции можно распределить между несколькими вычислительными блоками для параллельного выполнения. Операции, объединяющие несколько элементов в меньшее число результатов, называются редукциями; к ним относятся, например, суммирование и нахождение максимума. Редукция требует поэтапного объединения результатов отдельных блоков. Идеальная бинарная редукция выполняется за $\log_2 128 = 7$ раундов, причём каждый раунд ожидает результат предыдущего. Увеличение числа параллельных вычислительных блоков может сократить время обработки каждого раунда, но сами раунды по-прежнему должны выполняться последовательно.

Векторные блоки выполняют такие поэлементные операции и редукции, блоки специальных функций обрабатывают операции наподобие экспоненты, а управляющая логика организует циклы, адресацию, ветвления и синхронизацию. Вместе они определяют, когда результат матричных вычислений можно передать последующим операциям. Названия Ascend Cube, Vector и Scalar наглядно отражают такое разделение. В описанной в техническом документе Ascend 950 подсистеме AI один Cube Core сочетается с двумя Vector Core. Векторная часть использует преимущественно выполнение по схеме «одна инструкция — множество данных» (SIMD, то есть одна инструкция выполняет одну и ту же операцию над группой данных), дополненное схемой «одна инструкция — множество потоков» (SIMT, то есть одна инструкция передаётся группе потоков, каждый из которых имеет собственное состояние выполнения).¹³

Оценки Softmax меняются вместе со входными данными, поэтому экспоненты приходится вычислять заново при каждом запуске. Коэффициенты некоторых других

¹²Техническое описание архитектуры A100, техническое описание архитектуры H100, Hopper Tuning Guide, технический обзор Blackwell и возможности CUTLASS для Blackwell.

¹³Официальное техническое описание архитектуры Ascend 950, §4.1–4.1.6; постраничные ссылки на раннюю архитектуру DaVinci и разделённую архитектуру CANN см. в заметках по сравнению.

нематричных операций зависят только от позиции, поэтому их можно предварительно вычислить и использовать многократно. RoPE использует фиксированные синусные и косинусные коэффициенты позиции и выполняет над парой компонент следующие операции:

$$x'_0 = x_0 \cos \theta - x_1 \sin \theta, \quad x'_1 = x_0 \sin \theta + x_1 \cos \theta.$$

В одном слое Qwen3-8B имеются 32 головы Q и 8 голов K, каждая из которых поворачивает 128 измерений, что даёт $(32 + 8) \times 64 = 2560$ пар компонент. Для каждой пары требуются четыре умножения и два сложения или вычитания, то есть всего 15 360 FLOPs на токен. Все головы в одной позиции используют одинаковые коэффициенты, поэтому могут обращаться к общей таблице: если для каждой из 1 024 позиций сохранить 64 значения $\cos$ и 64 значения $\sin$, то при хранении в BF16 потребуется лишь $1024 \times 128 \times 2 = 256 \text{ KiB}$.¹⁴

Предварительное вычисление заменяет повторное вычисление тригонометрических функций чтением коэффициентов, однако само вращение по-прежнему выполняется для каждого токена. Таким образом, нагрузка переносится с вычисления специальных функций на доступ к таблице и векторные операции.

4.2.3 Как NVIDIA, Ascend и Apple выполняют вычисление внимания

Механизм внимания связывает матричные и векторные вычисления из двух предыдущих разделов: $Z = QK^T$ формирует оценки, $P = \text{Softmax}(Z)$ — вероятности, а $O = PV$ — выходные данные. Один и тот же блок данных должен последовательно пройти эти три этапа, тогда как несколько блоков можно продвигать внахлёт на разных ресурсах.

Работа	NVIDIA GPU	Ascend	Apple GPU
Скалярные произведения запросов и ключей QK, произведения вероятностей и значений PV	Tensor Core; для малых матриц могут использоваться универсальные вычислительные блоки	Матричное ядро Cube (AIC)	Матричные операции Metal GPU
Экспоненты, редукции, масштабирование	Универсальные ресурсы и ресурсы специальных функций SM	Векторное ядро Vector (AIV)	Потоки GPU и соответствующие библиотечные реализации
Промежуточное состояние	Регистры, общая память, хранилище аккумуляторов	Буфер нулевого уровня (L0) рядом с вычислительным блоком, унифицированный буфер и каналы обмена	Буферы GPU и память группы потоков

Это разделение определяет, какие операции могут выполняться одновременно. Например, пока один блок вычисляет экспоненты, матричный блок может обрабатывать другой блок. Однако если обеим сторонам требуется получать операнды через один и тот же внутрикристальный интерфейс, этому интерфейсу всё равно придётся последовательно передать все байты. То, какие блоки способны работать параллельно и какие

¹⁴От утилизации операторов к ожиданию во время выполнения и заметки по чтению статьи.

операции совместно используют один интерфейс, в совокупности определяет пропускную способность устойчиво работающего конвейера. Путь Apple в таблице относится к Metal GPU. Независимый ускоритель нейронных сетей Neural Engine и специализированные блоки внутри GPU отдельно рассматриваются в разделе 4.6.3.¹⁵

Семейство FlashAttention уменьшает обращения к видеопамяти для промежуточных результатов внимания за счёт разбиения на блоки. В четвёртой версии дополнительно исследуется распределение вычислений при использовании более мощных матричных блоков. Ниже на основе анализа одного SM NVIDIA Blackwell B200 в FlashAttention-4 вычисляется влияние такого распределения на пропускную способность. Рассмотрим прямоугольный блок внимания размером 128×128 с размерностью головы 128. Операции QK и PV в сумме требуют около 8,39 MFLOPs, а число результатов экспоненты равно 16 384. При производительности матричных вычислений 8 192 FLOPs на такт на каждый SM и производительности вычисления экспонент 16 результатов на такт матричные операции и вычисление экспонент занимают по 1 024 такта. Чтение матричных операндов из общей памяти составляет 96 KiB и при скорости 128 bytes за такт занимает 768 тактов.¹⁶

При последовательной обработке нескольких блоков матричный блок, интерфейс общей памяти и блок вычисления экспонент могут образовать конвейер. При обработке каждого блока эти ресурсы заняты соответственно 1 024, 768 и 1 024 такта. Если новые блоки поступают быстрее, чем их способен обработать хотя бы один из этих ресурсов, очередь ожидания будет неограниченно расти. Поэтому после выхода конвейера в устойчивый режим интервал между завершением двух соседних блоков составляет как минимум

$$\tau \geq \max(T_{\text{matrix}}, T_{\text{smem}}, T_{\text{exp}}).$$

Пример 4-1. Почему пропускная способность конвейера внимания ограничена вычислением экспонент и общей памятью?

Насколько увеличится пропускная способность конвейера после повышения производительности матричных вычислений и вычисления экспонент? Несколько блоков образуют конвейер между независимыми ресурсами. Сравним две конфигурации: в первой удвоена только производительность матричных вычислений, а во второй дополнительно удвоена производительность вычисления экспонент. Каков для каждой конфигурации минимальный интервал между завершением двух соседних блоков при устойчивой работе конвейера?

Интервал между завершением соседних блоков определяется самым медленным ресурсом. Для исходной конфигурации он равен $\max(1024, 768, 1024) = 1024$ такта. После увеличения только матричной производительности получаем $\max(512, 768, 1024) = 1024$ такта. После дополнительного увеличения производительности вычисления экспонент получаем $\max(512, 768, 512) = 768$ тактов.

После ускорения матричных вычислений узкое место смещается к вычислению экспонент и общей памяти. Первое изменение позволяет матричному блоку закончить работу раньше, но пропускная способность вычис-

¹⁵Официальные характеристики M2 Pro/Max, описание архитектуры Apple GPU, режимы хранения Metal, официальное описание Neural Accelerator в GPU M5.

¹⁶FlashAttention-4, MLSys 2026, статья, §2.2, §3.1.1, формулы 1–3 и таблица 1; независимый пересчёт для одного SM.

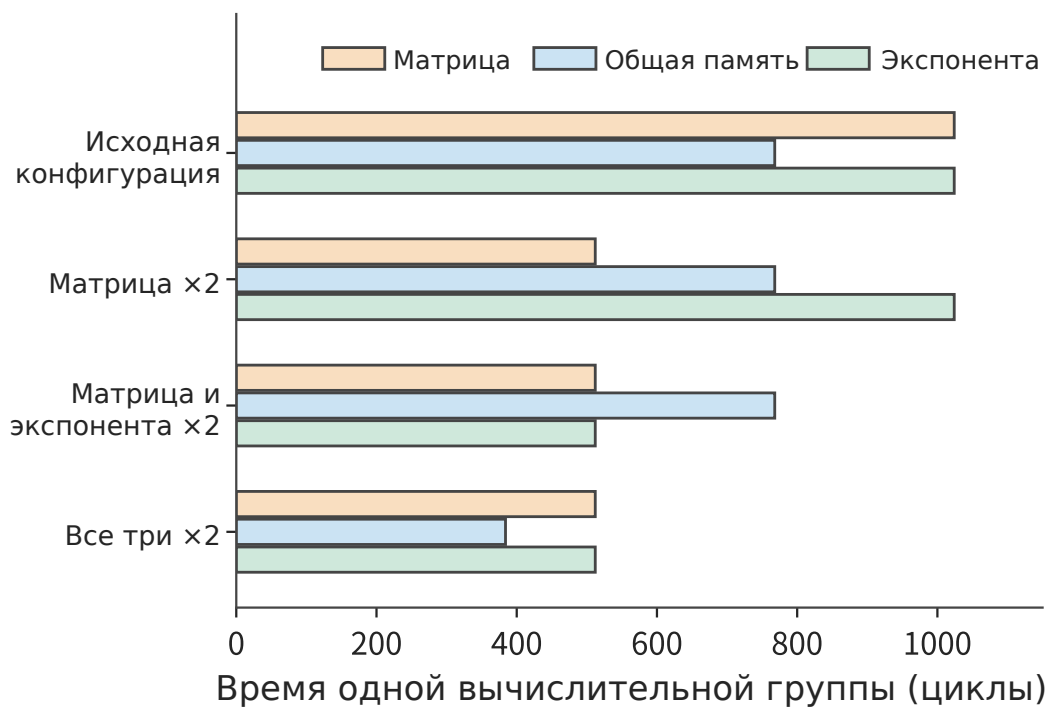


Рисунок 4-11. Число тактов обслуживания одного и того же блока внимания при четырёх конфигурациях ресурсов. Отдельно сравниваются матричные вычисления, общая память и вычисление экспонент. После увеличения производительности одного ресурса более продолжительным этапом может стать другой ресурс. «×2» означает удвоение пропускной способности соответствующего ресурса, а горизонтальная ось показывает число тактов, необходимое для обработки одного и того же вычислительного блока.

ления экспонент остаётся неизменной, поэтому нижняя граница интервала завершения не меняется. Второе изменение устраняет ограничение со стороны вычисления экспонент, и самым продолжительным этапом становится работа с общей памятью. Соответствующая идеальная пропускная способность увеличивается в $1024/768 = 4/3$ раза относительно исходной.

К каким блокам сместится узкое место после удвоения пропускной способности общей памяти? Если также удвоить пропускную способность общей памяти, три значения составят 512, 384 и 512 тактов, и только тогда нижняя граница снизится до 512 тактов. Новая точка равновесия совместно определяется матричными вычислениями и вычислением экспонент.

Повышение производительности одного ресурса сокращает ожидание до тех пор, пока узким местом не станет другой ресурс. Для одного блока операции QK, Softmax и PV по-прежнему выполняются последовательно. Чтобы достичь указанного интервала завершения, необходимо одновременно хранить несколько блоков, находящихся на разных этапах обработки. Требуемые буферы и временная последовательность будут совместно рассчитаны в разделе 4.4.

Эксперимент 4-1 · Основной: как разбиение внимания на блоки изменяет объём вычислений и чтения из общей памяти

Измените размер прямоугольного блока на 128×256 и 256×128 и отдельно вычислите время, необходимое для матричных операций, вычисления экспонент и чтения из общей памяти. Затем поочерёдно увеличивайте производительность матричных вычислений, производительность вычисления экспонент и пропускную способность общей памяти, указывайте величину каждого увеличения и после каждого изменения определяйте самый продолжительный этап. Объясните, как форма блока изменяет повторное использование данных. Наконец, выберите один тип оборудования и обозначьте на схеме выполнения три вида работы и места хранения данных.

4.2.4 Как низкая точность изменяет объём вычислений, данных и затраты на преобразование

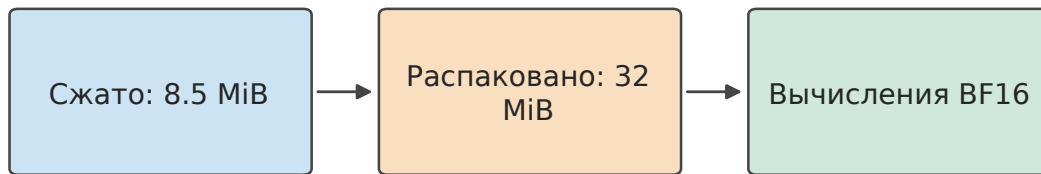
Форма матрицы определяет число операций умножения-сложения, а числовой формат — представление, используемое каждой такой операцией. При анализе низкой точности необходимо последовательно ответить на три вопроса: сколько места занимают данные, какие преобразования выполняются при обработке и какие преобразования допустимы с учётом требований к ошибке.

Сначала рассмотрим представление. Если каждый из n весов хранится в b bit, а каждые g значений совместно используют scale размером s bytes, то при условии, что общее число элементов делится на размер группы без остатка, объём хранения равен

$$V_W = \frac{nb}{8} + \frac{n}{g}s.$$

Для весов размером 4096×4096 из раздела 4.1.1 при хранении каждого значения в 4 bit и использовании одного однобайтового scale на каждые 32 значения данные весов занимают 8 MiB, а scale — 0,5 MiB, всего 8,5 MiB, что примерно в 3,8 раза меньше 32 MiB, необходимых для BF16. Если один scale используется на каждые 16 значений, общий объём увеличивается до 9 MiB. Чем мельче группы, тем точнее scale может соответствовать

Сначала распаковка в высокую точность



Вычисления в низкой точности

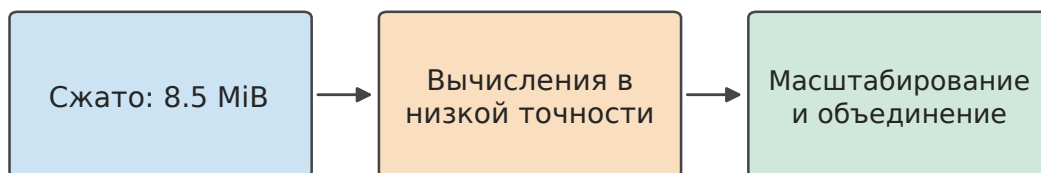


Рисунок 4-12. Два вычислительных пути для одних и тех же сжатых весов. Предварительное развёртывание создаёт копию BF16 размером 32 MiB, а путь низкой точности выполняет масштабирование и объединение в процессе вычислений. Сжатые веса вместе со scale занимают 8,5 MiB.

диапазону значений группы, но тем больше значений scale необходимо хранить.

Теперь рассмотрим выполнение. Сжатое значение q вместе со scale a представляет приближённое значение $\hat{w} = aq$. Его можно сначала развернуть в BF16, а затем выполнить матричное умножение либо обработать q матричной инструкцией низкой точности и после этого скорректировать результат с помощью scale. На рисунке 4-12 эти две реализации показаны рядом: первая сначала затрачивает время на развёртывание весов и сохраняет высокоточную копию, а вторая выполняет масштабирование и объединение непосредственно в процессе вычислений.

Форматы FP4, FP8 и восьмибитный целочисленный формат INT8 уже используются в kernel реальных моделей: одна из реализаций маршрутизируемых экспертов DeepSeek V4-Flash сначала преобразует веса FP4 в FP8, а затем выполняет GEMM. Хранение и чтение весов в FP4 сокращает число передаваемых байтов, после чего матричный блок выполняет вычисления в формате FP8. Если развёрнутые веса используются многократно, время однократного преобразования можно распределить между несколькими вызовами. Если веса разворачиваются заново при каждом вызове, время преобразования становится постоянной частью каждой задачи. Экономия времени благодаря низкой точности определяется разностью между сокращением времени чтения и дополнительным временем преобразования.¹⁷

¹⁷ Аудит аппаратной точности, поэтапные ресурсные условия DeepSeek V4-Flash и Qwen, исследование низкой точности и путей выполнения. Эволюцию аппаратного обеспечения и атрибуцию стоимости см. в исследовании снижения стоимости.

Сопутствующий эксперимент одновременно измеряет ошибку и затраты на выполнение. Для одной экспертной матрицы размером 2048×1536 модели Qwen3-VL-30B-A3B сравниваются два метода: непосредственное блочное вычисление в INT8 и предварительное развёртывание в BF16 с последующим однократным матричным умножением. Эксперимент выполняется на RTX PRO 6000, а входные данные получены из реальной маршрутизации экспертов. L2-норма вектора равна квадратному корню из суммы квадратов его элементов. Локальная относительная ошибка измеряется как отношение L2-нормы ошибки выходных данных к L2-норме эталонных выходных данных, а порог установлен на уровне 2%. При построчном масштабировании активаций ошибка составляет около 2,3–3,5%. Если отдельно масштабировать каждые 128 элементов активаций по измерению K, ошибка снижается примерно до 1,5–1,9%.¹⁸

Более мелкая группировка при квантовании изменяет способ накопления скалярного произведения. Для одной входной строки и одного столбца весов обозначим scale каждой группы активаций как a_g , а scale столбца весов — как b_j . Тогда скалярное произведение можно записать в виде

$$y_j \approx \sum_{g=1}^{16} a_g b_j \left(\sum_{k \in g} q_{x,k} q_{w,kj} \right).$$

Поскольку a_g различается между группами, при вычислении сначала находятся целочисленные частичные суммы каждой группы, затем каждая из них отдельно умножается на scale, после чего результаты складываются. Поэтому такая реализация требует 16 вызовов матричного умножения с преобразованием, масштабированием и объединением между вызовами. При небольшом числе входных строк также требуется дополнить их до минимального числа строк, необходимого kernel. Другой метод сначала разворачивает все группы в BF16, а затем накапливает результат по всему измерению K с помощью одного GEMM.

Для реальных входных данных декодирования одного токена полное время выполнения двух методов составляет соответственно около 2,81 ms и 2,52 ms, поэтому метод с предварительным развёртыванием оказывается быстрее. По объёму видеопамати наблюдается обратная картина: зарегистрированные распределителем памяти пиковые значения при последующих вызовах составляют соответственно около 11,4 MiB и 35,2 MiB, то есть метод прямого вычисления экономит примерно две трети памяти.¹⁹ Более мелкая группировка при квантовании уменьшает ошибку, групповые вызовы увеличивают время, а отказ от высокоточной копии сокращает потребление видеопамати. Таким образом, числовой формат и метод вычисления совместно определяют ошибку, время выполнения и потребление видеопамати.

Следовательно, такую реализацию следует улучшать путём слияния внутригрупповых вычислений, масштабирования и накопления, чтобы одновременно уменьшить ошибку квантования и накладные расходы на запуск kernel.

Эксперимент 4-2 · Расширенный: когда следует разворачивать квантованные веса, чтобы сократить затраты при повторном выполнении

Исходя из приведённой выше формулы группового скалярного произведения, сравните два метода: предварительное развёртывание весов с последующим

¹⁸Эксперимент 4-2, фактические активации маршрутизации, группировка по 128 элементов, замена внутри одноэкспертной модели.

¹⁹Пиковое значение аллокатора в эксперименте 4-2.

вычислением и непосредственное групповое вычисление. Для каждого метода перечислите данные, которые необходимо хранить, включая веса, scale, развёрнутую копию весов, частичные суммы и выходные данные. Вычислите полное время после R повторных использований для трёх стратегий: однократного развёртывания при загрузке, развёртывания при каждом вызове и непосредственного группового вычисления. Затем на основе экспериментальных записей с реальными входными данными сравните ошибку и пиковое потребление видеопамяти и объясните, когда ради сокращения затрат на повторное развёртывание оправдано выделить дополнительную память для хранения развёрнутых весов.

4.3 Иерархия памяти

В разделе 4.2.4 было показано, как числовой формат изменяет объём данных. В этом разделе анализируются чтение и запись данных проекции Q с разграничением трёх запросов: помещаются ли эти данные, с какого уровня они считываются при повторном использовании и достаточно ли независимых обращений для поддержания целевой пропускной способности.

4.3.1 Видеопамять и унифицированная память

Во время работы модели веса, KV-кэш и временная рабочая область одновременно занимают память. Объём весов относительно постоянен, KV-кэш растёт с количеством запросов и длиной контекста, а рабочая область хранит промежуточные результаты выполнения. Только определив эти три величины, можно решить, способен ли ускоритель принять ещё один запрос.

Веса Qwen3-8B в формате BF16 занимают около 16.4 GB. Модель содержит 36 слоёв, в каждом из которых 8 KV-голов размерностью 128; для K и V хранится по одной копии, каждый элемент занимает два байта. Поэтому объём KV-кэша на один токен контекста равен

$$m_{KV} = 36 \times 2 \times 8 \times 128 \times 2 = 147\,456 \text{ bytes} = 144 \text{ KiB}.$$

Если для каждого запроса сохраняется S позиций и одновременно обслуживается B запросов, условие вместимости имеет вид

$$W_{\text{resident}} + B S m_{KV} + M_{\text{workspace}} \leq C_{\text{available}}.$$

Решив его относительно B , получаем максимальное количество запросов при фиксированной длине:

$$B_{\text{max}} = \left\lfloor \frac{C_{\text{available}} - W_{\text{resident}} - M_{\text{workspace}}}{S m_{KV}} \right\rfloor.$$

Примем 24 GB видеопамяти RTX 4090 за доступную ёмкость и выделим 2 GiB под рабочую область. После вычета весов для KV-кэша остаётся около 5.47 GB. При $S = 8192$ каждый запрос требует 1.125 GiB, или около 1.21 GB; KV-кэш четырёх запросов занимает суммарно около 4.83 GB и помещается. После увеличения до пяти запросов потребуется

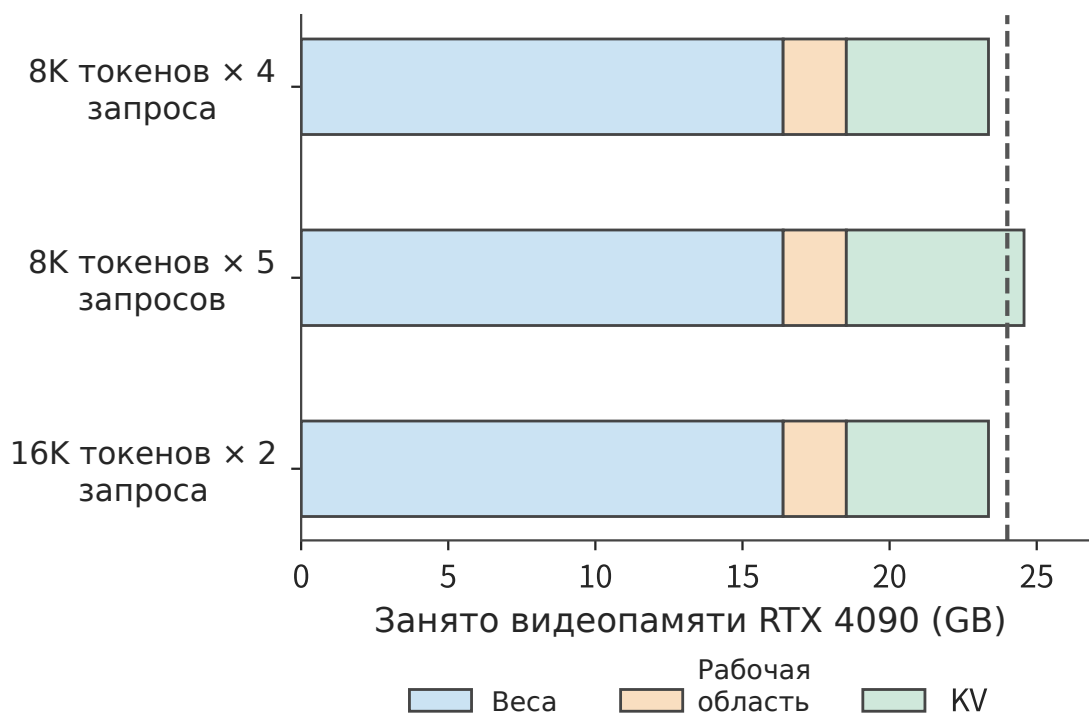


Рисунок 4-13. Веса, рабочая область и KV-кэш в 24 GB видеопамяти RTX 4090. Четыре запроса с контекстом 8K и два запроса с контекстом 16K помещаются, а пять запросов с контекстом 8K превышают обозначенный пунктиром предел ёмкости.

уже около 6.04 GB, что превышает оставшуюся ёмкость. При удвоении контекста объём KV-кэша каждого запроса также удваивается, а максимальное количество запросов уменьшается до двух.²⁰

На рисунке 4-13 ширина областей весов и рабочей памяти остаётся неизменной, а изменения происходят в расположенной справа области KV-кэша. Длинный контекст делает каждый блок KV шире, а увеличение параллелизма повышает количество блоков; и то и другое использует одно и то же оставшееся пространство.

Формула максимального количества запросов с округлением вниз выявляет ступенчатый эффект ёмкости. При небольшом увеличении объёма памяти $B_{\max}$ может остаться неизменным; принять дополнительный запрос можно лишь тогда, когда оставшегося пространства достаточно для всего его KV-кэша. Сжатие весов увеличивает остаток в числителе формулы, поэтому также позволяет вместить больше запросов. Для моделей с большим объёмом весов сокращение фиксированных затрат может заметно повысить параллелизм; в задачах с длинным контекстом KV-кэш уже занимает значительную долю памяти, поэтому выгода от дальнейшего сжатия весов постепенно уменьшается.

Снижение разрядности KV-кэша также высвобождает память. Рассмотренная в главе 2 модель DeepSeek V4.1-Flash одновременно использует глобальную историю и локальное окно SWA. Одна запись основного KV-кэша из 512 значений хранится в формате FP4 и вместе с групповым scale занимает 288 bytes; одна запись локального окна хранится в формате FP8 и вместе со scale занимает 528 bytes. Перед участием основного KV-кэша в умножении для механизма внимания выполняется обратное квантование, то есть значения с помощью scale восстанавливаются в числовом формате, используемом для вы-

²⁰Конфигурация Qwen3-8B; полные результаты по поколениям памяти.

числений. Формат хранения сжимает резидентное состояние, а во время выполнения операнды восстанавливаются.²¹

Выше учитывалось только состояние во время инференса. При обучении дополнительно используются градиенты, состояния оптимизатора и активации, сохранённые для обратного распространения. Объём этих состояний рассчитывается с учётом жизненного цикла тензоров, описанного в главе 3: все тензоры, которые требуется сохранять в конкретный момент, совместно определяют пиковое потребление памяти, а освобождённые промежуточные результаты предоставляют место для последующей работы. Поэтому планирование ёмкости зависит как от размера каждого тензора, так и от порядка выполнения.

Ёмкость видеопамати в конечном счёте обеспечивается устройствами памяти и интерфейсами. HBM обеспечивает пропускную способность благодаря многослойному размещению кристаллов памяти и широкому интерфейсу; графическая память с удвоенной скоростью передачи данных (GDDR) обычно формирует видеопамать из нескольких микросхем памяти, соединённых высокоскоростным интерфейсом. И HBM, и GDDR представляют собой технологии устройств памяти и интерфейсов, тогда как унифицированная память — это способ организации памяти, совместно используемой процессорами. CPU и GPU Apple обращаются к общей физической памяти и могут последовательно использовать один и тот же буфер, уменьшая количество отдельных копий; при этом CPU, GPU и другие приложения также совместно потребляют её ёмкость и пропускную способность.²² В разделе 4.5 влияние такой организации на время передачи будет проанализировано по пути от хоста к ускорителю.

Вопрос для размышления: можно ли принять ещё один запрос после сжатия KV-кэша? Если потребность каждого запроса в KV-кэше уменьшится на 10% с исходных 1.21 GB, сможет ли RTX 4090 вместить пятый запрос? Сначала рассчитайте суммарную потребность пяти запросов, затем сравните её с оставшимся пространством.

4.3.2 Кэши, внутрикристалльные буферы и регистры

При вычислении блока выходных данных одно и то же число из входных данных часто должно участвовать и в вычислении соседних выходных значений, а ещё не завершённые частичные суммы необходимо продолжать накапливать. Если при каждой операции умножения со сложением обращаться к внешней памяти за данными и записывать результат обратно, возникнет множество передач в обоих направлениях. Хранение часто используемых данных рядом с вычислительным блоком, подобно размещению нужных материалов на рабочем столе под рукой, позволяет сократить эти перемещения, однако ближайшее пространство ограничено.

Регистры хранят значения, используемые потоком и инструкцией, кэш автоматически сохраняет недавно запрошенные данные, а загрузкой и освобождением программно управляемого буфера явно распоряжается программа. Все три типа памяти могут хранить многократно используемые данные, но отличаются ёмкостью, способом доступа и механизмом управления. Кроме того, внутрикристалльная память занимает площадь

²¹Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и пересчёт сквозного межглавного примера.

²²Официальные характеристики M2 Pro/Max, описание архитектуры Apple GPU, режимы хранения Metal, официальное описание Neural Accelerator в GPU M5.

кристалла, поэтому её ёмкость обычно намного меньше ёмкости внешней памяти. Ниже проследим за двумя входными блоками и одной частичной суммой, рассчитав требуемое пространство.

Рассмотрим блок умножения со сложением с $m = n = 128$, $k = 64$. Входной блок и блок весов в формате BF16 занимают по 16 KiB, суммарно 32 KiB; выходной аккумулятор FP32 занимает $128 \times 128 \times 4 = 64$ KiB. В каждом SM V100 под разделяемую память можно выделить не более 96 KB, то есть 96 KiB; если поместить туда и операнды, и аккумулятор, она будет заполнена полностью.²³ Чтобы одновременно подготовить следующую пару операндов, потребуется $2 \times 32 + 64 = 128$ KiB.

Если для аккумулятора используется отдельное хранилище, те же 96 KiB смогут вместить три набора парных входных блоков и блоков весов. Общая ёмкость локального буфера не увеличилась, но разделение функций хранилищ изменило количество входных блоков, которые можно подготовить заранее. В разделе 4.6.1 такое разделение будет показано на примере специализированного хранилища аккумуляторов Blackwell.

В предыдущем примере частичная сумма вычислительного блока оставалась на кристалле, что исключало её многократную обратную запись. Этот же подход применим и к входным данным, совместно используемым несколькими вычислительными блоками. Предположим, что четырём выходным блокам требуется прочесть одни и те же входные данные объёмом 32 KiB. Суммарно они формируют логические запросы на 128 KiB. L2 — это кэш второго уровня, общий для вычислительных блоков. Если при первом чтении данные останутся в L2, а следующие три обращения попадут в кэш, внешняя память должна будет предоставить только 32 KiB, но L2 всё равно потребуется передать блокам все 128 KiB. Если четыре выходных блока последовательно обрабатываются одной вычислительной группой, а входные данные сохраняются в локальном буфере, последующие обращения смогут получать данные непосредственно из него.

Поэтому при анализе одних и тех же данных необходимо различать три величины: пространство, занимаемое самими данными, количество байтов, запрошенных вычислительными блоками для чтения, и количество байтов, фактически переданных через интерфейс конкретного уровня. Арифметическая интенсивность также изменяется в зависимости от уровня наблюдения. Интенсивность, рассчитанную по объёму запросов к L2, следует использовать вместе с пропускной способностью L2, а интенсивность, рассчитанную по трафику DRAM, — с пропускной способностью внешней памяти. В разделе 4.8 это различие будет показано на фактических показаниях счётчиков.

Ещё одна функция L2 определяется его расположением. SM GPU объединяются в кластеры графической обработки (Graphics Processing Cluster, GPC); L2 разделён на несколько секций, каждая из которых отвечает за свой диапазон адресов и соединена со всеми SM внутрикристалльной коммутационной матрицей (crossbar). RTX PRO 6000 содержит 188 SM, а общий объём L2 составляет 128 MiB. Регистры, разделяемая память и кэш первого уровня принадлежат отдельным SM. Начиная с Hopper несколько SM внутри одного GPC могут образовывать **кластер блоков потоков (thread block cluster)**, в котором SM способны напрямую читать и записывать разделяемую память друг друга, то есть использовать распределённую разделяемую память. За пределами такого кластера общей памяти между SM нет: чтобы передать результат от одного SM другому, его необходимо сначала записать в секцию L2, после чего другой SM должен прочесть его оттуда (рисунок 4-14). Таким образом, L2 служит как входом для поступающих на кристалл данных

²³Техническое описание архитектуры Volta, разделы о Tensor Cores и смешанной точности.

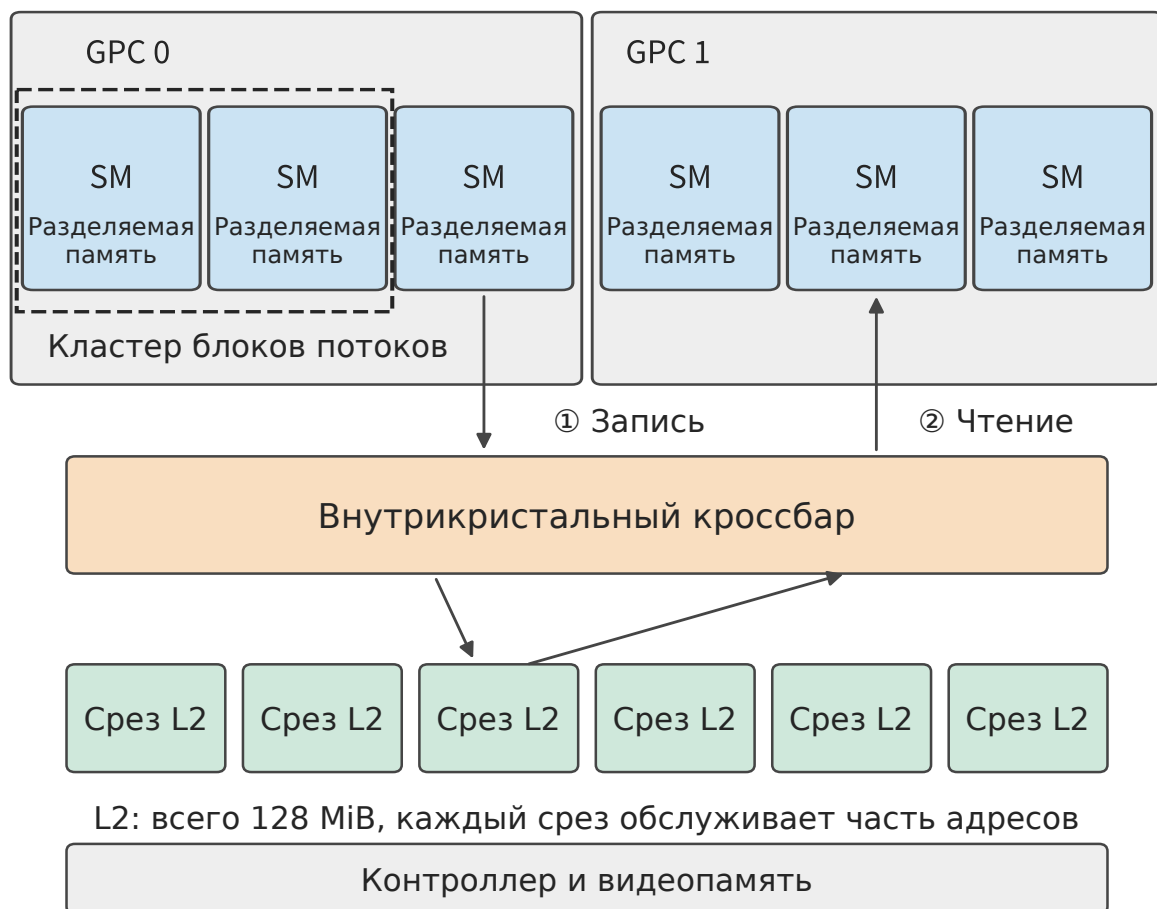


Рисунок 4-14. Внутрикристалльная память и пути передачи данных GPU. SM внутри пунктирной рамки образуют кластер блоков потоков и могут напрямую обращаться к разделяемой памяти друг друга; данные между остальными SM через коммутационную матрицу записываются в секцию L2 (①), а затем считываются получателем (②). Количество SM, GPC и секций на схеме приведено только для иллюстрации.

из внешней памяти, так и единственным путём обмена данными между SM, находящимися за пределами одного кластера.

Увеличение вычислительного блока обычно повышает повторное использование данных, но также требует более крупного буфера. RTX 5090 и RTX PRO 6000 используют архитектуру SM120. В каждом SM суммарный объём кэша данных первого уровня и разделяемой памяти составляет 128 KB, из которых не более 100 KB можно выделить под разделяемую память, совместно используемую вычислительными группами, резидентными на этом SM: если каждой группе требуется 32 KiB, резидентными могут быть три группы; после увеличения потребности каждой группы до 64 KiB останется только одна.²⁴ Пока одна группа ожидает данные, другая может продолжать вычисления. Поэтому с уменьшением количества одновременно работающих групп вычислительные блоки чаще простаивают. При выборе размера блока необходимо одновременно учитывать, насколько каждая группа сокращает повторные чтения и

²⁴Blackwell Tuning Guide, описание возможностей CUTLASS для Blackwell и пример SM100 TMEM; сведения о 128 KB кэша данных первого уровня и предельных 100 KB общей памяти на каждый SM в SM120 (вычислительная возможность 12.0) см. в таблице вычислительных возможностей CUDA Programming Guide.

сколько независимых вычислительных групп всё ещё можно разместить.

Ёмкость внутрикристалльных буферов также увеличивается с каждым поколением. Предельный объём разделяемой памяти на один SM составляет 164 KB у A100 и 228 KB у H100.²⁵ Большее количество вычислительных групп, резидентных на кристалле, также позволяет отправлять больше независимых запросов на чтение.

4.3.3 Что именно ограничивают ёмкость, пропускная способность и задержка

Ёмкость определяет, сколько данных можно вместить в конкретный момент, пропускная способность — сколько данных можно передать за единицу времени, а задержка — сколько времени проходит от отправки одного запроса до его завершения. Если одновременно отправлять достаточно независимых запросов, во время ожидания результата одного обращения можно продолжать обработку других.

Размещение модели и её пошаговое выполнение предъявляют разные требования к памяти. Согласно таблице конфигураций HBM из раздела 4.1.3, H100 SXM имеет 80 GB памяти с пропускной способностью 3.35 TB/s, а H200 SXM — 141 GB и 4.8 TB/s; ёмкость и пропускная способность увеличились в разной степени. В одной из 4-битных конфигураций Qwen3-235B-A22B веса и scale занимают около 123.1 GB, KV-кэш для восьми запросов по 8,192 токена — около 12.6 GB, а с добавлением 2 GiB рабочей области общий объём составляет около 137.9 GB. H100 не хватает почти 58 GB, тогда как у H200 остаётся около 3.1 GB. Увеличение ёмкости прежде всего позволяет всем этим данным одновременно оставаться резидентными.²⁶

Веса всех экспертов хранятся в памяти, а то, какие именно веса требуется считывать на каждом шаге, определяется результатом маршрутизации. Если выбор экспертов для восьми запросов различается, суммарный объём обращений к весам, scale и KV-кэшу на каждом шаге составляет около 75.967 GB; если запросы выбирают одних и тех же экспертов, он снижается примерно до 24.737 GB. На H200 одна только передача этих байтов занимает соответственно около 15.8 ms и 5.2 ms; при использовании HGX B200 с пропускной способностью 8 TB/s — около 9.5 ms и 3.1 ms. Повторное использование экспертов сокращает трафик примерно до одной трети, а замена ускорителя уменьшает время передачи одного байта примерно до 60% исходного значения. Эти два изменения воздействуют на разные части формулы $T = V/R$.

Приведённый выше расчёт V/R предполагает, что интерфейс способен непрерывно обеспечивать заданную пропускную способность. Для поддержания такой скорости необходимо продолжать отправлять другие запросы, пока ожидается ответ на одно обращение. Назовём транзакцией памяти базовую единицу, передаваемую интерфейсом памяти. Пусть интерфейс допускает не более N_o одновременно выполняющихся транзакций, каждая транзакция возвращает s bytes, а средняя задержка завершения равна L . Одна транзакция занимает одно место от отправки до возврата; за секунду может завершиться не более приблизительно N_o/L транзакций, поэтому

$$R_{\text{effective}} \leq \min \left(R_{\text{interface}}, \frac{N_o s}{L} \right).$$

Решив выражение относительно N_o , получаем количество одновременно выполняющихся транзакций, необходимое для поддержания целевой пропускной способности:

²⁵Понимание эволюции чипов через изменение нагрузки.

²⁶Результаты сравнения поколений памяти и описание расчётов.

Одновременно ожидающие ответа запросы занимают слоты



128 bytes на запрос, ответ через 500 ns

Устойчивая пропускная способность зависит и от числа одновременных запросов

Рисунок 4-15. Запись о запросе на чтение остаётся занятой от его отправки до получения ответа. Только перекрытие нескольких независимых запросов позволяет непрерывно использовать интерфейс во время ожидания отдельного обращения; на схеме показаны лишь четыре характерных запроса.

$$N_o \geq \left\lceil \frac{R_{\text{target}} L}{s} \right\rceil.$$

Это закон Литтла: среднее количество одновременно выполняющихся запросов равно произведению скорости завершения запросов на их среднюю задержку. Аппаратные очереди определяют, сколько таких транзакций можно вместить, а взаимная независимость обращений в программе — можно ли заполнить очередь. Если следующий адрес зависит от результата предыдущего чтения, заполнить очередь не удастся независимо от её глубины; несколько независимых вычислительных блоков, напротив, могут формировать запросы на чтение, не ожидающие друг друга.

Каждое обращение в цепочке зависимостей несёт полную задержку. Именно так измеряется и задержка при попадании в L2: один поток многократно выполняет чтение, причём адрес каждого обращения берётся из значения, полученного предыдущим чтением, а все данные резидентны в L2. На RTX PRO 6000 один полный цикл обращения к L2 занимает около 340–390 тактов, что при измеренной частоте SM 2.88 GHz соответствует приблизительно 120–135 ns. Хотя это меньше задержки обращения к видеопамяти, остановка всё равно длится сотни тактов. Задержка также зависит от секции, в которой находятся данные: одна и та же программа в разных процессах показала 354–870 тактов. Каждая передача между SM состоит из нескольких таких полных циклов, как показано в разделе 4.4.4.²⁷

²⁷Измерения автора на RTX PRO 6000 Blackwell Workstation (GB202, 188 SM, 128 MiB L2, измеренная частота SM 2,88 GHz): описание измерений и проверка методики, выдержка числовых значений, использованных в книге. Программа измерения — `tools/gpu_microbench/` из репозитория OpenTallas, CUDA 12.8. Во время измерений GPU использовался совместно с другой нагрузкой; в основном тексте взято минимальное значение из нескольких запусков, медиана выше на 10%–40%.

Пример 4-2. Ускорится ли чтение KV-кэша пропорционально после установки видеокарты с большей пропускной способностью?

Сравним ограничения чтения KV-кэша, обусловленные количеством одновременно выполняющихся транзакций и пропускной способностью интерфейса. KV-кэш, соответствующий 8,192 токенам контекста одного запроса, занимает 1.125 GiB. Возьмём интерфейс видеопамяти RTX 4090 с пропускной способностью 1,008 GB/s, размер транзакции 128 bytes и задержку завершения 500 ns: средство измерения производительности памяти Mess показало на H100 задержку незагруженной видеопамяти 363 ns, которая при приближении к предельной пропускной способности возрастала до 699–1,433 ns; значение 500 ns находится между ними.²⁸ Рассчитаем эффективную пропускную способность чтения и его продолжительность при максимально допустимых 128 и 4,096 одновременно выполняющихся транзакциях, а затем сравним с результатами после перехода на RTX 5090 с пропускной способностью 1,792 GB/s.

Рассчитаем эффективную пропускную способность чтения по количеству байтов в одновременно выполняющихся транзакциях и задержке доступа. Для поддержания 1,008 GB/s требуется $\lceil 1.008 \times 10^{12} \times 500 \times 10^{-9} / 128 \rceil = 3938$ транзакций. 128 транзакций могут обеспечить лишь около 32.8 GB/s, поэтому чтение займёт не менее 36.9 ms; 4,096 транзакций достаточно для поддержания 1,008 GB/s, и время сокращается примерно до 1.20 ms.

При недостаточном количестве одновременно выполняющихся транзакций польза от более высокой пропускной способности интерфейса ограничена. После перехода на RTX 5090 4,096 транзакций по-прежнему поддерживают не более примерно $4096 \times 128 / (500 \text{ ns}) = 1.05 \text{ TB/s}$, а время составляет около 1.15 ms. Пропускная способность интерфейса увеличилась примерно на 78%, но пропускная способность чтения ограничена количеством одновременно выполняющихся транзакций и возросла лишь приблизительно на 4%.

Сколько одновременно выполняющихся транзакций требуется для полного использования более высокой пропускной способности или компенсации большей задержки? Для поддержания 1,792 GB/s их количество необходимо увеличить как минимум до 7,000; если задержка возрастёт до 800 ns, потребуется уже 11,200 транзакций.²⁹

При малом параллелизме две кривые на рисунке 4-16 совпадают, поскольку ограничение определяется количеством байтов в одновременно выполняющихся транзакциях. После прохождения точки перегиба каждой кривой новым узким местом становится пропускная способность интерфейса. В реальной системе повышение параллелизма обращений также удлиняет очередь, а задержка изменяется с нагрузкой; Mess одновременно измеряет обе величины с помощью зонда задержки и регулируемого фонового трафика, получая кривую зависимости задержки от пропускной способности.³⁰ Эта кри-

²⁸Mess, MICRO 2024, принятая авторская версия, выборочно физические страницы 3–6; пример расчёта параллелизма доступа к памяти и его ограничения.

²⁹RTX 4090, 128 незавершённых транзакций, RTX 4090, 4 096 транзакций, RTX 5090, 4 096 транзакций, RTX 5090, задержка 800 ns; задержку H100 в Mess см. в таблице I и на рисунке 3(h).

³⁰Mess, MICRO 2024, принятая авторская версия, выборочно физические страницы 3–6; пример расчёта параллелизма доступа к памяти и его ограничения.

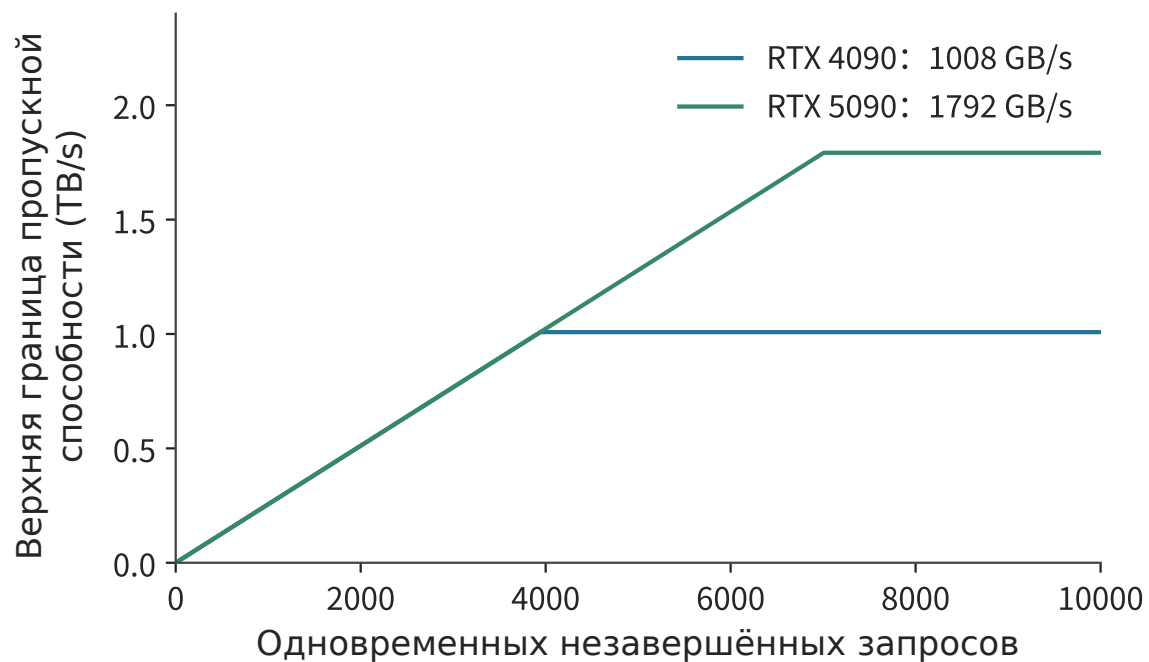


Рисунок 4-16. При размере транзакции 128 bytes и задержке возврата 500 ns увеличение количества одновременно выполняющихся запросов повышает верхнюю границу пропускной способности, пока она не достигнет предела самого интерфейса видеопамати RTX 4090 или RTX 5090.

вая объединяет на одном графике два эффекта: «одновременная отправка независимых запросов уменьшает простой во время ожидания» и «слишком большое количество запросов увеличивает время ожидания в очереди».

При анализе производительности памяти сначала по ёмкости определяют, могут ли данные оставаться резидентными, затем с учётом повторного использования рассчитывают трафик интерфейсов каждого уровня и, наконец, по уровню параллелизма и задержке оценивают, способен ли интерфейс непрерывно передавать данные.

Эксперимент 4-3 · Расширение: какие узкие места decode устраняются увеличением ёмкости памяти, а какие — повышением пропускной способности

Используя результаты расчётов памяти для Qwen3-8B и Qwen3-235B-A22B, по отдельности увеличьте только ёмкость и только пропускную способность, а затем измените batch и распределение экспертов. Сначала определите максимальное количество резидентных запросов, затем рассчитайте время чтения на каждом шаге и, наконец, количество одновременно выполняющихся транзакций, необходимое для поддержания целевой пропускной способности. Укажите, чем прежде всего ограничивается каждый вариант: ёмкостью, временем передачи или параллелизмом обращений к памяти.

4.4 Перемещение данных

В разделах 4.1–4.3 были рассчитаны объём данных и количество их повторных использований, а также объяснена степень параллелизма, необходимая для поддержания про-

пускной способности. В этом разделе дополнительно определяется, когда эти данные следует получать, использовать и освобождать. Главная задача перемещения данных — доставить следующий блок до того, как он понадобится вычислительному блоку, и при этом сохранять текущий блок до завершения его последнего использования.

4.4.1 Размещение данных и адресация

Если выделить на матрице небольшой квадрат, это определит лишь то, какие элементы нужно получить. В памяти эти элементы необязательно расположены рядом: если исходная матрица хранится построчно, после чтения одной строки квадрата нужно пропустить невыбранные элементы и продолжить чтение с соответствующей позиции следующей строки исходной матрицы. Математическое разбиение на блоки может изменять только индексы и диапазон доступа, не требуя предварительного копирования отдельной малой матрицы. Аппаратное обеспечение должно находить эти элементы по шагам. Для двумерного представления, в котором один элемент занимает b bytes, шаг строки равен s_r bytes, а шаг столбца — s_c bytes, адрес элемента определяется как

$$\text{addr}(i, j) = \text{base} + i s_r + j s_c.$$

При непрерывном построчном хранении матрицы $s_c = b$, а шаг строки определяется полной шириной строки. При извлечении подматрицы каждая её строка занимает лишь часть строки исходной матрицы, но расстояние между началами соседних строк по-прежнему равно шагу строки исходной матрицы. От этого зависит, какие обращения должно выполнить аппаратное обеспечение.

Например, возьмём из BF16-матрицы шириной 4 096 элементов 128 строк по 128 последовательных элементов в каждой. Объём полезных данных составляет $128 \times 128 \times 2 = 32$ KiB, каждая строка занимает лишь 256 bytes, а начала соседних строк находятся на расстоянии 8 192 bytes. Диапазон от начала первой строки до конца последней охватывает $127 \times 8192 + 256 = 1\,040\,640$ bytes. При построчном чтении передаются только полезные участки каждой строки; если соседние потоки обращаются к соседним элементам, их запросы также можно объединить в меньшее число транзакций.

Синяя область в каждой строке занимает лишь 256 bytes, но синяя область следующей строки начинается только через 8 192 bytes. Чтение всего диапазона адресов привело бы к перемещению и серых промежутков, тогда как построчное чтение получает только синие части. В этом и заключается назначение шага строки: он указывает блоку перемещения, где начинается следующий участок полезных данных.

Размещение данных влияет на стоимость доступа с двух сторон: шаг влияет на вычисление адресов, а непрерывность адресов — на возможность объединить несколько обращений в одну транзакцию. Одни и те же 32 KiB при одном размещении можно получить регулярными последовательными запросами, а при другом потребуются множество коротких запросов. Число bytes на транзакцию s из раздела 4.3 как раз зависит от того, как объединяются эти обращения.

Разбиение на блоки также определяет, сколько раз будет запрошен один и тот же адрес. Одно вычисление $w1$ общего эксперта DeepSeek V4-Flash имеет вид $[32, 4096] \times [4096, 2048]$, а FP8-вход занимает всего 128 KiB. Выходная ширина разбивается на 16 блоков по 128 столбцов, и каждый выходной блок должен полностью пройти по входу, поэтому суммарный объём запросов входных данных на уровне блоков равен 16×128

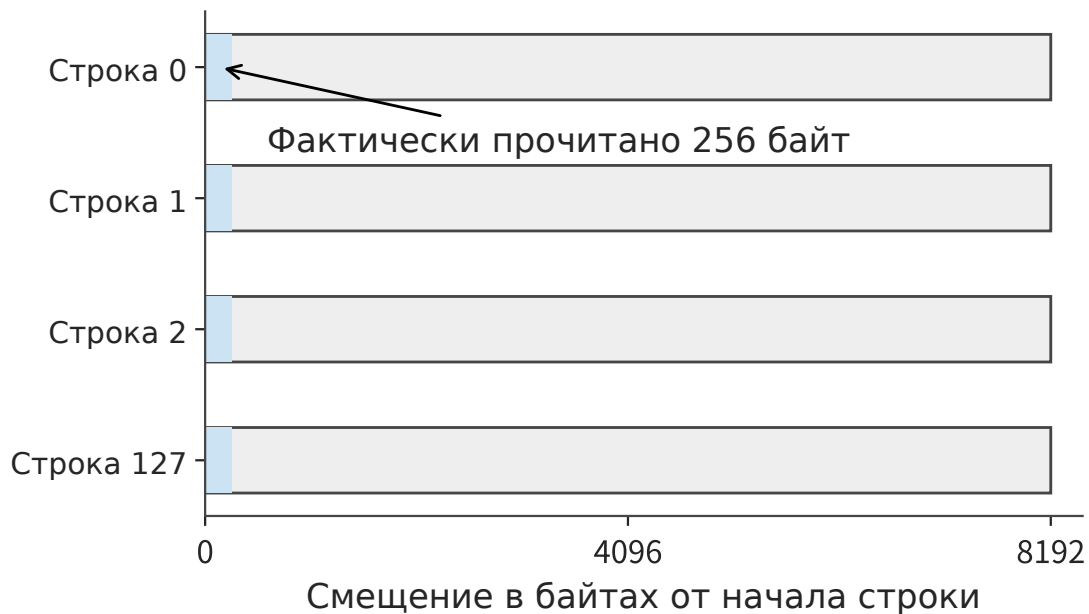


Рисунок 4-17. Первые 256 bytes каждой строки представляют фактически читаемый участок, а начала соседних строк находятся на расстоянии 8192 bytes. Для 128 строк суммарно считывается 32 KiB, а серые области между строками пропускаются благодаря шагу.

KiB, то есть 2 MiB. Веса разделяются по выходным столбцам, каждый блок считывает собственную часть, поэтому суммарный объём данных весов составляет 8 MiB.³¹

Размер входа по-прежнему равен 128 KiB, а дополнительные запросы возникают из-за разбиения по направлению выходных столбцов. Если выходные блоки используют общий кэш, повторные запросы могут получать данные непосредственно из внутрикристального кэша; если одна вычислительная группа сохраняет вход и последовательно обрабатывает несколько выходных блоков, повторное использование можно перенести в локальный буфер. Здесь соединяются адресация, разбиение на блоки и иерархия памяти: способ разбиения определяет, какие запросы чтения будут отправлены, а расположение данных — через интерфейс какого уровня пройдут эти запросы.

Обычные инструкции загрузки могут последовательно вычислять адреса и выполнять обращения; специализированный блок перемещения получает форму тензора, шаги и начальную точку, после чего многократно генерирует эти адреса. Набор параметров, описывающих задачу перемещения, называется дескриптором. Ускоритель тензорной памяти TMA архитектуры NVIDIA Hopper, к которой относится H100, и блок многомерного прямого доступа к памяти NDDMA в Ascend 950 поддерживают многомерное перемещение тензоров.³² Для описанной выше подматрицы достаточно указать начальную точку, 128 строк, 256 bytes в каждой строке и шаг строки 8192 bytes. Когда повторяющиеся вычисления адресов выполняет специализированный блок, вычислительные потоки могут продолжать другую работу.

³¹Результаты координат перемещения общих экспертов DeepSeek V4-Flash и описание подсчёта на уровне исходного кода.

³²Hopper Tuning Guide, техническое описание Ascend 950, официальное описание архитектуры Rubin.

4.4.2 Асинхронное перемещение и двойная буферизация

После нахождения и получения блока данных необходимо обеспечить своевременное прибытие следующего блока: если начинать чтение только после завершения вычислений над текущим блоком, вычислительный блок будет постоянно простаивать. При наличии двух буферов можно использовать данные из первого, одновременно загружая следующий блок во второй; после завершения работы с текущим блоком буферы меняются ролями. Это и есть двойная буферизация: дополнительное пространство обменивается на перекрытие загрузки и вычислений во времени.

Пусть имеется n блоков данных, загрузка каждого занимает t_l , а вычисление — t_c . Если получать один блок, полностью обрабатывать его и лишь затем переходить к следующему, общее время составит $n(t_l + t_c)$. Если загрузка и вычисление используют независимые ресурсы, при двойной буферизации сначала загружается и вычисляется первый блок, а последующие блоки завершаются в темпе более медленного этапа:

$$T_{\text{pipe}} = t_l + t_c + (n - 1) \max(t_l, t_c).$$

Если загрузка каждого из восьми блоков занимает $2 \mu\text{s}$, а вычисление — $1 \mu\text{s}$, последовательное выполнение потребует $24 \mu\text{s}$, а конвейерное — $2 + 1 + 7 \times 2 = 17 \mu\text{s}$. Экономия $7 \mu\text{s}$ достигается за счёт перекрытия, при этом число переданных bytes и операций умножения со сложением не меняется. Если затем вдвое ускорить вычисления, общее время сократится лишь до $16,5 \mu\text{s}$, поскольку загрузка по-прежнему может предоставлять только один блок каждые две микросекунды.

В приведённом выше анализе загрузка рассматривается как единый этап. Далее мы разделим её на передачу и ожидание ответа, чтобы определить, сколько буферов требуется при одновременном выполнении нескольких запросов. Здесь tick обозначает один такт SM B200; при этом различаются **интервал запуска** и **задержка завершения**: если запрос чтения отправляется каждые 64 tick, а данные каждого блока становятся доступны только через 192 tick после отправки запроса, несколько блоков будут одновременно ожидать ответа. До начала вычисления входные данные уже должны быть получены. Назовём буфер для хранения одного входного блока входным слотом: с момента отправки запроса чтения и до завершения вычисления этого блока слот остаётся занятым. Дополнительные слоты позволяют раньше запускать чтение последующих данных, но требуют больше локальной памяти.

Конкретно выведем это для вычисления QK в механизме внимания. Оба входа имеют размер 128×128 и разбиваются по размерности головы на четыре накапливаемых блока с $k = 32$. Общий объём входных данных Q/K каждого блока составляет 16 KiB, объём вычислений — 1 048 576 FLOPs; четыре вычисления последовательно обновляют один и тот же результат FP32, который хранится в отдельной памяти накопления объёмом 64 KiB. Скорость матричных вычислений принимается равной 8 192 FLOPs за такт на один SM B200 согласно статье FlashAttention-4; скорость передачи входных данных — 256 bytes/tick, после завершения передачи требуется ещё 128 tick ожидания, прежде чем данные станут доступны.³³³⁴

³³Базовый конвейер входа механизма внимания Qwen, удвоение скорости матричных операций, моделирование и независимая проверка.

³⁴FlashAttention-4, MLSys 2026, статья, §2.2, §3.1.1, формулы 1–3 и таблица 1; независимый пересчёт для одного SM.

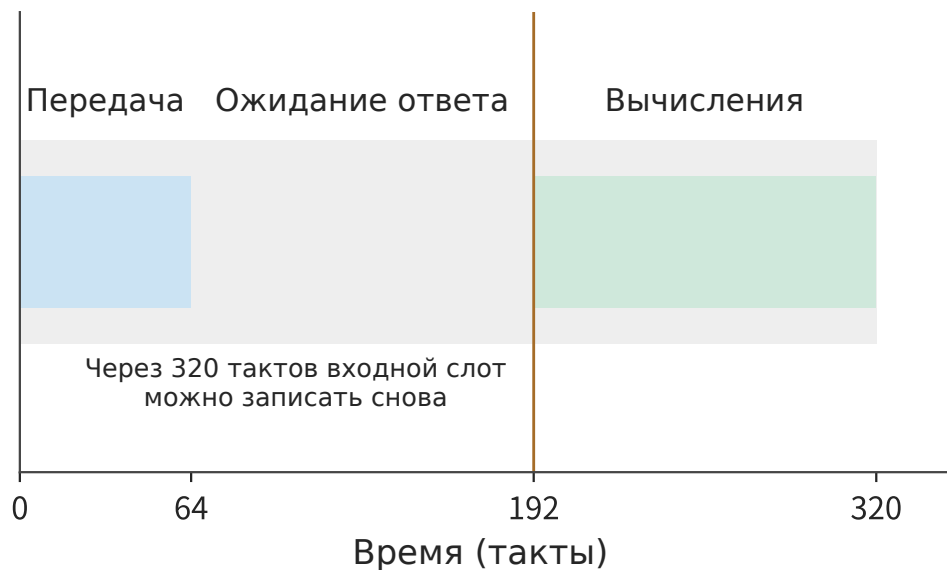


Рисунок 4-18. Полный жизненный цикл одного входного слота от запуска до освобождения. Передача занимает 64 tick, дополнительное ожидание — 128 tick, данные готовы через 192 tick, затем вычисление занимает 128 tick, и на отметке 320 tick слот освобождается. Один tick соответствует одному такту SM B200.

Таким образом, передача входных данных каждого блока занимает 64 tick, данные становятся готовы через 192 tick после запуска, а вычисление требует ещё 128 tick.

При наличии только одного входного слота он проходит загрузку, ожидание и вычисление и может быть использован повторно лишь через $64 + 128 + 128 = 320$ tick; четыре блока завершаются за 1280 tick.

На рисунках 4-19–4-21 последовательно показано выполнение с одним, двумя и тремя слотами при одинаковом масштабе времени. Если сначала находить свободные промежутки между зелёными отрезками вычислений, а затем смотреть выше, когда будут готовы входные данные следующего блока, можно определить источник ожидания.

Два слота позволяют отправить запросы чтения первых двух блоков в моменты 0 и 64. Вычисление первого блока начинается в момент 192 и завершается в момент 320, после чего освободившийся слот используется для третьего блока, который будет готов в момент 512; вычисление второго блока уже завершится в момент 448, поэтому между ними останется 64 tick простоя. При продолжении в том же порядке четвёртый блок завершится в момент 768. Второй слот позволяет частично перекрыть загрузку данных и вычисления, но матричный блок по-прежнему простаивает между вычислениями блоков.

Для непрерывного вычисления четырёх блоков минимальное время завершения равно ожиданию первого блока плюс время четырёх вычислений:

$$T_{\min} = 192 + 4 \times 128 = 704 \text{ tick.}$$

Трёх слотов уже достаточно для достижения этого времени. Запросы чтения первых трёх блоков отправляются соответственно в моменты 0, 64 и 128, а данные становятся готовы в моменты 192, 256 и 320; последовательные вычисления начинаются с момента 192. Первый блок освобождает слот в момент 320, после чего немедленно отправляется запрос чтения четвёртого блока. Он будет готов в момент 512, то есть раньше запла-

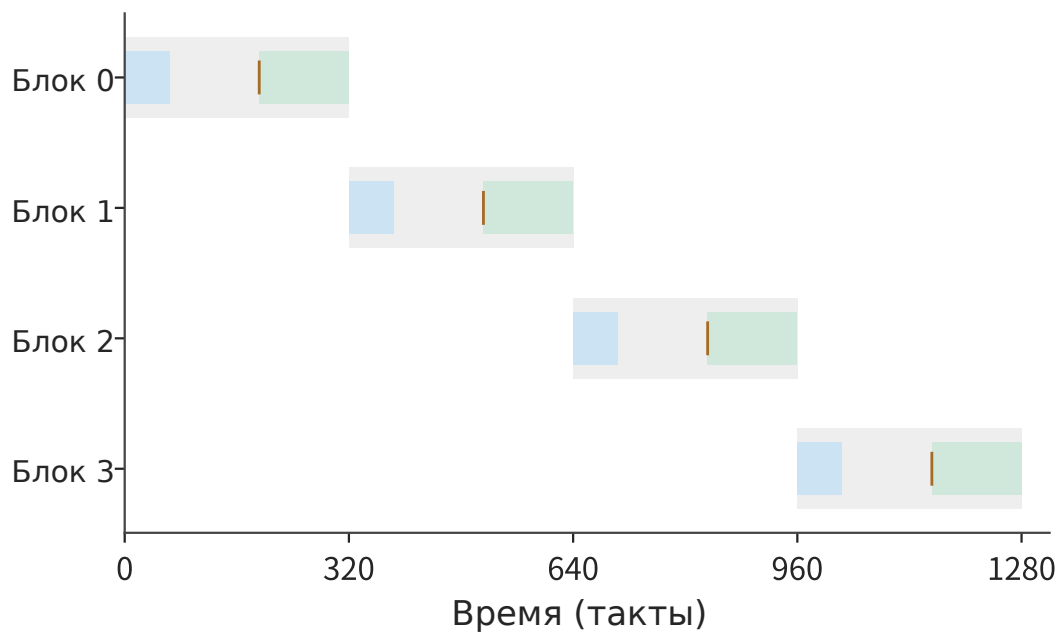


Рисунок 4-19. Временная диаграмма четырёх блоков с одним входным слотом. Синяя полоса обозначает передачу, оранжевая линия — готовность, зелёная полоса — вычисление, светло-серая — занятость слота; повторный запуск возможен лишь после завершения предыдущего блока, поэтому выполнение заканчивается на отметке 1280 tick. Один tick соответствует одному такту SM B200.

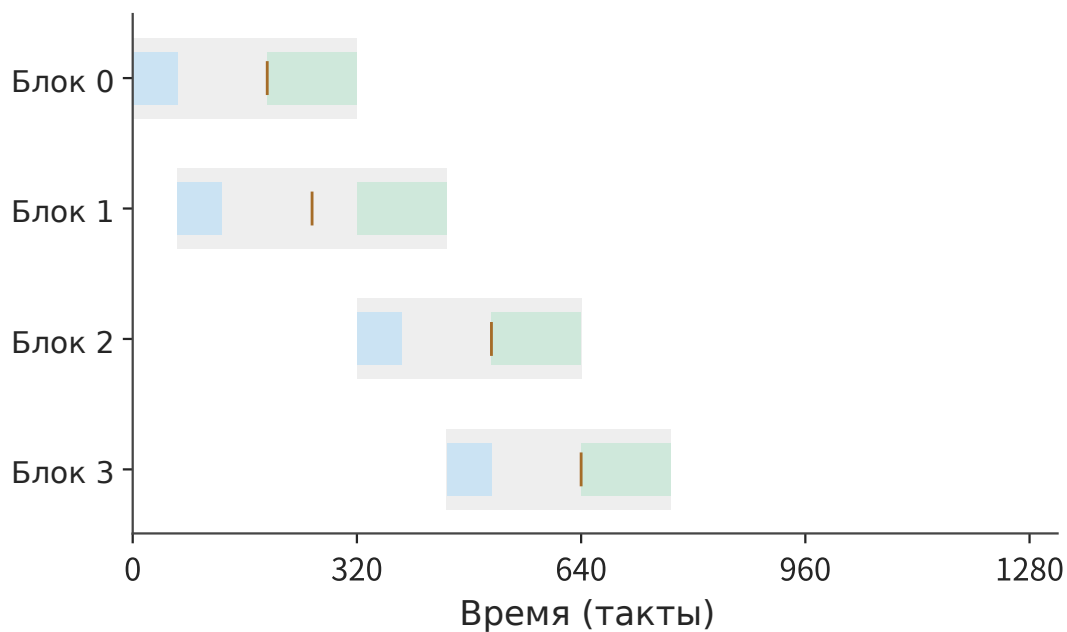


Рисунок 4-20. Для двух входных слотов используется тот же масштаб времени. Первые два блока можно запустить заранее, но третий будет готов только на отметке 512 tick, тогда как вычисление второго завершается уже на отметке 448 tick, оставляя 64 tick простаивания. Один tick на горизонтальной оси соответствует одному такту SM B200; каждая строка соответствует одному блоку данных, серый цвет обозначает занятость входного слота, синий — передачу, зелёный — вычисление, а вертикальная метка — готовность данных.

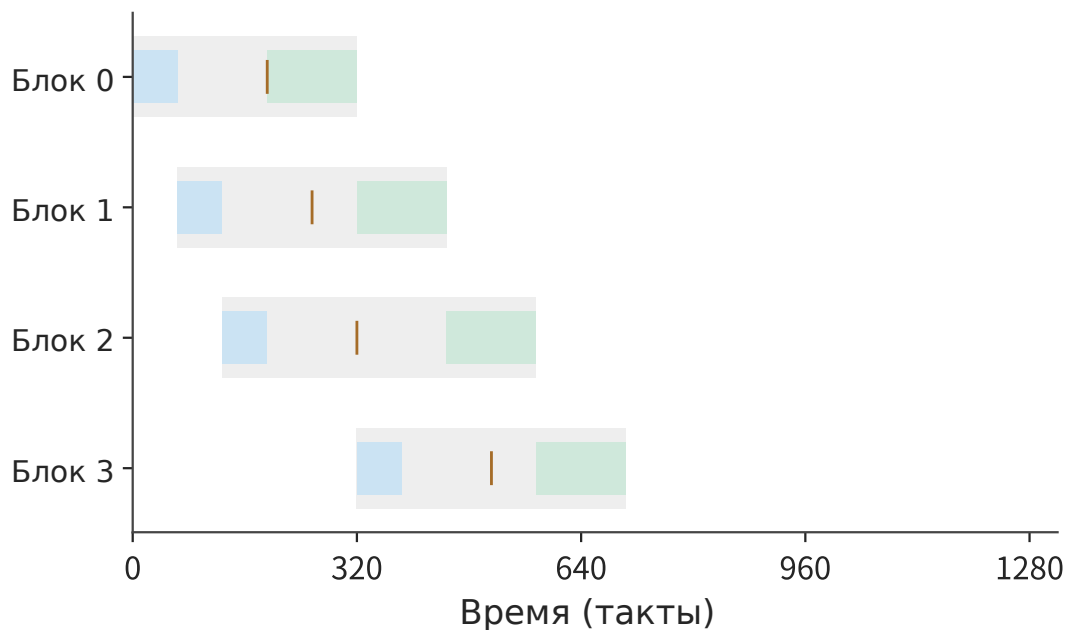


Рисунок 4-21. При трёх входных слотах первые три блока запускаются заранее, а после освобождения первого слота в него поступает четвёртый блок. Матричный блок непрерывно работает с 192 до 704 tick, поэтому четвёртый слот уже не сокращает время завершения. Один tick на горизонтальной оси соответствует одному такту SM B200; каждая строка соответствует одному блоку данных, серый цвет обозначает занятость входного слота, синий — передачу, зелёный — вычисление, а вертикальная метка — готовность данных.

нированного начала вычисления в момент 576. Поэтому матричный блок непрерывно работает с 192 до 704 tick, не простаивая.³⁵

Число входных слотов	Входной буфер	Время завершения	Экономия относительно предыдущего варианта
1	16 KiB	1280 tick	—
2	32 KiB	768 tick	512 tick
3	48 KiB	704 tick	64 tick
4	64 KiB	704 tick	0

Этот пример показывает один из способов подбора конфигурации буферов: сначала по времени прибытия первого блока и времени непрерывных вычислений определяется целевое время, затем для каждого блока проверяется, успеет ли он прибыть вовремя, после чего находится минимальное число слотов, позволяющее достичь цели. Хотя четвёртый слот обеспечивает более раннюю готовность последнего блока, начать вычисление раньше он не позволяет; полученные заранее данные лишь дольше ожидают использования.

Конфигурация буферов также зависит от скорости вычислений. Если удвоить скорость матричных вычислений, обработка каждого блока займёт только 64 tick, а целевое время непрерывных вычислений станет равным $192 + 4 \times 64 = 448$ tick. При наличии

³⁵Трёхслотовый конвейер, вариант с удвоенными вычислениями и точки перелома в проектировании сгенерированы учебным скриптом вывода; полные временные диаграммы см. в данных вывода.

только трёх слотов запрос чтения четвёртого блока можно отправить не раньше момента 256, поэтому данные будут доступны лишь в момент 448 и не успеют к первоначально запланированному началу вычисления в момент 384; выполнение завершится за 512 tick. Четыре слота позволяют заранее отправить все запросы и достичь времени 448 tick. После ускорения матричных вычислений прежняя схема загрузки уже не успевает подготавливать следующий блок данных, поэтому конфигурацию входных буферов также необходимо изменить.

Благодаря асинхронному перемещению вычислительные потоки после запуска загрузки могут продолжать другую работу и ждать события завершения только тогда, когда данные действительно потребуются. Блок перемещения записывает данные в свободный слот, событие завершения отмечает их готовность, а после окончания вычисления слот освобождается. Асинхронное копирование NVIDIA, MTE/NDDMA Ascend и механизм зависимостей задач Metal используются для организации таких передач.³⁶

Вопрос для размышления: достаточно ли трёх буферных слотов, если дополнительное ожидание до доступности входных данных увеличится вдвое? Сохранив исходные скорости передачи и вычислений, увеличьте дополнительное время ожидания между завершением передачи данных и доступностью входа со 128 до 256 tick. Смогут ли три слота по-прежнему обеспечить вычисления без простоев? Укажите момент готовности четвёртого блока и момент, к которому он должен быть готов.

4.4.3 Передача данных между вычислительными блоками

Входной конвейер сокращает время ожидания данных матричным блоком, но механизм внимания также должен передать матричный результат в векторный блок, а затем вернуть вероятности в матричный блок. Для полного хранения матрицы оценок FP32 размером 128×128 требуется 64 KiB; матричный блок один раз записывает её, а векторный блок один раз считывает, поэтому через один и тот же интерфейс памяти суммарно проходит 128 KiB данных. Если матричный блок напрямую передаст результат векторному блоку, можно избежать записи этого промежуточного результата в память и его последующего чтения.

Момент начала векторных вычислений также зависит от того, какие полные результаты им необходимы. Для обычного построчного Softmax сначала требуется получить все оценки строки, а затем вычислить максимальное значение и знаменатель нормализации. После обработки половины размерности внутреннего произведения QK каждая оценка остаётся лишь частичной суммой; если же группировать вычисления по строкам запросов, можно сначала полностью завершить одну группу строк и передать её в Softmax, продолжая матричные вычисления для остальных строк. Таким образом, направление группировки определяет, сможет ли векторный блок начать вычисления раньше.

Разделим 128 строк по направлению запросов на четыре группы по 32 строки. Для оценок FP32 и вероятностей BF16 одной группы суммарно требуется $32 \times 128 \times (4 + 2) = 24$ KiB. Далее сравниваются варианты с одним и двумя буферами для хранения передаваемых данных; каждый буфер занят с начала QK соответствующей группы до завершения PV. QK и PV используют общий матричный блок, Softmax — векторный блок,

³⁶Hopper Tuning Guide, техническое описание Ascend 950, официальное описание архитектуры Rubin.

Передаётся вся строка, две группы чередуют буферы

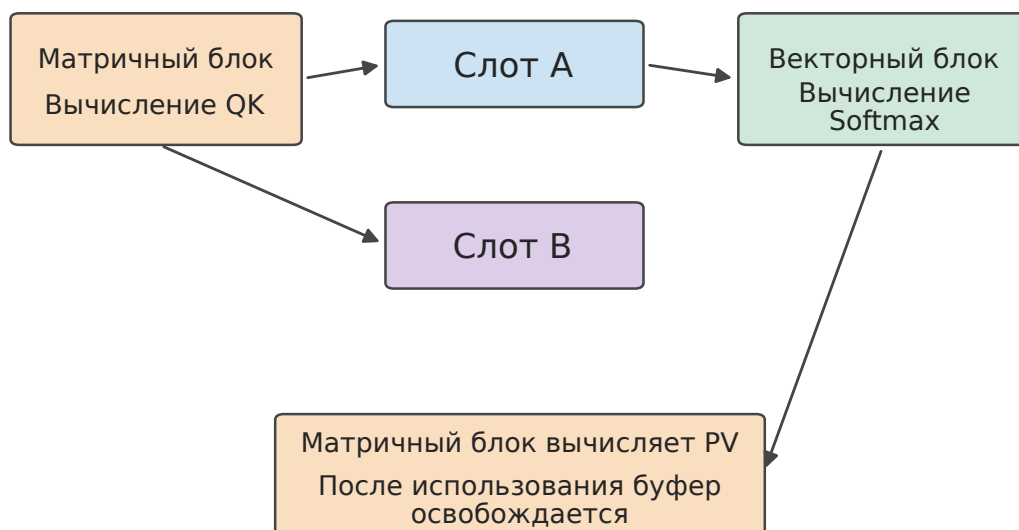


Рисунок 4-22. Матричный и векторный блоки передают данные группами полностью вычисленных строк. QK создаёт оценки, Softmax — вероятности, а после использования вероятностей в PV слот освобождается; соседняя группа строк размещается в другом слоте, благодаря чему разные группы могут обрабатываться с перекрытием.

а оба направления используют общий интерфейс передачи. При двух буферах разные группы могут одновременно находиться на разных этапах вычисления.³⁷

В этом примере QK, Softmax и PV выполняются над прямоугольными блоками оценок, а готовые операции планируются в порядке полных групп строк. tick по-прежнему обозначает такт SM B200; скорость матричных вычислений равна 8 192 FLOPs/tick, пропускная способность интерфейса передачи — 128 bytes/tick, а экспонента и деление выполняются по 16 операций за tick. Эти значения соответствуют приведённым в статье FlashAttention-4 скорости матричных вычислений одного SM, пропускной способности чтения общей памяти и скорости блока специальных функций³⁸; векторный блок последовательно выполняет масштабирование, вычитание и суммирование, сравнение, экспоненту и деление, причём обычные операции и сравнения выполняются по 128 операций за tick. При размере группы 32 строки QK и PV занимают по 128 tick; обычные операции, сравнение, экспонента и деление в Softmax занимают соответственно 96, 32, 256 и 256 tick, суммарно 640 tick. Прямая передача 16 KiB оценок и 8 KiB вероятностей занимает соответственно 128 и 64 tick, а путь с предварительной записью и последующим чтением — соответственно 256 и 128 tick. Длительность каждого этапа округляется вверх до целого числа tick.

Способ передачи	Размер группы строк	Число слотов	Начало векторных вычислений	Время завершения
Запись в память с последующим чтением	128	1	1536 tick	5118 tick
Запись в память с последующим чтением	32	1	384 tick	5120 tick
Запись в память с последующим чтением	32	2	384 tick	3200 tick
Прямая передача	32	2	256 tick	3008 tick
Прямая передача	32	4	256 tick	3008 tick

Сначала изменим только группировку. Векторный блок начинает вычисления раньше — в момент 384 вместо 1536 tick, однако общее время остаётся примерно равным 5120 tick. Причина заключается в наличии только одного слота: когда группа переходит к Softmax, находящийся в слоте результат всё ещё необходимо хранить, а результат следующей группы QK некуда записать, поэтому приходится ждать завершения PV предыдущей группы. Более раннее начало сокращает ожидание первой группы, но не обеспечивает перекрытия соседних групп.

После добавления второго слота следующая группа может обрабатываться в другом слоте, и общее время снижается до 3200 tick, то есть примерно на 38%. Последующий переход к прямой передаче уменьшает объём данных, проходящих через интерфейс памяти при каждой передаче, и дополнительно сокращает общее время до 3008 tick, то есть примерно на 6%. Дальнейшее увеличение числа слотов не меняет рабочий темп блока с наибольшей длительностью вычислений, поэтому время завершения остаётся прежним.

Группировка позволяет раньше получать полные результаты, второй слот обеспечивает перекрытие соседних групп, а прямой путь сокращает время передачи. Эти три из-

³⁷ Описание передачи между матрицей и вектором, прямой путь для 32 строк и двух слотов.

³⁸ FlashAttention-4, MLSys 2026, статья, §2.2, §3.1.1, формулы 1–3 и таблица 1; независимый пересчёт для одного SM.

менения соответственно влияют на зависимости, число одновременно обрабатываемых групп и объём работы интерфейса; в разделе 4.6.2 они будут сопоставлены с эволюцией архитектуры Ascend.

Эксперимент 4-4 · Основное задание: объём внутрикристальной буферной памяти для поддержания непрерывных вычислений

Нарисуйте временные диаграммы входного конвейера с одним, двумя, тремя и четырьмя буферными слотами. По моментам прибытия данных, завершения вычислений и освобождения буферов определите минимальное число слотов, необходимое для поддержания непрерывных вычислений. Затем удвойте скорость матричных вычислений и объясните изменение минимального числа слотов. После этого нарисуйте временные диаграммы прохождения каждой группы строк запросов через QK, Softmax и PV, отдельно показав экономию времени от группировки, добавления слотов и прямой передачи. Наконец, при фиксированном объёме внутрикристальной памяти проанализируйте, как размер буфера каждой группы ограничивает число групп, которые могут одновременно находиться в памяти.

4.4.4 Передача между SM: fence и уровни синхронизации

Передача данных в разделе 4.4.3 происходила внутри SM. При decode размер batch мал, поэтому матрично-векторное умножение необходимо распределить между всеми SM, чтобы полностью использовать пропускную способность видеопамати. Каждый SM вычисляет лишь часть выходного вектора, тогда как следующему оператору, например RMSNorm, требуется полный вектор. Такая передача происходит между SM и может выполняться только через L2.

Передача между SM состоит из двух этапов: производитель сначала записывает данные, а затем флаг; потребитель многократно читает флаг, то есть выполняет опрос, и после получения нового значения считывает данные. Ключевое значение имеет порядок. Данные и флаг могут находиться в разных сегментах L2, а две операции записи проходят через межсоединение независимо, поэтому отправленная первой запись необязательно прибывает первой. Если потребитель сначала увидит флаг, он может прочитать ещё не обновлённые данные. Поэтому перед записью флага производитель должен выполнить **fence** — инструкцию, гарантирующую порядок обращений к памяти, — и дождаться подтверждения L2 о завершении всех предшествующих записей; после того как потребитель увидит флаг, его последующее чтение также не должно выполняться раньше времени. Эти два ограничения называются соответственно семантикой освобождения (release) и получения (acquire); процесс показан на рисунке 4-23.

На RTX PRO 6000 одна передача «один к одному» занимает около 370 ns, или примерно 1 050 тактов, из которых около 600 тактов приходится на fence. Без fence требуется лишь около 140 ns, что близко к длительности одного полного обмена с L2, но корректность результата при этом не гарантируется. Аппаратно отправляемые сигналы подчиняются тому же ограничению: согласно измерениям, CUDA PDL (Programmatic Dependent Launch, при котором предыдущий kernel отправляет сигнал, после чего можно запустить следующий kernel) требует около 366 ns на каждую границу, что сопоставимо с программной передачей.

Сведение всех SM обходится дороже. После завершения каждый SM увеличивает один общий счётчик; когда его значение достигает 188, выполняющие опрос SM переходят к

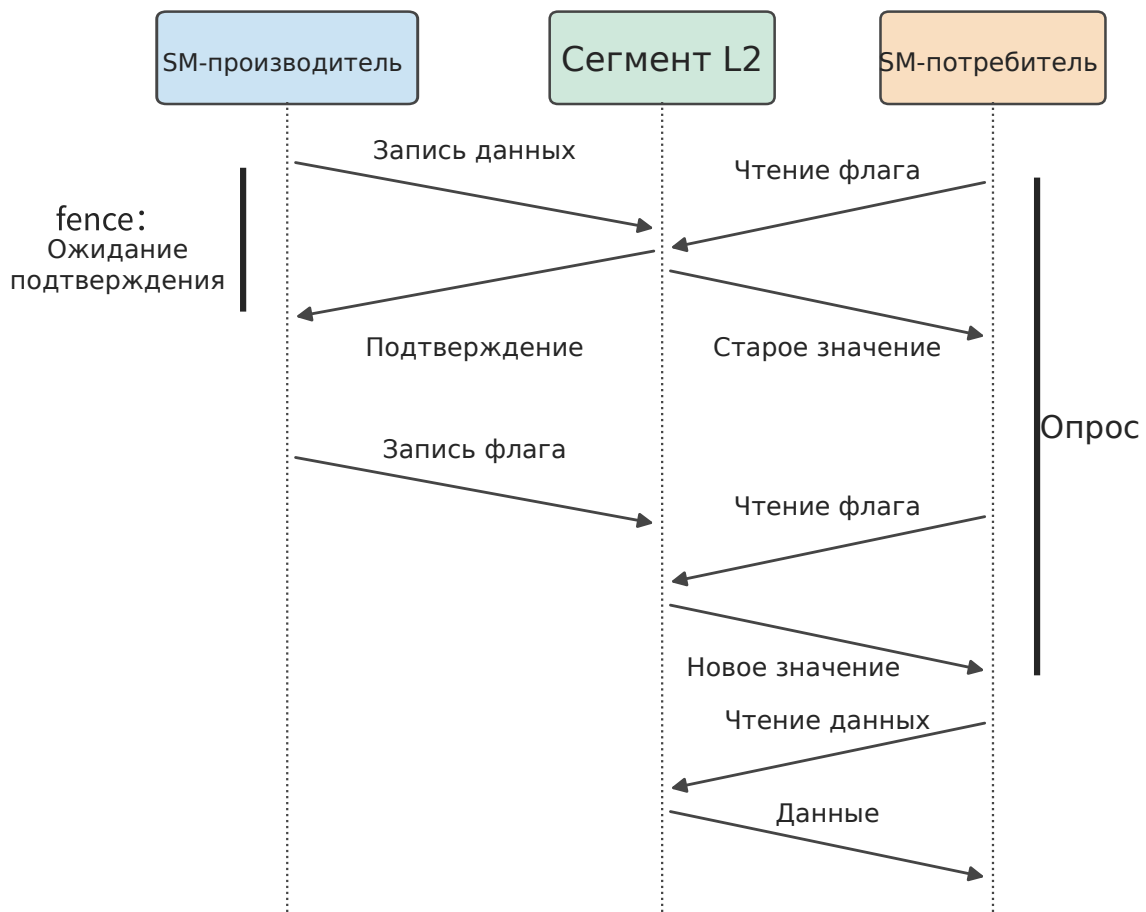


Рисунок 4-23. Временная диаграмма передачи между SM; время направлено сверху вниз. Благодаря fence производитель записывает флаг только после подтверждения L2 о записи данных; потребитель считывает данные после обнаружения нового значения флага при опросе. Каждая наклонная линия соответствует одному прохождению через межсоединение, то есть примерно половине полного обмена с L2.

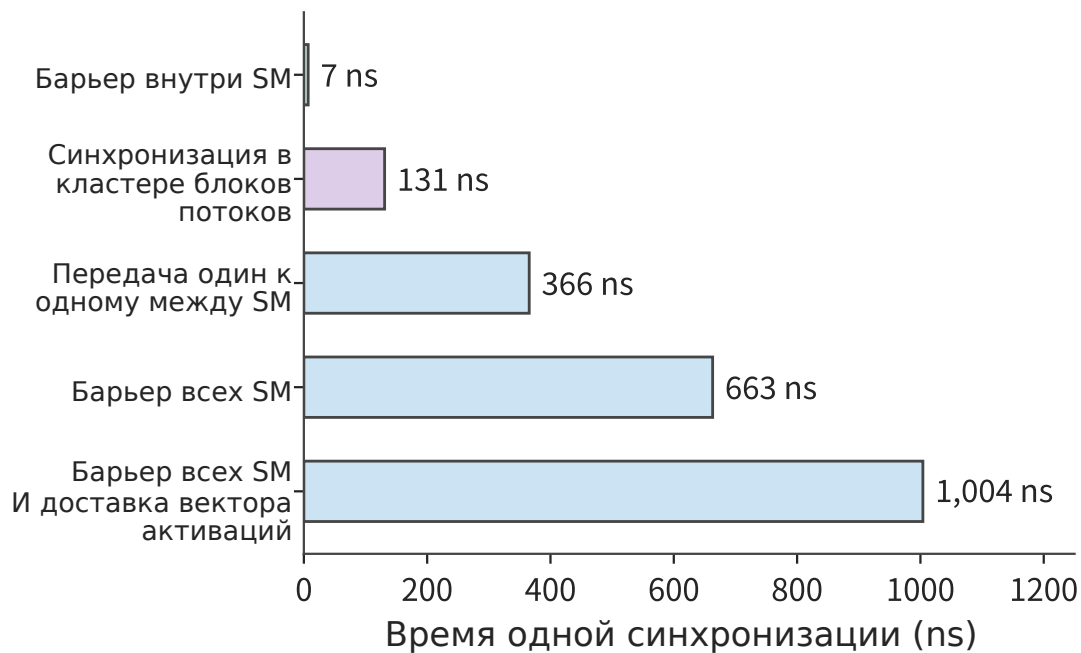


Рисунок 4-24. Время одной синхронизации разных уровней на RTX PRO 6000. Зелёный, фиолетовый и синий цвета соответствуют синхронизации внутри SM, внутри кластера блоков потоков и через L2; передача «один к одному» реализована с семантикой release-acquire, а при сведении всех SM доставляется вектор объёмом 8 KiB, размноженный в восьми экземплярах. Значения представляют собой минимумы из нескольких измерений во время совместного использования GPU.

следующему этапу. Очередь из 188 операций прибавления единицы по одному адресу в совокупности занимает менее 80 ns; основные затраты приходятся на fence с обеих сторон, примерно по 460 тактов каждый, обмены при прибытии и опросе, а также различия во времени завершения отдельных SM. В сумме это около 1 900 тактов, или примерно 660 ns. Если дополнительно требуется доставить полный вектор активаций каждому SM, все 188 SM одновременно считывают один и тот же набор строк кэша, причём кэш управляется строками фиксированного размера, в данном случае 128 bytes. Запросы концентрируются в нескольких сегментах и выстраиваются в очередь: когда все SM одновременно считывают одну строку, каждое чтение занимает около 3 400 тактов, что в девять раз больше, чем при чтении отдельной строки каждым SM. После создания восьми копий вектора и их распределения между разными сегментами одна передача данных всем SM занимает около 1,0 μ s.

На рисунке 4-24 сведена стоимость синхронизации разных уровней. Чем шире область действия зависимости и чем дальше расположена используемая память, тем выше затраты: барьер внутри SM, который пропускает участвующие потоки только после прибытия всех потоков, подсчитывается аппаратурой SM и занимает около 7 ns; синхронизация внутри кластера блоков потоков через сеть между SM — около 130 ns; за пределами кластера требуется обращение через L2, поэтому передача «один к одному» занимает около 370 ns, а сведение всех SM с доставкой данных — около 1 μ s.

Кластер блоков потоков не может заменить L2. Обмен между кластерами по-прежнему проходит через L2; пропускная способность чтения распределённой общей памяти внутри кластера также ниже: около 1,3 bytes за такт на каждый SM против

примерно 6,9 bytes для L2 и около 36 bytes для общей памяти собственного SM. Обмен вектором активаций объёмом 16 KiB внутри кластера более чем в десять раз медленнее обмена через L2.

Эти затраты не зависят от способа отправки kernel. Повторное воспроизведение графа и persistent kernel из раздела 5.5 позволяют устранить затраты на отправку с хоста и запуск kernel, но не могут устранить необходимые для самой зависимости обмена с L2: если следующему оператору требуются результаты всех SM, каждая такая граница по-прежнему занимает около 1 μ s. Второй тип ожидания, описанный в разделе 4.1.2, то есть ожидание завершения предыдущего вычисления, при пересечении границ SM проявляется именно в виде таких обменов с L2.³⁹

4.5 Многочиповая компоновка и межсоединения

Когда один ускоритель не может удовлетворить требования к вычислениям или памяти, вычисления и хранение данных можно распределить между несколькими чипами. Масштабирование увеличивает объём ресурсов, но также увеличивает расстояние между блоками, создающими и использующими данные. В этом разделе рассматриваются три вида соединений — внутри корпуса, между хостом и ускорителем и между ускорителями — и анализируется, как размещение данных и гранулярность передачи влияют на время выполнения.

4.5.1 Вычислительные кристаллы, память и соединения внутри корпуса

Корпус может содержать несколько кристаллов, совместно предоставляющих вычислительные блоки и интерфейсы памяти. Продукты Blackwell для центров обработки данных и Ascend 910C используют конструкцию с двумя кристаллами: каждый кристалл предоставляет локальные вычислительные ресурсы и интерфейсы памяти, а внутрикорпусные каналы отвечают за обмен данными между кристаллами.⁴⁰ Увеличение ёмкости позволяет размещать в памяти больше моделей; от того, на какой стороне выполняются вычисления, зависит количество обращений через канал.

Рассмотрим расчёты для двух продуктов с двумя кристаллами. Каждый GPU в HGX B200 имеет 180 GB HBM с пропускной способностью 8 TB/s. Восемь стеков HBM поровну распределены между двумя кристаллами, поэтому каждому локально доступны 90 GB и 4 TB/s, а пропускная способность NV-HBI между кристаллами составляет 10 TB/s. Каждый кристалл Ascend 910C локально располагает 64 GB памяти с пропускной способностью 1,6 TB/s, а пропускная способность канала между кристаллами составляет 270 GB/s в каждом направлении. Пусть на каждом из двух кристаллов хранится по 32 GiB весов, на некотором этапе требуется один раз прочитать все 64 GiB, а все вычисления выполняются на кристалле 0. Кристалл 0 читает собственные 32 GiB из локальной HBM

³⁹Измерения автора на RTX PRO 6000 Blackwell Workstation (GB202, 188 SM, 128 MiB L2, измеренная частота SM 2,88 GHz): описание измерений и проверка методики, выдержка числовых значений, использованных в книге. Программа измерения — `tools/gru_microbench/` из репозитория OpenTallas, CUDA 12.8. Во время измерений GPU использовался совместно с другой нагрузкой; в основном тексте взято минимальное значение из нескольких запусков, медиана выше на 10%–40%.

⁴⁰Технический обзор Blackwell, NV-HBI 10 TB/s; CloudMatrix384 v2, §3.3.1 (у 910C каждый die имеет 64 GB и 1,6 TB/s, между die — 270 GB/s в каждом направлении), начало §4.2 (при decode на каждом die размещается один эксперт) и §4.2.2; расчёты локальности die для обоих продуктов см. в элементе `die_locality` учебного скрипта вывода; платформа Vera Rubin, проверка UB и Ascend.

и одновременно через канал читает 32 GiB с кристалла 1. Передаваемые между кристаллами данные сначала считываются из НВМ кристалла 1, а затем проходят через канал, поэтому скорость определяется меньшей из двух пропускных способностей. Локальный и межкристальный потоки используют разные интерфейсы и могут обрабатываться одновременно, поэтому время чтения на этом этапе определяется более медленным потоком.

NV-HBI в B200 быстрее НВМ каждого отдельного кристалла: в техническом описании не указано, являются ли 10 TB/s суммарной двунаправленной пропускной способностью, но даже при таком предположении в каждом направлении доступно 5 TB/s, что выше 4 TB/s НВМ одного кристалла. Поэтому межкристальное чтение по-прежнему выполняется со скоростью НВМ кристалла 1 — 4 TB/s. Локальное и межкристальное чтение занимают примерно по 8,6 ms, а весь этап — около 8,6 ms, как если бы каждый кристалл читал собственные локальные веса: расположение вычислений не меняет время чтения. У 910C пропускная способность канала в каждом направлении составляет лишь около 17% пропускной способности локальной НВМ. Локальное чтение занимает примерно 21,5 ms, тогда как межкристальное — около 127 ms, поэтому время всего этапа определяется каналом. Если перенести вычисления, обрабатывающие веса кристалла 1, на сам кристалл 1, обе стороны будут читать локальные веса и потребуется лишь около 21,5 ms. Вместо 32 GiB весов через канал будут передаваться входные данные и результаты; если их общий объём составляет 64 MiB, передача по каналу со скоростью 270 GB/s займёт около 0,25 ms.⁴¹

Различие между двумя продуктами определяется отношением пропускной способности межкристального канала к пропускной способности локальной НВМ каждого кристалла. Для B200 это отношение равно 2,5: межкристальное и локальное чтение выполняются с одинаковой скоростью, поэтому два кристалла можно использовать как один GPU. Для 910C отношение составляет около 0,17, а межкристальное чтение занимает примерно в 5,9 раза больше времени, чем локальное. В суперузле Huawei CloudMatrix384 из 384 ускорителей Ascend 910C при декодировании DeepSeek-R1 эксперты также развёртываются на уровне отдельных кристаллов: на каждом кристалле размещается ровно один эксперт. Если отношение меньше 1, вычислительные блоки должны многократно использовать крупные блоки весов из локальной памяти, передавая между кристаллами только сравнительно небольшие входные данные и результаты.

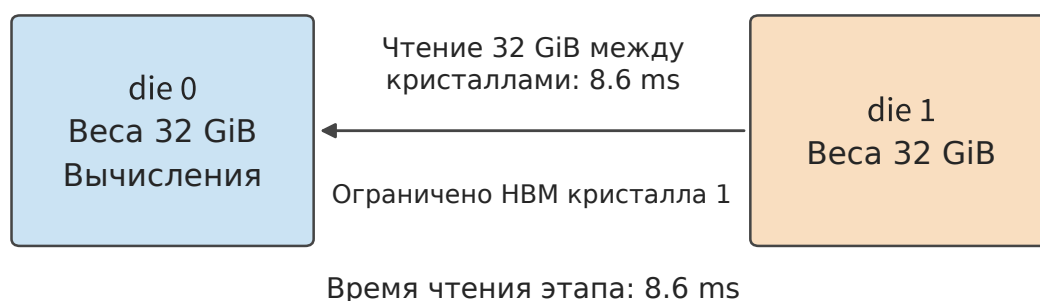
На рисунках 4-25 и 4-26 сравнивается взаимное расположение вычислительных блоков и весов: при централизованных вычислениях перемещаются веса, а при локальных — передаются входные данные и возвращаются результаты. Чтобы определить способ распределения вычислительных задач, сначала следует изобразить данные, проходящие через канал в каждом варианте, а затем сравнить их общий объём.

Такое размещение также создаёт возможности для параллелизма: локальное чтение и вычисления на обеих сторонах могут выполняться независимо, после чего передаются итоговые результаты. Необходимость ждать завершения обеих сторон определяется вычислительным графом модели; если на следующем шаге требуется объединить частичные суммы, точка объединения становится точкой синхронизации. В главе 6 эти пере-

⁴¹Технический обзор Blackwell, NV-HBI 10 TB/s; CloudMatrix384 v2, §3.3.1 (у 910C каждый die имеет 64 GB и 1,6 TB/s, между die — 270 GB/s в каждом направлении), начало §4.2 (при decode на каждом die размещается один эксперт) и §4.2.2; расчёты локальности die для обоих продуктов см. в элементе die_locality учебного скрипта вывода; платформа Vera Rubin, проверка UB и Ascend.

Все вычисления на кристалле 0

HGX B200



Ascend 910C

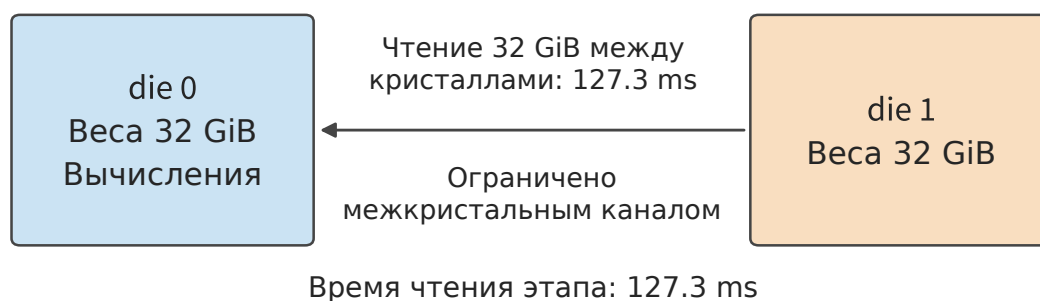
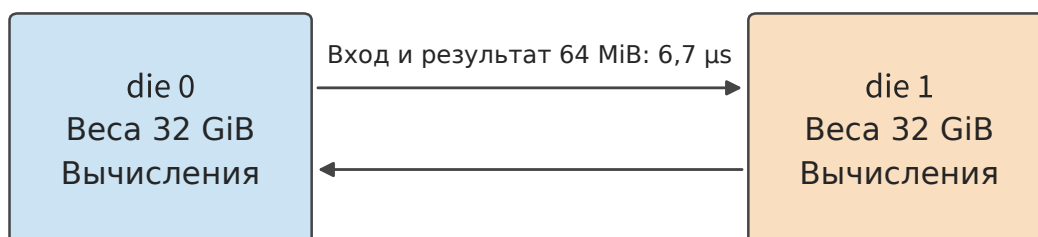


Рисунок 4-25. Когда все вычисления выполняются на кристалле 0, 32 GiB весов с кристалла 1 приходится читать через межкристальный канал. В B200 межкристальное чтение ограничено HBM кристалла 1 и занимает около 8,6 ms — столько же, сколько локальное чтение; в Ascend 910C оно ограничено межкристальным каналом и занимает около 127 ms.

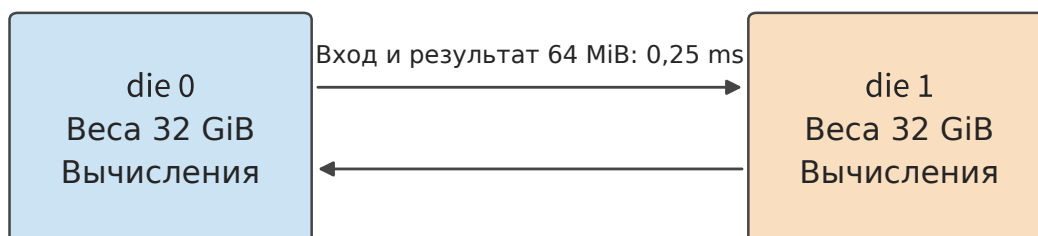
Вычисления на кристалле с весами

HGX B200



Чтение локальных весов с обеих сторон: 8,6 ms

Ascend 910C



Чтение локальных весов с обеих сторон: 21,5 ms

Рисунок 4-26. После переноса вычислений на кристалл, где находятся веса, через канал передаются только 64 MiB входных данных и результатов, а обе стороны читают локальные веса. Для B200 время по-прежнему составляет около 8,6 ms; для Ascend 910C оно сокращается примерно со 127 до 21,5 ms.

дачи будут выведены для конкретных способов параллельного разбиения.

4.5.2 Доступ к данным между CPU и ускорителем

Размещение данных внутри корпуса определяет передачу между кристаллами; аналогичная проблема существует и за пределами чипа. На рисунке 4-2 показана передача данных между хостом и ускорителем: подготовленные CPU данные необходимо передать GPU. Подготовленные CPU входные данные обычно сначала помещаются в память хоста, а GPU с отдельной видеопамятью получает их через PCIe или специализированное соединение платформы. Передача с хоста на устройство называется H2D, а с устройства на хост — D2H. Механизм копирования переносит данные по заранее подготовленным адресам посредством прямого доступа к памяти (DMA), поэтому после отправки команды копирования CPU может выполнять другую работу.

Один и тот же тензор сначала передаётся по каналу хоста, а затем считывается GPU из видеопамяти, поэтому время передачи и чтения нужно рассчитывать отдельно. Рассмотрим RTX 4090: эта карта подключается к хосту через PCIe 4.0 x16 (16 линий), номинальная суммарная двунаправленная пропускная способность составляет 64 GB/s, то есть 32 GB/s в каждом направлении; пропускная способность видеопамяти равна 1 008 GB/s.⁴² Пусть объём входных данных равен 64 MiB. Если сначала пренебречь накладными расходами на запуск, то

$$T_{\text{H2D}} = \frac{64 \text{ MiB}}{32 \text{ GB/s}} \approx 2.1 \text{ ms}, \quad T_{\text{read}} = \frac{64 \text{ MiB}}{1008 \text{ GB/s}} \approx 67 \text{ }\mu\text{s}.$$

Время загрузки в 31,5 раза превышает время одного чтения из видеопамяти, что точно соответствует отношению пропускных способностей двух путей. Если данные используются только один раз, основное время занимает передача с хоста; если после загрузки они повторно используются на ускорителе R раз, на каждое использование приходится T_{H2D}/R . При 100 повторных использованиях среднее приходящееся на одно использование время загрузки сокращается примерно до 21 μs . При каждом вызове достаточно передать адрес и параметры выполнения, а полный набор данных продолжает находиться в видеопамяти.

Можно также определить минимальное необходимое количество повторных использований: при $R \geq 32$ приходящееся на одно использование время загрузки уже не превышает время одного чтения ускорителем. Поэтому увеличение пропускной способности канала хоста особенно полезно для однократно используемых входных данных, а постоянное размещение весов на устройстве позволяет избежать повторных загрузок.

Механизм копирования и вычислительные блоки являются независимыми исполнительными ресурсами, поэтому во время вычислений GPU над текущим batch можно загружать следующий batch. Для построения конвейера два входных буфера отдельно хранят текущий и следующий batch, а после завершения вычислений текущего batch назначение буферов меняется. Это и есть применение двойной буферизации из раздела 4.4 к каналу хоста. Если запись при копировании и чтение при вычислении совместно используют интерфейс памяти, этот интерфейс должен обслуживать оба потока данных.

⁴²Техническое описание архитектуры RTX Blackwell, приложение A, таблица 3: RTX 4090 — 24 GB GDDR6X, 1 008 GB/s, PCIe Gen 4, TGP 450 W; RTX 5090 — 32 GB GDDR7, 1 792 GB/s, PCIe Gen 5, TGP 575 W; спецификация A100 80GB: PCIe 4.0 — 64 GB/s в обоих направлениях суммарно. Время H2D и чтения видеопамяти см. в элементе `host_link` учебного скрипта вывода.

Не на всех платформах требуется такое копирование. CPU и GPU Apple совместно используют физическую память. Если применяется доступный обеим сторонам буфер, после записи данных CPU и завершения синхронизации GPU может читать их из того же хранилища. По сравнению с подходом с отдельной видеопамью устраняются отдельная копия H2D и её передача; последующее чтение GPU по-прежнему обслуживается системой памяти.⁴³ Унифицированная память превращает передачу данных из «использования после копирования» в «использование после синхронизации».

Унифицированное адресное пространство позволяет CPU и GPU обращаться к данным по одному набору виртуальных адресов. CUDA — платформа программирования и выполнения для GPU NVIDIA; её механизм унифицированной памяти (Unified Memory) может перемещать страницы в соответствии с правилами платформы, сохраняя адрес неизменным при изменении физического расположения.⁴⁴ Расстояние между данными и использующим их процессором влияет на время доступа, а адреса и механизм синхронизации позволяют процессору найти данные и прочесть их после завершения записи.

4.5.3 Обмен данными между ускорителями

Пример загрузки с хоста подчёркивает важность размещения и повторного использования данных. Карты также часто обмениваются активациями, входными данными экспертов и частичными результатами. Объём данных при каждой передаче различается, поэтому помимо числа байтов необходимо учитывать фиксированные накладные расходы каждой передачи. Для одной передачи требуется отправить запрос, уведомить получателя о завершении и передать данные по каналу. Если обозначить фиксированные накладные расходы через α , эффективную однонаправленную пропускную способность через R , а объём данных через V , то время передачи между двумя точками равно

$$T_{\text{edge}} = \alpha + \frac{V}{R}.$$

Рассмотрим сервер HGX H100: внутри него каждая H100 соединена с другими GPU посредством NVLink, суммарная двунаправленная пропускная способность которого равна 900 GB/s, то есть $R = 450$ GB/s в каждом направлении; NVLink в A100 SXM обеспечивает 300 GB/s в каждом направлении.⁴⁵ Примем фиксированные накладные расходы $\alpha = 2$ μs , как и для запуска каждого раунда NVLink внутри сервера в главе 7. Пусть передаётся BF16-активация формы $[M, 4096]$, объём которой равен $8192M$ bytes. При $M = 1$ объём данных составляет всего 8 KiB, передача на H100 занимает около 0,02 μs , а общее время — около 2,02 μs . При $M = 256$ объём данных равен 2 MiB, передача занимает около 4,66 μs , а общее время — около 6,66 μs .

При переходе с A100 на H100 пропускная способность увеличивается в 1,5 раза, однако время передачи 8 KiB сокращается лишь с 2,03 до 2,02 μs . Для передачи 2 MiB время сокращается с 8,99 до 6,66 μs , то есть примерно на 26%. Если вместо этого вдвое уменьшить фиксированные накладные расходы на H100, время передачи небольшого объёма данных сократится примерно вдвое, а при большом объёме данных экономия составит

⁴³Официальные характеристики M2 Pro/Max, описание архитектуры Apple GPU, режимы хранения Metal, официальное описание Neural Accelerator в GPU M5.

⁴⁴Терминологию для хоста, DMA и единого адресного пространства см. в CUDA Programming Guide.

⁴⁵Спецификация RTX PRO 6000: ECC, MIG и конфигурация продукта; Hopper Tuning Guide: NVLink; техническое описание GA102: NVLink в 3090; Blackwell Tuning Guide и возможности Blackwell в CUTLASS: SM100/SM120.

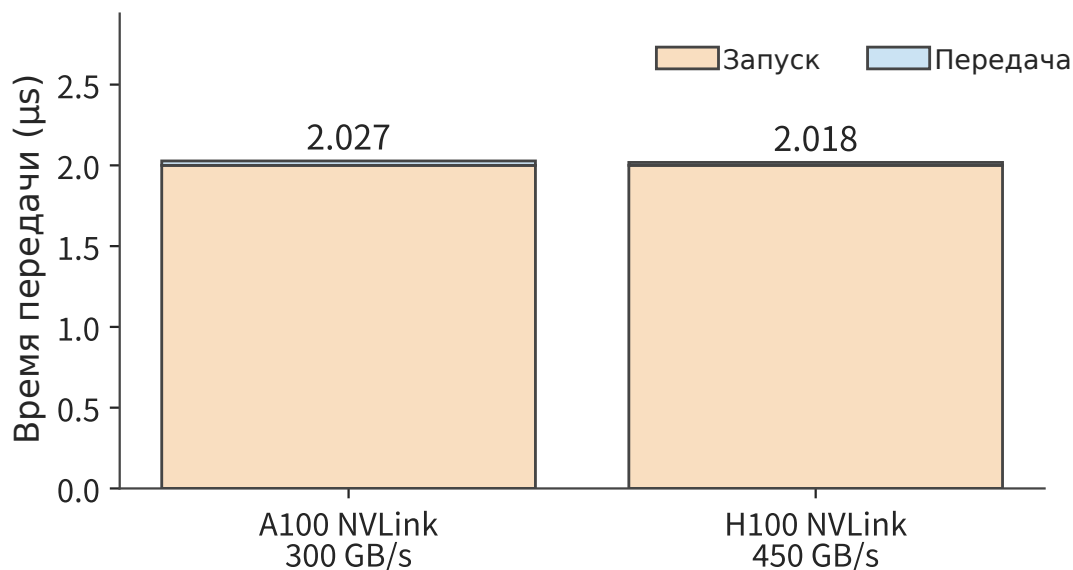


Рисунок 4-27. Время запуска и передачи 8 KiB данных. При фиксированных накладных расходах на запуск 2 µs замена NVLink A100 с пропускной способностью 300 GB/s в каждом направлении на NVLink H100 с 450 GB/s сокращает лишь тонкую синюю составляющую передачи.

около 15%. Эти два способа оптимизации воздействуют на разные доминирующие слагаемые формулы.

Приравняв два слагаемых, получим пороговый объём данных:

$$V^* = \alpha R = 2 \times 10^{-6} \times 4.5 \times 10^{11} = 900\,000 \text{ bytes} \approx 879 \text{ KiB}.$$

Для активации из этого примера он соответствует $M \approx 109.9$: только начиная со 110 строк время передачи превышает фиксированные накладные расходы. Чем быстрее канал, тем выше порог и тем больше передач попадает в область, где преобладают фиксированные накладные расходы. Для объёмов меньше этого порога объединение нескольких передач в одну уменьшает приходящиеся на каждый набор данных накладные расходы на запуск. Для объёмов выше порога больший эффект дают сокращение числа байтов или повышение пропускной способности. Объединённая отправка нескольких уже готовых наборов данных сокращает число запусков; ожидание ещё не готовых данных ради формирования более крупного пакета увеличивает время в очереди. Поэтому выбор гранулярности передачи также предполагает компромисс между пропускной способностью и временем отклика.

Если одно выполнение включает 100 обязательно последовательных обменов небольшими объёмами данных, каждый с фиксированными накладными расходами 2 µs, одни только эти расходы составят 200 µs. При передаче между серверами фиксированные накладные расходы выше: для операции AllReduce над 16 bytes между 16 картами двух серверов HGX H100 — объединения локальных вкладов всех карт с предоставлением полного результата каждой карте — опубликованное измерение составляет около 25 µs.⁴⁶ Сто таких операций сокращения займут 2,5 ms. В этом случае даже частичное уменьшение объёма передаваемых данных вряд ли заметно сократит общее время, тогда как

⁴⁶Проверка опубликованных записей запуска эксперимента 7-3, исходный журнал двух HGX H100: 16 rank, NCCL 2.26.2, AllReduce для 16 bytes — 24,96 µs вне буфера и 24,93 µs в исходном буфере.

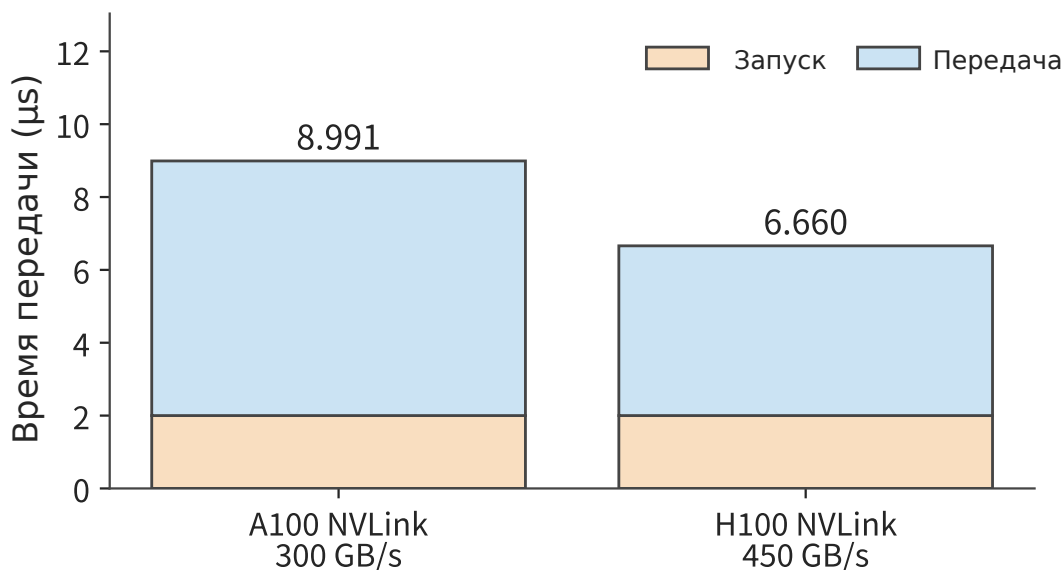


Рисунок 4-28. Время передачи 2 MiB данных при тех же условиях запуска. Синяя составляющая передачи занимает основную часть времени, поэтому переход на NVLink H100 даёт более заметный выигрыш; диапазон вертикальной оси на этом и предыдущем рисунках указан отдельно.

сокращение количества раундов обмена напрямую укорачивает цепочку зависимостей. AllReduce объединяет несколько передач и операций сокращения; в главе 6 будут отдельно рассчитаны число раундов, количество передаваемых по каждому каналу байтов и время, в течение которого передача может перекрываться вычислениями.

Такие межсоединения, как NVLink и Huawei Unified Bus, предоставляют пути передачи между устройствами, а уведомления о завершении позволяют получателю начать вычисления после поступления данных.⁴⁷

Вопрос для размышления: как объединённая отправка изменяет время завершения и время отклика для первого набора данных? Четыре набора данных объёмом 8 KiB становятся готовы последовательно в моменты 0, 1, 2 и 3 μs. Сохраняя для NVLink H100 пропускную способность 450 GB/s в каждом направлении и накладные расходы 2 μs на каждый запуск, сравните их отдельную отправку по мере готовности с ожиданием готовности всех наборов и объединённой отправкой по одному каналу: когда будут доставлены все данные? Когда первый набор данных станет доступен получателю?

4.6 Как требования моделей стимулируют эволюцию архитектур

В разделах 4.2–4.5 были отдельно рассмотрены вычисления, хранение, перемещение данных и межсоединения. В этом разделе мы вновь объединим эти компоненты в контексте смены поколений NVIDIA, Ascend и Apple, рассчитаем, какую именно работу

⁴⁷Технический обзор Blackwell, NV-HBI 10 TB/s; CloudMatrix384 v2, §3.3.1 (у 910C каждый die имеет 64 GB и 1,6 TB/s, между die — 270 GB/s в каждом направлении), начало §4.2 (при decode на каждом die размещается один эксперт) и §4.2.2; расчёты локальности die для обоих продуктов см. в элементе `die_locality` учебного скрипта вывода; платформа Vera Rubin, проверка UB и Ascend.

устраняют новые механизмы и требования к каким ресурсам они изменяют, а затем объясним, как эти изменения отвечают характеру модельной нагрузки.

Ниже последовательно сравниваются изменения матричных блоков, числовых форматов, трактов данных и ёмкости; в каждом случае объём работы модели остаётся неизменным, а меняется только исследуемое оборудование или способ представления. Параметры оборудования и повторные расчёты приведены в сопутствующих вычислительных материалах.⁴⁸

4.6.1 NVIDIA: какой выигрыш по отдельности дают матрицы, точность и тракт данных

Ценность Tensor Core сначала рассчитаем для двух трактов на одном и том же чипе. Обычная пиковая производительность Volta V100 SXM2 в FP32 составляет 15,7 TFLOP/s, пиковая производительность Tensor Core с входными данными FP16 и накоплением FP32 — 125 TFLOP/s, а пропускная способность HBM2 — 900 GB/s. Снова возьмём проекцию 4096×4096 из этой главы: входные данные и веса хранятся в FP16, результат также записывается в FP16. Обычный тракт преобразует операнды в FP32 и затем выполняет умножение со сложением, а тракт Tensor Core напрямую выполняет матричное умножение со сложением в смешанной точности, то есть использует FP16 для входных данных и FP32 для накопления. Оба тракта читают и записывают одинаковый объём данных.⁴⁹

При обработке 4 096 строк объём матричных вычислений составляет $2 \times 4096^3 \approx 137,4$ GFLOPs, а общий размер входных данных, весов и выходных данных — 96 MiB. Умножение со сложением в обычном FP32 требует $137,4/15,7 \approx 8,75$ ms, Tensor Core — $137,4/125 \approx 1,10$ ms, а чтение и обратная запись — около 0,112 ms. Время матричных вычислений сокращается примерно в восемь раз, поскольку большое количество регулярных операций умножения со сложением передаётся специализированному блоку.

Теперь сократим входные данные до одной строки. Время матричных вычислений по двум трактам станет около 2,14 μ s и 0,268 μ s, однако передача 32 MiB весов вместе с входными и выходными данными потребует около 37,3 μ s. Поэтому оба тракта прежде всего ограничены чтением данных. Одно и то же улучшение Tensor Core даёт значительный выигрыш для матричного умножения с большим batch, но мало помогает при данном однострочном вызове: ускорять по-прежнему необходимо чтение весов.

Эволюция точности одновременно изменяет числовой диапазон, объём хранения и исполняемые инструкции. В Turing для Tensor Core появились INT8 и INT4, расширившие возможности низкобитного целочисленного инференса; Ampere добавил BF16, TF32 и структурированную разреженность 2:4. В матричном умножении TF32 сохраняет диапазон экспоненты FP32, но сокращает мантиссу, а разреженность 2:4 организует вычисления так, что ненулевыми остаются два элемента из каждых четырёх. Transformer Engine в Hopper объединил матричные инструкции FP8 с управлением масштабированием, а Blackwell дополнительно поддерживает блочное масштабирование и

⁴⁸Программа количественного расчёта эволюции архитектуры и пояснения, подробные результаты расчётов. Параметры считываются из таблицы аппаратных характеристик; данные для 3090 дополнены по таблице 9 официального технического описания GA102. Полная конфигурация модели и перечень тензоров взяты из каталога calculations этой книги.

⁴⁹Техническое описание архитектуры Volta, разделы о Tensor Cores и смешанной точности.

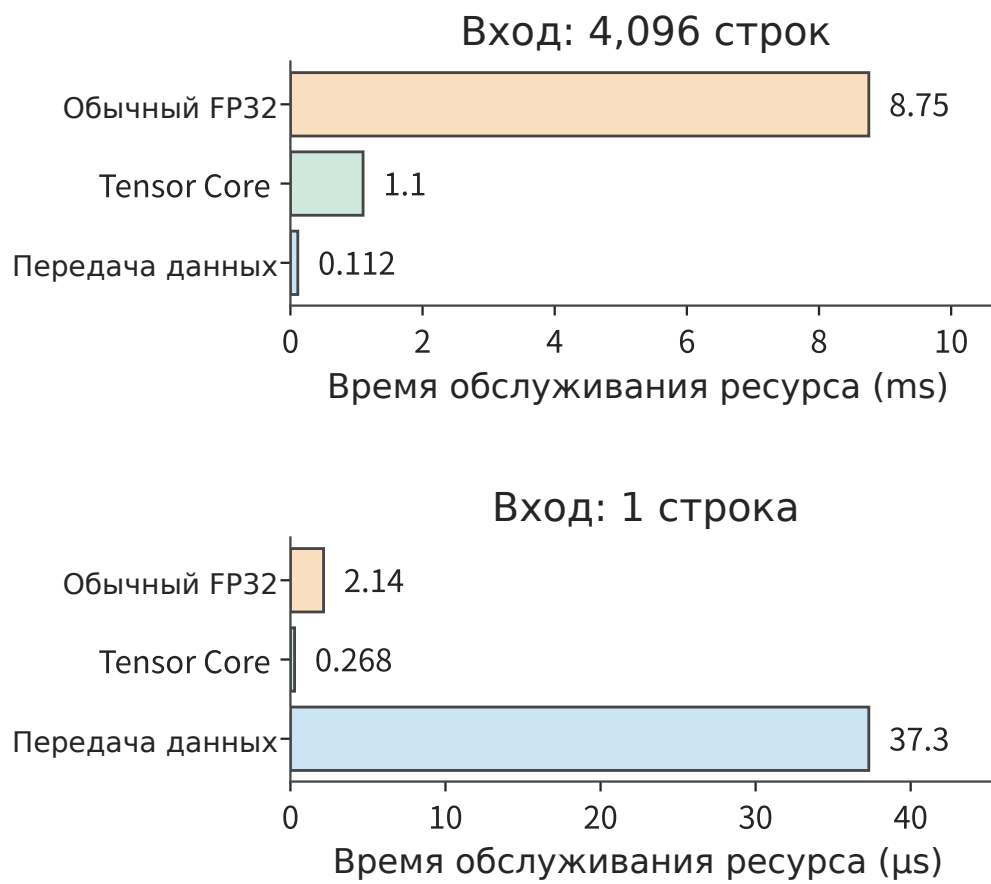


Рис. 4-29. Проекция Q на одном V100. При 4 096 строках Tensor Core значительно сокращает время матричных вычислений; при одной строке оба вычислительных тракта работают быстрее, чем выполняется передача весов. Для верхней и нижней групп отдельно указаны единицы времени.

FP4.⁵⁰⁵¹

Рассмотрим в качестве примера добавленный формат BF16. И FP16, и BF16 занимают два байта, поэтому после перехода на BF16 прежние 32 MiB весов не изменятся. Число с плавающей запятой представляет значение как произведение значащих цифр и степени двойки; у нормальных чисел значащие цифры нормализованы, а у субнормальных чисел количество значащих разрядов постепенно уменьшается в меньшем числовом диапазоне. BF16 использует восьмибитную экспоненту, а его минимальное нормальное число составляет около $1,18 \times 10^{-38}$; FP16 использует пятибитную экспоненту, его минимальное нормальное число составляет около $6,10 \times 10^{-5}$, а минимальное положительное субнормальное число — около $5,96 \times 10^{-8}$. Градиент величиной 10^{-8} при непосредственном округлении до FP16 превратится в ноль, тогда как BF16 всё ещё способен его представить. Это снижает потребность в корректировке масштабирования для предотвращения переполнения и исчезновения порядка при обучении. Цена этого — меньшее количество значащих цифр: в окрестности 1 интервал между соседними числами BF16 равен 2^{-7} , а для FP16 — 2^{-10} . Низкая точность входных данных в сочетании с накоплением FP32 позволяет раздельно работать с представлением входных данных и ошибкой накопления длинного скалярного произведения.⁵²

Теперь рассмотрим объём хранения. Сохраним блочный формат из раздела 4.2.4, в котором каждые 32 значения используют общий однобайтовый масштабный коэффициент; MX — название такого формата микромасштабирования (microscaling). Для MXFP8 требуется $16 + 0,5 = 16,5$ MiB, для MXFP4 — $8 + 0,5 = 8,5$ MiB. Чтение этих весов BF16 с пропускной способностью RTX 4090, равной 1 008 GB/s, занимает 33,29 μ s; чтение тех же весов BF16 на RTX 5090 с пропускной способностью 1 792 GB/s занимает 18,72 μ s, а чтение весов MXFP4 — 4,97 μ s. Первое сокращение обусловлено повышением пропускной способности, второе — изменением разрядности и группового представления.

При переходе с 33,29 μ s к 4,97 μ s бюджет чтения весов уменьшается примерно в 6,69 раза. Этот выигрыш складывается из произведения двух факторов: пропускная способность возрастает примерно в 1,78 раза, а объём хранения сокращается примерно в 3,76 раза. Нативный низкоточный матричный тракт непосредственно обрабатывает сжатое представление, устраняя высокоточные копии весов, необходимые при предварительной распаковке по схеме из раздела 4.2.4.

По мере ускорения матриц архитектура начинает сокращать подготовку и ожидание до и после их обработки. Асинхронное копирование в Ampere устраняет промежуточную передачу через регистры при копировании из глобальной памяти в разделяемую; TMA в Hopper берёт на себя генерацию многомерных адресов и перемещение данных, работая совместно с асинхронным выполнением матричных операций; в серверном Blackwell SM100 для хранения результатов матричного накопления применяется тензорная память (Tensor Memory, TMEM). Эти три изменения соответственно затрагивают инструкции перемещения, использование регистров общего назначения и место хранения промежуточных результатов.⁵³

⁵⁰Официальное техническое описание архитектуры Turing, Tensor Cores Turing и инференс с низкой точностью.

⁵¹Техническое описание архитектуры A100, техническое описание архитектуры H100, Hopper Tuning Guide, технический обзор Blackwell и возможности CUTLASS для Blackwell.

⁵²Техническое описание архитектуры A100, разделы о BF16 и числовых форматах; повторный расчёт числового диапазона, хранения и квантования.

⁵³Blackwell Tuning Guide, описание возможностей CUTLASS для Blackwell и пример SM100 TMEM; сведения о 128 KB кэша данных первого уровня и предельных 100 KB общей памяти на каждый SM в SM120 (вычисли-

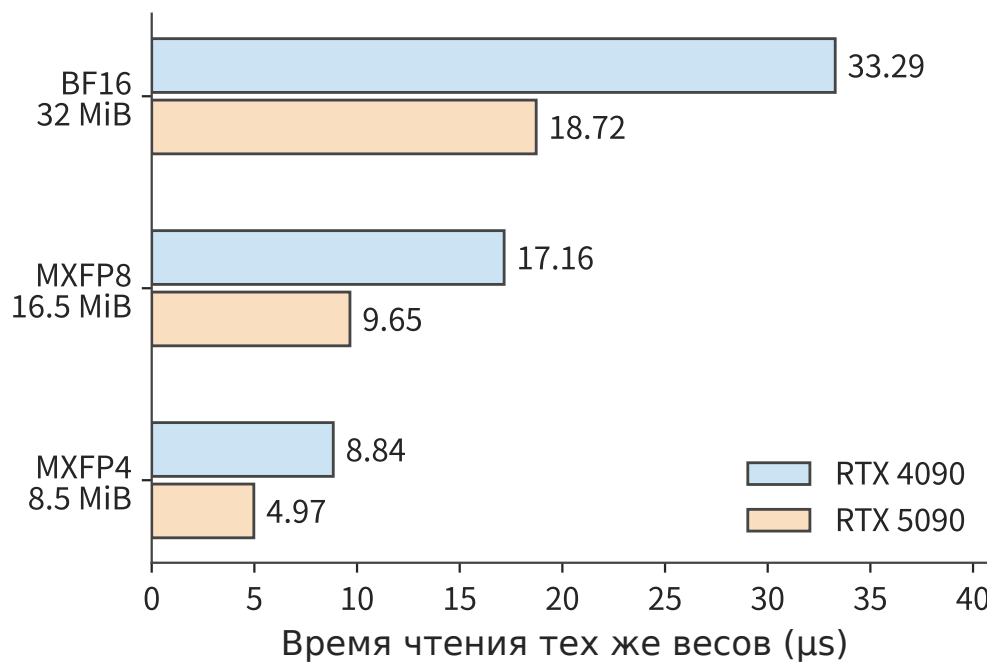


Рис. 4-30. Бюджет хранения и чтения одного и того же массива весов 4096×4096 . Каждые 32 низкоточных значения используют общий однобайтовый масштабный коэффициент. При снижении разрядности с BF16 до MXFP4 общий объем весов и масштабных коэффициентов сокращается с 32 MiB до 8,5 MiB.

Вычислительному блоку из раздела 4.3.2 требуется 32 KiB входных данных и 64 KiB результатов накопления. Три набора входных данных и один набор результатов в сумме занимают 160 KiB; ТМЕМ самостоятельно обслуживает 64 KiB из этого объема, поэтому регистры общего назначения можно использовать для состояния других потоков. Таково прямое влияние нового специализированного хранилища на резидентность рабочей нагрузки.

В следующем поколении пропускная способность Rubin для вычисления экспоненты FP32 увеличивается вдвое относительно Blackwell, а для BF16/FP16 — вчетверо, что устраняет узкое место при вычислении экспоненты, проявившееся после ускорения матриц в примере 4-1. Rubin также позволяет ТМА обновлять адреса и шаги экспертов: при переключении экспертов МоЕ форма матрицы может оставаться неизменной, достаточно сменить расположение весов, благодаря чему обновление дескриптора сокращает повторную подготовку адресов.⁵⁴

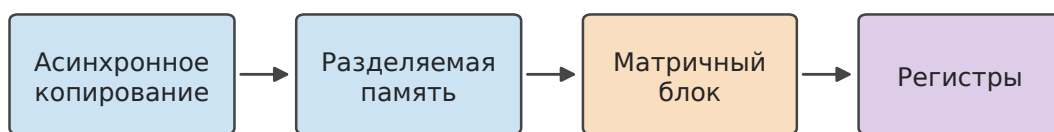
Как разместить потребительские, рабочие и серверные продукты в одной таблице? Зафиксируем входные данные BF16, накопление FP32 и вычисление плотных матриц. Параметры приведены ниже. RTX 3090, 4090 и 5090 относятся соответственно к Ampere, Ada и Blackwell; A100 и H100 — соответственно к Ampere и Hopper.⁵⁵

тельная возможность 12.0) см. в таблице вычислительных возможностей CUDA Programming Guide.

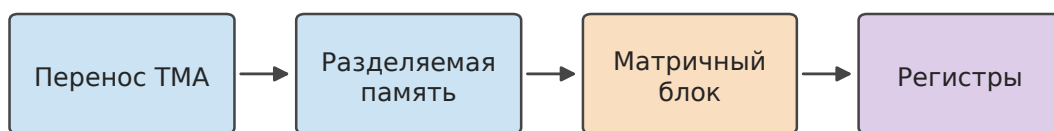
⁵⁴Описание архитектуры NVIDIA Rubin: перемещение данных МоЕ, пропускная способность инструкций по размерности K и ускорение механизма внимания.

⁵⁵Программа количественного расчёта эволюции архитектуры и пояснения, подробные результаты расчётов. Параметры считываются из таблицы аппаратных характеристик; данные для 3090 дополнены по таблице 9 официального технического описания GA102. Полная конфигурация модели и перечень тензоров взяты из каталога calculations этой книги.

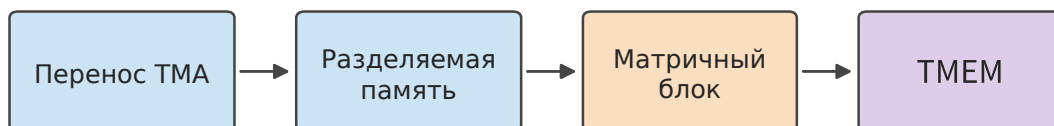
Ampere



Hopper



Blackwell SM100



Входной блок 32 KiB; состояние аккумуляции выхода 64 KiB

Рис. 4-31. От асинхронного копирования и ТМА до ТМЕМ: специализированные компоненты берут на себя всё больше задач по подготовке данных и управлению состоянием накопления. Синим показаны хранение и перемещение данных, оранжевым — матричные вычисления, фиолетовым — результаты накопления.

Модель и линейка продуктов	Ёмкость, GB	Пропускная способность, GB/s	Плотные матрицы BF16/FP32, TFLOP/s
RTX 3090, потребительская	24	936	71,0
RTX 4090, потребительская	24	1 008	165,2
RTX 5090, потребительская	32	1 792	209,5
RTX PRO 6000 Blackwell, рабочая станция	96	1 792	503,8
A100 80GB SXM, серверная	80	2 039	312,0
H100 SXM, серверная	80	3 350	989,4
H200 SXM, серверная	141	4 800	989,5

RTX 3090 и 4090 имеют одинаковую ёмкость, при этом скорость матричных операций BF16 увеличивается примерно в 2,33 раза, а пропускная способность — лишь примерно на 7,7%. Первое изменение главным образом сокращает время вычисления больших матриц, и только второе непосредственно сокращает время чтения весов для одной строки. У RTX 5090 ёмкость увеличена до 32 GB, а пропускная способность — ещё примерно в 1,78 раза; новый тракт FP4 также предоставляет дополнительный способ выполнения для моделей, допускающих низкую точность. Эти три поколения продуктов соответственно изменяют ресурсные бюджеты обработки матриц, чтения весов и резидентного размещения модели.

Пропускная способность RTX PRO 6000 и RTX 5090 одинакова и составляет 1 792 GB/s. Основные преимущества карты для рабочих станций включают 96 GB видеопамати с ECC, более высокую пропускную способность BF16 с накоплением FP32 и функцию многоэкземплярного GPU (MIG); MIG разделяет один GPU на экземпляры с изолированными друг от друга ресурсами. Для потокового чтения весов одинакового размера бюджет пропускной способности у них совпадает; для больших матриц или резидентных данных объёмом свыше 32 GB результаты будут различаться. SM120 и SM100 — две разные архитектуры инструкций SM. Потребительские и предназначенные для рабочих станций версии Blackwell используют SM120; SM100 в серверных Blackwell, таких как B200, применяет описанную выше TMEM и соответствующие матричные инструкции, а программное обеспечение выбирает kernel в соответствии с целевой архитектурой.⁵⁶

Серверные карты также оснащаются HBM и высокоскоростными межсоединениями для нескольких карт. Совокупная двунаправленная пропускная способность NVLink у A100 SXM составляет 600 GB/s, у H100 SXM — 900 GB/s, что при однонаправленной передаче соответствует 300 и 450 GB/s. По методике из раздела 4.5 время сериализации при передаче 64 MiB данных по каналу — объём данных, делённый на однонаправленную пропускную способность — составляет соответственно около 224 и 149 μ s. При увеличении масштаба вычислений обмен данными на каждом слое при тензорном параллелизме, когда матрица одного слоя распределяется между несколькими картами, обмен градиентами при обучении и диспетчеризация экспертов многократно используют эти соединения. RTX 4090 и 5090 не имеют NVLink; RTX 3090 поддерживает двухкарточный

⁵⁶ Спецификация RTX PRO 6000: ECC, MIG и конфигурация продукта; Hopper Tuning Guide: NVLink; техническое описание GA102: NVLink в 3090; Blackwell Tuning Guide и возможности Blackwell в CUTLASS: SM100/SM120.

NVLink. В главах 6 и 7 параметры этих соединений будут использованы для расчёта времени тензорного разбиения, обмена градиентами и диспетчеризации экспертов.⁵⁷

Вернёмся к сравнению вычислений и пропускной способности: относительно A100 скорость матричных операций BF16 у H100 возрастает примерно в 3,17 раза, пропускная способность — примерно в 1,64 раза, а также появляются FP8, TMA и новые механизмы совместной работы. В H200 относительно H100 главным образом увеличиваются ёмкость и пропускная способность, тогда как приведённая в таблице скорость матричных операций BF16 практически не меняется. Эти две группы изменений соответственно представляют подходы «совместно изменить вычисления и организацию выполнения» и «сохранить вычислительную мощность, усилив подсистему хранения». После подстановки модельной нагрузки в разделе 4.8.2 можно будет определить, какая группа улучшений полезнее.

4.6.2 Ascend: от развёртки свёрток CNN к взаимодействию матриц и векторов Transformer

При проектировании раннего Ascend 910, который в последующем сравнении поколений называется 910A, большую часть обучающей нагрузки составляли свёрточные нейронные сети (CNN), такие как ResNet. Авторский обзор разработки описывает этот контекст, а статья о DaVinci дополнительно раскрывает распределение задач между Scalar, Vector, Cube и MTE. MTE содержит модули `img2col`, транспонирования и распаковки; `img2col` во время перемещения данных организует окна свёртки в матричные входные данные. Сначала рассчитаем, какую работу эта специализированная возможность устранила в CNN, а затем заменим нагрузку на Transformer.⁵⁸

Почему развёртку свёртки стоит поручить оборудованию? Возьмём свёртку в стиле ResNet размером 3×3 со входом $56 \times 56 \times 64$, 64 выходными каналами и шагом 1, сохраняющую пространственный размер с помощью дополнения (`padding`). Вход хранится в FP16 и занимает $56 \times 56 \times 64 \times 2 = 401\,408$ bytes, то есть около 0,383 MiB. Для каждой выходной позиции используются 64 канала в девяти соседних позициях; развёрнутая матрица имеет размер 3136×576 и занимает около 3,445 MiB — в девять раз больше исходного входа.

Если сначала сформировать развёрнутую матрицу в HBM, а затем прочитать её для матричного умножения, дополнительно выполнятся одна запись и одно чтение по 3,445 MiB, всего 6,891 MiB. Пропускная способность HBM раннего 910 составляет 1,2 TB/s, поэтому только запись и чтение этой развёрнутой матрицы занимают около 6,02 μ s времени пропускной способности. Выполняя развёртку окна непосредственно при перемещении данных на кристалле, MTE устраняет запись и чтение этой промежуточной копии в HBM. После организации входного окна на кристалле оно напрямую передаётся в матричные вычисления, а в HBM сохраняются исходные входные данные и веса.⁵⁹

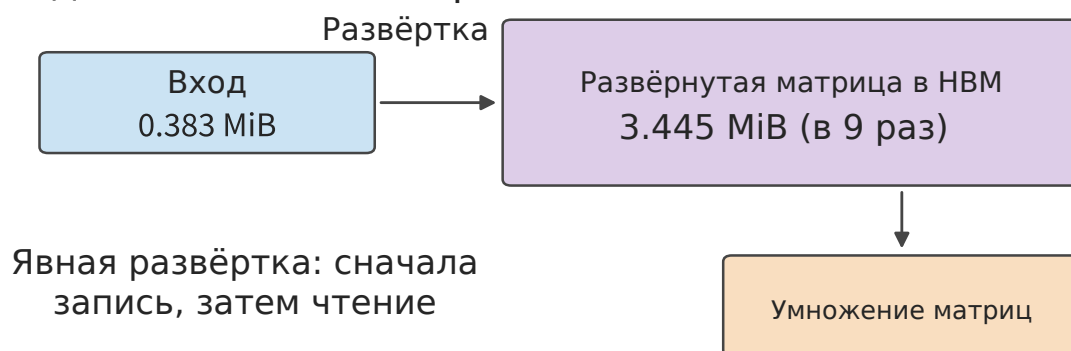
Теперь рассмотрим распределение вычислений. Эта свёртка требует $2 \times 3136 \times 576 \times$

⁵⁷ Спецификация RTX PRO 6000: ECC, MIG и конфигурация продукта; Hopper Tuning Guide: NVLink; техническое описание GA102: NVLink в 3090; Blackwell Tuning Guide и возможности Blackwell в CUTLASS: SM100/SM120.

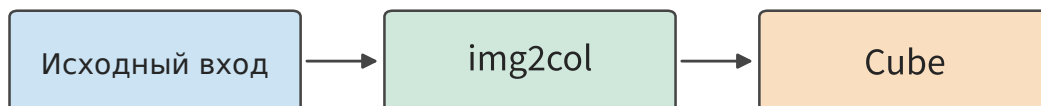
⁵⁸ Статья автора «Где должен находиться интеллект сети», обзор предпосылок проектирования ResNet в 2016 году из выступления 2023 года; статья об архитектуре DaVinci, §3.2, §3.4, таблица 5 и система Ascend 910: соответственно приведены `img2col`, соотношение ресурсов, пропускная способность каждого ядра и HBM с пропускной способностью 1,2 TB/s.

⁵⁹ Статья об архитектуре DaVinci, §3.1–3.4; руководство по Ascend C, глава 4 об интегрированной и разделённой архитектурах; CloudMatrix384 v2, §3.3.1 и §4.2.2.

Вход $56 \times 56 \times 64$, свёртка 3×3



МТЕ формирует окна на кристалле



Устранены запись и чтение 6.891 MiB в HBM $\approx 6.02 \mu s$

Рис. 4-32. Два тракта подготовки одной и той же свёртки 3×3 . Явная развёртка формирует промежуточную матрицу в девять раз большего размера; МТЕ организует окна на кристалле, устраняя запись и чтение 6,891 MiB промежуточных данных в HBM.

$64 \approx 231,2$ MFLOPs. Для Ascend-Max Cube в статье о DaVinci указано 8 192 FLOPs за такт, поэтому время обслуживания матрицы составляет 28 224 такта. Выходная ReLU обрабатывает лишь 200 704 элемента; при ширине вектора 256 bytes одна группа вмещает 128 элементов FP16, всего требуется 1 568 групп. Объем матричных вычислений свёртки и объем постобработки резко различаются, а Cube, Vector и оборудование развёртки выполняют чётко определённые задачи.

При переходе к вниманию одновременно меняются соотношение ресурсов и направление данных. Для блока внимания 128×128 с размерностью головы 128 операции QK и PV в сумме требуют около 8,39 MFLOPs, что при той же скорости Cube соответствует 1 024 тактам. Softmax должен вычислить 16 384 экспоненты, а также выполнить редукцию максимума, суммирование и нормализацию. При ширине вектора 256 bytes одна группа данных FP32 содержит 64 значения, поэтому для одних только экспонент необходимо обработать 256 групп. Чтобы этап вычисления экспоненты не превышал 1 024 такта матричной части, он должен в среднем обрабатывать одну группу за четыре такта, то есть выдавать 16 результатов экспоненты за такт. Затем выполняются редукция и нормализация, формирующие вход для PV.

На одном и том же Cube матричная часть свёртки CNN занимает 28 224 такта, а блок внимания — лишь 1 024 такта; при этом векторная работа меняется с простой ReLU на экспоненты, редукцию и нормализацию. В анализе BERT из статьи о DaVinci время большинства слоёв определяется Cube. Если отдельно рассмотреть внимание, становится видна новая потребность Softmax в пропускной способности векторных вычислений.⁶⁰

Развёртка окон свёртки, полезная в CNN, не выполняет никакой работы для проекций Q, K и V, поскольку вход проекции уже является матрицей. Внимание, напротив, требует передать результат QK в Vector для Softmax, а затем вернуть вероятности в Cube для PV. Работа по развёртке свёртки, устраняемая `img2col`, отличается от работы, необходимой для этих двух передач между матричными и векторными вычислениями; после изменения нагрузки меняется и приоритет новых аппаратных ресурсов.

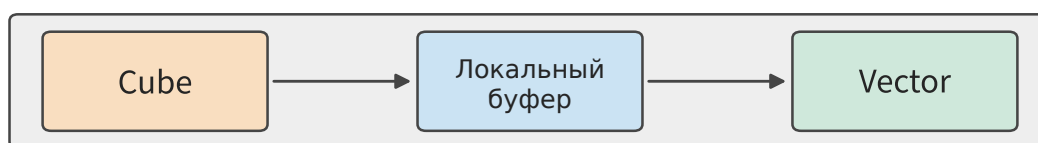
Как тракт передачи влияет на скорость при независимом планировании? В Atlas A2 с 910B матричные ядра AIC и векторные ядра AIV управляются отдельно; 910C дополнительно использует два кристалла, на каждом из которых находятся 24 AIC и 48 AIV. Раздельное управление позволяет разным видам работы продвигаться независимо, а AIC и AIV обмениваются результатами через глобальное адресное пространство, при этом фактические данные проходят через иерархию кэшей. CloudMatrix384 сжимает состояние KV в MLA и распределяет матричную и векторную работу с помощью слияния и динамического разбиения на блоки; MTP одновременно обрабатывает несколько прогнозируемых позиций, тем самым снова изменяя количество строк в этих вычислениях.⁶¹

Рассчитаем объем работы интерфейса для того же блока внимания. Оценки и вероятности передаются в FP32, каждый тензор занимает 64 KiB. Одна передача через внешний интерфейс обмена включает запись и чтение, всего 128 KiB; две передачи QK→Softmax и Softmax→PV в сумме составляют 256 KiB. Если обе передачи выполняются одним внешним интерфейсом, то для завершения за 1,024 μ s ему требуется около 256 GB/s. Если под-

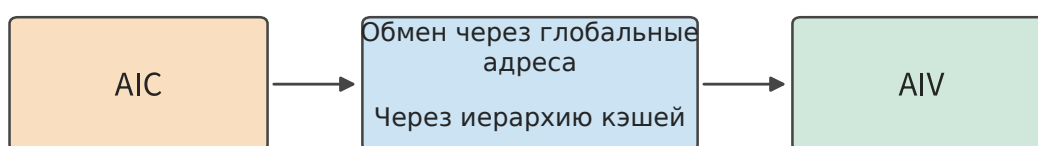
⁶⁰Статья об архитектуре DaVinci, §3.1–3.4; руководство по Ascend C, глава 4 об интегрированной и разделённой архитектурах; CloudMatrix384 v2, §3.3.1 и §4.2.2.

⁶¹Статья об архитектуре DaVinci, §3.1–3.4; руководство по Ascend C, глава 4 об интегрированной и разделённой архитектурах; CloudMatrix384 v2, §3.3.1 и §4.2.2.

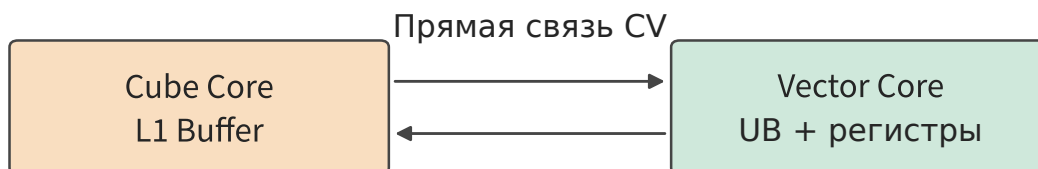
910A: разделение задач в одном ядре, MTE поддерживает img2col



910B / 910C: раздельное управление матрицами и векторами



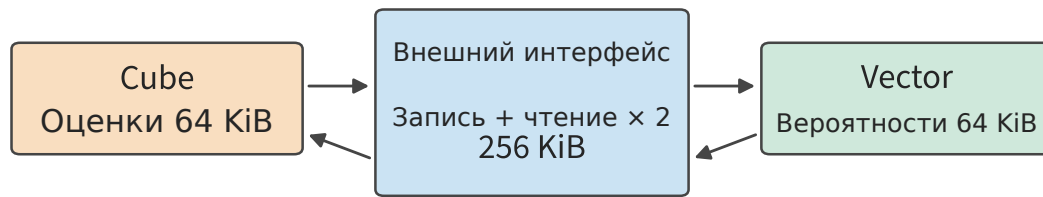
950: усиленный Vector, добавлена прямая связь CV



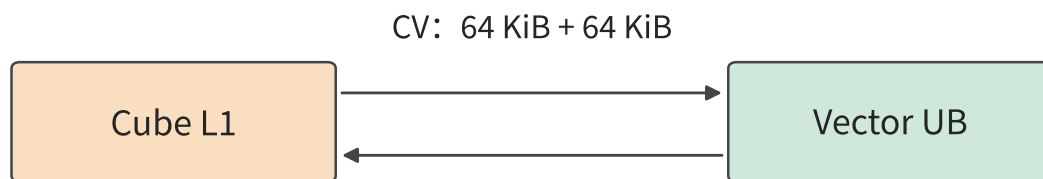
Передача между матрицами и векторами остаётся в ближнем тракте

Рис. 4-33. Раннее распределение задач внутри одного ядра, независимое управление в 910B/910C и добавленный в 950 прямой тракт CV. Соотношение вычислительных блоков и тракт обмена результатами совместно определяют эффективность объединённых операторов.

QK → Softmax → PV: две передачи



При 94 GB/s интерфейс занят $\approx 2.79 \mu\text{s}$



Для передачи за $1.024 \mu\text{s}$ CV требуется 128 GB/s

Внешний интерфейс освобождается от 256 KiB; прямой тракт передаёт 128 KiB

Рис. 4-34. Две передачи в одном блоке внимания. Внешний интерфейс обмена выполняет запись и чтение общим объёмом 256 KiB; прямой тракт CV передаёт два тензора по 64 KiB, выводя этот трафик из внешнего интерфейса. Завершение передачи за $1,024 \mu\text{s}$ требует пропускной способности прямого тракта 128 GB/s.

ставить указанную в статье о DaVinci пропускную способность кэша последнего уровня (LLC) на ядро, равную 94 GB/s, эта работа займёт около $2,79 \mu\text{s}$, что уже превышает время обслуживания матрицы Cube, равное $1,024 \mu\text{s}$ при частоте 1 GHz. Даже если матричные вычисления продолжат ускоряться, внешнему интерфейсу всё равно придётся переместить все эти байты.

Следующее изменение в Ascend 950 нацелено именно на этот внешний интерфейс: между буфером первого уровня Cube L1 и унифицированным буфером Vector (Unified Buffer, в документации Ascend сокращается до UB и не совпадает с межсоединением Unified Bus из главы 6) добавлен прямой тракт CV (Cube–Vector). Два промежуточных тензора по 64 KiB передаются через прямое соединение, всего 128 KiB, благодаря чему прежние 256 KiB записи и чтения выводятся из внешнего интерфейса. Чтобы этот прямой тракт выполнил обе передачи за $1,024 \mu\text{s}$, требуется около 128 GB/s; если формат передачи изменить на FP16/BF16, требование дополнительно сократится вдвое, примерно до 64 GB/s.

Кроме того, 950 вдвое увеличивает пропускную способность одного Vector Core для FP16/FP32, добавляет регистровый файл между унифицированным буфером и векторным арифметико-логическим устройством (ALU), а также оптимизирует выполнение Softmax и функции активации GELU. Для той же обычной векторной работы F_v величина F_v/P_v сокращается до $F_v/(2P_v)$; если ранее векторный этап

занимал вдвое больше времени, чем матричный, это изменение вновь уравнивает их. Для рассмотренного выше блока внимания пропускная способность вычисления экспоненты должна достигать 16 результатов за такт. Новый NDDMA берёт на себя подготовку многомерных адресов и раскладки, а BufferID организует синхронизированную передачу с помощью идентификаторов буферов.⁶²

Низкую точность 950 также можно рассматривать при том же объёме работы: при одинаковой частоте скорость матричных вычислений формата класса FP8 вдвое выше FP16, а MXFP4 — вчетверо выше. Для фиксированной матрицы F время обслуживания вычислений последовательно равно F/P , $F/(2P)$ и $F/(4P)$; однако размер двух передач оценок и вероятностей FP32 остаётся прежним. Работа интерфейса сократится только при одновременном изменении тракта передачи, формата или раскладки. Лишь совместная эволюция матриц, векторов и тракта CV позволяет превратить локальное ускорение в выигрыш всего фрагмента внимания.

4.6.3 Apple: как преобразовать большую ёмкость унифицированной памяти в возможности запуска моделей

CPU, GPU и отдельный Neural Engine Apple используют общую унифицированную память. Унифицированная память устраняет копирование данных между процессорами и позволяет локально выполняемым моделям использовать более крупный пул памяти. Сначала рассчитаем, какие модели помещаются в унифицированную память, а затем — какой объём данных необходимо прочитать для генерации одного токена.

Сначала сверим соотношение ёмкости и пропускной способности разных поколений. Типичная конфигурация M1 Ultra имеет 128 GB и 800 GB/s; M2 Ultra расширяет ёмкость до 192 GB при 800 GB/s; конфигурации M3 Ultra с 80-ядерным GPU имеют 256 GB и официально указанную конфигурацию 512 GB, обе с пропускной способностью 819 GB/s. M4 Max также имеет конфигурацию 128 GB, её пропускная способность составляет 546 GB/s.⁶³

Какие показатели времени изменяются при удвоении ёмкости? Сначала оставим модель, точность и batch неизменными. При увеличении ёмкости M3 Ultra с 256 до 512 GB пропускная способность остаётся равной 819 GB/s, поэтому величина V/R для одного и того же выполнения совершенно не меняется. При замене M1 Ultra на M3 Ultra пропускная способность возрастает с 800 до 819 GB/s, а скорость чтения увеличивается лишь примерно на 2,4%. Увеличение ёмкости позволяет разместить больше весов, KV или запросов, но только увеличение пропускной способности непосредственно изменяет время чтения того же объёма данных.

Возьмём Qwen3-8B в BF16, один запрос и 8 192 позиции. Приведённый в книге перечень тензоров даёт около 16,345 GB нагрузки на интерфейс весами и KV за один шаг. Нижняя граница времени чтения для M1/M2 Ultra составляет $16,345 \text{ GB} / (800 \text{ GB/s}) \approx 20,43 \text{ ms}$, для M3 Ultra — $16,345 \text{ GB} / (819 \text{ GB/s}) \approx 19,96 \text{ ms}$, для M4 Max — $16,345 \text{ GB} / (546 \text{ GB/s}) \approx 29,94 \text{ ms}$. При одинаковых 128 GB M1 Ultra

⁶²Официальное техническое описание архитектуры Ascend 950, §4.1–4.1.6; постраничные ссылки на раннюю архитектуру DaVinci и разделённую архитектуру CANN см. в заметках по сравнению.

⁶³Сверка официальных конфигураций Apple, сочетания GPU и объёма памяти, а также таблица аппаратных характеристик. Конфигурация M3 Ultra с 512 GB перекрёстно подтверждена официальным пресс-релизом и записью о тестовой конфигурации готовой системы с 80-ядерным GPU. 512 GB и 1 200 GB/s для M5 Ultra — характеристики, объявленные в августе 2026 года; поставка конфигурации с 512 GB запланирована на конец октября 2026 года.

читает данные этого шага примерно на 9,51 ms быстрее M4 Max.⁶⁴

Основную ценность ёмкости проще увидеть на примере крупной МоЕ. При использовании принятой в книге 4-битной схемы хранения резидентные веса Qwen3-72B-A22B вместе с масштабными коэффициентами и тензорами, сохраняемыми в высокой точности, занимают около 123,142 GB. Один запрос с контекстом 8K, KV и рабочей областью 2 GiB требует в сумме около 126,867 GB. В системе со 128 GB остаётся лишь около 1,13 GB; если дополнительно зарезервировать 8 GB для операционной системы и других приложений, потребуется около 134,87 GB. Конфигурации с 256 или 512 GB имеют достаточно места для этого бюджета.

На каждом шаге МоЕ обращается только к выбранным экспертам. Для того же единственного запроса нагрузка на интерфейс весами и KV за шаг составляет около 13,697 GB, что на M3 Ultra требует около 16,72 ms, а не около 150 ms, которые получились бы при чтении всех 123,142 GB весов на каждом шаге. Для МоЕ полный объём весов определяет необходимую ёмкость, а выбранные на текущем шаге эксперты — объём чтения весов за шаг. Поэтому крупная МоЕ может одновременно «требовать много памяти» и «читать сравнительно мало на каждом шаге».⁶⁵

Теперь подставим результаты маршрутизации восьми запросов из раздела 4.3.3. Объём чтения за шаг при концентрированной и распределённой маршрутизации составляет соответственно 24,737 GB и 75,967 GB, что на M3 Ultra соответствует 30,20 ms и 92,76 ms. За шаг выдаются восемь токенов, поэтому бюджет пропускной способности чтения составляет соответственно $8/0,03020 \approx 265$ token/s и $8/0,09276 \approx 86$ token/s. Конфигурации с 256 и 512 GB способны вместить одну и ту же задачу с восемью запросами, а различие между двумя бюджетами чтения здесь определяется повторным использованием при маршрутизации.

Как включить в этот расчёт локальное хранилище и специализированные блоки GPU? В M3 появился Dynamic Caching, распределяющий локальную память GPU в соответствии с потребностями выполнения; M4 сохраняет эту организацию. Такие механизмы влияют на резидентность работы и использование ресурсов на кристалле, тогда как ёмкость модели определяется унифицированной памятью. В M5 в каждое ядро GPU дополнительно встроен Neural Accelerator, доступный через такие интерфейсы, как Metal 4 Tensor API; отдельный Neural Engine при этом сохраняется. Специализированные блоки внутри GPU ускоряют обработку матриц, а подготовка данных и операции общего назначения по-прежнему организуются программой GPU.^{66,67}

Пропускная способность памяти также растёт от поколения к поколению: у базовых M4 и M5 пропускная способность унифицированной памяти увеличивается со 120 до 153 GB/s, то есть примерно на 27,5%. Время чтения рассмотренных выше 32 MiB весов сокращается примерно с 279,6 μ s до 219,3 μ s, то есть примерно на 21,6%. Заявленная Apple пиковая производительность GPU M5 в задачах AI более чем в четыре раза превышает M4; чтение весов для той же однострочной проекции по-прежнему определяется указанной выше пропускной способностью. При увеличении числа входных строк чтение

⁶⁴Программа количественного расчёта эволюции архитектуры и пояснения, подробные результаты расчётов. Параметры считываются из таблицы аппаратных характеристик; данные для 3090 дополнены по таблице 9 официального технического описания GA102. Полная конфигурация модели и перечень тензоров взяты из каталога calculations этой книги.

⁶⁵Результаты сравнения поколений памяти и описание расчётов.

⁶⁶Официальное описание архитектуры Apple M3, официальный анонс M4 и характеристики M4 Mac mini.

⁶⁷Официальные характеристики M2 Pro/Max, описание архитектуры Apple GPU, режимы хранения Metal, официальное описание Neural Accelerator в GPU M5.

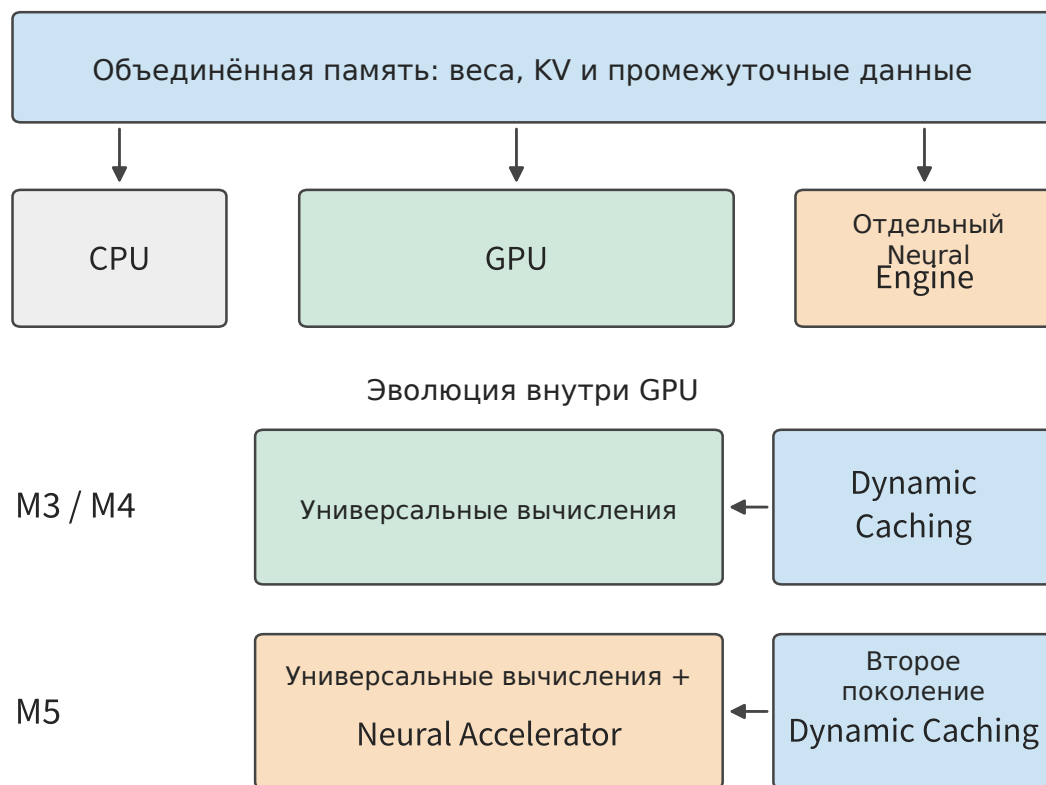


Рис. 4-35. Унифицированная память отвечает за общесистемную ёмкость данных, Dynamic Caching управляет локальными ресурсами GPU, а Neural Accelerator в M5 добавляет специализированные вычислительные возможности внутри GPU. Эти три компонента соответственно учитываются при анализе ёмкости, резидентности и времени вычислений.

весов распределяется на больший объём вычислений, и только тогда новые матричные блоки с большей вероятностью становятся основным источником выигрыша. Это явление полностью соответствует результатам для двух вариантов числа строк на V100.⁶⁸

Эти направления эволюции превращают обратную связь на рис. 4-3 в конкретные изменения ресурсов: большие матрицы стимулируют появление специализированных блоков умножения со сложением, Transformer увеличивает требования к векторным вычислениям и передаче данных, а крупные МоЕ повышают требования к ёмкости для резидентного размещения.

4.7 Специализированные архитектуры

В предыдущих разделах эффективность универсальных ускорителей повышалась за счёт повторного использования, конвейеризации и локальности. Если некоторые условия рабочей нагрузки длительное время остаются стабильными, их можно непосредственно учесть при проектировании аппаратуры: зафиксировать правила вычислений, порядок обмена данными и даже веса. Специализация сокращает объём одних видов работы и позволяет сосредоточить больше ресурсов на оставшихся.

4.7.1 TPU

Вычисления при инференсе на TPU v1 сосредоточены вокруг регулярного матричного умножения. Массив образуют 256×256 блоков умножения с накоплением; веса загружаются в него через расположенную на кристалле очередь «первым пришёл — первым вышел» (FIFO, буфер, из которого данные извлекаются в порядке поступления), входные данные подаются из Unified Buffer, а результаты сохраняются в отдельной памяти накопителей и затем передаются блоку активации. Одна инструкция может описывать крупную матричную операцию, а локальные соединения многократно передают операнды, сокращая объём управления и дальних обращений, необходимый для каждого умножения с накоплением.⁶⁹

Один блок 8-битных весов размером 256×256 занимает 64 KiB. Если обрабатывать только одну строку входных данных, выполняется $2 \times 256^2 = 131\,072$ операций — по две операции на каждый байт весов; при обработке 256 строк выполняется около 33,6 млн операций — по 512 операций на байт весов. Веса остаются неизменными, а объём вычислений возрастает в 256 раз — это и есть реализация в массиве соотношения повторного использования при проекции из раздела 4.1.

Для получения такого выигрыша массиву требуется непрерывный поток строк входных данных. FIFO весов вмещает четыре таких блока весов, а загрузку весов и вычисления можно согласовать посредством буферизации; пока входные данные непрерывно поступают в массив, отдельная память накопителей хранит частичные суммы. Поэтому большой массив, входной буфер и очередь весов работают совместно: массив предоставляет блоки умножения с накоплением, а буферы обеспечивают их непрерывное снабжение необходимыми операндами.

⁶⁸Официальные характеристики M2 Pro/Max, описание архитектуры Apple GPU, режимы хранения Metal, официальное описание Neural Accelerator в GPU M5.

⁶⁹Jouppi и др., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017, архивная версия статьи.

При запуске массива операнды должны распространиться по соединениям до всех вычислительных позиций, а по завершении результаты необходимо собрать для вывода. Чем больше блоков вычисляется непрерывно, тем меньше приходится на каждый из них времени заполнения и опустошения конвейера; небольшие задачи сильнее зависят от этих фиксированных этапов.

При изменении рабочей нагрузки необходимо корректировать и соотношение аппаратных блоков. Обучение увеличивает объём изменяемого состояния, градиентов и обмена между ускорителями; генерация с длинным контекстом увеличивает объём KV и требует на каждом шаге ожидать результат предыдущего. В Google TPU 8t/8i обучение и обслуживание сэмплирования организованы отдельно, что отражает различия в требованиях этих этапов к матричным и векторным вычислениям, памяти и межсоединениям.⁷⁰ Общий принцип TPU состоит в распределении ресурсов вокруг заранее известной работы, тогда как конкретные пропорции изменяются вместе с основной рабочей нагрузкой.

4.7.2 Groq, Graphcore и Cerebras

Сохранение большего объёма данных на кристалле позволяет сократить число дальних обращений. Развивая эту идею, можно распределить вычисления и память между множеством локальных блоков либо дополнительно увеличить размер кристалла, разместив на нём больше таких блоков. Оба подхода позволяют использовать большие объёмы данных расположенными поблизости вычислительными блоками и требуют организации обмена между ними.

В тензорном потоковом процессоре Groq (Tensor Streaming Processor, TSP) компилятор заранее планирует моменты создания, передачи и использования тензоров. Интеллектуальный процессор Graphcore (Intelligence Processing Unit, IPU) состоит из множества вычислительных блоков с локальной памятью, которые Graphcore называет tile; эти блоки совместно выполняют задачи посредством вычислений, синхронизации и обмена данными. Cerebras размещает большое количество вычислительных блоков и памяти в масштабе целой пластины; третье поколение пластинного процессора WSE-3 содержит 44 GB SRAM на кристалле. SRAM подходит для непосредственной интеграции в вычислительный кристалл и обеспечивает доступ с низкой задержкой ценой занимаемой площади.⁷¹ Эти архитектуры реализуют локальность в разных масштабах: они стремятся максимально сократить пути передачи данных и размещать многократно используемые данные рядом с вычислениями.

Сложность распределённой ёмкости состоит в том, что потребности разных участников неодинаковы. Если один блок освободил пространство в памяти, другие блоки не могут непосредственно использовать его как собственную локальную память; для переконфигурации необходимо перенести данные или вычисления. Особенно часто такая ситуация возникает в моделях с экспертами при неравномерной маршрутизации: при одинаковом общем объёме запросы, сосредоточенные на нескольких экспертах, сильнее загружают небольшое число вычислительных блоков.

Неравномерность использования локальной памяти можно смягчить перераспределением задач, однако по мере роста масштаба обслуживания обостряется другая проблема: увеличение общего объёма KV. Для той же Qwen3-8B запрос с 8 192 токенами содер-

⁷⁰Inside the Eighth-Generation TPU: An Architecture Deep Dive.

⁷¹Статья о Groq TSP, IPU Programming Model, спецификация Cerebras WSE-3, статья о SambaNova SN40L.

жит 1,125 GiB KV в формате BF16. Каждая микросхема Groq TSP имеет 220 MiB SRAM,⁷² поэтому для хранения одних лишь весов BF16 требуется 72 микросхемы. KV хранится на дополнительных микросхемах и может быть равномерно разделён между ними: одному запросу требуется $\lceil 1152/220 \rceil = 6$ микросхем; восьми запросам с общим объёмом 9 GiB — 42 микросхемы; если контекст каждого из восьми запросов увеличить до 32 768 токенов, потребуется 168 микросхем.

При переходе от 6 к 42, а затем к 168 увеличивается объём изменяемого состояния, а не весов модели. Хранение весов на кристалле устраняет один вид чтения, однако непрерывное обслуживание также требует ёмкости для растущего состояния. С увеличением числа микросхем распределённый между ними KV необходимо собирать посредством межкристального обмена; потребности в расширении ёмкости и передаче данных одновременно порождаются одной и той же рабочей нагрузкой.

Масштаб пластины сокращает часть соединений, которые иначе проходили бы между корпусами, но данные всё равно перемещаются по сети через несколько вычислительных блоков. Размещение поблизости вычислений, использующих одно и то же состояние, сокращает число задействованных каналов и синхронизаций; равномерное распределение задач, напротив, позволяет задействовать больше вычислительных позиций. При распределении вычислительных задач необходимо одновременно обеспечивать локальное повторное использование данных и не допускать, чтобы несколько блоков были перегружены, пока остальные простаивают. Этот принцип совпадает с принципом размещения на нескольких кристаллах.

4.7.3 Фиксированные потоки данных и веса

Специализация использует заранее определённые условия выполнения для перераспределения аппаратных ресурсов. Если зафиксировать набор операторов, можно упростить инструкции и управление; если зафиксировать поток данных, можно заранее спланировать передачу; если зафиксировать числовой формат, память и вычислительные блоки можно спроектировать под требуемую разрядность; если зафиксировать веса, для неизменяемых параметров можно создать специализированную структуру хранения. Управление и пути данных, изначально предназначенные для обработки множества вариантов, при этом можно соответствующим образом упростить.

При изменении модели реконфигурируемые архитектуры адаптируются к новым вычислениям посредством обновления соединений и расписания. Организация потоков данных SambaNova и реконфигурируемые пути FPGA позволяют перенастраивать часть соединений и порядок выполнения, тогда как архитектуры с фиксированными весами организуют хранение и вычисления вокруг конкретных параметров.⁷³

Фиксированные веса не означают возможность выполнять только одну задачу. Изменяя входные данные, одну и ту же модель можно использовать для разных высокоуровневых задач. Одинаковый набор весов модели способен как отвечать на вопросы по документам, так и предлагать изменения на основе кода и результатов тестов; каждый вызов использует собственное состояние контекста. Поэтому аппаратные пути весов можно спроектировать под выполнение общей модели, а контекст каждого запроса хранить в изменяемой памяти. Ниже веса и контекст измеряются отдельно, чтобы вычислить, какую ёмкость и пропускную способность позволяет высвободить такое разделение.

⁷²Статья о Groq TSP, IPU Programming Model, спецификация Cerebras WSE-3, статья о SambaNova SN40L.

⁷³Статья о Groq TSP, IPU Programming Model, спецификация Cerebras WSE-3, статья о SambaNova SN40L.

В проекте автора OpenTallas исследуется архитектура, в которой неизменяемые веса помещаются в масочную ROM; масочная ROM — это постоянное запоминающее устройство, содержимое которого фиксируется при производстве. Ниже на примере Qwen3-8B из этого проекта с помощью формул оценивается, как архитектура, в которой веса читаются из отдельной ROM, а KV читается и записывается в HBM, изменяет объём передачи через каждый интерфейс.⁷⁴ Для каждого пакета необходимые на текущем шаге веса читаются только один раз и составляют около $W = 15.1$ GB; для каждого запроса полностью считывается KV для 8 192 токенов объёмом около $K = 1.21$ GB. Полностью размещённые в памяти веса занимают около 16,4 GB, а на каждом шаге считывается участвующая в вычислениях их часть.

Сначала рассмотрим исходный объём чтения через один интерфейс. Когда веса и KV совместно используют HBM, на каждом шаге считывается $W + BK$. После переноса весов в отдельную ROM на HBM остаётся только BK . Отношение объёмов чтения из HBM в этих двух случаях равно

$$S_{\text{HBM}} = \frac{W + BK}{BK} = 1 + \frac{W}{BK}.$$

При $B = 1$ объём чтения из HBM сокращается примерно с 16,3 GB до 1,21 GB, а время чтения — приблизительно до $1/13.5$ исходного. При $B = 16$ объём KV уже составляет около 19,3 GB, а общий объём сокращается примерно с 34,5 GB до 19,3 GB, то есть приблизительно в 1,8 раза. Увеличение пакета уже распределяет веса между большим числом запросов, поэтому относительная выгода от дополнительного устранения чтения весов уменьшается.

Положив $BK = W$, можно найти точку перехода, в которой объёмы чтения KV и весов равны: она соответствует примерно $B = 12.5$; начиная с целочисленного размера пакета 13, объём чтения KV превышает объём чтения весов. На рисунке 4-36 этот переход показан горизонтальной и наклонной линиями.

При наличии отдельного интерфейса ROM веса и KV можно считывать одновременно. Пусть пропускная способность двух интерфейсов равна соответственно β_{ROM} и β_{HBM} . Если модель, точность, контекст и размер пакета остаются неизменными, нижняя граница времени работы памяти при перекрытии двух путей равна

$$t_{\text{memory}} \geq \max \left(\frac{W}{\beta_{\text{ROM}}}, \frac{BK}{\beta_{\text{HBM}}} \right).$$

Увеличение пропускной способности ROM сокращает первое слагаемое, пока более медленным путём не становится чтение KV. Если целевая скорость для одного пользователя составляет 10 000 токенов/с, на каждый токен приходится всего 100 μs , и даже для 1,21 GB KV требуется пропускная способность HBM около 12,1 TB/s.

Отдельная ROM также высвобождает ёмкость HBM, ранее занятую весами. При фиксированном контексте 8K, если использовать один стек восьмислойной HBM3E ёмкостью 24 GB из раздела 4.1.3 как хранилище KV, после вычета 2 GiB рабочей области и при бюджете KV 1,125 GiB на запрос можно разместить 18 запросов; если же в той же памяти требуется хранить полные веса BF16, помещается только 4 запроса.⁷⁵ При увеличении контекста до 32K объём чтения KV для каждого запроса возрастает в четыре раза, поэто-

⁷⁴Пример OpenTallas.

⁷⁵Поэлементный пересчёт сравнений условий в этой главе и программа расчёта.

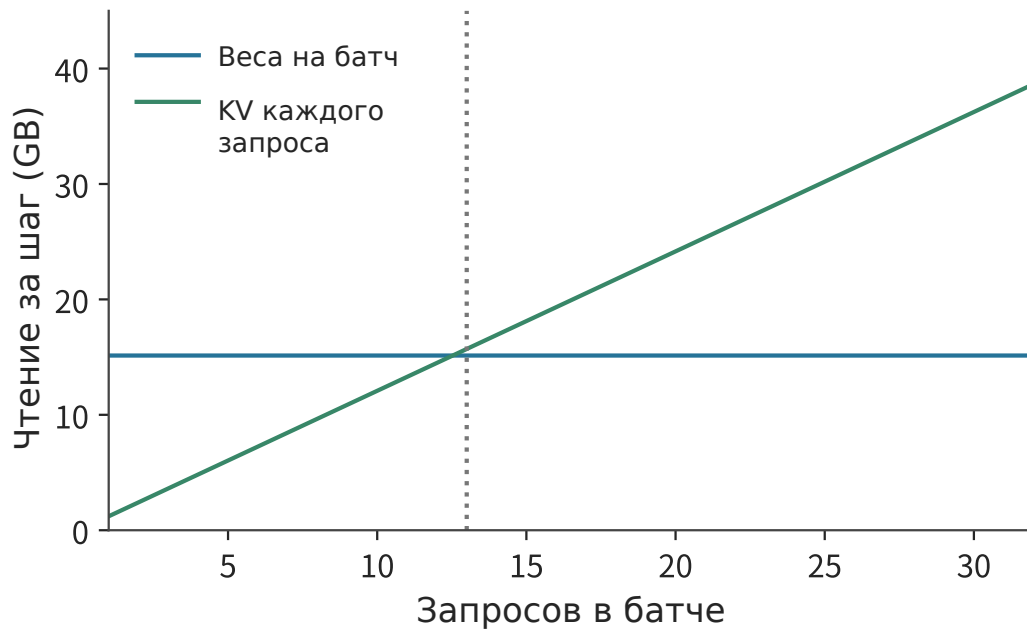


Рисунок 4-36. При фиксированных весах KV по-прежнему считается отдельно для каждого запроса. По условию для каждого запроса сохраняется контекст 8K; начиная с размера пакета 13, объем чтения KV превышает объем чтения общих весов.

му приведённая выше точка равенства объёмов весов и KV снижается примерно с 12,5 до 3,13 запроса.

То же ограничение в 100 μ s можно использовать для расчёта числа вычислительных блоков. Рабочая нагрузка взята из исторических записей выполнения проекта: набор тензорных операций общим объёмом 15 134 641 792 операции. Если каждый вычислительный канал (lane) при частоте 1 GHz выполняет одну операцию умножения со сложением (FMA) за такт, то необходимое для выполнения за 100 μ s число вычислительных каналов составляет не менее

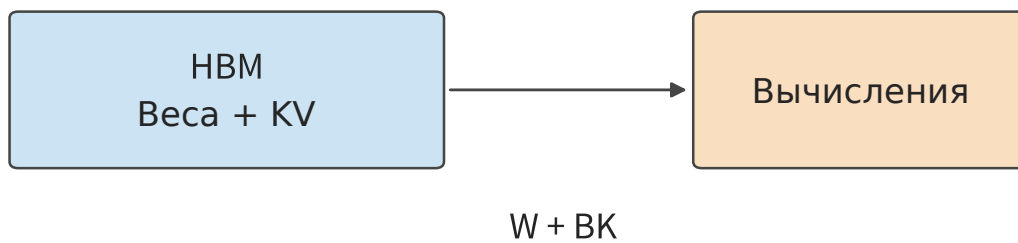
$$n_{\text{lane}} \geq \left\lceil \frac{15\,134\,641\,792}{2 \times 10^9 \times 100 \times 10^{-6}} \right\rceil = 75\,674.$$

256 вычислительных каналов выполняют эти операции примерно за 29,6 ms. Если на матричные вычисления отводится только 40 μ s, а эффективная утилизация составляет 60%, потребуется 315 306 вычислительных каналов — примерно в 4,2 раза больше первоначально необходимого количества. Чем жёстче временной бюджет, тем меньше остаётся запаса на ожидание и тем выше должны быть выделенные вычислительная мощность и пропускная способность памяти.

4.8 Анализ производительности и выбор ускорителя на основе архитектуры

В разделах 4.6 и 4.7 было рассчитано, как различные архитектуры изменяют объём вычислений, хранения и обмена данными. В этом разделе полученные результаты используются для выбора устройства: сначала строится сводная модель времени выполнения,

Общий интерфейс весов и состояния



Отдельный канал весов только для чтения

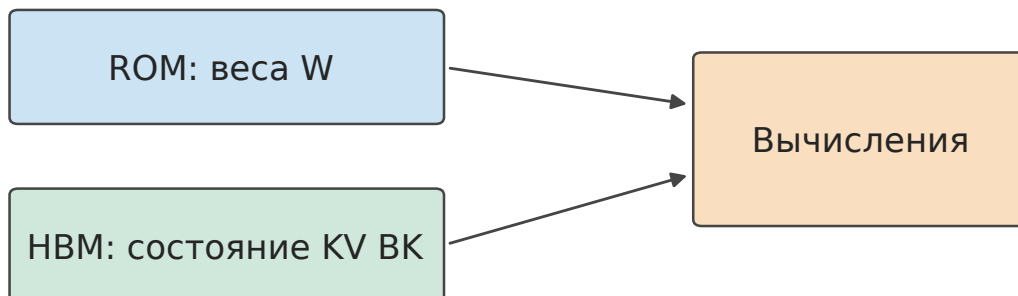


Рисунок 4-37. Отдельное постоянное хранилище весов изменяет два пути доступа к памяти. В верхней части веса и KV конкурируют за HBM; в нижней ROM предоставляет веса, а HBM хранит изменяемое состояние. Стрелки обозначают чтение; для KV также требуется запись нового состояния.

затем с помощью одной и той же модели сравниваются устройства-кандидаты, после чего модель калибруется по результатам фактических измерений и, наконец, сопоставляются стоимость и энергопотребление выполнения задачи.

При выборе оборудования также необходимо сохранить пригодное для последующего разбиения и планирования описание ресурсов: эффективную вычислительную производительность при целевой точности и заданной форме матриц, ёмкость и пропускную способность каждого уровня памяти, ограничения регистров и общей памяти, а также однонаправленную пропускную способность между устройствами, задержку запуска и общий выходной канал. В главе 5 локальные ограничения ресурсов используются для определения параметров *tile* (блоков данных, на которые разбивается матрица), буферов и параллелизма, а в главах 6 и 7 параметры межсоединений — для ограничения размещения между картами и коммуникационных групп. Требования одной и той же модели к вычислениям и данным остаются неизменными, однако то, на ресурсы какого уровня приходится эти требования и по каким путям проходят данные, влияет на время выполнения и выбор наиболее подходящего разбиения.

4.8.1 Построение модели времени выполнения на основе бюджета ресурсов

Время выполнения определяется двумя типами отношений: какие операции должны последовательно выполняться одним и тем же аппаратным блоком и каких результатов должна ожидать следующая задача. Первое определяет, как долго должен работать каждый блок, второе — порядок выполнения этапов.

Сначала рассмотрим совместно используемые ресурсы. Один и тот же матричный блок обрабатывает два типа задач объёмом по 1 млрд операций со скоростью соответственно 100 и 200 Gops/s. Они занимают 10 и 5 ms, то есть суммарно 15 ms. Если объём работы типа i на ресурсе r равен $w_{r,i}$, а соответствующая скорость — $p_{r,i}$, то время, необходимое этому аппаратному блоку для выполнения операций, равно

$$T_r = \sum_i \frac{w_{r,i}}{p_{r,i}}.$$

Когда веса и KV используют общий интерфейс памяти, их объёмы передачи данных складываются; когда QK и PV используют общий матричный блок, складывается время их вычисления. Даже если эти задачи распределены между разными операторами, они расходуют одну и ту же аппаратную мощность.

Теперь рассмотрим независимые ресурсы. Если матричные вычисления и передача данных могут выполняться одновременно, общее время выполнения не может быть меньше времени, необходимого блоку с наибольшим временем выполнения для завершения всех операций, поэтому $T \geq \max_r T_r$. Анализ установившегося режима в разделе 4.2 использовал именно это соотношение: если вычисление экспоненты не ускоряется, матричный блок после досрочного завершения работы простаивает, а темп завершения блоков по-прежнему определяется вычислением экспоненты.

Наконец, добавим зависимости. Предположим, что на первом этапе матричным вычислениям требуется 10 μ s, а памяти — 1 μ s; на втором этапе матричным вычислениям требуется 1 μ s, а памяти — 10 μ s. Внутри каждого этапа операции могут перекрываться, но второй этап должен дожидаться полного завершения первого. Поэтому первый этап занимает 10 μ s, второй — также 10 μ s, а всего требуется 20 μ s. Если просто суммировать нагрузку по ресурсам, получится по 11 μ s для матричного блока и памяти. Разница в 9

μs возникает из-за зависимости между этапами: матричные вычисления первого этапа и обращения к памяти второго должны выполняться последовательно и не могут перекрываться.⁷⁶

Чтобы применить эти соотношения к конкретному устройству, сначала нужно выбрать правильную скорость: производительность матричного блока также зависит от точности входных данных и аккумуляции. Пиковая производительность Tensor Core у RTX 4090 при выполнении плотных матричных операций составляет около 330 TFLOP/s для входных данных FP16 и аккумуляции FP16 и около 165 TFLOP/s для аккумуляции FP32; для проекции BF16/FP32 в этой главе используется 165,2 TFLOP/s.⁷⁷ Для 8,59 GFLOPs вычисление с этими двумя скоростями занимает соответственно около 26 и 52 μs . Рассчитанное далее время чтения составляет около 37 μs , поэтому выбор скорости напрямую меняет соотношение между длительностью вычислений и чтения.

После определения времени использования ресурсов и зависимостей выполнения можно также найти точку смены узкого места при изменении нагрузки. Для рассматриваемой в этой главе проекции Q модель Roofline выражает этот переход через соотношение арифметической интенсивности и пропускной способности.

Пусть пиковая производительность матричного блока равна P , пропускная способность внешней памяти — R , а объём вычислений и объём обращений к памяти для проекции — соответственно F и V . Время, необходимое для вычислений и передачи данных, равно соответственно F/P и V/R . Если вычисления и обращения к памяти полностью перекрываются, нижнюю границу времени определяет большее из этих значений, а соответствующая верхняя граница пропускной способности в модели Roofline равна

$$P_{\text{attainable}} \leq \min(P, RI), \quad I = F/V.$$

Здесь наклонная линия RI показывает, какой объём вычислений способна обеспечить пропускная способность памяти, а горизонтальная линия P — какой объём вычислений способен выполнить матричный блок. Они пересекаются при $I^* = P/R$. Ниже точки пересечения данные поступают медленнее; выше неё на каждый байт уже приходится достаточно много операций, поэтому новым узким местом становится скорость вычислений. Эта ломаная задаёт допустимую аппаратным обеспечением верхнюю границу, и любая измеренная точка находится ниже неё; расстояние от измеренной пропускной способности до ломаной соответствует коэффициенту использования, определённого в разделе 1.2.2. Если арифметическая интенсивность выше точки пересечения, применяется MFU, если ниже — MBU.

Для проекции из раздела 4.1, где входные данные и веса читаются один раз, а выходные данные записываются один раз, пиковая производительность матричных вычислений RTX 4090 при входных данных BF16 и аккумуляции FP32 составляет 165,2 TFLOP/s, а пропускная способность памяти — 1,008 TB/s. Подставив тот же набор значений F и V , получаем:⁷⁸

⁷⁶Описание моделирования нижней границы времени каждого этапа; фактический диапазон операторов модели см. в результатах по ресурсам Qwen3-8B prefill128.

⁷⁷Источники данных об аппаратном обеспечении и аудит точности, а также официальная базовая таблица.

⁷⁸Проекция Q, RTX 4090, M=1; M=256.

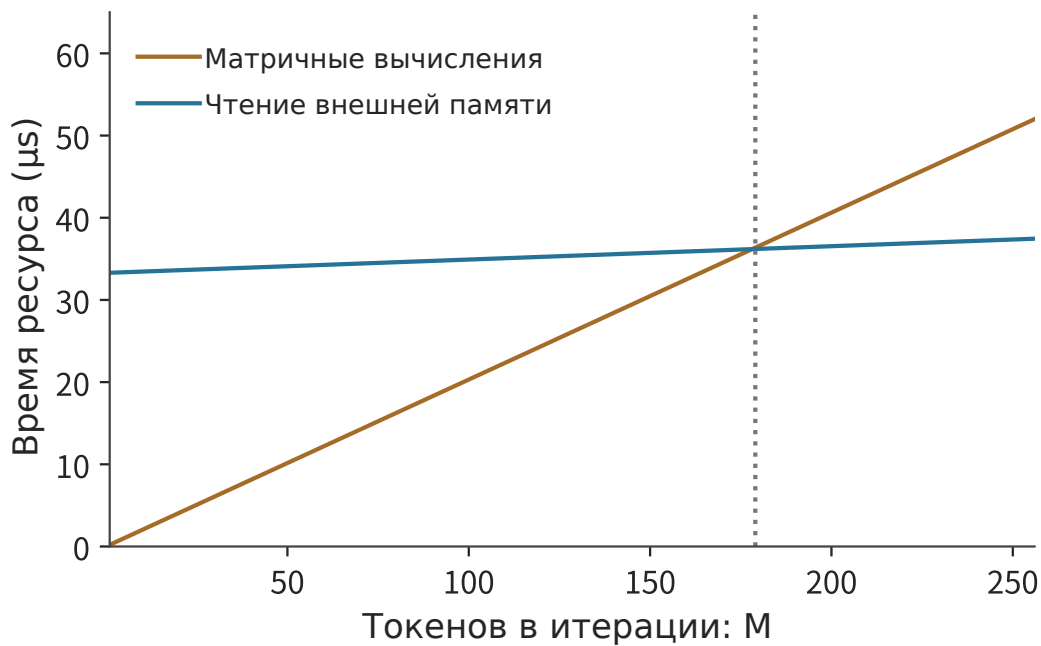


Рисунок 4-38. Изменение времени вычислений и обращений к памяти для одной и той же проекции Q при изменении числа входных строк. Объем вычислений растёт вместе с числом строк, а обращения к внешней памяти включают как фиксированный объем весов, так и растущие объёмы входных и выходных данных; начиная со 179 строк вычисления занимают больше времени.

Число обрабатываемых в этот раз токенов M	Время матричных вычислений	Время передачи данных между чипом и внешней памятью	Максимальное из двух значений времени	Доминирующий ресурс
$M = 1$	около 0,20 μs	около 33,3 μs	около 33,3 μs	память
$M = 256$	около 52 μs	около 37,4 μs	около 52 μs	матричный блок

Можно также найти число входных строк, при котором происходит переход. Пусть $d = 4096$; тогда интенсивность из раздела 4.1 можно записать как $I(M) = Md/(d + 2M)$. Приравняв её к $I^* = P/R$, получаем

$$M^* = \frac{I^* d}{d - 2I^*}.$$

Подставив неокруглённые значения скоростей, получаем $I^* \approx 164$ FLOPs/byte и $M^* \approx 178.1$. Следовательно, начиная с целого числа строк 179, время матричных вычислений превышает время передачи данных. Точка перехода превращает утверждение «более крупный батч лучше подходит для матричного блока» в вычисляемое пороговое число строк.⁷⁹

Эксперимент 4-5 · Основной: как батч изменяет узкое место производительности разных архитектур

Выведите формулу и постройте график арифметической интенсивности проекции Q для числа строк от 1 до 256. Затем по отдельности удвойте пропуск-

⁷⁹Трёхслотовый конвейер, вариант с удвоенными вычислениями и точки перелома в проектировании сгенерированы учебным скриптом вывода; полные временные диаграммы см. в данных вывода.

ную способность матричных вычислений и пропускную способность памяти и найдите число входных строк, при котором время вычислений равно времени чтения. После этого добавьте в таблицу ресурсов механизм внимания и матрицы экспертов и объясните, как длина контекста и распределение числа входных строк между экспертами изменяют время обработки каждого блока и расположение узкого места. Выберите два конкретных ускорителя, оцените время выполнения одной и той же задачи по параметрам их ресурсов и сравните узкие места.

В разделе 4.8.2 этот метод распространяется с одной проекции на всю модель: отдельно вычисляются основные объёмы чтения при декодировании и матричных вычислений при предварительном заполнении, а также проверяется, какие устройства способны вместить необходимое состояние.

4.8.2 Сравнение ускорителей на одной и той же модели

Сначала зафиксируем пример, удобный для повторного расчёта: Qwen3-8B, BF16, один запрос, 8 191 предшествующих позиций, после добавления на текущем шаге — 8 192 позиции. Постоянно размещённые веса занимают около 16,381 GB, KV-кэш — около 1,208 GB, рабочая область принята равной 2 GiB, общий бюджет — около 19,737 GB. Основная нагрузка на интерфейс памяти от весов и KV-кэша при одном шаге decode составляет 16,345 GB. На каждом шаге для эмбединга токена считывается строка, соответствующая текущему токenu, а выходная голова считывает веса целиком, поэтому объём постоянно размещённых весов отличается от объёма весов, считываемых на каждом шаге.⁸⁰

Теперь рассчитаем матричные операции. Qwen3-8B имеет 36 слоёв, ширину скрытого состояния 4 096, ширину слоя прямого распространения 12 288, 32 головы запросов и 8 KV-голов при размерности головы 128. Число матричных параметров проекций Q, K, V, O и трёх проекций слоя прямого распространения в каждом слое равно

$$W_{\text{layer}} = 2 \times 4096^2 + 2 \times 4096 \times 1024 + 3 \times 4096 \times 12288.$$

Линейные операции всех слоёв для каждого токена составляют $2 \times 36 W_{\text{layer}} \approx 13.89$ GFLOPs. Для QK и PV одного шага decode с контекстом 8K дополнительно требуется $4 \times 36 \times 8192 \times 4096 \approx 4.832$ GFLOPs, для выходной головы — около 1,245 GFLOPs, итого — около 19,97 GFLOPs.

Теперь рассмотрим prefill: при обработке 4 096 новых входных токенов объём линейных операций возрастает пропорционально числу токенов. Число пар видимых позиций в причинном внимании равно $4096 \times 4097/2$; для каждой пары в 32 головах выполняются одно скалярное произведение QK и одно накопление PV, что в сумме даёт 4×4096 операций. После умножения на 36 слоёв эффективный объём операций QK и PV составляет $2 \times 36 \times 4096 \times 4096 \times 4097$. Выходная голова вычисляется для последней позиции, а общий объём матричных операций всего prefill составляет около 61,85 TFLOPs. Подставив эти два этапа по отдельности в таблицу параметров оборудования, получим рис. 4-39.

Эти результаты позволяют сделать несколько непосредственных выводов. При замене 3090 на 4090 нижняя граница времени чтения для decode одного запроса сокращается лишь примерно на 7,1%, тогда как время матричных операций prefill для 4K умень-

⁸⁰Результаты сравнения поколений памяти и описание расчётов.

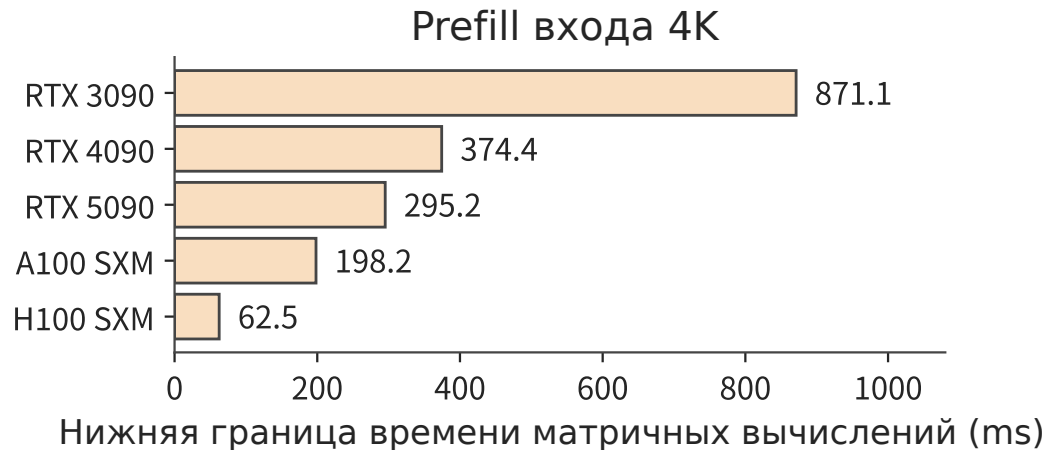
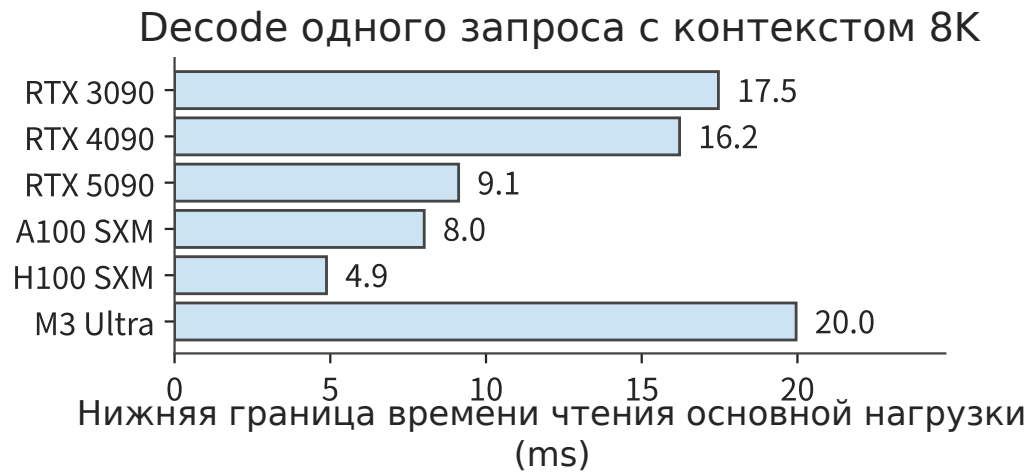


Рис. 4-39. Бюджеты двух этапов для одной и той же Qwen3-8B. Decode одного запроса более непосредственно отражает пропускную способность чтения, а матричный бюджет prefill для 4K — скорость матричных операций при соответствующей точности. На горизонтальных осях двух графиков указано рассчитанное время.

шается примерно на 57,0%. При замене A100 на H100 время чтения сокращается примерно на 39,1%, а время матричных операций — примерно на 68,5%; при последующей замене на H200 время чтения продолжает снижаться, а время матричных операций практически не меняется. У RTX 5090 и RTX PRO 6000 одинаковое время чтения, но у последней выше пропускная способность матричных операций и больше объём памяти. В задачах ответов на вопросы по документам с длинными входными данными прирост матричной производительности 4090 и H100 прежде всего сокращает prefill; при непрерывной генерации одного запроса прирост пропускной способности памяти 5090 и H200 непосредственно сокращает чтение на каждом шаге.

Как объём памяти влияет на выбор устройства? Бюджет BF16 Qwen3-8B для одного запроса составляет около 19,737 GB, поэтому модель помещается на указанных в таблице картах с 24 GB памяти; при увеличении до восьми запросов с контекстом 8K общий бюджет составит около 28,193 GB, что превышает 24 GB у 3090/4090, но укладывается в 32 GB у 5090.

Как выбрать между Qwen3-8B и Qwen3-235B-A22B при локальном запуске? Сначала сравним генерацию одного запроса. Qwen3-8B с BF16 и контекстом 8K помещается в памяти RTX 4090, 5090 и M3 Ultra. Если пересчитать приведённое в таблице время чтения на каждом шаге в число токенов в секунду, получим верхние границы пропускной способности чтения: соответственно $1000/16.22 \approx 61.7$, $1000/9.12 \approx 109.6$ и $1000/19.96 \approx 50.1$ токен/с. Эта плотная модель 8B не использует дополнительный объём памяти Ultra, а высокая пропускная способность памяти 5090 непосредственно выражается в более высоком бюджете генерации.

Теперь возьмём Qwen3-235B-A22B из раздела 4.6.3 с 4-битными весами, одним запросом и контекстом 8K. Для модели, KV-кэша и рабочей области требуется около 126,867 GB, поэтому на одной карте 3090, 4090 или 5090 она не помещается, но может быть размещена на M3 Ultra 256 GB и H200 141 GB. На каждом шаге считывается около 13,697 GB; на M3 Ultra это занимает около 16,72 ms, а на H200 — около 2,85 ms, что соответствует верхним границам пропускной способности чтения около 59,8 и 350,4 токен/с. Большой объём памяти Ultra позволяет запускать эту MoE-модель 235B на одном локальном компьютере, а высокая пропускная способность памяти H200 сокращает время того же чтения примерно до одной шестой.

Тензорное разбиение и диспетчеризация экспертов при работе на нескольких картах рассматриваются в главе 6.

4.8.3 Почему прогнозы расходятся с измерениями

Сравнение устройств в разделе 4.8.2 дало оценки ёмкости и бюджетов основных ресурсов. Библиотека времени выполнения выбирает конкретный kernel для математических операций, кэширование изменяет объём трафика за пределами кристалла, а отправка команд и синхронизация также занимают время. Измерения нужны, чтобы выявить эти этапы выполнения и дополнить временную модель из раздела 4.8.1.

Сначала преобразуем бюджет чтения Qwen3-8B в конкретную задачу генерации и выполним её на используемой в этой книге тестовой платформе RTX PRO 6000 Blackwell Workstation. По-прежнему используем BF16, один запрос и контекст 8K. Нагрузка на каждом шаге составляет 16,345 GB, поэтому чтение при пиковой пропускной способности 1 792 GB/s должно занимать 9,12 ms, что соответствует примерно 109,6 token/s. Для 128 шагов decode при таком контексте суммарное время составит около 1,17 s.

В эксперименте 8-1 из главы 8 это условие выполнялось на той же карте с помощью vLLM: контекст 8K, batch 1. Медиана времени чистой итерации decode по трём сериям измерений составила 25,83 ms. Результаты трёх серий — 26,43, 16,18 и 25,83 ms. Одна серия была заметно быстрее, а две другие различались менее чем на 3%; медиана позволяет исключить влияние отклонения отдельной серии на последующее сравнение.⁸¹ Скорость генерации составляет примерно 38,7 token/s, поэтому 128 шагов занимают около 3,31 s. Нижняя граница чтения составляет лишь 35,3% измеренного времени, а измеренное значение превышает её примерно на 16,7 ms.

Дополнительное время связано не со скоростью чтения: при контексте 2K в том же эксперименте на каждом шаге считывается на 0,906 GB KV меньше, что при пиковой пропускной способности должно экономить около 0,51 ms, однако измеренное время одной серии составило 26,26 ms — не меньше 25,83 ms при 8K. При увеличении batch до 64 объём чтения на каждом шаге в условии 2K возрастает до 34,46 GB, нижняя граница чтения составляет 19,23 ms, а измеренное время одной серии увеличивается лишь до 29,63 ms. Эквивалентная пропускная способность равна примерно 1,16 TB/s, или 65% пиковой. Следовательно, при batch 1 время каждого шага в основном определяется фиксированной работой, не зависящей от объёма чтения: эксперимент выполняется в режиме eager, kernel запускаются поочерёдно, а кроме того, выполняются векторные операции, сэмплирование и синхронизация. На этой программно-аппаратной платформе повышение утилизации пропускной способности почти не ускоряет генерацию одного запроса. Чтобы сократить время шага, сначала нужно уменьшить объём этой фиксированной работы; именно на это направлены рассматриваемые в главе 5 слияние kernel и сокращение числа запусков. При увеличении batch фиксированная работа распределяется между большим количеством токенов, и чтение снова становится основной составляющей.

Согласно определению из раздела 1.2.2, значения 35,3% и 65% являются MBU для этих двух условий. Здесь видны оба типа расхождений, описанные в разделе 1.3.4. Приведённые ниже показания счётчиков покажут, что логические запросы чтения в модели не равны трафику DRAM: данные, найденные в кэше, не используют пропускную способность видеопамати, поэтому здесь нужно скорректировать область учёта модели. Промежутки, возникающие из-за поочерёдного запуска kernel, отправки команд хостом и синхронизации, напротив, являются устранимыми накладными расходами реализации. Сначала нужно скорректировать область учёта, а затем устранить накладные расходы — необходимы оба этапа.

Чтобы разложить дополнительное время по конкретным этапам, нужны записи времени, трафика и kernel. Ниже на уже выполненных для книги измерениях проекции Q показано, как по этим данным определить источник расхождения во времени.

В сопутствующем эксперименте на M2 Max и RTX PRO 6000 Blackwell Workstation выполняется одна и та же проекция с $K = N = 4096$; входные данные, веса и выходные данные представлены в BF16. Для сравнения различных способов доступа к весам на каждой платформе подготовлено 16 копий весов размером 32 MiB с одинаковым содержанием, но разными адресами. В группе повторного использования всегда применяется один и тот же адрес, а в группе чередования последовательно используются разные адреса. В каждом раунде выполняется 16 вызовов, измеряется полное время раунда, включая

⁸¹ Сканирование batch в эксперименте 8-1 и его построчная сводка эффективности: RTX PRO 6000 Blackwell Workstation, Qwen3-8B BF16, vLLM 0.23.0, режим eager; для каждой строки берётся медиана итераций чистого decode в каждом прогоне, а затем медиана трёх прогонов. Отношение к нижней границе чтения вычислено элементом `measured_decode` учебного скрипта вывода.

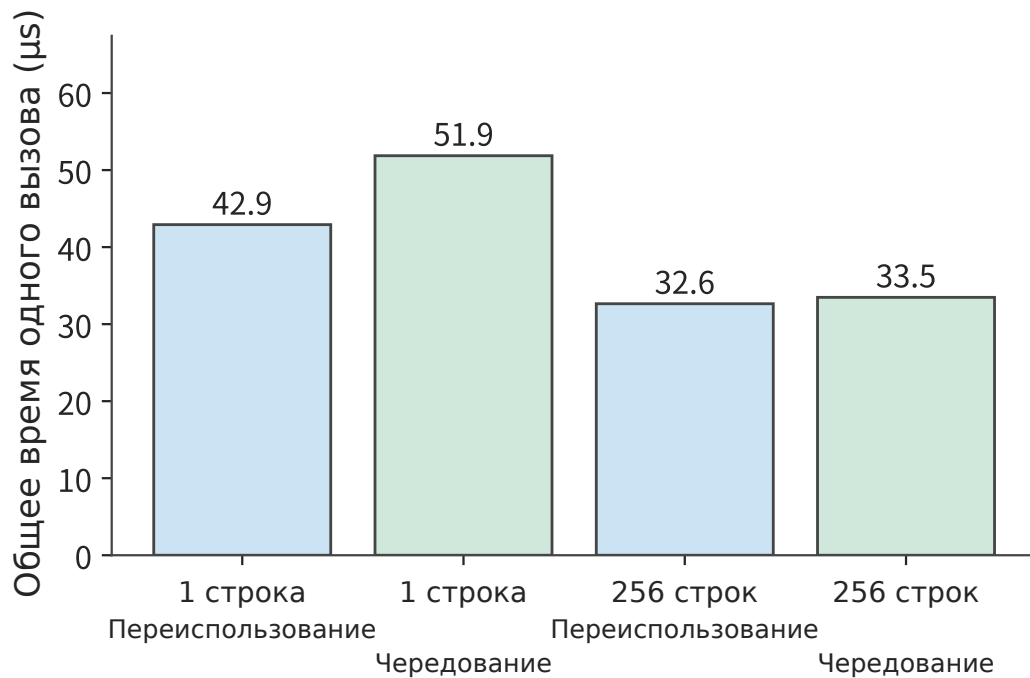


Рисунок 4-40. Полное время проекции на RTX PRO 6000. Для каждого условия выполнено одиннадцать раундов по шестнадцать вызовов; приведена медиана рассчитанного для каждого раунда среднего времени одного вызова. Измерение времени включает отправку команд и синхронизацию. «Повторное использование» означает, что несколько вызовов считывают веса по одному адресу; «чередование» означает смену адреса весов между вызовами.

отправку команд хостом и синхронизацию, после чего результат делится на 16 для получения среднего времени одного вызова в этом раунде. Всего выполняется 11 раундов, а итоговым значением служит медиана этих 11 средних.⁸²

Число обрабатываемых токенов M	M2 Max: повторное использование / чередование	RTX PRO 6000: повторное использование / чередование
$M = 1$	около 166 / 183 µs	около 42,9 / 51,9 µs
$M = 256$	в обоих случаях около 1,835 ms	около 32,6 / 33,5 µs

Сначала рассмотрим одну строку на RTX: чередование примерно на 21% медленнее повторного использования. Естественное объяснение состоит в том, что при многократном использовании одной и той же копии весов они остаются в кэше, что сокращает объём чтения из DRAM. Это объяснение можно проверить напрямую: если различие обусловлено сокращением чтения весов, то при их повторном использовании число байтов, считываемых за пределами кристалла, должно быть меньше. В эксперименте также использовался инструмент NVIDIA Nsight Compute для анализа производительности GPU kernel: сначала веса считывались заданным способом, затем записывался один исследуемый вызов и суммировались счётчики доступа всех kernel.

При обработке одной строки объём чтения DRAM составляет около 32 MiB при обоих способах доступа к весам, что точно соответствует размеру всей копии весов. В обоих

⁸² Полное описание и исходные записи эксперимента 4-6, сводка измерения времени проекции, фактические счётчики DRAM/L2.

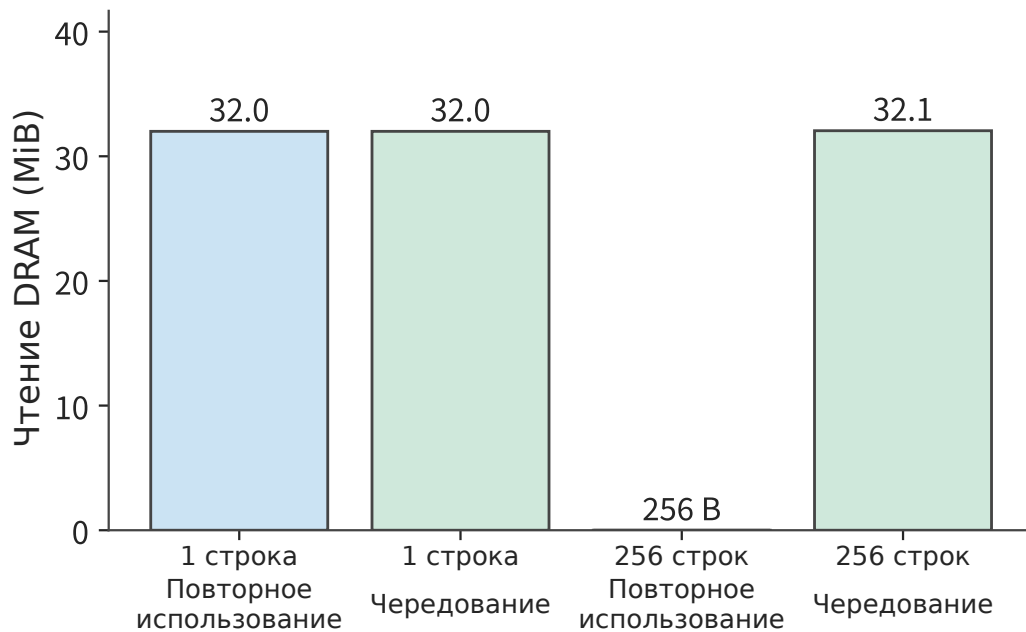


Рисунок 4-41. Отдельно собранные счётчики чтения DRAM для тех же четырёх условий. Для одной строки во всех случаях считывается около 32 MiB; для 256 строк при повторном использовании — 256 bytes, при чередовании — около 32,1 MiB. Счётчики доступа и обычные измерения времени собирались отдельно. «Повторное использование» и «чередование» означают соответственно сохранение и смену адреса весов.

условиях за пределами кристалла считывается одинаковый объём весов, поэтому причину различия во времени для одной строки нужно искать в этапах отправки команд, выполнения и ожидания. Для 256 строк ситуация иная: при повторном использовании одной копии весов считывается лишь 256 bytes, а при чередовании разных копий — около 32,1 MiB, однако обычные измерения времени в обоих случаях дают около 33 μ s. Кэширование изменило объём трафика за пределами кристалла, но полное время вызова включает также другие вычисления, передачи и ожидание.

Иерархия памяти, описанная в разделе 4.3, здесь становится наблюдаемой. При обоих способах доступа к весам для 256 строк объём запросов L2 составляет около 147 MiB, что намного больше почти нулевого объёма чтения DRAM в одном из случаев. Попадание в кэш означает лишь, что данные предоставляются на кристалле; запросы чтения вычислительных блоков и локальная передача данных никуда не исчезают. Одна и та же копия весов из исходной программы после разбиения выполнения на блоки многократно запрашивается несколькими вычислительными блоками.

Запросы на кристалле объясняют, почему после сокращения чтения DRAM доступ к данным сохраняется. Теперь рассмотрим вычисления. Библиотека разбивает одну проекцию на несколько этапов: вызов для одной строки использует один GEMV kernel, а для 256 строк — два kernel, GEMM и редукцию split-K. Split-K распределяет вычисления по измерению скалярного произведения между несколькими вычислительными блоками: сначала формируются частичные результаты, а затем они объединяются редукцией. Это позволяет большему числу вычислительных блоков работать параллельно, но требует сохранять и считывать частичные результаты, а в конце выполнять ещё одну редукцию. Таким образом, одна математическая операция умножения из раздела 4.1 здесь разво-

рачивается в несколько этапов.

Теперь оценим время выполнения на RTX PRO 6000 в предположении, что входные данные и веса считываются за пределами кристалла. Пиковая производительность плотных матричных операций с входными данными BF16 и накоплением FP32 составляет около 504 TFLOP/s, а пропускная способность — 1,792 TB/s. Нижние границы для двух значений числа строк составляют примерно 18,7 и 21,1 μ s. Модель даёт время, необходимое для матричных вычислений и чтения за пределами кристалла; счётчики показывают трафик на интерфейсах разных уровней при участии кэша; путь kernel объясняет дополнительную работу по объединению. Полное время также включает отправку команд, синхронизацию и промежутки между этапами выполнения.⁸³

Для более точной локализации расхождения можно разделить вопросы «как долго блок был занят» и «насколько быстро он работал во время активности». Пусть аппаратному блоку для выполнения всех операций с идеальной скоростью требуется время S_r , фактически в рабочем состоянии он находится в течение A_r , а полное время задачи равно T . Тогда

$$\frac{S_r}{T} = \frac{S_r}{A_r} \times \frac{A_r}{T}.$$

Первый множитель показывает отношение фактической скорости во время работы к идеальной, второй — долю рабочего времени в полном времени задачи. Пусть две задачи выполняются по 100 μ s, а идеальное время вычислений для каждой составляет 40 μ s. При выполнении первой задачи блок работает лишь 40 μ s и в это время достигает идеальной скорости, но остальные 60 μ s ожидает. При выполнении второй задачи блок работает 80 μ s со скоростью, составляющей лишь половину идеальной, а время ожидания равно 20 μ s. В обоих случаях итоговое отношение составляет 40%, но направления оптимизации различаются.

Для первой задачи важнее заранее подготовить данные, запланировать больше независимых друг от друга вычислений или сократить время ожидания результатов предыдущего этапа — аналогично переходу от двух слотов к трём в разделе 4.4. Для второй задачи важнее улучшить гранулярность выполнения, инструкции и локальный доступ — аналогично анализу заполнения неиспользуемых ресурсов и сгруппированных вызовов в разделе 4.2.⁸⁴

Эксперимент 4-6 · Продолжение: проверка прогнозов времени выполнения с помощью измерений на Mac и RTX

На основе измеренного полного времени проекции на двух платформах предложите две причины, способные объяснить различие во времени, а затем изучите записи DRAM, L2 и kernel и для каждого объяснения перечислите ожидаемые и фактические наблюдения. После этого изобразите путь матрицы, частичных результатов и редукции для вызова с 256 строками и укажите, время каких этапов нужно знать, чтобы различить эффективность выполнения во время работы вычислительных блоков и время ожидания. Сопоставьте полученный результат с моделью, предполагающей чтение входных данных и

⁸³Стоимость парных проекций и условия для средней мощности; значение 600 W для RTX PRO 6000 взято из таблицы исходных данных об аппаратном обеспечении, а энергопотребление каждого вызова и мощность Mac при равном энергопотреблении рассчитаны элементом `energy` учебного скрипта вывода; см. данные вывода.

⁸⁴От утилизации операторов к ожиданию во время выполнения и заметки по чтению статьи.

весов за пределами кристалла.

4.8.4 Выбор между ёмкостью, скоростью, энергопотреблением и стоимостью

В разделе 4.8.3 было получено время выполнения задачи, а в этом разделе оно пересчитывается в стоимость и энергопотребление. Сначала сравним варианты с оплатой по времени использования, а затем учтём время простоя и фиксированные вложения в специализированную архитектуру.

Пусть почасовая стоимость двух систем равна c_A, c_B , а время выполнения одной и той же задачи — t_A, t_B . При оплате по времени выполнения стоимость одной задачи пропорциональна ct , поэтому

$$K_A < K_B \iff \frac{c_A}{c_B} < \frac{t_B}{t_A}.$$

Правая часть показывает, во сколько раз может различаться почасовая стоимость: если А вдвое быстрее В, то при почасовой стоимости А меньше удвоенной почасовой стоимости В каждая задача всё равно будет обходиться дешевле. Если заменить почасовую стоимость средней мощностью, то то же неравенство будет сравнивать энергопотребление на задачу. В группе повторного использования из раздела 4.8.3 отношения времени Мас/RTX составляют примерно 3,9 для одной строки и 56 для 256 строк. Для RTX учитывается только видеокарта: при предельной мощности RTX PRO 6000 Workstation в 600 W один вызов для одной строки потребляет не более примерно 25,8 mJ, а вызов для 256 строк — примерно 19,6 mJ. В официальных материалах по M2 Max мощность всей системы не указана; из приведённого выше неравенства следует, что при средней мощности Мас во время однострочного вызова ниже примерно 155 W его энергопотребление на вызов будет меньше, чем у RTX, а для 256 строк мощность должна быть ниже примерно 10,7 W. При неизменной машине и изменении только размеров матрицы допустимая мощность, при которой Мас сохраняет преимущество по энергопотреблению, снижается примерно со 155 W до 10,7 W.⁸⁵

Для непрерывно работающего сервиса необходимо также учитывать стоимость времени простоя. Пусть при обработке запросов машина генерирует q полезных токенов в секунду, а доля часа, в течение которой она обрабатывает запросы, равна u . Тогда часовой объём вывода составляет $3600uq$, а стоимость одного токена — $c/(3600uq)$. Если скорость генерации удваивается, а утилизация уменьшается вдвое, часовой объём вывода и удельная стоимость не изменяются.

Энергопотребление также совместно определяется мощностью и временем. Рассмотрим выполнение одного шага decode для одного запроса с контекстом 8K из раздела 4.8.2 и предположим, что обе карты работают на предельной мощности, а время шага равно нижней границе времени чтения: для RTX 4090 это 450 W и 16,22 ms, то есть примерно 7,30 J на шаг; для RTX 5090 — 575 W и 9,12 ms, то есть примерно 5,24 J на шаг.⁸⁶ Мощ-

⁸⁵Стоимость парных проекций и условия для средней мощности; значение 600 W для RTX PRO 6000 взято из таблицы исходных данных об аппаратном обеспечении, а энергопотребление каждого вызова и мощность Мас при равном энергопотреблении рассчитаны элементом `energy` учебного скрипта вывода; см. данные вывода.

⁸⁶Техническое описание архитектуры RTX Blackwell, приложение А, таблица 3: RTX 4090 — 24 GB GDDR6X, 1 008 GB/s, PCIe Gen 4, TGP 450 W; RTX 5090 — 32 GB GDDR7, 1 792 GB/s, PCIe Gen 5, TGP 575 W; спецификация A100 80GB: PCIe 4.0 — 64 GB/s в обоих направлениях суммарно. Время H2D и чтения видеопамати см. в элементе `host_link` учебного скрипта вывода.

ность увеличивается примерно на 28%, но энергопотребление одного шага, наоборот, уменьшается примерно на 28%. В общем виде

$$E = \int_0^T P(t) dt.$$

Для непрерывно работающего сервиса можно измерить общее потребление электроэнергии за некоторый период и разделить его на полезный объём вывода за тот же период, чтобы учесть также энергопотребление во время простоя и прогрева. Сокращение времени выполнения снижает энергопотребление непосредственно при исполнении задачи, а повышение утилизации уменьшает долю энергопотребления простоя, приходящуюся на каждую задачу. Разбивка энергопотребления из раздела 4.1.3 даёт оценку порядка потребления электроэнергии за один шаг decode: для одного запроса и контекста 8K оно составляет 0,534 J на токен, из которых 0,481 J расходуется на чтение весов; при batch 32 энергопотребление снижается примерно до 0,068 J на токен. Эта разбивка не учитывает управление, тактирование, статические утечки и мощность во время простоя, поэтому энергопотребление на токен, полученное делением общего потребления электроэнергии на полезный объём вывода, будет выше этой оценки.⁸⁷

Специализация также требует фиксированных вложений. Пусть дополнительные фиксированные затраты относительно универсального решения равны F_0 , а экономия переменных затрат на единицу полезного вывода — Δc . Тогда объём вывода, необходимый для окупаемости, равен

$$V^* = \frac{F_0}{\Delta c}.$$

Модель может быть обновлена или заменена, поэтому срок её использования в специализированной системе ограничен. Пусть имеется D машин, скорость генерации каждой при обработке запросов равна q , утилизация — u , а срок использования модели — L . Тогда общий объём вывода составляет $DuqL$. Специализированное решение окупится только в том случае, если сэкономленные за этот период эксплуатационные расходы достигнут величины фиксированных вложений.

Пример 4-3. Успеет ли специализация окупиться за срок службы модели?

Дополнительные фиксированные затраты составляют 20 млн долларов, а экономия на каждый миллион токенов пригодного вывода — 0,50 доллара. Развёрнуто 100 машин, каждая из которых во время обработки запросов выдаёт 10 000 токенов в секунду; утилизация составляет 50%, экономический срок службы модели — один год, в году принимается 365 дней.

Сравнение объёма вывода, необходимого для окупаемости, с объёмом, достижимым за год. Для окупаемости требуется $20\,000\,000 / (0.50/10^6) = 4 \times 10^{13}$ токенов, то есть 40 трлн. Годовой объём вывода равен $100 \times 10\,000 \times 0.5 \times 31\,536\,000 \approx 1.58 \times 10^{13}$ токенов, то есть примерно 15,8 трлн.

Каков фактический масштаб сервиса, необходимый для окупаемости за один год? Годовой объём вывода составляет примерно 39% от необходимо-

⁸⁷Результаты отдельного учёта энергопотребления: количество байтов и FLOPs считывается из результатов decode одного запроса с контекстом 8K, после чего складывается энергопотребление на каждый байт и каждый FLOP согласно приведённой выше таблице. Расчёт можно повторить командой `python3 calculations/calc.py energy-ledger --format md`.

го для окупаемости. При неизменных скорости одной машины и утилизации потребуется не менее 254 машин, работающих в течение года.

Масштаб, необходимый для окупаемости при сокращении срока службы модели вдвое или снижении утилизации. Если модель заменяется через полгода, доступное время каждой машины уменьшается вдвое, поэтому для сохранения заданной утилизации потребуется не менее 508 машин; снижение утилизации с 50% до 25% даёт тот же результат.

В этом примере годовая экономия составляет примерно 7,88 млн долларов, что меньше фиксированных вложений в 20 млн долларов; для достижения объёма окупаемости в 40 трлн токенов потребуется примерно 2,54 года, что превышает заданный годичный срок службы модели.⁸⁸

Эксперимент 4-7 · Расширение: сохраняет ли изменение ресурсов чипа пользу после изменения нагрузки

Выберите одну архитектуру и одну типичную нагрузку, затем предложите одно изменение ресурсов. Сначала выведите изменение времени на основе объёма работы, трафика и зависимостей, а затем рассчитайте увеличение ёмкости и сокращение ожидания. После этого измените batch или длину контекста и найдите критическое значение, при котором исходное изменение перестает экономить время. Если решение требует дополнительных фиксированных вложений, определите минимальный объём обслуживания, необходимый для их окупаемости, исходя из экономии на каждой задаче, утилизации ускорителя и срока службы модели.

Эксперимент 4-8 · Расширение: от разбивки энергопотребления к ограничению мощности

Используйте таблицу энергопотребления из раздела 4.1.3 и число байтов, считываемых за один шаг decode Qwen3-8B для одного запроса с контекстом 8K. (а) Последовательно задайте batch равным 8, 32 и 128, рассчитайте для каждого случая энергопотребление весов, KV и вычислений на токен и найдите минимальный batch, при котором составляющая весов становится меньше вычислительной составляющей. (б) Измените контекст на 32K, найдите batch, при котором составляющая KV равна составляющей весов, и объясните, почему дальнейшее увеличение batch после этого уже не приводит к заметному снижению энергопотребления на токен. (в) Выберите TDP, пиковую вычислительную производительность и пропускную способность HBM одной карты и найдите энергетический бюджет на FLOP при пиковой скорости b ; примите фактическое энергопотребление одного FLOP равным $1.5b$ и при $P \propto fV^2$ найдите отношение устойчивой частоты к пиковой для двух случаев: при неизменном напряжении и при снижении напряжения вместе с частотой. Затем разделите энергию одного шага для batch 32 из пункта (а), то есть энергопотребление на токен, умноженное на 32, на время этого шага при устойчивой частоте, чтобы получить среднюю мощность, и сравните её с TDP.

⁸⁸Пример OpenTallas.

4.9 Итоги главы

Анализ ускорителей начинается с оценки рабочей нагрузки модели: объём состояния определяет требования к ёмкости, объём передаваемых данных и пропускная способность — время чтения, а форма матриц, точность и скорость выполнения инструкций — время вычислений; затем выполнение планируется с учётом зависимостей. Эволюция архитектур трёх производителей показывает, что требования моделей меняют соотношение ресурсов. Подсчёт обращений, журналирование kernel и замеры времени преобразуют эти оценки в скорость выполнения и стоимость задачи. Время и энергопотребление — две отдельные статьи: пропускная способность определяет скорость одного шага, энергозатраты на байт и количество повторных использований — сколько джоулей требует один шаг, а ограничение мощности, в свою очередь, снижает устойчиво поддерживаемую частоту. В главе 5 продолжится рассмотрение компоновки, слияния и планирования, чтобы показать, как программное обеспечение эффективно использует эти аппаратные ресурсы.

Операторы и среда выполнения

Одни и те же входные данные, только что использованные для получения одного результата, затем требуются для соседнего результата: если они по-прежнему находятся рядом с вычислительным блоком, их можно сразу использовать снова; если их уже вытеснили, придётся загрузить их заново. Программа матричного умножения должна упорядочить эти обращения и выделить подходящее пространство для входных данных и частичных сумм. Одно лишь изменение количества результатов, обрабатываемых за раз, или порядка вычисления блоков влияет и на число повторных переносов данных, и на количество задач, которые можно выполнять одновременно.

В рассматриваемом в этой главе примере матричного умножения требуется около 103 GFLOPs вычислений, а входные данные, веса и выходные данные вместе занимают 128 MiB, однако объём чтения и записи одной из блочных реализаций достигает примерно 3 GiB. Увеличение блока выходных данных позволяет многократно использовать входные данные и вдвое сократить объём чтения и записи, но потребность каждого блока в локальной памяти при этом возрастает с 24 KiB до 80 KiB. Зная эти показатели, можно определить, компенсирует ли время, сэкономленное благодаря сокращению переносов данных, потери из-за уменьшения числа одновременно выполняемых блоков.

В главе 4 были представлены вычислительная мощность ускорителя, ёмкость памяти и пропускная способность передачи данных. В этой главе рассматривается, как программа использует эти ресурсы. На примере фрагмента FFN сначала объясняется, как хост иницирует вычисления и как ускоритель выполняет задачу, затем анализируются циклы и блочное разбиение матричного умножения, а также совместное использование промежуточных результатов соседними операторами. Далее рассматривается, как компилятор и среда выполнения реализуют эти оптимизации, а в завершение рассчитывается, сколько времени они позволяют сэкономить при обработке полного запроса.

Основным примером в этой главе служит FFN модели Qwen3-8B: ширина скрытого состояния $H = 4096$, промежуточная ширина FFN $F = 12288$. Сначала одновременно обрабатываются $M = 1024$ токена: 4096-мерные векторы признаков каждого токена образуют входную матрицу X , после чего вычисляется одна ветвь проекции $C = XW$; X , W и C представлены в формате BF16, каждый элемент занимает 2 байта, а при матричном умножении накопление выполняется в FP32. M обозначает число токенов, обрабатываемых за один вызов; каждому токеноу соответствует одна строка входной матрицы. Эти токены могут относиться к одному фрагменту входных данных на этапе prefill либо к нескольким запросам в одном пакете; при decode одного запроса по одному токеноу значение M равно 1.¹

¹Фиксированная конфигурация Qwen3-8B; операторы модели и их реализации.

Объект	Форма	Размер	Роль в вычислении
Входная матрица X	[1024, 4096]	8 MiB	Вектор признаков каждого токена занимает одну строку; после проекции получается новый вектор признаков этого токена
Веса W	[4096, 12288]	96 MiB	Совместно используются при вычислении проекций 1024 токенов
Проекция C	[1024, 12288]	24 MiB	В дальнейшем подвергается активации и другой обработке

Эта глава посвящена трём вопросам: какая работа выполняется повторно, какие данные можно использовать повторно и какие вычисления или ожидания определяют общее время. При ответе на эти вопросы единицей выполнения программы на ускорителе служит `kernel`. Анализ объёма чтения и записи, ёмкости буферов и числа `kernel` позволяет понять, какой объём работы выполнила программа; однако рассчитать продолжительность её выполнения можно лишь с учётом порядка выполнения этой работы.

5.1 Отправка, перемещение и синхронизация при выполнении на ускорителе

Сначала рассмотрим умножение матриц, входные и выходные данные которого находятся в памяти CPU. CPU подготавливает входные данные, передаёт их в GPU, запускает вычисление, а затем получает результат. Этот процесс требует совместной работы двух процессоров. Понимание того, когда они работают независимо, а когда ожидают друг друга, служит основой для последующего рассмотрения слияния, конвейеризации и повторного воспроизведения графов.

5.1.1 От вызова фреймворка до `kernel launch`

PyTorch — программный фреймворк для выражения тензорных операций и организации выполнения модели. При вызове умножения матриц CPU сначала исполняет код фреймворка: считывает форму и тип данных тензоров, выбирает реализацию, подготавливает адреса и параметры, а затем через среду выполнения и драйвер отправляет задачу в GPU. Инициация одного выполнения `kernel` на ускорителе называется **kernel launch**. Среда выполнения отвечает за отправку задач, управление памятью и синхронизацию выполнения; компилятор заранее или при первом вызове генерирует код для ускорителя.

Здесь есть три уровня: оператор описывает математическую работу, например умножение матриц или активацию; `kernel` выполняет эту работу на ускорителе; `launch` — действие CPU, иницирующее это выполнение. Одно умножение матриц может выполняться несколькими `kernel`, а несколько поэлементных операторов — одним `kernel`. Поэтому можно как оптимизировать вычисления на ускорителе, так и сокращать накладные расходы на подготовку и отправку задач хостом.

При выполнении kernel CUDA запускает множество потоков. Несколько потоков образуют блок потоков (thread block), а все блоки потоков — сетку потоков (grid). При умножении матриц выходная матрица обычно делится на несколько tile, вычисляемых разными блоками потоков; потоки внутри блока совместно считывают входные данные и накапливают результат. Ускоритель размещает блоки потоков на SM со свободными ресурсами. Блоки, завершившиеся раньше, освобождают ресурсы, после чего могут запускаться следующие; поэтому блоки потоков в grid могут выполняться партиями.

Kernel launch использует асинхронную отправку. К моменту возврата из вызова CPU уже завершил отправку, а GPU выполняет работу в соответствии с очередью задач и зависимостями. CPU может сразу начать подготовку следующей задачи. Ему требуется ждать завершения соответствующей операции ускорителя только тогда, когда нужно использовать её результат или перезаписать данные, которые ускоритель ещё использует.²

Такое разделение работы образует простой конвейер: CPU подготавливает и отправляет задачи, а GPU их выполняет. Вычисление больших матриц занимает достаточно много времени, чтобы CPU успел подготовить следующий вызов; небольшие операторы завершаются быстро, поэтому GPU чаще успевает выполнить все задачи в очереди и останавливается в ожидании следующей отправки от CPU. Следовательно, ускорение возможно с двух сторон: за счёт сокращения времени ожидания хоста ускорителем и за счёт уменьшения времени выполнения самого kernel.

5.1.2 Копирование данных между хостом и ускорителем

CPU может заранее отправить следующую задачу, но ускоритель способен начать вычисление только после получения входных данных. Анализ процесса передачи данных позволяет понять, какую работу необходимо завершить между отправкой задачи и началом вычисления. Помимо H2D и D2H, представленных в главе 4, GPU с выделенной видеопамью также часто копирует данные внутри неё. Распространены три направления копирования:

Название	Направление копирования	Типичный объект
H2D	Память хоста → видеопамья	Входной тензор текущего batch
D2H	Видеопамья → память хоста	Результат вычисления, необходимый CPU
D2D	Видеопамья → видеопамья	Фиксированный входной буфер или результат перестановки

Описанное выше умножение матриц выполняется так:

CPU подготавливает входные данные → H2D → GPU вычисляет → D2H → CPU использует результат.

На рис. 5-1 показано расположение этих данных в памяти с обеих сторон и направления копирования.

При выполнении полной модели передавать все данные заново при каждом вызове не требуется. После загрузки веса могут постоянно оставаться в видеопамью, межслой-

²Модель выполнения CUDA; семантика синхронизации CUDA Runtime API; управление stream в CUDA и управление event; руководство PyTorch по закреплённой памяти и асинхронному копированию. О копировании на хосте и повторном использовании буферов см. также локальные расчётные материалы, цитируемые в этой главе.

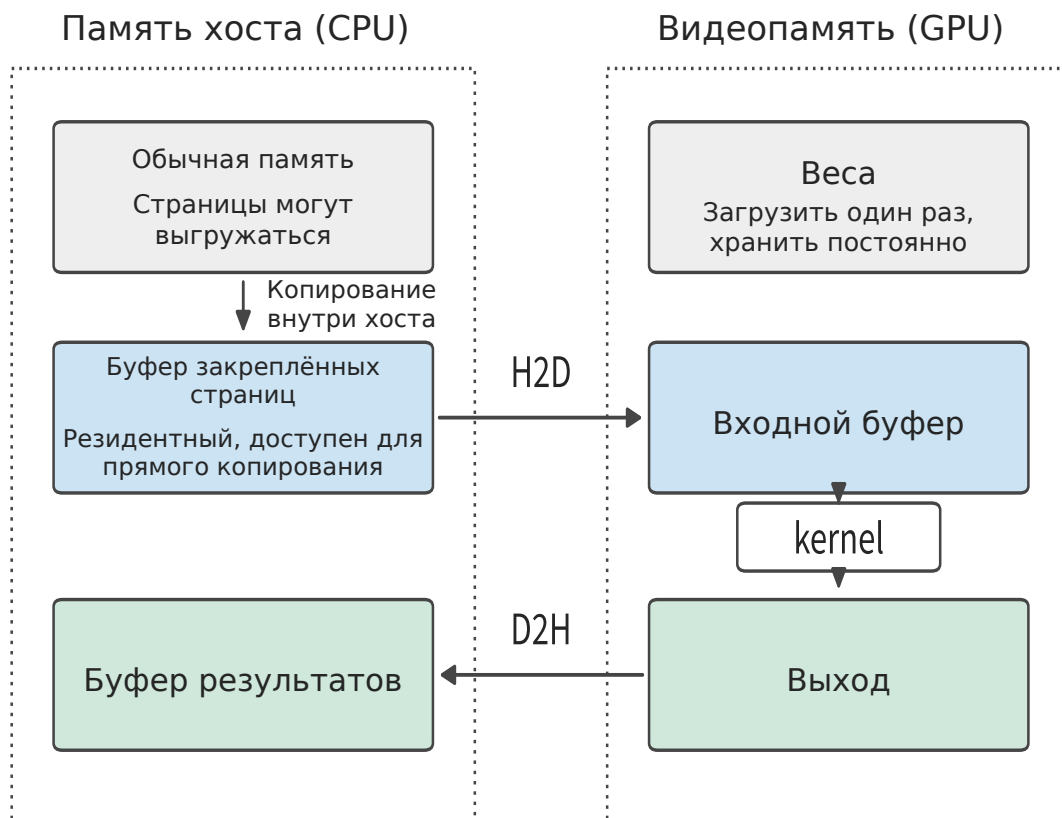


Рис. 5-1. H2D перемещает входные данные из закреплённого буфера хоста в видеопамять, а D2H возвращает результат на хост; после однократной загрузки веса остаются в видеопамяти, и kernel непосредственно считывает из неё входные данные и записывает выходные. Данные из обычной памяти необходимо сначала скопировать в закреплённый буфер, чтобы механизм копирования мог переместить их напрямую.

ные активации может непосредственно считывать следующий kernel, а KV можно продолжать использовать на последующих шагах decode. Если сэмплирование выполняется на GPU, CPU достаточно получить небольшое количество token ID. Поэтому число копирований между хостом и ускорителем не совпадает с числом чтений внутри ускорителя: веса можно передать один раз, но прочитать на ускорителе многократно.

Пример 5-1. Сколько времени занимает передача входных данных с хоста на GPU? Тензор BF16 формы [8192, 4096] занимает $8192 \times 4096 \times 2 = 64$ MiB. RTX PRO 6000 подключена к хосту через PCIe Gen5 x16 с номинальной пропускной способностью 64 GB/s в каждом направлении,³ поэтому время передачи равно:

$$T_{\text{H2D}} = \frac{64 \text{ MiB}}{64 \text{ GB/s}} \approx 1.05 \text{ ms}.$$

Если разделить входные данные на два тензора по 32 MiB, передача каждого займёт около 0,52 ms, а общий объём передаваемых данных останется прежним. Преимущество состоит в том, что первая часть данных поступит раньше: пока GPU вычисляет первую половину batch, механизм копирования продолжает передавать вторую. В разделе 5.3 будет дополнительно рассчитано общее время при перекрытии передачи и вычисления.⁴

На процесс копирования также влияет тип памяти хоста. Страницами обычной памяти управляет операционная система; **закреплённая память (pinned memory)** остаётся резидентной во время использования, благодаря чему ускоритель может перемещать данные напрямую. При передаче данных из обычной памяти среда выполнения часто сначала копирует их во внутренний закреплённый буфер: CPU выполняет одно копирование, после чего данные считывает ускоритель. Это и есть копирование внутри памяти хоста, показанное слева на рис. 5-1. Если для каждого batch копировать 64 MiB данных во вновь созданный закреплённый буфер, перед H2D появляется дополнительное копирование на хосте. Если CPU будет подготавливать входные данные непосредственно в повторно используемом закреплённом буфере, можно устранить это промежуточное копирование и повторные выделения памяти.⁵

5.1.3 Stream, event и порядок завершения

Место, куда переданы данные, определяет, какой процессор сможет их использовать; момент завершения передачи определяет, когда может начаться последующее вычисление. После возврата из асинхронного вызова копирование может ещё продолжаться, поэтому порядок выполнения должен гарантировать, что вычисление не прочтёт входные данные до завершения их передачи. **Stream (поток)** CUDA — это последовательность задач ускорителя, выполняемых по порядку. Если поместить H2D и kernel, считывающий эти входные данные, в один stream, вычисление окажется в очереди после копирования; CPU сможет последовательно отправить обе задачи.

Если копирование и вычисление находятся в разных stream, порядок выполнения между потоками задаётся с помощью **event (события)**. Поток копирования записывает

³В спецификации рабочей станции RTX PRO 6000 Blackwell системным интерфейсом указан PCIe 5.0 x16; согласно спецификации NVIDIA H100, пропускная способность PCIe Gen5 x16 составляет 128 GB/s суммарно в обоих направлениях, по 64 GB/s в каждом направлении.

⁴Порядок потоковой обработки и бюджет буфера; перемещение на хост и время занятости буфера.

⁵Модель выполнения CUDA; семантика синхронизации CUDA Runtime API; управление stream в CUDA и управление event; руководство PyTorch по закреплённой памяти и асинхронному копированию. О копировании на хосте и повторном использовании буферов см. также локальные расчётные материалы, цитируемые в этой главе.

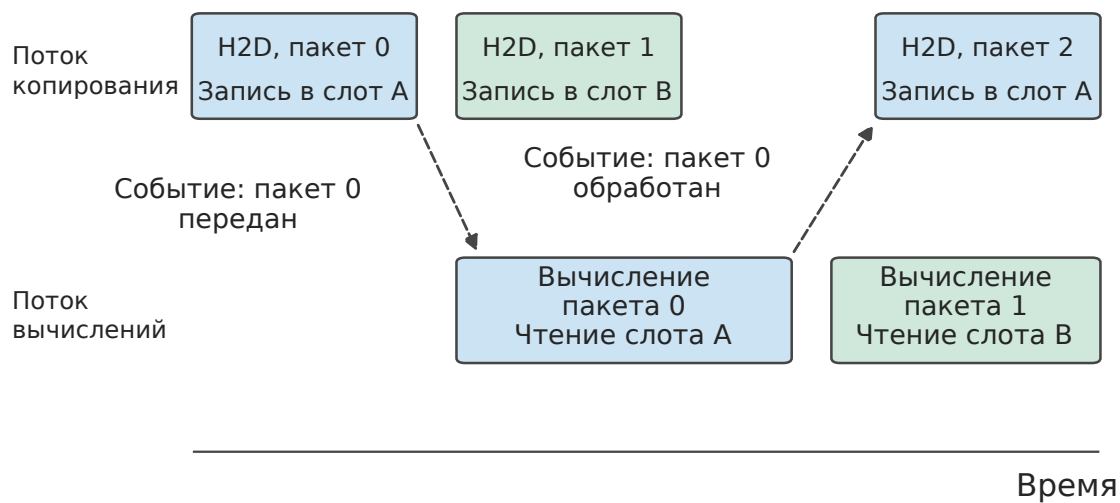


Рис. 5-2. Поток копирования последовательно передаёт входные данные каждого batch, а поток вычисления их считывает. Вычисление batch 0 должно ждать события «передача batch 0 завершена»; чтобы записать batch 2 в слот A, необходимо дождаться события «использование batch 0 завершено». Пунктирные стрелки обозначают ожидание события, а не перемещение данных. По-переменное использование двух слотов подробно рассматривается в разделе 5.3.2.

событие после H2D, а поток вычисления ожидает его и только затем использует входные данные. Ожидание происходит в последовательности задач ускорителя в потоке вычисления, поэтому CPU может продолжать отправлять другую работу.

На рис. 5-2 два потока и два события показаны на общей временной шкале.

События также позволяют определить, когда буфер можно использовать повторно. Предположим, текущие входные данные копируются из буфера хоста в буферный слот A на GPU. После завершения копирования CPU может записать в буфер хоста входные данные следующего batch; слот A можно перезаписать лишь после того, как GPU завершит использование текущих входных данных. Хотя оба буфера содержат данные одного batch, повторно использовать их можно в разные моменты.

Буфер	Последний пользователь	Последующее действие
Исходный буфер хоста для H2D	Считывается механизмом копирования	CPU записывает входные данные следующего batch
Входной буфер GPU	Kernel, считывающий эти входные данные	Механизм копирования записывает входные данные следующего batch
Исходный буфер ускорителя для D2H	Считывается механизмом копирования	GPU записывает новый результат
Целевой буфер хоста для D2H	Записывается механизмом копирования	CPU считывает текущий результат

Следовательно, событие следует записывать после последнего чтения или записи, а следующая операция, использующая этот буфер, должна ожидать данного события. Когда CPU требуется результат D2H, он может ждать соответствующего события, не оста-

навливая другие независимые задачи ускорителя. Двойная буферизация из раздела 5.3 как раз чередует две такие последовательности использования.

Этот порядок также объясняет, почему программа может полностью остановиться. Каждое ожидание относится к конкретному событию, и если оно так и не будет записано, ожидание не завершится; внешне это выглядит как остановка программы, а не как замедление. Последние пользователи четырёх буферов в таблице выше соответствуют четырём событиям, которые легко забыть записать: если пропустить событие «использование завершено» для входного буфера GPU, механизм копирования никогда не сможет записать следующий batch. Аналогично действует циклическое ожидание между потоками: поток А ожидает события, записываемого потоком В, но соответствующая задача потока В стоит в очереди после ещё не завершённой работы потока А, поэтому оба потока не могут продвинуться. При выполнении на нескольких ускорителях появляется ещё одна причина: коллективная коммуникация требует, чтобы каждая участвующая карта инициировала один и тот же вызов. Если хотя бы одна карта пропустит вызов, остальные будут бесконечно ждать участника, который так и не появится (раздел 6.4).

При диагностике сначала проверьте, продолжает ли работать сторона, события которой ожидают: если результаты всё ещё появляются, выполнение лишь замедлилось и следует вернуться к временной шкале из раздела 5.1.4, измерив длительность каждого участка; если движение полностью прекратилось, программа действительно ожидает событие, которое никогда не наступит. Медленный узел из раздела 10.4.5 относится к первому случаю. Тайм-аут ожидания и вывод после его истечения всех незавершённых задач каждого потока позволяют определить, какого именно события не хватает.

5.1.4 От времени отправки до момента готовности результата

Определить момент готовности результата можно только после объединения порядка копирования, вычисления и использования буферов. На рис. 5-3 эта последовательность представлена как временная шкала.

Пример 5-2. Почему время отправки, время работы ускорителя и полное время вызова различаются? Входные данные уже подготовлены на хосте. CPU отправляет все задачи в течение 0–3 μs , первая задача ускорителя начинается на отметке 3 μs ; H2D, kernel и D2H занимают соответственно 8, 20 и 4 μs и последовательно выполняются ускорителем в соответствии с зависимостями.

CPU тратит на отправку задач 3 μs , kernel выполняется 20 μs , а использовать результат CPU может лишь на отметке 35 μs . Если сократить время выполнения kernel с 20 до 5 μs , результат станет доступен через $3 + 8 + 5 + 4 = 20 \mu\text{s}$. Kernel ускорится в четыре раза, но полное время выполнения сократится с 35 до 20 μs , то есть лишь примерно в 1,8 раза; оставшиеся 15 μs приходятся на отправку и копирование.

Три вида времени из примера 5-2 соответствуют трём способам размещения точек измерения. Результат зависит от того, где начинается и заканчивается измерение. Если считать показания часов CPU до и после асинхронного вызова, будет измерено время отправки; если перед завершением измерения дождаться результата, будет измерено время от инициации до завершения. CUDA event отмечает начальную и конечную позиции в последовательности задач ускорителя и измеряет время выполнения этого участка на ускорителе. Инструмент анализа производительности (profiler) отображает вызовы хоста, вычисления ускорителя и копирование на общей временной шкале, позволяя отдельно увидеть постановку в очередь, ожидание зависимостей и выполнение после

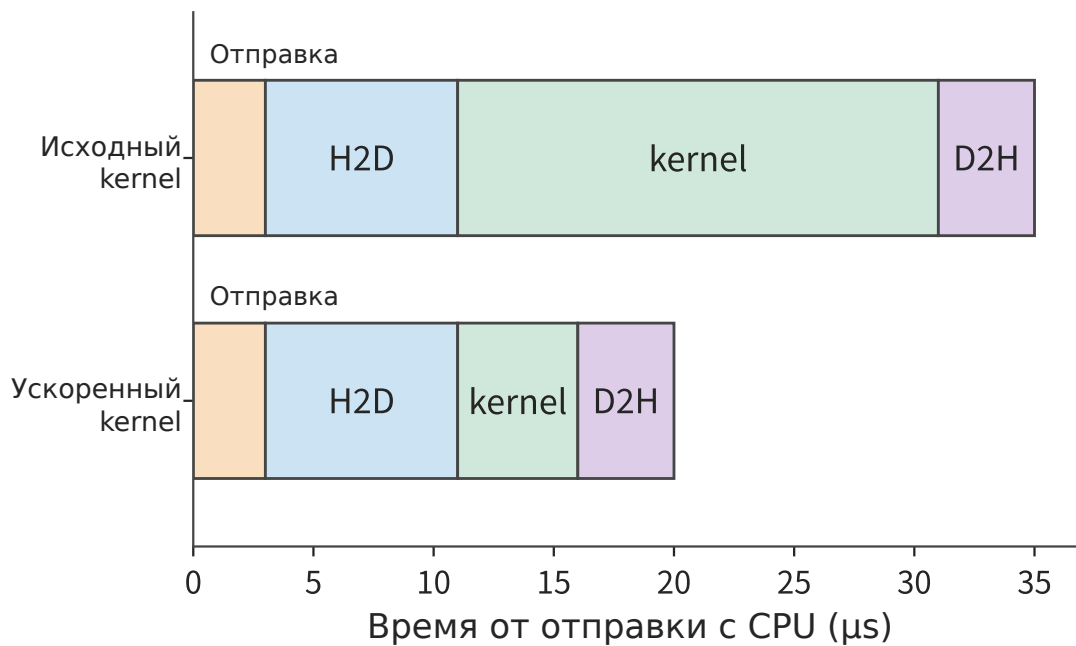


Рис. 5-3. Последовательно выполняются отправка задач CPU, копирование входных данных H2D, выполнение kernel и возврат результата D2H. Вверху kernel выполняется 20 μ s, и результат становится доступен на отметке 35 μ s; внизу kernel выполняется 5 μ s, и результат становится доступен на отметке 20 μ s.

отправки.

Временная шкала на рис. 5-3 начинается с момента готовности входных данных. Если программа ещё не скомпилирована, перед ней нужно добавить однократную подготовительную работу: первый вызов может потребовать компиляции кода или выделения рабочего пространства. Предположим, первая компиляция занимает 100 ms, а каждое последующее выполнение — 1 ms. Тогда один вызов займёт в общей сложности 101 ms, а 100 вызовов — 200 ms, в среднем по 2 ms на вызов. Чем больше вызовов, тем меньше средняя доля времени компиляции, тогда как 1 ms, необходимая для каждого выполнения, остаётся неизменной. В разделе 5.5 тем же способом будет определено, сколько раз должны повторяться бакетизация (дополнение входных данных до одного из небольшого числа репрезентативных размеров) и специализация (генерация отдельной программы для конкретной формы), чтобы окупить затраты.

5.1.5 Как один kernel выполняется на SM

На рис. 5-3 kernel показан как единый отрезок длительностью 20 μ s; возможность сократить его до 5 μ s зависит от того, что происходит на SM в течение этого времени. В этом разделе зафиксируем один kernel и разберём его выполнение на одном SM в пять этапов: потоки, резидентность, обращения к памяти, матричные инструкции и синхронизация. В качестве kernel возьмём блок умножения со сложением из раздела 4.3.2: выходной блок размером 128×128 и шаг 64 по K; блоки A и W в BF16 занимают по 16 KiB, аккумулятор FP32 — 64 KiB. Все три размещаются в общей памяти и образуют рабочий набор (данные, которые во время вычисления должны одновременно находиться в локальном хранилище) общим объёмом 96 KiB. Один блок потоков, включающий 256 потоков, отвечает за

один выходной блок. В качестве аппаратуры возьмём один SM H100: одновременно на нём могут быть резидентны не более 2048 потоков (то есть 64 warp; warp объясняется ниже) и 32 блоков потоков; доступно 64K регистров (по 32 бита каждый) и 228 KB общей памяти (в обозначениях документации CUDA, то есть 233,472 bytes). Kernel использует 128 регистров на поток, каждая инструкция загрузки получает по 16 bytes на поток, каждый warp может иметь не более четырёх незавершённых загрузок, форма матричной инструкции равна $16 \times 8 \times 16$, а задержка видеопамяти принимается равной 600 ns.⁶

Warp: 32 потока, инструкции для которых выдаются совместно. SM выдаёт инструкции не отдельным потокам, а группам по 32 потока; такая группа называется **warp**. Все 32 потока одного warp выполняют одну инструкцию, обрабатывая разные данные; поэтому одна инструкция загрузки сразу формирует 32 адреса. Блок из 256 потоков содержит 8 warp. Упомянутые в разделе 5.2.3 при анализе bank общей памяти (независимо обслуживающих запросы разделов общей памяти) 32 lane — это 32 потока одного warp: количество bank, на которые попадают одновременно сформированные ими 32 адреса, определяет число циклов использования интерфейса общей памяти этой инструкцией.

Резидентность и заполнение. Чтобы блок потоков был резидентен на SM, то есть оставался на нём и мог быть запланирован к выполнению в любой момент, ему должны одновременно выделяться слоты потоков (по одному на поток), регистры и общая память. Число блоков потоков, которое позволяет разместить каждый вид ресурсов, равно округлённому вниз отношению общего объёма ресурса к потребности одного блока; число резидентных блоков определяется минимумом среди всех видов ресурсов:

Ресурс	Общий объём SM	Потребность одного блока потоков	Допустимое число блоков потоков
Слоты потоков	2048	256	8
Регистры	64K	$256 \times 128 = 32K$	2
Общая память	228 KB (233,472 bytes)	Рабочий набор 96 KiB + резерв 1 KiB на блок = 97 KiB	2
Предельное число блоков потоков	32	1	32

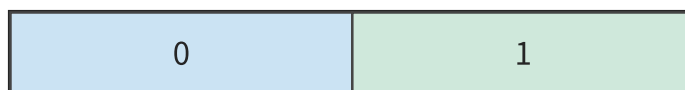
И регистры, и общая память допускают только 2 блока, поэтому на SM резидентны 2 блока потоков, то есть 16 warp. **Заполнение (occupancy)** — отношение числа резидентных warp к максимальному числу резидентных warp на SM; здесь оно равно $16/64 = 25\%$. На рис. 5-4 три вида ресурсов представлены тремя горизонтальными полосами: два блока потоков полностью занимают регистры и заполняют общую память, оставляя лишь 34 KiB, тогда как слоты потоков используются только на четверть.

Полезность изменения того или иного ресурса зависит от наиболее строгого ограничения. Если перенести аккумулятор объёмом 64 KiB из общей памяти в регистры, каждому потоку потребуется дополнительно хранить 64 значения FP32, то есть использовать

⁶Резидентность на SM, скрытие задержки и число инструкций MMA, размещение аккумулятора в регистрах, задержка 1000 ns и две загрузки на warp. Ограничения SM взяты из таблицы вычислительных возможностей руководства по программированию CUDA (столбец вычислительных возможностей 9.0) и руководства по настройке Норрег, определение занятости — из раздела руководства по написанию ядер, сведения об асинхронном копировании, барьерах, распределении работы между warp и swizzle — из раздела об асинхронном копировании данных, значение 3.35 TB/s — со страницы продукта H100, а 132 SM и пиковая производительность плотных матричных вычислений BF16, равная 989.4 TFLOP/s, — из описания архитектуры H100 (таблица 3). Число регистров на поток, задержка видеопамяти, число незавершённых загрузок на warp, ширина загрузки и форма инструкции MMA — значения, выбранные в этом разделе.

Два резидентных блока (синий:
блок 0, зелёный: блок 1)

64К регистров



Лимит резидентности: два блока заняли всё, третий не помещается

228 КВ общей
памяти



Лимит резидентности: осталось 34 KiB, третий блок не помещается

2048 слотов
потоков



Занято 512, поместятся ещё 6 блоков

Резидентны 2 блока = 16 варпов,
занятость $16/64 = 25\%$

Рис. 5-4. Три полосы соответствуют общему объёму регистров, общей памяти и слотов потоков SM; синим и зелёным показаны доли двух резидентных блоков потоков. Регистры заполнены полностью, а оставшихся 34 KiB общей памяти недостаточно для третьего блока, которому требуется 97 KiB, поэтому резидентность одновременно ограничивают оба ресурса; из слотов потоков использовано лишь 512. Каждый блок содержит 256 потоков, каждый поток использует 128 регистров, а блок — 96 KiB общей памяти и ещё 1 KiB резерва.

ещё 64 регистра. Потребность блока в общей памяти снизится до 32 KiB, что позволит разместить 6 блоков; однако если ранее каждый поток использовал только 64 регистра, то вместе с аккумулятором получится 128, и регистры по-прежнему допустят лишь 2 блока. Число резидентных блоков не изменится: вместо двух ограничивающих ресурсов останутся только регистры. Чтобы сделать резидентным третий блок, необходимо одновременно сократить число регистров на поток и объём общей памяти на блок либо уменьшить рабочий набор блока.

Скрытие задержки: резидентные warp обеспечивают запросы в полёте. Заполнение важно потому, что все независимые запросы к памяти исходят от резидентных warp: если запросов в полёте достаточно, то во время ожидания ответа на один запрос интерфейс остаётся полностью занят остальными — это называется скрыванием задержки. Закон Литтла из раздела 4.3.3 определяет объём данных в полёте, необходимый для поддержания пропускной способности: пропускная способность $\times$ задержка. При пропускной способности видеопамати H100 3,35 TB/s и задержке 600 ns всей карте требуется около 2,01 МВ данных в полёте, или примерно 15,2 KB на каждый из 132 SM. Теперь рассчитаем объём, обеспечиваемый резидентными warp: одна инструкция загрузки одного warp получает по 16 bytes для каждого из 32 потоков, то есть 512 bytes; каждый warp может иметь не более четырёх незавершённых загрузок; суммарно 16 warp дают 32 KiB.

$$B_{\text{need}} = \frac{3.35 \text{ TB/s} \times 600 \text{ ns}}{132} \approx 15.2 \text{ KB}, \quad B_{\text{have}} = 16 \times 4 \times 512 \text{ bytes} = 32 \text{ KiB}.$$

Доступный объём данных в полёте в 2,15 раза превышает требуемый; достаточно даже 8 резидентных warp, то есть одного блока потоков. В разделе 5.2.2 встретится явление, при котором «большой блок считывает меньше данных, но может работать медленнее»; здесь приведена его количественная формулировка: чем больше рабочий набор, тем меньше резидентных блоков и доступный объём данных в полёте. Как только он становится меньше требуемого, эффективная пропускная способность снижается до отношения доступного объёма данных в полёте к задержке, и более высокая пропускная способность интерфейса уже не может быть использована.

Гранулярность матричных инструкций. В разделе 4.2.1 показано, что матричные модули выполняют операции над блоками фиксированной формы. Здесь одна инструкция умножения матриц со сложением (ММА) имеет форму $16 \times 8 \times 16$: фрагмент A размером 16×16 и фрагмент W размером 16×8 ; одна инструкция выполняет $2 \times 16 \times 8 \times 16 = 4096$ операций с плавающей точкой. Для блока $128 \times 128 \times 64$ требуется выдать $(128/16) \times (128/8) \times (64/16) = 8 \times 16 \times 4 = 512$ инструкций, что соответствует 2,097,152 FLOPs этого блока. Эти 512 инструкций поровну распределяются между 8 warp: выходной блок содержит $8 \times 16 = 128$ выходных фрагментов размером 16×8 , каждый warp отвечает за 16 из них и выдаёт в общей сложности 64 инструкции за четыре шага по K. Инструкции ММА накапливают значения в регистрах, но блок из этого раздела хранит накопленный результат в общей памяти, поэтому до и после каждого шага по K необходимо перемещать эти 16 фрагментов между регистрами и общей памятью. Именно это перемещение устраняет описанный выше перенос аккумулятора в регистры.

Инструкции ММА можно выдавать двумя способами. Инструкция уровня warp выдаётся одним warp, а операнды предварительно загружаются в регистры этого warp. Инструкция уровня warpgroup в Norper выдаётся совместно четырьмя соседними warp, вместе называемыми warpgroup; она может напрямую получать операнды из

общей памяти и выполняется асинхронно: выдавшие её warp могут заниматься другой работой, ожидая результат. Именно такие инструкции FlashAttention-3 использует для организации перекрывающихся вычислений.⁷

Синхронизация: асинхронное копирование и барьеры. В разделе 4.4.2 два входных слота позволяют перекрыть загрузку и вычисление при условии, что известны два момента: «слот заполнен» и «слот освобождён». На SM эти моменты отмечаются **барьерами (barrier)**. Барьер — счётчик в общей памяти: участники достигают барьера после завершения своего этапа, а когда счётчик достигает заданного значения, ожидающие на барьере потоки освобождаются. Асинхронное копирование, которое после инициации выполняется аппаратно в фоне и не требует ожидания со стороны инициатора, по завершении также достигает барьера и увеличивает счётчик на число перенесённых байтов. Каждому слоту соответствует пара барьеров: барьер «заполнен» срабатывает после завершения копирования, и его ожидает вычисление; барьер «свободен» срабатывает после завершения вычисления, и его ожидает следующее копирование. Такой барьер действует только внутри одного SM, а однократное освобождение занимает лишь несколько десятков тактов; передача между SM должна проходить через L2 и стоит на один-два порядка дороже, см. раздел 4.4.4.

Барьеры позволяют разделить обязанности между warp: один warp только иницирует копирование (производитель), а остальные только вычисляют (потребители). Такая организация называется **warp specialization**. После инициации копирования блока $j+1$ warp-производитель не обязан ждать его завершения: он переходит к ожиданию барьера «свободен», чтобы подготовиться к записи блока $j+2$; warp-потребители после каждого срабатывания барьера «заполнен» начинают вычисление следующего блока.

Сначала рассчитаем длительность копирования и вычисления. Разделим пропускную способность видеопамати H100 3,35 TB/s и пиковую производительность плотных матричных вычислений BF16 989,4 TFLOP/s между 132 SM: на каждый SM приходится примерно 25,4 GB/s и 7,50 TFLOP/s. Ниже вычисления выполняются в предположении, что данный блок потоков единолично использует долю этого SM. Блоки A и W одного шага по K суммарно занимают 32 KiB, и их копирование требует около 1,29 μ s; умножение блока составляет 2,097,152 FLOPs и требует около 0,28 μ s. Копирование примерно в 4,6 раза медленнее вычисления, поскольку на каждый перемещённый byte в этом блоке приходится только 64 операции с плавающей точкой, тогда как отношение пиковой матричной производительности H100 к пропускной способности видеопамати составляет около 295 FLOP/byte. На рис. 5-5 с использованием этих двух значений времени показаны четыре блока K в двух слотах.

Такое разделение работы представляет собой программную реализацию конвейера из раздела 4.2.3. Там тремя видами ресурсов были матричные модули, интерфейс общей памяти и экспоненциальные модули; здесь warp-производитель управляет копированием и использует интерфейс общей памяти, warp-потребители управляют матричными и экспоненциальными вычислениями, а барьеры заставляют обе стороны ждать только в точках передачи данных. FlashAttention-3 построен по этой схеме: warpgroup-производитель иницирует копирование блоков K и V, а два warpgroup-потребителя попеременно выполняют умножение матриц и Softmax; пока один вычисляет экспоненту, другой умножает матрицы. В разделе 5.3.2 значения 1,29 μ s на копирование и 0,28 μ s на вычисление блока будут использованы для вывода общего времени конвейера, а в раз-

⁷FlashAttention, FlashAttention-2, FlashAttention-3.

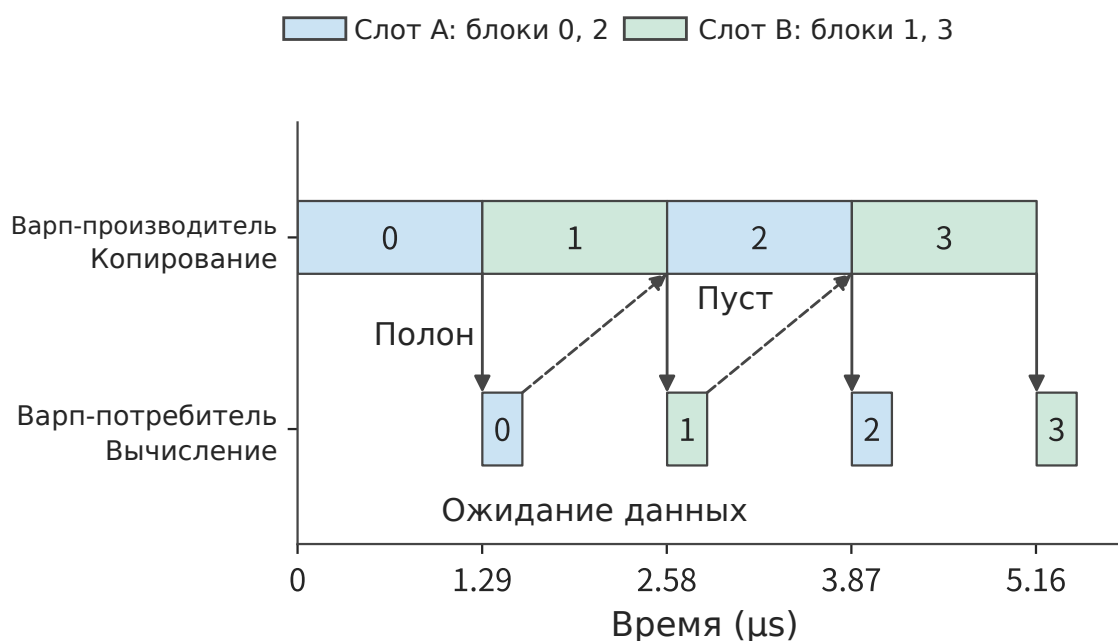


Рис. 5-5. В верхней строке показано асинхронное копирование, инициируемое warp-производителем, в нижней — вычисления warp-потребителей; синий и зелёный цвета соответствуют слотам А и В. Сплошные стрелки обозначают барьеры «заполнен»: вычисление может начаться только после завершения копирования; пунктирные стрелки обозначают барьеры «свободен»: блок 0 освобождает слот А на отметке 1,57 μs , но копирование блока 2 начинается только на отметке 2,58 μs — производителю не приходится ждать слот. На одном SM H100 копирование каждого блока занимает 1,29 μs , а вычисление — 0,28 μs ; копирования следуют одно за другим, а потребители после вычисления каждого блока ожидают очередного барьера «заполнен».

деле 5.3.3 будет подробно рассмотрено разбиение FlashAttention на блоки.⁸

Асинхронное пакетное копирование предъявляет ещё одно требование к расположению данных. В разделе 5.2.3 для распределения обращений по bank в конец каждой строки будет добавлено одно слово с помощью padding; асинхронное пакетное копирование перемещает целый блок по дескриптору тензора — структуре, содержащей форму и ша- ги тензора, — и не может вставлять заполнение между строками. Поэтому вместо этого адреса внутри блока переставляются по фиксированному правилу; такой приём называется **swizzle**: общий объём данных не меняется, а запросы одного столбца по-прежнему попадают в разные bank.

Соответствие Ascend. Пять рассмотренных в этом разделе этапов — потоки, резидентность, обращения к памяти, матричные инструкции и синхронизация — на Ascend соответствуют другому набору компонентов из разделов 4.2.3 и 4.6.2:

Механизм из этого раздела	NVIDIA GPU	Ascend
Выдача инструкций и разделение работы	Warp — единица выдачи инструкций для 32 потоков; warp-производители и warp-потребители разделяют работу внутри одного SM	AIC и AIV — отдельные ядра, а не warp: матричная работа выполняется на AIC, векторная — на AIV; у каждого свой поток инструкций
Резидентность и ёмкость	Регистры, общая память и слоты потоков определяют число резидентных блоков потоков на каждом SM	Ёмкость L0, L1 и унифицированного буфера определяет размер блоков, одновременно размещаемых на каждом ядре
Перемещение и синхронизация	Асинхронное копирование достигает барьера, warp ожидают барьер	Данные перемещаются с помощью MTE/NDDMA, а BufferID идентифицирует буферы и организует их передачу

Сделанный выше вывод о «достаточном объёме данных в полёте» основан на задержке 600 ns и четырёх незавершённых загрузках на warp. **Условие смены результата:** если задержка возрастёт до 1000 ns, а каждый warp сможет иметь только две незавершённые загрузки, каждому SM потребуется 25,4 KB данных в полёте, тогда как 16 warp смогут обеспечить лишь 16 KiB. Коэффициент покрытия, то есть отношение доступного объёма данных в полёте к требуемому, снизится до 0,65; для устранения дефицита потребуется как минимум 25 warp, то есть 4 блока потоков, однако регистры и общая память допускают лишь 2 блока. В таком случае заполнения 25% уже недостаточно: необходимо либо уменьшить рабочий набор, чтобы получить больше резидентных блоков, либо перейти на асинхронное пакетное копирование, при котором один поток запрашивает целый блок данных, а объём данных в полёте определяется размером блока и больше не зависит от числа warp.

Упражнение 5-1 · Расширение: какой ресурс определяет заполнение при уменьшении выходного блока до 64×64

Сохранив 256 потоков и 128 регистров на поток, измените размер выходного блока на 64×64, оставьте шаг по K равным 64 и продолжайте хранить аккумулятор в общей памяти. Рассчитайте потребность одного блока в общей памяти, допустимое число блоков потоков для каждого из четырёх видов ресур-

⁸FlashAttention, FlashAttention-2, FlashAttention-3.

сов и заполнение; укажите наиболее строгое ограничение. Затем уменьшите число регистров на поток до 64 и пересчитайте результат, объяснив, на какой ресурс переместилось ограничение. Наконец, при задержке 600 ns и четырёх загрузках на warp проверьте, по-прежнему ли объём данных в полёте покрывает потребность.

5.2 Разбиение отдельного оператора на блоки, размещение данных и фактический доступ к памяти

Вычисления с большими матрицами можно разбивать на блоки, как режут тофу: сначала разделить их на небольшие части, удобные для обработки, а затем вычислить каждую часть по очереди. Однако при выборе формы блоков необходимо учитывать и то, какие данные способны обрабатывать вычислительные блоки, и то, сколько данных помещается в ближайшей к ним памяти. В главе 4 уже было показано, что аппаратные матричные инструкции поддерживают ограниченный набор форм; множество вычислительных блоков всего ускорителя может одновременно обрабатывать множество блоков данных, но внутри каждой задачи работу всё равно необходимо организовать в небольшие шаги, которые способно выполнить оборудование.

Разбиение на блоки также должно позволять многократно использовать загруженные данные. Если продолжить аналогию с рабочим столом из главы 4, текущие входные данные и ещё не завершённые частичные результаты должны находиться рядом с вычислительным блоком и освобождать место только после использования. Если выходной блок слишком мал, одни и те же входные данные может потребоваться многократно переносить для разных выходных блоков; если он слишком велик, он займёт больше локальной памяти и вытеснит другие задачи, которые могли бы выполняться одновременно. Сначала проследим, какие данные нужны для одного выходного значения, затем объединим несколько выходных значений в блоки и разберём, как размер блока, порядок доступа и объём локальной памяти совместно влияют на повторное использование данных.

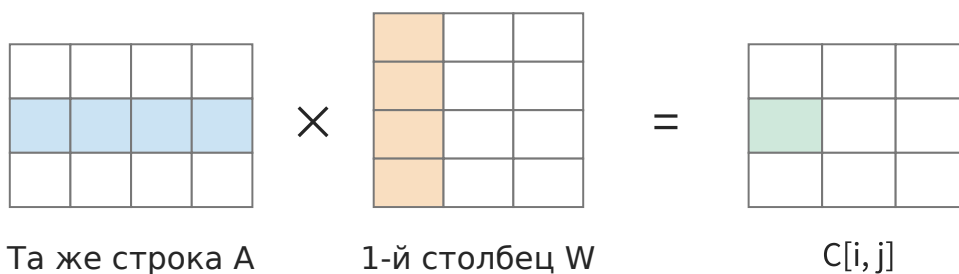
5.2.1 Форма матриц и повторное чтение

Ниже A обозначает вход X из начала главы, а проекция записывается как $C = AW$, где $M = 1024$, $K = 4096$, $N = 12288$. Матрицы A , W и C хранятся в BF16 и занимают соответственно 8, 96 и 24 MiB. Если A и W прочитать по одному разу, а C записать один раз, потребуется передать всего 128 MiB. Чтобы объяснить, почему одна из реализаций с разбиением на блоки создаёт около 3 GiB обращений к памяти, сначала рассмотрим вычисление одного выходного значения.

При вычислении $C[i,j]$ берутся строка i матрицы A и столбец j матрицы W , соответствующие элементы перемножаются, а затем суммируются вдоль измерения K . При следующем вычислении $C[i,j+1]$ справа используется соседний столбец W , но строка A остаётся той же. Синие области двух вычислений на рисунке полностью совпадают.

Если между двумя использованиями эта строка A остаётся в локальной памяти или кэше, её можно использовать повторно; если она уже была вытеснена, данные потребуются снова запросить со следующего уровня памяти. При переходе к следующей строке выходных данных повторно используются и элементы W . Математическая операция многократно использует одни и те же данные. От порядка выполнения операций в программе зависит, сколько раз эти данные пройдут через интерфейс памяти.

Сначала вычисляем один
результат



Затем вычисляем
соседний результат

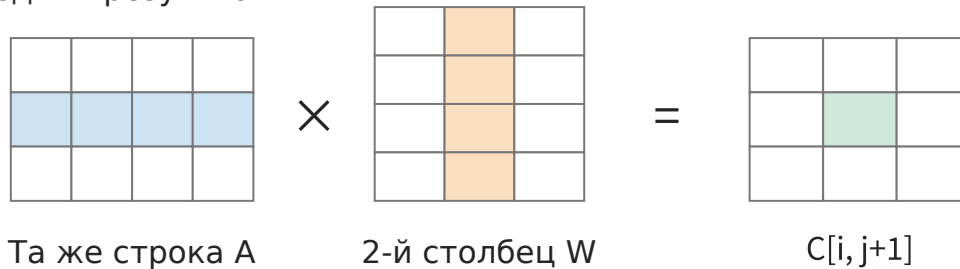


Рисунок 5-6. Сначала вычисляется одно выходное значение, затем соседнее. Синим показана используемая строка A, оранжевым — используемый столбец W, зелёным — полученный на этом шаге выходной элемент. Для наглядности матрицы на рисунке уменьшены; в фактическом примере K равно 4096.

В этом разделе подсчёт ведётся на интерфейсе между рабочим буфером и следующим уровнем памяти: суммируется число байтов при каждом чтении и записи программы. Указанный ниже объём обращений около 3 GiB также рассчитан для этого интерфейса.⁹

Если записать поэлементное вычисление с рисунка в виде циклов, получится:

```
for i in range(M):
    for j in range(N):
        acc = 0.0
        for k in range(K):
            acc += A[i, k] * W[k, j]
        C[i, j] = acc
```

После каждого полного выполнения внутреннего цикла k получается одно выходное значение. Одна операция умножения со сложением считается за два FLOP, поэтому для вычисления всех выходных значений требуется:

$$F_{\text{gemm}} = 2MKN \approx 103 \text{ GFLOPs.}$$

Базой для сравнения служит объём чтения и записи, при котором каждая из трёх матриц BF16 проходит через интерфейс один раз:

$$V_{\text{once}} = 2(MK + KN + MN) = (8 + 96 + 24) \text{ MiB} = 128 \text{ MiB.}$$

Здесь MiB равен 2^{20} bytes. В качестве базового случая принимается однократное чтение каждого входа и однократная запись выхода; далее относительно него рассчитывается дополнительный объём повторных обращений, вызванный разбиением на блоки.

Порядок циклов также определяет адрес каждого обращения. Если W хранится непрерывно по строкам, внутренний цикл выше последовательно читает $W[k,j]$, $W[k+1,j]$, а адреса различаются на N элементов. Если изменить порядок на i, k, j , внутренний цикл будет последовательно читать $W[k,j]$, $W[k,j+1]$, одновременно используя $A[i,k]$ для нескольких выходных значений. Тогда данные, полученные одной непрерывной передачей, можно сразу использовать в вычислении.

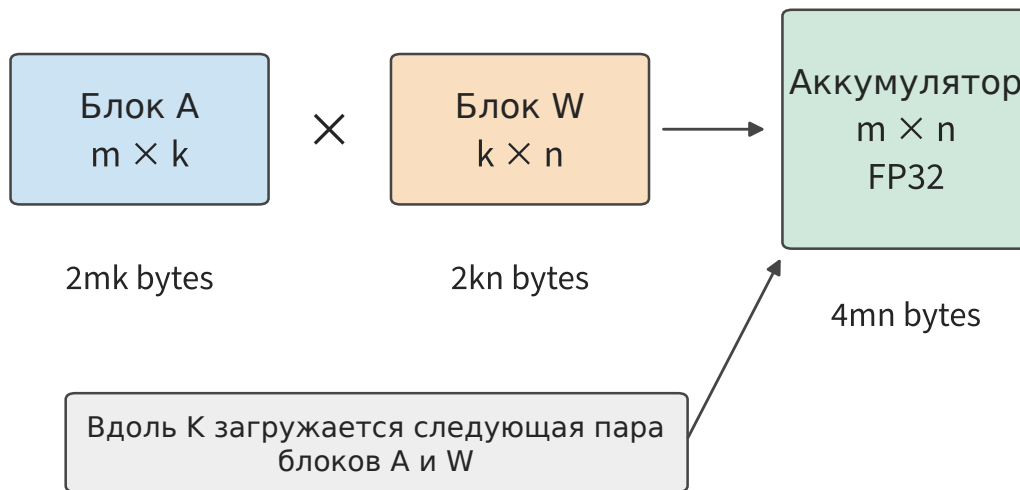
Однако при новом порядке необходимо одновременно хранить и результаты накопления для некоторого числа выходных значений. Если локальная память не вмещает эти результаты, каждое обновление может потребовать дополнительных операций чтения и записи на следующем уровне памяти. Далее с помощью разбиения на блоки мы ограничим объём одновременно хранимых данных, согласовав повторное использование входов с хранением выходов.

5.2.2 Повторное использование блоков при ограниченной ёмкости

Сначала выберем небольшой блок C в качестве текущего выходного блока и выделим буфер накопления для этого блока размером $m \times n$. На каждом шаге вдоль измерения K считывается пара входных блоков: блок A размером $m \times k$ и блок W размером $k \times n$. Их произведение обновляет один и тот же выходной аккумулятор; при загрузке следующей пары входных блоков уже полученная частичная сумма сохраняется. Только после завершения всего накопления вдоль K результат преобразуется в BF16 и записывается.

⁹Ёмкость блоков и повторное использование данных, включая подсчёт итераций циклов, дополнительные материалы по Orojenes и отображение банков памяти.

Пара входных блоков
обновляет весь выходной
блок



Входные блоки сменяются; выходной аккумулятор сохраняется

Рисунок 5-7. Текущий выходной блок $m \times n$ фиксируется, а соответствующие блоки A и W последовательно загружаются вдоль K и многократно обновляют одну и ту же частичную сумму; результат записывается обратно только после завершения всего накопления. Входы A и W представлены в BF16, аккумулятор — в FP32. На рисунке показано программное разбиение на блоки: одно блочное умножение может включать несколько аппаратных матричных инструкций; обозначения размеров указывают форму, а площади прямоугольников не масштабированы по числу байтов.

Эти блоки программно организуются для повторного использования данных, а внутри могут дополнительно разбиваться на несколько аппаратных матричных инструкций. Само разбиение на блоки не требует предварительно копировать исходные матрицы во множество независимых малых матриц; конкретные перемещения данных определяет исполняющая программа.

Каждый элемент блока A можно использовать для n столбцов выходных данных, а каждый элемент блока W — для m строк. Увеличение выходного блока позволяет задействовать однократно прочитанные данные в большем числе умножений со сложением, но одновременно требует хранить больше входных данных и выходных частичных сумм.

Пример 5-3. Насколько увеличение выходного блока матрицы сокращает чтение входных данных? Показанные на рисунке входной блок A, блок весов W и выходной аккумулятор занимают соответственно $2mk$, $2kn$ и $4mn$ bytes. При использовании одного набора входных буферов требуется следующий объём локальной памяти:

$$S = \underbrace{2mk}_{\text{блок A}} + \underbrace{2kn}_{\text{блок W}} + \underbrace{4mn}_{\text{аккумулятор}} \quad \text{bytes.}$$

Пусть $k = 32$. Сначала возьмём выходной блок 64×64 . Блоки A и W занимают по 4 KiB, аккумулятор — 16 KiB, итого 24 KiB. После увеличения выходного блока до 128×128 каждый из двух входных блоков занимает 8 KiB, аккумулятор — 64 KiB, а общий объём

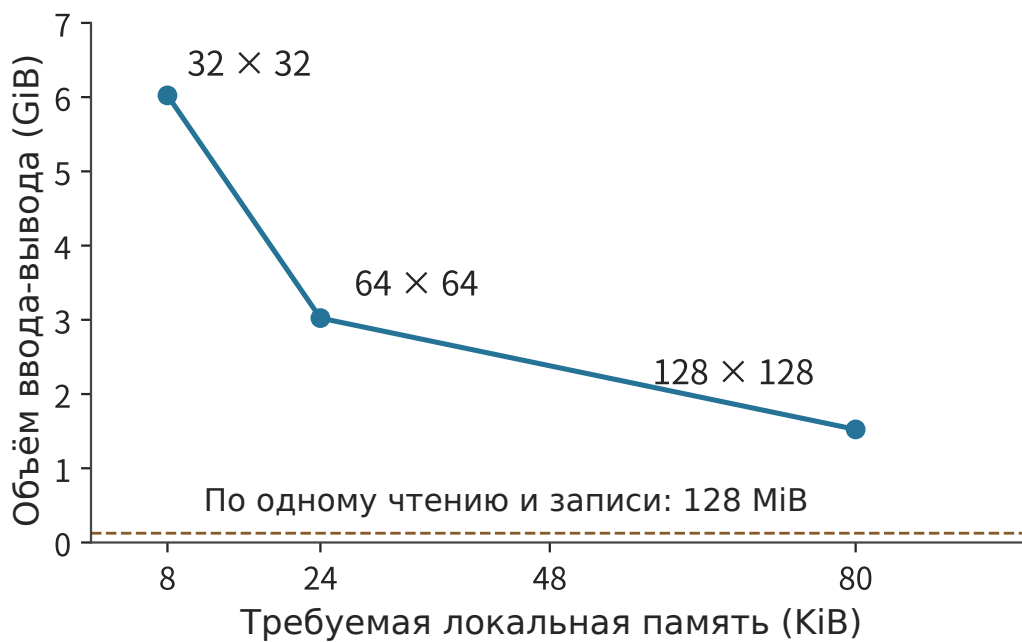


Рисунок 5-8. Каждая точка соответствует форме выходного блока. Увеличение выходного блока сокращает повторное чтение через интерфейс, но требует больше локальной памяти. Пунктирная линия показывает 128 MiB, соответствующие однократному прохождению трёх матриц; условия сравнения: $k=32$, один набор входных буферов, каждый выходной блок независимо читает входные данные.

возрастает до 80 KiB. Повторное использование входов увеличивается, но выходной аккумулятор растёт ещё быстрее.

Теперь рассчитаем чтение и запись для всей матрицы. Предположим, что каждый выходной блок независимо читает собственные входные данные, без учёта повторного использования кэша между разными выходными блоками. В направлении N имеется N/n блоков столбцов, и каждый из них должен полностью пройти по A ; в направлении M имеется M/m блоков строк, и каждый из них должен полностью пройти по W . Матрица C в итоге записывается только один раз, поэтому:

$$V = \underbrace{2MK \frac{N}{n}}_{\text{повторное чтение } A} + \underbrace{2KN \frac{M}{m}}_{\text{повторное чтение } W} + \underbrace{2MN}_{\text{запись } C}.$$

При выходном блоке 64×64 выход разбивается на 16 блоков строк и 192 блока столбцов. Каждый из 192 блоков столбцов один раз читает матрицу A размером 8 MiB, что даёт 1536 MiB; каждый из 16 блоков строк один раз читает матрицу W размером 96 MiB — также 1536 MiB. Вместе с 24 MiB выходных данных общий объём составляет 3096 MiB. Отсюда и берутся упомянутые в начале раздела примерно 3 GiB.

После перехода к блокам 128×128 число блоков строк сокращается с 16 до 8, а число блоков столбцов — со 192 до 96, поэтому объём чтения каждого из двух входов уменьшается вдвое. Размер выхода не меняется, и общий объём снижается до $768 + 768 + 24 = 1560$ MiB. На рисунке 5-8 этот выигрыш сопоставлен с объёмом занятой локальной памяти.

Полные значения приведены ниже. Арифметическая интенсивность рассчитывается

как отношение числа FLOP того же матричного умножения к указанному числу байтов, прошедших через интерфейс:

Выходной блок	Требуемая локальная память	Обращения между рабочим буфером и следующим уровнем памяти	Арифметическая интенсивность
32×32	8 KiB	6168 MiB	15.9 FLOP/byte
64×64	24 KiB	3096 MiB	31.7 FLOP/byte
128×128	80 KiB	1560 MiB	63.0 FLOP/byte

Арифметическая интенсивность всех трёх выходных блоков намного ниже приведённого в разделе 5.1.5 отношения пиковой производительности матричных блоков H100 к пропускной способности видеопамати, равного 295 FLOP/byte. Если учитывать только трафик на этом уровне интерфейса, все три варианта разбиения находятся в области ограничения пропускной способностью, а верхняя граница MFU матричных блоков составляет соответственно лишь 5%, 11% и 21%. Увеличение выходного блока сдвигает арифметическую интенсивность к точке пересечения, но одного разбиения на этом уровне недостаточно, чтобы её преодолеть: оставшееся повторное использование должны обеспечивать уровни памяти, расположенные ближе к вычислительным блокам.

Даже если объём обращений сокращается почти вдвое, время не обязательно уменьшится вдвое: это зависит от того, сколько задач ускоритель способен выполнять одновременно. На GPU входные буферы размещаются в общей памяти, а аккумуляторы — в общей памяти или регистрах. Здесь используется тот же подход, что и в разделе 5.1.5: каждый блок потоков отвечает за один выходной блок, а входные буферы и аккумуляторы размещаются в общей памяти. В качестве оборудования рассматривается один SM RTX PRO 6000: эта карта относится к compute capability 12.x, предел общей памяти на SM равен 100 KB, то есть 102,400 bytes, а каждый блок потоков дополнительно занимает 1 KiB резерва.¹⁰

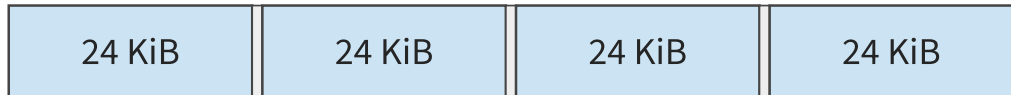
Одновременно резидентные блоки потоков могут чередовать запросы к памяти: пока одни блоки ожидают данные, другие продолжают работу. Если число одновременно резидентных рабочих наборов сокращается с четырёх до одного, уменьшается и объём других вычислений, которые можно выполнять во время ожидания памяти. Поэтому большой блок, хотя и читает меньше данных, фактически может получить меньшую пропускную способность. В разделе 5.1.5 это уже было рассчитано: резидентные warp обеспечивают запросы к памяти в полёте, а если объём байтов в полёте недостаточен, эффективная пропускная способность оказывается ниже пропускной способности интерфейса.

Предположим, что обе реализации ограничены пропускной способностью этого уровня, а их эффективная пропускная способность равна соответственно B_{64} и B_{128} . Тогда:

$$\frac{T_{128}}{T_{64}} = \frac{1560}{3096} \frac{B_{64}}{B_{128}}.$$

¹⁰ RTX PRO 6000 Blackwell использует архитектуру SM120 (см. среду выполнения эксперимента 5-2), то есть имеет вычислительные возможности 12.0; согласно таблице вычислительных возможностей руководства по программированию CUDA, в этом столбце верхний предел разделяемой памяти составляет 100 KB на SM и 99 KB на блок потоков, поэтому разница между ними соответствует резерву 1 KiB на блок.

Сколько наборов поместится в 100 KB общей памяти SM?



Четыре рабочих набора для
выходного блока 64 × 64



Один рабочий набор для
выходного блока 128 × 128

Серый: резерв 1 KiB на блок

Рисунок 5-9. Полная длина каждой полосы соответствует 100 KB общей памяти одного SM RTX PRO 6000. Ровно четыре рабочих набора по 24 KiB помещаются вместе с резервом 1 KiB на каждый блок; рабочий набор размером 80 KiB помещается только в одном экземпляре.

Если пропускная способность одинакова, время сокращается почти вдвое; если большой блок получает лишь 40% прежней пропускной способности, отношение времени составит примерно $0.504/0.4 = 1.26$, то есть выполнение, наоборот, замедлится примерно на 26%. Сначала по ограничению ёмкости можно исключить варианты, которые не помещаются, затем по объёму обращений сузить область сравнения и наконец измерить фактическую эффективность выполнения. В разделе 5.4 этот процесс будет поручен компилятору.

5.2.3 Размещение данных: распределение параллельных запросов по разным bank

В предыдущем подразделе большой блок, несмотря на меньший объём чтения, мог замедлиться из-за снижения параллелизма. Параллелизм зависит не только от числа резидентных блоков потоков, но и от того, могут ли одновременно обслуживаться создаваемые этими потоками запросы на чтение. Сначала рассмотрим размещение данных в локальной памяти. Возьмём временный блок FP32 размером 32×32, локальная память которого разделена на 32 bank; каждый bank за один раунд выдаёт одно 32-bit слово, а номер bank равен адресу слова по модулю 32. В разделе 5.1.5 уже говорилось, что 32 потока одного warp образуют 32 lane. Когда все 32 lane читают по одному слову из одной строки, запросы распределяются по 32 bank; при чтении столбца шаг между строками равен 32 словам, поэтому все адреса попадают в один bank и требуют 32 раунда.

Если добавить в конец каждой строки одно слово, шаг строки станет равен 33. Тогда номера bank соседних строк одного столбца последовательно увеличиваются на единицу, и 32 запроса распределяются по 32 bank, поэтому их можно выполнить за один раунд. Занимаемый объём возрастает с 4096 bytes до 4224 bytes, то есть примерно на 3.1%. Полезные данные и число операций чтения не меняются, но чтение столбца, прежде требовавшее 32 раунда, теперь завершается за один.

Добавленное в конец строки слово и есть padding. Оно не изменяет само вычисление,

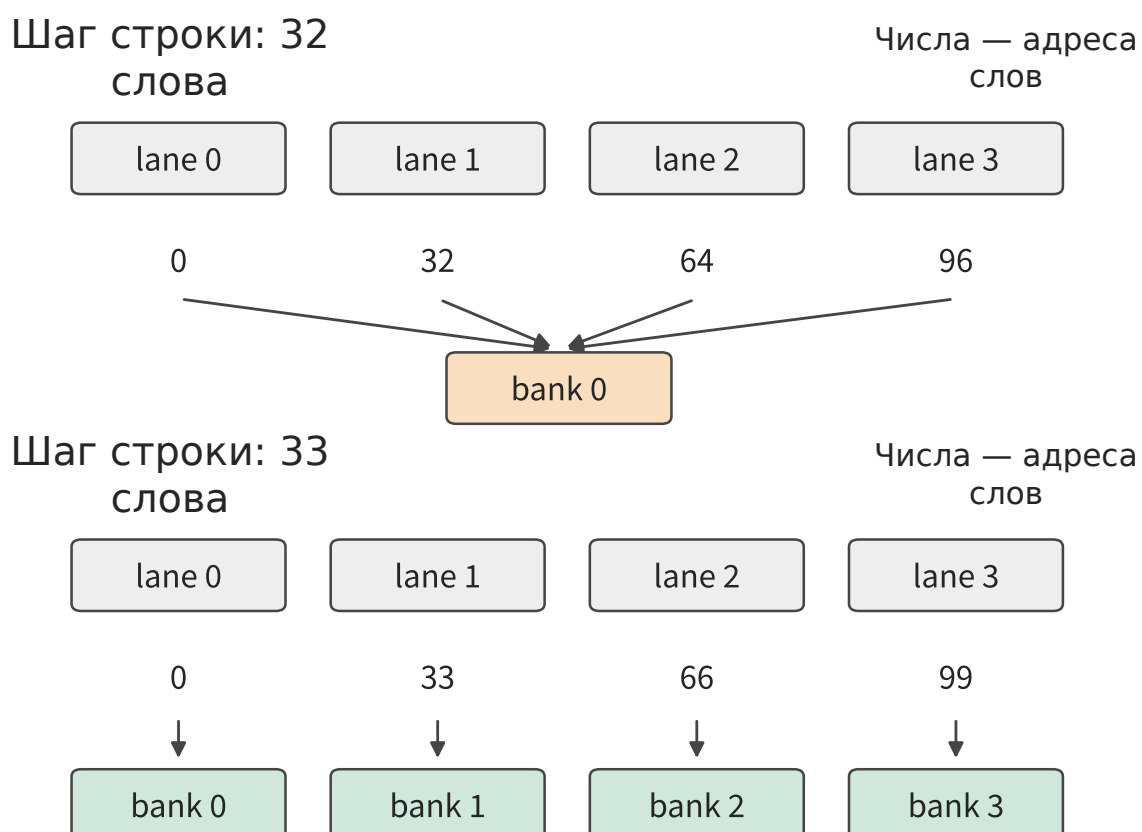


Рисунок 5-10. Тридцать два разных слова одного столбца одновременно запрашиваются 32 lane. В верхней части шаг строки равен 32 словам, поэтому запросы сосредоточены в одном bank; в нижней части шаг дополнен до 33 слов, и запросы распределяются по 32 bank. На рисунке показаны первые четыре запроса; предполагается, что каждый bank за один раунд выдаёт одно 32-bit слово, без учёта широковещательной передачи.

а лишь распределяет существующие параллельные запросы по разным bank.

Padding обеспечивает распределение запросов ценой дополнительных байтов. Можно не увеличивать объём, а лишь изменить места хранения: поместить элемент строки r и столбца c в столбец $c \oplus r$, где $\oplus$ — побитовое исключающее ИЛИ. Шаг строки по-прежнему равен 32 словам, но при чтении столбца 32 lane получают номера столбцов $c \oplus r$, которые при изменении r принимают 32 разных значения, поэтому номера bank также различаются и чтение завершается за один раунд. Библиотека шаблонов матричного умножения NVIDIA CUTLASS называет такое изменение мест хранения **swizzle**. Swizzle экономит место, требуемое для дополнения, но при каждом обращении сначала приходится вычислять номер столбца по формуле.

5.2.4 Выбор измерения для разбиения: параллельные задачи и частичные суммы

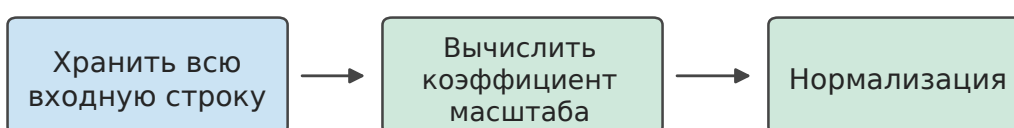
Padding решает вопрос одновременного обслуживания запросов. Если программа изначально создаёт лишь небольшое число задач, одного изменения размещения адресов недостаточно — вычисление также необходимо разделить. Рассмотрим RMSNorm: сначала по среднему квадратов элементов строки вычисляется общий коэффициент масштабирования, а затем масштабируется вся строка. Вычисление коэффициента масштабирования является редукцией: он получается из суммы квадратов всей строки; после этого каждый элемент умножается на этот коэффициент и соответствующий обучаемый параметр γ . Для входа BF16 формы [1024, 4096] можно назначить по одной группе на строку и независимо запланировать 1024 группы. Однако на этапе decode для одного запроса за шаг генерируется только один токен, поэтому такое же разбиение создаёт всего одну группу, оставляя значительную часть исполнительных ресурсов ускорителя незанятой.

Если разделить каждую строку на восемь сегментов по 512 элементов, число параллельных задач первого этапа увеличится. Полная редукция тогда разбивается на три этапа: каждый сегмент вычисляет локальную сумму квадратов, восемь локальных сумм объединяются для получения коэффициента масштабирования строки, после чего этот коэффициент используется для нормализации входа. Между первыми двумя этапами передаётся лишь небольшое число значений, но на последнем этапе исходные данные приходится читать повторно.

Подсчёт вдоль стрелок на рисунке 5-11 позволяет определить цену увеличения числа параллельных задач в виде дополнительных обращений к памяти. По-прежнему возьмём 1024 токена, причём вектор признаков каждого токена занимает одну строку входной матрицы. Число параллельных задач первого этапа увеличивается с 1024 до 8192 групп; 8192 локальные суммы FP32 занимают 32 KiB, а 1024 коэффициента масштабирования FP32 — 4 KiB. Программа с одной группой на строку сохраняет вход локально: она читает 8 MiB входных данных, дополнительно читает по одной копии BF16 γ для каждой строки, суммарно 8 MiB, а затем записывает 8 MiB результатов. Общий объём обращений равен $8 + 8 + 8 = 24$ MiB. После разбиения заключительный этап нормализации повторно читает 8 MiB входных данных; запись и чтение локальных сумм и коэффициентов масштабирования добавляют ещё около 0.1 MiB, поэтому общий объём достигает 32.1 MiB, а число запусков kernel увеличивается с одного до трёх.¹¹

¹¹Трёхэтапное разбиение вычисления RMSNorm. Входные данные и γ имеют формат BF16, γ считывается для каждой строки; вариант «одна группа на строку» сохраняет входные данные, а трёхэтапный

Одна группа обрабатывает строку



Разбить строку на восемь частей

512 элементов на часть → локальная сумма квадратов

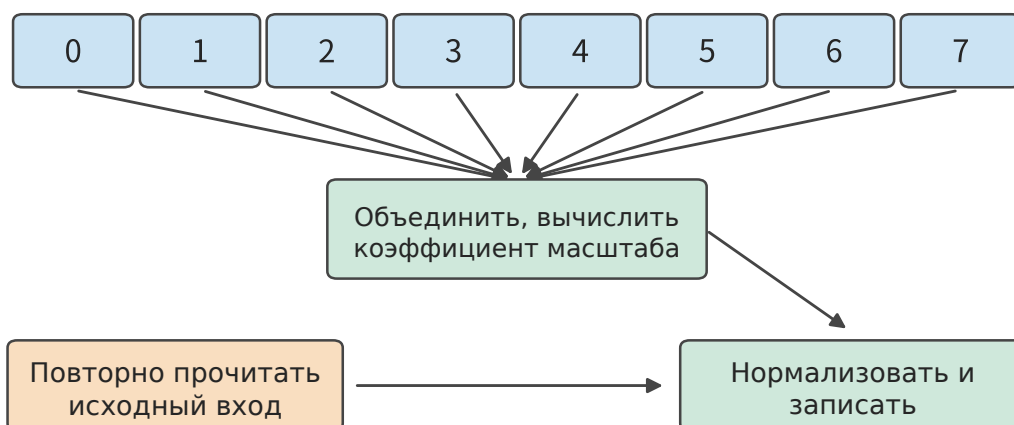
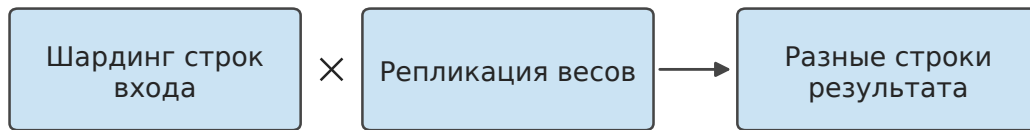
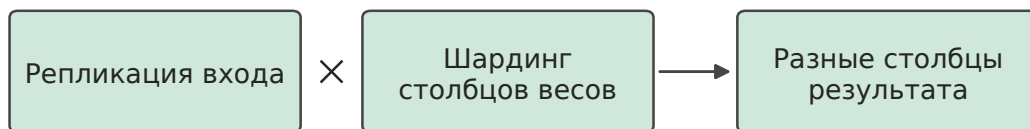


Рисунок 5-11. В верхней строке одна группа обрабатывает всю строку, а вход сохраняется до завершения нормализации; в нижней строке одна строка разделена на восемь сегментов: сначала вычисляются локальные суммы, затем они объединяются, после чего вход повторно читается и нормализуется. В каждой строке находится 4096 элементов BF16; на рисунке показана только одна строка, а при распространении на 1024 строки число задач первого этапа возрастает с 1024 до 8192. Оранжевая рамка обозначает дополнительное чтение исходного входа.

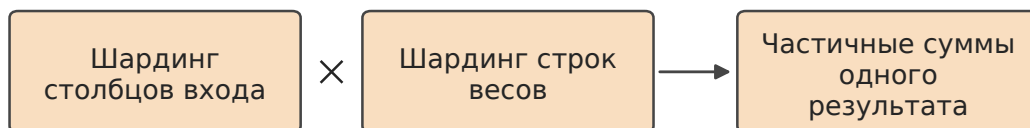
Разбиение по M :
строки результата



Разбиение по N :
столбцы результата



Разбиение по K : ось
редукции



Шарды результата собираются по мере надобности; шарды редукции нужно суммировать

Рисунок 5-12. При разбиении по выходным строкам M или выходным столбцам N каждый исполнитель получает полный результат в собственной области, после чего при необходимости эти области объединяются; при разбиении по измерению редукции K исполнители получают частичные суммы одного и того же выхода, которые перед использованием необходимо сложить. Прямоугольники обозначают принадлежность данных и не масштабированы по числу элементов матрицы.

Обработка каждой группой лишь одной восьмой строки, как на рисунке, даёт ещё одно преимущество: сокращается объём необходимых временных данных. Для хранения полной строки в FP32 требуется 16 KiB, а для одной восьмой — лишь 2 KiB. Благодаря этому временные данные, которые прежде из-за нехватки регистров записывались в более далёкую память, могут остаться в регистрах. Выигрыш от разделённой редукции возникает по двум причинам: увеличивается число параллельных задач и уменьшается объём временных данных каждой группы. Цена — дополнительное чтение входа и передача данных между этапами.

В RMSNorm разбиение выполняется вдоль измерения редукции: сумма квадратов строки разделяется на восемь локальных сумм, которые затем необходимо объединить. В матричном умножении имеется такой же выбор. Для $C_{M \times N} = A_{M \times K} W_{K \times N}$ работу можно распределить между разными исполнителями вдоль любого из трёх измерений. На рисунке 5-12 показано, какие входные данные требуются и какие результаты получаются при каждом из трёх вариантов разбиения.

вариант повторно считывает их на этапе применения.

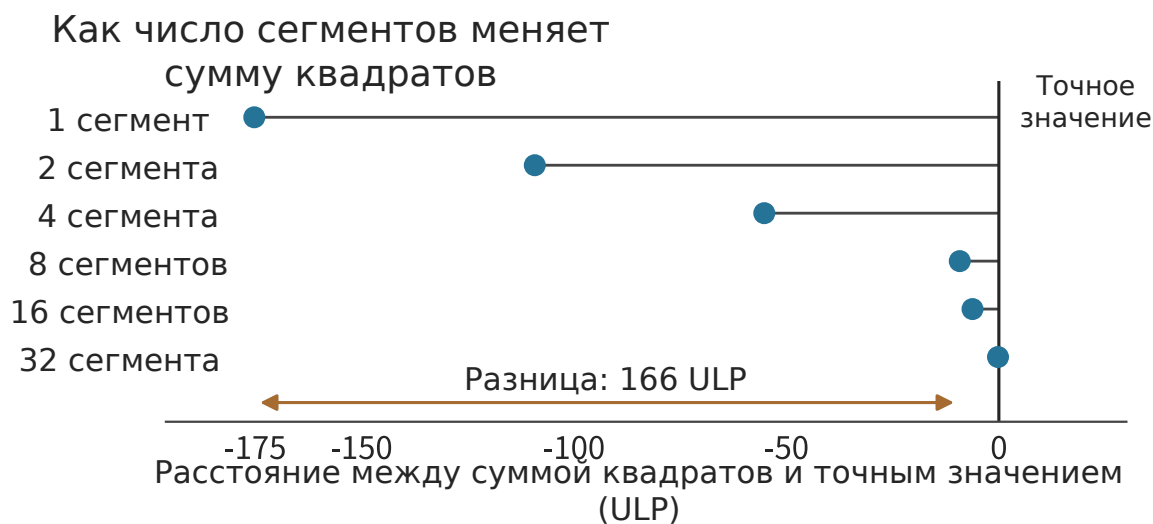
Первые два варианта требуют лишь копировать или делить входные данные по мере необходимости, а выход может оставаться разделённым, пока последующему оператору не потребуются полные данные. Третий вариант избавляет от копирования входа, но, как и в случае локальных сумм RMSNorm, добавляет этап объединения. Выходные блоки из раздела 5.2.2 одновременно разбиваются вдоль M и N , после чего накапливаются блок за блоком вдоль K : аккумулятор остаётся внутри одного блока потоков, поэтому объединение частичных сумм не пересекает никаких границ.

Порядок объединения также изменяет результат. Сложение чисел с плавающей точкой не обладает свойством ассоциативности: при вычислении $1 + 2^{-24} + 2^{-24}$ в FP32, если сначала сложить первые два слагаемых, сумма останется равной 1, а после добавления третьего также останется равной 1; если сначала сложить последние два слагаемых, получится 2^{-23} , а после прибавления к 1 — $1 + 2^{-23}$. Результаты двух порядков различаются на один ULP (unit in the last place — расстояние между двумя соседними числами с плавающей точкой при одинаковом показателе степени). Биты, утраченные при округлении, восстановить невозможно, а каждое последующее сложение выполняется уже с округлённым числом.

Для RMSNorm из этого подраздела можно вычислить величину такого различия. Возьмём строку из 4096 элементов BF16. Квадрат каждого элемента точно представим в FP32, поэтому различия между приведёнными ниже способами возникают только из-за порядка сложения. В верхней части рисунка 5-13 сравниваются суммы квадратов, вычисленные при разделении на 1, 2, 4, 8, 16 и 32 сегмента; по горизонтальной оси отложено их расстояние до точного значения. Когда одна группа обрабатывает всю строку, аккумулятор постепенно возрастает примерно до 2038, тогда как каждое добавляемое значение составляет лишь около 0.5, поэтому младшие разряды многократно отбрасываются. В итоге результат оказывается ниже точного значения на 175.2 ULP. После деления на восемь сегментов каждый аккумулятор возрастает лишь примерно до 255, после чего восемь локальных сумм складываются один раз; результат отличается от точного значения лишь на 9.2 ULP. Две траектории различаются на 166 ULP, а относительная разница составляет примерно 9.9×10^{-6} . Таким образом, разбиение изменяет не только число параллельных задач, но и точность.

В нижней части зафиксированы восемь локальных сумм, а изменяется только порядок их объединения. 40,320 вариантов порядка дают лишь три разных результата FP32, различающихся между собой на один ULP. Если накапливать локальные суммы по одному адресу с помощью атомарного сложения, порядок их поступления определяется планированием; поэтому для одних и тех же входных данных и одной и той же программы два запуска могут дать разные результаты. Если сначала записать восемь локальных сумм в буфер, дождаться завершения всех записей, а затем поручить одному исполнителю сложить их в фиксированном порядке, каждый запуск будет давать одинаковый результат. Это имеет две составляющие стоимости: дополнительную запись и последующее чтение; кроме того, объединение может начаться только после завершения самого медленного сегмента, тогда как атомарное сложение накапливает значения в порядке поступления и не требует ждать готовности всех результатов.

Различия не ограничиваются этапом вычисления суммы квадратов. Если продолжить вычисление с rms_norm_eps из Qwen3-8B, коэффициенты масштабирования двух траекторий различаются на 59 ULP, а первый выходной элемент этой строки — на 50 ULP; это



Те же 8 сегментов: 40,320 порядков слияния, всего 3
результата

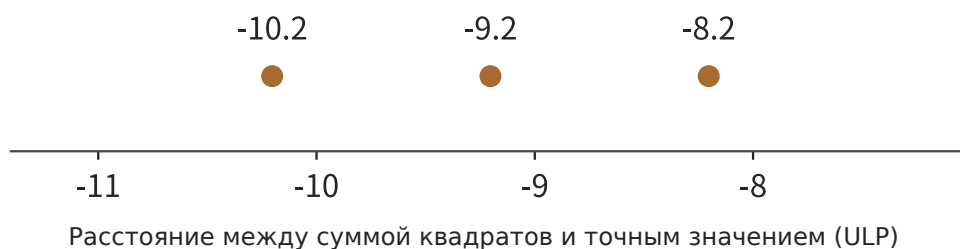


Рисунок 5-13. Расстояние между суммой квадратов одной строки из 4096 элементов BF16 и точным значением; единица горизонтальной оси — ULP формата FP32. В верхней части сравниваются шесть вариантов разбиения на 1–32 сегмента; в нижней части зафиксированы восемь локальных сумм и перечислены результаты для всех порядков их объединения. Точное значение вычислено с помощью рациональных чисел и используется только как база сравнения.

различие передаётся и в следующий слой.¹²

Следовательно, способ разбиения влияет не только на скорость. В начале этого подраздела уже было показано, что число сегментов зависит от доступного числа параллельных задач: для 1024 строк одной группы на строку уже достаточно, а при decode с одной строкой её необходимо разделить. Поэтому один и тот же kernel изменяет число сегментов в зависимости от размера batch, и одна и та же входная строка при разных batch даёт разные выходы. Чтобы выход определялся только входом, ни число сегментов, ни порядок объединения не должны зависеть от batch. Цена этого требования — возврат параллелизма при малом batch к уровню до разбиения. Именно этим в разделе 10.5.3 объясняется, почему генерация и обучение могут загружать одинаковые веса, но вычислять разные logprob для одного и того же префикса.

Разбиение на блоки может пересекать и границу одной карты. В пределах одной карты вычисления блоков совместно выполняются блоками потоков, общей памятью и локальными редукциями; после распределения по нескольким картам исполнители, разделённые по выходному измерению, должны каждый получить входные данные, а исполнители, разделённые по измерению редукции, должны передать частичные суммы в одно место для сложения. Все эти перемещения проходят через межсоединение карт. Математические зависимости не меняются — меняются расстояние и стоимость перемещения данных.

Планирование после разбиения определяет порядок выполнения, размещение, буферизацию и перекрытие операций. Например, после разбиения вдоль K можно дожидаться получения всех входных данных и только затем начать вычисление либо накапливать результат по мере поступления данных; от этого изменяются использование памяти и фактическое время ожидания. Образцы, последовательности, признаки, слои и эксперты модели предоставляют дополнительные направления распределения работы; слой представляет собой последовательный этап графа вычислений, а эксперт — ветвь, выбрannую маршрутизацией. В разделе 6.1.4 эти направления распределения будут сопоставлены с тремя рассмотренными здесь вариантами разбиения, а в разделе 6.2 последовательно будет выведен соответствующий каждому из них объём коммуникации.

Разбиение матриц на блоки, bank padding и разделение редукции по-разному упорядочивают одну и ту же математическую работу. Разбиение позволяет нескольким выходным значениям повторно использовать входные данные, padding распределяет запросы на чтение по разным bank, а разделение редукции увеличивает число задач, которые можно выполнять параллельно. При анализе программы необходимо отдельно вычислять выигрыш и накладные расходы каждого изменения, а также проверять, не изменяет ли оно результат вычислений с плавающей точкой. Сопутствующие эксперименты, сравнивающие непрерывный доступ и редукцию широких строк, содержат соответствующие записи выполнения.¹³

¹²Повторный расчёт с плавающей точкой для разного числа частей и порядка объединения. Эта учебная строка получена по заданной формуле с последующим округлением до BF16 и не является полученной при измерениях активацией; каждый квадрат точно представим в FP32, а различия между путями обусловлены исключительно порядком сложения. Точное значение вычислено с помощью рациональных чисел и используется только как эталон для сравнения; перебор порядков объединения охватывает возможные последовательности атомарного сложения, но не представляет фактическое распределение какого-либо бэкенда.

¹³Эксперимент 5-1, разбиение на блоки на GPU, разбиение редукции.

5.3 Слияние соседних операторов, буферизация и конвейерное выполнение

Внутри одного оператора входные данные можно повторно использовать в локальном хранилище. Такая же возможность существует и между соседними операторами: выход предыдущего оператора часто сразу становится входом следующего. Если следующий оператор может непосредственно прочитать результат из локального хранилища, можно избежать одной записи и повторного чтения. Для этого необходимо определить, какие данные нужны следующему оператору и когда все эти данные будут полностью вычислены.

5.3.1 Промежуточные тензоры и границы слияния

Расширим пример до двух проекций FFN. Пусть $G = XW_g$, $U = XW_u$, цепочка активации вычисляет $Z = \text{SiLU}(G) \odot U$, после чего Z передаётся в down-проекцию, которая уменьшает результат до ширины скрытого слоя. Формы G , U и Z равны $[1024, 12288]$; при хранении в BF16 каждый из них занимает 24 MiB.

SiLU и $\odot$ — поэлементные операции. $Z[i,j]$ зависит только от $G[i,j]$ и $U[i,j]$, между различными позициями нет редукции. Поэтому после чтения двух элементов в одной позиции можно сразу вычислить окончательный результат для этой позиции, не вычисляя предварительно все результаты SiLU.

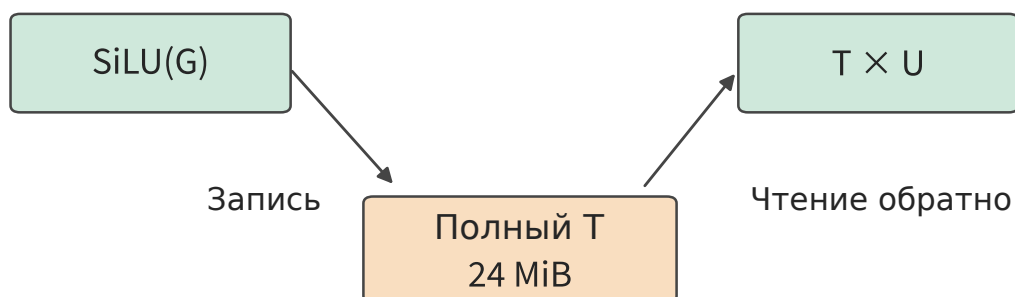
Сначала рассмотрим, куда направляется промежуточный результат $T = \text{SiLU}(G)$. При раздельном выполнении полный T записывается в хранилище следующего уровня, откуда затем считывается операцией умножения. После слияния небольшой фрагмент результатов SiLU можно напрямую передать умножению в том же kernel, а после использования освободить локальное пространство. Это похоже на передачу только что обработанного на верстаке материала непосредственно на следующую операцию без лишней отправки в удалённое место и обратной доставки. В данном случае устраняются запись и повторное чтение промежуточного результата, но обе операции по-прежнему должны быть выполнены.

Пример 5-4. Как слияние оператора активации сокращает чтение и запись промежуточных тензоров? При раздельном выполнении сначала вычисляется $T = \text{SiLU}(G)$: считывается G и записывается T , всего 48 MiB. Затем вычисляется $Z = T \odot U$: считываются T и U и записывается Z , всего 72 MiB. Общий объём чтения и записи составляет 120 MiB. Если слить обе операции, каждый раз считываются фрагменты G и U , локально выполняются SiLU и умножение, после чего записывается Z ; требуется лишь 72 MiB.

Сэкономленные 48 MiB — это ровно одна запись и одно чтение T . Если использовать модель пропускной способности из предыдущего раздела и считать эффективную пропускную способность обеих программ одинаковой, отношение времени составит $72/120 = 0.6$, то есть ускорение будет примерно 1,7 раза. В измерениях время цепочки активации снизилось примерно с 14,2 μs до 7,7 μs , что соответствует ускорению примерно в 1,8 раза. И теоретический расчёт, и результаты измерений указывают на один источник выигрыша: устранение записи и повторного чтения полного T .¹⁴

¹⁴Исходные условия, данные и анализ эксперимента 5-2. RTX PRO 6000, совместно используемый GPU, горячий кэш и воспроизведение графа; исходные медианы составили 14.20/7.68 μs , измерялась только под-цепочка активации.

Раздельно: промежуточный результат проходит через следующий уровень памяти



Слияние: фрагменты передаются следующему шагу

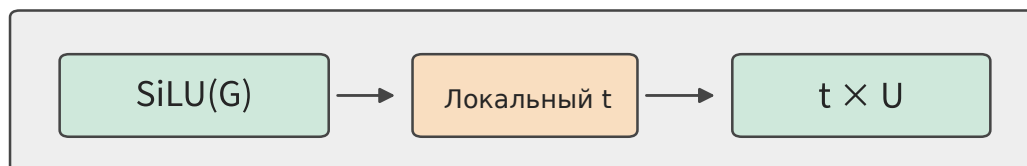


Рисунок 5-14: в верхней части полный T один раз записывается и один раз считывается; в нижней части локальный фрагмент t напрямую передаётся умножению. В обоих вариантах по-прежнему считываются G и U и записывается Z; эти общие рёбра ввода-вывода на рисунке опущены. После слияния сохраняются исходные правила округления промежуточного результата.

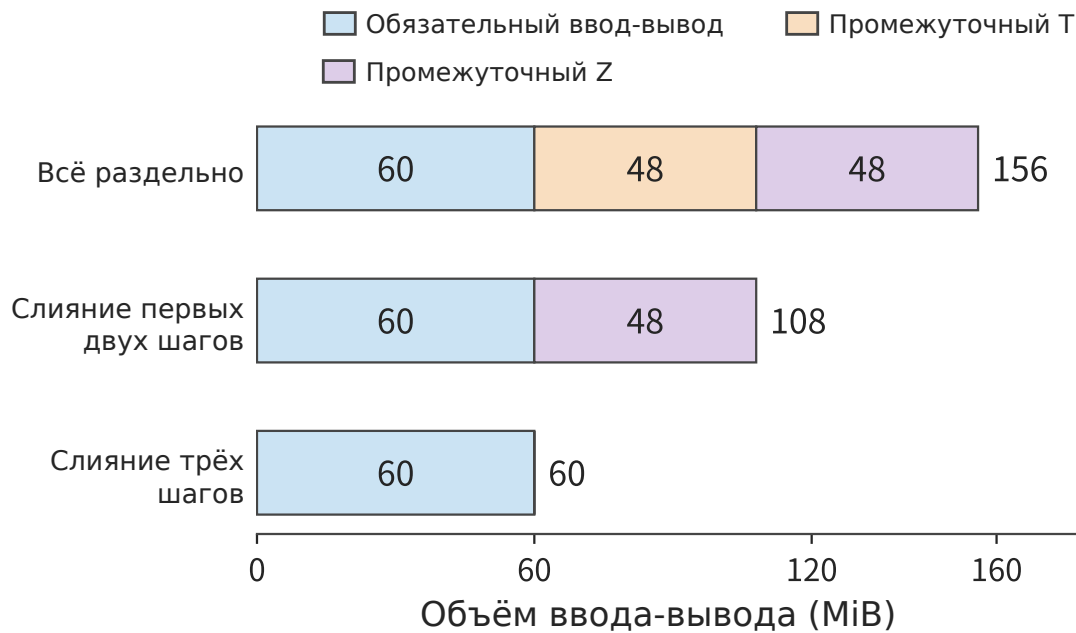


Рисунок 5-15: устранение сохранения каждого промежуточного результата размером 24 MiB исключает одну запись и одно чтение, то есть в сумме 48 MiB. G и U представлены в BF16, конечный результат занимает 1 byte на элемент, scale квантования задан заранее. Во всех вариантах используются одинаковые операции и порядок округления.

Далее Z квантуется с фиксированным scale, чтобы каждый элемент занимал только 1 byte. Отдельное преобразование требует считать 24 MiB Z и записать 12 MiB квантованного результата, добавляя 36 MiB. При полностью раздельном выполнении трёх операций общий объём составляет 156 MiB; при слиянии первых двух — 108 MiB; при слиянии всех трёх — $24 + 24 + 12 = 60$ MiB. На рисунке 5-15 слева в каждом столбце показаны обязательные чтение и запись входных и выходных данных, после чего последовательно добавлены чтение и запись двух промежуточных результатов.

Ёмкость промежуточных тензоров и объём обращений к ним здесь — разные величины. Предположим, что на каждом шаге сначала выделяется память под выходные данные, а после завершения освобождаются уже использованные входные данные. При отдельном выполнении SiLU в памяти одновременно находятся G, U и T, поэтому пиковый объём равен 72 MiB. При слиянии трёх операций одновременно находятся G, U и конечный результат, поэтому пиковый объём равен 60 MiB. Объём чтения и записи уменьшается на 96 MiB, а пиковое потребление памяти — лишь на 12 MiB, поскольку одну и ту же область хранилища можно многократно читать и записывать.¹⁵

Каждый результат поэлементной операции зависит только от входных данных в соответствующей позиции, поэтому его можно вычислить сразу после чтения, сократив необходимость сохранять промежуточные результаты. Для успешного повторного использования также необходимо, чтобы два последовательных шага применяли совместимые компоновки данных, а буфер не перезаписывался до завершения последнего чтения. Далее рассмотрим эти два условия на протяжении жизненного цикла одного блока данных.

¹⁵Слияние фиксированного scale и жизненный цикл тензоров.

5.3.2 Преобразование компоновки, повторное использование буферов и двойная буферизация

Соседние операторы иногда используют разные компоновки. Предыдущему оператору удобно записывать данные построчно, а следующему требуется считывать их в другом порядке блоков. Если создать отдельный результат перестановки размером 24 MiB, потребуется прочитать исходный результат и записать новый, всего 48 MiB. Вместо этого предыдущий оператор может сразу записывать данные в компоновке, необходимой последующим вычислениям, устраняя отдельную перестановку. Сравнение общего времени вариантов «сначала вычислить, затем переставить» и «при вычислении сразу записать в новой компоновке» позволяет определить, какой из них быстрее.

Буфер также определяет, могут ли два этапа выполняться с перекрытием. Предположим, что модуль перемещения помещает блок 0 в слот А, после чего вычислительный модуль считывает А. Пока вычислительный модуль читает А, модуль перемещения помещает блок 1 в слот В. После перехода вычислений к В в А можно загрузить блок 2. Так два слота используются поочерёдно: пока вычислительный модуль читает текущий блок, модуль перемещения может записывать следующий.

Важно различать моменты «данные полностью перемещены» и «данные полностью использованы» для слота А. Вычисления начинают чтение только после завершения перемещения, а записать следующий блок в слот А, перезаписав прежнее содержимое, можно лишь после завершения последнего чтения. Используем значения для одного SM H100 из раздела 5.1.5: перемещение каждого блока занимает 1,29 μ s, вычисление — 0,28 μ s, причём модуль перемещения и вычислительный модуль работают независимо. На рисунке 5-16 показано состояние каждого слота в трёх временных интервалах, а на рисунке 5-17 на этой основе построена полная временная шкала перемещений и вычислений.

Пример 5-5. Как двойная буферизация перекрывает перемещение данных и вычисления? Перемещение каждого блока занимает 1,29 μ s, а вычисление — 0,28 μ s. Модуль перемещения и вычислительный модуль работают независимо, накладные расходы событий считаются нулевыми. Последовательная обработка четырёх блоков требует $4 \times (1.29 + 0.28) \approx 6.28 \mu$ s.

При использовании двух слотов блок 0 перемещается в интервале 0–1,29 μ s и вычисляется в интервале 1,29–1,57 μ s; одновременно в интервале 1,29–2,58 μ s перемещается блок 1. Вычисление блока 0 завершается уже в момент 1,57 μ s, и слот А сразу освобождается, но модуль перемещения лишь в момент 2,58 μ s завершает перемещение блока 1 и начинает записывать блок 2. Затем модуль перемещения работает непрерывно, выдавая по одному блоку каждые 1,29 μ s, тогда как вычислительный модуль каждый раз занят лишь 0,28 μ s. Четвёртый блок завершается в момент 5,44 μ s, поэтому время уменьшается всего примерно на 13%.

Обобщим четыре блока до n блоков. Сначала необходимо переместить первый блок, затем каждый следующий блок завершается через время, требуемое более медленному этапу, а для последнего блока нужно дополнительно завершить вычисление. Поэтому:

$$T_{\text{pipe}} = t_m + (n - 1) \max(t_m, t_c) + t_c.$$

Здесь t_m — время перемещения одного блока, а t_c — время его вычисления. В предыдущем примере $t_m > t_c$, поэтому модуль перемещения непрерывно занят от 0 до 5,16 μ s, а общее время равно $4 \times 1.29 + 0.28 \approx 5.44 \mu$ s. Вычислительный модуль работа-

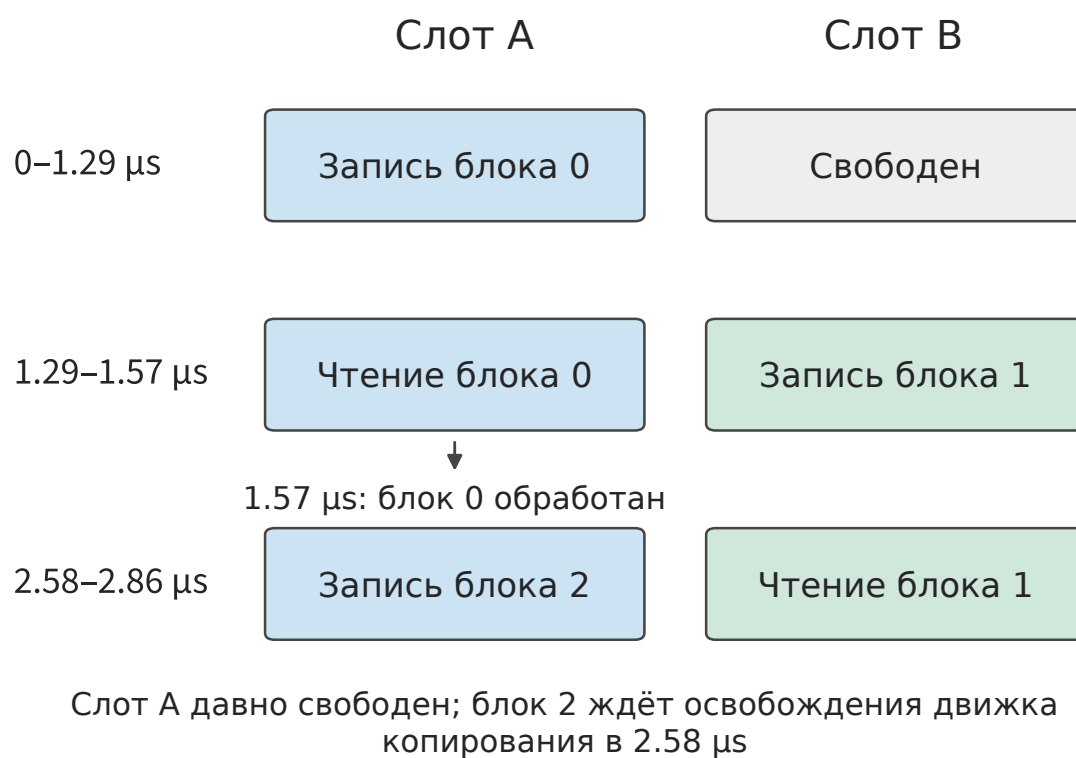


Рисунок 5-16: блок 0 использует слот А в интервале 1,29–1,57 μs , после чего слот А освобождается; блок 2 начинает записываться в А только после того, как модуль перемещения завершит перемещение блока 1 в момент 2,58 μs . На рисунке выделены три временных интервала; в интервале 1,57–2,58 μs ни в одном слоте нет данных, готовых к вычислению, поэтому вычислительный модуль простаивает. Цвет неизменно обозначает слот А или В, а текст указывает, какой блок в данный момент читается или записывается.

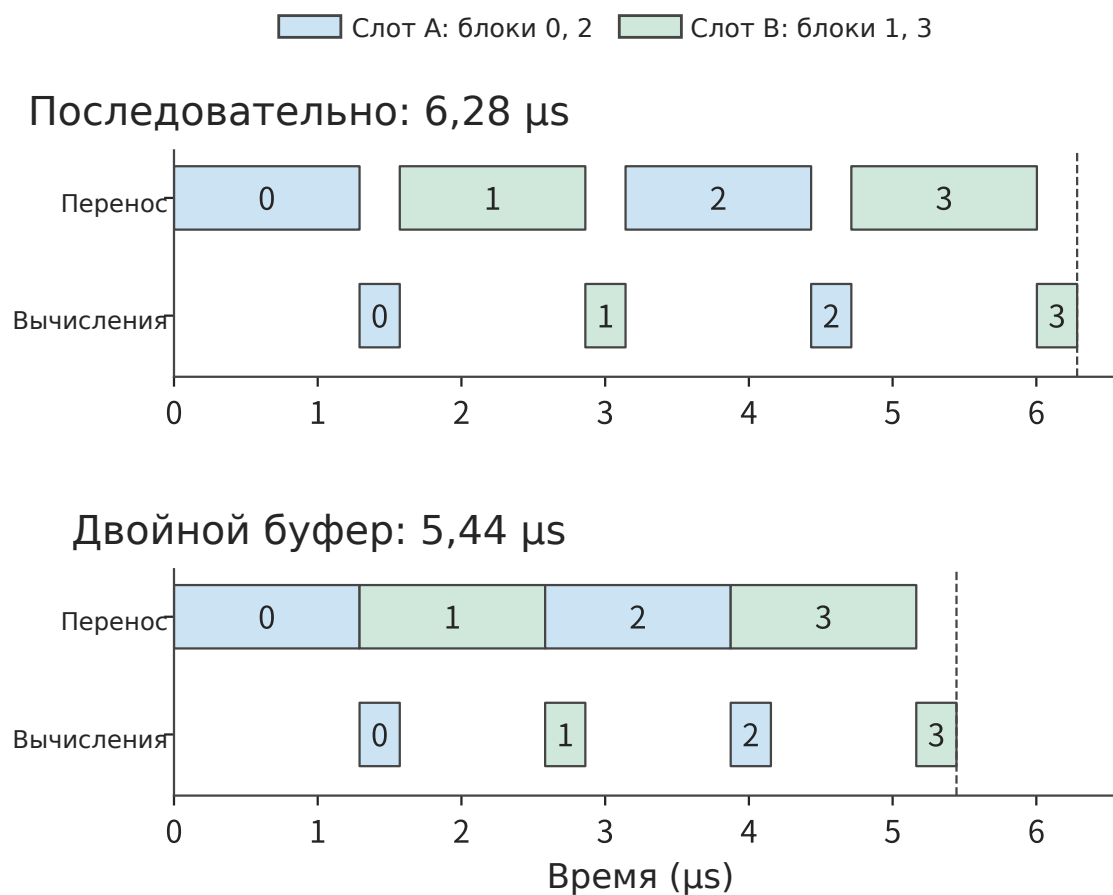


Рисунок 5-17: поочерёдное повторное использование двух входных слотов позволяет перекрыть перемещение и вычисления. На одном SM H100 перемещение каждого блока занимает 1,29 μs , а вычисление — 0,28 μs ; ресурсы независимы, накладные расходы синхронизации не учитываются; одинаковый цвет обозначает один и тот же слот. Перекрытие скрывает только вычисления, а время завершения определяется непрерывной последовательностью перемещений.

ет лишь около одной пятой времени. Добавление третьего слота ничего не даст: двух слотов уже достаточно, чтобы модуль перемещения не простаивал, а дополнительные слоты не ускорят перемещение; ускорение вычислений также не поможет. Два слота — минимальная конфигурация, позволяющая организовать перекрытие. Увеличение числа слотов до n , при котором перемещение начинается с опережением на $n - 1$ блоков, в CUTLASS называется использованием n **stage**. В этом примере модуль перемещения уже работает непрерывно, поэтому дополнительные stage лишь занимают больше памяти. Они могут заполнить простои только тогда, когда время перемещения отдельных блоков нестабильно или задержка одного перемещения превышает время вычисления одного блока; при этом требуемая ёмкость растёт пропорционально числу stage. Для дальнейшего сокращения времени необходимо уменьшить число перемещаемых байтов на блок: например, увеличить выходной блок согласно разделу 5.2.2, чтобы каждый входной байт участвовал в большем числе умножений со сложением, либо позволить одновременно выполняющимся блокам потоков совместно использовать входной блок в L2. Другой вариант — перейти на карту с более высокой пропускной способностью видеопамяти.

Вернёмся к перестановке, рассмотренной в начале подраздела. Если два этапа используют разный порядок доступа, буфер должен хранить больше ещё не использованных данных. Три ожидающих блока по 16 KiB занимают 48 KiB, а вместе с 32 KiB пространства для перестановки требуется 80 KiB. Один блок потоков RTX PRO 6000 может использовать не более 99 KB общей памяти, поэтому при одиночном размещении этот объём помещается. Если на одном SM должны одновременно находиться два таких блока потоков, каждому достанется не более 49 KiB: 100 KB за вычетом 1 KiB резерва для каждого из двух блоков делятся пополам. После вычета 32 KiB пространства для перестановки остаётся только один ожидающий слот размером 16 KiB. Перед записью второго блока модуль перемещения должен дожидаться, пока вычислительный модуль использует и освободит этот слот. Механизм, при котором нижележащий этап не успевает обрабатывать данные и вынуждает вышележащий этап приостановиться, называется **обратным давлением (backpressure)**. Выбор общей компоновки позволяет сократить очередь, а увеличение буфера — разместить больше уже записанных, но ещё не использованных данных.¹⁶

Ввод с хоста из раздела 5.1 также можно организовать таким образом. Поток копирования передаёт следующую партию, а вычислительный поток обрабатывает текущую. Вычислительный поток ожидает событие «копирование завершено», прежде чем читать новые входные данные, а поток копирования ожидает событие «вычисление завершено», прежде чем перезаписывать старый буфер. Эти два события соответственно гарантируют готовность входных данных и завершение использования старых данных, а выигрыш по времени возникает благодаря одновременной работе двух ресурсов.

5.3.3 FlashAttention: разбиение на блоки и онлайн-Softmax

В слитых поэлементных операциях из раздела 5.3.1 каждый выходной элемент зависит только от входных данных в соответствующей позиции, однако Softmax в механизме внимания использует всю строку оценок. При вычислении одной строки выходных данных внимания можно сначала преобразовать оценки каждой позиции в ещё не нормализованные веса, вычислить с ними взвешенную сумму значений, а затем разделить её

¹⁶Порядок потоковой обработки и бюджет буфера; перемещение на хост и время занятости буфера.

на сумму весов. Можно сначала обработать один блок, запомнить его вклад во взвешенную сумму и сумму весов, а затем перейти к следующему. Если вклады удаётся правильно объединить, нет необходимости постоянно хранить все уже обработанные оценки. Сложность заключается в том, что Softmax вычисляет экспоненты оценок: если в последующем блоке встретится более высокая оценка, числовую базу ранее накопленного результата потребуется скорректировать.

FlashAttention организует хранение вокруг такого поблочного вычисления, сокращая чтение и запись промежуточных матриц внимания для длинных последовательностей в видеопамяти. Этот метод был предложен в 2022 году Tri Dao и другими исследователями Стэнфордского университета совместно с коллегами из Университета штата Нью-Йорк в Буффало.¹⁷

При знакомстве с FlashAttention автор вспомнил свой опыт работы над AKG в 2019 году. Тогда для слияния оператора Softmax автор долго искал подходящий онлайн-алгоритм. Впоследствии он нашёл исследование NVIDIA, опубликованное в 2018 году, и смог, используя возможности AKG по разбиению на блоки и слиянию, объединить предшествующее матричное умножение с последующим Softmax. В то время автор ещё даже не был знаком с механизмом внимания и лишь позднее осознал, что случайно пришёл к одной из ключевых идей FlashAttention. Однако для перехода от этого шага к полноценному FlashAttention необходимо также включить последующее взвешенное суммирование с V в онлайн-обновление и организовать внутрикристальное хранение и перемещение данных для всего вычисления внимания. Приведённый ниже вывод показывает, как это кажущееся локальным изменение алгоритма устраняет крупные промежуточные матрицы.

Вернёмся к самому вычислению внимания. Стандартный механизм внимания сначала вычисляет $S = QK^T / \sqrt{d}$, затем $P = \text{softmax}(S)$ и наконец $O = PV$.

Если поручить эти три операции отдельным универсальным операторам, программисту можно непосредственно составить по формулам, а результаты между операторами передавать через промежуточные тензоры. Зная все зависимости данных в вычислении внимания, можно выбрать другой способ компиляции и выполнения: использовать редукционную зависимость Softmax, перейти границы отдельных операторов и непрерывно выполнять вычисления над небольшими блоками данных. Сначала вычислим затраты на полное сохранение промежуточных тензоров, а затем выведем состояние, которое необходимо сохранять при поблочном вычислении.

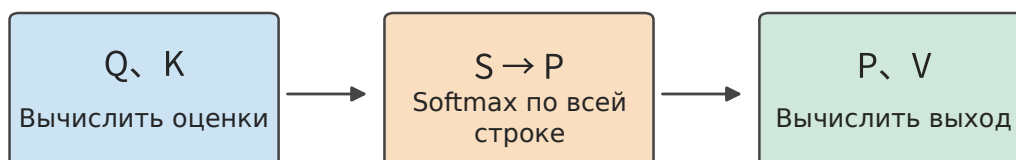
Рассмотрим одну голову с длиной последовательности $L = 8192$ и размерностью головы $d = 128$, вычисляя внимание для всех токенов. Q , K , V и O используют BF16 и занимают по 2 MiB; полные S и P используют FP32 и занимают по 256 MiB. Однократное чтение Q , K и V и однократная запись O дают в сумме 8 MiB. Однако S и P по одному разу записываются и считываются, из-за чего требуется передать 1 GiB данных. Размер промежуточных матриц растёт квадратично с длиной последовательности, что приводит к большому объёму чтения и записи.¹⁸

Чтобы устранить этот 1 GiB промежуточного чтения и записи, можно воспользоваться идеей из раздела 5.3.1, однако прямому применению поэлементного слияния меша-

¹⁷FlashAttention, FlashAttention-2, FlashAttention-3.

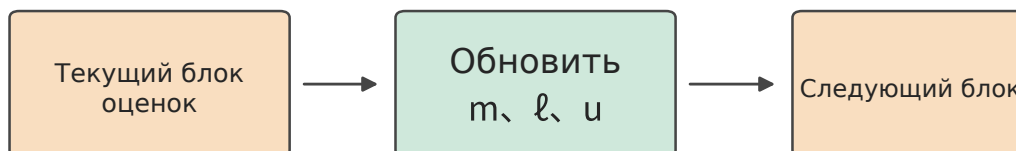
¹⁸Разбиение attention на блоки и онлайн-состояние; размер блока и объём обращений при буфере 99 KB. Модель ёмкости предполагает одну голову, отсутствие маски, повторное использование входного слота для K/V и временного слота для S/P ; предел разделяемой памяти 99 KB на блок потоков взят из таблицы вычислительных возможностей руководства по программированию CUDA (столбец вычислительных возможностей 12.x).

Хранить полные
промежуточные
матрицы



S и P по 256 MiB: записать, затем прочитать

Обработать текущий блок, сохранив статистики



Передать статистики; буфер оценок оставить следующему блоку

Рисунок 5-18: в верхней части сохраняются полные S и P — две матрицы FP32 по 256 MiB каждая, запись и повторное чтение которых в сумме составляют 1 GiB. В нижней части передаются только максимальное значение m, сумма экспонент ℓ и взвешенное значение u для уже обработанной части; после завершения обработки каждого блока его буфер оценок можно использовать повторно. Стрелки обобщённо показывают порядок обработки; полное вычисление также требует чтения Q, K и V.

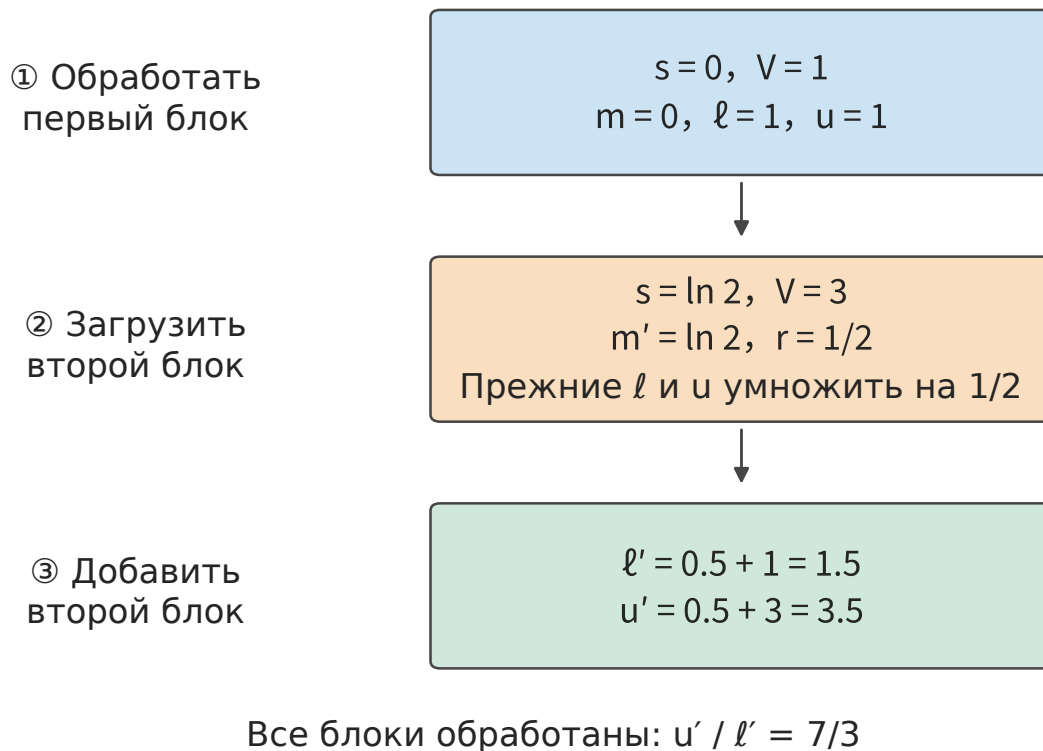


Рисунок 5-19: оценки двух блоков равны соответственно 0 и $\ln 2$, а значения — 1 и 3. После увеличения максимального значения прежнюю сумму экспонент и прежнее взвешенное значение одновременно умножают на $1/2$, затем добавляют вклад нового блока и лишь в конце выполняют деление. По стрелкам передаётся статистика, поэтому старые оценки хранить не требуется. m — максимальная из обработанных оценок, ℓ — сумма экспонент относительно m , u — соответствующая взвешенная сумма векторов значений, r — коэффициент масштабирования при переходе к новой базе максимального значения.

ет одно обстоятельство: знаменатель Softmax представляет собой сумму экспонент всей строки. После чтения только предыдущего блока неизвестны ни окончательный знаменатель, ни наличие более высокой оценки в следующих блоках.

Чтобы отбрасывать уже обработанные оценки, необходимо хранить три величины: максимальную оценку m , сумму экспонент ℓ относительно m и сумму значений u с теми же экспоненциальными весами. Здесь u обычно является вектором; сначала рассмотрим процесс обновления на небольшом примере со скалярными значениями.

Пример 5-6. Как онлайн-Softmax объединяет результаты блоков, сохраняя нормализацию? Оценки равны $[0, \ln 2]$, соответствующие значения — $[1, 3]$, а каждый блок содержит только один элемент. Для первого блока получаем $m = 0, \ell = 1, u = 1$. После обработки второго блока максимальное значение увеличивается до $\ln 2$, поэтому прежние ℓ и u умножаются на $r = 1/2$. Экспонента нового элемента равна 1, следовательно, $\ell' = 1/2 + 1 = 1.5$, $u' = 1/2 + 3 = 3.5$, а результат равен $7/3$. При обработке всей строки за один раз два веса пропорциональны 1 и 2, поэтому результат также равен $(1 + 2 \times 3)/3 = 7/3$.

В небольшом примере второй блок увеличивает максимальное значение с 0 до $\ln 2$.

Экспонента старого элемента, ранее равная 1, при переходе к новой базе становится равной 1/2. Старые вклады в знаменатель и числитель необходимо уменьшить одновременно, чтобы сохранить неизменными их относительные веса.

Обобщим это на строку оценок. Вычитание общего максимального значения позволяет избежать переполнения экспоненты. Пусть $\ell = \sum_j \exp(s_j - m)$, $u = \sum_j \exp(s_j - m)V_j$, а окончательный результат равен u/ℓ . После поступления нового блока старое состояние сначала переводится к новой базе максимального значения, а затем к нему добавляется вклад нового блока:

$$m' = \max(m, \max(s)), \quad r = \exp(m - m'), \quad p_j = \exp(s_j - m'),$$

$$\ell' = r\ell + \sum_j p_j, \quad u' = ru + \sum_j p_j V_j.$$

Суммирование в правой части выполняется только по j из нового блока. Полный вклад старых блоков уже сохранён в ℓ и u , поэтому повторно считывать старые оценки не требуется.

Именно это рекуррентное соотношение позволяет FlashAttention отказаться от хранения полных S и P . После обработки каждого блока обновляются m , ℓ и u ; использованные оценки и вероятности этого блока можно отбросить, а их буфер передать следующему блоку. Сопоставление запросов и ключей и взвешивание значений в пределах той же области внимания по-прежнему должны быть выполнены полностью. Меняются порядок вычислений и способ хранения промежуточных результатов; из-за вычислений с плавающей точкой также возникают различия в округлении.

Таким образом, задача хранения полной матрицы оценок превращается в задачу размещения текущего блока в быстром хранилище. Далее определим, сколько строк Q и сколько строк K/V следует обрабатывать за один раз, чтобы эффективнее использовать доступное пространство. Возьмём 99 KB общей памяти, доступной одному блоку потоков RTX PRO 6000, то есть 101,376 bytes, в качестве быстрого буфера. Одновременно будем хранить a строк Q и накопитель выходных данных FP32; размер блока оценок составит $a \times b$. Блоки K и V поэтапно используют один и тот же входной слот размером $b \times d$, оценки и вероятности — один слот размером $a \times b$, а для построчной статистики и временных данных обновления выделяются ещё три вектора FP32. Требуемый объём локального хранилища равен:

$$S = \underbrace{2ad}_Q + \underbrace{2bd}_{K/V \text{ слот}} + \underbrace{4ab}_{\text{оценки/вероятности}} + \underbrace{4ad}_{\text{накопитель результата}} + \underbrace{12a}_{\text{построчное состояние}} \leq 101376.$$

Чем меньше блок K/V , тем меньше места занимают он сам и блок оценок, а значит, можно разместить больше строк Q . Большее число строк Q совместно использует один проход по K/V , поэтому вся совокупность K/V считывается повторно меньшее число раз. При $b=64$ помещаются $a=82$ строки Q , что требует $\lceil 8192/82 \rceil = 100$ полных проходов. Каждый проход считывает 4 MiB K/V ; с учётом однократного чтения Q и записи O общий объём равен $100 \times 4 + 4 = 404$ MiB.

Если уменьшить b до 1, помещаются $a=128$ строк Q , число проходов снижается до 64, а объём обращений — до $64 \times 4 + 4 = 260$ MiB. Однако каждый проход теперь состоит не из

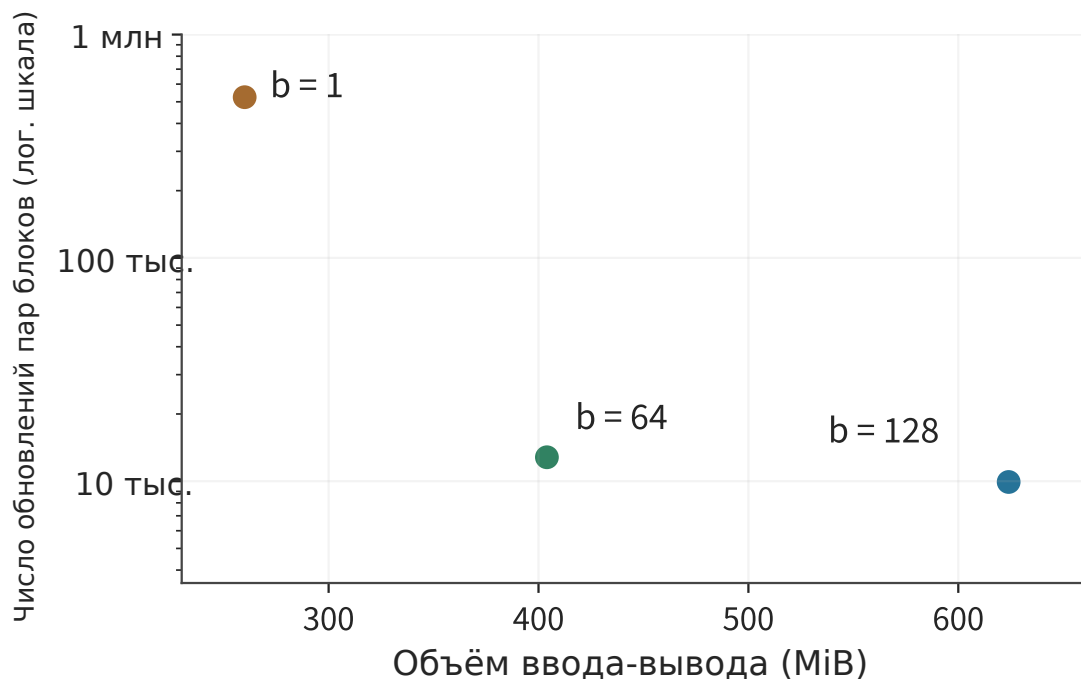


Рисунок 5-20: быстрый буфер представляет собой 99 KB общей памяти одного блока потоков RTX PRO 6000; длина последовательности равна 8192, размерность головы — 128, маска отсутствует. Каждая точка соответствует одному размеру блока K/V из таблицы в тексте. По горизонтальной оси отложен объём обращений между буфером и хранилищем следующего уровня, по вертикальной — число онлайн-обновлений; используется логарифмический масштаб. При переходе от точки $b=64$ к точке $b=1$ объём чтения уменьшается, но число обновлений возрастает примерно в 41 раз. b обозначает число токенов в одном блоке K/V.

128, а из 8192 блоков K/V, поэтому онлайн-состояние приходится обновлять значительно чаще.

Число строк в блоке K/V, b	Число помещающихся строк Q , a	Число полных проходов по K/V	Объём обращений между буфером и хранилищем следующего уровня	Число обновлений состояния между блоками Q и K/V
1	128	64	260 MiB	524288
64	82	100	404 MiB	12800
128	53	155	624 MiB	9920

На рисунке 5-20 две составляющие затрат показаны на одной плоскости: чем левее находится точка, тем меньше данных считывается, а чем ниже — тем меньше выполняется обновлений. При уменьшении блока с 64 строк до одной считывается на 144 MiB меньше, но число обновлений состояния возрастает с 12800 до 524288, то есть примерно в 41 раз. Каждое обновление требует управления циклом, сравнения максимальных значений, вычисления экспонент и масштабирования выходных данных. Кроме того, обработка лишь одной строки K/V за раз делает одно из измерений матричного умножения слишком малым, что мешает полноценно использовать матричные вычислительные модули. Меньшие блоки K/V сокращают чтение, но увеличивают число обновлений; большие блоки благоприятны для матричных вычислений и уменьшают накладные расходы

циклов. Поэтому при выборе размера блока необходимо одновременно сравнивать оба вида затрат.

Онлайн-редукция устраняет чтение и запись полных промежуточных матриц, но внутриблочные матричные умножения, вычисления экспонент и обновления статистики всё равно должны выполняться. Дальнейшая оптимизация направлена именно на эту оставшуюся работу. FlashAttention-2 улучшает распределение задач между потоками, сокращая обмен промежуточными данными и число нематричных операций. FlashAttention-3 использует асинхронное выполнение, перекрывая перемещение, матричное умножение и Softmax. После устранения огромных промежуточных матриц эти усовершенствования дополнительно сокращают оставшееся время вычислений и перемещений.¹⁹

Чтобы проверить влияние этих механизмов в реальной программе, в сопутствующем эксперименте сравнивались две реализации одного интерфейса внимания в PyTorch. Здесь **бэкенд (backend)** означает вычислительную программу, которую фреймворк фактически выбирает внутри интерфейса: вызывающая сторона передаёт одинаковые Q, K и V, но фреймворк может выполнить вычисление внимания разными программами.

Математический бэкенд (SDPBackend.MATH) составляет вычисление по формуле внимания из универсальных операций матричного умножения, Softmax и других, сохраняя полные промежуточные матрицы оценок и вероятностей. **Бэкенд FlashAttention (SDPBackend.FLASH_ATTENTION)** использует описанные выше разбиение на блоки и онлайн-редукцию, избегая хранения полных промежуточных матриц. В эксперименте эти два бэкенда выбирались явно, после чего сравнивались время выполнения одного и того же вычисления внимания и дополнительное потребление видеопамяти.

В эксперименте использовалась RTX PRO 6000 Blackwell; входные и выходные данные имели формат BF16, за один раз обрабатывались одна последовательность и одна голова внимания с размерностью 128. Длина последовательности составляла 8192 токена. Использовалось каузальное внимание: каждая позиция учитывала только себя и предшествующие позиции. После прогрева группа операций GPU предварительно записывалась с помощью CUDA Graph и затем многократно выполнялась, чтобы исключить влияние повторной отправки операций с хоста. В пересчёте на одно вычисление внимания математическая реализация с сохранением полных промежуточных матриц занимала около 2,6 ms, а реализация FlashAttention с разбиением на блоки и онлайн-редукцией — около 80 µs; ускорение последней составляло примерно 33 раза.²⁰

Также сравним пиковое дополнительное потребление видеопамяти при одном обычном вызове. Входные Q, K и V заранее находились в видеопамяти, а измерение охватывало выходные данные и временную рабочую область: математической реализации требовалось около 848 MiB, а реализации FlashAttention — около 22 MiB.

Это сравнение полноценных реализаций показывает, как совместно работают разбиение на блоки, онлайн-редукция и слияние: разбиение позволяет разместить текущие данные в быстром хранилище, онлайн-редукция позволяет вычислять Softmax поблочно, а слияние устраняет запись внутриблочных промежуточных результатов. Вместе эти три механизма изменяют время выполнения и требования к промежуточному хранилищу.

¹⁹FlashAttention, FlashAttention-2, FlashAttention-3.

²⁰Эксперимент 5-3. Сравнение для одной головы на RTX PRO 6000 с горячим кэшем: 2.64 ms/79.5 µs; пиковый объём дополнительных выделений памяти — около 848/22.2 MiB, включая временную память, выделенную каждым бэкендом.

5.4 Компилятор: представление, преобразование и выбор

В двух предыдущих разделах циклы, размеры блоков, буферизация и границы операторов выбирались вручную. Реальные модели используют различные формы входных данных и комбинации операторов, а каждый ускоритель имеет собственную иерархию памяти и вычислительные ресурсы. Задача компилятора — представить эти способы реализации в виде программы, исключить некорректные преобразования с учётом зависимостей, а затем сравнить затраты на выполнение оставшихся реализаций.

5.4.1 Зачем сравнивать несколько реализаций

Вернёмся к цепочке поэлементных операций: SiLU выполняет нелинейное преобразование каждого числа, затем результат поэлементно умножается на другой вход, после чего квантуется в представление с низкой разрядностью. На каждой из двух границ между соседними операторами можно выбрать раздельное или слитное выполнение, поэтому возможны четыре варианта разбиения: три отдельных шага, слияние первых двух шагов, слияние последних двух шагов и слияние всех трёх шагов. Если цепочка содержит десять операторов, существует $2^9 = 512$ вариантов разбиения по границам между соседними операторами. Для каждого варианта также можно выбирать размер блока, локальную раскладку и глубину конвейера.

Эти решения влияют друг на друга. Слияние двух шагов устраняет промежуточную запись, но одновременно увеличивает количество значений, которые необходимо хранить в один момент времени; рост объёма временных данных, в свою очередь, влияет на допустимый размер блока. Поэтому компилятор должен одновременно представлять содержание вычислений, порядок выполнения и места хранения данных. По одному лишь имени оператора невозможно оценить эти затраты.

Компилятор может выбирать реализацию в три этапа. Сначала математические операции представляются в виде циклов и обращений к массивам с явным указанием преобразуемых циклов и отношений чтения и записи. Затем корректность преобразований проверяется с учётом зависимостей чтения и записи и численных правил. Наконец, с учётом доступной ёмкости и измеренного времени сравниваются затраты на выполнение. Ниже первые два этапа сначала показаны на понятных циклах, а затем объясняется, как система автоматической генерации кода операторов AKG организует весь процесс с помощью полиэдральной компиляции — метода, описывающего итерации циклов и обращения к массивам посредством линейных ограничений.

5.4.2 Представление разбиения на блоки и слияния с помощью преобразований циклов

В разделе 5.2 определялось, блок какого размера обрабатывать за один раз, но необходимо также решить, в каком порядке вычислять эти блоки. Даже при одинаковом разбиении данных ненужные обращения всё равно возникнут, если только что использованные данные вытеснить, а вскоре снова загрузить. Преобразования циклов позволяют выразить такой порядок в программе: `split` разделяет цикл на «номер блока» и «позицию внутри блока», а `reorder` изменяет порядок обхода, чтобы данные, которые вскоре понадобятся снова, могли остаться в локальной памяти. Выбрав также уровень цикла, на котором промежуточный результат сохраняется или используется, можно явно задать продолжительность его хранения.

Рассмотрим $Y = \text{SiLU}(AW)$ при прежних $M = 1024, K = 4096, N = 12288$: матричное умножение использует накопление в FP32, после чего результат округляется до BF16 и передаётся функции активации. В самой прямолинейной программе сначала полностью вычисляется C, а затем обход C формирует Y:

```
for i in range(M):
    for j in range(N):
        acc = 0.0
        for k in range(K):
            acc += A[i, k] * W[k, j]
        C[i, j] = round_to_bf16(acc)

for i in range(M):
    for j in range(N):
        Y[i, j] = silu(C[i, j])
```

Первое преобразование — **разделение (split)**. Разделим i на номер блока i_o и координату внутри блока i_i , задав $i = 64i_o + i_i$. Исходные 1024 строки превратятся в 16 блоков по 64 строки. Аналогичная обработка j даёт 192 блока столбцов. Одновременное разделение двух выходных измерений образует тайл 64×64 .

Сначала разделение изменяет способ представления индексов. Чтобы непрерывно вычислять один тайл, необходимо также **изменить порядок (reorder)**: во внешних циклах обходить выходные блоки, а во внутренних — элементы внутри блока. Затем K делится с шагом 32 на 128 блоков редукции. Программа выбирает выходной тайл, последовательно загружает требуемые блоки A и W и накапливает результат в одном и том же выходном блоке.

После перехода к такой структуре циклов можно определить, как долго должны храниться входные блоки. Текущие блоки A и W занимают по 4 КиБ и могут заменяться после вычисления одного блока редукции; 4096 аккумуляторов FP32 занимают 16 КиБ и должны сохраняться до завершения вычисления всех 128 блоков редукции. Поэтому буферы этих двух типов следует выделять и освобождать на разных уровнях циклов.

Наконец, изменяется **место вычисления** функции активации. После выполнения всей редукции по K для одного выходного тайла все его 4096 значений C уже готовы, поэтому можно сразу вычислить SiLU и записать Y ; незавершённость остальных выходных тайлов не влияет на эту часть результата. Благодаря этому локально требуется хранить только результат текущего выходного блока, а полная матрица C размером 24 МиБ больше не нужна, что устраняет 48 МиБ записи и повторного чтения.

```
for io in range(ceil_div(M, BM)):
    for jo in range(ceil_div(N, BN)):
        acc = zeros_fp32(BM, BN)
        for ko in range(ceil_div(K, BK)):
            a = load_tile(A, io, ko, BM, BK)
            w = load_tile(W, ko, jo, BK, BN)
            matmul_accumulate(acc, a, w)
        c = round_to_bf16(acc)
        store_tile(Y, io, jo, silu(c))
```

Здесь `load_tile` и `matmul_accumulate` обозначают соответственно блочное чтение и

Уровень вложенности задаёт срок хранения

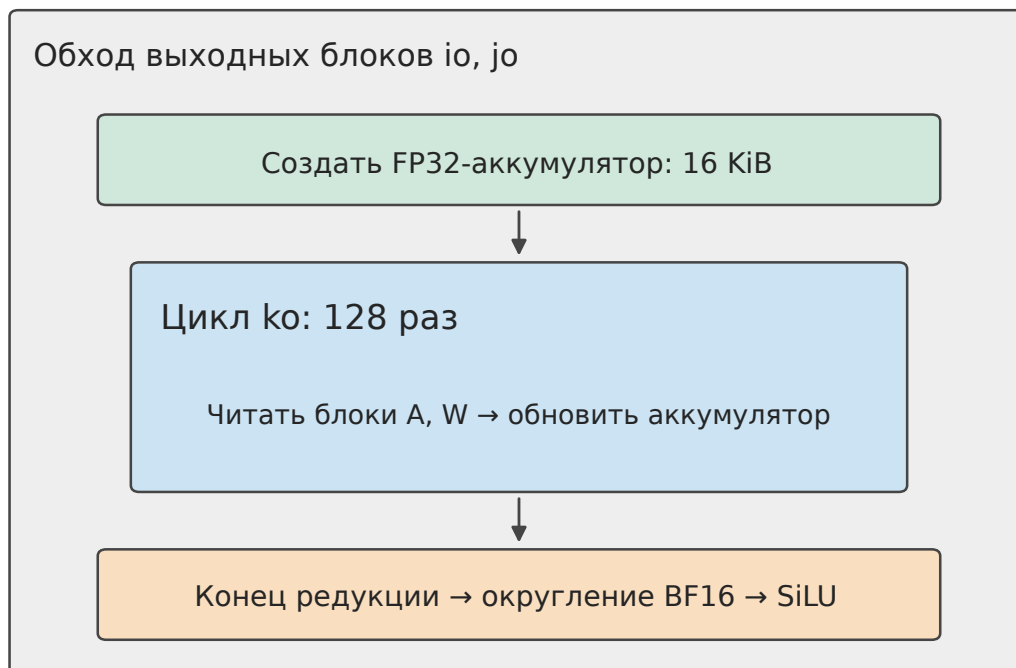


Рисунок 5-21. Иерархия циклов определяет, как долго должны храниться временные данные. Внешний цикл выбирает выходной блок и создаёт аккумулятор размером 16 КиБ; внутренний цикл ko многократно считывает блоки A и W , а после завершения всей редукции выполняются округление и вычисление функции активации.

матричное накопление; допустимые позиции в последнем неполном тайле отмечаются маской. Отступы в псевдокоде наглядно показывают порядок действий: входные блоки обновляются внутри цикла ko , аккумулятор сохраняется на протяжении всего цикла ko , а функция активации вычисляется после его завершения.

В системах программирования этим действиям планирования соответствуют определённые конструкции. Halide — система программирования, в которой отдельно описываются «что вычислять» и «как планировать вычисления»; программист изменяет способ выполнения с помощью таких действий, как `split`, `tile` и `reorder`. Оператор, создающий промежуточный результат, здесь называется производителем, а использующий этот результат оператор — потребителем. `compute_at` задаёт, внутри какого уровня циклов потребителя выполняется производитель. TVM — система компиляции тензорных вычислений, которая также задаёт место вычисления и операции чтения и записи кэша посредством расписания, позволяя соседним операторам последовательно обрабатывать один и тот же блок данных.²¹

Место вычисления одновременно определяет повторное использование и повторное вычисление. Если вычисление промежуточного результата поместить на самый внешний уровень, сначала будет полностью вычислен результат, который затем смогут счи-

²¹TVM, TensorIR.

Сначала сложить,
затем активировать

$$1 + (-1) = 0 \rightarrow \text{SiLU}(0) = 0$$

Сначала активировать
по отдельности, затем
сложить

$$\text{SiLU}(1) + \text{SiLU}(-1) \approx 0.4621$$

Рисунок 5-22. Если сначала сложить те же две частичные суммы и получить ноль, применение SiLU по-прежнему даст ноль; если сначала применить SiLU к каждой части, а затем сложить результаты, получится приблизительно 0,4621. Эти два числа показывают, что перенос вычисления функции активации перед суммированием изменяет результат.

тивать все последующие вычисления. Если поместить его внутрь блочного цикла последующего вычисления, будет вычисляться только требуемая в данный момент часть. Если одна и та же часть промежуточного результата нужна нескольким выходным блокам, её вычисление внутри каждого блока приведёт к повторам. Поэтому компилятор должен сравнивать два подхода: сохранить промежуточный результат для многократного чтения или вычислять его заново перед каждым использованием.

Операция fuse для циклов также позволяет объединить несколько измерений итерации в одно. Например, двумерные координаты 16×192 выходных блоков можно преобразовать в одномерные номера от 0 до 3071, а затем распределить их между рабочими группами ускорителя. Это изменяет порядок обхода выходных блоков и способ распределения задач; выполнение SiLU непосредственно после тайла матричного умножения, в свою очередь, устраняет запись и повторное чтение промежуточной матрицы. Оба преобразования можно использовать совместно в одном расписании.

5.4.3 Как зависимости и округление ограничивают преобразования

В предыдущем подразделе функция активации была перенесена внутрь выходного тайла, но по-прежнему выполнялась после завершения цикла ko. Причину можно показать на двух числах. Пусть две частичные суммы одного выходного значения равны 1 и -1 , а их полная сумма — 0, поэтому $\text{SiLU}(0)=0$. Если сначала применить SiLU к каждой частичной сумме, а затем сложить результаты, получится $\text{SiLU}(1) + \text{SiLU}(-1) \approx 0.4621$. Перенос вычисления функции активации перед полным суммированием уже изменяет операцию.

Таким образом, зависимости, которые должен соблюдать компилятор, существуют на двух уровнях: частичные суммы одного выходного значения должны объединяться по правилам редукции, а последующий оператор должен дожидаться вычисления требуемого результата. Разные выходные значения можно вычислять параллельно; редукция одного выходного значения и последующие операции над ним должны выполняться в порядке зависимостей.

Блочная редукция подчиняется тому же ограничению зависимостей. FlashAttention из раздела 5.3.3 может выполнять вычисления блок за блоком, поскольку сохраняет m , ℓ и u , а при изменении максимума одновременно корректирует ранее накопленную сумму экспонент и взвешенное значение. Если сохранять только уже нормализованный результат каждого блока, информация об относительных величинах знаменателей блоков будет потеряна. Например, пусть каждый из двух блоков содержит только одно значение — соответственно 1 и 3. Результаты внутри блоков по-прежнему равны 1 и 3; по этим двум результатам невозможно определить, должна ли Softmax всей строки назначить им веса в отношении 1:2 или 2:1. Поэтому блочная редукция должна сохранять достаточный набор статистик, позволяющий корректно объединить результаты блоков.

Округление чисел с плавающей точкой дополнительно ограничивает преобразования. Исходная программа округляет накопленный в FP32 результат до BF16, а затем применяет функцию активации; слитная программа также должна выполнить `round_to_bf16` до активации. Промежуточное значение можно передать функции активации непосредственно из регистра, но само округление всё равно должно произойти. Место хранения данных и числовой формат — два независимых решения.

Пример 5-7. Как коэффициент масштабирования для всей строки ограничивает порядок блочных вычислений? Строка содержит 256 элементов и разделена на два блока по 128 элементов. Первые элементы блоков равны соответственно 1 и 10, остальные — нулю. Для квантования используется формат E4M3FN: восьмибитное представление с одним знаковым битом, четырьмя битами экспоненты и тремя битами мантиссы, максимальное конечное значение которого равно 448. Коэффициент масштабирования определяется максимальным абсолютным значением во всей строке; затем выполняется скалярное произведение, причём в весах только первый элемент равен 1. Поэтому результат полностью определяется способом квантования первого элемента.

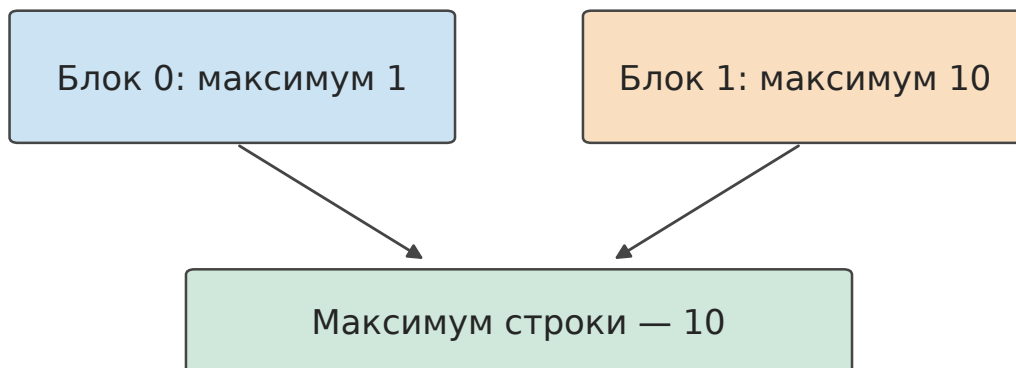
Если сначала прочитать всю строку, максимальное значение будет равно 10. Первый элемент масштабируется до $1 \times 448/10 = 44.8$, округляется до представимого в этом формате значения 44, а после деквантования получается $44 \times 10/448 = 55/56$. Если квантовать данные сразу после чтения первого блока, текущий максимум будет равен 1, первый элемент можно представить значением 448, а результат деквантования будет в точности равен 1.

После чтения значения 10 из второго блока уже невозможно заставить первый элемент повторно пройти округление $44.8 \rightarrow 44$, даже если скорректировать коэффициент масштабирования для последующих вычислений. Результаты двух подходов отличаются на $1/56$, то есть примерно на 1,8%. Поэтому, если коэффициент масштабирования определяется максимумом всей строки, сначала следует прочитать всю строку и найти максимум, а уже затем выполнять квантование.²²

Эти примеры показывают, какие преобразования допустимы, а какой порядок опе-

²² Допустимость слияния, округление FP8 и проекция квантования; изучение бэкенда RedFuser. В объёмы 444/620 MiB входят A, W и выходные данные; дополнительные операции чтения и записи построчного scale приведены в сопутствующем расчёте.

Масштаб квантования строки
определяется лишь после чтения
всей строки



Первый элемент: $1 \rightarrow 44.8 \rightarrow$ округление до 44 $\rightarrow 55/56$

Рисунок 5-23. Строка разделена на два блока, и значение 10 из второго блока определяет коэффициент масштабирования для всей строки. Первый элемент необходимо сначала масштабировать с этим коэффициентом, затем округлить до допустимого в формате значения и, наконец, деквантовать.

раций необходимо сохранять. Порядок вычисления разных выходных значений можно менять, входные данные можно кэшировать блоками, а готовые результаты можно непосредственно передавать последующим операциям; способ объединения при редукции и место округления при квантовании относятся к определению операции и должны сохраняться при преобразованиях.

5.4.4 AKG: планирование вычислений и перемещений данных с помощью `isl schedule`

AKG — разработанный Huawei компилятор с открытым исходным кодом для автоматической генерации операторов нейронных сетей, их оптимизации и генерации кода для NPU, в том числе Ascend. Автор книги участвовал в его разработке. Блочные циклы из предыдущего подраздела уже показывают, какие вычисления требуются; AKG передаёт тензорную программу полиэдральному компилятору, который автоматически организует порядок циклов, разбиение на блоки, внутрикристальные буферы, перемещения данных и целевой код. Сначала на конкретных числах рассмотрим, что даёт такая организация, а затем разберём представление `isl`.²³

Пусть $C = AB$, а все три матрицы имеют форму 4096×4096 . А и В используют BF16, то есть 2 байта на элемент; С — FP32, то есть 4 байта на элемент. Одно матричное умножение требует

²³Zhao и др., 2021, AKG: Automatic Kernel Generation for Neural Processing Units using Polyhedral Transformations.

$$F = 2 \times 4096^3 \approx 137.4 \text{ GFLOPs.}$$

Простейшая запись вычисления имеет вид:

```
for i in range(4096):
    for j in range(4096):
        acc = 0
        for k in range(4096):
            acc += A[i, k] * B[k, j]
        C[i, j] = acc
```

Если для каждого выходного элемента считывать операнды А и В из внешней памяти, объём чтения входных данных составит

$$4096^2 \times 4096 \times (2 + 2) \text{ байт} = 256 \text{ ГиБ.}$$

Эти циклы выражают математическое вычисление, но не повторное использование данных. Возьмём выходной тайл $B_M = B_N = 64$ и тайл редукции $B_K = 32$. Всего получим $64 \times 64 = 4096$ выходных тайлов; каждому из них нужны $4096/32 = 128$ тайлов редукции. Один тайл А и один тайл В занимают по

$$64 \times 32 \times 2 = 4096 \text{ байт} = 4 \text{ КиБ.}$$

Если для каждого выходного тайла считывать набор тайлов А/В вдоль измерения К, суммарный объём его входных данных равен $128 \times (4 + 4) = 1024$ КиБ. Для всех выходных тайлов объём чтения составит 4096×1024 КиБ = 4 ГиБ. Итоговая запись С требует $4096^2 \times 4 = 64$ МиБ. Таким образом, блочное расписание с удержанием выходного тайла сокращает чтение входных данных с 256 ГиБ примерно до 4 ГиБ — в 64 раза. Взамен для каждого тайла необходимо удерживать аккумулятор FP32 объёмом $64 \times 64 \times 4 = 16$ КиБ.

Соответствующая блочная программа:

```
for io in range(64):
    for jo in range(64):
        acc = zeros_fp32(64, 64)
        for ko in range(128):
            a = load_tile(A, io*64, ko*32, 64, 32)
            b = load_tile(B, ko*32, jo*64, 32, 64)
            acc += a @ b
        store_tile(C, io*64, jo*64, acc)
```

Отношений доступа в этой программе уже достаточно для определения рабочего набора тайла: область А задаётся координатами (io, ko), область В — (ko, jo), а аккумулятор acc сохраняется на протяжении 128 обновлений по ko. Далее AKG должен преобразовать эти отношения в исполнимое расписание и схему размещения данных.

5.4.4.1 Что такое isl schedule

Выполнение цикла состоит из множества **экземпляров оператора**. Оператор умножения со сложением выше обозначим $S(i, j, k)$; например, $S(17, 29, 4)$ — это экземпляр,

вычисляющий $A[17, 4] \times B[4, 29]$. Все допустимые экземпляры образуют область итераций:

$$D = \{S(i, j, k) \mid 0 \leq i < 4096, 0 \leq j < 4096, 0 \leq k < 4096\}.$$

`isl_schedule` назначает каждому экземпляру одну или несколько целочисленных координат. Их лексикографическое упорядочение определяет новый порядок выполнения. Исходные циклы можно записать так:

$$S(i, j, k) \mapsto (i, j, k).$$

После разбиения на блоки расписание имеет вид:

$$S(i, j, k) \mapsto ([i/64], [j/64], [k/32], i \bmod 64, j \bmod 64, k \bmod 32).$$

Первые три координаты — это `io`, `jo`, `ko`, а последние три — внутриблочные координаты `ii`, `jj`, `kk`. Лексикографический порядок задаёт вложенность: выбор выходного тайла 64×64 , перебор его 128 тайлов по `K`, а внутри каждого из них — перебор элементов. Отображение сохраняет экземпляры и индексы массивов, меняя лишь временные координаты выполнения. Зависимость редукции требует объединять обновления вдоль `K` для каждой пары (i, j) в допустимом порядке.

«Аффинное» означает, что координаты расписания выражаются линейными комбинациями переменных циклов, например $2*i+j$; частное и остаток при разбиении на блоки относятся к квазиаффинным выражениям. `isl` использует целочисленные множества и отображения для представления границ, обращений к данным и зависимостей, после чего ищет расписание, удовлетворяющее ограничениям зависимостей. Это не таблица производительности, а математическое отображение, назначающее координаты каждому вычислению.

Дерево расписания организует это отображение в виде дерева. Для рассматриваемого GEMM его можно прочитать так:

```
Domain {S(i,j,k)}
└─ Band {io, jo, ko}
   └─ Band {ii, jj, kk}
```

`Domain` перечисляет существующие экземпляры; `Band` представляет группу измерений циклов, которые совместно преобразуются и упорядочиваются. В подграфах АКГ с несколькими операторами также используются `Sequence` для задания последовательности, `Filter` для выбора множества экземпляров и `Extension` для включения экземпляров производителя в тайл потребителя. Дерево расписания можно понимать как структурированное представление вложенности циклов и последовательности операторов, а аффинное отображение — как координаты каждого экземпляра в этом дереве.

5.4.4.2 От расписания к данным на кристалле

То же расписание можно непосредственно записать в виде отображений перемещения данных:

$$\begin{aligned}
A_{L1}[io, ko, ii, kk] &\leftarrow A[64\ io + ii, 32\ ko + kk], \\
B_{L1}[ko, jo, kk, jj] &\leftarrow B[32\ ko + kk, 64\ jo + jj], \\
C_{L0}[ii, jj] &\leftarrow \sum_{kk} A_{L0}[ii, kk] B_{L0}[kk, jj].
\end{aligned}$$

Слева записаны логические координаты внутрикристальных тайлов, справа — адреса в глобальных матрицах. Для A_{L1} и B_{L1} требуется по 4 КиБ, для C_{L0} — 16 КиБ; при двойной буферизации входные слоты занимают по 8 КиБ. По этим размерам компилятор проверяет вместимость буферов, вставляет перемещения и определяет точки повторного использования данных.

В расписании в стиле Halide/TVM те же решения можно записать так:

```
# compute: математические отношения
C[i, j] = sum(k, A[i, k] * B[k, j])

# schedule: координаты циклов и размещение данных
io, ii = split(i, factor=64)
jo, jj = split(j, factor=64)
ko, kk = split(k, factor=32)
reorder(io, jo, ko, ii, jj, kk)

A_L1 = cache_read(A, "L1")
B_L1 = cache_read(B, "L1")
C_L0 = cache_write(C, "L0C")
compute_at(A_L1, ko)
compute_at(B_L1, ko)
compute_at(C_L0, jo)
```

`split` создаёт координаты частного и остатка, `reorder` размещает циклы тайлов снаружи внутриблочных циклов, `compute_at` указывает, на каком уровне цикла потребителя формируется результат производителя, а `cache_read/cache_write` задают размещение во внутрикристальной памяти. Полиэдральное планирование AKG, описание аппаратных ресурсов и управление памятью автоматически преобразуют эти отношения, после чего генерируется код для MTE, Cube, Vector и механизмов синхронизации.

5.4.5 TileLang-Ascend: запись того же расписания в блочной программе

TileLang создан преимущественно совместными усилиями исследователей Пекинского университета и Microsoft Research. Он отделяет поток данных от пространства расписаний: `T.soru`, `T.gemm`, `T.reduce` выражают поток данных на уровне тайлов, а `T.Pipelined`, аннотации раскладки, привязка потоков и перестановки данных — расписание. Проект TileKernels компании DeepSeek реализует операторы обучения и инференса на TileLang; 30 сентября 2026 года в нём появился бэкэнд Huawei Ascend, позволяющий использовать один набор интерфейсов Python для NVIDIA GPU и Huawei Ascend NPU. Отдельный проект `tilelang-ascend` предоставляет варианты адаптации через Ascend C/PTO и

AscendNPU IR.²⁴²⁵²⁶

Для того же $C = AB$ поток данных можно выразить следующим псевдокодом в стиле Ascend:

```
with T.Kernel(core_num, is_npu=True) as (cid, _):
    A_L1 = T.alloc_L1((stages, BM, K_L1), "float16")
    B_L1 = T.alloc_L1((stages, K_L1, BN), "float16")
    A_L0 = T.alloc_L0A((stages, BM, BK), "float16")
    B_L0 = T.alloc_L0B((stages, BK, BN), "float16")
    C_L0 = T.alloc_L0C((BM, BN), "float32")

    for ko1 in T.Pipelined(ceil_div(K, K_L1), num_stages=stages):
        T.copy(A[...], A_L1[ko1 % stages, :, :])
        T.copy(B[...], B_L1[ko1 % stages, :, :])
        for ko0 in T.serial(ceil_div(K_L1, BK)):
            T.copy(A_L1[ko1 % stages, ...], A_L0[ko0 % stages, ...])
            T.copy(B_L1[ko1 % stages, ...], B_L0[ko0 % stages, ...])
            T.mma(A_L0[ko0 % stages, ...],
                  B_L0[ko0 % stages, ...], C_L0,
                  init=(ko1 == 0 and ko0 == 0))

    T.copy(C_L0, C[...])
```

Этот псевдокод и расписание isl выше — два представления одного вычисления. $ko1$ и $ko0$ соответствуют двум уровням разбиения по K ; A_L1/B_L1 — внешним входным тайлам, A_L0/B_L0 — входным тайлам Cube, а C_L0 удерживает частичную сумму на протяжении циклов редукции. Слоты `stages` позволяют перекрывать следующее перемещение с текущей операцией `mma`. Для завершённого блочного расписания также нужны отображение номера ядра в (io, jo) , обработка краевых блоков, раскладки, `swizzle` и флаги синхронизации.

Обе системы выражают аппаратный поток данных. AKG главным образом опирается на isl, описание аппаратных ресурсов и управление памятью для автоматического формирования расписания и перемещений; TileLang позволяет программисту явно записывать больше решений о расписании в блочной программе. В таблице сопоставлены представления одного GEMM в двух системах.

Одно решение	Представление в isl/AKG	Представление в TileLang-Ascend
Выходной тайл	$\text{floor}(i/BM)$, $\text{floor}(j/BN)$ в аффинном расписании	Координаты тайлов и циклы в <code>T.Kernel</code>
Разбиение по K	$\text{floor}(k/BK)$ и внутриблочная координата	Два уровня циклов: $ko1$, $ko0$
Входная область	Выводится из отношений доступа к массивам	Исходный срез и целевой буфер в <code>T.copy</code>
Внутрикристальная память	Управление памятью переносит данные в <code>UB/L1/L0</code>	Объявления <code>alloc_L1/L0A/L0B/L0C</code>

²⁴Wang и др., TileLang: A Composable Tiled Programming Model for AI Systems, 2025.

²⁵DeepSeek TileKernels, README, прочитано 30 сентября 2026 года.

²⁶TileLang-Ascend, адаптеры Ascend C/PTO и AscendNPU IR, README, прочитано 30 сентября 2026 года.

Одно решение	Представление в isl/AKG	Представление в TileLang-Ascend
Матричные вычисления	Генератор кода отображает их на инструкции Cube	T.mma выражает операцию Cube
Конвейер и синхронизация	Формируются по зависимостям, ёмкости и бэкэнду	T.Pipelined, флаги или автоматическая синхронизация

5.4.5.1 Слияние и повторное использование данных: пример 5-8

Редакционное примечание русского издания. Следующий пример сохранён из предыдущей редакции книги: в обновлённом оригинале он отсутствует, хотя следующий подраздел по-прежнему на него ссылается.

Совместный анализ вычислений и памяти также объясняет, почему слияние иногда увеличивает объём обращений к памяти. В рассмотренной ранее цепочке поэлементных операций слияние сократило чтение промежуточных значений; однако если промежуточный результат представлен меньшим числом битов и многократно считывается последующими вычислениями, его сохранение, напротив, может уменьшить объём чтения.

Пример 5-8. Что требует меньше операций чтения и записи: отдельное квантование или его слияние с матричным умножением? Возьмём вход A в формате FP16 с формой [4096, 4096], занимающий 32 МиБ; после квантования в FP8 он занимает 16 МиБ. Веса FP8 с формой [4096, 1536] занимают 6 МиБ, а выход FP16 — 12 МиБ. Выход разбивается на блоки 128×128: всего 32 блока строк и 12 блоков столбцов. Каждый выходной блок независимо считывает необходимые данные, а коэффициент масштабирования определяется максимумом всей строки.

Сначала рассмотрим отдельное выполнение квантования. При обработке каждой строки для её входных данных выделяется 8 КиБ локальной памяти, после нахождения коэффициента масштабирования выполняется квантование. Весь вход считывается в объёме 32 МиБ, а результаты квантования записываются в объёме 16 МиБ. Затем каждый из 12 выходных блоков столбцов один раз считывает вход FP8, что в сумме составляет 192 МиБ. Таким образом, объём чтения и записи, связанный со входом, равен $32 + 16 + 192 = 240$ МиБ.

Если объединить квантование с матричным умножением, сначала исходный вход объёмом 32 МиБ считывается для определения коэффициента масштабирования каждой строки; затем каждый из 12 блоков столбцов считывает те же 32 МиБ исходного входа и выполняет локальное квантование. Связанный со входом объём обращений равен $32 + 12 \times 32 = 416$ МиБ. Хотя запись 16 МиБ результатов квантования устраняется, объём входа, считываемого каждым блоком столбцов, возрастает с 16 до 32 МиБ, поэтому 12 чтений добавляют 192 МиБ.

В обоих вариантах каждый из 32 блоков строк считывает 6 МиБ весов, после чего записывается выход объёмом 12 МиБ: всего $32 \times 6 + 12 = 204$ МиБ. Поэтому общий объём обращений составляет соответственно $240 + 204 = 444$ МиБ и $416 + 204 = 620$ МиБ, то есть после слияния он возрастает примерно на 40%.

На рисунке 5-24 различие определяется повторным чтением данных для 12 блоков столбцов: объём данных при каждом чтении отличается вдвое. Поэтому число выходных блоков столбцов непосредственно определяет разницу в объёме обращений к памяти между двумя вариантами. При добавлении каждого блока столбцов вариант с сохране-

Хранить промежуточный результат низкой разрядности

Чтение входа 32 + запись квант. 16 + повторное чтение 192

Для входа: 240 MiB

Повторное квантование для каждого блока столбцов

Чтение входа для scale 32 + повторное чтение входа 384

Для входа: 416 MiB

Рисунок 5-24. В обоих вариантах сначала считывается весь вход для определения коэффициента масштабирования каждой строки. После сохранения результата FP8 суммарное повторное чтение 12 блоками столбцов составляет 192 МиБ; в слитном варианте повторно считывается 384 МиБ входа FP16. На веса и выход в обоих случаях дополнительно приходится по 204 МиБ.

нием результата квантования считывает дополнительно 16 МиБ, а слитный вариант — 32 МиБ. Если поместить все 1536 выходных столбцов в один блок, такое повторное чтение между блоками столбцов будет устранено; однако соответствующий выходной аккумулятор расширится со 128 до 1536 столбцов и потребует в 12 раз больше памяти. Поэтому при выборе, следует ли объединять квантование с матричным умножением, необходимо по-прежнему сравнивать использование локальной памяти и повторное использование данных, рассмотренные в разделе 5.2.²⁷

AKG совместно рассматривает слияние и разбиение на блоки именно для учёта таких взаимосвязей. При изменении места вычисления промежуточного результата меняются данные, считываемые каждым выходным блоком, и число их чтений; вслед за этим изменяются требования к буферам, что влияет на допустимые размеры блоков.

5.4.6 Оценка стоимости и выбор по результатам измерений

В примере 5-8 две реализации уже можно сравнить по объёму чтения и записи. При большем количестве комбинаций разбиения и слияния компилятор таким же образом постепенно сужает область поиска: сначала проверяет зависимости и численные правила, затем исключает реализации, которым недостаточно локальной памяти, и, наконец, оценивает затраты на выполнение, определяя, какие реализации следует тестировать в первую очередь. В разделе 5.2 уже был приведён один способ оценки: сравнение двух тайлов по объёму обращений и эффективной пропускной способности. Компилятор может также учесть степень использования матричных инструкций, требования к регистрам и затраты на запуск, чтобы определить порядок тестирования реализаций.

Для матрицы из раздела 5.2 и общей памяти RTX PRO 6000 процесс выбора можно представить следующей таблицей. В указанном разделе входные буферы и аккумуляторы размещались в 100 КиБ общей памяти каждого SM; если перенести аккумуляторы в регистры, как в разделе 5.1.5, необходимо отдельно проверить общую память и регистры, поскольку ёмкость одного типа ресурсов нельзя использовать вместо другого. Если перевести двух кандидатов на двойную буферизацию, объём входных буферов удвоится, рабочие наборы составят соответственно 32 КиБ и 96 КиБ, а число одновременно размещаемых на каждом SM блоков потоков изменится с 4 и 1 на 3 и 1.

Этап выбора	Данные для матричного примера этой главы	Основание для исключения или сравнения
Зафиксировать цель	Форма входа, точность, ошибка, частота вызовов	Сравнение одинаковой математической работы и одинаковой нагрузки
Перечислить кандидатов	Тайл, порядок циклов, число буферов, область слияния	Поддержка компилятором и оборудованием
Проверить выполнимость	Общая память, регистры, потоки, выравнивание и зависимости	Отчёт о ресурсах компиляции и проверка корректности
Оценить время	Объём вычислений, фактический объём чтения и записи, запуск и преобразование раскладки	Время обслуживания каждым ресурсом и критический путь

²⁷ Допустимость слияния, округление FP8 и проекция квантования; изучение бэкенда RedFuser. В объёмы 444/620 MiB входят A, W и выходные данные; дополнительные операции чтения и записи построчного scale приведены в сопутствующем расчёте.

Этап выбора	Данные для матричного примера этой главы	Основание для исключения или сравнения
Измерить и выбрать	Время и разброс для репрезентативных форм, а также полная цепочка операторов	Минимизация стоимости рабочей нагрузки с сохранением близких по результату кандидатов

Сначала по отчёту о ресурсах исключаются кандидаты, которым недостаточно доступной ёмкости, а затем для оставшихся измеряются эффективная пропускная способность, время вычислений и время полной цепочки операторов. Если разлив регистров, дополнение краёв матрицы или снижение параллелизма сводят на нет выигрыш от повторного использования, следует вернуться к таблице кандидатов и изменить форму блока либо число буферов. В главе 6 используется тот же процесс, но размещение тайлов распространяется на несколько карт, а межблочные зависимости преобразуются в коллективные коммуникации — операции, при которых несколько карт обмениваются данными или агрегируют их по фиксированной схеме.

Этап измерений можно проиллюстрировать программой транспонирования матриц из сопутствующего эксперимента 5-5: одна и та же программа на RTX PRO 6000 Blackwell выполняется приблизительно за 7,9 мкс с блоками 16×16 и за 2,9 мкс с блоками 32×32 , что даёт ускорение примерно в 2,7 раза. При удвоении каждого измерения блока число элементов внутри него увеличивается в четыре раза, а вместе с ним изменяются раскладка памяти и взаимодействие потоков.²⁸

Автоматическая настройка TVM ищет варианты расписания, включая разбиение на блоки и порядок циклов, по результатам измерений на оборудовании. AI-агент, способный вызывать инструменты компиляции и тестирования, может изменять расписание или код ядра и тем самым пробовать реализации за пределами исходных шаблонов расписания. В обоих случаях объектом оценки служит скомпилированная программа: корректные реализации отбираются с помощью численной проверки, а расписания оцениваются по времени выполнения. Оценка проводится именно для скомпилированной программы, поскольку компилятор может объединить операторы, отдельно представленные в исходном коде, в одно ядро.²⁹ Метод поиска должен решить ещё одну задачу: на какой нагрузке оценивать эти реализации.

Пример 5-9. Как соотношение вызовов с разными формами входа влияет на выбор реализации оператора? У оператора есть две формы входа, A и B, каждую из которых исходная реализация обрабатывает за 10 мкс. Новая реализация обрабатывает A за 5 мкс, а B — за 20 мкс: время для A уменьшается вдвое, а для B удваивается. Если каждая форма встречается один раз, суммарное время увеличивается с 20 до 25 мкс, то есть новая реализация работает на 25% дольше. Если A вызывается 100 раз, а B — один раз, суммарное время уменьшается с 1010 до 520 мкс, то есть новая реализация даёт ускорение примерно в 1,9 раза.

Пусть доля вызовов с формой входа A равна p . Среднее время выполнения исходной реализации равно 10 мкс, а новой:

$$\overline{T}_{\text{new}} = 5p + 20(1 - p) = 20 - 15p \quad \mu\text{s}.$$

²⁸Сгенерированный код и сравнение транспонирования в эксперименте 5-5.

²⁹Различия в компиляции, округлении и выполнении в эксперименте 5-4, подцепочка экспертов DeepSeek V4-Flash.

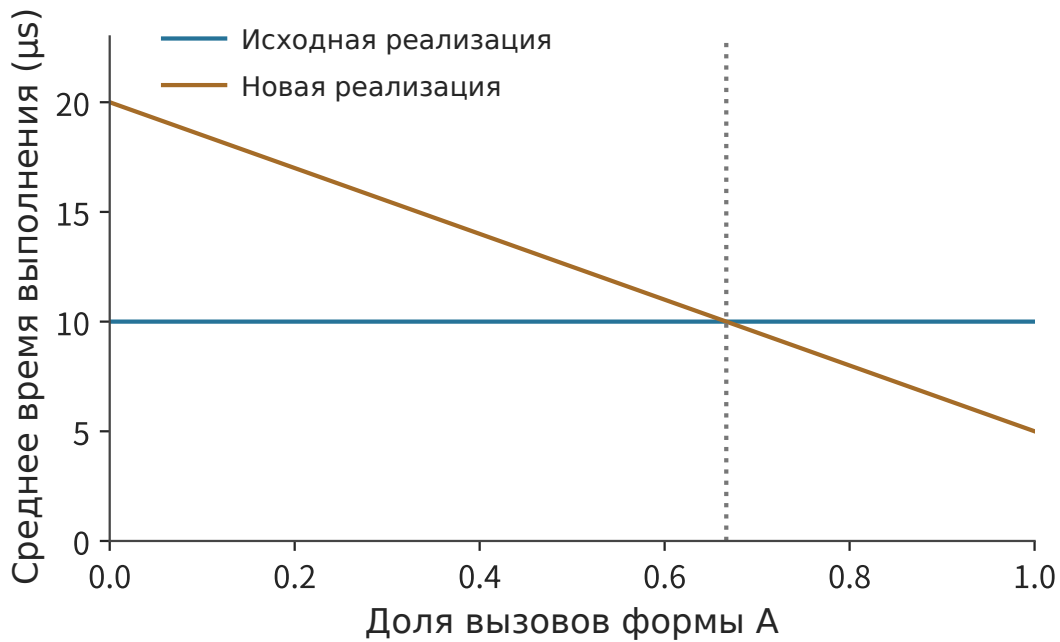


Рисунок 5-25. Когда доля формы А превышает $2/3$, суммарное время выполнения новой реализации становится меньше. Исходное время для А и В равно 10 мкс, а в новой реализации — соответственно 5 и 20 мкс; вызовы выполняются последовательно, и на рисунке сравнивается установившийся режим выполнения. Точка пересечения определяется равенством среднего времени двух реализаций.

Чтобы среднее время новой реализации было меньше, должно выполняться $20 - 15p < 10$, то есть $p > 2/3$. Точка пересечения на рисунке 5-25 показывает, что новая реализация становится быстрее, только когда доля А превышает $2/3$ всех вызовов.

Таким образом, настройка может выбирать отдельную реализацию для каждой формы либо одну реализацию с меньшей совокупной стоимостью с учётом фактической частоты вызовов. Затраченное самим поиском время компиляции и измерений относится к подготовительным затратам. В сопутствующем эксперименте с агентом сохраняются результаты генерации кода, повторных отправок и измерений; в следующем разделе с учётом числа вызовов будет рассчитано, сколько времени потребуется, чтобы экономия времени выполнения окупил затраты на поиск.³⁰

5.5 Среда выполнения: отправка, повторное воспроизведение и динамические формы

После того как компилятор выбрал программу для ускорителя, среда выполнения должна многократно подготавливать входные данные, выбирать программу, отправлять задачи и управлять буферами. Даже при использовании одного и того же кода kernel изменение способа отправки или повторное использование уже полученных результатов подготовки влияет на общее время. В этом разделе мы продолжим рассмотрение временной шкалы из раздела 5.1 и проанализируем эту работу хоста.

³⁰ Журнал поиска и измерений эксперимента 5-6; расчёты для оценки и развёртывания. Зафиксированы два раунда, всего 12 вызовов агента; новых более быстрых реализаций получено не было.

5.5.1 Перекрытие подготовки на хосте и вычислений на ускорителе

В разделе 5.1 время отправки с CPU было отделено от времени выполнения на ускорителе, а в разделе 5.3 было показано, что ресурсы этих двух типов могут работать параллельно. Применим оба положения к одному прямому проходу модели: CPU проверяет формы и подготавливает параметры или метаданные для следующего этапа, пока GPU выполняет текущий этап. Если подготовку следующего этапа можно начать заранее, обе стороны образуют конвейер; если CPU каждый раз начинает подготовку только после завершения работы GPU, ускоритель будет останавливаться между этапами.

Пример 5-10. Почему после ускорения ускорителя узким местом становится хост? Предположим, работа состоит из 100 сегментов, а подготовка на хосте и вычисления на ускорителе для каждого сегмента занимают по 20 μs . Все данные, необходимые для подготовки следующего сегмента, уже известны, буферов достаточно, ресурсы двух типов независимы. При строго последовательном выполнении потребуется $100 \times (20 + 20) = 4000 \mu\text{s}$. При использовании конвейера сначала завершается подготовка первого сегмента, после чего каждые 20 μs завершается один сегмент, поэтому общее время составляет $20 + 100 \times 20 = 2020 \mu\text{s}$.

Сократим вычисления на ускорителе до 5 μs , сохранив время подготовки одного сегмента на хосте равным 20 μs . Время конвейера станет равно $100 \times 20 + 5 = 2005 \mu\text{s}$, то есть сократится лишь на 15 μs . Скорость отправки задач с CPU не увеличилась: чем быстрее считает GPU, тем дольше он ждёт следующую задачу. Только после сокращения времени подготовки на хосте до 5 μs обе стороны смогут завершать по сегменту каждые 5 μs , а общее время снизится до $5 + 100 \times 5 = 505 \mu\text{s}$.³¹

На рисунке 5-26 показаны временные шкалы первых четырёх сегментов в трёх случаях.

Этот пример показывает два направления оптимизации хоста. Первое — сократить объём подготовки, например повторно использовать уже определённые формы и планы выполнения (заранее определённые с учётом длины запроса, размещения кэша и других условий выбор kernel, разбиение задач и организация рабочей области). Второе — перенести подготовку на время выполнения предыдущего сегмента на ускорителе, например асинхронно обрабатывать результаты предыдущего шага. Эти подходы соответственно уменьшают объём работы в конвейере и промежутки между этапами.

При авторегрессионной генерации возможность заблаговременной подготовки зависит от того, получены ли уже необходимые данные. Для вычислений на следующем шаге требуется токен, выбранный на предыдущем шаге; формирование batch, обновление состояния грамматики — отслеживание того, до какого места грамматики дошло соответствие уже сгенерированного содержимого при ограничении формата вывода — и проверка условия остановки также требуют соответствующих результатов. Среда выполнения может заранее выполнить работу, не зависящую от этих результатов, сократив ожидание между двумя вычислительными шагами. Развитие асинхронного выполнения и планирования в vLLM и других движках направлено на решение этой задачи.³²

Когда специализированный ускоритель сокращает выполнение модели до времени, значительно меньшего, чем подготовка каждого шага на хосте, нижняя граница интервала между шагами устойчивого конвейера начинает определяться подготовкой на хо-

³¹Исследование временной шкалы на хосте; конвейер конфигурации и выполнение графа.

³²Способ выполнения и обратная связь, эволюция фреймворка и разделы 7–8 исследования стоимости токенов.

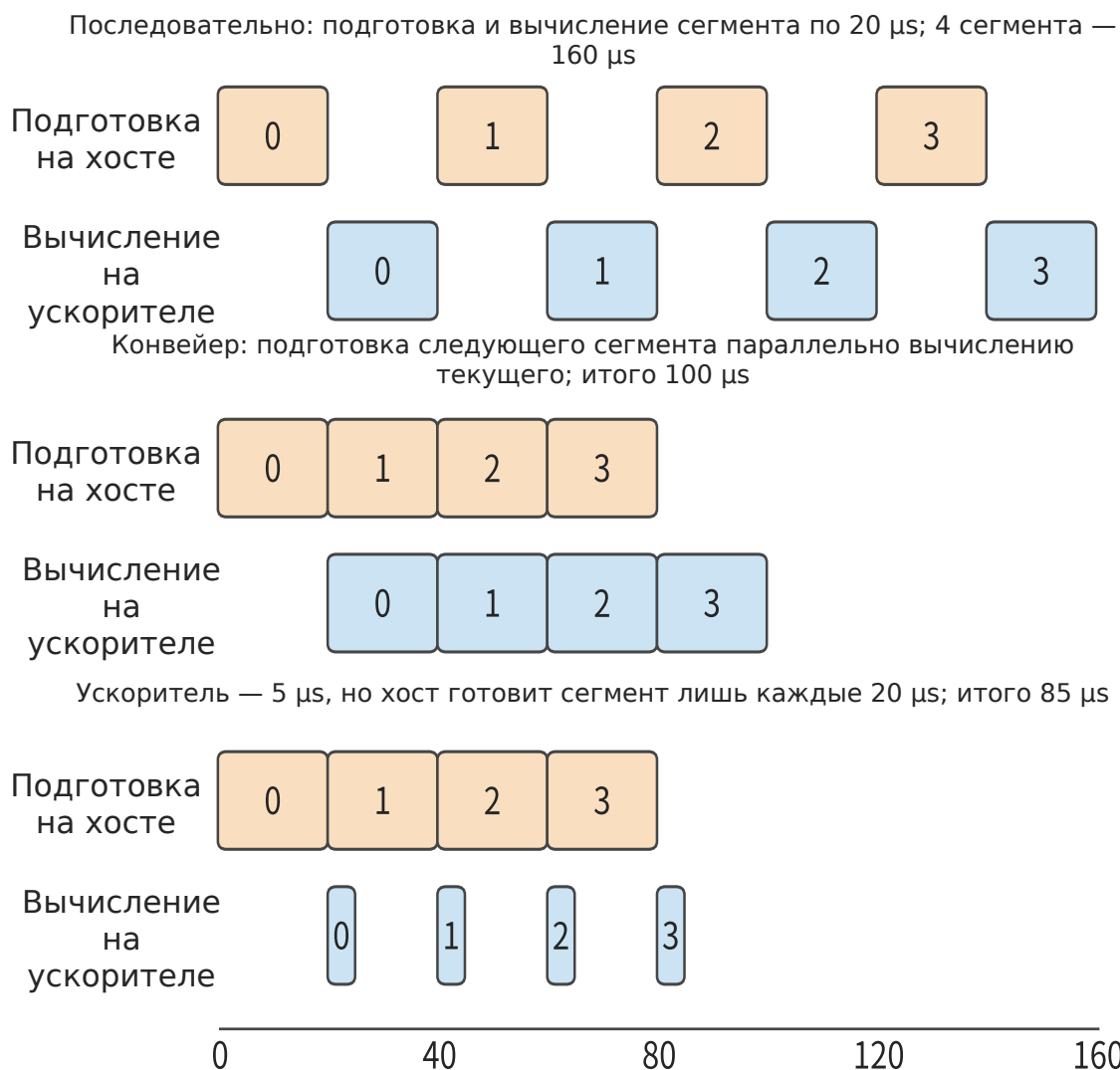


Рисунок 5-26. Подготовка каждого сегмента на хосте занимает 20 μ s. При последовательном выполнении ресурсы двух типов работают по очереди; в конвейере хост подготавливает следующий сегмент, пока ускоритель вычисляет текущий, поэтому каждые 20 μ s завершается один сегмент; после ускорения ускорителя до 5 μ s каждому сегменту по-прежнему приходится ждать подготовки на хосте, поэтому большую часть времени ускоритель простаивает. На рисунке показаны только первые четыре сегмента, а в примере в тексте их 100.

сте. Чтобы дополнительно сократить этот интервал, необходимо совместно оптимизировать подготовку динамического batch — batch, в который на каждом шаге могут добавляться запросы или удаляться из него, — обновление адресов и отправку задач.

Выпущенная в сентябре 2024 года версия vLLM v0.6.0 служит примером ситуации, когда затраты хоста занимали основную часть интервала между шагами. Профилирование перед выпуском показало, что при запуске Llama 3 8B на одной H100 в пределах одного шага API-сервер занимал 33%, планирование — 29%, а выполнение на ускорителе — лишь 38%: API-сервер и движок инференса работали в одном процессе Python, их рутины конкурировали за GIL (глобальную блокировку интерпретатора), а ускорителю оставалось лишь простаивать в ожидании хоста. В v0.6.0 API-сервер перенесли в отдельный процесс, организовав обмен с движком через очередь сообщений, что позволило избежать конкуренции за GIL. Кроме того, было введено многошаговое планирование: после одного планирования последовательно выполнялось несколько шагов без повторной подготовки входных данных. Наконец, обработку вывода сделали асинхронной и перекрыли с выполнением следующего шага. В совокупности эти три изменения повысили пропускную способность Llama 3 8B в 2,7 раза относительно v0.5.3, а 70B — в 1,8 раза.³³ Эти изменения не затронули ни одного kernel и не потребовали замены оборудования. Согласно критерию из раздела 1.3.4, когда накладные расходы программных абстракций на стороне хоста достигают 60% интервала между шагами, их устранение оказывается гораздо эффективнее замены ускорителя на более быстрый.

Подготовительную работу можно повторно использовать не только между соседними шагами одного запроса, но и между разными вызовами. Два вызова могут соответственно требовать от модели объяснить код и проверить тесты; содержимое входных данных различается, но они всё равно проходят через один и тот же набор слоёв модели. Если длина и batch находятся в диапазоне, поддерживаемом существующим планом выполнения, среда выполнения может повторно использовать программу, обновив токены, индексы позиций и адреса состояния; при появлении новой формы или пути управления она выбирает другой план выполнения. Приложение выражает изменения через контекст, а среда выполнения использует устойчивую структуру выполнения, чтобы сократить повторную подготовку. Далее отдельно рассчитаем пользу двух способов такого повторного использования: повторного воспроизведения графа и специализации по форме.

5.5.2 CUDA Graph: выигрыш при отправке и затраты на копирование входных данных

В предыдущем подразделе после ускорения GPU всё равно ожидал хост. Когда модель многократно выполняет похожую последовательность задач, значительную часть работы по отправке можно использовать повторно: сначала записать задачи ускорителя и зависимости между ними в граф, а затем многократно отправлять его на выполнение. При использовании CUDA Graph хосту достаточно каждый раз инициировать одно повторное воспроизведение графа, не подготавливая и не отправляя заново каждый kernel по отдельности. Ускоритель по-прежнему выполняет все узлы графа: сокращается именно

³³ Архив объявления о выпуске vLLM v0.6.0; разбор атрибуции прироста см. в разделе 8.2 исследования стоимости токенов. Указанная в объявлении пропускная способность измерена при одновременном поступлении запросов и с параметром `--num-scheduler-steps 10`; ускорение в 2.7 раза представляет собой совокупный эффект трёх изменений и не может быть полностью отнесено на счёт GIL. При низкой нагрузке многошаговое планирование может увеличить ожидание первого токена.

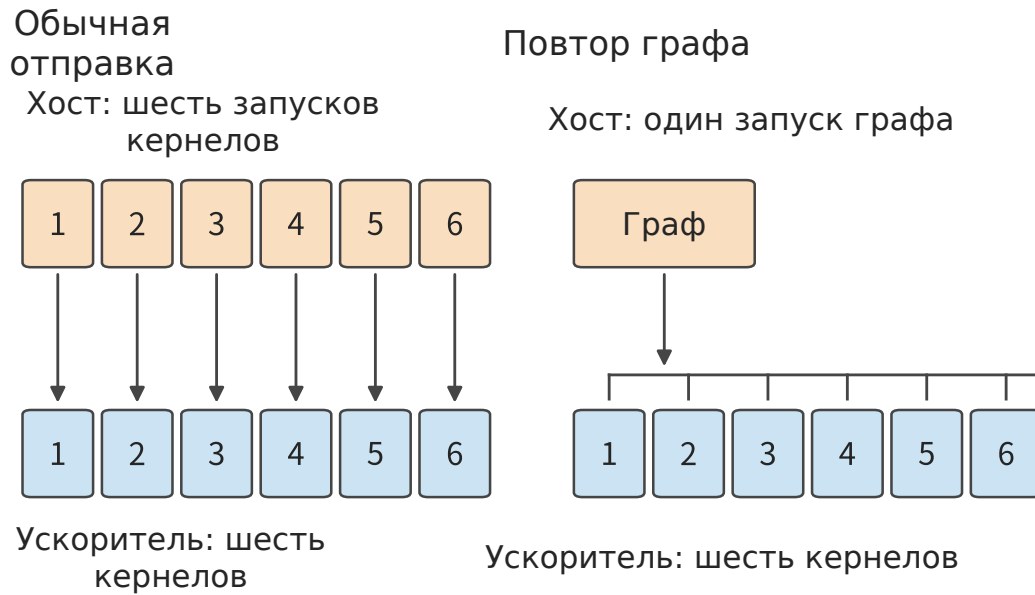


Рисунок 5-27. При обычной отправке хост выполняет отдельный launch для каждого kernel; при повторном воспроизведении графа хост выполняет только один graph launch, а ускоритель по-прежнему исполняет шесть записанных в графе kernel. Количество kernel сокращается посредством слияния, а не повторного воспроизведения графа.

повторяющаяся работа хоста. На рисунке 5-27 различие между двумя способами отправки показано на примере шести kernel одного FFN.

На рисунке 5-28 для сравнения взяты три выполнения FFN. При обычной отправке выполняются 18 операций kernel launch на хосте и 18 kernel на ускорителе; при повторном воспроизведении графа вместо них выполняются три операции graph launch, но ускоритель по-прежнему исполняет 18 kernel. После слияния цепочки активаций число kernel на ускорителе уменьшается до 15; если затем применить повторное воспроизведение графа, хосту по-прежнему потребуются лишь три операции graph launch. Слияние объединяет вычисления нескольких операторов в одном kernel, а повторное воспроизведение графа сокращает работу хоста по подготовке и отправке отдельных задач.³⁴

Перед повторным воспроизведением графа новые входные данные необходимо разместить по записанным в нём адресам. В типичной реализации для графа резервируется фиксированный буфер, в который вышестоящий оператор при каждом выполнении записывает новые данные, а граф считывает их по записанным адресам. Если результат вышестоящего оператора находится в другом тензоре, перед выполнением графа добавляется копирование; если вышестоящий оператор записывает данные непосредственно в фиксированный буфер, это копирование можно исключить.

Пример 5-11. Когда копирование входных данных сводит на нет выигрыш от повторного воспроизведения графа? При обычном выполнении вычисления на ускорителе занимают 20 μs , а ожидание ускорителем отправки задач с хоста — ещё 20 μs , итого 40 μs . Повторное воспроизведение графа и подготовка метаданных занимают 5 μs , а вычисления на ускорителе по-прежнему требуют 20 μs . Таким образом, граф изначально позволяет сэкономить 15 μs .

Пусть необходимые для выполнения графа входные данные BF16 имеют форму

³⁴Эксперимент 5-8 и JSON с анализом трассировки.

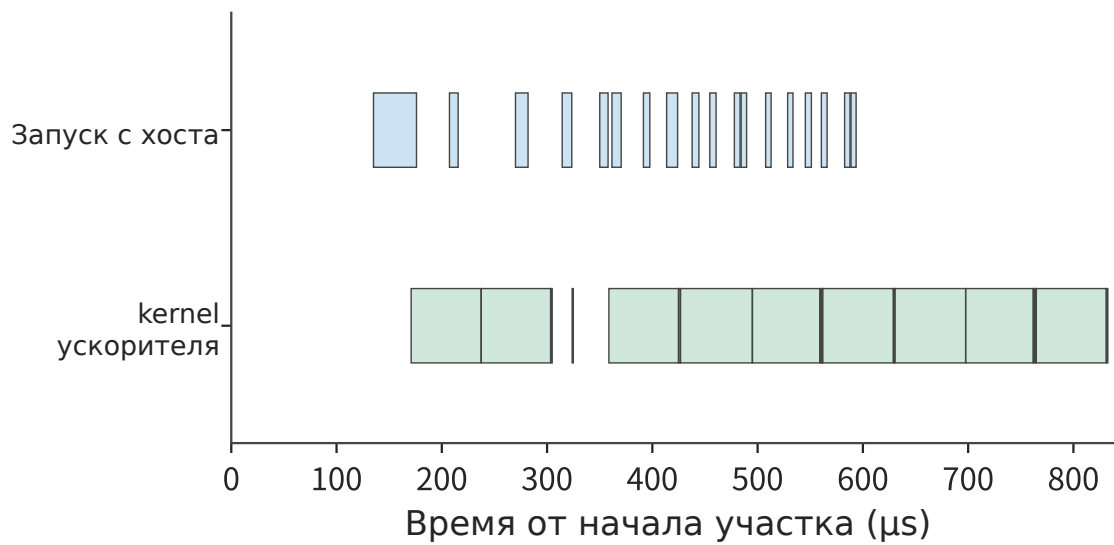


Рисунок 5-28. Измерения трёх выполнений FFN при обычной отправке. В верхней строке показаны вызовы API kernel launch на хосте, в нижней — kernel на ускорителе; время по горизонтальной оси отсчитывается от отмеченной начальной точки этого фрагмента. 18 операций kernel launch соответствуют 18 kernel на ускорителе.

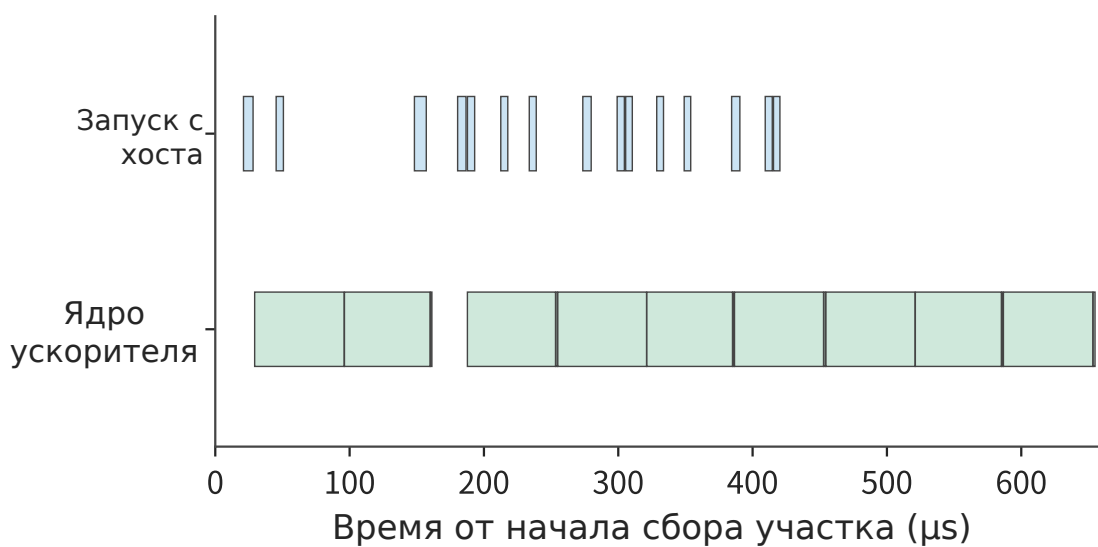


Рисунок 5-29. После слияния цепочек активаций в трёх выполнениях FFN хост выполняет 15 операций kernel launch, а ускоритель — соответствующие 15 kernel. Данные получены в отдельно отмеченном интервале сбора из того же эксперимента, что и на предыдущем рисунке.

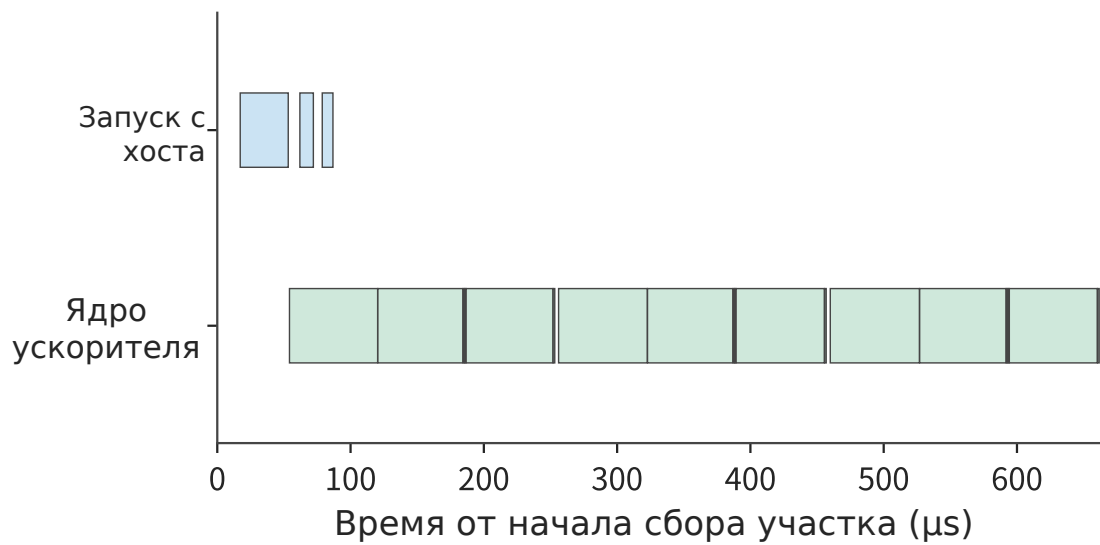


Рисунок 5-30. Три операции graph launch соответствуют 18 kernel на ускорителе. Повторное воспроизведение графа сокращает число отправок с хоста, но ускоритель по-прежнему выполняет все узлы исходного графа; на рисунке показано реальное время в пределах собственного интервала сбора.

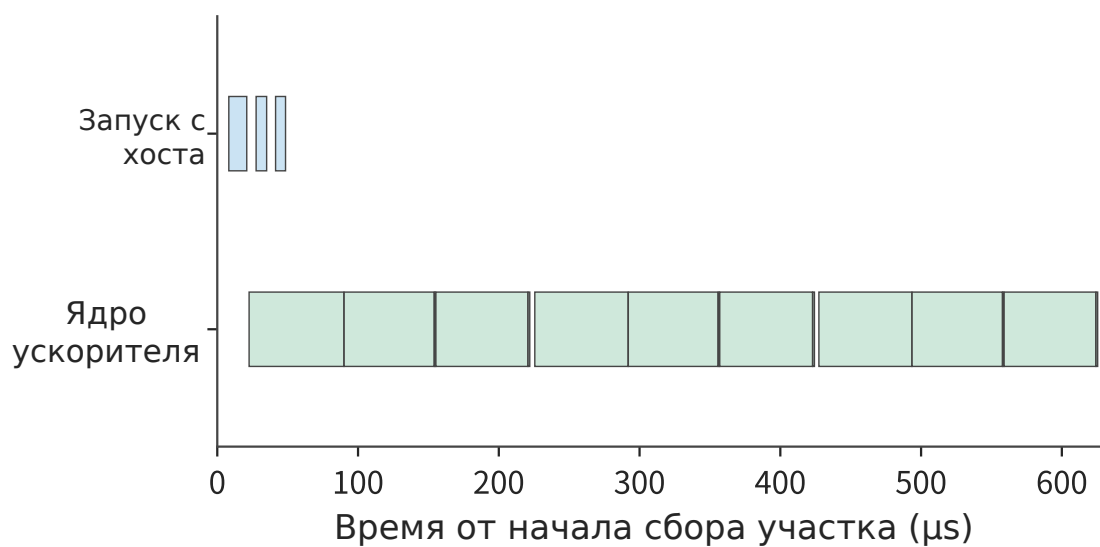


Рисунок 5-31. После слияния и последующего повторного воспроизведения хост трижды запускает граф, а ускоритель выполняет 15 kernel. Слияние сокращает число kernel на ускорителе, а повторное воспроизведение графа — число отправок с хоста.

При повторе граф читает вход по
записанному адресу

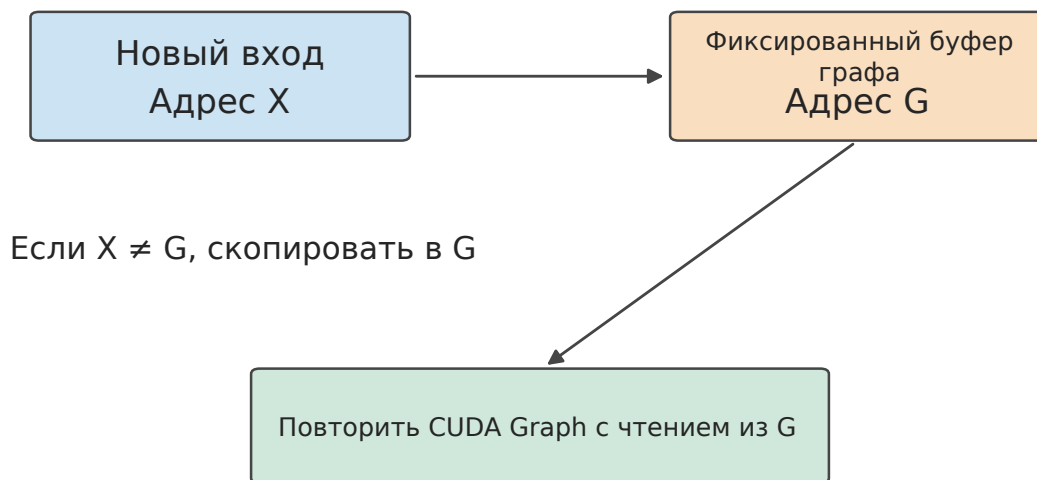


Рисунок 5-32. При выполнении граф считывает данные по записанному адресу G. Если новые входные данные находятся в X, они сначала копируются в G; если вышестоящий оператор записывает данные непосредственно в G, используется тот же буфер и промежуточное копирование не требуется.

[256, 4096] и объём 2 MiB. При копировании одновременно выполняются чтение из источника и запись в назначение, поэтому суммарный объём чтения и записи составляет 4 MiB. При пропускной способности видеопамати RTX PRO 6000, равной 1792 GB/s, которая совместно используется для чтения и записи, копирование занимает около 2,3 μ s, а общее время выполнения графа — около 27,3 μ s, то есть примерно на 12,7 μ s меньше, чем при обычном выполнении.

Сохраним время вычисления и подготовки, но увеличим входные данные до 2048 строк, то есть до 16 MiB. Суммарный объём чтения и записи при копировании возрастёт до 32 MiB, а время — примерно до 18,7 μ s; общее время выполнения графа составит около 43,7 μ s. Теперь копирование занимает больше изначально сэкономленных 15 μ s.

На рисунке 5-33 сэкономленное благодаря повторному воспроизведению графа время подготовки представляет собой отрезок фиксированной длины, а полоса времени копирования удлиняется с ростом входных данных. Когда их длины сравниваются, повторное воспроизведение графа перестаёт экономить время. Обозначим объём входных данных через X, а пропускную способность копирования — через B. Граф выполняется быстрее при условии $2X/B < 15 \mu$ s. Подставив B=1792 GB/s, получим:

$$X < \frac{15 \times 10^{-6} \times 1.792 \times 10^{12}}{2} \text{ bytes} \approx 12.8 \text{ MiB.}$$

Объём 2 MiB ниже этого порога, а 16 MiB — выше. Затраты на копирование можно сократить, если предыдущий оператор будет записывать результат непосредственно в буфер графа либо если начать захват с места, где входной тензор меньше. Поэтому при выборе операций, включаемых в один граф, необходимо сопоставлять сэкономленное

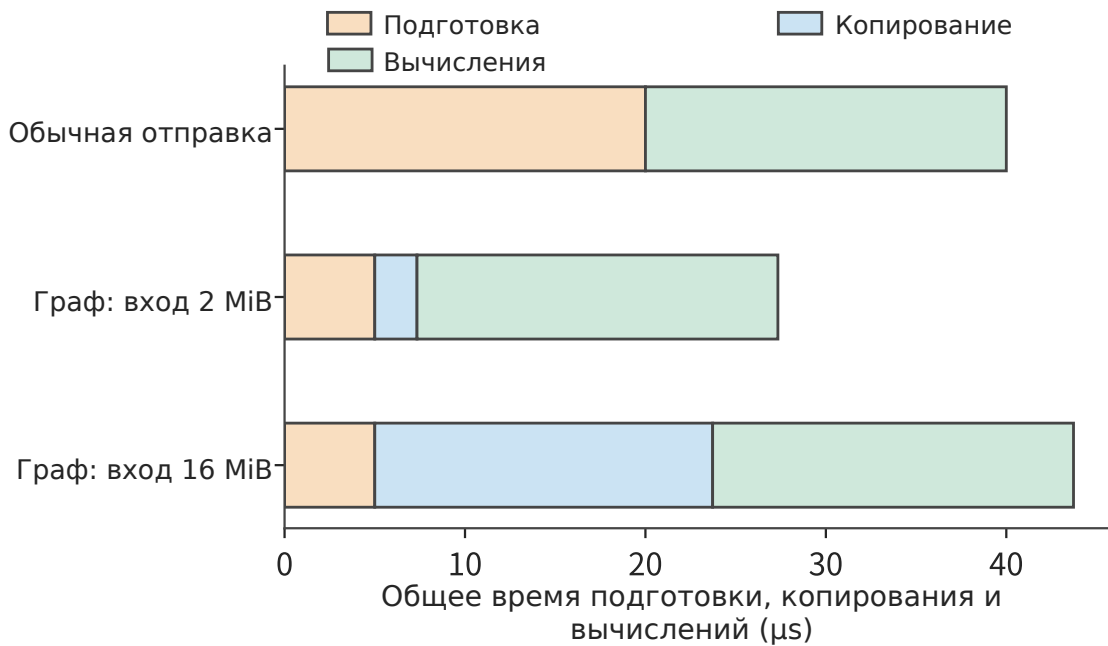


Рисунок 5-33. Оранжевым показана подготовка, синим — дополнительное копирование входных данных, зелёным — вычисления на ускорителе. Обычный способ занимает 40 μs ; граф с входными данными объёмом 2 MiB — около 27 μs , а с 16 MiB — около 44 μs . Время копирования рассчитано по пропускной способности видеопамяти RTX PRO 6000, равной 1792 GB/s.

время отправки с хоста с дополнительным временем копирования входных данных.³⁵

Повторное воспроизведение графа позволяет использовать последовательность отправки повторно; тот же принцип применим и к подготовительной работе, предшествующей созданию этих задач. Например, один план выполнения может совместно использоваться несколькими слоями модели. Библиотека операторов инференса FlashInfer разделяет построение плана (plan) и выполнение (run): plan выбирает kernel, разбивает работу на задачи, организует рабочую область с учётом длин запросов на текущем шаге и размещения KV в видеопамяти, а затем копирует эти метаданные на GPU; run выполняет внимание в соответствии с планом. На одном шаге длины запросов для всех 36 слоёв одинаковы, как и способ размещения KV, поэтому они могут использовать общий план. Если заново выполнять plan для каждого слоя, одна и та же подготовительная работа повторится 36 раз. В двух контрольных измерениях на RTX PRO 6000 переход к однократному выполнению plan сократил время 36 вызовов внимания примерно с 2,0 и 2,4 ms до 0,72 и 0,75 ms, а число копирований с хоста на GPU — со 144 до 4; число kernel на ускорителе не изменилось. Однако после изменения длины запроса plan необходимо выполнить заново: старый план будет считать по прежней длине и даст ошибочный результат.³⁶

³⁵Копирование малых входных данных при воспроизведении графа, копирование больших входных данных и компромиссы при выполнении графа. Пропускная способность копирования определяется как трафик интерфейса, включающий чтение источника и запись назначения.

³⁶Практическое измерение повторного использования плана FlashInfer.

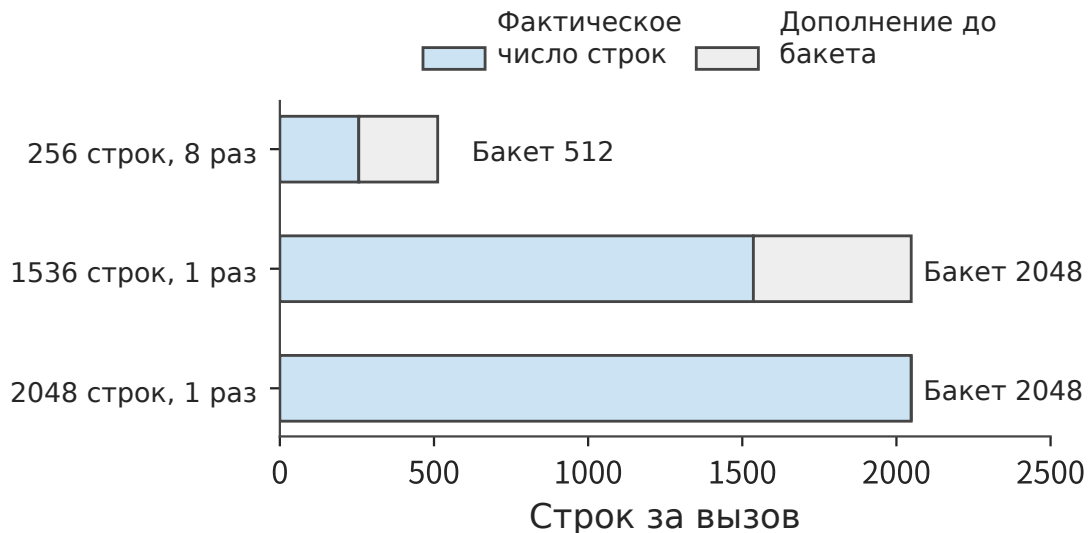


Рисунок 5-34. Вызовы на 256 строк дополняются до корзины на 512 строк, вызов на 1536 строк — до корзины на 2048 строк, а вызов на 2048 строк точно совпадает с границей корзины. Серым показаны дополнительные строки, обрабатываемые после дополнения; в 10 вызовах фактически содержится 5632 строки, но выполняется обработка 8192 строк.

5.5.3 Динамические формы и затраты на компиляцию

Совместное использование плана несколькими слоями модели опирается на одинаковую структуру вычислений; для совместного использования скомпилированной программы разными вызовами необходимо также учитывать изменения формы входных данных. При изменении числа строк среда выполнения может выбрать один из трёх способов. Универсальная программа вычисляет индексы и границы по текущей форме; разбиение по корзинам дополняет входные данные до нескольких репрезентативных форм и повторно использует программы для этих форм; специализация использует известные условия входных данных для создания специализированных реализаций — здесь под этим понимается отдельная компиляция программы для каждой часто встречающейся формы. Чем полнее используются известные формы, тем больше возможностей упростить работу ускорителя, но тем больше программ приходится компилировать и хранить.

Пример 5-12. Сколько повторных выполнений требуется, чтобы специализация стала оправданной? Возьмём группу из 10 выполнений FFN: 8 по 256 строк, одно на 1536 строк и одно на 2048 строк, всего $8 \times 256 + 1536 + 2048 = 5632$ строки. При использовании двух корзин на 512 и 2048 строк фактически обрабатывается $8 \times 512 + 2048 + 2048 = 8192$ строки, то есть примерно на 45% больше. На рисунке 5-34 показано, до какой корзины дополняется каждая форма.

Для трёх проекций полного FFN на каждую строку требуется суммарно $6HF$ FLOPs. Предположим, что на RTX PRO 6000 универсальная реализация, реализация с корзинами и специализированная реализация достигают соответственно около 20%, 40% и 50% от пиковых 503,8 TFLOP/s для плотных вычислений BF16, то есть их MFU составляет 20%, 40% и 50%, а эффективная скорость матричной обработки — 100, 200 и 250 TFLOP/s. Тогда время одной группы составит соответственно около 17,0, 12,4 и 6,8 ms. Хотя при использовании корзин выполняется на 45% больше вычислений, скорость обработки удва-

ивается, поэтому этот вариант всё равно быстрее универсальной реализации. Зададим время подготовки этих вариантов равным соответственно 100, 400 и 900 ms. Получим следующую таблицу.³⁷

³⁷ Полные входные данные и точки пересечения для специализации по форме.

Стратегия	Однократная подготовка Р	Выполнение одной группы Т	Способ обработки форм
Универсальная	100 ms	17,0 ms	Выполнение по фактическим 5632 строкам
Корзины	400 ms	12,4 ms	Дополнение до двух корзин, выполнение для 8192 строк
Специализация по каждой форме	900 ms	6,8 ms	Отдельная генерация программы для трёх форм

Общее время выполнения r групп равно $T_{\text{total}} = P + rT$. На рисунке 5-35 точка пересечения каждой прямой с вертикальной осью обозначает время подготовки, а наклон — время выполнения одной группы. В этом примере чем быстрее вариант выполняет одну группу, тем меньше наклон прямой, но тем выше её начальная точка из-за более длительной подготовки.

Подготовка варианта с корзинами занимает на 300 ms больше, чем универсального, но экономит около 4,6 ms на каждой группе, поэтому дополнительные затраты окупаются примерно после 65 групп. Подготовка специализированного варианта занимает ещё на 500 ms больше, чем варианта с корзинами, но экономит ещё около 5,6 ms на каждой группе, поэтому требуется около 90 групп. Расчёт по полным значениям показывает, что для 1–64 групп быстрее всего универсальный вариант, для 65–89 групп — вариант с корзинами, а начиная с 90 групп — специализированный.

Если экземпляр выполняет всего 70 групп, общее время трёх вариантов составит соответственно около 1,29, 1,27 и 1,38 s. Вариант с корзинами оказывается самым быстрым, поскольку экономия времени выполнения уже превысила дополнительные затраты на подготовку; хотя специализированный вариант выполняет каждую группу быстрее, накопленная экономия ещё не компенсирует дополнительную подготовку. Если программы всех трёх типов уже находятся в кэше и затраты на подготовку в этом вызове равны нулю, специализированный вариант будет самым быстрым с первой же группы. Таким образом, выбор, совершаемый средой выполнения, зависит одновременно от частоты форм и наличия уже скомпилированной программы в кэше.

Аналогично можно рассчитать, сколько вызовов требуется для окупаемости затрат на автоматическую настройку. Если поиск занимает 10 минут, а новая реализация экономит 5 μ s на каждом вызове, то для компенсации времени поиска накопленной экономией потребуется $600 / (5 \times 10^{-6}) = 1.2 \times 10^8$ вызовов. При 10 000 вызовов в секунду это займёт около 3,3 часа, а при 100 вызовах в секунду — около 14 дней. Чем дольше выполняется настройка, тем важнее в первую очередь оптимизировать часто вызываемые и длительно используемые формы.

В главе 8 выбор и кэширование программ будут подробнее рассмотрены с учётом поступления запросов и динамической пакетной обработки. Если определённая форма многократно появляется за короткое время, сохранение соответствующего результата компиляции сокращает повторную подготовку; если форма долго не используется, её кэш можно удалить, освободив место для других форм.

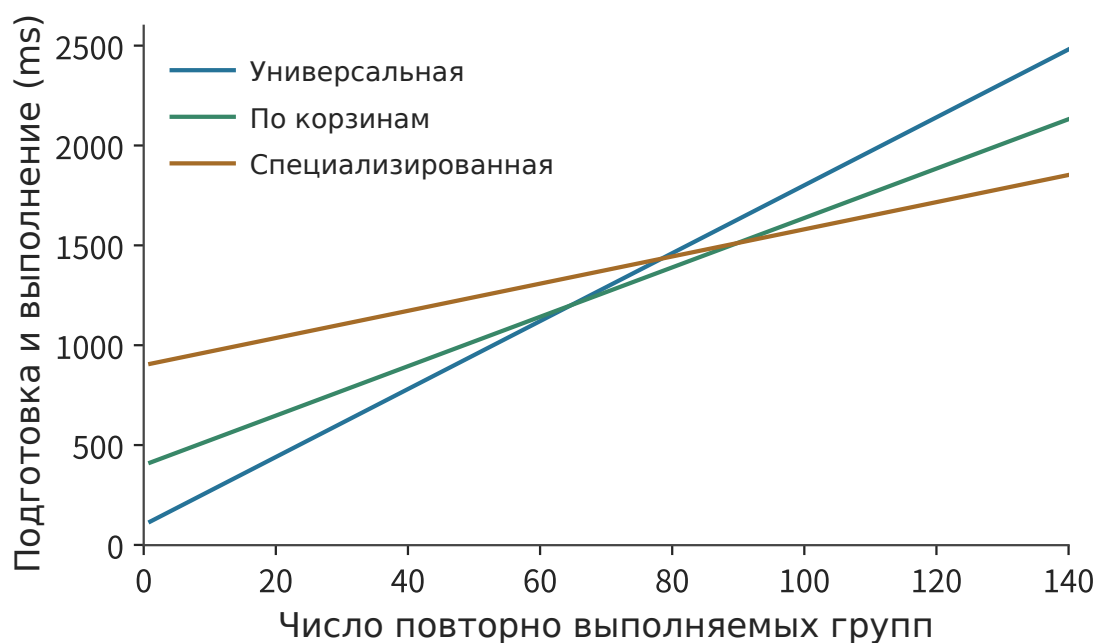


Рисунок 5-35. При многократном выполнении одной и той же группы вызовов наиболее быстрая стратегия меняется в зависимости от количества повторных использований. Кривые построены по формам, скоростям обработки и времени подготовки из примера 5-12; диапазоны целого числа вызовов, для которых применимы разные стратегии, рассчитаны по неокруглённым значениям. «Универсальная» стратегия использует один план выполнения для всех форм; «корзины» объединяют близкие формы в группы; «специализация» выбирает специализированный план для конкретной формы.

5.5.4 Persistent Kernel: планирование задач по блокам данных

Повторное воспроизведение графа и кэширование программ сокращают повторную подготовку на хосте, но на ускорителе существует другой вид ожидания: если два kernel зависят друг от друга, второй обычно может начать работу только после полного завершения первого. Если второму оператору нужен лишь один блок результата первого, условие ожидания можно сузить до блока данных: как только рассчитан первый блок, для него сразу начинается последующая операция, а вычисление остальных блоков продолжается параллельно.

persistent kernel постоянно работает на ускорителе, извлекает задачи из очереди и по флагам завершения проверяет готовность необходимых данных. Это не только сокращает повторные операции kernel launch на хосте, но и позволяет перекрывать выполнение соседних операторов по блокам. Mirage Persistent Kernel (MPK) — исследовательская система, использующая такое планирование задач.³⁸

Снова рассмотрим пример проекции `up` в Qwen3-8B с последующей активацией `SiLU`. Возьмём векторы признаков для 512 токенов и объединим в один блок векторы каждого 64 токенов, получив восемь блоков. Проекция каждого блока требует $2 \times 64 \times 4096 \times 12288 \approx 6.44$ GFLOPs, что при пиковой производительности плотных вычислений BF16 на RTX PRO 6000, равной 503,8 TFLOP/s, занимает около 12,8 μ s. При активации каждого блока обрабатывается 64×12288 элементов BF16, причём для каждого элемента считывается 2 bytes и записывается 2 bytes; при пропускной способности видеопамяти 1792 GB/s это занимает около 1,76 μ s. Предположим, что матричные и векторные ресурсы независимы, буферов достаточно, а каждая отдельная операция kernel launch с хоста занимает 5 μ s. Если активация начинается лишь после завершения всех проекций, общее время равно $2 \times 5 + 8 \times (12.8 + 1.76) \approx 126 \mu$ s.

Теперь разобьём вычисления на восемь задач проекции и восемь задач активации. Перед началом каждая задача извлекается из общей очереди: выполняется атомарное сложение с возвратом значения для счётчика очереди, что требует примерно одного обращения к L2 туда и обратно и занимает 0,15 μ s. После завершения публикуется флаг готовности — передача данных один к одному между SM, описанная в разделе 4.4.4, которая занимает 0,37 μ s. Таким образом, проекция каждого блока требует около 13,3 μ s, а активация — около 2,28 μ s. Нужна лишь одна операция kernel launch; восемь блоков проекции выполняются последовательно без промежутков, активация каждого блока начинается сразу после завершения его проекции, а активация последнего блока завершается уже после всех проекций. Общее время равно $5 + 8 \times 13.3 + 2.28 \approx 114 \mu$ s, что даёт ускорение примерно в 1,1 раза.

Более медленный этап здесь — проекция. Экономия примерно в 12,6 μ s складывается из двух составляющих: исключается одна операция kernel launch длительностью 5 μ s, а активация семи блоков, занимающая около 12,3 μ s, скрывается временем проекции. При этом накладные расходы задач возвращают в критический путь 4,7 μ s. Активация одного блока занимает лишь 1,76 μ s, поэтому её ранний запуск по блокам позволяет скрыть не больше этого времени. Чем ближе длительность двух этапов, тем больше времени экономится за счёт перекрытия по блокам; если один этап значительно длиннее другого,

³⁸ Организация задач на устройстве и MPK; расчёт для восьми блоков задач. Проекция вычислена исходя из пиковой производительности плотных вычислений BF16 на RTX PRO 6000, равной 503.8 TFLOP/s; активация — исходя из пропускной способности видеопамяти 1792 GB/s и 4 bytes трафика чтения и записи на элемент (448 G элементов/s). Оба значения взяты из таблицы оборудования; независимость ресурсов, накладные расходы задач и условия буферизации соответствуют условиям примера.

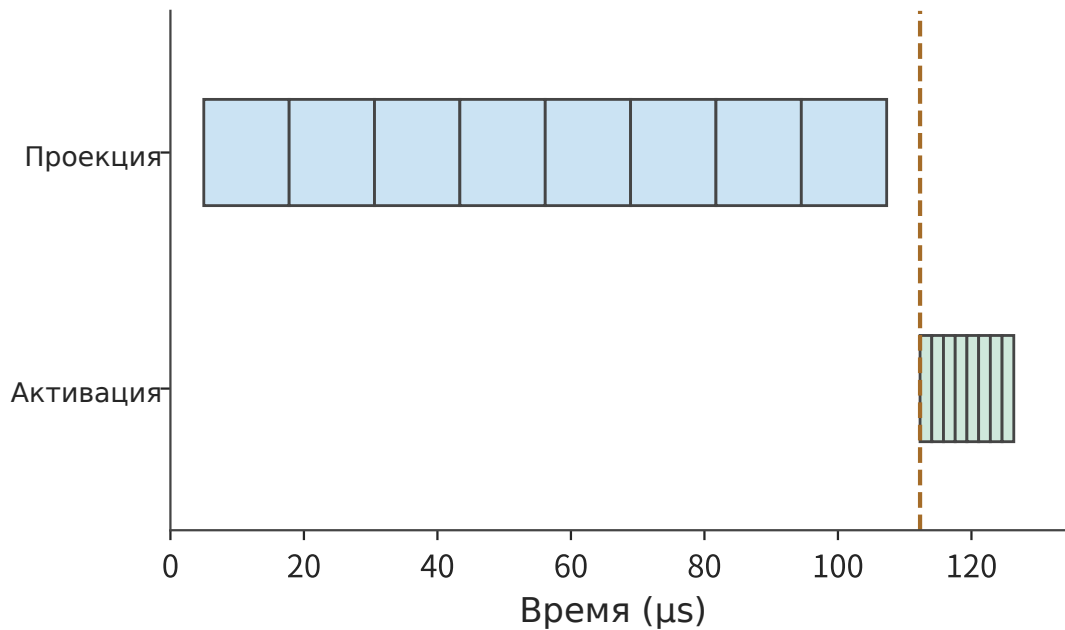


Рисунок 5-36. При крупнозернистом выполнении сначала завершаются проекции всех восьми блоков, после чего запускаются восемь активаций. При пиковой матричной производительности и пропускной способности видеопамати RTX PRO 6000 проекция одного блока занимает около 12,8 μs , активация — около 1,76 μs , а каждая из двух операций kernel launch — 5 μs ; вертикальная линия отмечает начало активации.

выигрыш практически сводится к исключению одной операции kernel launch.

Если проследить временную шкалу проекций на рисунке 5-37, видно, что восемь блоков проекции выполняются последовательно без промежутков. Здесь действует тот же принцип, что и при двойной буферизации: сначала завершается предыдущий этап для первого блока, после чего по одному блоку завершается с интервалом, равным времени более медленного этапа. Здесь медленнее проекция, подобно тому как в разделе 5.3.2 медленнее было перемещение; именно она определяет такт конвейера. Каждый слот буфера занят с начала проекции до завершения активации; если свободных слотов нет, задача проекции вынуждена ждать. Поэтому момент запуска следующего блока совместно определяется очередью задач, событиями завершения и наличием свободных буферов.

В предыдущем примере 5 μs — это стоимость отдельной отправки kernel с хоста. На стороне ускорителя сама граница между kernel значительно короче: на RTX PRO 6000 интервал между двумя пустыми kernel в CUDA Graph составляет около 430 ns, а при использовании PDL — около 370 ns, что сопоставимо с передачей один к одному между SM из раздела 4.4.4. persistent kernel исключает затраты на запуск, но обращения к L2 туда и обратно, необходимые для зависимостей, сохраняются.

При decode это проявляется ещё сильнее. Когда batch равен 1, каждое умножение матрицы на вектор распределяется по всем SM, а следующее умножение должно дожидаться результатов от всех SM, поэтому каждая граница требует передачи между всеми SM. Рассмотрим проекцию BF16 размером 4096×4096 в Qwen3-8B: 64 умножения образуют цепочку зависимостей, все веса считываются из видеопамати, а веса для следующего умножения заранее загружаются в общую память. Без зависимостей каждое умножение занимает около 21 μs и ограничено пропускной способностью видеопамати; после до-

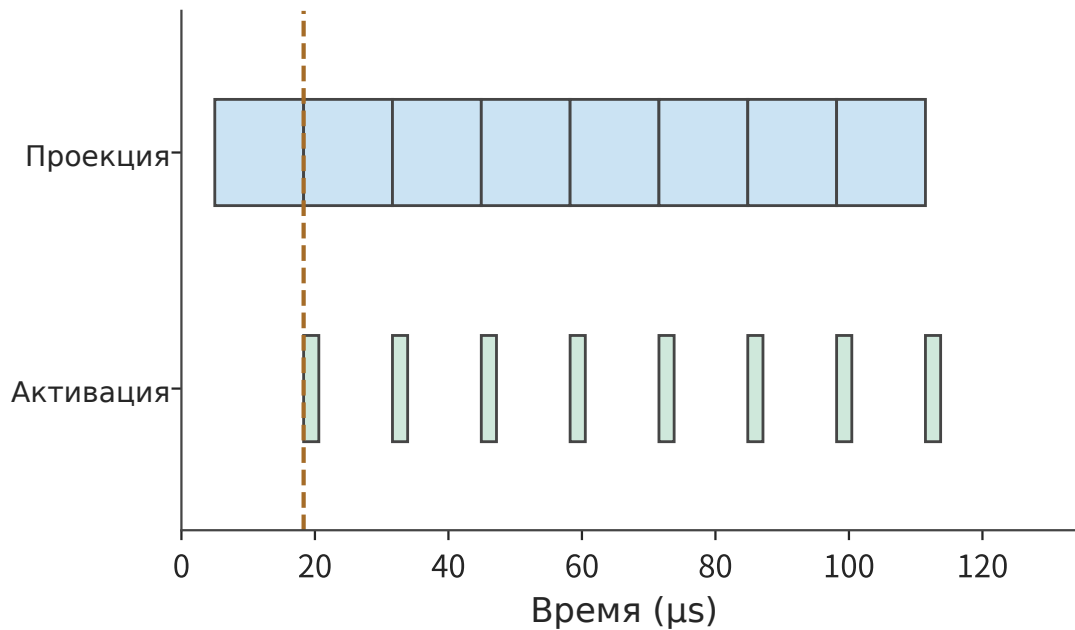


Рисунок 5-37. Матричные и векторные ресурсы независимы, буферов достаточно. Для каждой задачи дополнительно учитывается $0,52 \mu\text{s}$ на извлечение и уведомление; активация может начаться после завершения проекции первого блока. Проекция выполняется последовательно без промежутков, а активация ненадолго включается после проекции каждого блока; конвейер из восьми блоков завершается примерно за $114 \mu\text{s}$.

бавления зависимостей каждая граница добавляет $1,2\text{--}2,3 \mu\text{s}$. Дополнительное время в основном расходуется на ожидание самого медленного SM, а не на саму передачу: каждый SM обрабатывает фиксированные 21 или 22 строки, при этом видеопамять обслуживает разные SM неравномерно, и предварительная загрузка скрывает лишь часть этого времени. Если вместо этого каждый SM при освобождении слота буфера получает следующую группу строк через счётчик, быстрые SM выполняют больше работы, а медленные — меньше; дополнительное время на каждой границе снижается до $0,74\text{--}1,06 \mu\text{s}$, приближаясь к примерно $1 \mu\text{s}$ для передачи между всеми SM из раздела 4.4.4. Это стоимость самой зависимости, и изменение способа отправки не может её устранить.³⁹

5.6 От локальной оптимизации к полному запросу

Каждая из рассмотренных выше оптимизаций имеет чёткий объект воздействия: разбиение на блоки сокращает повторное чтение входных данных, слияние сокращает промежуточные операции чтения и записи, а оптимизация среды выполнения уменьшает время подготовки и ожидания. Однако полный запрос включает и другую работу, поэтому локально сэкономленное время составляет лишь часть его общей длительности. В этом разделе сначала объясняется, как по локальному ускорению оценить время запро-

³⁹Измерения автора на RTX PRO 6000 Blackwell Workstation: раздел «Under a real weight stream» в описании измерений и выдержка числовых данных. Поочерёдно использовались 16 матриц размером 4096×4096 , всего 512 MiB , что в четыре раза превышает ёмкость L2; дополнительное время на каждой границе равно медианному времени каждого умножения в цепочке зависимостей за вычетом времени ядра той же структуры без зависимостей. Границы CUDA Graph и PDL измерялись с помощью пустого ядра; для фактического ядра также необходимо учитывать время завершения и подготовки.

са, а затем разбирается реальный запрос Qwen3-8B.

5.6.1 Доля узкого места определяет потенциал ускорения

Пусть запрос состоит из последовательных этапов, время остальных этапов остаётся неизменным, доля узкого места в исходном времени равна f , а коэффициент его ускорения — s . Нормируем исходное время запроса к 1. Тогда новое время составит $(1 - f) + f/s$, а общее ускорение:

$$S_{\text{request}} = \frac{1}{(1 - f) + f/s}.$$

Если узкое место занимает 20%, то после двукратного ускорения общее время станет равным $0.8 + 0.2/2 = 0.9$, а коэффициент ускорения составит примерно 1,11. Если продолжить и ускорить узкое место в десять раз, общее время составит 0,82; даже если время узкого места уменьшится до нуля, остальные этапы всё равно потребуют 0,8 исходного общего времени, поэтому предельное общее ускорение равно 1,25 раза.

Это закон Амдала, представленный в главе 1. Он объясняет, почему отдача от оптимизации постепенно уменьшается: чем меньше времени занимает узкое место, тем выше доля остальных этапов в общем времени.

5.6.2 Параллельные ветви и смена критического пути

В соотношении Амдала время последовательных этапов складывается. Если две ветви запроса выполняются одновременно, после сокращения одной ветви может потребоваться ожидание другой; в таком случае нужно найти в графе зависимостей самый длинный путь, определяющий время завершения. Пусть после подготовки длительностью 10 μs одновременно запускаются узкое место А и ветвь В, занимающие соответственно 60 и 40 μs ; завершающий этап занимает 10 μs и ожидает окончания обеих ветвей. Исходное время запроса:

$$T = 10 + \max(60, 40) + 10 = 80 \mu\text{s}.$$

Если ускорить А в четыре раза, до 15 μs , тогда как В по-прежнему занимает 40 μs , новое время составит $10 + \max(15, 40) + 10 = 60 \mu\text{s}$. Для А экономия равна 45 μs , но запрос сокращается лишь на 20 μs , поскольку В сменяет А на критическом пути.

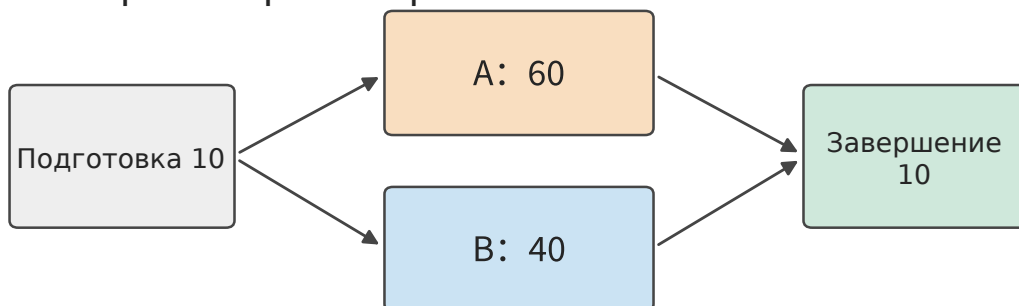
Когда А сокращается до 40 μs , она завершается одновременно с В; если продолжать оптимизировать только А, время завершения останется равным 60 μs . Эта точка пересечения задаёт максимальную выгоду, которую можно получить от изолированной оптимизации А. Чтобы дополнительно сократить запрос, нужно оптимизировать В либо уменьшить время подготовки и завершения.

Общие ресурсы также могут изменить время выполнения самих ветвей. Предположим, новая реализация А интенсивно использует пропускную способность, из-за чего время параллельно выполняемой В увеличивается до 90 μs . Тогда время запроса составит $10 + \max(15, 90) + 10 = 110 \mu\text{s}$. Дополнительное время В превысит время вычислений, сэкономленное в А, и запрос в целом станет медленнее.⁴⁰

Критический путь показывает, что время, сэкономленное в одной ветви, необходимо рассматривать в контексте всего графа зависимостей: только так можно определить,

⁴⁰ Учебный расчёт критического пути запроса, сценарий конкуренции за ресурсы.

A = 60 μ s: запрос 80 μ s



A = 15 μ s: запрос 60 μ s

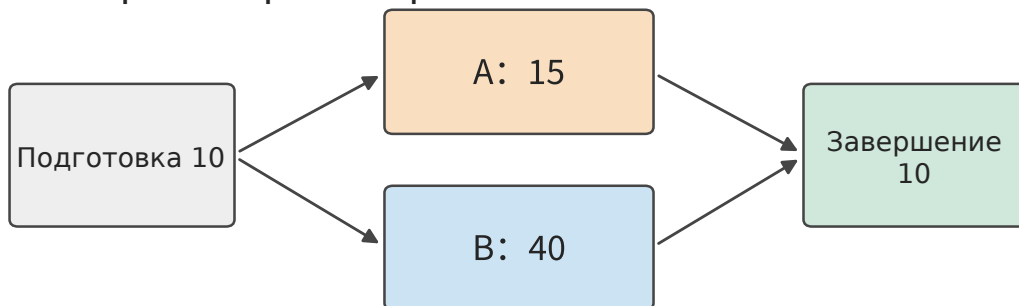


Рисунок 5-38. Завершающий этап ожидает обе ветви, А и В. После сокращения А с 60 до 15 μ s более медленной становится ветвь В вместо А, а время запроса уменьшается с 80 до 60 μ s. Подготовка и завершение занимают по 10 μ s, две ветви используют независимые ресурсы.

насколько ускорится запрос. Экономия памяти также нужно рассматривать в контексте всего процесса выполнения: уменьшение объёма, занимаемого состоянием, не обязательно равно сокращению числа байтов, считываемых на каждом шаге, поскольку объём чтения зависит от количества читающих его слоёв и от того, какие записи читает каждый слой. Для сессии V4.1 в разделе 2.3.6 уже приведён набор чисел для условия 8K: V4.1 позволяет 38 слоям глобального внимания совместно использовать 4 набора глобальных KV, благодаря чему резидентный объём снижается примерно до четверти от V4-Flash, однако логическое чтение по слоям сокращается лишь примерно на 9% и при этом превышает резидентный объём. Для общего состояния не нужно хранить отдельную копию на каждый слой, но каждый использующий его слой всё равно должен его читать, поэтому экономия объёма не превращается в равную ей экономию чтения.⁴¹

При увеличении контекста дополнительные накладные расходы также создаёт сканирование индекса, которое всё равно необходимо выполнять. При 128K объём логического чтения глобального исторического состояния и индекса у двух поколений моделей составляет соответственно примерно 65,085 и 33,422 MiB. В последующих слоях V4.1 индекс обращается только к пулу кандидатов, но начальный индекс всё равно должен обходить глобальные данные. Поэтому даже при фиксированной области доступа к индексу в последующих слоях затраты всего decode по-прежнему растут с длиной контекста.

На затраты влияет не только источник данных, но и то, действительно ли программа устраняет вычисления. Пул кандидатов сокращает область поиска в индексе для последующих слоёв, соответственно уменьшая объём работы. В открытой эталонной реализации сначала вычисляется скалярное произведение для полного кэша индекса, после чего маскируются позиции за пределами пула кандидатов; в описанной в статье производственной реализации непосредственно уменьшается число записей, сканируемых в индексе последующими слоями. В первом случае скалярные произведения за пределами пула уже вычислены, а во втором эта работа исключается ещё при запуске вычислений. Таким образом, одному и тому же правилу выбора соответствуют две разные стоимости выполнения.

5.6.3 Комплексный пример: выбор оптимизаций для одного запроса Qwen3

Применим этот метод к Qwen3-8B на RTX PRO 6000. На вход запроса поступает 7239 токенов, генерируется 32 выходных токена, конкурентность равна 1, используются BF16 и выполнение eager; здесь eager означает, что хост последовательно, в порядке программы, отправляет задания ускорителю. Модель содержит 36 слоёв: сначала выполняется один prefill, формирующий первый выходной токен, а затем — 31 decode. В каждом слое на каждом шаге один раз выполняется активационная часть SwiGLU, то есть $Z = \text{SiLU}(G) \odot U$ из раздела 5.3.1. Всего получается $36 \times 32 = 1152$ вызова: 36 для prefill и 1116 для decode.

Эти два вида вызовов выполняют разную работу. Prefill одного запроса применяет этот оператор к 7239 входным токенам в каждом из 36 слоёв модели, обрабатывая в общей сложности представления $36 \times 7239 = 260604$ токенов; за 31 шаг decode в каждом слое обрабатывается один новый токен, что даёт $36 \times 31 = 1116$ вызовов однотокенного оператора. Здесь работа оператора суммируется как «число слоёв $\times$ число токенов», поэтому один и тот же токен, участвующий в вычислениях разных слоёв, учитывается

⁴¹Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт для межглавной сессии.

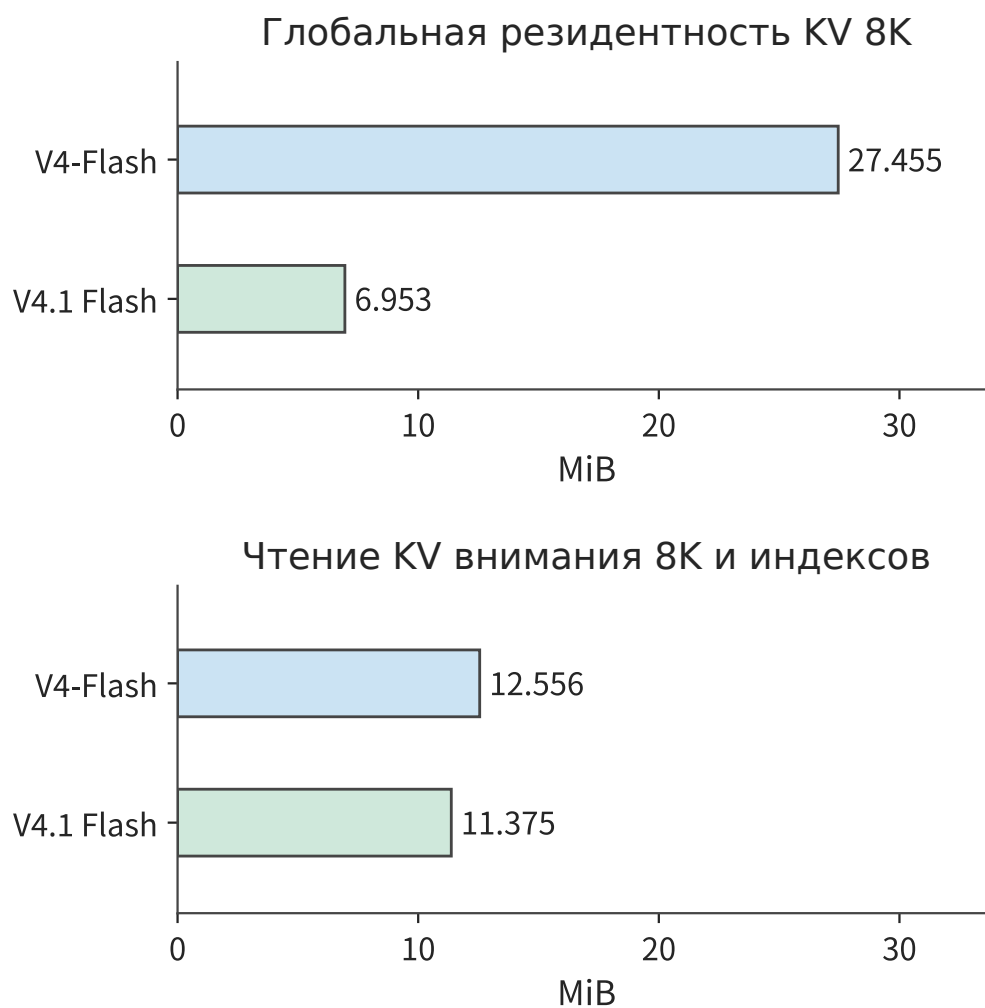


Рисунок 5-39. Глобальный резидентный объём и послойное логическое чтение сравниваются отдельно. В верхней части учитывается только объём глобальных KV, а чтение в нижней части включает глобальные записи, локальное окно и индекс; для двух столбцов используются одинаковые масштабы горизонтальной оси. Значения рассчитаны для производственной компоновки.

повторно. Число вызовов decode в 31 раз больше числа вызовов prefill, но число обрабатываемых строк намного меньше. Это именно то различие форм, которое неоднократно встречалось в этой главе: большие матрицы предоставляют больше параллельной работы, а для маленьких матриц время вычислений короче, накладные расходы на запуск ядра и распределение задач заметнее, и полностью загрузить ускоритель сложнее.

В том же движке SwiGLU во всех 36 слоях заменили ядром с другой схемой диспетчеризации. Запись выполнения подтвердила, что во всех 1152 вызовах использовалось заменённое ядро. Времена этапов приведены ниже.⁴²

Этап SwiGLU	Форма и количество вызовов	Исходное ядро	Заменённое ядро
Prefill	7239 строк, 36 раз	около 11,2 ms	около 11,2 ms
Decode	1 строка, 1116 раз	около 2,4 ms	около 0,92 ms

Заменённое ядро главным образом сокращает время активации decode для однострочного входа: коэффициент ускорения составляет примерно 2,6, а суммарная экономия в части decode — около 1,45 ms. Время активации prefill практически не изменилось и после замены даже увеличилось примерно на 0,03 ms. Эти активации и прочая работа ускорителя в трассе выполняются последовательно. Все вычисления активаций исходным ядром занимают около 13,6 ms, то есть 1,6% от примерно 851 ms интервала сбора данных; если остальная работа не изменится, даже полное устранение всех вычислений активаций сэкономит не более 13,6 ms. Сейчас сокращена только часть decode, поэтому суммарное время всех вычислений активаций уменьшилось с 13,6 до 12,2 ms, а ожидаемое сокращение всего запроса составляет примерно 1,4 ms.

При чередующемся измерении запросов до и после замены на клиенте получено 11 пар результатов. Время до первого токена в обоих случаях составило около 436 ms, а медиана полного времени запроса — примерно 817 и 815 ms соответственно. Если для каждого раунда вычислить «время до замены минус время после замены», медиана парной разницы составит около 1,3 ms, или примерно 0,16% исходного времени запроса, что согласуется с ожидаемыми 1,4 ms. На рисунке 5-40 приведена разница для каждого раунда: в 9 парах после замены время сократилось, а в 2 — увеличилось.

Улучшение decode происходит после первого выходного токена, поэтому в текущем запросе с длинным входом время до первого токена по-прежнему определяется вычислениями и ожиданием в prefill. Чем длиннее выходные данные, тем больше раз повторяются вычисления активации для однострочного входа и тем больше накапливается экономия времени на каждом шаге.

При анализе полного запроса сначала подсчитывают количество вызовов для каждой формы, затем вычисляют время, сэкономленное на каждом вызове, и наконец с учётом зависимостей определяют, может ли эта экономия сократить запрос целиком. Такой подход позволяет связать коэффициент ускорения ядра, длительность отдельных этапов и фактическое время ожидания пользователя.

⁴²Реализация ядра, журнал выполнения и 11 пар запросов в эксперименте 5-9. vLLM 0.23; заменяющее ядро использует планирование block256/4-warps, выходные токены совпали во всех 11 парах. Медианы на клиенте составили 816.640/815.406 ms, медиана парной разницы времени — 1.343 ms, медиана времени до первого токена — 435.674/436.353 ms. В отдельно собранном профиле производительности prefill составил 11.218659/11.244366 ms, decode — 2.369803/0.920376 ms. Время исходного ядра составило 13.588462 ms, после замены — 12.164742 ms, интервал сбора данных — 850.848989 ms; пересечение по времени между каждой горячей точкой и другой наблюдаемой работой GPU равно нулю.

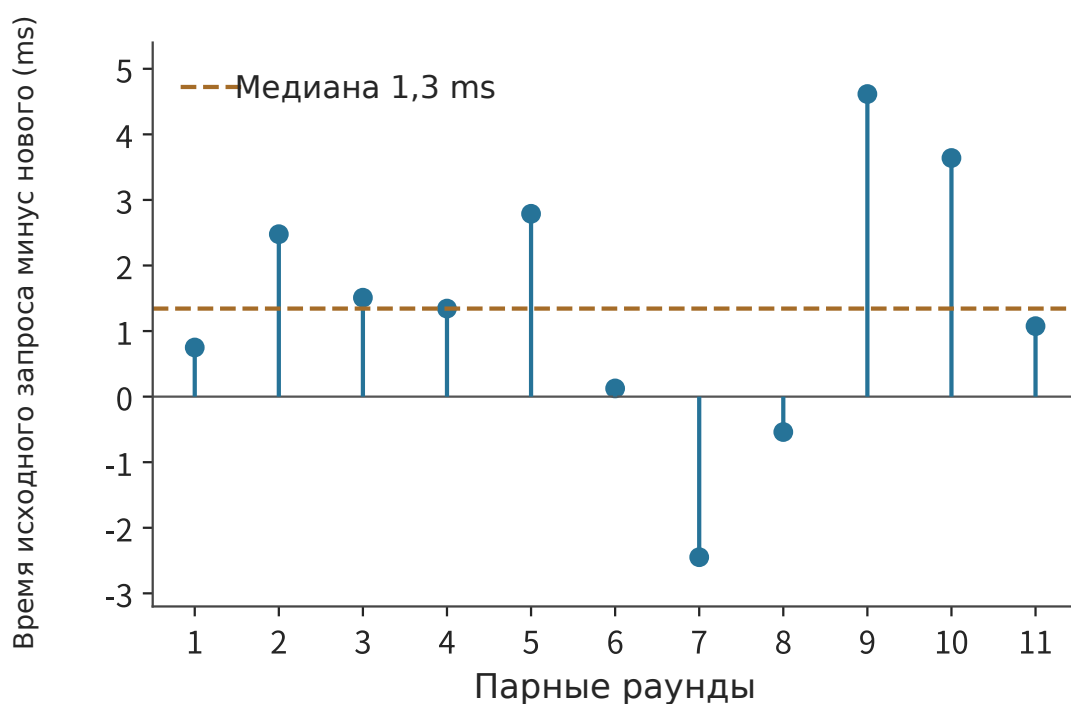


Рисунок 5-40. Время запроса до замены в том же раунде минус время после замены; положительное значение означает, что после замены запрос выполнялся быстрее. Qwen3-8B на RTX PRO 6000, вход из 7239 токенов, принудительный вывод 32 токенов, конкурентность 1, BF16, eager, кэш префиксов отключён; 11 пар чередующихся измерений, порядок внутри раунда случайный, во время измерений работали другие резидентные сервисы. Пунктирная линия обозначает медиану парной разницы времени, составляющую около 1,3 ms. Таблица этапов основана на отдельно собранной записи профилирования.

5.7 Распространённые заблуждения

Заблуждение: стоимость выполнения можно оценить лишь по числу запусков kernel. Повторное воспроизведение графа сокращает количество отправок трёх FFN с хоста с 18 до 3, но ускоритель по-прежнему выполняет 18 запусков kernel; только слияние сокращает количество запусков kernel на ускорителе до 15. Отправки с хоста и вычисления на ускорителе следует рассматривать отдельно, каждое на собственной временной шкале.

Заблуждение: реализация с наименьшим объёмом обращений к памяти обязательно самая быстрая. При сокращении блоков K/V механизма внимания с 64 строк до 1 строки объём обращений уменьшается на 144 MiB, однако количество обновлений состояния возрастает примерно в 41 раз. Стоимость выполнения в совокупности определяется чтением, обновлением состояния и скоростью обработки матриц.

Заблуждение: увеличение числа буферов повышает пропускную способность. На одном SM H100 перенос каждого блока занимает 1,29 μ s, а вычисление — 0,28 μ s; двух слотов уже достаточно для непрерывной работы механизма переноса, и четыре блока обрабатываются за 5,44 μ s. Третий слот не ускорит перенос, как не поможет и ускорение вычислений; чтобы сократить время, нужно уменьшить количество переносимых байтов на блок либо увеличить пропускную способность.

Заблуждение: если установившийся режим выполняется быстрее всего, то и общее время будет минимальным. При выполнении 70 групп вызовов разбиение по корзинам занимает около 1,27 s, а специализация для каждой формы — около 1,38 s. Каждая группа со специализацией выполняется быстрее, но выигрыш во времени пока недостаточен, чтобы компенсировать дополнительные затраты на подготовку.

5.8 Упражнения и сопутствующие эксперименты

В следующих заданиях условия примеров этой главы последовательно изменяются. Сначала выполните расчёты на бумаге, а затем объясните различия с помощью сопутствующего кода или журналов выполнения. Точки входа экспериментов сохраняют исходную нумерацию; конфигурации, порядок действий и полные журналы приведены в соответствующих материалах.

Эксперимент 5-1 · Расширенный: как размер матричного блока изменяет объём локального хранилища и чтения

Измените выходной блок в примере 5-3 на 64×128 , сохранив размер блока редукции равным 32, и вычислите требуемый объём локального хранилища, количество чтений A и количество чтений W, а также общий объём обращений. Какой минимальной доли эффективной пропускной способности варианта 64×64 должен достичь вариант 64×128 , чтобы время доступа к памяти стало меньше? Затем сравните журналы выполнения для двух порядков циклов и объясните, как последовательный доступ влияет на пропускную способность обработки.⁴³

Эксперимент 5-2 · Основной: насколько слияние операторов активации сокращает обращения к памяти и потребление видеопамати

⁴³Эксперимент 5-1, разбиение на блоки на GPU, разбиение редукции.

Для цепочки активаций, в которой G и U занимают по 24 MiB, изобразите порядок выделения, использования и освобождения каждого тензора при раздельном выполнении трёх операторов и при их полном слиянии. Выведите объёмы обращений 156 MiB и 60 MiB, а также пиковое потребление видеопамати в обоих вариантах. Если реализация со слиянием при каждом вызове добавляет фиксированные накладные расходы 5 μ s, а пропускная способность интерфейса принимается равной пропускной способности видеопамати RTX PRO 6000 — 1792 GB/s, то при каком минимальном числе входных строк экономия времени на обращениях к памяти превысит эти накладные расходы? Затем, опираясь на результаты сопутствующего эксперимента, объясните связь между долей сокращения объёма обращений и долей сокращения времени выполнения.⁴⁴

Эксперимент 5-3 · Расширенный: как блочная редукция и предвыборка изменяют вычисление внимания и обращения к памяти

Добавьте ко входным данным примера 5-6 ещё один элемент с оценкой $\ln 4$ и соответствующим значением 5, затем обновите m , ℓ и u и сравните результат с одновременной обработкой трёх элементов. Сохраните общую ёмкость буфера внимания равной 99 KB (101,376 байт), дополнительно выделив из него слот предвыборки BF16 размером $b \times d$. При $b=64$ найдите новое максимальное число строк Q , количество проходов и объём обращений. Объясните, почему буфер предвыборки изменяет количество повторных чтений данных.⁴⁵

Эксперимент 5-4 · Расширенный: изменяет ли слияние операторов результат вычислений или стоимость выполнения

В псевдокоде раздела 5.4.2 поочерёдно перенесите SiLU внутрь цикла ko , удалите промежуточное округление BF16 и переместите вычисление квантования внутрь цикла по каждому блоку выходных столбцов. Укажите, что именно изменяется в каждом случае: математические операции, численное округление или стоимость выполнения. Приведите входные данные, позволяющие различить результаты или объёмы обращений до и после изменения.⁴⁶

Эксперимент 5-5 · Расширенный: как блочные циклы изменяют порядок доступа и временное хранение

Запишите блочный цикл $i-k-j$ через номера блоков и координаты внутри блока, отметив в программе места создания, последнего использования и освобождения входного кэша и аккумулятора. Затем сопоставьте процессы выполнения сопутствующей программы транспонирования с двумя размерами блоков, сравните непрерывность доступа, операции синхронизации и объём временных данных и на этой основе объясните различие во времени выполнения.⁴⁷

Эксперимент 5-6 · Расширенный: как доля вызовов для разных форм

⁴⁴Исходные условия, данные и анализ эксперимента 5-2. RTX PRO 6000, совместно используемый GPU, горячий кэш и воспроизведение графа; исходные медианы составили 14.20/7.68 μ s, измерялась только подцепочка активации.

⁴⁵Эксперимент 5-3. Сравнение для одной головы на RTX PRO 6000 с горячим кэшем: 2.64 ms/79.5 μ s; пиковый объём дополнительных выделений памяти — около 848/22.2 MiB, включая временную память, выделенную каждым бэкэндом.

⁴⁶Различия в компиляции, округлении и выполнении в эксперименте 5-4, подцепочка экспертов DeepSeek V4-Flash.

⁴⁷Сгенерированный код и сравнение транспонирования в эксперименте 5-5.

входных данных влияет на выгоду от оптимизации

Используйте две формы входных данных из примера 5-9. Примите долю вызовов формы А равной соответственно 1/2, 2/3 и 9/10 и вычислите среднее время исходной и новой реализаций. Если выбор реализации по форме добавляет 1 μ s на каждый вызов, причём для А используется новая реализация, а для В — исходная, то при каких условиях этот подход лучше постоянного использования исходной реализации? Затем по сопутствующему журналу поиска подсчитайте его общую продолжительность и вычислите, сколько времени выполнения найденная реализация экономит при повторных вызовах.⁴⁸

Эксперимент 5-7 · Расширенный: сколько повторных выполнений требуется, чтобы специализация стала выгодной

Изобразите полную временную шкалу подготовки и выполнения из примера 5-12. Измените число вызовов малой и средней форм в каждой группе с 8 на 4, сохранив по одному вызову для каждой из двух остальных форм. Попарно сравните три варианта и найдите для каждой пары число повторений, при котором их полное время совпадает. Затем предположите, что программы для двух корзин уже скомпилированы и заэкшированы, и после изменения времени подготовки найдите диапазоны числа повторений, в которых каждый вариант обеспечивает минимальное полное время.⁴⁹

Эксперимент 5-8 · Основной: какие накладные расходы сокращают воспроизведение графа и слияние операторов

Изучите рисунок 5-28 и отдельно подсчитайте для каждого из четырёх вариантов число отправок с хоста и количество kernel на ускорителе. Перенесите условия примера 5-11 на RTX 4090, приняв пропускную способность копирования равной пропускной способности его видеопамати — 1008 GB/s, и найдите размер входных данных, при котором затраты на копирование в точности компенсируют время, сэкономленное воспроизведением графа. Затем организуйте прямую запись из вышестоящего этапа в буфер графа и сравните время завершения при обычной отправке и воспроизведении графа для входных данных размером 2 MiB и 16 MiB.⁵⁰

Эксперимент 5-9 · Основной: как ускорение kernel изменяет задержку запроса и критический путь

По сопутствующим журналам 11 пар запросов сначала вычислите разность времени выполнения в каждой паре, а затем найдите медиану этих разностей. Объясните, чем она отличается от разности медиан времени выполнения, отдельно вычисленных для двух групп запросов. Используя условия комплексного примера этой главы и сохраняя pfill без изменений, увеличьте число шагов decode с 31 до 127. Предполагая неизменную экономию времени на каждом шаге, оцените общее время, которое удастся сэкономить на вычислении активаций. Затем постепенно сокращайте время выполнения А на рисунке 5-38 и найдите его значение, ниже которого полное время запроса больше не уменьшается.⁵¹

⁴⁸Журнал поиска и измерений эксперимента 5-6; расчёты для оценки и развёртывания. Зафиксированы два раунда, всего 12 вызовов агента; новых более быстрых реализаций получено не было.

⁴⁹Полные входные данные и точки пересечения для специализации по форме.

⁵⁰Эксперимент 5-8 и JSON с анализом трассировки.

⁵¹Реализация ядра, журнал выполнения и 11 пар запросов в эксперименте 5-9. vLLM 0.23; заменяющее

5.9 История и материалы для дальнейшего чтения

Halide разделяет алгоритм и планирование, превращая разбиение на блоки, переупорядочивание и выбор места вычисления в комбинируемые программистом действия; TVM связывает тензорные вычисления, генерацию кода для бэкендов и автоматическую настройку; AKG объединяет полиэдральное планирование, многоуровневое слияние и размещение в локальной памяти для генерации кода нейронных процессоров. Изучение первичных материалов по этим трём системам позволяет подробнее проследить, как способ представления влияет на преобразования, которые компилятор способен выполнять автоматически.^{52,53}

FPGA — программируемое логическое устройство, а высокоуровневый синтез (HLS) преобразует программы более высокого уровня в аппаратные тракты данных. Ранние работы автора с FPGA/HLS также прошли путь от обёртывания интерфейсов к преобразованию кода: для формирования требуемого конвейера необходимо было распознавать примитивы и зависимости чтения и записи, изменяя генерируемый тракт данных. Этот опыт показывает, что акцент исследований сместился с упрощения программирования на улучшение способа выполнения.

Система оптимизации оркестрации операторов Korch даёт ещё один показательный пример: в одном историческом случае подграф первоначально выполнялся тремя ядрами с суммарным временем около 91 μs ; после перехода к четырём ядрам суммарное время составило около 69 μs . Повторное разбиение и объединение операторов сократили время выполнения на ускорителе настолько, что компенсировали дополнительные накладные расходы на запуск ядер и передачу данных.⁵⁴ Вместе с приведённым в этой главе контрпримером для слияния это направление показывает, что при оптимизации следует сравнивать затраты всего процесса вычисления.

5.10 Итоги главы

В этой главе мы начали с одной матричной проекции, затем рассмотрели доступ к памяти, слияние операторов, компиляцию и среду выполнения, а в завершение проанализировали запрос целиком. Во всех разделах неоднократно использовались три основных принципа.

Во-первых, **повторное использование требует пространства**. Более крупный выходной tile позволяет одним и тем же входным данным участвовать в большем числе операций умножения-сложения, но требует больше памяти для аккумуляторов; сохранение результатов квантования позволяет нескольким блокам столбцов считывать данные меньшей разрядности, но также требует промежуточной памяти. Чтобы определить, оправданы ли эти затраты памяти, следует оценить, сколько повторных чтений

ядро использует планирование block256/4-warps, выходные токены совпали во всех 11 парах. Медианы на клиенте составили 816.640/815.406 ms, медиана парной разницы времени — 1.343 ms, медиана времени до первого токена — 435.674/436.353 ms. В отдельно собранном профиле производительности prefill составил 11.218659/11.244366 ms, decode — 2.369803/0.920376 ms. Время исходного ядра составило 13.588462 ms, после замены — 12.164742 ms, интервал сбора данных — 850.848989 ms; пересечение по времени между каждой горячей точкой и другой наблюдаемой работой GPU равно нулю.

⁵²TVM, TensorIR.

⁵³Zhao и др., 2021, AKG: Automatic Kernel Generation for Neural Processing Units using Polyhedral Transformations.

⁵⁴Пример оркестрации Korch: подграф из статьи для V100/FP32; времена трёх ядер — 38.7, 24.2 и 28.2 μs , четырёх ядер — 7.6, 18.4, 7.5 и 35.7 μs .

или вычислений они устраняют. На SM объём этой памяти также определяет, сколько блоков потоков могут одновременно находиться на нём, а значит, сколько находящихся в обработке запросов к памяти способно скрывать задержку.

Во-вторых, **зависимости по данным определяют порядок вычислений**. Поэлементные операции можно выполнять сразу после подготовки входных данных для соответствующей позиции, тогда как редукции требуют сохранять частичные суммы или текущую статистику. Компилятор перемещает вычисления в соответствии с зависимостями, а среда выполнения с помощью событий определяет, готовы ли данные и можно ли освободить буфер. И слияние, и конвейеризация основаны на этих зависимостях.

В-третьих, **общее время зависит от порядка выполнения всех работ**. Пропускная способность определяет длительность передачи порции данных, более медленный этап — интервал завершения двух соседних блоков в конвейере, число повторных использований — смогут ли последующие сокращения времени выполнения компенсировать первоначальные затраты на подготовку, а критический путь — позволит ли локальное улучшение сократить время обработки запроса. Поэтому необходимо не только вычислить, какой объём работы удалось устранить, но и определить, влияла ли эта работа изначально на итоговое время завершения.

В следующей главе эти взаимосвязи будут распространены на несколько ускорителей. Момент готовности части результата, то, кому потребуется её прочитать, и срок её хранения определяют, когда начнётся обмен данными и сколько буферной памяти потребуется на обеих сторонах.

Глава 6

Суперузлы

В предыдущей главе матрица разбивалась на небольшие блоки, чтобы входные данные можно было повторно использовать вблизи вычислительного модуля. Теперь передадим один из этих блоков другой карте-ускорителю: какие входные данные ей потребуются, будет ли результат вычислений полным выходом или частичной суммой и кто продолжит использовать этот результат? Математика не изменилась, но данные, ранее проходившие через общую память, теперь могут передаваться по межсоединению ускорителей или даже по сети дата-центра. Так один и тот же способ разбиения превращается в схему параллельных вычислений на нескольких картах.

Как только веса модели перестают помещаться в видеопамять одной карты, подход «одна карта — одна модель» становится неприменим: веса Qwen3-235B-A22B в формате BF16 занимают около 470 ГБ, что почти в шесть раз превышает объём памяти одной H100. Имея восемь карт, можно поручить каждой независимо обрабатывать запросы или объединить их для ускорения одного запроса; способ разбиения должен одновременно обеспечить размещение модели и достижение целевых показателей задержки и стоимости. В этой главе мы начнём с размерностей и зависимостей операторов, постепенно выведем коммуникации, возникающие при различных способах распараллеливания, а затем отберём подходящие схемы для заданной модели, восьмикартной аппаратной конфигурации и длительности сессии. Прочитав главу, читатель должен уметь перечислить возможные схемы для своей модели и аппаратной платформы, исключить неприменимые и сделать выбор с учётом узких мест и результатов измерений, а не просто запомнить несколько сокращений, обозначающих способы распараллеливания.

В предыдущей главе рассматривались вычисления, повторное использование данных и выполнение задач внутри одного ускорителя. При параллельной работе на нескольких картах остаются те же вопросы: по каким измерениям выполнять разбиение, где размещать каждый блок, какие входные данные нужны блокам, где объединяются результаты и в каком порядке выполняются операции. В этой главе разбиение и планирование операторов на одной карте расширяются до нескольких карт: сначала матрицы и состояние распределяются между картами, затем выводятся возникающие при этом коммуникации, после чего коммуникации сопоставляются с реальными соединениями и, наконец, определяется, сколько карт должен использовать один экземпляр и сколько экземпляров следует развернуть одновременно. В качестве аппаратной платформы рассматривается распространённый восьмикартный сервер HGX H100, а все рассуждения сопровождают две модели: веса Qwen3-235B-A22B занимают весь сервер — на её примере разбираются ёмкость, распределение экспертов и коммуникации MoE; веса Qwen3-32B помещаются на одной карте, однако KV-кэш

при длинном контексте вынуждает распределять модель между картами — на её примере сравниваются количество карт для тензорного параллелизма и количество экземпляров. Другие модели и системы используются для демонстрации того, как изменения структуры влияют на проектирование.

6.1 От одной карты к нескольким

6.1.1 Независимое развёртывание и совместная работа восьми карт

Экземпляр инференса, представленный в главе 1, может занимать одну карту либо распределять модель по нескольким картам. Суперузел предоставляет физические ресурсы для частого взаимодействия внутри экземпляра: его ускорители тесно связаны между собой высокоскоростными соединениями с низкой задержкой.

Экземпляр инференса описывает, как развёртывается и выполняется модель, а суперузел — как соединены ускорители. На суперузле с восемью картами можно разместить восемь однокарточных экземпляров или два четырёхкарточных. В первом случае восемь запросов независимы друг от друга; во втором каждый запрос требует обмена промежуточными результатами между четырьмя картами. Серверы и стойки служат единицами монтажа и электропитания; все карты одного экземпляра могут находиться как внутри одного сервера, так и на нескольких серверах.

Большее число карт обычно требуется по трём причинам: веса модели превышают объём памяти одной карты и должны храниться распределённо; отдельный запрос выполняется слишком медленно и вычисления нужно распределить между несколькими картами; одновременно поступает слишком много запросов и необходимо увеличить число экземпляров, способных обрабатывать запросы независимо. Эти три потребности соответствуют трём целям: ёмкости, задержке и пропускной способности.

6.1.2 Сквозной пример: ёмкость и время одного шага Qwen3-235B-A22B

Сквозной пример этой главы — запуск Qwen3-235B-A22B на одном сервере HGX H100 после завершения обработки входных данных, когда в кэше находятся 8192 контекстных токена и требуется продолжить генерацию. HGX H100 — распространённый сервер с восемью картами: каждая из восьми H100 SXM имеет 80 GB HBM с пиковой пропускной способностью 3350 GB/s и пиковой производительностью 989.4 TFLOP/s при операциях с плотными матрицами в BF16. Восемь карт соединены через NVSwitch — коммутационные микросхемы, организующие сеть NVLink, — посредством NVLink четвёртого поколения с пропускной способностью 450 GB/s на каждую карту в каждом направлении. Кроме того, каждая карта оснащена сетевым адаптером ConnectX-7 на 400 Gbit/s (50 GB/s) для подключения к другим серверам.¹

¹Техническое описание HGX H100: восемь GPU соединены через NVSwitch, пропускная способность NVLink между GPU составляет 900 GB/s, а скорость сети — до 400 Gbit/s; в спецификации NVIDIA H100 для H100 SXM указаны 80 GB, 3,35 TB/s, NVLink 900 GB/s и PCIe Gen5 128 GB/s, причём последние два значения представляют суммарную пропускную способность в направлениях приёма и передачи; описание архитектуры H100, стр. 47: 18 линий NVLink четвёртого поколения, по 25 GB/s в каждом направлении на линию; техническое описание ConnectX-7: до 400 Gbit/s на порт; руководство пользователя DGX H100: 8 ускорителей H100, 640 GB памяти GPU. Конфигурация с одной сетевой картой 400 Gbit/s на каждый ускоритель также приведена в публичном журнале выполнения эксперимента 7-3. Пропускная способность HBM и значение 989,4 TFLOP/s взяты из строки h100-sxm в таблице оборудования.

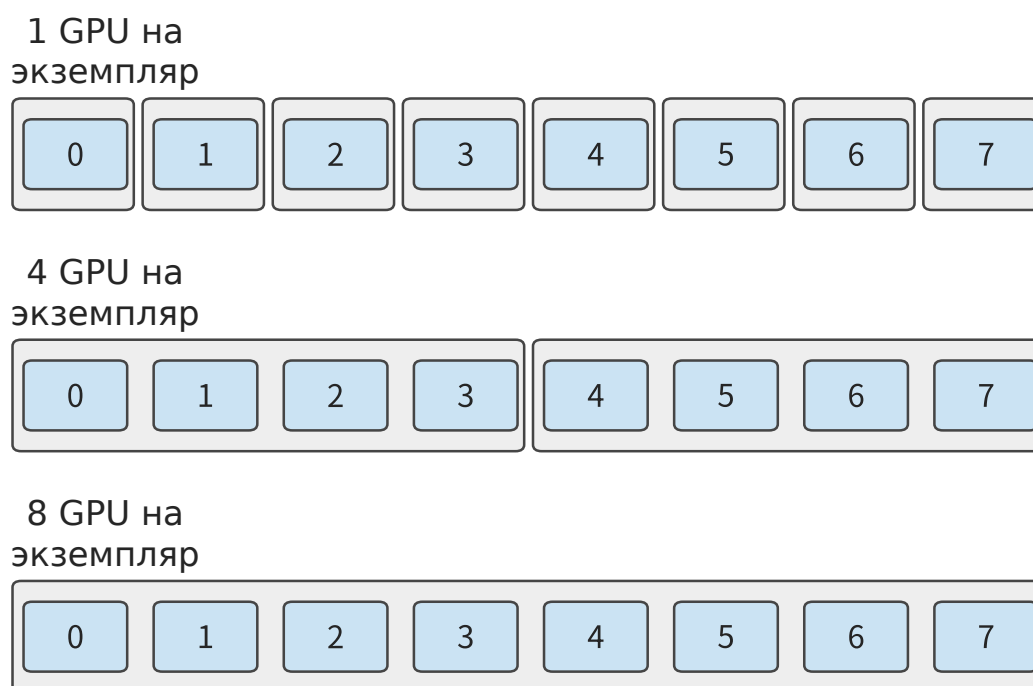


Рисунок 6-1. Три варианта группировки экземпляров на одних и тех же восьми картах. Каждая внешняя рамка обозначает отдельный экземпляр инференса, а внутренние линии — взаимодействие, необходимое для выполнения запроса. Если каждый экземпляр обрабатывает по одному запросу за раз, эти три варианта развёртывания позволяют одновременно обрабатывать соответственно восемь, два и один запрос.

При проектировании многокарточной системы сначала необходимо определить размещение данных. Одновременно хранящиеся на карте i веса W_i , состояние KV K_i , активации A_i и остальные рабочие области U_i должны удовлетворять условию

$$W_i + K_i + A_i + U_i \leq C_i. \quad (6-1)$$

Здесь C_i — доступная задаче память этой карты. Объём весов определяется моделью, KV растёт вместе с контекстом сеанса, а объёмы активаций и рабочих областей зависят от способа выполнения. Все четыре составляющие одновременно используют физическую память одной карты.

Размер модели Qwen3-235B-A22B. Модель хранит веса и KV в BF16, содержит 94 слоя и имеет скрытую размерность 4096; в каждом слое 64 головы запросов и 4 головы KV, размерность головы равна 128. FFN состоит из 128 маршрутизируемых экспертов, для каждого токена выбираются 8 из них, а промежуточная размерность каждого эксперта равна 1536. GB и TB используются в десятичном смысле, MiB и GiB — в двоичном.

Сначала рассчитаем ёмкость. Веса всей модели в BF16 занимают приблизительно 470.2 GB. Для каждого контекстного токена необходимо хранить K и V каждого слоя, поэтому объём KV всей модели на один токен равен

$$2 \times 94 \times 4 \times 128 \times 2 = 188 \text{ KiB}.$$

KV для 8192 токенов занимает в общей сложности около 1.58 GB. Если зарезервировать для активаций и остальных рабочих областей 2 GiB, то есть приблизительно 2.15 GB, общая потребность одной карты составит около 473.9 GB — почти шестикратный объём памяти одной H100.

Чтобы разместить модель, эти данные необходимо распределить между восемью картами: веса разбиваются на восемь частей по строкам или столбцам матриц, и каждая карта хранит одну часть; рабочая область по-прежнему резервируется отдельно на каждой карте. KV разделяется по головам внимания, но у этой модели всего 4 головы KV: при использовании восьми карт каждые две карты совместно используют одну голову KV, причём каждая хранит собственную копию, поэтому на каждой карте находится четверть всего KV. Как такое разбиение сохраняет неизменным результат вычислений, объясняется в разделе 6.2.2. Требования к памяти каждой карты становятся следующими:

Размещение	Веса	KV	Резерв для активаций и рабочих областей	Итого
Одна карта	470.2 GB	1.58 GB	2.15 GB	473.9 GB
Каждая из восьми карт	58.96 GB	0.39 GB	2.15 GB	61.50 GB

Каждой карте требуется около 61.50 GB, что меньше 80 GB у H100; весь сервер использует суммарно около 492.0 GB, и из 640 GB остаётся примерно 148 GB. Объём весов на каждой карте немного превышает одну восьмую от 470.2 GB, поскольку параметры нормализации и маршрутизатор реплицируются на всех картах. Оставшиеся на каждой карте примерно 18.5 GB позволяют хранить дополнительные сеансы: при таком разбиении один сеанс из 8192 токенов занимает на каждой карте 0.39 GB, поэтому весь сервер

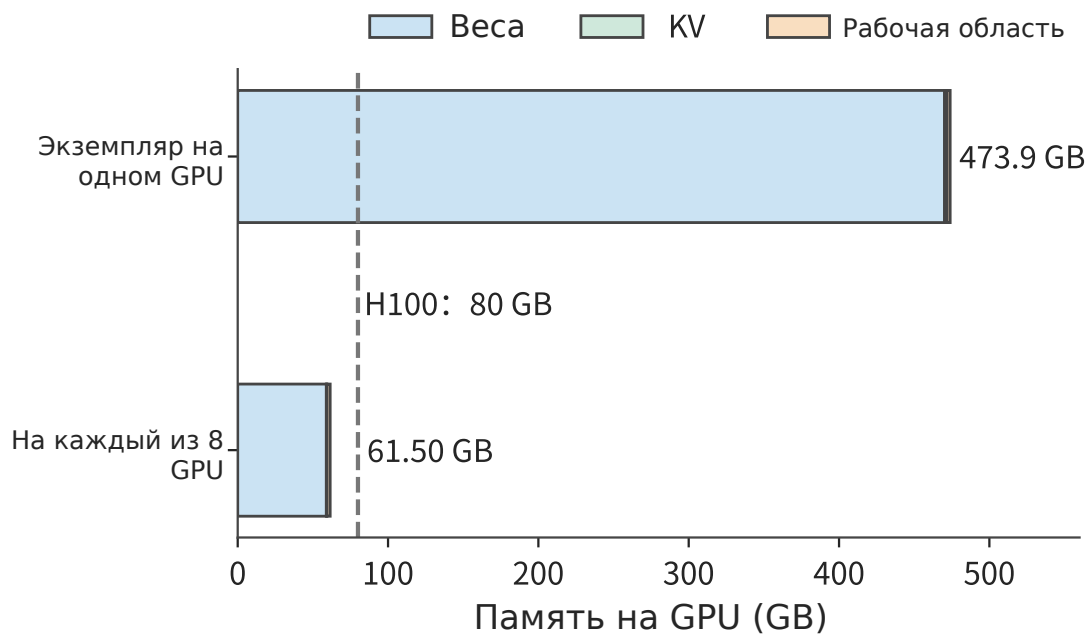


Рисунок 6-2. Требования к памяти однокарточного экземпляра и каждой карты после распределения по восьми картам. Одной карте требуется около 473.9 GB, что намного превышает 80 GB у H100 (пунктирная линия); после распределения по восьми картам каждой требуется около 61.50 GB. Рабочая область объёмом 2 GiB резервируется отдельно на каждой карте.

может одновременно хранить не более 47 таких сеансов. Распределённое хранение снижает требования к памяти каждой карты, тогда как общий объём памяти экземпляра равен сумме объёмов, занятых на всех картах.²

Теперь рассчитаем время выполнения. Ёмкость памяти определяет, можно ли сохранить эти данные, а объём обращений — сколько данных необходимо прочитать или записать за одно выполнение. Рассмотрим вычисления экспертов одного слоя: при генерации одного токена маршрутизатор выбирает 8 экспертов, у каждого из которых есть три матрицы с общим объёмом весов $8 \times 3 \times 4096 \times 1536 \times 2 = 288 \text{ MiB}$; все они должны быть один раз прочитаны из HBM. При скорости 3350 GB/s одно чтение занимает около 90.1 μs ; матричные умножения требуют примерно 302 млн FLOPs и при производительности 989.4 TFLOP/s выполняются всего за 0.31 μs . Чтение весов занимает более чем на два порядка больше времени, чем вычисления, поэтому для сокращения времени этого вызова прежде всего следует уменьшить объём весов, читаемых каждой картой. При генерации одного токена всей модели необходимо прочитать около 43.1 GB весов и 1.58 GB KV. Если всё читает одна карта, это занимает около 13.3 ms; при равномерном распределении между восемью картами описанным выше способом каждая карта читает около

²Полный набор весов BF16 для одной карты занимает 470 187 269 120 bytes, а KV для 8192 токенов — 1 577 058 304 bytes. При восьмикарточном TP каждая карта хранит 58 959 617 024 bytes весов, включая отдельную копию параметров нормализации и маршрутизатора на каждой карте, и 394 264 576 bytes KV; на каждой карте помещается не более 47 сеансов по 8192 токена. В таблице GB указаны в десятичной системе, а GiB/MiB — в двоичной; сначала точные значения суммируются, а затем результат округляется. В 2 GiB включён резерв для активаций и прочего рабочего пространства. Подробности приведены в материалах «Размещение Qwen3-235B-A22B на восьми картах» и соответствующем JSON, где считываются зафиксированные официальные конфигурация и индекс весов. Объём чтения и число операций за один шаг приведены в материалах «Decode одного запроса с контекстом 8K» и «Prefill 8192 токенов».

5.79 GB, что занимает примерно 1.73 ms.

При перекрытии вычислений оператора и обращений к памяти время выполнения можно оценить по формуле:

$$T_{\text{op}} = \max \left(\frac{F}{P}, \frac{V}{B_{\text{HBM}}} \right). \quad (6-2)$$

F — объём вычислений, P — производительность матричных операций, V — объём обращений к HBM в байтах. Вычисления и обращения к памяти выполняются одновременно, поэтому время выполнения оператора определяется более длительной из этих операций. Следующий оператор должен ждать результата предыдущего, поэтому длительности последовательных этапов складываются. Формула (6-2) представляет собой модель Roofline из раздела 4.8.1. Согласно определению из раздела 1.2.2, MFU вызова для одного токена не превышает 0.31/90.1, то есть составляет около 0.3%. Это аппаратный предел при batch, равном 1, не зависящий от качества реализации; все рассматриваемые в этой главе способы распараллеливания по-разному организуют чтение и повторное использование весов в рамках данного ограничения. В разделе 6.2 будет показано, как распределить веса и вычисления этого слоя между несколькими картами, а в разделе 6.4 будет рассчитано время обмена результатами между картами.

6.1.3 Различия в требованиях к ресурсам для Prefill, Decode и обучения

Значения 90.1 μs и 0.31 μs из раздела 6.1.2 соответствуют вызову для одного токена. Если за один раз обрабатывать больше токенов, одни и те же веса можно использовать для нескольких строк входных данных, а соотношение времени вычислений и чтения изменится. Когда один вызов обрабатывает m токенов, объём вычислений экспертов пропорционален m , но веса выбранных экспертов могут повторно использоваться для этих токенов. Если заменить вызов для одного токена в приведённом выше примере на prefill для 8192 токенов, будут выбраны все 128 экспертов, а объём чтения весов экспертов одного слоя вырастет до 4.5 GiB, что при 3350 GB/s займёт около 1.44 ms. Объём матричных вычислений при этом вырастет примерно до 2.47 TFLOPs, что при 989.4 TFLOP/s потребует около 2.50 ms и превысит время чтения. При большом batch одни и те же веса используются для множества строк входных данных, поэтому основным ограничением времени выполнения вместо чтения весов становятся матричные вычисления.

Характер коммуникации также меняется в зависимости от формы данных. Скрытый тензор в BF16 имеет форму $m \times 4096$: один токен занимает 8 KiB, а 8192 токена — 64 MiB. Оба вызова проходят одинаковое число слоёв и выполняют одинаковое число обменов, однако объём данных каждой передачи различается в 8192 раза. На передачу малого объёма данных сильнее влияют накладные расходы запуска, а передача большого объёма в большей степени зависит от устойчивой пропускной способности.

Decode также имеет временную зависимость: текущий выходной токен определяет вход следующего шага. Один сеанс должен продвигаться последовательно, тогда как несколько сеансов могут одновременно предоставлять уже готовые входные данные. Чем больше сеансов, тем больше возможностей для повторного использования весов, но тем выше ёмкость и объём чтения KV; следовательно, одновременно изменяются время вычислений, время обращений к памяти и время ожидания в очереди.

При обучении после прямого прохода необходимо выполнить обратное распространение, вычислить градиенты входов и весов, а затем обновить параметры с помощью

градиентов; активации прямого прохода должны сохраняться до завершения их использования при обратном проходе. Поэтому при многокарточном обучении также требуется синхронизировать градиенты и хранить больше состояний. В этой главе сначала рассматриваются разбиение модели и создаваемый им обмен данными, а в главе 10 — то, как эти операции образуют полный шаг обучения.

6.1.4 Обзор способов распараллеливания: какое измерение разбивать

Все три потребности из раздела 6.1.1 требуют распределения работы между несколькими картами, но способы распределения различаются. Они зависят от обрабатываемых моделью данных и структуры самой модели. Входные активации одного слоя имеют три измерения: число обрабатываемых за один раз образцов B , которое при инференсе равно числу одновременно обрабатываемых запросов; число позиций последовательности в каждом образце S ; число скрытых признаков в каждой позиции H . В матричном умножении из главы 5 измерения B и S объединялись в число строк M , однако механизму внимания необходимо различать позиции внутри одной последовательности, поэтому здесь они рассматриваются отдельно. В структуре самой модели есть ещё два измерения, допускающих разбиение: слои L и множество экспертов E каждого слоя в модели МоЕ. На рисунке 6-3 эти пять измерений показаны вместе с местами разбиения для шести распространённых способов распараллеливания.

По разбиваемым измерениям шесть способов распараллеливания можно свести к следующей таблице. Под «обменом» понимаются данные, которые необходимо передавать между картами; их объём будет последовательно рассчитан в подразделах раздела 6.2.

Способ распараллеливания	Разбиваемое измерение	Что хранит и вычисляет каждая карта	Чем необходимо обмениваться между картами
Параллелизм по данным (DP)	Образцы B	Полная копия модели, разные образцы	При инференсе обмена нет; при обучении объединяются градиенты всех карт
Тензорный параллелизм (TP)	Признаки H : столбцы или строки весов, головы внимания	Часть матриц одного слоя	Фрагменты выходных данных при необходимости собираются; частичные суммы необходимо сложить
Параллелизм по последовательности (SP)	Позиции последовательности S для поточечных операторов над токенами	Отрезок позиций одной последовательности в этих операторах	Перед матричным умножением собираются все позиции; после суммирования частичных сумм данные снова разбиваются по позициям
Контекстный параллелизм (CP)	Позиции последовательности S в механизме внимания	Q , K и V для отрезка позиций одной последовательности	Удалённые K и V либо статистики внимания для каждого отрезка

Способ распараллеливания	Разбиваемое измерение	Что хранит и вычисляет каждая карта	Чем необходимо обмениваться между картами
Конвейерный параллелизм (PP)	Слои L	Непрерывная группа слоёв	Между соседними этапами передаются активации; при обучении в обратном направлении также передаются градиенты
Параллелизм экспертов (EP)	Эксперты E	Часть экспертов	Токен отправляется на карту с выбранным экспертом, а результат возвращается обратно

Зачем нужны шесть способов распараллеливания: от узких мест к композиции.

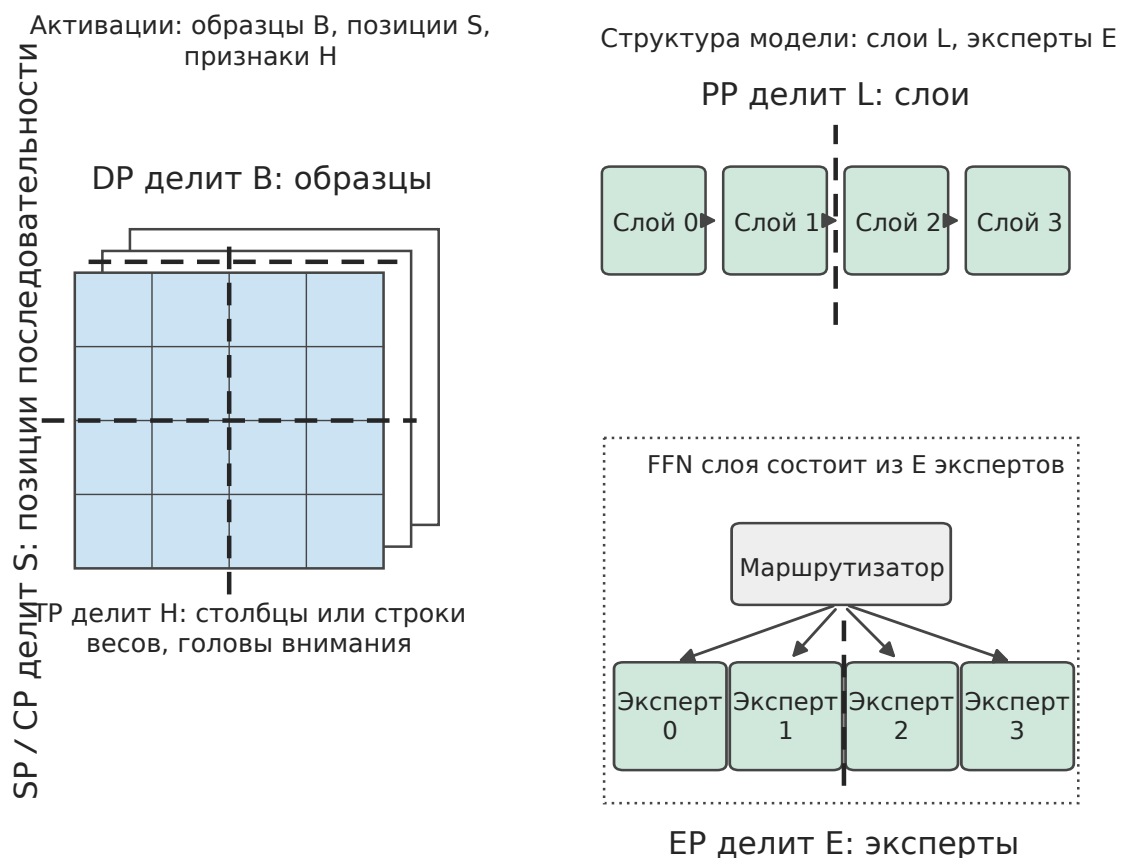


Рисунок 6-3. Слева показаны входные активации одного слоя: три измерения соответствуют местам разбиения при параллелизме по данным, параллелизме по последовательности и контексту, а также тензорном параллелизме. Справа показана структура модели: конвейерный параллелизм разделяет соседние слои, а параллелизм экспертов — экспертов одного слоя. Жирная пунктирная линия обозначает место разбиения, а не направление потока данных.

Таблица показывает точки разделения и связанные с ними узкие места. Эти связи удобно представить в виде русской матрёшки. Внешний слой управляет образцами или запросами, поэтому параллелизм по данным (DP) определяет, сколько независимых работ продвигается одновременно. Затем одна работа проходит модель слой за слоем, а конвейерный параллелизм (PP) передаёт последовательные группы слоёв разным этапам. Внутри слоя параллелизм экспертов (EP) распределяет вычисления МоЕ по экспертам, тензорный параллелизм (TP) разделяет матрицы по признакам, а параллелизм по последовательности и контекстный параллелизм (SP/CP) распределяют активации и внимание по позициям последовательности. Каждый уровень имеет собственные зависимости данных, точки синхронизации и границы коммуникации. В сентябре 2026 года главный учёный Huawei по полупроводниковым технологиям доктор Ляо Хэн предложил Nested BSP, рекурсивно повторяющую «параллельную работу, синхронизацию, обмен и агрегацию» на разных уровнях; на рисунке 1 этой статьи используется та же программная матрёшка.³

Матрёшка показывает структуру композиции. DP может тиражировать полный экземпляр $TP \times PP \times EP$; SP обычно использует ту же группу карт, что и TP; CP добавляется при ограничении ёмкости состояния или вычислений внимания длинным контекстом; EP применяется к моделям с набором экспертов. Разные способы разделяют разные координаты одного пространства выполнения:

- DP обслуживает большое число образцов или запросов и повышает общую пропускную способность;
- TP распределяет вычисления, веса и промежуточные результаты внутри слоя и отдельного крупного запроса;
- SP распределяет хранение и обработку активаций для операторов над отдельными токенами внутри группы TP;
- CP распределяет состояние контекста и вычисления внимания для длинных последовательностей, обмениваясь удалённым контекстом;
- PP размещает большое число слоёв и крупные веса в конвейере с заданным порядком вычислений;
- EP распределяет веса экспертов и маршрутизацию токенов в МоЕ.

Каждый способ отвечает за своё основное узкое место. DP увеличивает число независимых работ, TP уменьшает размер матриц внутри слоя, SP снижает затраты на активации отдельных токенов, CP расширяет ёмкость контекста, PP позволяет разместить больше слоёв, а EP использует разреженность МоЕ. Их сочетание определяет конкретную раскладку по устройствам. Например, 16 карт можно организовать как $DP2 \times PP2 \times TP4$: две копии модели обрабатывают разные примеры, каждая копия разделена на два конвейерных этапа, а каждый этап использует четыре карты для матриц внутри слоя. Группы TP из четырёх карт могут одновременно применять SP, сохраняя общее число карт равным 16. При проектировании следует распределять работу, создающую основное узкое место, на подходящем уровне и размещать возникающий обмен на соответствующем физическом соединении.

Таблица также показывает два способа организации результатов. При разделении по

³Главный учёный Huawei по полупроводниковым технологиям доктор Ляо Хэн, Nested Parallel von Neumann Architecture and Nested BSP, arXiv:2609.16787, 15 сентября 2026 года. Раздел 2 и рисунок 1 описывают рекурсивное вложение программного параллелизма; снаружи внутрь на рисунке расположены DP, PP, EP, FSDP, CP и TP. Глава 6 использует иерархический взгляд статьи для объяснения сочетаний, а глава 10 — для различения FSDP и SP; пример с 16 картами является учебным примером этой книги.

примерам, позициям последовательности, слоям или экспертам каждая карта отвечает за разные данные или слои, а результаты передаются через сбор или межэтапную передачу. Если признаки разделены вдоль измерения редукции, каждая карта вносит часть одного и того же выхода, поэтому перед дальнейшим использованием результата необходимо сложить частичные суммы. При разбиении выходного измерения получаются разные выходные элементы, а не слагаемые одной суммы. В разделе 5.2.4 это различие рассматривалось внутри одной карты; между картами действует то же правило, а редукция проходит через межкарточное соединение.

Уточнение русского издания: необходимость суммирования относится к разбиению измерения редукции, а не к любому разбиению по признакам.

Эти способы объединяются в одной конфигурации. Параллелизм по последовательности обычно использует ту же группу карт, что и тензорный параллелизм; параллелизм экспертов может использовать карты параллелизма по данным для части внимания; конвейерный параллелизм разделяет вычислительный граф по его последовательным зависимостям. В разделе 6.3 показано, как нумеровать карты в таких комбинациях.

> Эксперимент 6-1 · Углублённое задание: как длина контекста и число входных строк влияют на ёмкость и узкое место выполнения > > (а) Рассчитайте объём KV одного запроса Qwen3-235B-A22B при 4096, 8192 и 16384 контекстных токенах. > > (b) При описанном в этом разделе разбиении по восьми картам и резервировании 2 GiB рабочей области на каждой карте рассчитайте, сколько сеансов по 32768 токенов может одновременно хранить один HGX H100; точное значение объёма весов приведено в примечании. > > (c) Для 1, 64 и 8192 токенов отдельно рассчитайте объём матричных вычислений экспертов одного слоя и время чтения весов выбранных экспертов. Для 64 токенов оцените число выбранных экспертов по формуле (6-8) из раздела 6.3.2. Используя параметры H100, определите, чем главным образом ограничено время выполнения: вычислениями или обращениями к памяти. > > (d) Уменьшите пропускную способность HBM вдвое и сравните увеличение времени вызова для трёх указанных количеств токенов.

6.2 Шесть способов распараллеливания

В этом разделе в порядке, показанном на рис. 6-3, последовательно рассматриваются шесть способов распараллеливания. Чтобы наглядно продемонстрировать распределение работы, в каждом подразделе сначала приводится пример с двумя ускорителями, а затем он обобщается на большее их количество.

6.2.1 Параллелизм по данным: копирование модели, разделение выборки

Параллелизм по данным (data parallelism, DP) предполагает, что каждый GPU хранит полную копию модели и обрабатывает отдельную часть выборки. На рис. 6-4 два GPU обрабатывают соответственно первые четыре и последние четыре примера из batch, каждый получая свою часть выходных данных.

Параллелизм по данным при инференсе — это развёртывание нескольких экземпляров. Если модель помещается на одном GPU и имеется несколько независимых запросов, самый прямой подход — распределить разные запросы между разными экземплярами. Каждый экземпляр хранит собственные веса и состояние сессии и независимо формирует результат. Если один GPU способен обрабатывать r запросов в секунду, то восемь

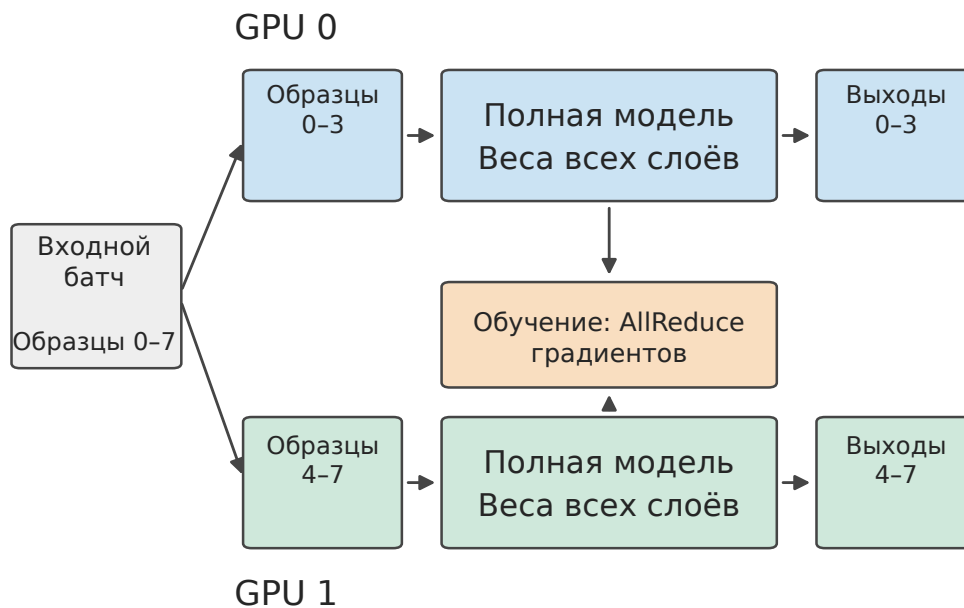


Рис. 6-4. Каждый из двух GPU хранит полную копию модели и обрабатывает отдельные примеры. При инференсе GPU не ждут друг друга; при обучении их градиенты сначала объединяются, после чего каждый GPU обновляет свою копию параметров.

экземпляров на отдельных GPU при постоянном наличии запросов смогут обрабатывать $8r$ запросов в секунду.

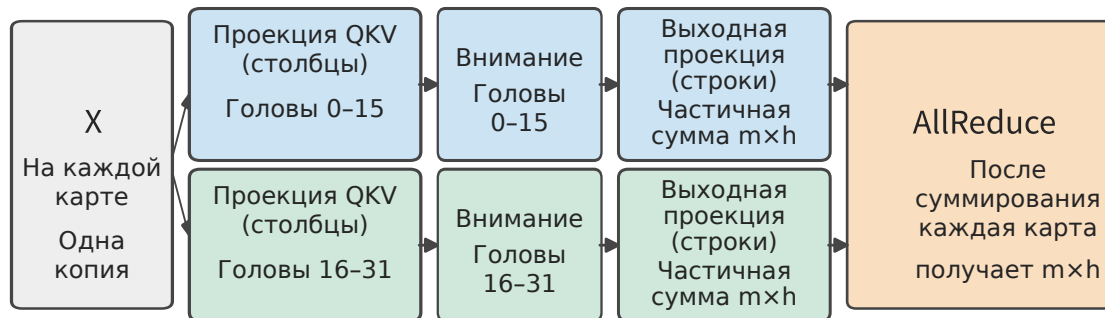
Если объединить восемь GPU в один экземпляр и обозначить коэффициент ускорения отдельного запроса как $S(8)$, этот экземпляр сможет обрабатывать $S(8)r$ запросов в секунду. Оба варианта развёртывания используют одни и те же восемь GPU, поэтому отношение их пропускной способности равно $S(8)/8$: пропускная способность совпадёт лишь в том случае, если отдельный запрос ускорится ровно в восемь раз. При наличии обмена данными и работы, не поддающейся распараллеливанию, совместно работающий экземпляр жертвует частью общей пропускной способности ради сокращения времени обработки отдельного запроса.

При параллелизме по данным во время обучения на нескольких GPU также размещается один и тот же набор параметров, однако градиенты со всех GPU совместно определяют следующее обновление. Например, если два GPU обрабатывают по четыре примера и вычисляют свои средние градиенты g_0, g_1 , то средний градиент по всем восьми примерам равен $(g_0 + g_1)/2$. Каждый GPU вычисляет градиенты для половины примеров, после чего градиенты объединяются — так выполняется одно и то же обновление. Между экземплярами, выполняющими инференс, такой зависимости нет.

6.2.2 Тензорный параллелизм: разделение матриц внутри слоя

Несколько экземпляров позволяют одновременно обрабатывать больше запросов, но не сокращают время выполнения отдельного запроса на одной карте и не помогают с сессиями, которые не помещаются на одну карту. Чтобы две карты совместно обрабатывали один запрос, необходимо разделить вычисления и состояние внутри слоя. **Тензорный параллелизм** (tensor parallelism, TP) распределяет матрицы одного слоя между несколькими картами: каждая карта вычисляет свою часть, после чего карты обмениваются ре-

Подслой внимания: деление по головам, выходная проекция даёт частичные суммы



Подслой FFN: деление по промежуточной размерности, проекция вниз даёт частичные суммы

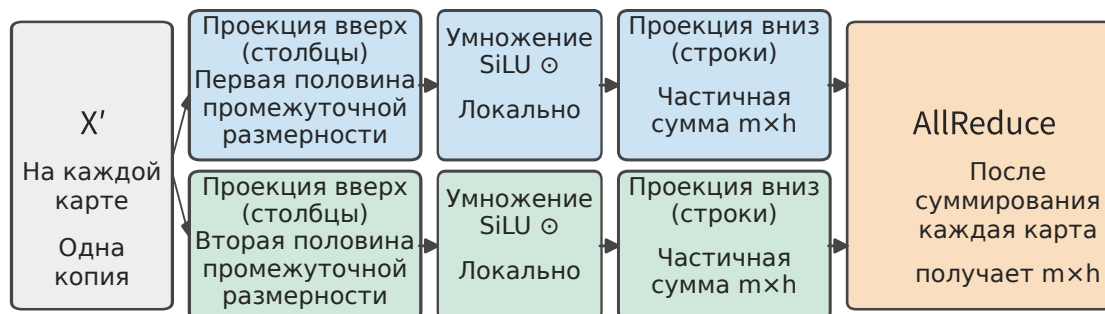


Рис. 6-5. Головы внимания распределены между двумя картами, а промежуточное измерение сети прямого распространения разделено между двумя картами. Последняя проекция в обоих местах даёт лишь частичную сумму, поэтому для сложения частичных сумм двух карт каждый раз требуется один AllReduce. m — число токенов, обрабатываемых за один раз, h — скрытая размерность.

зультатами. На рис. 6-5 показаны две точки разделения внутри слоя.

Как длинный контекст вынуждает распределять плотную модель между картами. BF16-веса плотной модели Qwen3-32B занимают около 65,52 GB и помещаются на одной H100 с 80 GB памяти. Модель изначально поддерживает контекст длиной 32K токенов, а после четырёхкратной экстраполяции позиционного кодирования с помощью YaRN длина достигает 128K.⁴ KV-кэш одного токена занимает $2 \times 64 \times 8 \times 128 \times 2 = 256$ KiB, поэтому для одной сессии 128K требуется сохранить около 34,36 GB. При развёртывании на одной карте веса, KV-кэш одной сессии 128K и рабочая область объёмом 2 GiB вместе занимают около 102,03 GB, что превышает 80 GB. При распределении на две карты на каждую приходится около 52,09 GB, и система вмещает две такие сессии; на четырёх картах на каждую приходится около 27,12 GB, и система вмещает семь сессий; на восьми картах помещается шестнадцать сессий. В этом случае тензорный параллелизм прежде всего нужен ради ёмкости: TP1 (число после TP обозначает количество карт в

⁴В таблице 1 технического отчёта Qwen3 для Qwen3-32B указаны 64 слоя, 64/8 голов запросов/KV и контекст 128K; в §3.2 сказано, что длина последовательности при предварительном обучении составляет 32,768, а во время инференса YaRN и DCA увеличивают поддерживаемую длину последовательности в четыре раза.

тензорном параллелизме, то есть TP1 — одна карта) не вмещает даже одну сессию 128K.

Однако некоторые модели используют новые механизмы внимания, уменьшающие требования к хранению KV-кэша при длинном контексте. Объём KV-кэша определяется устройством внимания: числом слоёв, количеством KV-голов, размерностью головы и тем, требуется ли сохранять KV на каждом слое. В следующей таблице сравниваются три модели близкого масштаба: плотная Qwen3-32B; МоЕ-модель того же поколения Qwen3-30B-A3B, по-прежнему использующая обычный GQA с 48 слоями, 4 KV-головами и размерностью головы 128; а также МоЕ-модель Qwen3.6-35B-A3B с гибридным вниманием, у которой лишь 10 из 40 слоёв используют полное внимание — по 2 KV-голова с размерностью 256 на слой, тогда как остальные 30 слоёв используют линейное внимание и сохраняют только рекуррентное состояние фиксированного размера.

Показатель	Qwen3-32B	Qwen3-30B-A3B	Qwen3.6-35B-A3B
Постоянно размещённые BF16-веса	65,52 GB	61,06 GB	69,32 GB
KV-кэш на токен	256 KiB	96 KiB	20 KiB (только 10 слоёв полного внимания)
Фиксированное состояние одного запроса	—	—	62,9 MB рекуррентного состояния (FP32) + 2,0 MB состояния свёртки
Состояние одного запроса 128K	34,36 GB	12,88 GB	2,75 GB
Чтение весов за шаг (batch 1, контекст 32K)	63,97 GB	6,08 GB	5,89 GB
Чтение весов за шаг (batch 64, контекст 32K)	63,97 GB	60,44 GB	68,30 GB
Матричные вычисления за шаг (batch 64, контекст 32K)	8,49 TFLOPs	2,04 TFLOPs	0,73 TFLOPs
Чтение состояния за шаг (batch 64, контекст 32K)	549,8 GB	206,2 GB	47,1 GB
Число запросов 128K/32K на одной H100	0/1	1/5	3/11
Число запросов 128K/32K на двух H100	2/10	7/29	31/117

Из таблицы следуют три вывода. Во-первых, при batch 1 обе МоЕ-модели на каждом шаге считывают лишь около одной десятой весов; при batch 64 эксперты, выбранные 64 токенами, охватывают почти всех экспертов, поэтому объём чтения весов возвращается к 60–68 GB, что сопоставимо с 64 GB у плотной модели. Во-вторых, объём матричных вычислений всегда составляет лишь от 1/12 до 1/4 объёма плотной модели — это постоянное преимущество МоЕ. В-третьих, при длинном контексте и большом batch чтение состояния становится основной частью каждого шага: при 3350 GB/s для batch 64 и контекста 32K чтение состояния трёх моделей занимает соответственно около 164, 62 и 14 ms, тогда как чтение весов требует лишь 18–20 ms. Эта разница обусловлена устройством внимания, а не МоЕ: хотя Qwen3-30B-A3B также является МоЕ-моделью, для каждого то-

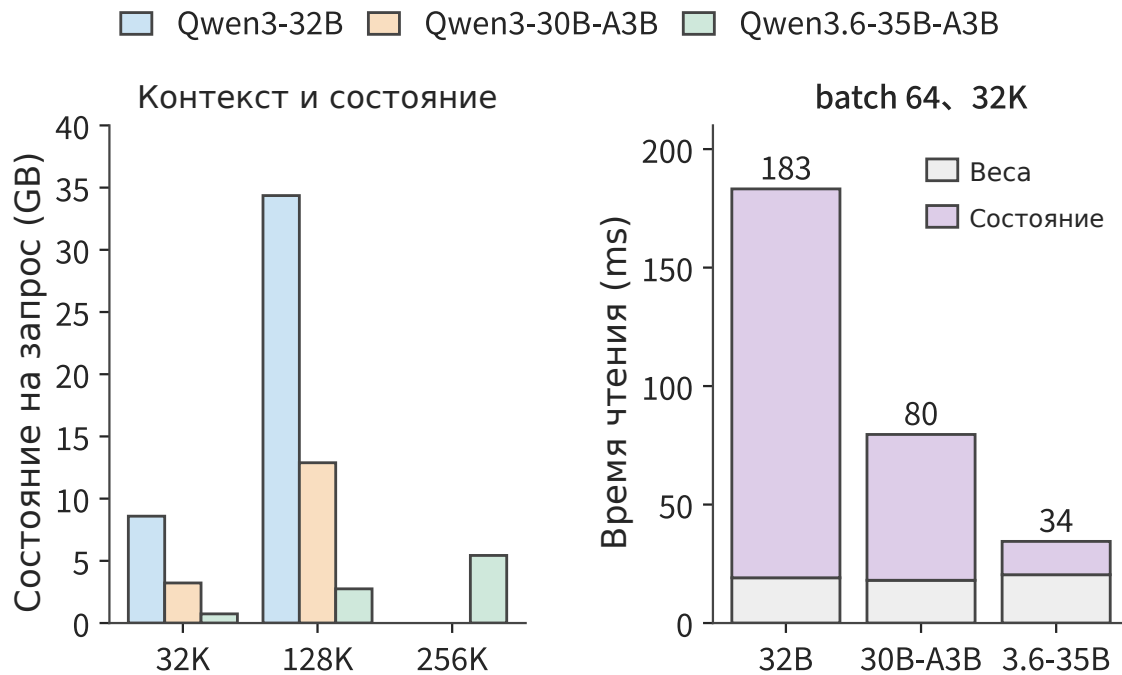


Рис. 6-6. Слева показано состояние одного запроса при контексте 32K, 128K и 256K; предел контекста Qwen3-32B и Qwen3-30B-A3B равен 128K. Справа показано время чтения весов и состояния за один decode-шаг при batch 64 и контексте 32K, рассчитанное для пропускной способности H100 в 3350 GB/s. Объем чтения весов у трёх моделей близок, тогда как объем чтения состояния различается более чем в десять раз.

кена ей всё равно требуется сохранять 96 KiB KV-кэша.

Соответственно различается и ёмкость. Если на каждой карте зарезервировать 2 GiB рабочей области, одна H100 вмещает соответственно 0, 1 и 3 запроса 128K, а две H100 — 2, 7 и 31 запрос. Qwen3.6-35B-A3B изначально поддерживает 262 144 токена, причём состояние одного такого запроса занимает лишь около 5,43 GB. Поэтому необходимость многокарточного TP для сервиса масштаба 30B с контекстом 128K определяется прежде всего устройством внимания, а не тем, является ли модель плотной или МоЕ-моделью. Такие модели, как DeepSeek V4-Flash, дополнительно сжимают KV-кэш; в разделе 2.6.1 состояние и вычисления пяти моделей сравниваются при контексте 8K, 200K и 1M, поэтому здесь мы не будем повторять это сравнение.⁵

Вернёмся к двум точкам разделения внутри слоя, показанным на рис. 6-5. На последнем шаге в обеих точках получается лишь частичная сумма, поэтому необходимо с помощью AllReduce сложить частичные суммы всех карт, а затем предоставить каждой карте итоговую сумму. В каждом слое Qwen3-32B это требуется дважды, то есть 128 раз для 64 слоёв. Это число будет неоднократно использоваться далее при расчёте времени выполнения. Чтобы понять, почему при одном способе разделения достаточно конкатенации, а при другом обязательно требуется суммирование, рассмотрим минимальное матричное умножение. Когда его совместно выполняют две карты, сначала нужно определить,

⁵Qwen3-32B, batch 1 и batch 64, Qwen3-30B-A3B, batch 1 и batch 64 с равномерной маршрутизацией, Qwen3.6-35B-A3B, batch 1 и batch 64, состояние Qwen3.6 при 128K и состояние при 256K. Число помещающихся запросов рассчитывается как $\lfloor (n \times (80 \text{ GB} - 2 \text{ GiB}) - W) / S \rfloor$, где n — число ускорителей, W — постоянно размещённые в памяти веса, а S — состояние на запрос; сводные данные приведены в поле dense_тое результатов расчётов для главы 6. Чтение состояния Qwen3.6 включает KV-кэш слоёв полного внимания, рекуррентное состояние слоёв линейного внимания и состояние свёртки.

То же умножение: $[2, 3] \times [[1, 4], [2, 5]]$

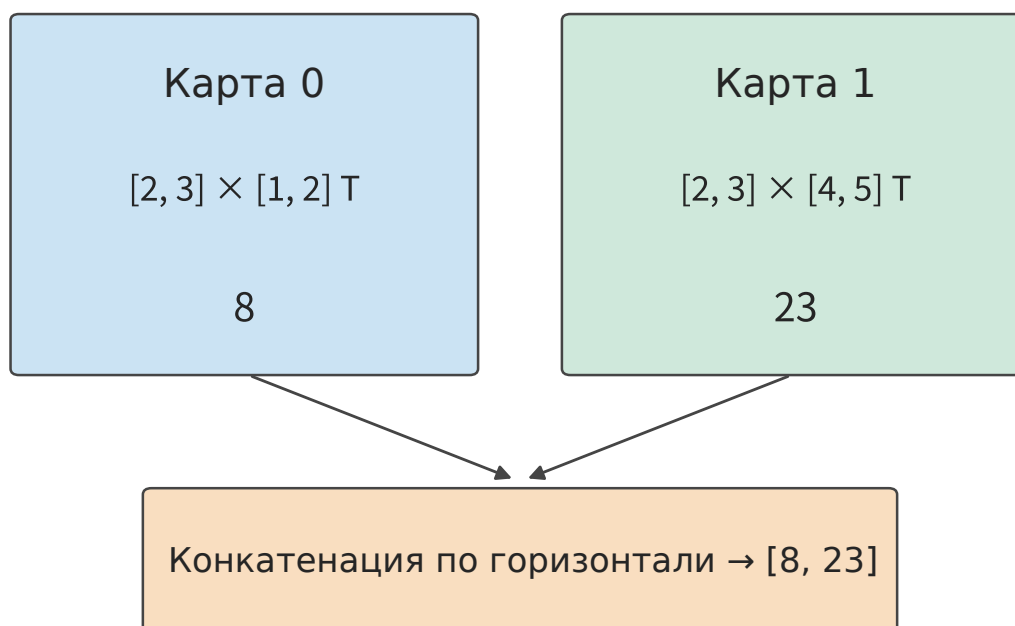


Рис. 6-7. При разделении матрицы весов по столбцам каждая карта использует полный вход и вычисляет разные элементы результата. Символ транспонирования T внутри ячейки означает, что записанные горизонтально в квадратных скобках числа следует рассматривать как вектор-столбец.

что вычисляет каждая из них: карты могут вычислять разные элементы результата либо каждая может вычислять часть одного и того же результата. В первом случае результаты требуется объединить, во втором — сложить.

Рассмотрим оба способа разделения на полном примере. Пусть вход равен $x = [2, 3]$, матрица весов — $W = \begin{bmatrix} 1 & 4 \\ 2 & 5 \end{bmatrix}$, а результат — $xW = [8, 23]$. Разделим матрицу весов по столбцам, то есть распределим выходные признаки между разными картами: карта 0 хранит первый столбец и получает первый элемент результата 8; карта 1 хранит второй столбец и получает второй элемент 23. Конкатенация этих двух элементов даёт полный результат.

Теперь разделим матрицу весов по строкам, то есть распределим входные признаки одного токена между разными картами: карта 0 умножает входное значение 2 на первую строку и получает $[2, 8]$; карта 1 умножает входное значение 3 на вторую строку и получает $[6, 15]$. Обе карты получают вектор из двух элементов, но каждый вектор содержит вклад лишь одной строки весов. Полный результат $[8, 23]$ получается только после поэлементного сложения.

В FFN эти два способа разделения можно соединить последовательно, как показано в нижней части рис. 6-5. Здесь используется SwiGLU, представленная в главе 2: вход передаётся в управляющую и повышающую проекции, к результату управляющей проекции

То же умножение: $[2, 3] \times [[1, 4], [2, 5]]$

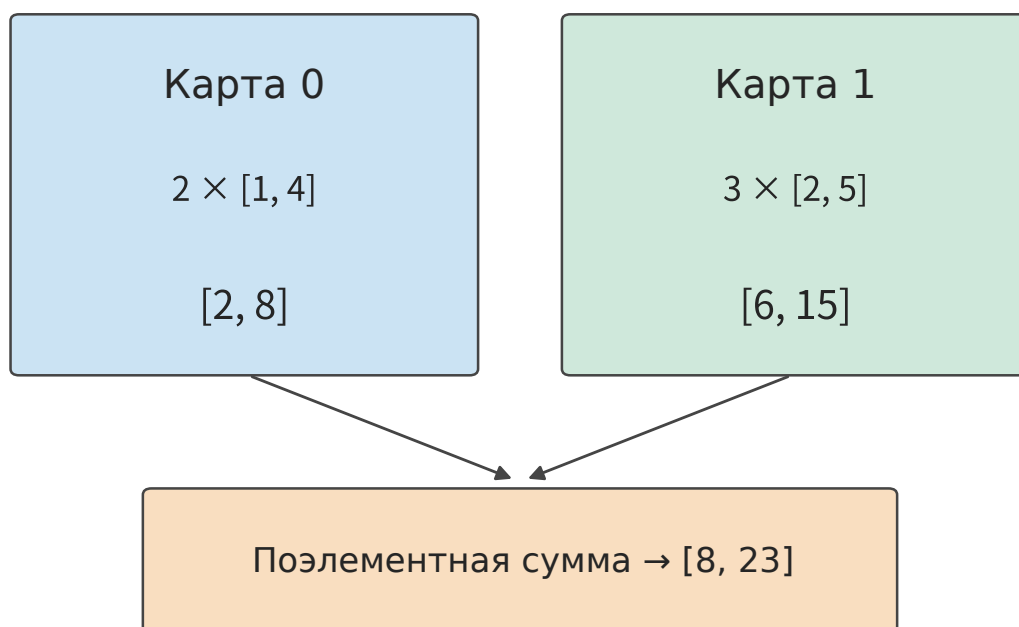


Рис. 6-8. При разделении матрицы весов по строкам две карты создают частичные суммы результата одинаковой формы. Поэлементное сложение восстанавливает полный результат умножения.

применяется функция активации SiLU, затем результаты двух ветвей поэлементно перемножаются, после чего выполняется понижающая проекция. Пусть одновременно обрабатывается m токенов, скрытая размерность равна h , а промежуточная размерность — f . Вход имеет вид $X \in \mathbb{R}^{m \times h}$, повышающие проекции — $W_g, W_u \in \mathbb{R}^{h \times f}$, а понижающая проекция — $W_d \in \mathbb{R}^{f \times h}$:

$$Y = [\text{SiLU}(XW_g) \odot (XW_u)] W_d.$$

Разделим промежуточную размерность f на левую и правую половины. Карта 0 хранит столбцы левых половин двух матриц повышающих проекций и строки верхней половины матрицы понижающей проекции, а карта 1 — столбцы правых половин двух матриц повышающих проекций и строки нижней половины матрицы понижающей проекции. Обе карты хранят полный вход X и получают соответственно промежуточные активации Z_0, Z_1 . SiLU и умножение выполняются поэлементно, поэтому каждая карта может непосредственно обработать свою половину.

Понижающая проекция соединяется вдоль той же промежуточной размерности:

$$Y = [Z_0 \ Z_1] \begin{bmatrix} W_{d,0} \\ W_{d,1} \end{bmatrix} = Z_0 W_{d,0} + Z_1 W_{d,1}. \quad (6-3)$$

Оба произведения в правой части имеют форму $m \times h$, и каждое содержит вклад лишь половины компонентов промежуточной размерности. Только после их сложения получается вход, необходимый следующему слою.

Если полный результат нужен обоим картам, выполняется AllReduce. Если следующий оператор может продолжить работу с разделёнными данными, выполняется **ReduceScatter (редукция с распределением)**: сначала данные суммируются, а затем разные фрагменты суммы остаются на разных картах. Для восстановления полных данных из фрагментов выполняется **AllGather (полный сбор)**, который объединяет фрагменты, хранящиеся на всех картах.

Пример: насколько разделение между четырьмя картами сокращает чтение весов и насколько увеличивает обмен данными? Возьмём $h = 5120$, $f = 25600$ модели Qwen3-32B и поровну разделим промежуточную размерность между четырьмя картами.

Решение: на каждой карте матрицы управляющей и повышающей проекций имеют форму 5120×6400 , а понижающая — 6400×5120 . Три матрицы вместе занимают 187,5 MiB, то есть четверть от исходных 750 MiB. При 3350 GB/s время чтения для одного токена сокращается примерно с 234,8 μs до 58,7 μs : каждая карта считывает на 562,5 MiB меньше, что экономит около 176,1 μs . Цена этого — необходимость сложить частичные суммы результата объёмом 10 KiB с каждой из четырёх карт. В разделе 6.4 время такого объединения будет рассчитано с помощью конкретных алгоритмов.

Внимание также можно разделять по головам: проекции Q, K и V создают только головы, за которые отвечает текущая карта, а каждая карта независимо вычисляет внимание для этих голов; выходная проекция отображает результаты голов обратно в скрытую размерность, однако каждая карта по-прежнему получает лишь частичную сумму, и полный результат возникает только после сложения. Поэтому на каждом слое Qwen3-32B выполняется по одной редукции для результата внимания и FFN, всего 128 редукций для 64 слоёв. Эмбединги слов разделяются по словарю; выходная проекция на словарь также распределяется между картами по словарю.

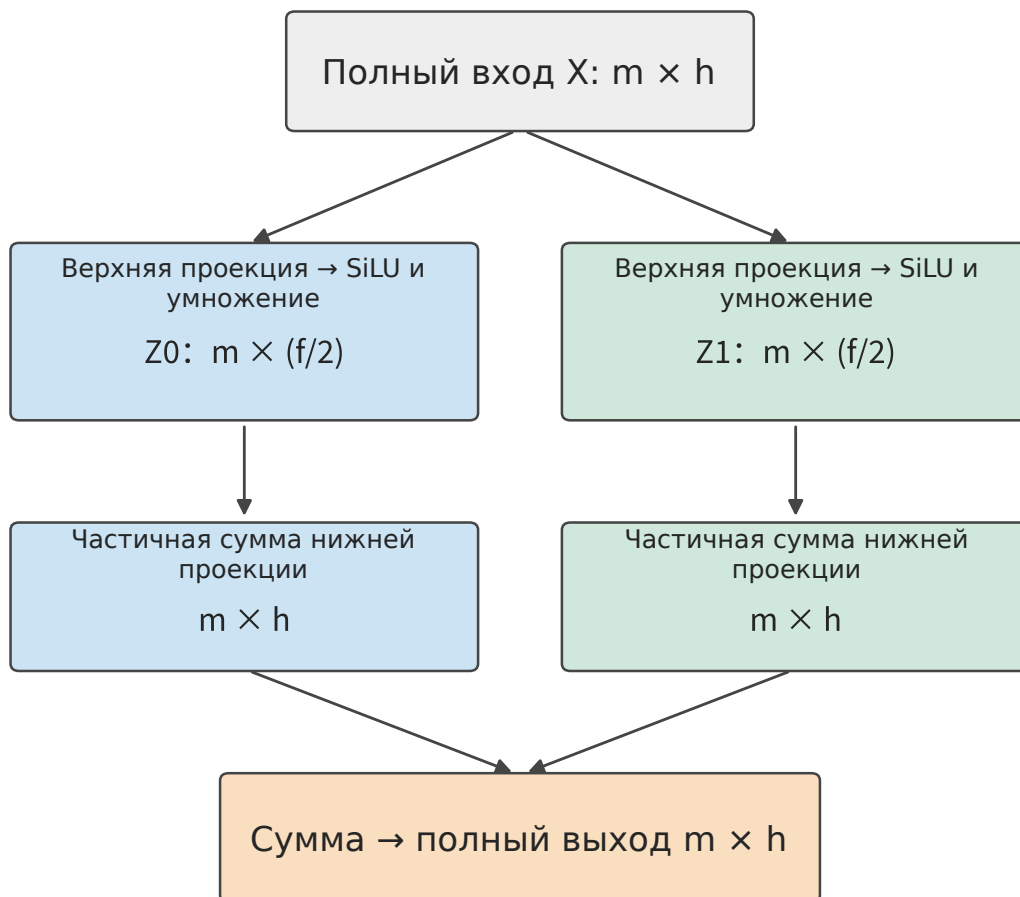


Рис. 6-9. Разделение SwiGLU между двумя картами. В повышающих проекциях выходные признаки разделяются по столбцам матрицы весов, а поэлементные операции выполняются локально; в понижающей проекции входные признаки разделяются по строкам матрицы весов. На обоих шагах сохраняются все m токенов, а разделяется размерность признаков каждого токена. Промежуточная активация на каждой карте имеет форму $m \times (f/2)$, а частичная сумма результата по-прежнему имеет форму $m \times h$; в конце выполняется поэлементное суммирование.

Голову нельзя разделить дальше: это минимальная единица распределения. У Qwen3-32B есть 8 KV-голов, поэтому при TP8 на каждую карту приходится по одной. При TP16 запросные головы, совместно использующие одну KV-голову, распределяются между двумя картами, и обе карты должны хранить состояние этой KV-головы. При контексте 128K состояние каждой KV-головы занимает 4 GiB: суммарно 32 GiB на восьми картах и 64 GiB на шестнадцати. У Qwen3-235B-A22B всего 4 KV-головы, поэтому при распределении по восьми картам из раздела 6.1.2 каждая KV-голова уже хранится в двух экземплярах на двух картах. Как только число карт TP превышает количество KV-голов, дальнейшее увеличение числа карт приводит к дублированию KV-кэша.

В этой главе «числом экземпляров» называется количество независимых сервисов инференса, а p обозначает число карт TP в одном экземпляре. При наличии восьми карт число экземпляров равно $8/p$. Далее сначала вычисляется время $T(p)$, за которое один экземпляр генерирует один токен, а затем оно пересчитывается во время завершения группы запросов.

Вывод времени полного decode-шага из вычислений одного слоя. Рассмотрим одну сессию 128K модели Qwen3-32B. На каждом шаге веса каждой проекции один раз считываются из HBM, старый KV-кэш один раз считывается, а данные одного нового токена записываются. BF16-веса проекций внимания занимают

$$2(2 \times 5120 \times 8192 + 2 \times 5120 \times 1024) = 180 \text{ MiB}.$$

Здесь два слагаемых 5120×8192 соответствуют Q и выходной проекции, а два 5120×1024 — K и V. Если добавить 750 MiB весов FFN на слой, получится 930 MiB. При контексте из 131 064 токенов объём считываемого KV-кэша на каждом слое составляет около 512 MiB. Словарь содержит 151936 элементов, поэтому веса выходной проекции занимают около 1,56 GB. Таким образом, общий объём обращений к HBM за шаг можно записать как

$$V(s) = 64 \times 930 \text{ MiB} + 256 \text{ KiB} \times (s + 1) + 151936 \times 5120 \times 2 \text{ bytes}, \quad (6-4)$$

где s — длина контекста перед началом текущего шага. При $s = 131\,064$ получается около 98,32 GB, из которых 63,97 GB приходится на веса, а 34,36 GB — на KV-кэш. При пропускной способности H100 в 3350 GB/s чтение занимает около 29,35 ms. Матричные вычисления на том же шаге составляют около 0,34 TFLOPs и при 989,4 TFLOP/s занимают лишь около 0,34 ms, поэтому время выполнения внимания и FFN на каждом слое ограничено пропускной способностью памяти. При TP на p карт ($p \leq 8$) веса и KV-кэш поровну распределяются между картами, поэтому эта часть занимает $29,35/p$ ms. Нормализация, поэлементные операции и сэмплирование читают и записывают лишь несколько скрытых векторов, поэтому здесь они не учитываются.⁶ Вычисления и редукции выполняются последовательно, что даёт следующую модель времени выполнения, используемую в последующих разделах:

⁶Сквозная модель и пошаговые расчёты приведены в описании модели, скрипте расчёта и полных результатах. Конфигурация Qwen3-32B взята из зафиксированных настроек, а объём HBM, пропускная способность и пиковая производительность матричных вычислений H100 — из таблицы оборудования. Время матричного оператора определяется как максимальное из времени доступа к памяти и времени вычисления, причём оба рассчитаны по пиковым значениям; коммуникации выполняются последовательно по слоям, а контекстный KV-кэш растёт на каждом шаге.

$$T(p) = \frac{T_{\text{local}}}{p} + 128 T_{\text{AR}}(p), \quad T_{\text{local}} \approx 29.35 \text{ ms.} \quad (6-5)$$

Здесь $T_{\text{AR}}(p)$ — время одного AllReduce на p картах. Формула (6-5) разделяет влияние увеличения числа карт ТР на две составляющие: локальное время выполнения уменьшается с ростом числа карт, а время обмена данными зависит от алгоритма и способа соединения. При удвоении числа карт с p до $2p$ экономия локального времени составляет $T_{\text{local}}/(2p)$. Следовательно, увеличение числа карт сокращает время выполнения при условии

$$128 [T_{\text{AR}}(2p) - T_{\text{AR}}(p)] < \frac{T_{\text{local}}}{2p}. \quad (6-6)$$

Чем больше p , тем меньше правая часть. Даже если каждая редукция становится медленнее лишь на один и тот же небольшой интервал, накопленная задержка может свести на нет сокращение локального времени выполнения.

При большем числе карт линейный слой также можно одновременно разделить по двум измерениям матрицы, расположив карты в группы обмена данными по строкам и столбцам. Рассмотрим проекцию $[8192, 4096] \times [4096, 12288]$: разделим результат на 16 частей и разместим их на 16 картах, каждая из которых изначально содержит лишь фрагмент входа объёмом 4 MiB и фрагмент весов объёмом 6 MiB. При размещении карт в сетке 4×4 каждая карта должна собрать по горизонтали фрагменты входа с трёх других карт той же строки, отправив 12 MiB, и собрать по вертикали фрагменты весов с трёх других карт того же столбца, отправив 18 MiB. В сетке 2×8 объёмы по двум направлениям составляют соответственно 28 MiB и 6 MiB, а в сетке 8×2 — 4 MiB и 42 MiB. Если каналы двух направлений независимы и имеют одинаковую скорость, время передачи определяется большим из двух объёмов, поэтому сетка 4×4 оказывается самой быстрой. MeshSlice продолжает разбивать данные на блоки вдоль такой раскладки, конвейерно совмещая сбор данных с матричным умножением; система инференса Arctic переключает распределение работы между разными этапами выполнения, а перенос её состояния рассматривается в главе 9.⁷

6.2.3 Параллелизм по последовательности: разбиение операторов, работающих с отдельными токенами, вдоль последовательности

При тензорном параллелизме каждый GPU хранит только часть матрицы, однако внутри слоя остаются операторы, не выполняющие матричное умножение, например LayerNorm (нормализация слоя), Dropout и остаточное сложение. Эти операторы обрабатывают каждый токен отдельно: результат для каждого токена зависит только от его собственного скрытого вектора. **Параллелизм по последовательности** (sequence parallelism, SP) разбивает работу этих операторов по позициям последовательности: каждый GPU в группе ТР обрабатывает лишь часть позиций и больше не хранит собственную копию полных активаций. В этой книге используется это распространённое определение из фреймворка обучения больших моделей Megatron.⁸

⁷Формы матриц и конвейерные расчёты MeshSlice, состояние и дополнительное резидентное размещение при Shift Parallelism. Частичное перекрытие в MeshSlice является прогнозом; определение SP и границы динамического переключения Arctic взяты из описания зафиксированной реализации.

⁸Korthikanti и др., Reducing Activation Recomputation in Large Transformer Models, tensor parallelism и sequence parallelism в §3; документация Megatron Core по Context Parallelism; Liu и др., Ring Attention with

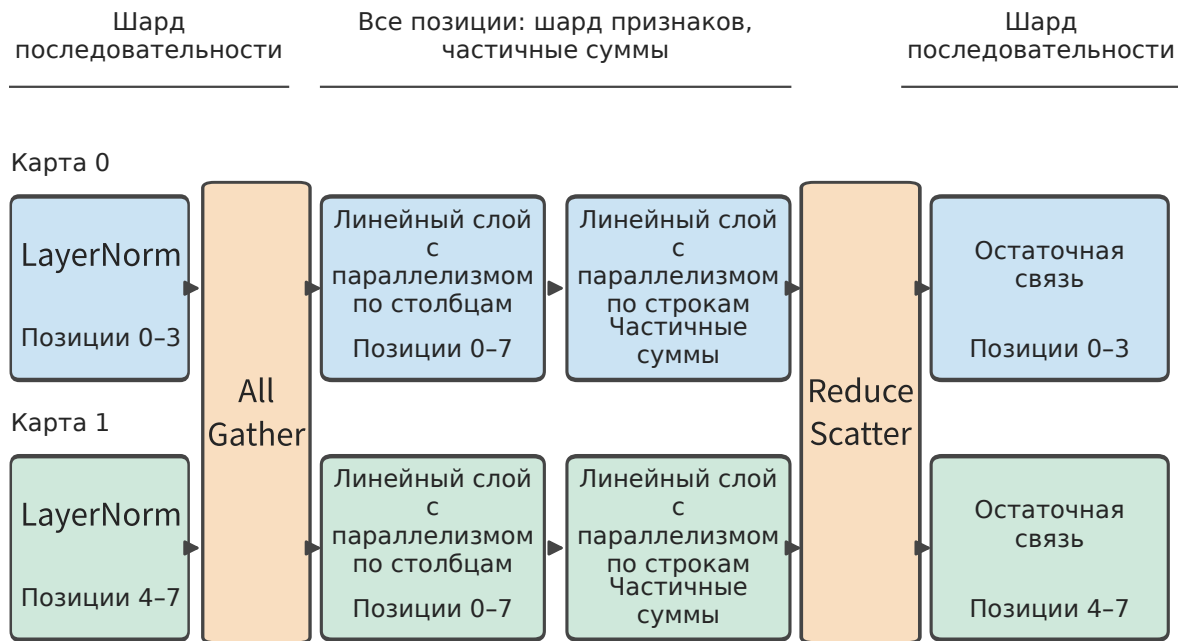


Рисунок 6-10. Для LayerNorm и остаточного сложения нужны только данные текущего токена, поэтому каждый из двух GPU обрабатывает половину позиций; перед входом в линейный слой с параллелизмом по столбцам все позиции собираются с помощью AllGather, а частичные суммы линейного слоя с параллелизмом по строкам суммируются с помощью ReduceScatter и снова разбиваются по позициям.

Линейному слою с параллелизмом по столбцам требуются полные строки входных данных, поэтому перед входом в него фрагменты последовательности со всех GPU сначала собираются с помощью AllGather. После того как линейный слой с параллелизмом по строкам сформирует частичные суммы, ReduceScatter одновременно суммирует их и снова разбивает по позициям. AllReduce по определению эквивалентен последовательности ReduceScatter и AllGather; параллелизм по последовательности лишь размещает эти две части на разных границах операторов, благодаря чему промежуточным операторам, работающим с отдельными токенами, не требуется хранить на каждом GPU полную копию активаций.

Например, BF16-активации размером $[8192, 4096]$ занимают 64 MiB. Если четыре GPU хранят непрерывные фрагменты размером $[2048, 4096]$, каждый GPU использует лишь 16 MiB, а суммарно — 64 MiB; если же каждый из четырёх GPU хранит полный тензор, суммарный объём составляет 256 MiB. Эта экономия достигается только на участке, где сохраняется разбиение последовательности; это не означает, что все активации и веса модели уменьшаются в четыре раза. Соотношение между измерением редукции и выходным измерением остаётся таким же, как на рисунке 6-9; меняется лишь раскладка данных, которую получает следующий оператор. В сопутствующем эксперименте на JAX (фреймворке численных вычислений с поддержкой автоматического дифференцирования и компилируемого выполнения) показаны два варианта прохождения через FFN: с сохранением разбиения и с повторной сборкой данных.⁹

Blockwise Transformers for Near-Infinite Context. В этом разделе SP используется в специфическом значении из Megatron, а пример CP с восемью позициями выведен в этой книге.

⁹Восемь итоговых конфигураций прошли проверку по эталону FP64; выходы парных TP/SP побитно совпали. Полные формы тензоров и критерии приёмки приведены в материале «Эксперимент 6-2: фактический

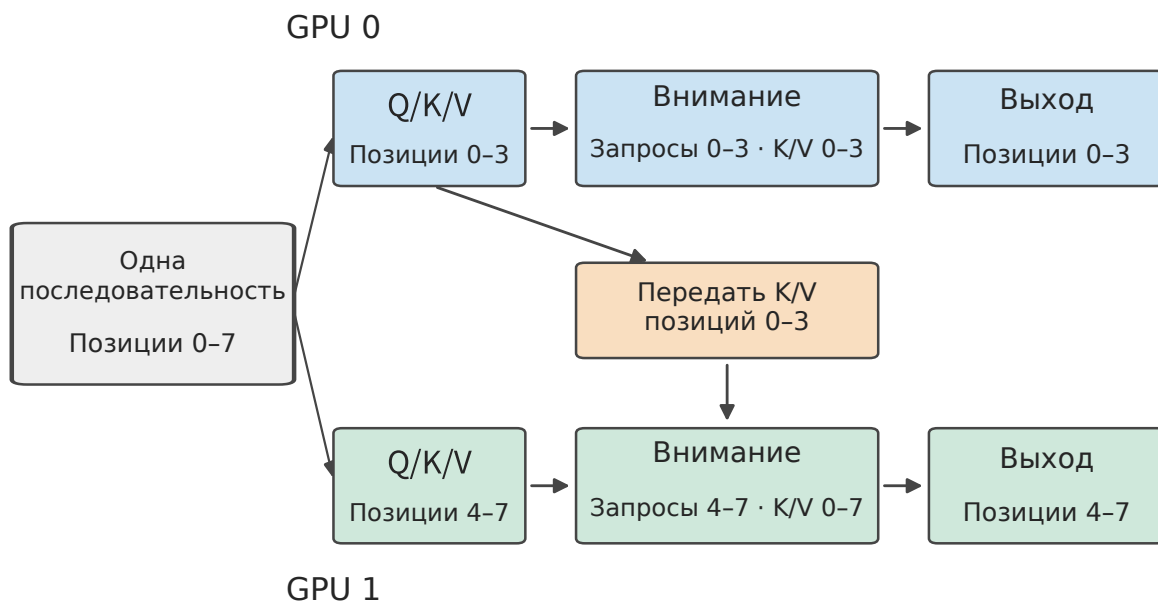


Рис. 6-11. Каждая карта хранит Q, K и V для своего участка позиций. При причинном внимании запросам карты 1 также нужны K и V карты 0, поэтому K и V необходимо передавать между картами; при неперчинном внимании передача требуется в обоих направлениях.

6.2.4 Параллелизм контекста: разделение внимания одной последовательности

Параллелизм по последовательности охватывает только операции, выполняемые отдельно для каждого токена, тогда как при вычислении внимания каждая позиция должна видеть другие позиции последовательности. **Параллелизм контекста** (context parallelism, CP) распределяет позиции одной длинной последовательности между несколькими картами. Каждая карта хранит Q, K и V для своего участка позиций и вычисляет для него выход внимания. На рис. 6-11 разделение работы между двумя картами показано на примере восьми позиций.

Запросам на текущей карте по-прежнему нужны удалённые K и V в пределах, разрешённых маской. Можно сначала собрать все K и V либо передавать блоки K и V по кольцу между картами и по мере их поступления с помощью онлайн-softmax из раздела 5.3.3 накапливать максимум, сумму экспонент и взвешенный выход. Кольцевое внимание (Ring Attention) — это способ планирования и передачи данных, а параллелизм контекста — способ разделения работы; это не синонимы.¹⁰

На рис. 6-12 эта зависимость представлена как отношения видимости между восемью позициями.

Непрерывное разделение позиций пополам также приводит к неравномерной вычислительной нагрузке. Причинное внимание для восьми позиций содержит всего $1 + \dots + 8 = 36$ пар «запрос — ключ»: на первые четыре позиции приходится лишь 10 пар,

путь FFN с JAX TP/SP». Эксперимент выполнялся на логических устройствах CPU.

¹⁰Korthikanti и др., Reducing Activation Recomputation in Large Transformer Models, tensor parallelism и sequence parallelism в §3; документация Megatron Core по Context Parallelism; Liu и др., Ring Attention with Blockwise Transformers for Near-Infinite Context. В этом разделе SP используется в специфическом значении из Megatron, а пример CP с восемью позициями выведен в этой книге.

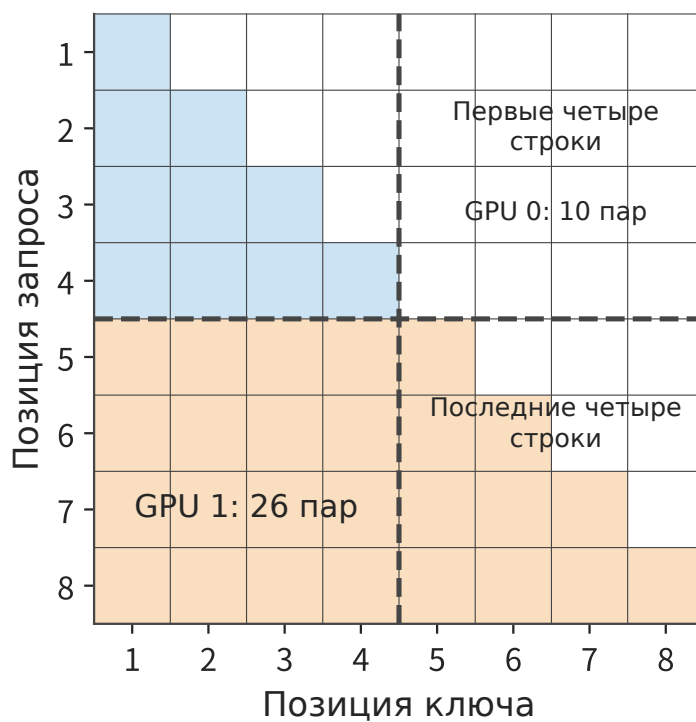


Рис. 6-12. Причинное внимание для восьми токенов: по горизонтальной оси расположены позиции ключей, по вертикальной — позиции запросов. Закрашенные ячейки обозначают пары «запрос — ключ», которые необходимо вычислить; первые четыре строки относятся к карте 0, последние четыре — к карте 1. Пунктирная линия лишь обозначает границу между устройствами, поэтому удалённые зависимости в левой нижней области никуда не исчезают.

а на последние четыре — 26. Хотя количество токенов на двух картах одинаково, объём работы различается в 2,6 раза. Если объединить ранние и поздние позиции, например передать карте 0 позиции 1, 2, 7, 8, а карте 1 — позиции 3, 4, 5, 6, то каждая карта обрабатывает по 18 пар. На практике часто применяется чередующаяся блочная раскладка «зиг-заг», однако при этом необходимо сохранить исходные позиции и причинную маску, а также перестроить обмен данными; удалённый контекст нельзя отбросить как padding.

При Decode запросов для нового токена мало, а историческая последовательность K и V очень длинна. После её разделения по историческим позициям каждая карта сначала вычисляет для своего участка максимум m_r , сумму экспонент l_r и ненормализованный взвешенный вектор u_r , после чего они объединяются:

$$m = \max_r m_r, \quad l = \sum_r e^{m_r - m} l_r, \quad u = \sum_r e^{m_r - m} u_r, \quad o = u/l.$$

o — это выход внимания нового токена по всем историческим позициям. Это тот же способ объединения softmax между tile, что и в главе 5, но статистики и векторы требуется передавать между картами. Нельзя сначала независимо выполнить softmax на каждой карте, а затем просто усреднить выходы карт. Параллелизм контекста сокращает объём состояния контекста, хранящегося на каждой карте, и объём локальных вычислений внимания, но требует обмена K и V либо редукции статистик; зависимости между предыдущими и последующими токенами при генерации от этого не исчезают. В разных фреймворках названия параллелизма по последовательности и параллелизма контекста частично пересекаются, поэтому при анализе конфигурации следует смотреть, какие операции и тензоры разделяются и какая коммуникационная группа используется, а не только на аббревиатуру.

6.2.5 Конвейерный параллелизм: разделение этапов по слоям

Все описанные выше способы распределяют работу внутри одного слоя. **Конвейерный параллелизм** (pipeline parallelism, PP) вместо этого распределяет работу по слоям: слои модели последовательно разделяются на несколько этапов, за каждый из которых отвечает один или несколько ускорителей. На рис. 6-13 64 слоя Qwen3-32B разделены на два этапа.

Когда предыдущий этап завершает вычисления для относящихся к нему слоёв, он передаёт активации следующему этапу, который продолжает вычисления. Каждый этап хранит только веса своих слоёв, а между этапами передаются только промежуточные результаты.

Входные данные подаются в конвейер в виде микробатчей: один batch разделяется на несколько micro-batch, благодаря чему разные этапы могут одновременно обрабатывать разные micro-batch. Пусть модель разделена на q этапов, время выполнения каждого этапа, включая передачу активаций, равно t , а входные данные состоят из b уже готовых и не зависящих друг от друга micro-batch. Рассмотрим случай с четырьмя этапами, каждый из которых выполняется за 1 ms. micro-batch 0 поступает на этап 0 в момент 0 ms и, пройдя четыре этапа, завершается в момент 4 ms; уже в момент 1 ms этап 0 может принять micro-batch 1, который завершится в момент 5 ms. Остальные micro-batch последовательно следуют за ними, и с интервалом 1 ms завершается один из них.

Первый micro-batch требует qt , а каждый следующий micro-batch увеличивает общее время лишь на t , поэтому

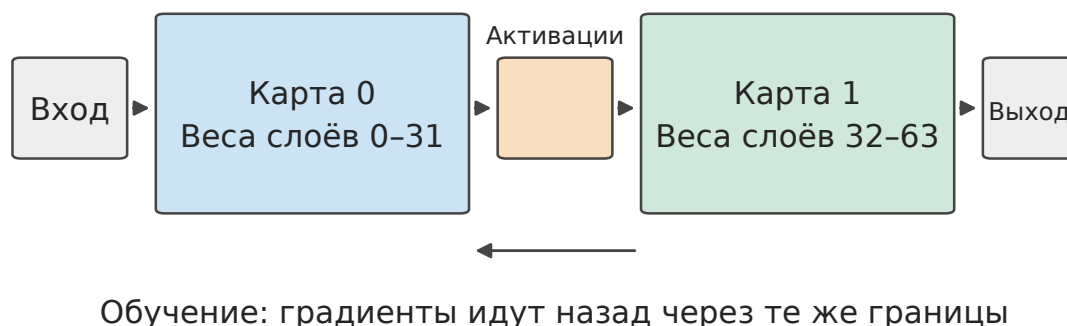


Рис. 6-13. Каждый из двух этапов хранит веса относящихся к нему слоёв. Между соседними этапами передаются только активации; при обучении градиенты передаются в обратном направлении через ту же границу.

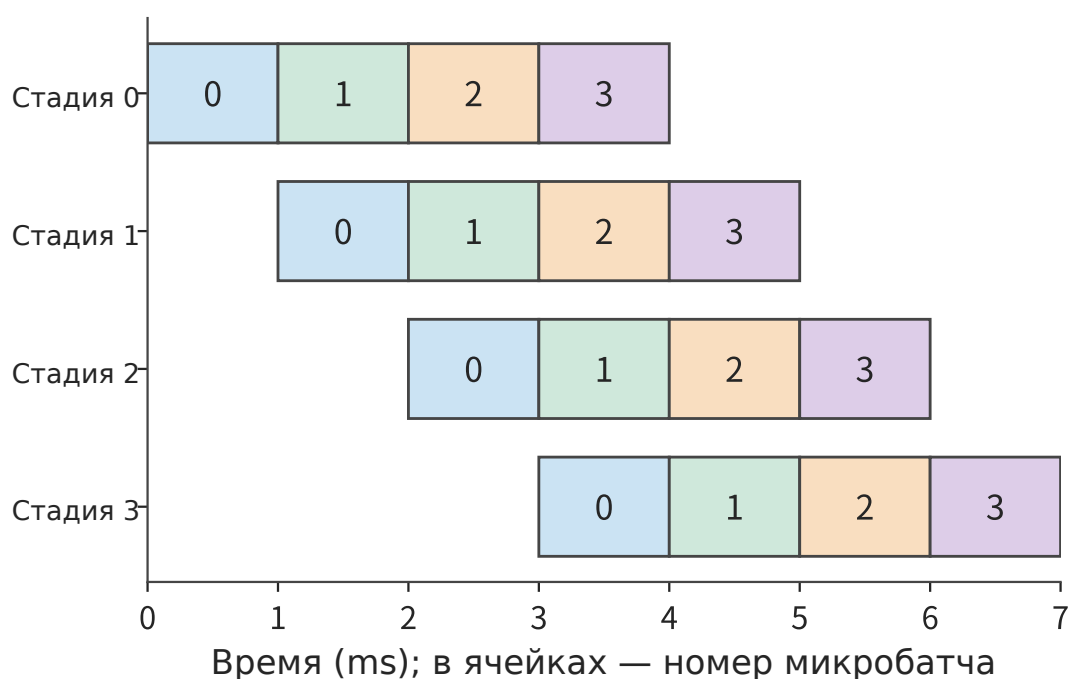


Рис. 6-14. Четыре этапа с одинаковым временем выполнения обрабатывают четыре независимых micro-batch. Каждая ячейка соответствует 1 ms, одинаковый цвет обозначает один и тот же micro-batch. Первый результат появляется в момент 4 ms, последний — в момент 7 ms; пустые области от левого верхнего до правого нижнего угла возникают из-за заполнения и опустошения конвейера.

$$T_{pp} = (q + b - 1)t, \quad \eta_{pp} = \frac{qbt}{q(q + b - 1)t} = \frac{b}{q + b - 1}. \quad (6-7)$$

В числителе указано суммарное время фактической работы всех этапов, а в знаменателе — произведение числа этапов на время обработки всех micro-batch. Для четырёх этапов и четырёх micro-batch коэффициент использования равен $4/7$, то есть около 57 %; при увеличении числа micro-batch до 16 он равен $16/19$, то есть около 84 %. Чем больше независимых micro-batch, тем меньше доля простоя, вызванного заполнением и опустошением конвейера.

Неодинаковое время выполнения этапов также приводит к простоям ускорителей. Если четыре этапа требуют соответственно 1, 1, 2 и 1 ms, первый micro-batch завершится за 5 ms, а затем результаты смогут появляться не чаще одного раза в 2 ms. Входные данные поступают на этап длительностью 2 ms быстрее, чем он успевает их обрабатывать, поэтому очередь постепенно растёт; после заполнения буфера предшествующие этапы вынуждены приостановиться. Разделение этапов с учётом фактического времени выполнения каждого слоя позволяет выравнивать длительность этапов и уменьшить накопление очереди.¹¹

При авторегрессионном сеансе входные данные для следующего шага становятся готовы только после генерации текущего токена. Четыре независимых сеанса могут предоставить четыре micro-batch, тогда как четыре будущих токена одного сеанса образуют последовательную цепочку зависимостей. Поэтому коэффициент использования конвейера напрямую зависит от числа micro-batch, готовых к одновременному запуску.

6.2.6 Экспертный параллелизм: разбиение набора экспертов

Последний способ разбиения применяется к набору экспертов, характерному для МоЕ. В плотной модели для каждого токена используется один и тот же набор весов FFN; в модели МоЕ предусмотрено несколько сетей прямого распространения, выступающих в роли экспертов, а маршрутизатор вычисляет оценки выбора и для каждого токена определяет экспертов, которых необходимо выполнить. Каждый токен использует только выбранных экспертов, однако весь набор экспертов должен храниться на ускорителях, чтобы их могли выбирать последующие токены. Таким образом, МоЕ предоставляет ещё один способ распределения работы: **экспертный параллелизм** (expert parallelism, EP) размещает разных экспертов на разных картах, каждая из которых обрабатывает направленные ей входные данные. На рис. 6-15 восемь экспертов распределены между двумя картами.

При экспертном параллелизме сначала необходимо определить, где будут находиться веса экспертов, и только затем — как выполнять обработку текущего токена.

Рассмотрим такое распределение работы на примере четырёх карт и восьми экспертов. На карте 0 хранятся эксперты 0 и 1, на карте 1 — эксперты 2 и 3 и так далее. В данный момент на карте 0 находится токен А, для которого маршрутизатор выбрал экспертов 1 и 6: эксперт 1 находится на этой же карте, а эксперт 6 — на карте 3. Карта 0 должна не только выполнить локального эксперта 1, но и отправить входные данные А на карту 3, а затем получить оба выходных результата и объединить их с учётом весов маршрутизации.

¹¹Расчёт ограниченного числа слотов и обратного давления, базовые тракты коммуникации TP/PP.

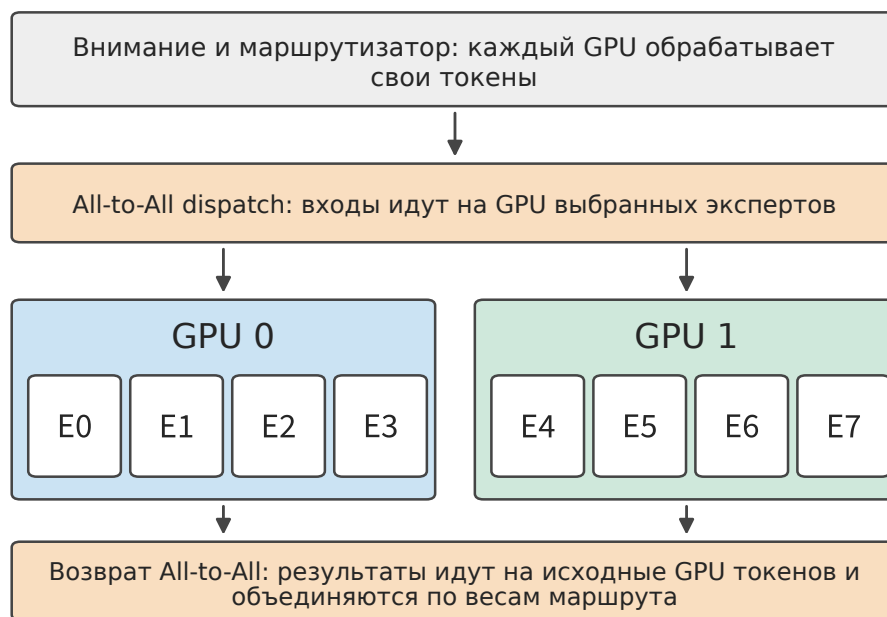


Рис. 6-15. Вычисления внимания и маршрутизатора выполняются на карте, где находится токен; входные данные отправляются на карту с выбранным экспертом, а после вычислений результат эксперта возвращается на исходную карту и объединяется с учётом весов маршрутизации. Оба обмена представляют собой All-to-All (полный обмен), то есть каждая карта отправляет каждой другой карте разные данные.

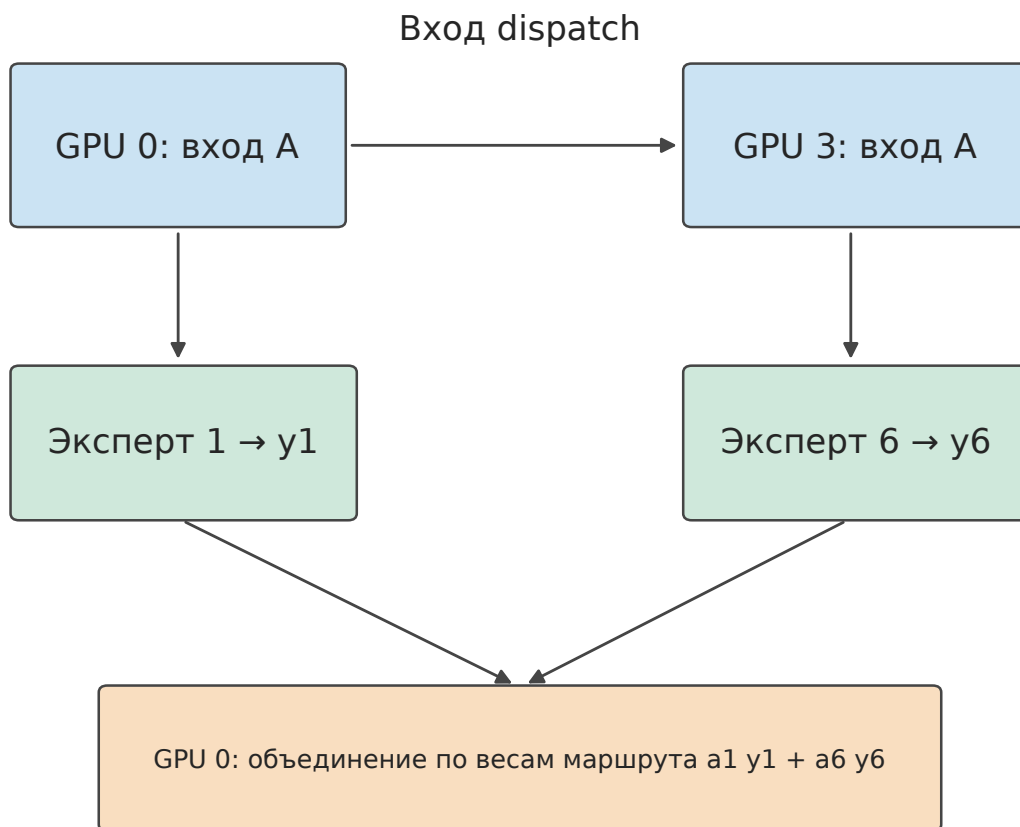


Рис. 6-16. На карте 0 находится токен A, для которого выбраны локальный эксперт 1 и эксперт 6 на карте 3. При dispatch входные данные отправляются по верхнему пути, а при combine выходы экспертов возвращаются по нижнему пути и объединяются с весами. Сколько экспертов выбрано для каждого токена, столько вычислений и требуется выполнить; расположение эксперта определяет, куда следует отправить входные данные.

Процесс отправки входных данных эксперта на карту, где он находится, называется **dispatch (диспетчеризация)**, а процесс возврата выходных данных на исходную карту и их объединения — **combine (объединение)**. Продолжим рассматривать токен A: если выходы двух экспертов равны соответственно y_1 , y_6 , а веса маршрутизации — a_1 , a_6 , то полный выход МоЕ равен $a_1 y_1 + a_6 y_6$. Карта-отправитель сохраняет номер токена и информацию о маршрутизации, карта-получатель выполняет эксперта, после чего возвращённый результат снова сопоставляется с тем же токеном.

Если на одной карте находится несколько экспертов, достаточно отправить одну копию входных данных. Если восемь экспертов, выбранных для одного токена, попарно распределены между четырьмя удалёнными картами, то вход размерности 4096 в формате BF16 занимает 8 KiB: при отправке отдельно каждому эксперту потребуется 64 KiB, а при отправке по одной копии на каждую целевую карту — только 32 KiB, поскольку два эксперта на одной карте совместно используют одну копию входных данных.

При совместной обработке нескольких токенов каждая карта должна отправлять другим картам разное количество данных — это и есть All-to-All на рис. 6-15. Получатель сначала перегруппировывает полученные строки входных данных по экспертам, выполняет матричное умножение, а затем восстанавливает исходный порядок выходных

данных по номерам токенов. Карте, получившей больше всего входных данных, обычно приходится выполнять и больше всего вычислений экспертов.

В Qwen3-235B-A22B этот пример масштабирован до 128 маршрутизируемых экспертов в каждом слое, из которых для каждого токена выбираются 8; всего модель содержит 94 слоя. Каждый эксперт включает три проекции; при скрытой размерности 4096, промежуточной размерности 1536 и весах BF16 их объём составляет $3 \times 4096 \times 1536 \times 2 = 36$ MiB. Все эксперты одного слоя занимают 4,5 GiB, а 94 слоя — около 423 GiB. При равномерном распределении экспертов между восемью картами на каждую приходится около 52,9 GiB; кроме того, каждая карта должна хранить веса слоёв внимания, маршрутизаторов, входного и выходного слоёв, а также KV-кэш и рабочую область. Набор экспертов можно распределить между картами, тогда как головы и состояния внимания хранятся согласно собственной схеме распараллеливания внимания.

Эксперимент 6-2 · Основной: как тензорное разбиение и конвейер микробатчей изменяют время выполнения

- (a) Укажите формы шести фрагментов весов и двух частичных сумм выходов для SwiGLU на двух картах. С помощью формулы (6-3) объясните, почему частичные суммы выходов с двух карт необходимо сложить.
- (b) Для TP1, TP2, TP4 и TP8 рассчитайте время доступа к локальной памяти при обработке одного сеанса Qwen3-32B с контекстом 128K, пока не учитывая затраты на обмен данными; определите, сколько времени экономит каждое удвоение числа карт.
- (c) Возьмите время редукции из раздела 6.4, подставьте его в формулу (6-5), повторно рассчитайте ускорение и объясните причину расхождения.
- (d) Постройте временную диаграмму обработки восьми микробатчей в четыре этапа для двух случаев: когда каждый из четырёх этапов занимает 1 ms и когда этапы занимают соответственно 1, 1, 2 и 1 ms. Сравните время завершения в обоих случаях и укажите, перед каким этапом будут накапливаться микробатчи.
- (e) Усложнённое задание: для одного и того же FFN сравните объём данных, который необходимо передавать между соседними операторами, в двух вариантах — при формировании полного выходного тензора и при сохранении фрагментов выхода. Можно также выбрать соответствующую схему с двумерной сеткой и дополнительно вывести способ разбиения передаваемых данных.

6.3 Комбинирование видов параллелизма и масштаб модели

Шесть подходов редко используются по отдельности. В этом разделе сначала объясняется, как комбинировать эти подходы и нумеровать каждую карту, а затем рассматриваются два вопроса, характерных для MoE: как выбор экспертов для батча токенов влияет на чтение весов и нагрузку каждой карты и как меняются комбинации при дальнейшем увеличении модели.

6.3.1 Схемы комбинирования и координаты устройств

Тензорный и конвейерный параллелизм можно комбинировать: например, разделить восемь карт на четыре конвейерных этапа, внутри каждого из которых две карты исполь-

зуют тензорный параллелизм. Редукция между двумя картами для каждого слоя выполняется внутри этапа, а между соседними этапами передаются активации. Если каждая из двух карт этапа хранит полный скрытый вектор размером 8 KiB и передаёт его соответствующей карте следующего этапа, то между двумя этапами необходимо отправить в сумме 16 KiB. Если между этапами передаются шарды, следующий этап повторно объединяет их в соответствии с требованиями вычислений. Поэтому при комбинировании этих двух видов параллелизма необходимо явно указать, какие шарды выводит каждый этап и какие шарды получает каждая карта следующего этапа.

При комбинировании нескольких подходов сначала каждой карте назначают координаты, а затем для каждого оператора указывают разбиение входных данных, весов и выходных данных, а также группы карт, между которыми требуется обмен данными. Например, схема с параллелизмом данных 2, конвейерным параллелизмом 2 и тензорным параллелизмом 4 обозначается как DP2×PP2×TP4 и занимает $2 \times 2 \times 4 = 16$ карт. Если внутри той же группы TP4 включить параллелизм последовательности, количество карт не изменится, поэтому дополнительно умножать его на SP4 не нужно. Разбиение определяет, кому назначается каждая часть работы, а планирование — порядок готовности, выполнения, обмена данными и освобождения ресурсов. Только вместе они образуют полную схему параллелизма.

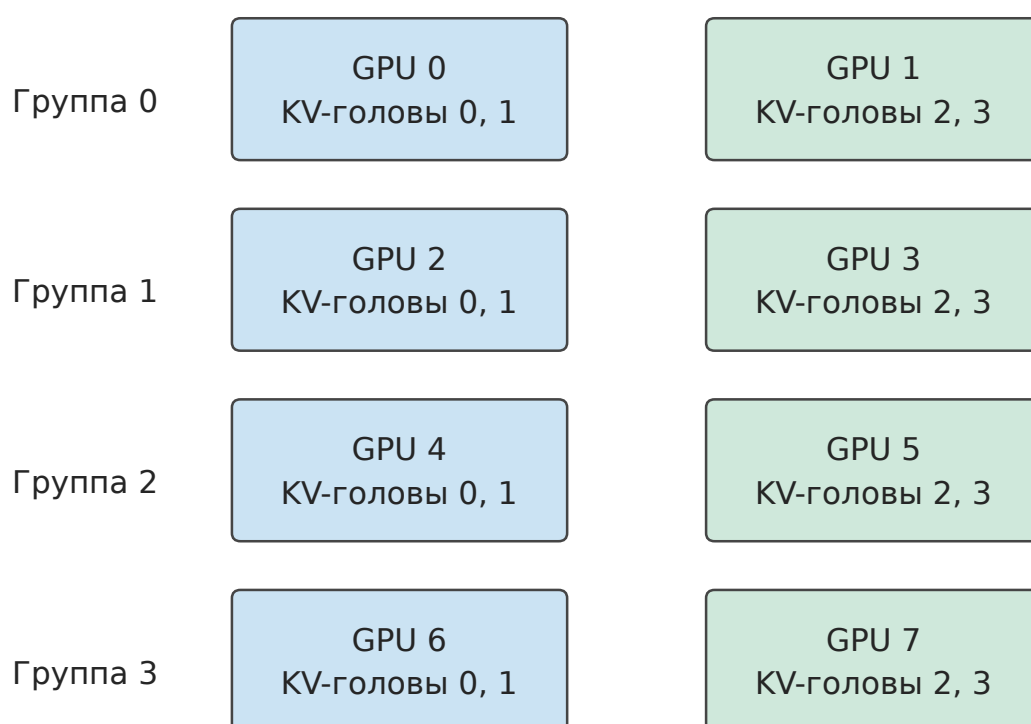
Экспертный параллелизм также можно комбинировать с тензорным параллелизмом внутри эксперта. Например, четыре группы экспертов содержат по 32 эксперта, а внутри каждой группы две карты делят промежуточную размерность экспертов, обрабатывая по 768 измерений каждая. Весь экземпляр по-прежнему занимает восемь карт, и каждая карта идентифицируется двумя номерами: номер группы экспертов указывает, за каких экспертов отвечает карта, а номер тензорного параллелизма внутри группы — какую половину матриц этих экспертов она обрабатывает. Далее обмен данными выводится для этой схемы TP2×EP4.

Входные данные экспертов формируются следующим образом: все четыре группы экспертов выполняют внимание для одного и того же запроса, причём каждая сохраняет KV этого запроса. После завершения внимания каждая группа содержит полный вход экспертов и может непосредственно выбрать строки, необходимые её экспертам, не рассылая входные данные между группами. В конце результаты всех групп объединяются посредством редукции.

Группа экспертов	Карты	Номера экспертов	Промежуточная размерность эксперта на карту	KV-головы на карту
0	0, 1	0—31	768	чётная карта: 0, 1; нечётная карта: 2, 3
1	2, 3	32—63	768	то же
2	4, 5	64—95	768	то же
3	6, 7	96—127	768	то же

После определения размещения данных необходимо проследить, как объединяются выходные данные. Две карты внутри группы вычисляют каждая свою половину промежуточной размерности экспертов, после чего результаты сначала суммируются внутри группы, образуя полный вклад экспертов этой группы. Затем суммируются вклады четырёх групп.

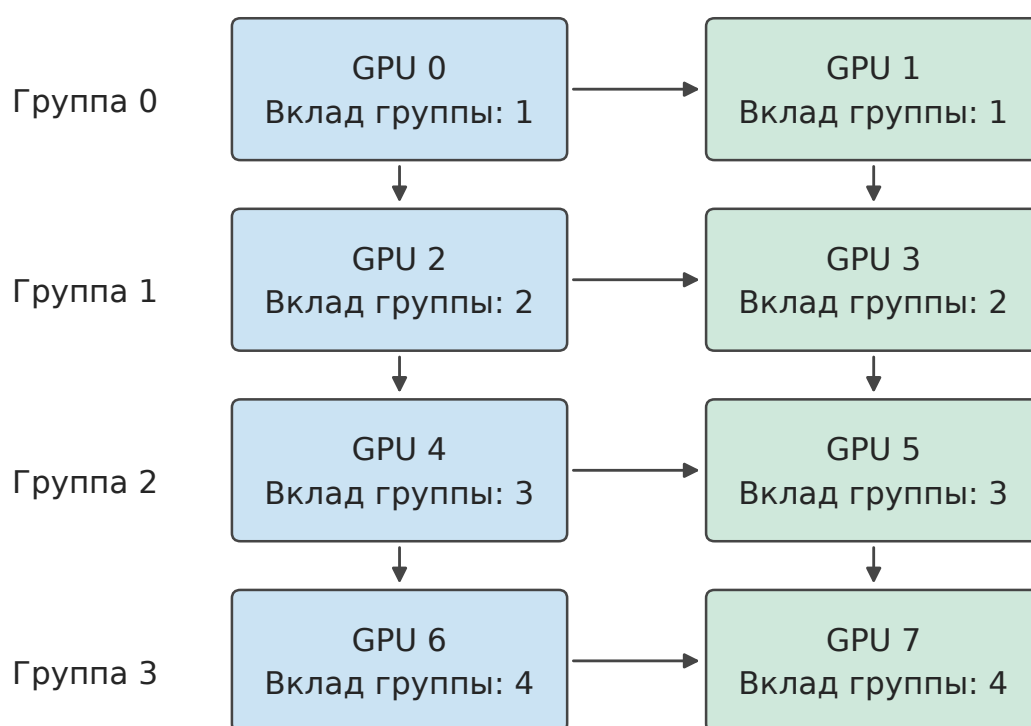
В каждой строке своя группа экспертов, KV дублируется по столбцам



Размерность экспертов строки: слева 768, справа 768

Рисунок 6-17. Восемь карт образуют четыре группы экспертов, в каждой из которых две карты делят работу по промежуточной размерности экспертов. Головы внимания и KV в одном столбце дублируются четыре раза, поэтому каждая группа может локально получить одинаковый вход экспертов.

Вклады экспертов: сумма сначала по строкам, затем по столбцам



Каждый столбец получает полный выход: $1 + 2 + 3 + 4 = 10$

Рисунок 6-18. Горизонтальные стрелки обозначают суммирование результатов двух карт внутри группы, вертикальные — суммирование результатов четырёх групп. Число в ячейке — значение некоторого элемента выхода после суммирования внутри группы. Оба столбца получают одинаковый полный результат 10.

На втором этапе суммирование выполняется между картами с одинаковыми номерами внутри групп. Например, если вклады четырёх групп в некоторый элемент выхода равны соответственно 1, 2, 3 и 4, то одна группа из четырёх чётных карт получает посредством редукции 10, а другая группа из четырёх нечётных карт также получает 10. В этот момент каждая карта содержит одинаковый полный выход и может перейти к вычислению следующего слоя.

Отказ от рассылки входных данных требует дополнительных затрат на хранение и вычисления. В этой схеме каждая карта хранит примерно 64,8 GB весов, а все восемь карт — 518,6 GB, что приблизительно на 48,4 GB больше 470,2 GB, необходимых при хранении каждого параметра только в одном экземпляре. При контексте из 8192 токенов объём KV на каждой карте равен 752 MiB, а на восьми картах — 5,875 GiB, то есть в четыре раза больше фактического объёма KV данного запроса. Каждая группа повторно выполняет внимание, получая взамен локальный вход экспертов.¹²

Пример: какой объём трафика редукции требуется при дублировании внимания и разбиении экспертов? Для редукции используется FP32, поэтому полный скрытый вектор одного токена занимает $4096 \times 4 = 16$ KiB. При редукции между двумя картами кольцевым алгоритмом из раздела 6.4.2 каждая карта отправляет объём одного полного вектора; при редукции между четырьмя картами — в 1,5 раза больше. На каждом слое выполняются три объединения результатов: внутригрупповая редукция внимания, внутригрупповая редукция экспертов и редукция между четырьмя группами экспертов. Поэтому каждая карта отправляет

$$16 + 16 + 24 = 56 \text{ KiB.}$$

В сумме восемь карт отправляют 448 KiB. При prefill для 8192 токенов используется то же распределение работы, и каждая карта отправляет 448 MiB.

6.3.2 Повторное использование внутри батча и дисбаланс нагрузки

В предыдущем подразделе было определено, на каких картах размещаются эксперты, но не было определено, какие карты наиболее загружены на каждом шаге: это зависит от того, какие эксперты выбраны в текущем батче. Один токен выбирает только восемь экспертов, однако батч может охватить весь набор экспертов. Пусть 64 токена выбирают по восемь экспертов, что даёт в общей сложности 512 назначений пар «токен — эксперт».

Если эти назначения равномерно распределены между 128 экспертами, каждый эксперт обрабатывает векторы признаков четырёх токенов. Если все токены выбирают одних и тех же восемь экспертов, каждый эксперт обрабатывает векторы признаков 64 токенов. Полезный объём матричных вычислений одинаков в обоих случаях. Если предположить, что в пределах одного батча веса каждого выбранного эксперта считываются из HBM только один раз, в первом случае потребуется 128×36 MiB, то есть 4,5 GiB, а во втором — 8×36 MiB, то есть 288 MiB. Объёмы чтения различаются в 16 раз.

Чем меньше активных экспертов, тем для большего числа входных строк можно повторно использовать веса каждого эксперта. Это одновременно сокращает повторное чтение и увеличивает количество строк в матричном умножении. Однако время завершения зависит и от того, на какой карте выполняются эти строки. В схеме TP2×EP4 при равномерном распределении каждая группа обрабатывает $32 \times 4 = 128$ назначений.

¹²Упражнение по связям владения в Qwen235 и результаты независимой проверки. В этом примере используется заданная в основном тексте схема TP2×EP4.

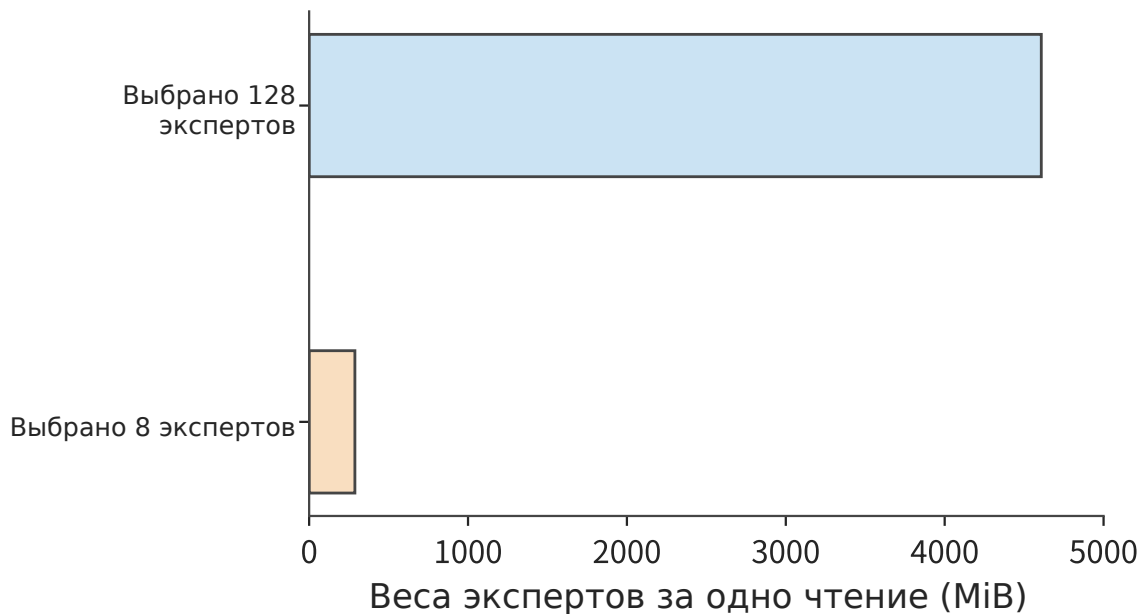


Рисунок 6-19. 64 токена, по восемь экспертов на токен, всего 512 назначений. При равномерном распределении и концентрированном выборе полезный объём вычислений одинаков, однако объём чтения весов выбранных экспертов различается в 16 раз. BF16-веса каждого эксперта занимают 36 MiB и считываются один раз на батч.

Если все восемь активных экспертов находятся в одной группе экспертов, эта группа должна выполнить все 512 назначений — в четыре раза больше, чем при равномерном распределении, — а остальные три группы могут только ждать.

Поэтому для анализа времени выполнения необходимо одновременно вычислять объём чтения весов для всего батча и объём вычислений наиболее загруженной группы экспертов. Небольшие матричные операции decode чаще ограничены скоростью чтения весов, тогда как крупные матричные операции prefill чаще ограничены временем вычислений наиболее загруженной группы экспертов. Распределение популярных экспертов между разными группами EP позволяет сохранить повторное использование внутри батча и одновременно снизить вычислительную нагрузку на отдельную группу экспертов.

В двух последних схемах на рисунках считывается одинаковый объём весов, однако время завершения может различаться, поскольку в вычислениях участвует разное количество карт. Поэтому повторное использование внутри батча следует анализировать совместно с распределением экспертов между картами.

Помимо случаев равномерного покрытия и концентрированного выбора можно вычислить средний результат при равномерном случайном выборе экспертов. Пусть каждый токен выбирает k экспертов из E , а выборы разных токенов независимы. Вероятность того, что один токен не выберет некоторого эксперта, равна $1 - k/E$, а вероятность того, что этот эксперт не будет выбран ни одним из m токенов батча, равна $(1 - k/E)^m$. Для каждого эксперта определим индикаторную переменную: она равна 1, если эксперт выбран внутри батча, и 0 в противном случае. Суммирование этих переменных с последующим вычислением математического ожидания даёт ожидаемое количество выбранных в батче экспертов:

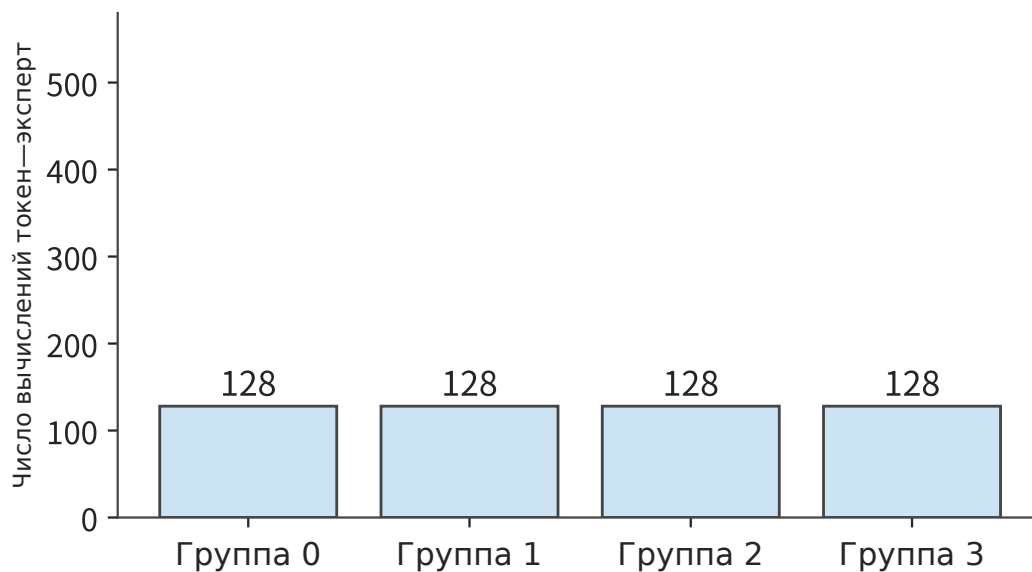


Рисунок 6-20. При равномерном использовании 128 экспертов каждая из четырёх групп EP выполняет 128 вычислений «токен — эксперт», а общий объём чтения весов составляет 4,5 GiB.

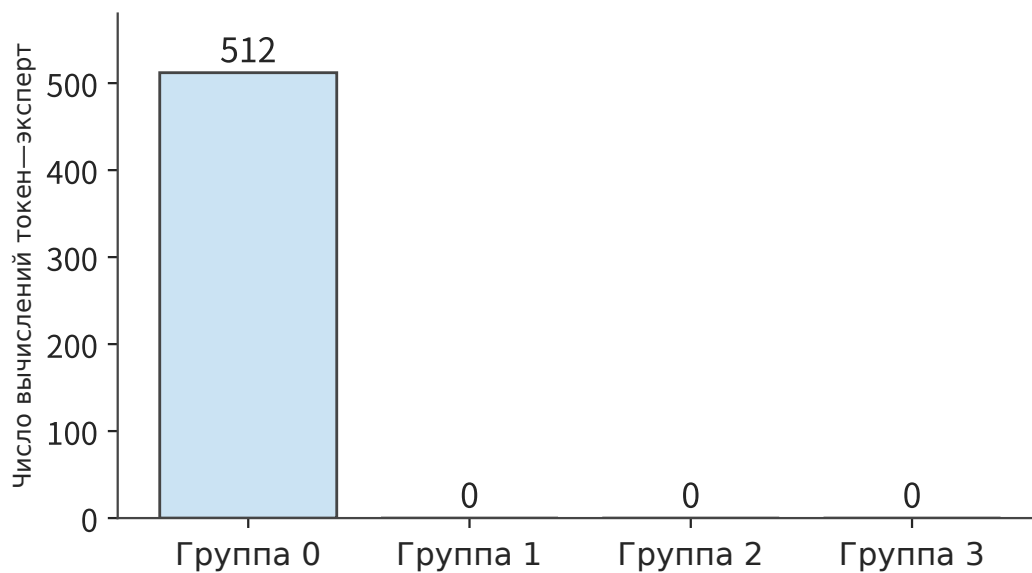


Рисунок 6-21. Все восемь выбранных экспертов находятся в группе 0: объём чтения весов снижается до 288 MiB, однако все 512 вычислений приходятся на одну группу.

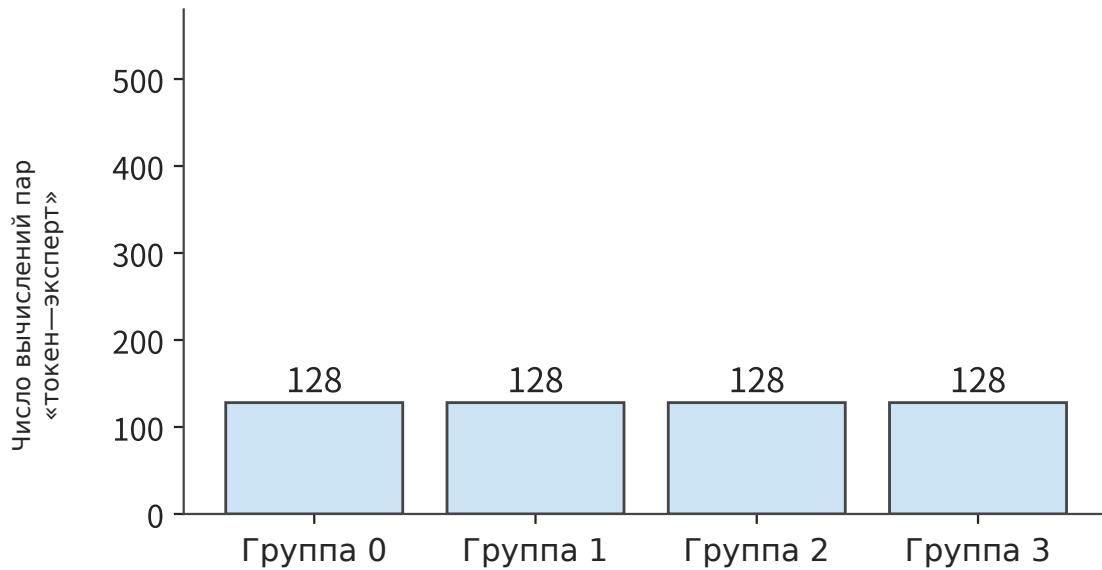


Рисунок 6-22. Распределение тех же восьми экспертов между четырьмя группами сохраняет объём чтения на уровне 288 MiB, при этом каждая группа выполняет 128 вычислений. Диапазон вертикальной оси одинаков на всех трёх рисунках.

$$\mathbb{E}[E_{\text{active}}] = E \left[1 - \left(1 - \frac{k}{E} \right)^m \right]. \quad (6-8)$$

При $E = 128$, $k = 8$, $m = 8$ ожидается около 52 активных экспертов, а идеальный объём чтения составляет приблизительно 1,8 GiB. Если все восемь токенов выбирают одних и тех же восемь экспертов, по-прежнему считывается только 288 MiB. С увеличением батча количество выбранных экспертов постепенно приближается к общему количеству экспертов, и дальнейшее увеличение числа входных строк главным образом повышает степень повторного использования каждого набора весов.¹³

Фактические результаты маршрутизации совместно определяются входными данными и моделью. Записи маршрутизации DeepSeek V4-Flash из сопроводительных материалов охватывают четыре входа и 43 слоя и содержат в общей сложности около 1,31 миллиона выборов экспертов. Частоты выбора разных экспертов различаются. Если преобразовать количество полученных каждым экспертом токенов в количество строк матрицы, а затем просуммировать значения по набору экспертов на каждой карте, по этим записям можно получить распределение вычислительной нагрузки между картами. Пойслойная тепловая карта приведена в сопроводительных материалах.¹⁴

¹³Материал об использовании экспертов и MoE Serving Tax. В формуле (6-8) предполагается, что каждый токен независимо и равномерно выбирает экспертов; в исходной статье сопоставление с плотной контрольной моделью выполняется отдельно по FLOPs активаций и по общему числу параметров.

¹⁴График и описание выборки приведены в материалах «Тепловая карта маршрутизации» и «Описание сопроводительной иллюстрации». Записи охватывают 43 слоя и четыре фактических входа decode для каждого задания, включая преждевременно выполненный планировщиком EOS forward. Источники: «Наблюдение за маршрутизацией DeepSeek V4-Flash» и анализ исходных счётчиков.

6.3.3 От сотен миллиардов до триллионов параметров: как меняются комбинации параллелизма

Анализ повторного использования и нагрузки в разделе 6.3.2 основан на предположении, что весь набор экспертов уже помещается в память ускорителей. При дальнейшем увеличении модели сначала необходимо повторно проверить общую ёмкость памяти, а затем решить, как комбинировать TP, PP и EP. Все BF16-веса Qwen3-235B-A22B занимают около 470 GB, если каждый параметр учитывать только один раз. DeepSeek V4-Flash содержит около 284B параметров, что в том же формате соответствует примерно 568 GB. Восемь H100 в одном HGX H100 располагают в общей сложности 640 GB памяти, поэтому для состояния и рабочего пространства остаётся уже немного места. Каждый слой DeepSeek V4-Flash содержит 256 маршрутизируемых экспертов, из которых каждый токен выбирает шесть; кроме того, выполняются общие эксперты. Поскольку набор экспертов больше, при размещении весов необходимо одновременно учитывать общих и маршрутизируемых экспертов.¹⁵

DeepSeek V4-Pro содержит около 1,6T параметров, и даже при половине байта на параметр веса занимают приблизительно 800 GB. Для примерно 2,8T параметров Kimi K3 требуется около 1,4 TB. Требования этих моделей к памяти уже превышают общую ёмкость одного HGX H100. Увеличение количества групп EP позволяет распределить экспертов между большим количеством карт. Увеличение числа карт TP внутри эксперта уменьшает объём матриц экспертов, сохраняемый на каждой карте. Увеличение количества этапов PP сокращает количество слоёв модели, хранящихся на каждом этапе.

Представление входных данных экспертов также влияет на объём обмена между картами. Маршрутизируемые эксперты Kimi K3 работают в латентном пространстве размерностью 3584, тогда как скрытый вектор основной части модели имеет размерность 7168. Если сначала выполнить проекцию, а затем отправку, размер BF16-вектора уменьшается с 14 KiB до 7 KiB. Если сначала отправить данные, а затем выполнить проекцию, передаваться будет вход без уменьшения размерности. При одинаковых вычислениях экспертов размещение проекции до отправки или после получения определяет размерность вектора, передаваемого между картами. На каждом слое модель выбирает 16 из 896 маршрутизируемых экспертов. Кроме того, присутствуют два общих эксперта, обрабатывающих полный вектор основной части модели; эти виды экспертов выполняют разные вычисления. Из 93 слоёв 69 слоёв KDA рекуррентно агрегируют контекст с помощью матрицы фиксированного размера, а 24 слоя MLA сохраняют контекст в низко-размерном представлении. Требования к памяти для состояния KDA главным образом определяются его фиксированной размерностью, тогда как требования к памяти для состояния MLA дополнительно растут вместе с контекстом сеанса.¹⁶

Таким образом, изменения модели влияют на проектирование системы с двух сторон: увеличение количества весов повышает необходимую общую ёмкость памяти, а различия в графе вычислений изменяют состав данных, которыми необходимо обмениваться. Первое определяет минимальный объём необходимых ресурсов, а второе — способ их соединения.

¹⁵Расчёт параллелизма для конкретных моделей, конфигурация Qwen3-235B-A22B, отчёт DeepSeek V4, таблица 1 и §2.3 отчёта Kimi K3. DeepSeek-V3 с 671B параметров используется как историческая модель для сравнения.

¹⁶Расчёт параллелизма для конкретных моделей, конфигурация Qwen3-235B-A22B, отчёт DeepSeek V4, таблица 1 и §2.3 отчёта Kimi K3. DeepSeek-V3 с 671B параметров используется как историческая модель для сравнения.

Эксперимент 6-3 · Основной: как назначение и размещение экспертов изменяют нагрузку карт и объём обмена данными

- (a) По рисунку 6-16 запишите последовательность отправки входа токена A, двух выполнений экспертов и взвешенного объединения.
- (b) Повторно рассчитайте два варианта назначения экспертов для 64 токенов, определив объём чтения весов и количество задач наиболее загруженной группы EP.
- (c) Равномерно распределите восемь популярных экспертов между четырьмя группами EP, сравните общий объём чтения и вычислительную нагрузку наиболее загруженной группы экспертов и объясните источник изменений.
- (d) Используя таблицу $TP2 \times EP4$ из этого раздела, вычислите объём KV, который должна хранить каждая карта при длине контекста 8192 и 16384, и выведите объём отправки для трёх объединений результатов.
- (e) Усложнённое задание: для DeepSeek V4-Flash, DeepSeek V4-Pro или Kimi K3 выберите конкретный формат хранения и по отдельности разместите маршрутизируемых экспертов, общих экспертов, состояние внимания и рабочее пространство.

6.4 Реализация коллективных коммуникаций и стоимость их выполнения

6.4.1 Определение требований к коммуникации по способу разбиения модели

В разделе 6.2 были определены такие операции коллективной коммуникации, как AllReduce и All-to-All; в этом разделе рассматриваются их реализация и стоимость выполнения. Сначала рассмотрим, как четыре карты могут свести четырёхэлементный вектор. Каждая карта содержит локальный вклад в четыре элемента, а цель состоит в том, чтобы каждая карта получила полную поэлементную сумму. Это можно сделать в два этапа: сначала выполнить суммирование так, чтобы на карте 0 осталась полная сумма элемента 0, на карте 1 — полная сумма элемента 1 и так далее; затем обменяться этими уже просуммированными элементами, чтобы все четыре карты получили полный вектор.

Первый этап — это ReduceScatter, второй — AllGather; вместе они выполняют одну операцию AllReduce. Эти два этапа изменяют соответственно значения и расположение значений. Плотным моделям и MoE также требуется объединять сегменты и отправлять входные данные экспертов на разные карты. По тому, какими данными располагает каждая карта в начале и в конце коммуникации, можно выделить следующие четыре операции коллективной коммуникации.

Операция	Что находится на каждой карте в начале	Что находится на каждой карте после завершения	Назначение
AllReduce	Частичные результаты одного тензора	Полный результат редукции	Частичные суммы TP, градиенты обучения

Операция	Что находится на каждой карте в начале	Что находится на каждой карте после завершения	Назначение
ReduceScatter	Частичные результаты одного тензора	Разные сегменты результата редукции	Сегментированный вывод, сегменты градиентов
AllGather	Разные сегменты одного тензора	Полный объединённый результат	Восстановление полного входа
All-to-All	Данные для разных получателей	Данные, отправленные этой карте всеми исходными картами	Dispatch входов экспертов и возврат результатов

Broadcast (широковещательная рассылка) отправляет данные одной указанной карты всей группе, а Reduce (редукция) передаёт результат редукции только одной указанной карте; в отличие от операций в таблице, обе они не оставляют результат у всей группы. AllGather объединяет данные по rank (номеру каждой карты внутри коммуникационной группы), не выполняя суммирование; All-to-All перераспределяет данные по получателям и также не объединяет автоматически выходы экспертов с учётом весов маршрутизации. Поэтому combine в MoE включает три этапа: возврат результатов, восстановление порядка токенов и суммирование с учётом весов маршрутизации. Нельзя считать его AllReduce только на основании названия.¹⁷

Название операции определяет, что вычисляется и на каких картах размещается результат, а конкретный алгоритм — на сколько раундов делятся данные и как они передаются. В следующем подразделе прослеживается передача одного блока по кольцу.

Алгоритм определяет продолжительность самого обмена, а момент запуска коммуникации каждой картой — длительность ожидания каждой карты: группа может начать обмен только после запуска последней картой, а карты, пришедшие раньше, ожидают, пока она завершит предшествующие вычисления. Поэтому чем сильнее различаются моменты запуска, тем меньше выигрыш от ускорения самого обмена. В разделе 7.6.1 это сравнение подробно разбирается на одном наборе данных.¹⁸

6.4.2 Кольцо, дерево и раунды коммуникации

Кольцевой AllReduce выстраивает участвующие карты в ориентированное кольцо. В каждом раунде каждая карта отправляет один блок следующей карте и получает блок от предыдущей. Пусть число участвующих карт равно n , а полный объём частичных сумм на каждой карте — M байт. Разделим его на n блоков по M/n каждый.

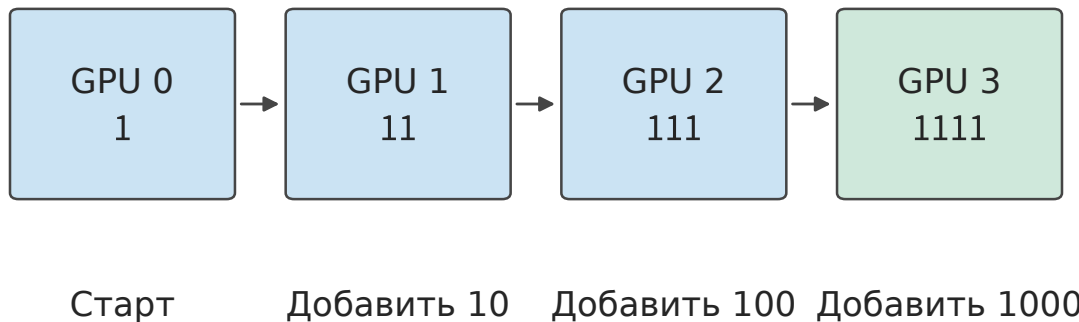
Рассмотрим кольцо из четырёх карт $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$. Изначально каждая карта содержит локальные вклады в блоки 0, 1, 2 и 3. В первом раунде карта r отправляет блок r ; получатель складывает полученный блок со своим соответствующим вкладом. Во втором раунде пересылается только что сформированная частичная сумма, а в третьем пересылка продолжается с добавлением последнего вклада.

Проследим за блоком 0: он отправляется с карты 0, проходит через карты 1 и 2 и в итоге попадает на карту 3. Если вклады четырёх карт в один из элементов блока 0 равны

¹⁷NVIDIA: семантика коллективных операций NCCL, операции «точка — точка» и обмен данными разной длины, методика измерения пропускной способности nccl-tests; снимки прочитанных материалов и хеши см. в реестре источников.

¹⁸Тракты коллективной коммуникации и диагностика, включая зафиксированный исходный код фреймворка, диспетчеризацию custom AllReduce, расхождения в моментах готовности и область измерения.

По пути $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ суммируется
вклад блока 0



После трёх раундов GPU 3 хранит полную сумму блока 0

Рисунок 6-23. Отслеживается только один элемент блока 0: при прохождении каждой карты к нему добавляется вклад этой карты. После трёх раундов получается 1111, сохранённое на карте 3. Остальные три блока одновременно продвигаются по кольцу.

1, 10, 100 и 1000, то по пути значение последовательно принимает значения 1, 11, 111 и 1111. После третьего раунда карта 3 содержит полную сумму блока 0; одновременно карты 0, 1 и 2 содержат полные суммы блоков 1, 2 и 3 соответственно.

Трёх раундов достаточно, поскольку блок уже содержит один вклад с начальной карты, а после прохождения остальных трёх карт — все четыре вклада. В общем случае ReduceScatter требует $n - 1$ раундов. Затем каждая карта пересылает завершённый ею блок следующей карте, получая в каждом раунде ещё один блок; ещё через $n - 1$ раундов каждая карта собирает все блоки. Поэтому общий объём отправки с каждой карты равен $2(n - 1)M/n$, а общее число раундов — $2(n - 1)$.

Обозначим фиксированные накладные расходы каждого раунда через α , эффективную пропускную способность каждого ориентированного ребра — через B и предположим, что каждая карта переходит к следующему раунду только после завершения отправки в текущем. Тогда модель времени имеет вид

$$T_{\text{ring}} = 2(n - 1)\alpha + \frac{2(n - 1)M}{nB}. \quad (6-9)$$

Первое слагаемое растёт с числом карт, а второе постепенно приближается к $2M/B$. Кольцевой алгоритм делит большие данные на небольшие блоки, позволяя картам передавать их одновременно; при малом объёме данных основную долю времени, наоборот, занимают запуски отдельных раундов.

Пример: насколько увеличение числа карт ТР всё ещё ускоряет выполнение с



В ячейках — номера уже собранных блоков

Рисунок 6-24. После завершения ReduceScatter карты 0, 1, 2 и 3 содержат блоки 1, 2, 3 и 0 соответственно. Затем в каждом раунде пересылается один блок; через три раунда каждая карта содержит все четыре блока с полными результатами.

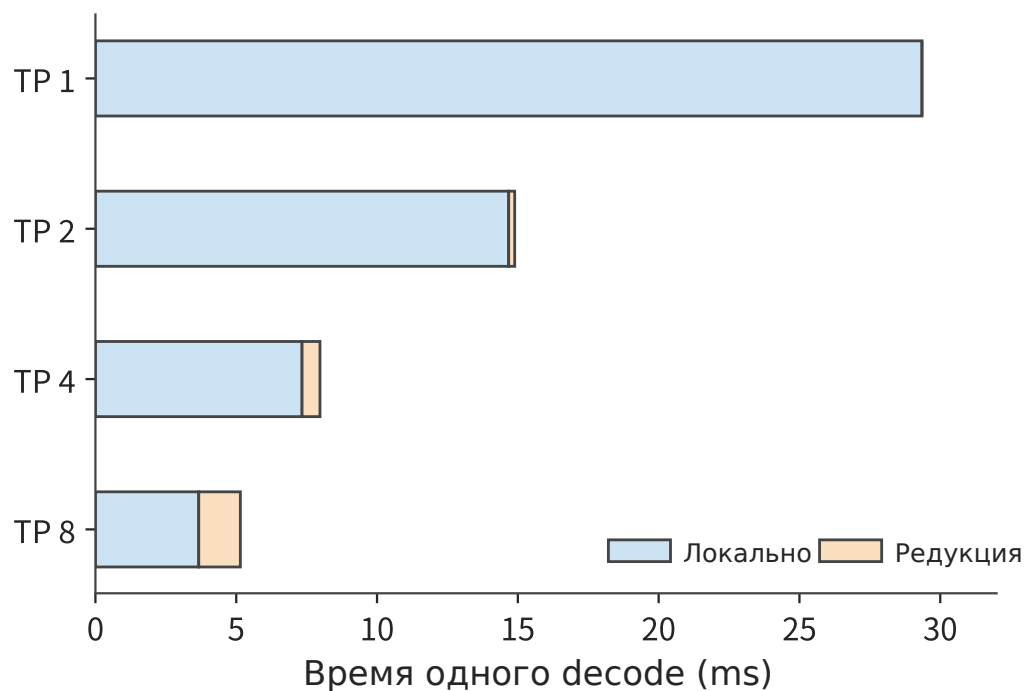


Рисунок 6-25. Время одного шага, рассчитанное по формуле (6-5) и модели кольцевой редукции. Время локального доступа к памяти уменьшается с ростом числа карт, а время редукции увеличивается; полная длина столбца соответствует времени выполнения одного шага. Локальная составляющая включает чтение весов и KV, а составляющая редукции — коллективную коммуникацию между картами.

учёт коммункации редукции? Снова рассмотрим сеанс Qwen3-32B длиной 128K. Входом каждой редукции служит BF16-вектор скрытого состояния одного токена, $M = 10 \text{ KiB}$; в HGX H100 пропускная способность каждой карты через NVSwitch в каждом направлении равна $B = 450 \text{ GB/s}$; фиксированные накладные расходы раунда α примем равными измеренной в статье о MSCCL++ односторонней задержке NVLink $0,822 \mu\text{s}$ на таком же восьмикарточном сервере H100. Найдём время одной редукции и время по формуле (6-5) для TP2, TP4 и TP8.

Решение: для TP8 требуется четырнадцать раундов, суммарные накладные расходы запуска равны $11,51 \mu\text{s}$, каждая карта отправляет $17,5 \text{ KiB}$, передача занимает около $0,04 \mu\text{s}$, а одна операция — около $11,55 \mu\text{s}$. Для остальных конфигураций аналогично подставим значения в формулу (6-9). Строка TP1 приведена только для сравнения: одна карта не способна вместить даже один сеанс длиной 128K.

Число карт TP	Локальный доступ к памяти	128 редукций	Время шага
1	29,35 ms	0	29,35 ms
2	14,68 ms	0,21 ms	14,89 ms
4	7,34 ms	0,64 ms	7,97 ms
8	3,67 ms	1,48 ms	5,15 ms

На рисунке синяя часть сокращается с увеличением числа карт, а оранжевая постепенно растёт. При переходе от двух карт к четырём время локального доступа к памяти

уменьшается примерно на 7,34 ms, а коммуникация увеличивается приблизительно на 0,42 ms, что даёт чистую экономию около 6,92 ms. При переходе от четырёх карт к восьми время локального доступа к памяти уменьшается ещё лишь примерно на 3,67 ms, а коммуникация увеличивается примерно на 0,84 ms, поэтому чистая экономия снижается приблизительно до 2,83 ms. Левая и правая части формулы (6-6) постепенно сближаются, но восемь карт всё ещё работают быстрее.

Повышение пропускной способности здесь почти ничего не даёт. При одной редукции на восьми картах 11,51 μ s приходится на запуск; даже если время передачи снизить до нуля, всё равно придётся ждать завершения этих четырнадцати раундов. Удвоение пропускной способности экономит за весь шаг лишь около 2,5 μ s, тогда как сокращение фиксированных накладных расходов каждого раунда вдвое позволяет сэкономить около 0,74 ms на 128 редукциях.¹⁹

Древовидный алгоритм сокращает число раундов. Четыре карты сначала выполняют попарную редукцию, после чего корневой узел объединяет результаты — для редукции требуется два раунда; широковещательная рассылка занимает ещё два раунда, итого четыре. Когда n является степенью двойки, несегментированное биномиальное дерево выполняет $2 \log_2 n$ раундов, причём в каждом раунде на критическом пути передаётся полный тензор:

$$T_{\text{tree}} = 2 \log_2 n \left(\alpha + \frac{M}{B} \right). \quad (6-10)$$

Для восьми карт и 10 KiB время составляет около 5,07 μ s, что меньше 11,55 μ s у кольцевого алгоритма. Если объём данных увеличить до 80 MiB, соответствующих 8192 токенам, кольцевому алгоритму потребуется около 0,34 ms, а древовидному — около 1,12 ms: экономия времени за счёт меньшего числа раундов не компенсирует время, добавленное многократной передачей полного большого тензора по критическому пути. Если приравнять две формулы, точка пересечения будет находиться примерно на 680 KiB; после превышения этого значения более быстрым становится другой алгоритм.

Эксперимент 6-4 · Расширение: при каком объёме данных кольцевая редукция становится быстрее древовидной?

- (a) Следуя правилам отправки на рисунке 6-23, проследите три раунда редукции блока 1 и укажите, какая карта содержит этот блок после каждого раунда и вклады каких карт уже были в нём накоплены.
- (b) Найдите объём данных, отправляемых каждой картой при кольцевой редукции на четырёх и восьми картах, а затем выведите время древовидной редукции для того же объёма данных.
- (c) Найдите объём данных, при котором в восьмикарточной конфигурации времена двух алгоритмов редукции равны; повторите расчёт после уменьшения α вдвое и объясните, почему изменяется объём данных, при котором времена алгоритмов совпадают.

¹⁹Пораундовые результаты Ring и результаты несегментированного биномиального дерева, рассчитанные без конкуренции за ресурсы. Пропускная способность 18 каналов NVLink 4 карты H100 в каждом направлении в сумме составляет 450 GB/s; см. страницу 47 описания архитектуры H100. Накладные расходы каждого раунда взяты из работы С. Hwang и соавторов «MSCCL++: Rethinking GPU Communication Abstractions for AI Inference», ASPLOS 2026, таблицы 1 и 2: на платформе с 8 картами H100, NVLink 4 и одной сетевой картой ConnectX-7 400 Gbit/s на каждую карту утилита nvbandwidth показала задержку NVLink 822 ns и пропускную способность 397,5 GB/s, а RDMA perftest — задержку InfiniBand 3,76 μ s и пропускную способность 48,94 GB/s.

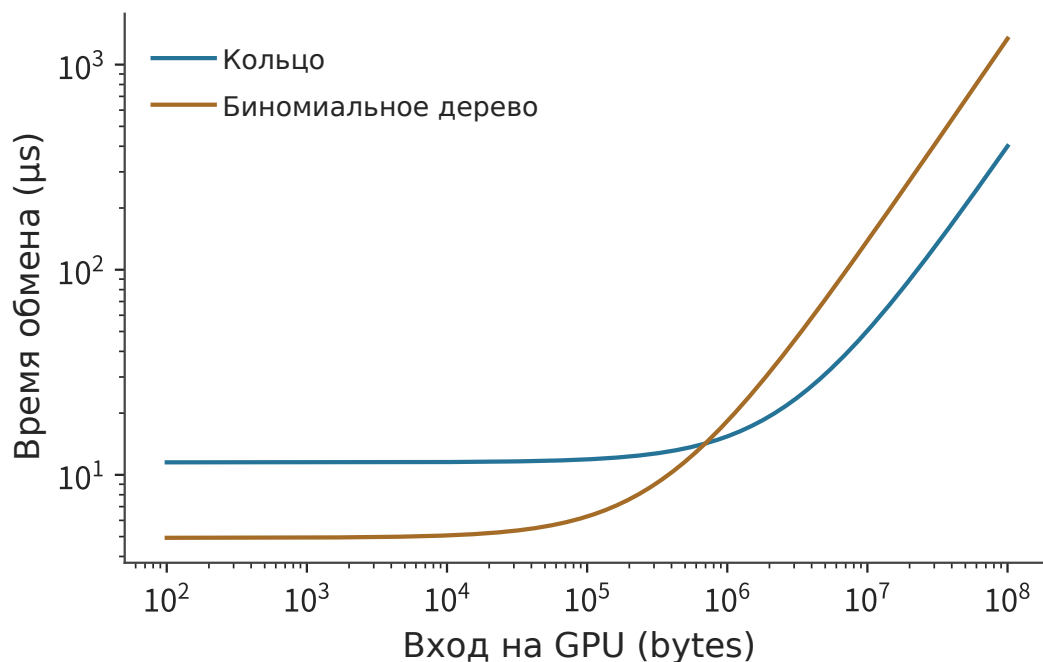


Рисунок 6-26. Модели времени для кольцевого алгоритма на восьми картах и несегментированного биномиального дерева при $0,822 \mu\text{s}$ на раунд и 450 GB/s в каждом направлении. Точка пересечения находится примерно на 680 KiB ; вход decode размером 10 KiB расположен в области преимущественного влияния накладных расходов запуска, а вход prefill размером 80 MiB — в области преимущественного влияния времени передачи данных.

- (d) Подставьте время древовидной редукции 10 KiB данных в рассматриваемый в этой главе пример Qwen3-32B, найдите время одного шага на восьми картах и сравните выигрыш от выбора алгоритма с выигрышем от удвоения пропускной способности.

6.4.3 Конкуренция библиотеки коммуникации и вычислений за ресурсы

Кольцевой и древовидный алгоритмы должны фактически выполняться программным обеспечением на каждой карте. **Библиотека коммуникации** — это программное обеспечение, реализующее операции коллективной коммуникации, например NVIDIA Collective Communications Library (NCCL). Соответствующая реализация для Ascend — Huawei Collective Communication Library (HCCL). Приложение сначала создаёт коммуникационную группу (communicator), назначает каждой карте внутри группы свой rank, а затем отправляет операцию, указывая буфер, количество элементов, тип данных, операцию редукции и CUDA stream, в котором она должна выполняться. Вызовы на всех картах группы должны соответствовать друг другу согласно требованиям интерфейса, включая порядок вызовов и объём данных; возврат из вызова обычно означает лишь, что операция отправлена в stream, но не что ещё участвующий в передаче буфер уже можно использовать повторно. Сторона, использующая результат, должна дождаться его посредством порядка операций в том же stream или явной зависимости от event. Коллективную коммуникацию также нельзя использовать как глобальную точку синхронизации, доступную произвольному коду на хосте.

В МоЕ число токенов, отправляемых между каждой парой карт при `dispatch`, меняется в зависимости от `batch`. Равномерный All-to-All не может выразить такой обмен с разными объёмами, поэтому среда выполнения должна самостоятельно организовать счётчики, смещения, упаковку и пространство приёма. NCCL позволяет выразить обмен между любыми двумя картами с помощью сгруппированных двухточечных операций `Send/Recv` (отправки/приёма); каждой отправке должен соответствовать приём, а коммуникации, которые должны продвигаться совместно, необходимо помещать в одну группу, иначе они будут последовательно ожидать операций, ещё не запущенных другой стороной. Специализированные библиотеки коммуникации для экспертного параллелизма дополнительно используют сведения о маршрутизации при реализации `dispatch` и `combine`. Независимо от выбранной реализации необходимо фиксировать версию библиотеки и конкретный интерфейс; тест All-to-All с фиксированным размером нельзя считать измерением реальной коммуникации МоЕ.²⁰

При измерении NCCL также нужно различать два сообщаемых ею вида пропускной способности: **algbw** — объём данных, определённый тестом, делённый на время, а **busbw** пересчитывается с учётом фактического объёма передачи коллективной операции. Для AllReduce на n картах при входе объёмом M на каждой карте $\text{algbw} = M/T$, а $\text{busbw} = 2(n - 1)M/(nT)$. Например, в общедоступной записи, сохранённой в эксперименте 7-3, AllReduce с 16 rank на двух HGX H100 и 64 MiB на карту занял 466,1 μs : **algbw** составил 143,99 GB/s, а **busbw** — в 1,875 раза больше, то есть 269,98 GB/s. **busbw** — нормализованный показатель, его нельзя напрямую считать фактической пропускной способностью каждой сетевой карты; иерархическую коммуникацию, редукцию внутри коммутатора и совместное использование каналов всё равно необходимо учитывать по отдельности согласно физическому пути, описанному в главе 7. При малом объёме данных следует смотреть на задержку, при большом — на устойчивую пропускную способность, а в контексте выполнения модели — на весь интервал от готовности данных до завершения их использования.²¹

При сравнении кольца и дерева в разделе 6.4.2 одна коммуникация рассматривалась как отдельная операция. На практике коммуникация часто перекрывается вычислениями другого `micro-batch`, и они конкурируют за ресурсы ускорителя. Для редукции требуется прочитать частичные суммы, выполнить сложение, записать результат и продвинуть коммуникацию. Эти действия занимают SM, HBM и межсоединение ускорителей; одновременно выполняемое матричное умножение также нуждается в этих ресурсах. Когда коммуникация и вычисления перекрываются, скорость, наблюдавшаяся при монопольном использовании ресурсов, изменяется.

Сопутствующий эксперимент 6-5 измеряет это влияние. В эксперименте Gloo — библиотека коллективной коммуникации с поддержкой сред выполнения, включая CPU, — выполняет AllReduce между четырьмя процессами CPU, одновременно с чем каждый процесс выполняет одно матричное умножение. Время обеих операций фиксируется как при раздельном, так и при одновременном выполнении; берётся медиана пяти серий:

²⁰NVIDIA: семантика коллективных операций NCCL, операции «точка — точка» и обмен данными разной длины, методика измерения пропускной способности `nccl-tests`; снимки прочитанных материалов и хеши см. в реестре источников.

²¹NVIDIA: семантика коллективных операций NCCL, операции «точка — точка» и обмен данными разной длины, методика измерения пропускной способности `nccl-tests`; снимки прочитанных материалов и хеши см. в реестре источников.

Объём данных редукции	Коммуникация отдельно	Вычисление отдельно	Коммуникация при одновременном выполнении	Вычисление при одновременном выполнении	Завершение всей группы при одновременном выполнении
4 MiB	2,67 ms	11,53 ms	6,14 ms	11,52 ms	11,58 ms
64 MiB	36,92 ms	11,03 ms	45,21 ms	12,89 ms	45,24 ms

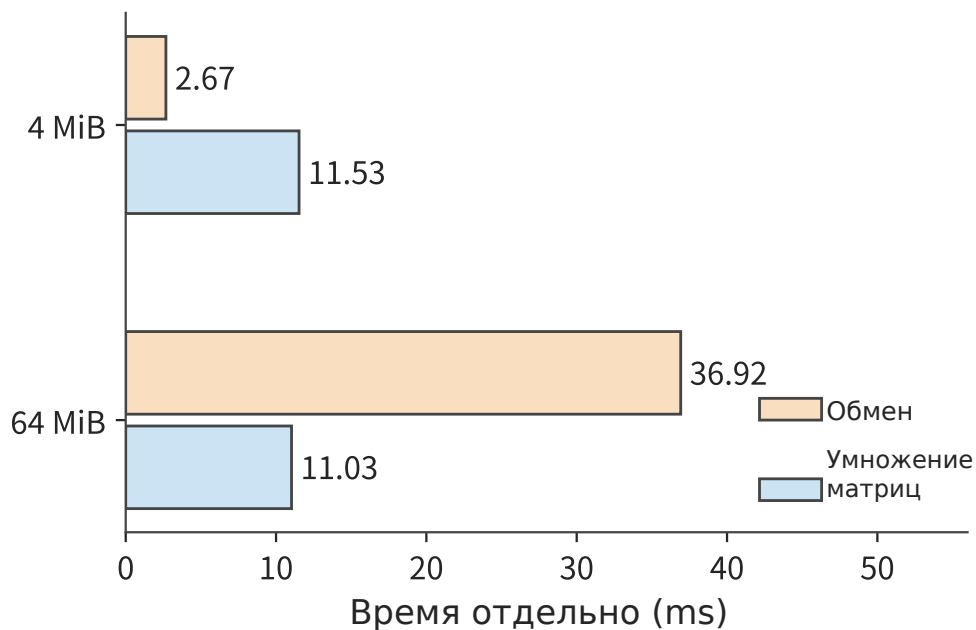


Рисунок 6-27. Время раздельного выполнения коммуникации и матричного умножения в эксперименте 6-5: четыре процесса CPU, медиана пяти серий. Коммуникация занимает около 2,67 ms для 4 MiB и около 36,92 ms для 64 MiB, а матричное умножение — около 11 ms.

При 4 MiB одновременное выполнение коммуникации и матричного умножения замедляет коммуникацию более чем вдвое, однако она всё равно завершается раньше матричного умножения. Вся группа выполняется за 11,58 ms — почти столько же, сколько одно матричное умножение: коммуникация скрыта вычислениями. При 64 MiB время коммуникации увеличивается с 36,92 до 45,21 ms, а время матричного умножения — с 11,03 до 12,89 ms; раздельное выполнение занимает в сумме около 48,0 ms, а одновременное экономит лишь около 2,7 ms, а не все 11 ms матричного умножения. Для оценки перекрывающегося выполнения необходимо сравнивать время при одновременном запуске обеих операций, а не просто вычитать измеренные по отдельности времена.²²

Для GPU действует тот же принцип. Увеличение числа коммуникационных потоков и каналов может ускорить отдельно выполняемую передачу, но одновременно занимает исполнительные ресурсы, необходимые для вычислений. Поэтому выбор конфигурации только по пропускной способности отдельно запущенной коммуникации может замедлить выполнение в целом. По этой причине система автоматической оптимизации коллективной коммуникации AutoCCL во время обучения фиксирует фактическое время коммуникации, чтобы обратная связь учитывала помехи от параллельного выполнения; сначала она выбирает алгоритм, протокол и реализацию передачи, а затем подбирает потоки, каналы и разбиение на блоки. Согласно статье, при помехах от по-

²²Эксперимент 6-5: фактическое перекрытие коммуникации и вычислений в четырёх процессах. Эксперимент выполнялся на одном M2 Max с версией PyTorch для CPU; четыре процесса обменивались данными по локальному TCP. Итоговые измерения проводились в пяти группах. В каждой группе по одному разу запускались три режима: только коммуникация, только вычисления и одновременное выполнение; порядок внутри группы выбирался случайно. Для каждого показателя сначала выбиралось время самого медленного процесса внутри группы, а затем медиана по пяти группам. Время коммуникации измерялось с момента отправки асинхронного AllReduce до получения хостом callback о завершении, которое могло произойти немного позже фактического окончания передачи данных.

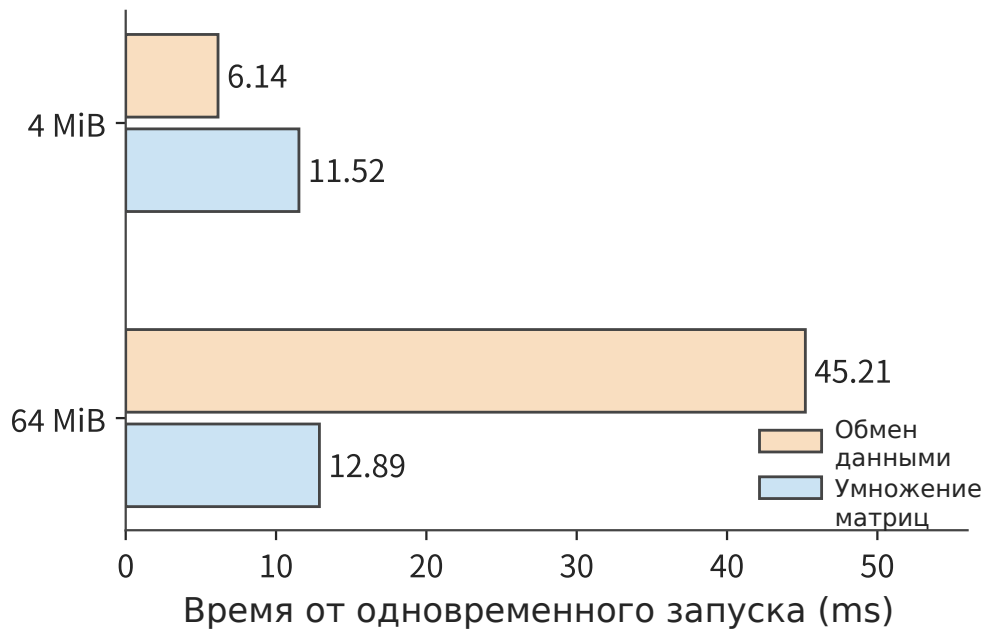


Рисунок 6-28. В том же эксперименте обе операции запускаются одновременно. Оранжевым показана коммуникация, синим — вычисление; последующая работа ожидает завершения обеих операций. Время коммуникации для 4 MiB увеличивается до 6,14 ms, но она по-прежнему скрыта вычислением; время коммуникации для 64 MiB возрастает до 45,21 ms, а вычисления — до 12,89 ms.

вторных вычислений пропускная способность AllGather выросла с 18,26 до 32,44 GB/s. Сам поиск и переключение также требуют времени: если поиск дополнительно занимает S , а каждый последующий вызов экономит Δ , для окупаемости требуется не менее S/Δ вызовов.²³

6.4.4 Слияние коммуникаций, запуск ускорителем и специализированная разгрузка

Ускорение коммуникации само по себе не обязательно ускоряет выполнение в целом, поэтому следует различать два вида улучшений: сокращение числа запусков коммуникации и операций чтения и записи данных, а также снижение потребления вычислительных ресурсов коммуникацией. В восьмикарточном примере с Qwen3-32B накладные расходы запуска 128 редукций составляют около 1,47 ms. Эти расходы можно уменьшать на трёх разных уровнях: сколько работы выполняется за один запуск, кто его инициирует и кто выполняет перемещение и редукцию.

Слияние позволяет выполнять смежные операции последовательно внутри одного ядра. Например, если после редукции требуется прибавить остаточную связь и выполнить RMSNorm, раздельное выполнение приводит к записи результата в память, его повторному чтению и нескольким запускам; после слияния обработку можно

²³В AutoCCL используются 16/32 карты A40 и NCCL 2.18.3, а в восьмикарточной конфигурации NVLink карты соединены четырьмя парами. Непосредственной целью поиска по-прежнему остаётся производительность коммуникации; рост с 18,26 до 32,44 GB/s — это приведённое в таблице 6 статьи измерение AllGather при помехах от повторных вычислений. Путь AllGather/AlltoAll без CTA через сеть в NCCL 2.31.2 включает симметричное окно регистрации, внутрисерверный CE и межсерверный CPU проху. Подробности приведены в материале «Настройка коммуникации, конкуренция с вычислениями и разгрузка».

продолжить, пока данные всё ещё находятся рядом с вычислительными блоками. Нормализацию необходимо выполнять только после того, как редукция сформирует полный вход: если пренебречь малой константой ϵ для предотвращения деления на ноль и обучаемым коэффициентом масштабирования, то $\text{RMSNorm}([1, 0] + [0, 1]) = [1, 1]$, тогда как раздельная нормализация с последующим сложением даёт $[\sqrt{2}, \sqrt{2}]$. Чтобы получить тот же результат вычисления, слияние должно сохранять этот порядок выполнения.

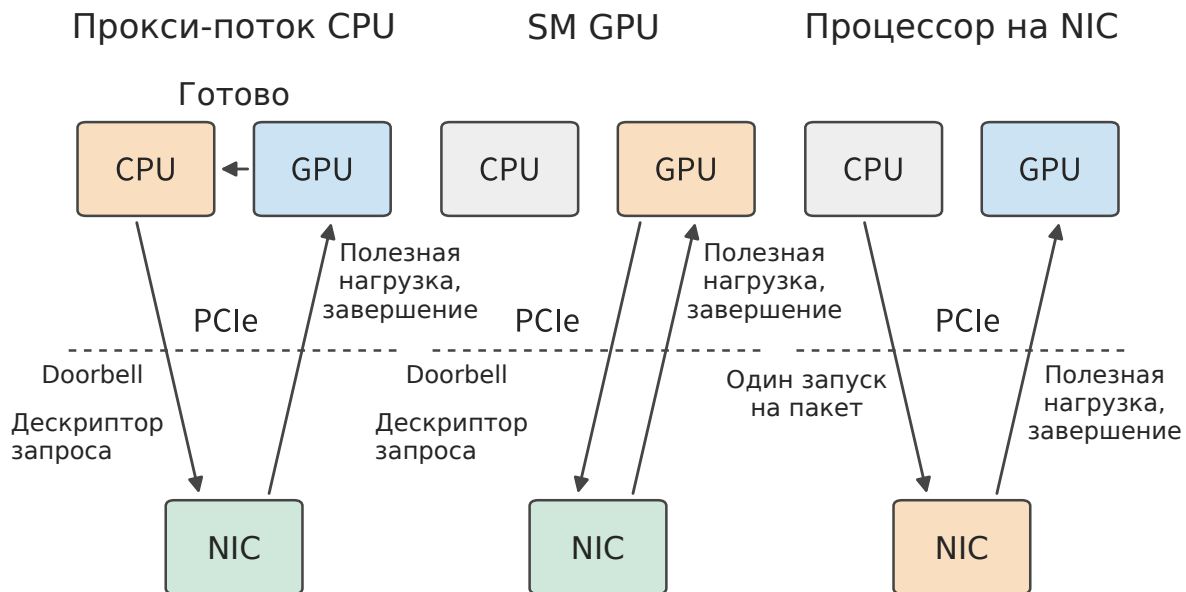
Запуск ускорителем устраняет обмен между отправкой с хоста и выполнением на ускорителе: вычислившее данные ядро может напрямую запустить последующую коммуникацию, сократив ожидание между завершением вычислений и началом коммуникации. **Специализированная разгрузка** передаёт перемещение, редукцию или синхронизацию движку копирования, коммуникационному процессору либо логике внутри коммутатора, освобождая универсальные вычислительные блоки для матричных вычислений.

Коллективный коммуникационный блок Ascend (CCU) сосредоточивает перемещение, редукцию, синхронизацию и обработку завершения в коммуникационном блоке; в некоторых путях NCCL для перемещения данных используются внутрисерверный движок копирования (Copy Engine) и межсерверный прокси-поток CPU — поток хоста, отправляющий запросы сетевой карте от имени GPU. Эти механизмы меняют исполнителя такой работы. Если вычисления замедляются из-за конкуренции со связью за SM, освобождение SM сокращает время вычислений; если узким местом служит общий сетевой выход, объём передаваемых через него данных не уменьшается.^{24,25}

Сторона, запускающая коммуникационный запрос, определяет, сколько раз управляющий путь пересечёт PCIe и чьи ядра будут заняты. На рисунке 6-29 сравниваются три варианта размещения. **Прокси-поток CPU**: после вычисления данных GPU записывает в память хоста флаг готовности; поток CPU обнаруживает его при опросе, формирует дескриптор запроса — запись, описывающую одну передачу, — и звонит в «дверной звонок» сетевой карты, то есть записывает в один из её регистров, уведомляя о новом запросе; затем сетевая карта считывает дескриптор запроса из памяти хоста. Запись о завершении также помещается в память хоста, после чего CPU уведомляет GPU. Управляющий путь пересекает PCIe три раза, и каждое пересечение занимает сотни наносекунд; число запросов, которые один поток CPU способен обрабатывать за секунду, также ограничено, поэтому множество мелких запросов образует здесь очередь. **SM GPU**: GPU напрямую формирует дескриптор запроса в собственной памяти и звонит в «дверной звонок»; сетевая карта считывает дескриптор запроса из памяти GPU, а запись о завершении возвращается в память GPU, где её опрашивает SM. CPU исключается из критического пути, но управляющий путь по-прежнему дважды пересекает PCIe; цена этого решения — выделение части SM для формирования запросов и опроса завершения, из-за чего эти SM не могут одновременно выполнять матричные вычисления. NVIDIA GPUDirect Async и низкозадержечное ядро библиотеки коммуникации для экспертно-

²⁴Проверка материалов об UB и Ascend 950, §4 эталонного проекта UB OS, §4.6 официального технического документа 950.

²⁵В AutoCCL используются 16/32 карты A40 и NCCL 2.18.3, а в восьмикарточной конфигурации NVLink карты соединены четырьмя парами. Непосредственной целью поиска по-прежнему остаётся производительность коммуникации; рост с 18,26 до 32,44 GB/s — это приведённое в таблице 6 статьи измерение AllGather при помехах от повторных вычислений. Путь AllGather/AlltoAll без CTA через сеть в NCCL 2.31.2 включает симметричное окно регистрации, внутрисерверный CE и межсерверный CPU проху. Подробности приведены в материале «Настройка коммуникации, конкуренция с вычислениями и разгрузка».



Пересечений PCIe: 3 Пересечений PCIe: 2 Пересечений PCIe: 0

Рисунок 6-29. Три варианта размещения инициатора. Оранжевым обозначен компонент, формирующий дескриптор запроса; пунктиром — границы PCIe. Управляющий путь прокси-потока CPU пересекает PCIe три раза, при запуске с GPU — два раза, а при запуске процессором сетевой карты — ни разу; полезная нагрузка инициатора и запись о завершении во всех трёх вариантах пересекают PCIe по одному разу.

го параллелизма DeepEP используют именно этот путь. **Процессор на сетевой карте:** сетевые карты наподобие BlueField содержат многоядерный многопоточный процессор тракта данных, поэтому формирование дескриптора запроса, звонок в «дверной звонок» и получение дескриптора могут выполняться внутри сетевой карты, а хосту достаточно один раз запустить пакет операций; управляющий путь больше не пересекает PCIe и не занимает ядра SM и CPU. При всех трёх вариантах полезная нагрузка на стороне инициатора перемещается из памяти GPU, а запись о завершении возвращается в память GPU, по одному разу пересекая PCIe; эти две операции не зависят от места запуска. Сетевая карта другой стороны при чтении и записи памяти своего хоста должна ещё раз пересечь PCIe уже на той стороне. Поэтому выбор заключается в том, кто предоставляет ядра, какова длина управляющего пути и сколько запросов можно запустить в секунду. В разделе 7.3.3 будет просуммировано время каждого пересечения, а в разделе 7.3.4 скорость запуска будет включена в модель пропускной способности.²⁶

Вернёмся к 128 редукциям малого объёма, рассмотренным в начале раздела. Фреймворк выбирает реализацию коммуникации в зависимости от объёма данных и способа соединения. Когда Qwen3-32B обрабатывает за раз 1, 4 или 16 токенов, вход редукции

²⁶Технический блог NVIDIA, Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async (этапы двух путей: через прокси-поток CPU и с иницированием со стороны GPU); документация NVIDIA DOCA DPA (модель программирования, переносящая коммуникационный код на встроенный процессор сетевой карты BlueField-3); снимок README DeepEP. Количество переходов для трёх вариантов размещения рассчитано по соответствующим путям управления; конкретная реализация может объединять этапы или добавлять новые. Снимки полученных материалов см. в реестре источников.

составляет соответственно 10, 40 или 160 KiB; такие фреймворки, как vLLM, могут выбирать для этих малых объёмов специализированную реализацию. Таким образом, у T_{AR} в формуле (6-5) есть два направления улучшения: уменьшение числа раундов алгоритма или сокращение программных и исполнительных накладных расходов каждого раунда.²⁷

Эксперимент 6-5 · Расширение: сколько времени экономит оптимизация коммуникации и когда окупаются затраты на подготовку?

- (a) По измеренным значениям на рисунках 6-27 и 6-28 найдите, сколько времени одновременное выполнение экономит по сравнению с отдельным для 4 MiB и 64 MiB, и укажите, что в каждом случае определяет общее время завершения: вычисление или коммуникация.
- (b) Пусть одна оптимизация требует дополнительного времени S на поиск и переключение, после чего каждый вызов экономит Δ . Запишите минимальное число вызовов, после которого накопленная экономия впервые превысит S ; используя измеренный выигрыш от перекрытия для 64 MiB в качестве Δ , найдите это число при $S = 1$ s.
- (c) На примере двух векторов по два элемента сравните результат выполнения RMSNorm после редукции с результатом редукции после отдельного выполнения RMSNorm и определите, можно ли менять эти операции местами.
- (d) Дополнительное задание для ускорителя: сравните монопольное и параллельное выполнение для одной и той же формы, зафиксируйте события готовности, начала и завершения и объясните, какой интервал изменился сильнее всего после совместного использования ресурсов.

6.5 Физическая организация суперузлов

6.5.1 Прямые соединения, коммутация и многоуровневые топологии

Алгоритмы и библиотеки связи из раздела 6.4 определяют, кто, кому и сколько отправляет на каждом раунде, но не рассматривают, через какие физические соединения проходят данные. Коллективный алгоритм задаёт логических отправителей и получателей, а физическая топология определяет, через какие устройства и каналы фактически проходят данные. Две карты могут быть соединены напрямую либо передавать данные через коммутатор, другие карты или память хоста. Если несколько передач используют один интерфейс, они вынуждены делить его пропускную способность по очереди.

Пример: станет ли TP16 быстрее TP8, если добавить ещё один сервер? Два HGX H100 содержат в общей сложности 16 карт, а серверы соединены через отдельный сетевой адаптер ConnectX-7 каждой карты. Если перевести один сеанс Qwen3-32B с контекстом 128K на TP16, одна группа тензорного параллелизма охватит два сервера, и AllReduce на каждом слое будет проходить через сеть.

Решение: для доступа к локальной памяти объём весов на карту снова уменьшается вдвое, но у Qwen3-32B всего 8 KV-голов. При TP16 каждая KV-голова хранится по одной копии на двух картах, поэтому каждая карта считывает столько же KV-данных, сколько

²⁷Тракты коллективной коммуникации и диагностика, включая зафиксированный исходный код фреймворка, диспетчеризацию custom AllReduce, расхождения в моментах готовности и область измерения.

при TP8, а локальное время сокращается лишь с 3,67 до 2,48 мс. Что касается связи: если выстроить 16 карт в кольцо, на каждом раунде придётся ждать прохождения участка через сетевой адаптер. В статье о MSCCL++ для той же платформы измерена односторонняя задержка InfiniBand, то есть используемой между серверами коммутируемой сети с низкой задержкой, равная 3,76 мкс; одно сокращение требует 30 раундов, то есть около 113 мкс. Реальная библиотека связи так работать не будет. Сохранённая в эксперименте 7-3 общедоступная запись nccl-tests, программы тестирования производительности коллективных операций NCCL, относится как раз к AllReduce на двух HGX H100 с 16 rank. Для сообщения размером 10 KiB берётся строка записи для 16 KiB; измеренное значение составляет 32,74 мкс:

$$T(16) \approx 2.48 \text{ ms} + 128 \times 32.74 \mu\text{s} \approx 6.67 \text{ ms}.$$

TP8 внутри одного сервера занимает 5,15 мс, поэтому TP16, наоборот, медленнее примерно на 1,52 мс. Дополнительные карты уменьшают объём чтения весов каждой картой, но взамен требуют более дорогой синхронизации между серверами. Даже с многоуровневым алгоритмом реальной библиотеки связи каждое сокращение по-прежнему занимает около 33 мкс — почти втрое больше, чем кольцевое сокращение внутри сервера, занимающее 11,55 мкс.

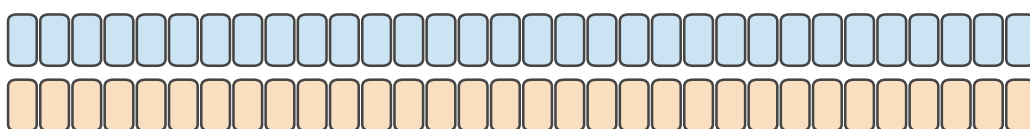
Чтобы большее число карт могло эффективно работать совместно, необходимы достаточное количество портов и достаточная коммутационная ёмкость. Если карта напрямую соединена со всеми остальными $N - 1$ картами, ей требуется $N - 1$ соединений. Коммутатор объединяет соединения: сначала карты подключаются к коммутационному уровню, который затем пересылает данные получателю. Число портов коммутационной микросхемы называется **радиксом** (radix).

Рассмотрим NVSwitch третьего поколения, используемый в HGX H100. Одна микросхема имеет 64 порта NVLink 4, каждый с пропускной способностью 25 GB/s в каждом направлении. Если использовать её как **коммутатор уровня leaf**: 32 порта подключить вниз к GPU, а остальные 32 — вверх к **коммутаторам уровня spine**, то совокупная пропускная способность со стороны GPU и совокупная пропускная способность восходящих соединений составят по 800 GB/s. Если выделить 48 нисходящих и 16 восходящих портов, число подключённых линий GPU возрастёт, но все они вместе смогут передавать вверх не более 400 GB/s — лишь треть совокупной пропускной способности нисходящих соединений. При неизменном общем числе портов подключение большего числа GPU оставляет меньше восходящих портов и меньшую пропускную способность восходящих соединений. В NVLink Switch System для DGX H100, готовой системы NVIDIA на базе HGX H100, несколько серверов подключаются к одной коммутируемой сети NVLink, и там применяется именно такой компромисс: за пределы каждого узла выводится лишь половина всей пропускной способности NVLink, то есть используется сужение 2:1.²⁸

Расширим коммутационный уровень за пределы одной микросхемы: четыре коммутационные микросхемы уровня leaf имеют по 32 нисходящих порта, всего 128 нисходящих портов. К ним добавляются 32 коммутационные микросхемы уровня spine, каждая из которых соединена одной линией с каждой из четырёх микросхем уровня leaf; таким образом, все 32 восходящих соединения каждой микросхемы уровня leaf полностью за-

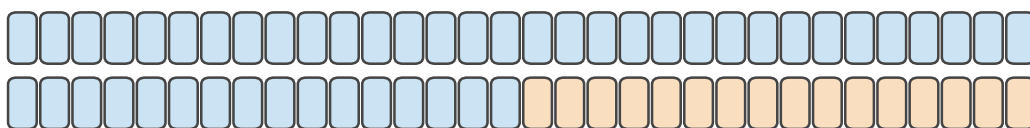
²⁸Описание архитектуры H100, стр. 47–48: один NVSwitch третьего поколения предоставляет 64 порта NVLink четвёртого поколения, а каждая линия NVLink — по 25 GB/s в каждом направлении; в NVLink Switch System вся внутренняя пропускная способность NVLink каждого узла выводится за его пределы с коэффициентом консолидации 2:1.

32 вниз + 32 вверх = 64 порта



Вниз 800 GB/s → вверх 800 GB/s

48 вниз + 16 вверх = 64 порта



Вниз 1200 GB/s → вверх 400 GB/s

Рис. 6-30. Два варианта распределения портов одной 64-портовой NVSwitch; каждый порт обеспечивает 25 GB/s в каждом направлении. Восходящие линии служат общим выходом для всего трафика между коммутаторами. Распределение 32/32 обеспечивает 800 GB/s восходящей пропускной способности, а 48/16 — только 400 GB/s. Число ячеек на цветных полосах соответствует числу портов.

действованы. Если разделить систему на две части, совокупность всех соединяющих их линий называется **разрезом**. Если на некотором этапе $V_{A \rightarrow B}$ байт должны пройти через разрез со стороны А на сторону В, а совокупная пропускная способность разреза в этом направлении равна $B_{A \rightarrow B}$, передача займёт не менее $V_{A \rightarrow B} / B_{A \rightarrow B}$. Распределение портов коммутатора определяет совокупную пропускную способность разреза.

Число портов определяет общее количество линий, а маршрутизация — то, на каких линиях концентрируются данные. Даже если все карты отправляют одинаковый объём данных, разные маршруты создают разные узкие места. Соединим 16 карт в двунаправленное физическое кольцо и рассмотрим первые три раунда ReduceScatter для входных данных размером 8 MiB. В рекурсивном алгоритме расстояние до партнёра удваивается на каждом раунде. Swing — алгоритм коллективной связи, разработанный для топологии тора; он распределяет нагрузку по линиям, меняя партнёра на каждом раунде. В обоих алгоритмах каждая карта на каждом раунде отправляет соответственно 4, 2 и 1 MiB, но получатели различаются:

Первые три раунда	Рекурсивные маршруты	Маршруты Swing
Число физических переходов на передачу	1, 2, 4	1, 1, 3
Максимальный объём передачи по однонаправленной линии	4, 4, 4 MiB	4, 2, 2 MiB
Совокупный объём передачи по физическим линиям всей сети	64, 64, 64 MiB	64, 32, 48 MiB

На третьем раунде данные рекурсивного алгоритма проходят четыре перехода, а самая загруженная линия одновременно переносит четыре порции по 1 MiB. Данные Swing проходят три перехода, а самая загруженная линия переносит лишь две порции. Если принять пропускную способность каждой линии ICI (межчипового соединения) TPU v4 равной 50 GB/s в каждом направлении, то время передачи первых трёх раундов, определяемое наиболее загруженной линией, составит соответственно около 252 и 168 мкс. Время передачи сократилось на треть благодаря уменьшению объёма данных на общих линиях.²⁹

Передача через память хоста — другая форма той же проблемы. Архитектура неоднородного доступа к памяти (NUMA) размещает память хоста рядом с разными CPU, а чтение через границу CPU проходит по интерфейсу между процессорами. Пусть карты 0 и 1 подключены к NUMA-узлу А, а карты 2 и 3 — к узлу В; при кольцевом сокращении четыре карты отправляют в общей сложности 48 MiB. Если разместить все промежуточные буферы в А, межпроцессорное соединение должно передать по 24 MiB в каждом направлении. Если разместить буферы рядом с отправляющими картами, а карты одного NUMA-узла расположить в кольце рядом, объём в каждом направлении снизится до 12 MiB. Если же изменить кольцо так, чтобы оно попеременно пересекало NUMA-узлы, объём снова вырастет до 24 MiB в каждом направлении. Хост DGX H100 также имеет двухпроцессорную структуру: каждый из двух Xeon 8480C обслуживает свой NUMA-узел, между двумя CPU может быть до четырёх линий UPI (межпроцессорного соединения Intel) по 16 GT/s каждая, а каждая GPU подключается к хосту через PCIe Gen5 x16 с пропускной способностью

²⁹ Физические тракты коллективной коммуникации и разнесение заданий по времени, расчёт физических трактов 96 сообщений, избранные материалы NSDI 2024, версия Morphlux и заметки по материалам. OCS в TPU v4 основан на MEMS-зеркалах, а переключение занимает миллисекунды; см. §2 статьи о TPU v4.



Рис. 6-31. На третьем раунде рекурсивного алгоритма карта 0 отправляет данные карте 4. Тонкие линии показывают физическое кольцо из 16 карт, а стрелками отмечены четыре проходимые линии.

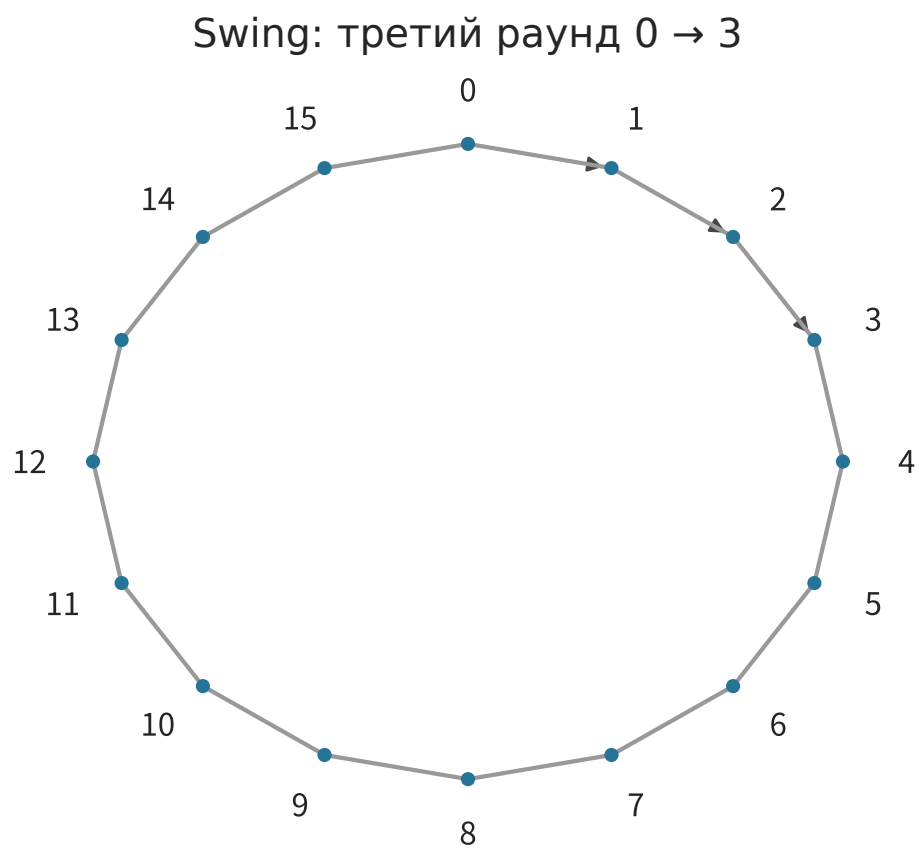


Рис. 6-32. На том же физическом кольце в третьем раунде Swing карта 0 отправляет данные карте 3 через три линии. Объем отправки каждой карты на этом раунде по-прежнему равен 1 MiB.

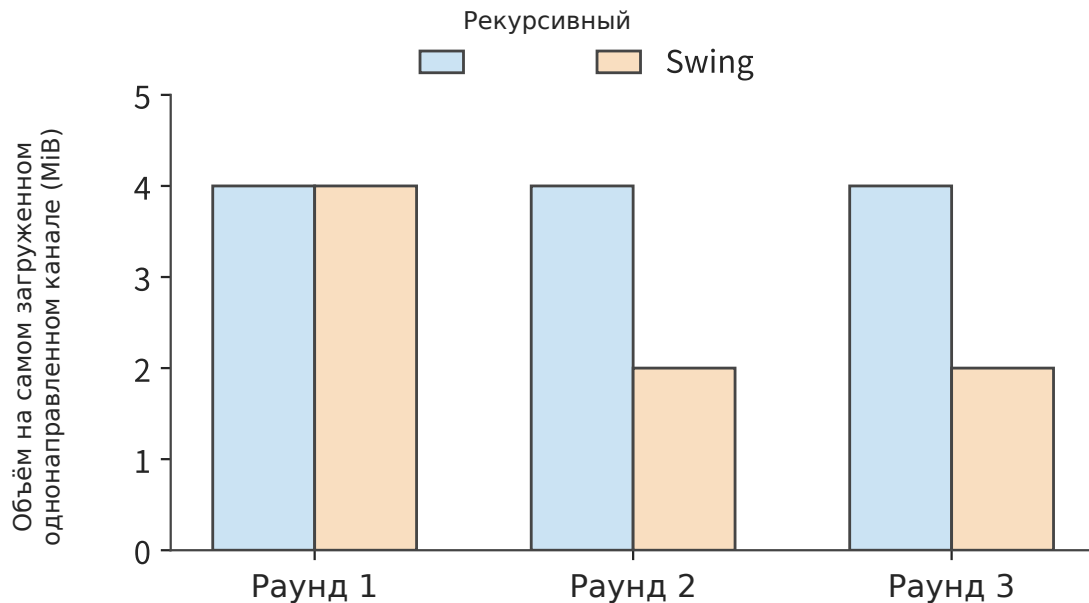


Рис. 6-33. Данные всех отправителей суммируются на каждой направленной линии, после чего для каждого раунда выбирается максимальное значение. В первых трёх раундах рекурсивного алгоритма оно равно 4 MiB, а для Swing — 4, 2 и 2 MiB.

64 GB/s в каждом направлении. Во всех трёх вариантах каждая карта передаёт через PCIe в хост и обратно одинаковый объём данных, но объём данных, проходящих через UPI, изменяется в отношении 2:1:2, а вместе с ним меняется и время передачи между CPU. Положение буферов и порядок карт в кольце совместно определяют маршруты перемещения данных.³⁰

Пример проектирования: тор с прямыми соединениями или коммутируемая сеть? В предыдущих примерах данные либо напрямую проходили по одной линии, либо шли через один коммутатор. При объединении 64 карт в один суперузел обычно выбирают один из двух подходов. Первый — напрямую соединить каждую карту с соседями: расположить 64 карты в кубе $4 \times 4 \times 4$, подключить каждую карту к двум соседям вдоль каждого из трёх измерений и замкнуть начало и конец каждого измерения. Такая топология называется **трёхмерным тором** (torus). Тор — класс топологий, давно применяемый в суперкомпьютерах; его унаследовали TPU. Второй подход — подключить каждую карту только к коммутационным микросхемам, чтобы коммутационный уровень пересылал данные получателю. Этот подход применяется в NVL72 и CloudMatrix384. В обоих вариантах каждая карта имеет одинаковые шесть портов по 50 GB/s в каждом направлении — именно так сконфигурирован ICI каждой микросхемы TPU v4. Различие состоит в том, что на другом конце порта находится соседняя карта или коммутационная микросхема. На рис. 6-34 показан маршрут одной передачи при двух способах соединения.

³⁰Промежуточная передача через PCIe и размещение NUMA, расчёт буфера рядом с отправителем, из которого в этой главе используется только количество байтов. Модель CPU хоста HGX H100 указана в руководстве пользователя DGX H100. Xeon 8480C — специализированная версия Xeon Platinum 8480+, поддерживающего до 4 каналов UPI по 16 GT/s каждый; см. спецификацию Intel и таблицу 1 технического обзора Xeon четвёртого поколения. Intel не публикует пропускную способность UPI в GB/s для каждого направления, поэтому время передачи здесь не пересчитывается. Пропускная способность PCIe Gen5 x16 в каждом направлении составляет 64 GB/s; см. страницу 48 описания архитектуры H100.

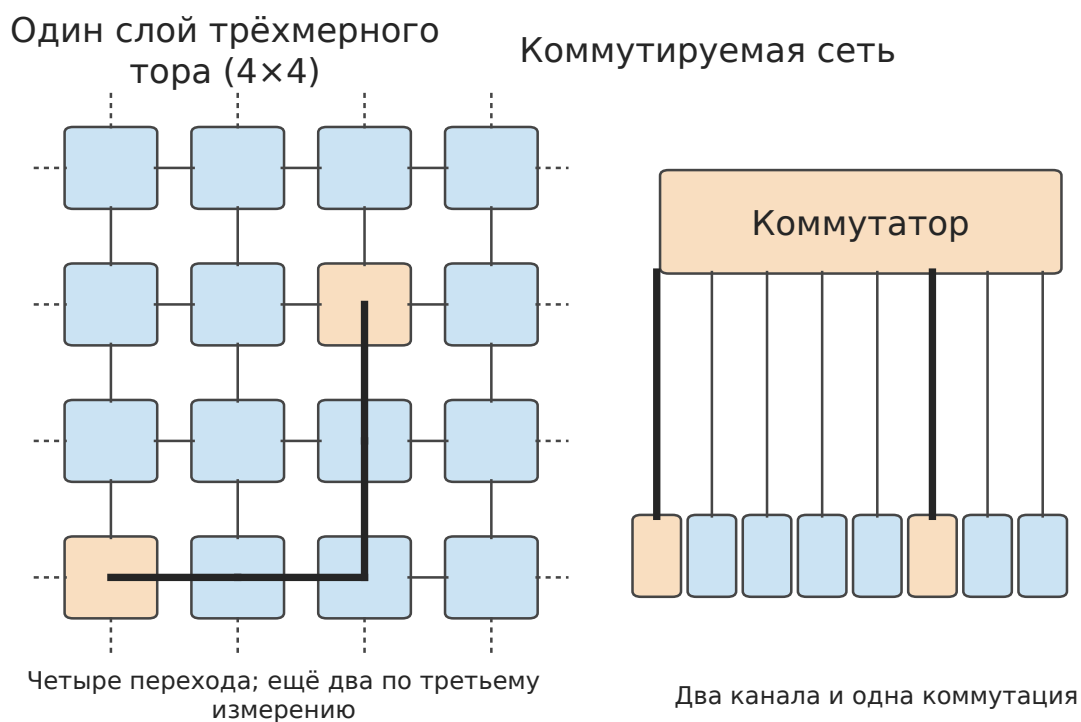


Рис. 6-34. Слева показан один слой трёхмерного тора: каждая карта напрямую соединена с четырьмя соседями, а пунктир показывает замыкание начала и конца каждого измерения. От угловой карты до центральной требуется четыре перехода, а с учётом третьего измерения — не более шести. Справа показана коммутируемая сеть: между любыми двумя картами находятся две линии и одна коммутация. Оранжевым обозначена наиболее удалённая пара карт, а толстыми линиями — маршрут передачи между ними.

Сначала сравним **число переходов**. В торе наиболее удалённые карты находятся друг от друга на расстоянии двух переходов в каждом измерении, всего шести по трём измерениям. В коммутируемой сети между любыми двумя картами находятся две линии и одна пересылка. Предположим, что прохождение данных через маршрутизатор микросхемы или коммутационную микросхему добавляет 200 нс. Это целевая задержка без нагрузки, заданная спецификацией открытого стандарта межсоединений ускорителей UALink 2.0 для коммутационной микросхемы со 128 lane, то есть последовательными каналами; временем распространения по линиям пренебрежём. Между наиболее удалёнными картами тора находятся 5 промежуточных микросхем, а в коммутируемой сети — лишь 1 коммутационная микросхема, поэтому каждая передача в торе занимает на 0,8 мкс больше. Разница для одной передачи невелика, но при decode на каждом слое выполняются два сокращения, то есть 128 операций для 64 слоёв Qwen3-32B. Если каждый раз приходится ждать наиболее удалённую карту, общее дополнительное ожидание в торе составляет около 0,10 мс — примерно 7% от полного времени сокращений для восьми карт TP, равного 1,48 мс. Когда число карт возрастает до $8^3 = 512$, максимальное расстояние в торе достигает двенадцати переходов. В коммутируемой сети достаточно добавить ещё один уровень коммутации, и маршрут будет состоять из четырёх линий и трёх пересылок.

Теперь сравним **паттерны трафика**. Первый — сокращение вдоль измерений: AllReduce разбивается на кольцевые ReduceScatter и AllGather вдоль направлений x , y и z , причём на каждом шаге данные передаются только между соседними картами, а все шесть линий работают одновременно. В этом случае тор задействует всю пропускную способность портов каждой карты и работает так же быстро, как коммутируемая сеть. Именно так компилятор TPU сопоставляет группы параллелизма данных и модели с измерениями тора. Второй паттерн — равномерный All-to-All: каждая карта отправляет по одной порции каждой из остальных 63 карт; именно так работает dispatch в MoE. В торе каждая порция данных проходит в среднем три перехода, по одному на каждое измерение. Если каждая из 64 карт отправляет M байт, все линии в общей сложности должны перенести $3 \times 64M$; при распределении по 384 направленным линиям на каждую приходится $M/2$. В коммутируемой сети на каждый из шести портов карты приходится $M/6$. При $M = 32$ MiB тору требуется не менее примерно 336 мкс, а коммутируемой сети — около 112 мкс, что даёт трёхкратную разницу. Для тора из 512 карт среднее число переходов равно шести, поэтому разница становится шестикратной. Оба паттерна сопоставлены на рис. 6-35.

Наконец, сравним **стоимость оборудования**. Для тора из 64 карт требуется $64 \times 6/2 = 192$ линии и не требуется ни одной коммутационной микросхемы, поскольку маршрутизатор встроен в каждую микросхему. Для коммутируемой сети нужны 384 линии и шесть 64-портовых коммутационных микросхем. Когда число карт превышает число портов одной коммутационной микросхемы, коммутируемой сети требуется ещё один уровень. В CloudMatrix384 для 384 NPU внутри каждого из 48 узлов установлено по 7 коммутационных микросхем первого уровня, которые соединены через 7 подплоскостей, содержащих по 16 коммутационных микросхем второго уровня. Всего используется 448 коммутационных микросхем, причём микросхемы второго уровня размещены в 4 коммуникационных стойках. В торе при любом масштабе каждой карте по-прежнему достаточно шести линий, но число переходов растёт вместе с масштабом, а приходящаяся на карту пропускная способность разреза уменьшается. Это будет рассчитано в разделе 6.5.4.

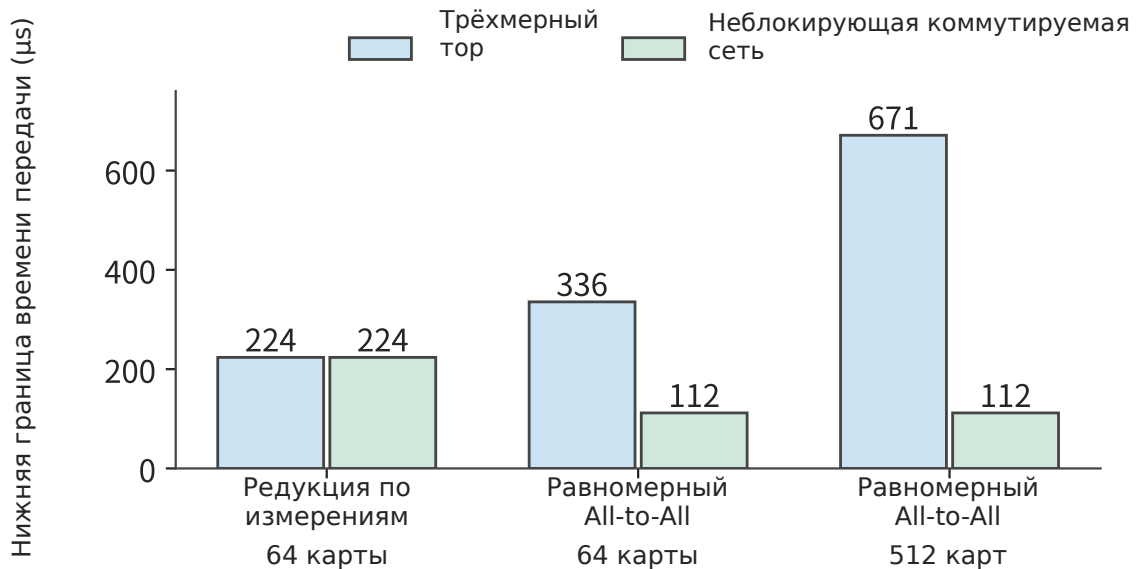


Рис. 6-35. Каждая карта имеет шесть портов по 50 GB/s и отправляет 32 MiB. При сокращении вдоль измерений обе топологии полностью задействуют пропускную способность портов. При равномерном All-to-All данные в торе проходят в среднем три перехода, из-за чего возрастает объём данных на общих линиях; при 512 картах среднее число переходов увеличивается до шести. На рисунке показана нижняя граница, определяемая нагрузкой на линии, без учёта накладных расходов запуска и очередей. Коммутлируемая сеть считается неблокирующей, то есть коммутационный уровень не мешает всем портам одновременно принимать и передавать данные на полной скорости.

В совокупности три сравнения задают критерии выбора топологии: преобладают ли в нагрузке регулярные коллективные операции либо присутствует много небольших передач произвольным получателям и операций All-to-All; сколько карт должен охватывать домен совместной работы; готовы ли мы использовать коммутационные микросхемы и больше кабелей, чтобы расстояние между любыми двумя картами было одинаковым. В разделах 6.5.3–6.5.5 мы увидим, какой выбор по этим трём вопросам сделали NVIDIA, Google и Huawei.³¹

Эксперимент 6-6 · Продолжение: как топология связи и положение буферов влияют на время передачи

- (а) Повторно рассчитайте время одного шага TP16 на двух HGX H100, а затем определите, до какого значения должно снизиться время одного AllReduce между серверами, чтобы время шага TP16 точно сравнялось со временем шага TP8 внутри одного сервера.

³¹В проектном примере используются 6 портов на чип и 50 GB/s на порт в каждом направлении — параметры взяты из TPU v4. В таблице 1 обзора поколений TPU для каждого чипа TPU v4 указаны 6 каналов ICI по 50 GB/s, а в сноске 4 уточняется, что значение приведено для каждого направления; у TPU v5p и Ironwood каждый канал обеспечивает 100 GB/s. В таблице 4 статьи о TPU v4 также указаны 6 каналов по 50 GB/s. Задержка каждой пересылки 200 ns взята из §9.9.2 спецификации UALink 2.0, где она задана как целевая задержка без нагрузки для коммутатора на 128 линий; здесь это же значение применяется к маршрутизаторам внутри чипов тора. 32 MiB — заданный условием объём отправляемых данных на карту; нагрузка на каналы при равномерном All-to-All рассчитывается по среднему числу переходов при маршрутизации по измерениям. Остальные системные данные взяты из §2 статьи о TPU v4 — стоимость оптических каналов, доля OCS и сечение трёхмерного тора; из официальной спецификации NVLink — 18 каналов на GPU и неблокирующее соединение 72 GPU; из §3.3.2–3.3.3 статьи CloudMatrix384 v2 — число коммутаторов первого и второго уровней и неблокирующая конструкция.

- (b) Для 64-портовой NVSwitch сравните два варианта распределения нисходящих и восходящих портов: 32/32 и 48/16. Рассчитайте среднюю доступную пропускную способность каждой линии, когда все линии GPU одновременно передают данные вверх.
- (c) По рис. 6-31 суммируйте объём данных третьего раунда, проходящий по одной направленной линии, и объясните различие между числом переходов и объёмом передачи по линии.
- (d) В примере с NUMA сначала переместите буферы, а затем измените порядок кольца; отдельно изобразите появившиеся и исчезнувшие обращения через границу CPU.
- (e) Замените в примере проектирования 64 карты на тор $8 \times 8 \times 8$ и найдите максимальное число переходов, нагрузку на каждую направленную линию при равномерном All-to-All и число 64-портовых коммутационных микросхем, необходимое для двухуровневой неблокирующей коммутируемой сети. Объясните, какая из этих величин растёт быстрее всего при увеличении числа карт.

6.5.2 Среда соединения, электропитание и физический масштаб

В разделе 6.5.1 линии связи рассматривались как соединения с заданными пропускной способностью и задержкой. Однако для создания таких соединений в реальной системе необходимо также решить вопросы дальности передачи, прокладки кабелей и электропитания. При распространении низколатентного взаимодействия на большие расстояния должна меняться и среда соединения. На небольших расстояниях медные проводники могут непосредственно передавать высокоскоростные электрические сигналы. С увеличением расстояния усиливаются затухание сигнала и перекрёстные помехи, из-за чего приёмнику становится сложнее восстанавливать данные и тактовый сигнал, поэтому требуются более мощные схемы эквализации или ретайминга. Более толстые кабели и большее число соединений также занимают пространство в корпусах и кабельных трассах. При выборе среды соединения необходимо одновременно учитывать дальность передачи, потери сигнала и плотность кабельной разводки.

В оптических соединениях данные модулируют оптическую несущую, что подходит для передачи на большие расстояния, но одновременно увеличивает затраты на оптоэлектронное преобразование, электропитание компонентов и обслуживание. В подключаемых оптических модулях преобразование выполняется внутри заменяемого модуля, а **совместно упакованная оптика** (CPO) переносит оптический движок ближе к коммутационному или вычислительному чипу, сокращая высокоскоростной электрический тракт. Последний вариант повышает плотность соединений внутри корпуса, однако при совместной упаковке оптических компонентов с вычислительным или коммутационным чипом необходимо также совместно проектировать охлаждение, тестирование и замену компонентов. Среда соединения определяет стоимость реализации при заданных расстоянии и пропускной способности, а топология — то, как эти соединения образуют пути передачи данных.³²

Возможности электропитания дополнительно ограничивают число устанавливаемых карт. Карты устанавливаются комплектами по числу карт в сервере. Пусть бюджет мощ-

³² Анализ числа портов и среды интерконнекта. В примерах расчёта числа портов и мощности используются входные данные из условия.

ности системы равен P_b , фиксированная мощность вспомогательного оборудования, такого как коммутаторы и хранилища, равна P_0 , в каждом сервере установлено g карт, а его мощность равна P_s . Тогда

$$N \leq g \left\lfloor \frac{P_b - P_0}{P_s} \right\rfloor. \quad (6-11)$$

Предельная мощность системы DGX H100 с восемью H100, а также CPU, NVSwitch, сетевыми картами и вентиляторами составляет 10,2 кВт, то есть в среднем 1,275 кВт на карту. Предельная потребляемая мощность самой отдельной H100 SXM составляет 700 Вт, а остальная мощность приходится на вспомогательные компоненты сервера.³³ Пусть бюджет мощности группы стоек составляет 120 кВт. Если в этом примере учитывать только серверы и принять P_0 равным 0, можно установить не более 11 серверов, то есть 88 карт. Для 64 карт (8 серверов) требуется 81,6 кВт, для 72 карт (9 серверов) — 91,8 кВт, а для 96 карт (12 серверов) — 122,4 кВт, что превышает бюджет мощности. Конкретную конфигурацию вариантов с 64 и 72 картами необходимо дополнительно определить с учётом числа портов коммутаторов и доступного пространства для установки.

Охлаждение задаёт ещё один верхний предел. В стойках с воздушным охлаждением вентиляторы отводят тепло в воздух машинного зала. В техническом документе ASHRAE по жидкостному охлаждению указано, что стойке мощностью от 40 до 50 кВт требуется воздушный поток до 5 000 cfm (кубических футов в минуту), тогда как лучшие напольные вентиляционные решётки способны обеспечить лишь 1 900 cfm. В стойке мощностью 50 кВт сами вентиляторы потребляют не менее 5 кВт. В этом примере примем 40 кВт в качестве порога для стойки с воздушным охлаждением. В одну стойку можно установить не более трёх DGX H100, то есть 24 карты с общей мощностью 30,6 кВт. Четырём системам потребуется уже 40,8 кВт. В примере компоновки стоек из эталонной архитектуры DGX SuperPOD H100 мощность каждой стойки превышает 40 кВт. Если сосредоточить 64 карты мощностью 81,6 кВт или 72 карты мощностью 91,8 кВт в небольшом числе стоек, этот порог будет значительно превышен. Потребуется перейти на жидкостное охлаждение, передающее тепло непосредственно охлаждающей жидкости, а не воздуху машинного зала.³⁴

Если один коммутационный блок способен подключить только 64 конечные точки, то для подключения 65-й карты может потребоваться полный дополнительный комплект коммутационного оборудования. При последовательном увеличении числа карт стоимость сопутствующего коммутационного оборудования растёт не плавно, а скачкообразно при пересечении подобных границ комплектных групп. После определения приблизительного диапазона по формуле (6-11) необходимо скомпоновать систему из таких единиц установки, как коммутаторы, лотки и стойки, и лишь затем можно получить конфигурацию, которую реально построить.

Эксперимент 6-7 · Дополнительное задание: как бюджет мощности и

³³Руководство пользователя DGX H100, таблица 3: максимальная мощность системы — 10,2 кВт; значение 700 W для H100 SXM приведено в таблице оборудования. Предельная мощность в расчёте на сервер приведена в поле `gask` результатов расчётов для главы 6.

³⁴ASHRAE TC 9.9, «Emergence and Expansion of Liquid Cooling in Mainstream Data Centers», 2021, страницы 14 и 28: в техническом документе не указан единый предел воздушного охлаждения; значение 40 kW выбрано в этом примере как порог на основе приведённого сравнения воздушных потоков. В примере компоновки стоек на странице 10 эталонной архитектуры DGX SuperPOD H100 мощность каждой стойки превышает 40 kW. Ограничение мощности в расчёте на сервер приведено в поле `gask` результатов вычислений для главы 6.

требования комплектного расширения ограничивают число ускорителей

- (а) Отдельно пересчитайте общую мощность при установке 64, 72 и 96 карт. По формуле (6-11) определите максимальное число карт, которое можно установить при увеличении бюджета мощности до 150 кВт.
- (b) Если после превышения 64 карт для коммутационного оборудования второго уровня необходимо зарезервировать 10,2 кВт, что соответствует мощности одного сервера, определите максимальное число карт при бюджете мощности 120 кВт.
- (c) Используя бюджет мощности и условие о дополнительном коммутационном оборудовании из пункта (b), предположите, что расширение возможно только полными группами по четыре сервера (32 карты). Определите максимальное число устанавливаемых карт и объясните, почему верхние пределы в пунктах (a)–(c) различаются.
- (d) Используя полученное ранее время обслуживания одного экземпляра, сравните обслуживающую способность двух из рассмотренных конфигураций ускорителей.

6.5.3 NVIDIA: от восьми GPU в сервере до серверной стойки

Порты, расстояние и мощность совместно ограничивают масштаб системы. В этих условиях несколько поколений систем NVIDIA развили межсоединение от прямых подключений внутри корпуса до коммутационной сети масштаба серверной стойки. В DGX-1 восемь GPU образуют гибридную кубическую ячеистую сеть NVLink: некоторые GPU соединены напрямую, а обмен данными между остальными требует промежуточной передачи. Размещение вычислительных задач с интенсивным обменом на соседних GPU позволяет снизить нагрузку на промежуточную передачу и общие каналы связи.

В DGX-2 используются 16 GPU V100 и NVSwitch. Коммутационная структура заменяет вопрос «соединена ли конкретная пара GPU напрямую» вопросом «сколько данных коммутационный уровень способен передавать одновременно», расширяя доступные возможности совместной работы. DGX A100 снова использует сервер с восемью GPU в качестве базового блока и оснащается NVLink/NVSwitch. Эти системы различаются количеством GPU и структурой межсоединений с учётом интерфейсов чипов, пространства в корпусе и требований приложений.³⁵

GB200 NVL72 объединяет 72 GPU и 36 CPU Grace в системе NVLink масштаба серверной стойки. Благодаря этому большее число GPU внутри стойки может интенсивно обмениваться данными через NVLink. Вернёмся к примеру из раздела 6.5.1: когда TP16 для Qwen3-32B распределён между двумя HGX H100, 128 операций редукции по сети занимают около 4,19 мс, а один шаг — около 6,67 мс. Если эти 16 GPU находятся в одном коммутационном домене NVLink (NVLink Switch System в DGX H100 поддерживает подключение до 256 GPU H100), то при 0,822 мкс на раунд кольцевая редукция на 16 GPU требует в общей сложности 30 раундов, 128 операций занимают около 3,16 мс, а один шаг — около 5,64 мс, что всё ещё медленнее TP8 внутри одного сервера с его 5,15 мс. У этой модели всего 8 KV-голов: при TP16 объём KV, считываемый каждым GPU, уже

³⁵Описание архитектуры V100, топология NVLink и приложение о DGX-1; описание архитектуры A100, приложения о NVLink/NVSwitch и DGX A100. Историческая 16-карточная конфигурация DGX-2 также приведена в описании платформы в статье о ZeRO.

не уменьшается, тогда как число раундов в кольце удваивается. Для таких моделей ценность более крупного домена межсоединения заключается не в ещё более мелком разделении одного сеанса, а в размещении большего числа экземпляров либо модели, размер которой превышает ёмкость одного сервера.³⁶

Межсоединение масштаба серверной стойки позволяет большему числу GPU участвовать в работе одного экземпляра. Для небольших моделей, помещающихся на одном GPU, число экземпляров можно выбирать исходя из требований к задержке и пропускной способности; для крупных моделей, веса которых превышают суммарную ёмкость восьми GPU, можно использовать больше GPU для хранения весов, а обмен данными на каждом слое выполнять через внутристоечное межсоединение.

6.5.4 TPU: согласование топологии и нагрузки

Подход NVIDIA заключается в расширении области связности GPU с помощью коммутационного слоя. Тензорные процессоры Google TPU, напротив, используют тор, рассмотренный в примере проектирования из раздела 6.5.1, чтобы разбиение модели, раскладка компилятора и физические коммуникации соответствовали друг другу. Двумерный тор соединяет чипы по двум измерениям, причём в каждом измерении начало соединено с концом; в трёхмерном торе добавляется ещё одно измерение, поэтому данные могут передаваться в большем числе направлений.

В основе TPU v4 лежит модуль электрического соединения из 64 чипов размером $4 \times 4 \times 4$, а несколько таких модулей соединяются с помощью оптических коммутаторов каналов (OCS, соединяющих порты устройств посредством изменения оптических маршрутов). Такое разбиение учитывает одновременно топологию сети и размещение оборудования: 64 чипа и 16 CPU-хостов помещаются в одну стойку, тогда как более крупный модуль 8³ содержит 512 чипов и требует межстоечных соединений.³⁷

Пропускная способность разреза позволяет анализировать коммуникационные возможности такой регулярной топологии после масштабирования. Если трёхмерный тор размером $k \times k \times k$ разделить пополам вдоль одного из измерений, через две поверхности разреза пройдут в общей сложности $2k^2$ физических соединений — рёбер этого разреза, называемых рёбрами бисекции. При увеличении k с 4 до 8 число чипов возрастает с 64 до 512, то есть в восемь раз, тогда как число рёбер разреза увеличивается лишь с 32 до 128, то есть в четыре раза. Если каждому чипу требуется передать в другую половину одинаковый объём данных, средняя пропускная способность разреза, приходящаяся на один чип, уменьшается вдвое. Различие между темпами роста числа чипов и пропускной способности разреза определяет, какие схемы параллелизма подходят для такой формы топологии.

Фиксированная топология определяет пропускную способность между разделами, а расположение свободных чипов влияет на то, сможет ли задача получить требуемую конфигурацию. OCS на более крупном масштабе перекоммутирует эти модули электрического соединения, выделяя различным задачам подмножества чипов подходящей конфигурации; такие подмножества называются срезами. Рассмотрим

³⁶Официальный архив GB200 NVL72: в зафиксированном снимке описаны 72 GPU, 36 CPU и организация NVLink в масштабе стойки. NVLink Switch System поддерживает подключение до 256 карт H100; см. страницу 48 описания архитектуры H100. Время кольцевой редукции на 16 картах приведено в поле `two_servers.nvlink_domain_tp16` результатов вычислений для главы 6.

³⁷Статья о TPU v4, обзор нескольких поколений систем TPU: конструкция электрического интерконнекта cube, OCS, разделение на срезы и изоляция отказов.

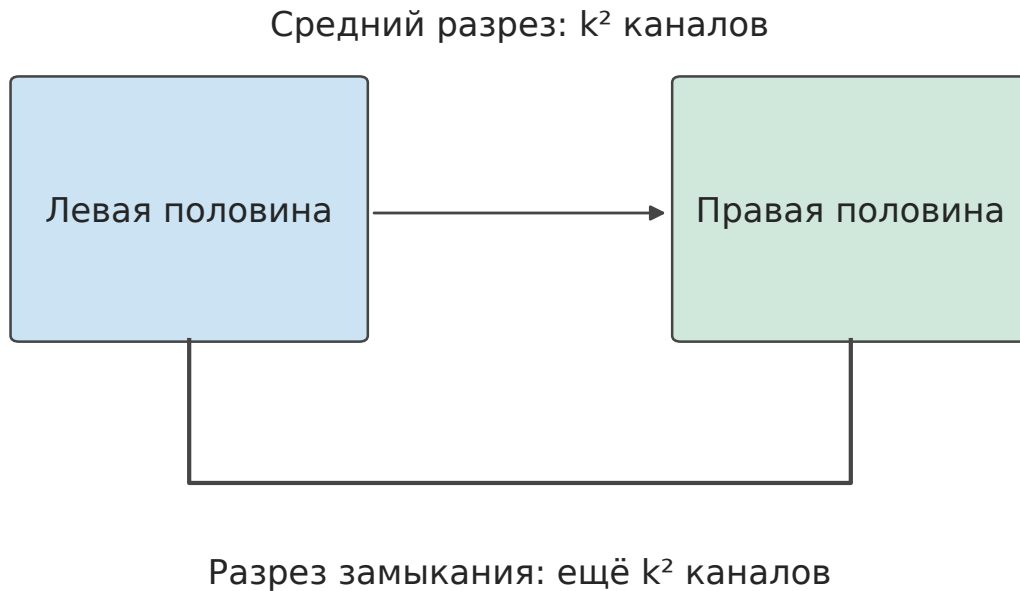


Рисунок 6-36. Чтобы разделить трёхмерную тороидальную сеть пополам вдоль одного измерения, необходимо разорвать соединения посередине и соединения между началом и концом. В каждом месте находится k^2 соединений, всего — $2k^2$.

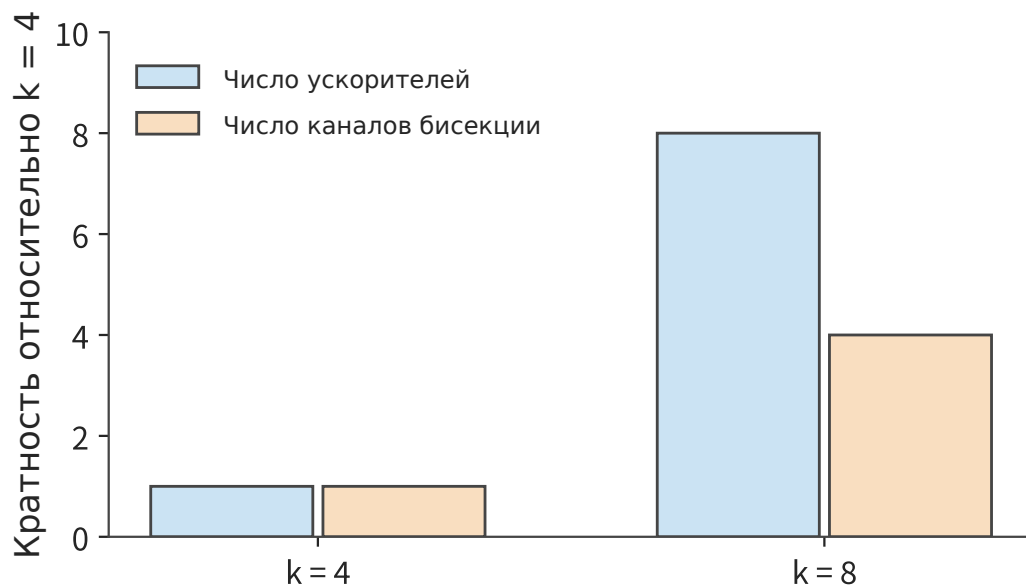


Рисунок 6-37. При увеличении k с 4 до 8 число чипов возрастает с 64 до 512, а число рёбер бисекции — с 32 до 128. Для обоих показателей за основу взято значение при $k = 4$. k обозначает число узлов в каждом измерении трёхмерного тора, а общее число узлов равно k^3 .

сетку из 4×4 чипов: если свободны две смежные строки, на них можно разместить две задачи, каждой из которых требуется смежный блок из 2×2 чипов; но если те же восемь свободных чипов расположены в шахматном порядке, ни одного смежного блока 2×2 найти не удастся. Перекоммутация позволяет изменить доступную для выделения конфигурацию.

Перекоммутация целесообразна, только если сэкономленное время выполнения превышает затраты времени на неё саму. В исследовании Morphlux топология оптических соединений перестраивается под задачу, чтобы соединения соответствовали её коммуникационным потребностям. Предположим, восемь чипов TPU v4 выполняют кольцевую операцию AllReduce для входных данных объёмом 8 MiB, и каждый чип передаёт 14 MiB. Если изначально в этом срезе для данной коммуникационной группы каждому чипу доступно только одно соединение ICI со скоростью 50 GB/s, а после перекоммутации — три, то составляющая времени передачи снижается примерно с 293,6 μ s до 97,9 μ s, что даёт экономию около 195,7 μ s за одну операцию. Переключение оптических маршрутов посредством OCS в TPU v4 занимает миллисекунды: если перекоммутация требует 1 ms, затраты окупятся после 6 повторений; если 100 ms — после 511 повторений. Чем дольше выполняется задача, тем выгоднее устанавливать более подходящие соединения.³⁸

6.5.5 Unified Bus и CloudMatrix

Системы NVIDIA и TPU в основном исходят из задачи соединения ускорителей. UB компании Huawei расширяет постановку задачи до доступа к ресурсам: если соединить два хоста, сможет ли устройство на одном хосте напрямую использовать память другого? Первые размышления автора о Unified Bus возникли из-за границы между внутривостовой шиной и межхостовой сетью. Даже при чтении одних и тех же данных после пересечения границы хоста программному обеспечению обычно приходится переходить на интерфейс передачи сообщений и заново организовывать буферы, а каждый дополнительный уровень абстракции увеличивает время. UB стремится устранить эти уровни абстракции и расширить область прямого доступа к ресурсам, чтобы вычислительные ресурсы, память и хранилища можно было перекомпоновывать под конкретные задачи.

В этой книге UB рассматривается в двух местах. В данном разделе UB представлен как межсоединение суперузла, что позволяет ответить на три вопроса, связанные с масштабом: где подключается контроллер, межсоединение какого размера помещается в кэш на кристалле и сколько времени занимает установление всех соединений. В главе 7 рассматривается путь одного удалённого обращения: в разделе 7.1.3 объясняются мотивы разработки унифицированного интерфейса, в разделе 7.3.3 поэтапно выводится задержка одного чтения, в разделе 7.3.4 вычисляется частота запросов, в разделе 7.4.2 выводятся рост состояния соединений и точка переполнения кэша на кристалле, а в разделе 7.4.3 обсуждается порядок указания по требованию. В обоих случаях используются один набор параметров и один вычислительный скрипт, а результаты расчётов сопоставляются со значениями, полученными при моделировании реализации UB автора — OpenURMA.³⁹

³⁸Физические тракты коллективной коммуникации и разнесение заданий по времени, расчёт физических трактов 96 сообщений, избранные материалы NSDI 2024, версия Morphlux и заметки по материалам. OCS в TPU v4 основан на MEMS-зеркалах, а переключение занимает миллисекунды; см. §2 статьи о TPU v4.

³⁹Параметры и результаты для числа хостов и времени установления соединения приведены в расчёте межсоединения UB; расчёт можно воспроизвести командой `python3 calculations/calc.py ub-fabric --format md`. Размер записи, задержки этапов и количество записей кэша взяты из таблиц 3 и 7, а также §7.2 статьи об OpenURMA. Когерентное межсоединение с каталогом рассчитано по формуле, согласно которой

Такая унификация прежде всего требует ответить на два разных вопроса: к кому обращается операция, что она читает или записывает и когда можно использовать результат; как данные этой операции проходят по сети и надёжно доставляются. В UB обязанности первого типа относятся к **транзакционному уровню**, а второго — к **транспортному уровню**. Здесь «транзакция» означает одну коммуникационную операцию, а не транзакцию базы данных с семантикой фиксации и отката.

Jetty на транзакционном уровне предоставляет приложению абстракцию конечной точки для отправки, приёма и уведомления о завершении, сохраняя идентичность приложения и необходимый для операции контекст. Транспортный канал на транспортном уровне (в UB он называется TP Channel, где TP — сокращение от Transport Protocol и не связано с тензорным параллелизмом) хранит состояние порядковых номеров пакетов, подтверждений, повторных передач и управления перегрузкой. Несколько конечных точек приложений, обращающихся к одному удалённому узлу, могут совместно использовать нижележащий транспортный канал: каждой связи между приложениями не требуется собственная полная копия состояния надёжной передачи. На рис. 6-38 показано это разделение. QP с надёжным соединением (queue pair, пара очередей — коммуникационная конечная точка, состоящая из очередей отправки и приёма) в традиционном удалённом прямом доступе к памяти (RDMA, позволяющем сетевому адаптеру напрямую читать и записывать разрешённую удалённую память) довольно тесно связывает отношения между конечными точками приложений с состоянием транспорта. UB позволяет независимо задавать количество, время жизни и область совместного использования состояний этих двух типов.⁴⁰

Ещё один ключевой выбор — **семантика памяти**. В односторонней операции инициатор напрямую указывает уже разрешённый адрес удалённой памяти, а удалённому приложению не требуется подготавливать соответствующий запрос на приём для каждой операции. UB поддерживает два способа инициации: Load/Store иницируются инструкциями процессора, а Read/Write — явно отправленными запросами. Оба способа позволяют обращаться к удалённой памяти, но ни один из них не предполагает автоматического поддержания когерентности кэшей между устройствами. В разделе 7.3.3 сравниваются эти способы, а в разделе 7.4.1 объясняется, когда данные становятся видимыми и когда операция считается завершённой.

Расположение контроллера. Традиционный сетевой адаптер представляет собой периферийное устройство PCIe; этот путь инициации уже был прослежен в разделе 6.4.4. Когда процессор иницирует удалённое чтение, дверной звонок, чтение дескриптора, чтение данных на целевом узле, обратная запись данных и запись сведений о завершении в сумме пять раз пересекают PCIe, причём каждый переход занимает несколько сотен наносекунд. UB подключает контроллер к шине процессора на кристалле: процессор выполняет инструкцию Load, запрос по шине на кристалле напрямую поступает в контроллер, а возвращённые данные непосредственно записываются в регистр. На рис. 6-39 сопоставлены эти два пути инициации. В разделе 7.3.3 этапы обоих путей последовательно суммируются: при односторонней задержке линии 100 ns чтение 64 B через периферийный сетевой адаптер PCIe занимает около 2,2 µs, из которых около 1,65 µs приходится

для каждой строки кэша хранится 1 бит на каждый другой узел; 72 GPU в NVL72 — публично известная конфигурация продукта этого поколения, а не результат, выведенный в этой книге.

⁴⁰Ли Боцзе, «Размышления о принципах Unified Bus», разделы о Jetty, порядке транзакций и Load/Store; статья об OpenURMA, редакция от 2026-06-02 (arXiv:2605.28717), §3 — проектирование, §7–§9 — состояние и задержка, §11 — полносистемная проверка, §13 — сводка результатов. Источники и область применимости данной интеграции.

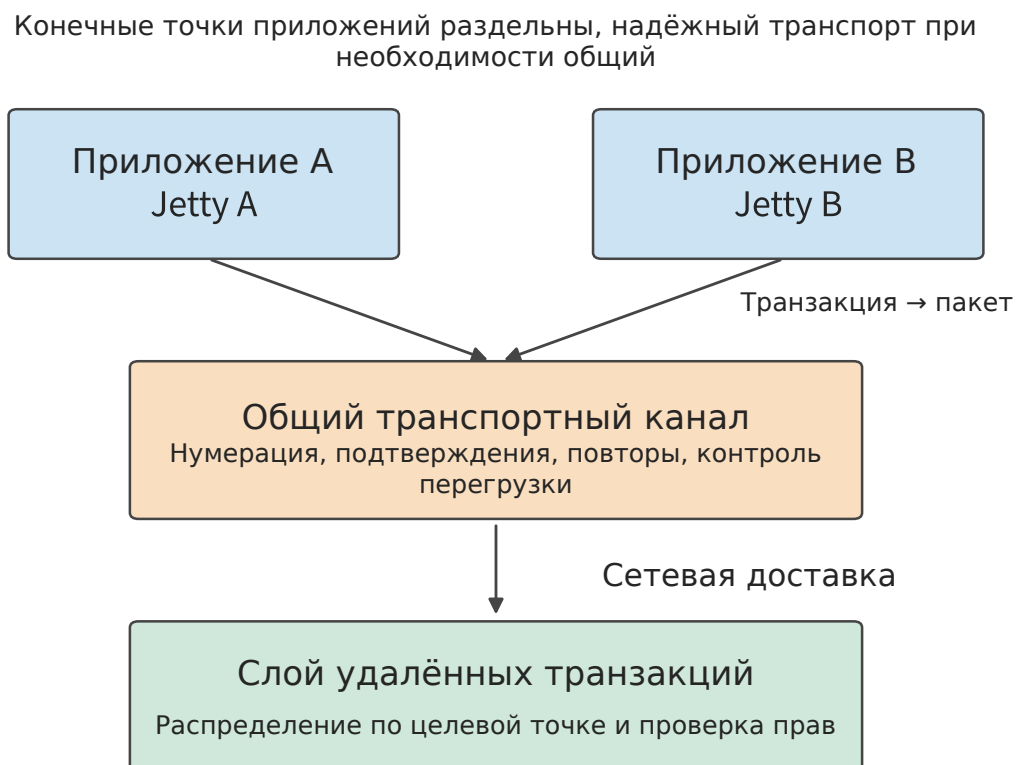


Рис. 6-38. Два локальных приложения сохраняют собственные Jetty и обращаются к удалённой конечной точке через общий транспортный канал. Транзакционный уровень различает операции и получателей уведомлений об их завершении, а транспортный уровень обеспечивает надёжную доставку пакетов. Совместное использование канала не отменяет проверку разрешений и не устанавливает автоматически глобальный порядок между приложениями.

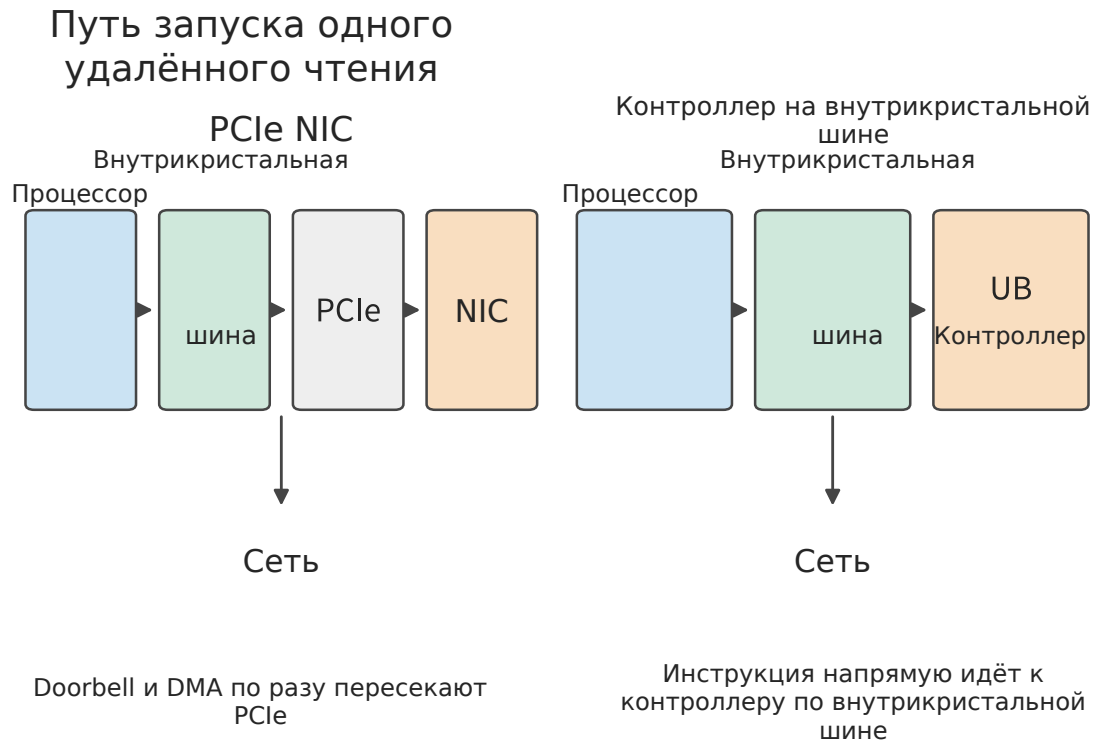


Рис. 6-39. Слева сетевой адаптер подключён через PCIe: дверной звонок, дескриптор запроса, возвращаемые данные и сведения о завершении по одному разу пересекают PCIe между процессором инициирующего узла и сетевым адаптером, а при чтении данных сетевым адаптером целевого узла происходит ещё одно пересечение. Справа контроллер подключён к шине на кристалле, поэтому инструкция процессора поступает в него напрямую. Участки обоих путей от контроллера до сети одинаковы.

на пять пересечений PCIe; чтение через контроллер на шине кристалла занимает около 0,42 μs . Экономия времени достигается не сокращением продолжительности прежних этапов, а их устранением благодаря общему адресному пространству процессора и контроллера. Устранение уровней абстракции, позволяющее процессору обращаться к удалённой памяти как к локальной, и есть источник преимуществ UB. Оставшиеся 0,42 μs в основном определяются задержкой линии и доступом к памяти удалённого узла, что уже близко к аппаратному пределу.

Масштаб межсоединения и ёмкость состояния на кристалле. Сетевой адаптер кэширует на кристалле контекст каждой коммуникационной связи; здесь принимается ёмкость кэша 256 KiB. Пусть межсоединение содержит H хостов, а каждый хост — A конечных точек приложений, причём конечные точки любых двух хостов могут обмениваться данными. Состояние соединений можно организовать двумя способами. Первый — **попарные соединения**, примером которых служат надёжные соединения RoCE (RDMA поверх Ethernet): каждая локальная конечная точка устанавливает отдельное соединение с каждой конечной точкой на остальных $H - 1$ хостах, а для каждого соединения хранится контекст размером 512 B. Второй — применяемая в UB схема **«конечная точка плюс канал»**: для каждой локальной конечной точки хранится одна запись Jetty размером 20 B и одна запись сегмента памяти размером 32 B, а каждому удалённому хосту соответствует транспортный канал размером 56 B. Объём состояния, хранимого каждым сетевым адаптером при этих двух способах, соответственно равен

$$S_{\text{pair}}(H) = 512A^2(H - 1) + 32A, \quad S_{\text{UB}}(H) = 52A + 56(H - 1).$$

Пусть $A = 8$. При попарных соединениях каждый дополнительный хост увеличивает состояние на 32 KiB, поэтому кэш размером 256 KiB вмещает состояние только для 8 хостов. При схеме «конечная точка плюс канал» каждый дополнительный хост увеличивает состояние лишь на 56 B, поэтому тот же кэш способен обслуживать более четырёх тысяч хостов. Для 192 хостов CloudMatrix384 попарным соединениям требуется около 6 MiB — в 24 раза больше ёмкости кэша, тогда как схеме «конечная точка плюс канал» нужно лишь около 11 KiB. Состояние, не помещающееся в кэш, приходится заново считывать из памяти хоста при каждой операции. Его стоимость вычисляется в разделе 7.4.2: при попарных соединениях к каждой операции добавляются два чтения PCIe DMA, занимающие около 1 μ s.

Третий способ организации — межсоединение с когерентными кэшами, к которому относится NVLink: ускоритель может помещать данные из памяти удалённого узла в собственный кэш. Чтобы поддерживать согласованность копий, требуется каталог, указывающий, какие удалённые узлы хранят каждую строку кэша; для каждого удалённого узла необходимо не менее 1 bit на строку. Пусть каталог отслеживает $W = 2^{20}$ строк кэша (64 MiB данных), а каждая строка дополнительно имеет тег размером 8 B. Тогда каталог занимает $W((H - 1)/8 + 8)$ B: около 8 MiB для двух хостов и около 136 MiB для 1024 хостов. При каждой записи также требуется отправлять сообщения об инвалидации всем удалённым узлам, хранящим копии, поэтому число сообщений линейно растёт с H . Эти две статьи расходов ограничивают масштаб межсоединения с когерентными кэшами несколькими десятками удалённых узлов; 72 GPU системы NVL72 находятся именно в этом диапазоне. UB не поддерживает когерентность кэшей между хостами: приложение самостоятельно определяет, когда данные становятся видимыми другим устройствам (раздел 7.4.1). Поэтому из трёх кривых состояния только кривая UB простирается до тысяч хостов. Эти три кривые показаны на рис. 6-40.

Время установления соединений. Количество состояний также определяет число управляющих операций, выполняемых при запуске задания. Каждое попарное соединение проходит создание и три перехода между состояниями, то есть требует в общей сложности 4 системных вызова. Кроме того, с удалённым узлом необходимо один раз обменяться идентификаторами соединения, что требует одного внеполосного кругового обмена (одного обмена туда и обратно по каналу вне тракта данных). Если один системный вызов занимает 5 μ s, а один внеполосный круговой обмен — 500 μ s, установление одного соединения занимает около 0,52 ms. Для доступа N локальных конечных точек к M удалённым конечным точкам необходимо установить NM соединений. В схеме «конечная точка плюс канал» достаточно выполнить по 1 системному вызову для каждой из N конечных точек, а для каждого из M удалённых хостов — по 1 системному вызову и 1 круговому обмену. При $N = M = 1024$ для попарной схемы необходимо установить 1 048 576 соединений, что последовательно занимает около 545 s, а параллельно на 32 ядрах — около 17 s. В схеме «конечная точка плюс канал» требуется создать лишь 2048 объектов, что при параллельном выполнении занимает около 16 ms. На рис. 6-41 показан весь диапазон N от 1 до 1024. Этот процесс повторяется при каждом перезапуске задания и каждом масштабировании, поэтому напрямую влияет на время восстановления после сбоя, рассматриваемое в разделе 6.7.2.

Конкретным воплощением этих механизмов UB служит система Huawei

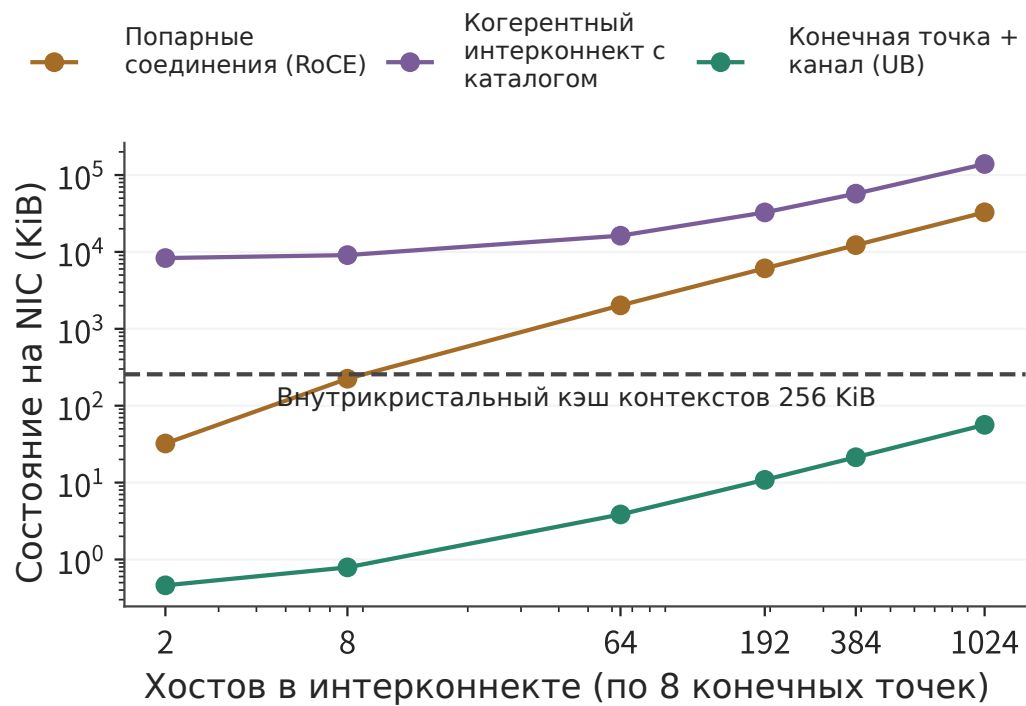


Рис. 6-40. Изменение состояния каждого сетевого адаптера в зависимости от числа хостов в межсоединении при 8 конечных точках на хост для трёх способов организации. Пунктирная линия обозначает кэш на кристалле размером 256 KiB. Парные соединения превышают его ёмкость при 8 хостах, межсоединение с каталогом когерентности кэшей значительно превышает её уже при двух хостах, а схема «конечная точка плюс канал» помещается в кэш даже при тысячах хостов.

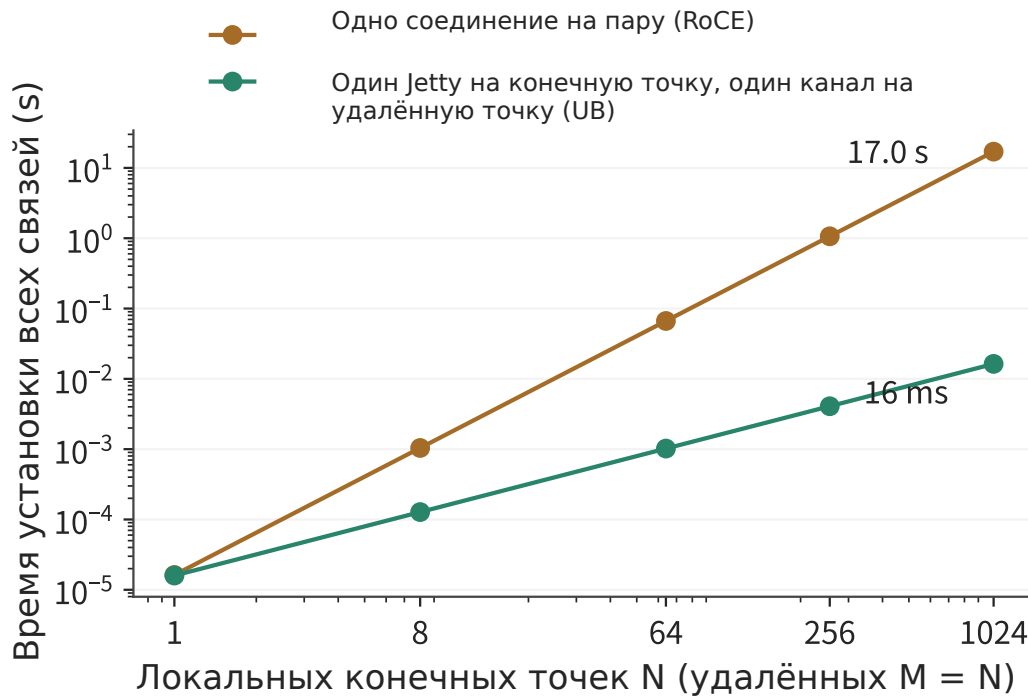


Рис. 6-41. Время установления всех соединений между N локальными и N удалёнными конечными точками при параллельном выполнении на 32 ядрах. Время для попарных соединений растёт пропорционально N^2 , а для схемы «конечная точка плюс канал» — пропорционально N. При N = 1024 значения различаются примерно в тысячу раз.

CloudMatrix384: в ней 384 NPU Ascend 910C и 192 CPU Kunpeng объединены коммутационной структурой UB, а за пределами суперузла используется сеть RDMA. Подключение сервисов обеспечивается сетью виртуального частного облака (VPC), которая изолирует и соединяет ресурсы арендаторов. Если проложить отдельное прямое соединение между каждой парой из 384 NPU, потребуется $384 \times 383 / 2 = 73\,536$ соединений; коммутационный уровень обеспечивает связь между устройствами посредством портовых соединений и маршрутизации.⁴¹

Во всех этих системах можно применять схему параллелизма TP2×EP4 из раздела 6.3.1, однако соответствие между группами и физическими границами различается. Частые операции редукции внутри группы тензорного параллелизма целесообразно размещать в пределах соединения с низкой задержкой, а для обмена результатами между различными группами экспертов требуется достаточная межгрупповая пропускная способность. Только сопоставив коммуникационные группы алгоритма с физическими коммутационными уровнями, можно определить, какие порты задействованы при каждой операции объединения.

При расчёте времени передачи по пропускной способности интерфейса необходимо различать однонаправленную и суммарную двунаправленную пропускную способность. Например, суммарная двунаправленная линейная скорость UB в Ascend 950 составляет 2016 GB/s, а при 72 линиях по 112 Gbit/s однонаправленная скорость равна 1008 GB/s. UBоЕ — способ передачи трафика UB поверх Ethernet, а UB Link — собственный тип канала UB. Они совместно используют один набор сериализаторов/десериализаторов (SerDes

⁴¹ §3.2, рисунок 2 и таблица 1 статьи CloudMatrix384 v2. Некоторые измерения NPU в таблице статьи приведены в расчёте на die, тогда как система из 384 компонентов считается по NPU.

GB200 NVL72



TPU v4



Unified Bus



Рис. 6-42. Уровни соединений в трёх публично представленных системах. NVIDIA соединяет устройства посредством NVLink, TPU v4 использует OCS для соединения электрических блоков межсоединения, а Unified Bus подключает вычислительные ресурсы и память хостов к единому межсоединению.

— схемы, преобразующие параллельные данные внутри микросхемы в последовательный сигнал канала и обратно) в соответствии с предварительной конфигурацией, поэтому один набор физических каналов необходимо распределять между этими двумя назначениями. Способ распределения определяет пропускную способность, фактически доступную заданию.⁴²

Эксперимент 6-8. Расширение: объяснение структуры межсоединения суперузла через требования к обмену данными

- (a) Выберите DGX-1, DGX-2 или DGX A100 и изобразите прямые соединения или коммутационные уровни, через которые проходит одна коммуникационная группа.
- (b) Повторно вычислите количество микросхем и число сбалансированных бисекционных каналов для торов размером 4^3 и 8^3 и объясните, почему доступная каждой микросхеме пропускная способность между разделами снижается.
- (c) Разместите TP2×EP4 на двух хостах с четырьмя картами на каждом и отметьте межхостовую редукцию группы экспертов.
- (d) Увеличьте число конечных точек на каждом хосте до 64 и определите, сколько хостов ещё помещается в кэш размером 256 KiB при попарных соединениях. Затем увеличьте кэш до 1 MiB и определите, поместится ли состояние 192 хостов CloudMatrix384.
- (e) Измените длительность одного внеполосного кругового обмена на 50 μ s, заново вычислите время установления соединений обоими способами при $N = M = 1024$ и укажите, какой из них чувствительнее к задержке кругового обмена.
- (f) Выберите одну из конструкций UB или TPU и объясните, какое требование к установке, обмену данными или доступу к ресурсам привело к такой структуре системы.

6.6 Пул памяти

6.6.1 Нехватка локальной ёмкости при наличии свободной памяти в системе

После того как UB расширил доступ к ресурсам за пределы одного хоста, у него появляется конкретное применение: узлы с недостатком памяти могут заимствовать свободную память других узлов. Тензорный параллелизм, конвейерный параллелизм и параллелизм экспертов из раздела 6.2 распределяют вычисления и веса между разными ускорителями, снижая требования к памяти каждого из них. Другой подход — не менять место вычислений, а разместить часть данных на других узлах. В этом и состоит основное назначение пула памяти: предоставлять распределённую свободную ёмкость задачам, которым она нужна.

Рассмотрим четыре узла, каждый с одним H100 на 80 GB. Задачам требуется соответственно 100, 60, 40 и 40 GB. Общая потребность составляет 240 GB, что меньше совокупной ёмкости 320 GB, однако задаче 0 требуется на 20 GB больше, чем доступно на её узле. На остальных узлах свободно соответственно 20, 40 и 40 GB — всего 100 GB.

⁴²Проверка материалов об UB и Ascend 950, §4 эталонного проекта UB OS, §4.6 официального технического документа 950.

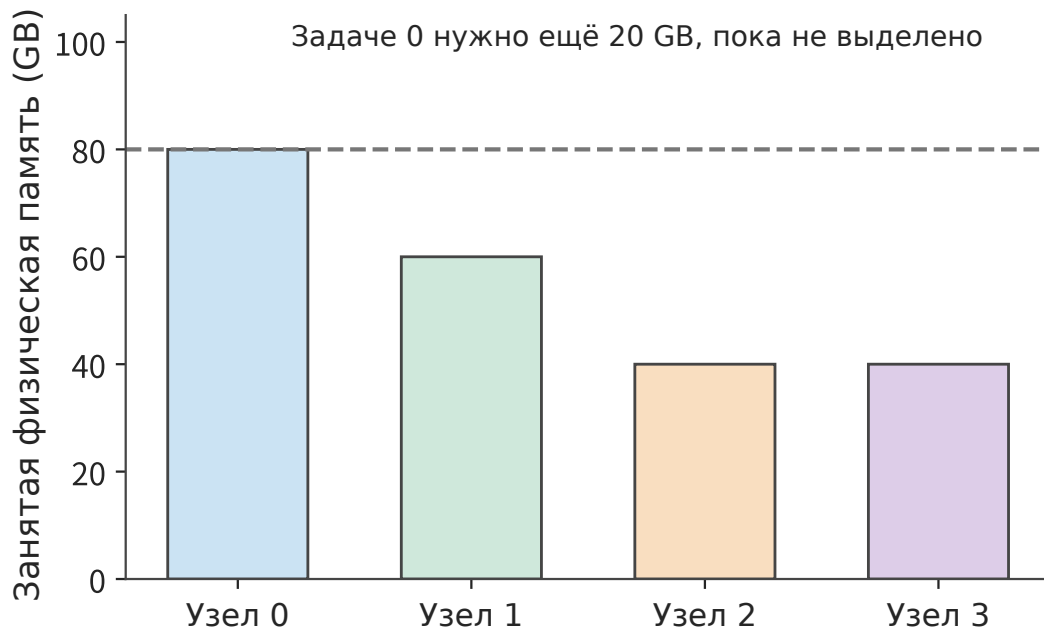


Рисунок 6-43. Физическое использование памяти до заимствования. Ёмкость каждого узла составляет 80 GB, а задаче 0 требуется 100 GB, из которых для 20 GB место хранения ещё не найдено. Цвет обозначает задачу, которой принадлежат данные.

Если целиком перенести задачу 0 на другой узел, её потребность всё равно превысит ёмкость одного узла, равную 80 GB. Если же разместить 20 GB её данных в свободной области узла 1, физическое использование памяти составит 80, 80, 40 и 40 GB. Рабочий набор задачи 0 по-прежнему равен 100 GB, из которых 80 GB находятся локально, а 20 GB — удалённо. Так задача 0 сможет использовать память, которая прежде оставалась свободной на других узлах.

Следует различать свободную ёмкость до и после заимствования: изначально на других узлах было свободно 100 GB памяти, а после передачи 20 GB задаче 0 на четырёх узлах в совокупности остаётся 80 GB. Заимствование не увеличивает объём физической памяти, а лишь позволяет задаче использовать свободное пространство других узлов. Именно такой дисбаланс между серверами находился в центре ранней концепции автора о пуле памяти UB; позднее конкретным вариантом его применения стало хранение и повторное использование KV.

6.6.2 Заимствование памяти и границы доступа

Для показанных на рисунке 20 GB уже найдено место хранения; следующий шаг — предоставить узлу 0 корректный доступ к этой памяти. При заимствовании прежде всего необходимо определить, какой узел предоставляет область памяти и какой её использует. Узел, предоставляющий память, открывает доступ к определённой области, узел-заёмщик создаёт соответствующее отображение адресов, а система управляет правами доступа и отслеживает, когда эту память можно освободить и перераспределить. На время эксклюзивного заимствования право использования передаётся заёмщику, а первоначальный владелец ждёт возврата памяти; при совместном доступе несколько

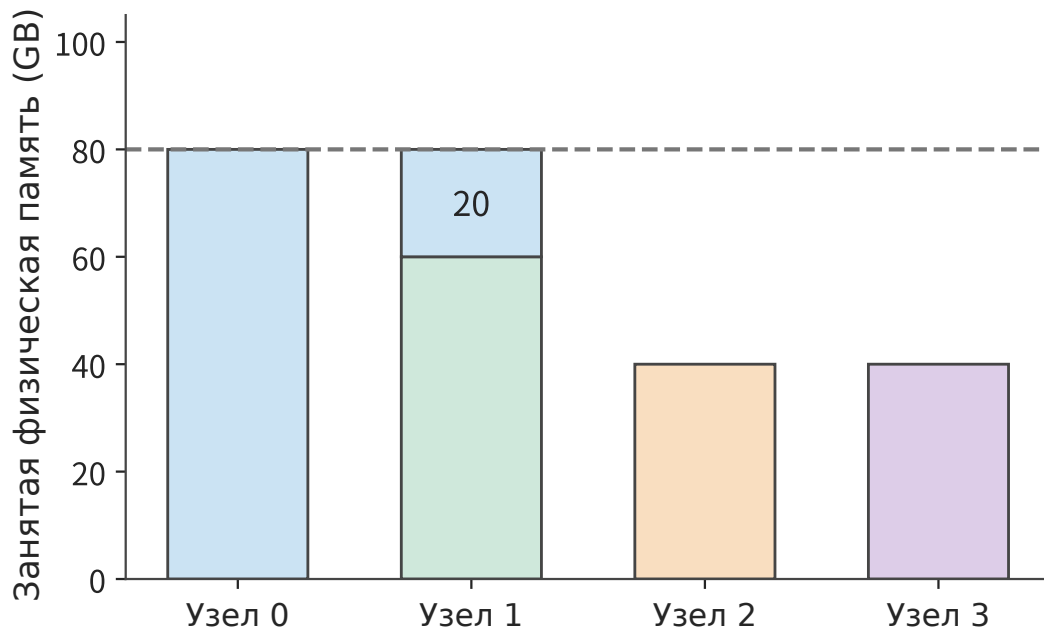


Рисунок 6-44. Дополнительные 20 GB задачи 0 размещены на узле 1. Зелёные 60 GB на узле 1 принадлежат задаче 1, а синие 20 GB — задаче 0; ни один узел не превышает ёмкость 80 GB.

потребителей могут обращаться к ней в соответствии с согласованными правилами согласованности и синхронизации.⁴³

Например, если узел 1 предоставляет область памяти узлу 0, узел 0 должен сначала записать новые KV и лишь затем начать вычисление внимания. Вычисление можно начинать только после подтверждения завершения записи и доступности новых KV для чтения; в свою очередь, узел 1 должен дождаться окончания заимствования, прежде чем перераспределять эту область. Адрес отвечает на вопрос «где обращаться к памяти», права — «кто может обращаться», а порядок завершения операций и освобождения памяти — «когда её можно использовать». Удалённую память можно использовать корректно лишь при совместном учёте адреса, прав и порядка доступа.

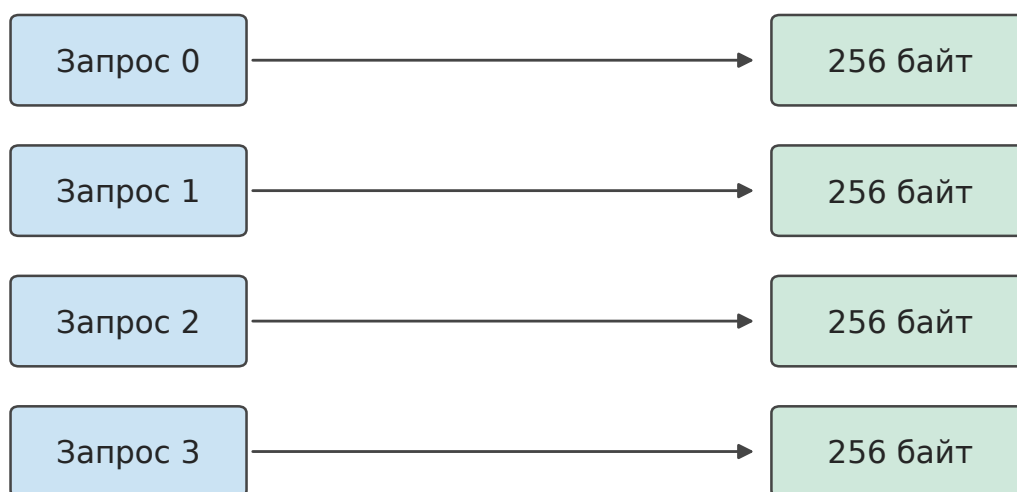
Права доступа и порядок завершения операций обеспечивают корректность данных, а показанное на рисунке ожидание определяет скорость чтения. Между отправкой запроса и получением результата проходит некоторое время; чтобы канал оставался загруженным во время ожидания, необходимо одновременно отправлять несколько запросов. Пусть каждая транзакция чтения возвращает q байт, от отправки до завершения проходит время L , а одновременно в процессе выполнения — то есть уже отправленными, но ещё не завершёнными — могут находиться не более u транзакций. Тогда за каждые L секунд можно вернуть не более uq байт. Совместно учитывая пропускную способность пути и ограничение на количество транзакций в процессе выполнения, получаем

$$B_{\text{window}} = \frac{uq}{L}, \quad B_{\text{usable}} = \min \left(B_{\text{path}}, \frac{uq}{L} \right). \quad (6-12)$$

Пусть два узла соединены путём RDMA через сетевые адаптеры ConnectX-7 с про-

⁴³Проверка материалов об UB и Ascend 950, §4 эталонного проекта UB OS, §4.6 официального технического документа 950.

Запросы в полёте: отправлены,
результат ещё не получен



Туда-обратно 7.52 μ s; до 128 запросов в полёте

Рисунок 6-45. После отправки группы запросов чтения до получения результатов проходит время кругового пути L . Если одновременно в процессе выполнения может находиться не более u запросов, каждый из которых возвращает q байт, то за один круговой путь можно вернуть не более uq байт. Четыре запроса на рисунке служат лишь для демонстрации процесса; в расчёте в основном тексте используются 128 запросов по 256 байт и время кругового пути 7.52 μ s.

пускной способностью 50 GB/s. Один запрос чтения от отправки до завершения проходит путь туда и обратно; примем $L = 7.52 \mu\text{s}$ — удвоенную одностороннюю задержку InfiniBand в $3.76 \mu\text{s}$, измеренную MSCCL++ на платформе с H100 и ConnectX-7. Для каждой транзакции чтения примем $q = 256$ байт — ровно столько занимает вектор К одной KV-головы Qwen3-32B на одном слое для одного токена: 128 измерений BF16. При $u = 128$ объём данных в процессе выполнения в каждой группе составляет 32 KiB, а верхний предел пропускной способности, определяемый количеством таких транзакций, равен приблизительно 4.36 GB/s. Для достижения пропускной способности пути 50 GB/s требуется как минимум

$$u \geq \left\lceil \frac{50 \times 10^9 \times 7.52 \times 10^{-6}}{256} \right\rceil = 1469.$$

Если увеличить только линейную скорость, по-прежнему допуская не более 128 одновременно выполняющихся транзакций, эффективная пропускная способность чтения всё равно будет ограничена их количеством. Если увеличить допустимое количество до 4096, определяемый им предел составит приблизительно 139 GB/s, превысив пропускную способность пути, и узким местом станет сам путь с пропускной способностью 50 GB/s. Пропускная способность пути, размер транзакции и время кругового пути связаны объёмом данных в процессе передачи.

6.6.3 Общая ёмкость и KV-кэш

Чтобы определить, может ли удалённая память заменить локальную, необходимо сравнить рассчитанную в разделе 6.6.2 пропускную способность чтения с потребностью приложения. Даже для одних и тех же 20 GB удалённых данных требуемая средняя пропускная способность зависит от частоты чтения. Полное чтение раз в минуту требует в среднем около 0.33 GB/s, раз в секунду — 20 GB/s, а двадцать раз в секунду — 400 GB/s. При одинаковой ёмкости требуемая средняя пропускная способность различается более чем в тысячу раз.

При достаточном количестве одновременно выполняющихся транзакций полное чтение по пути с пропускной способностью 50 GB/s занимает около 0.40 s. Если начинать чтение раз в секунду, между двумя чтениями остаётся около 0.60 s простоя; если начинать его каждые 0.05 s, новая работа поступает ещё до завершения предыдущей и очередь непрерывно растёт. Если одновременно можно отправить не более 128 транзакций, чтение занимает около 4.59 s, поэтому даже при одном запуске в секунду очередь будет постепенно накапливаться.⁴⁴

Способ использования KV определяет, где их целесообразно размещать. Пока сеанс ожидает следующего ввода пользователя, KV могут долго не считываться; в этом случае удалённая память используется главным образом для хранения состояния, которое извлекается при возобновлении сеанса. В активном сеансе состояние контекста считывается на каждом шаге, поэтому повторное сканирование удалённой памяти постоянно занимает пропускную способность. Размер самого состояния зависит от способа представления контекста в модели: в разделе 9.2.2 параллельно сравниваются число байт и

⁴⁴Поле `memory_pool` расчётов пула памяти для главы 6: занятое пространство четырёх узлов по 80 GB до и после заимствования, пропускная способность окна uq/L , время чтения и сравнение с повторным чтением. Пропускная способность тракта принимается равной 400 Gbit/s для ConnectX-7, время обмена туда и обратно — удвоенной односторонней задержке InfiniBand $3.76 \mu\text{s}$ из таблицы 1 MSCCL++, а локальная пропускная способность — 3350 GB/s для H100.

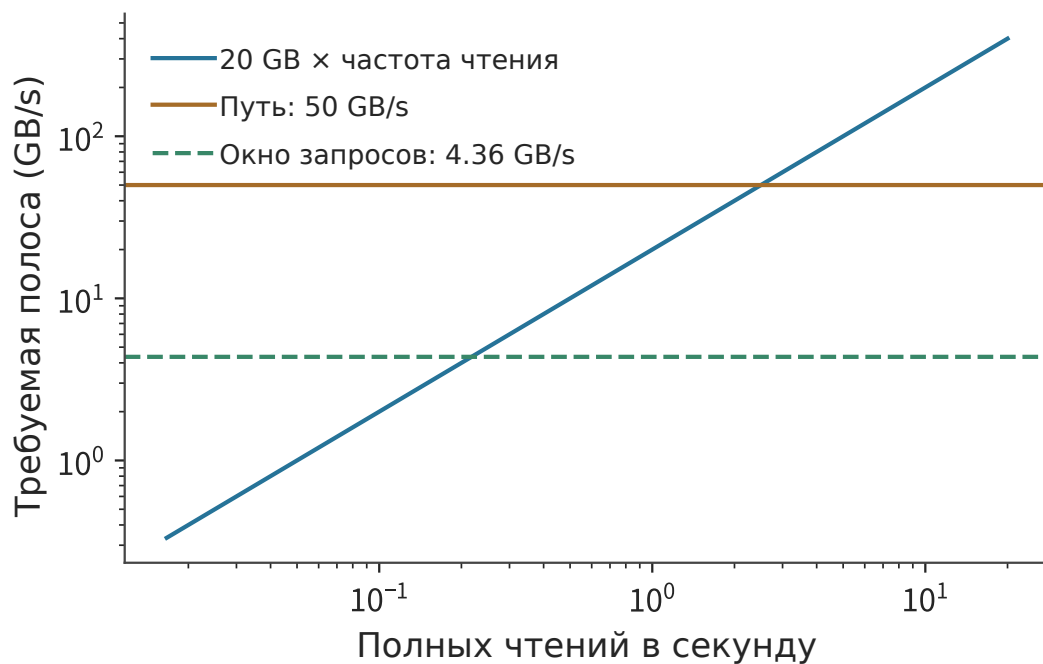


Рисунок 6-46. Частота полного чтения 20 GB и требуемая средняя пропускная способность. Пропускная способность пути составляет 50 GB/s; если каждая транзакция возвращает 256 байт, а время кругового пути равно 7.52 μ s, то 128 одновременно выполняющихся транзакций ограничивают эффективную пропускную способность приблизительно до 4.36 GB/s.

время извлечения одного и того же контекста из 8192 токенов для GQA в Qwen3-8B и компактной MLA в DeepSeek-V3; во втором случае объём составляет приблизительно половину от первого.

Данные также можно сначала извлечь, а затем многократно использовать локально. Если один и тот же снимок объёмом 20 GB требуется прочитать десять раз, последовательное чтение по пути с пропускной способностью 50 GB/s займёт около 4.0 s. Однократное извлечение займёт 0.40 s, а десять последующих сканирований в локальной HBM H100 с пропускной способностью 3350 GB/s — около 0.06 s, то есть всего приблизительно 0.46 s. Второй вариант занимает 20 GB локальной памяти, но экономит около 3.54 s времени доступа. Поэтому для многократно считываемых данных необходимо также сравнивать время удалённого чтения со временем доступа к локальному кэшу, чтобы определить, когда данные следует извлекать в локальную память.

Эксперимент 6-9 · Расширение: когда удалённую память следует читать напрямую, а когда — кэшировать локально

- (a) Повторно рассчитайте физическое использование памяти четырёх узлов до и после заимствования: отдельно определите свободную ёмкость остальных узлов до заимствования и суммарную оставшуюся ёмкость всех узлов после него.
- (b) Каждая транзакция передаёт 256 байт, а время кругового пути составляет соответственно 7.52 μ s и 15.04 μ s. Определите минимальное количество одновременно выполняющихся транзакций, необходимое для достижения пропускной способности 50 GB/s в каждом случае.
- (c) Сравните время однократного чтения при 128 и 4096 одновременно вы-

полняющихся транзакциях. Предполагая, что следующее чтение начинается только после завершения предыдущего, определите максимальное количество полных чтений в секунду при непрерывном чтении.

- (d) Используя приведённые в этом разделе значения пропускной способности удалённой и локальной памяти, определите, начиная с какого количества повторных чтений предварительное извлечение данных в локальную память даёт меньшее суммарное время доступа; также укажите необходимую ёмкость локальной памяти.

6.7 От параллельных решений к масштабу суперузлов

6.7.1 Более крупные группы совместной работы и больше экземпляров инференса

В этом разделе результаты предыдущих разделов используются для размещения нескольких сеансов Qwen3-32B на восьми GPU одного HGX H100. Во всех четырёх сеансах уже завершён prefill, и для каждого сохранён KV-кэш из 131 064 токенов контекста. В каждом сеансе также есть один уже сгенерированный, но ещё не записанный в KV-кэш токен, который служит входными данными для следующего шага. Все четыре сеанса одновременно запрашивают генерацию ещё восьми токенов, в точности заполняя 131 072 позиции, поддерживаемые YaRN. Один экземпляр одновременно обслуживает один сеанс и, завершив эти восемь шагов, переходит к следующему; запросы распределяются между экземплярами по кругу. Все восемь GPU находятся на одном сервере и остаются выделенными этой группе задач до завершения всех четырёх сеансов.

Время каждого шага определяется по формуле (6-5). При продолжении генерации длина контекста в формуле (6-4) последовательно принимает значения от 131 064 до 131 071, а объём обращений к KV-кэшу постепенно растёт. Сумма времени восьми шагов даёт время обслуживания сеанса. Ожидающие сеансы хранят только собственные KV-кэши, а сеансы одного экземпляра совместно используют веса модели. Проверка ёмкости сразу исключает восемь экземпляров на одном GPU: на одном GPU не помещается даже один сеанс с контекстом 128K. Согласно алгоритму из раздела 6.2.2, экземпляр на двух GPU вмещает не более двух сеансов, экземпляр на четырёх GPU — семь, а экземпляр на восьми GPU — шестнадцать; в трёх вариантах развёртывания на каждый экземпляр приходится соответственно один, два и четыре сеанса, поэтому во всех случаях ёмкости достаточно.

Вариант развёртывания	Восемь шагов одного сеанса	Моменты завершения четырёх сеансов	Завершение всех сеансов
Восемь экземпляров на одном GPU	Не помещается	—	—
Четыре экземпляра на двух GPU	119,1 мс	119,1, 119,1, 119,1, 119,1 мс	119,1 мс
Два экземпляра на четырёх GPU	63,8 мс	63,8, 63,8, 127,6, 127,6 мс	127,6 мс
Один экземпляр на восьми GPU	41,2 мс	41,2, 82,4, 123,5, 164,7 мс	164,7 мс

Совместная работа восьми GPU сокращает время продолжения генерации одного сеанса примерно со 119 до 41 мс, однако четыре сеанса выстраиваются в очередь, и все

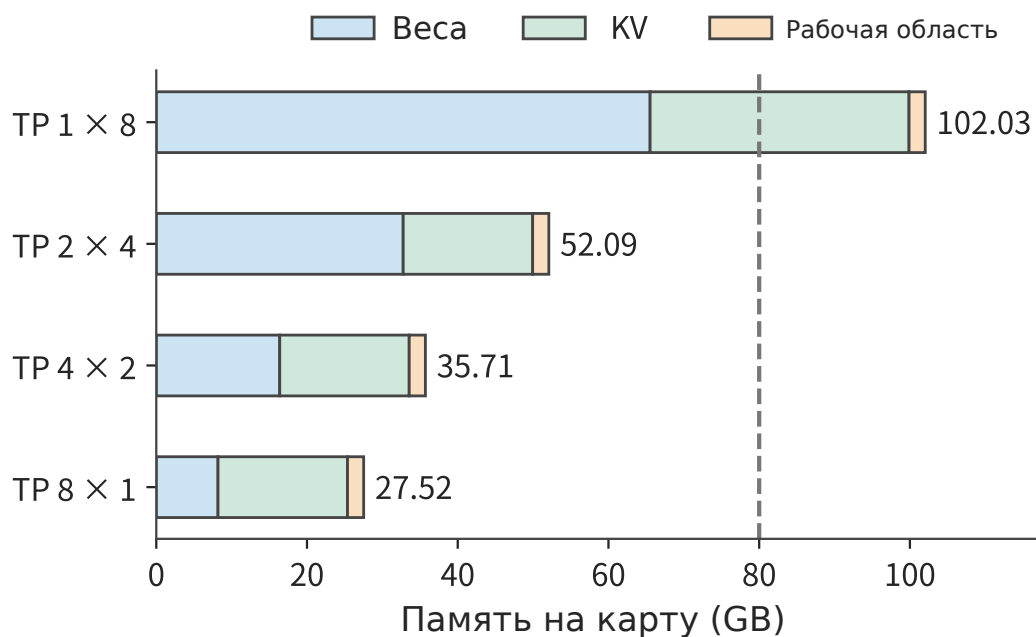


Рисунок 6-47. Требования к памяти каждого GPU для четырёх вариантов развёртывания при сеансах с контекстом 128K, включая веса, KV-кэши всех ожидающих сеансов внутри экземпляра и рабочую область объёмом 2 GiB. Экземпляру на одном GPU требуется 102,03 GB на каждый GPU, что превышает 80 GB у H100 (пунктирная линия); экземплярам на двух, четырёх и восьми GPU требуется соответственно около 52,09, 35,71 и 27,52 GB.

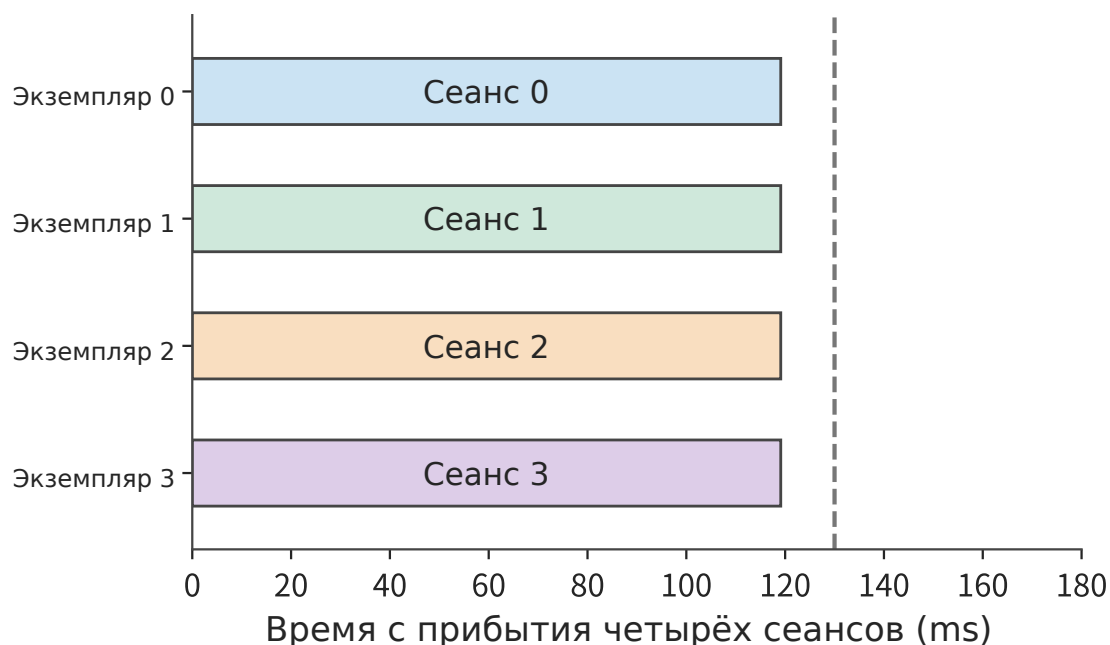


Рисунок 6-48. Четыре сеанса выполняются одновременно, каждый завершается примерно за 119,1 мс. Цвета сеансов совпадают с цветами на соседних схемах конфигураций. Здесь используются четыре экземпляра TP2, по два GPU на экземпляре; в каждом сеансе генерируется продолжение из восьми токенов. Пунктирной линией показан предельный срок 130 мс.

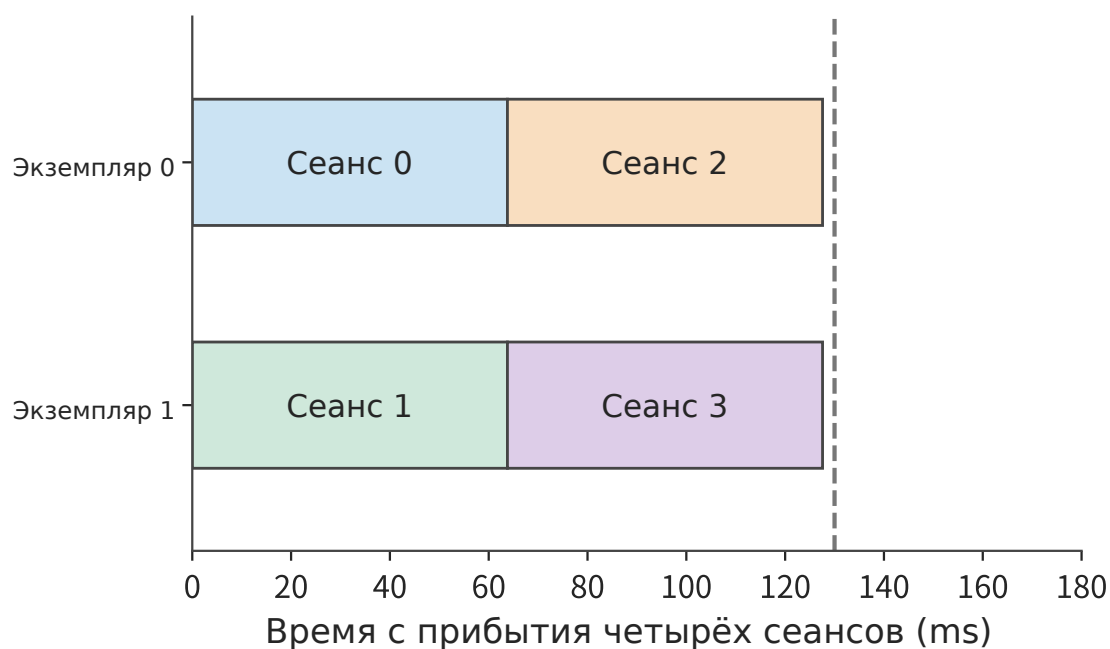


Рисунок 6-49. Каждый экземпляр последовательно обрабатывает два сеанса, которые завершаются примерно за 63,8 и 127,6 мс соответственно. Здесь используются два экземпляра TP4, по четыре GPU на экземпляр; в каждом сеансе генерируется продолжение из восьми токенов. Пунктирной линией показан предельный срок 130 мс.

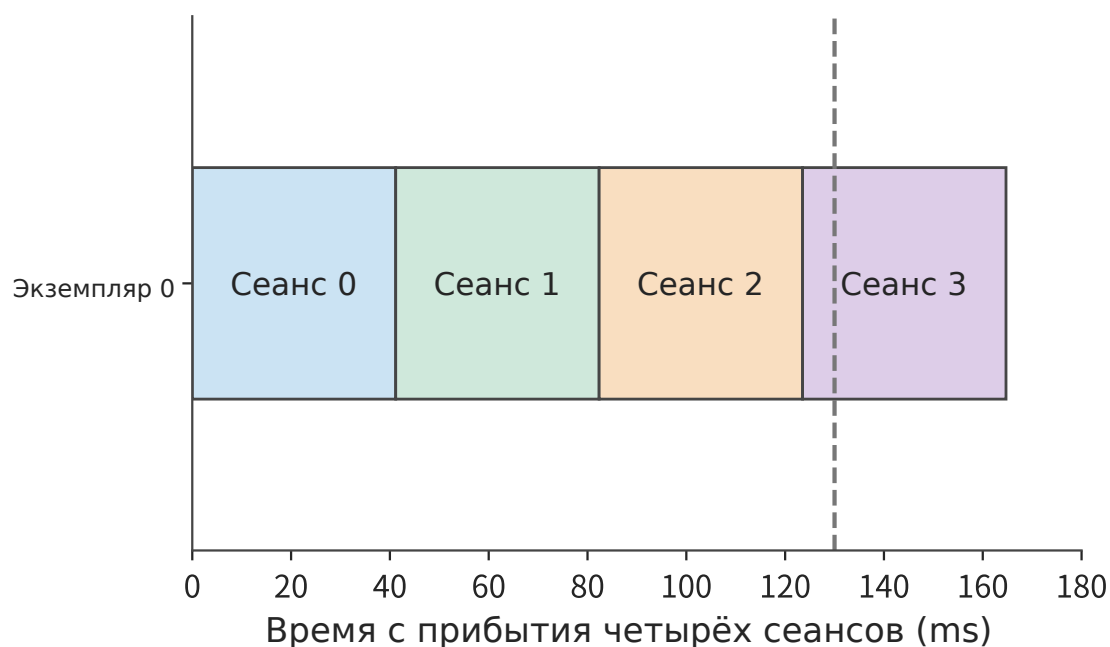


Рисунок 6-50. Время одного сеанса сокращается примерно до 41,2 мс, но четыре сеанса выполняются последовательно, и последний завершается примерно за 164,7 мс. Во всех трёх схемах используется одинаковый масштаб горизонтальной оси. Здесь используется один экземпляр TP8 на восьми GPU; в каждом сеансе генерируется продолжение из восьми токенов. Пунктирной линией показан предельный срок 130 мс.

они завершаются лишь примерно за 165 мс. При использовании четырёх экземпляров на двух GPU продолжение генерации одного сеанса занимает больше времени, зато четыре сеанса могут выполняться одновременно и завершаются примерно за 119 мс. Можно было бы запустить восемь экземпляров на одном GPU, но ни один из них не способен вместить даже один сеанс.

Здесь одновременно используются два вида параллелизма: параллельные вычисления внутри модели и конкурентное выполнение разных запросов. Внутрислойные матрицы можно распределить между несколькими GPU, а независимые сеансы — между несколькими экземплярами, но оба подхода конкурируют за одни и те же восемь GPU. Только разделив эти два уровня, можно выбирать размер группы совместной работы в соответствии с предельными сроками завершения запросов. Планирование обслуживания определяет количество одновременно выполняемых сеансов, разбиение модели — объём коммуникации на каждом шаге, а топология соединений — затраты времени на обмен данными и синхронизацию; вместе эти три фактора определяют приведённые в таблице моменты завершения.

Горизонтальная длина цветных блоков на рисунках обозначает время выполнения одного сеанса, а количество вертикальных дорожек — число сеансов, которые могут выполняться одновременно. Предельный срок завершения определяет компромисс между ними. Если сеанс всего один и его требуется завершить за 50 мс, этому требованию отвечает экземпляр на восьми GPU. Если нужно завершить четыре сеанса не более чем за 130 мс, требованиям отвечают варианты на двух и четырёх GPU, причём вариант на двух GPU завершает работу раньше. Чем жёстче срок для одного сеанса, тем крупнее должна быть группа совместной работы; если независимых сеансов достаточно много, можно использовать больше экземпляров.

При изменении способа соединения расчёты по-прежнему выполняются по исходным формулам. Если, как в разделе 6.5.1, объединить 16 GPU на двух серверах в TP16, каждый шаг займёт около 6,67 мс, а восемь шагов — около 53,3 мс, поэтому достичь цели в 50 мс уже не удастся. Второй сервер лучше использовать для запуска других экземпляров; если же для одного сеанса непременно требуется задействовать больше GPU, необходимо уменьшить дополнительные коммуникационные затраты: улучшить соединение между серверами, изменить алгоритм коллективных коммуникаций либо разместить эти GPU в домене с более тесными связями.

Архитектура с фиксированными весами из главы 4 меняет описанный выше компромисс. Когда веса предоставляет отдельное ROM, сокращается объём чтения весов из HBM, ранее распределённый между несколькими GPU, и доля времени синхронизации TP в длительности одного шага возрастает. Уменьшение количества GPU в TP позволяет исключить часть синхронизации, а освободившиеся GPU — объединить в большее число независимых экземпляров; при этом каждому GPU потребуется хранить бóльшую долю состояния запросов. Приведённые в этом разделе таблицы ёмкости каждого GPU и времени сеансов по-прежнему можно использовать для сопоставления этих двух изменений; конкретные числовые значения приведены в разделе 6.7.4.

Применим тот же анализ к модели с более компактным состоянием. При расчёте количества GPU, необходимого для DeepSeek V4.1, помимо логической ёмкости состояния в 890 В на токен необходимо учитывать распределение состояния между GPU. В этом значении состояние, совместно используемое разными слоями, учитывается только один раз; если слои размещены на разных GPU, последующим слоям всё равно потребуется обращаться к данным, сохранённым тем слоем, который создал эти состояния. Связан-

ные слои можно разместить на соседних GPU, реплицировать состояние на целевых GPU либо организовать путь удалённого доступа. Первый вариант ограничивает размещение слоёв, второй увеличивает затраты на хранение и обновление, а третий добавляет зависимость от коммуникаций, поэтому потребность в ресурсах следует рассчитывать отдельно по данным, которые каждый GPU фактически хранит и к которым обращается.⁴⁵

6.7.2 Как мощность, стоимость и область распространения отказа меняют выбор

Временная шкала показывает момент завершения каждого сеанса и длительность использования восьми GPU, поэтому по ней можно непосредственно рассчитать стоимость и сравнить, сколько ресурсов требуется для своевременной обработки одного и того же набора запросов. Стоимость измеряется в GPU-s для H100, то есть одна секунда использования одного GPU. Оплата восьми GPU начинается с одновременного поступления четырёх сеансов и продолжается до завершения всех сеансов. Предельное время завершения составляет 130 ms; вовремя должны завершиться как минимум три из четырёх сеансов.

Общая стоимость четырёх экземпляров с двумя GPU составляет примерно $8 \times 0.1191 \approx 0.953$ GPU-s. Все четыре сеанса завершаются вовремя, а средняя стоимость каждого своевременно завершённого сеанса составляет примерно 0.238 GPU-s. Общая стоимость двух экземпляров с четырьмя GPU составляет примерно 1.021 GPU-s. Все четыре сеанса также завершаются вовремя, а средняя стоимость каждого составляет примерно 0.255 GPU-s. Один экземпляр с восемью GPU завершает в установленный срок три сеанса, точно достигая минимально допустимой доли, однако восемь GPU остаются заняты до 164.7 ms. Общая стоимость составляет примерно 1.318 GPU-s, а средняя стоимость каждого своевременно завершённого сеанса — примерно 0.439 GPU-s. Восемь экземпляров с одним GPU не могут вместить сеанс.

Если в качестве знаменателя использовать число своевременно завершённых сеансов, средняя стоимость определяется как

$$C_{\text{on time}} = \frac{C_{\text{occupied}}}{N_{\text{on time}}}. \quad (6-13)$$

Стоимость занятых ресурсов рассчитывается по полной продолжительности работы, включая время простоя из-за отказа и повторного выполнения; в знаменателе учитываются только сеансы, завершённые в установленный срок. Стоимость неудачных и повторных выполнений, а также запросов, превысивших предельное время, входит в числитель и поэтому распределяется между своевременно завершёнными сеансами.

В приведённом выше сравнении предполагается, что все GPU постоянно работают исправно. При отказе одного GPU содержащий его экземпляр приостанавливается, а назначенные этому экземпляру последующие сеансы также задерживаются. Добавим один отказ на ту же временную шкалу: в момент 20 ms GPU 0 выходит из строя, содержащий его экземпляр останавливается на 40 ms, а после восстановления в момент 60 ms повторно выполняет восьмиступенчатое продолжение генерации с начальной точки текущего сеанса. Остальные экземпляры продолжают работу и не затрагиваются этим отказом.

⁴⁵Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт сквозного межглавного сеанса.

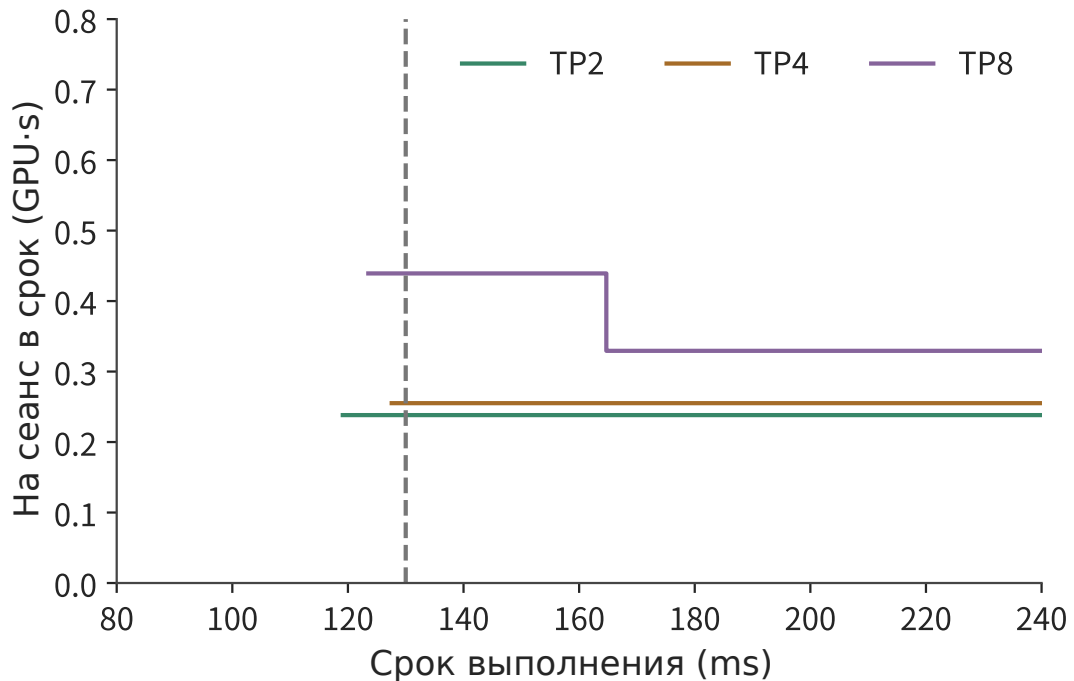


Рис. 6-51. Зависимость средней стоимости своевременно завершённых сеансов от предельного времени при отсутствии отказов. Восемь GPU оплачиваются до завершения всех сеансов; каждая кривая начинается с точки, в которой вовремя завершаются как минимум три сеанса. Значение TP указывает число GPU, используемых каждым экземпляром; стоимость измеряется в GPU·s для H100. TP1 не вмещает сеанс с контекстом 128K, поэтому соответствующая кривая отсутствует.

При использовании четырёх экземпляров с двумя GPU остальные три сеанса по-прежнему завершаются примерно за 119 ms, а затронутый сеанс — за $60 + 119.1 \approx 179.1$ ms; доля своевременно завершённых сеансов составляет 75%. При использовании двух экземпляров с четырьмя GPU незатронутый экземпляр завершает два сеанса примерно за 64 и 128 ms, а затронутый — остальные два примерно за 124 и 188 ms; доля своевременно завершённых сеансов также составляет 75%. У экземпляра с восемью GPU четыре сеанса последовательно завершаются примерно за 101, 142, 184 и 225 ms; за 130 ms завершается только один.

Четыре экземпляра с двумя GPU по-прежнему удовлетворяют минимальному требованию, но стоимость возрастает до $8 \times 0.1791 \approx 1.433$ GPU·s. Средняя стоимость каждого своевременно завершённого сеанса составляет примерно $1.433/3 \approx 0.478$ GPU·s, то есть в 2.0 раза больше, чем без отказа; для двух экземпляров с четырьмя GPU она составляет примерно 0.500 GPU·s. Отказ изолирован внутри одного экземпляра, поэтому остальные три сеанса по-прежнему завершаются вовремя; однако из-за увеличения времени использования ресурсов и сокращения числа своевременно завершённых сеансов стоимость каждого такого сеанса всё равно удваивается.

Область распространения отказа зависит от того, используют ли экземпляры общие устройства. Если два экземпляра используют один источник питания или коммутатор, отказ этого общего устройства затронет оба экземпляра. Разделение на экземпляры позволяет сохранить прогресс остальных экземпляров только в сочетании с изоляцией ресурсов. Чем больше устройств использует один экземпляр, тем от исправной работы большего числа устройств зависит выполнение запроса. Поэтому при разделении

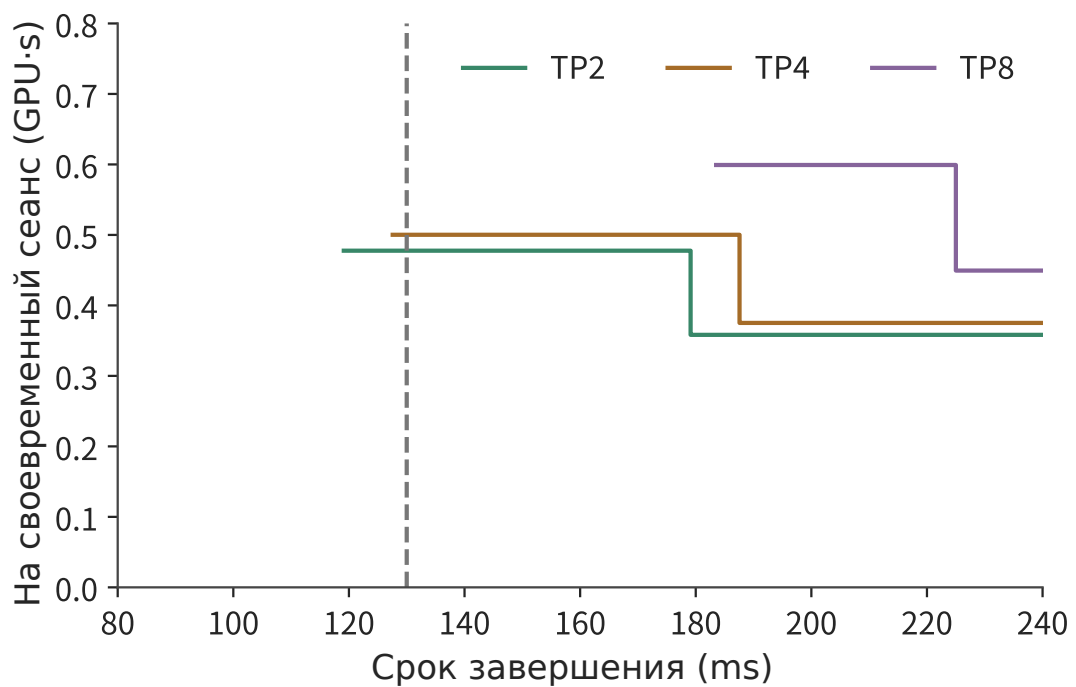


Рис. 6-52. В момент 20 ms происходит отказ GPU 0, а в момент 60 ms затронутый экземпляр начинает повторное выполнение с начальной точки текущего сеанса. Используются тот же метод расчёта стоимости и тот же диапазон вертикальной оси, что и на графике без отказов. Значение TP указывает число GPU, используемых каждым экземпляром; стоимость измеряется в GPU·s для H100.

на экземпляры также необходимо учитывать совместное использование источников питания и коммутаторов.

Энергопотребление этих конфигураций развёртывания также можно рассчитать по мощности. Если принять указанный в разделе 6.5.2 предел мощности системы DGX H100, равный 10.2 kW, то без отказов четыре экземпляра с двумя GPU работают примерно 119.1 ms и потребляют около 1.21 kJ, а два экземпляра с четырьмя GPU работают примерно 127.6 ms и потребляют около 1.30 kJ. При одинаковом оборудовании и одинаковой мощности чем раньше завершается задача, тем меньше электроэнергии расходуется.

Требования модели к памяти ограничивают доступные конфигурации развёртывания. В данном примере первой исключается TP1: KV-кэш сеанса с контекстом 128K увеличивает потребность одного GPU в памяти до 102.03 GB. Если сократить контекст до 32K, один GPU сможет вместить один сеанс, однако восьмишаговое продолжение генерации занимает 173.3 ms, что всё равно превышает предельное время 130 ms. В этом случае TP1 исключается уже по времени. Сначала следует определить конфигурации развёртывания с достаточным объёмом памяти, а затем сравнить их время выполнения и стоимость.⁴⁶

Эксперимент 6-10 · Основной: выбор конфигурации параллелизма по предельному времени завершения сеанса и стоимости

- (a) Используя формулы (6-4) и (6-5), пошагово вычислите время восьмишагового продолжения генерации для четырёх конфигураций TP, а затем рассчитайте моменты завершения каждого из четырёх сеансов.
- (b) Поочерёдно задайте предельное время 100, 130 и 170 ms, потребовав своевременного завершения как минимум 75% сеансов. Найдите удовлетворяющие требованиям конфигурации развёртывания и сравните их стоимость.
- (c) Перейдите на TP16 на двух системах HGX H100, возьмите измеренное в эксперименте 7-3 значение межсерверного AllReduce, заново рассчитайте время продолжения генерации для одного сеанса и определите, можно ли уложиться в предельное время 50 ms.
- (d) Повторно рассчитайте моменты завершения сеансов и стоимость после отказа; сократите время простоя с 40 до 10 ms и объясните, как изменится набор удовлетворяющих требованиям конфигураций развёртывания.
- (e) Измените контекст на 32K, добавьте ограничение по мощности и укажите, какие конфигурации развёртывания первыми исключаются соответственно по ёмкости памяти, времени и мощности.

6.7.3 Как выбрать стратегию разбиения для заданных модели и оборудования

Вернёмся к вопросу, поставленному в начале главы. Выбор стратегии разбиения можно сформулировать как задачу оптимизации с входными данными и ограничениями: множество вариантов π включает степень параллелизма, физическое размещение, алгоритм коммуникации, разбиение на micro-batch и реализацию локальных операторов. Сначала определяется целевая функция J , после чего ищется

⁴⁶Поля candidates и candidates_32k в повременной шкале сеансов, сбоев и GPU-s сгенерированы скриптом расчёта; отбор для четырёх наборов условий развёртывания приведён в результатах выбора разбиения. Энергопотребление рассчитано исходя из предельной мощности системы DGX H100, равной 10,2 кВт; см. результаты расчётов для главы 6.

$$\pi^* = \arg \min_{\pi \in \mathcal{P}_{\text{feasible}}} J(\pi).$$

Каждый вариант из допустимого множества $\mathcal{P}_{\text{feasible}}$ должен удовлетворять ограничениям на объём памяти каждой карты, зависимости данных при разбиении модели, поддержку со стороны ядер и предельному сроку обслуживания. Целевой функцией J может быть задержка отдельного запроса или рассмотренная в предыдущем разделе стоимость каждого сеанса, завершённого в срок. Если целью является пропускная способность, следует при одинаковом целевом уровне обслуживания (SLO) сравнивать частоту запросов, которую система способна стабильно принимать. Если нагрузка и цель не зафиксированы, единственного ответа о «наилучшей степени параллелизма» не существует.

Шаг 1. Зафиксировать модель, нагрузку и ресурсы. Для модели необходимо указать число слоёв, размеры матриц, число голов внимания и KV-голов, число экспертов и выбираемых для каждого токена экспертов, точность, общие веса и межслойное состояние. Для нагрузки необходимы длина prefill, контекст decode, число одновременно готовых запросов, распределение маршрутизации, целевая задержка и процесс поступления запросов. Для оборудования необходимы доступный объём НВМ каждой карты, эффективные вычислительная мощность и пропускная способность для различных форм, а также пути в каждом направлении внутри узла и между узлами, накладные расходы на запуск и общие узкие места. Параметры оборудования приведены в главе 4, а фактические формы и локальное время выполнения — в главе 5. Нельзя делить общую пропускную способность кластера поровну между всеми коммуникационными группами.

Шаг 2. Перечислить варианты с учётом узких мест, а не выбирать сначала аббревиатуру. В следующей таблице перечислены варианты, которые следует рассматривать при различных ограничениях и узких местах. Она предназначена лишь для сужения области поиска: окончательное решение всё равно требует сравнения времени завершения.

Текущее ограничение или узкое место	Приоритетные варианты	Затраты, которые необходимо дополнительно рассчитать
Веса не помещаются в память либо при малом batch их повторное чтение слишком медленно	ТР, РР; для МоЕ можно использовать ЕР	Редукции, последовательное выполнение стадий, трафик экспертов и рабочая область
Много независимых запросов, полная модель уже помещается в память	Больше DP/сервисных реплик, меньшие группы ТР	Очереди, переиспользование внутри batch, объём KV на каждую реплику
Активации в определённом диапазоне хранятся с дублированием	Включить SP внутри одной группы ТР	Сбор и редукции на границах раскладки, поддержка со стороны ядер
При длинном контексте доминируют KV и внимание	Сочетание СР и ТР	Обмен K и V, редукции статистик, неравномерность нагрузки при каузальном внимании
Большой объём весов множества слоёв, дорогие межузловые редукции	ТР внутри узла + РР между узлами	Достаточно ли micro-batch, самая медленная стадия и простои конвейера

Текущее ограничение или узкое место	Приоритетные варианты	Затраты, которые необходимо дополнительно рассчитать
Большой набор экспертов, выраженные горячие точки маршрутизации	ЕР, реплики экспертов, деление экспертов (размещение маршрутизируемых экспертов в отдельном пуле ресурсов экспертов)	Трафик dispatch/combine в обоих направлениях, наиболее загруженная карта и очередь пула экспертов

Шаг 3. Для каждого варианта описать принадлежность данных и временную диаграмму. Для каждой карты укажите, какие слои, головы, эксперты, активации и KV она хранит. При расчёте пикового потребления памяти необходимо также учитывать буферы коллективных коммуникаций, двойную буферизацию, padding, находящиеся в обработке micro-batch и пространство для миграции реплик. Затем для каждого оператора рассчитайте время локальных вычислений и обращений к памяти, а для каждой межкарточной операции — число раундов, объём данных и общие пути; разместите их на временной шкале в соответствии с зависимостями. Последовательные слои одного токена не завершатся одновременно лишь благодаря конвейерному параллелизму, а результаты выбранных этим токеном экспертов не будут готовы раньше времени лишь потому, что сеть свободна. Для МоЕ необходимо подставлять показатели самой загруженной группы экспертов, а не среднее значение; конкретная модель skew (перекоса нагрузки) приведена в разделе 9.4.

Шаг 4. Ранжировать допустимые варианты по одной и той же цели. В примере этой главы с восемью картами зафиксированы плотная модель Qwen3-32B и сеансы с контекстом 128K; каждый экземпляр одновременно обрабатывает только один сеанс. В первом сравнении рассматриваются TP1, TP2, TP4 и TP8, а оставшиеся карты образуют независимые экземпляры. Параллелизм экспертов неприменим к плотной модели. Последовательный параллелизм изменяет только хранение части активаций и не даёт дополнительной выгоды в этом примере с последовательной обработкой токенов и фиксированной рабочей областью. Если при конвейерном параллелизме не чередовать несколько сеансов, отдельный сеанс всё равно должен последовательно пройти все стадии, поэтому этот вариант не рассматривается как приоритетный для сокращения времени одного сеанса. Контекстный параллелизм, межсеансовый конвейер и непрерывная пакетная обработка (перепланирование активных запросов на границе каждой итерации) могут быть включены в расширенный поиск, однако для них необходимо отдельно учесть временные модели контекстных редукций, планирования стадий и изменения batch; применять к ним масштабирование тензорного параллелизма $1/p$ нельзя.

В двух предыдущих разделах уже приведены временная шкала и стоимость каждого сеанса. Фильтрация по этим полным значениям непосредственно даёт приведённый ниже выбор; при изменении цели, физических границ или ёмкости изменится и выбор.⁴⁷

⁴⁷Входные данные для выбора разбиения, скрипт отбора вариантов, полные результаты. Используется модель времени выполнения из этой главы; объём KV-кэша рассчитывается по пиковому контексту всех поставленных в очередь сеансов внутри экземпляра. Ранжирование выполняется только среди явно указанных комбинаций TP и количества экземпляров.

Заданные условия	Выбор с учётом ограничений	Почему
Четыре сеанса, 128K, не менее 75% должны завершиться за 130 ms	Четыре экземпляра TP2	TP1 не помещается в память; с TP2 все сеансы завершаются примерно за 119.1 ms, а стоимость каждого своевременно завершённого сеанса составляет около 0.238 GPU-s, что ниже 0.255 для TP4 и 0.439 для TP8
Один сеанс, 128K, срок 50 ms	Один экземпляр TP8	Около 41.2 ms; TP4 требует около 63.8 ms, что уже превышает срок
Два сервера HGX H100, всего 16 карт; один сеанс, 128K, срок 50 ms	По-прежнему один экземпляр TP8	Межсерверный TP16 по измеренному времени AllReduce требует около 53.3 ms, что превышает срок; на другом сервере можно запускать другие экземпляры
Четыре сеанса, 32K, не менее 75% должны завершиться за 130 ms	Четыре экземпляра TP2	TP1 помещается в память, но восемь шагов требуют 173.3 ms; с TP2 все сеансы завершаются примерно за 88.3 ms, а стоимость каждого своевременно завершённого сеанса составляет около 0.177 GPU-s

Шаг 5. Корректировать оценки измерениями, не считая ранжирование по оценкам окончательным выводом. Для лидирующих вариантов проведите измерения при одинаковых запросах, маршрутизации и требованиях к качеству, зафиксировав пиковое потребление памяти, моменты готовности и завершения каждой стадии, объём принятых и отправленных данных на каждом порту и хвостовую задержку сервиса. Если фактическая эффективность матричных операций, конкуренция за коммуникационные ресурсы или очереди отличаются от оценок, подставьте измеренные параметры и выполните ранжирование заново. Если различие между двумя вариантами меньше разброса измерений, сохраните оба и проверьте их на более репрезентативной нагрузке. Итоговый результат должен включать выбранный вариант, исключённые варианты, пороговые значения, при которых вывод изменится, и диапазон измеренных значений. Сопутствующий скрипт позволяет пересчитать четыре группы условий из таблицы, но перебирает только явно заданные варианты и не является глобальным планировщиком для произвольных моделей и фреймворков.

Различные проектные решения влияют на разные временные затраты. Изменение TP прежде всего меняет объём вычислений и обращений к памяти на каждой карте, а также число карт, участвующих в редукции; изменение алгоритма — главным образом число раундов и объём отправляемых данных; изменение топологии — общие пути и фиксированное время ожидания; увеличение числа экземпляров — порядок независимых сеансов в очереди. Подставив эти изменения в одну и ту же модель времени выполнения, можно сравнить производительность различных проектных решений.

Для дальнейшей оптимизации можно выполнить анализ чувствительности формулы (6-5). В примере с восемью картами HGX H100 и малым объёмом данных удвоение пропускной способности NVLink экономит лишь около 2.5 μ s, тогда как уменьшение вдвое накладных расходов на запуск каждого раунда экономит около 0.74 ms. Поэтому

в первую очередь следует сокращать накладные расходы на запуск каждого раунда коммуникации. Если производительность МоЕ ограничена самой загруженной группой экспертов, сбалансированное распределение напрямую уменьшает объём вычислений этой группы. Если при чтении удалённого KV пропускная способность ограничена числом находящихся в обработке транзакций, полностью задействовать доступную пропускную способность канала можно лишь увеличением параллелизма транзакций. Это позволяет определить, какие затраты следует сокращать и какие ресурсы увеличивать в первую очередь.

6.7.4 Увеличение суперузла и размещение весов, таблиц Engram и KV: комплексный пример V4.1 Flash

В трёх предыдущих разделах обсуждалось, как распределить восемь карт одного сервера между несколькими экземплярами. Теперь изменим сам суперузел: сначала рассмотрим, что даёт его увеличение, затем, следуя подходу из раздела 4.7.3, зафиксируем веса в ROM и, наконец, обсудим размещение таблиц Engram и KV. В качестве примера по-прежнему используется DeepSeek V4.1 Flash из главы 2, а задача — decode с контекстом 200K. Аппаратная платформа состоит из 256 карт H100 SXM, размеры суперузла составляют соответственно 8, 64, 128 и 256 карт, причём каждый экземпляр инференса занимает ровно один суперузел. Внутри экземпляра внимание размещается с использованием параллелизма по данным: каждая карта обрабатывает и хранит внимание и KV только своих сеансов. Маршрутизируемые эксперты и таблицы Engram равномерно распределяются между картами с использованием экспертного параллелизма; на каждом слое выполняется по одному вызову dispatch и combine.⁴⁸

Размер суперузла и число сеансов на карту. Веса на каждой карте состоят из двух частей: внимание, общие эксперты и выходная голова хранятся на каждой карте отдельной копией и занимают в сумме 9,5 GB; маршрутизируемые эксперты объёмом 288,8 GB и таблицы Engram объёмом 203,1 GB распределяются между S картами экземпляра. После вычета весов и рабочего пространства U вся оставшаяся НВМ отводится под KV. Каждый сеанс занимает $K = 180,7$ MB, поэтому

$$W_{\text{card}} = 9.5 \text{ GB} + \frac{288.8 \text{ GB} + 203.1 \text{ GB}}{S}, \quad N_{\text{session}} = \left\lfloor \frac{C - W_{\text{card}} - U}{K} \right\rfloor. \quad (6-14)$$

Время шага по-прежнему оценивается по формуле (6-2): веса текущей карты и KV

⁴⁸Фиксированный сценарий, скрипт расчёта и полные результаты. Веса суммируются по каждому тензору на основе заголовков шардов зафиксированного checkpoint V4.1 Flash; объём KV-кэша и количество матричных операций рассчитаны соответственно с помощью kv-comparison и v41-forward при контексте 200,000; параметры H100 взяты из таблицы оборудования; накладные расходы на запуск каждого раунда для NVLink и сетевой карты совпадают со значениями из разделов 7.6.3 и 7.6.4. Строки о пластине ROM и 58 ускорителях B200 взяты из rooffline-анализа DeepSeek-V4.1-Flash в репозитории автора OpenTallas, commit c7093ba; использованные точки анализа сохранены в выдержке. Это результаты анализа при одинаковой площади кремния: чип OpenTallas ещё не изготовлен, а плотность ROM и пропускная способность чтения для техпроцесса N5 не измерялись. Число строк таблицы Engram, количество байтов в строке и число строк, считываемых двумя модулями, взяты из того же checkpoint и той же конфигурации; в развёртывании DeepSeek таблица размещается в памяти хоста и предварительно загружается по RDMA, см. технический отчёт V4.1, §2.4.2 и §3.1.3; время одного обращения к памяти хоста туда и обратно и скорость чтения, ограниченная числом меток запросов в обработке, взяты из измерений KV-Direct в разделе 7.3.4; площадь ROM пересчитана по плотности масочного ROM для N5 из OpenTallas, а проект одной пластины после переноса таблицы из ROM взят из анализа engram-host в том же commit. Объём встроенной SRAM WSE-3, равный 44 GB, указан в разделе 4.7.2.

всех её сеансов считываются по одному разу, из времени чтения и вычислений выбирается большее, после чего добавляются по одному `dispatch` и `combine` на каждом слое — всего 80 операций All-to-All. Результаты для четырёх размеров суперузла приведены в таблице.

Суперузел	Веса на карту	Сеансов на карту	Время шага	Пропускная способность на карту
8 карт	71,0 GB	37	14,1 ms	2 632 токена/s
64 карты	17,2 GB	335	17,3 ms	19 420 токенов/s
128 карт	13,4 GB	356	18,4 ms	19 392 токена/s
256 карт	11,5 GB	367	18,9 ms	19 378 токенов/s

Когда суперузел состоит всего из 8 карт, каждая карта хранит по 48 экспертов на слой, и за один шаг выбираются почти все они, поэтому чтение весов экспертов занимает подавляющую часть времени шага. При увеличении до 64 карт число экспертов на карту сокращается в восемь раз, время их чтения падает более чем в десять раз, а освободившаяся НВМ позволяет разместить в девять раз больше сеансов. В результате пропускная способность на карту возрастает в 7,4 раза. Дальнейшее увеличение уже не приносит пользы: чтение экспертов стало незначительным, тогда как чтение KV, вычисления и All-to-All растут пропорционально числу сеансов. Этот выигрыш в 7,4 раза обеспечен главным образом ёмкостью. Если перенести таблицы Engram в память хоста, каждая карта 8-карточного суперузла также сможет разместить 178 сеансов, и отставание от 64-карточного суперузла сократится до 1,9 раза.

Скорость отдельного пользователя от размера суперузла не зависит. При `batch`, равном 1, критический путь одного токена включает однократное чтение 8,2 GB весов, копия которых хранится на каждой карте, а затем последовательное чтение одного эксперта на каждом слое. Это занимает 2,76 ms, то есть даёт 362 токена/s, совершенно одинаково для 8 и 256 карт. Увеличение суперузла повышает ёмкость и общую пропускную способность, но не скорость отдельного сеанса; сократить время отдельного сеанса можно лишь с помощью тензорного параллелизма внимания, как в разделе 6.7.1. И наоборот, если те же 64 карты распределить по восьми серверам HGX H100, семь восьмых байтов `dispatch` и `combine` придётся передавать через сетевые карты. Время коммуникации возрастёт с 3,7 до 29,2 ms, а пропускная способность на карту снизится до 7 828 токенов/s. Именно эти затраты и устраняет суперузел.

Фиксация весов в ROM. На рисунке 6-54 показаны три варианта размещения весов и KV. GPU размещает и то и другое в НВМ, поэтому на каждом шаге приходится заново считывать веса. В архитектуре из раздела 4.7.3 веса, которые после развёртывания больше не изменяются, записываются в масочную ROM, а в НВМ остаётся только KV. KV также можно дополнительно перенести во внутрикристальную SRAM.

Два последних варианта рассчитаны по результатам анализа V4.1 Flash в OpenTallas: две пластины ROM, изготовленные по техпроцессу N5, тот же контекст 200K и `batch`, равный 1. У весов и KV есть отдельные тракты передачи, поэтому чтение весов, чтение KV и вычисления могут перекрываться, а коллективная коммуникация начинается лишь после завершения всех трёх операций:

$$T_{\text{token}} = \frac{\max(T_{\text{weight}}, T_{\text{KV}}, T_{\text{compute}})}{0.9} + T_{\text{comm}} + T_{\text{fixed}}. \quad (6-15)$$

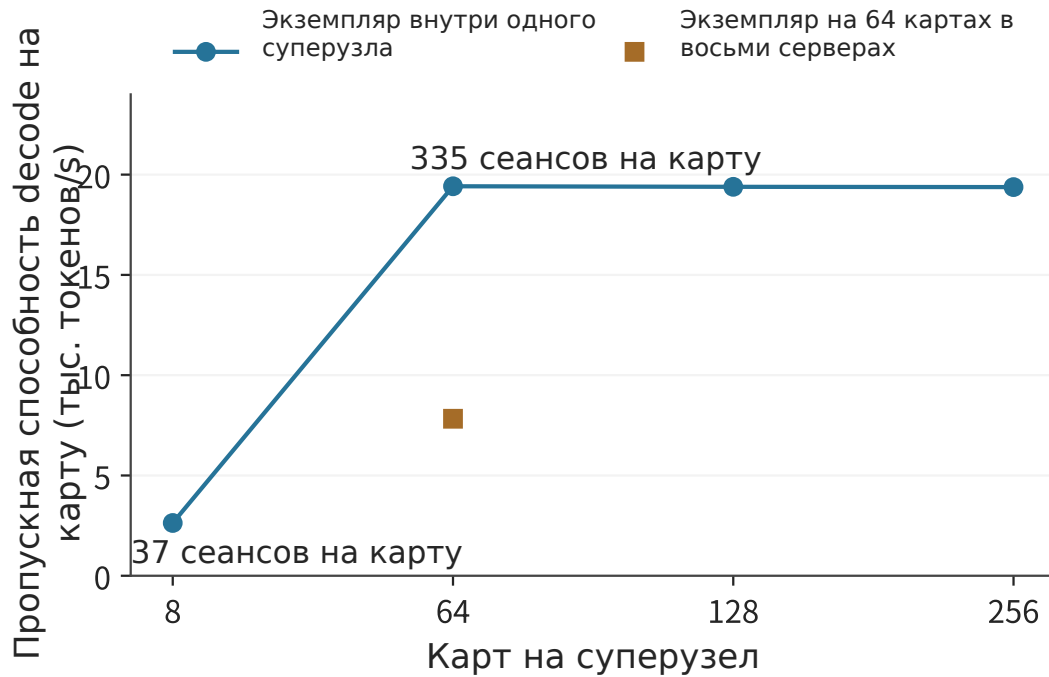


Рисунок 6-53. Изменение пропускной способности decode на карту для V4.1 Flash на 256 картах H100 в зависимости от размера суперузла при целевом времени обслуживания 50 ms на токен. Синяя линия: все экземпляры находятся внутри одного суперузла; при переходе от 8 к 64 картам показатель возрастает примерно в 7,4 раза, после чего перестаёт расти. Оранжевая точка: те же 64 карты распределены по восьми серверам и образуют группу экспертного параллелизма.

Коэффициент 0,9 учитывает невозможность совершенно равномерно распределить слои между двумя пластинами, а T_{fixed} — фиксированную конвейерную задержку каждого слоя. В строке для GPU веса и KV совместно используют HBM, поэтому времена двух операций чтения складываются.

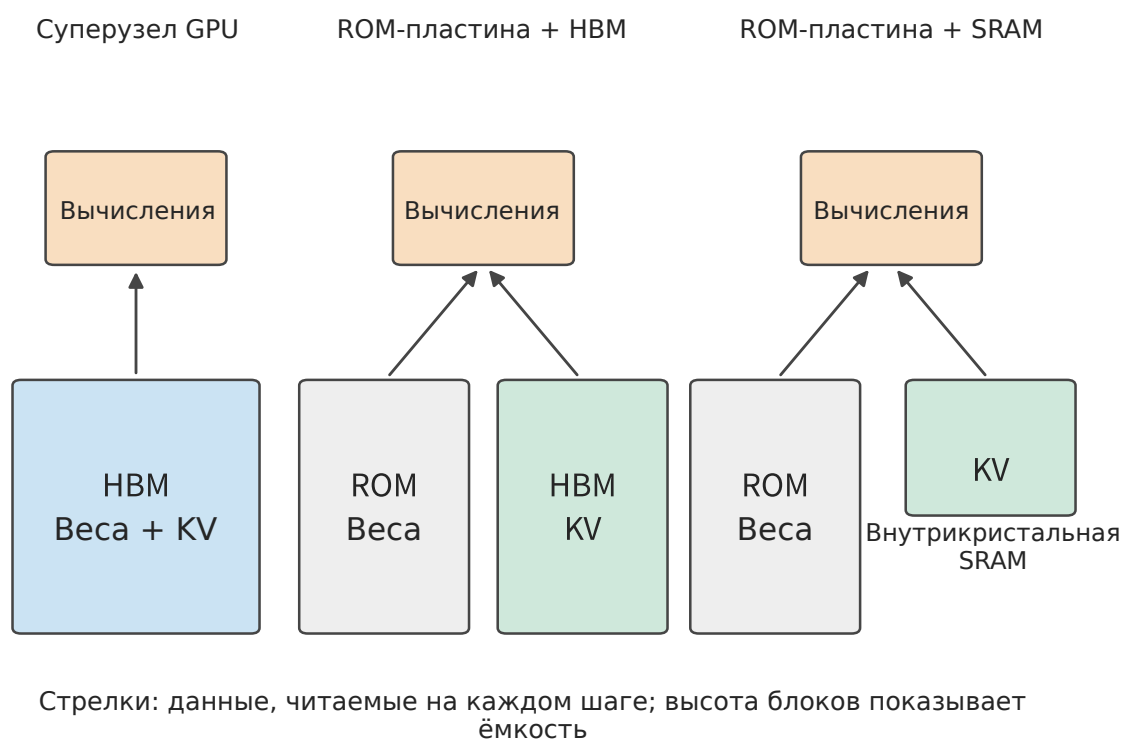


Рисунок 6-54. Три варианта размещения весов и KV. Слева: веса и KV GPU совместно используют HBM. В центре: веса записываются в ROM при производстве, а KV размещается в HBM. Справа: веса находятся в ROM, а KV — во внутрикристалльной SRAM, ёмкость которой значительно меньше. Стрелками обозначены данные, считываемые на каждом шаге, а высота прямоугольников схематично показывает ёмкость.

Машина	Хранение и вычисления	Коллективная коммуникация	Скорость одного пользователя	Доля коммуникации
8 карт H100, веса в HBM	2 689 μ s	76 μ s	362 токена/s	3%
58 карт B200 с тензорным параллелизмом, веса в HBM	165 μ s	562 μ s	1 357 токенов/s	76%
Две пластины ROM, веса в ROM	77 μ s	159 μ s	4 070 токенов/s	65%

После переноса весов из HBM время их чтения сокращается с 2,7 ms до 69 μ s, но 80 внутрикристалльных операций AllReduce по-прежнему требуют 159 μ s и занимают 65% времени обработки каждого токена. При использовании тензорного параллелизма на 58 картах B200 время чтения весов также можно сократить до 148 μ s, однако время коммуникации возрастает до 562 μ s. Как только чтение весов перестаёт быть самой длительной операцией, скорость отдельного пользователя начинает определяться коллективной коммуникацией, и ключевым становится метод оценки коммуникации по числу переходов и задержке каждого перехода из раздела 6.5.1.

Где размещать таблицы Engram. V4.1 Flash содержит два модуля Engram, каждый из которых снабжён таблицей на 384 миллиона строк. Каждая строка содержит 256 значений FP8 и 8-байтовый scale; общий объём двух таблиц составляет 203,1 GB. При генерации каждого токена каждый из двух модулей считывает по 24 строки, всего 12,7 KB. Эти таблицы также являются весами и после развёртывания не изменяются. Выбор считываемых строк зависит только от последовательности токенов, а не от активаций, поэтому предварительную выборку можно начать в самом начале каждого шага. Если строки будут получены до завершения вычислений первого блока Transformer, поиск в таблице не увеличит время шага. Доступное для получения данных время составляет 69 μ s на H100 и всего 6,1 μ s на пластине ROM. На рисунке 6-56 показаны три варианта размещения и их стоимость.

Размещение	Занимаемая ёмкость	Получение 48 строк
Память хоста, предварительная выборка через PCIe или RDMA	Не занимает HBM	Один обмен туда и обратно: 1,05 μ s
Шарды в HBM каждой карты суперузла	Для 8 карт — 25,4 GB на карту, что эквивалентно 140 сеансам; для 64 карт — 3,2 GB на карту, что эквивалентно 18 сеансам	Один раунд обмена через NVLink: 0,83 μ s
Масочная ROM	21 651 mm ² , примерно половина пластины	Внутрикристалльное чтение

На H100 обе траектории получения данных значительно короче 69 μ s, поэтому при выборе размещения достаточно учитывать только ёмкость. В 8-карточном суперузле эта таблица не помещается; после её переноса в память хоста число сеансов на карту возрастает с 37 до 178. Начиная с 64 карт на каждую приходится лишь 3,2 GB, и размещение таблицы в HBM сокращает число сеансов всего на 5%. Память хоста не становится узким местом и при пакетном обслуживании: в 64-карточном экземпляре каждая карта выполняет за шаг 16 080 случайных чтений. При измеренной в разделе 7.3.4 производи-

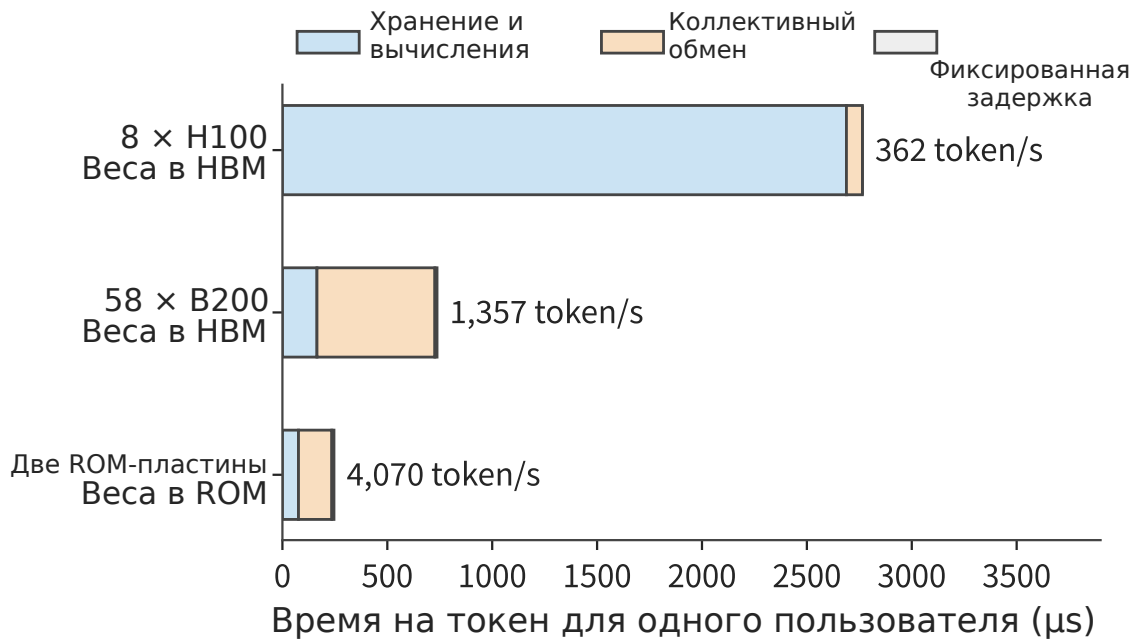


Рисунок 6-55. Состав времени обработки одного токена V4.1 Flash для одного пользователя на трёх машинах при контексте 200K и batch, равном 1. Когда веса находятся в HBM, основную часть времени занимает чтение из хранилища; после переноса весов в ROM самой длительной операцией становится коллективная коммуникация.

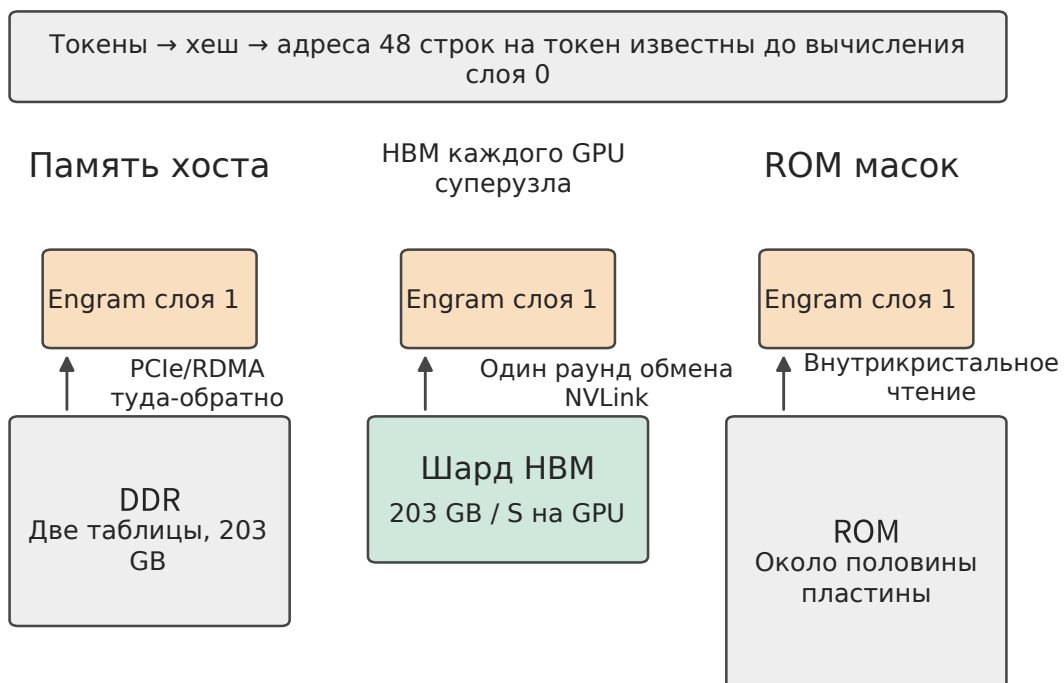


Рисунок 6-56. Три варианта размещения таблиц Engram. Считываемые строки определяются ещё до начала вычислений первого блока Transformer. Варианты отличаются трактом получения этих 48 строк и типом хранилища, ёмкость которого занимает таблица.

тельности 61 миллион операций в секунду это занимает всего 0,26 ms, или 1,5% времени шага. В развёртывании DeepSeek таблицы действительно размещаются в памяти хоста и предварительно выбираются через RDMA параллельно с вычислениями первого блока Transformer.

Для пластины ROM компромисс меняется. Вычисление первого блока занимает всего 6,1 μ s, поэтому один обмен с памятью хоста туда и обратно отнимает шестую часть этого времени, а когда время шага становится меньше 42 μ s, скрыть задержку уже невозможно. Запись таблицы в ROM требует дополнительной половины пластины, хотя на каждый токен из неё считывается всего 12,7 KB. Размещение таблицы в HBM на краю пластины занимает пространство лишь 1 124 сеансов, а чтение не проходит через интерконнект. После переноса таблицы из ROM весь checkpoint помещается на одной пластине, а скорость отдельного пользователя даже возрастает до 5 037 токенов/s. Чем быстрее машина, тем ближе к вычислениям следует размещать эту таблицу — в доступном для записи хранилище.

Перенос KV в SRAM. Внутрикристалльная SRAM значительно быстрее HBM, однако прежде всего должна обладать достаточной ёмкостью. Checkpoint V4.1 Flash занимает 510,3 GB, поэтому даже при использовании 44 GB внутрикристалльной SRAM, как в WSE-3 из раздела 4.7.2, только для весов потребуются 12 пластин. Таким образом, вариант с размещением весов в SRAM сразу исключается из-за недостаточной ёмкости. После переноса весов в ROM оставшейся на пластине SRAM хватает лишь для KV одного сеанса с контекстом 200K; если же разместить KV в HBM, те же две пластины смогут одновременно удерживать 9 637 сеансов. Скорость отдельного пользователя у двух вариантов одинакова, но общая пропускная способность различается на три порядка. Скорость одного сеанса определяется хранилищем и коммуникацией на критическом пути, а число одновременно обслуживаемых сеансов — ёмкостью для изменяемого состояния. Это и есть два уровня параллелизма, разделённые в разделе 6.7.1.

Во всех трёх шагах применяется один и тот же метод: сначала по формуле (6-1) проверяется ёмкость, затем по формуле (6-2) отдельно оценивается время каждой операции и, наконец, определяется самая длительная операция на критическом пути. Увеличение суперузла повышает пропускную способность, но не ускоряет отдельный сеанс. После фиксации весов задержка каждого перехода в интерконнекте ограничивает скорость отдельного пользователя. Такие доступные только для чтения большие структуры, как таблицы Engram, по мере ускорения машины следует размещать всё ближе к вычислениям. Ёмкость SRAM определяет число одновременно обслуживаемых сеансов.

Эксперимент 6-11. Расширение: комплексный пример с суперузлом, ROM и SRAM

- (a) По формуле (6-14) повторно рассчитайте число сеансов в каждой строке таблицы. Измените целевое время обслуживания на 20 ms на токен и определите число сеансов, которое сможет обслуживать каждая карта при каждом размере суперузла.
- (b) Переведите внимание внутри экземпляра на TP8, не меняя остальные условия. Повторно рассчитайте скорость одного пользователя при batch, равном 1, и сравните её с 362 токенами/s.
- (c) Измените накладные расходы запуска каждого межсерверного раунда с 5 μ s на 2 μ s и установите на каждую карту по две сетевые карты. Определите, сможет ли межсерверный 64-карточный экземпляр сравняться с экземпляром внутри суперузла.

- (d) Уменьшите вдвое задержку каждого внутрикристального перехода и по формуле (6-15) повторно рассчитайте скорость одного пользователя и долю коммуникации для строки с пластинами ROM.
- (e) Измените размер контекста на 1М и повторно рассчитайте число одновременно размещаемых сеансов при хранении KV в SRAM и в HBM.
- (f) Измените время одного обмена с памятью хоста туда и обратно на 2 μ s. Определите, начиная с какой скорости отдельного пользователя поиск в таблице Engram начнёт увеличивать время шага на H100 и на пластине ROM.

Если объём весов, состояния или требования к обслуживанию продолжают расти, одного суперузла может оказаться недостаточно для работы экземпляра. В следующей главе к проанализированным здесь трактам передачи данных и зависимостям добавляются сетевые карты, коммутируемая сеть центра обработки данных, уведомления о завершении удалённых операций и сетевая перегрузка, после чего продолжается расчёт выгод и затрат от расширения области совместной работы.

6.8 Итоги главы

Параллельная работа на нескольких ускорителях продолжает принципы разбиения и планирования операций, применяемые на одном ускорителе. При разбиении по примерам, позициям последовательности, признакам, слоям и экспертам на каждом ускорителе различаются хранимые данные и зависимости; распространёнными способами организации такого разбиения служат параллелизм данных, тензорный параллелизм, параллелизм последовательности, параллелизм контекста, конвейерный параллелизм и параллелизм экспертов. Разбиение порождает обмен данными, планирование определяет моменты его запуска и синхронизации, а совокупность алгоритма и физических маршрутов определяет затраты. При выборе решения сначала проверяют, достаточна ли ёмкость и допустимо ли выполнение, а затем сравнивают варианты по задержке, пропускной способности или стоимости при полной нагрузке.

По мере масштабирования выигрыш от параллелизма уменьшается: объём вычислений и обращений к памяти на каждом ускорителе постепенно сокращается, но затраты времени на последовательную обработку сохраняются, а раунды редукции и ожидание обмена данными между серверами продолжают отнимать время. Повторное использование данных внутри батча, распределение горячих точек, транзакции в процессе выполнения и физические разрезы сети также по-разному влияют на скорость передачи данных и время выполнения самого медленного этапа. При анализе этих факторов следует исходить из того, где данные создаются, где хранятся и когда используются в следующий раз: это позволяет объяснить, как каждая статья затрат влияет на последующие вычисления. Пример расчёта для UB показывает: складывается ли состояние соединений по числу конечных точек или перемножается по числу пар конечных точек — от этого зависит, сколько хостов поместится в кэш на кристалле и сколько времени займёт установление всех соединений; подключён ли контроллер к шине на кристалле или расположен за PCIe — от этого зависит, сколько уровней приходится пересекать при каждом обращении.

В рассмотренной в этой главе задаче продолжения текста с контекстом 128К один сеанс не помещается на одном ускорителе; экземпляр с восемью ускорителями обеспе-

чивает минимальное время продолжения одного сеанса, тогда как четыре экземпляра по два ускорителя позволяют раньше завершить все четыре сеанса. Сбои, длина контекста и маршруты между серверами дополнительно влияют на доступные варианты развёртывания. В конечном счёте масштаб суперузла служит конкретной задаче: обеспечить достаточный объём памяти, выполнить запросы в установленный срок и снизить ресурсные затраты на один запрос.

Глава 7

Сети центров обработки данных

В предыдущей главе матрицы, слои и эксперты распределялись между разными ускорителями, а размер экземпляра выбирался с учётом стоимости коммуникации. Теперь модель и общее число ускорителей остаются неизменными, но часть ускорителей переносится в другой суперузел: подходит ли прежнее разбиение? Частичные суммы, ранее передававшиеся по высокоскоростному внутрисерверному соединению, теперь, возможно, придётся передавать через InfiniBand или RoCE; поэтому одна и та же синхронизация будет использовать другую полосу пропускания, очереди и домены отказа. Пересчитать нужно размещение разбиения и стоимость его выполнения, а не заново осваивать концепции параллелизма.

После распределения модели между несколькими серверами для продолжения обучения прежде всего необходимо объединить градиенты всех реплик. Несколько реплик параллелизма по данным обрабатывают разные примеры, после чего их градиенты объединяются, а параметры обновляются по одному и тому же батчу. Сначала градиенты реплик объединяются внутри суперузла, а затем передаются между узлами — аналогично описанному в главе 5 локальному накоплению частичных сумм для сокращения внешних перемещений данных. Локальное объединение также требует времени, поэтому такая схема выгодна, только если сэкономленное время удалённой передачи и ожидания превышает эти затраты. Сеть центра обработки данных общего назначения обслуживает множество независимых потоков данных, и вызванная перегрузкой задержка распределяется между ними; при синхронной редукции все ускорители ожидают одну и ту же порцию данных, поэтому момент завершения каждого раунда определяется самой медленной передачей (раздел 7.2.1).

В этой главе исследуется путь данных за пределы одного сервера и то, как ускорители, распределённые между несколькими серверами, совместно выполняют работу. На протяжении всей главы рассматриваются два сервера с восемью ускорителями на каждом: сначала определяется способ редукции градиентов между серверами, затем обсуждается непрерывная отправка и обработка коммуникационных запросов, после чего объясняется, когда данные доступны для чтения, когда буфер можно использовать повторно и как справляться с перегрузкой общей сети. Наконец, этот анализ распространяется на задачу обучения с фиксированными 1024 ускорителями: сравниваются иерархия коммуникаций, пропускная способность и стоимость восстановления при разных размерах суперузлов. Также сравнивается передача небольших объёмов данных при инференсе с низким уровнем параллелизма и объясняется, что следует оптимизировать в первую очередь для одной и той же сети при разных нагрузках.

В главе используются три аналитические модели. **Модель трафика и ресурсов рас-**

считывает время прохождения данных через каждое соединение и каждый интерфейс и выявляет узкие места; **модель параллелизма и пропускной способности** объясняет, сколько запросов нужно обрабатывать одновременно для полного использования пропускной способности; **модель зависимостей и критического пути** помогает определить, какие операции можно выполнять одновременно и сокращение какой операции уменьшит общее время выполнения. Несколько приведённых в книге примеров проектирования основаны на работах, в которых участвовал автор: KV-Direct позволяет программируемым сетевым адаптерам напрямую обрабатывать запросы к хранилищу ключей и значений; 1Pipe использует сеть для задания глобального порядка операций, упрощая координацию распределённых программ; UB предоставляет унифицированное соединение устройств. Участвуя в этих исследованиях и разработках, автор неизменно размышлял над тремя вопросами: как сократить перемещение данных, как выполнять другую работу во время ожидания завершения операции и как упростить программирование, не допуская остановки системы.

В этой главе GB/s обозначает десятичную пропускную способность, а KiB, MiB и GiB — двоичную ёмкость, $1 \text{ GiB} = 2^{30}$ байт.

7.1 От суперузла к кластеру

7.1.1 Как требования к хранилищу приводят к взаимодействию между серверами

Одна из непосредственных причин взаимодействия между серверами состоит в том, что требуемый модели объём хранилища превышает ёмкость одной машины. Рассмотрим модель с 8B параметров, обучаемую с помощью Adam в смешанной точности. **Смешанная точность** означает использование при обучении разной числовой точности для разных объектов. Если параметры, градиенты, основные веса FP32, а также первые и вторые моменты оптимизатора суммарно занимают 16 байт/параметр, потребуется 128 GB. Одна карта на 80 GB этого не вместит; две карты суммарно дают 160 GB, и после размещения этих состояний остаётся 32 GB для активаций и рабочей области. При увеличении числа параметров до 284B те же состояния в таком же представлении данных потребуют около 4.5 TB, поэтому их придётся распределить между большим числом карт для хранения и обработки.¹

При инференсе не требуется сохранять градиенты и состояния оптимизатора, однако веса большой модели всё равно могут не помещаться на одном сервере. Если хранить параметры DeerpSeek V4-Pro, содержащей около 1.6T параметров, и Kimi K3, содержащей около 2.8T параметров, из расчёта 0.5 байта на параметр, их веса займут примерно 800 GB и 1.4 TB соответственно. Сервер с восемью картами по 80 GB предоставляет суммарно 640 GB, поэтому только для хранения весов потребуется не менее двух и трёх серверов соответственно. Маршрутизатор MoE выбирает для каждого токена лишь часть экспертов, уменьшая объём вычислений в данном прямом проходе; веса невыбранных экспертов всё равно необходимо хранить.

После разделения весов программа должна связать вычисления их отдельных частей.

¹Методика расчёта размера модели и объёма состояния приведена в разделе «Расчёт ресурсов модели» и в расширенных материалах к главе 7. В примере с Adam смешанной точности для хранения состояния обучения отводится 16 bytes на параметр; в примере инференса модели с триллионом параметров ёмкость весов рассчитывается исходя из 0,5 byte на параметр.

При разбиении по слоям между стадиями необходимо передавать активации; при разбиении внутри слоя — объединять локальные результаты вычислений; если несколько копий модели обрабатывают разные примеры — объединять градиенты. Поэтому даже для одной и той же модели разные способы разбиения создают разные требования к коммуникации.

Для сравнения этих способов коммуникации в этой главе используется коммуникационная группа из шестнадцати карт на двух серверах H100 спецификации HGX, причём на каждой карте запущен один коммуникационный процесс. Эти процессы совместно выполняют коллективные операции; далее они называются участниками и различаются по rank. Участники с 0 по 7 находятся на сервере А, а с 8 по 15 — на сервере В. Внутри сервера восемь карт полностью соединены через NVSwitch с помощью NVLink. Совокупная пропускная способность NVLink каждой H100 составляет 900 GB/s — это сумма приёма и передачи, то есть 450 GB/s в каждом направлении. Каждая карта оснащена сетевой картой (network interface card, NIC) на 400 Gbit/s, обеспечивающей 50 GB/s в каждом направлении; NIC соединена с этой картой через коммутатор PCIe и не используется совместно с другими картами. Сетевые карты с одинаковыми номерами на двух серверах подключены к одному коммутатору; такой путь от одной сетевой карты до другой называется **рельсом (rail)**. Коммутационная сеть способна передавать двусторонний трафик между двумя серверами. Время запуска одного раунда коллективной операции принимается равным 0.83 μ s: опубликованные результаты nccl-tests показывают, что один AllReduce данных объёмом от 16 В до 128 В на шестнадцати картах двух таких серверов занимает около 25 μ s. Для столь малого объёма данных временем передачи можно пренебречь; если равномерно распределить 25 μ s между 30 раундами кольцевого AllReduce из следующего раздела, получится около 0.83 μ s на раунд.²

Граница серверов на рис. 7-1 — это также коммуникационная граница, неоднократно анализируемая в этой главе. Возьмём тензор градиентов объёмом 192 MiB, результат редукции которого должны получить все шестнадцать участников. В примере 7.6 (раздел 7.6.2) вычисления шага обучения занимают 20 ms, а градиенты готовы через 17 ms. Чтобы полностью скрыть коммуникацию за вычислениями, её необходимо завершить за 3 ms; в этой главе 3 ms принимаются за бюджет данной коммуникации. Этот пример удобен для расчётов по раундам и наглядно показывает главную сложность взаимодействия между серверами: локальные результаты вычислений необходимо передать на другой сервер через сетевую карту, пропускная способность которой значительно ниже, чем у NVLink.

Сетевая карта гораздо медленнее NVLink, поэтому сначала необходимо решить, ка-

²Техническое описание HGX H100: восемь GPU соединены через NVSwitch, пропускная способность NVLink между GPU составляет 900 GB/s, а скорость сети достигает 400 Gbit/s; в спецификации NVIDIA H100 для H100 SXM указаны NVLink 900 GB/s и PCIe Gen5 128 GB/s, причём оба значения представляют суммарную пропускную способность приёма и передачи; на странице спецификаций NVLink приведена пропускная способность NVLink на один GPU для каждого поколения; техническое описание ConnectX-7: один порт до 400 Gbit/s, интерфейс хоста PCIe Gen5 x16; техническое описание DGX H200: восемь GPU и восемь ConnectX-7 по 400 Gbit/s. Конфигурация с одной сетевой картой 400 Gbit/s на каждый GPU взята из опубликованных журналов запусков experiments/ch07/07-03: в каждом из двух серверов HGX установлено по 8 H100 и 8 сетевых карт ConnectX-7 400 Gbit/s InfiniBand. Rail-топология, в которой сетевые карты с одинаковыми номерами подключены к одному коммутатору, приведена в эталонной архитектуре DGX SuperPOD H100. Пропускная способность PCIe Gen5 x16 в одном направлении, равная 64 GB/s, превышает 50 GB/s сетевой карты, поэтому ограничением этого пути служит сетевая карта. Время запуска каждого раунда 0,833 μ s выведено из тех же опубликованных записей: в исходном журнале невнутриместный AllReduce для 16 rank при объёме от 16 В до 128 В занимает 24,93–25,68 μ s; берётся 25 μ s и делится на 30 раундов кольцевого AllReduce с шестнадцатью участниками. Фактически выбранный NCCL для малых сообщений алгоритм в журнале не опубликован, поэтому здесь выполнен лишь пересчёт по кольцевой модели этой главы.

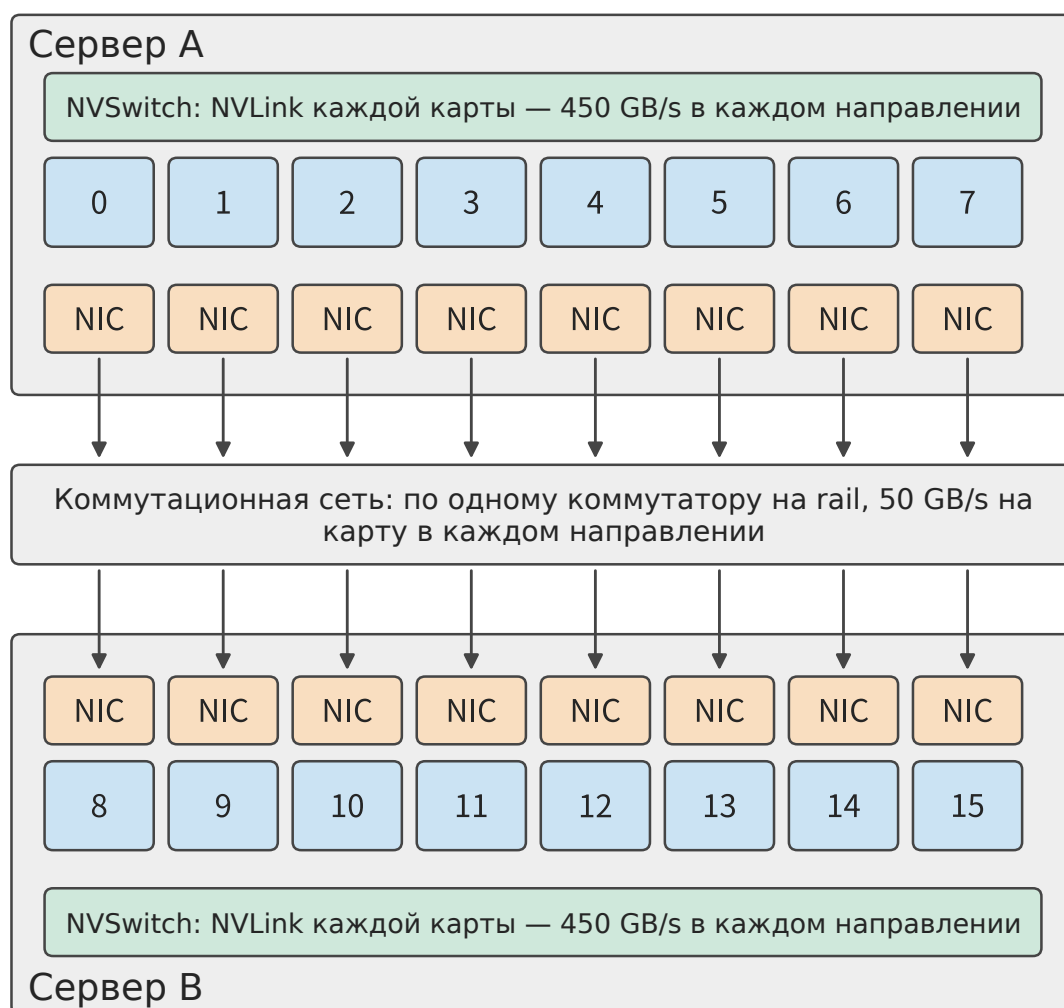


Рис. 7-1. Два фиксированных сервера, по восемь карт на каждом. Карты внутри сервера соединены через NVLink и NVSwitch; у каждой карты есть собственная сетевая карта. Данные между серверами проходят от сетевой карты отправителя через коммутационную сеть к сетевой карте получателя; каждая сетевая карта обеспечивает 50 GB/s в каждом направлении.

кие виды коммуникации размещать между серверами. В этой главе используется без повторного определения обзор способов параллелизма из раздела 6.1.4. В первую очередь следует рассмотреть варианты, в которых частые редукции тензорного параллелизма остаются внутри более быстрого соединения, а между серверами размещаются синхронизация градиентов при параллелизме по данным или передача между стадиями конвейерного параллелизма. Если локальной ёмкости недостаточно, контекстному параллелизму требуется контекст большего масштаба либо маршрутизация при экспертном параллелизме пересекает несколько узлов, область коммуникации приходится расширять. Целесообразность такого размещения определяется фактическим трафиком и критическим путём.

Само пересечение границы суперузла не изменяет объём обмениваемых данных. Число передаваемых байтов определяется алгоритмом, размещением, точностью и устранением дубликатов у получателя, а путь определяет, через какой выход пройдут эти байты и сколько времени придётся ждать. Одного знания о том, что сеть использует InfiniBand или RoCE, недостаточно для определения эффективной пропускной способности, задержки передачи, коэффициента переподписки (отношения пропускной способности нисходящих и восходящих подключений коммутатора), размера окна данных в пути и времени восстановления после сбоя. Далее расчёты выполняются по явно заданным параметрам сети; при реальном развёртывании их следует заменить измеренными значениями соответствующей системы.

Эксперимент 7-1. Расширение: разбиение модели между восьмикартowymi серверами

Рассчитайте минимальное число серверов для двух указанных выше моделей инференса, дополнительно выделив на каждой карте 8 GB для KV-кэша и рабочей области. Отдельно изобразите распределение работы при тензорном или экспертном параллелизме внутри сервера и конвейерном параллелизме между серверами, а также при тензорном параллелизме между серверами. Отметьте места межсерверной передачи при одном выполнении `prefill` и одном выполнении `decode`.

7.1.2 Модель трафика и ресурсов

После определения расположения карт и размера градиентов можно рассчитать, сколько времени займёт прохождение данных через каждый участок пути. Блок данных выходит из памяти GPU, через интерфейс ускорителя поступает на сетевую карту, затем проходит через коммутационную сеть и в конце записывается в память получателя. Один и тот же блок данных должен последовательно пройти через эти интерфейсы и соединения, но разные блоки могут одновременно передаваться на разных участках, образуя конвейер. При длительной передаче большого объёма данных общую пропускную способность определяет участок с наименьшей пропускной способностью. Например, карта может отправлять через NVLink 450 GB/s в каждом направлении, но её сетевая карта способна передавать лишь 50 GB/s в каждом направлении, поэтому скорость длительной передачи с этой карты на другой сервер ограничена значением 50 GB/s сетевой карты.

При переходе от одного пути ко всему кластеру можно сначала выбрать анализируемую границу сети, а затем подсчитать все проходящие через неё данные. Если разделить сетевые узлы на две группы, соединения между ними образуют разрез из главы 6. Если для выполнения задачи требуется передать в некотором направлении через разрез V_c

байт, а его эффективная пропускная способность в этом направлении равна B_c , время передачи составляет не менее

$$T_c = \frac{V_c}{B_c}.$$

Эта формула задаёт нижнюю границу времени передачи: при любом планировании число байтов, передаваемых по соединению за секунду, не может превышать его пропускную способность. Для полнодуплексного соединения оба направления рассчитываются отдельно; если несколько портов совместно используют внутренний интерфейс, для этого интерфейса рассчитывается суммарный трафик. Один и тот же блок данных должен последовательно пройти через все участки цепочки, поэтому сквозная пропускная способность ограничена самым медленным участком, а пропускные способности участков нельзя складывать.

Ширина разреза зависит от организации коммутационной сети. **Сеть Clos** состоит из нескольких уровней коммутаторов и предоставляет конечным узлам несколько путей через промежуточные уровни коммутации. В иерархической сети Clos leaf-коммутаторы соединены с серверами, а коммутаторы верхнего уровня предоставляют несколько путей между leaf-коммутаторами. Например, каждый порт коммутатора NVIDIA Quantum QM9700 InfiniBand, используемого в эталонной архитектуре DGX SuperPOD, работает на скорости класса NDR для InfiniBand — 400 Gbit/s, то есть 50 GB/s в каждом направлении. Если leaf-коммутатор использует 48 портов для подключения серверов и 16 портов для подключения к верхнему уровню, совокупная нисходящая пропускная способность составляет 2.4 TB/s, а восходящая — 0.8 TB/s. Их отношение 3:1 называется **коэффициентом переподписки**. Эти восходящие соединения с совокупной пропускной способностью 0.8 TB/s образуют разрез, через который должен пройти весь трафик, покидающий данный leaf-коммутатор. Серверы, подключённые к одному leaf-коммутатору, могут обмениваться данными напрямую; пропускную способность восходящих соединений использует только коммуникация между разными leaf-коммутаторами.

Пример 7.1. Когда выигрыш от увеличения числа карт ограничивается межсерверной коммуникацией? Расположим шестнадцать участников в кольцо для выполнения редукции с расчётом по раундам, описанным в разделе 7.2.1. Каждый сервер отправляет другому 360 MiB, и все эти байты проходят через одну сетевую карту с пропускной способностью 50 GB/s в каждом направлении. При исходном числе карт вычисления занимают 20 ms. При увеличении числа карт вычисления равномерно распределяются между ними, а объём и путь межсерверной передачи не изменяются. Найдите время завершения при полностью последовательном выполнении и при полном перекрытии.

Решение: нижняя граница времени передачи в одном направлении равна

$$T_c = \frac{360 \times 2^{20}}{50 \times 10^9} \approx 7.5 \text{ ms}.$$

Обозначим отношение нового числа карт к исходному через x ; тогда время вычислений станет равным $20/x$ ms. При последовательном выполнении потребуется $20/x + 7.5$ ms; при полном перекрытии вычислений и коммуникации общее время равно большему из этих двух значений. При удвоении числа карт результаты снижаются примерно с 27.5 и 20.0 ms до 17.5 и 10.0 ms соответственно. При четырёхкратном увеличении последовательное выполнение занимает около 12.5 ms, а выполнение с полным перекрытием — около 7.5 ms. На рис. 7-2 показаны зависимости для обоих вариантов от числа карт.

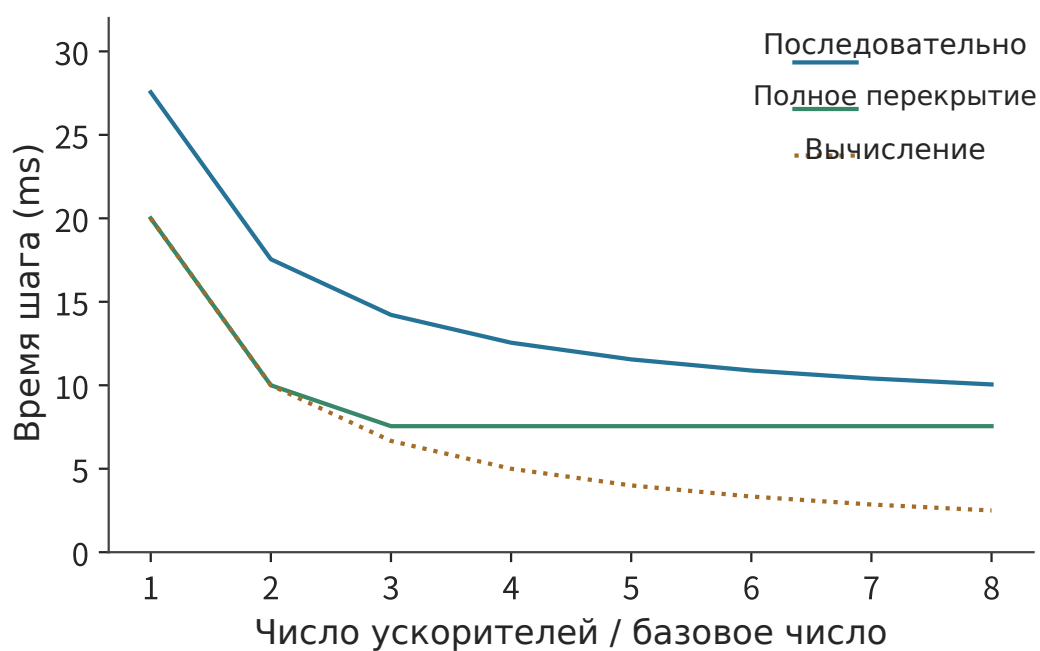


Рис. 7-2. Эффект увеличения числа карт при фиксированном объеме межсерверной передачи. В каждом направлении через одну сетевую карту с пропускной способностью 50 GB/s передается 360 MiB; исходное время вычислений составляет 20 ms и сокращается обратно пропорционально увеличению числа карт. Кривые рассчитаны для последовательного выполнения вычислений и коммуникации и для их полного перекрытия; коммуникационная составляющая равна времени передачи через разрез.

Когда время вычислений равно времени передачи, $20/x = 7.5$, откуда $x \approx 2.6$. После этой точки вычисления уже занимают меньше времени, чем передача, поэтому дальнейшее добавление карт не сокращает время выполнения при полном перекрытии. Для дальнейшего сокращения времени необходимо уменьшить объём данных, проходящих через разрез, либо повысить его эффективную пропускную способность, задействовав одновременно больше сетевых карт. В разделе 7.2 будет показано, как реорганизация той же редукции позволяет добиться обоих результатов одновременно.

Подход, при котором общий объём задачи фиксирован, а число карт увеличивается, называется **сильным масштабированием**; при **слабом масштабировании** объём задачи увеличивается вместе с числом карт. Например, если объём передачи в каждом направлении также удвоится, а выход останется прежним, время коммуникации увеличится вдвое. Форма кривой масштабирования зависит от того, как с увеличением числа карт изменяются объём вычислений и объём межсерверной передачи.

Эксперимент 7-2. Расширение: как объём коммуникации и пропускная способность выхода ограничивают выигрыш от масштабирования

Используя условия примера 7.1, отдельно рассмотрите два изменения: уменьшение вдвое трафика в каждом направлении и одновременную работу двух сетевых карт, удваивающую пропускную способность выхода. Для каждого случая постройте зависимости времени завершения от кратности увеличения числа карт при последовательном выполнении и полном перекрытии, а также найдите кратность числа карт, при которой время вычислений равно времени коммуникации. Затем примите, что трафик растёт линейно с кратностью числа карт, и найдите кратность, минимизирующую время выполнения при полном перекрытии.

Число уровней сети Clos и бисекционная пропускная способность. Выше восходящие соединения leaf-коммутатора рассматривались как разрез. Чтобы определить ширину разреза всего кластера, нужно исходить из числа портов одного коммутатора. Пусть у каждого коммутатора имеется k портов с пропускной способностью 50 GB/s в каждом направлении; для QM9700 $k = 64$. Leaf-коммутатор использует d портов для подключения серверов и u портов для подключения к spine-коммутаторам верхнего уровня; отношение d/u и есть коэффициент переподписки. В неблокируемой сети $d = u = k/2$. В двухуровневой сети Clos каждый spine-коммутатор использует все свои k портов, подключая каждый к отдельному leaf-коммутатору, а $k/2$ восходящих соединений каждого leaf-коммутатора подключаются к $k/2$ spine-коммутаторам. Поэтому такая сеть может содержать не более k leaf-коммутаторов и $k^2/2$ конечных узлов. Трёхуровневая сеть с организацией pod по схеме fat-tree, то есть многоуровневой Clos без сужения совокупной пропускной способности между уровнями, где несколько leaf-коммутаторов и группа коммутаторов промежуточного уровня образуют один pod, а разные pod соединяются через коммутаторы верхнего уровня, поддерживает не более $k^3/4$ конечных узлов. При $k = 64$ двухуровневая сеть соединяет 2048 конечных узлов с помощью 96 коммутаторов. Именно такая конфигурация из 64 leaf-коммутаторов и 32 spine-коммутаторов используется для 2048 GPU в эталонной архитектуре DGX SuperPOD. Трёхуровневая сеть соединяет 65536 конечных узлов с помощью 5120 коммутаторов.³

³Неблокирующий Clos на 64 порта, переподписка 3:1: leaf-коммутатор имеет d нисходящих и u восходящих линий, верхний уровень не блокирует восходящие линии leaf-коммутаторов, а три уровня следуют структуре fat-tree; пропускная способность половинного разреза рассчитывается как половина линий верх-

Совокупная пропускная способность соединений между двумя половинами кластера с одинаковым числом конечных узлов называется **бисекционной пропускной способностью** (bisection bandwidth). Бисекционная пропускная способность неблокируемой сети равна суммарной пропускной способности половины конечных узлов: для двухуровневой сети с 2048 конечными узлами она составляет $1024 \times 50 = 51.2$ TB/s, а для трёхуровневой — 1638.4 TB/s. При коэффициенте переподписки 3 leaf-коммутатор использует 48 нисходящих и 16 восходящих портов; двухуровневая сеть может подключить 3072 конечных узла, используя всего 80 коммутаторов. У 64 leaf-коммутаторов имеется в общей сложности 1024 восходящих соединения, поэтому бисекционная пропускная способность равна суммарной пропускной способности половины из них — 25.6 TB/s. Это лишь треть от 76.8 TB/s, которые обеспечила бы неблокируемая сеть для тех же 3072 конечных узлов; доля пропускной способности бисекционного разреза на один конечный узел снижается с 50 GB/s до 16.7 GB/s. На рис. 7-3 показана двухуровневая сеть Clos и её бисекционный разрез.

Когда задание обучения на 1024 картах из раздела 7.6.4 занимает целое число leaf-коммутаторов, его разрез образуют все восходящие соединения этих leaf-коммутаторов:

него уровня; разрез при разбиении 1024 карт рассчитывается для целого числа занятых leaf-коммутаторов; исходящая пропускная способность суперузла равна 50 GB/s на карту из сценария с 1024 картами, умноженным на число карт и поделённым на коэффициент переподписки, а число байтов, пересекающих домены, взято из расчёта для 1024 карт. Используется коммутатор NVIDIA Quantum QM9700 (NDR 400 Gbit/s) из эталонной архитектуры DGX SuperPOD H100: в её таблице 3 для 2048 GPU используются 64 leaf-коммутатора и 32 spine-коммутатора, что соответствует числу коммутаторов в двухуровневой 64-портовой сети Clos. Расчёт можно повторить командой `python3 calculations/calc.py clos-cut --inputs calculations/scenarios/clos-cut-example.json --format md`.

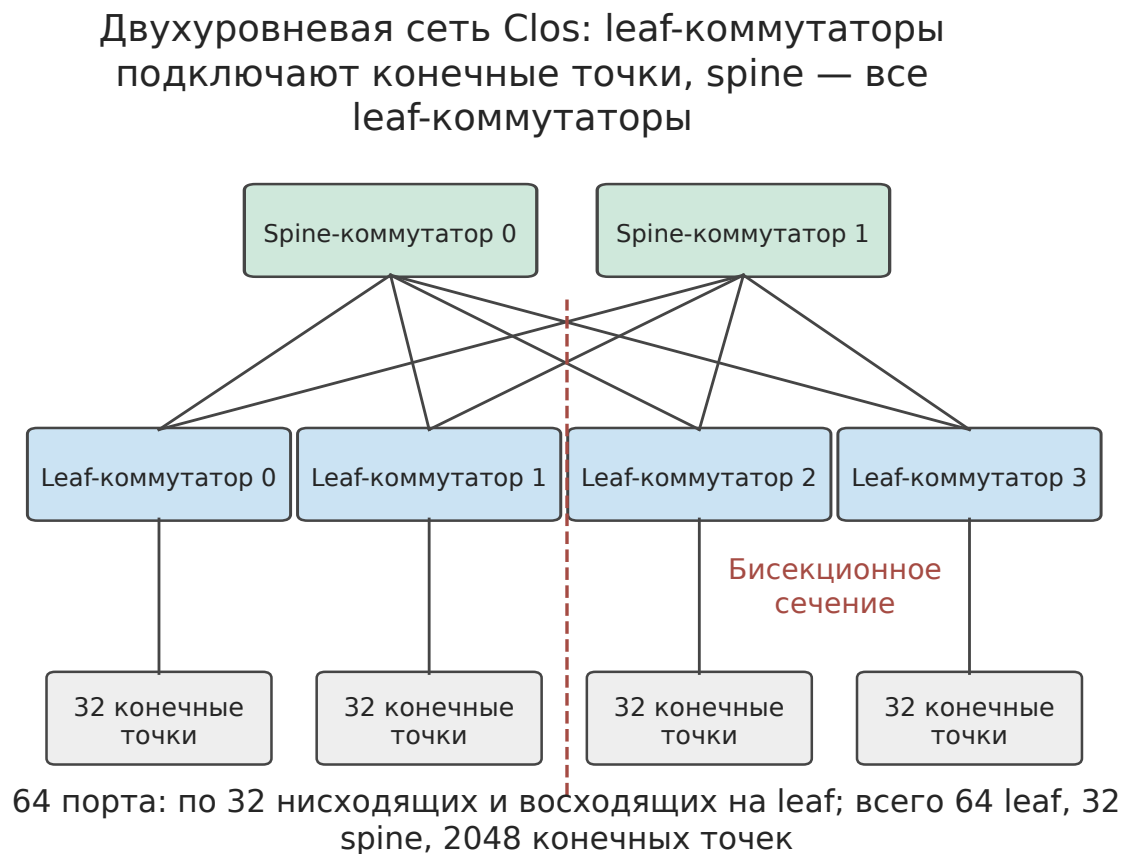


Рис. 7-3. Двухуровневая сеть Clos. Половина портов каждого leaf-коммутатора подключена к серверам, а другая половина — к spine-коммутаторам; любые два leaf-коммутатора могут обмениваться данными через любой spine-коммутатор. Пунктирная линия делит конечные узлы на две половины, а пересекающие её соединения между leaf- и spine-коммутаторами образуют бисекционный разрез. На рисунке для наглядности показаны четыре leaf-коммутатора и два spine-коммутатора; при 64 портах фактически используются 64 leaf-коммутатора и 32 spine-коммутатора.

Коэффициент переподписки	Нисходящие/восходящие порты на leaf-коммутатор	Число leaf-коммутаторов для 1024 карт	Число соединений в разрезе	Пропускная способность разреза	Доля на карту
1:1	32/32	32	1024	51.2 TB/s	50 GB/s
3:1	48/16	22	352	17.6 TB/s	17.2 GB/s

Коэффициент переподписки также изменяет значения в столбце «выход растёт вместе с числом карт» таблицы из раздела 7.6.4. В этом столбце пропускная способность выхода каждого суперузла принимается равной числу карт, умноженному на 50 GB/s, что подразумевает неблокируемую коммутационную сеть. При коэффициенте переподписки 3 пропускная способность выхода уменьшается до одной трети, а время междоменной передачи утраивается:

Число карт в суперузле	Междоменная передача на узел	Выход без блокировки	Выход при переподписке 3:1	Время междоменной передачи (без блокировки $\rightarrow$ 3:1)
8	127 GB	400 GB/s	133 GB/s	0.318 $\rightarrow$ 0.953 s
64	120 GB	3.2 TB/s	1.07 TB/s	37.5 $\rightarrow$ 112.5 ms
128	112 GB	6.4 TB/s	2.13 TB/s	17.5 $\rightarrow$ 52.5 ms
256	96 GB	12.8 TB/s	4.27 TB/s	7.5 $\rightarrow$ 22.5 ms

В таблице указано чистое время передачи. Междоменная составляющая в разделе 7.6.4 также включает время запуска $2(H - 1)$ раундов по 0.83 μ s каждый. Для суперузла на 128 карт $H = 8$, поэтому суммарное время запуска составляет около 12 μ s и почти не влияет на приведённое в таблице время порядка миллисекунд.

Обсуждение: когда разрез сети с переподпиской становится узким местом? Коэффициент переподписки r уменьшает приходящуюся на каждую карту долю пропускной способности разреза до $50/r$ GB/s. Если только доля f байтов, отправляемых сетевой картой, должна покинуть её leaf-коммутатор, условие отсутствия узкого места на восходящих соединениях имеет вид $f \leq 1/r$: в неблокируемой сети ограничений нет при любом f , а при $r = 3$ ограничение отсутствует, только если между leaf-коммутаторами проходит менее трети байтов. Если на межсерверной стадии иерархической редукции два парных сервера не подключены к одному leaf-коммутатору — в многорельсовой топологии из раздела 7.2.5 сетевые карты с одинаковыми номерами на разных серверах подключаются к одному leaf-коммутатору, по 32 сервера в группе, поэтому данный случай означает, что серверы находятся в разных группах, — все байты каждой сетевой карты должны пройти между leaf-коммутаторами, то есть $f = 1$. В таком случае разрез сети с переподпиской непосредственно увеличивает длительность этой стадии в r раз. Если два сервера находятся в одной группе, согласованное образование пар из раздела 7.2.5 позволяет байтам каждой пары пройти только через один leaf-коммутатор, то есть $f = 0$, и восходящие соединения не передают ни одного байта этой редукции. Размещение серверов одной группы синхронизации в одной группе — один из способов уменьшить f .

Эксперимент 7-3. Расширение: как число портов коммутатора и коэффициент переподписки определяют разрез

Измените число портов коммутатора на 128 и отдельно рассчитайте число конечных узлов, число коммутаторов и бисекционную пропускную способность для неблокируемых двухуровневой и трёхуровневой сетей Clos. Сохранив 64 порта, измените коэффициент переподписки на 2:1, рассчитайте пропускную способность разреза раздела на 1024 карты и долю на одну карту, а также время междоменной передачи для суперузла на 128 карт из раздела 7.6.4. Наконец, при коэффициенте переподписки 3 найдите, какая доля байтов каждой сетевой карты на межсерверной стадии иерархической редукции должна

оставаться внутри своего leaf-коммутатора, чтобы разрез больше не был узким местом.

7.1.3 Унифицированное соединение

Приведённые выше нижние границы показывают ограничения физических соединений. Однако перед программой стоит и другая проблема: для передачи одних и тех же данных внутри машины и между серверами часто приходится использовать разные интерфейсы доступа. Это связано с давно сложившимся разделением ролей между шинами и сетями: тесно связанные соединения подключают устройства к процессору и системе памяти и подходят для частого мелкогранулярного взаимодействия; кластерные сети соединяют множество независимых узлов и должны также обрабатывать отказы узлов и соединений. Когда модель размещается на нескольких серверах, одно вычисление одновременно использует оба механизма. Локальный доступ к памяти и удалённая передача выражают в программе сходные зависимости по данным, но используют разные пути отправки и завершения операций.

Когда автор участвовал в раннем проектировании UB, одной из ключевых задач было создание единого интерфейса для локального и удалённого взаимодействия. Приложению требуется выразить такие действия, как «прочитать эти данные», «передать результат следующей стадии» и «после завершения повторно использовать пространство». Унифицированное соединение предоставляет общий интерфейс для адресации, доступа к данным и коммуникации между устройствами, позволяя участвовать в ней CPU, ускорителям и другим устройствам.

Для понимания этой унификации необходимо различать две вещи. Интерфейс определяет, как программа выражает работу, а физический путь — через какие ресурсы эта работа проходит. Удалённое чтение можно инициировать одной инструкцией, но данные всё равно должны пройти через выход сетевой карты с пропускной способностью 50 GB/s. Асинхронную запись можно отправить заранее, однако получатель сможет начать использовать данные только после их готовности. Поэтому в этой главе сначала определяется обязательный объём трафика, затем — число параллельных запросов, необходимое для полного использования пропускной способности, и наконец анализируется требуемый приложением порядок операций. Преимущества унифицированного интерфейса также проявляются на этих трёх уровнях: уменьшение числа перемещений, сокращение пути инициирования и упрощение взаимодействия. В разделе 6.5.5 уже рассматривались с точки зрения масштаба суперузла два проектных решения UB: подключение контроллера к внутрикристальной шине и отдельное хранение состояния соединения в записях конечных точек и транспортных каналах. В этой главе эффекты этих решений поэтапно рассчитываются вдоль пути одного удалённого доступа.

7.2 Межузловой трафик и физические маршруты

7.2.1 Межсерверный трафик при параллелизме по данным

Унифицированный интерфейс не позволяет избежать необходимой передачи, однако алгоритм редукции может изменить то, какие данные требуется передавать между серверами. Начнём с параллелизма по данным: каждый участник вычисляет локальные градиенты, затем соответствующие элементы суммируются, после чего каждый участник

должен получить полный результат редукции. В качестве примера возьмём градиент проекции gate в FFN первого слоя Qwen3-8B размерности [12288, 4096], хранящийся в FP32 по 4 байта на элемент. Входные данные каждого участника имеют объём

$$M = 12288 \times 4096 \times 4 = 192 \text{ MiB}.$$

В разделе 6.4.2 был выведен кольцевой AllReduce: сначала выполняется ReduceScatter, после которого каждая карта хранит одну часть результата редукции, а затем AllGather собирает все эти части на каждой карте; тензор поровну делится на p частей, оба этапа состоят из $p - 1$ раундов, и в каждом раунде следующему участнику отправляется одна часть объёмом M/p . Поэтому каждый участник отправляет

$$V_{\text{rank}} = 2 \frac{p-1}{p} M.$$

Каждый из шестнадцати участников отправляет 360 MiB, суммарно отправляется 5760 MiB за тридцать раундов. Алгоритм определяет общий объём отправки, а распределение участников по серверам — какая его часть должна передаваться между серверами и через какие сетевые карты.

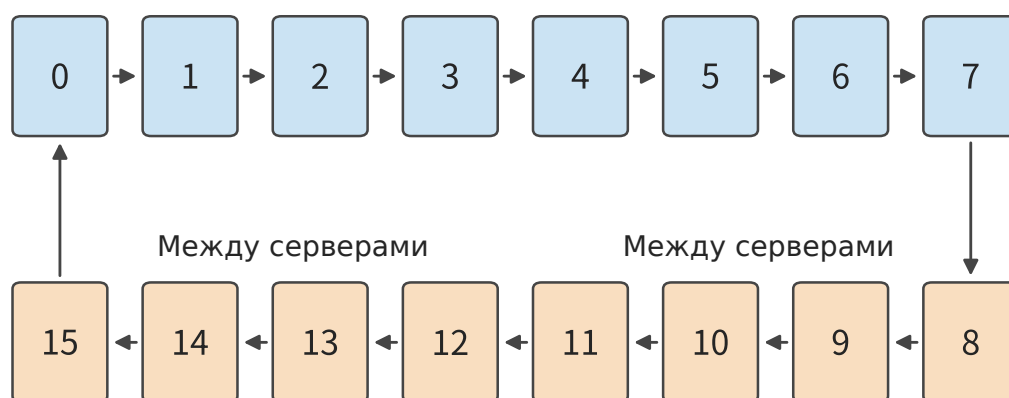
Задача 7.2: какой алгоритм редукции способен завершить передачу за 3 ms? На двух серверах из раздела 7.1 сравним непрерывное кольцо без иерархии, чередующееся кольцо и иерархическую редукцию.

Решение: вычислим объём межсерверной передачи для каждого варианта редукции. В непрерывном кольце участники расположены в порядке 0, 1, ..., 15. Из шестнадцати ориентированных рёбер между серверами проходят только $7 \rightarrow 8$ и $15 \rightarrow 0$. В каждом раунде по каждому ребру отправляется 12 MiB; за тридцать раундов в каждом направлении отправляется по 360 MiB, всего в двух направлениях — 720 MiB. На сервере А эти байты проходят только через сетевую карту участника 7, а на сервере В — только через сетевую карту участника 15; остальные четырнадцать сетевых карт простаивают. Если вместо этого расположить участников с чередованием 0, 8, 1, 9, ..., 7, 15, все шестнадцать рёбер будут проходить между серверами; в каждом раунде каждая сетевая карта отправляет по 12 MiB, а общий объём межсерверной передачи возрастает до 5760 MiB.

При иерархической редукции восемь участников на каждом сервере сначала выполняют локальный ReduceScatter. После семи раундов у каждого участника остаётся одна восьмая результата редукции своего сервера, то есть 24 MiB. Затем два участника с разных серверов, отвечающие за одну и ту же часть, выполняют один AllReduce: каждый делит свои 24 MiB на две части, за один раунд обменивается одной из них и выполняет её редукцию, а за второй раунд получает другую часть. Каждая пара отправляет в обоих направлениях суммарно 48 MiB, а восемь пар — 384 MiB; каждая пара использует собственный rail, поэтому восемь сетевых карт работают одновременно. Наконец, на каждом сервере за семь раундов AllGather собирается полный результат. На рисунках 7-4–7-6 показаны пути трёх вариантов, а на рисунке 7-7 сравнивается количество передаваемых между серверами байтов.

Решение: вычислим время редукции по нагрузке на каналы в каждом раунде. Раунды выполняются последовательно, а разные передачи внутри одного раунда — параллельно. Пусть в раунде r через ресурс e проходит объём данных $V_{r,e}$, пропускная способность этого ресурса равна B_e , а время запуска каждого раунда — α . Тогда модель передачи имеет вид

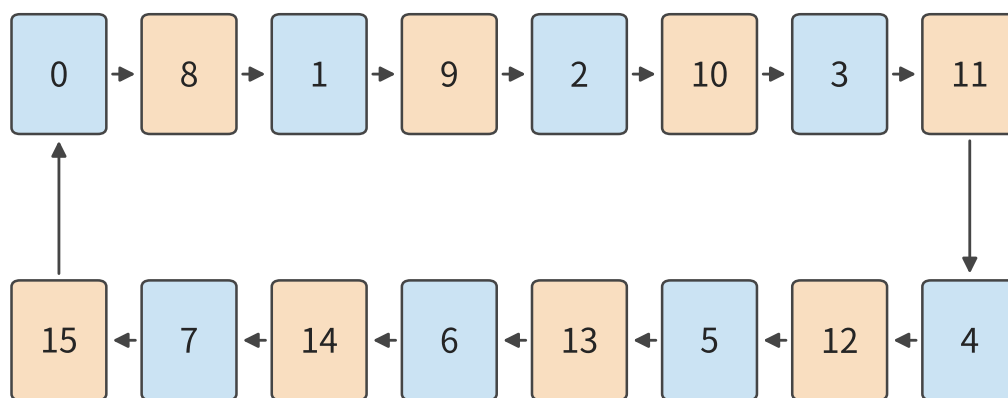
Непрерывное кольцо: лишь 2 из 16 рёбер проходят между серверами



Синий: сервер А; оранжевый: сервер В

Рисунок 7-4. В непрерывном кольце участники одного сервера расположены рядом. Из шестнадцати ориентированных рёбер между серверами проходят только $7 \rightarrow 8$ и $15 \rightarrow 0$, поэтому межсерверный трафик сосредоточен на двух сетевых картах. Цвет обозначает сервер, на котором находится участник.

Чередующееся кольцо: все 16
рёбер межсерверные



Синий: сервер А; оранжевый: сервер В

Рисунок 7-5. При чередующемся размещении участников на серверах А и В все шестнадцать ориентированных рёбер проходят между серверами и работают все шестнадцать сетевых карт. Общий объём отправки каждого участника остаётся неизменным.

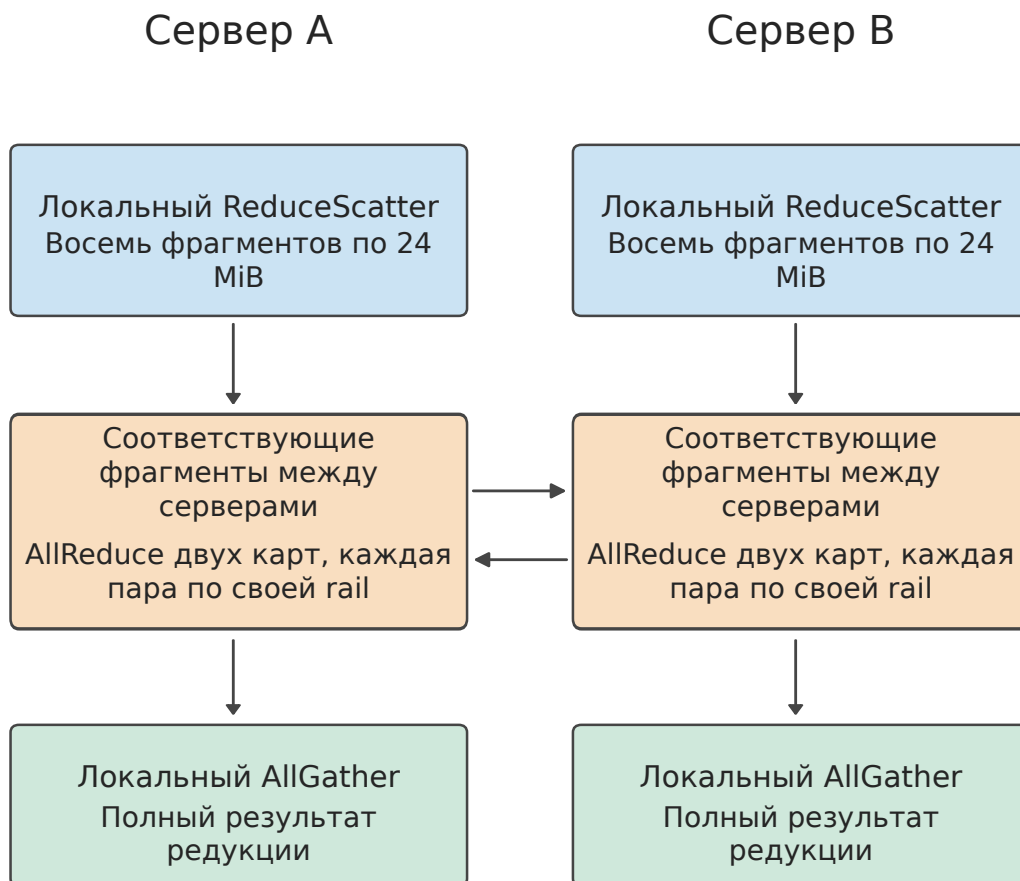


Рисунок 7-6. Каждый сервер сначала выполняет редукцию восьми локальных вкладов в части результата, затем обменивается соответствующими частями с другим сервером и, наконец, локально собирает полный результат. Каждый столбец выполняется сверху вниз; горизонтальные стрелки обозначают межсерверный обмен, а каждая пара участников использует отдельный rail.

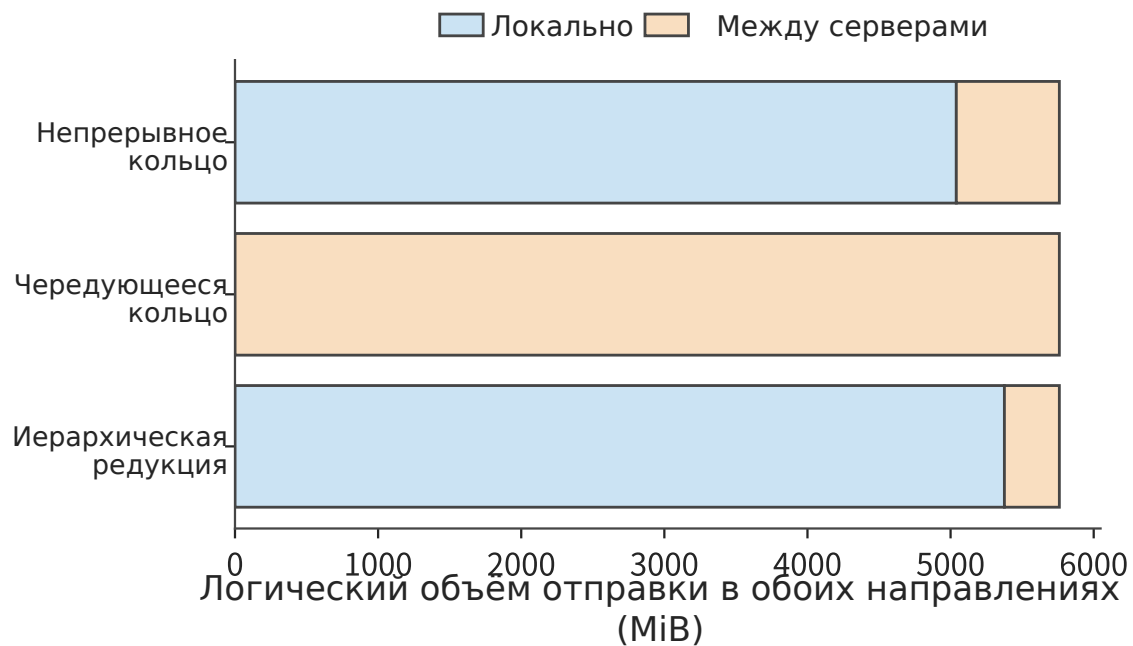


Рисунок 7-7. Логический объем отправки во всех трёх вариантах одинаков и составляет 5760 MiB; оранжевые межсерверные части составляют соответственно 720, 5760 и 384 MiB. Указана сумма по двум направлениям.

$$T_{\text{comm}} = \sum_r \left(\alpha + \max_e \frac{V_{r,e}}{B_e} \right).$$

Последовательно рассчитав каждый раунд по стрелкам трёх описанных вариантов, можно преобразовать объём трафика во время. Формула разделяет две зависимости: внутри раунда время определяется самой медленной передачей, а времена разных раундов складываются последовательно. Здесь учитываются только передача данных и время запуска каждого раунда.⁴

В каждом раунде непрерывного кольца через одну сетевую карту в каждом направлении отправляется 12 MiB, что занимает около 0.25 ms; остальные четырнадцать рёбер того же раунда проходят через NVLink, и передача 12 MiB по каждому из них занимает лишь около 0.03 ms, однако все они должны ждать это одно ребро. Поэтому тридцать раундов занимают около 7.6 ms. В чередующемся кольце каждое из шестнадцати рёбер в каждом раунде отправляет 12 MiB через собственную сетевую карту, поэтому раунд также занимает около 0.25 ms, а тридцать раундов — около 7.6 ms: чередующееся кольцо задействует все сетевые карты, но увеличивает межсерверный трафик в восемь раз, не сокращая время.

В двух межсерверных раундах иерархической редукции каждая сетевая карта передаёт по 12 MiB за раунд, что суммарно занимает около 0.50 ms. Локальные ReduceScatter и

⁴Расчёт двухуровневой редукции градиентов, непрерывное кольцо, чередующееся кольцо. Форма модели фиксирована; каждый раунд синхронизируется барьером, а указанное время представляет собой сумму времени передачи полезной нагрузки и запуска при отсутствии конкуренции: NVLink — 450 GB/s в каждом направлении, одна NIC на карту — 50 GB/s в каждом направлении, запуск каждого раунда — 0,833 μs. Расчёт можно повторить командой `python3 calculations/calculations.py hierarchical-gradient --inputs calculations/scenarios/hierarchical-gradient-example.json --format md`.

AllGather вместе состоят из четырнадцати раундов, в каждом из которых каждый участник отправляет через NVLink 24 MiB; при 450 GB/s это суммарно занимает около 0.78 ms. Запуск шестнадцати раундов занимает около 13 μ s, поэтому

$$T_{\text{hier}} = 14 \frac{24 \text{ MiB}}{450 \text{ GB/s}} + 2 \frac{12 \text{ MiB}}{50 \text{ GB/s}} + 16 \times 0.83 \mu\text{s} \approx 1.3 \text{ ms}.$$

Передача в непрерывном кольце уже превышает бюджет в 3 ms, тогда как иерархическая редукция оставляет около 1.7 ms на вычисления редукции, распространение данных и ожидание в очереди. Иерархическая редукция уменьшает объём межсерверной передачи примерно на 47%, а время коммуникации — примерно на 83%. Столь значительное различие между этими долями объясняется не количеством байтов, а путём их передачи: в каждом из тридцати раундов непрерывного кольца приходится ждать, пока межсерверное ребро передаст через сетевую карту 12 MiB; остальные четырнадцать рёбер проходят через NVLink и после завершения примерно за 0.03 ms ожидают это ребро, а другие четырнадцать сетевых карт весь раунд не передают данные. Иерархическая редукция сжимает межсерверную передачу до двух раундов, в каждом из которых восемь сетевых карт одновременно передают по 12 MiB, поэтому эффективная пропускная способность сечения возрастает с 50 GB/s до 400 GB/s.

Обсуждение: насколько быстрым должно быть локальное соединение, чтобы иерархия была выгодна? Обозначим локальную пропускную способность как B_L , оставив пропускную способность сетевой карты равной 50 GB/s в каждом направлении. В четырнадцати локальных раундах иерархической редукции передаётся суммарно 336 MiB, а два межсерверных раунда занимают около 0.5 ms. Пока B_L не ниже пропускной способности сетевой карты, время непрерывного кольца всегда определяется одним межсерверным ребром и составляет около 7.6 ms. Приравняв времена двух вариантов, $336 \text{ MiB}/B_L + 0.5 \text{ ms} = 7.6 \text{ ms}$, получаем $B_L \approx 50 \text{ GB/s}$: иерархическая редукция быстрее, если локальное соединение не медленнее одной сетевой карты, а 450 GB/s у NVLink намного выше этой границы. На данном сервере иерархическая редукция не несёт дополнительных издержек, поскольку локальный объём отправки каждого участника, 336 MiB, не превышает 360 MiB у непрерывного кольца, а на межсерверном этапе задействованы все восемь сетевых карт.

Иерархическая редукция не всегда обходится без дополнительных затрат. Для сравнения рассмотрим распространённый сервер PCIe, далее называемый контрольной конфигурацией: в каждом сервере установлены четыре A100 80GB PCIe, между которыми данные передаются напрямую через PCIe Gen4 x16 со скоростью 32 GB/s в каждом направлении; также на каждом сервере установлена одна двухпортовая ConnectX-7 с двумя портами по 200 Gbit/s, или 25 GB/s каждый, которые совместно используют слот PCIe Gen4 x16 этой сетевой карты, также ограниченный 32 GB/s в каждом направлении. В непрерывном кольце из восьми участников каждый из четырнадцати раундов передаёт 24 MiB; локальные рёбра ограничены пропускной способностью прямых соединений, межсерверные — слотом сетевой карты, поэтому для каждого раунда используется значение 32 GB/s, а общее время составляет около 11.0 ms. В двух межсерверных раундах иерархической редукции через слот требуется передать по 96 MiB в каждом направлении, что занимает около 6.3 ms; ещё около 9.4 ms занимают шесть раундов локальной коммуникации, и суммарное время составляет около 15.7 ms — больше, чем у непрерывного кольца. Равенство достигается при локальной пропускной способности около

64 GB/s, то есть примерно вдвое выше пропускной способности слота сетевой карты.⁵ Мост NVLink у A100 PCIe обеспечивает 300 GB/s в каждом направлении, что намного выше этой границы, однако один мост соединяет только две карты, поэтому в кольце из четырёх карт два ребра всё равно проходят через PCIe; если бы такую пропускную способность имело каждое ребро кольца из четырёх карт, время иерархической редукции снизилось бы примерно до 7.3 ms и стало бы меньше времени непрерывного кольца. Причина различия — общий интерфейс: межсерверный этап концентрирует части восьми участников в двух раундах, и весь трафик обоих раундов должен пройти через один слот с пропускной способностью 32 GB/s, поэтому более длинные межсерверные раунды частично сводят на нет экономию удалённо передаваемых байтов; локальные каналы также не быстрее этого слота, поэтому компенсировать дополнительное время локальной редукции не удаётся.

Иерархическая редукция сначала выполняет частичную редукцию через более быстрое локальное соединение, а затем отправляет результат через сетевые карты. Она уменьшает количество передаваемых между серверами байтов, но ещё важнее то, что у каждой сетевой карты появляется собственная часть для передачи. Если межсерверную пропускную способность обеспечивают несколько независимых сетевых карт, иерархия почти ничего не стоит; если же межсерверная пропускная способность предоставляется общим интерфейсом, иерархическая редукция будет быстрее только тогда, когда дополнительное время локальной редукции меньше сэкономленного времени удалённой передачи.

Четвёртый вариант: редукция в сети. В первых трёх вариантах участники передают данные попарно, а коммутатор лишь пересылает их. **Редукция в сети** позволяет коммутатору выполнять сложение одновременно с пересылкой: NVIDIA SHARP (Scalable Hierarchical Aggregation and Reduction Protocol, масштабируемый протокол иерархической агрегации и редукции) выполняет в микросхеме коммутатора редукцию данных, поступающих с нескольких портов по дереву агрегации, а затем распределяет результат обратно, поэтому конечным узлам не требуется многократно отправлять одни и те же данные.⁶ Сохраним локальные ReduceScatter и AllGather из иерархической редукции, изменив только межсерверный этап: каждый участник один раз отправляет коммутатору свою часть размером 24 MiB и один раз получает обратно результат редукции; всё выполняется за один раунд. При двух серверах каждая сетевая карта по-прежнему передаёт по 24 MiB в каждом направлении, а общий объём межсерверной передачи по-прежнему составляет 384 MiB, как и при иерархической редукции; время сокращается с 0.505 ms за

⁵Иерархическая редукция в контрольной конфигурации, непрерывное кольцо, чередующееся кольцо: в каждом сервере установлены четыре A100 80GB PCIe; двухточечные соединения между картами через PCIe Gen4 x16 обеспечивают 32 GB/s в каждом направлении; одна двухпортовая ConnectX-7 с двумя портами по 200 Gbit/s, каждый из которых обеспечивает 25 GB/s, совместно использует слот PCIe Gen4 x16 с пропускной способностью 32 GB/s в каждом направлении; запуск каждого раунда — 0,833 μ s. В техническом описании A100 80GB для PCIe 4.0 указано 64 GB/s, а для моста NVLink между двумя картами — 600 GB/s; в обоих случаях это суммарная пропускная способность приёма и передачи. В техническом описании ConnectX-7 указаны конфигурации с 1/2/4 портами, суммарная пропускная способность одной карты до 400 Gbit/s и интерфейс хоста PCIe Gen5 x16/x32; контрольная конфигурация рассчитана для установки этой карты в слот PCIe Gen4 x16. Значение локальной пропускной способности 300 GB/s получено повторным расчётом после изменения `local_bytes_per_second` на 300 GB/s.

⁶Контрольный вариант редукции в сети: локальный этап совпадает с иерархической редукцией, а на межсерверном этапе каждая сетевая карта один раз отправляет фрагмент и один раз получает результат; кольцо из S серверов учитывает по $2(S - 1)/S$ фрагмента и $2(S - 1)$ раунда на каждую сетевую карту, запуск каждого раунда занимает 0,833 μ s; пропускная способность механизма редукции коммутатора в модель не включена. Определение SHARP взято из документации NVIDIA SHARP, а измеренные значения — из аннотации статьи о SHARP.

два раунда до 0.504 ms за один раунд, то есть экономится только одно время запуска в 0.83 μ s.

Различие проявляется с увеличением числа серверов. Пусть одну и ту же часть хранят S серверов. При кольцевом AllReduce каждая сетевая карта отправляет $2 \frac{S-1}{S} \times 24$ MiB за $2(S-1)$ раундов; при редукции в сети каждая сетевая карта всегда отправляет данные один раз и получает их один раз. Время межсерверного этапа приведено в следующей таблице и на рисунке 7-8:

Число серверов	Кольцо: отправка на сетевую карту	Кольцо: число раундов	Кольцо: время	Редукция в сети: отправка на сетевую карту	Редукция в сети: время
2	24 MiB	2	0.505 ms	24 MiB	0.504 ms
4	36 MiB	6	0.760 ms	24 MiB	0.504 ms
8	42 MiB	14	0.892 ms	24 MiB	0.504 ms
16	45 MiB	30	0.969 ms	24 MiB	0.504 ms
32	46.5 MiB	62	1.027 ms	24 MiB	0.504 ms

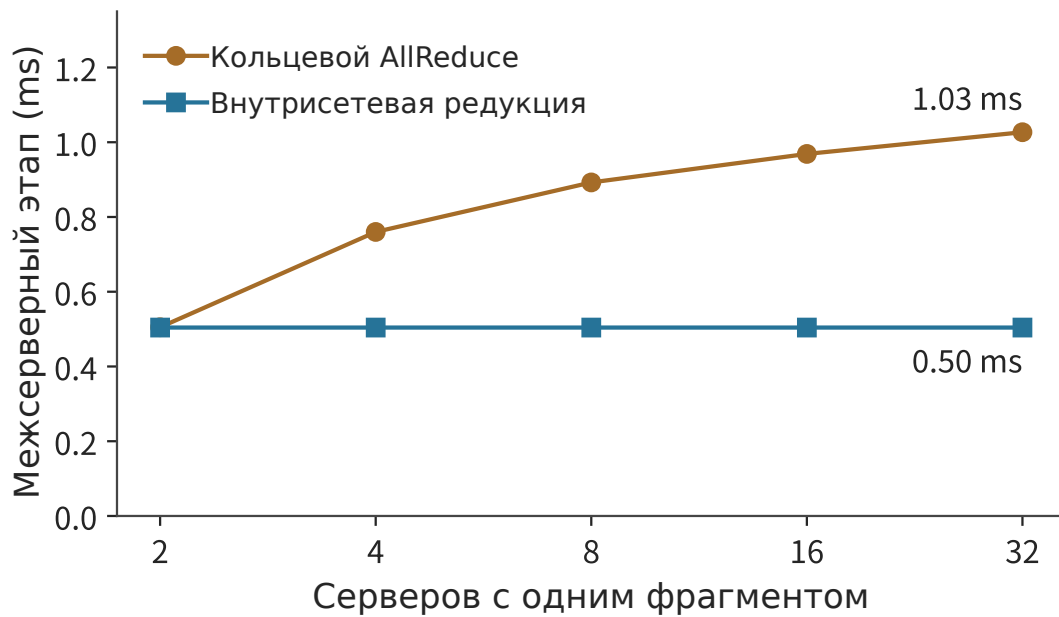


Рисунок 7-8. Время межсерверного этапа при увеличении числа серверов, хранящих одну и ту же часть, с 2 до 32. Объём отправки на сетевую карту в кольце стремится к двум частям, а число раундов растёт как $2(S - 1)$; редукция в сети сохраняет одну часть и один раунд. При двух серверах варианты различаются только одним запуском.

Обсуждение: когда редукция в сети становится заметно быстрее? При двух серверах варианты различаются только одним запуском; начиная с $S = 3$ объём данных кольца в $2(S - 1)/S > 1$ раза больше, чем при редукции в сети, и дополнительно требуется $2S - 3$ запусков раундов, поэтому разрыв увеличивается вместе с числом серверов. При 32 серверах время редукции в сети составляет лишь 49% времени кольца. Эта модель учитывает только последовательную отправку через сетевые карты и запуски; количество байтов, которое механизм редукции коммутатора способен обрабатывать в секунду, и поддерживаемые типы данных должны задаваться отдельно. Если трафик превышает его возможности, узким местом вместо сетевых карт становится коммутатор. В статье о SHARP на 128 хостах время AllReduce для 8 байтов снизилось с $6.01 \mu s$ до $2.83 \mu s$, а для 4096 байтов — с $46.93 \mu s$ до $14.48 \mu s$: для небольших сообщений основная экономия достигается за счёт сокращения числа запусков раундов.⁷

Эксперимент 7-4 • Расширение: когда редукция в сети оправдана

Замените градиент на BF16 (96 MiB на участника) и повторно вычислите время межсерверного этапа кольцевой редукции и редукции в сети для двух и восьми серверов. Считая, что пропускная способность механизма редукции коммутатора в каждом направлении равна 200 GB/s, определите, станет ли он узким местом при одновременной редукции по восьми rail, и найдите минимальную пропускную способность механизма, при которой редукция в сети

⁷Контрольный вариант редукции в сети: локальный этап совпадает с иерархической редукцией, а на межсерверном этапе каждая сетевая карта один раз отправляет фрагмент и один раз получает результат; кольцо из S серверов учитывает по $2(S - 1)/S$ фрагмента и $2(S - 1)$ раунда на каждую сетевую карту, запуск каждого раунда занимает $0,833 \mu s$; пропускная способность механизма редукции коммутатора в модель не включена. Определение SHARP взято из документации NVIDIA SHARP, а измеренные значения — из аннотации статьи о SHARP.

всё ещё будет быстрее кольца. Наконец, для входных данных размером 8 KiB из раздела 7.6.3 сравните разницу в суммарном времени запуска двух вариантов для 36 слоёв и двух редукций на слой.

7.2.2 Требования к обмену данными при тензорном и конвейерном параллелизме

При параллелизме по данным градиенты всех участников объединяются с помощью редукции. Тензорный параллелизм распределяет вычисления одного слоя между несколькими участниками, а последующие операторы зачастую могут начать работу только после объединения локальных результатов, поэтому обмен данными внутри слоя непосредственно задерживает последующие вычисления. Конвейерный параллелизм разделяет модель на стадии, сосредоточивая обмен между серверами на границах стадий. В этих двух подходах данные пересекают границы серверов разное количество раз.

Сначала рассчитаем объём данных при одной передаче между слоями. Ширина скрытого состояния Qwen3-8B равна 4096, поэтому один скрытый вектор BF16 занимает 8 KiB. При prefill аналогичный тензор для 8192 токенов занимает 64 MiB, а при decode одного запроса — 8 KiB. Примем пропускную способность соединения B равной 50 GB/s для рассматриваемого в этой главе сетевого адаптера, а время запуска каждой передачи α — равным 5 μ s. Тогда передача M bytes занимает

$$T_{\text{msg}} = \alpha + \frac{M}{B}.$$

Для 64 MiB требуется приблизительно 1,3 ms, а для 8 KiB — приблизительно 5,2 μ s. В первом случае время расходуется главным образом на передачу данных, во втором около 97% времени занимает запуск. Время запуска равно времени передачи при $M = \alpha B = 250000$ bytes, то есть приблизительно 244 KiB. Если объём данных значительно превышает это значение, доминирует время передачи и эффективнее повышать пропускную способность; если он значительно меньше, эффективнее сокращать количество передач.

Приведённая выше формула определяет стоимость каждой передачи; для конвейерного параллелизма необходимо также учитывать, как многократные передачи чередуются с вычислениями. При конвейерном параллелизме несколько последовательных слоёв размещаются на одной стадии, а между стадиями передаются активации. Пока предыдущая стадия обрабатывает следующий микробатч, последующая может обрабатывать предыдущий. Рассмотрим четыре стадии прямого прохода, каждая из которых занимает 1 ms. Один микробатч последовательно проходит от первой до четвёртой стадии и завершается через 4 ms. Если последовательно подать восемь независимых микробатчей, первая стадия каждую миллисекунду будет передавать результат одного микробатча следующей стадии, а последний микробатч завершится через 11 ms.

Если проследить на рисунке 7-9 за одинаковыми номерами, каждый микробатч должен пройти все четыре стадии; если проследить вдоль одной строки, одна стадия может последовательно обработать восемь микробатчей. Пустые области слева внизу и справа вверху соответствуют простоям при запуске и завершении конвейера.

В общем случае для p равных по длительности стадий, m независимых микробатчей и длительности каждой стадии τ общее время и коэффициент использования каждой стадии соответственно равны

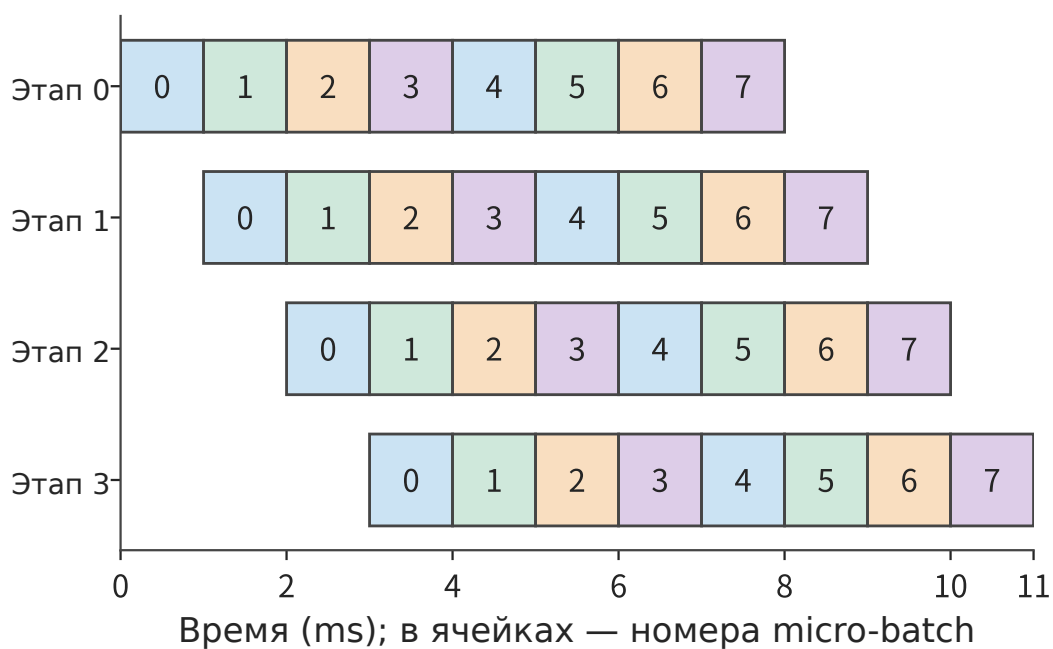


Рисунок 7-9. Обработка восьми микробатчей четырьмя стадиями прямого прохода. Каждая ячейка соответствует 1 ms, числа обозначают номера микробатчей; одинаковые номера перемещаются вправо и вниз, показывая последовательное прохождение микробатчем всех стадий. Каждая стадия выполняет вычисления в течение 8 ms и простаивает 3 ms из общего времени 11 ms. Стадии занимают одинаковое время, а дополнительные затраты на передачу на рисунке не учитываются.

$$T_{\text{pipe}} = (m + p - 1)\tau, \quad U_{\text{pipe}} = \frac{m}{m + p - 1}.$$

Для четырёх стадий и восьми микробатчей коэффициент использования составляет около 73%. Чтобы достичь 90%, из $m/(m + 3) \geq 0.9$ следует, что необходимо не менее 27 независимых микробатчей. Для непрерывной работы всех стадий конвейера требуется достаточно много независимых микробатчей. При decode с малым числом одновременных запросов следующий токен одного запроса зависит от результата предыдущего токена, поэтому добавление серверов не создаёт новых независимых микробатчей; их количество в конвейере увеличивается только при росте числа одновременных запросов.

Таким образом, эти два вида параллелизма порождают разные виды ожидания: при тензорном параллелизме необходимо часто объединять локальные результаты, а при конвейерном параллелизме часть ускорителей простаивает во время запуска и завершения. Тензоры при prefill имеют большой размер, поэтому в первую очередь следует сокращать объём данных, передаваемых между серверами; при decode с малым числом одновременных запросов токены генерируются по одному, поэтому в первую очередь следует сокращать количество запусков обмена данными на каждом слое. Число одновременных запросов, в свою очередь, определяет коэффициент использования конвейера.

7.2.3 Коммуникационная нагрузка при экспертном параллелизме

Коммуникация при тензорном и конвейерном параллелизме определяется способом разбиения; в МоЕ появляется ещё одна переменная: каждый токен может выбирать разных экспертов, поэтому меняются и получатели данных. При экспертном параллелизме эксперты размещаются на разных ускорителях, а токены направляются к соответствующим экспертам согласно маршрутизации. Возьмём 1024 токена, каждый из которых выбирает восемь экспертов: всего получается 8192 операции dispatch. Размер каждой активации составляет 8 KiB, половина пересекает границу серверов, поэтому на вход удалённым экспертам посредством dispatch передаётся 4096 активаций общим объёмом 32 MiB; combine возвращает результаты такого же размера, передавая ещё 32 MiB.⁸

Пусть число токенов равно n , количество экспертов, выбираемых каждым токеном, — k , размер каждой активации — d bytes, а доля передач через границу — f . Если каждая операция dispatch передаётся отдельно, объём передачи для dispatch равен $nkfd$, а вместе с возвращаемыми посредством combine результатами экспертов — $2nkfd$. Размещение популярных экспертов на сервере отправителя уменьшает f ; изменение типа данных или способа кодирования активаций изменяет d .

Распределение этих 32 MiB между получателями также влияет на время передачи. Если весь объём отправить на одну сетевую карту с пропускной способностью 50 GB/s, приём займёт не менее приблизительно 0,67 ms (рис. 7-10); если равномерно распределить данные между восемью сетевыми картами принимающих серверов, каждая из них получит 4 MiB, а время приёма снизится приблизительно до 0,08 ms (рис. 7-11). В

⁸Пример параллелизма модели, основной текст главы 6. В примере EP этой главы используются 1024 токена, восемь операций dispatch на токен, по 8 KiB на каждую часть и доля пересечения границы, равная одной второй; маршрутизация сконструирована в учебных целях.

Всё на одну сетевую карту



Приём: не менее 0.67 ms

Рис. 7-10. При отправке всех 32 MiB на одну сетевую карту с пропускной способностью 50 GB/s приём занимает не менее приблизительно 0,67 ms.

конфигурации для сравнения два порта сетевой карты совместно используют один слот PCIe с пропускной способностью 32 GB/s, поэтому независимо от распределения между двумя портами приём 32 MiB займёт не менее приблизительно 1,05 ms. Равномерное распределение трафика между сетевыми картами устраняет узкое место одной сетевой карты; если порты совместно используют слот, следующим узким местом становится общий слот.

После распределения получателей между несколькими сетевыми картами необходимо продолжить отслеживание общих интерфейсов, через которые проходит трафик. На рис. 7-12 отдельно показан общий слот из конфигурации для сравнения: независимо от последующего распределения между двумя портами все 32 MiB сначала должны пройти через один и тот же слот.

Сравнение этих трёх схем маршрутов показывает, что распределение получателей уменьшает объём данных на каждой сетевой карте; однако через общую точку входа на последней схеме по-прежнему должны пройти все 32 MiB. При расчёте нерегулярной коммуникации также можно сначала отдельно определить объём трафика в каждой из этих точек.

Обозначим полезную нагрузку, передаваемую с ускорителя i на ускоритель j , через V_{ij} . Сумма элементов одной строки равна объёму данных, отправляемых соответствующим отправителем, а сумма элементов одного столбца — объёму данных, принимаемых соответствующим получателем; сумма элементов, пересекающих определённое сечение, равна объёму передачи через это сечение. Если разделить каждую из этих величин на соответствующую пропускную способность, а затем выбрать максимальное значение, получится применение ресурсной модели из раздела 7.1 к нерегулярному обмену.

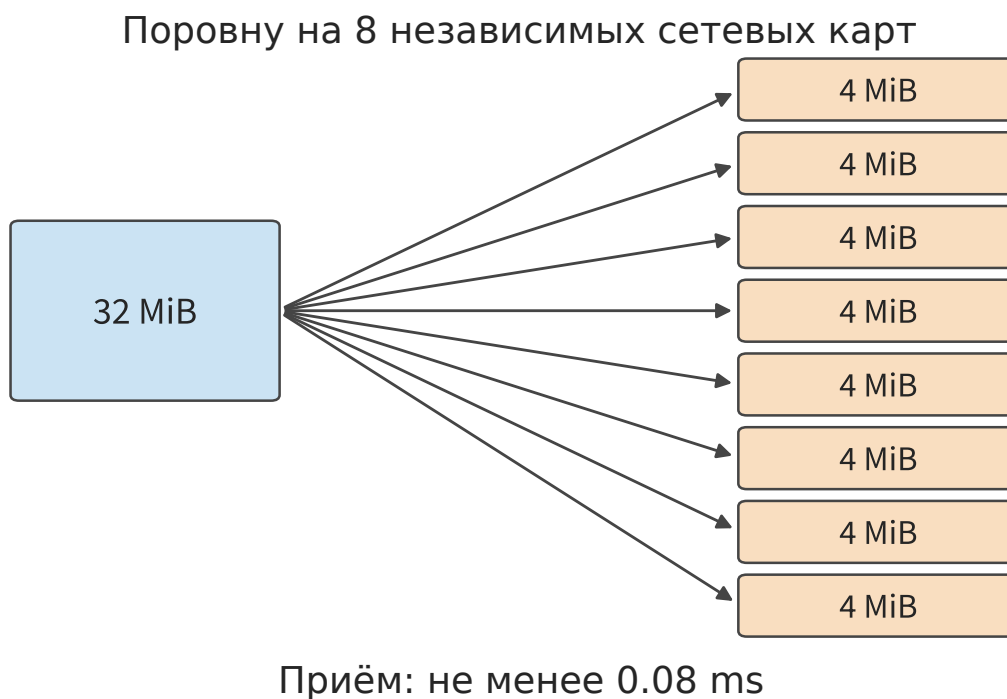


Рис. 7-11. Восемь независимых сетевых карт получают по 4 MiB каждая, сокращая этап приёма приблизительно до 0,08 ms.

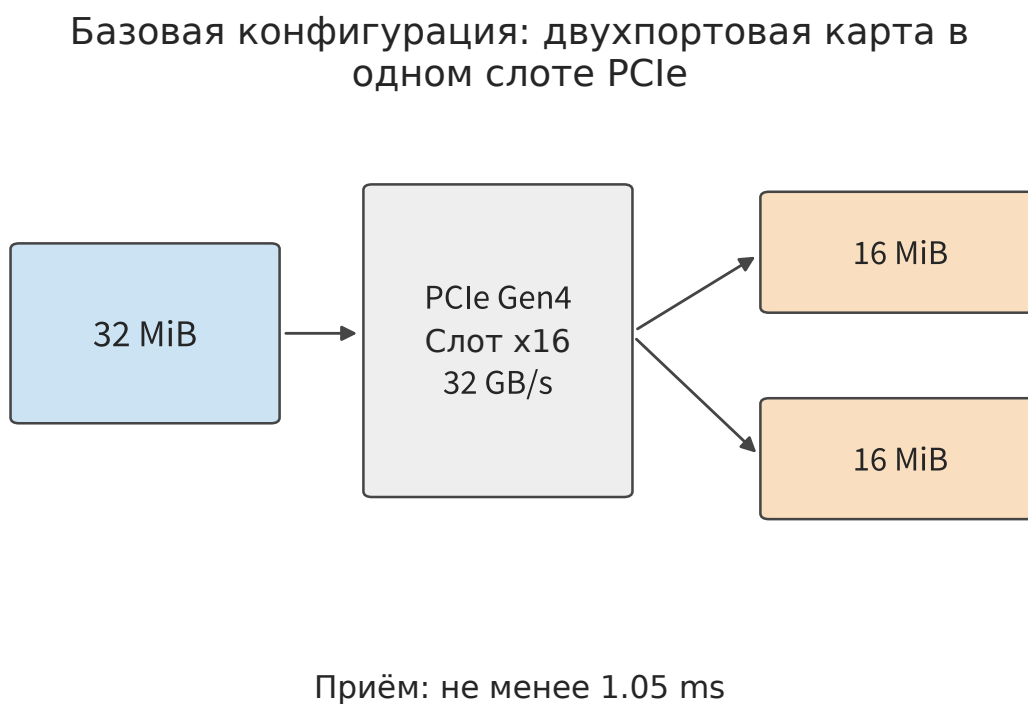


Рис. 7-12. В конфигурации для сравнения все 32 MiB сначала должны пройти через слот PCIe сетевой карты с пропускной способностью 32 GB/s, поэтому этап приёма ограничен приблизительно 1,05 ms. На всех этих схемах учитывается только передача принимаемой полезной нагрузки.

По этой матрице также видно, как соотносятся горячие точки `dispatch` и `combine`. Если данные отправляются отдельно для каждой пары «токен — эксперт», обратно возвращается вектор того же размера, а получатель не выполняет объединение, то матрица возврата представляет собой транспонированную матрицу отправки: получатель, являющийся горячей точкой для `dispatch`, становится отправителем — горячей точкой для `combine`. Одинаковое общее число переданных байтов на двух этапах коммуникации не означает одинакового времени: возвращаемые данные становятся доступны только после завершения вычислений экспертом, а доступная пропускная способность, формат данных и место редукции в двух направлениях также могут различаться. Если `dispatch` использует FP8, а `combine` — BF16, возвращаемая полезная нагрузка будет приблизительно вдвое больше полезной нагрузки `dispatch`; метаданные, такие как `scale` квантования, учитываются отдельно. В разделе 9.4 эти объёмы трафика и моменты готовности результатов вычислений будут объединены в одном примере расчёта.

Таким образом, размещение экспертов преследует две взаимосвязанные цели: сохранить больше операций `dispatch` локальными и распределить удалённые операции `dispatch` между получателями, располагающими свободной пропускной способностью. Репликация популярных экспертов обменивает дополнительную ёмкость на сокращение удалённых передач; входные данные для нескольких экспертов на одном ускорителе можно передать как одну активацию, а после доставки на этот ускоритель распределить между экспертами. Оба способа уменьшают объём передачи по соответствующим маршрутам.

7.2.4 Несколько сетевых адаптеров

Распределение по экспертам (`expert dispatch`) показывает преимущества нескольких параллельных входов и ограничения общего входа. Тот же анализ применим и к отправителю: добавление сетевого адаптера сокращает время передачи, только если он действительно предоставляет дополнительную доступную пропускную способность и на него распределяются данные. Непрерывное кольцо из раздела 7.2.1 — один из примеров: на сервере каждый из восьми сетевых адаптеров подключён к отдельному ускорителю, однако при непрерывном кольце данные для передачи получает только один из них; лишь иерархическая редукция распределяет фрагменты между восемью парами участников, сокращая этап между серверами с 7,6 ms примерно до 0,5 ms. Наличие сетевых адаптеров не означает, что они используются: объём байтов, приходящийся на каждый адаптер, определяется алгоритмом коммуникации.

Контрольная конфигурация демонстрирует другое ограничение. Её двухпортовый сетевой адаптер имеет два порта по 25 GB/s, использующих общий слот PCIe с пропускной способностью 32 GB/s. При использовании одного порта четырнадцать раундов непрерывного кольца с восемью участниками занимают примерно 14,1 ms. Если разделить передаваемые за раз данные на две части и направить их через разные порты, пропускная способность вырастет лишь до разрешённых слотом 32 GB/s, а время сократится примерно до 11,0 ms, но не до приблизительно 7,1 ms, соответствующих суммарным 50 GB/s двух портов. Даже если заменить адаптер на четырёхпортовый ConnectX-7 и задействовать третий порт, слот останется тем же PCIe Gen4 x16, поэтому сократить эту передачу ещё сильнее не получится. После исчерпания пропускной способности общего интерфейса добавление портов больше не повышает пропускную способность.⁹

⁹Непрерывное кольцо в контрольной конфигурации с одним портом, двумя портами и тремя портами:

Возможна и другая ситуация: сетевой адаптер, напрямую подключённый к одной GPU, занят, а адаптер соседней GPU простаивает. Чтобы воспользоваться соседним выходом, сначала необходимо передать данные через NVLink на соседнюю GPU, добавив ещё один этап локальной передачи. Именно такой путь использует система обмена данными с несколькими сетевыми адаптерами FuseLink: с помощью внутрисерверного соединения GPU он переназначает сетевой буфер на свободный сетевой адаптер.¹⁰

Ниже сравниваются два варианта: прямая отправка и ретрансляция через соседнюю GPU. Пропускная способность напрямую подключённого сетевого адаптера составляет 50 GB/s в каждом направлении; каждый из двух доступных соседних адаптеров также обеспечивает 50 GB/s. Пропускная способность NVLink до соседней GPU составляет 450 GB/s в каждом направлении.

Рассчитаем ветвление на рисунке 7-13: прямой путь и путь ретрансляции могут передавать данные одновременно, однако внутри пути ретрансляции данные должны последовательно пройти два этапа. Поэтому прямой путь и путь ретрансляции в сумме обеспечивают $50 + \min(450, 100) = 150$ GB/s. Обозначим суммарную пропускную способность, допускаемую внешней сетью и трактом приёма, через B_D . Тогда фактическая пропускная способность отправки равна

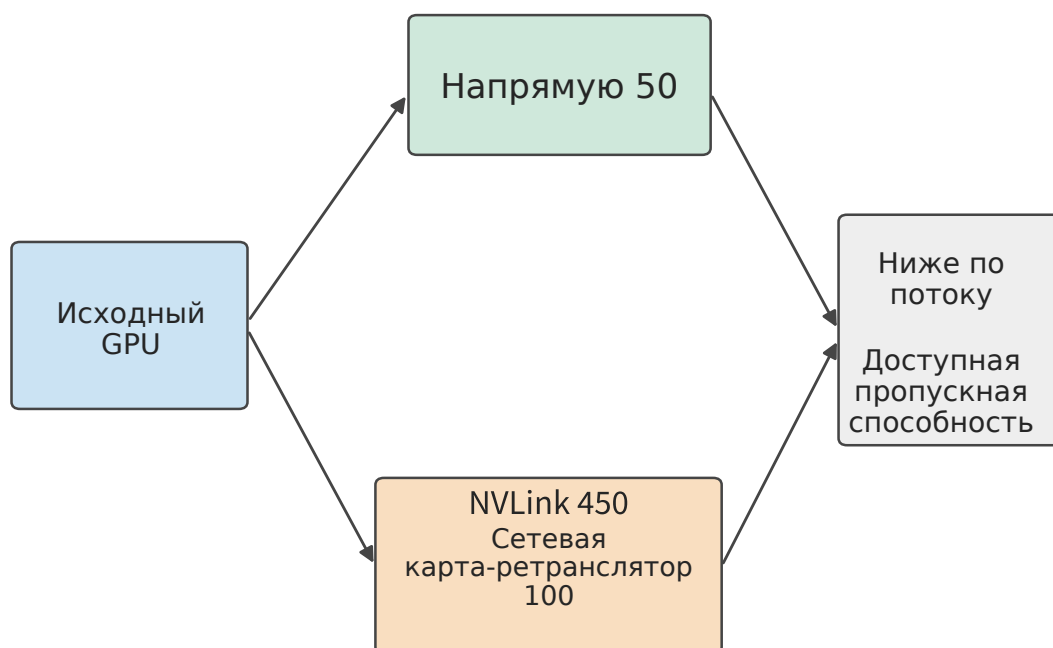
$$B_{\text{eff}} = \min(B_D, 50 + \min(450, 100)) \text{ GB/s},$$

где B_D также измеряется в GB/s. При $B_D \leq 50$ напрямую подключённый сетевой адаптер уже полностью использует пропускную способность последующего тракта, поэтому ретрансляция не даёт выигрыша. При $50 < B_D < 150$ выигрыш от ретрансляции растёт вместе с доступной пропускной способностью последующего тракта. После достижения 150 GB/s новым узким местом становится суммарная пропускная способность двух задействованных соседних адаптеров. Если NVLink одновременно передаёт трафик локальной редукции и доступная для ретрансляции локальная пропускная способность падает ниже 100 GB/s, путь ретрансляции начинает ограничиваться этим локальным участком. Передача полезной нагрузки объёмом 1,125 GiB по прямому пути с пропускной способностью 50 GB/s занимает примерно 24,2 ms; при доступной пропускной способности последующего тракта 100 GB/s время снижается до 12,1 ms, а при 150 GB/s — до 8,1 ms.

Приведённый анализ использования соседних сетевых адаптеров по пропускной способности предполагает, что объём данных каждой передачи достаточно велик. При распределении (dispatch) и объединении (combine) на этапе декодирования МоЕ каждый ускоритель должен отправлять тысячи небольших сообщений. В таком случае необходимо также проверить, сколько запросов в секунду способен инициировать сетевой адаптер, то есть величину $1/\delta$ из раздела 7.3.4. При $\delta \approx 18.6$ ns для реализации надёжного соединения RoCE, рассмотренной в том разделе, сетевой адаптер способен инициировать примерно 54 миллиона запросов в секунду. На линии с пропускной способностью

межсерверные сообщения распределяются полосами по портам, а пропускная способность слота PCIe Gen4 x16 в каждом направлении остаётся равной 32 GB/s независимо от числа портов. Иерархическая редукция из раздела 7.6.2, когда остаётся только одна сетевая карта, приведена в расчёте иерархической редукции с одной сетевой картой.

¹⁰Ren и др., *Enabling Efficient GPU Communication over Multiple NICs with FuseLink*, OSDI 2025; официальная публикация конференции, расчёт пути передачи данных. Для ретрансляционного пути в этой книге используются значения из конфигурации данной главы: сетевая карта — 50 GB/s в каждом направлении, NVLink — 450 GB/s в каждом направлении; реализация плагина и экспериментальная платформа описаны в исходной статье.



Единицы: GB/s; ретрансляция ограничена суммарными 100 задействованных карт

Рисунок 7-13. Путь с использованием соседнего сетевого адаптера. Прямой путь передаёт не более 50 GB/s; два ретранслирующих адаптера обеспечивают в сумме 100 GB/s, причём данные сначала должны пройти по NVLink с пропускной способностью 450 GB/s до соседней GPU. Два пути объединяются далее по потоку, где итоговая пропускная способность дополнительно ограничивается внешней сетью и доступной пропускной способностью получателя. Стрелки показывают направление передачи.

50 GB/s скорость инициирования становится узким местом раньше пропускной способности только для сообщений размером менее приблизительно 0,93 KB. Одно сообщение при распределении MoE содержит скрытый вектор одного токена: при размерности скрытого состояния 7168 его размер составляет около 7 KiB для FP8 и около 14 KiB для BF16. Оба значения значительно превышают эту точку пересечения, поэтому на ускорителе с востребованным экспертом первой по-прежнему исчерпывается пропускная способность; достаточно распределять данные между соседними сетевыми адаптерами по числу байтов. Скорость инициирования исчерпывается раньше пропускной способности только при размере сообщения менее 1 KB, например при поштучной отправке управляющих сообщений. При рассмотрении размещения реплик экспертов в разделе 9.4.2 способность сетевого адаптера обрабатывать пакеты в такой ситуации также учитывается как ресурс, для которого необходимо предусмотреть резерв.¹¹

И для иерархической редукции, и для размещения экспертов, и для ретрансляции через несколько сетевых адаптеров порядок анализа одинаков: изобразить путь, рассчитать объём данных, передаваемый по каждой линии и через каждый интерфейс, а затем найти операцию с наибольшей продолжительностью.

Эксперимент 7-5 · Основное задание: как локальная пропускная способность и общий выход влияют на выбор алгоритма редукции

Полностью повторите расчёты примера 7.2, а затем пересчитайте непрерывное кольцо и иерархическую редукцию для контрольной конфигурации: четыре A100 80GB PCIe на каждом сервере, двухточечные соединения между ускорителями через PCIe Gen4 с пропускной способностью 32 GB/s в каждом направлении и два порта двухпортового ConnectX-7 по 25 GB/s, использующие общий слот PCIe с пропускной способностью 32 GB/s. Для контрольной конфигурации увеличьте локальную пропускную способность с 32 GB/s до 300 GB/s, обеспечиваемых мостом NVLink, предполагая, что такую пропускную способность имеет каждое ребро между четырьмя ускорителями. Объясните, почему выбор алгоритма редукции в контрольной конфигурации изменится на противоположный, а в конфигурации этой главы — нет. Затем увеличьте число используемых портов на каждом сервере контрольной конфигурации с одного до двух и трёх, сохраняя пропускную способность слота равной 32 GB/s. Постройте график зависимости времени редукции от числа портов и укажите, после какого числа портов дальнейшее увеличение уже не сокращает время.

7.2.5 Многорельсовая топология

Иерархическая редукция из раздела 7.2.1 направляет каждую из восьми пар участников по отдельной rail. В эталонной архитектуре DGX SuperPOD вычислительная сеть организована в виде **многорельсовой топологии** (rail-optimized): восемь сетевых адаптеров каждого сервера подключены к восьми разным leaf-коммутаторам, причём i -й сетевой адаптер каждого сервера подключён к i -му leaf-коммутатору. Этот leaf-коммутатор вместе со всеми подключёнными к нему сетевыми адаптерами образует i -ю rail. В пределах одной группы из 32 серверов трафик одной rail достигает адресата за один переход че-

¹¹Точка пересечения скорости обработки пакетов и пропускной способности, около 931 B, выведена из интервала запуска 18,63 ns для надёжного соединения RoCE из раздела 7.3.4 и пропускной способности сетевой карты 50 GB/s из этой главы; заимствование пропускной способности в FuseLink описано в [^fuselink], а скрытая размерность 7168 для MoE dispatch, FP8 dispatch и BF16 combine — в снимке README DeepEP.

рез leaf-коммутатор, а трафик между разными rail проходит через spine-коммутатор.¹² Именно так устроена сеть коммутации «один коммутатор на каждую rail» на рис. 7-1.

На межсерверном этапе иерархической редукции rank i объединяется в пару с rank $i + 8$: rank i использует i -й сетевой адаптер сервера А, а rank $i + 8$ — i -й сетевой адаптер сервера В; оба адаптера подключены к i -му leaf-коммутатору. Поэтому трафик каждой из восьми пар участников остаётся внутри своей rail и проходит только через один leaf-коммутатор; ни один байт не проходит через spine-коммутатор. Каждая rail передаёт по 24 MiB в каждом направлении, две фазы занимают в сумме 0,505 мс. Если сосредоточить эти 192 MiB на одном сетевом адаптере, потребуется 4,03 мс; одновременная работа восьми rail сокращает это время примерно в восемь раз.¹³ В верхней части рис. 7-14 показано такое согласованное формирование пар, а в нижней — рассматриваемое далее несогласованное формирование пар.

Если изменить формирование пар так, чтобы rank i объединялся с rank $i + 9$ (rank 7 — с rank 8), rank i по-прежнему использует i -ю rail, но узел на другом конце использует уже $i + 1$ -ю rail. Байты каждой пары должны сначала попасть в i -й leaf-коммутатор, а затем через spine-коммутатор перейти в $i + 1$ -й leaf-коммутатор. Это относится ко всем восьми парам без исключения, поэтому все 192 MiB в каждом направлении проходят через spine-уровень. Объём данных, отправляемых каждым сетевым адаптером, не изменился, и нижняя граница времени межсерверного этапа по-прежнему составляет 0,505 мс. Изменилось то, что теперь эти байты занимают каналы spine-коммутатора, которые приходится совместно использовать с межрельсовым трафиком других заданий; кроме того, трафик проходит через многопутевое хеширование, рассматриваемое в разделе 7.5.3. Согласованность формирования пары зависит только от совпадения номеров rank внутри соответствующих серверов: если значения $i \bmod 8$ равны, обмен остаётся внутри одной rail; если не равны — проходит через spine-уровень. Чередующееся кольцо (рис. 7-5) в многорельсовой топологии также несёт эти издержки: все шестнадцать его рёбер пересекают границы серверов, причём ребро $i \rightarrow i + 8$ согласовано, а ребро $i + 8 \rightarrow i + 1$ — нет, поэтому половина байтов проходит через spine-уровень.

Применим это правило к суперузлу из 128 карт, описанному в разделе 7.6.4. В каждом из шестнадцати серверов установлено по восемь карт, а группа тензорного параллелизма занимает все восемь карт одного сервера. Карта с координатой тензорного параллелизма t на каждом сервере является t -й картой и использует t -ю rail. После редукции внутри суперузла шестнадцать участников параллелизма по данным с одной координатой обмениваются между суперузлами шардом градиента этой координаты. Шарды всех восьми координат имеют одинаковый размер — на каждой карте $G = 8$ GB, — поэтому каждая из восьми rail передаёт одну восьмую объёма передачи между суперузлами: 14 GB из 112 GB. Ни одна rail не загружена сильнее остальных. Более загруженная rail появ-

¹²Согласованное попарное соединение, попарное соединение со смещением: по восемь сетевых карт на каждом из двух серверов; i -я сетевая карта подключена к leaf-коммутатору i -й rail. Межсерверный этап представляет собой ReduceScatter и AllGather для двух rank; учитываются только последовательная передача сетевыми картами и запуск каждого раунда длительностью 0.833 μ s. Многорельсовая топология, соединение в один переход по одной rail внутри группы из 32 серверов и соединение между rail через spine-уровень взяты из эталонной архитектуры DGX SuperPOD H100, страницы 8 и 14 PDF.

¹³Согласованное попарное соединение, попарное соединение со смещением: по восемь сетевых карт на каждом из двух серверов; i -я сетевая карта подключена к leaf-коммутатору i -й rail. Межсерверный этап представляет собой ReduceScatter и AllGather для двух rank; учитываются только последовательная передача сетевыми картами и запуск каждого раунда длительностью 0.833 μ s. Многорельсовая топология, соединение в один переход по одной rail внутри группы из 32 серверов и соединение между rail через spine-уровень взяты из эталонной архитектуры DGX SuperPOD H100, страницы 8 и 14 PDF.

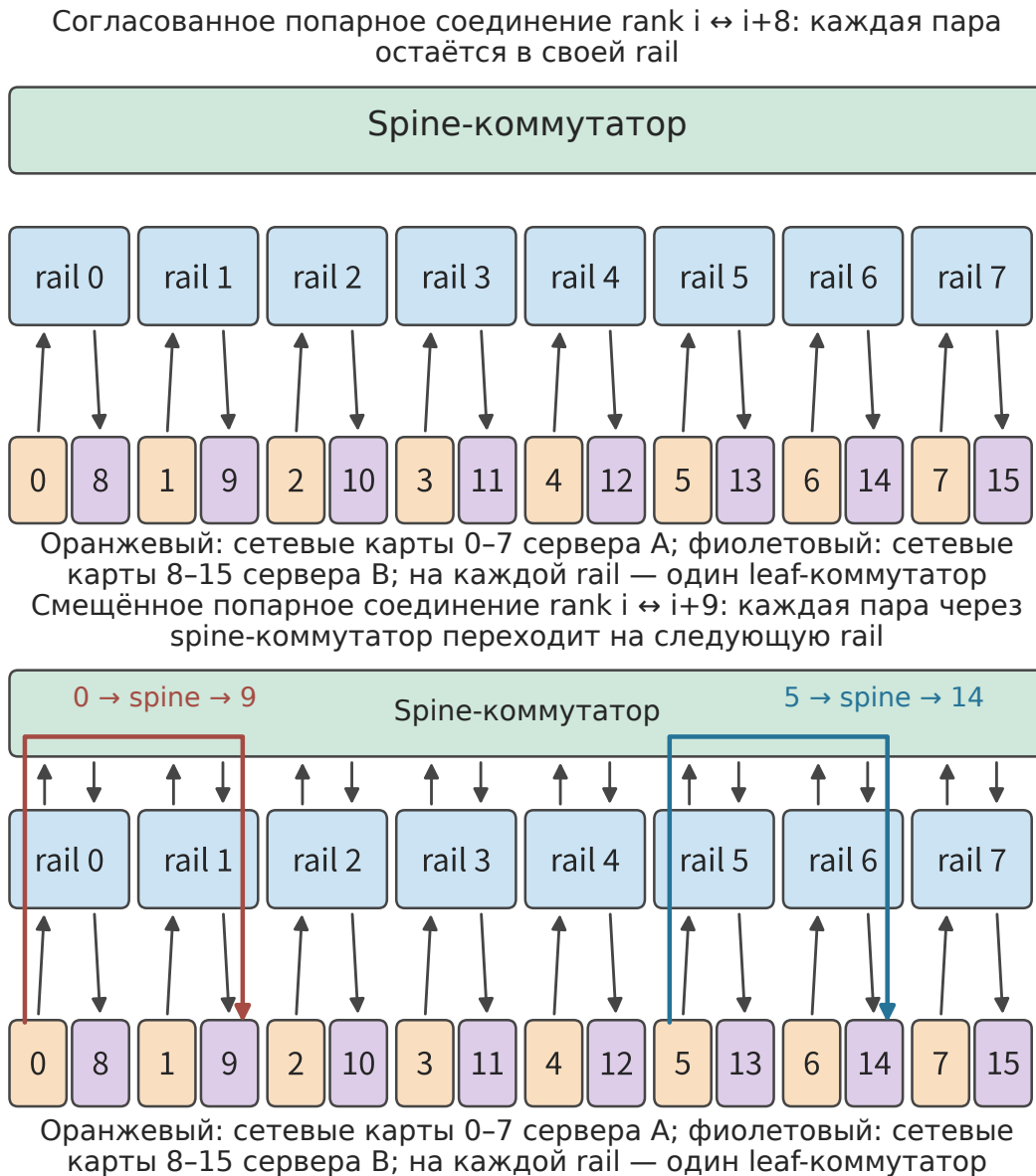


Рис. 7-14. Восемь rail изображены как восемь вертикальных полос; на каждой полосе находятся leaf-коммутатор и сетевые адаптеры с одинаковыми номерами на двух серверах. В верхней части rank i объединяется в пару с rank $i + 8$: байты каждой пары остаются на своей полосе и проходят только через один leaf-коммутатор. В нижней части rank i объединяется в пару с rank $i + 9$: каждая пара должна перейти с i -й rail через spine-коммутатор на $i + 1$ -ю rail, поэтому все 192 MiB в каждом направлении проходят через spine-уровень.

ляется только при разных размерах шардов, например при неравномерном разбиении тензора, либо при отказе сетевого адаптера, когда его трафик через NVLink перенаправляется на соседний адаптер, как описано в разделе 7.2.4.

Эксперимент 7-6 · Продолжение: согласование rail и трафик spine-уровня

На двух серверах измените формирование пар так, чтобы rank i объединялся с rank $((i + 4) \bmod 8) + 8$, и вычислите число байтов, проходящих через spine-уровень. Затем смоделируйте отказ сетевого адаптера 3 на сервере B: трафик rank 11 через NVLink поровну использует сетевые адаптеры 2 и 4. Вычислите объём байтов, передаваемый каждой rail, и время межсерверного этапа. Наконец, в суперузле из 128 карт из раздела 7.6.4 замените конфигурацию на TP16, где одна группа охватывает два сервера, и объясните, шарды каких координат передаёт каждая rail и остаётся ли нагрузка равномерной.

7.3 Пути удалённого доступа и параллелизм

7.3.1 Передача данных и уведомление о завершении

В разделе 7.2 лишь рассчитывалось, по каким каналам проходят байты и сколько времени занимает их передача, но не объяснялось, как именно эти байты передаются получателю. На межсерверном этапе иерархической редукции ускоритель 0 уже получил сегмент после локальной редукции, и теперь его необходимо передать ускорителю 8. Передача данных включает две операции: записать сегмент в место, доступное для чтения ускорителем 8, и сообщить ускорителю 8, что данные готовы. Первая операция изменяет местоположение данных, а вторая сообщает получателю, когда можно начинать вычисления.

При одностороннем чтении и записи из раздела 6.5.5 инициатор напрямую обращается к заранее разрешённой удалённой памяти. Например, ускоритель 0 записывает сегмент в область приёма ускорителя 8: сетевая карта инициатора считывает сегмент из локальной памяти, а сетевая карта целевой стороны записывает его в память ускорителя 8; программа на другой стороне в этом перемещении не участвует. **Двусторонний обмен сообщениями** требует взаимодействия отправителя и получателя: получатель заранее отправляет запрос на приём и предоставляет буфер, а коммуникационная система сопоставляет поступившее сообщение с соответствующим запросом. Крупные сегменты с заранее известным целевым адресом удобно передавать прямым чтением или записью. Уведомления от нескольких отправителей лучше помещать в очередь сообщений, которая различает отдельные уведомления и передаёт их соответствующим потребителям данных, далее называемым потребителями.

Таким образом, крупные данные и уведомления можно обрабатывать отдельно: сначала записать сегмент, а затем отправить короткое сообщение о том, что сегмент доступен. Объединение записи и уведомления в одну операцию отправки также позволяет избежать одной дополнительной операции инициирования. Получив уведомление, потребитель начинает редукцию; порядок записи и уведомления подробно рассматривается в разделе 7.4.

Запись и уведомление завершают передачу данных. Если получатель также должен обработать данные заданным способом, используется **удалённый вызов процедур** (remote procedure call, RPC): другой стороне отправляется запрос на выполнение

указанной функции, а один вызов включает передачу параметров, удалённое выполнение и возврат результата. Способ кодирования параметров напрямую влияет на накладные расходы этого пути: JSON представляет собой текстовый формат обмена данными, а base64 кодирует каждые 3 байта двоичных данных четырьмя текстовыми символами. При помещении 1 MiB двоичных данных в поле JSON в формате base64 объём закодированной полезной нагрузки увеличивается примерно на треть. Переход на двоичные запросы одновременно сокращает число передаваемых байтов и объём работы по кодированию.

В серии прилагаемых к книге измерений RPC между хостами размер запроса уменьшился приблизительно с 1,33 MiB до 1,00 MiB, а медианное время обработки на CPU клиента — примерно с 9,8 ms до 1,1 ms. В 11 из 20 пар полноценных вызовов общее время сократилось; длительность сетевой передачи и ожидания пересылки колебалась сильнее.¹⁴ Эти измерения разделяют затраты времени на два вида: сокращение объёма кодирования экономит локальное время CPU, но для уменьшения длительности всего вызова необходимо также сократить ожидание при сетевой передаче, пересылке и удалённом выполнении.

Эксперимент 7-7 · Дополнительное задание: насколько устранение преобразования при кодировании сокращает время удалённого вызова

Выберите из прилагаемых записей пары вызовов с одинаковой полезной нагрузкой и постройте временную шкалу кодирования на клиенте, отправки, ожидания, декодирования и выполнения на сервере. Объясните, почему время выполнения на сервере уже входит во время ожидания на клиенте. Рассчитайте, на сколько уменьшается число передаваемых байтов после исключения кодирования base64, и проанализируйте, как доля затрат на кодирование влияет на ускорение полного вызова.

7.3.2 Инициирование обмена данными непосредственно ускорителем, который их сформировал

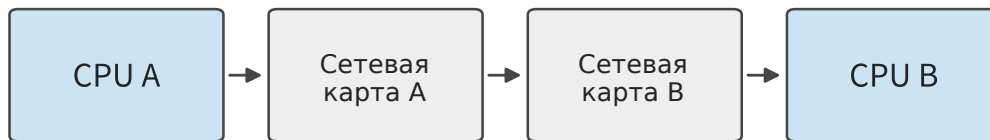
В разделе 7.3.1 чтение и запись, сообщения и RPC рассматривались с точки зрения получателя. На стороне отправителя возникает другой вопрос: кто первым узнаёт, что данные готовы, и кто отвечает за отправку запроса. Шард карты 0 формируется на GPU. Если обмен данными иницирует CPU, то GPU сначала сообщает о готовности, CPU формирует запрос и передаёт его сетевой карте, а после завершения операции возвращает статус GPU. На этом пути управления приходится несколько раз передавать управление, и даже если полезная нагрузка никогда не проходит через память CPU, сами эти передачи требуют времени.

Сетевая карта RDMA уже может обращаться к авторизованной удалённой памяти; технология NVIDIA GPUDirect RDMA дополнительно позволяет сетевой карте напрямую обращаться к зарегистрированной и авторизованной памяти GPU, поэтому полезную нагрузку не нужно передавать через память хоста. Когда обмен данными иницирует CPU, он по-прежнему отвечает за отправку запроса и обработку уведомления о завершении; при иницировании обмена ускорителем GPU самостоятельно формирует или

¹⁴Повторный расчёт этапов RPC и парных вызовов, исходные записи 264 вызовов. Изменения этапов CPU и числа байтов запроса, а также изменения парных вызовов подсчитаны отдельно. Измеряемый путь проходит с Мас через SSH-переадресацию к хосту Linux и включает шифрование, переадресацию и сетевые колебания; медианная экономия полного парного вызова составляет около 10,1 ms.

RPC хоста

Инициатор отправляет операцию, получатель использует результат после завершения



Сплошная: данные; пунктир: управляющая команда

Рисунок 7-15. Параметры RPC между хостами отправляются с хоста, на котором находится CPU A, проходят через сетевую карту и сеть, достигают хоста с CPU B, после чего удалённая сторона выполняет запрос. Стрелка обозначает путь данных параметров, а результат возвращается в обратном направлении.

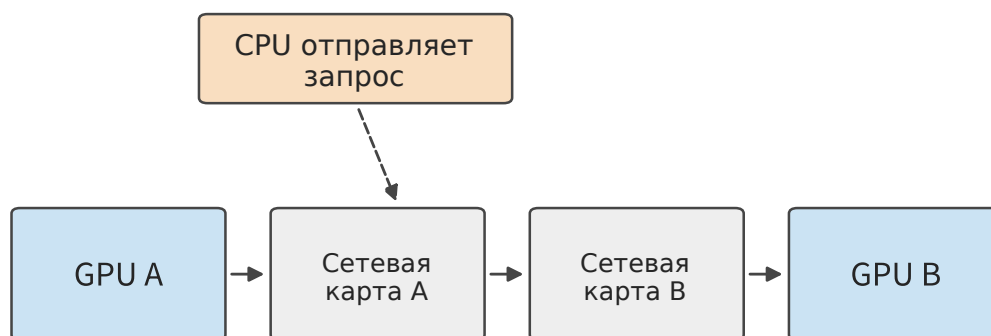
запускает запрос и считывает статус завершения, поэтому данные можно отправить сразу после окончания вычислений. URMA — предоставляемый UB унифицированный интерфейс удалённого доступа к памяти. Обращение GPU к памяти другого устройства через NVLink и инициированный устройством асинхронный доступ через UB представляют собой реализации такого непосредственного взаимодействия для соответствующих путей передачи. На рисунках 7-15–7-18 последовательно показаны RPC между хостами, GPUDirect RDMA с отправкой запроса через CPU, доступ GPU через NVLink и инициированный устройством доступ URMA.

После исключения памяти хоста из пути данных можно также изменить сторону, инициирующую операцию. На рисунке 7-17 GPU самостоятельно инициирует обращение к памяти другого устройства, устраняя необходимость каждый раз передавать запрос через CPU.

На рисунке 7-18 те же отношения инициирования и завершения перенесены на интерфейс асинхронного доступа UB. Если проследить полезную нагрузку по сплошной линии, а управление — по связям отправки запроса и завершения, можно разделить затраты на передачу данных и управление запросами.

Это изменение можно объяснить с помощью формулы времени передачи из раздела 7.2. Если воспользоваться примером из раздела 7.2.2 с каналом 50 GB/s, передача 8 KiB данных занимает около 0,16 μ s; если время запуска сократить с 5 μ s до 2 μ s, общее время уменьшится примерно с 5,2 μ s до 2,2 μ s, то есть более чем вдвое. Передача полезной нагрузки размером 64 MiB сама по себе занимает около 1,3 ms, поэтому экономия тех же 3 μ s оказывается пренебрежимо малой. Следовательно, при передаче небольших объёмов

CPU инициирует GPUDirect RDMA

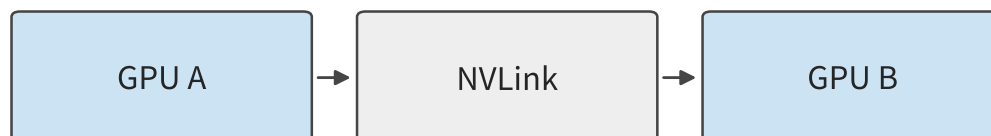


Сплошная: данные; пунктир: управляющая команда

Рисунок 7-16. Данные передаются непосредственно из памяти GPU через сетевую карту в память удалённого GPU; пунктирная линия обозначает отправку запроса через CPU, при этом полезная нагрузка не проходит через память CPU. Отображение памяти и права доступа настраиваются заранее.

Доступ GPU через NVLink

Инициатор отправляет операцию, получатель использует результат после завершения

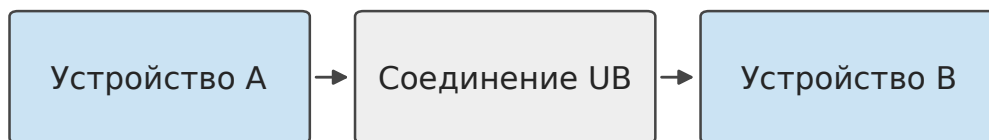


Сплошная: данные; пунктир: управляющая команда

Рисунок 7-17. GPU обращается к памяти другого GPU через NVLink, а инициирующий GPU выполняет авторизованную операцию доступа.

Устройство инициирует асинхронный доступ URMA

Инициатор отправляет операцию, получатель использует результат после завершения



Сплошная: данные; пунктир: управляющая команда

Рисунок 7-18. Устройство отправляет через URMA асинхронные операции чтения и записи, а полезная нагрузка проходит по соединению UB до целевого устройства; после завершения запроса статус и результат проверяются в соответствии с требованиями интерфейса.

мов данных в первую очередь следует сокращать затраты на отправку запросов, а при передаче больших объёмов — увеличивать пропускную способность.

Централизованная организация обмена данными на стороне хоста позволяет предоставить различным программам универсальный механизм отправки запросов и обработки завершения. При выполнении модели производителя и потребителя данных, а также условия их готовности чётко определены, поэтому часто выполняемые операции инициирования и обработки завершения можно перенести на устройство: система заранее настраивает отображение адресов и права доступа, после чего устройство непрерывно инициирует вычисления и обмен данными без промежуточной передачи управления хосту при каждой операции. Выигрыш заключается в сокращении времени запуска, как в приведённом выше примере.

Операционная система отвечает за настройку отображения памяти, прав доступа и выделение ресурсов; устройство — за быстрое выполнение уже авторизованных операций; уведомление о завершении возвращает результат операции среде выполнения. При таком разделении обязанностей поток на устройстве, инициирующий обмен данными, также может ограничивать пропускную способность. Чтобы проанализировать это ограничение, сначала нужно определить, сколько времени занимает ожидание одного запроса и можно ли во время ожидания инициировать другие запросы, а затем вычислить количество запросов, обрабатываемых за единицу времени.

7.3.3 Семантика памяти: Load/Store и Read/Write

Чтобы различать эти два варианта, сначала нужно разделить способы выражения доступа. Удалённый доступ обычно выражается двумя способами: инструкциями процессора

для чтения и записи (Load/Store) и явными асинхронными запросами на чтение и запись (Read/Write). В обоих случаях необходимо регистрировать ещё не завершённые запросы; различие заключается в том, кто управляет этими записями.

Результат Load может непосредственно стать входными данными последующих инструкций. Процессор отслеживает эту зависимость и во время ожидания выполняет другие независимые инструкции. В случае асинхронного запроса целевой адрес, длина и условие завершения помещаются в очередь; программное обеспечение может последовательно отправить несколько задач, а перед использованием результатов дожидаться завершения соответствующих запросов. В первом случае зависимости передаются механизму исполнения инструкций, во втором — событиям и среде выполнения. В этой книге Load/Store называется синхронным доступом, чтобы подчеркнуть, что зависимостями инструкций управляет аппаратное обеспечение, а не то, что у процессора в каждый момент может быть только одна незавершённая операция доступа к памяти. Современные процессоры способны параллельно выполнять несколько независимых Load, а Store могут сначала попадать в буфер.

Критерий сравнения	Load/Store	Асинхронные Read/Write
Инициация и путь	Инструкция процессора, направляемая посредством отображения адресов в доступную память; в UB её может обслуживать контроллер на кристалле	Отправка запроса с адресом, длиной и другими параметрами; продвижение через очередь, doorbell или интерфейс инициации устройства
Зависимости и завершение	Аппаратное обеспечение отслеживает потребителей Load; для удалённой видимости Store отдельно требуются гарантии порядка	Среда выполнения ожидает заданного события завершения, после чего использует результат или освобождает буфер
Типичная область применения	Мелкогранулярное чтение, доступ по указателям, часто используемые данные, для которых важны низкие накладные расходы на отправку	Пакетное перемещение активаций, KV, весов и явные конвейеры
Ограничения параллелизма	Зависимости инструкций, слоты незавершённых обращений к памяти, преобразование адресов и аппаратные ресурсы	Очереди запросов и завершений, зарегистрированная память, возможности отправки программного обеспечения или устройства
Обработка сбоев	Зависит от механизмов платформы для тайм-аутов, исключений и ошибок памяти	Ошибки могут сообщаться отдельно для каждого запроса; всё равно необходимо обрабатывать тайм-ауты, частичную запись и освобождение буферов

Оба интерфейса могут использовать семантику памяти, однако по одному лишь названию интерфейса нельзя судить о когерентности кэшей. Односторонние Read/Write не требуют, чтобы удалённое приложение для каждого запроса отправляло соответствующий запрос приёма (Receive), но всё равно требуют предварительного разрешения и гарантии действительности памяти на всё время операции; Load/Store также не гарантируют автоматическую когерентность общих кэшей между узлами. Реализованный в OpenURMA путь Load/Store не поддерживает когерентность кэшей между узлами: зави-

симости между записью, публикацией и чтением должно организовывать само приложение. Исключение обменов через PCIe достигается благодаря изменению расположения контроллера и пути запроса; простое переименование программного интерфейса в Load не даст тех же преимуществ.¹⁵

Разложение задержки одного удалённого чтения. Разобьём критический путь одного удалённого чтения 64 В на несколько этапов и зададим задержку каждого из них; сумма задержек всех этапов даст полное время чтения. Здесь сравниваются три пути: асинхронное чтение через периферийную сетевую карту PCIe, асинхронное чтение через контроллер шины на кристалле (интерфейс URMA в UB) и инструкция Load через контроллер шины на кристалле. Заданы следующие условия: одно прохождение по шине на кристалле занимает 30 ns; одна запись doorbell через PCIe — 150 ns, одно чтение PCIe DMA — 500 ns, запись — 250 ns; прохождение линии в одном направлении — 100 ns; попадание в строку памяти на целевой стороне (нужная строка DRAM уже открыта) — 30 ns; задержка конвейера сетевой карты вычисляется по числу тактов при частоте 322 MHz: 9 тактов для периферийной сетевой карты, 25 тактов для асинхронного пути UB и 8 тактов для пути Load; отправка через интерфейсную библиотеку, построение дескриптора запроса и опрос записи о завершении относятся к программным накладным расходам. В следующей таблице этапы перечислены в порядке прохождения критического пути; пустая ячейка означает, что на данном пути этого этапа нет.

Этап	Категория	Периферийная сетевая карта PCIe	Асинхронное чтение UB	UB Load
Отправка через интерфейсную библиотеку	Программная отправка	50	50	
Построение де- скриптора запро- са	Программная отправка	30	30	
Запись doorbell	Прохождение PCIe	150		
Чтение дескрип- тора сетевой кар- той через DMA	Прохождение PCIe	500		
Отправка по шине на кристал- ле	Шина на кристал- ле		30	30
Передающий конвейер сете- вой карты	Конвейер сете- вой карты	28	78	25
Линия в прямом направлении	Линия	100	100	100
Приёмный кон- вейер сетевой карты на целе- вой стороне	Конвейер сете- вой карты	28	78	25

¹⁵Bojie Li, «Размышления, лежащие в основе Unified Bus», разделы Jetty, порядка транзакций и Load/Store; статья OpenURMA, редакция от 2026-06-02 (arXiv:2605.28717), §3 — проектирование, §7–§9 — состояние и задержка, §10 — порядок, §12 — передача, §13 — сводка результатов. Источники и область применимости данной интеграции.

Этап	Категория	Периферийная сетевая карта PCIe	Асинхронное чтение UB	UB Load
Чтение памяти хоста сетевой картой через DMA на целевой стороне	Прохождение PCIe	500		
Доступ к памяти по шине на кристалле на целевой стороне	Шина на кристалле		30	30
Попадание в строку памяти на целевой стороне	Память и завершение	30	30	30
Отправка ответа сетевой картой на целевой стороне	Конвейер сетевой карты	28	78	25
Линия в обратном направлении	Линия	100	100	100
Получение ответа сетевой картой	Конвейер сетевой карты	28	78	25
Запись полезной нагрузки ответа через DMA	Прохождение PCIe	250		
Запись сведений о завершении через DMA	Прохождение PCIe	250		
Завершение по шине на кристалле	Шина на кристалле		30	30
Опрос очереди завершений	Память и завершение	70	5	
Опрос через интерфейсную библиотеку	Память и завершение	30	30	
Сериализация выделения порядкового номера	Память и завершение	50		
Итого (расчёт)		2222	746	419
Измерено в симуляции		2236	757	500

Время конвейера сетевой карты в таблице округлено, а итоговые значения получены суммированием до округления. Суммарное время пути через периферийную сетевую карту PCIe составляет 2222 ns, из которых 1650 ns приходится на пять прохождений PCIe; в асинхронном пути UB этапов PCIe нет, а его суммарное время составляет 746 ns; путь Load исключает также программную отправку и опрос, поэтому его суммарное время составляет 419 ns. На рис. 7-19 три пути представлены в виде составных полос по категориям этапов. В реализации OpenURMA автора при потактовой симуляции двух узлов

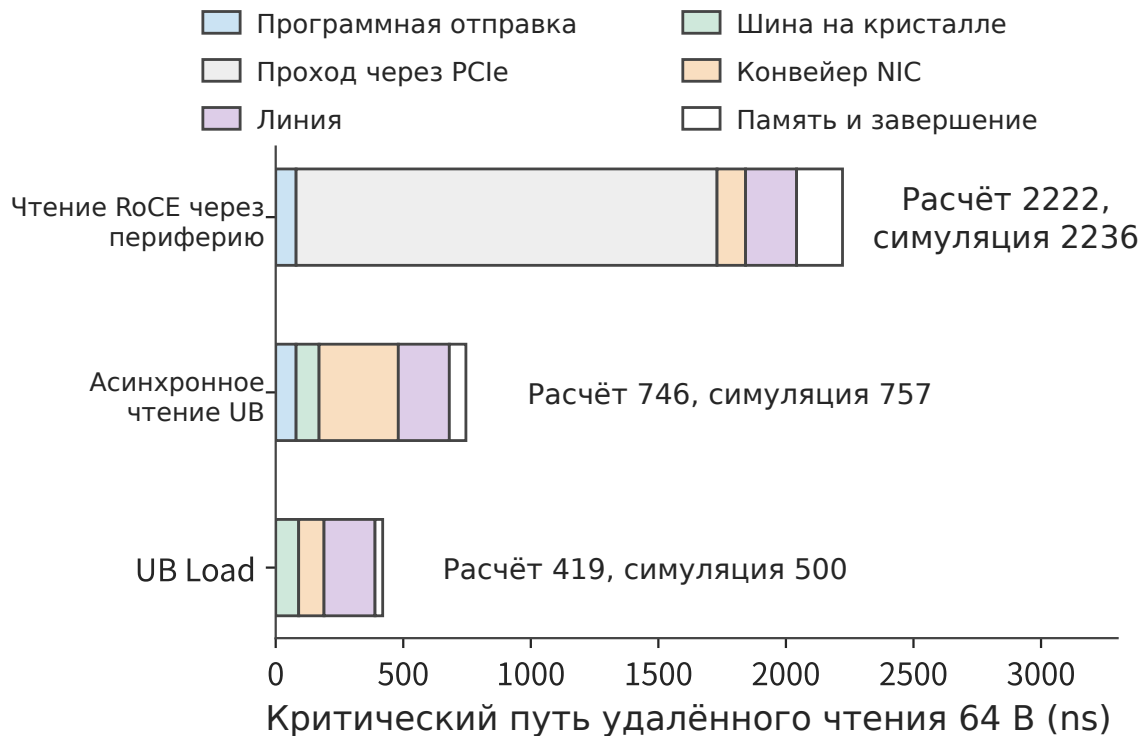


Рис. 7-19. Критические пути трёх вариантов представлены составными полосами по категориям этапов. На пути через периферийную сетевую карту PCIe светло-серый участок соответствует пяти прохождениям PCIe; у двух путей через шину на кристалле этого участка нет, а путь Load дополнительно исключает программную от отправку и опрос завершения. Справа от полос указаны расчётные значения и результаты симуляции.

в тех же условиях были измерены значения 2236, 757 и 500 ns, которые отличаются от расчётных соответственно на 14, 11 и 81 ns. Разница обусловлена постоянными накладными расходами симулятора на передачу данных между модулями; у пути Load меньше всего этапов и самое короткое общее время, поэтому доля этих накладных расходов в нём максимальна.¹⁶

Дополнительные этапы пути через периферийную сетевую карту PCIe — не девять отдельных неэффективных участков, каждый из которых можно оптимизировать, а следствие трёх структурных причин. Во-первых, запрос сначала необходимо записать как дескриптор в память хоста, а затем сетевая карта должна его прочитать, поэтому возникают два этапа: отправка через интерфейсную библиотеку и построение дескриптора. Во-вторых, сетевая карта и процессор имеют собственные адресные пространства, поэтому возникают три этапа: запись doorbell, DMA дескриптора и DMA на целевой стороне. В-третьих, после завершения операции необходимо уведомить процессор через границу адресных пространств, поэтому возникают DMA ответа, DMA записи о завершении и два опроса. После подключения контроллера к шине на кристалле этапы второй категории объединяются в одно прохождение по этой шине. После перехода на инструкцию Load собственный механизм процессора для отслеживания зависимостей заменяет механизм

¹⁶Параметры и результаты для задержки этапов, частоты запросов и роста состояния приведены в расчёте системы межсоединений UB; расчёт можно повторить командой `python3 calculations/calc.py ub-fabric --format md`. Задержки каждого этапа взяты из таблицы 7 статьи OpenURMA, размеры записей — из таблицы 3, а контрольные значения моделирования — из §8.1 и §8.3. В выкладках все этапы складываются последовательно без учёта перекрытия параллельных запросов.

уведомления о завершении, поэтому этапы первой и третьей категорий исчезают.

Существует и четвёртый путь: сетевая карта остаётся периферийным устройством PCIe, но дескриптор запроса создаётся и используется процессором внутри неё, то есть применяется третье место инициации из раздела 6.4.4. Согласно расчёту по той же таблице, этот путь исключает два прохождения — запись *doorbell* и чтение дескриптора сетевой картой через DMA, — что в сумме даёт 650 ns. Отправка через программный интерфейс и построение дескриптора запроса, занимающие 80 ns, выполняются процессором сетевой карты; для них используются те же значения. Запись полезной нагрузки через DMA, запись сведений о завершении через DMA и чтение через DMA на целевой стороне сохраняются. Итог составляет около 1572 ns — между 2222 ns для периферийного варианта и 746 ns для пути через шину на кристалле. Исключаются прохождения на пути управления, но передачу данных и уведомление о завершении всё равно приходится выполнять между двумя адресными пространствами. Этот путь не симулировался: значение получено только расчётом по данным таблицы.

При изменении задержки линии меняется и соотношение между тремя путями. Общее время каждого пути равно фиксированным накладным расходам плюс удвоенная задержка линии в одном направлении L : для периферийной сетевой карты PCIe это 2022 ns плюс $2L$, для асинхронного пути UB — 546 ns плюс $2L$, для пути Load — 219 ns плюс $2L$. На рис. 7-20 показан случай изменения L от 50 ns до 500 ns: три прямые имеют одинаковый наклон, но разные точки пересечения с осью; абсолютная разница всегда составляет около 1.8 μ s, тогда как относительная разница сокращается с 5.3 раза при $L = 100$ ns до 2.5 раза при $L = 500$ ns. Чем короче линия, тем больше относительная выгода от расположения контроллера, поэтому наиболее заметна она внутри суперузла; на длинных линиях между дата-центрами основную часть общего времени составляет задержка самой линии. Эти две составляющие имеют разную природу. Величина $2L$ определяется скоростью распространения сигнала, её невозможно исключить ни на одном пути, и она представляет собой физический нижний предел времени одного удалённого чтения. Часть фиксированных накладных расходов, превышающая фиксированные накладные расходы пути Load, — это цена уровня абстракции, причём только пять прохождений PCIe составляют 1650 ns. Разница между расчётными значениями и результатами симуляции, равная 14, 11 и 81 ns, относится к первому типу расхождений, описанному в разделе 1.3.4: модель не учитывает постоянные накладные расходы на передачу между модулями симулятора. После добавления этой величины расхождение исчезает, и искать дополнительные накладные расходы реализации не требуется.

Далее при одинаковой задержке доступа сравним параллелизм и зависимости, а затем рассмотрим повторное использование данных. Первое применимо к обоим типам интерфейсов, второе относится к выбору размещения: «обращаться к удалённым данным по требованию или заранее переносить их в локальную память». Прямое удалённое чтение не равнозначно Load, а предварительный перенос не связан с каким-либо одним конкретным интерфейсом.

Рассмотрим восемь независимых удалённых чтений. Аппаратное обеспечение способно одновременно обрабатывать восемь операций, каждая из которых при прохождении по указанному в таблице пути через периферийную сетевую карту PCIe занимает около 2.2 μ s с учётом накладных расходов на отправку. Последовательное выполнение по схеме «отправить, дождаться, использовать» займёт около 17.8 μ s; если отправить все восемь операций одновременно, их результаты поступят вместе примерно через 2.2 μ s. Если первое чтение возвращает адрес для второго, а второе — адрес для третьего, то

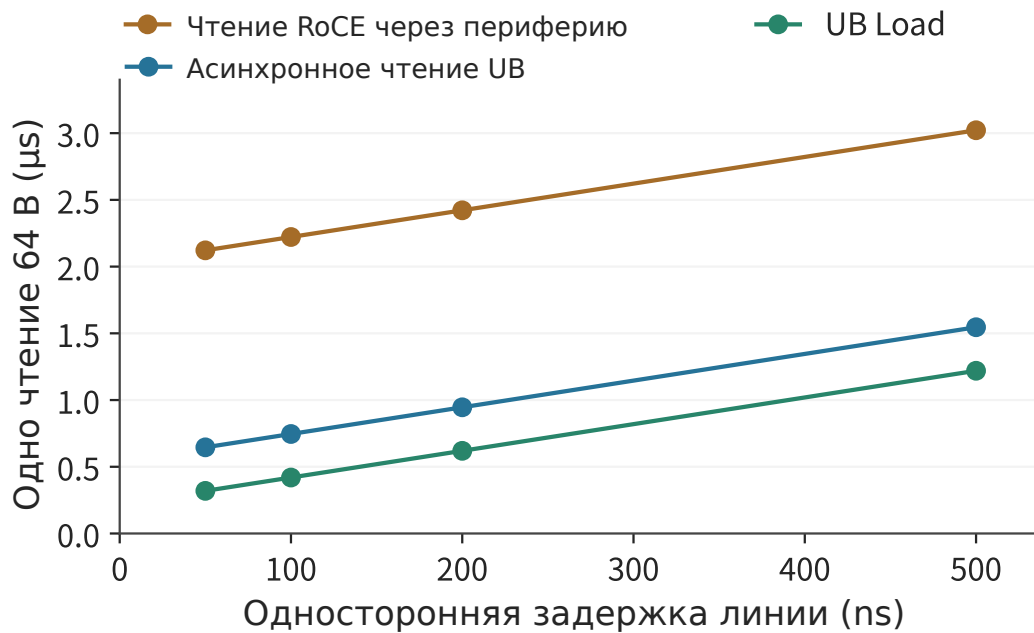


Рис. 7-20. Зависимость общего времени одного чтения 64 В от задержки линии в одном направлении. Наклон всех трёх прямых равен 2, а точки пересечения с осью определяются соответствующими фиксированными накладными расходами; разница между путями обусловлена фиксированными накладными расходами и не зависит от задержки линии.

каждое следующее чтение можно отправить лишь после завершения предыдущего. Число чтений, которые можно инициировать одновременно, зависит от числа запросов с известными адресами и без взаимных зависимостей.

Параллельное чтение использует независимость между разными запросами. Если несколько запросов обращаются к одним и тем же данным, можно воспользоваться другим отношением: при первом обращении перенести данные в локальную память, а при последующих чтениях многократно использовать эту копию. Предположим, что потребитель полностью считывает неизменяемый снимок KV объёмом 144 MiB и в локальной памяти достаточно места для его размещения. Удалённое чтение и перенос выполняются через описанную в этой главе сетевую карту с пропускной способностью 50 GB/s в каждом направлении; для локального чтения используется пропускная способность HBM в H100, равная 3350 GB/s. С учётом запуска и записи на целевой стороне одно удалённое чтение занимает около 3.02 ms, фиксированные затраты на один перенос — около 3.07 ms, а каждое последующее локальное чтение — около 0.046 ms.¹⁷

Обозначим число повторных использований через r . Приближённое время двух путей равно

¹⁷Удалённое чтение и локальное получение неизменяемого снимка KV. Снимок представляет собой KV всех слоёв Qwen3-8B для 1024 токенов в BF16 и занимает 144 MiB; удалённое чтение и локальное получение рассчитаны для ConnectX-7 с пропускной способностью 50 GB/s в каждом направлении и 391 транзакцией по 256 В в пути, а локальные чтение и запись — для HBM H100 SXM с пропускной способностью 3350 GB/s. Фиксированная стоимость одного локального получения составляет около 3,075 ms, каждого удалённого чтения — около 3,025 ms, локального чтения — около 0,046 ms. В контрпримере с доступом к 10% данных стоимость каждого обращения масштабируется в той же пропорции, но стоимость локального получения по-прежнему рассчитывается для всего снимка.

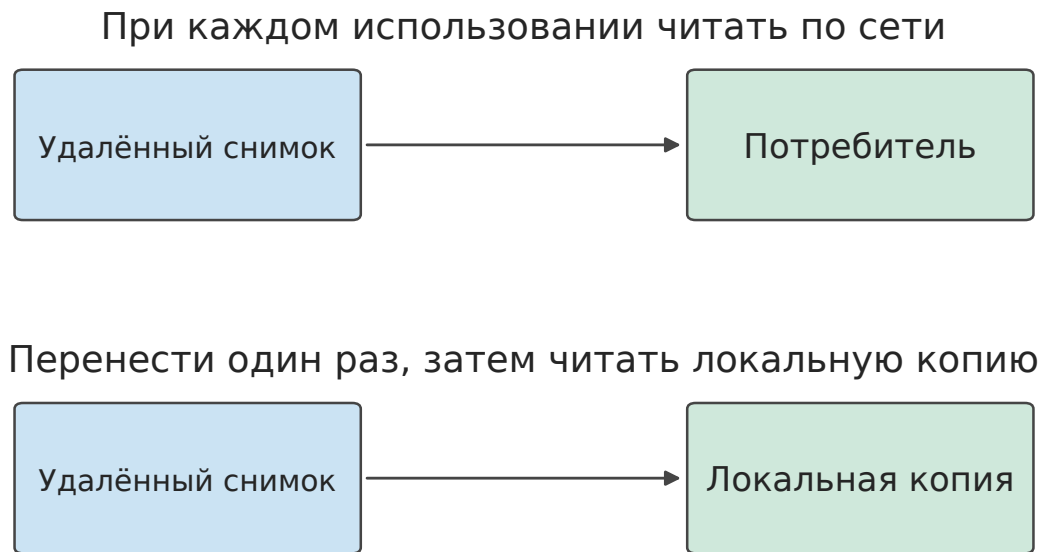


Рис. 7-21. При каждом обращении к удалённой стороне одни и те же данные передаются повторно; в нижней части рисунка данные сначала переносятся в локальную память, а затем многократно используются. Оба пути обрабатывают один и тот же неизменяемый снимок.

$$T_{\text{remote}} = 3.02r \text{ ms}, \quad T_{\text{stage}} = (3.07 + 0.046r) \text{ ms}.$$

Чтение после переноса становится быстрее при $r > 3.07/(3.02 - 0.046) \approx 1.03$, то есть выгода появляется начиная со второго полного чтения. При однократном чтении прямое удалённое чтение немного быстрее (3.02 ms против 3.12 ms); при четырёх чтениях предварительный перенос в локальную память сокращает общее время примерно с 12.1 ms до 3.3 ms. По сети снимок передаётся только один раз, а объём передачи уменьшается с 576 MiB до 144 MiB. На рис. 7-21 показаны два пути использования, а на рис. 7-22 — точка пересечения накопленного времени.

Различие между двумя кривыми обусловлено выбором: «однократно оплатить перенос или каждый раз выполнять чтение через сеть». Теперь изменим объём данных, читаемых за одно обращение. Если один вызов обращается лишь к небольшому числу участков снимка, его можно передать по ссылке, чтобы удалённая функция читала данные по требованию. Предположим, что при каждом обращении читается только 10% снимка, а вариант с переносом всё равно копирует его целиком. Тогда стоимость удалённого чтения по требованию составляет около $0.302r \text{ ms}$, каждое локальное обращение после переноса — около 0.0046 ms , а точка пересечения соответствует примерно 10.3 обращения. Следовательно, предварительный перенос в локальную память начинает сокращать общее время чтения лишь с 11-го обращения (рис. 7-23).

Таким образом, выбор между удалённым чтением по требованию и переносом всего снимка в локальную память следует делать с учётом доли данных, охватываемой каж-

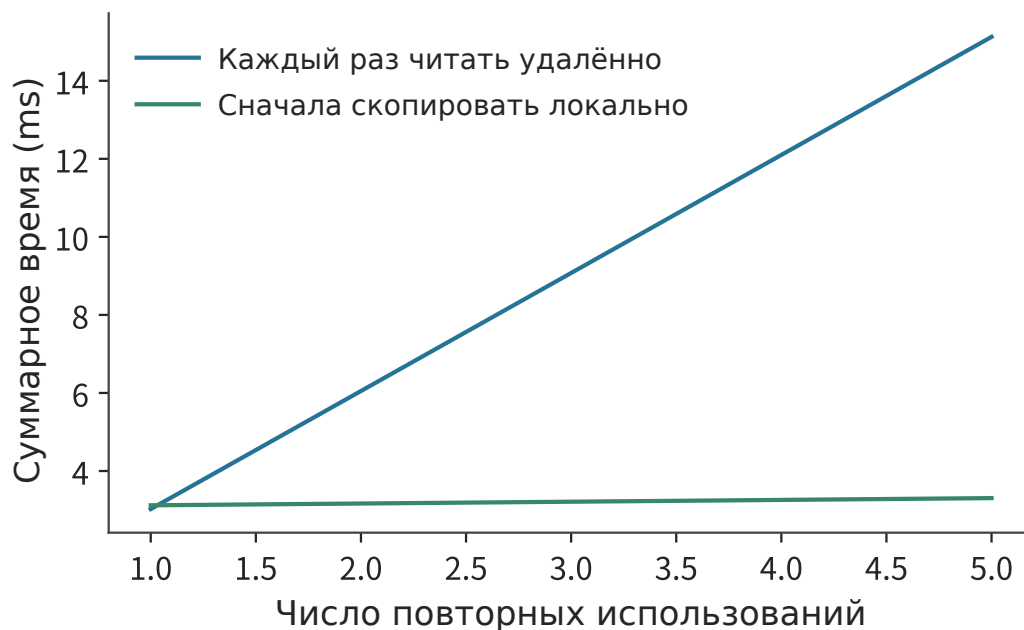


Рис. 7-22. Накопленное время двух методов при каждом полном чтении снимка объемом 144 MiB. Предварительный перенос в локальную память требует однократных фиксированных затрат, но начиная со второго чтения экономит время.

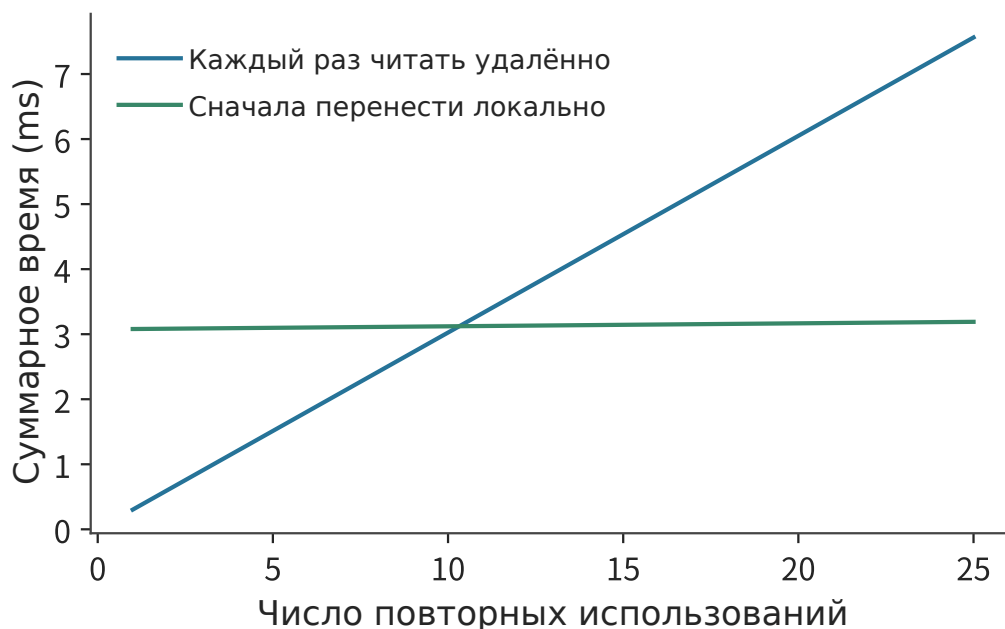


Рис. 7-23. При каждом удалённом и локальном чтении используется только 10% данных, однако при переносе снимок всё равно копируется целиком; поэтому для компенсации фиксированных затрат требуется больше повторных использований.

дым обращением, и числа повторных использований. URPC — это интерфейс удалённого вызова процедур поверх унифицированной системы соединений, который предоставляет удалённый доступ в форме вызова функций и поддерживает такой способ работы; передача по ссылке позволяет вызываемой стороне получать только фактически необходимые данные. В разделе 7.4 будет объяснено, когда следует уведомлять другие устройства об использовании этих объектов и когда освобождать занимаемое ими пространство.¹⁸

7.3.4 Модель параллелизма и пропускной способности

В разделе 7.3.3 повторное использование сокращало количество удалённых чтений; для запросов, которым всё же приходится проходить по сети, необходимо обеспечить постоянное наличие данных для передачи по каналу. В этом разделе рассчитывается, сколько параллельных запросов требуется для канала с пропускной способностью 50 GB/s в каждом направлении на одной сетевой карте из примера этой главы. **Слот запроса** — это запись, в которой хранятся адрес, длина и состояние незавершённого запроса: она выделяется при отправке запроса и освобождается после обработки состояния завершения, а до освобождения не может использоваться новым запросом. Предположим, что при каждом удалённом обращении передаётся 256 В, а от отправки запроса до освобождения его слота проходит 2 μ s. За эти 2 μ s канал может передать 100 000 В, что соответствует объёму данных 390,6 обращения. Чтобы во время ожидания по каналу постоянно передавались данные, необходимо поддерживать не менее 391 транзакции в полёте, то есть уже отправленных, но ещё не полностью обработанных запросов. На рис. 7-24 показан жизненный цикл слота от выделения до освобождения.

В общем случае, если полезная нагрузка одной транзакции равна m , время занятости слота — T , а целевая пропускная способность — B , требуемый параллелизм составляет

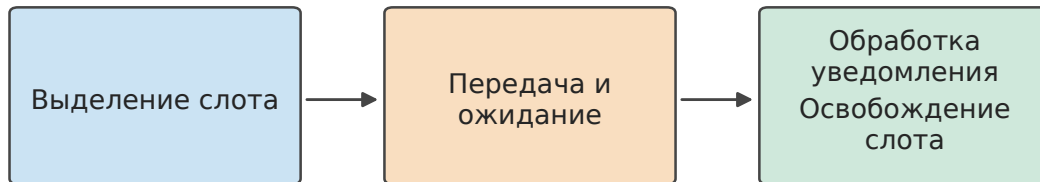
$$N \geq \left\lceil \frac{BT}{m} \right\rceil.$$

Произведение пропускной способности на задержку — это произведение целевой пропускной способности и времени ожидания, выражающее объём данных в полёте, необходимый для непрерывного использования канала во время ожидания. Если дополнительно разделить его на полезную нагрузку одной транзакции, получится количество запросов. Чем быстрее канал, чем дольше ожидание и чем меньше данных возвращает каждый запрос, тем больше незавершённых запросов система должна учитывать одновременно.

Пример 7.3. Почему недостаточное количество слотов удалённого чтения в полёте не позволяет полностью использовать исходящую пропускную способность? Каждая транзакция передаёт 256 В и занимает слот в течение 2 μ s, а пропускная способность канала равна 50 GB/s. Найдите эффективную пропускную способность чтения при использовании 128 активных слотов. Затем предположите, что минимальный интервал запуска двух последовательных транзакций блоком обработки запросов равен 18,6 ns — это значение реализации надёжного соединения RoCE, рассматриваемой далее в этом разделе, — и найдите верхнюю границу эффективной пропускной способности при одновременном ограничении количеством слотов и скоростью запуска.

¹⁸Записи сверки спецификации UB и эталонного проекта операционной системы.

Слот хранит один незавершённый запрос



Занят 2 μ s: от выделения до освобождения

Рис. 7-24. Запрос занимает слот с момента выделения записи, проходит этапы передачи и ожидания, а после обработки состояния завершения записи возвращается. Здесь 2 μ s охватывают весь период занятости слота.

Решение: 128 слотов оборачиваются раз в 2 μ s и обеспечивают полезную нагрузку 128×256 В, поэтому максимальная пропускная способность составляет приблизительно 16,4 GB/s. Если в полёте находится только одна транзакция, она составит 0,128 GB/s. Количество выделенных слотов и количество слотов, фактически используемых одновременно, — разные величины.

Серые интервалы на рис. 7-25 возникают из-за того, что слоты ещё не освободились; однако даже при достаточном количестве слотов запросы могут не успевать отправляться. Если блок обработки запросов запускает одну транзакцию каждые 18,6 ns, он может запускать не более примерно 54 миллионов транзакций в секунду, поэтому транзакции размером 256 В способны обеспечить лишь около 13,7 GB/s — меньше 16,4 GB/s, доступных при 128 слотах. Даже если увеличить количество слотов до 391, блок обработки запросов не успеет отправлять их с требуемой скоростью. Объединив ограничения пропускной способности канала, количества запросов в полёте и скорости запуска, получим

$$B_{\text{eff}} \leq \min \left(B_{\text{path}}, \frac{Nm}{T}, \frac{m}{\delta} \right),$$

где T — время, в течение которого запрос занимает слот, а δ — минимальный интервал запуска двух последовательных запросов. Первая величина определяет, через какое время слот можно использовать повторно, а вторая — сколько запросов можно запускать в секунду. На рис. 7-26 сравнивается эффективная пропускная способность при учёте только слотов и при дополнительном учёте ограничения скорости запуска.¹⁹

¹⁹Окно удалённого чтения, последовательное ожидание: путь 50 GB/s, 256 В на транзакцию, время занято-

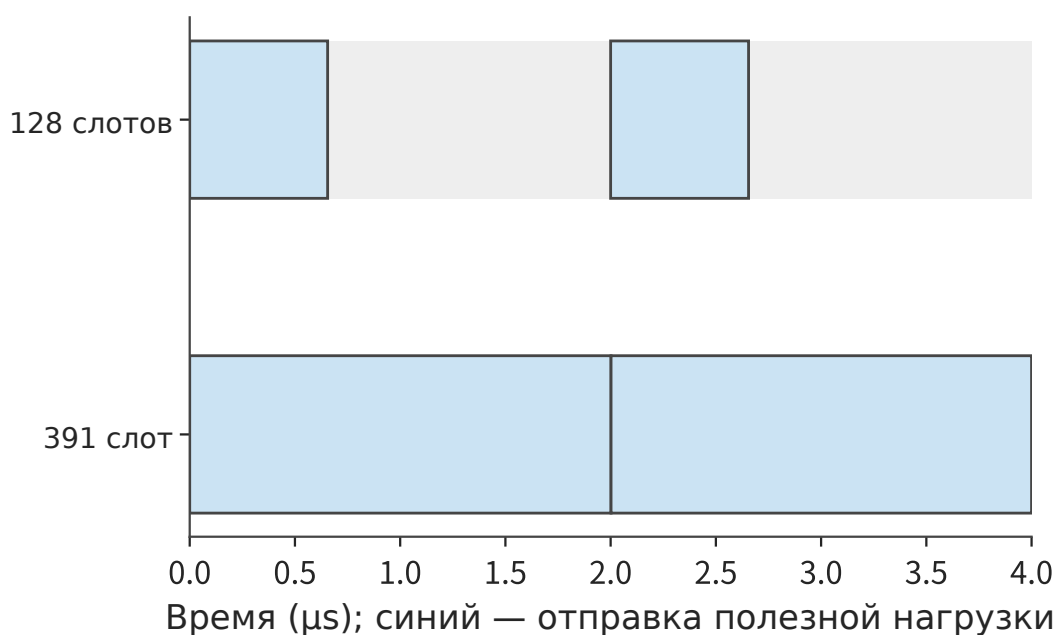


Рис. 7-25. Все 128 слотов расходуется приблизительно за 0,66 μs , после чего для продолжения отправки приходится ждать освобождения первого слота через 2 μs . Для покрытия времени ожидания достаточно 391 слота. Синим показана передача полезной нагрузки, серым — простой канала.

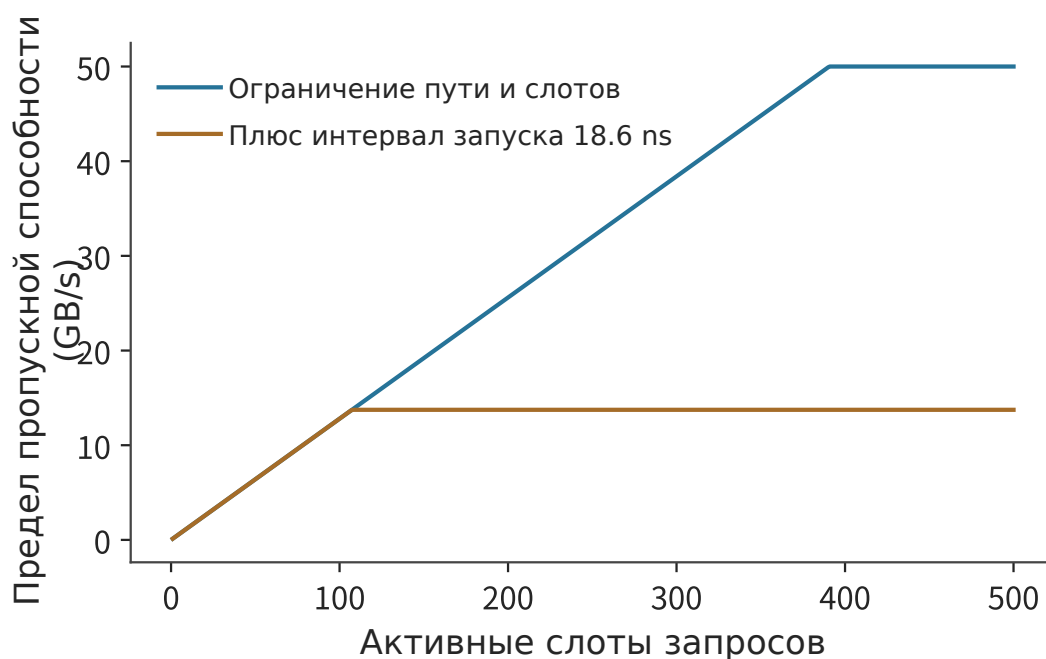


Рис. 7-26. Синяя линия учитывает только ограничения канала и слотов, а оранжевая дополнительно учитывает ограничение на запуск одного запроса каждые 18,6 ns. Увеличение количества слотов поднимает синюю линию, тогда как оранжевая по-прежнему ограничена скоростью отправки около 13,7 GB/s.

Обсуждение. Какой параллелизм удалённого чтения необходим для достижения 50 GB/s? При сохранении размера транзакции 256 В интервал запуска требуется сократить до $256 / (50 \times 10^9) = 5.12 \text{ ns}$, что даже меньше 6,2 ns в описанной ниже реализации UB; кроме того, необходимо поддерживать не менее 391 операции в полёте. Другой вариант — объединять соседние обращения: если передавать по 4 KiB за одну транзакцию, для достижения 50 GB/s достаточно интервала запуска не более 82 ns, тогда как интервал 18,6 ns соответствует приблизительно 220 GB/s, что намного превышает возможности канала; при времени ожидания 2 μs для поддержки 50 GB/s потребуется всего 25 слотов. В первом случае повышается частота транзакций, а во втором каждый запрос передаёт больше данных, снижая накладные расходы на отправку в расчёте на байт.

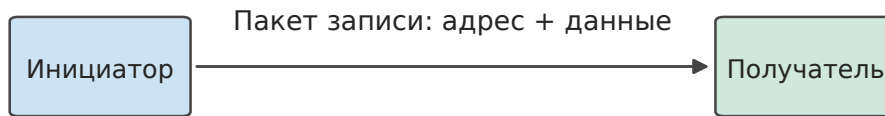
Один из конкретных источников интервала запуска δ виден в реализации OpenURMA: самая медленная ступень конвейера отправки принимает один запрос каждые 2 такта, поэтому при частоте 322 MHz $\delta \approx 6.2 \text{ ns}$, а максимальная частота составляет около 161 миллиона запросов в секунду. В реализации надёжного соединения RoCE, созданной с помощью того же набора инструментов для сравнения, распределение порядковых номеров в каждом соединении имеет последовательную зависимость, каждая транзакция занимает 6 тактов, $\delta \approx 18.6 \text{ ns}$, а максимальная частота составляет около 54 миллионов транзакций в секунду. При моделировании непрерывной пачки из 256 транзакций измеренная установившаяся скорость составила соответственно 150 и 54 транзакции на микросекунду: для RoCE результат совпадает с расчётными 53,7 транзакции, а результат UB приблизительно на 7% ниже расчётных 161 транзакции; при увеличении пачки до 1000 транзакций скорость UB возрастает примерно до 159 транзакций на микросекунду, отличаясь от расчётного значения приблизительно на 1%. Подстановка в m/δ даёт для транзакций размером 64 В соответственно 10,3 GB/s и 3,4 GB/s, а для транзакций размером 4 KiB — 660 GB/s и 220 GB/s; последние два значения намного превышают 50 GB/s в каждом направлении для сетевой карты 400 Gbit/s из этой главы. Иными словами, при малой полезной нагрузке узким местом служит скорость запросов, а при большой — линия передачи.²⁰

Почему чтение и запись подчиняются разным ограничениям. В PCIe величины N и δ в формуле имеют конкретное происхождение. PCIe — это шина с коммутацией пакетов, и каждое чтение или запись DMA представляет собой пакет уровня транзакций (TLP). Запись является posted-транзакцией: пакет с адресом и данными отправляется и считается завершённым сразу после отправки, не ожидая ответа. Чтение является non-posted-транзакцией: пакет запроса отправляется с тегом, а данные возвращаются в одном или нескольких пакетах завершения и сопоставляются с исходным запросом по этому тегу. Количество одновременно находящихся в полёте запросов чтения у инициатора ограничивается двумя факторами: кредитами, заранее предоставленными получателем для каждого класса транзакций, и количеством тегов, которые может выделить инициатор. На рис. 7-27 показаны оба вида транзакций.

сти слота 2 μs . Интервалы запуска 18,6 ns и 6,2 ns взяты из показателя частоты запросов в расчёте системы межсоединений UB: в наборе инструментов OpenURMA надёжное соединение RoCE требует 6 тактов на запрос, а UB — 2 такта при частоте 322 MHz. Ожидание запроса и интервал запуска обслуживания в формулах определены раздельно.

²⁰Параметры и результаты для задержки этапов, частоты запросов и роста состояния приведены в расчёте системы межсоединений UB; расчёт можно повторить командой `python3 calculations/calc.py ub-fabric --format md`. Задержки каждого этапа взяты из таблицы 7 статьи OpenURMA, размеры записей — из таблицы 3, а контрольные значения моделирования — из §8.1 и §8.3. В выкладках все этапы складываются последовательно без учёта перекрытия параллельных запросов.

Запись (posted): завершена
после отправки



Чтение (non-posted): запросы в полёте
ограничены тегами и кредитами

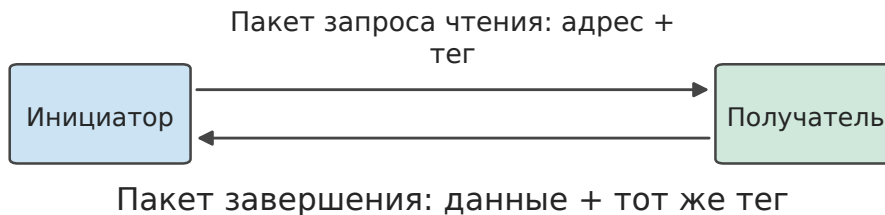


Рис. 7-27. Запись является posted-транзакцией и завершается сразу после отправки одного пакета; при чтении сначала отправляется пакет запроса с тегом, а затем ожидается пакет завершения с тем же тегом, доставляющий данные. Количество операций чтения в полёте не может превышать количество тегов и кредитов.

Ограничения по кредитам и тегам можно наблюдать на реальной платформе. Автор измерил оба ограничения в KV-Direct. Используемая программируемая сетевая карта обращалась к памяти хоста через PCIe Gen3 x8 с теоретической пропускной способностью 7,87 GB/s; к каждому чтению или записи DMA размером 64 В добавлялись 26 В заголовка пакета и заполнения, поэтому верхняя граница с учётом пакетных накладных расходов составляла 5,6 GB/s, или 87 миллионов операций в секунду. Время кругового обхода случайного DMA-чтения составляло около 1050 ns, поэтому для полного использования канала требовалось 92 одновременно находящихся в полёте чтения; однако хост предоставлял для DMA-чтения только 84 кредита, а DMA-движок FPGA поддерживал лишь 64 тега. Таким образом, в полёте могло находиться не более 64 чтений, что при делении на время кругового обхода 1050 ns давало верхнюю границу около 61 миллиона операций в секунду; измеренная частота случайных чтений по 64 В составила 60 миллионов операций в секунду, что практически совпадает с этой границей. Запись не требует ответа и не занимает тег, поэтому измеренный результат близок к верхней границе, определяемой пакетными накладными расходами. На рис. 7-28 эти границы показаны вместе: чтение ограничивается количеством операций в полёте, а запись — скоростью обработки пакетов; это пример раздельного действия членов Nm/T и m/δ в формуле. Сетевая карта самостоятельно и конвейерно выполняет вычисление адреса, обращение к памяти и обработку результата именно для того, чтобы во время ожидания результата одного чтения продолжать отправлять другие запросы. Эти числа также иллюстрируют порядок проверки из раздела 1.3.4: модель последовательно уточняется с учётом трёх верхних границ — пропускной способности канала, пакетных накладных расходов и количества тегов в полёте, — пока расхождение с измерениями не сокращается приблизительно до 1,6%; на этом этапе в модели уже нет пропущенных факторов,

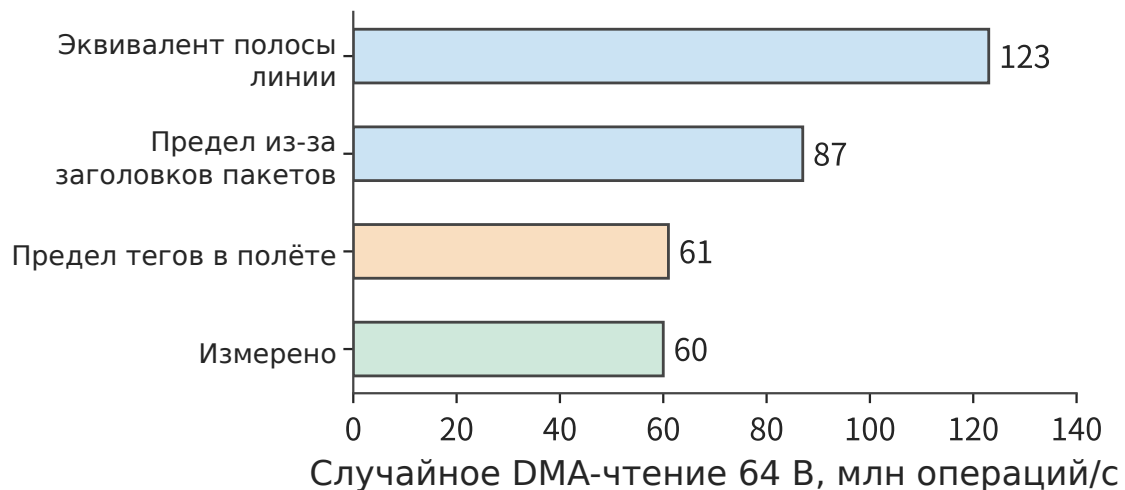


Рис. 7-28. Верхние границы и измеренный результат случайного DMA-чтения по 64 В на платформе KV-Direct. Пропускная способность канала соответствует 123 миллионам операций в секунду, накладные расходы заголовка пакета снижают верхнюю границу до 87 миллионов, а 64 тега при времени кругового обхода 1050 ns допускают лишь 61 миллион операций; измеренный результат составляет 60 миллионов операций, следовательно, количество тегов в полёте является узким местом чтения на этой платформе.

поэтому оставшееся расхождение не требует дальнейшего исследования.²¹

Когда один канал PCIe совместно используется двумя типами трафика, это различие приводит к асимметричной конкуренции. Предположим, что в направлении к GPU по каналу PCIe одновременно идут два потока: сетевая карта отправляет удалённые данные непосредственно в память GPU с помощью posted-записи, а движок копирования GPU переносит данные из памяти хоста в GPU, отправляя запросы чтения, ответы на которые возвращаются в GPU в виде пакетов завершения. Когда канал насыщается, posted-запись согласно протоколу должна продолжать продвигаться, тогда как сторона чтения после исчерпания тегов для операций в полёте не может отправлять новые запросы и вынуждена ждать возвращения пакетов завершения, поэтому ей достаётся значительно меньше половины пропускной способности. В направлении от GPU ситуация иная: копирование из GPU в память хоста представляет собой posted-запись, инициируемую GPU, а при удалённом чтении памяти GPU он должен отправлять пакеты завершения. Оба потока должны сначала получить данные из НВМ, а затем отправить их в канал через тракт передачи GPU, поэтому узкое место находится внутри GPU, а не в канале, и оба потока деградируют более сходным образом. Оба направления показаны на рис. 7-29. При анализе такой конкуренции сначала необходимо определить, является каждый поток posted-записью или чтением с пакетами завершения, а затем выяснить, на какой стороне канала и в каком общем компоненте они объединяются. Более позднее исследование общих путей PCIe с помощью инструмента тестирования производительности rPCIEBench также показывает, что объём работы в полёте и конкуренция на входе влия-

²¹Bojie Li и др., KV-Direct: High-Performance In-Memory Key-Value Store with Programmable NIC, SOSIP 2017, §2.4 и рис. 3; описание кредитов и тегов PCIe также приведено в докторской диссертации автора, глава 5. Числа приведены для исходной платформы (PCIe Gen3 x8 и сетевая карта FPGA). На других платформах число кредитов, число тегов и задержки отличаются, но сохраняется та же структура: чтение ограничено числом запросов в пути, а запись — скоростью обработки пакетов.

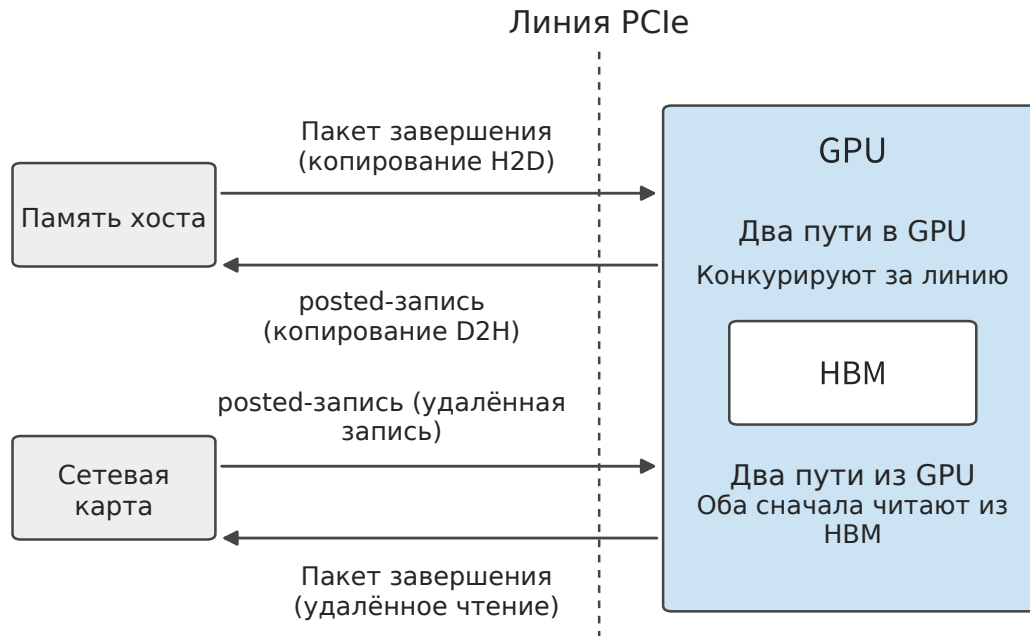


Рис. 7-29. В направлении к GPU posted-запись сетевой карты конкурирует за канал с пакетами завершения, возвращаемыми памятью хоста движку копирования: первая не ожидает ответа, а вторые ограничены количеством тегов для операций в полёте. В направлении от GPU posted-запись при копировании D2H и пакеты завершения удалённого чтения должны сначала получить данные из HBM и совместно использовать внутренний тракт передачи GPU.

ют на распределение пропускной способности.²² Приведённая выше формула позволяет определить узкое место: исчерпана ли пропускная способность канала, ещё не освободились ли слоты запросов или слишком низка скорость запуска запросов.

Иерархическая редукция сокращает объём передаваемых между серверами данных, а параллельная отправка обеспечивает непрерывную передачу по сети; кроме того, программе необходимо знать, когда эти данные можно использовать и когда можно освободить занимаемое ими пространство.

Эксперимент 7-8 · Расширение: сколько транзакций в полёте требуется для полного использования пропускной способности удалённого чтения

В примере 7.3 увеличьте время занятости слота с 2 μ s до 4 μ s; это время отсчитывается от отправки запроса до момента, когда слот можно использовать повторно. Отдельно рассчитайте количество транзакций в полёте, необходимое для поддержания целевой пропускной способности при передаче 256 B и 4 KiB за одну транзакцию. При фиксированном интервале запуска 18,6 ns найдите минимальный размер транзакции, необходимый для достижения 50 GB/s; затем повторите расчёт для интервала 6,2 ns реализации UB. Наконец, замените независимые адреса цепочкой указателей и объясните, как время чтения всей цепочки зависит от её длины.

²²Hou и др., *Understanding Routable PCIe Performance for Composable Infrastructures*, NSDI 2024; в экспериментах использовалась платформа PCIe Gen3; пути PCIe и диагностика.

7.4 Передача данных, порядок и управление состоянием

7.4.1 Создание и использование сегментов данных, повторное использование буферов

После доставки данных получатель может ещё не начать их использовать либо уже использовать их. Это похоже на передачу следующему производственному этапу поддона с материалами: поддон уже доставлен, но получателю всё ещё требуется время на обработку материалов, и до её завершения их нельзя заменить следующей партией. Аналогично, при передаче данных необходимо отдельно определять завершение отправки, видимость данных для получателя и завершение их потребления получателем. В разделе 7.3.4 время занятости слота считалось известным; здесь на основе этих событий определяется момент освобождения буфера.

При иерархической редукции карта 0 передаёт локальный сегмент карте 8, а карта 8 складывает его со своим сегментом. Отправителю необходимо знать, когда можно перезаписать исходный буфер; получателю — когда можно читать целевой буфер; следующей операции записи — когда этот целевой буфер можно перезаписать. Этим трём случаям соответствуют разные события.

От создания до освобождения сегмент проходит следующие этапы: создание данных, запись на удалённую сторону, обеспечение видимости данных для получателя, отправка уведомления о готовности, чтение и обработка данных и, наконец, освобождение пространства. Возможность повторного использования исходного буфера зависит от того, гарантирует ли семантика завершения интерфейса, что передача больше не читает из него. Прежде чем получатель начнёт использовать данные, необходимо гарантировать, что запись уже видна ему, а уведомление о готовности опубликовано и проверено в соответствии с соглашением. Перезаписывать целевой буфер можно только после того, как получатель закончит использовать находящиеся в нём данные. Само по себе завершение сетевой передачи не заменяет уведомления «использование завершено».

Например, отправка завершается на отметке 5 μs , потребитель начинает чтение на отметке 8 μs и заканчивает на отметке 12 μs . После завершения отправки отправитель может повторно использовать исходный буфер, но целевой буфер можно перезаписать только на отметке 12 μs . Если следующая запись перезапишет целевой адрес на отметке 6 μs , потребитель прочитает уже следующие данные. Каждый байт будет правильно передан по сети, однако принимающая программа использует неверные данные.

На рисунке 7-30 два интервала занятости завершаются при наступлении разных событий. Если представить стрелками требования «какого события необходимо дожидаться», получится граф зависимостей, описывающий корректную передачу: узлы обозначают работу, а ребро $u \rightarrow v$ означает, что v должно начаться после завершения u . Момент завершения каждого узла получается сложением наиболее позднего момента завершения всех его предшественников и времени выполнения самого узла. Цепочка с наибольшей длительностью от начального до конечного узла является критическим путём.

Эта модель одновременно объясняет корректность и производительность. Отсутствие зависимости «данные видимы $\rightarrow$ отправить уведомление» может привести к чтению потребителем ещё не опубликованных данных; лишнее ребро «необходимо также дождаться несвязанной передачи» увеличивает ожидание. Задача управления состоянием — регистрировать, какие узлы завершены, какие зависимости удовлетворены и какие ресурсы можно вернуть.

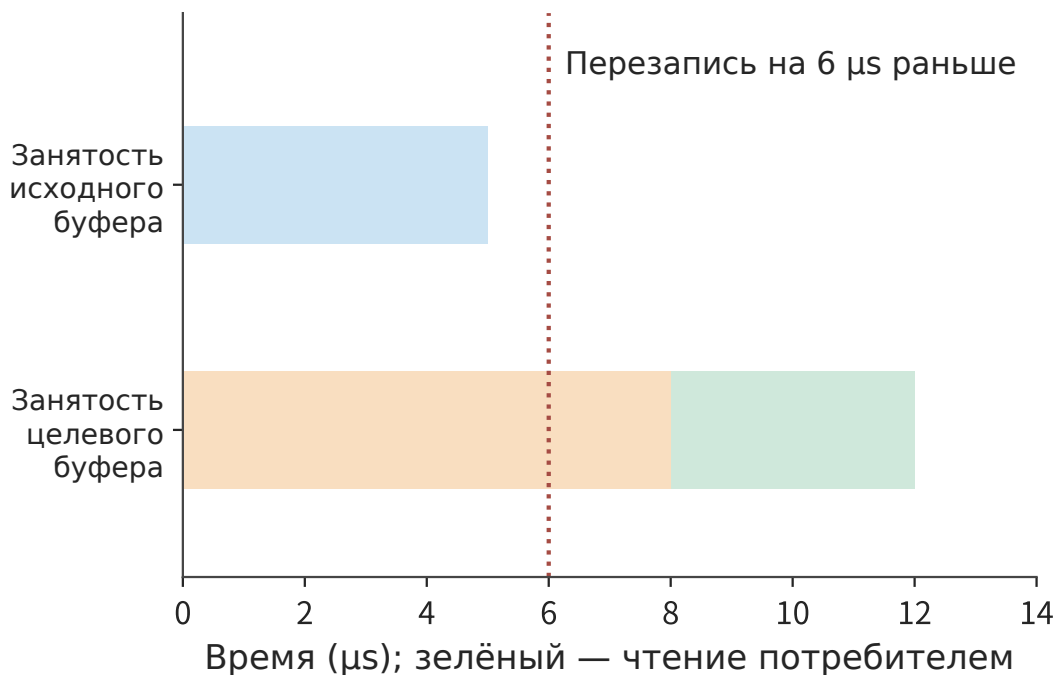


Рисунок 7-30. Исходный буфер занят до завершения отправки на отметке 5 μs ; целевой буфер остаётся занят до завершения использования потребителем на отметке 12 μs . Зелёным показан интервал чтения 8–12 μs . Пунктиром отмечена ошибочная преждевременная перезапись целевого адреса на отметке 6 μs .

Принадлежность буфера также влияет на количество копирований. Если коммуникационная библиотека выделяет для отправки отдельный зарегистрированный буфер, результат вычислений сначала потребуется скопировать в него, метаданные — упаковать вместе с данными, а на стороне получателя — распаковать и перенести данные в непрерывную область. Все эти операции копирования и упаковки занимают время SM. Прямая запись результатов вычислений в зарегистрированный буфер позволяет исключить этот этап, но порядок событий, рассматриваемый в данном разделе, становится строже: отправки можно инициировать только после завершения вычислительной записи и обеспечения её видимости для сетевой карты; до фактического завершения отправки этот буфер нельзя перезаписывать; получатель должен по тем же правилам определять, когда данные можно читать, а буфер — освободить.²³

7.4.2 Разделение уровня транзакций и транспортного уровня: Jetty и общее состояние

Система должна сохранять эти зависимости и состояния завершения. Нужно ли при добавлении каждого коммуникационного отношения сохранять ещё один одинаковый набор транспортных записей? Редукция между шестнадцатью участниками затрагивает лишь небольшое количество коммуникационных отношений. По мере масштабирования системы один процесс получает несколько потоков, каждый из которых обращается

²³Размеры буферов, отдельно зарезервированных для NVLink и RDMA, приведены в снимке README Deerp; накладные расходы на копирование и упаковку выведены из событийной модели этого раздела и не являются результатами измерения конкретной версии.

к нескольким удалённым целям, поэтому количество отношений растёт как произведение. Приложение должно регистрировать, кто отправил запрос и кому следует направить уведомление о завершении; транспортный уровень должен поддерживать порядковые номера, подтверждения, повторные передачи и состояние перегрузки. Первое идентифицирует приложение, второе обеспечивает надёжную передачу данных.

Конечная точка приложения — логическая сущность, от имени которой программа отправляет коммуникационные операции и получает уведомления о завершении; **транспортное состояние** хранит ход передачи, включая порядковые номера, подтверждения, повторные передачи и скорость отправки. Возьмём 64 конечные точки приложения, каждая из которых обращается к 128 удалённым целям: всего получается 8192 отношения. Если для каждого отношения отдельно хранить транспортное состояние размером 1 KiB, только транспортная часть займёт 8 MiB. Если отношения, направленные к одной цели, совместно используют одно транспортное состояние, потребуется лишь 128 экземпляров общим объёмом 128 KiB. Экономия достигается за счёт отказа от отдельных транспортных состояний для разных коммуникационных отношений с одной и той же целью.

Идентичность приложения по-прежнему необходимо сохранять. Каждая конечная точка занимает 256 B, всего 16 KiB; привязка каждого отношения занимает 64 B, всего 512 KiB. С учётом этих частей общий объём состояния при отдельном состоянии каждого отношения составляет около 8,52 MiB, а при совместном использовании по целям — около 0,64 MiB. Количество экземпляров транспортного состояния уменьшается до 1/64 исходного значения, а общий объём состояния — приблизительно до 1/13.²⁴

Пусть количество конечных точек равно L , количество целей — P , а каждая конечная точка, привязка отношения и экземпляр транспортного состояния занимают соответственно e, r, t bytes. Тогда

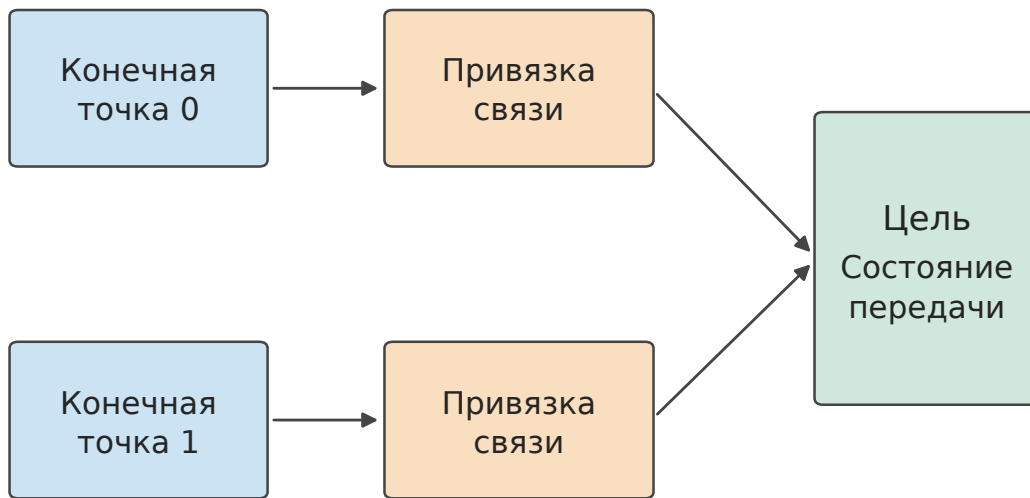
$$S_{\text{private}} = Le + LP(r + t), \quad S_{\text{shared}} = Le + LPr + Pt.$$

В статье об OpenURMA это разделение описывается с помощью другой оценки: если локальный интерфейс содержит L конечных точек приложения, обращающихся к P удалённым хостам, количество основных записей конечных точек и транспортных контекстов растёт как $O(L + P)$; если же каждое прикладное отношение использует отдельное надёжное соединение, эта часть растёт как $O(LP)$. Этот вывод относится только к основному аппаратному состоянию, которое допускает совместное использование: программные отображения, разрешения, незавершённые запросы и кэши не обязательно занимают пространство лишь аддитивного порядка. В приведённой выше формуле сохранён член LPr , поскольку в данной учебной схеме явно учитываются записи привязки каждого отношения. В конце этого подраздела расчёт повторяется с размерами записей из реализации.²⁵

Две формулы непосредственно показывают экономию пространства благодаря совместному использованию: LPr превращается в Pt , тогда как прикладные отношения LPr сохраняются. В UB Jetty отделяется от транспортного канала (раздел 6.5.5) именно

²⁴ Активные отношения и состояние передачи, изоляция восьми категорий. Модели отдельно учитывают три категории ёмкости: конечные точки, привязки отношений и состояние передачи.

²⁵ Wojie Li, «Размышления, лежащие в основе Unified Bus», разделы Jetty, порядка транзакций и Load/Store; статья OpenURMA, редакция от 2026-06-02 (arXiv:2605.28717), §3 — проектирование, §7–§9 — состояние и задержка, §10 — порядок, §12 — передача, §13 — сводка результатов. Источники и область применимости данной интеграции.



Идентификаторы точек хранятся отдельно, состояние передачи к цели — общее

Рисунок 7-31. Две конечные точки приложения по отдельности хранят данные идентификации и записи привязки, а привязки отношений указывают на общее транспортное состояние одной цели. Общая часть поддерживает ход передачи, но конечные точки по-прежнему могут различать свои запросы.

по этой схеме: одно транспортное состояние может обслуживать несколько прикладных отношений.²⁶

На рисунке 7-31 несколько отношений совместно используют одно транспортное состояние. Это экономит пространство, но может привести к конкуренции за окно отправки этого канала, возможности планирования и ресурсы восстановления. Разделение уровней не требует помещать все транзакции в одну глобально упорядоченную очередь: порядок выполнения прикладных транзакций, порядок их завершения и порядок прибытия сообщений по-прежнему необходимо задавать отдельно. Если реализация использует общую очередь FIFO или последовательное ожидание, потеря пакета или длительный запрос приведут к блокировке начала очереди (head-of-line blocking: если первый запрос останавливается, находящиеся за ним несвязанные запросы также вынуждены ждать). Возможность продолжить выполнение независимых транзакций уменьшает последствия, но не устраняет конкуренцию за общую пропускную способность и окно. Если большое количество незавершённых запросов одного коммуникационного отношения занимает общую очередь, короткие запросы других отношений вынуждены ждать. Если для разных видов нагрузки создать K групп независимого транспортного состояния, общий объём состояния составит

²⁶Записи сверки спецификации UB и эталонного проекта операционной системы.

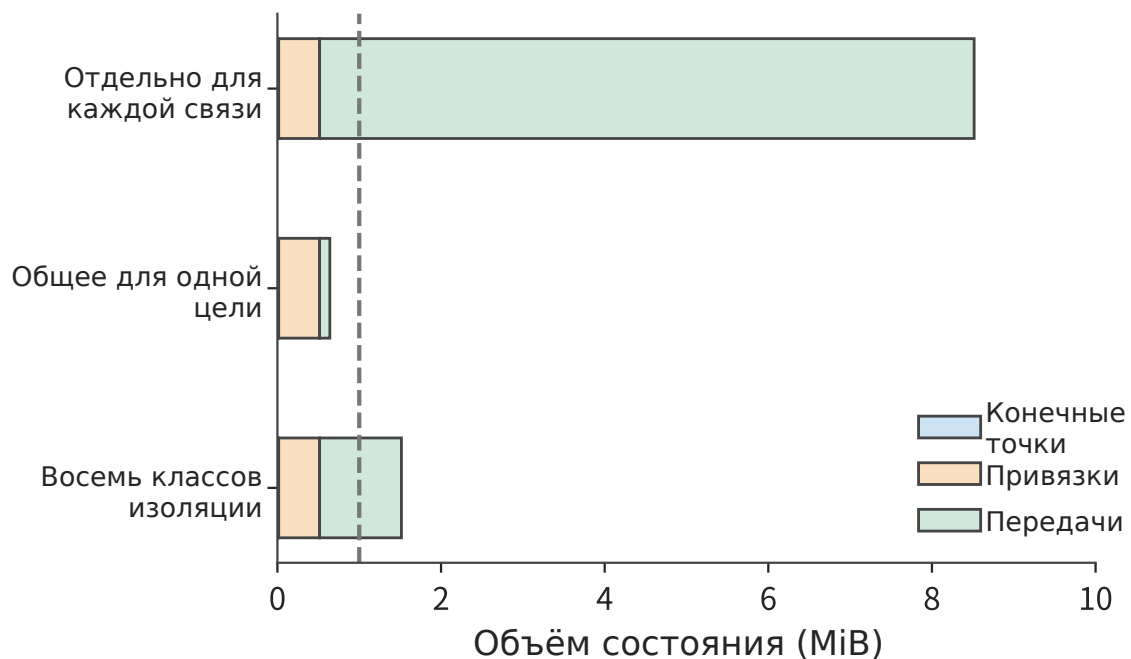


Рисунок 7-32. 64 конечные точки обращаются к 128 целям. Синим показаны записи конечных точек, оранжевым — привязки отношений, зелёным — транспортное состояние. Пунктирная линия обозначает объём 1 MiB; при разделении на восемь изолированных групп для каждой цели сохраняется восемь независимых экземпляров транспортного состояния.

$$S(K) = Le + LPr + KPt.$$

Для приведённой выше конфигурации фиксированная часть равна 528 KiB, а каждая дополнительная группа независимого транспортного состояния добавляет 128 KiB. В хранилище объёмом 1 MiB помещается не более трёх полных групп независимого транспортного состояния; четырём группам требуется 1040 KiB, что уже превышает доступный объём; восемью группам требуется около 1,52 MiB. На рисунке 7-32 показан объём состояния для этих стратегий совместного использования. Следовательно, изоляцию видов нагрузки необходимо организовывать в пределах доступного хранилища: выделение отдельного транспортного состояния критически важной нагрузке и совместное использование состояния остальными видами нагрузки позволяет уменьшить помехи для критически важной нагрузки.

Помимо совместного использования разными отношениями, состояние можно хранить иерархически. Полное состояние размещается в основной памяти, а часто используемая часть кэшируется на сетевой карте. Совместное использование уменьшает дублирование записей, позволяя кэшу того же размера вместить больше коммуникационных отношений. Если часто используемое состояние остаётся в кэше, сокращается и количество его чтений из основной памяти. Таким образом, экономия пространства состояния позволяет также снизить накладные расходы на обработку запросов.

Размеры записей в реализации и общий объём состояния. Приведённые выше значения 256, 64 и 1024 В заданы в этом разделе. Структуры состояния реализации OpenURMA дают проверяемый набор размеров: каждый Jetty занимает 20 В, каждый зарегистрированный сегмент памяти — 32 В, каждый транспортный канал — 56 В; для

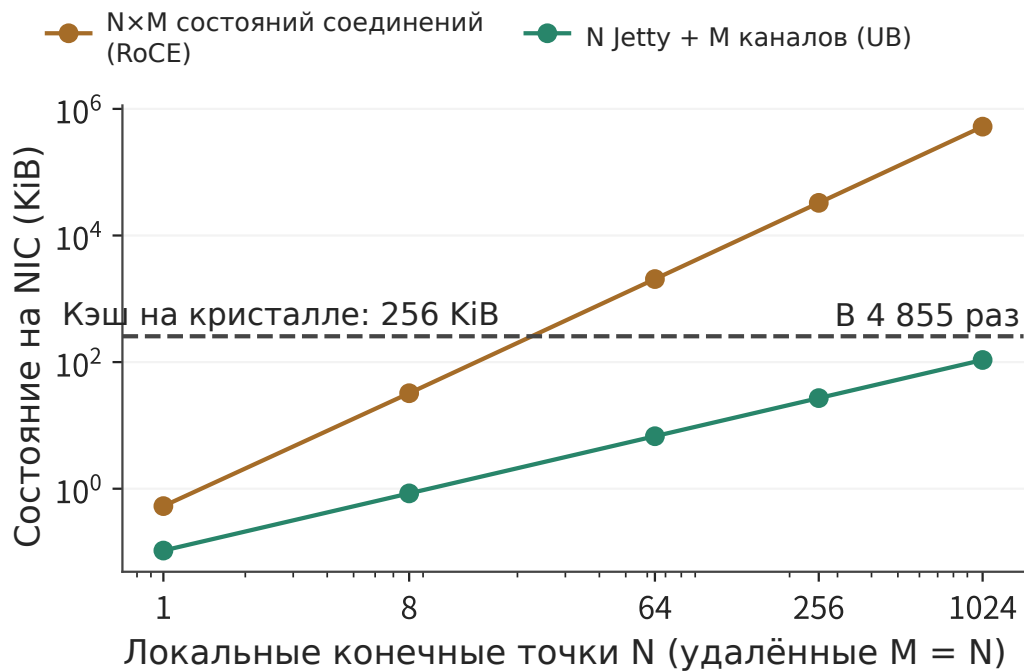


Рисунок 7-33. Состояние, сохраняемое сетевой картой при обращении N локальных конечных точек к $M = N$ удалённым конечным точкам для двух вариантов организации. Попарные соединения превышают объём внутрикристального кэша 256 KiB уже при N немногим больше 20, тогда как конечные точки и каналы даже при $N = 1024$ занимают меньше половины кэша.

сравнения, реализация надёжного соединения RoCE хранит для каждого соединения контекст QP размером 512 В. Пусть N локальных конечных точек обращаются к M удалённым конечным точкам. При двух вариантах организации объём состояния, сохраняемого сетевой картой, равен

$$S_{\text{pair}} = 512NM + 32N, \quad S_{\text{UB}} = 52N + 56M.$$

В сопоставлении с предыдущими формулами здесь $e = 52$, $t = 56$, а $r = 0$: эта реализация размещает привязки отношений в программном отображении, поэтому сетевая карта не хранит аппаратное состояние для каждого отношения. При $N = M = 1024$ попарным соединениям требуется около 512 MiB, а конечным точкам и каналам — лишь 108 KiB, то есть приблизительно в 4855 раз меньше. Даже если дополнить запись Jetty всеми полями спецификации до размера 48 В, отношение всё равно превышает три тысячи. На рисунке 7-33 эти две кривые представлены в логарифмическом масштабе: наклон одной равен 2, другой — 1, а их отношение линейно растёт с N .

Общий объём состояния и переполнение внутрикристального кэша. Сетевая карта кэширует контексты на кристалле, а не поместившаяся часть остаётся в памяти хоста. При объёме кэша 256 KiB попарные соединения переполняют его при $512N^2 + 32N > 262144$, то есть при $N \geq 23$; конечные точки и каналы — при $108N > 262144$, то есть при $N \geq 2428$. После переполнения при каждой операции приходится заново считывать контекст. Цена зависит от места его хранения: сетевая карта — периферийное устройство PCIe — читает его из памяти хоста, поэтому инициатор и целевая сторона выполняют по одной операции чтения PCIe DMA, что добавляет к каждой операции около 1000 ns; контроллер внутрикристальной шины читает его из локальной памяти, поэто-

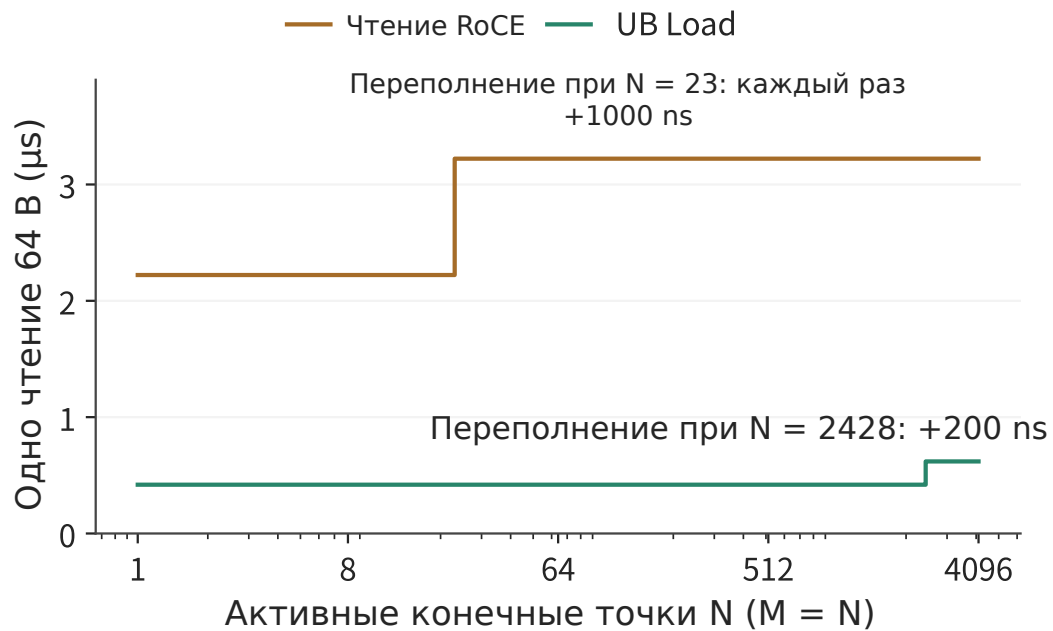


Рисунок 7-34. Зависимость задержки одного чтения 64 В от количества активных конечных точек при внутрикристальном кэше объёмом 256 KiB. Для попарных соединений задержка возрастает на 1000 ns при 23 конечных точках, а для конечных точек и каналов — на 200 ns при 2428 конечных точках. Ось абсцисс имеет логарифмический масштаб.

му на каждой стороне выполняется один переход через внутрикристальную шину и одно обращение к памяти, что добавляет около 200 ns. На рисунке 7-34 задержка двух путей, выведенная в разделе 7.3.3, представлена как функция количества активных конечных точек. Коммуникация «все со всеми» при обучении обычно задействует от нескольких десятков до нескольких тысяч конечных точек — именно этот диапазон. В нём каждое чтение при попарных соединениях требует заново считывать контекст. Симулятор автора определяет переполнение по количеству записей, а не по количеству байтов: 512 для RoCE и 2048 для UB. Поэтому точка переполнения UB в нём соответствует 1024 конечным точкам. При обоих способах оценки вывод одинаков: точка переполнения для конечных точек и каналов отстоит более чем на порядок, а цена после переполнения также на порядок меньше.²⁷

Эксперимент 7-9 · Расширенный: сколько изолированных групп может поддерживать общее транспортное состояние

Сохранив 64 конечные точки, обращающиеся только к одной удалённой цели, вычислите общий требуемый объём хранилища для отдельного и общего транспортного состояния. Затем увеличьте количество активных целей до 128, доступный объём хранилища — до 2 MiB и определите максимальное количество полных изолированных групп. Сравните влияние увеличения количества конечных точек и количества целей на фиксированное и транспортное

²⁷Параметры и результаты для задержки этапов, частоты запросов и роста состояния приведены в расчёте системы межсоединений UB; расчёт можно повторить командой `python3 calculations/calc.py ub-fabric --format md`. Задержки каждого этапа взяты из таблицы 7 статьи OpenURMA, размеры записей — из таблицы 3, а контрольные значения моделирования — из §8.1 и §8.3. В выкладках все этапы складываются последовательно без учёта перекрытия параллельных запросов.

состояние. Наконец, используйте размеры записей из реализации — Jetty 20 В, транспортный канал 56 В, контекст соединения 512 В — и определите количество конечных точек, при котором каждый из двух вариантов организации переполнит внутрикристальный кэш объёмом 1 MiB.

7.4.3 Зависимости операций и изоляция отказов

В разделе 7.4.2 независимые транспортные группы использовались для уменьшения взаимных помех между разными видами нагрузки. Внутри одной группы операций также может возникнуть лишнее ожидание — это зависит от того, какие операции система принудительно выполняет последовательно. Рассмотрим в качестве примера публикацию сегмента картой 0. Операция А записывает данные D, операция В публикует уведомление «D готовы», а операция С передаёт другие, несвязанные данные. Для корректного использования D необходимо сохранить порядок А→В, тогда как С может использовать другой независимый транспортный путь.

Пример 7.4. Какие операции могут завершиться раньше при ослаблении ограничений порядка? Запись А занимает 20 μ s, последующее восстановление — 80 μ s, после его завершения D становятся видимыми; В занимает 2 μ s; С занимает 10 μ s. Сравните два варианта: последовательное завершение А, В и С и требование выполнять В только после завершения А.

Решение: цепочка зависимостей записи, восстановления и уведомления занимает $20 + 80 + 2 = 102 \mu$ s. При последовательном выполнении С начинается лишь на отметке 102 μ s, и вся группа завершается на отметке 112 μ s. Если сохранить только необходимую зависимость, С начинается в начальный момент и завершается за 10 μ s; цепочка публикации по-прежнему завершается на отметке 102 μ s. Вся группа завершается на 10 μ s раньше, но С — на 102 μ s раньше. Эти варианты показаны соответственно на рисунках 7-35 и 7-36.

Если время восстановления увеличится до 200 μ s, при последовательном выполнении С завершится лишь на отметке 232 μ s, а при независимом — по-прежнему на отметке 10 μ s. Удаление лишней зависимости ограничивает вызванную восстановлением остановку цепочкой, которой действительно нужны эти данные. Для последующей задачи, ожидающей С, такая изоляция гораздо важнее, чем ускорение всей группы на 10 μ s.²⁸

Глобальный порядок упрощает описание последовательности операций верхним уровнем, но помещает независимую работу в ту же цепочку ожидания. Если граф вычислений уже явно задаёт зависимость публикации А→В, интерфейс и среда выполнения могут гарантировать порядок именно для этой зависимости, позволяя С выполняться независимо. Приложение предоставляет более точные сведения о зависимостях, а сеть на их основе гарантирует порядок только для связанных операций. Это одновременно гарантирует, что после получения уведомления получатель прочитает уже записанные данные, и уменьшает ненужное ожидание.

Теперь предположим, что все операции используют один обрабатывающий модуль:

²⁸Необходимые зависимости и контрпример со старым значением, вариант с общим ресурсом, исследование упорядочивания удалённых операций и разбор фиксированной реализации NVSHMEM. Упорядочивание на целевом узле относится к решениям, требующим новой аппаратной поддержки; контрольные эксперименты с современными сетевыми картами используются для наблюдения за временным порядком запросов и завершений. Значения $20 + 80 + 2 \mu$ s для цепочки публикации и $1-6 \mu$ s для контрпримера со старым значением относятся к двум независимым наборам входных данных. Стратегия «выполнить всё последовательно» используется как последовательный контрольный вариант.

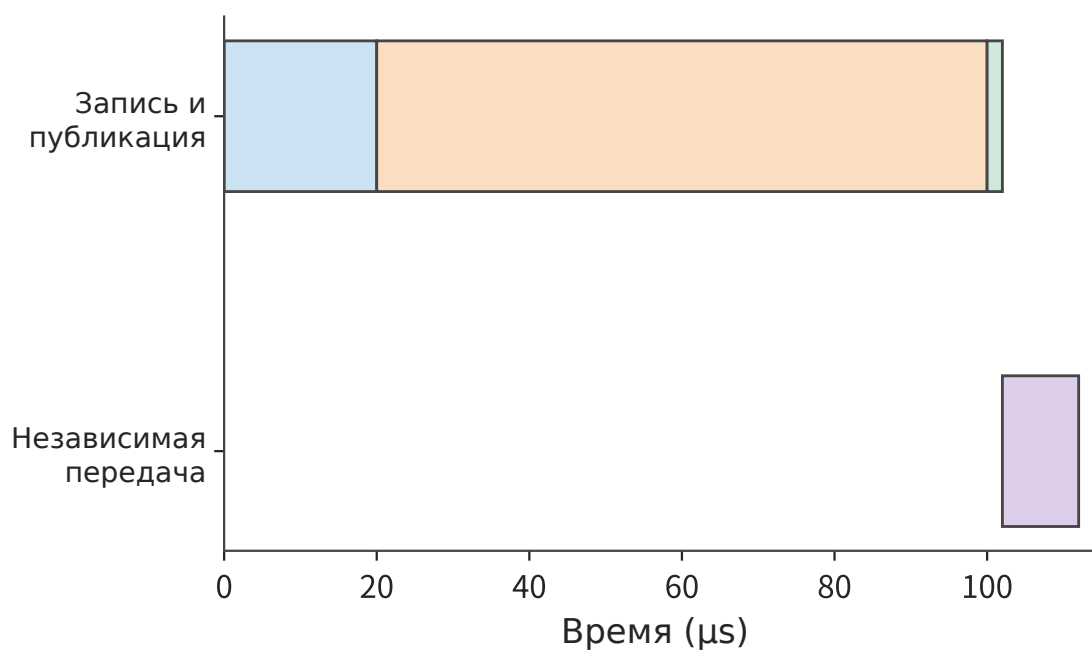


Рисунок 7-35. Независимая передача также поставлена после публикации: сначала запись 20 μs , затем восстановление и обеспечение видимости данных 80 μs , уведомление 2 μs и, наконец, независимая передача 10 μs .

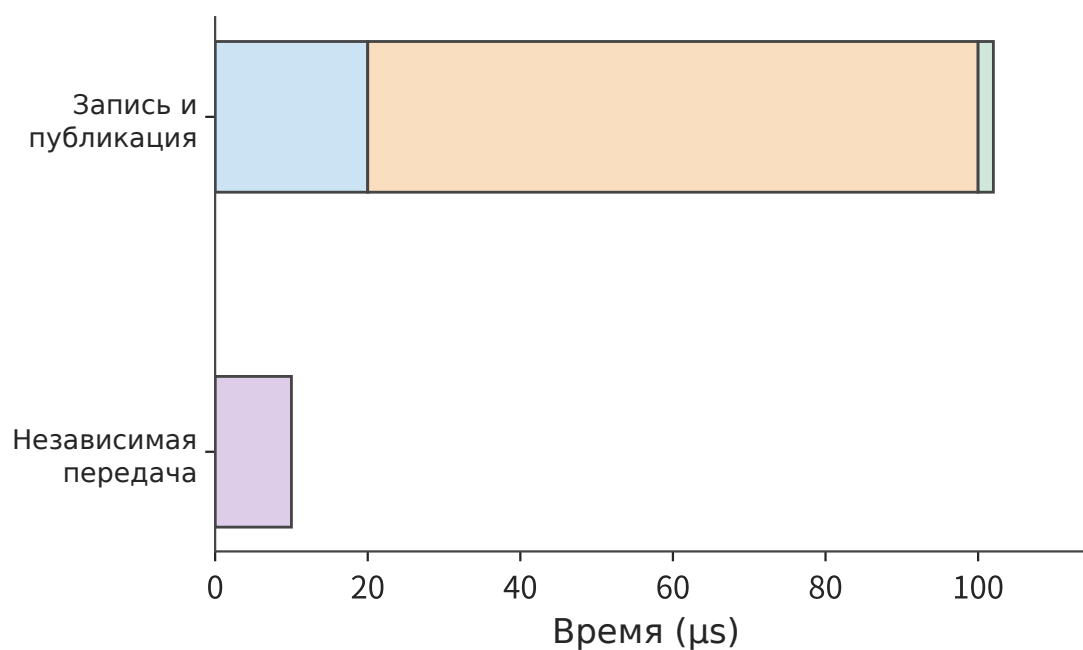


Рисунок 7-36. Запись, восстановление и уведомление по-прежнему выполняются последовательно; передача, использующая независимые ресурсы, начинается на отметке 0 μs . Синим показана запись, оранжевым — восстановление, зелёным — уведомление, фиолетовым — независимая передача.

он занят с начала записи А до завершения уведомления В, поэтому С может выполняться только после них. В этом случае само использование ресурса добавляет ребро порядка от цепочки публикации к С, и при обоих вариантах выполнение завершается на отметке 112 μ s. Следовательно, программные и ресурсные зависимости необходимо изображать на одном графе: даже после ослабления ограничений порядка в программе ресурсные ограничения по-прежнему ограничивают параллелизм.

Пример проектирования: аппаратные издержки задания порядка по требованию. В авторской реализации OpenURMA четыре режима обслуживания порядка UB и три метки выполнения реализованы в одном конвейере отправки. Тактовая симуляция показывает: независимо от требуемого запросом режима порядка от отправки запроса до выхода первого пакета проходит 24 такта. Трекер порядка заставляет запрос ждать только тогда, когда тот явно требует завершения предшествующих операций; если незавершённых предшествующих операций не больше четырёх, ожидание не превышает 50 тактов, то есть около 155 ns. Запросы, не требующие порядка, этих издержек не несут. Ещё важнее эффект изоляции: когда запрос одного инициатора останавливается в ожидании предшествующей операции, восемь запросов без требований к порядку от четырёх других инициаторов всё равно полностью отправляются за 78 тактов, то есть около 242 ns; остановленный запрос не задерживает остальных инициаторов. Для сравнения, в надёжном соединении RoCE запросы одного QP должны выполняться строго по порядку: если один запрос ожидает, все последующие также ожидают. Именно так на аппаратном уровне проявляется вариант из примера 7.4, в котором С поставлена после А и В.²⁹

Предпосылки проектирования: как глобальный порядок упрощает репликацию и координацию?

В исследовании 1Pipe автор изучал архитектуру, в которой сеть обеспечивает глобальный полный порядок. Если все участники наблюдают операции в одной последовательности, верхний уровень может использовать более простую модель порядка для репликации и координации. При дальнейшей обработке отказов порядок и доставку необходимо рассматривать отдельно: если отправитель завершит работу посреди отправки, система должна определить, какие узлы уже получили данные; если получатель окончательно завершит работу, система должна решить, какие задачи продолжать выполнять. Для решения этих проблем необходимо отделить гарантии порядка от механизмов доставки и восстановления и явно выразить действительно необходимые приложению зависимости.

В приведённом выше примере уведомление можно отправить только после завершения записи. Получатель также должен прочитать данные в правильный момент, иначе даже при корректном порядке уведомлений он может использовать старое значение. Пусть начальное значение D равно 0 и на отметке 2 μ s изменяется на 1; флаг готовности становится видимым на отметке 3 μ s. Потребитель преждевременно читает D на отметке 1 μ s, а флаг — на отметке 4 μ s. В результате у него сохраняются новый флаг и старые данные. Даже если на отметке 5 μ s вернуть результаты чтения в порядке «флаг, данные», сохранённое значение D всё равно будет равно 0.

На рисунке 7-37 перенос вправо пустой точки, обозначающей момент возврата ре-

²⁹Bojie Li, «Размышления, лежащие в основе Unified Bus», разделы Jetty, порядка транзакций и Load/Store; статья OpenURMA, редакция от 2026-06-02 (arXiv:2605.28717), §3 — проектирование, §7–§9 — состояние и задержка, §10 — порядок, §12 — передача, §13 — сводка результатов. Источники и область применимости данной интеграции.



Рисунок 7-37. Фактическое чтение, обновление данных, уведомление о готовности и возврат результата перечислены во временном порядке. На отметке 5 μ s по-прежнему возвращается старое значение, прочитанное на отметке 1 μ s; если на отметке 4 μ s обнаружить конфликт и выполнить повторное чтение, новое значение будет получено на отметке 6 μ s.

зультата, не перемещает закрашенную точку, обозначающую фактический момент получения значения. Этот контрпример показывает, что требование порядка должно применяться именно к моменту чтения данных. Потребитель может прочитать D после наблюдения флага либо выполнить предварительное чтение, проверить наличие конфликта и при необходимости прочитать повторно. Если после обнаружения конфликта на отметке 4 μs повторное чтение занимает 2 μs , правильные данные будут получены на отметке 6 μs . Упорядочивание на стороне получателя переносит ожидание на принимающую сторону, а проверка конфликтов определяет, какие результаты предварительного чтения остаются действительными. Чтобы гарантии порядка действительно упрощали программирование, необходимо сохранять эти состояния и выполнять соответствующие проверки.

7.4.4 Освобождение слотов запросов и обратное давление

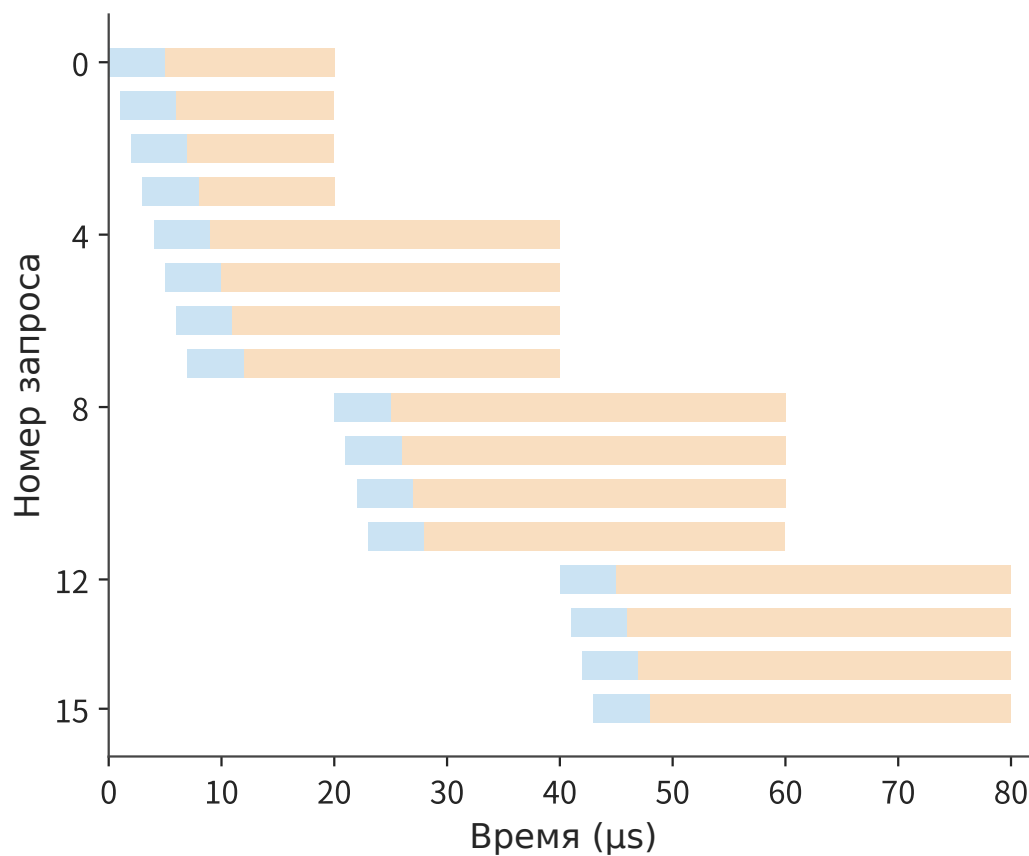
Слоты запросов из раздела 7.3.4 освобождаются только после того, как программное обеспечение обработает уведомление о завершении; лишь затем ими могут воспользоваться новые запросы. У буферов данных, рассмотренных в разделе 7.4.1, другой срок использования: пока получатель не закончил использовать данные, буфер необходимо сохранять. Если передача ускоряется, а уведомления о завершении обрабатываются недостаточно быстро, накапливается множество запросов со статусом «операция завершена, слот не освобождён».

Возьмём 16 операций по 8 KiB каждая, которые можно отправлять не чаще одного раза в 1 μs и которые завершаются через 5 μs после отправки. При наличии 16 слотов последняя операция отправляется на отметке 15 μs , а к отметке 20 μs завершаются все передачи. Программное обеспечение каждые 20 μs читает и обрабатывает четыре уведомления о завершении. Для 16 уведомлений требуется четыре цикла, поэтому все соответствующие слоты будут освобождены только на отметке 80 μs .

Если слотов всего восемь, первые восемь отправок быстро занимают их все. Девятая отправка должна ждать до отметки 20 μs , когда программное обеспечение обработает уведомления о завершении и освободит слоты; последующие запросы продолжают отправляться партиями, и все передачи завершаются только на отметке 48 μs . Удвоение количества слотов переносит момент завершения передачи с 48 на 20 μs , но полное освобождение всех слотов всё равно происходит лишь на отметке 80 μs .³⁰

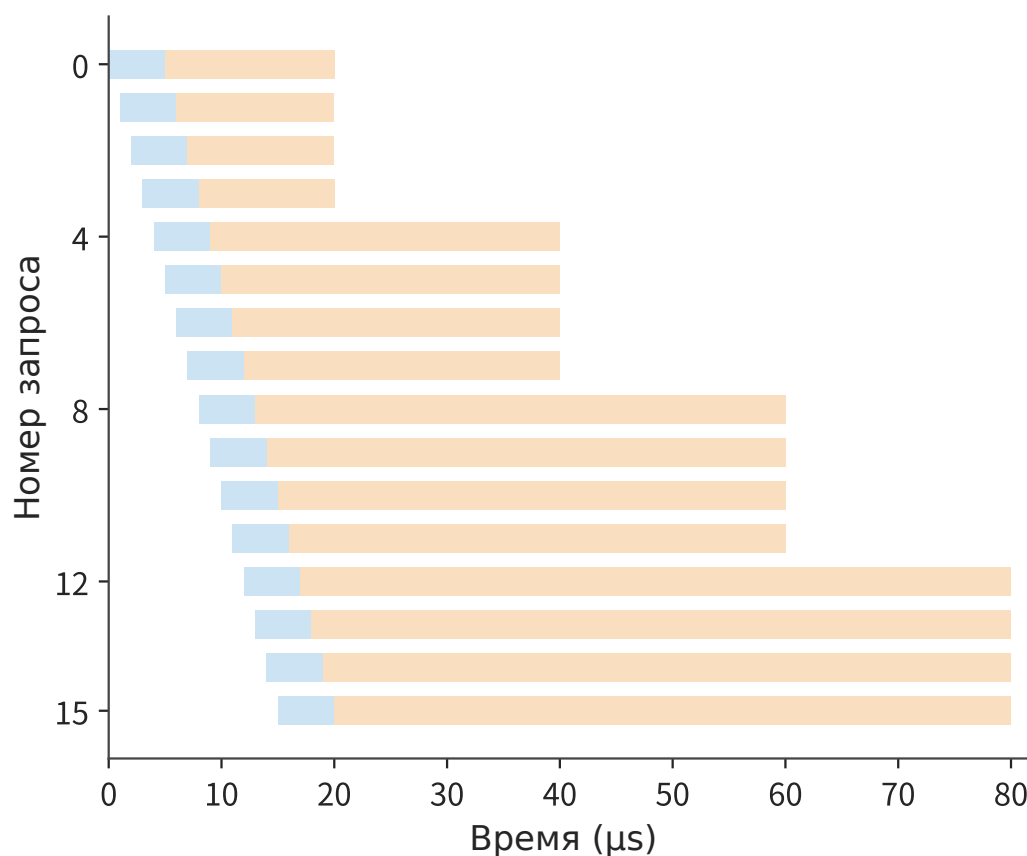
Сравнение рисунков 7-38 и 7-39 показывает: увеличение количества слотов сдвигает синие полосы передачи влево, но длительное оранжевое ожидание справа сохраняется. Устойчивую пропускную способность определяет скорость окончательного освобождения этих слотов. Согласно модели параллелизма и пропускной способности поток обработки уведомлений о завершении обрабатывает лишь $4/(20 \times 10^{-6}) = 200000$ элементов в секунду. При размере элемента 8 KiB это соответствует устойчивой пропускной способности обработки около 1,64 GB/s. Чтобы при таком размере операций достичь 50 GB/s одной сетевой карты, в среднем необходимо обрабатывать около 6,1 миллиона элементов в секунду, то есть приблизительно один элемент каждые 0,164 μs . Увеличение количества слотов позволяет лишь принять кратковременный всплеск запросов; при длительной нагрузке необходимо повысить скорость обработки уведомлений о завершении

³⁰Обработка уведомлений о завершении и освобождение слотов, увеличение числа слотов. Период опроса, число обрабатываемых за один опрос уведомлений о завершении и время занятости слотов заданы в учебных целях.



Синий: передача; оранжевый: ожидание освобождения слота

Рисунок 7-38. Синим показан интервал от отправки до завершения передачи, оранжевым — ожидание обработки уведомления о завершении программным обеспечением. Слот освобождается только после завершения оранжевого интервала. Моменты времени всех запросов взяты из журнала событий.



Синий: передача; оранжевый: ожидание освобождения слота

Рисунок 7-39. После увеличения количества слотов последующие запросы отправляются раньше. Уведомления о завершении по-прежнему обрабатываются партиями по четыре каждые 20 μs , поэтому последний слот всё равно освобождается на отметке 80 μs . Временная шкала одинакова на обоих рисунках.

или увеличить объём данных одной операции.

Если уведомления о завершении обрабатываются недостаточно быстро, отправку вышестоящим компонентом необходимо ограничивать. Это и есть обратное давление из главы 5: когда слоты запросов заполнены, конечная точка приостанавливает или замедляет отправку, пока не освободится слот. Обратное давление управляет вышестоящим компонентом, ограничивая количество незавершённых запросов и предотвращая неограниченный рост объёма незавершённой работы. Если буфер можно освободить только после завершения использования данных удалённой программой, обратное давление распространяется между устройствами; когда множество задач совместно используют транспортное состояние, область его распространения становится ещё шире.

После отказа также необходимо сначала завершить старые запросы и только затем освободить занимаемое ими пространство. После отмены запросов система сначала прекращает новые отправки, затем ожидает завершения старых обращений, находящихся в пути, либо изолирует их и лишь после этого освобождает пространство. Иначе запоздавшая старая запись перезапишет новый объект. Присвоение нового номера версии при каждом повторном использовании объекта, отзыв прав доступа и изоляция потерявшей связь конечной точки служат одной цели: не позволить старым запросам обратиться к уже перераспределённому пространству. После запуска восстановления по тайм-ауту система должна перед повторным использованием пространства подтвердить завершение старых запросов либо запретить им дальнейший доступ. Если несколько конечных точек совместно используют сеть, такая же разница между скоростью поступления и скоростью обработки превращается в очереди внутри коммутатора.

Эксперимент 7-10 · Основной: как ослабление порядка операций уменьшает ожидание, сохраняя корректность данных

Измените длительность независимой операции *C* в примере 7.4 на 50 μ s и определите время завершения всей группы операций и независимой операции при двух вариантах выполнения. Затем предположите, что после получения уведомления потребителю требуется 30 μ s на обработку данных, и отметьте наиболее ранний момент, когда в целевой буфер можно записать новые данные. Для контрпримера со старым значением постройте два пути выполнения: чтение после ожидания и предварительное чтение с последующим повторным чтением.

7.5 Перегрузка и надёжность в общей сети

7.5.1 От фиксированного трафика к изменяющемуся во времени спросу

Добавим к конфигурации из предыдущих разделов ещё одно задание обучения на двух серверах и разместим оба задания в многорельсовой топологии из раздела 7.2.5: два сервера каждого задания относятся к разным группам (по 32 сервера в каждой), поэтому межсерверный трафик в пределах одной *rail* уже не достигает цели за один переход, а проходит через восходящий канал *leaf*-коммутатора на уровень *spine*. Потоки обоих заданий на этой *rail* хешируются на один восходящий канал 400 Gbit/s, обеспечивающий 50 GB/s в каждом направлении. На межсерверном этапе иерархической редукции каждая сетевая карта отправляет данные со скоростью 50 GB/s, поэтому при совпадении пиков

двух заданий спрос превышает пропускную способность канала.

Пример 7.5. Почему очередь возникает, даже если в среднем используется только 40% пропускной способности канала? Каждое из двух заданий раз в 100 ms создаёт пик передачи длительностью 20 ms со скоростью 50 GB/s, а в остальное время ничего не передаёт. Предположим, что изначально очередь пуста, задания отправляют данные по указанному расписанию, а буфер достаточно велик, чтобы вместить весь накопившийся объём.

Решение: средний суммарный спрос двух заданий равен

$$\bar{\lambda} = 2 \times 50 \times \frac{20}{100} = 20 \text{ GB/s}.$$

По сравнению с 50 GB/s средняя утилизация составляет всего 40%. Однако при одновременном начале двух пиков скорость поступления превышает скорость выходного канала на 50 GB/s, и за 20 ms объём накопившихся данных достигает

$$Q_{\max} = (100 - 50) \text{ GB/s} \times 20 \text{ ms} = 1 \text{ GB}.$$

После завершения пика для опустошения очереди потребуется ещё 1 GB/50 GB/s = 20 ms. Оба задания прекращают отправку на отметке 20 ms, но выходной канал продолжает передавать эти данные ещё 20 ms. Если задержать второе задание на 20 ms, пики полностью разойдутся, в любой момент будет поступать не более 50 GB/s, а очередь останется пустой. В обоих вариантах передаётся одинаковый объём данных; различается только относительное время начала отправки двух заданий. На рисунке 7-40 показана скорость поступления при разной степени перекрытия, а на рисунке 7-41 — соответствующий объём накопившихся данных.³¹

Обсуждение. При каком смещении времени отправки в общем выходном канале начнут накапливаться данные? При длительности перекрытия h дополнительный объём накопившихся данных равен $(100 - 50)h$. Перекрытие на 5 ms создаёт очередь объёмом 250 MB. Если для этого всплеска выделен буфер всего 512 KiB и изначально очередь пуста, допустимое время перекрытия составляет приблизительно 512 KiB/50 GB/s = 10.5 μ s. Даже если погрешность времени отправки приводит лишь к перекрытию порядка миллисекунд, это намного превосходит возможности такого буфера.

Идея разведения пиков уже реализована в существующих системах. CASSINI использует периодичность коммуникаций при обучении, размещая задания и планируя время обмена так, чтобы развести пики, а также отслеживает дрейф и корректирует расписание.³² В сопутствующем эксперименте этой книги сравнивались два задания распределённого параллельного обучения по данным (DDP), выполнявшиеся на CPU, с однократным разведением пиков: после задержки одного из заданий на 50 ms их фазы продолжили смещаться во время выполнения, и в трёх измерениях время завершения всей группы заданий, напротив, увеличилось примерно на 2–3%.³³ При периодическом планирова-

³¹Периодический спрос и очередь, разнесение по времени, дрейф фазы. Скорость поступления в примере изменяется в соответствии с заданным периодом.

³²Rajasekaran и др., CASSINI: Network-Aware Job Scheduling in Machine Learning Clusters, NSDI 2024; размещение заданий и планирование фаз. Основные эксперименты статьи проводились на 24 серверах с одной A100 40 GB, сетевыми картами 50 Gbps и логической сетью с переподпиской 2:1; каждое задание монополично использовало устройства обучения, но сеть была общей.

³³experiments/ch07/07-08: реальное обучение на CPU и однократное разнесение запуска. В результаты трёх раундов включена начальная задержка 50 ms; состояния модели и оптимизатора проверены и совпадают.

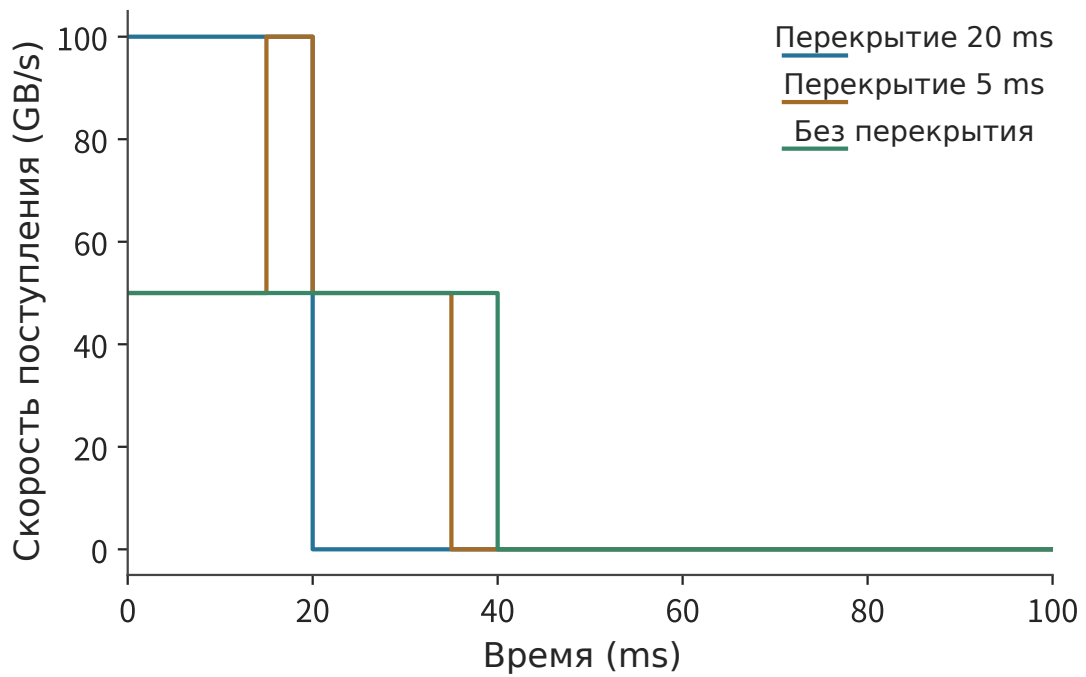


Рисунок 7-40. Каждое из двух заданий отправляет данные по 20 ms каждые 100 ms со скоростью 50 GB/s. Разная степень перекрытия пиков создаёт разную суммарную скорость поступления при общем выходном канале 50 GB/s.

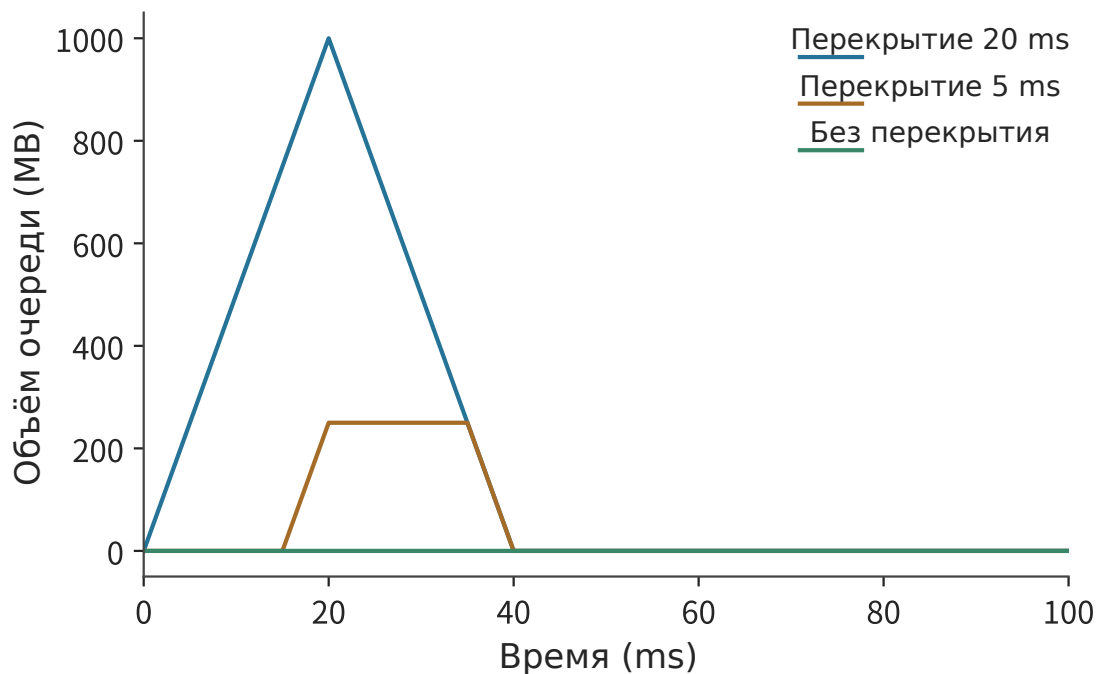


Рисунок 7-41. Накопление во времени разности между скоростью поступления и скоростью выходного канала даёт три пиковых значения: 1000, 250 и 0 MB. В этом расчёте предполагается бесконечный буфер.

нии фазу необходимо поддерживать постоянно, тогда как управление перегрузкой корректирует отклонившуюся от ожидаемой скорость передачи по обратной связи в реальном времени.

7.5.2 Задержка обратной связи и ёмкость буфера

Разведение пиков уменьшает их перекрытие, но фактическое время отправки всё равно может отклоняться от расписания. Поэтому, когда очередь начинает расти, необходимо своевременно замедлить отправителей. Если данные поступают быстрее, чем передаются выходным каналом, избыток накапливается в очереди. Обозначим длину очереди через Q , скорость поступления через $\lambda(t)$, а скорость выходного канала через B . Пока очередь не пуста и не заполнена,

$$\frac{dQ}{dt} = \lambda(t) - B.$$

Если скорость поступления выше скорости передачи, очередь растёт; если ниже — накопившийся объём уменьшается.

Для своевременного снижения скорости поступления применяются механизмы двух типов. **Управление потоком на уровне канала** позволяет соседнему получателю при нехватке буфера приостановить вышестоящий узел и предотвратить переполнение приёмного буфера. **Сквозное управление перегрузкой** передаёт сведения об узком месте отправителю, который снижает скорость передачи. Первый механизм быстро предотвращает локальное переполнение, второй адаптирует скорость источника к пропускной способности всего маршрута.

Продолжим пример со скоростью поступления 100 GB/s и выходным каналом 50 GB/s. Пусть общий объём буфера равен 512 KiB, из которых уже занято 256 KiB. Каждую микросекунду дополнительные 50 KB данных занимают оставшееся пространство, которого хватит примерно на 5,2 μ s. Если обратная связь приведёт к снижению скорости лишь через 20 μ s, возникнет 1 MB избыточных данных; после вычета оставшегося пространства около 738 KB будет отброшено.³⁴

Обозначим оставшийся буфер через Q_{free} , а задержку от обнаружения перегрузки до снижения скорости — через T_f . Чтобы избежать переполнения, должно выполняться условие

$$T_f \leq \frac{Q_{\text{free}}}{\lambda - B}.$$

Таким образом, при заданных скоростях поступления и передачи допустимую задержку обратной связи можно вычислить по размеру буфера. Если удвоить свободное пространство, допустимая задержка обратной связи также удвоится; если удвоить разность между скоростью поступления и скоростью выходного канала, допустимая задержка сократится вдвое. При указанных выше скоростях снижение скорости через 50 μ s приведёт к накоплению дополнительных 2,5 MB данных.

После получения обратной связи накопившиеся данные ещё нужно удалить из очереди. Если снизить скорость отправки только до 50 GB/s, скорость поступления сравня-

Этот контрольный вариант используется для анализа порядка выполнения на CPU. В среде с общим CPU и Gloo loopback время выполнения трёх раундов увеличилось на 2,27–3,32%.

³⁴Ограниченный буфер и задержка обратной связи. Момент обратной связи установлен на 20 μ s, а скорость отправки после обратной связи изменяется согласно условию задачи.

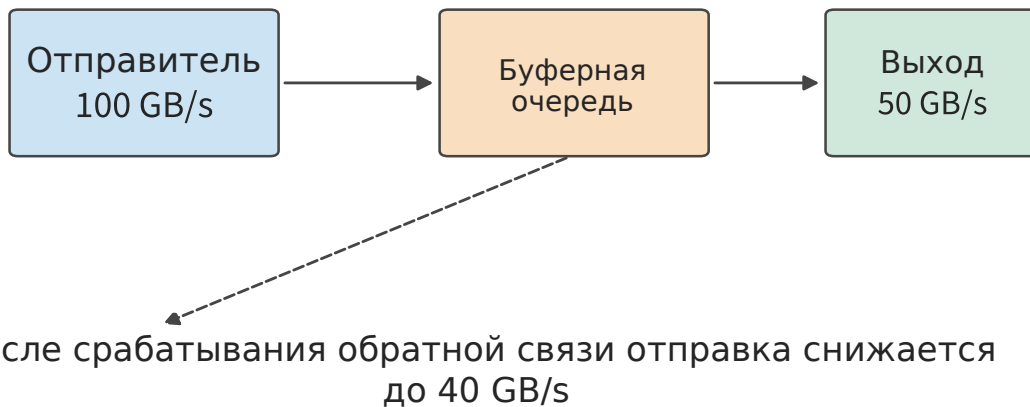


Рисунок 7-42. Данные по сплошной линии поступают в очередь и передаются через выходной канал; пунктирная линия обозначает обратную связь о перегрузке, возвращающуюся отправителю. До получения и применения обратной связи прежняя скорость отправки продолжает заполнять очередь.

ется со скоростью выходного канала, а уже существующая очередь не изменится. После снижения до 40 GB/s каждую секунду появляется запас 10 GB для опустошения очереди, поэтому для 512 KiB потребуется около 52 μ s. От начальной точки очередь вернётся к нулю примерно через 72 μ s. Управление перегрузкой должно не только останавливать рост очереди, но и оставлять запас пропускной способности для её опустошения. На рисунке 7-42 показан этот контур обратной связи, а на рисунке 7-43 — заполнение и опустошение буфера.

На рисунке 7-43 момент применения обратной связи определяет, как долго очередь остаётся заполненной, а величина снижения скорости — крутизну нисходящего участка. Все механизмы управления перегрузкой воздействуют на очередь именно посредством этих двух величин. **Явное уведомление о перегрузке (ECN)** позволяет коммутатору помечать пакеты, возвращая отправителю сведения о состоянии очереди; **DCQCN** — алгоритм, использующий такие уведомления для регулирования скорости передачи RDMA; методы на основе задержки выявляют образование очереди по изменению времени кругового обхода; **C-AQM** в UB обеспечивает совместное активное управление очередью конечными узлами и коммутационной сетью: отправитель замедляется на основании состояния очереди ещё до её переполнения. Чтобы понять эти механизмы, сначала нужно определить, какую часть пути обратной связи сокращает каждый из них и что именно меняет отправитель после получения информации. Пороговые значения, шаг обновления и частота обратной связи совместно определяют скорость и амплитуду реакции очереди.

Схождение «многие к одному» и управление потоком на уровне канала. В при-

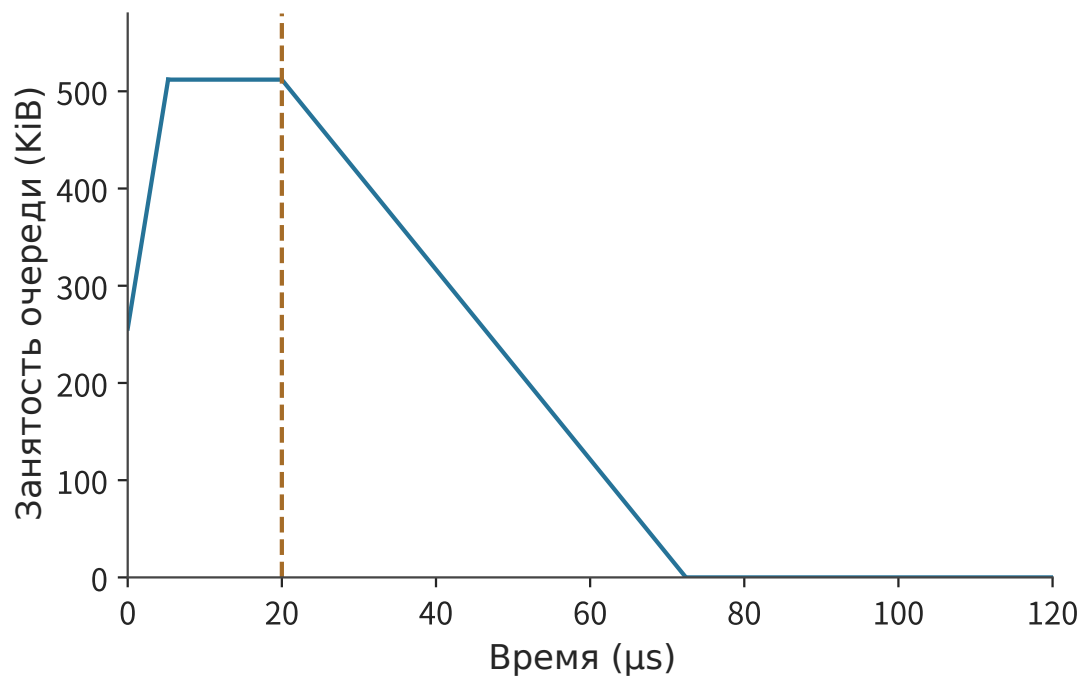


Рисунок 7-43. До применения обратной связи буфер продолжает заполняться. Скорость поступления равна 100 GB/s, скорость выходного канала — 50 GB/s, а буфер объемом 512 KiB изначально заполнен наполовину. На отметке 5,2 μs он заполняется полностью, и до снижения скорости на отметке 20 μs избыточные входящие данные продолжают отбрасываться; после снижения до 40 GB/s выходной канал опустошает очередь с чистой скоростью 10 GB/s. Время обратной связи и величина снижения скорости заданы условиями.

ведённом выше примере скорость поступления 100 GB/s создавалась двумя заданиями. Однако коллективные коммуникации сами по себе могут создавать ещё более резкие всплески. Если ReduceScatter реализован напрямую, каждый участник отправляет фрагмент j непосредственно его владельцу j , завершая операцию за один раунд. Если N участников поочерёдно отправляют данные целевым узлом в одинаковом порядке, причём каждый делает это на полной скорости сетевой карты B , то в период, когда очередь доходит до владельца j , остальные $N - 1$ отправителей одновременно передают данные ему, и ведущий к нему выход коммутатора получает данные со скоростью $(N - 1)B$. Если порядок отправки сместить по rank, каждый целевой узел в любой момент будет получать только один поток. Ситуация, когда несколько отправителей одновременно передают данные в один выходной канал, называется **incast**. Подставим $\lambda = (N - 1)B$ в приведённую выше формулу: каждый отправитель передаёт с $B = 50$ GB/s, выходной канал также имеет скорость 50 GB/s, а свободный буфер равен 1 MiB. При $N = 8$ скорость поступления составляет не 50 GB/s одного потока, а суммарные 350 GB/s семи потоков:³⁵

N	Число отправителей	Скорость поступления	Избыточная скорость	Допустимая задержка обратной связи
8	7	350 GB/s	300 GB/s	3,50 μ s
16	15	750 GB/s	700 GB/s	1,50 μ s
64	63	3150 GB/s	3100 GB/s	0,34 μ s

Управление потоком на уровне канала и сквозное управление перегрузкой соответствуют здесь двум расстояниям обратной связи. Одна из реализаций управления потоком на уровне канала — **управление потоком на основе приоритетов** (Priority-based Flow Control, PFC) из IEEE 802.1Qbb: когда очередь входного порта превышает порог, он отправляет соседнему вышестоящему порту кадр приостановки, останавливающий канал для конкретного приоритета. Расстояние обратной связи составляет всего один переход: для кабеля длиной 30 м при 5 ns/м кадр приостановки идёт вверх 150 ns, уже находящиеся в линии данные идут вниз 150 ns, а сериализация пакета размером 1500 В при 50 GB/s занимает ещё 30 ns — итого 0,33 μ s. Избыточные данные, продолжающие поступать в течение этих 0,33 μ s, должны поместиться в буфер. Зарезервированное для них пространство называется headroom, то есть буферным запасом: при $N = 8$ требуется $300 \text{ GB/s} \times 0,33 \mu\text{s} = 99 \text{ KB}$, при $N = 16$ — 231 KB, при $N = 64$ — 1,02 MB; все эти значения не превышают 1 MiB. В статье о DCQCN для максимального размера передаваемого блока (MTU) 1500 В приводится пример с 22,4 KB на порт для каждого приоритета. Для сквозного управления перегрузкой, то есть маркировки ECN и снижения скорости DCQCN, расстоянием обратной связи служит круговой обход длительностью 20 μ s. За эти 20 μ s объём поступивших избыточных данных составит соответственно 6,0, 14 и 62 MB, то есть 5,7, 13,4 и 59,1 MiB, и ни одно из этих значений не помещается в буфер размером 1 MiB. На рисунке 7-44 показаны рост очереди и два расстояния обратной связи для трёх значений N .

³⁵ Обратная связь incast, буфер 4 MiB: каждый из $N - 1$ отправителей передаёт со скоростью 50 GB/s на выход со скоростью 50 GB/s; используется гидродинамическая модель. Расстояние обратной связи в один переход складывается из времени распространения туда и обратно по кабелю длиной 30 м и времени сериализации одного пакета размером 1500 В; сквозное расстояние обратной связи принято равным 20 μ s туда и обратно. Определение PFC как приостановки полнодуплексного канала по классу трафика взято из раздела IEEE 802.1Qbb; MTU 1500 В и headroom 22.4 KB на каждый приоритет каждого порта взяты из статьи о DCQCN, §4.

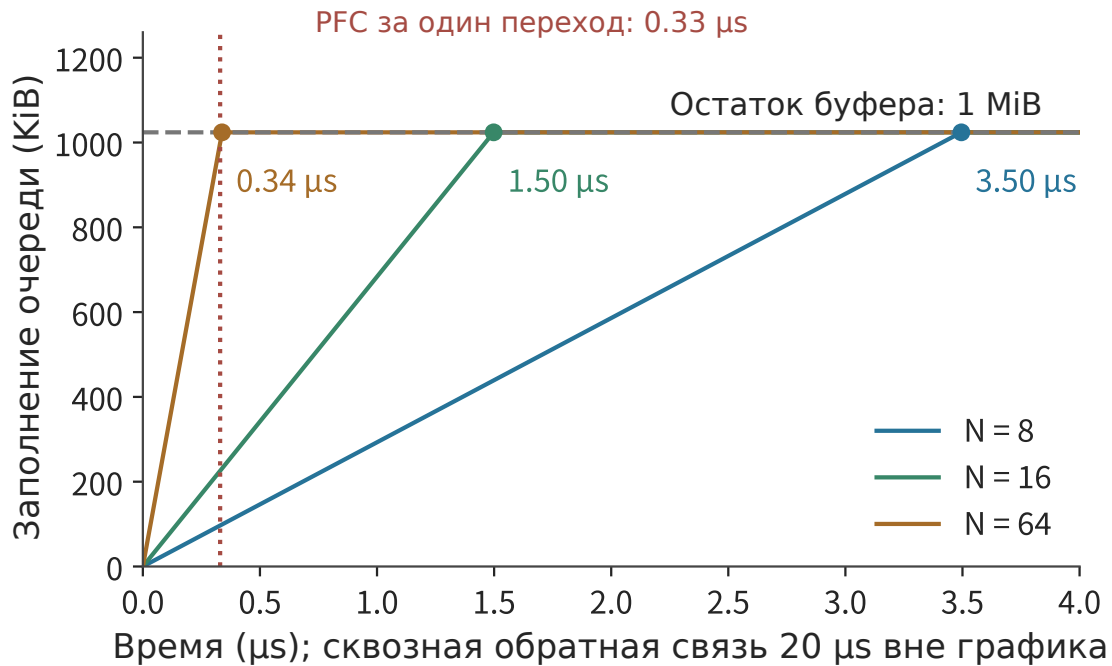


Рисунок 7-44. При $N = 8, 16$ и 64 очередь растёт с избыточной скоростью 300, 700 и 3100 GB/s; точками отмечены моменты заполнения свободного буфера объёмом 1 MiB: 3,50, 1,50 и 0,34 μs . Вертикальная пунктирная линия обозначает расстояние обратной связи PFC в один переход, равное 0,33 μs : до этого момента ни одна из трёх линий не достигает заполнения. Сквозная обратная связь длительностью 20 μs находится за пределами горизонтальной оси рисунка.

Два расстояния определяют распределение обязанностей: PFC за 0,33 μs останавливает соседний узел на расстоянии одного перехода, не позволяя буферу переполниться в течение 20 μs , необходимых DCQCN; затем DCQCN снижает скорость источника до уровня, который способен выдержать выходной канал, после чего приостановку можно снять. Условие работы без потери пакетов только за счёт сквозной обратной связи имеет вид $(N - 2) \times 50 \text{ GB/s} \times 20 \mu\text{s} \leq Q_{\text{free}}$: буфер размером 1 MiB допускает только $N \leq 3$, а 4 MiB — лишь $N \leq 6$. Если увеличить буфер до 4 MiB, допустимая задержка обратной связи составит 13,98, 5,99 и 1,35 μs , что по-прежнему намного меньше 20 μs .³⁶ Цена PFC рассматривается в разделе 7.5.4: приостановка может по переходам распространяться вверх по сети, а при образовании циклической зависимости возникает взаимная блокировка.

Эксперимент 7-11 · Продолжение: как размер буфера и величина снижения скорости влияют на устранение очереди

Пусть скорость поступления равна 100 GB/s, скорость выходного канала — 50 GB/s, а начальный объём накопившихся данных — 256 KiB. Найдите допустимую задержку обратной связи при общем размере буфера 512 KiB и 1 MiB. Предположите, что обратная связь применяется точно в момент заполнения

³⁶Обратная связь incast, буфер 4 MiB: каждый из $N - 1$ отправителей передаёт со скоростью 50 GB/s на выход со скоростью 50 GB/s; используется гидродинамическая модель. Расстояние обратной связи в один переход складывается из времени распространения туда и обратно по кабелю длиной 30 м и времени сериализации одного пакета размером 1500 В; сквозное расстояние обратной связи принято равным 20 μs туда и обратно. Определение PFC как приостановки полнодуплексного канала по классу трафика взято из раздела IEEE 802.1Qbb; MTU 1500 В и headroom 22.4 KB на каждый приоритет каждого порта взяты из статьи о DCQCN, §4.

буфера, а скорость поступления после замедления составляет соответственно 50, 45 и 40 GB/s. Найдите время от применения обратной связи до полного опустошения очереди и объясните, почему при одном из условий очередь опустошить невозможно.

Эксперимент 7-12 · Продолжение: буфер и расстояние обратной связи при incast

Измените скорость выходного канала на 100 GB/s, что соответствует двум сетевым картам, оставив скорость каждого отправителя равной 50 GB/s. Пересчитайте допустимую задержку обратной связи и headroom PFC для $N = 8$, 16 и 64. Затем увеличьте длину кабеля до 100 m, найдите расстояние обратной связи в один переход и определите, останется ли headroom при $N = 64$ в пределах 1 MiB. Наконец, для времени кругового обхода 5 μ s найдите максимальное N , при котором избежать потери пакетов можно только за счёт сквозной обратной связи.

7.5.3 Многопутевая передача и повторная передача

Управление по обратной связи снижает скорость отправки и тем самым ослабляет перегрузку. Если на других маршрутах остаётся свободная пропускная способность, трафик можно перенаправить туда. Сеть Clos предоставляет несколько возможных маршрутов. Распределение разных соединений по разным маршрутам позволяет рассредоточить трафик с перегруженных каналов. Распределение пакетов одной передачи по нескольким маршрутам позволяет одновременно использовать пропускную способность нескольких каналов, но также переносит на получателя различия в задержках этих маршрутов.

Возьмём BF16-векторы скрытых состояний для четырёх токенов общим объёмом 32 KiB, разделим их на восемь пакетов по 4 KiB и циклически распределим по двум независимым маршрутам со скоростью 50 GB/s. Сериализация каждого пакета занимает около 82 ns, а отправка четырёх пакетов по каждому маршруту — около 0,33 μ s. Если задержка распространения на обоих маршрутах равна 1 μ s, все данные будут доставлены примерно за 1,33 μ s; передача тех же восьми пакетов по одному маршруту потребует около 1,66 μ s. Каждый из двух маршрутов выполняет половину работы по отправке, и передача завершается параллельно.³⁷

Изменим задержку распространения на втором маршруте на 9 μ s. Последний пакет этого маршрута прибывает только примерно через 9,33 μ s. Последующие пакеты, раньше прибывшие по быстрому маршруту, должны ждать заполнения предшествующего пробела, а получателю потребуется буферизовать до 12 KiB данных, пришедших не по порядку. При том же общем числе байтов использование дополнительного маршрута оказывается намного медленнее передачи по одному маршруту, поскольку экономия времени сериализации составляет лишь около 0,33 μ s и намного меньше дополнительных 8 μ s задержки маршрута.

Горизонтальные линии на рисунках 7-45 и 7-46 показывают, что завершение передачи по быстрому маршруту ещё не означает, что все данные можно передать прило-

³⁷ Одинаковая задержка двух путей, разные задержки путей, восстановление после заданной потери пакета. Число байтов подсчитывается по полезной нагрузке приложения. Оба пути обеспечивают по 50 GB/s, размер пакета составляет 4 KiB, используются фиксированная полезная нагрузка и циклическое распределение; момент восстановления задан условием задачи. Для расчёта дополнительной полезной нагрузки используются 32 KiB данных, повторно отправляемых начиная с потерянного пакета.

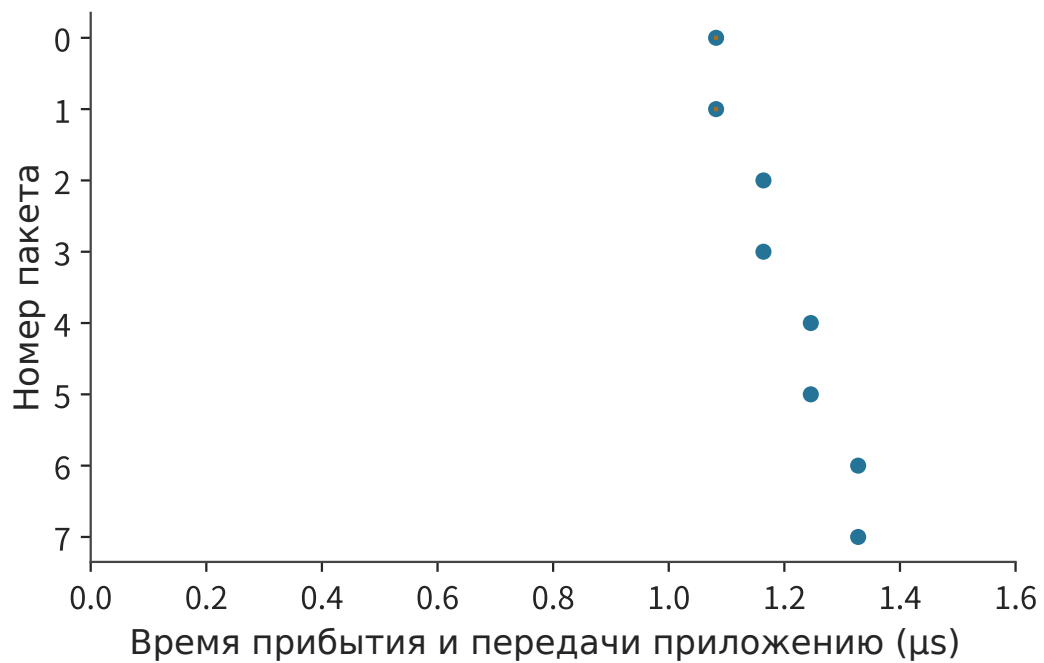


Рисунок 7-45. При задержке распространения $1 \mu s$ на обоих маршрутах все восемь пакетов прибывают примерно за $1,33 \mu s$. Точки обозначают прибытие, а горизонтальные линии — ожидание заполнения пробела после прибытия.

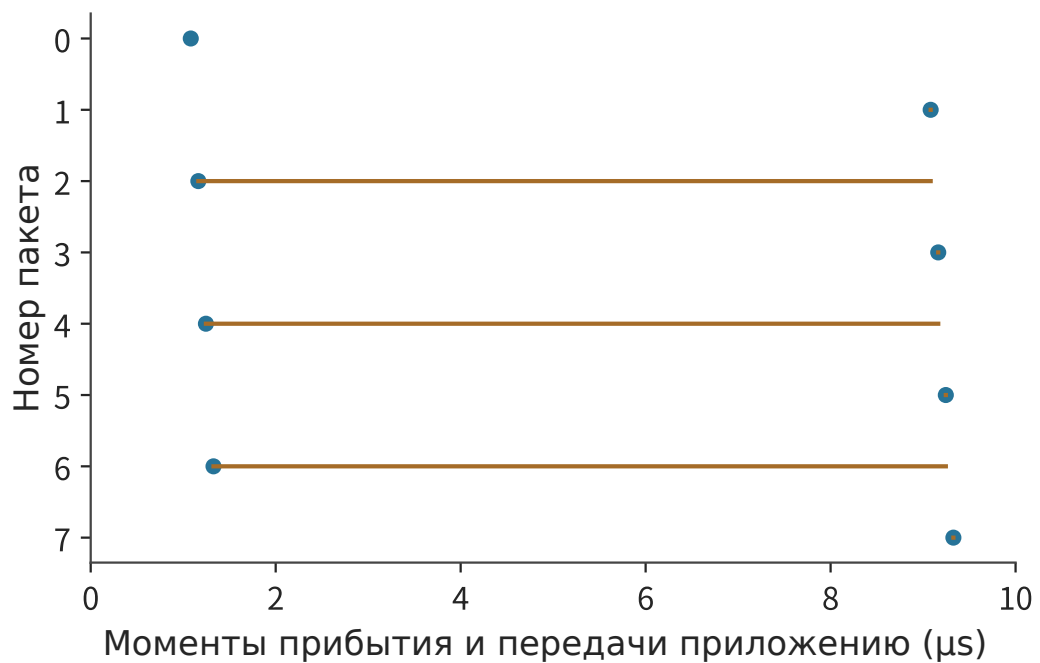


Рисунок 7-46. Задержка распространения на втором маршруте увеличена до $9 \mu s$. Последующие пакеты быстрого маршрута прибывают раньше, но всё равно ждут заполнения предшествующего пробела; все данные становятся доступны для доставки примерно через $9,33 \mu s$. Точки обозначают прибытие пакетов, горизонтальные линии — ожидание заполнения пробела после прибытия, вертикальная ось — номер пакета.

жению. Два маршрута сокращают время отправки, но могут увеличить время ожидания отсутствующих пакетов. Условие, при котором двухпутевая передача оказывается быстрее, можно вычислить напрямую. При одном маршруте необходимо отправить восемь пакетов, а при двух — по четыре пакета на каждом. Сериализация одного пакета занимает около 82 ns, поэтому два маршрута экономят примерно 0,33 μ s. Если дополнительная задержка распространения второго маршрута относительно первого превышает эти 0,33 μ s, равномерное распределение теряет преимущество по времени. В канале со скоростью 50 GB/s этот запас лишь немногим больше удвоенной задержки распространения по кабелю длиной 30 м, равной примерно 150 ns, поэтому даже небольшое различие между маршрутами сводит выигрыш на нет. Равномерно делить передачу объёмом в несколько десятков KiB между маршрутами с разными задержками обычно невыгодно. Выбор маршрута или неравномерное распределение должно уменьшать число пакетов на медленном маршруте, чтобы последние пакеты обоих маршрутов прибывали как можно ближе по времени. Многопутевая передача выгодна, только когда время сериализации выделенного каждому маршруту объёма намного превышает различие в задержках маршрутов. Рассматриваемое ниже по пакетное распыление 8 MiB относится именно к этому случаю.

Помимо медленного распространения возможна потеря пакетов, которая приводит к ожиданию другого характера. Пусть пакет с номером 0 потерян и повторно передаётся через 20 μ s после завершения его первоначальной отправки. Тогда все данные можно передать приложению лишь примерно через 21,2 μ s: доставка ждёт заполнения пробела и не зависит от того, насколько рано прибыли остальные пакеты. К этому моменту другие семь пакетов уже находятся у получателя, который буферизовал 28 KiB полезной нагрузки; повторно передать нужно только отсутствующие 4 KiB. Если же начиная с отсутствующего пакета повторно отправить его и все последующие пакеты, придётся снова передать 32 KiB. **Выборочная повторная передача** использует состояние, фиксирующее ход приёма, и благодаря этому уменьшает объём повторной передачи. На рисунке 7-47 показан случай, когда повторно отправляется только пакет с номером 0. Двухузловая симуляция OpenURMA демонстрирует такое же различие: транспортный уровень UB использует выборочное подтверждение и повторно отправляет только отсутствующие пакеты, поэтому пропускная способность плавно снижается с ростом доли потерянных пакетов; при возврате на N шагов с той же долей потерь приходится повторно передавать всё окно.³⁸

У повторной передачи есть ещё одна независимая проблема: когда считать, что пробел требует восстановления. Если ждать слишком долго, пауза после фактической потери будет большой. Если повторно передать пакет, который лишь задержался на медленном маршруте, возрастёт объём дублирующего трафика. Распределение задержек маршрутов определяет допустимую степень нарушения порядка при обнаружении потерь, а состояние, хранимое получателем, — объём данных, которые можно буферизовать после отсутствующего пакета. Надёжная доставка совместно с зависимостями приложений из раздела 7.4 определяет, какие уже прибывшие операции можно продолжать выполнять.

Конфликты хеширования на нескольких маршрутах. Выше пакеты одной передачи распределялись по нескольким маршрутам. Другой подход, упомянутый в начале

³⁸Bojie Li, «Размышления, лежащие в основе Unified Bus», разделы Jetty, порядка транзакций и Load/Store; статья OpenURMA, редакция от 2026-06-02 (arXiv:2605.28717), §3 — проектирование, §7–§9 — состояние и задержка, §10 — порядок, §12 — передача, §13 — сводка результатов. Источники и область применимости данной интеграции.

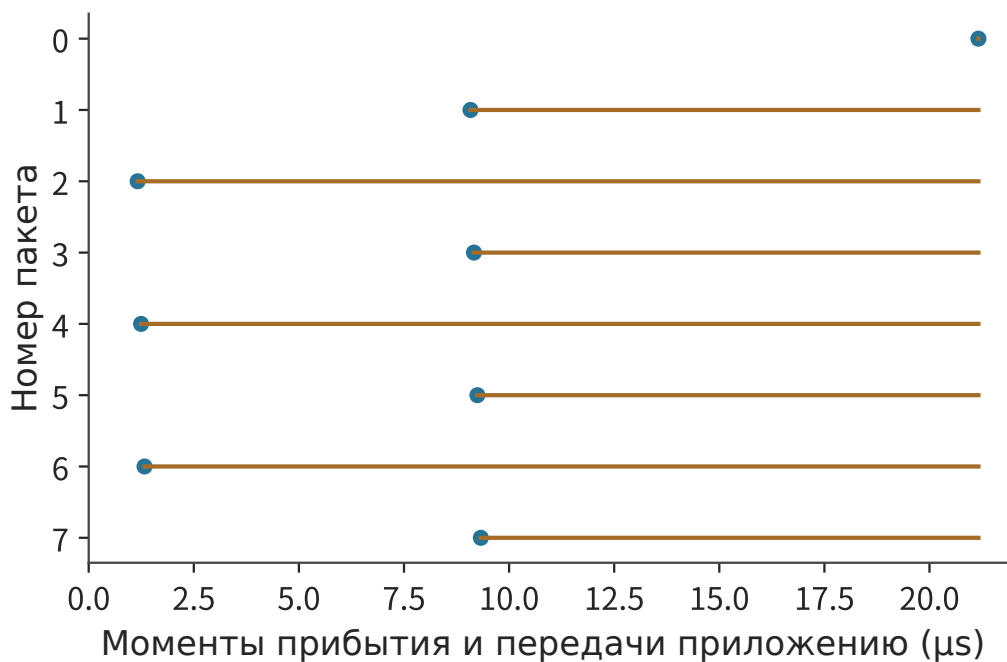


Рисунок 7-47. После потери пакета с номером 0 он повторно передаётся через заданное в примере время ожидания, а остальные семь пакетов сохраняются у получателя. После повторной отправки только отсутствующего пакета все данные становятся доступны для доставки примерно через 21,2 μs . Точки обозначают прибытие пакетов, горизонтальные линии — ожидание заполнения пробела после прибытия, вертикальная ось — номер пакета.

раздела, распределяет маршруты по соединениям, то есть потокам. Он не нарушает порядок пакетов, но несколько потоков могут оказаться на одном канале. Когда межгрупповые потоки покидают leaf-коммутатор, для каждого из них необходимо выбрать один из его восходящих каналов. Обычно коммутатор хеширует поля адресов и портов заголовка пакета и по результату выбирает восходящий канал. Этот механизм называется **многопутевой маршрутизацией с равной стоимостью** (equal-cost multipath, ECMP): все пакеты одного потока проходят по одному маршруту, а разные потоки распределяются по разным восходящим каналам в соответствии с хешем. Хеширование носит случайный характер, поэтому вероятность попадания двух потоков на один восходящий канал ненулевая. Конфликтовать друг с другом могут межгрупповые потоки одного leaf-коммутатора. Хотя смещённое сопоставление из раздела 7.2.5 направляет восемь потоков на уровень spine, они выходят из восьми разных leaf-коммутаторов, на каждом из которых есть только один межгрупповой поток, поэтому на восходящих каналах одного leaf-коммутатора они не встретятся. Выравнивание rail сводит к минимуму число межгрупповых потоков на каждом leaf-коммутаторе именно для предотвращения таких конфликтов. Пусть n потоков одинаковой скорости на одном leaf-коммутаторе независимо и равномерно хешируются на его m восходящих каналов: у неблокирующего leaf-коммутатора при $k = 64$ их 32, а у leaf-коммутатора с переподпиской 3:1 — 16. Обозначим через $L_{\max}$ число потоков на самом загруженном восходящем канале. При отсутствии конфликтов самый загруженный восходящий канал несёт $\lceil n/m \rceil$ потоков. Если потоки справедливо делят пропускную способность своего канала, а вся группа вынуждена ждать завершения самого медленного потока, её скорость составляет лишь

$\lceil n/m \rceil / E[L_{\max}]$ от скорости без конфликтов. Распределение максимальной нагрузки вычисляется точно; r_{99} в таблице — 99-й процентиль $L_{\max}$, то есть в 99% результатов хеширования число потоков на самом загруженном восходящем канале не превышает это значение.³⁹

³⁹ 8 потоков и 32 восходящих канала, 8 потоков и 16 восходящих каналов, 32 потока и 32 восходящих канала, 32 потока и 16 восходящих каналов, 128 потоков и 16 восходящих каналов: каждый поток на одном leaf-коммутаторе независимо и равномерно хешируется на один из его восходящих каналов. Распределение максимальной нагрузки точно вычисляется по усечённому экспоненциальному многочлену; скорость относительно случая без коллизий равна $\lceil n/m \rceil$, делённому на математическое ожидание максимальной нагрузки (при $n \geq m$ это эффективный cut-set в файле результатов). При по пакетному распределении используется принятая в этом разделе модель изменения порядка пакетов: 8 MiB разбиваются на пакеты по 4 KiB, которые циклически распределяются по восьми каналам со скоростью 50 GB/s; задержки каналов от 1 до 8 μ s заданы во входных данных. Попакетное распределение и значения энтропии взяты из спецификации Ultra Ethernet v1.0.1, §3.6.5.2, а стоимость отката на N шагов — из обзора UEC, страница 5.

Число потоков n	Число восходящих каналов m	Число потоков на самом загруженном восходящем канале без конфликтов	$E[L_{\max}]$	p99	Вероятность отсутствия конфликтов	Скорость относительно варианта без конфликтов
8	32	1	1,66	3	38,6%	60,1%
8	16	1	2,06	4	12,1%	48,5%
32	32	1	3,53	6	1.8×10^{-13}	28,3%
32	16	2	4,83	8	0	41,4%
128	16	8	13,36	18	0	59,9%

Если на одном leaf-коммутаторе есть только восемь межгрупповых потоков, вероятность полного отсутствия конфликтов на неблокирующем leaf-коммутаторе составляет 38,6%. При возникновении конфликта два потока попадают на один восходящий канал и получают по половине его пропускной способности. На самый загруженный восходящий канал в среднем приходится 1,66 потока, поэтому средняя скорость всей группы составляет лишь 60,1% от скорости без конфликтов. Если число восходящих каналов leaf-коммутатора с переподпиской уменьшено вдвое, вероятность отсутствия конфликтов падает до 12,1%, а скорость — до 48,5%. Хуже всего ситуация, когда каждый из 32 серверов группы создаёт один межгрупповой поток, а неблокирующий leaf-коммутатор имеет ровно 32 восходящих канала: поскольку число потоков равно числу каналов, практически неизбежно, что некоторые восходящие каналы останутся свободными, а на других скопятся по три-четыре потока; скорость всей группы снизится до 28,3%. Если потоков намного больше, чем восходящих каналов, относительные колебания случайного распределения уменьшаются: для 32 потоков на 16 восходящих каналах скорость составляет 41,4%, а для 128 потоков на 16 каналах — 59,9%. На рисунке 7-48 показана нагрузка самого загруженного канала для этих конфигураций. **Условие смены характера зависимости:** цена конфликтов максимальна, когда число потоков близко к числу восходящих каналов, и меньше, когда потоков намного меньше или намного больше каналов. Избежать этой цены можно двумя способами: использовать выравнивание rail, уменьшая число межгрупповых потоков на одном leaf-коммутаторе, либо отказаться от распределения по потокам в пользу описанного ниже по пакетному распылению.

Попакетное распыление. Распределение пакетов одного потока по нескольким маршрутам устраняет конфликты хеширования потоков ценой нарушения порядка, рассчитанного в начале этого раздела. Разделим prefill-активации объёмом 8 MiB, то есть BF16-векторы скрытых состояний для 1024 токенов, на 2048 пакетов по 4 KiB и циклически распределим их по восьми маршрутам со скоростью 50 GB/s и задержками распространения от 1 до 8 μ s. Каждый маршрут передаёт 1 MiB, сериализация которого занимает 20,97 μ s; с учётом дополнительных 8 μ s на самом медленном маршруте все данные прибывают через 28,97 μ s, или приблизительно 29,0 μ s. Для передачи тех же 8 MiB по одному маршруту требуется 167,8 μ s на сериализацию и 1 μ s на распространение — всего 168,8 μ s. Распыление экономит время сериализации, но добавляет различие в задержках маршрутов. Согласно условию из начала раздела распыление будет быстрее, пока задержка самого медленного маршрута меньше $168,8 - 21,0 = 147,8 \mu$ s; здесь она составляет всего 8 μ s. Цена нарушения порядка состоит в необходимости буферизовать раньше прибывшие пакеты: пакеты быстрого маршрута ждут пакеты с меньшими номерами на медленном маршруте, поэтому получатель одновременно хранит до 339 пакетов, или 1,39 MB.⁴⁰

Ultra Ethernet, спецификация Ethernet для AI и высокопроизводительных вычислений, включает оба этих механизма в транспортный уровень: пакеты распыляются по

⁴⁰8 потоков и 32 восходящих канала, 8 потоков и 16 восходящих каналов, 32 потока и 32 восходящих канала, 32 потока и 16 восходящих каналов, 128 потоков и 16 восходящих каналов: каждый поток на одном leaf-коммутаторе независимо и равномерно хешируется на один из его восходящих каналов. Распределение максимальной нагрузки точно вычисляется по усечённому экспоненциальному многочлену; скорость относительно случая без коллизий равна $\lceil n/m \rceil$, делённому на математическое ожидание максимальной нагрузки (при $n \geq m$ это эффективный cut-set в файле результатов). При по пакетному распределении используется принятая в этом разделе модель изменения порядка пакетов: 8 MiB разбиваются на пакеты по 4 KiB, которые циклически распределяются по восьми каналам со скоростью 50 GB/s; задержки каналов от 1 до 8 μ s заданы во входных данных. Попакетное распределение и значения энтропии взяты из спецификации Ultra Ethernet v1.0.1, §3.6.5.2, а стоимость отката на N шагов — из обзора UEC, страница 5.

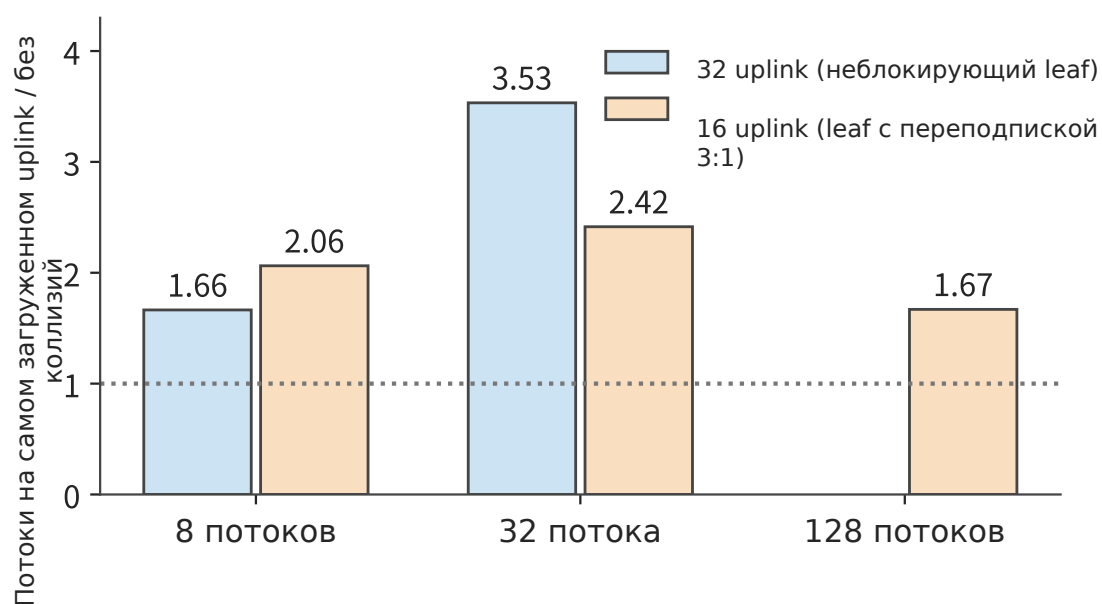


Рисунок 7-48. Когда n межгрупповых потоков одного leaf-коммутатора хешируются на его m восходящих каналов, величина, обратная отношению ожидаемого числа потоков на самом загруженном восходящем канале к числу потоков на нём при отсутствии конфликтов, равна скорости всей группы относительно варианта без конфликтов. Непроницаемый leaf-коммутатор имеет 32 восходящих канала, а leaf-коммутатор с переподпиской 3:1 — 16. При распределении 32 потоков по 32 восходящим каналам отношение максимально и равно 3,53; при распределении 128 потоков по 16 восходящим каналам оно снижается до 1,67.

нескольким маршрутам с помощью значений энтропии (entropy — поле заголовка пакета, по которому коммутатор хешированием выбирает маршрут); типичные конфигурации используют от 64 до 256 значений энтропии, а когда обратная связь указывает на перегрузку маршрута, на него направляется меньше пакетов. Потерянные пакеты восстанавливаются с помощью выборочного подтверждения: повторно передаются только отсутствующие пакеты, а не всё окно с возвратом на N шагов.⁴¹ Это в точности соответствует модели на рисунке 7-47: распыление предоставляет пропускную способность всех каналов, выборочная повторная передача ограничивает цену потери отсутствующими пакетами, а получатель ради этого хранит состояние для обработки нарушения порядка.

Эксперимент 7-13 · Продолжение: как различия в задержках маршрутов и повторная передача откладывают доставку по порядку

Изменяйте задержку распространения на втором маршруте от 1 до 9 μs и найдите её значение, при котором время завершения при равномерном распределении по двум маршрутам равно времени завершения при использовании только первого маршрута. Затем распределите пять пакетов на первый маршрут и три на второй и найдите время завершения, если задержка распространения первого маршрута равна 1 μs , а второго — 9 μs . Сохранив условие потери первого пакета, увеличьте время ожидания перед восстановлением с 20 до 40 μs и вычислите момент, когда данные впервые можно будет доставить по порядку, а также объём данных, который к этому моменту потребуются буферизовать.

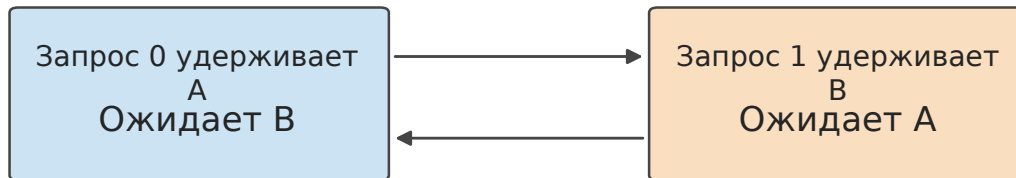
Эксперимент 7-14 · Продолжение: границы конфликтов хеширования и распыления

Найдите ожидаемую нагрузку самого загруженного канала и скорость относительно варианта без конфликтов при распределении 16 потоков по 32 восходящим каналам. Сравните результат с приведённым в таблице распределением восьми потоков по 16 восходящим каналам, имеющим такое же $n/m = 1/2$. Затем задайте задержки маршрутов от 1 до 30 μs с равными интервалами и найдите время завершения и пиковый объём буфера для пакетов, пришедших не по порядку. После этого найдите задержку самого медленного маршрута, при которой распыление перестаёт быть быстрее одного маршрута. Наконец, измените размер пакета на 1 KiB и объясните, как изменятся число пакетов и объём в байтах при пиковом нарушении порядка.

7.5.4 Взаимная блокировка

В разделе 7.5.3 уже прибывшие пакеты приходилось хранить до заполнения пробела, прежде чем освободить пространство. Противодействие конечных узлов и управление

⁴¹8 потоков и 32 восходящих канала, 8 потоков и 16 восходящих каналов, 32 потока и 32 восходящих канала, 32 потока и 16 восходящих каналов, 128 потоков и 16 восходящих каналов: каждый поток на одном leaf-коммутаторе независимо и равномерно хешируется на один из его восходящих каналов. Распределение максимальной нагрузки точно вычисляется по усечённому экспоненциальному многочлену; скорость относительно случая без коллизий равна $\lceil n/m \rceil$, делённому на математическое ожидание максимальной нагрузки (при $n \geq m$ это эффективный cut-set в файле результатов). При по пакетному распределении используется принятая в этом разделе модель изменения порядка пакетов: 8 MiB разбиваются на пакеты по 4 KiB, которые циклически распределяются по восьми каналам со скоростью 50 GB/s; задержки каналов от 1 до 8 μs заданы во входных данных. По пакетное распределение и значения энтропии взяты из спецификации Ultra Ethernet v1.0.1, §3.6.5.2, а стоимость отката на N шагов — из обзора UEC, страница 5.



Каждый ждёт, пока другой освободит ресурс

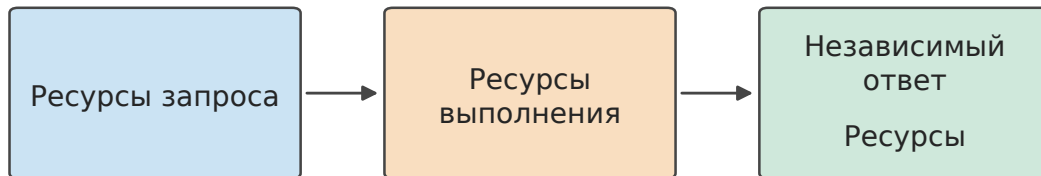
Рисунок 7-49. Два запроса удерживают соответственно А и В, причём каждый ожидает ресурс, удерживаемый другим. Стрелки обозначают отношения ожидания; ни один запрос не может завершиться и освободить ресурс.

поток на уровне канала также могут распространять такое ожидание вверх по сети: если у получателя нет свободного места, вышестоящий узел не может продолжать передачу. Если действие, освобождающее ресурс, само требует этого ресурса, возникает цикл. Рассмотрим два запроса, занимающих ресурсы А и В. Для завершения первому запросу необходимо получить В, а второму — А. Каждый ждёт, пока другой первым освободит ресурс.

В системе редукции такой цикл может охватывать несколько уровней: запросы заполняют приёмный буфер, ответ о завершении ждёт очереди отправки, а очередь отправки, в свою очередь, ждёт освобождения удалённым узлом пространства, занятого запросом. Если изобразить связь «удерживает предыдущий ресурс и запрашивает следующий» в виде ребра, получится граф зависимостей ресурсов. Когда каждый ресурс цикла удерживается, все условия освобождения одновременно перестают выполняться. На рисунке 7-49 показан такой цикл, а на рисунке 7-50 — описанное ниже решение с резервированием пути для ответа.

Один из способов решения — установить строгий порядок запроса ресурсов: например, все задания сначала запрашивают А, а затем В. Тогда задание, удерживающее В, уже не возвращается к ожиданию А, и цикл разрывается. Другой способ — зарезервировать для ответа о завершении отдельный буфер и возможность отправки. Пусть у получателя заполнены все восемь слотов данных, а отправитель ждёт их освобождения. Если ответ тоже должен занимать эти слоты данных, всё остановится. Добавление отдельного слота ответа позволяет вернуть сообщение об освобождении после обработки данных, благодаря чему освобождается пространство хотя бы одного запроса и очередь данных может продолжить работу.

Виртуальные каналы разделяют ресурсы с помощью логически обособленных очере-



Для ответа зарезервирован путь: он возвращается и освобождает исходный запрос

Рисунок 7-50. Запрос, выполнение и ответ используют независимые ресурсы, запрашиваемые в установленном порядке. Для ответа зарезервированы буфер и возможность отправки, поэтому он может вернуться и освободить исходный запрос.

дей. Если направить запросы и ответы в разные каналы, а затем установить направление запроса ресурсов, можно построить ациклическую структуру зависимостей. Реальный эффект здесь обеспечивают независимая ёмкость и порядок использования.

При обращении к памяти также может потребоваться преобразование адреса и чтение таблицы страниц. Если исходный запрос ждёт преобразования адреса, а для преобразования требуется ещё одно удалённое обращение, они могут образовать такой же цикл, конкурируя за один и тот же исчерпанный набор слотов. При участии автора в проектировании UB приходилось совместно анализировать зависимости между обработкой транзакций, обращениями к памяти и передачей по каналам. Только резервирование ресурсов для уведомлений о завершении, преобразования адресов и восстановления после сбоев позволяет предотвратить остановку системы при перегрузке или отказе.

7.6 От совершенствования коммуникации к выполнению задачи

7.6.1 Критический путь начинается с готовности данных

В разделах 7.2–7.5 передача, ожидание, уведомление о завершении и освобождение пространства прослеживались на примере одного набора данных. Чтобы определить, сколько времени обучения позволят сэкономить эти улучшения, необходимо также учесть вычисления, создающие и использующие эти данные, и установить момент начала коммуникации. Одна карта может очень рано войти в вызов коллективной коммуникации, но, если другая карта ещё не вычислила градиент, первая будет ждать её внутри вызова. Это ожидание учитывается во времени выполнения коммуникационного вызова, хотя

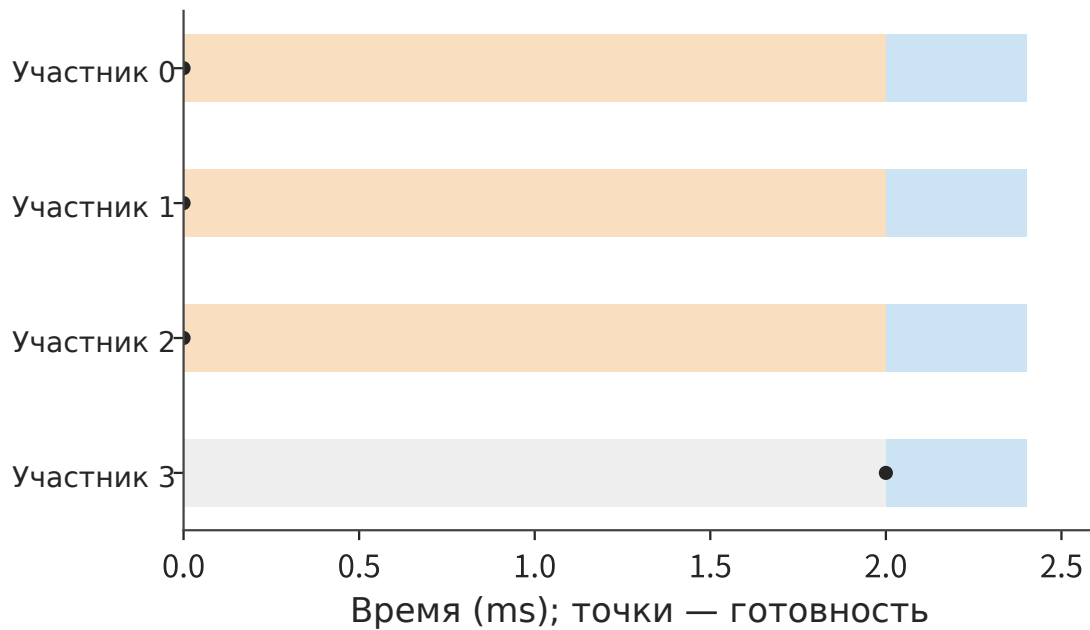


Рисунок 7-51. В исходном расписании первые три участника ждут, пока четвёртый будет готов через 2 ms, после чего выполняют обмен длительностью 0.4 ms. Серым показано состояние до готовности, оранжевым — ожидание других участников, синим — обмен.

его первопричина находится в предшествующих вычислениях или подготовке данных.

Рассмотрим четырёх участников, которым необходимо выполнить локальную редукцию и которые готовы соответственно в моменты 0, 0, 0 и 2 ms. После готовности всех участников выполняется обмен длительностью 0.4 ms. Согласно графу зависимостей из раздела 7.4, момент начала обмена равен максимальному из четырёх моментов готовности, поэтому вся группа завершает работу через 2.4 ms. Если сократить длительность обмена вдвое, время завершения составит 2.2 ms; если все четыре участника будут готовы в начальный момент, оно составит 0.4 ms. Эти две оптимизации воздействуют на разные узлы графа зависимостей; три соответствующих случая последовательно показаны на рисунках 7-51–7-53.⁴²

Такая рассинхронизация готовности встречается и в крупномасштабных системах обучения. При диагностике производительности обучения система крупномасштабного распределённого обучения MegaScale обнаружила, что пропускная способность сети оставалась стабильной, однако разница во времени начала коммуникации между участниками постоянно увеличивалась, а вместе с ней росло и ожидание в ReduceScatter. Дальнейший анализ показал, что эта разница связана с операциями хоста во время прямого прохода.⁴³ Измерения для четырёх процессов, выполненные на CPU с помощью библиотеки коллективной коммуникации Gloo в сопутствующем эксперименте к этой книге, также показывают ту же зависимость: при входных данных размером 4 KiB, когда один участник опаздывает примерно на 25.1 ms, медиана времени завершения всей группы возрастает приблизительно с 1.6 ms до 26.6 ms; хвостовой участок после

⁴² Учебные записи разброса времени готовности, 100 смешанных записей, две записи восстановления. За p99 принимается элемент с номером $\lceil 0.99N \rceil$ в отсортированной последовательности.

⁴³ Jiang и др., *MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs*, NSDI 2024; официальная статья, диагностика коллективных операций.

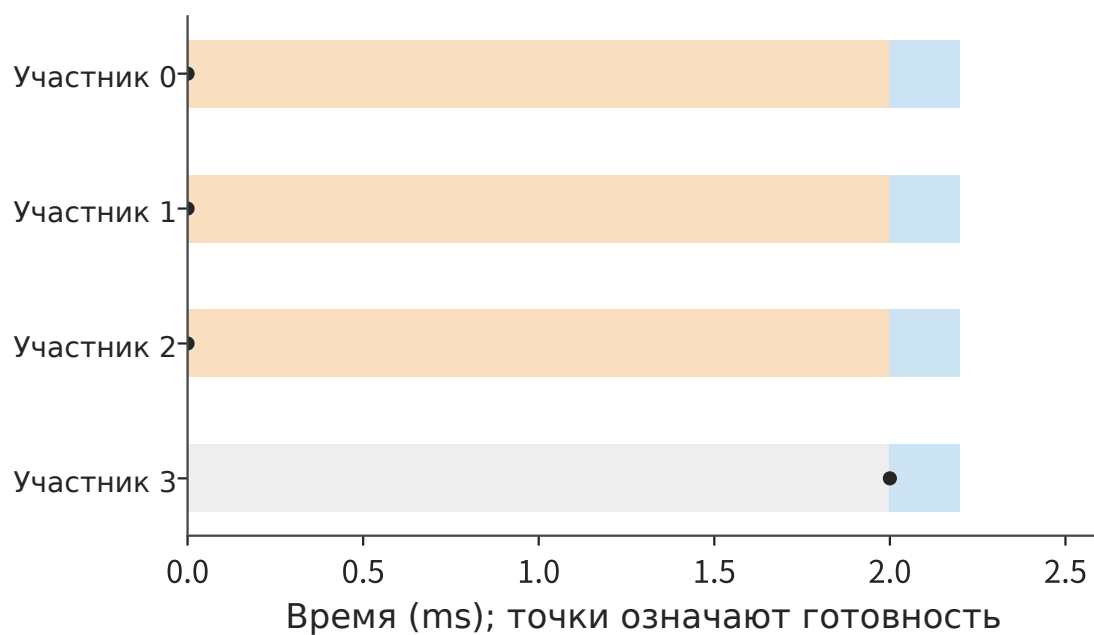


Рисунок 7-52. Моменты готовности не изменились, длительность обмена сократилась с 0.4 ms до 0.2 ms, поэтому вся группа завершает работу через 2.2 ms вместо 2.4 ms. Серым показано состояние до готовности, оранжевым — ожидание других участников, синим — обмен.

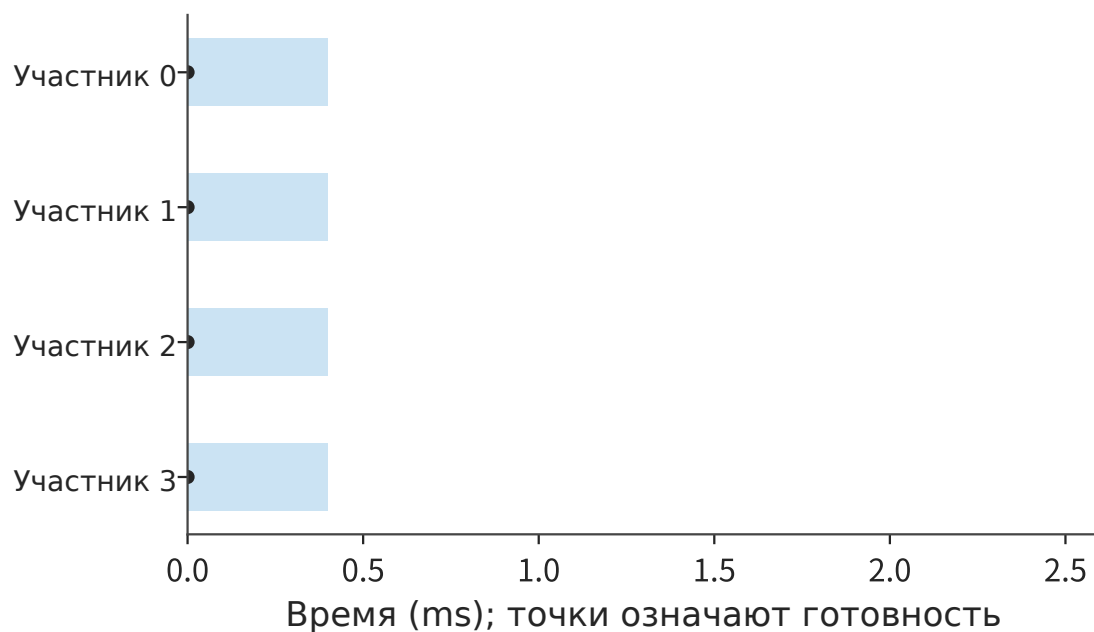


Рисунок 7-53. Все четыре участника одновременно готовы в начальный момент, обмен по-прежнему занимает 0.4 ms, и вся группа завершает работу через 0.4 ms. Масштаб временной оси одинаков на всех трёх рисунках. Синим показан обмен; каждая строка соответствует одному участнику.

прибытия последнего участника по-прежнему занимает около 1.6 ms.⁴⁴

Поэтому временную шкалу задачи следует строить начиная с момента создания данных. Градиенты разделяются на несколько бакетов, и редукцию каждого бакета можно начинать сразу после вычисления входящих в него градиентов; пока вычисляется следующий бакет, предыдущий может передаваться. Коммуникации разных бакетов конкурируют за сетевую карту и каналы связи, а обновление параметров должно ждать завершения редукции последнего бакета. Разделение на бакеты позволяет начать коммуникацию раньше и выполнять её одновременно с последующими вычислениями градиентов.

7.6.2 Расчёт времени обмена в иерархической сети

Пример 7.6. Что следует оптимизировать в первую очередь: алгоритм редукции, порядок выполнения или пропускную способность канала? Используем градиент размером 192 MiB из предыдущего примера. Градиент готов через 20 ms после начала вычислений, а обновление после редукции занимает 2 ms. Сравним плоское кольцо с иерархической редукцией, а затем рассмотрим недостаточный параллелизм, раннее начало обмена и деградацию каналов.

Решение. Сначала вычислим длительность шага обучения при последовательном выполнении редукции и вычислений. Вычисление → редукция → обновление образуют последовательную цепочку. Согласно модели передачи из раздела 7.2, общее время для плоского кольца составляет приблизительно $20 + 7.6 + 2 = 29.6$ ms, а для иерархической редукции — около $20 + 1.3 + 2 = 23.3$ ms. Время обмена уменьшается примерно на 83%, но длительность всего шага — лишь приблизительно на 21%. Остальное время занимают вычисление длительностью 20 ms и обновление длительностью 2 ms.

В общем случае, если время, не затрагиваемое оптимизацией, равно T_c , время обмена — T_n , а обмен ускоряется в s раз, то общее ускорение составляет

$$S = \frac{T_c + T_n}{T_c + T_n/s}.$$

Теперь рассмотрим ограничение числа параллельных запросов из раздела 7.3. Если на каждой сетевой карте одновременно находятся в обработке только 128 транзакций по 256 B, а время оборота слота составляет 2 μs, то без учёта скорости отправки удалённая пропускная способность каждой сетевой карты ограничена 16.4 GB/s. При иерархической редукции каждая сетевая карта передаёт между серверами в общей сложности 24 MiB, поэтому одна только удалённая передача занимает около 1.5 ms. С учётом локального обмена и запуска общее время обмена составляет около 2.3 ms, а длительность всего шага — приблизительно 24.3 ms. Чтобы вернуться примерно к 23.3 ms, необходимо увеличить число активных транзакций до 391 и обеспечить достаточную скорость отправки. После замены алгоритма редукции нужно также добиться такой скорости отправки и обработки запросов, которая позволит реализовать преимущества этого алгоритма.

⁴⁴experiments/ch07/07-10/rank-readiness: разброс времени готовности четырёх процессов Gloo. Apple M2 Max, локальный CPU, 180 групп записей основного эксперимента; время завершения всей группы измеряется от возврата из самого раннего барьера до возврата самого позднего вызова, поэтому хвост после прибытия последнего участника всё ещё включает редукцию, планирование и пробуждение. В основном тексте используется медиана статистик по различным выборкам; временная шкала каждой выборки приведена в исходных записях.

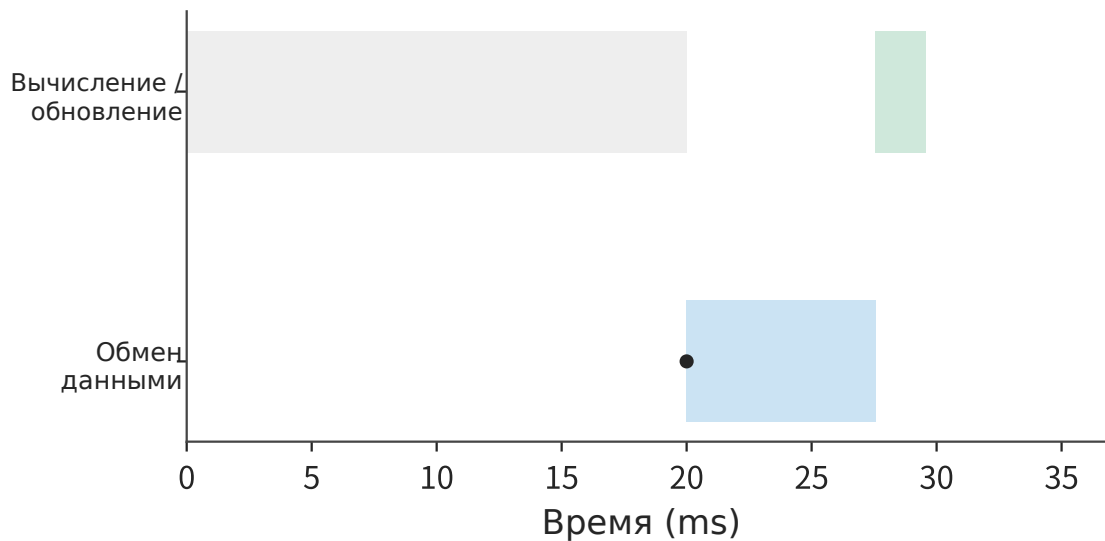


Рисунок 7-54. Последовательное выполнение с плоским кольцом: вычисление — 20 ms, обмен — около 7.6 ms, обновление — 2 ms. Серым показано вычисление, синим — обмен, зелёным — обновление; точка обозначает готовность данных для обмена.

Теперь рассмотрим перекрытие. Предположим, что через 17 ms после начала вычислений данные для обмена уже готовы, а остальные вычисления требуют ещё 3 ms. Эти 3 ms вычислений можно выполнять одновременно с обменом. Они используют независимые аппаратные ресурсы, не конкурирующие друг с другом, а обновление должно дождаться завершения обоих процессов. Тогда

$$T_{\text{step}} = \max(20, 17 + T_n) + 2 \text{ ms}.$$

На рисунках 7-54–7-59 последовательно показаны шесть вариантов организации выполнения, причём во всех используется одинаковый масштаб времени. Серый интервал вычислений остаётся неизменным, длина синего интервала обмена определяется объёмом передачи и пропускной способностью, а его начало — моментом готовности данных.

Если данные готовы через 17 ms и обмен перекрывается с вычислениями, весь шаг с плоским кольцом занимает около 26.6 ms, а с иерархической редукцией — приблизительно 22.0 ms. Иерархический обмен завершается примерно через 18.3 ms, то есть раньше, чем заканчиваются вычисления через 20 ms, поэтому обновление начинается сразу после них. Даже если данные будут готовы не позднее чем через 18.7 ms, иерархический обмен всё ещё полностью перекроется вычислениями. Дальнейшее сокращение времени обмена уже не изменит этот критический путь: для дальнейшего уменьшения общего времени потребуется ускорить вычисление или обновление.

Обсуждение. Станет ли редукция снова узким местом шага обучения после сокращения числа сетевых карт? Если из восьми линий rail останется доступна только одна, все восемь фрагментов каждого сервера придётся передавать через оставшуюся сетевую карту. В каждом раунде в каждом направлении передаётся 96 MiB, а за два раунда — в общей сложности 192 MiB, поэтому время удалённой передачи увеличивается приблизительно с 0.5 до 4.0 ms, а всё время обмена — примерно до 4.8 ms. Если данные готовы через 17 ms, обмен завершается только через 21.8 ms, а весь шаг занимает около

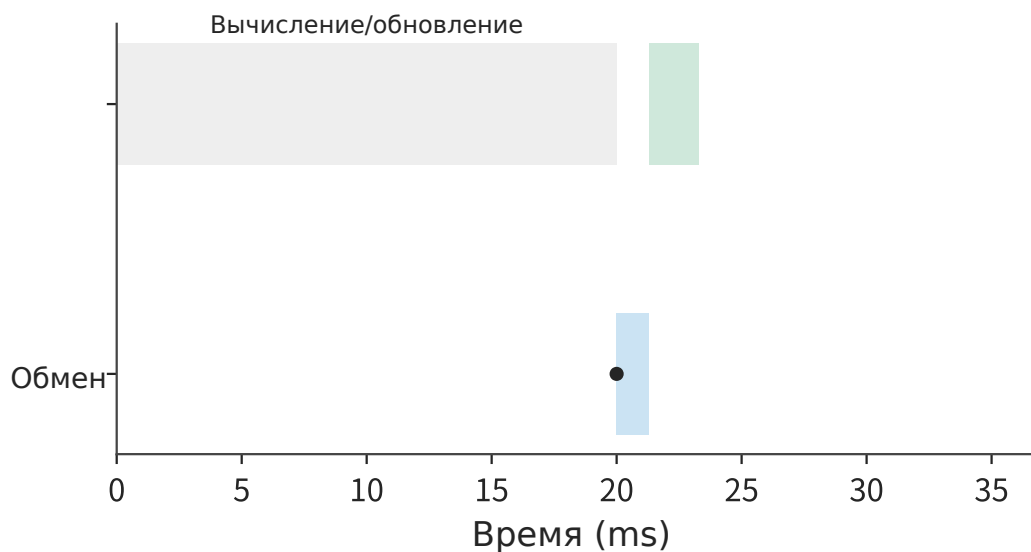


Рисунок 7-55. Здесь используется иерархическая редукция, но вычисление, обмен и обновление по-прежнему выполняются последовательно. Обмен сокращается приблизительно до 1.3 ms, а весь шаг занимает около 23.3 ms. Серым показано вычисление, синим — обмен, зелёным — обновление; точка обозначает готовность данных для обмена.

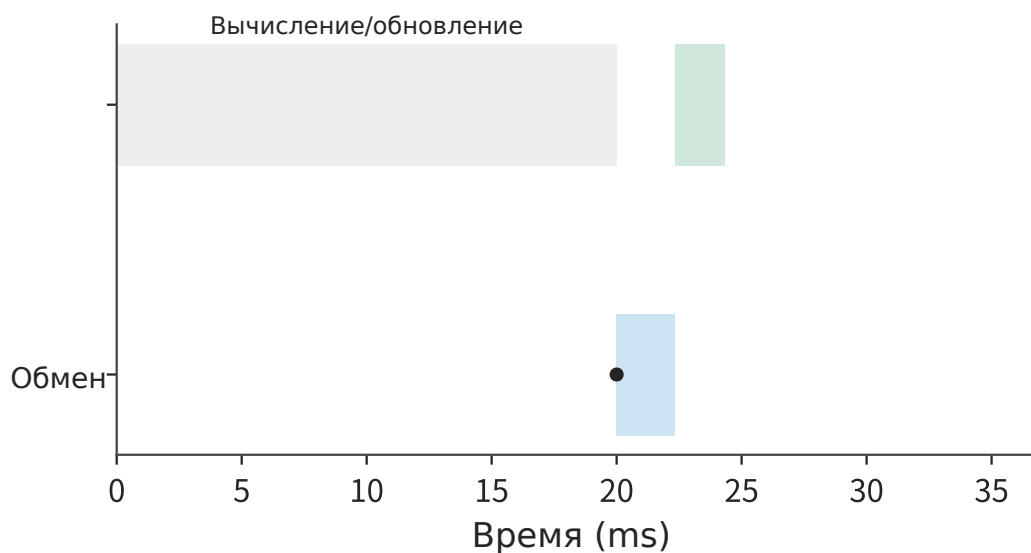


Рисунок 7-56. Здесь используется иерархическая редукция, а длительность вычислений и обновления остаётся неизменной. Ограничение в 128 транзакций, одновременно обрабатываемых каждой сетевой картой, снижает удалённую пропускную способность, из-за чего обмен растягивается приблизительно до 2.3 ms, а весь шаг — до 24.3 ms. Серым показано вычисление, синим — обмен, зелёным — обновление; точка обозначает готовность данных для обмена.

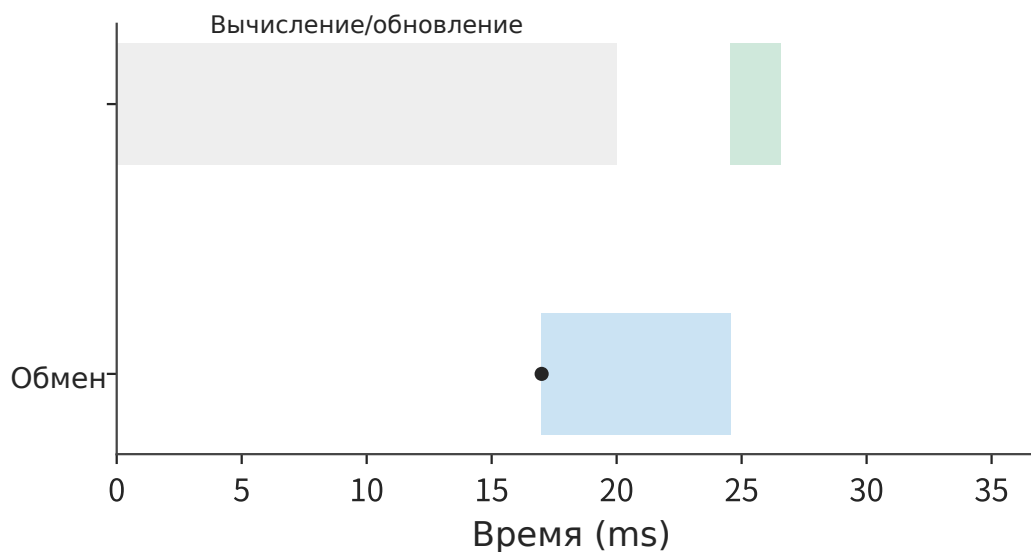


Рисунок 7-57. Здесь обмен по плоскому кольцу начинается раньше, через 17 ms, и перекрывается с ещё не завершившимися вычислениями. Обмен длительностью 7.6 ms выходит за пределы интервала вычислений, поэтому обновление ожидает его завершения; весь шаг занимает около 26.6 ms. Серым показано вычисление, синим — обмен, зелёным — обновление; точка обозначает готовность данных для обмена.

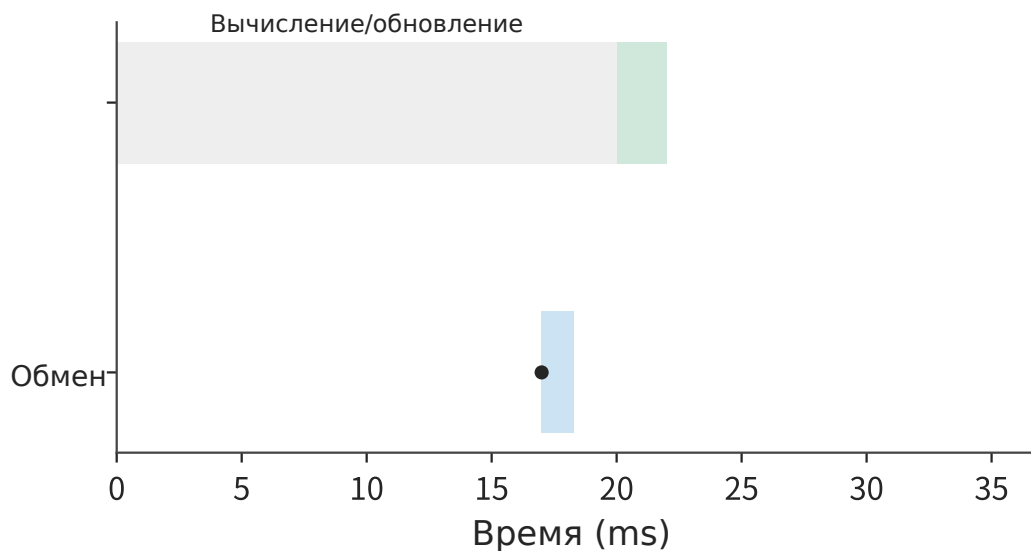


Рисунок 7-58. Здесь обмен для иерархической редукции начинается раньше, через 17 ms. Он завершается через 18.3 ms, раньше вычислений, поэтому обновление начинается через 20 ms, а весь шаг занимает 22 ms. Серым показано вычисление, синим — обмен, зелёным — обновление; точка обозначает готовность данных для обмена.

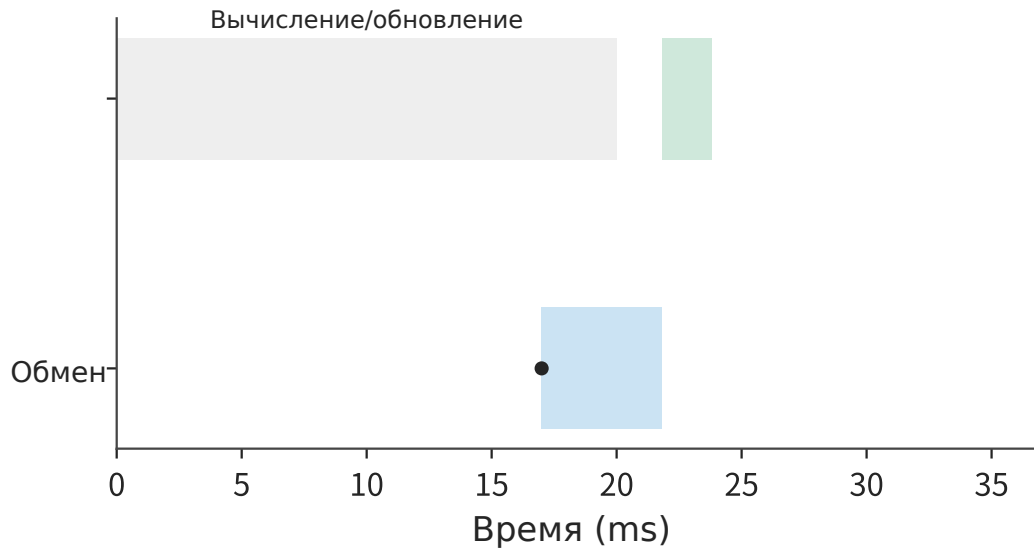


Рисунок 7-59. Здесь за основу взята иерархическая конфигурация с ранним началом обмена, но число доступных сетевых карт сокращено до одной. В каждом из двух межсерверных раундов через неё приходится передавать 96 MiB, поэтому обмен занимает около 4.8 ms, выходит за пределы интервала вычислений, а весь шаг длится приблизительно 23.8 ms. На всех приведённых временных диаграммах используется одинаковый диапазон горизонтальной оси. Серым показано вычисление, синим — обмен, зелёным — обновление; точка обозначает готовность данных для обмена.

23.8 ms. После увеличения времени обмена возрастает его доля, выполняемая уже после окончания вычислений.

Последовательно сравним эти временные диаграммы: сокращение объёма обмена укорачивает синюю полосу, недостаточный параллелизм удлиняет её, а более ранняя готовность данных сдвигает её влево. Зелёное обновление может начаться только после завершения как вычислений, так и обмена. Модели трафика, параллелизма и зависимостей также задают понятный порядок улучшений. Сначала следует с помощью иерархической редукции сократить межсерверную передачу и обеспечить каждую сетевую карту фрагментом для передачи, затем — полностью задействовать пропускную способность каналов за счёт параллельной отправки и обработки запросов. После достижения предела пропускной способности нужно искать обмены, которые можно начать раньше. Когда время обмена полностью перекрывается вычислениями, следует оптимизировать вычисление или обновление, которые теперь определяют момент завершения.

7.6.3 Коммуникационные узкие места при обучении и инференсе

Синяя полоса на рис. 7-54 может значительно сократиться, поскольку передача больших объёмов градиентов занимает основную часть времени коммуникации. При decode с низким параллелизмом скрытый вектор размером 8 KiB многократно передаётся между слоями; сама передача данных занимает крайне мало времени, а основная часть задержки приходится на запуск каждой операции. Рассмотрим кольцо из восьми участников: запуск каждого раунда занимает 5 μ s, эффективная пропускная способность каждого участника равна B , а при входном объёме M время одной редукции составляет

$$T_{AR} = 14 \times 5 \mu s + \frac{1.75M}{B}.$$

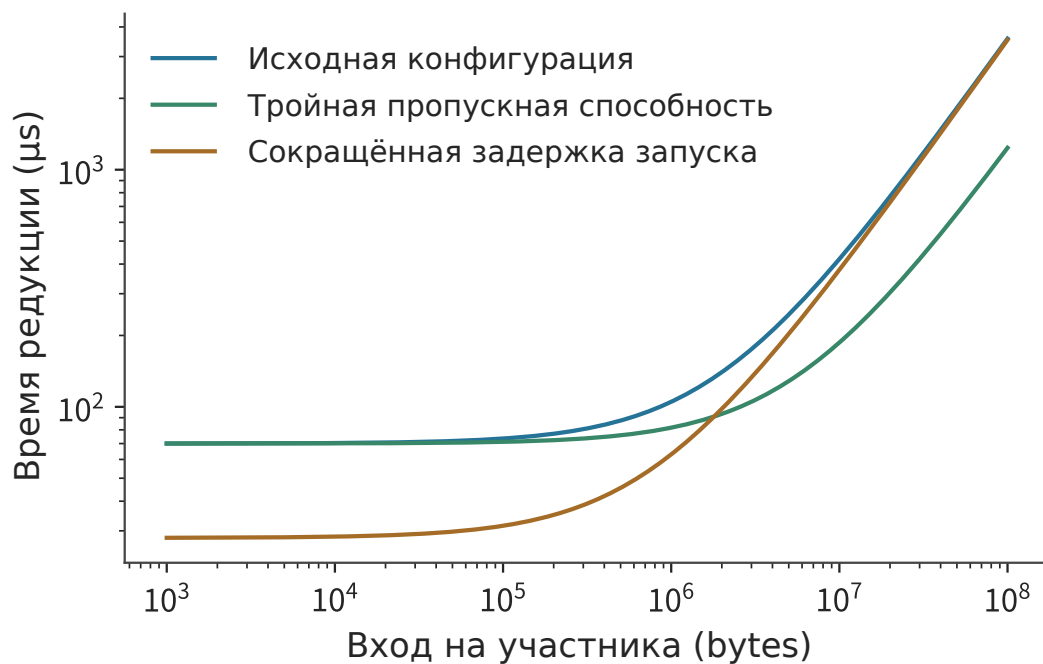


Рис. 7-60. В исходной конфигурации запуск каждого раунда занимает 5 μs , а пропускная способность равна 50 GB/s. На зелёной кривой пропускная способность увеличена втрое, до 150 GB/s; на оранжевой время запуска каждого раунда сокращено до 2 μs . При малом объёме данных основное влияние оказывает запуск, при большом — передача.

Четырнадцать запусков суммарно занимают 70 μs . Если принять пропускную способность сетевого адаптера из этой главы равной 50 GB/s и приравнять время передачи полезной нагрузки ко времени запусков, получим $M = 2 \text{ MB}$, то есть примерно 1,91 MiB. При входном объёме выше этого порядка величины растёт доля времени, затрачиваемая на передачу данных; 8 KiB значительно меньше, поэтому основное время уходит на запуски.

Если двигаться слева направо по горизонтальной оси рис. 7-60, то на участке с малым объёмом данных общее время остаётся почти постоянным, а на участке с большим объёмом растёт вместе с полезной нагрузкой. Повышение пропускной способности главным образом уменьшает наклон правой части графика, а сокращение времени запуска сдвигает вниз кривую на плоском участке слева.

Для входного объёма 8 MiB увеличение пропускной способности с 50 до 150 GB/s сокращает время одной редукиции примерно с 364 до 168 μs ; для входного объёма 8 KiB — лишь примерно с 70,3 до 70,1 μs . Предположим, выполняется модель из 36 слоёв, в каждом из которых внимание и FFN требуют по одной такой редукиции. Тогда 72 последовательные редукиции при обоих значениях пропускной способности занимают примерно 5,1 ms, а разница составляет около 14 μs . Однако сокращение времени запуска каждого раунда с 5 до 2 μs позволяет уменьшить время 72 редукиций примерно на 3,0 ms.⁴⁵

⁴⁵Пример планирования сети и коллективных операций. В примере с малыми сообщениями запуск каждого раунда занимает 5 μs , а эффективная пропускная способность каждого участника равна 50 GB/s для сетевой карты из этой главы либо втрое большей величине 150 GB/s; кольцевая модель учитывает передачу полезной нагрузки и запуск. Значения приведены в элементе `small_messages` учебного примера этой главы. Для упомянутой выше редукиции крупных блоков используется время запуска каждого раунда 0,833 μs , выведенное из записей `nccl-tests`.

Поэтому обучение и prefill с большим batch легче выигрывают от сокращения объёма передаваемых данных, увеличения пропускной способности и перекрытия операций; для decode с низким параллелизмом важнее сократить синхронизацию между серверами внутри слоя и ускорить пути запуска и завершения операций. Этим также объясняется схема конвейерного параллелизма, рассмотренная в разделе 7.2: размещение одной стадии локально позволяет сократить удалённые коммуникации внутри неё, а количество одновременно обрабатываемых независимых запросов определяет утилизацию конвейера.

Помимо среднего времени, завершение задачи могут задерживать и редкие длительные ожидания. Рассмотрим 100 коммуникаций: 98 из них занимают по 0,4 ms, одна из-за поздней готовности требует дополнительных 2 ms, а ещё одна из-за восстановления — дополнительных 10 ms. Среднее время составляет примерно 0,52 ms; p99 соответствует 99-му элементу отсортированного по возрастанию ряда и равен 2,4 ms. Если сократить обычное время передачи вдвое, p99 снизится до 2,2 ms; если же число событий восстановления увеличится до двух, p99 будет определяться коммуникацией, включающей восстановление после сбоя, и составит 10,4 ms.⁴⁶

Обычная передача, задержка подготовки данных и восстановление после сбоя — каждый из этих факторов может стать основным при определении времени завершения задачи. Для частых передач малых объёмов данных необходимо снижать накладные расходы каждого запуска; для длительного восстановления после сбоя — ограничивать область его воздействия. После коммуникационного сбоя во время обучения также требуется заново создать коммуникационную группу и восстановить состояние обучения; этот процесс подробнее рассматривается в главе 10.

При сравнении пропускной способности и накладных расходов на запуск схема коммуникации остаётся неизменной. Однако можно изменить и саму схему: корректировка схемы параллелизма и распределения экспертов позволяет сократить число передач; непосредственный запуск коммуникации ускорителем либо перенос обработки запуска и завершения на модуль разгрузки позволяет сократить время каждой передачи. Маршрутизация MoE, dispatch и combine предъявляют требования к сетевым операциям; способность сети эффективно выполнять эти операции, в свою очередь, влияет на гранулярность экспертов и их размещение между узлами. Если специализированное оборудование ускоряет вычисления модели, а время коммуникации остаётся прежним, её доля на критическом пути возрастает.

Такой же анализ трафика применим к переносу состояния при инференсе. Продолжим анализ одного и того же сеанса DeepSeek V4.1, рассматриваемого на протяжении всей книги, и предположим, что этот запрос необходимо перенести на экземпляр, не содержащий его глобальный KV-кэш. Сначала вычислим время передачи полезной нагрузки, соответствующей глобальной истории: контекст равен 128K, а передача выполняется через один ConnectX-7, работающий на скорости 200 Gbit/s и обеспечивающий эффективную пропускную способность 25 GB/s. Передача 439,281 MiB для V4-Flash занимает примерно 18,425 ms, а 111,250 MiB для V4.1 — примерно 4,666 ms. Сжатие состояния сокращает время передачи по этому пути примерно на 13,759 ms.⁴⁷

Эти 13,759 ms — экономия времени, которую схема миграции может обеспечить на

⁴⁶ Учебные записи разброса времени готовности, 100 смешанных записей, две записи восстановления. За p99 принимается элемент с номером $\lceil 0.99N \rceil$ в отсортированной последовательности.

⁴⁷ Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт сквозной межглавной сессии.

этапе передачи. Если запрос последовательно проходит через очередь, поиск в кэше, передачу и восстановление локального состояния SWA, время восстановления равно сумме этих четырёх этапов; насколько сокращается передача, настолько же сокращается общее время. В главе 9 этот путь восстановления будет сопоставлен с ожиданием на исходном экземпляре и повторным вычислением из входных данных, чтобы определить, где следует выполнять запрос.

7.6.4 Фиксированные 1024 ускорителя: как пересмотреть выбор при увеличении суперузла

В предыдущих разделах путь анализировался на примере шестнадцати участников. В этом разделе масштаб увеличивается до 1024 ускорителей: общее число ускорителей остаётся неизменным, а размер суперузла увеличивается с 8 до 64, 128 и 256 ускорителей, чтобы выяснить, какие временные показатели изменятся. Сначала зафиксируем объём вычислений и схему параллелизма, затем по отдельности изменим сетевые условия и в конце заново сравним степени параллелизма.⁴⁸

Фиксированные модель и объём обучения. Используется конфигурация Qwen3-32B: 64 слоя, размерность скрытого состояния 5120, число параметров $P = 32 \times 10^9$. 1024 ускорителя H100 SXM 80 GB организованы с тензорным параллелизмом 8 и параллелизмом по данным 128, что обозначается как TP8×DP128; каждая восьмиускорительная группа тензорного параллелизма целиком находится внутри одного суперузла. За одно обновление всегда обрабатывается 2^{20} токенов, а каждая реплика параллелизма по данным обрабатывает один микробатч из 8192 токенов; глобальный размер батча, точность и смысл обновления оптимизатора остаются неизменными. Состояние обучения оценивается в 16 bytes на параметр, то есть 64 GB на ускоритель; дополнительно предполагается, что 8 GB достаточно для активаций и рабочей области. ZeRO (Zero Redundancy Optimizer, оптимизатор с нулевой избыточностью, который распределяет состояния оптимизатора, градиенты и параметры между репликами параллелизма по данным) пока не используется.

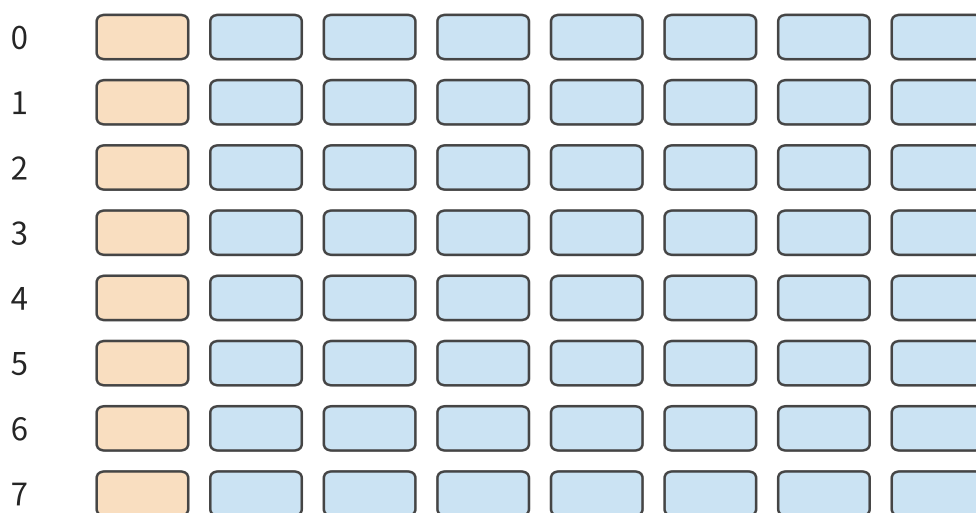
Градиенты передаются в BF16, размер хранящегося на каждом ускорителе фрагмента составляет $G = 2P/8 = 8$ GB. Пусть суперузел содержит S ускорителей, тогда число суперузлов $H = 1024/S$, а для каждой координаты тензорного параллелизма внутри суперузла имеется $q = S/8$ участников параллелизма по данным. На рис. 7-61 показана группировка внутри суперузла из 64 ускорителей: восемь ускорителей в одной строке образуют одну реплику модели, а ускорители в одном столбце принадлежат одной группе синхронизации градиентов.

Объём передачи между суперузлами. Сначала q участников параллелизма по данным внутри узла выполняют ReduceScatter, после чего на каждом ускорителе остаётся фрагмент градиента размером G/q ; затем H суперузлов, хранящих один и тот же фрагмент

⁴⁸Фиксированный сценарий с 1024 картами, полный расчёт и сравнение кандидатов, скрипт. Форма взята из конфигурации Qwen3-32B, общее число параметров приближённо принято равным 32B. Пиковая производительность H100 SXM для плотных операций BF16 приведена в таблице оборудования; MFU 41% взят из таблицы 4 статьи Llama 3; максимальное число подключаемых к NVLink Switch System GPU — 256, а полная коммутационная пропускная способность — 115,2 TB/s, согласно снимку статьи об архитектуре Grace Hopper; пропускная способность NVLink пятого поколения на один GPU составляет 1800 GB/s суммарно в обоих направлениях согласно странице спецификаций NVLink; задание на 1024 картах в среднем прерывается раз в 7,9 часа, а частота прерываний пропорциональна числу карт согласно рис. 7 статьи о надёжности кластера Meta. Интервал checkpoint, длительность паузы и фиксированное время восстановления являются заданными входными данными.

Суперузел из 64 карт: 8
групп TP8

Одна строка: TP-фрагменты совместно обрабатывают один ввод



Один столбец: соответствующие градиенты сначала
редуцируются локально

↓ Затем обмен между узлами

Рис. 7-61. Суперузел из 64 ускорителей состоит из восьми групп тензорного параллелизма. Восемь ускорителей в одной строке обрабатывают разные фрагменты модели для одного микро-батча; восемь ускорителей в одном столбце сначала объединяют соответствующие градиенты внутри узла, а затем обмениваются тем же фрагментом с другими суперузлами. Использование всех ускорителей одним заданием обучения не означает, что все они образуют одну группу тензорного параллелизма.

мент, выполняют AllReduce; наконец, внутри узла выполняется AllGather. При кольцевом алгоритме каждый ускоритель отправляет между узлами $2(H-1)G/(Hq)$ данных. Умножив это значение на $8q$ ускорителей внутри узла, получим объём отправки суперузла в каждом направлении:

$$V_{\text{out}} = 2 \frac{H-1}{H} 8G.$$

При увеличении узла с 8 до 128 ускорителей q возрастает с 1 до 16, и число байтов, передаваемых каждым ускорителем между узлами, уменьшается, однако суммарный объём отправки одного суперузла снижается лишь со 127 GB до 112 GB. Таким образом, увеличение суперузла в 16 раз не означает, что объём данных, проходящих через внешний интерфейс, также уменьшится в 16 раз.

Для внутриузловой сети NVLink принимается пропускная способность 450 GB/s на ускоритель в каждом направлении: 8 ускорителей соответствуют одному HGX H100; конфигурации от 64 до 256 ускорителей соответствуют NVLink Switch System, которая может объединить до 256 GPU Hopper в один домен NVLink с полной коммутируемой пропускной способностью 115.2 TB/s — ровно по 450 GB/s для каждого из 256 ускорителей. Для межузловой сети на каждый ускоритель приходится одна ConnectX-7 с пропускной способностью 50 GB/s в каждом направлении. Задержки запуска одного раунда внутри узла и между узлами, α_L и α_R , принимаются равными 0.83 μ s, как в разделе 7.1.1. Обозначим внутриузловую пропускную способность на один ускоритель в каждом направлении через B_L , пропускную способность одной сетевой карты в каждом направлении через B_{NIC} , а доступную одностороннюю пропускную способность внешнего интерфейса всего суперузла через B_{out} . Тогда время иерархической синхронизации градиентов равно

$$T_{\text{grad}} = 2(q-1)\alpha_L + \frac{2(q-1)G}{qB_L} + 2(H-1)\alpha_R + \max\left(\frac{2(H-1)G}{HqB_{\text{NIC}}}, \frac{V_{\text{out}}}{B_{\text{out}}}\right).$$

Слагаемые последовательно соответствуют локальной редукции и сборке, запуску межузлового обмена и более медленному из двух ограничений: сетевой карты или общего внешнего интерфейса. В реальной системе также могут сказываться сечение коммутационной сети и параллельный локальный обмен; в этом примере предполагается, что сеть обеспечивает указанную пропускную способность, а этапы не перекрываются.

Полное время шага обучения. Эффективная вычислительная производительность каждого ускорителя принимается равной 41% от пиковой производительности H100 SXM для плотных вычислений BF16, составляющей 989.4 TFLOP/s, то есть около 405.7 TFLOP/s; 41% — это заявленный BF16 MFU при обучении Llama 3 405B на 16384 ускорителях H100 с TP8, PP16 (конвейерный параллелизм из 16 стадий) и DP128. Согласно определению из раздела 1.2.2, значение 41% означает, что 59% пиковой вычислительной мощности не преобразуется в вычисления модели. Часть этой мощности уходит на работу, не учитываемую в подобных оценках первого порядка, например на квадратичную составляющую механизма внимания и повторные вычисления для экономии видеопамати во время обучения; другая часть теряется из-за ожидания, которое можно сократить, например из-за разброса времени готовности из раздела 7.6.1 и недостаточного числа запросов в пути из раздела 7.3.4. Время вычислений оценивается как $6P \times 2^{20} / (1024 \times 405.7 \text{ TFLOP/s}) \approx 0.485 \text{ s}$ без учёта квадратичной

составляющей механизма внимания. Для каждого слоя прямой и обратный проходы в сумме считаются как четыре операции AllReduce тензорного параллелизма; скрытый тензор BF16 для 8192 токенов занимает 80 MiB, что для 64 слоёв даёт около 0.086 s. На обновление оптимизатора дополнительно отводится 0.050 s. Тогда время шага и пропускная способность Θ при фиксированном расписании, выраженная в токенах в секунду, равны

$$T_{\text{step}} = 0.485 + 0.086 + T_{\text{grad}} + 0.050, \quad \Theta = 2^{20}/T_{\text{step}}.$$

Это оценка для фиксированного расписания, в котором вычисления, обмен тензорного параллелизма, синхронизация градиентов и обновление выполняются последовательно; в реальной системе из временной шкалы готовности градиентов вычитается уже перекрытая часть. Для сравнения также рассчитывается непрерывное кольцо параллелизма по данным без иерархии: для каждой координаты тензорного параллелизма имеется только одно межузловое ребро кольца, нагрузка которого не распределяется между другими сетевыми картами. В таблице ниже для каждого случая выбран более быстрый из двух алгоритмов.

Ускорителей в суперузле	Суперузлов	Локальных участников DP q	Междоменная отправка на узел	Внешний интерфейс масштабируется с числом ускорителей: время шага / пропускная способность	Внешний интерфейс фиксирован на 400 GB/s: время шага / пропускная способность
8	128	1	127 GB	0.939 s / 1.117 млн токенов/s	0.939 s / 1.117 млн токенов/s
64	16	8	120 GB	0.690 s / 1.520 млн токенов/s	0.939 s / 1.117 млн токенов/s
128	8	16	112 GB	0.672 s / 1.560 млн токенов/s	0.935 s / 1.122 млн токенов/s
256	4	32	96 GB	0.663 s / 1.581 млн токенов/s	0.896 s / 1.171 млн токенов/s

В сценарии «внешний интерфейс масштабируется с числом ускорителей» принимается $B_{\text{out}} = S \times 50 \text{ GB/s}$, то есть сетевые карты каждого ускорителя со скоростью 400 Gbit/s подключены к коммутационной сети. Это означает, что для более крупных узлов предусмотрено больше одновременно работающих внешних портов и достаточная пропускная способность коммутационной сети; сам по себе рост узла не делает внешний интерфейс шире. В сценарии «внешний интерфейс фиксирован» каждый суперузел подключён к коммутационной сети QM9700 только восемью линиями по 400 Gbit/s, то есть суммарно 400 GB/s, как у одного сервера HGX. В последнем случае для конфигурации с 64 ускорителями иерархическая редукция занимает около $31.1 + 300.0 = 331.1 \text{ ms}$ и оказывается медленнее непрерывного кольца, занимающего около 317.7 ms, поэтому в таблице выбрано непрерывное кольцо. Это соответствует критерию целесообразности локальной редукции из раздела 7.2. Столбец «внешний интерфейс масштабируется с числом ускорителей» также подразумевает неблокирующую коммутационную сеть: как рассчитано в разделе 7.1.2, переподписка 3:1 уменьшает внешний интерфейс каждого суперузла втрое, из-за чего междоменная составляющая для узла из 128 ускорителей возрастает примерно с 17.5 ms до 52.5 ms, а время шага увеличивается приблизительно на 35 ms; в разделе 7.2.5 объясняется, как эти междоменные байты равномерно распределяются по восьми магистралям в многорельсовой топологии.

На рис. 7-62 показано изменение пропускной способности с размером суперузла при трёх сетевых конфигурациях. При увеличении размера суперузла с 8 до 64 ускорителей и масштабировании внешнего интерфейса пропускная способность возрастает примерно на 36%. Суперузел из 8 ускорителей состоит всего из одного сервера, поэтому каждому ускорителю приходится синхронизировать через сетевую карту весь собственный фрагмент градиента размером 8 GB, что занимает 0.318 s — треть времени шага. При 64 ускорителях градиент сначала редуцируется по NVLink до одной восьмой исходного размера, и каждый ускоритель обменивается между узлами лишь этим небольшим фрагментом, благодаря чему синхронизация градиентов сокращается примерно до 69 ms. Увеличение размера суперузла со 128 до 256 ускорителей повышает пропускную способность лишь приблизительно на 1.3%, поскольку к этому моменту вычисления и обмен тензорного параллелизма уже занимают 86% времени шага. Если дополнительно увеличить внутреннюю пропускную способность с 450 до 900 GB/s, то есть до пропускной способности пятого поколения NVLink в каждом направлении, для узла из 128 ускорителей получаем около 0.614 s и 1.709 млн токенов/s; этот выигрыш обусловлен повышением внутренней пропускной способности, а не самим размером узла.

Повторное сравнение степеней параллелизма. Более крупный высокоскоростной домен может вместить более крупную группу тензорного параллелизма, но заполнять его целиком необязательно. Для суперузла из 128 ускорителей при масштабируемом внешнем интерфейсе переберём тензорный параллелизм 8, 16, 32 и 64, которому соответствует параллелизм по данным 128, 64, 32 и 16; глобальное число токенов остаётся неизменным. Число последовательностей, обрабатываемых каждой репликой параллелизма по данным, соответственно возрастает, при этом каждая последовательность по-прежнему содержит 8192 токена; число строк локальных матриц и объём передачи при тензорном параллелизме необходимо пересчитать. Если предположить, что все эти формы обеспечивают принятую в примере эффективную вычислительную производительность, время шага составит приблизительно 0.672, 0.753, 0.942 и 1.333 s соответственно. В этом случае TP8 по-прежнему оказывается лучшим вариантом: чем крупнее группа тензорного параллелизма, тем большим объёмом активаций

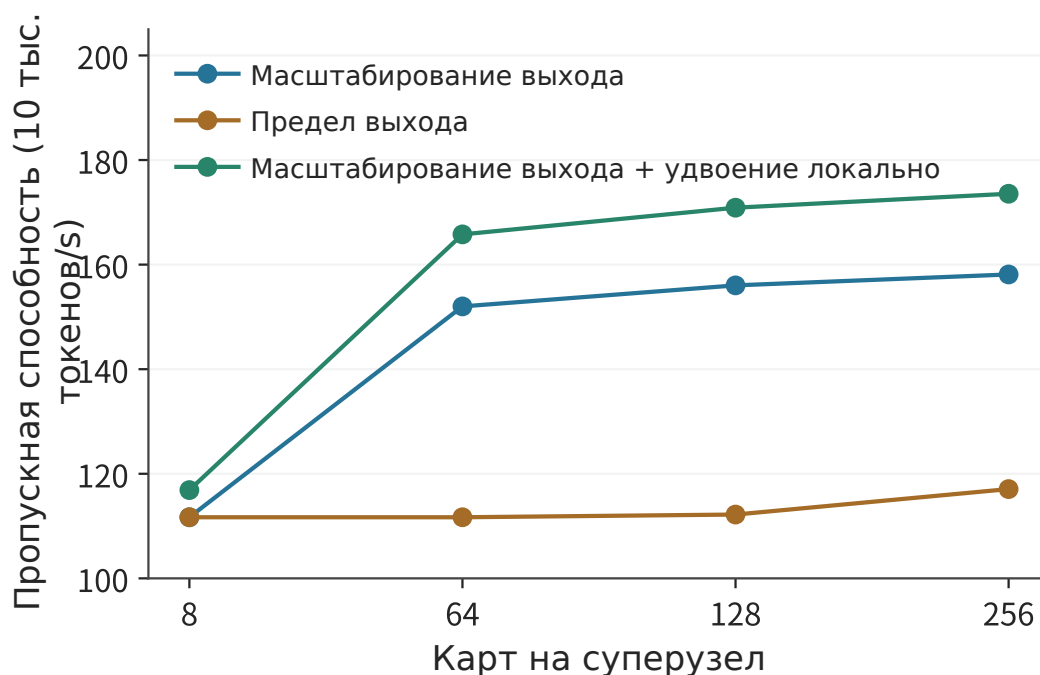


Рис. 7-62. Глобальное число токенов и общее число ускорителей фиксированы. Синяя линия соответствует увеличению внешней пропускной способности каждого узла, оранжевая — ограничению внешней пропускной способности уровнем 400 GB/s, зеленая — увеличению локальной пропускной способности с 450 до 900 GB/s при масштабируемом внешнем интерфейсе. В каждой точке выбирается более быстрый из указанных вариантов: непрерывное кольцо или иерархическая редукция; по вертикальной оси отложена оценка пропускной способности в штатном режиме при фиксированном расписании.

приходится обмениваться каждой реплике, и экономия на локальной синхронизации градиентов не компенсирует увеличение обмена активациями. Когда степень тензорного параллелизма превышает число KV-голов, равное восьми, также необходимо поддерживать схемы с репликацией KV; в этом примере предполагается, что kernel поддерживает такие схемы и располагает достаточным запасом ресурсов.

Этот отбор показывает, что 1024 ускорителя могут совместно обучать одну модель, но это не означает, что каждый слой модели следует разбивать между всеми 1024 ускорителями. При переходе к инференсу несколько более мелких экземпляров также могут независимо обрабатывать запросы и изолировать сбои; в разделе 6.7.4 тем же методом рассчитано влияние размера суперузла на пропускную способность декодирования. В реальном обучении при иной эффективности локальных матриц, других ограничениях памяти или возможностях перекрытия лучшими могут оказаться комбинации тензорного, контекстного и конвейерного параллелизма; следует расширить набор вариантов и пересчитать их, как в разделе 6.7.3, а не считать TP8 из этого примера универсальным ответом.

Затраты на восстановление после сбоев. Все приведённые выше значения пропускной способности предполагают, что задание никогда не прерывается. При синхронном обучении отказ одного ускорителя может остановить всё задание; реплику параллелизма по данным нельзя просто отбросить, как независимый экземпляр инференса, поскольку это изменит образцы и градиенты, используемые в текущем обновлении. Если система не поддерживает локальное восстановление или эластичную схему без изменения смысла обучения, необходимо откатиться к согласованной контрольной точке. Пусть через каждые I секунд полезных вычислений задание приостанавливается на C секунд для сохранения контрольной точки, частота прерываний задания равна λ , а восстановление занимает R . Если сбои происходят редко и их моменты приблизительно равномерно распределены внутри интервала между контрольными точками, долю накладных расходов можно грубо оценить как

$$\epsilon \approx C/I + \lambda(I/2 + R), \quad \Theta_{\text{effective}} \approx \Theta/(1 + \epsilon).$$

Здесь оценивается только средний полезный прогресс без гарантий хвостовой поддержки. Для независимых отказов ускорителей частоту прерываний можно суммировать по числу ускорителей, а отказы общего оборудования суперузла следует учитывать по фактическим доменам отказа: считать все отказы независимыми нельзя. Частота прерываний взята из статистики исследовательского кластера Meta: задание на 1024 ускорителях в среднем прерывается раз в 7.9 часа, причём частота прерываний пропорциональна числу ускорителей, что соответствует среднему интервалу между отказами каждого ускорителя около 337 дней, или около 8090 часов; эти же данные используются в главе 10. Эта статистика уже включает отказы общего оборудования, поэтому в примере не добавляется отдельная составляющая отказов общего оборудования суперузла и частота прерываний не зависит от размера суперузла; любое прерывание запускает восстановление всего задания. Через каждые 600 секунд полезных вычислений обучение приостанавливается на 10 секунд для сохранения контрольной точки. Время восстановления включает фиксированные 60 секунд плюс время чтения 64 GB состояния каждого ускорителя затронутого суперузла через одну линию PCIe Gen5 x16 с пропускной способностью 64 GB/s в каждом направлении. Для узлов из 128 и 256 ускорителей время восстановления составляет соответственно 188 и 316 секунд, а дополнительные наклад-

ные расходы — около 3.4% и 3.8%. При масштабируемом внешнем интерфейсе эффективная пропускная способность составляет приблизительно 1.509 и 1.523 млн токенов/s: восстановление узла из 256 ускорителей занимает больше времени, но дополнительные расходы всё ещё меньше прироста его штатной пропускной способности.

Частота прерываний получена из статистики одного кластера, а интервал между контрольными точками и канал восстановления заданы как входные параметры. Увеличение узла может сократить число общих устройств, но также может увеличить ущерб от одного сбоя и объём данных восстановления; при наличии резервных ускорителей, локального восстановления или иной частоты отказов значения следует подставить заново. Поэтому при сравнении вариантов необходимо одновременно указывать штатную пропускную способность, модель восстановления и эффективный прогресс обучения за целевой период; протоколы сохранения контрольных точек и восстановления подробнее рассматриваются в главе 10.

7.6.5 Развёртывание и распространённые заблуждения

Большие градиенты, передача малых объёмов данных и восстановление после сбоев подчёркивают соответственно значимость пропускной способности, накладных расходов на запуск и ожидания зависимостей. Сравнить варианты развёртывания также следует исходя из этих рабочих нагрузок. InfiniBand, RoCE и UB предоставляют конкретные механизмы соединения и обмена данными; при выборе варианта развёртывания сначала сравните пути передачи данных с помощью трёх моделей из этой главы. Соединения между GPU и сетевыми картами, общие интерфейсы и топология коммутаторов определяют доступную пропускную способность; механизмы отправки запросов и обработки завершений определяют фактическую пропускную способность; механизмы синхронизации и восстановления приложения определяют порядок выполнения.

При расчёте доступной пропускной способности сначала необходимо определить схему подключения физических портов. Например, линии UBoE и UB Link в Ascend 950 совместно используют SerDes в соответствии с предварительно заданной конфигурацией, а назначение группы SerDes определяется конфигурацией портов. Фактическую доступную пропускную способность линий в данном направлении можно рассчитать лишь после выбора назначения портов.⁴⁹ После унификации интерфейсов программы могут обращаться к ним единообразно, а конфигурация портов определяет, по каким линиям проходят данные и какова пропускная способность этих линий.

Опубликованные журналы запусков AllReduce позволяют проанализировать совместное влияние этих факторов. При входных данных объёмом 16 GiB и использовании разных буферов для входных и выходных данных двум восьмикарточным серверам H100 с теми же характеристиками, что и в примере этой главы, требуется около 68,7 ms, а четырёх — около 91,0 ms.⁵⁰ Добавление серверов увеличивает число участников и длину пути взаимодействия.

⁴⁹Сверка первичных материалов по UB и Ascend 950, включающая базовую спецификацию UB 2.0.1, эталонный проект операционной системы 2.0 и официальный технический документ Ascend 950. Домены, режимы передачи, двунаправленная пропускная способность и совместное использование SerDes рассчитаны отдельно.

⁵⁰Проверка опубликованных журналов запусков experiments/ch07/07-03. Данные взяты из отправленных пользователями записей официального проекта.

Выкладки этой главы также позволяют прояснить четыре распространённых заблуждения.

Заблуждение: если общий объём отправленных данных одинаков, то одинаково и время обмена данными. Непрерывное кольцо, чередующееся кольцо и иерархическая редукция отправляют по 5760 MiB, но на межсерверную часть приходится соответственно 720, 5760 и 384 MiB, причём задействуются соответственно одна, восемь и восемь сетевых карт. Время передачи зависит от того, через какие ресурсы проходят данные, а расположение участников изменяет число байтов, приходящихся на каждую сетевую карту.

Заблуждение: низкой средней загрузки линии достаточно, чтобы справиться с пиками обмена данными. Двум периодическим заданиям в среднем требуется лишь 20 GB/s, но при наложении пиков обмена нагрузка достигает 100 GB/s, из-за чего на выходе с пропускной способностью 50 GB/s накапливается очередь в 1 GB. Буфер хранит накопившуюся за определённый период разницу, а фаза и обратная связь определяют, как долго эта разница будет накапливаться.

Заблуждение: если результаты возвращаются по порядку, значит прочитаны правильные данные. Старое значение, прочитанное до обновления данных, не станет новым из-за задержки возврата. Зависимость между публикацией и чтением должна ограничивать фактический момент чтения, а при опережающем выполнении необходимы проверки и повторное чтение.

Заблуждение: ускорение обмена данными равно ускорению задачи. Иерархическая редукция сокращает обмен данными примерно с 7,6 до 1,3 ms, а последовательный шаг обучения — примерно с 29,6 до 23,3 ms; более ранняя готовность данных дополнительно сокращает шаг иерархической схемы до 22,0 ms. Одна и та же оптимизация обмена данными при разном порядке выполнения экономит разное суммарное время.

В понимании автора исходная идея унифицированной системы межсоединений состоит в том, чтобы устройства могли напрямую обращаться к удалённым данным и инициировать обмен, верхний уровень мог явно выражать зависимости, а нижний — повторно использовать одно и то же состояние передачи. Для оценки таких решений нужно ответить на три вопроса: какие байты больше не требуется перемещать, какого ожидания удалось избежать и какие дополнительные состояния запросов необходимо для этого хранить и какую обработку выполнять.

Эксперимент 7-15 · Основное: насколько оптимизация обмена данными сокращает время обучения и инференса

Сначала повторите расчёт примера с 1024 картами, сохраняя неизменным глобальное число токенов, поочерёдно изменяйте размер суперузла, ограничение исходящего трафика и локальную пропускную способность и фиксируйте, какой алгоритм обмена данными и какой кандидат тензорного параллелизма оказывается лучше; затем вдвое уменьшите пропускную способность канала восстановления и сравните эффективный прогресс. После этого повторите расчёт примера 7.6 и постройте полную временную шкалу шага для непрерывного кольца, иерархической редукции, недостаточного числа запросов в пути, опережающего обмена данными и случая, когда остаётся лишь одна сетевая карта. Найдите самый поздний момент готовности данных, при котором иерархический обмен данными полностью перекрывается вычислениями длительностью 20 ms. Затем рассмотрите 36 слоёв и две редукции по 8 KiB на каждом слое, отдельно увеличьте

пропускную способность втрое и сократите накладные расходы на каждый запуск до 2 μ s и объясните, почему порядок улучшений для двух типов нагрузки различается.

7.7 Итоги главы

От операторов на одной карте к суперузлам, а затем и к центрам обработки данных — разбиение и планирование всегда строятся вокруг одного и того же набора зависимостей данных. При анализе задач, охватывающих несколько серверов, можно сначала проследить путь одной порции данных: определить по алгоритму, каким участникам её необходимо передать, а затем по топологии выяснить, через какие ресурсы она проходит. В примере с двумя серверами сначала выполняется локальная редукция, благодаря чему объём межсерверной передачи сокращается с 720 MiB до 384 MiB, а все восемь сетевых карт работают одновременно, уменьшая время передачи по каналу, являющемуся узким местом. Сама коммутационная сеть также является ресурсом, который необходимо учитывать в расчётах: коэффициент переподписки определяет ширину разреза, выравнивание rail — пройдут ли байты за один переход или через уровень spine, конфликты хеширования и incast — эффективную пропускную способность и требования к буферам, а редукция внутри сети сокращает межсерверный этап до одного раунда.

После выбора пути обмена необходимо непрерывно отправлять и обрабатывать запросы. Производство пропускной способности на задержку определяет требуемый объём данных в передаче, объём данных в каждом запросе и интервал запуска определяют, сколько данных можно отправлять в секунду, а скорость обработки уведомлений о завершении — как скоро можно повторно использовать слоты запросов. Совместно используемое состояние экономит ёмкость, изолированное состояние ограничивает взаимное влияние; моменты, когда данные становятся доступными для чтения и когда их можно освободить, определяют, как долго необходимо хранить эти состояния. Поэтапное суммирование времени одного удалённого доступа показывает, подключён ли контроллер после PCIe или непосредственно к внутрикристальной шине, а значит, каковы фиксированные накладные расходы каждого обращения; складывается ли объём состояния соединений по числу конечных точек или умножается по числу их пар, определяет, поместится ли оно во внутрикристальном кэше.

Пример с 1024 картами дополнительно показывает, что размер суперузла, внутрисерверная пропускная способность и внешний канал — это разные ограничения. При расширении высокоскоростного домена с одного восьмикарточного сервера до 64 карт локально выполняется больше операций редукции, доля времени шага, занятая синхронизацией градиентов, снижается с одной трети примерно до одной десятой, а пропускная способность возрастает приблизительно на 36%; при дальнейшем расширении выигрыш быстро уменьшается. После увеличения высокоскоростного домена можно также заново сравнить сочетания тензорного параллелизма и параллелизма по данным, однако ограниченная пропускная способность внешнего канала, стоимость локальной редукции и накладные расходы восстановления могут изменить выбор. Использование заданием всех карт не означает, что каждый вид обмена должен охватывать их все.

Наконец, последовательность подготовки, передачи, использования и восстановления данных следует представить в виде графа зависимостей. Сокращение объёма передаваемых данных и работы по обработке запросов, более ранний запуск независимых

операций и сокращение критического пути — три способа превратить оптимизацию сети в выигрыш для задачи. В следующей главе на основе этих возможностей выполнения и межсоединений будет построен сервис инференса для одного экземпляра, а в главе 9 будет рассмотрена организация передачи данных между экземплярами.

Глава 8

Оптимизация инференса

Когда одна и та же модель работает на одном и том же ускорителе, отдельный пользователь может получать лишь несколько десятков токенов в секунду, тогда как все пользователи в совокупности — тысячи токенов в секунду. При декодировании каждое чтение весов сопровождается небольшим объёмом вычислений, поэтому скорость ограничивается пропускной способностью видеопамяти, а не пиковой вычислительной производительностью. Если объединить несколько запросов для совместного выполнения и использовать однократно прочитанные веса, общую пропускную способность можно увеличить в несколько раз. В то же время длина контекста, время ожидания в очереди и порядок выполнения каждого запроса определяют, когда пользователь получит ответ. В этой главе рассматривается, как организовать обработку таких запросов, чтобы за заданное время возвращать больше правильных ответов.

Экземпляр инференса может использовать одну видеокарту или выполняться совместно несколькими видеокартами и серверами. В главах 6 и 7 уже объяснялось, как разделяются вычисления модели и как взаимодействуют ускорители. В этой главе за основу берётся заданная конфигурация ускорителей и исследуются пакетная обработка, планирование запросов, управление памятью, сжатие, выгрузка и спекулятивное декодирование, а также сравнивается эффективность выполнения при различных конфигурациях. В главе 9 будет рассмотрено, где следует размещать вычисления и состояние, как организовать полные реплики, пулы сервисов для отдельных этапов и распределение операторов, а затем на этой основе определить масштаб сервиса.

Эта глава начинается с конкретной задачи проектирования. В примерах экземпляр Qwen3-8B работает на одной видеокарте RTX PRO 6000 Blackwell Workstation Edition: 96 Гб видеопамяти, пропускная способность 1792 Гб/с, пиковая производительность плотных вычислений с входными данными BF16 и накоплением FP32 — 503,8 TFLOP/s, предельная мощность — 600 Вт.¹ За исключением эксперимента 8-7, выполненного на Apple M2 Max, все эксперименты этой главы проводились на этой видеокарте. Веса модели хранятся в формате с плавающей точкой BF16 по 2 байта на элемент и занимают около 15,3 GiB. KV-кэш хранит ключи (K) и значения (V) токенов контекста, которые считываются при последующих вычислениях внимания. Как и в эксперименте 8-9, этому экземпляру выделено 32 GiB видеопамяти: 12 GiB зарезервировано для KV-кэша и вспомогательных буферов, а оставшиеся примерно 4,7 GiB используются для активаций, фиксированных буферов воспроизведения вычислительного графа (раздел 5.5) и других рабочих областей выполнения.

¹Спецификация NVIDIA RTX Blackwell PRO, приложение A, таблица 4; данные сведены в строку rtx-pro6000-blackwell-ws таблицы параметров оборудования.

Экземпляр обрабатывает запросы двух типов: короткий диалог получает на вход 2048 токенов, а запрос с длинным контекстом — 8192 токена; оба типа генерируют по 256 токенов. Первые 6144 токена длинных запросов представляют собой одинаковый системный промпт и определения инструментов. Для каждого запроса время от поступления до возврата полного правильного ответа не должно превышать 7 секунд. Один короткий запрос при самостоятельном выполнении на этой видеокарте занимает около 6,95 секунды (раздел 8.1.1), поэтому до предельного срока остаётся лишь около 0,05 секунды. Далее эта совокупность условий будет использоваться неоднократно.

Условие проектирования	Короткий диалог	Длинный контекст
Длина входных данных	2048 токенов	8192 токена
Длина выходных данных	256 токенов	256 токенов
Длина общего префикса	0	6144 токена
Срок завершения	7 s	7 s

Сначала рассчитаем, KV-кэши скольких запросов экземпляр может хранить одновременно, а затем — насколько увеличение размера пакета сокращает объём чтения весов в расчёте на каждый выходной токен. После этого введём страничную организацию памяти, совместное использование префикса, сжатие, выгрузку и спекулятивное декодирование и отдельно проанализируем, что позволяет сэкономить каждый из этих методов и какие дополнительные затраты он создаёт. В конце главы результаты этого анализа будут применены к одной и той же задаче: какую конфигурацию следует выбрать, если одновременно поступают 16 длинных запросов?

8.1 Процесс выполнения запросов на инференс и требования к ресурсам

8.1.1 Ресурсы выполнения и жизненный цикл запроса

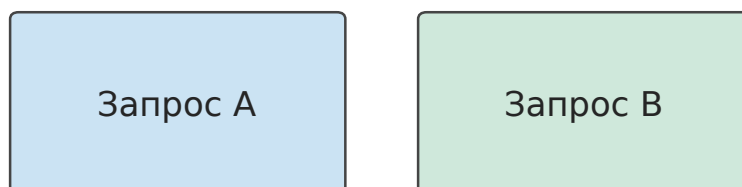
Запрос — это задача инференса с независимыми входными данными, результатом и условием завершения. Под **batch** в этой главе понимается группа входных токенов, совместно обрабатываемых в рамках одного выполнения на ускорителе; **итерация планирования** — это цикл, в котором планировщик выбирает задачи для текущего раунда, отправляет их на выполнение и обрабатывает результаты. Запрос может выполняться на протяжении многих итераций, причём набор других запросов, объединённых с ним в **batch**, может меняться в каждом раунде.

После поступления запрос сначала попадает в очередь ожидания. Модель начинает обрабатывать входные данные, то есть переходит к этапу **prefill**, только после того, как планировщик добавит запрос в **batch** и выделит пространство для KV. Поскольку все входные токены уже заданы, модель может одновременно вычислять несколько токенов и сохранять ключи и значения каждого слоя. После завершения вычислений для последнего токена промпта модель генерирует первый выходной токен.

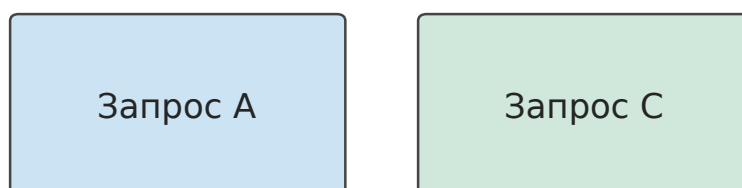
Затем начинается этап **decode**. Модель использует только что сгенерированный токен как вход для следующего прямого прохода, считывает KV контекста и генерирует следующий токен. Для одного запроса эти шаги должны выполняться последовательно, однако текущие шаги разных запросов можно вычислять совместно. Поэтому, даже если отдельный запрос может генерировать токены только по одному, экземпляр всё рав-

Запросы сохраняют идентичность,
batch пересобирается на каждой
итерации

Итерация 0



Итерация 1



В завершён → освободившееся место получает С

Рисунок 8-1. А продолжает выполняться на протяжении двух итераций. В завершается в первом раунде, а в следующем его вычислительную позицию занимает С (в каждом раунде для одного активного запроса выделяется одна строка вычислений); состояние каждого запроса по-прежнему сохраняется отдельно в соответствии с его идентичностью.

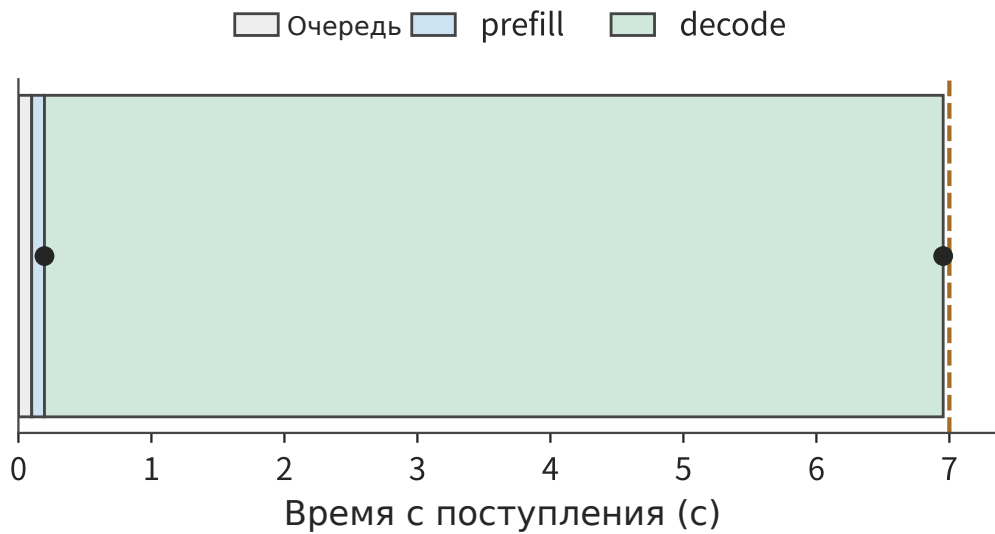


Рисунок 8-2. Запрос, выводящий 256 токенов. Ожидание в очереди занимает 0,1 секунды, prefill — 0,096 секунды, а каждый из последующих 255 выходных интервалов — 26,5 ms (измерено в эксперименте 8-1 при batch 1). Полоса построена в реальном временном масштабе, точками отмечены первый и последний токены; пунктирная линия на отметке 7 секунд обозначает предельный срок завершения.

но способен повышать общую пропускную способность за счёт объединения запросов в batch.

На этой карте в эксперименте 8-1 отдельно выполняется короткий запрос с входом длиной 2К: этап prefill и возврат первого токена занимают около 0,096 секунды, после чего токены возвращаются в среднем с интервалом 26,5 ms. Если предположить, что запрос сначала проводит в очереди 0,1 секунды, пользователь получит первый токен через 0,196 секунды после поступления запроса. Для вывода 256 токенов требуется 255 выходных интервалов, поэтому полное время запроса составляет

$$0.1 + 0.096 + 255 \times 0.0265 = 6.95 \text{ s.}$$

Этот запрос успеет завершиться за 7 секунд. Если во время генерации вставить этап prefill другого короткого запроса, один из выходных интервалов увеличится примерно на 0,096 секунды — с 26,5 ms приблизительно до 123 ms, а полное время возрастет до 7,05 секунды. Состав вычислений для каждого токена не изменился, однако одного лишь изменения порядка выполнения достаточно, чтобы запрос не уложился в срок.

На рисунке 8-2 эти 6,95 секунды представлены в виде временной шкалы. Момент возврата первого токена делит её на две части: предшествующие ему ожидание в очереди и обработка входных данных определяют, когда пользователь увидит ответ, а последующие 255 интервалов — когда он получит полный ответ. Все рассматриваемые далее оптимизации планирования можно понимать как изменение длины того или иного участка этой временной шкалы.

После завершения запроса планировщик больше не назначает для него вычисления, а диспетчер состояния решает, как поступить с оставшимся KV. KV, принадлежащий только этому запросу, освобождается после его завершения, а KV общего префикса можно сохранить для повторного использования последующими запросами; дальнейшее управ-

ление им выполняет политика кэширования. В разделе 8.3 подробнее объясняется, когда эти кэши используются совместно, а когда освобождаются.

8.1.2 Требования к объёму хранения весов, KV и буферов среды выполнения

Чтобы несколько запросов могли продвигаться по собственным временным шкалам, экземпляр должен одновременно хранить веса, состояния запросов и промежуточные результаты вычислений. В примере этого раздела используется только одна карта, поэтому приводится только объём памяти на ней. Для многокарточного экземпляра необходимо отдельно составлять выражения для фрагментов весов, KV и буферов, хранящихся на каждой карте: свободную память одной карты можно использовать для вычислений на другой только после изменения размещения данных.

Обозначим через M_w объём постоянно размещённых в памяти весов, через M_{KV} — общий объём физических блоков KV без повторно учитываемых ссылок (KV выделяется блоками фиксированного размера, а блок, совместно используемый несколькими запросами, учитывается только один раз), через M_a — объём активаций и обычных рабочих областей, а через M_u — объём других буферов, например для повторного выполнения вычислительного графа. Сумма этих объёмов памяти должна быть меньше или равна доступной ёмкости ускорителя C :

$$M_w + M_{KV} + M_a + M_u \leq C.$$

Среди перечисленных потребителей памяти веса загружаются в неё ещё до поступления запросов, а KV непрерывно растёт по мере поступления запросов и генерации. Чтобы вычислить, сколько запросов экземпляр способен обрабатывать одновременно, сначала определим, какой объём KV требуется сохранять для каждого токена. Qwen3-8B содержит 36 слоёв, в каждом из которых имеется 8 KV-голов размерностью 128. Для каждого токена на каждом слое необходимо сохранять K и V, а каждый элемент BF16 занимает 2 bytes, поэтому требуемый объём KV на один токен равен

$$k = 2 \times 36 \times 8 \times 128 \times 2 = 147\,456 \text{ bytes} = 144 \text{ KiB}.$$

После ввода 2048 токенов KV занимает 288 MiB, а после ввода 8192 токенов — 1152 MiB. Все запросы совместно используют около 15,3 GiB весов, однако каждый дополнительный длинный запрос с независимым контекстом требует ещё 1,125 GiB KV. Только для хранения KV входных данных 16 длинных запросов потребуется 18 GiB, что превышает зарезервированные 12 GiB.²

Во время генерации выходных данных KV продолжает расти. Этап prefill создаёт первый выходной токен, после чего первые 255 выходных токенов последовательно возвращаются модели для генерации остальных 255 токенов. Для короткого запроса всего вычисляется $2048 + 255 = 2303$ токена, а для длинного — $8192 + 255 = 8447$ токенов. Если один блок вмещает 16 токенов, для них необходимо выделить пространство соответственно на 2304 и 8448 токенов.

²Вычисления с пакетной обработкой для фиксированной модели на RTX PRO 6000 и сравнение с 8K. Полные веса BF16 занимают 16,381,470,720 bytes, идеальный объём однократного чтения общих матричных весов за один шаг составляет 15,136,811,008 bytes; строки embedding выбираются для каждого запроса, однако в памяти всё равно необходимо хранить полную матрицу embedding. Таблица чтения сначала рассчитывается в точных байтах, после чего каждое значение округляется.

Независимый запрос	KV после prefill	Выделенный объём KV после вывода 256 токенов	Число запросов, одновременно помещающихся в 12 GiB
Короткий диалог	288 MiB	324 MiB	37
Длинный контекст	1152 MiB	1188 MiB	10

Значения в последнем столбце получены соответственно как $\lfloor 12288/324 \rfloor$ и $\lfloor 12288/1188 \rfloor$. Если учитывать только размер KV на момент завершения prefill, в 12 GiB поместятся 42 коротких запроса; однако после генерации каждым из них 256 выходных токенов требуемый объём увеличится до 13 608 MiB. Резервирование пространства, необходимого на этапе генерации, ещё при приёме запроса позволяет избежать ситуации, когда нехватка памяти обнаруживается посреди выполнения.

Совместное использование общего префикса позволяет дополнительно экономить память. Первые 6144 токена длинного запроса занимают 864 MiB KV, а оставшаяся часть входных данных — 288 MiB. Если каждый из 16 запросов хранит префикс самостоятельно, в памяти окажутся 16 одинаковых копий KV; если же они ссылаются на один и тот же KV, освободившееся пространство можно использовать для индивидуальных суффиксов. В разделе 8.3 будет описан конкретный способ отображения блоков и рассчитан общий объём после завершения генерации.

8.1.3 От времени отдельного запроса к целевым показателям сервиса

Представим две части временной шкалы из раздела 8.1.1 в виде метрик, для которых можно задавать предельные значения. Пусть запрос поступил в момент t_a и всего вывел G токенов, а первый и последний токены были возвращены пользователю соответственно в моменты t_1 и t_G . Тогда

$$\text{TTFT} = t_1 - t_a,$$

а j -й выходной интервал равен $\text{ITL}_j = t_{j+1} - t_j$. Сложив время до первого вывода и время после него, получим полное время запроса:

$$T_{\text{request}} = \text{TTFT} + \sum_{j=1}^{G-1} \text{ITL}_j.$$

В примере из раздела 8.1.1 TTFT составляет 0,196 секунды, а суммарная длительность последующих 255 выходных интервалов — 6,76 секунды. Более короткий TTFT позволяет пользователю раньше увидеть ответ, равномерные выходные интервалы обеспечивают удобное непрерывное чтение, а полное время определяет, когда будет доступен весь ответ. SLO количественно выражает эти требования в виде предельных значений. В примере из начала главы полное время не должно превышать 7 секунд, а в примере с планированием также будет сравниваться максимальный выходной интервал.

Эти метрики относятся к отдельному запросу; общая производительность экземпляра зависит ещё и от числа одновременно обрабатываемых запросов, то есть от batch size (далее — batch). Пропускная способность показывает объём работы, выполненной за единицу времени. Когда в эксперименте 8-1 одновременно обрабатываются 16 запросов длиной 2K, каждый раунд decode движка занимает около 27,44 ms и выводит в общей

сложности 16 токенов, поэтому пропускная способность генерации составляет приблизительно $16/0.02744 \approx 583$ токенов/s; когда один запрос выводит по токену каждые 26,26 ms, это значение составляет лишь около 38. Увеличение числа токенов, выводимых за раунд, или сокращение длительности раунда повышает пропускную способность; фактическое время ожидания пользователя зависит также от того, сколько запрос провёл в очереди перед началом выполнения.

Для оценки эффективности сервиса необходимо также проверять правильность ответов. Фиксированная длина вывода позволяет сравнивать время выполнения при одинаковом объёме работы; если же разрешить модели завершать генерацию естественным образом, можно наблюдать за правильностью ответов, их длиной и числом повторных попыток. В разделе 8.6 будет рассчитано число правильных и своевременно завершённых запросов за единицу времени, что позволит одновременно оценить скорость и качество.

8.2 Пакетная обработка и планирование запросов

8.2.1 Как повторное использование весов повышает эффективность пакетной обработки

Рассмотрим умножение матриц $Y = XW$, где входная матрица X содержит b строк. Один и тот же элемент весов участвует в вычислениях для всех строк, поэтому при совместной обработке нескольких строк после однократного чтения весов можно выполнить больше операций умножения со сложением. Предположим, что каждый элемент весов считывается внутри пакета только один раз, каждая операция умножения со сложением считается двумя операциями, а элемент весов занимает s_w байт. Если учитывать только чтение весов, арифметическая интенсивность равна

$$I_{\text{weight}} \approx \frac{2b}{s_w}.$$

Для BF16 $s_w = 2$. Если увеличить число запросов, одновременно выполняющих один шаг decode, с 1 до 16, причём каждый запрос предоставляет один новый токен, а вектор признаков каждого токена занимает отдельную строку входной матрицы, то число строк матрицы увеличится с одной до 16, а объём вычислений на одно и то же чтение весов возрастёт в 16 раз. Разные запросы используют общую матрицу весов, однако при вычислении внимания для каждого из них по-прежнему требуется отдельно считывать KV собственного контекста.

Условие совместного использования состоит в том, что все запросы в пакете используют одни и те же веса. **Низкоранговая адаптация (LoRA)** ослабляет это условие: корректировка весов представляется произведением двух небольших матриц, а базовые веса остаются неизменными. Благодаря этому разные задачи могут совместно использовать одну базовую модель, добавляя лишь собственный небольшой adapter — адаптер, то есть модуль обучаемых параметров, подключаемый к базовой модели. Если для проекций Q и V модели Qwen3-8B использовать ранг 16 и BF16, один адаптер занимает 14,6 MiB, а 100 адаптеров — 1,43 GiB, что значительно меньше 14,1 GiB матричных весов.

Если запросы одного пакета используют разные адаптеры, общими остаются только базовые веса. Низкоранговые вычисления приходится группировать по адаптерам, причём число строк в каждой группе равно числу запросов данного пакета, относящихся

к соответствующему адаптеру. Чем сильнее запросы распределены между адаптерами, тем меньше строк в каждой группе и тем хуже повторное использование весов в этой части. Поэтому количество разновидностей адаптеров и количество запросов в пакете — две величины, которые следует выбирать независимо.

Ёмкость также нужно рассчитывать отдельно. Сто адаптеров занимают 1,43 GiB, тогда как KV ста независимых контекстов длиной 8K в сумме занимают 112,5 GiB. Число размещаемых адаптеров определяется небольшими матрицами, а число одновременно обслуживаемых запросов — по-прежнему объёмом KV. Избыток ёмкости для первых не означает её избытка для вторых.³

Пусть объём матричных весов, которые за один раунд достаточно считать один раз, равен D_w , каждый запрос содержит L токенов контекста, запросы не используют общий KV, а KV одного токена занимает k . Тогда общий объём чтения за раунд и объём чтения в расчёте на один выходной токен соответственно равны

$$D(b, L) = D_w + bLk, \quad d(b, L) = \frac{D_w}{b} + Lk.$$

В объёме чтения на один выходной токен первое слагаемое представляет распределённое чтение весов, а второе — чтение KV контекста данного запроса. Для Qwen3-8B D_w составляет около 14,1 GiB, а KV контекста длиной 2K — 0,28125 GiB. При увеличении размера пакета с 1 до 16 чтение весов в расчёте на выходной токен снижается примерно до 0,88 GiB, а при увеличении до 64 — примерно до 0,22 GiB. На этом этапе на каждый выходной токен уже считывается больше KV, чем весов.⁴

На рис. 8-3 показано это изменение. Соседние отметки на горизонтальной оси соответствуют удвоению размера пакета, поэтому составляющая весов каждый раз уменьшается вдвое. Составляющая KV остаётся неизменной, а их сумма постепенно приближается к горизонтальной линии, соответствующей KV. При контексте длиной 2K увеличение пакета с 1 до 64 снижает объём чтения на один выходной токен примерно с 14,38 до 0,50 GiB, то есть приблизительно в 29 раз. Если увеличить длину контекста до 8K, объём KV возрастает в четыре раза, поэтому даже после объединения запросов в пакет для каждого выходного токена приходится считывать больше данных.

С увеличением размера пакета экономия от распределения чтения весов постепенно сокращается. Это изменение можно описать размером пакета, при котором объёмы чтения весов и KV становятся равными. Минимальный размер пакета, при котором объём чтения KV всего пакета достигает или превышает объём чтения общих весов, равен

$$b_{KV} = \left\lceil \frac{D_w}{Lk} \right\rceil.$$

При расчёте с использованием неокруглённого числа байтов весов для контекста длиной 2K получается 51, а для 8K — 13. Когда длина контекста увеличивается в четыре раза, объём чтения KV превосходит объём чтения весов уже при значительно меньшем размере пакета. В то же время более крупному пакету требуется больше памяти: веса и KV после одного шага decode для 64 запросов с контекстом длиной 2K в сумме занимают

³Обзор MLSys 2024 и вычисления с несколькими LoRA.

⁴Вычисления с пакетной обработкой для фиксированной модели на RTX PRO 6000 и сравнение с 8K. Полные веса BF16 занимают 16,381,470,720 bytes, идеальный объём однократного чтения общих матричных весов за один шаг составляет 15,136,811,008 bytes; строки embedding выбираются для каждого запроса, однако в памяти всё равно необходимо хранить полную матрицу embedding. Таблица чтения сначала рассчитывается в точных байтах, после чего каждое значение округляется.

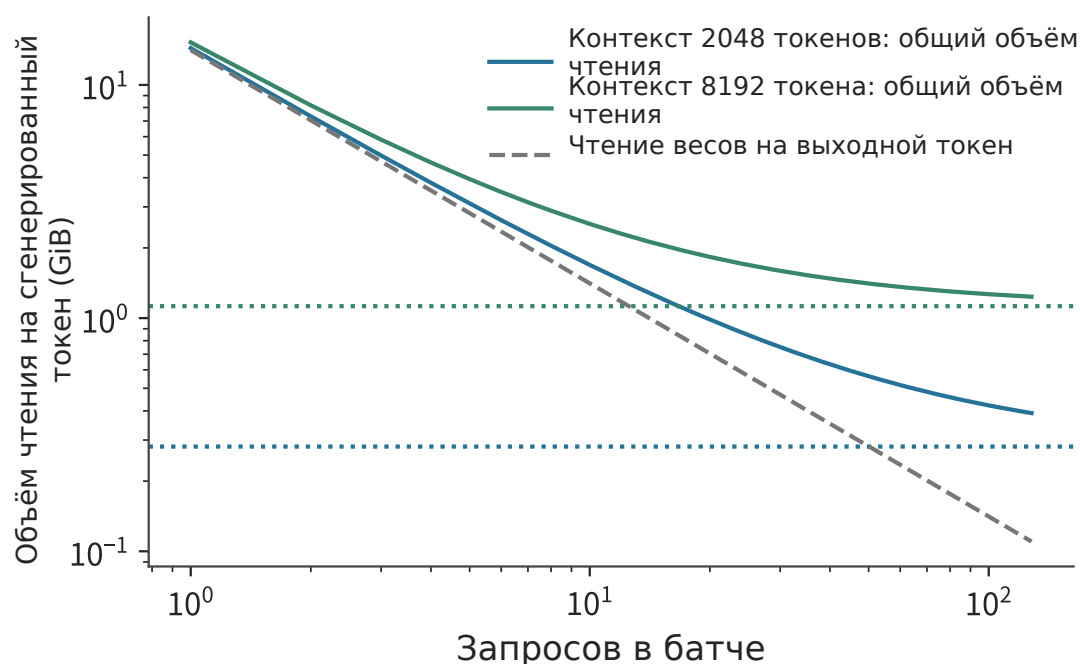


Рис. 8-3. Как пакетная обработка уменьшает объем чтения, необходимый для генерации одного токена. По горизонтальной оси отложено число одновременно генерируемых запросов, по вертикальной — объем байтов, считываемых для всего пакета, разделенный на число выходных токенов в текущем раунде. Синяя и зеленая сплошные линии соответствуют запросам с контекстом длиной 2048 и 8192 токена и учитывают чтение весов и KV; пунктирные горизонтальные линии тех же цветов показывают только объем чтения KV при соответствующей длине. Серая штриховая линия показывает объем чтения общих для пакета весов, разделенный на число запросов. Используется модель Qwen3-8B с BF16; каждый набор матричных весов считывается один раз за раунд, а KV запросов независимы. Обе оси имеют логарифмическую шкалу.

около 35,7 GB. Объединение в пакет уменьшает объём чтения на один выходной токен, но увеличивает объём одновременно размещённых данных.

Теперь оценим время, необходимое для этого чтения. Пусть объём вычислений для одного запроса равен F , ускоритель выполняет P операций в секунду, а пропускная способность памяти равна β . Оценим время одного раунда как максимум из отдельно рассчитанных времени вычислений и времени чтения:

$$T_{\text{step}} \approx \max \left(\frac{bF}{P}, \frac{D_w + bLk}{\beta} \right).$$

Приравняв два слагаемых и преобразовав выражение, получим $b(F/P - Lk/\beta) = D_w/\beta$. Добавление каждого запроса увеличивает время вычислений на F/P , а время чтения KV — на Lk/β . Если первое значение больше, то с увеличением размера пакета время вычислений догонит время чтения; в противном случае чтение всегда будет занимать больше времени. Приведённые выше значения 51 и 13 сравнивают объёмы чтения двух типов данных, тогда как это уравнение сравнивает ограничения времени выполнения, связанные с вычислениями и пропускной способностью памяти.

Оба слагаемых рассчитываются соответственно по пиковой вычислительной мощности и пиковой пропускной способности, поэтому результат представляет физическую нижнюю границу для данной карты в данных условиях. Отношения соответствующих нижних оценок времени вычислений и чтения к измеренному времени одного раунда — это соответственно MFU и MBU, определённые в разделе 1.2.2. В разделе 8.6.3 эти два отношения будут откалиброваны по пошаговым записям, а затем использованы в остальных расчётных примерах этой главы.

Применительно ко времени обработки запроса нужно также проверить, не задерживает ли увеличение пакета первый токен и не возрастает ли ожидание между соседними выходными токенами. В эксперименте со сканированием размеров пакета фиксировались оба изменения. При входе длиной 2K и выходе длиной 256 токенов для Qwen3-8B увеличение размера пакета с 1 до 64 повысило общую пропускную способность примерно с 37 до 1165 токенов/с, однако время возврата первого токена выросло примерно с 96 мс до 3,29 с, а средний интервал между выходными токенами — примерно с 27 до 40 мс. Хотя каждый раунд выдавал больше токенов, обработка большего объёма входных данных и выполнение более крупного пакета также занимали больше времени. В этом эксперименте все запросы пакета отправлялись одновременно. Если онлайн-сервис дополнительно ожидает заполнения пакета, это время также следует включать в задержку первого токена.⁵

Упражнение 8-1 · Расчёт: изменение объёма чтения при увеличении размера пакета. Примите $D_w = 15, 136, 811, 008$ байт и $k = 144$ KiB. Рассчитайте объём чтения на один выходной токен для контекстов длиной 2K и 8K при размерах пакета 1, 4, 16 и 64, а также для каждой из двух длин контекста найдите минимальный целый размер пакета, при котором объём чтения KV впервые достигает или превышает распределённый объём чтения весов. Затем, используя 12 GiB доступной памяти для KV, блоки по 16 токенов и выход длиной 256 токенов, найдите максимальное число независимых запросов, которое можно принять при каждой из двух длин контекста. Сравните верхний

⁵Исходные записи сканирования batch. RTX PRO 6000, vLLM 0.23.0, BF16, три прогона; в основном тексте приведена медиана, пропускная способность охватывает оставшийся prefill и генерацию, исключая загрузку и прогрев; для условия 8K предварительно создавался общий префикс.

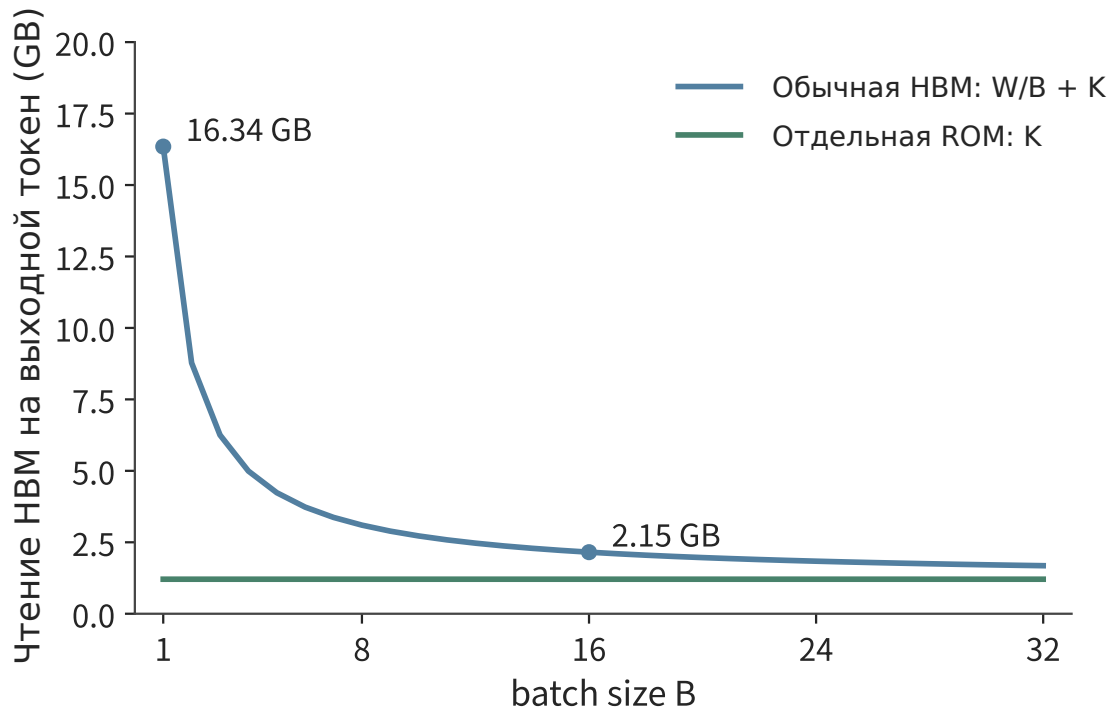


Рис. 8-4. Распределённый объём чтения из HBM на один выходной токен. При фиксированном контексте длиной 8K традиционному пути соответствует $W/B + K$, а пути с независимым быстрым ROM — K . Значения по вертикальной оси рассчитаны по приведённым выше формулам. W — число байтов весов, считываемых для всего пакета, B — число запросов в пакете, K — число байтов KV, считываемых на текущем шаге для одного запроса; $W/B + K$ — распределённый объём чтения на один выходной токен.

предел числа размещаемых запросов с размером пакета, при котором объёмы чтения двух типов становятся равными, и определите, помешает ли ограничение ёмкости дальнейшему увеличению пакета раньше этого момента.

Если веса предоставляются достаточно быстрым независимым хранилищем, время их чтения сокращается, поэтому выигрыш от пакетной обработки необходимо пересчитать. Используем пример с контекстом 8K из главы 4: пусть W — число байтов весов, считываемых для всего пакета, B — число запросов в пакете, а K — число байтов KV, считываемых на текущем шаге для одного запроса. В традиционной схеме объём чтения из HBM в расчёте на один выходной токен равен $W/B + K$. Если веса предоставляются достаточно быстрым независимым ROM, то на каждый выходной токен из HBM по-прежнему требуется считывать K .⁶ На рис. 8-4 расстояние между двумя кривыми сокращается с увеличением размера пакета: это означает, что прежний выигрыш от распределения чтения весов между запросами пакета исчезает.

Тем не менее размер пакета по-прежнему влияет на загрузку вычислительных блоков при матричных операциях и на объём чтения KV внутри пакета. Подставив новый объём чтения в приведённую выше модель времени, можно найти точку пересечения ограничений вычислений и чтения KV, а затем выбрать размер пакета с учётом задержки первого токена и интервала между выходными токенами.

⁶Покомпонентный повторный расчёт сравнений условий в этой главе и программа расчёта.

8.2.2 От фиксированного пакета к непрерывной пакетной обработке

Два запроса одновременно начинают генерацию: одному требуется восемь токенов, другому — только два. После завершения короткого запроса длинному предстоит выполнить ещё шесть шагов. Если принимать следующую группу только после завершения обоих запросов, освободившаяся позиция короткого запроса всё это время будет простаивать. **Фиксированная пакетная обработка** принимает и завершает запросы целыми группами, тогда как непрерывная пакетная обработка перераспределяет активные запросы на границах итераций и принимает новые запросы сразу после освобождения позиций.

Анализ размера пакета в разделе 8.2.1 предполагает, что в одном раунде участвуют b запросов, однако фактическое число строк изменяется по мере завершения и добавления запросов. Позиция выполнения на рис. 8-1 соответствует здесь одной строке вычислений и представляет другой ресурс, нежели слот для регистрации коммуникационного запроса из главы 7. От своевременного добавления новых запросов зависит, сколько строк сможет участвовать в следующем раунде вычислений.

Более раннее включение нового запроса также означает, что обработка его входа раньше займёт ускоритель. Если у нового запроса длинный пром프트, `prefill` увеличит время ожидания между двумя шагами `decode` уже выполняющихся запросов. Поэтому планировщик должен распределять время ускорителя между двумя задачами: обработкой входа новых запросов и продолжением генерации уже выполняющихся.⁷

Расчётный пример: почему непрерывная пакетная обработка сокращает общее время, но увеличивает интервал между выходными токенами? Одновременно могут выполняться не более двух запросов. `r0` и `r1` поступают в нулевой момент времени; каждый содержит 2048 входных токенов, а их выходы составляют соответственно 8 и 2 токена. `r2` поступает через 20 мс, содержит 8192 входных и 2 выходных токена. `r3` поступает через 30 мс, содержит 2048 входных и 4 выходных токена. Время каждого раунда аппроксимировано по измерениям эксперимента 8-1 для пакета размера 1: фиксированные 26,22 мс, дополнительно 30,4 мкс на каждый новый токен и 3 нс на каждую причинно допустимую пару запроса и ключа. Эти параметры воспроизводят время одного раунда `decode` 26,26 мс и время `prefill` 94,8 мс при отдельном выполнении запроса длиной 2K. Фиксированная часть — это не зависящее от числа токенов время одного раунда, включающее чтение всех весов, которое при пиковой пропускной способности 1792 GB/s занимает около 8,4 мс, а также накладные расходы на последовательный запуск ядер и другие операции.⁸

При фиксированном пакете `r2` и `r3` ожидают завершения всей группы `r0` и `r1`, поэтому общее время составляет около 870 мс. Непрерывная пакетная обработка принимает `r2` после завершения `r1` и сокращает общее время примерно до 766 мс, однако следующий выход `r0` должен ждать завершения `prefill` длиной 8K для `r2`, поэтому максимальный интервал между выходными токенами увеличивается примерно с 26 до 376 мс. Общее время сокращается примерно на 12%, но одна пауза в выдаче возрастает в 14 раз.

Теперь дополнительно разделим `prefill` `r2` на блоки по 2048 токенов и будем обрабатывать не более 4096 токенов за раунд, как в эксперименте 8-1. В каждом раунде сна-

⁷Оригинальная статья Orca и блочные вычисления с примечаниями о версиях.

⁸Фиксированный batch, непрерывная пакетная обработка, стратегия разбиения на блоки. Параметры длительности каждого раунда аппроксимированы по поэтапным записям эксперимента 8-1: один раунд `decode` для 2K при batch 1 занимает 26.26 ms, а `prefill` — 94.8 ms; последний выходной токен к моменту завершения запроса ещё не записан в KV.

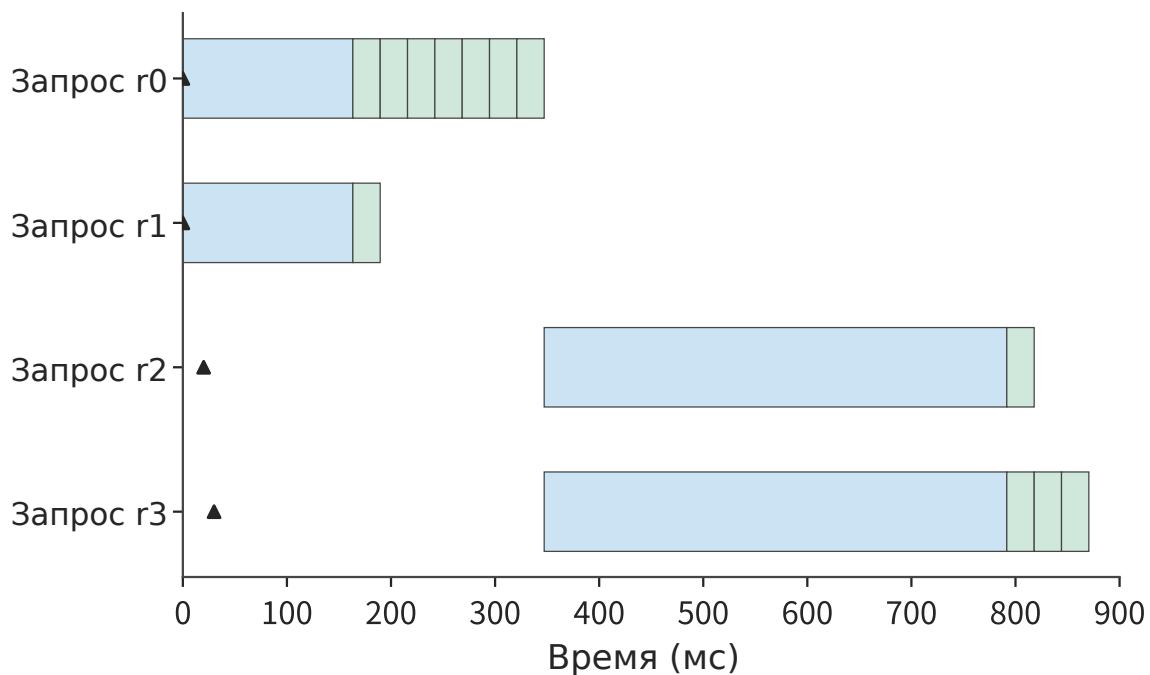


Рис. 8-5. Фиксированный пакет ожидает завершения всей группы и только затем принимает r2 и r3. Синим показан prefill, зелёным — decode, треугольниками — моменты поступления запросов. Ширина каждой цветной полосы соответствует длительности всей итерации планирования и рассчитана по аппроксимации измерений на RTX PRO 6000.

чала продолжим decode уже выполняющихся запросов, а затем обработаем часть входа. Максимальный интервал между выходными токенами сократится примерно до 133 мс, а общее время завершения всех запросов составит около 844 мс. После разделения на блоки планировщик может чаще продолжать генерацию уже выполняющихся запросов, благодаря чему интервалы между выходными токенами становятся равномернее.

8.2.3 Влияние длинного prefill на генерацию и блочное выполнение

Чем крупнее блок prefill, обрабатываемый за один раз, тем меньше раундов обработки входа требуется новому запросу. Чем меньше блок, тем раньше уже выполняющиеся запросы получают следующую возможность выполнения. Обычно планировщик сначала запускает запросы, находящиеся на этапе decode, а остаток лимита токенов текущего раунда использует для prefill. Таким образом, вычисление длинного входа распределяется между несколькими интервалами выдачи.

Даже при одинаковой длине блока объём вычислений внимания увеличивается с ростом контекста. Пусть текущий блок содержит c новых токенов, а длина существующего контекста равна h . Первый новый токен обращает внимание на $h + 1$ токенов, второй — на $h + 2$, а последний — на $h + c$. Просуммируем число этих пар:

$$N_{\text{pair}} = (h + 1) + (h + 2) + \dots + (h + c) = ch + \frac{c(c + 1)}{2}.$$

Слагаемое ch соответствует обращениям новых токенов к старому контексту, а треугольное слагаемое — причинному вниманию внутри блока. При $c = 512$ все 131 328

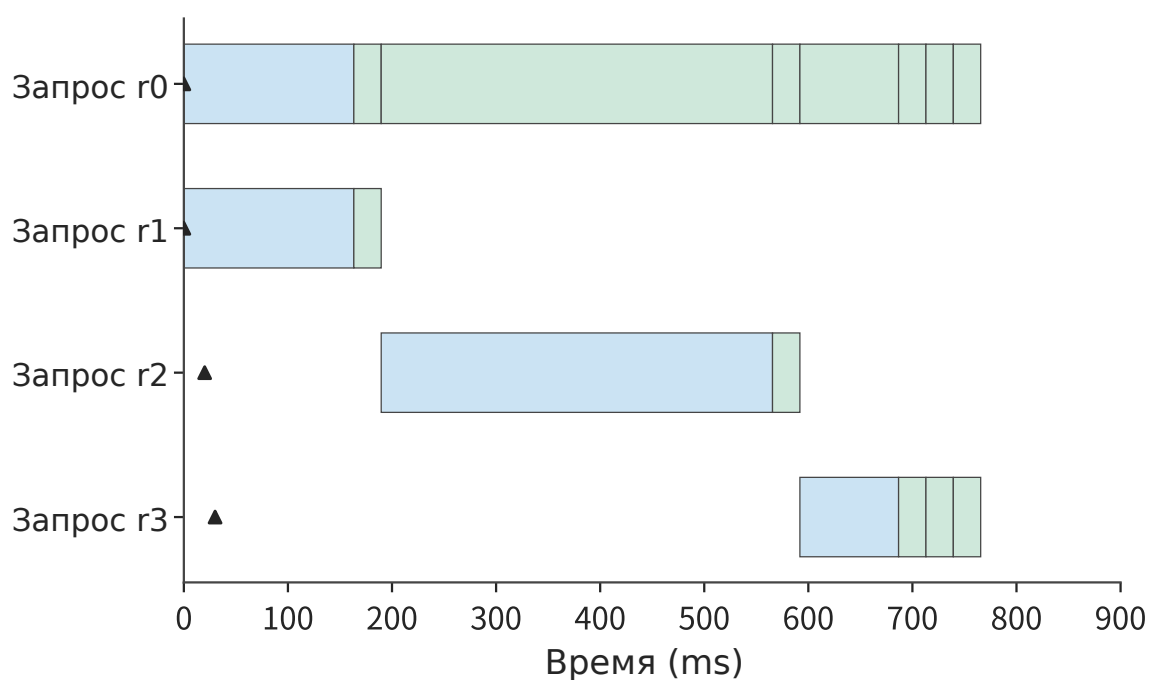


Рис. 8-6. Непрерывная пакетная обработка принимает r2 после завершения r1. Каждая строка соответствует одному запросу; синим показана обработка входа (prefill), зелёным — генерация выхода (decode), треугольниками — моменты поступления запросов, а ширина цветной полосы соответствует длительности содержащей её итерации планирования. r0 ожидает обработки входа r2 длиной 8K, поэтому максимальный интервал между выходными токенами увеличивается примерно до 376 мс; все запросы завершаются примерно за 766 мс.

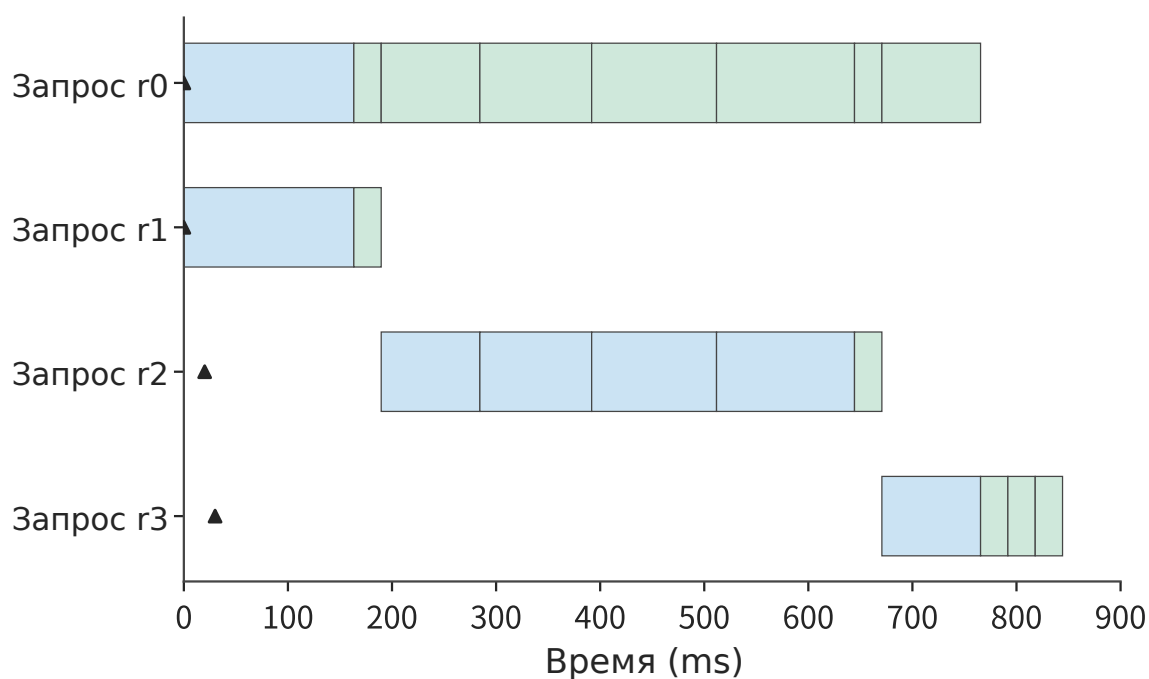
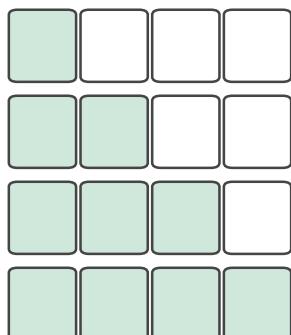


Рис. 8-7. Длинный вход обрабатывается блоками: в каждом раунде сначала планируется генерация для уже выполняющихся запросов, а затем обрабатывается один блок нового входа. Каждая строка соответствует одному запросу; синим показана обработка входа, зелёным — генерация выхода, треугольниками — моменты поступления запросов. Разделение на блоки сокращает максимальный интервал между выходными токенами примерно до 133 мс, а все запросы завершаются примерно за 844 мс. Горизонтальная ось использует тот же масштаб времени, что и на рис. 8-5 и 8-6.

Старый контекст: 0 токенов +
новый ввод: 4 токена



Пары со старым контекстом: 0 + пары внутри блока: 10 = 10

Рис. 8-8. При первой обработке четырёх токенов старого контекста нет, поэтому присутствуют только причинно допустимые пары внимания между новыми токенами, образующие $1 + 2 + 3 + 4 = 10$ зелёных ячеек. Белые ячейки соответствуют будущим токенам и не учитываются при вычислении внимания для текущей позиции.

пар первого блока образуются внутри самого блока. В последнем блоке входа длиной 8K уже имеется 7680 токенов контекста, поэтому число пар возрастает до 4 063 488, что примерно в 31 раз больше, чем для первого блока.

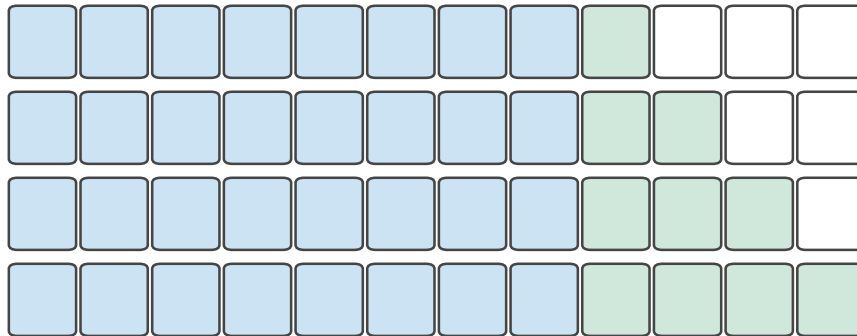
На рис. 8-8 число пар показано в виде площади: каждый новый токен должен обращаться ко всему старому контексту, образуя прямоугольник слева; внутри блока он может обращаться только к текущей и предшествующим позициям токенов, образуя треугольник справа. При фиксированной длине блока правый треугольник не изменяется, а левый прямоугольник расширяется с ростом контекста. Именно поэтому последнему блоку требуется больше вычислений внимания.

Модель также выполняет проекции и FFN. Эти преобразования признаков производятся отдельно для каждого токена, поэтому в каждом блоке обрабатываются 512 новых токенов и объём работы остаётся практически одинаковым. Согласно имеющимся пошаговым записям, объём матричных операций основного тракта модели в последнем блоке примерно на 32% больше, чем в первом, а медиана времени выполнения возрастает примерно с 25,3 до 35,1 мс, то есть приблизительно на 39%. Поэтому время выполнения всего блока определяется двумя составляющими: одна изменяется преимущественно с числом новых токенов, а другая дополнительно возрастает с длиной существующего контекста.⁹

Рост контекста объясняет, почему блоки одинакового размера становятся всё медленнее. Уменьшение длины блока создаёт другой вид накладных расходов. Если разделить

⁹Измерения поблочных вычислений и фиксации состояния. 11 запросов, по 16 блоков в каждом; медианное время первого и последнего блоков составляет 25.30 и 35.07 ms. Интервал CUDA event включает полный путь модели и возможные паузы на стороне хоста, но не внешние logits/сэмплирование. Блоки выполняются в исходном порядке, а входные токены и содержимое меняются от блока к блоку.

Старый контекст: 8 токенов +
новый ввод: 4 токена



Пары со старым контекстом: $32 + \text{внутри блока: } 10 = 42$

Рис. 8-9. При существующем контексте из восьми токенов четыре новых токена образуют со старым контекстом $4 \times 8 = 32$ синие ячейки, а внутри блока по-прежнему остаются 10 зелёных ячеек. При одинаковой длине блока общее число пар возрастает с 10 до 42.

один блок из 256 токенов на два блока по 128 токенов, появляется одна дополнительная возможность планирования, число строк матрицы уменьшается вдвое, но каждый небольшой блок должен использовать один и тот же набор весов. Например, если веса одного слоя FFN занимают 288 MiB и в обоих запусках считываются из видеопамати, объём этого чтения возрастёт с 288 до 576 MiB. Малые блоки позволяют уже выполняющимся запросам раньше получить следующую возможность выполнения, но одновременно увеличивают число запусков. Планировщику следует выбирать максимально крупный блок, удовлетворяющий требованиям к интервалу между выходными токенами, чтобы минимизировать число дополнительных запусков и чтений.¹⁰

8.2.4 Бюджет токенов, динамические формы и время выполнения

Планировщик обычно задаёт верхний предел числа обрабатываемых за раунд токенов, называемый бюджетом токенов. Примером служит предел в 4096 токенов за раунд из расчётного примера с разделением на блоки в разделе 8.2.2. При фактическом выполнении векторы признаков этих токенов нужно объединить в матрицу и выбрать ядро либо заранее захваченный CUDA Graph (раздел 5.5.2). Реальная матрица выполнения может быть больше, чем требуется в текущем раунде, поэтому уменьшение объёма работы на один токен не обязательно сокращает вычисления на соответствующую величину.

Например, можно заранее захватить графы вычислений на 16 и 32 строки, дополняя до 32 строк любой вход размером от 17 до 32 строк. При сокращении работы с 18 до 17 строк всё равно выполняется граф на 32 строки. Только после сокращения с 17 до 16 строк можно использовать меньший граф. Если отдельно захватить граф для 17 строк, можно

¹⁰Обзор OSDI 2025, окончательная версия статьи NanoFlow и пример расчёта совместного использования и размещения ресурсов.

избежать дополнения на 15 строк, однако подготовка такого графа требует времени, а используемые при его воспроизведении буферы будут постоянно занимать память. Поэтому при выборе предустановленных форм CUDA Graph необходимо также учитывать накладные расходы памяти среды выполнения, рассмотренные в разделе 8.1.

Форма выполнения определяет длительность раунда, а лимит токенов — объём работы для новых запросов, выделяемый в этом раунде. Увеличение лимита позволяет раньше завершить обработку длинного входа, однако уже выполняющиеся запросы смогут продолжить выдачу только после окончания раунда. В записи воспроизведения шести запросов новый запрос поступал каждые 80 мс. При увеличении лимита токенов на раунд с 512 до 8192 время ожидания поздних запросов в очереди движка сократилось примерно со 149 мс до значения менее 0,1 мс, однако максимальный интервал между выходными токенами вырос примерно с 32 до 183 мс. Чем выше лимит, тем больше входных данных обрабатывается за один раунд и тем раньше новый запрос покидает очередь, но тем дольше уже выполняющиеся запросы ждут следующего выходного токена.¹¹

Поэтому число обрабатываемых за раунд токенов можно определять в два этапа. Сначала на основании допустимого интервала между выходными токенами нужно установить максимальную длительность одного раунда, а затем, с учётом длины контекста и предустановленных форм выполнения, пересчитать её в число новых токенов, которые можно обработать. За одинаковое время для запроса с более длинным контекстом можно обработать меньше новых токенов, а для запроса с более коротким контекстом — использовать более крупный блок. После определения порядка выполнения необходимо выделить этим запросам пространство KV. В следующем разделе рассматривается распределение и повторное использование этой памяти.

Упражнение 8-2 · Основное · Анализ: определение размера блока prefill по ограничению интервала между выходными токенами. Используя четыре запроса и модель времени выполнения из раздела 8.2.2, вручную рассчитайте длительность первоначального prefill и следующего шага decode для r_0 и r_1 , а затем длительность раунда, в котором непрерывная пакетная обработка принимает r_2 . Объясните происхождение максимального интервала между выходными токенами. При фиксированной длине нового блока $s = 512$ рассчитайте число пар внимания для длин существующего контекста $h = 0$ и $h = 7680$. Наконец, проанализируйте соответствующую запись воспроизведения шести запросов, выберите лимит токенов на раунд отдельно для целей «максимальный интервал менее 50 мс» и «как можно более ранний приём поздних запросов» и укажите, какая составляющая времени определяет выбор.

8.3 Распределение, повторное использование и освобождение KV-кэша

8.3.1 Резервирование, фрагментация и страничное распределение

В разделе 8.1 для каждого запроса резервировалось пространство исходя из максимальной длины вывода, чтобы запрос мог непрерывно генерировать токены. Если запрос

¹¹Натурные измерения для шести запросов и компромиссы при выполнении графов. В эксперименте шесть запросов с синтетическим текстом выполнялись на общей GPU в фиксированном порядке.

Логические блоки упорядочены, физические — разнесены

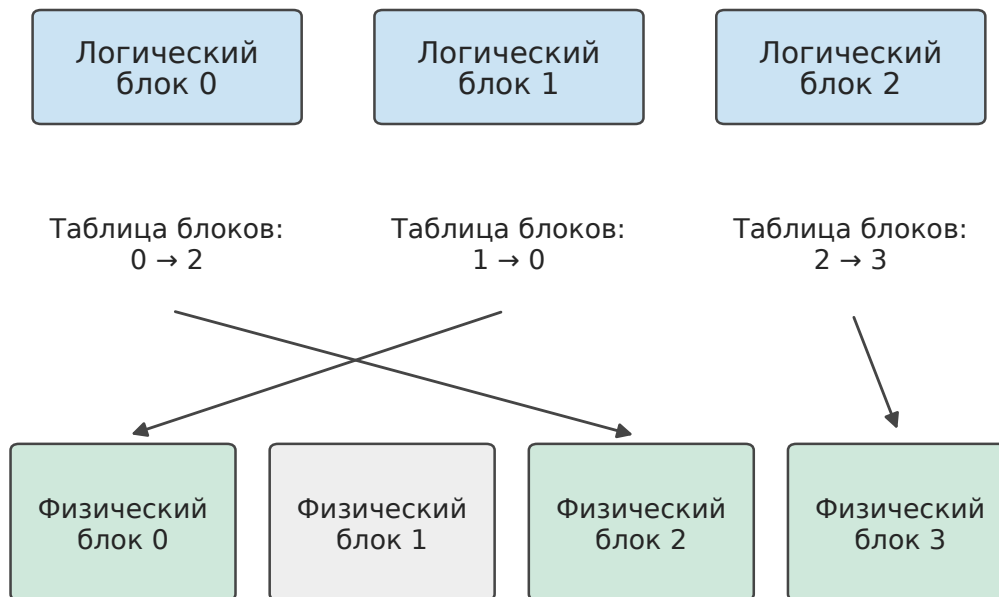


Рисунок 8-10. Логические блоки 0, 1 и 2 последовательно образуют последовательность, а таблица блоков указывает соответственно на физические блоки 2, 0 и 3. Физический блок 1 свободен; логический порядок восстанавливается для вычисления внимания по таблице блоков.

завершается раньше, конец зарезервированного пространства остаётся неиспользованным. Другой подход — постепенно расширять память по мере генерации, но если пространство должно оставаться непрерывным, то при занятой другими запросами соседней области придётся переносить данные или ждать.

Такое распределение можно сравнить с разъёмной папкой: порядок чтения определяется номерами страниц, но сами страницы необязательно должны находиться рядом — достаточно помнить, где находится каждая из них. Когда последовательность растёт, новую страницу можно поместить в любое свободное место и записать её расположение, не перемещая всё содержимое папки ради непрерывного размещения.

Страничное распределение делит последовательность на блоки фиксированного размера. **Логические блоки** нумеруются по позициям токенов в последовательности и соответствуют номерам страниц; **физические блоки** — это области памяти, в которых фактически хранятся KV; поддерживаемая для каждого запроса **таблица блоков** указывает, какому физическому блоку соответствует каждый логический блок. Когда последовательность вырастает настолько, что требуется новый блок, движок выделяет физический блок из пула свободных блоков и добавляет отображение. При вычислении внимания нужные позиции KV находятся по таблице блоков, поэтому физические блоки могут храниться раздельно. Здесь рассматривается блочная адресация KV в памяти; подкачка между уровнями хранения обсуждается в разделах 8.3.4 и 8.4.3.

Пусть каждый блок может хранить KV для p токенов, а текущая длина равна L . Тогда

требуется $\lceil L/p \rceil$ блоков. Неиспользованные слоты KV остаются только в конце последнего блока, а потери составляют

$$p \left\lceil \frac{L}{p} \right\rceil - L,$$

то есть не более пространства KV для $p - 1$ токенов. Чем меньше блоки, тем ниже эта верхняя граница, но тем больше записей в таблице блоков и операций распределения. В 2023 году исследователи из Беркли и других организаций представили в vLLM технологию PagedAttention, применив к KV-кэшу принципы страничного управления памятью из операционных систем, чтобы снизить влияние резервирования и фрагментации памяти на параллельную обработку запросов сервисом. PagedAttention переносит такое отображение в выполнение внимания, тогда как FlashAttention организует блочные вычисления и промежуточные результаты внутри оператора. Таким образом, первая технология управляет распределением состояния, а вторая — вычислениями и обращениями к памяти.

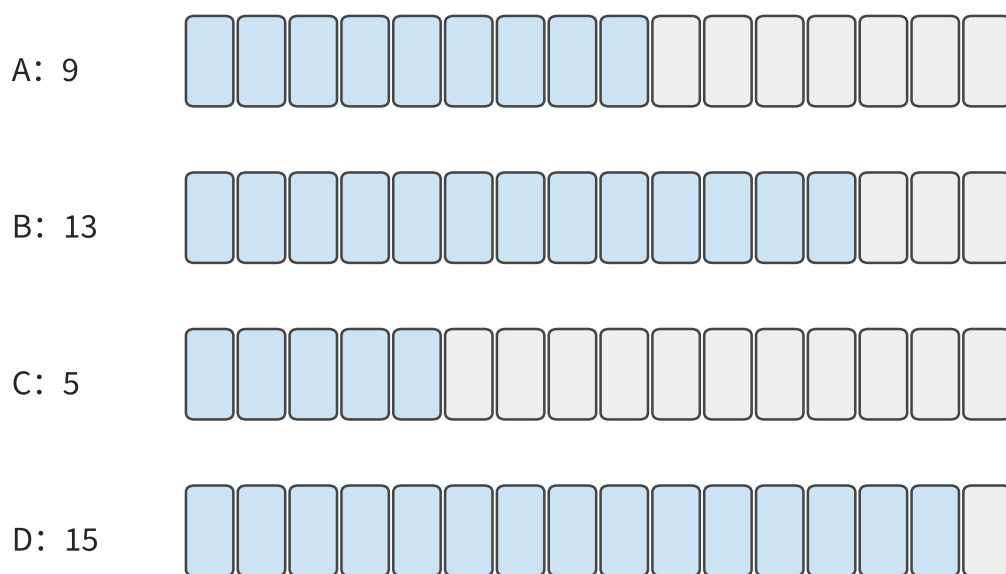
Пример расчёта: насколько страничное распределение KV сокращает резервируемое пространство? Пусть текущие длины A, B, C и D равны соответственно 9, 13, 5 и 15, а для каждой последовательности разрешено не более 16 токенов. При резервировании целыми сегментами четыре запроса совместно резервируют пространство KV для 64 токенов. При $p = 4$ страничное распределение выделяет им пространство соответственно для 12, 16, 8 и 16 токенов — всего для 52 токенов. При этом фактически сохранённый KV по-прежнему соответствует 42 токенам, а неиспользованная ёмкость сокращается с 22 до 10 токенов.

Если первые восемь токенов A и B совпадают, они могут совместно использовать два полных блока, что дополнительно снижает общий объём выделенного пространства до 44 токенов. Страничное распределение сокращает ещё не использованное зарезервированное пространство, а совместное использование устраняет повторное хранение контекста. Эти механизмы экономят разные части памяти.

8.3.2 Совместное использование ветвями, копирование при записи и освобождение памяти

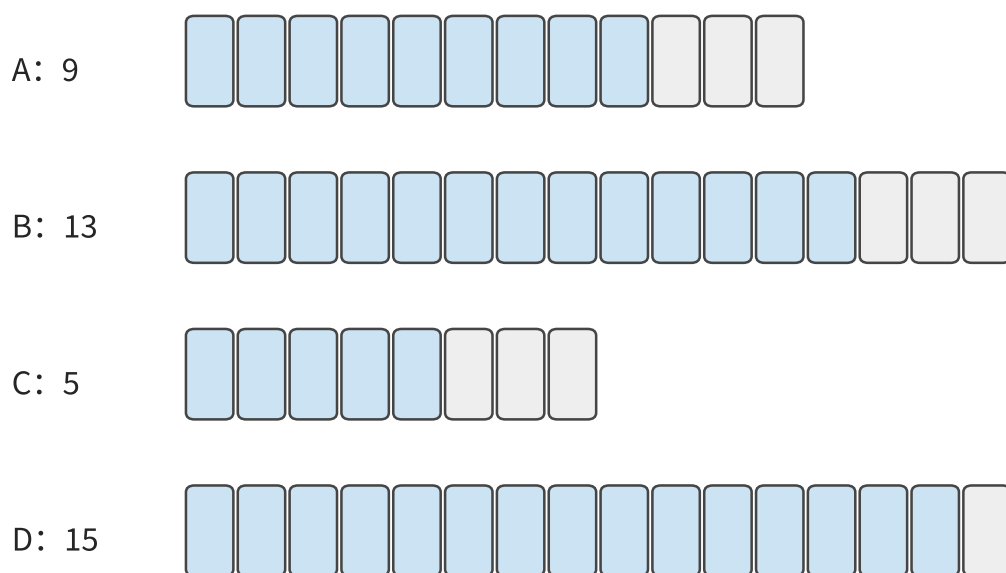
На схеме совместного использования префикса B и A ссылаются на одни и те же два физических блока. Если A завершится первым, эти два блока всё равно нужно сохранить для B; если B потребует изменить их содержимое, это не должно повлиять на A. Для реализации совместного использования необходимо определить, когда освобождать общие блоки и как предотвращать перезапись данных разными запросами. Каждая запись таблицы блоков, указывающая на физический блок, называется **ссылкой**; на общий блок ссылаются несколько таблиц блоков. **Счётчик ссылок** показывает, скольким пользователям он ещё нужен. Когда запрос получает ссылку, её счётчик увеличивается на единицу; при завершении запроса ссылка освобождается. Блок может вернуться в пул свободных блоков только после освобождения последней ссылки; кроме того, к этому моменту нужно убедиться, что операции ускорителя больше к нему не обращаются. Благодаря этому одна копия общего контекста может обслуживать несколько ветвей и продолжать обслуживать оставшиеся ветви после завершения одной из них.

Для записи в общий хвостовой блок необходимо получить отдельную копию. Например, в блок, вмещающий KV четырёх токенов, уже записаны KV трёх общих токенов,



Выделено под KV: 64 токена

Рисунок 8-11. Четыре запроса содержат соответственно 9, 13, 5 и 15 токенов. Для каждого резервируется пространство KV на 16 токенов, поэтому суммарно выделяется пространство на 64 токена. Синим показано используемое пространство, серым — зарезервированное, но неиспользуемое.



Выделено под KV: 52 токена

Рисунок 8-12. При распределении пространства KV блоками по 4 токена четыре запроса получают ёмкость соответственно на 12, 16, 8 и 16 токенов — всего на 52 токена. Серым показана неиспользованная ёмкость в конце блоков, которая сократилась с 22 до 10 токенов.

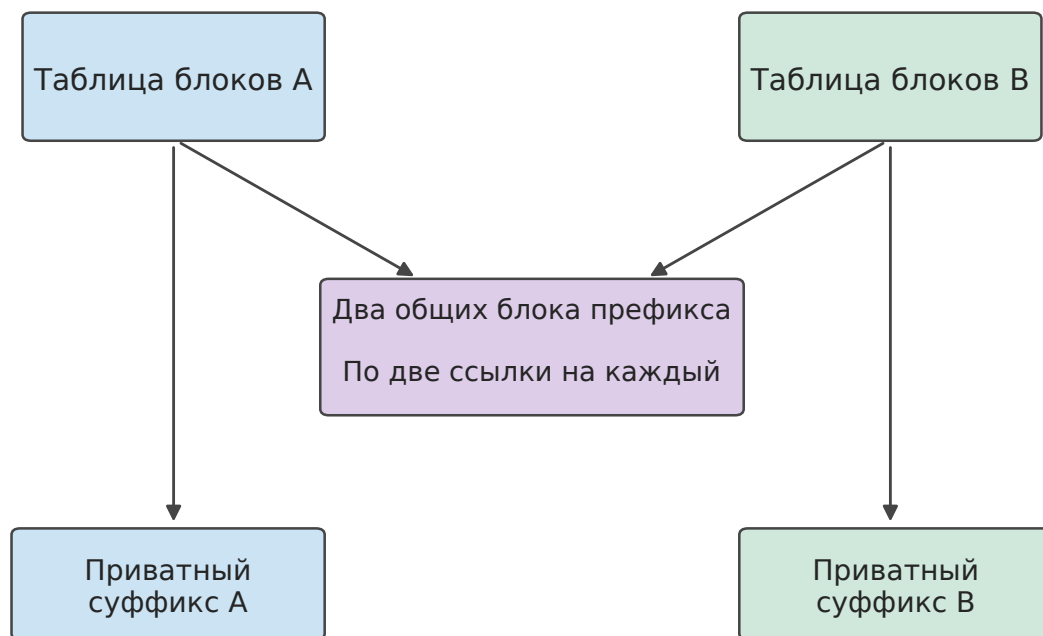


Рисунок 8-13. Два общих префиксных блока А и В сохраняются только один раз, а таблицы блоков обоих запросов ссылаются на них. У А и В остаются собственные хвостовые блоки; фактически выделенная для четырёх запросов ёмкость KV дополнительно сокращается до 44 токенов.

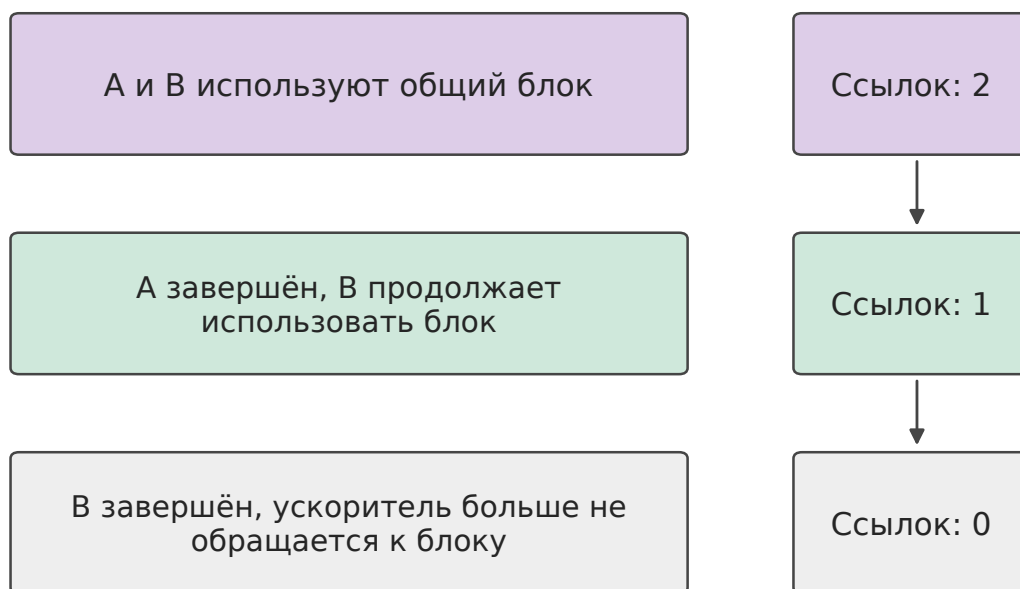


Рисунок 8-14. После завершения А счётчик ссылок уменьшается с 2 до 1, и В по-прежнему может использовать блок. Блок возвращается в пул свободных блоков только после освобождения последней ссылки и завершения работы ускорителя с ним.

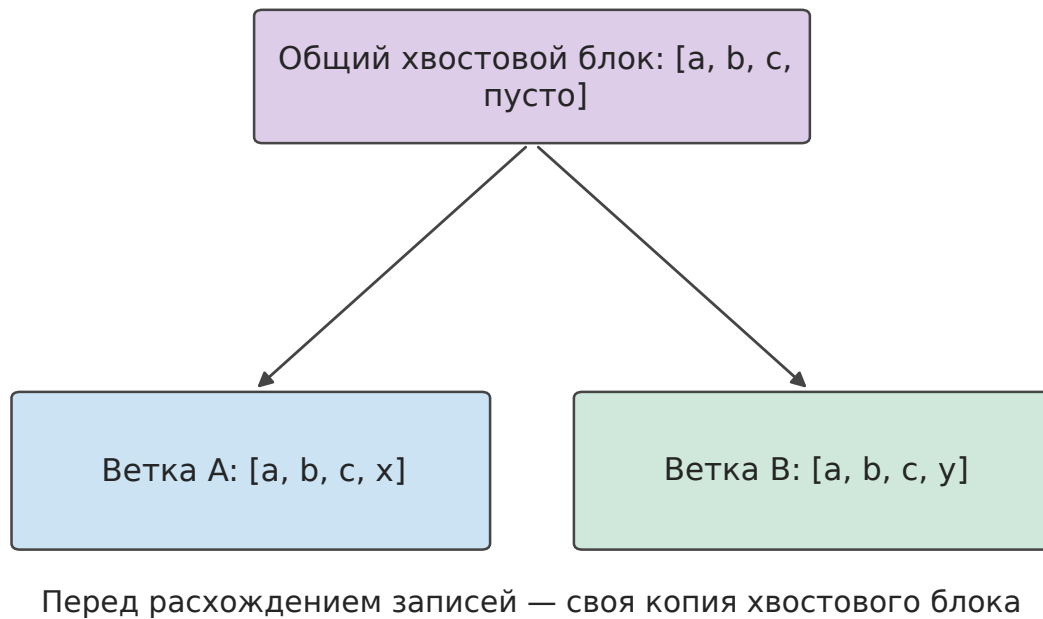


Рисунок 8-15. В хвостовой блок размером 4 токена уже записаны общие a, b и c. Ветви добавляют соответственно x и y, поэтому им требуются разные физические хвостовые блоки; ранее заполненные блоки продолжают использоваться совместно.

а затем две ветви должны записать разные токены. Если писать в исходный блок, обе ветви будут конкурировать за четвёртый токен в блоке. **Копирование при записи** — это механизм получения собственной копии перед изменением общего содержимого. В данном случае сначала копируются эти три токена, после чего каждая ветвь отдельно добавляет свой токен; уже заполненные доступные только для чтения блоки продолжают использоваться совместно. По мере роста ветвей общий префикс хранится в одном экземпляре, а каждая ветвь отдельно сохраняет добавленный ею суффикс.

В проведённом ранее эксперименте с четырьмя ветвями общий префикс занимает 95 блоков, а каждая ветвь дополнительно использует 9 собственных блоков. Четыре таблицы блоков содержат в общей сложности $4(95 + 9) = 416$ ссылок, но фактическое число физических блоков равно лишь

$$95 + 4 \times 9 = 131.$$

При независимом хранении потребовалось бы 416 блоков, поэтому совместное использование сокращает их число примерно на 69%. Эта экономия достигается за счёт устранения дублирования общего префикса; при четырёх ветвях остаются 36 собственных хвостовых блоков, и в общем случае их число по-прежнему растёт вместе с числом ветвей.¹²

¹²Страничная организация, отмена и вытеснение, совместное использование четырьмя ветвями, сравнение с холодными ветвями. Результаты общих ветвей совпадают; холодные и горячие ветви имеют одинаковый пиковый объём блоков, но различный объём prefill.

Система обслуживания знает, какие токены относятся к общему префиксу, а какие после ветвления будут записываться независимо, поэтому может воплотить отношения повторного использования модели в совместном использовании физических блоков. Таблица блоков отвечает за отображение адресов, отношения между префиксами определяют область совместного использования, а собственная копия создаётся только при записи в ветвь. Таким образом, пространство, ранее независимо резервировавшееся для каждого запроса целиком, начинает распределяться в соответствии с фактически создаваемым и повторно используемым состоянием.

Полная ёмкость KV для пакета после совместного использования общего префикса. 16 длинных запросов совместно используют первые 6144 входных токена, KV которых занимает в общей сложности 864 MiB. Собственная часть каждого запроса содержит $8192 - 6144 + 255 = 2303$ токена. При распределении блоками по 16 токенов требуется вместить 2304 токена, то есть 324 MiB. Поэтому после завершения генерации всей группой запросов необходимое пространство KV составляет

$$M_{KV} = 864 + 16 \times 324 = 6048 \text{ MiB.}$$

Ранее этой группе запросов требовалось $16 \times 1188 = 19008$ MiB, а после совместного использования — около 5,91 GiB, что помещается в зарезервированные 12 GiB памяти. В общем случае b однотипных длинных запросов занимают $864 + 324b$ MiB, поэтому предел ёмкости увеличивается с 10 независимых запросов до 35 запросов с совместным использованием.

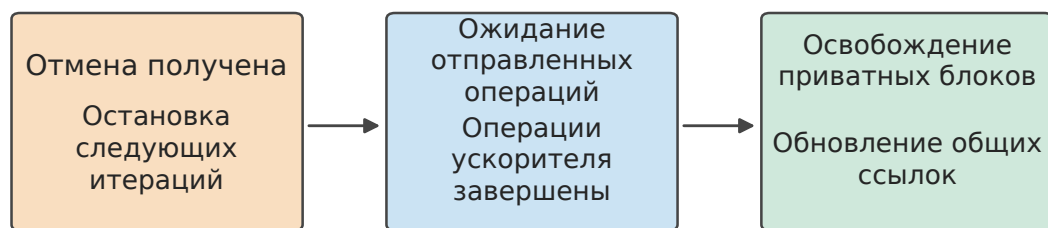
После использования блоки необходимо корректно освободить, чтобы передать последующим запросам. Для освобождения памяти должны одновременно выполняться два условия: ни один пользователь больше не удерживает ссылку, а уже отправленные операции ускорителя больше не обращаются к блоку. После отмены запроса пользователем планировщик перестаёт назначать новые вычисления, ожидает завершения уже отправленных операций, а затем освобождает собственные блоки. В одном наблюдении вызов отмены вернулся примерно через 1,6 ms, но соответствующие 104 блока были освобождены лишь примерно через 31 ms. Последующие запросы начинают использовать эти блоки после освобождения и потому не перезаписывают данные, которые ещё читает старый запрос.¹³

Помимо обычного освобождения после завершения или отмены запроса, при нехватке памяти движок может вытеснить запрос: приостановить его и освободить KV, а затем повторно вычислить состояние по сохранённым входным и выходным данным. В одной и той же серии экспериментов при использовании пула KV размером 1 GiB произошло одно вытеснение, из-за которого по сравнению с обычным выполнением пришлось дополнительно запланировать обработку 1805 токенов; с пулом размером 2 GiB этого повторного вычисления не было. Вытеснение временно освобождает память для других запросов, но при возобновлении выполнения приходится повторять вычисления.¹⁴

Упражнение 8-3. Расчёт: как размер блока KV и совместное использование префикса влияют на ёмкость. Разбейте четыре последовательности длиной 9, 13, 5 и 15 на страницы по 2, 4 и 8 токенов. Для каждого варианта

¹³Наблюдения за отменой получены из эксперимента с состоянием. Зафиксированы значения около 1.55 ms и 31.41 ms; измеренное время включает накладные расходы наблюдателя.

¹⁴Страничная организация, отмена и вытеснение, совместное использование четырьмя ветвями, сравнение с холодными ветвями. Результаты общих ветвей совпадают; холодные и горячие ветви имеют одинаковый пиковый объём блоков, но различный объём prefill.



Отмена вернулась за 1.6 ms; блоки освобождены примерно за 31 ms

Рисунок 8-16. Возврат из интерфейса отмены означает, что запрос на отмену принят. Собственные блоки и связанные ссылки освобождаются после завершения уже отправленных операций ускорителя; наблюдавшиеся значения 1,6 ms и 31 ms относятся к разным событиям.

вычислите число неиспользованных слотов KV в хвостовом блоке каждой последовательности, а затем общее число неиспользованных слотов KV и записей в таблицах блоков. Затем по примеру расчёта в начале главы определите полную ёмкость для 8, 16 и 32 длинных запросов при независимом и совместном хранении состояния. Сохраняя общую длину каждого запроса неизменной, сделайте один ранее собственный полный блок частью общего префикса и определите, на сколько физических блоков меньше потребуется выделить в общей сложности для b запросов.

8.3.3 Повторное использование префиксов в диалогах и AI-агентах

В разделе 8.3.2 было рассчитано, как одновременно выполняемые запросы могут совместно использовать физические блоки. Такое совместное использование может продолжаться и после завершения запроса: оставшийся от старого запроса KV префикса позволяет последующему запросу не вычислять повторно тот же фрагмент входных данных. Если префикс последующего запроса полностью совпадает и используются те же модель, правила позиционного кодирования, адаптер и формат состояния, можно напрямую применить сохранённый в кэше KV и продолжить `prefill` с конца префикса. Общий префикс из примера в начале главы содержит 6144 токена; при попадании в кэш для каждого длинного запроса потребуется обработать лишь оставшиеся 2048 входных токенов, после чего можно начинать генерацию.

Такое повторное использование непрерывно распространяется вдоль префикса. Предположим, у двух промптов совпадают первые 100 токенов, а 101-й различается. Даже если последующий текст снова совпадает, начиная со 101-го токена он уже зависит от разных контекстов. Если поместить динамическую временную метку в начало промпта, расположенные далее длинные общие определения инструментов нельзя будет использовать повторно. Если же сначала разместить стабильный системный промпт и определения инструментов, а затем добавить изменения текущего раунда, сохранится более длинный общий префикс.¹⁵

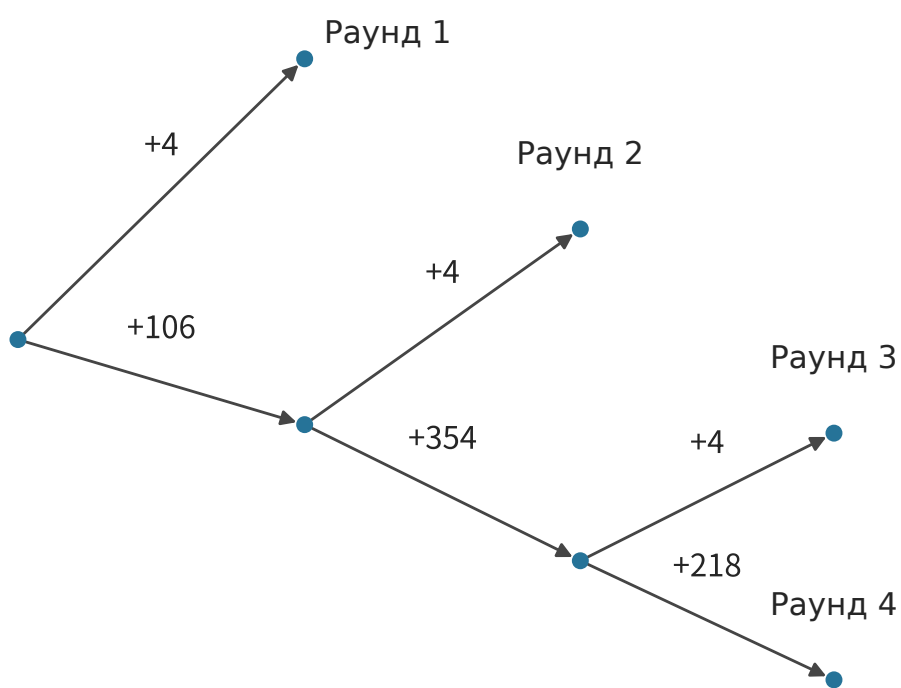
Префиксное дерево непосредственно выражает эту структуру: путь от корня до точки ветвления представляет общий контекст, а пути от точки ветвления до листьев — собственные суффиксы. Первые четыре раунда входных данных на рисунке 8-17 сначала совместно используют 206 токенов, а следующие три продолжают тот же путь. Рёбра дерева помечены числом добавленных токенов, а листья указывают полную длину входных данных.

Добавление содержимого в конец каждого раунда позволяет сохранить существующий общий префикс, но чем длиннее контекст, тем больше объём KV и объём последующего чтения.

Расширение: как модель с гибридным вниманием определяет позицию восстанавливаемого префикса. Полное внимание сохраняет KV для каждого токена, тогда как рекуррентная модель обычно сохраняет только текущее состояние. Пусть текст совпадает до токена 10 752, а рекуррентные снимки сохранены на токенах 4096 и 8192. Система восстанавливает состояние с позиции 8192, а затем вычисляет его до позиции 10 752, повторно обрабатывая 2560 токенов. Дополнительные снимки позволяют сократить число повторно вычисляемых токенов, но требуют хранения большего объёма состояния. Например, для представленной в главе 2 KDA одна из реализаций распределяет

¹⁵Пример проектирования контекста.

Первые 4 раунда: общий префикс
из 206 токенов



Числа на рёбрах: новые токены

Рисунок 8-17. Сжатое префиксное дерево первых четырёх раундов входных данных агента, работающего с кодом. В корне уже имеется 206 общих токенов, на рёбрах указано число добавленных токенов, а листья соответствуют раундам ввода. Ветвление означает различие последующего содержимого.

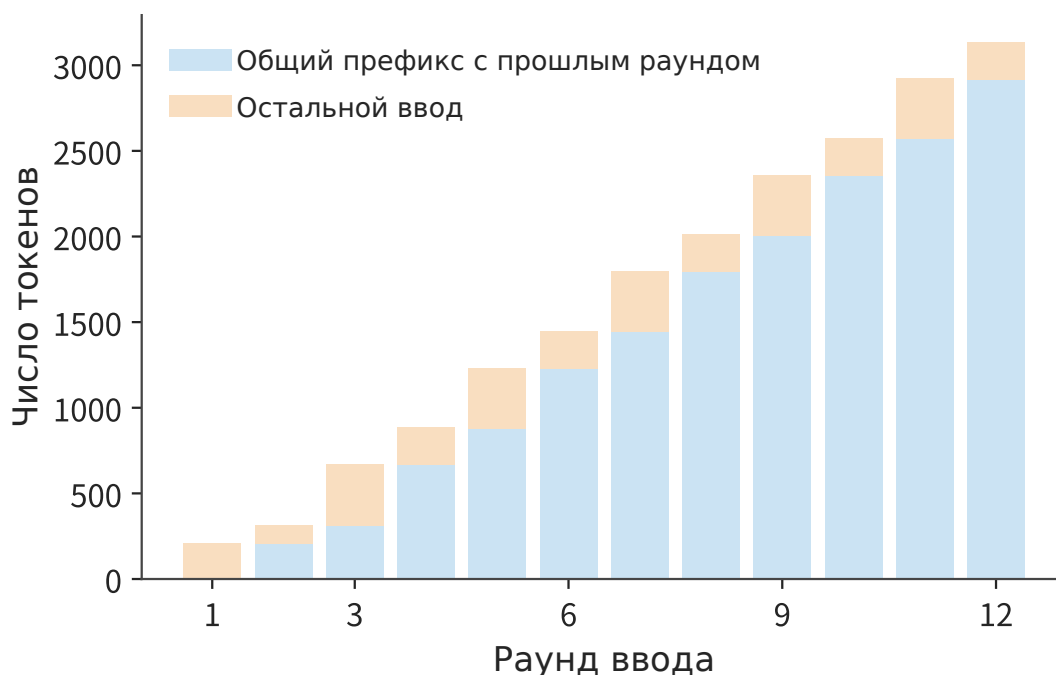


Рисунок 8-18. Синим показан префикс, совпадающий с предыдущим раундом token за tokenом, оранжевым — остальная часть входных данных. Рисунок отражает степень потенциального повторного использования содержимого; фактическое попадание в кэш зависит также от того, сохранилось ли состояние.

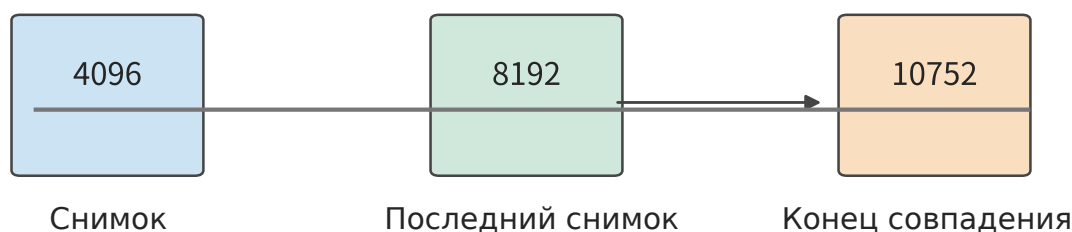
внутрислойные тензоры по восьми картам (TP=8). В этом случае один снимок занимает на каждой карте около 53,6 MiB, два снимка — около 107 MiB, а 32 снимка — около 1714 MiB. Чем чаще сохраняются снимки, тем меньше токенов приходится повторно вычислять при восстановлении, но тем больше сохраняется состояния.¹⁶

Оркестрация контекста также может целенаправленно изменять возможности повторного использования. Предположим, уже имеется контекст из 8K токенов, а в следующем раунде добавляется результат работы инструмента размером 1K токенов. В этом случае можно повторно использовать прежний префикс длиной до 8K и обработать только добавленную часть. Если же переписать инструкции в самом начале, последующее содержимое нельзя будет повторно использовать, даже если текст совпадает. Другой вариант — свести историю к резюме объёмом 2K токенов, а затем добавить ещё 1K. После замены исходной истории резюме последующие вычисления используют более короткий контекст, но возникают дополнительные затраты на создание резюме и повторную обработку. Поэтому при организации контекста необходимо одновременно учитывать объём памяти, время повторных вычислений и объём последующего чтения.

Повторное использование префикса на стороне модели требует более детального различения состояний. При повторном использовании префикса DeepSeek V4.1 отдельно обрабатывает два типа локального состояния. Представленная в главе 2 структура CED содержит каузальный кодировщик и декодировщик, у каждого из которых имеется локальное состояние SWA: **SWA кодировщика** можно кратковременно хранить в

¹⁶Восстановление гибридного состояния и расчёт checkpoint, обзор MLSys 2025. Ёмкость Kimi K3 рассчитана для заданного формата хранения и конфигурации параллелизма.

Текст совпал до 10752, состояние
восстановлено с 8192



Пересчёт 2560 токенов

Рисунок 8-19. Текст совпадает до позиции 10 752, а последний снимок состояния находится на позиции 8192. После восстановления всё ещё требуется повторно вычислить 2560 токенов, чтобы получить рекуррентное состояние в конце совпавшего фрагмента. Значения 10 752 и 8192 на рисунке — это накопленное от начала последовательности число токенов.

пуле DRAM хоста, чтобы в следующем раунде продолжить обработку новых входных данных вдоль существующего префикса; **SWA декодировщика** используется только для последующей генерации в текущем раунде, и в описанном в статье развёртывании оно не сохраняется в качестве кэша префикса. Глобальный KV помещается в кэш для более длительного повторного использования. Поэтому ёмкость резидентного во время генерации SWA 40 слоёв и ёмкость сохраняемого между раундами кэша префикса необходимо рассчитывать отдельно.¹⁷

Попадание этих двух типов состояния в кэш определяет, какую часть префикса кодировщику придётся восстанавливать в следующем раунде. Предположим, после возврата результата инструментом необходимые для восстановления токены или входные представления по-прежнему доступны, а версия модели, индексы позиций токенов и входные данные не изменились. Если в кэш попали и глобальный KV, и SWA кодировщика, кодировщик напрямую обрабатывает добавленные входные данные. Если попал только глобальный KV, кодировщик повторно обрабатывает не более 128 последних токенов сохранённого в кэше префикса вместе с отсутствующим в кэше суффиксом. Повторная обработка восстанавливает SWA кодировщика, при этом существующий глобальный KV продолжает использоваться; новый суффикс создаёт новые глобальный KV и SWA. Если же нет попадания и по глобальному KV, отсутствующий префикс вычисляется заново.

Затем каждый prefill включает общий этап: берутся выходные данные кодировщика не более чем для 128 последних токенов полного промпта и пропускаются через 20

¹⁷Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия межглавного сеанса и повторный расчёт.

Одна история, три способа обновления



Рисунок 8-20. Три способа обновления одной истории. Синим показан повторно используемый префикс, оранжевым — входные данные, которые требуется обработать заново; в варианте с резюме сначала создаётся резюме, а затем заново строится кэш. Длина условно указана в K токенов.

слоёв декодировщика, приблизительно формируя SWA декодировщика и подготавливая локальное состояние для первого шага decode. Поэтому при полном попадании состояния кодировщика в кэш можно избежать восстановления префикса кодировщика, но декодировщику всё равно необходимо повторно обработать окно.

Состояние, полученное короткой повторной обработкой окна, восстанавливается приближённо: ошибка возникает из-за отсечения более ранних локальных зависимостей. Хотя каждый слой обращается только к последним 128 токенам, при наложении нескольких слоёв локальное состояние всё равно может косвенно зависеть от более ранних входных данных через предыдущий слой. Повторное выполнение от границы окна изменяет эту часть исторической информации. Поэтому в разделе 3.2.2 технического отчёта восстановленное состояние кодировщика определяется как приближённое; оно участвует в последующих вычислениях, из-за чего глобальный KV и SWA нового суффикса зависят от начальной точки восстановления.

Различие между этими двумя типами состояния в конечном счёте отражается на ёмкости кэша. При нагрузке и стратегии кэширования из технического отчёта DeepSeek V4 SWA занимает около половины долговременного кэша. В V4.1 эта часть удалена из постоянного кэша, а глобальный KV сжат примерно до четверти прежнего объёма, поэтому долговременное хранилище составляет около $1/2 \times 1/4 = 1/8$ исходного. Кратковременно сохраняемое SWA кодировщика размещается в DRAM, а SWA декодировщика остаётся на ускорителе во время генерации текущего раунда. Разделение кэша по уровням хранения в зависимости от периода использования дополнительно снижает требования к долговременной ёмкости.

8.3.4 Допуск в кэш, вытеснение, выгрузка и повторное вычисление

Префикс сохраняется после завершения запроса, чтобы избежать повторных вычислений при его будущем использовании. Пусть вероятность повторного использования префикса равна p_h , время повторного вычисления — T_r , время получения состояния из кэша и его подготовки — T_f , а время обслуживания кэша — T_m . Каждое попадание экономит $T_r - T_f$; умножив эту величину на вероятность попадания и вычтя время обслуживания, получим ожидаемую чистую экономию:

$$V_h = p_h(T_r - T_f) - T_m.$$

Возьмём общий префикс из 6144 токенов для длинных запросов из примера в начале главы. Его KV занимает 864 MiB (0,906 GB). Для повторного вычисления этого префикса на RTX PRO 6000 требуется выполнить 96,5 TFLOP матричных операций. Если исходить из 62% пиковой производительности BF16, достигнутой для batch 1 prefill в эксперименте 8-1 (раздел 8.6.3), это занимает около 307,9 ms. После выгрузки префикса в память хоста его получение через линию PCIe Gen5 x16 этой карты с пропускной способностью 64 GB/s в каждом направлении занимает около 14,2 ms;¹⁸ затраты на обслуживание принимаются равными времени одной передачи при выгрузке в память хоста, то есть также

¹⁸Спецификация рабочей станции RTX PRO 6000 Blackwell: системный интерфейс PCIe 5.0 x16 с удвоенной относительно PCIe Gen4 пропускной способностью; спецификация A100 80GB: совокупная пропускная способность PCIe 4.0 на приём и передачу составляет 64 GB/s, то есть 32 GB/s в каждом направлении для x16. Следовательно, для PCIe Gen5 x16 она составляет 64 GB/s в каждом направлении. Расчёты в основном тексте используют номинальные значения и дают нижнюю границу времени передачи. Время повторного вычисления префикса рассчитано по таблице эффективности эксперимента 8-1.



Рисунок 8-21. Три пути восстановления префикса кодировщика и общая повторная обработка декодировщиком. SWA в состоянии попадания в кэш относится только к кодировщику; при любом пути во время prefill по-прежнему строится SWA декодировщика, после чего начинается генерация. Стрелки обозначают порядок выполнения, а размеры блоков не отражают затраченное время.

14,2 ms. При вероятности попадания 50% средняя экономия составляет 132,7 ms. Чтобы сохранение префикса сэкономило время, должно выполняться условие

$$p_h > \frac{T_m}{T_r - T_f} = \frac{14.2}{293.7} = 4.8\%.$$

Если заменить линию связи с хостом на PCIe Gen4 x16 с пропускной способностью 32 GB/s в каждом направлении, получение и выгрузка займут по 28,3 ms, а каждое попадание будет экономить лишь 279,6 ms. Необходимая минимальная вероятность попадания возрастёт до 10,1%. Выгрузка освобождает пространство ускорителя, но компенсировать время получения и обслуживания можно лишь при достаточно частом использовании префикса.

То, что префикс стоит сохранять, ещё не означает, что ему следует отдавать приоритет. При нехватке пространства большой префикс может занять место, в котором можно было бы хранить несколько малых префиксов. Пусть А — описанный выше префикс из 6144 токенов. Каждый префикс типа В содержит 2048 токенов и занимает 288 MiB. При таком же получении и выгрузке через PCIe Gen5 повторное вычисление занимает 94,7 ms, получение и обслуживание — по 4,7 ms, а при той же вероятности попадания 50% ожидаемая экономия составляет 40,3 ms. На рисунке 8-22 оба варианта помещены в кэш одинакового размера: сохранение одного А экономит 132,7 ms, а сохранение трёх В — в общей сложности 120,9 ms. Время повторного вычисления растёт сверхлинейно относительно длины префикса: prefill для 6144 токенов занимает в 3,25 раза больше времени, чем для 2048 токенов, поэтому использование одного и того же объёма памяти для длинного префикса приносит большую выгоду.

Это сравнение можно также представить как выгоду на единицу ёмкости: А экономит около 0,154 ms на MiB, а В — около 0,140 ms на MiB. Сортировка по этому значению позволяет сравнивать префиксы разных размеров.

У приведённого выбора есть предпосылка: префикс должен сохраняться до следующего использования. При недостаточной ёмкости кэша состояние может быть вытеснено, и тогда его придётся вычислять заново, даже если следующий раунд входных данных сохраняет тот же префикс. Одно воспроизведение 12 раундов входных данных агента содержало 19 556 входных токенов. При одинаковых мешающих запросах в пуле кэша размером 6 GiB в кэш попало 16 304 токена, поэтому заново потребовалось обработать лишь 3252 токена. В пуле размером 1 GiB попаданий не было, и пришлось заново обработать все входные данные. После увеличения времени ожидания между соседними раундами на 0,2 секунды число попаданий не изменилось. В этом сравнении увеличение ёмкости сохранило пригодный для повторного использования префикс, тогда как простая задержка следующего запроса не дала такого эффекта.¹⁹

Допуск в кэш определяет, сохранять ли префикс после завершения запроса; политика вытеснения определяет, какой префикс удалить первым при нехватке пространства; политика выгрузки — на какой уровень хранения переместить состояние; политика повторного вычисления — сколько входных токенов обработать заново. Для 16 длинных запросов из примера в начале главы сохранение одной копии общего префикса требует лишь 864 MiB, но одновременно сокращает дублирование при хранении и вычисления prefill для последующих запросов.

¹⁹Кэшированное воспроизведение фактических входных данных за 12 раундов. При воспроизведении используется исходная последовательность входных данных задачи, запросы выполняются последовательно, а на каждом раунде генерируется один токен.

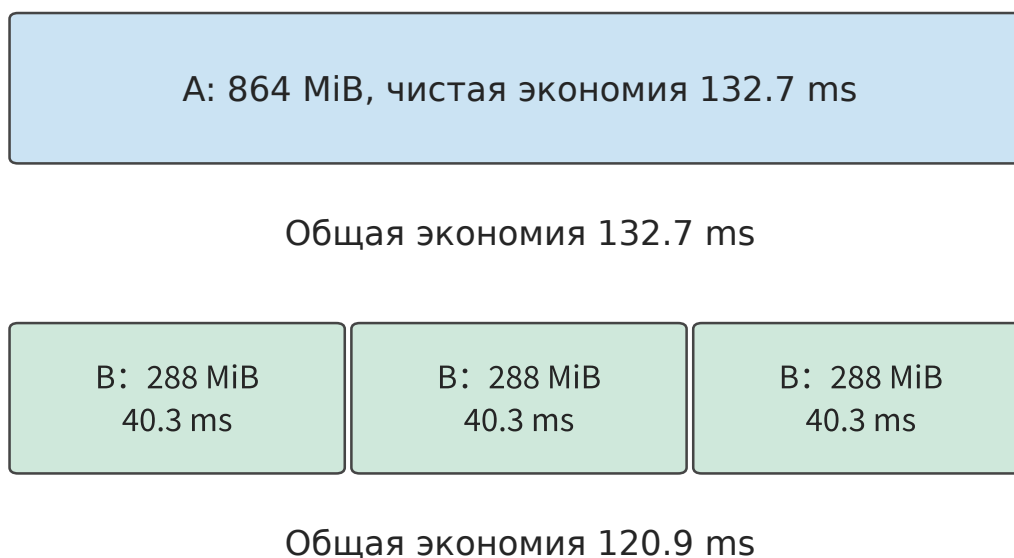


Рисунок 8-22. В обоих случаях ёмкость кэша составляет 864 MiB. А занимает 864 MiB и обеспечивает ожидаемую чистую экономию 132,7 ms; каждый из трёх префиксов типа В занимает 288 MiB и экономит 40,3 ms, вместе — 120,9 ms. Ширина на рисунке обозначает ёмкость; время рассчитано по времени повторного вычисления на RTX PRO 6000 и времени получения через PCIe Gen5, а вероятность попадания в обоих случаях равна 50%.

Упражнение 8-4. Ключевое. Анализ: какие префиксы следует сохранять в ограниченном кэше, чтобы максимально сэкономить время? Доступная ёмкость кэша составляет 864 MiB. Префикс A содержит 6144 токена и занимает 864 MiB; три независимых префикса типа B содержат по 2048 токенов и занимают по 288 MiB. Для RTX PRO 6000 рассчитайте время повторного вычисления по числу матричных операций, приняв производительность равной 62% пиковой производительности BF16 в 503,8 TFLOP/s. Время получения рассчитайте для PCIe Gen5 x16 с пропускной способностью 64 GB/s в каждом направлении; время обслуживания примите равным времени одной выгрузки, совпадающему со временем получения. Вероятность попадания во всех случаях равна 50%. Сравните ожидаемую чистую экономию времени при сохранении одного префикса A и трёх префиксов типа B. Сохраняя вероятность попадания для префиксов типа B неизменной, определите, ниже какого значения вероятности попадания для A следует перейти к сохранению трёх B. Затем замените линию на PCIe Gen4 x16 с пропускной способностью 32 GB/s в каждом направлении и повторите сравнение. В части анализа данных прочитайте прилагаемые данные 12 раундов входных данных и воспроизведения кэша, отдельно вычислите долю общего префикса и фактическую долю попаданий и объясните, почему использование пространства кэша другими запросами влияет на фактическую долю попаданий.

8.4 Сжатие и выгрузка

8.4.1 Потребление памяти после квантования весов

Совместное использование префикса экономит пространство за счёт устранения дублирующихся KV. Даже если запросы не имеют одинаковых префиксов, объём хранения весов и KV можно уменьшить с помощью квантования. Обычно при квантовании числовые значения кодируются группами: для каждой группы сохраняются значения с меньшей разрядностью и метаданные, такие как scale; во время вычислений по этой информации восстанавливаются приближённые значения.

Q2_K — один из форматов группового квантования, реализованных в семействе библиотек тензорных вычислений GGML. Основное кодирование использует 2 бита, а для scale внутри группы сохраняется дополнительная информация. Например, в этом формате каждая группа содержит 256 значений: 2-битное кодирование занимает $256 \times 2/8 = 64$ байта, а метаданные, включая scale и минимальное значение, — ещё 20 байт. Следовательно, одна группа занимает 84 байта, в среднем $84 \times 8/256 = 2.625$ бита на значение. При уменьшении разрядности доля метаданных в общем объёме соответственно возрастает и в этом формате составляет около 24%.²⁰

Расширенный пример: контекст какой длины ещё поместится в памяти после загрузки квантованных весов? В одном из вариантов файла Q2_K для Qwen3-235B-A22B применяется преимущественно 2-битное групповое квантование со смещением нескольких разрядностей. Квантованные данные вместе с неквантованными данными

²⁰Потензорная компоновка Q2_K, размер файлов для 8K и сравнение с 32K. Общий размер файлов составляет 85,691,002,112 bytes, полезная нагрузка кодовых значений/чисел с плавающей точкой — 69,178,275,840 bytes, метаданные квантования — 16,506,720,256 bytes, заголовки файлов и заполнение — 6,006,016 bytes. Приведённые размеры файлов рассчитаны по архивным заголовкам файлов и опубликованным метаданным.

с плавающей точкой занимают около 64.43 GiB, метаданные квантования — около 15.37 GiB, а вместе с заголовком файла и заполнением получается около 79.81 GiB. Другой вариант, UD-Q2_K_XL, занимает около 81.97 GiB. У Apple M2 Max, использованного в эксперименте 8-7, заявлено 96 GB объединённой памяти в десятичном исчислении. Если зарезервировать 8 GiB для системы и рабочей области, останется около 81.41 GiB.²¹

После загрузки первого файла в память останется 1.60 GiB, тогда как второй файл уже превышает доступный объём примерно на 0.56 GiB. BF16 KV для одного контекста 8K этой модели занимает около 1.47 GiB, поэтому в первом случае можно обработать ещё один запрос, оставив около 0.13 GiB. Если увеличить длину контекста до 32K, для KV потребуется около 5.88 GiB, поэтому придётся увеличить объём памяти или изменить способ размещения весов. Разница в размере двух файлов составляет всего 2.16 GiB, но её уже достаточно, чтобы определить, поместится ли запрос.

В системах с объединённой памятью CPU и GPU используют общую физическую память. После отображения файла весов в адресное пространство его страницы должны находиться в физической памяти только при обращении к соответствующим данным. Если общий объём данных, к которым требуется обращаться во время выполнения, превышает объём памяти, страницы будут постоянно подгружаться. Поэтому проблему нехватки памяти можно решить либо сокращением числа байтов самих данных, либо сохранением только данных, необходимых в текущий момент, с переносом остальных перед использованием. Сначала рассчитаем выигрыш от сжатия KV, а затем проанализируем время, необходимое для переноса весов.

8.4.2 Сжатие KV и стоимость чтения

Если применить к KV тот же метод группирования, можно рассчитать, сколько дополнительных запросов поместится благодаря сэкономленному пространству. Веса совместно используются множеством запросов, тогда как KV непрерывно растёт вместе с входными и выходными данными. Для модели из этой главы BF16 KV одного контекста 8K занимает 1152 MiB. Квантование изменяет разрядность каждого значения, а представление контекста определяет, сколько значений нужно сохранять для каждого токена: в разделе 9.2.2 состояние GQA и компактное состояние MLA для одного и того же контекста 8K приведены рядом, причём последнее занимает всего 549 MiB. Каждые 32 значения BF16 занимают 64 байта. **q8_0** — это 8-битный формат квантования, в котором 32 значения используют один общий scale. При переходе на **q8_0** сохраняются 32 байта кодированных значений и 2 байта scale, то есть объём уменьшается до 34/64 исходного; **q4_0** использует тот же способ группирования, но снижает разрядность кодированных значений до 4 бит, сохраняя 16 байт кодированных значений и 2 байта scale, то есть объём уменьшается до 18/64.

Следовательно, для того же контекста потребуется

$$M_{q8} = 1152 \times \frac{34}{64} = 612 \text{ MiB}, \quad M_{q4} = 1152 \times \frac{18}{64} = 324 \text{ MiB}.$$

По сравнению с BF16 формат **q8_0** экономит 540 MiB, а **q4_0** — 828 MiB. С учётом scale эти форматы занимают в среднем соответственно 8.5 бита и 4.5 бита на значение. Поскольку для каждой группы необходимо сохранять scale, объём метаданных также растёт

²¹Фиксированные файлы модели и рабочий бюджет, а также метаданные шардов 235B. Точные значения в байтах, GB и GiB приведены отдельно.

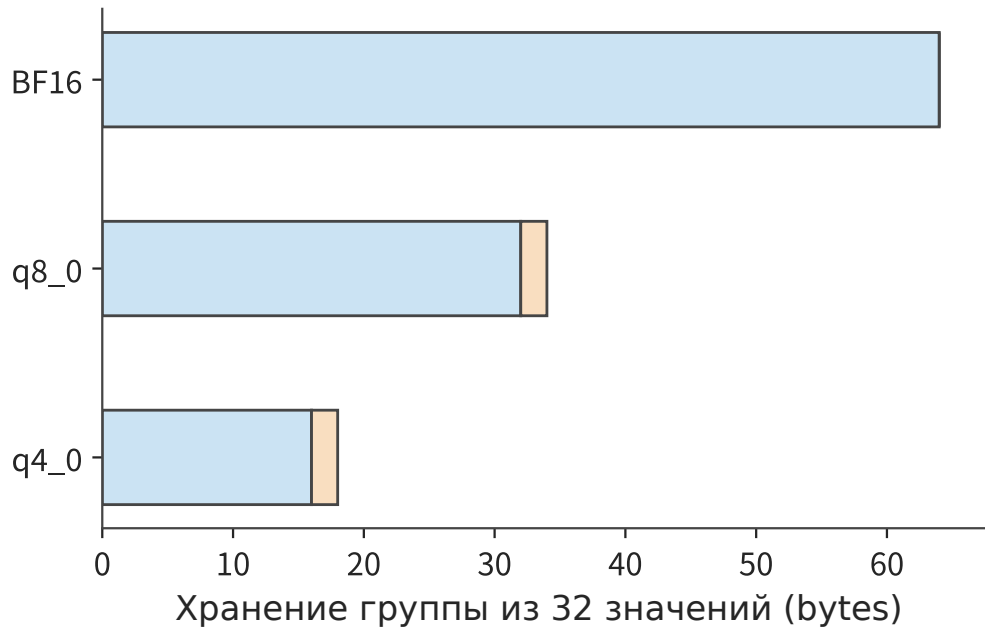


Рисунок 8-23. Пространство, необходимое форматам BF16, q8_0 и q4_0 для хранения одной и той же группы из 32 числовых значений. Длина полос пропорциональна числу байтов; оранжевым показаны 2 байта scale для каждой группы. Если умножить общую длину группы на число групп, получится объём KV для всего контекста.

вместе с длиной KV.²²

На рисунке 8-23 этот расчёт представлен в виде байтовой компоновки. В каждой строке сохраняются одни и те же 32 значения: кодированная часть сокращается, но scale остаётся. Поэтому общая длина q8_0 и q4_0 составляет соответственно 34 и 18 байт, а не 32 и 16 байт, получаемых при учёте только разрядности кодирования.

После завершения генерации длинного запроса из примера в начале главы BF16 KV при блочном распределении занимает в общей сложности 1188 MiB. При использовании q8_0 с той же компоновкой потребуется $1188 \times 34/64 = 631.125$ MiB, поэтому 16 независимых длинных запросов в сумме займут около 9.86 GiB и поместятся в зарезервированные 12 GiB памяти. Таким образом, совместное использование и сжатие экономят пространство по-разному: совместное использование устраняет повторяющиеся префиксы, а сжатие уменьшает число байтов на каждое значение.

При вычислении внимания со сжатым KV объём чтения уменьшается, но преобразование формата требует дополнительного времени. Пусть за одно вычисление считывается на ΔD байт меньше, эффективная пропускная способность равна β , а преобразование добавляет T_c последовательного времени выполнения. Тогда чистая экономия времени составит

$$\Delta T = \frac{\Delta D}{\beta} - T_c.$$

Например, при переводе одного контекста 8K на RTX PRO 6000 из BF16 в q8_0 считывается на 540 MiB меньше. При эффективной пропускной способности около 1.16 TB/s для

²²Форматы KV и стоимость выполнения. Материалы охватывают групповые форматы GGML, гранулярность scale для FP8 и результаты аппроксимации на H100.

операции, ограниченной чтением (раздел 8.6.3, 65% от пиковых 1792 GB/s), экономится около 0.487 ms. Если преобразование добавляет 0.2 ms, итоговая экономия составит около 0.287 ms; если оно добавляет 0.8 ms, время, наоборот, увеличится примерно на 0.313 ms. Ускорит ли один и тот же формат сжатия выполнение, зависит от того, меньше ли время преобразования сэкономленного времени чтения.

Чем длиннее контекст, тем больше времени экономится за счёт сокращения чтения. В одном из ранних исследований на H100 с Llama-3.1-8B интервалы выдачи токенов для BF16 и FP8 аппроксимировались соответственно выражениями $6.44 + 4.37 \times 10^{-5}L$ и $6.58 + 2.37 \times 10^{-5}L$ ms. Постоянная составляющая FP8 больше на 0.14 ms, а прирост на каждый токен контекста меньше на 2×10^{-5} ms. Если разделить разницу постоянных накладных расходов на разницу времени, добавляемого каждым токеном контекста, точка пересечения составит около 7000 токенов. При 2K получается около 6.53 ms для BF16 и 6.63 ms для FP8, а при 16K — около 7.16 и 6.97 ms соответственно. При достаточно длинном контексте экономия времени чтения превышает дополнительные постоянные накладные расходы.²³

Как отмечалось в начале раздела, представление контекста определяет, сколько значений необходимо сохранять для каждого токена; таким же способом можно сравнивать объём локального и глобального состояний моделей со смешанным вниманием. Для базового контекста 8K сложим объёмы глобальной истории и действующего SWA. У DeepSeek V4.1 они составляют соответственно 6.953 MiB и 2.578 MiB, всего 9.531 MiB; соответствующие два компонента V4 в сумме занимают 30.521 MiB, примерно в 3.20 раза больше, чем у V4.1. Локальное окно занимает фиксированное пространство, поэтому при коротком контексте разница общего объёма меньше 3.95-кратной разницы глобальной истории.

8.4.3 Выгрузка весов, предвыборка и перенос на каждом шаге

Сжатие уменьшает размер самих данных. Другой способ увеличить доступный объём — поочерёдно размещать данные на GPU: переносить их только тогда, когда они нужны для вычислений, а после использования повторно задействовать это пространство. При выгрузке весов часть весов, обычно постоянно находящихся на GPU, перемещается в основную память и возвращается на GPU перед вычислением соответствующего слоя. Освободившуюся память ускорителя можно использовать для KV, но при каждом прямом проходе эти веса придётся переносить заново. При пошаговой генерации ответа перенос повторяется на каждом цикле decode.

Три BF16-матрицы FFN одного слоя Qwen3-8B вместе занимают 288 MiB. Если из 36 слоёв выгружать один слой из каждых четырёх, будут выгружены девять наборов весов FFN, что освободит 2592 MiB. На GPU также необходимо зарезервировать буфер для предвыборки — заблаговременного копирования весов слоя на GPU перед его вычислением. Один набор буферов занимает 288 MiB, поэтому чистая экономия составляет 2304 MiB; два набора занимают 576 MiB, оставляя чистую экономию 2016 MiB. Если считать единицей KV размером 1152 MiB для одного входа 8K, в первом случае поместятся ещё два запроса, а во втором — только один. Если резервировать по 1188 MiB, необходимых после завершения генерации, обе схемы позволят добавить только один полный

²³Форматы KV и стоимость выполнения. Материалы охватывают групповые форматы GGML, гранулярность scale для FP8 и результаты аппроксимации на H100.

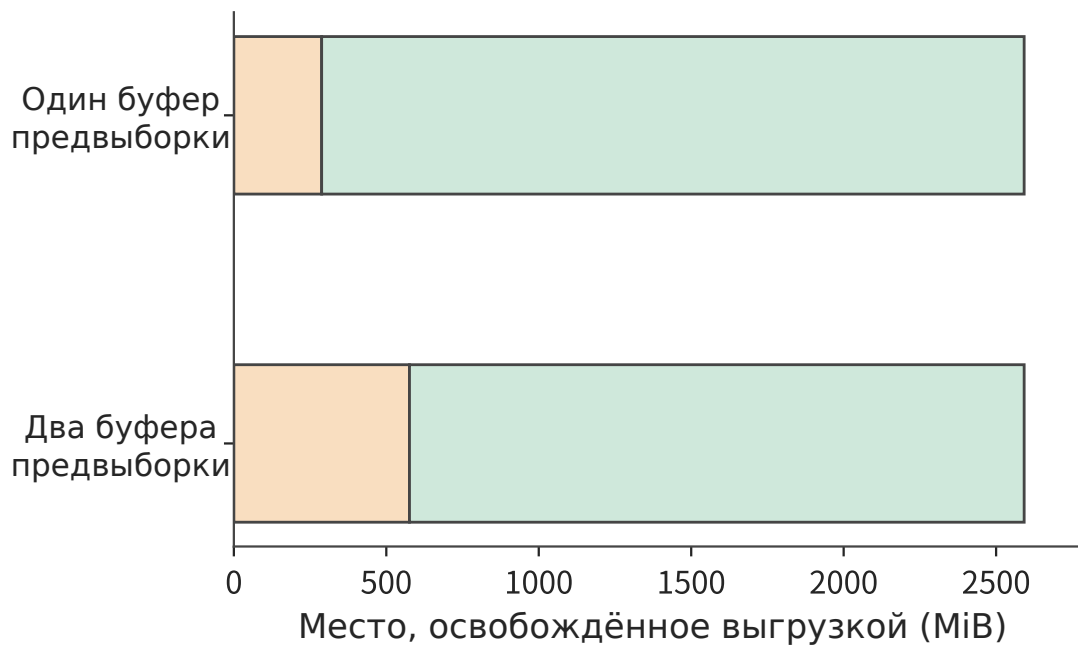


Рисунок 8-24. Выгрузка девяти наборов FFN освобождает в общей сложности 2592 MiB. Оранжевым показан буфер предвыборки, остающийся на ускорителе, зелёным — чистый объём, доступный для перераспределения; с одним набором буферов чистая экономия составляет 2304 MiB, с двумя — 2016 MiB.

запрос.²⁴

Предвыборка должна соблюдать две зависимости по порядку выполнения: вычисление может читать веса только после завершения их копирования, а буфер можно перезаписать следующим набором весов только после того, как вычисление перестанет использовать текущий набор. При одном наборе буферов один и тот же буфер последовательно используется для копирования, вычисления и следующего копирования. При двух наборах можно вычислять текущий слой, одновременно копируя веса последующего слоя в другой набор буферов. Это позволяет перекрыть копирование вычислениями, но сами буферы занимают больше памяти.

Сначала рассчитаем стоимость переноса по каналу связи. Эта карта подключена к хосту через PCIe Gen5 x16 с заявленной пропускной способностью 64 GB/s в каждом направлении. На каждом цикле необходимо переносить 2592 MiB, то есть 2.72 GB, для чего требуется

$$T_{\text{copy}} \geq \frac{2.72 \text{ GB}}{64 \text{ GB/s}} \approx 42.5 \text{ ms}.$$

Это уже превышает 26.26 ms, необходимые для одного цикла decode при batch 1. Даже если все остальные вычисления удастся полностью перекрыть копированием, каждый цикл всё равно займёт не менее 42.5 ms. Если за цикл генерируется по одному токenu для каждого из четырёх запросов, максимальная общая пропускная способность вывода

²⁴Расчёт выгрузки и предварительной загрузки весов; расписание копирования и предварительной загрузки для двух вариантов канала — PCIe Gen5 и NVLink-C2C. Время вычислений одного слоя получено делением 26.26 ms одного раунда decode при batch 1 на 36 слоёв; описание архитектуры GH200: совокупная пропускная способность NVLink-C2C составляет 900 GB/s, по 450 GB/s в каждом направлении.

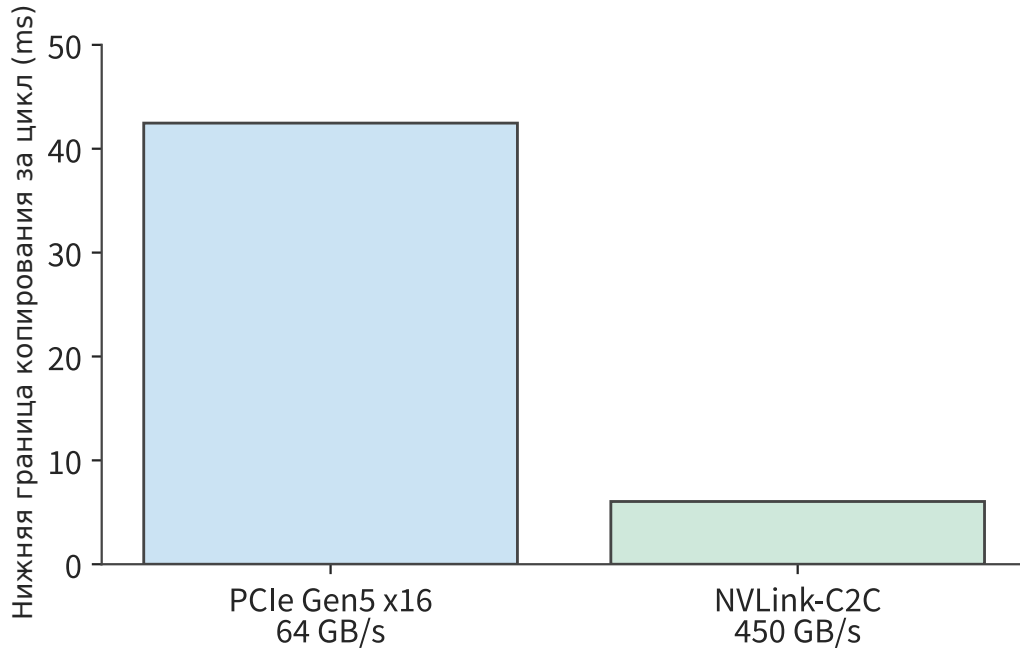


Рисунок 8-25. На каждом цикле копируется 2592 MiB (2.72 GB). Через PCIe Gen5 x16 с пропускной способностью 64 GB/s в каждом направлении требуется не менее 42.5 ms, а через NVLink-C2C в GH200 с пропускной способностью 450 GB/s в каждом направлении — не менее 6.0 ms.

составит около $4/0.0425 \approx 94$ token/s, а интервал выдачи токенов для каждого запроса — не менее 42.5 ms. Для генерации 256 выходных токенов требуется 255 последующих прямых проходов, поэтому один только перенос весов займёт около 10.8 секунды. Следовательно, выгрузка этой части весов через PCIe не позволяет уложиться в предельное время выполнения 7 секунд.

Между CPU и GPU бывают и значительно более быстрые соединения. В GH200 CPU Grace соединён с GPU Hopper посредством NVLink-C2C с пропускной способностью 450 GB/s в каждом направлении. Перенос тех же 2.72 GB занимает не менее 6.0 ms за цикл и около 1.54 секунды за 255 циклов. Если копирование на каждом цикле выполняется быстрее вычислений одного цикла decode, предвыборка может скрыть его за вычислениями.

Решающим здесь является отношение объёма переносимых на каждом цикле данных к эффективной пропускной способности. Предвыборка позволяет выполнять перенос одновременно с вычислениями, но число передаваемых за цикл байтов остаётся прежним.

Если сначала сжать веса, а затем переносить их, необходимо также учесть время распаковки. Пусть исходные данные занимают S байт, после сжатия — rS байт, пропускная способность канала равна B_{link} , а пропускная способность распаковки — R_{dec} в пересчёте на выходные исходные байты. Последовательные передача и распаковка займут $rS/B_{\text{link}} + S/R_{\text{dec}}$, тогда как прямая передача потребует S/B_{link} . Следовательно, перенос после сжатия будет быстрее, только если

$$R_{\text{dec}} > \frac{B_{\text{link}}}{1 - r}$$

Прямая передача 96 MiB весов через PCIe Gen5 x16 занимает около 1.57 ms; после

сжатия весов до четверти исходного объёма передача сжатых данных занимает всего 0.39 ms. Чтобы сохранить эту экономию в 1.18 ms, распаковщик должен выдавать более 85.3 GB исходных данных в секунду. Чем быстрее канал, тем выше требования к пропускной способности распаковки.²⁵

При описанной в этом разделе выгрузке вычисления модели по-прежнему полностью выполняются на GPU, а основная память хранит временно неиспользуемые веса. В разделе 9.3 будет изменено и место выполнения вычислений: если веса экспертов уже находятся в основной памяти, вычисления можно выполнить непосредственно на CPU и вернуть на GPU только меньшие по объёму активации. Тогда потребуется сравнить число входных строк для каждого эксперта, время вычислений на CPU и время передачи весов.

Упражнение 8-5. Вычисление: объём памяти, сэкономленный выгрузкой весов, и дополнительный перенос. Пусть выгружаются девять наборов весов FFN по 288 MiB каждый. Предположим, что каждый набор буферов предвыборки вмещает один набор весов FFN. Рассчитайте чистую экономию памяти ускорителя при использовании одного и двух наборов буферов, а также определите, сколько дополнительных входных состояний по 1152 MiB или полных состояний запросов по 1188 MiB поместится в освобождённом пространстве. Затем, используя пропускную способность 64 GB/s в каждом направлении для PCIe Gen5 x16 и 450 GB/s в каждом направлении для GH200 NVLink-C2C, рассчитайте нижнюю границу времени копирования на одном шаге и за 255 шагов. Если на все операции копирования отводится одна секунда, определите минимальную необходимую однонаправленную пропускную способность.

8.4.4 Выбор объёма и скорости с учётом ограничений качества

В разделе 8.4.2 были рассчитаны экономия пространства при сжатии и дополнительные затраты времени на преобразование, но ещё не был дан ответ на вопрос, останется ли ответ тем же после изменения числового формата. Под исполнительным бэкэндом здесь понимается реализация kernel, непосредственно выполняющая вычисление внимания. Сжатие KV изменяет сохраняемые числовые значения, а замена исполнительного бэкенда может одновременно изменить формат вычисления Q, процесс преобразования и числовую точность операций редукции. Раздельное сравнение форматов хранения тензоров и точности вычислений позволяет определить источник различий в результатах.

Ниже такое раздельное сравнение проводится на восьми фиксированных задачах, расположенных в одинаковом порядке на трёх рисунках. Задачи основаны на документах двух длин: каждый документ содержит 128 или 512 записей из шестизначных чисел, по четыре независимых документа каждой длины. В журнале эксперимента они обозначены как n128-r0 — n128-r3 и n512-r0 — n512-r3, а на рисунках последовательно представлены как задачи 1–8. Каждая задача запускалась по два раза при каждом из двух уровней параллелизма, что позволяет последовательно сравнить влияние различных числовых форматов на одни и те же задачи.

В серии экспериментов с Qwen3-8B веса BF16 оставались неизменными, а сравнивались три способа вычисления внимания. Из 32 свободных генераций с BF16 KV правиль-

²⁵Сжатие и передача тензоров. Материалы охватывают реализацию LLM.265 на видеодвижке, проектирование специализированного оборудования и оценку производительности.

Задача 1	○	○	○	○
Задача 2	○	○	○	○
Задача 3	○	○	○	○
Задача 4	○	○	○	○
Задача 5	×	×	×	×
Задача 6	○	○	○	○
Задача 7	○	○	○	○
Задача 8	○	○	○	○
	1 / 1	1 / 2	4 / 1	4 / 2
Параллелизм / № повтора				

Рисунок 8-26. Базовая конфигурация с BF16 KV. Каждая строка соответствует одной фиксированной задаче, а четыре столбца соответствуют двум свободным генерациям при каждом из уровней параллелизма 1 и 4. Зелёный круг означает правильный ответ, оранжевый крест — ошибку; всего правильно выполнено 28 из 32 запусков.

ными были 28; после перехода на исходную реализацию FP8 правильными оказались 26; когда FP8 KV был сохранён, а Q возвращён к BF16, число правильных ответов снова составило 28. В последней конфигурации изменилась только точность Q, но результаты стали другими. Это показывает, что формат хранения KV и точность вычисления Q необходимо исследовать отдельно.²⁶

На рисунке 8-28 дополнительно восстановлена точность BF16 только для Q, тогда как KV по-прежнему хранится в FP8. Сравнение одинаковых столбцов на трёх рисунках показывает, результаты каких именно задач изменились, а не только общее число правильных ответов.

Задача 5 на рисунках — n512-r0 в журнале эксперимента, то есть первый длинный документ из 512 записей, — во всех четырёх запусках с BF16 KV была выполнена неверно, а в конфигурации «FP8 KV + BF16 Q» — правильно. Для задачи 6, n512-r1, наблюдается строго обратная картина. Обе конфигурации дали по 28 правильных ответов, но ошиблись в разных задачах. Расположение результатов одной задачи рядом позволяет увидеть, где именно возникли различия и сохранялись ли они при повторных запусках.

²⁶Сравнение точности KV и Q, контроль Q при фиксированных весах FP8, данные о ёмкости при конкурентном выполнении. В основном тексте используется группа с весами BF16. В другой группе с фиксированными весами FP8 результаты для BF16 Q и стандартного пути составляют соответственно 32/32 и 28/32.

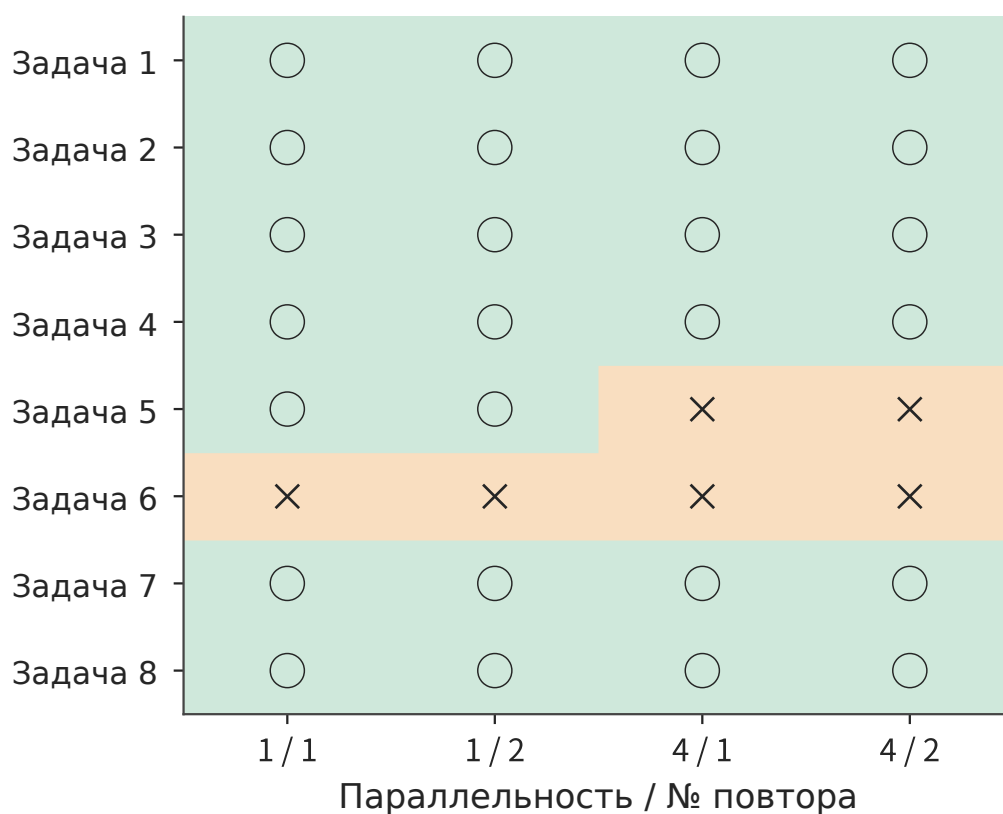


Рисунок 8-27. Для тех же задач и порядка запусков исходная реализация FP8 дала 26 правильных ответов из 32. Это сравнение включает влияние выбора реализации на точность Q. Круг означает правильный ответ, крест — ошибку; каждая строка соответствует одной задаче, а столбцы обозначают уровень параллелизма и номер повторного запуска.

Задача 1	○	○	○	○
Задача 2	○	○	○	○
Задача 3	○	○	○	○
Задача 4	○	○	○	○
Задача 5	○	○	○	○
Задача 6	×	×	×	×
Задача 7	○	○	○	○
Задача 8	○	○	○	○
	1 / 1	1 / 2	4 / 1	4 / 2
Параллельность / № повтора				

Рисунок 8-28. Q сохраняет точность BF16, а KV по-прежнему использует FP8; правильно выполнено 28 из 32 запусков, однако ошибки возникли в других задачах, чем в базовой конфигурации BF16. На трёх рисунках используются одинаковые задачи и порядок столбцов, а веса модели во всех случаях представлены в BF16. Круг означает правильный ответ, крест — ошибку; каждая строка соответствует одной задаче, а столбцы обозначают уровень параллелизма и номер повторного запуска.

Ошибки также увеличивают время выполнения задачи. Если после первого неправильного ответа производится повторная попытка, общее время до получения правильного ответа должно включать первую генерацию, проверку ответа и повторную генерацию. В проведённых экспериментах две повторные попытки исправили ошибки, но дополнительно потребовали около 2.0 секунды генерации и 92 токенов. Если первый ответ генерируется быстрее, но чаще оказывается неправильным, полное выполнение задачи в итоге может занять больше времени.²⁷

Сравним две описанные выше схемы размещения KV: после перехода на q8_0 шестнадцать независимых длинных запросов занимают около 9.86 GiB, а при совместном использовании префикса BF16 — около 5.91 GiB. Чтобы определить, какая схема предпочтительнее, необходимо включить время преобразования, генерации и повторных попыток в общее время обработки запроса. Это будет сделано в комплексном примере раздела 8.6.

Упражнение 8-6. Анализ данных: влияние сжатия KV на объём, правильность ответов и стоимость повторных попыток. Предположим, что каждая группа содержит 32 значения, а для хранения scale дополнительно используются 2 байта. Повторно рассчитайте объём KV для входа 8K и полного длинного запроса в форматах q8_0 и q4_0. Затем изучите результаты по отдельным задачам для трёх конфигураций точности KV и Q и перечислите задачи, правильность которых различается между BF16 KV и «FP8 KV + BF16 Q». Наконец, изучите журнал повторных попыток, для каждой успешно выполненной задачи сложите время первого ответа и повторной попытки и сравните результат со временем только последней успешной генерации.

8.5 Спекулятивное декодирование

8.5.1 Черновик, проверка, откат и корректность

В двух предыдущих разделах главным образом изменялись способы хранения и чтения данных, но каждый запрос по-прежнему требовал пошаговой генерации. Если за одно вычисление целевой модели можно определить несколько выходных значений, то для генерации 256 токенов уже не потребуется выполнять столько же шагов. При обычном декодировании целевая модель за каждое прямое вычисление генерирует один токен. В разделе 8.2 вычисления для разных запросов объединялись, чтобы одно чтение весов использовалось для большего числа строк. Спекулятивное декодирование использует другой подход: сначала для одного запроса генерируется **черновая последовательность**, после чего целевая модель одновременно проверяет несколько входящих в неё токенов. Если целевая модель принимает несколько черновых токенов, одно вычисление определяет сразу несколько выходных токенов.

Сначала рассмотрим **жадную генерацию**: на каждом шаге выбирается токен с наибольшей вероятностью по версии целевой модели. Предположим, черновая последовательность содержит четыре токена, а целевая модель вычисляет для каждой позиции, какой токен должен быть выведен. Первые два совпадают с черновиком, а третий отличается. Тогда первые два черновых токена сохраняются, а в третьей позиции выводится

²⁷Естественные результаты и стоимость повторных попыток. В эксперименте ответы проверяются по заранее известному результату, а стоимость повторной попытки включает затраты на генерацию первого ошибочного ответа.

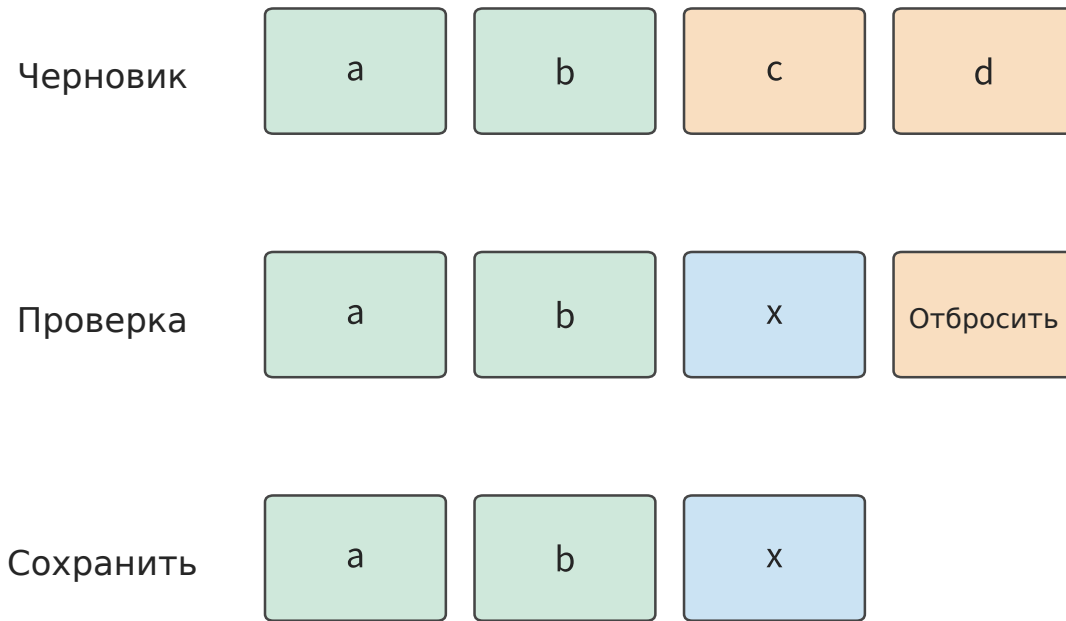


Рис. 8-29. Процесс жадной проверки. a , b , c , d и x обозначают токены; в третьей позиции целевая модель выбирает x , который отличается от чернового c . Четвёртый токен и последующие результаты проверки аннулируются, а следующий раунд генерации продолжается с a , b , x . Прямоугольники обозначают позиции токенов в последовательности, а не время выполнения.

токен, выбранный целевой моделью. Четвёртый черновой токен был сгенерирован после ошибочного третьего токена, поэтому он также отбрасывается. Следующий раунд генерации продолжается с исправленной последовательности. Если совпали все четыре черновых токена, целевая модель обычно может дополнительно сгенерировать после них ещё один токен.

На рис. 8-29 показан порядок действий при расхождении в третьем токене. Хотя целевая модель уже вычислила результаты проверки последующих токенов, четвёртый токен зависит от ошибочного третьего чернового токена, поэтому использовать его нельзя. После одной проверки остаются два совпавших черновых токена и один исправленный токен — всего три выходных токена.

При случайном сэмплировании необходимо обеспечить, чтобы итоговый результат по-прежнему соответствовал распределению вероятностей целевой модели. Обозначим целевое распределение как $p(x)$, а черновое — как $q(x)$. Сначала из q сэмплируется черновой токен x , который затем принимается со следующей вероятностью:

$$\alpha(x) = \min \left(1, \frac{p(x)}{q(x)} \right).$$

Вероятность сэмплировать x и сразу принять его равна $q(x)\alpha(x) = \min(p(x), q(x))$.

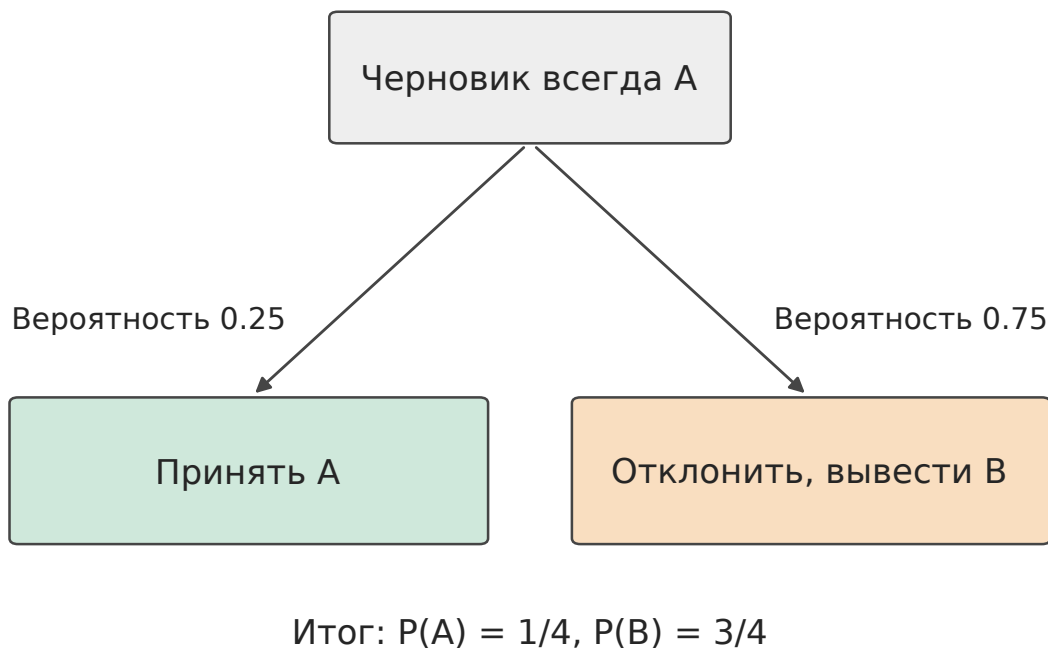


Рис. 8-30. Вероятность принять А равна 1/4, а вероятность вывести В после отклонения — 3/4. Вместе эти два пути дают целевое распределение.

Для достижения целевой вероятности $p(x)$ недостаёт $[p(x) - q(x)]_+$. Здесь $[z]_+ = \max(z, 0)$ означает, что сохраняется только положительная разность. Поэтому после отклонения чернового токена эта положительная разность нормализуется, а из полученного распределения выполняется повторное сэмплирование. Если объединить случаи непосредственного принятия и повторного сэмплирования после отклонения, итоговая вероятность вывода x будет равна $p(x)$.²⁸

Поясним этот процесс на примере всего с двумя символами — А и В. Целевое распределение задаётся как $p(A) = 1/4$ и $p(B) = 3/4$. Если черновик всегда генерирует А, то А принимается с вероятностью 1/4, а в остальных 3/4 случаев А отклоняется и вместо него выводится В. Если черновик всегда генерирует В, то В принимается с вероятностью 3/4, а в остальных случаях вместо него выводится А. Оба способа дают одно и то же целевое распределение, однако вероятность принятия черновика различается, поэтому различается и эффективность выполнения.

После проверки действительная длина KV корректируется до позиции уже определённого результата, а KV, соответствующие отброшенным черновым токенам, больше не участвуют в последующих вычислениях. Менеджер страниц обновляет ссылки на блоки в соответствии с длиной уже определённого результата, а сэмплер и средство проверки грамматики — компонент, ограничивающий набор доступных токенов согласно заданному формату вывода, — также возвращаются в соответствующее состояние. После этого следующий раунд вычислений продолжается с исправленной последовательности.

²⁸Оригинальная статья о спекулятивном сэмплировании, оригинальная статья о спекулятивном декодировании, заметки о подсчёте и бюджете фреймворков, охват фреймворков в обзоре. Базовый вероятностный алгоритм, подходы к формированию черновика и конкретные реализации цитируются отдельно.

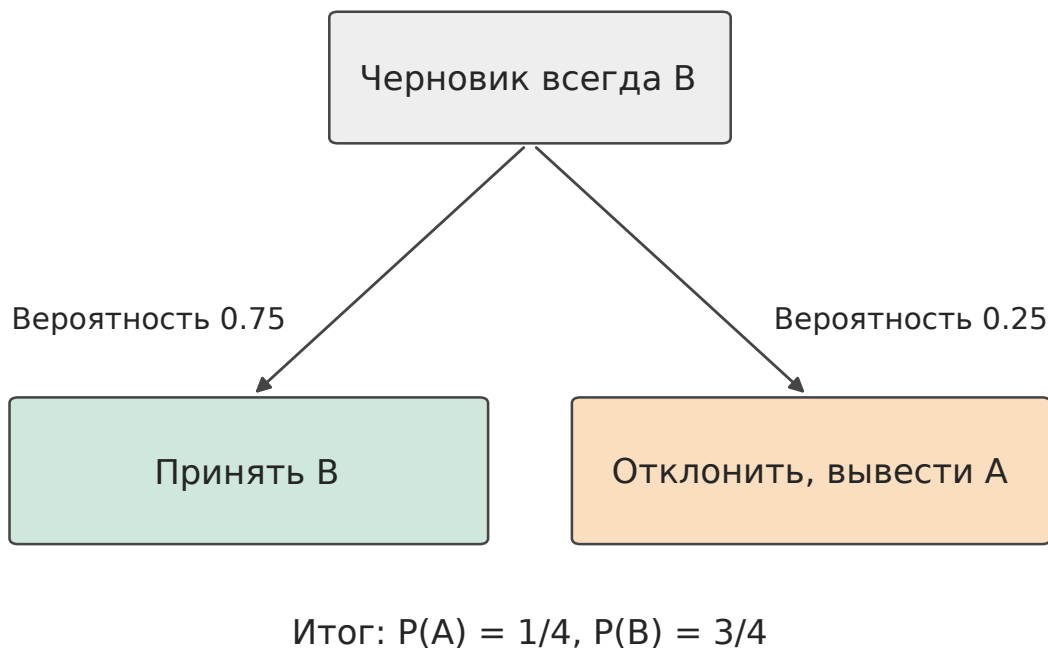


Рис. 8-31. Вероятность принять В равна 3/4, а вероятность вывести А после отклонения — 1/4. Целевое распределение не изменилось, но вероятность принятия черновика стала выше.

8.5.2 Время раунда и число выходных токенов

Спекулятивное декодирование изменяет как продолжительность каждого раунда, так и итоговое число выведенных за раунд токенов. Пусть раунд r занимает время T_r и выводит N_r токенов. Тогда после нескольких последовательных раундов среднее время на один выходной токен равно

$$\bar{t}_{\text{спец}} = \frac{\sum_r T_r}{\sum_r N_r}.$$

Например, два раунда длительностью по 1,5 мс выводят соответственно 1 и 5 токенов. Суммарно получаются 3 мс и 6 токенов, то есть в среднем 0,5 мс на токен. Если сначала вычислить среднее значение для каждого раунда, а затем с одинаковыми весами усреднить 1,5 и 0,3 мс/токен, получится 0,9 мс. Это равносильно присвоению одинакового веса раунду, выводящему один токен, и раунду, выводящему пять токенов. Среднее время на токен можно получить только прямым делением общего времени на общее число токенов.

Пример расчёта: как доля принятых черновых токенов определяет среднее число выходных токенов за раунд? Продолжим использовать целевое распределение с двумя символами и предположим, что позиции токенов независимы. В каждом раунде черновик генерирует четыре одинаковых символа, а итоговый результат дополнительно включает исправленный токен при отклонении либо дополнительный токен после принятия всего черновика. Пусть при условии, что все предшествующие черновые токены уже приняты, вероятность принятия токена в текущей позиции

равна a . Каждый раунд выводит не менее одного токена. Чтобы вывести второй токен, необходимо принять первый черновой токен, что происходит с вероятностью a . Чтобы вывести третий, необходимо принять первые два, что происходит с вероятностью a^2 . Последовательно складывая эти значения, получаем среднее число выходных токенов за раунд:

$$\mathbb{E}[N] = 1 + a + a^2 + a^3 + a^4.$$

Каждый член этого выражения обозначает вероятность того, что в итоге будет выведено не менее соответствующего числа токенов. Для черновика AAAA значение $a = 1/4$, а среднее число выходных токенов за раунд составляет около 1,33. Для BBBB значение $a = 3/4$, а среднее число выходных токенов — около 3,05. На RTX PRO 6000 один раунд обычного декодирования при batch 1 и контексте 2K занимает 26,26 мс (эксперимент 8-1). При проверке пяти позиций считываются те же веса и KV, что и при обычном декодировании. Дополнительные матричные операции для четырёх строк требуют около 65 GFLOP, что при 62% от пиковой производительности BF16 занимает лишь около 0,2 мс, поэтому продолжительность раунда проверки по-прежнему принимается равной 26,26 мс. В эксперименте 8-5 наблюдалось то же явление: DFlash — черновая сеть, параллельно генерирующая весь черновой фрагмент за одно прямое вычисление, — в каждом раунде генерировала 7 черновых токенов и проверяла 8 позиций; медианное время составляло 15,2 мс, что сопоставимо с 16,9 мс на один шаг обычного декодирования при той же конфигурации. С учётом 0,1 мс на запрос черновика один раунд занимает в общей сложности 26,36 мс, а среднее время на токен для двух вариантов черновика составляет соответственно около 19,8 и 8,64 мс. Оба результата лучше 26,26 мс/токен при обычном декодировании. Проверка почти не увеличивает время раунда, а даже при отклонении большей части черновика каждый раунд всё равно даёт как минимум один токен, поэтому черновик AAAA с низкой вероятностью принятия также обеспечивает ускорение.²⁹

Экономия времени за счёт принятия большего числа черновых токенов может быть сведена на нет затратами на запрос черновика. Чтобы превзойти обычное декодирование, полное время раунда должно быть меньше произведения среднего числа выходных токенов на 26,26 мс. После вычета 26,26 мс, необходимых для проверки, на запрос черновика остаётся около 8,7 мс для AAAA и около 53,9 мс для BBBB. На рис. 8-32 эти две точки пересечения показаны на оси стоимости запроса.

Подготовительные операции перед использованием черновика также необходимо учитывать в общем времени. Если построение индекса контекста занимает 2 секунды, а черновик BBBB экономит около $26.26 - 8.64 \approx 17.6$ мс на каждый выходной токен, то для компенсации этих двух секунд потребуется сгенерировать около 114 токенов. Генерация 256 выходных токенов с помощью обычного декодирования занимает около 6,72 секунды. Если сначала за 2 секунды построить индекс, а затем использовать черновик BBBB, ожидаемое полное время составит около 4,22 секунды. Если индекс можно построить заранее и совместно использовать для нескольких запросов, эти затраты распределятся между ещё большим числом выходных токенов.

²⁹Расчёт распределения и стоимости черновиков из контекста и результаты полного перебора рациональных чисел. В примере расчёта используется целевое распределение по двум символам; длительность каждого раунда на RTX PRO 6000 приведена для AAAA, BBBB и запроса на 256 токенов с двухсекундным индексированием. Длительность обычного раунда decode, равная 26.26 ms, взята из поэтапных записей эксперимента 8-1, а длительность каждого раунда DFlash — из пошаговых записей эксперимента 8-5.

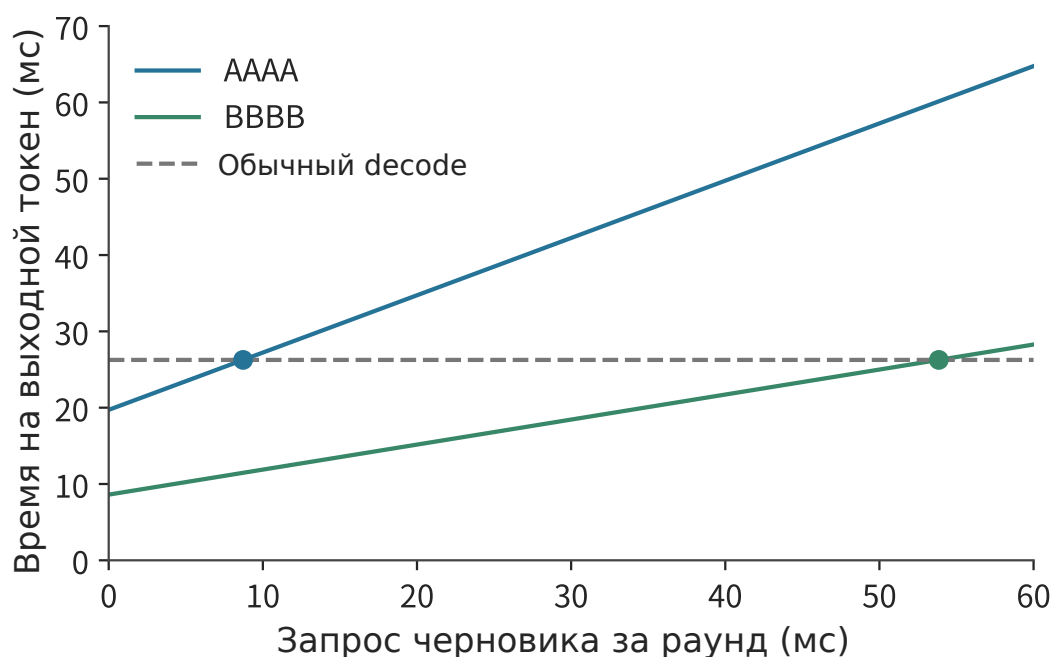


Рис. 8-32. Целевое распределение с двумя символами, в каждом раунде генерируются четыре черновых токена. Проверка каждого раунда занимает 26,26 мс, а время запроса изменяется вдоль горизонтальной оси; время на один выходной токен вычисляется делением времени раунда на среднее число выходных токенов. Обычное декодирование занимает 26,26 мс/токен (RTX PRO 6000, batch 1, контекст 2K). Досрочная остановка отсутствует, дополнительный токен учитывается в числе выходных токенов. Точки на графике отмечают пример расчёта со временем запроса 0,1 мс; AAAA и BBBB пересекаются с обычным выполнением при времени запроса около 8,7 и 53,9 мс соответственно.

8.5.3 Источники черновика и последовательная либо параллельная генерация

Сколько времени можно сэкономить с помощью четырёх черновых токенов, зависит как от числа токенов, прошедших проверку, так и от времени, необходимого для генерации этого чернового фрагмента. В реальной системе черновик можно искать в контексте либо генерировать с помощью дополнительной сети. Эти варианты требуют разного времени подготовки и объёма памяти.

При поиске в контексте повторяющиеся фрагменты находятся во входных данных или уже сформированном ответе; основные затраты приходятся на сопоставление и построение индекса. Небольшая независимая модель генерирует черновик последовательно и требует дополнительно хранить собственные веса и KV. Методы семейства EAGLE передают промежуточные представления целевой модели в меньшую предсказывающую сеть, а методы семейства DFlash параллельно генерируют несколько черновых токенов. Встроенные в модель головы MTP генерируют черновик с помощью предсказывающей способности, полученной при совместном обучении.³⁰

Если разделить каждый раунд на четыре этапа — генерацию черновика, подготовку проверки, проверку целевой моделью и определение результата, — эти методы можно сравнивать поэтапно. Например, черновая сеть DFlash из эксперимента 8-5 имеет веса объёмом около 1,95 GiB (2,10 GB). При эффективной пропускной способности для batch 1 — 15,44 GB за 26,26 мс, то есть около 588 GB/s, — одно прямое вычисление этой сети занимает около 3,57 мс. Если с её помощью последовательно сгенерировать четыре черновых токена, потребуется четыре прямых вычисления общей длительностью 14,28 мс. При параллельной генерации всего чернового фрагмента, как в DFlash, достаточно одного прямого вычисления длительностью около 3,57 мс. Проверка целевой моделью в обоих случаях занимает 26,26 мс. Если каждый раунд в среднем выводит 3 токена, то при последовательном подходе время на токен составит около $(14.28 + 26.26)/3 \approx 13.5$ мс, а при параллельном — около $(3.57 + 26.26)/3 \approx 9.9$ мс. В этом случае ускорение достигается за счёт сокращения времени генерации черновика.

Теперь предположим, что параллельно сгенерированный черновик принимается реже и среднее число выходных токенов снижается до 2. Тогда среднее время на один выходной токен возрастает до $(3.57 + 26.26)/2 \approx 14.9$ мс, что уже хуже последовательного подхода. Более быстрая генерация черновика экономит 10,7 мс, но число выходных токенов за раунд уменьшается, поэтому время, распределяемое на каждый выходной токен, возрастает и весь запрос выполняется медленнее.

8.5.4 Динамический выбор длины черновика и конкуренция при параллельной обработке

После выбора источника черновика необходимо определить его длину для каждого раунда. Чем длиннее черновик, тем больше максимальное число токенов, которое можно вывести за один раунд. Однако последующие токены могут попасть в итоговый результат, только если приняты все предшествующие. Пусть при условии, что все предыдущие черновые токены уже приняты, условная вероятность принятия j -го токена равна a_j . Тогда среднее число выходных токенов для черновика длиной m составляет

³⁰Оригинальная статья о спекулятивном сэмплинге, оригинальная статья о спекулятивном декодировании, заметки о подсчёте и бюджете фреймворков, охват фреймворков в обзоре. Базовый вероятностный алгоритм, подходы к формированию черновика и конкретные реализации цитируются отдельно.

$$\mathbb{E}[N_m] = 1 + \sum_{j=1}^m \prod_{i=1}^j a_i.$$

При увеличении длины черновика с $m - 1$ до m среднее число выходных токенов возрастает на $\Delta N = \prod_{i=1}^m a_i$. Пусть для этого требуется дополнительное время ΔT , а исходный раунд занимает T и в среднем выводит N токенов. Тогда среднее время на один выходной токен равно T/N . Чтобы после увеличения длины черновика это время сократилось, должно выполняться неравенство $(T + \Delta T)/(N + \Delta N) < T/N$, которое после преобразования принимает вид

$$\frac{\Delta T}{\Delta N} < \frac{T}{N}.$$

Иными словами, среднее время, затрачиваемое на дополнительные выходные токены, должно быть меньше исходного среднего времени на токен. Поскольку вероятности принятия последовательно перемножаются, вероятность сохранения более поздних черновых токенов снижается, поэтому добавление таких токенов всё реже обеспечивает достаточную экономию времени.

Заранее заданная форма выполнения также влияет на дополнительное время ΔT . Например, если при проверке четырёх или пяти токенов в обоих случаях векторы признаков токенов дополняются до восьми строк матрицы и используется заранее подготовленный вычислительный граф для восьми строк, добавление одного токена требует мало дополнительных вычислений. Но если число проверяемых токенов увеличивается с восьми до девяти, матрицу приходится дополнять до шестнадцати строк и использовать соответствующий вычислительный граф, поэтому объём вычислений резко возрастает. Число параллельных запросов также влияет на выгоду от спекулятивного декодирования: при низкой нагрузке многострочная проверка может использовать простаивающие вычислительные ресурсы; при высокой нагрузке обычная пакетная обработка уже в полной мере обеспечивает совместное чтение весов, а генерация и проверка черновиков дополнительно конкурируют с другими запросами за ускоритель и память.

В одной серии измерений сравнивались черновики разной длины. K7 и K15 обозначают конфигурации, генерирующие соответственно 7 и 15 черновых токенов за раунд. Для короткой задачи извлечения Qwen3-8B/DFlash при одном одновременно выполняемом запросе медианное полное время обычного декодирования, K7 и K15 составляло около 90,0, 30,6 и 29,4 мс соответственно. Переход от обычного выполнения к K7 сэкономил около 59 мс, а переход от K7 к K15 — лишь ещё около 1,2 мс. В записях по раундам также встречались раунды, в которых при обеих длинах блока принимался только один черновой токен: K7 отбрасывал шесть черновых токенов, K15 — четырнадцать, а итоговая длина результата оставалась одинаковой. Более крупный блок проверки увеличивал объём вычислений, но не давал дополнительных выходных токенов в этом раунде.³¹

Таким образом, при динамической настройке длины черновика можно использовать приведённое выше условие: для запросов с высокой вероятностью принятия и небольшими затратами времени на добавление проверяемых позиций следует генерировать

³¹Сравнение DFlash на одном исполнителе и наблюдения за этапами отдельных запросов. В эксперименте заданный набор коротких задач выполнялся в фиксированном порядке при низкой конкурентности. При конкурентности 1 медианное время обычного режима, K7 и K15 для успешно решённых коротких задач составляет соответственно 90.02, 30.58 и 29.43 ms; всего получено 32 пары результатов.

более длинный черновик; если черновик часто не проходит проверку уже в первых позициях либо требует перехода к более крупной форме выполнения, длину блока следует уменьшить. При выборе длины черновика также необходимо оставить место для KV целевой и черновой моделей, а затем определить число принимаемых запросов методом из раздела 8.3.

Упражнение 8-7 · Вывод: какая длина черновика минимизирует среднее время на один выходной токен? При фиксированной вероятности принятия чернового токена $a = 3/4$ вычислите среднее число выходных токенов за раунд для блоков длиной 1, 2, 4 и 8. Используя данные для RTX PRO 6000 из раздела 8.5.3 и округляя до одного знака после запятой, примите время проверки одного раунда равным 26,3 мс, а время последовательной генерации каждого чернового токена черновой сетью — 3,6 мс, то есть $T_m = 26.3 + 3.6m$ мс. Найдите среднее время на один выходной токен для каждой длины блока. Затем предположите, что черновик обрабатывается группами по 4 позиции, причём неполная группа требует столько же времени, сколько полная, то есть $T_m = 26.3 + 3.6 \times 4 \lceil m/4 \rceil$ мс. Сравните длины блока 4 и 5 и объясните выбор с помощью предельного условия.

8.6 Производительность и выбор конфигурации при потоке запросов

8.6.1 Интенсивность поступления, очереди, допуск и отмена

В разделах 8.2–8.5 рассматривалось, как изменять потребление памяти экземпляром и способ выполнения. Онлайн-сервис также должен обрабатывать непрерывное поступление запросов: один запрос ещё не завершён, а следующий уже попал в очередь; пока запрос ожидает ответа инструмента, вычисления модели приостановлены, но KV необходимо сохранять. Поэтому планировщик должен не только распределять текущие вычисления, но и сохранять состояние запросов, которые генерируют ответ или ожидают продолжения выполнения.

Объём памяти, необходимый для хранения состояния этих незавершённых запросов, зависит от среднего числа запросов в системе. В стабильном сервисе среднее число запросов в системе подчиняется закону Литтла:

$$\bar{n} = \lambda \bar{T},$$

где λ — средняя интенсивность поступления, а $\bar{T}$ — среднее время пребывания запроса в системе. Если сложить время пребывания всех запросов за период наблюдения и разделить на продолжительность наблюдения, получится среднее число запросов в системе. Например, если каждую секунду поступают 4 запроса и каждый в среднем находится в системе 0,5 секунды, то среднее число запросов в системе равно 2; если среднее время пребывания увеличится до 2 секунд, их станет 8.

Одни из этих запросов находятся в очереди, другие вычисляются, а для третьих лишь сохраняется состояние. Увеличение batch может повысить пропускную способность вывода, однако чтение KV, вычисления и прочие накладные расходы постепенно ограничивают этот рост. Когда скорость обработки перестаёт расти вместе с интенсивностью

поступления, новые запросы накапливаются в очереди, а время ожидания увеличивается.

Управление допуском определяет, начинать ли обработку нового запроса. Перед приёмом запроса проверяются доступное пространство KV и оставшееся до крайнего срока время. Для короткого запроса из раздела 8.1 первый вывод за 0,196 секунды и генерация за 6,76 секунды уже занимают 6,95 секунды; дополнительные 0,1 секунды в очереди приведут к превышению лимита времени. Заблаговременное выявление такого запроса позволяет отклонить его, передать другому экземпляру или применить более быструю конфигурацию выполнения. Отмена прекращает планирование последующих итераций, когда результат запроса теряет ценность, и освобождает пространство после завершения операций ускорителя.

Эти оптимизации влияют друг на друга. Сжатие KV позволяет экземпляру одновременно принимать больше запросов, а более крупный batch помогает обычному decode эффективнее совместно использовать веса, из-за чего меняется и относительная выгода спекулятивного декодирования. Далее эти изменения применяются к одному и тому же набору запросов, чтобы вычислить итоговое время выполнения и потребление памяти.

8.6.2 Эффективная пропускная способность с учётом качества и ограничений SLO

По одной лишь длине очереди и скорости завершения нельзя определить, получил ли пользователь пригодный ответ. В эксперименте по качеству из раздела 8.4.4 встречались ошибочные ответы, а из-за очереди правильный ответ может прийти слишком поздно. Эффективность сервиса можно оценить, лишь учитывая оба вида потерь. Пусть длительность окна наблюдения равна T_{obs} , для правильного ответа на запрос i значение $Q_i = 1$, а для завершённого вовремя — $D_i = 1$. Если измерять по числу завершённых запросов, эффективная пропускная способность равна

$$R_{\text{good}} = \frac{\sum_i Q_i D_i}{T_{\text{obs}}}.$$

В числителе учитываются только правильные и завершённые вовремя запросы. Если вместо этого поместить в числитель количество всех завершённых запросов, получится пропускная способность завершения. Разница между ними показывает, сколько вычислительной мощности потрачено на ошибочные или запоздавшие ответы.

Серия измерений показывает величину обеих потерь. В эксперименте по поиску с Qwen3-8B использовались одни и те же восемь вопросов, причём запрос должен был завершиться с правильным ответом в течение 3 секунд после поступления. Сравнивались три режима: последовательный приём запросов, непрерывная пакетная обработка и одновременная отправка всех запросов. В каждом эксперименте отправлялось 16 запросов; всего во всех экспериментах было сгенерировано 336 ответов, из которых 294 оказались правильными, но лишь 155 одновременно уложились в срок. Остальные 139 правильных ответов вернулись слишком поздно и потому не удовлетворили требованиям сервиса.³²

³²Исходные данные о сервисе и энергопотреблении. Одно устройство, восемь задач, по три окна из 16 запросов для каждого из семи условий; ответы на одну и ту же задачу совпадают при разных условиях, а все 42 ошибки относятся к одной задаче. В таблице пропускной способности приведена медиана метрик по окнам. Каждое окно содержит 16 запросов; p95 рассчитывается по позиции с округлением вверх в отсортированном списке и потому равен максимальному значению. Окно подсчёта компонентов NVML и RAPL немного

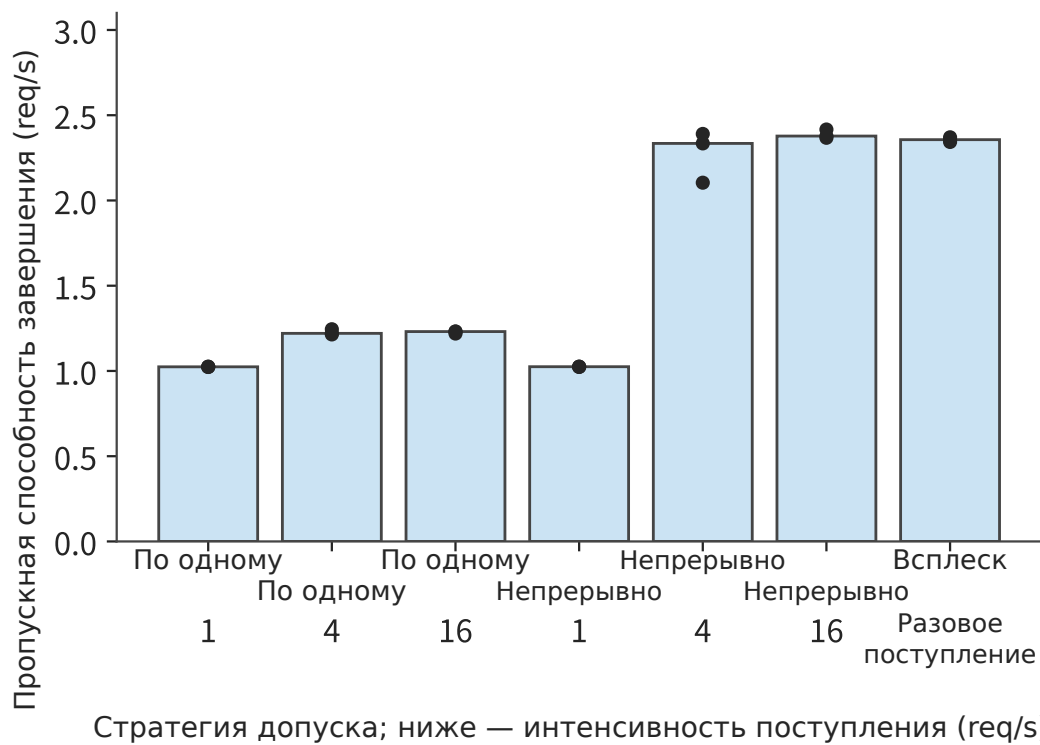
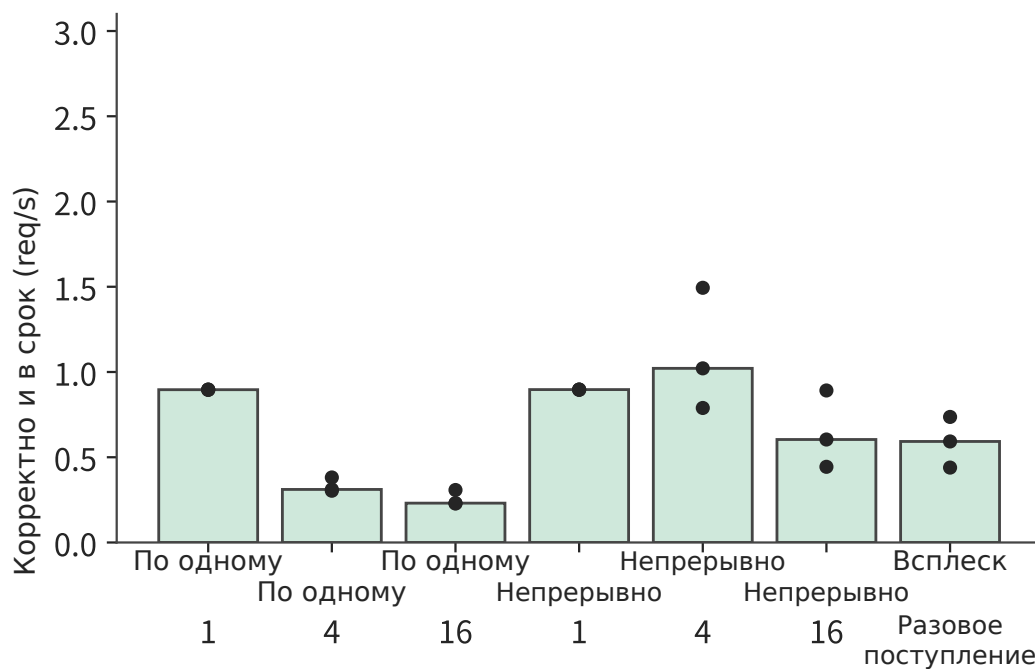


Рисунок 8-33. Для каждого сочетания режима допуска и условий поступления используются три окна по 16 запросов. Столбцы показывают медианную пропускную способность завершения всех запросов, точки — результаты отдельных окон. Время отсчитывается от запланированного момента поступления.



Стратегия допуска; ниже — интенсивность поступления (req/s)

Рисунок 8-34. В числителе учитываются только правильные запросы, завершённые в течение 3 секунд. На обоих графиках используются одинаковые порядок категорий по горизонтальной оси и диапазон вертикальной оси; разница обусловлена ошибочными или запоздавшими ответами.

При интенсивности поступления 1/s эффективная пропускная способность обеих онлайн-стратегий составляет около 0,90 req/s. Объём работы в каждом batch невелик, поэтому даже при последовательном допуске большинство запросов с правильными ответами удаётся обработать вовремя. Когда интенсивность поступления возрастает до 4/s, пропускная способность завершения при последовательном допуске составляет около 1,22 req/s, однако эффективная пропускная способность падает до 0,31 req/s: очередь растёт, и многие правильные результаты приходят после крайнего срока. Непрерывная пакетная обработка повышает пропускную способность завершения примерно до 2,33 req/s, а эффективную пропускную способность — примерно до 1,02 req/s, поэтому больше запросов завершается в срок.

Если затем увеличить интенсивность поступления в непрерывном режиме до 16/s, пропускная способность завершения возрастет лишь примерно до 2,38 req/s, а эффективная пропускная способность, напротив, упадёт примерно до 0,60 req/s, то есть приблизительно на 41%. Пропускная способность завершения практически перестаёт расти, и поступление дополнительных запросов главным образом увеличивает время ожидания в очереди. Для этой серии окон при 4/s вовремя возвращается больше правильных ответов, чем при 16/s. При выборе конфигурации сервиса следует сначала определить интенсивность поступления, при которой эффективная пропускная способность начинает снижаться, а затем ограничить количество вновь принимаемых запросов, чтобы избежать слишком долгого ожидания в очереди.

При последовательном приёме остальные запросы ещё не переданы движку, но про-
 шире окна запросов; другие сервисы продолжали работать.

должают ждать в очереди приложения. Поэтому время следует отсчитывать от запланированного момента поступления запроса, включая ожидание вне движка. С учётом измерений из этого раздела нужно отдельно вычислять количество токенов, выдаваемых ускорителем в секунду, время ожидания одного запроса и число правильных, своевременно завершённых запросов в секунду.

8.6.3 Конфигурация, ускоритель и полная стоимость задачи

Перед сравнением конфигураций преобразуем результаты измерений в скорость обработки. После выбора пакетной обработки, точности и настроек кэша фактическая скорость каждого этапа определяется делением объёма завершённой работы на время использования ускорителя: скорость prefill r_P измеряется числом новых входных токенов, обрабатываемых в секунду, а скорость decode r_D — числом вызовов decode, завершаемых в секунду. Если несколько запросов в batch совместно используют одно выполнение, считается, что каждый запрос завершил один вызов, а время использования учитывается только один раз. Для запроса с L новыми входными токенами и G выходными токенами время использования ускорителя в GPU-секундах равно

$$t_{\text{GPU}} = \frac{L}{r_P} + \frac{G - 1}{r_D}.$$

Если измерений в тех же условиях нет, время можно оценить, используя произведение характеристик оборудования и эффективности: время prefill определяется делением количества матричных операций на произведение пиковой вычислительной мощности и эффективности её использования, а время одного раунда decode — делением количества прочитанных байтов весов и KV на произведение пиковой пропускной способности памяти и эффективности её использования. В таблице ниже приведена эффективность, измеренная в эксперименте 8-1 на RTX PRO 6000. Время одного раунда decode берётся по журналу движка для раундов без prefill; интервал между результатами, наблюдаемый клиентом, дополнительно включает время доставки и потому немного больше, например 26,5 ms в разделе 8.1.1.

Контекст 2K, без общего префикса	batch 1	batch 4	batch 16	batch 64
Весы и KV, читаемые за раунд	15.44 GB	16.34 GB	19.97 GB	34.46 GB
Время чтения при 1792 GB/s	8.62 ms	9.12 ms	11.14 ms	19.23 ms
Измеренное время одного раунда decode	26.26 ms	26.62 ms	27.44 ms	29.63 ms
Эффективность пропускной способности decode	33%	34%	41%	65%
Эффективность вычислений prefill	62%	61%	61%	58%

Эффективность decode растёт вместе с batch не потому, что чтение становится быстрее. В этом эксперименте CUDA Graph был отключён, поэтому в каждом раунде каждый

kernel отправлялся отдельно: объём чтения вырос с 15,44 GB до 34,46 GB, а время одного раунда — лишь с 26,26 ms до 29,63 ms. Чем больше batch, тем больше байтов приходится на этот почти постоянный промежуток времени. Эффективность prefill при этом стабильно составляет 58–62% от пиковой производительности BF16; для входа 8K с уже кэшированными первыми 6K она равна 52–58%.

Эффективность пропускной способности в таблице — это MBU из раздела 1.2.2, а эффективность вычислений — MFU. При batch 1 один раунд decode занимает примерно на 17,6 ms больше нижней границы чтения в 8,62 ms, причём это дополнительное время практически не зависит от объёма чтения. Согласно критерию из раздела 1.3.4, причина заключается не в пропущенной составляющей модели, а в накладных расходах на очередную отправку kernel, планирование на стороне хоста и синхронизацию. Их можно устранить с помощью CUDA Graph и более качественного планирования; пример vLLM v0.6.0 из раздела 5.5.1 показывает, насколько можно сократить накладные расходы на стороне хоста.

Комплексный пример: выбор конфигурации с общим префиксом, сжатием и спекулятивным декодированием при ограничениях памяти и времени. Эффективная пропускная способность задаёт критерий оценки сервиса, а анализ ёмкости и времени позволяет ещё до запуска исключить неподходящие конфигурации. Используем экземпляр инференса, заданный в начале главы: экземпляру выделено 32 GiB на RTX PRO 6000, а после вычета весов и базовой рабочей области для KV и вспомогательных буферов остаётся 12 GiB. Одновременно поступают 16 длинных запросов, каждый с 8192 входными и 256 выходными токенами; первые 6144 входных токена у них совпадают, а предельное время завершения равно 7 секундам. Ниже сравниваются четыре варианта, причём предполагается, что каждый из них даёт правильные ответы. В вариантах с общим префиксом он уже кэширован до поступления запросов. В спекулятивном варианте используется черновая модель DFlash из эксперимента 8-5. После её включения пиковое потребление видеопамати процессом возрастает примерно на 3,1 GiB — с 17 706 MiB до 20 894 MiB, причём веса черновой модели занимают около 1,95 GiB. Черновая модель также должна обработать вход: в эксперименте 8-5 при входе примерно из 2300 токенов время до первого токена выросло со 129,4 ms до 141,6 ms, то есть на 9,4%. Предположим, что в этой группе запросов каждый раунд в среднем выдаёт 3 токена.

Вариант с общим префиксом для этой группы запросов точно соответствует условиям эксперимента 8-1: вход 8K, первые 6K уже кэшированы, batch 16. На этапе prefill обработка 32 768 новых токенов заняла 2,04 s, $r_P \approx 16,059$ новых токенов/s, что соответствует 58% пиковой вычислительной мощности; после этого каждый раунд decode занимал 27,35 ms, а $r_D = 16/0.02735 \approx 585$ вызовов/s. Каждый запрос занимает $t_{GPU} = 2048/16059 + 255/585 \approx 0.563$ GPU-секунды, а все 16 — около 9,01 GPU-секунды. Остальные варианты выводятся из этой таблицы и измерений в тех же условиях; ниже приведено время каждого этапа для всего batch из 16 запросов.³³

33

Таблица эффективности эксперимента 8-1 экспортирована скриптом расчёта из поэтапных записей сканирования batch; пиковые значения 1792 GB/s и 503.8 TFLOP/s для RTX PRO 6000 взяты из таблицы оборудования. Пик видеопамати, время до первого токена и длительность каждого раунда DFlash взяты из эксперимента 8-5. Память и время для каждого варианта повторно рассчитаны посредством проверки примера.

Вариант	Организация состоя- ния	До возврата первого токена (prefill всего batch)	Выполнение после возврата первого то- кена
A	BF16, независимые контексты	7.34 s	255 раундов по 29.63 ms
B	BF16, общий префикс	2.04 s	255 раундов по 27.35 ms
C	q8_0, независимые контексты	7.34 s	255 раундов пример- но по 28.26 ms
D	BF16, общий префикс и спекулятивное деко- дирование	2.23 s	85 раундов по 3 токе- на, каждый раунд по 27.35 ms

В A и C префикс не используется совместно, поэтому для каждого запроса приходится полностью обрабатывать 8192 токена. Общее число матричных операций всего batch равно 2137,5 TFLOP, что при эффективности вычислений 58% в тех же условиях требует около 7,34 s. В каждом раунде A считывается 34,46 GB — столько же, сколько для 64 запросов по 2K, поэтому берётся указанное в таблице время 29,63 ms. В C размер KV уменьшается в 34/64 раза, поэтому в каждом раунде считывается 25,40 GB; интерполяция по объёму чтения между столбцами 19,97 GB и 34,46 GB даёт около 28,26 ms, без учёта времени преобразования формата. Prefill в D на 9,4% дольше, чем в B, и занимает около 2,23 s. В каждом раунде D для каждого из 16 запросов проверяется 8 позиций, что требует около 2,56 TFLOP матричных операций и при эффективности 58% занимает примерно 8,8 ms — меньше 27,35 ms на один раунд B. Согласно разделу 8.2.1 берётся большее из двух значений, поэтому раунд по-прежнему занимает 27,35 ms. Прямой проход черновой сети также выполняется в рамках того же раунда, а в эксперименте 8-5 медианное время раунда DFlash не превышало времени одного обычного шага decode (раздел 8.5.2), поэтому дополнительное время здесь не учитывается.

Сначала проверим, достаточно ли памяти. Для A требуется $16 \times 1188 = 19008$ MiB, что превышает 12 GiB, поэтому одновременно принять всю группу запросов невозможно. Для B нужна лишь одна копия префикса и 16 частных состояний — всего 6048 MiB, или около 5,91 GiB. Состояние C сжато в 34/64 раза и занимает в общей сложности около 9,86 GiB. D добавляет к B 3188 MiB, поэтому занимает всего 9236 MiB, или около 9,02 GiB. B, C и D помещаются в память; теперь сравним время, необходимое каждому из них для завершения запросов.

Общее время B равно $2.04 + 255 \times 0.02735 = 9.01$ секунды, что совпадает с измеренной в эксперименте 8-1 медианной длительностью запроса 9,02 секунды, но превышает лимит в 7 секунд. На этой карте только 255 раундов decode занимают 6,97 секунды, поэтому 16 совместно обрабатываемых запросов, каждый из которых генерирует по одному токену за раунд до получения 256 выходных токенов, не успевают завершиться вовремя. В C повторное использование префикса отсутствует, поэтому время prefill возрастает с 2,04 s до 7,34 s, а общее время достигает $7.34 + 255 \times 0.02826 = 14.55$ секунды. D выдаёт по 3 токена за раунд и за 85 раундов генерирует оставшиеся $85 \times 3 = 255$ токенов, поэтому общее время равно

$$T_D = 2.23 + 85 \times 0.02735 = 4.56 \text{ s.}$$

A исключается из-за нехватки памяти, а B и C — из-за превышения лимита времени; только D возвращает 16 правильных ответов вовремя. В этом сравнении последова-

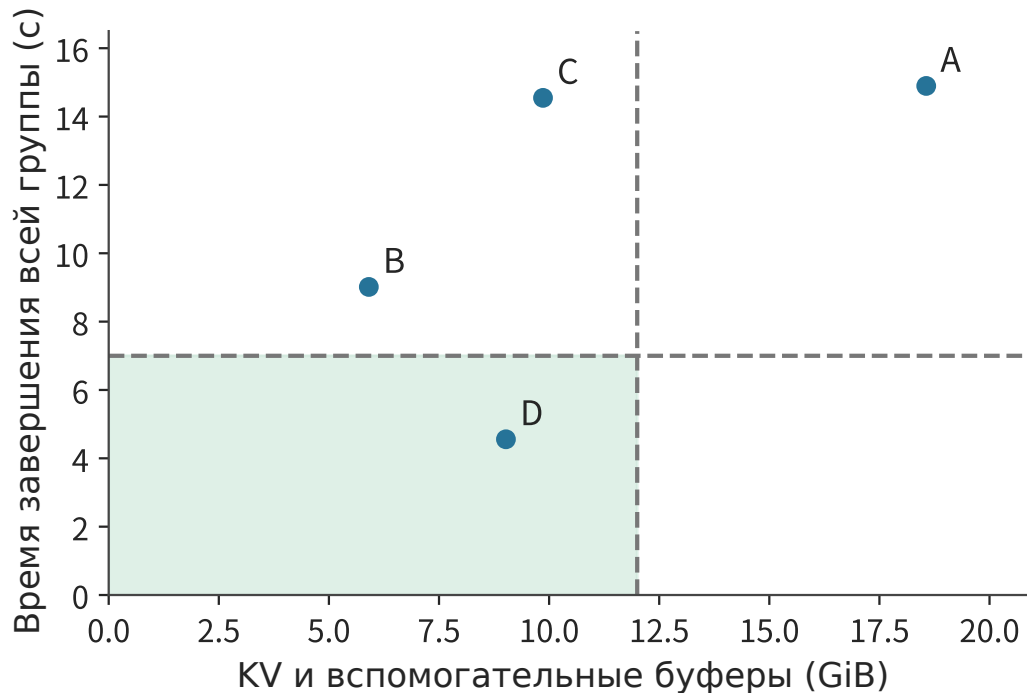


Рисунок 8-35. Сравнение конфигураций для 16 длинных запросов на RTX PRO 6000. По горизонтальной оси отложено потребление памяти KV и вспомогательными буферами после вычета общих весов и базовой рабочей области; по вертикальной — общее время обработки всего batch, выведенное из измерений экспериментов 8-1 и 8-5. Для A недостаточно памяти экземпляра, поэтому на графике показано лишь необходимое ему время. Затенённая область одновременно удовлетворяет ограничениям 12 GiB и 7 секунд. A — BF16 с независимыми контекстами, B — BF16 с общим префиксом, C — q8_0 с независимыми контекстами, D — BF16 с общим префиксом и спекулятивным декодированием.

тельно учитывались ёмкость после совместного использования префикса, размер сжатого формата, длительность раунда и число токенов, выдаваемых за раунд. На этой карте время одного раунда decode почти не зависит от batch и контекста, поэтому заметно сократить этап генерации можно только уменьшением числа раундов.

На рисунке 8-35 обе проверки представлены на одной плоскости. Выход за пунктирную линию по горизонтали означает, что KV и вспомогательные буферы занимают больше 12 GiB; выход за пунктирную линию по вертикали означает, что время завершения превышает 7 секунд. Только D удовлетворяет обоим ограничениям.

Наконец, сравним стоимость. Время использования карты на один соответствующий требованиям результат равно времени от поступления всего batch до полного завершения, делённому на число таких результатов. Для D это $4.56/16 \approx 0.285$ GPU-секунды; при предельной мощности 600 W энергопотребление на один соответствующий требованиям результат не превышает примерно 171 J. В занимает карту 9,01 секунды, но при лимите в 7 секунд не даёт ни одного соответствующего требованиям результата, поэтому всё это время и энергия расходуются впустую.

Как изменится выбор при уменьшении доступной памяти или сокращении вывода? Если доступная для KV и вспомогательных буферов память уменьшится с 12 GiB до 6 GiB, необходимые D примерно 9,02 GiB уже не поместятся. Требуемые B примерно 5,91 GiB поместятся, но запросы не уложатся в срок. Для этой группы запросов не оста-

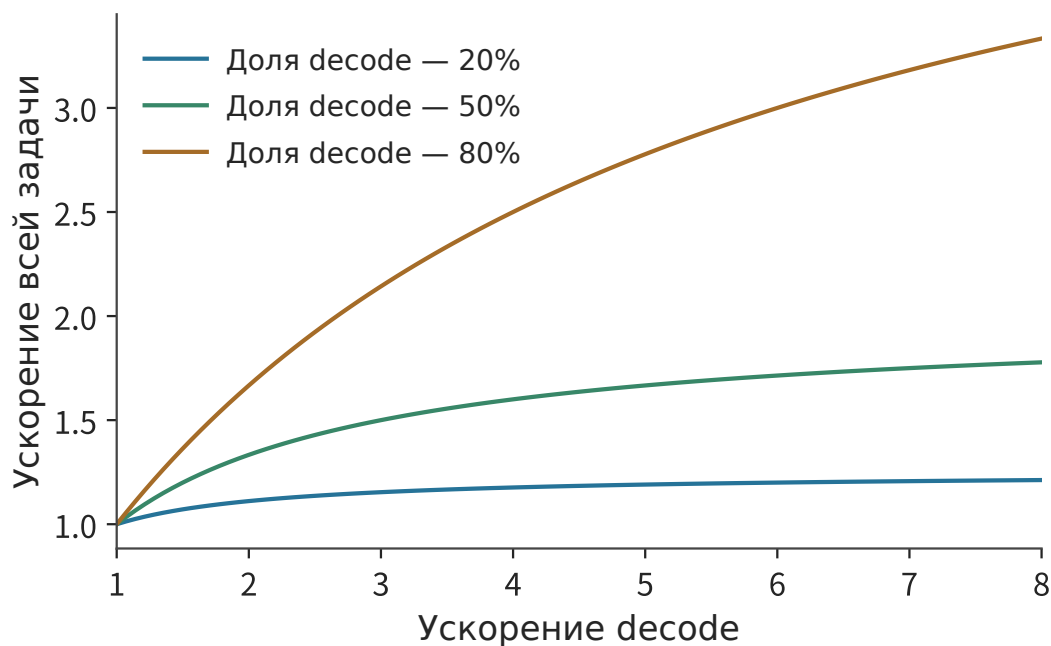


Рисунок 8-36. Кривые ускорения по закону Амдала при фиксированном времени остальных этапов и отсутствии новой подготовительной работы. Для трёх кривых доля decode в базовом времени равна соответственно 20%, 50% и 80%. Чем выше исходная доля времени этого этапа, тем больше выигрыш от его локального ускорения.

нется допустимого варианта: придётся увеличить лимит времени, сократить вывод или использовать источник черновиков, не занимающий видеопамять (см. упражнение 8-9). Если объём памяти оставить прежним, но сократить вывод до 16 токенов, В потребует-ся $2.04 + 15 \times 0.02735 = 2.45$ секунды, а D — $2.23 + 5 \times 0.02735 = 2.37$ секунды. Оба варианта завершатся вовремя, причём D будет быстрее лишь на 0,08 секунды: дополнительные 0,19 секунды prefill для черновой модели почти полностью компенсируют время, сэкономленное за счёт исключения 10 раундов decode.

Приведённый выше анализ охватывает время выполнения внутри экземпляра. Чтобы вычислить длительность полной задачи AI-агента, необходимо также учесть выполнение инструментов и внешнее ожидание. Предположим, задача занимает 100 секунд, из которых 50 секунд приходится на decode. Если ускорить decode в четыре раза, общее время составит $50 + 50/4 = 62.5$ секунды, то есть ускорение всей задачи будет равно 1,6 раза. Если изначально decode занимал лишь 20 секунд, общее время станет равным $80 + 20/4 = 85$ секундам, а суммарное ускорение составит всего около 1,18 раза. В общем случае, если доля decode в исходном общем времени равна f , этот этап ускоряется в s раз, а время остальных этапов не меняется, то по закону Амдала

$$S_{\text{task}} = \frac{1}{(1 - f) + f/s}.$$

На рисунке 8-36 чем меньше доля decode, тем раньше кривая выходит на плато: даже при дальнейшем ускорении этапа генерации вызовы инструментов и прочее ожидание продолжают занимать прежнее время. При сравнении использования ускорителя и энергопотребления необходимо также учитывать эти этапы и вести расчёт по итоговому числу соответствующих требованиям запросов. Пусть суммарное время использования

ускорителя за период наблюдения равно T_{GPU} , общее энергопотребление — E_{obs} , а число соответствующих требованиям результатов — N_g . Тогда один такой результат в среднем занимает T_{GPU}/N_g GPU-времени и расходует E_{obs}/N_g энергии. В эксперименте 8-9 на той же карте считывались счётчики энергии GPU и CPU package, то есть всего корпуса процессора: при непрерывной пакетной обработке и интенсивности поступления 1, 4 и 16 req/s на один соответствующий требованиям результат расходовалось соответственно 615,0, 652,0 и 1083,7 J; при последовательном допуске и 4 req/s — 1982,3 J. При 16/s пропускная способность завершения в непрерывном режиме близка к показателю при 4/s, но запоздавших ответов больше, поэтому энергопотребление на один соответствующий требованиям результат примерно на 66% выше. Даже если ответ ошибочен или возвращён слишком поздно, уже израсходованная энергия должна учитываться в общей стоимости.³⁴

Упражнение 8-8 · Расчёт: как ускорение этапа и подготовка черновиков изменяют общее время задачи. Задача длительностью 100 секунд состоит из prefill продолжительностью 20 секунд, decode продолжительностью 50 секунд и выполнения инструментов вместе с ожиданием продолжительностью 30 секунд. По отдельности сократите длительность каждой из трёх частей вдвое и вычислите общее время задачи в каждом случае. Затем, начиная с исходной задачи, сократите время decode до четверти от исходного и добавьте 8 секунд на подготовку черновиков; вычислите итоговый коэффициент ускорения и определите, при какой длительности подготовки это изменение перестанет экономить время.

Упражнение 8-9 · Основное · Комплексное проектирование: сокращение GPU-времени на один соответствующий требованиям результат при ограничениях ёмкости и времени. Используя ёмкость и временные параметры комплексного примера для RTX PRO 6000, постройте на основе В вариант Е: общий префикс BF16, черновики подбираются поиском в контексте (раздел 8.5.2), индекс размещается в памяти хоста, не занимает видеопамять и не увеличивает время prefill; каждый запрос в среднем выдаёт 2 токена за раунд, последующее выполнение занимает 128 раундов, а каждый раунд всего batch занимает 27,35 ms плюс 0,1 ms на поиск. Вычислите потребление видеопамати, общее время и количество GPU-секунд на один соответствующий требованиям результат для Е и сравните с В и D. Затем по отдельности измените объём памяти, доступной для KV и вспомогательных буферов, на 6 GiB и длину вывода на 16 токенов и заново выберите вариант. В открытой экспериментальной части запустите две конфигурации с одинаковыми наборами вопросов и траекториями поступления, зарегистрируйте для всех запросов моменты поступления, первого вывода и завершения, правильность и пиковый размер состояния, после чего вычислите эффективную пропускную способность по формулам этого раздела.

³⁴Исходные данные о сервисе и энергопотреблении. Одно устройство, восемь задач, по три окна из 16 запросов для каждого из семи условий; ответы на одну и ту же задачу совпадают при разных условиях, а все 42 ошибки относятся к одной задаче. В таблице пропускной способности приведена медиана метрик по окнам. Каждое окно содержит 16 запросов; p95 рассчитывается по позиции с округлением вверх в отсортированном списке и потому равен максимальному значению. Окно подсчёта компонентов NVML и RAPL немного шире окна запросов; другие сервисы продолжали работать.

8.7 История и дальнейшее чтение

Приём новых запросов на каждой итерации, страничное внимание и кэширование префиксного дерева способствовали развитию соответственно диспетчеризации запросов, распределения состояния и повторного использования вычислений. Соответствующие решения Orca, PagedAttention и SGLang можно изучать с точки зрения этих механизмов; Sarathi-Serve и NanoFlow дополнительно исследуют разбиение на блоки, конвейерную обработку и перекрытие различных ресурсов.³⁵³⁶

Дополнительные сведения о сервисе с несколькими адаптерами из раздела 8.2.1 можно найти в работах Punica и S-LoRA. Они управляют соответственно адаптерами и состоянием запросов, выполняя низкоранговые вычисления каждого адаптера при совместном использовании базовой модели.³⁷

Гибридное внимание и рекуррентные модели расширяют задачу индексирования кэша до восстановления состояния; в таких работах, как Marconi, рассматриваются возможности восстановления и ценность кэша. Выгрузка весов, кодирование тензоров и выполнение с унифицированной памятью, в свою очередь, связывают ёмкость ускорителя со стоимостью канала передачи; сопутствующие записи для M2 Max демонстрируют реальное распределение KV и обработку входных данных малой моделью.³⁸³⁹⁴⁰

Правила принятия и корректировки при спекулятивном сэмплировании заложили основу для сохранения целевого распределения. Последующие методы, формирующие черновик посредством поиска в контексте, прогнозирования по промежуточным признакам или параллельной генерации, главным образом изменяют затраты на один раунд и длину принятого черновика. Полная задача AI-агента также включает инструменты и внешние этапы; опубликованные данные о скорости и выполнении задач можно рассматривать вместе с декомпозицией по этапам из раздела 8.6.⁴¹⁴²

8.8 Итоги главы

В этой главе на примере выполнения отдельного запроса последовательно разбирались взаимосвязи между объёмом памяти, организацией вычислений и эффективностью сервиса. Разные запросы совместно используют веса, тогда как KV-кэш каждого запроса растёт вместе с его контекстом. Формирование batch позволяет сократить объём чтения весов в расчёте на каждый выходной токен, однако для каждого запроса по-прежнему требуется читать его собственный контекст. Выгода зависит от производительности низкоуровневых ресурсов: если веса предоставляются более быстрым отдельным хранили-

³⁵Оригинальная статья Orca и блочные вычисления с примечаниями о версиях.

³⁶Обзор OSDI 2025, окончательная версия статьи NanoFlow и пример расчёта совместного использования и размещения ресурсов.

³⁷Обзор MLSys 2024 и вычисления с несколькими LoRA.

³⁸Восстановление гибридного состояния и расчёт checkpoint, обзор MLSys 2025. Ёмкость Kimi K3 рассчитана для заданного формата хранения и конфигурации параллелизма.

³⁹Сжатие и передача тензоров. Материалы охватывают реализацию LLM.265 на видеодвижке, проектирование специализированного оборудования и оценку производительности.

⁴⁰Данные о ёмкости малой модели на M2 Max и дополнительные измерения для 32K. В каждом слоте последовательно выполняется prefill.

⁴¹Оригинальная статья о спекулятивном сэмплировании, оригинальная статья о спекулятивном декодировании, заметки о подсчёте и бюджете фреймворков, охват фреймворков в обзоре. Базовый вероятностный алгоритм, подходы к формированию черновика и конкретные реализации цитируются отдельно.

⁴²Эксперимент по генерации и журнал выполнения агента. В журнале агента используются непотоковые вызовы; диаграмма длительности этапов построена по параметрам из условия задачи.

щем, формирование batch больше не позволяет распределить затраты на их чтение, поэтому разные размеры batch и схемы спекулятивного декодирования необходимо сравнивать заново. Непрерывная пакетная обработка своевременно занимает освободившиеся после коротких запросов позиции выполнения, а блочный prefill уменьшает влияние длинного ввода на генерацию уже обрабатываемых запросов.

Способ управления KV-кэшем определяет, сколько запросов экземпляр может обрабатывать одновременно. Страничное управление выделяет пространство по мере необходимости, совместное использование позволяет хранить общую часть префикса только в одном экземпляре, а кэширование префиксов дополнительно устраняет повторные вычисления. Дополнение, изменение и суммаризация контекста влияют на повторное использование кэша, а также на затраты последующего сохранения и чтения состояния. При сравнении этих способов организации контекста с другими методами оптимизации сервиса следует предъявлять одинаковые требования к качеству выполнения задачи. Сжатие дополнительно уменьшает объём хранения, а выгрузка переносит часть весов за пределы ускорителя ценой их многократного перемещения. Спекулятивное декодирование увеличивает затраты на черновую генерацию и проверку, но позволяет за одно вычисление целевой модели определить несколько выходных токенов. Все эти изменения можно сравнивать по объёму используемой памяти, длительности каждого раунда и числу выходных токенов за раунд.

В итоговом комплексном примере сначала проверяется достаточность памяти, затем — возможность завершить работу в срок, после чего сравнивается время GPU, затрачиваемое на каждый результат, соответствующий требованиям. Благодаря совместному использованию префикса 16 длинных запросов, которые прежде не помещались в памяти, теперь могут выполняться одновременно. Однако на RTX PRO 6000 последовательная генерация 256 выходных токенов всё ещё не укладывается в срок; только спекулятивное декодирование сокращает число раундов decode настолько, чтобы запросы завершились вовремя. При уменьшении доступной памяти для этой группы запросов невозможно найти подходящее решение, а при сокращении выходных данных преимущество спекулятивного декодирования почти исчезает. Причины изменения выбора можно найти в выведенных в этой главе объёмах данных, времени выполнения и числе выходных токенов.

После завершения описанного в этой главе сравнения конфигураций следует зафиксировать три типа результатов: состояния скольких запросов можно одновременно хранить при заданных ускорителе, точности и длине запроса; с какой скоростью prefill и decode обрабатывают работу по отдельности; как задержка запросов изменяется в зависимости от размера batch и частоты их поступления. Эти результаты уже учитывают влияние планирования, кэширования и эффективности kernel.

В следующей главе на основе этих результатов определяется, какую работу будут выполнять разные типы ускорителей, сколько потребуется вычислительных блоков и как будет передаваться и совместно использоваться KV-кэш. Несколько ускорителей могут совместно выполнять один экземпляр, образовывать несколько полных реплик либо использоваться как раздельно планируемые пулы сервисов для отдельных этапов. Пакетная обработка определяет, как имеющиеся ресурсы обрабатывают запросы, а способ развёртывания — как между ними распределяется работа; вместе они влияют на производительность всего сервиса.

Глава 9

Распределённый инференс

В главе 8 рассматривалось, как повысить эффективность инференса на заданном наборе вычислительных ресурсов с помощью пакетной обработки, диспетчеризации запросов, кэширования и спекулятивного декодирования. Эти методы применимы как к одной карте, так и к экземплярам, совместно выполняемым на нескольких картах. В этой главе подробнее исследуется размещение вычислений и состояния: какие этапы и операторы выполняют разные вычислительные ресурсы, как распределяются веса и KV, а также как передаются данные, распределяется работа и обрабатываются сбои между местами выполнения. С увеличением контекста состояние может стать больше весов: в примере из раздела 9.2.3 при batch size, равном 32, KV занимает в сумме 43,5 ГБ — примерно в 2,7 раза больше весов Qwen3-8B. Поэтому размещение и передача состояния не менее важны, чем распределение вычислений.

В этой главе **P** обозначает этап prefill, а **D** — этап decode. При **разделении PD** эти два этапа передаются разным ресурсам. При **разделении AF** внимание и сеть прямого распространения каждого слоя, включая сеть экспертов, передаются разным ресурсам.

В связи с размещением вычислений и состояния в этой главе рассматриваются четыре основных способа организации. Добавление полных реплик позволяет разным группам ресурсов обрабатывать запросы независимо. Разделение prefill и decode позволяет отдельно выделять вычислительные ресурсы для каждого этапа; разделение вычислений внимания и экспертов позволяет использовать вычислительные возможности и возможности хранения данных разных ресурсов; совместное использование KV позволяет запросам применять состояние, оставленное в других местах выполнения. Эти способы организации могут применяться как внутри одного экземпляра инференса, так и для соединения нескольких независимо диспетчеризируемых пулов сервисов.

Проектная задача этой главы такова: один сервер оснащён четырьмя A100 80GB SXM, другой — четырьмя H20 SXM5 96GB. Как этим восьми картам обрабатывать 3,5 запроса, поступающих каждую секунду? Каждый запрос содержит 8192 входных токена и 1025 выходных токенов и предполагает рассуждение: модель сначала генерирует достаточно длинные рассуждения, а затем выдаёт ответ. Используется модель Qwen3-8B; веса и KV хранятся в формате BF16 по 2 байта на элемент. Каждая карта вмещает полную модель, поэтому одна карта представляет собой один исполнитель (worker). Между двумя серверами нет NVLink, и обмен данными возможен только через сетевые адаптеры RDMA. Далее последовательно определяется, как распределить вычисления, где передавать контекст, когда имеет смысл сохранять кэш и за какое время после запуска удастся устранить накопившуюся очередь. В разделе 9.7 эти результаты будут объединены для выбора схемы развёртывания на данном наборе ресурсов. Для сравнения в разделе 9.2

также анализируется однородный кластер из восьми A100.

Другой пример, рассматриваемый в разделах 9.3 и 9.4, использует Qwen3-235B-A22B и исследует размещение весов экспертов. При одних и тех же 512 назначениях экспертов в зависимости от результатов назначения может потребоваться считать веса множества экспертов либо многократно использовать веса небольшого числа часто востребованных экспертов. На этой основе определяется, где следует выполнять вычисления экспертов — на CPU или GPU, а затем рассматривается, почему при концентрации часто востребованных экспертов на одной карте необходимы их реплики. В обоих примерах применяется один и тот же метод анализа: сначала определяется объём работы и состояния, необходимый для каждого запроса, а затем оценивается скорость, с которой различные типы процессоров могут выполнить эту работу, а системы хранения — предоставить необходимые данные.

9.1 Способы распределения вычислений и состояния

9.1.1 Вычислительные ресурсы, экземпляры инференса и реплики сервиса

Сначала разграничим организацию оборудования и организацию сервиса. Ускорители, такие как GPU и NPU, а также CPU предоставляют вычислительные мощности и соответствующие ресурсы хранения; процессы выполняют на этих устройствах модель и коммуникационную логику. Каждый процесс в коммуникационной группе имеет свой rank. Один экземпляр инференса может совместно выполняться несколькими такими процессами. Полная реплика сервиса обладает весами и вычислительными возможностями, необходимыми для выполнения всего инференса модели, и может обрабатывать запросы независимо от других реплик.

На рис. 9-1–9-4 показаны четыре способа организации. В первых двух случаях выполняется полная модель; различие состоит в том, что группа из восьми совместно работающих карт обрабатывает один и тот же пакет запросов, а несколько полных реплик независимо принимают запросы. В следующих двух случаях разделение задач меняется ещё сильнее: PD разделяет их по этапам генерации, а AF — по операторам внутри слоя. У каждого способа организации есть собственный планировщик, управляющий закреплёнными за ним вычислениями.

Полные реплики разделяют работу по запросам; на рис. 9-3 один и тот же запрос последовательно проходит через два пула сервисов. Если проследить стрелку от P к D, передаётся контекстный KV, оставшийся после обработки входных данных, а каждый из двух пулов по-прежнему выполняет полную модель для соответствующего этапа.

На рис. 9-4 граница разделения перенесена внутрь слоя модели: активации, созданные на стороне attention, передаются на сторону feed-forward, после чего результат возвращается обратно. В отличие от однократной передачи между этапами, данные проходят по этому пути снова и снова на каждом слое и каждом шаге генерации.

При сравнении этих способов развёртывания сначала следует использовать конфигурацию выполнения и скорость обработки, полученные в главе 8, а затем изменять распределение работы между вычислительными ресурсами. Способ, при котором P и D обрабатываются одной и той же группой исполнительных ресурсов, называется **совместным размещением**. Рассмотрим уже оптимизированный экземпляр с совместным размещением. Длина входа запроса равна L , а полная длина выхода — G ; prefill вычисляет logits для последнего входного токена, из которых затем сэмпляется первый выходной

Несколько GPU совместно исполняют одну полную модель

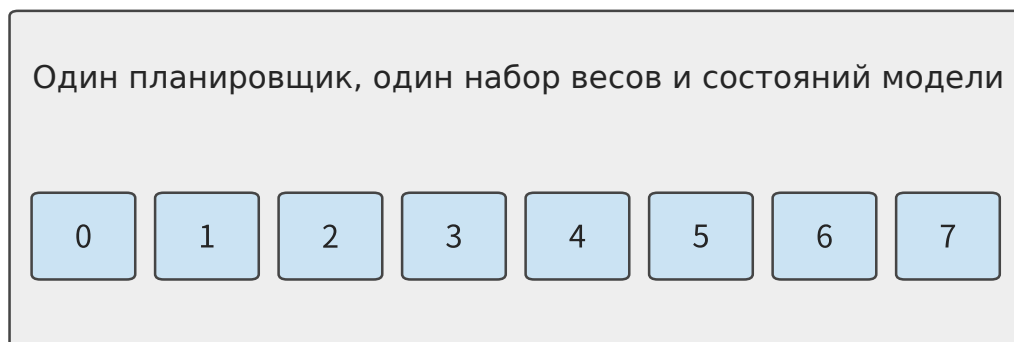


Рис. 9-1. Восемь карт совместно выполняют один полный экземпляр инференса и принимают один и тот же пакет запросов. Номера карт обозначают участников группы.

Полные копии принимают запросы отдельно

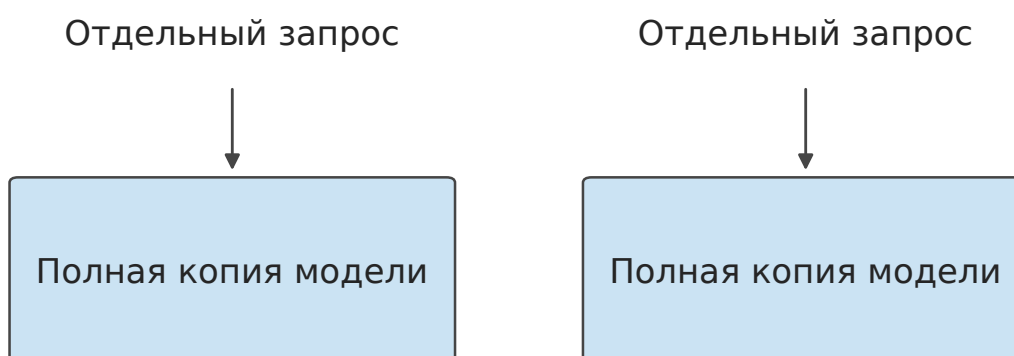


Рис. 9-2. Каждая реплика способна выполнить инференс целиком и может независимо принимать запросы. Сама реплика также может состоять из нескольких карт.

PD: разделение обработки входа
и генерации

Передача KV-кэша контекста



Рис. 9-3. P обрабатывает входные данные, D продолжает генерацию. Два пула сервисов планируются независимо, а контекстный KV передаётся от P к D.

AF: разделение
операторов в каждом
слое

Послойная передача активаций

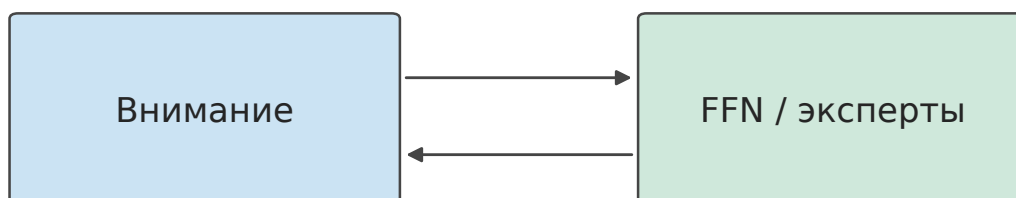


Рис. 9-4. Attention и FFN/эксперты выполняются отдельно, а скрытое состояние в виде активаций передаётся между двумя сторонами туда и обратно. Такая передача требуется на каждом слое.

токен. В соответствии с принятым в этой главе соглашением о генерации последующих вызовов `decode` будет $G - 1$: последний возвращённый токен ещё не был передан модели, поэтому число выходных токенов на единицу больше числа последующих вызовов `decode`.

Для заданного распределения запросов обозначим через d_r среднее время обслуживания одного запроса на ресурсе r , а через n_r — число независимо работающих единиц ресурсов этого типа. Интенсивность поступления запросов и время занятости ресурсов должны удовлетворять условию

$$\lambda d_r < n_r,$$

где λ — интенсивность поступления запросов. Если несколько операций используют один ресурс, сначала следует сложить время, в течение которого каждая из них занимает этот ресурс; если операции используют независимые друг от друга ресурсы, ограничение следует записать отдельно для каждого типа ресурсов. Объём использования ресурса рассчитывается как «число ресурсов $\times$ количество секунд занятости»; в этой главе ресурсом служит GPU, поэтому это произведение измеряется в GPU-секундах. Например, если каждую секунду поступает четыре запроса и каждый занимает 0,5 GPU-секунды, потребуются два непрерывно работающих GPU. Если карт ровно две, дополнительные запросы из всплеска нагрузки будут продолжать накапливаться, поскольку обе карты заняты непрерывно поступающими последующими запросами; ликвидировать накопившуюся после всплеска очередь можно только после добавления GPU.

Пусть `prefill` и `decode` одного запроса занимают на реплике данного типа соответственно d_P и d_D секунд, а два этапа выполняются на этой реплике последовательно. Тогда верхний предел интенсивности обработки запросов для N одинаковых реплик с совместным размещением равен

$$\mu_{\text{co}} = \frac{N}{d_P + d_D}.$$

В разделе 9.2.3 время обслуживания на каждом этапе для двух типов карт, упомянутых в начале главы, будет выведено из пиковых значений в таблице данных. Один A100 выполняет `prefill` для 8192 токенов примерно за 0,86 секунды, а 1024 вызова `decode` одного запроса в сумме занимают около 1,76 GPU-секунды; на одном H20 `prefill` занимает около 1,81 секунды, а `decode` — лишь около 0,88 GPU-секунды. Каждая полная реплика должна выполнять оба этапа, поэтому на один запрос A100 расходует 2,62 GPU-секунды, а H20 — 2,68 GPU-секунды. Восемь карт в совокупности выполняют примерно 3,02 запроса в секунду, что ниже интенсивности поступления в 3,5 запроса в секунду, поэтому очередь растёт примерно на 0,5 запроса каждую секунду. Теперь задача сформулирована чётко: позволит ли иное распределение работы между теми же восемью картами устранить дефицит в 0,5 запроса/с.

Реплика может состоять из нескольких совместно работающих карт. Например, один из вариантов развёртывания Qwen3-72B-A22B с TP2 $\times$ EP4, где TP делит матрицу экспертов на две части, а EP распределяет экспертов по четырём группам, использует восемь карт для обработки одного и того же пакета запросов: каждые две карты делят между собой матрицу экспертов, а четыре группы EP содержат разные наборы экспертов. Attention и KV реплицируются в каждой из четырёх групп EP, поэтому контекст одного запроса длиной 8192 токена занимает во всём экземпляре 5,875 GiB — в четыре раза

больше одной логической копии KV. Эти восемь карт совместно образуют один экземпляр инференса; для добавления реплики, способной независимо обрабатывать запросы, потребуется отдельно разместить ещё один полный набор весов и состояния.¹

9.1.2 Разделение вычислений и совместное использование состояния

Добавление полных реплик изменяет распределение запросов; разделение PD изменяет место выполнения prefill и decode; разделение AF изменяет место выполнения attention, FFN и экспертов; совместное использование KV изменяет область хранения и чтения контекстного состояния. Эти способы развёртывания изменяют разные части системы: их можно вводить по отдельности или комбинировать.

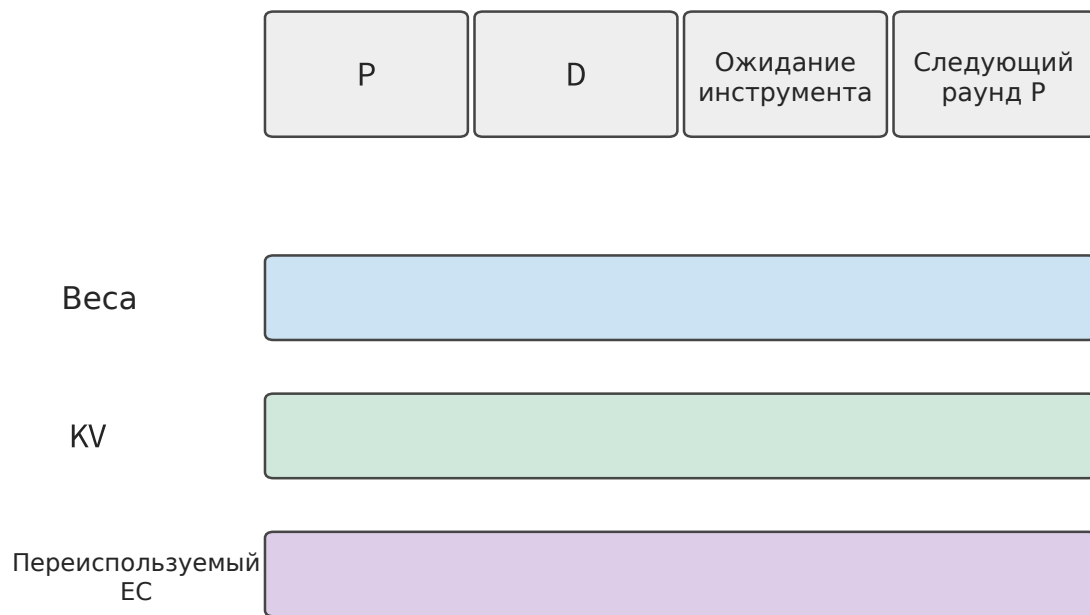
Способ развёртывания	Ожидаемая выгода	Новые основные ограничения
Добавление полных реплик	Обработка большего числа независимых запросов, распределение очереди	Копирование модели, разрозненные кэши, запуск и поддержание работы экземпляров
Разделение PD	Независимая настройка мощности двух этапов	Передача KV, одновременное использование памяти на обеих сторонах, две очереди
Разделение AF	Использование операторами подходящих вычислительных ресурсов и ресурсов хранения	Передача активаций туда и обратно на каждом слое, синхронизация, балансировка нагрузки между этапами конвейера
Совместное использование KV	Повторное использование контекста между экземплярами, сокращение повторных prefill	Идентификация кэша, чтение, уведомление о готовности, инвалидация и освобождение пространства

При выборе способа развёртывания сначала следует проанализировать потребности текущей нагрузки в ресурсах. Если потребности двух этапов в ресурсах близки, полные реплики уже могут обеспечивать сбалансированное обслуживание, а разделение способно лишь добавить накладные расходы на передачу. Если многие запросы используют общий длинный префикс, межэкземплярный кэш может оказаться полезнее дальнейшего разделения вычислений. Раздельный расчёт занятости ресурсов и времени передачи позволяет определить, что сейчас требуется: добавить реплики, разделить этапы или организовать совместное использование состояния.

9.1.3 Путь выполнения одного запроса и время пребывания состояния

После выбора способа разделения работы следует проследить временную шкалу одного запроса: через какие этапы он проходит и какое состояние оставляет. На рис. 9-5 этапы вычисления многораундового запроса и время пребывания состояния показаны на одной временной шкале. При наличии визуального входа этап кодирования E преобразует изображение в признаки, используемые моделью, а ЕС сохраняет результаты кодирования для повторного использования. Р обрабатывает результаты кодирования и текст, D

¹Принадлежность весов, состояния и коммуникации одного МоЕ-запроса. TP2×EP4, обработка одной и той же группы запросов и формат передачи FP32 являются явно заданными учебными условиями для конкретной демонстрации принадлежности коммуникации.



Ширина этапов условно отражает порядок; при ожидании состояние сохраняется

Рис. 9-5. Во время приостановки вычислений состояние всё равно необходимо сохранять. Показано время пребывания состояния одного запроса от prefill и decode до ожидания инструмента и следующего раунда; горизонтальная ось упорядочена по этапам и не означает равную длительность, ЕС присутствует только при наличии визуального входа. E — визуальное кодирование, EC — сохранённый результат визуального кодирования, P — prefill, D — decode.

пошагово генерирует выходные данные; после возврата результата инструментом в новом раунде P обрабатывает добавленную информацию. Веса обычно сохраняются между запросами, активации в основном передаются между соседними операторами, а KV растёт по мере увеличения числа обработанных позиций. На время ожидания инструмента вычисления можно приостановить, но контекстное состояние при этом всё равно может занимать память.

На примере используемой в этой главе Qwen3-8B одна копия контекстного KV в формате BF16 для 8192 токенов занимает 1,125 GiB. Если инструмент выполняется 10 секунд, GPU может обрабатывать другие запросы, но этот контекстный KV всё равно занимает 1,125 GiB памяти, а произведение объёма на время использования составляет 11,25 GiB·с; десять одновременно ожидающих таких сеансов займут 11,25 GiB. После приостановки вычислений состояние всё равно необходимо сохранять — именно эту задачу решают общий кэш и стратегии выгрузки.

Если затем этот запрос вернёт 1025 токенов, P сначала создаст первый выходной токен, после чего 1024 вызова decode обработают первые 1024 сгенерированных токена. В этот момент непрерывный KV охватывает $8192 + 1024 = 9216$ токенов, а последний возвращённый токен ещё не передан обратно модели. Следующий раунд продолжается после этих 9216 токенов: сначала обрабатываются ещё не записанный в KV последний токен и новые входные данные, после чего генерация возобновляется с этой позиции.

9.2 Разделение Prefill-Decode и распределение ресурсов

9.2.1 Зачем разделять Prefill и Decode

Prefill может за один вызов обработать множество новых токенов, а крупные матричные вычисления обычно позволяют эффективнее переиспользовать веса. На этапе decode каждый запрос за один шаг обычно обрабатывает лишь один новый токен, поэтому приходится многократно считывать веса и состояние контекста. При совместном выполнении двух этапов длительный prefill может задержать вывод следующего токена для уже обрабатываемых запросов, а чрезмерный приоритет decode увеличивает время ожидания новых запросов.

В разделе 8.2 блочный prefill уже применялся для сокращения блокировки decode длинным вводом. В этом разделе при тех же модели и запросах дополнительно изменяется место выполнения. Разделение PD размещает два этапа в разных пулах обслуживания с независимыми очередями, пакетной обработкой и распределением ресурсов. DistServe и Splitwise — системы инференса, исследующие раздельное развёртывание P и D. В однородном кластере из восьми A100 разделение устраняет влияние планирования одного этапа на другой благодаря независимым очередям. В гетерогенном кластере из A100 и H20 каждый тип карт может выполнять только тот этап, для которого он подходит лучше всего.²

Такое разделение труда похоже на ситуацию, когда один человек сначала полностью читает материал, а затем передаёт его другому для продолжения работы: принимающей стороне нужны уже накопленные рабочие записи. Для модели из этого примера передаваемой записью служит контекстный KV-кэш. P и D обрабатывают разные этапы одного запроса, причём на каждом этапе выполняется полный прямой проход. P для Qwen3-8B обрабатывает 8192 входных токена, после чего D вызывается 1024 раза; каждый вызов проходит через attention и feed-forward network. Поэтому обоим пулам нужны полные веса модели, а KV-кэш передаётся от P к D. Выигрыш от независимой пакетной обработки прежде всего должен компенсировать эту дополнительную передачу.

Приложение одним вызовом запрашивает полный инференс модели, тогда как внутри сервиса разные ресурсы могут назначаться по этапам. Зависимости данных модели определяют точку разделения этапов, KV-кэш задаёт состояние, которое необходимо передать между ними, а соотношение нагрузок определяет требуемый объём ресурсов каждого пула. Используя эти сведения, сервис может за единым интерфейсом вызова отдельно выбирать batch и вычислительные ресурсы для обработки ввода и пошаговой генерации.

9.2.2 Передача KV-кэша и потребление памяти на обеих сторонах

После завершения обработки ввода на P этапе D требуется получить контекстный KV-кэш, чтобы продолжить генерацию. В GQA несколько голов запросов используют общий набор ключей и значений, а для каждого токена контекста сохраняются полные ключи и значения. Пусть число слоёв равно n , длина контекста — L , число KV-голов — h_{kv} , размерность одной головы — d_h , а размер одного элемента — b байт. Тогда логический размер состояния равен

²Архивные материалы по DistServe, Splitwise и размещению этапов приведены в материалах расширения этой главы и исследовании разделения этапов и передачи состояния.

$$V_{KV} = 2nLh_{kv}d_hb.$$

Коэффициент 2 соответствует K и V. Фиксированная конфигурация Qwen3-8B содержит 36 слоёв, 8 KV-голов и размерность головы 128. KV-кэш в формате BF16 занимает 144 KiB на токен; для 8192 токенов он занимает

$$V_{KV} = 8192 \times 144 \text{ KiB} = 1.125 \text{ GiB}.$$

Пример 9.1. Сколько времени занимает передача KV-кэша между prefill и decode? Сервер A100 и сервер H20 соединены сетевой картой RDMA на 200 Gbit/s, однонаправленная пропускная способность полезной нагрузки принимается равной 25 GB/s, а состояние передаётся целиком напрямую. Время передачи только полезной нагрузки равно

$$T_{\text{payload}} = \frac{1.125 \times 2^{30}}{25 \times 10^9} \approx 48.3 \text{ ms}.$$

После добавления 5 μ s на запуск и синхронизацию общее время по-прежнему составляет около 48.3 ms: для полного объёма этих KV-данных фиксированные накладные расходы составляют лишь около одной десятичной, поэтому увеличение пропускной способности полезной нагрузки эффективнее сокращает время ожидания. Однако при послойной передаче AF небольшими объёмами доля фиксированных накладных расходов будет выше.³

Приведённый выше размер состояния обусловлен способом хранения в GQA. MLA (раздел 2.3.2) сохраняет не развёрнутые ключи и значения, а латентные переменные до повышающей проекции: для каждого токена на каждом слое хранятся только одна латентная переменная размерности d_c и одна позиционная ветвь размерности d_r . Размер состояния равен

$$V_{KV, \text{MLA}} = nL(d_c + d_r)b.$$

Фиксированная конфигурация DeepSeek-V3 содержит 61 слой, $d_c = 512$ и $d_r = 64$. В формате BF16 каждый токен занимает $61 \times 576 \times 2 = 70,272$ байт, или около 68.6 KiB; контекст из тех же 8192 токенов занимает 549 MiB. При использовании канала 25 GB/s из примера 9.1 передача только полезной нагрузки занимает около 23.0 ms; при увеличении скорости канала до 50 GB/s — 11.5 ms; состояние GQA по тому же каналу передаётся за 24.2 ms.

Передаваемое состояние	На токен	8192 токена	25 GB/s	50 GB/s
Qwen3-8B, BF16 GQA	144 KiB	1.125 GiB	48.3 ms	24.2 ms
DeepSeek-V3, компактное BF16 MLA	68.6 KiB	549 MiB	23.0 ms	11.5 ms

³Расчёт передачи PD/AF для Qwen3-8B с фиксированными официальной формой модели, числом байтов и предположениями о запуске. Для построения графика используется одноимённый JSON. Для 25 GB/s применяются десятичные единицы, а для 1.125 GiB — двоичные; точный объём полезной нагрузки составляет 1207959552 bytes, чистое время передачи — 48.31838208 ms, а с добавлением 5 μ s — 48.32338208 ms. Суммарная передача AF за 1024 шага decode одного запроса приведена в соответствующем результате.

Начало передачи: полный объём
занят с обеих сторон

Qwen3-8B, GQA



DeepSeek-V3, компактный
MLA



Копирование данных; ширина блоков пропорциональна числу
байтов

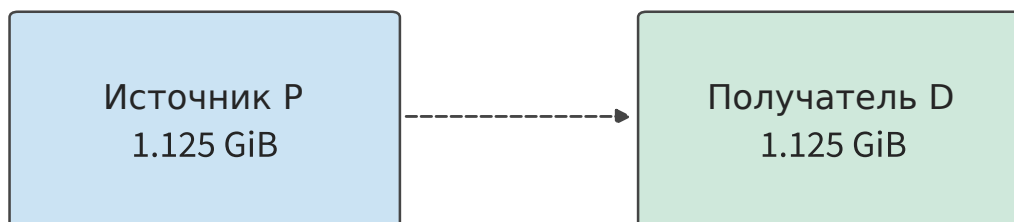
Рисунок 9-6. Источник Р сохраняет полное состояние, а получатель D одновременно выделяет приёмный буфер того же размера. Состояние GQA занимает по 1.125 GiB на каждой стороне, всего 2.25 GiB во время передачи; компактное состояние MLA занимает по 549 MiB, всего 1.07 GiB. Ширина прямоугольников пропорциональна числу байтов.

Состояние MLA меньше, поскольку кэш размещается по другую сторону линейного преобразования: GQA сохраняет развёрнутые K и V для каждой KV-головы, а MLA — только латентную переменную до повышающей проекции. Благодаря ассоциативности повышающую проекцию можно перенести на сторону текущего запроса. Если так же хранить в развёрнутом виде 128 голов DeepSeek-V3, каждый токен будет занимать $61 \times 128 \times (192 + 128) \times 2$ байт, или около 4.77 MiB, что в 71 раз больше компактного представления; при этом компактное MLA занимает лишь 0.48 от объёма GQA в Qwen3-8B, где всего 8 KV-голов. Далее везде, где в формулы входит объём KV-кэша в байтах, параллельно приводятся результаты для обоих состояний.⁴

При передаче всего состояния с двойной буферизацией источник сохраняет его полностью до окончания передачи, а получатель выделяет полный приёмный буфер. На рисунках 9-6–9-8 последовательно показаны три момента: начало передачи, её завершение и освобождение памяти источника. После записи всей полезной нагрузки на стороне получателя Р отправляет маркер завершения, и D считывает состояние только по-

⁴Расчёт передачи PD/AF для компактного состояния MLA, а также сравнение MLA и GQA на канале 50 GB/s; число байтов на токен согласуется со строкой `deepseek-v3` в расчёте кэша для разных моделей. Значения $d_c = 512$ и $d_r = 64$ взяты со страницы 12 статьи DeepSeek-V2; V3 использует ту же конфигурацию attention. Точный объём полезной нагрузки составляет 575,668,224 bytes, чистое время передачи при 25 GB/s — 23.02672896 ms, а критическое время запуска — 23003136/71 ns.

Передача завершена: уведомить
получателя о доступности KV



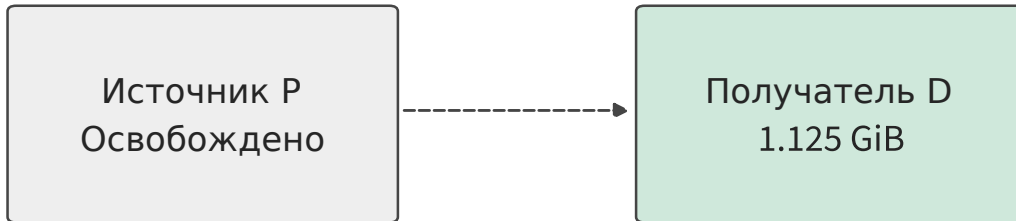
Метка завершения задаёт порядок использования

Рисунок 9-7. Полная полезная нагрузка объёмом 1.125 GiB уже записана на стороне получателя, после чего Р отправляет маркер завершения. Сам маркер больше не перемещает данные, а лишь задаёт порядок: сначала завершить запись, затем использовать данные. D считывает состояние только после получения маркера.

сле получения этого маркера (рисунок 9-7). Р освобождает исходный буфер лишь после передачи управления D (рисунок 9-8). При блочной передаче источник может заранее освободить блок после того, как получатель подтвердит его получение, сократив время дублирующего размещения на двух сторонах; по маркерам завершения получатель определяет, какие последовательные блоки уже доступны. Если данные передаются через основную память, состояние последовательно занимает буферы источника, основной памяти и получателя, а копирование из основной памяти в GPU становится новой зависимостью.

При непрерывном поступлении запросов пропускная способность канала ограничивает как число состояний, которые можно передать за секунду, так и время передачи каждого состояния. Для 8 таких передач в секунду требуется около 9.7 GB/s, а для 16 — около 19.3 GB/s. Порт 100 GbE (Ethernet 100 Gbit/s) обеспечивает 12.5 GB/s в каждом направлении и удовлетворяет средней потребности в первом случае, однако каждый запрос всё равно должен дожидаться завершения передачи. Во втором случае ситуация иная: даже при наличии свободных вычислительных ресурсов в обоих пулах сетевая очередь будет непрерывно расти. В первом случае передача каждого состояния занимает около 96.6 ms, а во втором каждую секунду будет накапливаться около 6.8 GB данных, ожидающих передачи. Время передачи определяет минимальное ожидание отдельного запроса, а разность между скоростью поступления данных и скоростью их передачи — темп роста очереди. Для компактного состояния MLA объёмом 549 MiB 8 и 16 передач в секунду требуют лишь около 4.6 и 9.2 GB/s соответственно. Один порт 100 GbE удовлетворяет средней потребности в обоих случаях, а передача каждого состояния занимает

Источник освобождает буфер,
D продолжает генерацию



Следующий запрос может использовать память источника

Рисунок 9-8. В этом примере исходный буфер P освобождается после передачи управления D, а D сохраняет контекст и продолжает генерацию. Память источника используется для последующих запросов.

около 46.1 ms. Рост очереди совместно определяется числом байтов состояния и пропускной способностью канала: при смене представления контекста модели вывод для того же канала может стать противоположным.

9.2.3 Производительность этапов и соотношение числа экземпляров

Перед сравнением скорости обработки на разных этапах следует сначала вычислить объём работы, который один запрос требует на каждом из них. Пусть запрос содержит L_{new} новых токенов, которые должен обработать P, и $G - 1$ вызовов decode. Эффективная производительность worker определённого типа равна соответственно r_P новых токенов/s и r_D вызовов decode/s. Тогда требуемое этому worker время на один запрос равно

$$d_P = \frac{L_{new}}{r_P}, \quad d_D = \frac{G - 1}{r_D}.$$

Это время занятости ресурсов можно складывать напрямую; показатели входных токенов/s и выходных токенов/s сначала необходимо преобразовать в потребности одного и того же запроса. Когда несколько запросов в batch совместно используют один вызов decode, завершённый шаг decode следует учитывать для всех запросов, но время занятости ресурсов учитывается один раз — по единственному пакетному выполнению.

Значения r_P и r_D для двух типов карт можно вывести из таблицы характеристик. Модель Roofline из главы 4 задаёт нижнюю границу времени каждого этапа как максимум из времени вычислений и времени доступа к памяти. В следующей таблице приведены пиковые показатели двух карт. Вычислительная производительность H20 составля-

ет менее половины производительности A100, но пропускная способность видеопамати примерно вдвое выше. Именно поэтому H20 хорошо подходит для decode и плохо — для prefill.

Карта	Плотная производи- тельность BF16	Пропускная способ- ность видеопамати	Объём видеопамати
A100 80GB SXM	312 TFLOP/s	2039 GB/s	80 GB
H20 SXM5 96GB	148 TFLOP/s	4096 GB/s	96 GB

Реальные kernel не достигают пиковых показателей. В разделе 8.6.3 две эффективности были откалиброваны по последовательным замерам на RTX PRO 6000: эффективность использования пропускной способности при decode возрастает с 33% при batch size 1 до 65% при batch size 64, а при batch size 32 интерполяция между измеренными значениями даёт около 49%; эффективность использования вычислительной производительности при prefill стабильно составляет 58%–62%. В этой главе для обоих показателей принимается 50% от пикового значения: на стороне decode это соответствует интерполяции для batch size 32, а на стороне prefill примерно на одну десятую ниже измеренного значения. При таком допущении эффективные вычислительная производительность и пропускная способность A100 равны 156 TFLOP/s и 1020 GB/s, а H20 — 74 TFLOP/s и 2048 GB/s.

Принять 50% означает задать MFU и MBU из раздела 1.2.2 равными 0.5: половина возможностей оборудования не преобразуется в вычисления модели. Согласно критериям раздела 1.3.4, у этой половины есть два источника. Первый — работа, не учтённая моделью первого порядка в этой главе, например нулевые строки, которыми kernel дополняет данные до размера tile, или коммуникации, не перекрытые вычислениями. Второй — устранимые накладные расходы на стороне хоста, например последовательный запуск отдельных kernel, пересоздание коммуникационных групп и графов выполнения перед переключением. По одному примеру приведено в разделах 9.4.3 и 9.6.2.

Подставим эти две группы эффективных характеристик в запрос из начала главы. Prefill обрабатывает 8192 входных токена, требуя 133.6 TFLOPs матричных вычислений, но лишь однократно считывает около 15 GB весов, поэтому ограничен вычислительной производительностью. На A100 требуется 0.856 секунды, что соответствует примерно 9570 новым токенам в секунду; на H20 — 1.81 секунды и около 4540 токенов в секунду. Один шаг decode генерирует по одному токену для каждого из 32 запросов в batch. Если принять средний контекст в процессе генерации равным $8192 + 512 = 8704$ токенам, за один шаг считывается около 15.1 GB весов и 41.1 GB KV-кэша, а объём матричных вычислений составляет лишь 0.65 TFLOPs, поэтому этап ограничен пропускной способностью видеопамати. На A100 один шаг занимает 55.1 ms, что соответствует примерно 580 вызовам decode в секунду; на H20 — всего 27.4 ms и около 1166 вызовов в секунду. Сильные и слабые стороны двух карт прямо противоположны: A100 выполняет prefill более чем вдвое быстрее H20, а H20 выполняет decode вдвое быстрее A100. При batch size 32 к концу генерации KV-кэш всех запросов занимает 43.5 GB, а вместе с весами — около 59.9 GB, что помещается на обоих типах карт.⁵

⁵Производительность этапов выводится расчётом rd-pool из таблицы оборудования и пооперационного учёта прямого прохода Qwen3-8B. Пиковое значение для A100 80GB SXM взято из спецификации NVIDIA A100. NVIDIA не публиковала спецификацию H20; модель и объём памяти взяты из документации AI Enterprise vGPU, а вычислительная производительность BF16 и пропускная способность памяти — из таблицы 3 на странице 8 MegaScale-Infer, где строки A800 и H800 согласуются со спецификациями NVIDIA. Ка-

Сначала рассмотрим однородный сценарий: восемь A100 выполняют одну и ту же модель. При совместном размещении на один запрос каждая карта расходует $0.856 + 1.764 = 2.62$ GPU-секунды, а суммарная производительность восьмью карт составляет 3.05 запроса/s. После разделения два пула можно формировать только из целого числа карт: три карты для P обеспечивают 3.50 запроса/s, а пять карт для D — 2.83 запроса/s. Верхний предел равен 2.83 запроса/s, что на 7% ниже совместного размещения. В этом расчёте потребность каждого этапа в GPU-секундах на запрос до и после разделения принята неизменной, поэтому само разделение не повышает расчётный предел пропускной способности; выбранное целочисленное разбиение также не позволяет точно уравнивать производительность пулов, и часть мощности остаётся неиспользованной. Этот вывод относится к условиям примера, а не к любому однородному кластеру: независимый выбор batch size и параллелизма для P и D может изменить потребность этапов в GPU-секундах, и её нужно оценивать заново. Кроме общей пропускной способности (throughput), следует отдельно сравнивать goodput — производительность при соблюдении заданных SLO: изоляция очередей может улучшить её даже без роста общего throughput.⁶

Взамен однородное разделение обеспечивает стабильность задержки. При совместном размещении без разбиения на блоки один prefill длиной 8192 токена монопольно занимает A100 примерно на 0.86 секунды, останавливая генерацию всех находящихся в обработке запросов на этой карте, что эквивалентно пропуску примерно 16 шагов decode длительностью 55 ms. Если эти восемь карт принимают 2.5 запроса в секунду, на каждой карте такой prefill в среднем приходится вставлять раз в 3.2 секунды. После разделения каждый шаг на картах D стабильно занимает 55 ms; карты P специализируются на обработке ввода, а время до первого токена складывается из 0.86 секунды prefill и передачи KV-кэша.

Блочный prefill из раздела 8.2 — альтернативный разделению подход. На A100 доступ к памяти при одном шаге decode занимает 55.1 ms, а матричные вычисления — лишь 4.2 ms, поэтому вычислительные ресурсы в оставшееся время простаивают. При производительности 156 TFLOP/s этого простоя достаточно для обработки примерно 490 токенов prefill. Ввод из 8192 токенов можно разделить на 17 блоков и обработать попутно за 17 шагов decode. Время до первого токена составит около 0.94 секунды, что сопоставимо с разделением, а интервал вывода для уже обрабатываемых запросов не изменится. Если попутные вычисления полностью скрываются временем доступа к памяти, верхний предел производительности восьмью совместно используемых A100 возрастает до 4.50 запроса/s, значительно превышая 2.83 при разделении. Фактический объём скрываемых вычислений зависит от того, могут ли одновременно выполняемые блочный prefill и decode полностью задействовать вычислительные ресурсы и пропускную способность; это следует определять по измеренному времени смешанного шага. Поэтому в однородном кластере обычно сначала применяют блочный prefill. Ценность разделения состоит в возможности независимо выбирать способ параллелизма и batch size для двух этапов, а также полностью изолировать их очереди друг от друга.

Пример 9.2. Как распределение этапов в гетерогенном кластере определяет

либровка в 50% взята из раздела 8.6.3 и записей эффективности по итерациям эксперимента 8-1.

⁶Целочисленное распределение гетерогенных P и D, однородный вариант с восьмью A100, однородный вариант с восьмью H20, попадание в префиксный кэш, 129 выходных токенов, 4097 выходных токенов. Поле derived_stage_rates каждого результата отдельно для каждой GPU перечисляет объём вычислений и чтения на этапах prefill и decode, обе временные составляющие Roofline и ограничивающий ресурс.

верхний предел пропускной способности? Используются 8192 входных токена и 1025 выходных токенов. Производительность этапов для четырёх A100 и четырёх H20 выводится описанным выше способом и приведена в следующей таблице; при последующем изменении состава запросов расчёт аналогично повторяется с новым объёмом работы.

Карта	Производительность Р, новых токенов/s	Производительность D, вызовов/s	GPU-секунд Р на запрос	GPU-секунд D на запрос
A100 80GB SXM	9566	580	0.856	1.764
H20 SXM5 96GB	4538	1166	1.805	0.878

При совместном размещении один запрос занимает 2.62 GPU-секунды на одной A100 и 2.68 GPU-секунды на одной H20, а восемь карт в сумме обеспечивают 3.02 запроса/s. Если назначить четыре A100 пулу Р, а четыре H20 — пулу D, пул Р обеспечит $4/0.856 \approx 4.67$ запроса/s, а пул D — $4/0.878 \approx 4.55$ запроса/s. На каждый запрос передаётся 1.125 GiB. Если между двумя серверами установлена одна сетевая карта на 200 Gbit/s, пропускная способность 25 GB/s позволяет передавать около 20.7 состояния в секунду. Тогда общий верхний предел пропускной способности равен

$$\mu_{PD} = \min \left(\mu_P, \mu_D, \frac{B_{net}}{V_{KV}} \right) \approx 4.55 \text{ запроса/s.}$$

Здесь μ_P и μ_D — верхние пределы частоты запросов пулов Р и D, а B_{net} — пропускная способность канала передачи между двумя пулами.

После разделения оба типа карт избегают более медленного для себя этапа, и верхний предел частоты запросов возрастает с 3.02 до 4.55 запроса/s, то есть в 1.51 раза относительно совместного размещения, превышая скорость поступления 3.5 запроса в секунду. Модель по-прежнему обрабатывает те же 8192 входных токена и 1024 вызова decode; меняется лишь исполнитель. Если поменять роли местами, назначив H20 для Р, а A100 для D, производительность пула Р составит всего 2.22 запроса/s, то есть верхний предел составит менее половины уровня при правильном распределении.

Попутный блочный prefill, рассмотренный выше для однородного кластера, на H20 даёт ограниченный эффект: свободной вычислительной производительности за один шаг decode хватает лишь примерно на 85 токенов prefill. Даже при идеальном попутном выполнении верхний предел для четырёх A100 и четырёх H20 при совместном размещении составляет 4.17 запроса/s, что по-прежнему ниже 4.55 при разделении. Выигрыш гетерогенного кластера обусловлен различиями самого оборудования, поэтому блочное выполнение не может заменить разделение.

Ещё одно изменяемое условие — само передаваемое состояние. При переходе на компактное состояние MLA слагаемое канала в μ_{PD} возрастает с 20.7 до 43.4 запроса/s, а при канале 50 GB/s — с 41.4 до 86.9 запроса/s. Верхний предел по-прежнему определяется пулом D с производительностью 4.55 запроса/s, поэтому распределение четырёх A100 для Р и четырёх H20 для D не меняется. Канал становится ограничивающим фактором при $B_{net} < \mu V_{KV}$: при 4.55 запроса/s состояние GQA требует не менее примерно 5.5 GB/s, а компактное состояние MLA — всего около 2.6 GB/s. Выше этого значения дальнейшее увеличение пропускной способности не меняет общий предел; ниже него все ячейки на рисунке 9-10 со значением выше B_{net}/V_{KV} ограничиваются каналом до одного и того

Пул Р: четыре A100



Пропускная способность пула: 4.67 запроса/с

Пул D: четыре H20



Пропускная способность пула: 4.55 запроса/с

Рисунок 9-9. Каждая A100 обеспечивает около 1.17 запроса/с на этапе Р, а каждая H20 — около 1.14 запроса/с на этапе D. Производительность пула Р равна 4.67 запроса/с, пула D — 4.55 запроса/с; обе ниже пропускной способности сетевой карты, составляющей около 20.7 запроса/с в пересчёте на передачу состояний.

же уровня.⁷

Компактный путь экономит байты, но не устраняет всю работу. Согласно ассоциативности из раздела 2.3.2, повышающая проекция переносится на сторону запроса: для каждого токена запроса каждый слой DeepSeek-V3 должен дополнительно выполнить одно преобразование запроса размером $128 \times 128 \times 512$ и одно восстановление значения того же размера. В сумме это 33.5 MFLOPs, а для 61 слоя — 2.05 GFLOPs, что совпадает с объёмом операций повышающей KV-проекции (`kv_b_proj`) для одного токена в таблице 2-3. Объём этих вычислений линейно растёт с числом запросов в batch: если один вызов `decode` обслуживает B запросов, добавляется $2.05B$ GFLOPs. При эффективной вычислительной производительности H20 в 74 TFLOP/s это добавляет 27.7 μ s на каждый шаг каждого запроса, или 28.3 ms за 1024 шага. Если эти вычисления нельзя скрыть временем доступа к памяти, затраты пула D на один запрос возрастут с 0.878 до примерно 0.907 GPU-секунды, а производительность пула D снизится с 4.55 до примерно 4.41 запроса/s, по-прежнему превышая скорость поступления 3.5 запроса в секунду. Пятая карта D потребуется лишь тогда, когда дополнительные вычисления на каждом шаге каждого запроса превысят примерно 258 μ s. Компактный путь меняет положение узкого места экземпляра D: объём чтения на токен сокращается с 4.77 MiB развёрнутого состояния до 68.6 KiB, но вычислительная составляющая растёт вместе с batch size, поэтому экземпляр D становится ограничен вычислениями, а не чтением, уже при меньшем batch size. Значение r_D в примере 9.2 выведено для batch size 32; после перехода на компактное MLA его необходимо пересчитать с учётом новой точки перехода между режимами.

На рисунке 9-10 также показано изменение пропускной способности при отклонении от оптимального распределения. Если перенести одну A100 из P в D, производительность P снизится с 4.67 до 3.50 запроса/s, а D возрастет с 4.55 до 5.12 запроса/s; в результате система будет ограничена P, и её производительность снизится до 3.50. Если перенести одну H20 из D в P, производительность P возрастет до 5.22 запроса/s, но D снизится до 3.42 запроса/s, поэтому система также замедлится. При оптимальном распределении производительности двух пулов близки. После изменения распределения избыточные ресурсы одного пула не могут выполнить работу другого.

9.2.4 Границы выигрыша при разных запросах и нагрузках

В примере 9.2 состав запроса был фиксирован. Теперь изменим только одно условие — попадание в кэш префикса. P уже кэшировал первые 6144 токена, поэтому ему нужно обработать лишь оставшиеся 2048. Число новых токенов сокращается в четыре раза, время `prefill` на A100 снижается с 0.856 до 0.238 секунды, а производительность P на одной A100 возрастает с 1.17 до 4.20 запроса/s. Если по-прежнему выделять P четыре A100, вместе они смогут обрабатывать 16.8 запроса/s, тогда как D останется ограничен 4.55 запроса/s.

Если перенести две A100 в D, оставшиеся две обеспечат производительность P в 8.41 запроса/s; D на двух A100 и четырёх H20 обеспечит $2/1.764 + 4/0.878 \approx 5.69$ запроса/s. Общий верхний предел пропускной способности возрастет до 5.69 запроса/s. Если перенести ещё одну A100, производительность P снизится до 4.20 запроса/s, и система, наоборот, замедлится. Предполагается, что D не кэшировал этот префикс, поэтому

⁷Целочисленное распределение P и D с компактным состоянием MLA, вариант с каналом 50 GB/s и сравнение с состоянием GQA при 50 GB/s. Дополнительные 2,046,820,352 FLOPs на шаг компактного пути указаны в поле `mha_compact_path` результата передачи MLA и на каждом слое совпадают со строкой `kv_b_proj` в таблице операторов одного шага `decode V3`.

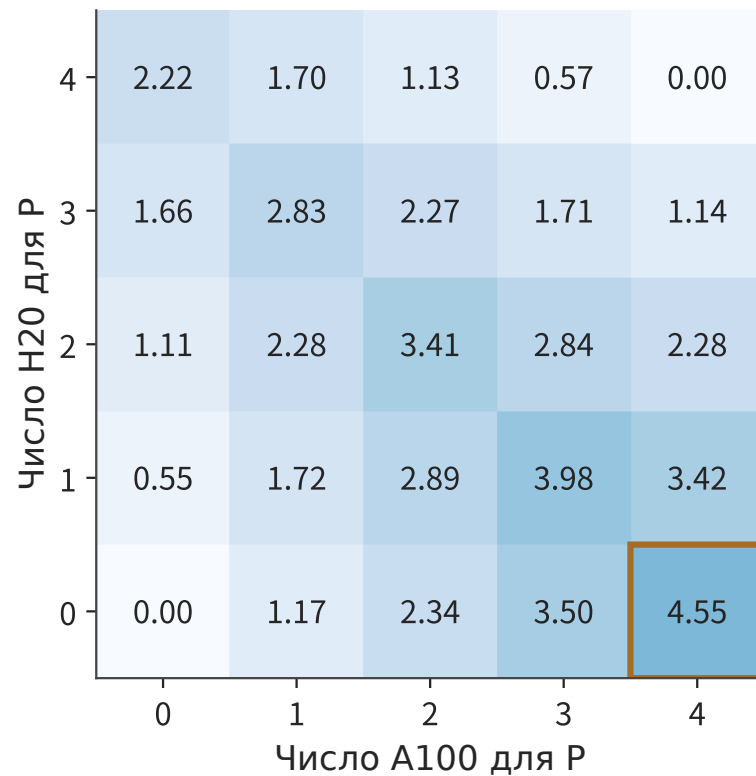
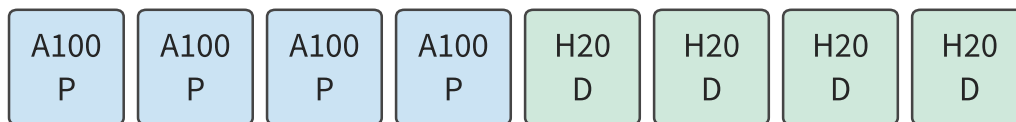


Рисунок 9-10. Влияние распределения этапов на ограничение частоты запросов. Используются четыре A100 и четыре H20, производительность этапов приведена в примере 9.2; по горизонтальной оси отложено число A100, назначенных пулу P, по вертикальной — число H20, назначенных пулу P, остальные карты назначены пулу D. В каждой ячейке берётся минимум из производительности двух пулов и частоты передачи состояний через сетевую карту с пропускной способностью 25 GB/s, без учёта очередей; интенсивность цвета обозначает устойчивую частоту запросов, единица измерения — запрос/s, рамкой выделены оптимальные распределения.

Инференс-запрос: 1025

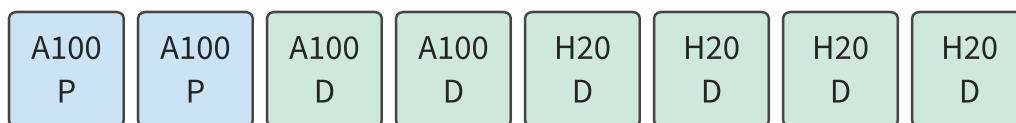
ВЫХОДНЫХ ТОКЕНОВ



Предел частоты запросов: 4.55/s

Попадание префикса: 6144

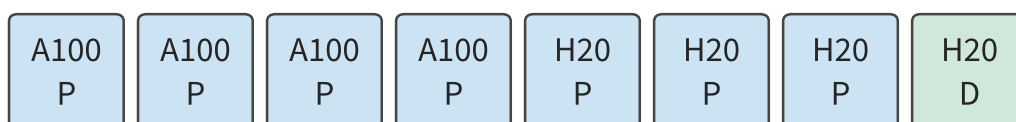
токена



Предел частоты запросов: 5.69/s

Выход сокращён до 129

токенов



Предел частоты запросов: 6.33/s

Рисунок 9-11. Изменение распределения ресурсов в зависимости от состава запросов. Используются четыре A100 и четыре H20, производительность этапов рассчитана методом из примера 9.2; первая строка соответствует запросу на рассуждение с 8192 входными и 1025 выходными токенами, во второй строке сокращён только новый ввод для P, а в третьей восстановлен полный ввод и вывод сокращён до 129 токенов. Прямоугольники обозначают карты, под каждой строкой указан соответствующий верхний предел пропускной способности. P — пул prefill, D — пул decode.

при обоих распределениях по-прежнему требуется передавать полные 1.125 GiB. Объём вычислений P сократился более чем на 70%, но размер передаваемого D KV-кэша не изменился.

Теперь восстановим исходный ввод, но сократим вывод до 129 токенов, то есть рассмотрим обычный диалог без процесса рассуждения. Число вызовов D снизится с 1024 до 128, а производительность D на одной H20 возрастет примерно до 9.46 запроса/s, поэтому D почти перестанет быть узким местом. Оптимальным становится распределение четырёх A100 и трёх H20 для P и одной H20 для D; пул P обеспечивает $4 \times 1.168 + 3 \times 0.554 \approx 6.33$ запроса/s. При совместном размещении производительность уже составляет 5.84 запроса/s, поэтому разделение повышает её лишь на 8.5%.

На рисунке 9-11 три варианта показаны для одного и того же набора вычислительных ресурсов. Каждый прямоугольник обозначает одну исходную карту; меняется только выполняемый ею этап. После попадания префикса в кэш объём работы P сокращается, поэтому можно высвободить две A100; после сокращения вывода объём работы D уменьшается, и три H20 переходят к P.

Для этих трёх видов запросов оптимальное число карт P равно соответственно четырём, двум и семи. Попадание префикса в кэш сокращает новую работу P и смещает ресурсы к D ; сокращение вывода уменьшает повторяющуюся работу D и смещает ресурсы к P . Чем длиннее вывод запроса на рассуждение, тем выше доля D и тем ценнее карты наподобие H2O с высокой пропускной способностью, но низкой вычислительной производительностью. Поэтому планирование можно разделить на два временных масштаба: при поступлении запроса выбирать очередь, а при устойчивом изменении распределения нагрузки корректировать размеры пулов. Распределение по очередям определяет текущее ожидание, а размеры пулов — способность накопившейся очереди сокращаться в долгосрочной перспективе.

9.3 Разделение механизма внимания и FFN и гетерогенное выполнение

9.3.1 Границы выполнения механизма внимания, FFN и экспертов

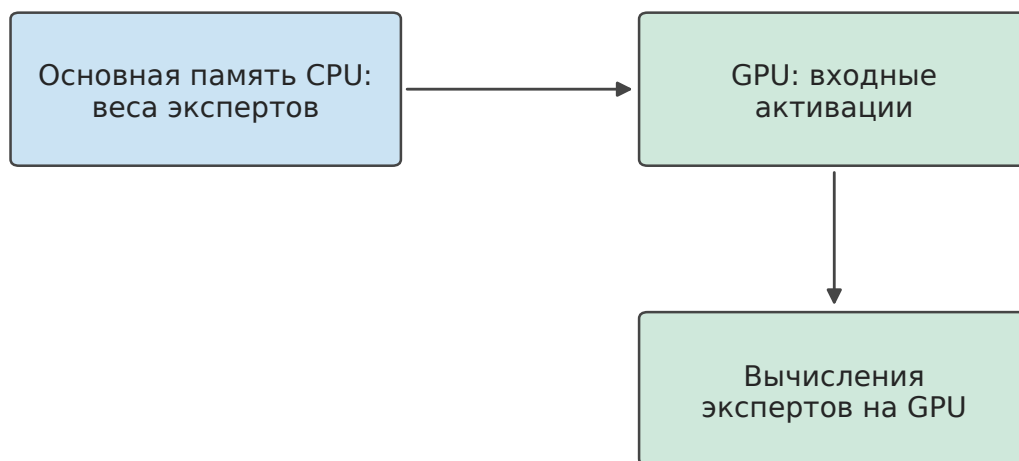
AF означает разделение задач между механизмом внимания и сетью прямого распространения. В отличие от PD, однократно разделяющего два этапа генерации, AF разделяет вычисления внутри слоя модели: механизм внимания создаёт скрытое состояние, сеть прямого распространения или выбранный эксперт обрабатывает его, после чего результат возвращается для использования последующими операторами. Следующий слой обычно должен ждать этого результата, поэтому запуск передачи небольших объёмов данных и синхронизация повторяются многократно.

МоЕ особенно хорошо подходит для такого разделения: общий объём весов экспертов велик, но каждый токен выбирает лишь их часть. Если большинство экспертов хранится в основной памяти, можно либо передавать выбранные веса на GPU, либо выполнять вычисления на CPU с локальными весами, передавая активации меньшего размера. Первый путь расходует пропускную способность канала ради вычислительной мощности GPU, второй — вычислительную мощность CPU ради уменьшения трафика по каналу. Выбор зависит от числа токенов, назначенных одному эксперту в текущем распределении: вектор признаков каждого токена занимает одну строку входной матрицы эксперта, и эти строки повторно используют один и тот же набор весов эксперта.

9.3.2 Совместная работа CPU и GPU

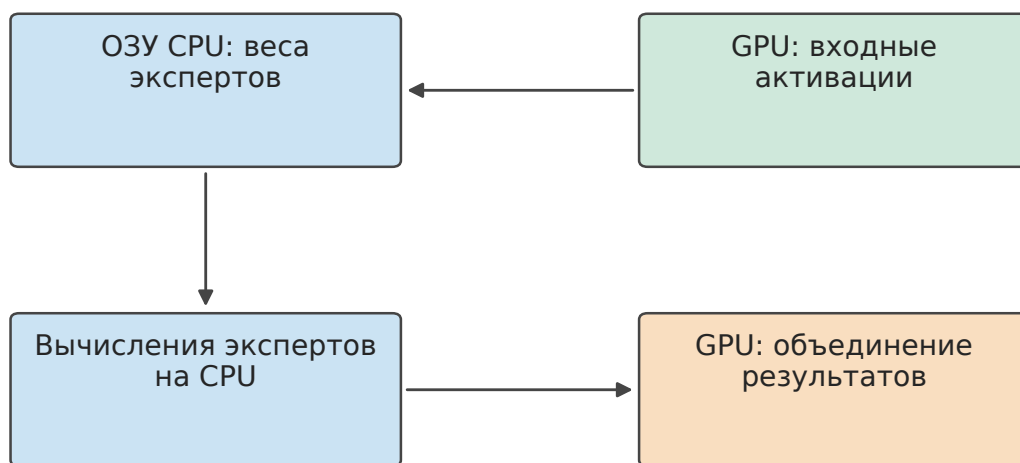
Два пути из предыдущего подраздела выдают одинаковый результат и различаются тем, что передаётся по каналу: на рис. 9-12 передаются веса, а на рис. 9-13 — входные и выходные данные. В разделе 8.4.3 используется первый путь: веса временно хранятся в основной памяти и при необходимости переносятся на GPU. KTransformers — реальная система, использующая второй путь: GPU обрабатывает механизм внимания и часть постоянно размещённых экспертов, а CPU — назначенных ему экспертов; после отправки работы на CPU GPU продолжает выполнять ветви, допускающие параллельное выполнение, а в конце результаты синхронизируются и объединяются. Для горячих и общих экспертов, а также этапа prefill можно применять различное распределение вычислительных ресурсов. В точке слияния ветвей следующий слой может начать работу лишь после завершения обеих сторон.⁸

⁸Выгрузка весов и место выполнения, а также описание версии реализации.



Перенос весов одного эксперта: 36 MiB

Рис. 9-12. Набор весов эксперта размером 36 MiB передаётся из основной памяти на GPU, после чего GPU считывает локальные входные данные и выполняет вычисления.



Активации туда-обратно: 16 KiB на строку

Рис. 9-13. Входные данные передаются с GPU на CPU, CPU выполняет эксперта с весами из основной памяти, а результат возвращается на GPU. При каждом назначении токена эксперту суммарный размер входящего и исходящего векторов признаков составляет 16 KiB; результаты объединяются после завершения параллельной ветви постоянно размещённых на GPU экспертов.

Затраты обоих путей зависят от размера набора весов одного эксперта. В Qwen3-235B-A22B эксперт содержит три матрицы с общим числом параметров $3 \times 4096 \times 1536 = 18874368$. В формате BF16 веса занимают 36 MiB.

Пример 9.3. Как повторное использование весов эксперта меняет выбор между выполнением на CPU и GPU? Возьмём экспериментальную машину из статьи о KTransformers: два Intel Xeon Platinum 8452Y по 36 ядер и 1 TB DDR5 каждый, подключённые по PCIe 4.0 к одной A100 40GB PCIe. С помощью утилиты тестирования памяти Intel MLC авторы статьи измерили пропускную способность памяти в пределах одного сокета CPU: 220 GB/s. При выполнении слоя MoE на одном CPU ядро PyTorch на основе AVX-512, 512-битного набора векторных инструкций, достигало максимум 1.8 TFLOP/s, а ядро KTransformers на основе матричных инструкций AMX, Advanced Matrix Extensions, — максимум 21.3 TFLOP/s. Для GPU возьмём 50% от пиковых значений A100 40GB PCIe, равных 312 TFLOP/s и 1555 GB/s, то есть 156 TFLOP/s и 778 GB/s. Теоретическая пропускная способность PCIe 4.0 x16 равна 32 GB/s; для передачи примем 25 GB/s и 5 μ s на каждый запуск. Ширина активации BF16 равна 4096. Восемь экспертов выполняются последовательно, поэтому общее время равно сумме времени всех экспертов, а веса считываются один раз на batch. Когда каждый эксперт получает вектор признаков только одного токена, CPU напрямую считывает веса из локальной основной памяти, избегая переноса 36 MiB по более медленному каналу. По мере увеличения числа входных строк каждого эксперта стоимость однократного переноса весов распределяется на большее число строк, тогда как CPU раньше достигает предела вычислительной мощности.

Сначала рассмотрим восемь горячих экспертов, каждый из которых обрабатывает один токен: суммарный размер весов составляет 288 MiB. Их чтение из основной памяти с пропускной способностью 220 GB/s занимает около 1.37 ms, а весь путь CPU с передачей активаций — около 1.46 ms. Для пути с переносом на GPU одна только передача весов занимает около 12.1 ms, а вместе с запусками и вычислениями GPU — около 12.5 ms. При слабом повторном использовании избежать переноса большого блока весов важнее, чем увеличить производительность матричных вычислений.

Если каждый из этих восьми экспертов обрабатывает по 128 токенов, размер весов по-прежнему равен 288 MiB, но объём вычислений возрастает в 128 раз. С ядром AVX-512 время пути CPU увеличивается примерно до 22.2 ms, а путь GPU по-прежнему занимает около 12.5 ms, поэтому вычисления на GPU становятся быстрее. На рис. 9-14 показан промежуточный процесс: когда число входных токенов на эксперта возрастает с 71 до 72, GPU начинает опережать CPU. С ядром AMX при 128 токенах путь CPU занимает лишь около 2.57 ms — примерно пятую часть времени пути с переносом весов, а точка пересечения сдвигается до 689 токенов на эксперта. На одной и той же машине при обработке одного batch выбор набора инструкций CPU определяет, на какой стороне следует разместить эксперта. Каждый дополнительный токен, обрабатываемый экспертом, увеличивает время вычислений CPU сильнее, чем GPU, тогда как затраты на перенос весов остаются неизменными, поэтому после точки пересечения преимущество получает GPU.⁹

Запишем приведённые числа в общем виде. Обозначим число параметров эксперта

⁹Kernel на AVX-512, 1 токен на эксперта, 128 токенов, kernel на AMX, 1 токен, 128 токенов; поле `locality_reuse_regions` каждого результата содержит две точки пересечения: 71/72 и 688/689. Пропускная способность kernel CPU приведена на страницах 4 и 6 статьи KTransformers, а пропускная способность памяти и конфигурация PCIe — на странице 10; пиковое значение A100 40GB PCIe приведено в таблице оборудования.

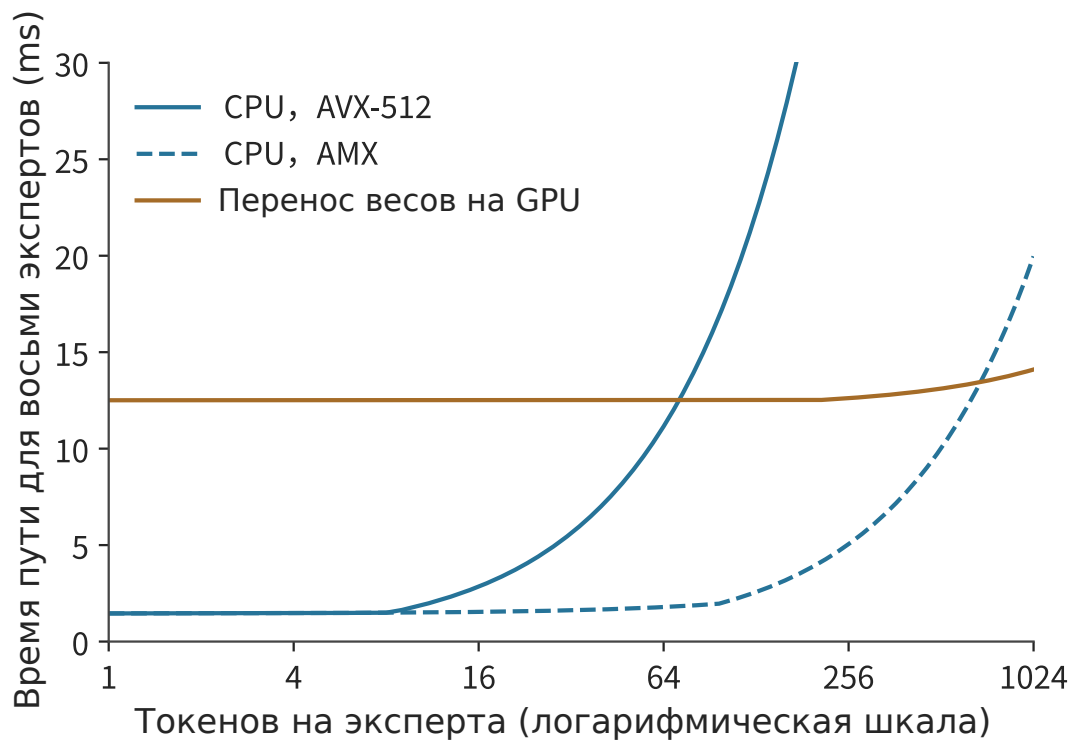


Рис. 9-14. Повторное использование экспертов меняет выбор места выполнения. Каждый из восьми экспертов имеет 36 MiB весов BF16. CPU — один Xeon Platinum 8452Y: для ядер AVX-512 и AMX используются измеренные в статье значения 1.8 и 21.3 TFLOP/s соответственно, а для памяти в пределах одного сокета — 220 GB/s. GPU — A100 40GB PCIe: 50% от пиковых значений, то есть 156 TFLOP/s и 778 GB/s. Передача по PCIe — 25 GB/s с запуском по 5 μ s. Горизонтальная ось логарифмическая; кривые рассчитаны по примеру 9.3 для пути экспертов одного слоя без преобразования форматов.

через P_e , размер весов в байтах — через W_e , ширину входа — через h , а число байтов на элемент — через b . Если один эксперт получает в текущем batch m токенов, объём его матричных вычислений составляет $F_e(m) = 2mP_e$ FLOPs, а веса необходимо прочитать как минимум один раз. Пусть эффективная вычислительная мощность CPU и пропускная способность DRAM равны соответственно C_C, B_D , а вычислительная мощность GPU и пропускная способность HBM — C_G, B_H ; пропускную способность канала передачи обозначим через B_L . Предположим, что веса и активации уже преобразованы в необходимый для выполнения формат, передача и вычисления эксперта происходят последовательно, а время оператора определяется максимумом времени вычислений и чтения весов. Тогда время выполнения одного эксперта двумя способами соответственно равно

$$T_C(m) = 2\alpha + \frac{2mhb}{B_L} + \max\left(\frac{2mP_e}{C_C}, \frac{W_e}{B_D}\right),$$

$$T_G(m) = \alpha + \frac{W_e}{B_L} + \max\left(\frac{2mP_e}{C_G}, \frac{W_e}{B_H}\right).$$

В первой формуле складывается время трёх этапов: передачи входных данных, вычисления эксперта на CPU и передачи выходных данных. Во второй сначала переносится один набор весов, а затем выполняются вычисления на GPU. Величина α в формулах — фиксированная стоимость каждой передачи. Входная активация сначала поступает на CPU, а после вычисления выходная активация возвращается, поэтому два запуска и передача полезной нагрузки туда и обратно в формуле соответствуют этим двум рёбрам зависимостей.

Влияние изменений аппаратного обеспечения можно оценивать теми же формулами. В архитектуре NUMA поток, читающий память другого CPU, должен обращаться к ней через межпроцессорное соединение. В статье измеренная межсокетная пропускная способность этой машины составила лишь 125 GB/s. Время пути CPU определяется большей из двух величин: временем чтения весов и временем матричных вычислений. На рис. 9-15 эти величины показаны рядом для случая 128 токенов на эксперта. При использовании ядра AVX-512 вычисления намного продолжительнее чтения, поэтому сокет, в памяти которого размещены данные, почти не имеет значения. При переходе на ядро AMX время вычислений сокращается почти до времени чтения в пределах одного сокета, а при межсокетном чтении именно чтение становится более продолжительной операцией. Выгодность локальных вычислений определяется текущим основным узким местом: при ограничении скоростью чтения помогает повышение пропускной способности основной памяти, а при ограничении вычислениями — повышение пропускной способности матричных операций. Формат квантования одновременно изменяет обе величины: размер весов в байтах уменьшается, а эффективная пропускная способность матричного ядра также меняется.¹⁰

9.3.3 Влияние ёмкости, параллелизма и повторного использования экспертов на возможности обслуживания

Возможность разместить веса и возможность своевременно обработать запрос — две разные задачи. В Qwen3-235B-A22B все 128 экспертов BF16 одного слоя занимают 4.5 GiB.

¹⁰Публичные записи экспериментов KTransformers. Учебная платформа использует два 6454S и 4090, а платформа из статьи — два 8452Y; опубликованные результаты Expert Deferral включают как повышение, так и снижение качества.

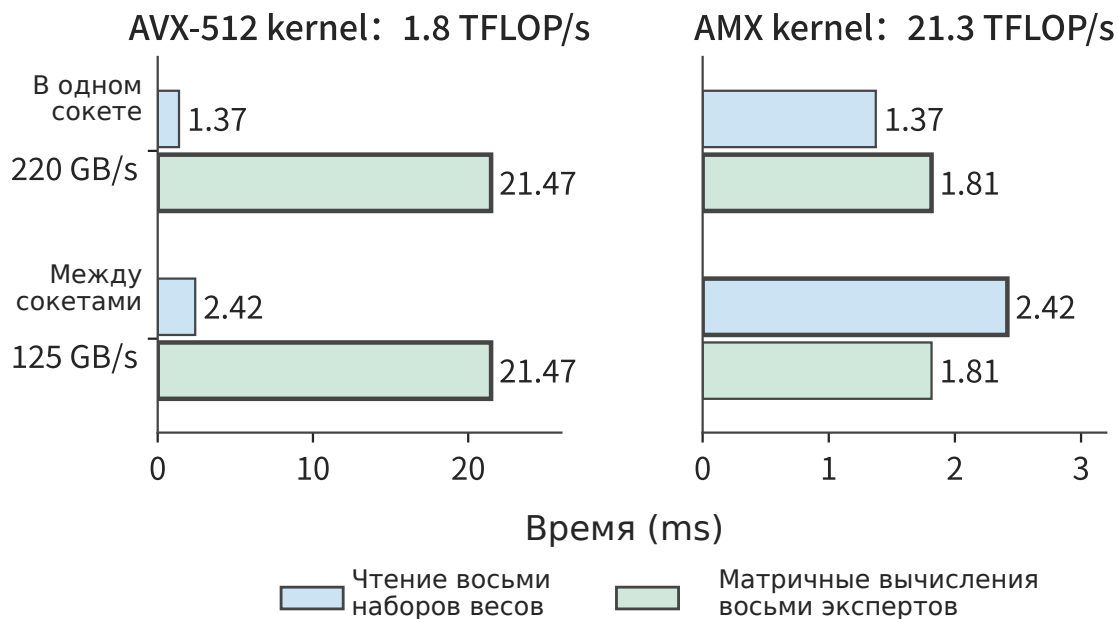


Рис. 9-15. Два компонента времени на одном Xeon Platinum 8452Y при обработке 128 токенов каждым из восьми экспертов Qwen3-235B-A22B: чтение восьми наборов весов BF16 общим размером 288 MiB и матричные вычисления объёмом 38.7 GFLOPs. Толстой рамкой выделен более продолжительный компонент, задающий нижнюю границу времени этого пути. Масштабы горизонтальных осей на левом и правом графиках различаются; вычислительная мощность ядер AVX-512 и AMX, а также пропускная способность памяти внутри сокета и между сокетами взяты из измерений статьи.

Если на GPU постоянно размещать по 32 эксперта каждого слоя, один слой займёт 1.125 GiB, а все 94 слоя — около 106 GiB. Даже если предоставить экспертам все 80 GB видеопамети A100 80GB, то есть около 74.5 GiB, разместить такой набор постоянно находящихся на GPU экспертов не удастся: потребуется распределить их между несколькими картами или сократить число размещённых экспертов на слой. Если уменьшить это число до 16, соответствующий объём весов снизится примерно до 53 GiB, но больше экспертов придётся выполнять на стороне основной памяти. Таким образом, выбор ёмкости напрямую меняет путь выполнения запроса.

То, что один токен выбирает восемь экспертов, не означает, что один batch также обращается лишь к восьми экспертам. В разделе 6.3.2 это распределение уже было рассчитано: 64 токена выбирают по восемь экспертов, что даёт 512 назначений. При равномерном распределении по 128 экспертам каждый эксперт обрабатывает четыре строки, а при концентрации на восьми экспертах — 64 строки. В обоих случаях полезный объём матричных вычислений экспертов равен $512 \times 2 \times 18874368$, то есть около 19.3 GFLOPs. Но если каждый набор весов эксперта считывается в пределах batch только один раз, объёмы чтения весов составляют соответственно 4.5 GiB и 288 MiB — различие в 16 раз. При пропускной способности основной памяти 220 GB/s из примера 9.3 одно только чтение весов занимает соответственно около 22.0 ms и 1.37 ms: объём вычислений одинаков, а время чтения различается на порядок.

Общее число экспертов определяет, сколько весов необходимо разместить, число назначений — полезный объём матричных вычислений текущего batch, а множество экспертов внутри batch — к каким весам потребуется обратиться. На рис. 9-14 уже показано, что число входных строк определяет, какая сторона, CPU или GPU, будет быстрее. Необ-

ходимо учесть и ещё одну величину: к скольким экспертам фактически обратится batch запросов. Только зная это число, результат для одного эксперта можно распространить на весь batch.

Помимо случаев полностью равномерного распределения и полной концентрации можно проанализировать среднее число экспертов, задействованных при случайной маршрутизации. Согласно выводу из раздела 6.3.2, если каждый токен независимо и равномерно выбирает K различных экспертов из E , то математическое ожидание числа активных экспертов для batch из M токенов равно

$$\mathbb{E}[U] = E \left[1 - \left(1 - \frac{K}{E} \right)^M \right].$$

Подставив $E = 128$, $K = 8$, $M = 64$, получим ожидаемое значение около 126 экспертов и идеальный объём чтения весов около 4.43 GiB, что близко к описанному выше случаю равномерного покрытия. При дальнейшем увеличении batch size число различных задействованных экспертов достигает максимума 128 и перестаёт расти, тогда как число назначений продолжает увеличиваться линейно. Поэтому всё больше новых токенов повторно используют уже считанные веса.

Этот анализ также показывает, что серверы с основной памятью большой ёмкости и небольшим числом GPU лучше подходят для нагрузок с малым числом входных строк на эксперта. Для опубликованной конфигурации DeepSeek V4-Flash с одной RTX 5090 требуется не менее 200 GB основной памяти, а для конфигурации Kimi K2 с групповой квантизацией Q4_K_M, преимущественно 4-битной, но с более высокой разрядностью для части тензоров, — около 600 GB. Основная память решает задачу размещения всего набора весов, однако с увеличением batch size запросов узкое место всё равно может сместиться с чтения весов на матричные вычисления CPU. Точка пересечения на рис. 9-14 соответствует именно этому изменению: увеличение параллелизма одновременно амортизирует стоимость чтения весов и постепенно исчерпывает вычислительные возможности CPU.¹¹

9.3.4 Межсерверное AF, разделение экспертов и конвейер микробатчей

В двух предыдущих подразделах задачи всё ещё распределялись между CPU и GPU одного сервера. В MoE маршрутизируемых экспертов также можно вынести в отдельный пул ресурсов, которому воркер, сохраняющий механизм внимания и KV, отправляет активации и от которого получает результаты экспертов. Это называется **разделением экспертов** (Expert Disaggregation), а также часто разделением EP. В этой книге термин «разделение экспертов» обозначает границу развёртывания, а EP — степень параллелизма экспертов. Разделение экспертов представляет собой одну из форм рассматриваемого в этом разделе распределения задач между механизмом внимания и экспертными вычислениями; для конкретной системы также необходимо указать, на какой стороне размещены общие эксперты, маршрутизатор, а также предшествующая и последующая проекции.

На примере двух серверов HGX H100 из главы 7 рассмотрим выполнение одного слоя MoE с разделением экспертов (рис. 9-16). Четыре карты A0–A3 сервера A выполняют ме-

¹¹Записи конфигураций DeepSeek V4-Flash и Kimi K2. Верхние пределы конкурентности в таблице взяты из конфигурации развёртывания.

ханизм внимания и хранят KV соответствующих запросов; четыре карты В0–В3 сервера В содержат по четверти маршрутизируемых экспертов. Внутри слоя последовательно выполняются четыре шага. На первом карты А завершают вычисление механизма внимания, после чего маршрутизатор выбирает для каждого токена восемь экспертов. Второй шаг — описанный в разделе 6.2.6 *dispatch*: карты А группируют скрытые состояния каждого токена по картам, на которых находятся выбранные эксперты, и отправляют их соответствующим картам В. Каждая из четырёх карт А отправляет каждой из четырёх карт В различный объём данных, образуя операцию All-to-All. На третьем шаге карты В получают входные данные от всех источников, переставляют входные строки по экспертам и выполняют матричные умножения экспертов. Четвёртый шаг — *combine*: выходные данные экспертов по обратному маршруту возвращаются на карту А, которой принадлежит токен. Карта А должна получить все восемь результатов экспертов для токена и вычислить их взвешенную по весам маршрутизации сумму, прежде чем перейти к механизму внимания следующего слоя.

На схеме есть две точки ожидания. Обычно карта В должна дожидаться входных данных от всех источников, прежде чем начать вычисления, а карта А — результатов всех экспертов выбранного токена, прежде чем вычислить сумму. Если одна карта В получает больше данных и вычисляет медленнее, она одновременно задерживает собственные вычисления и возврат результатов, а также все карты А, ожидающие эти результаты. Именно через эти две точки ожидания обсуждаемый в разделе 9.4 *skew* влияет на время выполнения всего слоя.

Большой ЕР означает, что одна группа параллелизма экспертов охватывает много карт. Благодаря большому ЕР каждая карта хранит лишь небольшое число экспертов и собирает входные данные от нескольких воркеров механизма внимания, обеспечивая экспертам достаточно большой матричный *batch*. Большой ЕР не обязательно требует отдельного пула экспертов: одна карта может одновременно выполнять механизм внимания и размещённых на ней экспертов. Отдельный пул экспертов также не обязательно состоит только из одной большой группы ЕР. В опубликованном отчёте об инференсе DeepSeek-V3/R1 указаны соответственно ЕР32 для *prefill* и ЕР144 для *decode*, то есть степени параллелизма экспертов 32 и 144, а также используются избыточные эксперты, дополнительные копии экспертов с высокой нагрузкой, и несколько видов балансировки нагрузки. Это подтверждает практическую применимость большого ЕР, но сам по себе такой масштаб не доказывает, что механизм внимания и эксперты распределены между двумя наборами устройств.¹²

Расширение ЕР не увеличивает *batch size* каждого эксперта автоматически. Если текущий *batch* содержит n токенов, каждый токен выбирает k экспертов, а общее число логических экспертов равно E , то при равномерной маршрутизации на одного эксперта в среднем приходится лишь nk/E строк. Например, при $n = 1024$, $k = 8$, $E = 256$ получается 32 строки. Если просто распределить те же 256 экспертов по 128 картам вместо 32, среднее значение останется равным 32, а уменьшится лишь число экспертов на карту. Увеличить *batch* каждого эксперта можно только за счёт объединения большего числа входных данных, однако при этом также расширяются область коммуникации и область синхронизации: две точки ожидания на рис. 9-16 должны охватывать каждую карту группы ЕР. Сокращение числа экспертов на карту также увеличивает межкарточный *skew*; в разделе 9.4.1 на тех же значениях n , k , E рассчитывается его рост при уве-

¹²Инженерный отчёт о системе инференса DeepSeek-V3/R1: большой ЕР, три типа балансировки нагрузки и точность коммуникации; снимок материалов по DeepEP V2. Границы источников и версий.

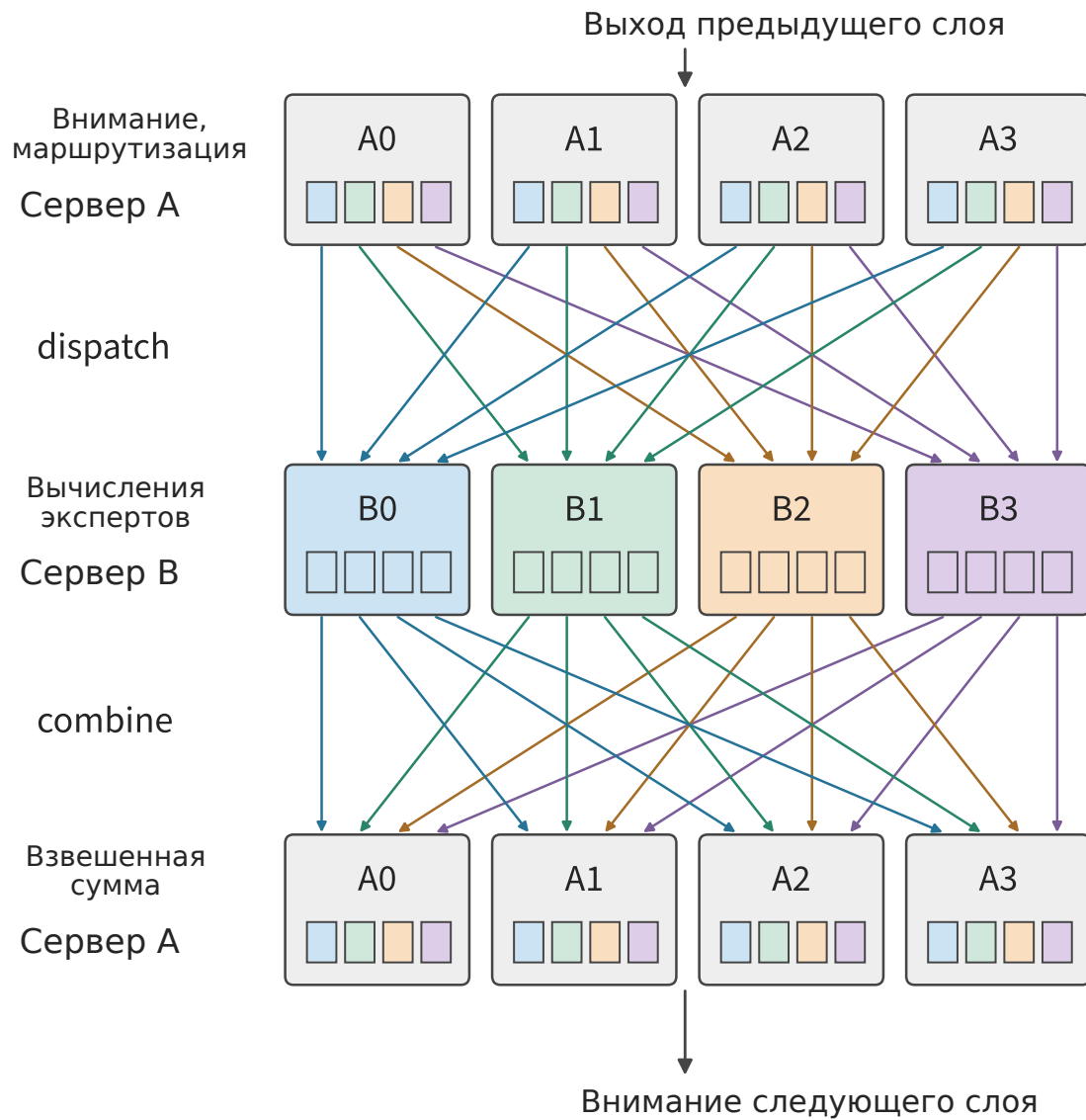


Рис. 9-16. Порядок выполнения одного слоя MoE между двумя серверами; время направлено сверху вниз. A0–A3 выполняют механизм внимания и маршрутизацию, B0–B3 — экспертов. Маленький прямоугольник обозначает группу входных данных, которую одна карта A отправляет одной карте B; цвет показывает целевую карту B. Dispatch доставляет прямоугольники картам B соответствующего цвета, а combine возвращает результаты исходным картам A. Оба обмена представляют собой All-to-All размером 4×4.

личении EP.

Пул экспертов позволяет независимо наращивать число копий, выбирать другое соотношение вычислительной мощности и памяти и собирать задачи от нескольких воркеров механизма внимания. Цена этого решения состоит в том, что входные данные экспертов, которые ранее можно было обрабатывать локально, теперь также приходится передавать между пулами, а сеть становится общим ресурсом обоих пулов. Отдельными репликами сервиса, способными независимо принимать задачи, являются лишь несколько независимых групп EP, каждая из которых хранит полный набор экспертов. Простое добавление карт, содержащих часть экспертов, не позволяет каждому запросу произвольно обходить занятых экспертов. Очередью, видеопамятью, версией модели и политикой допуска общего пула экспертов необходимо управлять как единым целым.

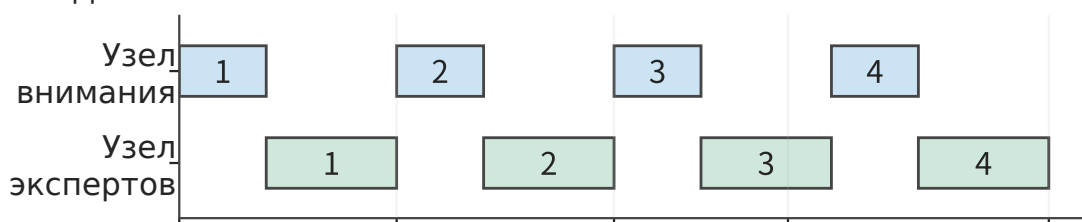
После расширения AF с одного сервера на несколько сильнее проявляется влияние времени запуска каждой передачи, пропускной способности канала и разницы во времени начала выполнения на разных узлах. MegaScale-Infer — распределённая система инференса с разделением вычислений механизма внимания и экспертов. В ней микробатчи чередуются между узлами механизма внимания и узлами экспертов, образуя попеременный конвейер: пока один микробатч выполняет механизм внимания, другой выполняет вычисления экспертов. В идеальном случае, если два этапа требуют для каждого микробатча времени t_A, t_F , то без учёта передачи обработка q независимых микробатчей двухэтапным конвейером занимает $t_A + t_F + (q - 1) \max(t_A, t_F)$. Первый микробатч требует $t_A + t_F$, после чего один микробатч завершается каждый период обслуживания более медленного этапа. При переходе между слоями возвращаемые активации связывают конец предыдущего слоя с началом следующего.

Эксперименты MegaScale-Infer дают практические доказательства эффективности такого развёртывания. В статье 2025 года использовались Mixtral-8×22B, DBRX и Scaled-MoE размером 317B, а веса, активации и KV имели формат BF16. Медианная длина входа и выхода производственных запросов составляла соответственно 571 и 159 токенов. Ограничение оценки требовало TPOT не более 150 ms. В однородном кластере из восьми узлов с восемью GPU Ampere 80 GB на каждом пропускная способность decode на один GPU для Scaled-MoE достигла уровня, в 1.90 раза превышающего показатель фреймворка инференса NVIDIA TensorRT-LLM. Этот результат включает совокупный эффект раздельного размещения, выбора параллелизма, конвейера и коммуникационной библиотеки, поэтому его нельзя считать универсальным ускорением от одного лишь включения разделения EP. В статье также явно указано, что избыточные копии размещаются в соответствии с популярностью экспертов, а стоимость наиболее загруженного узла минимизируется; следовательно, отдельный пул экспертов всё равно должен учитывать skew.¹³

Рассчитаем этот конвейер на конкретном примере: пусть этапы механизма внимания и экспертов занимают для каждого микробатча соответственно 2 ms и 3 ms. Последовательное выполнение четырёх микробатчей занимает 20 ms, причём в каждый момент работает только один узел. Идеальному двухэтапному конвейеру требуется лишь $2 + 3 + 3 \times 3 = 14$ ms, что экономит 6 ms (рис. 9-17). Если после уменьшения микробатчей фактическое время обслуживания каждого этапа увеличится или не перекрываемая вы-

¹³Zhu и др., MegaScale-Infer, arXiv:2504.02263v1: Load balance в §6, условия экспериментов и пропускная способность в §7.1–7.2; CRAFT, MLSys 2026, аннотация и §3–5; Demystifying the Mixture of Experts Serving Tax, MLSys 2026, §3–5. Выдержки из оригиналов, охват прочитанных материалов и условия применимости чисел приведены в записях этого исследования.

Последовательное выполнение: 20 ms



Чередующийся конвейер: 14 ms

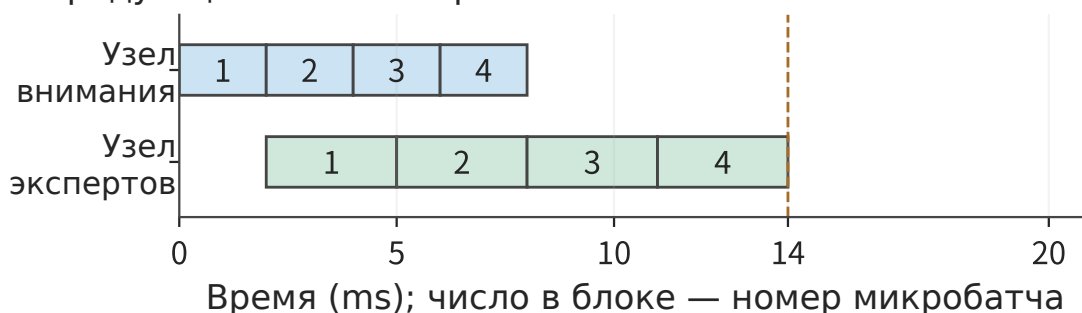


Рис. 9-17. Четыре микробатча выполняются между узлами механизма внимания и экспертов; для каждого микробатча механизм внимания занимает 2 ms, а эксперты — 3 ms, без учёта передачи. На верхней схеме выполнение последовательное, поэтому два узла поочерёдно простаивают. На нижней схеме выполнение чередуется: пока микробатч 1 проходит вычисления экспертов, микробатч 2 выполняет механизм внимания. После стабилизации конвейера узел экспертов работает непрерывно, завершая один микробатч каждые 3 ms.

числениями передача туда и обратно займёт больше 6 ms, преимущество исчезнет. Следовательно, конвейерное выполнение будет быстрее лишь в том случае, если суммарные дополнительные затраты из-за коммуникации и снижения эффективности вычислений останутся меньше 6 ms.¹⁴

Пример 9.4. Насколько послойная передача между механизмом внимания и сетью прямого распространения увеличивает коммуникационные затраты? В примере развёртывания Qwen3-8B все 36 слоёв плотной сети прямого распространения размещены на другой стороне. На каждом слое туда отправляется полный вектор скрытого состояния BF16, а обратно возвращается результат того же размера. Объём одной однонаправленной передачи равен $4096 \times 2 = 8192$ bytes; за один шаг decode выполняется в сумме 72 передачи объёмом 576 KiB. При использовании сетевой карты с пропускной способностью 25 GB/s из примера 9.1 передача полезной нагрузки занимает около 23.6 μ s, а 72 запуска по 5 μ s — в сумме 360 μ s. Последовательная передача занимает около 0.384 ms.

Это время меньше времени одной передачи PD из примера 9.1, но объём соответствующей работы различается: PD выполняет передачу для каждого запроса лишь один раз, тогда как AF делает это на каждом шаге decode. Если prefill для 8192 входных токенов также использует передачу плотного AF для всего batch, объём активаций составляет 4.5 GiB, а одна только передача занимает около 193 ms плюс затраты на запуск. Однократная передача PD растёт вместе с длиной контекста, тогда как послойные передачи туда

¹⁴Межмашинное выполнение и эволюция фреймворков, а также основания для расширенного описания AF.

и обратно в AF дополнительно повторяются с каждым шагом генерации. На первый путь прежде всего влияет перенос большого блока KV, а на второй — затраты на запуск передачи небольших объёмов данных и зависимости между операциями.

Послойная передача AF не зависит от представления KV: передаётся скрытое состояние размерности 4096, не связанное с числом голов KV или шириной латентного представления, поэтому один шаг decode по-прежнему требует 72 передач общим объёмом 576 KiB. Меняется сторона PD. Приравняем последовательное время одной передачи PD ко времени одного шага передачи AF:

$$\frac{V_{KV}}{B} + \alpha = \frac{V_{AF}}{B} + 72\alpha \implies \alpha^* = \frac{V_{KV} - V_{AF}}{71B},$$

где V_{AF} — суммарная полезная нагрузка передачи AF за один шаг, равная 576 KiB, а B — пропускная способность канала. Если время запуска меньше α^* , передача AF за один шаг быстрее одной передачи PD; если оно больше α^* , суммарные затраты 72 запусков уже превышают время передачи всего состояния. Для состояния GQA $\alpha^* \approx 680 \mu\text{s}$ при 25 GB/s и $340 \mu\text{s}$ при 50 GB/s; для компактного состояния MLA — соответственно 324 и $162 \mu\text{s}$. Чем меньше состояние и быстрее канал, тем меньше допустимый бюджет каждого запуска. Время запуска $5 \mu\text{s}$ намного меньше всех четырёх критических значений, поэтому, если рассматривать передачу за один шаг, AF во всех случаях оказывается быстрее. Однако для запроса целиком вывод иной: суммарное время передач AF за 1024 шага составляет около 393 ms — в 8.1 раза больше одной передачи PD для GQA длительностью 48.3 ms и в 17 раз больше одной передачи PD для компактного MLA длительностью 23.0 ms. PD выполняет передачу лишь один раз, и MLA вдвое сокращает её стоимость; AF выполняет передачу на каждом шаге, а ширина скрытого состояния остаётся неизменной, поэтому ни одного байта не экономится. Чем длиннее выход, тем больше разница.

В следующем разделе рассматривается передача активаций в МоЕ: каждый вход может отправляться в несколько мест выполнения, а перед объединением необходимо дождаться результатов от всех выбранных им экспертов.

9.4 Размещение и репликация экспертов, балансировка нагрузки

9.4.1 Перекос вычислений и коммуникаций при большом EP

В предыдущем разделе централизованное повторное использование уменьшило объём чтения весов. Однако если все эти горячие эксперты размещены на одной карте, остальные карты не могут разделить нагрузку. В этом разделе мы продолжим отслеживать те же 512 назначений и рассмотрим, когда каждая карта завершает вычисления. Даже если общий объём вычислений во всей системе не меняется, самая загруженная карта может задержать завершение всего слоя. Для карты r обозначим объём матричных вычислений через F_r , объём считываемых весов в байтах — через W_r , а эффективную вычислительную мощность и пропускную способность — через C_r и B_r соответственно. Время, необходимое этой карте для завершения вычислений и чтения весов, составляет не менее $\max(F_r/C_r, W_r/B_r)$; при объединении результатов всех карт необходимо ждать завершения самой медленной.

Если распределить 512 назначений из предыдущего раздела по восьми картам, проявится другая сторона централизованной маршрутизации. Если каждая карта получает

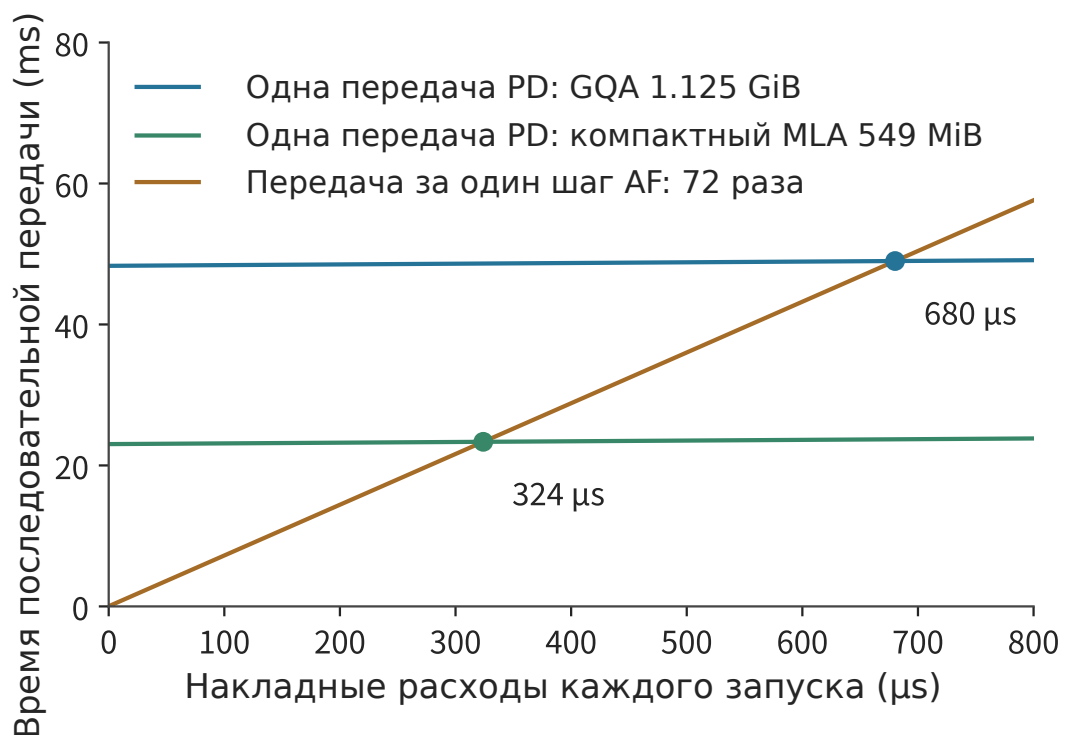


Рис. 9-18. Зависимость последовательного времени одной передачи PD и одного шага передачи AF от затрат на каждый запуск при канале 25 GB/s. PD запускается лишь один раз, поэтому наклон равен 1; один шаг AF требует 72 запусков, поэтому наклон равен 72. Две линии PD соответствуют состоянию GQA размером 1.125 GiB и компактному состоянию MLA размером 549 MiB; точки их пересечения с линией AF задают критическое время запуска 680 и 324 μs . Для канала 50 GB/s точки пересечения смещаются к 340 и 162 μs .

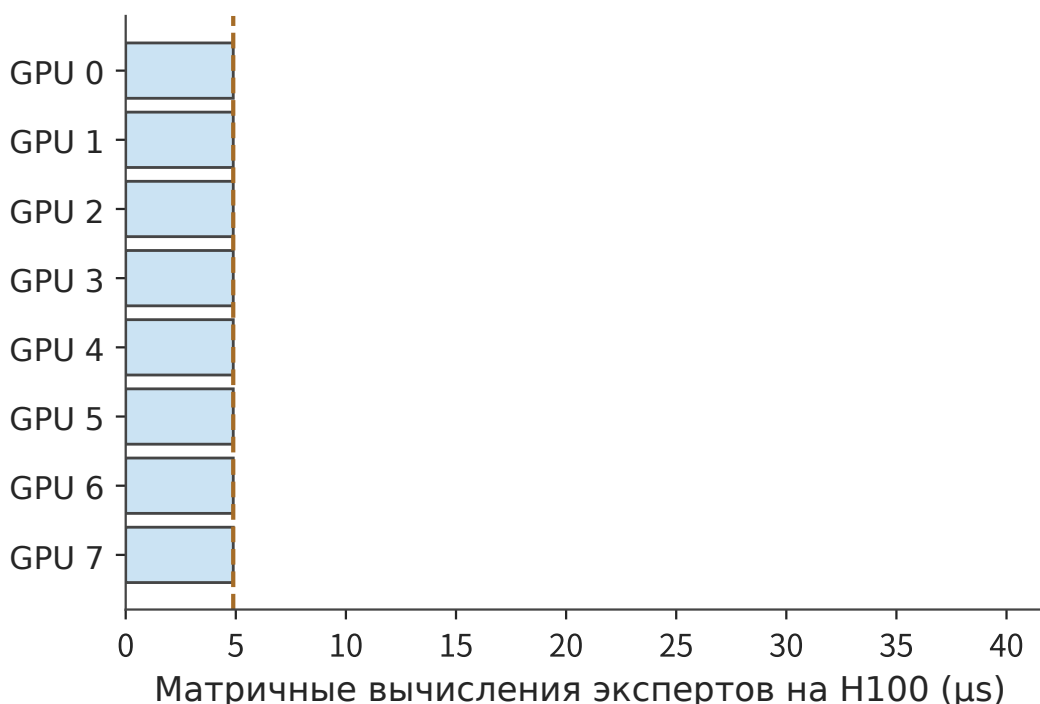


Рис. 9-19. В общей сложности 512 назначений экспертов равномерно распределены по восьми картам, по 64 назначения на карту; пунктирная линия обозначает момент, когда все вычисления завершены и результаты можно объединить.

ровно 64 задачи, она выполняет примерно 2,42 GFLOPs; если все восемь горячих экспертов находятся на одной карте, эта карта выполняет все примерно 19,3 GFLOPs. Возьмём сервер HGX H100 из главы 7 с восемью картами и для каждой H100 SXM примем 50% от пиковой производительности плотных вычислений BF16, равной 989,4 TFLOP/s, то есть 494,7 TFLOP/s. Если учитывать только вычисления, время обслуживания самой загруженной карты увеличивается примерно с 4,9 μ s до 39,1 μ s, то есть в восемь раз.

На рис. 9-19 длина каждой полосы показывает время, необходимое соответствующей карте для завершения вычислений. Перейти к следующему слою можно лишь после объединения всех результатов, поэтому момент завершения определяется самой длинной полосой, а не средним значением восьми полос.

Централизованная маршрутизация уменьшает объём считываемых весов с 4,5 GiB до 288 MiB, но может увеличить вычислительную нагрузку самой загруженной карты в восемь раз. Первое полезно при выполнении, ограниченном чтением, второе вредно при выполнении, ограниченном вычислениями. Поэтому стратегия балансировки должна сначала определить узкое место и лишь затем выбирать между повторным использованием и распределением работы.

Даже при неизменном общем объёме работы большой EP усиливает такое ожидание. Возьмём параметры из раздела 9.3.4: $n = 1024$, $k = 8$, $E = 256$ — в каждом слое DeepSeek-V3 также есть 256 маршрутизируемых экспертов, а для каждого токена выбираются 8. Всего в пакете 8192 назначения, в среднем по 32 строки на эксперта. Предположим, что один эксперт горячий и получает 128 строк, то есть в четыре раза больше среднего; оставшиеся 8064 строки равномерно распределяются между другими 255



Рис. 9-20. Все 512 назначений попадают на карту 0, остальные карты простаивают. При том же общем объёме вычислений приходится ждать завершения карты 0; в обоих рисунках используется одинаковая шкала времени.

экспертами, примерно по 31,6 строки на каждого. Если равномерно распределить 256 экспертов по номерам между картами группы EP, то карта с горячим экспертом должна обработать его 128 строк, а также строки остальных экспертов на этой карте. При EP8 на каждой карте размещено 32 эксперта, поэтому горячая карта обрабатывает около 1108 строк — лишь на 8% больше среднего значения 1024 строки на карту. При EP32 на каждой карте размещено 8 экспертов, поэтому горячая карта обрабатывает около 349 строк — на 36% больше среднего значения 256 строк. При EP256 на каждой карте остаётся лишь один эксперт, поэтому горячая карта обрабатывает 128 строк — в четыре раза больше среднего значения 32 строки. Один и тот же горячий эксперт при малом EP теряется на фоне других экспертов той же карты, а при большом EP единолично занимает карту, из-за чего дисбаланс между экспертами без изменений превращается в дисбаланс между картами (рис. 9-21).

Число строк на карте одновременно определяет три вида работы: объём байтов, принимаемых на этапе *dispatch*, объём байтов, отправляемых на этапе *combine*, и время матричных вычислений эксперта, когда они ограничены вычислительной мощностью. При 8 KiB на строку и сетевой пропускной способности 50 GB/s для каждой карты горячая карта в EP256 должна принять 1 MiB, что занимает около 21 μs , тогда как средняя карта принимает лишь 256 KiB примерно за 5,2 μs . Если *batch* мал и вычисления эксперта ограничены чтением весов, время матричных вычислений на карте в основном зависит от количества наборов весов экспертов, которые нужно прочесть, и слабо связано с числом строк, однако объём байтов при обоих обменах всё равно пропорционален числу строк. При переходе от EP8 к EP256 среднее число строк на карту уменьшается в 32 раза,

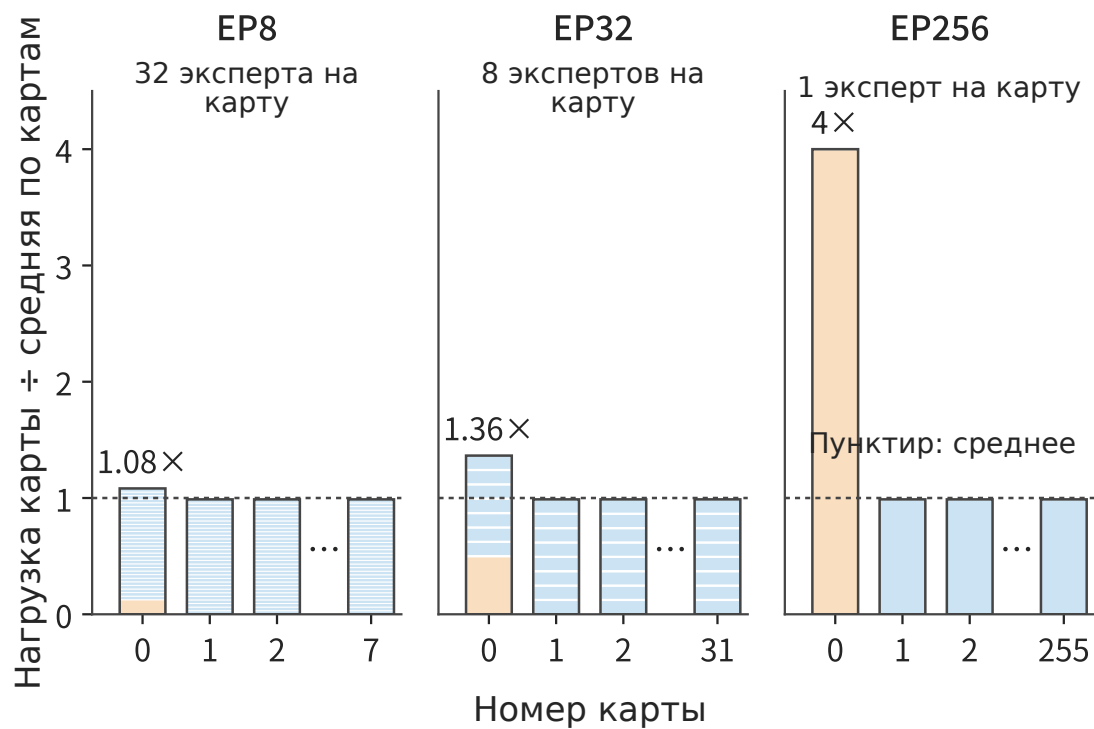


Рис. 9-21. Перекос нагрузки между картами, создаваемый одним горячим экспертом при трёх масштабах EP. Из 256 экспертов каждый из 1024 токенов выбирает 8, а эксперт 0 получает 128 строк — в четыре раза больше среднего. Каждый столбец соответствует одной карте, каждый сегмент столбца — одному эксперту на этой карте, оранжевым обозначен горячий эксперт; по вертикальной оси отложено отношение числа строк на карте к среднему числу строк на карту. На каждом графике показаны только карты 0, 1, 2 и последняя карта; остальные карты эквивалентны карте 1.

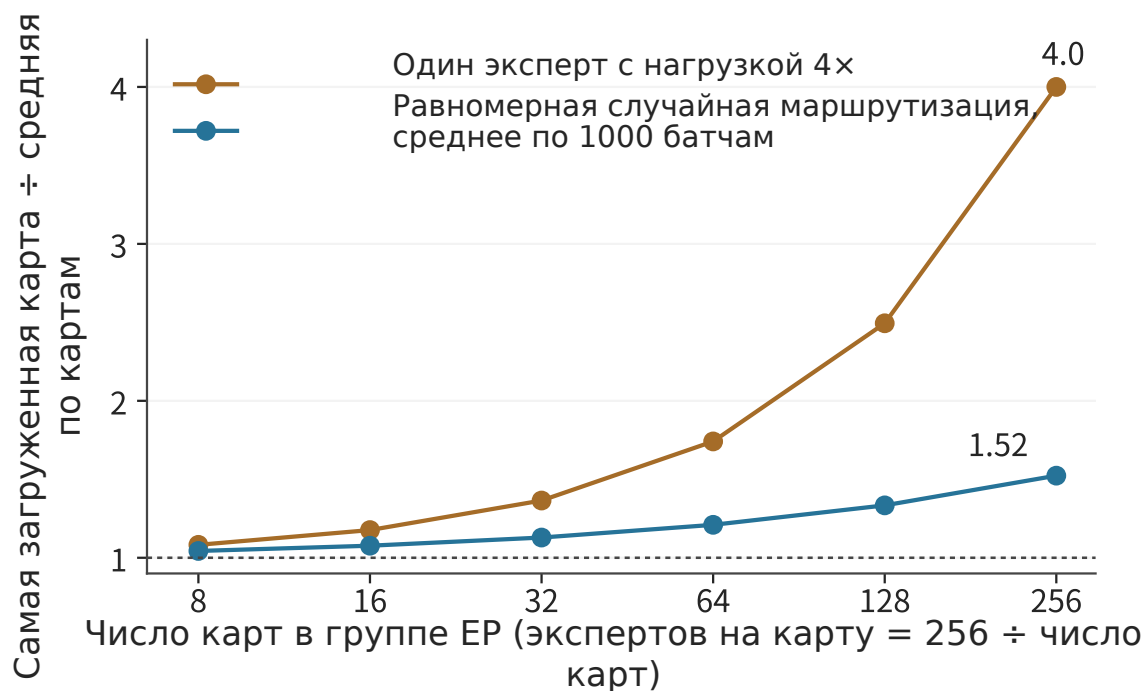


Рис. 9-22. Зависимость отношения числа строк на самой загруженной карте к среднему числу строк на карту от количества карт в группе EP. Оранжевая линия соответствует одному горячему эксперту с четырёхкратной нагрузкой без случайных колебаний и вычисляется вручную; синяя линия соответствует равномерной случайной маршрутизации и показывает среднее по 1000 пакетам при фиксированном seed. 256 экспертов равномерно распределены по номерам, реплики отсутствуют; горизонтальная ось логарифмическая.

тогда как число строк на самой загруженной карте снижается лишь с 1108 до 128, примерно в 8,7 раза. Время всего слоя определяется самой загруженной картой, поэтому в EP256 средняя карта три четверти времени этого этапа проводит в ожидании.

Даже без горячих экспертов увеличение EP также усиливает перекося. Предположим, что каждый токен независимо и равномерно случайно выбирает 8 экспертов; тогда число строк у каждого эксперта всё равно будет случайно колебаться около 32. Чем больше экспертов размещено на одной карте, тем лучше эти колебания компенсируют друг друга; чем больше карт, тем выше вероятность появления в одном пакете особенно загруженной карты. При моделировании 1000 пакетов с фиксированным seed самая загруженная карта в EP8 в среднем превышает среднюю нагрузку на карту лишь на 4%, а в EP256 — на 52% (рис. 9-22). Обе причины сводятся к одному: чем больше EP, тем меньше экспертов на каждой карте могут компенсировать колебания друг друга и тем больше карт приходится ждать. В опубликованной конфигурации DeepSeek для decode используется EP144, причём на каждой карте размещено лишь 2 маршрутизируемых эксперта, что соответствует области около правого края кривой. Поэтому система добавляет 32 избыточных эксперта и корректирует реплики по нагрузке — именно этот приём рассматривается в разделе 9.4.2.¹⁵

Объём коммуникаций также зависит от того, на какой карте изначально находился

¹⁵ Фиксированные входные данные для масштаба EP и нагрузки наиболее занятой GPU, скрипт расчёта, результат повторного расчёта. Для сценария с горячими точками используются точные доли, а для случайного сценария зафиксирован seed; учитываются только фактические строки, без padding, чтения весов и коммуникации. EP144 при развёртывании decode DeepSeek, 32 резервных эксперта и 2 маршрутизируемых эксперта на GPU описаны в инженерном отчёте о системе инференса DeepSeek-V3/R1.

токен. В конфигурации TR2×EP4 из раздела 9.1 каждая группа EP обрабатывает один и тот же пакет запросов, а участники каждой группы EP уже содержат одинаковые входные скрытые состояния, поэтому dispatch между группами EP может отсутствовать, а результаты объединяются редукцией частичных сумм. В другой конфигурации каждая карта содержит собственные входные токены, поэтому активации необходимо отправлять удалённым экспертам через dispatch, как на рис. 9-16. Если входы уже реплицированы, коммуникации сосредоточены на объединении результатов; если входы распределены между разными картами, перед выполнением добавляется удалённый dispatch. Объём коммуникаций совместно определяется исходным расположением токенов, расположением экспертов и местом назначения результатов.

Перекося при большом EP необходимо рассматривать как минимум на трёх уровнях. Частота выбора логических экспертов определяет эффективное число строк каждого эксперта; размещение экспертов и выбор реплик преобразуют эти строки в нагрузку на каждую карту; источник токенов, назначение и сетевой путь затем преобразуют нагрузку карты в трафик каждого направления. Неравномерная востребованность экспертов не обязательно приводит к неравномерности между картами, а равномерные вычисления на картах не обязательно обеспечивают равномерную нагрузку на общие каналы. Повторное использование внутри пакета из раздела 6.3.2 объясняет чтение весов, а здесь мы продолжим рассчитывать вычисления, коммуникации и время завершения для каждой карты.

Пусть a_{ij} — число назначений «токен — эксперт», отправляемых источником i группе экспертов j , а размеры каждой входной и выходной порции данных равны соответственно d_D, d_C байт. Для схемы с передачей по назначениям, без устранения дубликатов и предварительной редукции:

$$V_{ij}^D = a_{ij}d_D, \quad V_{ji}^C = a_{ij}d_C, \quad m_j = \sum_i a_{ij}, \quad s_{\text{expert}} = \frac{\max_j m_j}{nk/R}.$$

Здесь R — число принимающих групп экспертов, а $\sum_j m_j = nk$. Значение $s_{\text{expert}} = 1$ означает баланс эффективных задач между группами; лишь для однотипных экспертов с одинаковой точностью и без учёта padding оно приблизительно равно перекося вычислительной нагрузки. Для коммуникационного этапа $X \in \{D, C\}$, если учитывать матрицу по отправителям, получателям и физическим сечениям сети, получим:

$$T_X \geq \max \left(\max_i \frac{\sum_j V_{ij}^X}{B_i^{\text{send}}}, \max_j \frac{\sum_i V_{ij}^X}{B_j^{\text{recv}}}, \max_c \frac{V_c^X}{B_c} \right).$$

Здесь V_c^X и B_c — соответственно нагрузка, проходящая через физическое сечение c , и пропускная способность его каналов. Это нижняя граница времени передачи нагрузки, не учитывающая запуск, очереди и позднее поступление данных. Параллельный трафик одного физического направления необходимо суммировать: нельзя делить каждый поток на полную пропускную способность и предполагать, что все потоки завершатся одновременно. Под перекося пропускной способности здесь понимается неравномерность спроса или использования разных карт и каналов; номинальная пропускная способность оборудования не меняется.

Пример 9.5. Почему при одинаковом общем объёме коммуникаций горячая точка замедляет оба обмена? Возьмём из раздела 7.2.3 1024 токена, выбор 8 экспертов

для каждого токена и вектор размером 8 KiB и разместим их на двух серверах HGX H100 с рис. 9-16. Четыре источника — это четыре карты сервера А, выполняющие attention, а каждая из четырёх групп экспертов выполняется одной картой сервера В; все назначения проходят между серверами. Каждая карта отправляет и принимает данные через собственную сетевую карту ConnectX-7 400 Gbit/s с пропускной способностью 50 GB/s в каждом направлении; коммутируемая сеть между двумя серверами обеспечивает $4 \times 50 = 200$ GB/s в каждом направлении. Каждый из четырёх источников отправляет 2048 порций, общий объём dispatch составляет 64 MiB, как и объём combine. При равномерном распределении каждая группа принимает 2048 порций, то есть 16 MiB; при горячем распределении [5120, 1024, 1024, 1024] группа 0 принимает 40 MiB, а остальные три — по 8 MiB. Общий объём коммуникаций и общий объём полезных вычислений у двух распределений одинаков, но перекоз экспертов увеличивается с 1 до 2,5.

Для каждого назначения объём вычислений эксперта примем равным $6 \times 4096 \times 1536$ FLOPs, как в примере Qwen3 MoE, а эффективную вычислительную мощность каждой H100 — 494,7 TFLOP/s. Пока не учитывая чтение весов, padding, запуск и очереди, получаем следующую таблицу. Последний столбец применим только к выполнению с барьером всего пакета между тремя этапами и равен сумме трёх нижних границ; при перекрытии необходимо отдельно рассчитывать критический путь.¹⁶

Распределение и путь	Нижняя граница dispatch	Нижняя граница вычислений самой загруженной карты	Нижняя граница combine	Общая нижняя граница с барьерами между этапами
Четыре равномерные группы, между серверами	0,336 ms	0,156 ms	0,336 ms	0,827 ms
Горячая группа, между серверами	0,839 ms	0,391 ms	0,839 ms	2,068 ms
Четыре равномерные группы, между двумя пулами остался лишь один канал 400 Gbit/s	1,342 ms	0,156 ms	1,342 ms	2,841 ms
Горячая группа, dispatch переведён на FP8	0,419 ms	0,391 ms	0,839 ms	1,649 ms
Горячая группа, оба пула находятся в одном HGX (NVLink)	0,093 ms	0,391 ms	0,093 ms	0,577 ms

При обмене между серверами два коммуникационных этапа составляют 80% общей нижней границы с барьерами между этапами, а вычисления самой загруженной карты — лишь 20%: вычисление одного назначения эксперта на H100 занимает около 76 ns, тогда как передача одной активации размером 8 KiB через сетевую карту — около 164 ns.

¹⁶Фиксированные входные данные для большого EP и перекоза при разделении экспертов, скрипт расчёта, результат повторного расчёта. Пропускная способность сетевых адаптеров и NVLink приведена в спецификации HGX H100, спецификации NVIDIA H100 и спецификации ConnectX-7; производительность H100 принята равной 50% от пиковой величины из таблицы оборудования. Результат вычислений является нижней границей для приведённой модели выполнения; конвейер для двух micro-batch и коэффициент достижения равенства из раздела 9.4.3 также получены из этого результата.

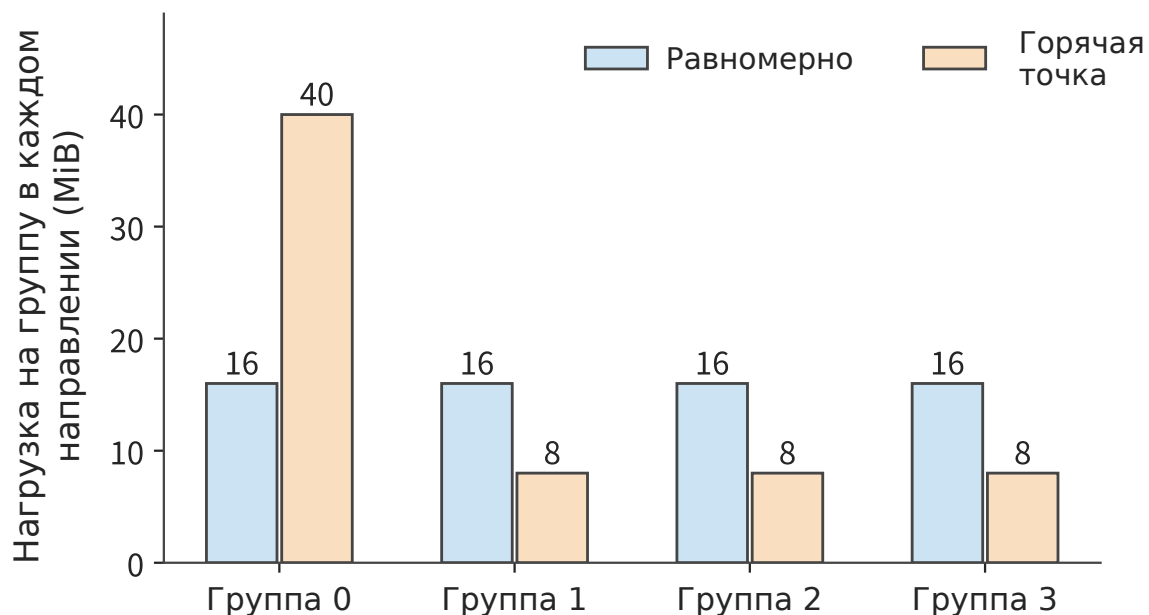


Рис. 9-23. Нагрузка, которую четыре группы экспертов принимают на этапе dispatch и отправляют на этапе combine. Общий объём в каждом направлении при равномерном и горячем распределении равен 64 MiB; горячая группа в обоих этапах обрабатывает 40 MiB. Высота столбца обозначает требуемое число байтов, а не измеренную мгновенную пропускную способность.

Горячая точка одновременно удлиняет приём, вычисление и возврат, и каждый из трёх этапов становится в 2,5 раза дольше. Если распределить горячую нагрузку, но оставить между двумя пулами лишь один канал 400 Gbit/s, в каждом направлении всё равно потребуется не менее 1,342 ms. Это показывает, что добавление карт экспертов не заменяет пропускную способность между пулами. Перевод только dispatch на FP8 также не обеспечивает пропорционального сокращения BF16 combine или вычислений экспертов; в таблице предполагается, что квантование не меняет время вычислений, а метаданные масштаба игнорируются, чтобы отдельно рассмотреть влияние каждого фактора. Если разместить оба пула в одном HGX, NVLink с пропускной способностью 450 GB/s в каждом направлении сокращает оба обмена до 0,093 ms, и самым длинным этапом становятся вычисления горячей карты длительностью 0,391 ms. В этом случае уменьшение вычислительной нагрузки горячей карты эффективнее дальнейшего увеличения пропускной способности; именно на такую ситуацию рассчитаны реплики экспертов из следующего раздела.

9.4.2 Реплики экспертов, размещение и динамическая корректировка

Сосредоточение работы на нескольких картах на рис. 9-20 указывает направление улучшения: разместить реплики горячих экспертов на свободных картах, чтобы они также могли обрабатывать горячие задачи. Репликация эксперта создаёт для одного логического эксперта несколько физических реплик, между которыми планировщик может распределять задачи. Если веса реплик одинаковы, веса маршрутизации применяются правильно, а результаты объединяются без ошибок, репликация не требует изменять

правило выбора для каждого токена k экспертов с максимальными оценками (top- k). Изменение правила выбора экспертов меняет поведение модели, поэтому эти два подхода следует оценивать раздельно. Механизмы балансировки нагрузки экспертов (EPLB) в vLLM и SGLang корректируют расположение реплик логических экспертов и распределение задач на основе статистики маршрутизации.

Реплики экспертов занимают видеопамять, вытесняя KV и буферы. Добавление на каждую карту одного эксперта размером 36 MiB для каждого из 94 слоёв требует в сумме 3384 MiB, то есть около 3,30 GiB. Если добавить по одной реплике только для восьми наиболее перегруженных слоёв, потребуется лишь 288 MiB. Эти объёмы различаются почти в 12 раз. CRAFT — исследовательская система обслуживания, посвящённая репликации экспертов и распределению видеопамяти. Она решает именно задачу распределения памяти между слоями: пространство следует выделять слоям, где оно сильнее всего сокращает ожидание, а не механически создавать одинаковое число реплик в каждом слое.¹⁷

Пример 9.6. Сколько пакетов требуется реплике горячего эксперта, чтобы окупить стоимость копирования? Восемь карт образуют один сервер HGX H100. Все 128 токенов пакета выбирают восемь горячих экспертов на карте 0, что даёт в общей сложности 1024 назначения. Скопируем по одной реплике семи из этих экспертов на карты с 1 по 7, всего 252 MiB. Каждой из семи принимающих карт требуется дополнительно 36 MiB, что помещается в доступные 64 MiB. Для каждой H100 примем эффективную вычислительную мощность 494,7 TFLOP/s, равную 50% пиковой, и пропускную способность видеопамяти 1675 GB/s. Карта 0 использует планирование по tile и для каждого пакета читает и записывает в видеопамять около 921 MB, поэтому ограничена её пропускной способностью. После репликации на карте 0 остаётся лишь 576 назначений, и время на пакет сокращается примерно с 0,550 ms до 0,309 ms, то есть экономится около 0,240 ms. Все семь реплик последовательно отправляются с карты 0 через NVLink с пропускной способностью 450 GB/s в каждом направлении; на запуск каждой передачи дополнительно отводится 5 μ s, поэтому копирование занимает около 0,622 ms. При использовании неокруглённого времени выполнения минимальное число пакетов, после которого экономия превышает время копирования, удовлетворяет условию:

$$N\Delta t > T_{copy}, \quad N_{min} = \left\lceil \frac{T_{copy}}{\Delta t} \right\rceil + 1 = 3.$$

В пределах одного сервера стоимость копирования реплик окупается на третьем пакете: если горячая нагрузка сохраняется 16 пакетов, чистая экономия составляет около 3,2 ms, а если 64 пакета — около 14,8 ms. Если исходная реплика горячего эксперта находится на другом сервере, копирование проходит через сетевую карту ConnectX-7 400 Gbit/s с пропускной способностью 50 GB/s в каждом направлении. Время подготовки увеличивается примерно до 5,32 ms, и затраты окупаются лишь на 23-м пакете: если горячая нагрузка сохраняется только 16 пакетов, чистая потеря составляет около 1,5 ms, а при 64 пакетах чистая экономия достигает примерно 10,1 ms. На рис. 9-24 единовременные затраты и выгода от каждого пакета показаны на одном графике. Если на принимающей карте свободно лишь 32 MiB, она не вместит даже одного эксперта размером 36 MiB. Тогда независимо от длительности горячей нагрузки сначала необходимо изменить размещение или освободить память.¹⁸

¹⁷Исследовательские заметки о MoE Serving Tax, CRAFT и запуске.

¹⁸Расчёты окупаемости копирования внутри одного HGX через NVLink и копирования между серверами через ConnectX-7. В примерах зафиксированы маршрутизация и дополнительное доступное простран-

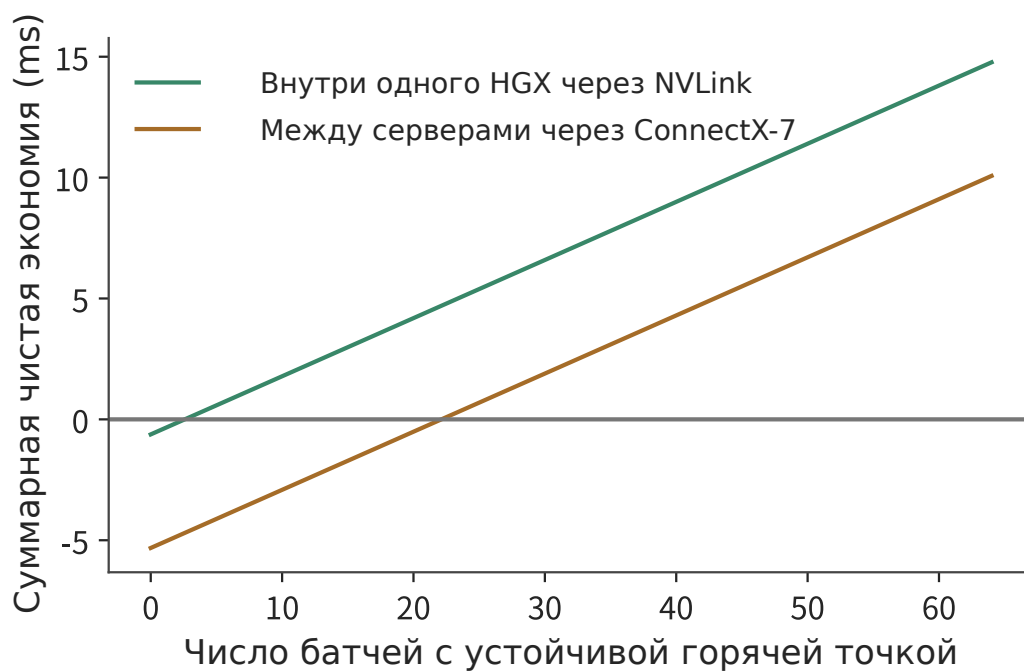


Рис. 9-24. Как долго должна сохраняться горячая нагрузка, чтобы репликация экспертов была оправданна. Каждый пакет экономит около 0,240 ms; однократное копирование внутри одного HGX H100 через NVLink занимает около 0,622 ms и начинает давать чистую выгоду с третьего пакета; копирование между серверами через сетевую карту ConnectX-7 занимает около 5,32 ms и начинает давать чистую выгоду с 23-го пакета. Кривые рассчитаны по неокруглённым значениям времени из примера 9.6; каждой из семи принимающих карт дополнительно требуется 36 MiB, а горячая нагрузка предполагается неизменной.

Измерения CRAFT также показывают, что выгода репликации зависит от слоя. В этом исследовании на основе SGLang v0.4.8 модели DeepSeek-R1 и Kimi K2 в BF16 запускались на кластере A100 80GB из восьми узлов AWS p4de.24xlarge. Распределение реплик экспертов по слоям в пределах бюджета видеопамати сравнивалось с существующими стратегиями репликации. Согласно статье, сквозная пропускная способность в среднем достигала 1,14-кратного контрольного значения, а максимум — 1,2-кратного. Это показывает, что бюджет реплик следует в первую очередь выделять слоям с высокой выгодой, но не означает, что произвольная репликация горячих экспертов даст такой же прирост. Экспериментальные условия, включая разбиение входа на блоки по 4096 токенов и фиксированную длину выхода 256 токенов, а также точку перегиба ёмкости, определённую через полезную пропускную способность (goodput), нельзя напрямую подставлять вместо производственных показателей р99.¹⁹

Стратегия репликации должна предсказывать, как долго сохранится горячая нагрузка. При копировании между серверами каждое новое размещение реплик занимает 5,32 ms, поэтому частая миграция реплик вслед за горячей нагрузкой откладывает окупаемость. Внутри одного сервера копирование занимает лишь 0,622 ms, поэтому время почти не является ограничением; число реплик ограничивается объёмом видеопамати, который можно освободить на каждой карте. Выделение памяти прежде всего постоянно перегруженным слоям позволяет накопленной экономии времени ожидания превысить затраты на подготовку.

Стратегия балансировки также должна выбирать объект управления с учётом полного пути. На стороне источников нужно балансировать число токенов и время готовности attention; на стороне экспертов — число эффективных строк, фактическое время GEMM и объём принимаемых данных; на стороне сети — проверять общие исходящие каналы, межстоечный трафик и обратное направление. Именно поэтому в опубликованном отчёте DeepSeek балансировка prefill, decode и нагрузки EP рассматривается отдельно. При decode с длинным контекстом даже при одинаковом числе запросов на карту объём чтения KV для attention может различаться, что приводит к разным моментам запуска dispatch.²⁰

В независимом пуле экспертов также необходимо контролировать очереди от разных источников. Несколько attention worker могут одновременно выбрать одного горячего эксперта. Даже если их собственные небольшие batch выглядят сбалансированными, объединённый поток может превысить пропускную способность обслуживания этого эксперта. Если пул принимает λ пакетов в секунду, а каждый пакет в среднем создаёт на карте экспертов j вычислительную нагрузку $\mathbb{E}[F_j]$ FLOPs, то для стабильной работы как минимум необходимо $\lambda \mathbb{E}[F_j] < C_j$; для каждого канала ℓ также должно выпол-

ство на каждой GPU, а время подготовки рассчитано для последовательного копирования. Значения 989.4 TFLOP/s и 3350 GB/s для H100 SXM приведены в таблице оборудования и используются с эффективностью 50%; пропускная способность NVLink в каждом направлении 450 GB/s приведена в спецификации NVIDIA H100, а пропускная способность сетевого адаптера в каждом направлении 50 GB/s — в спецификации ConnectX-7. Время подготовки составляет соответственно 0.62220256 ms и 5.31982304 ms, экономия на один батч — 402784256/1675 ns, то есть примерно 0.2404682 ms; минимальные количества батчей — 3 и 23 — вычислены по значениям без округления.

¹⁹Zhu и др., MegaScale-Infer, arXiv:2504.02263v1: Load balance в §6, условия экспериментов и пропускная способность в §7.1–7.2; CRAFT, MLSys 2026, аннотация и §3–5; Demystifying the Mixture of Experts Serving Tax, MLSys 2026, §3–5. Выдержки из оригиналов, охват прочитанных материалов и условия применимости чисел приведены в записях этого исследования.

²⁰Инженерный отчёт о системе инференса DeepSeek-V3/R1: большой EP, три типа балансировки нагрузки и точность коммуникации; снимок материалов по DeepEP V2. Границы источников и версий.

няться $\lambda \mathbb{E}[V_\ell] < B_\ell$. Соответствие средней нагрузки этим условиям лишь необходимо: всплески и входные данные с тяжёлыми хвостами всё равно могут создавать длинные очереди. Общий пул иногда позволяет объединять batch и сглаживать колебания разных источников, а иногда усиливает коррелированные горячие точки, поэтому нельзя автоматически считать, что разделение устранит перекос.

Реплики горячих экспертов следует размещать там, где одновременно имеется запас вычислительной мощности и пропускной способности каналов. Сообщение dispatch в этом разделе представляет собой вектор скрытого состояния одного токена: 8 KiB для BF16 и 4 KiB для FP8. Это значительно больше точки пересечения около 0,93 KB, рассчитанной в разделе 7.2.4 для сетевой карты 50 GB/s. Поэтому горячая карта сначала исчерпывает пропускную способность, а не скорость запуска запросов сетевой картой. Запас по способности сетевой карты запускать около 54 миллионов запросов в секунду необходимо учитывать только при поштучной отправке управляющих сообщений размером менее 1 KB. При динамическом выборе реплик необходимо учитывать очередь и путь, а также сохранять используемое текущим batch отображение маршрутизации до завершения combine; перед миграцией или удалением старой реплики необходимо дождаться завершения задач в пути. Квоты приёма для горячих точек и ограничение числа операций в пути от одного источника предотвращают исчерпание буфера всего пула. При увеличении ёмкости пула необходимо соответствующим образом корректировать квоты приёма и обратное давление. Отбрасывание токенов при переполнении, изменение top-k или замена логического эксперта меняют поведение модели и не могут считаться прозрачной балансировкой ресурсов.

9.4.3 Перекрытие распределения входов, вычислений и объединения результатов

Расчёты в двух предыдущих подразделах не различали prefill и decode, хотя концентрация назначений влияет на эти этапы по-разному. В работе «Demystifying the Mixture of Experts Serving Tax» это различие объясняется микробенчмарками каждого этапа: вычислительный дисбаланс при prefill заставляет весь пакет ждать самую медленную карту, тогда как концентрация экспертов при decode может уменьшать затраты на доступ к памяти благодаря сокращению объёма активных весов и padding. Это согласуется с анализом повторного использования в разделе 6.3.2. В статье Mixtral и Qwen2 MoE выполняются на восьми A100, DeepSeek-V3 — на восьми B200, а для коммуникационных микробенчмарков отдельно используются 8 или 16 H200. Эти результаты нельзя объединять в единый показатель сквозного ускорения между кластерами. Представленные в статье данные показывают, что маршрутизацию, фактическую форму матриц, коммуникации и доступ к памяти необходимо измерять совместно.²¹

Влияние фактической формы матриц главным образом обусловлено дополнением. Grouped GEMM объединяет матричные умножения нескольких экспертов в одном запуске kernel, причём строки каждой матрицы дополняются до размера tile. Например, восемь локальных экспертов получают [32, 16, 8, 4, 2, 1, 1, 0] токенов, то есть всего 64 значения «токен — эксперт», соответствующие 64 полезным строкам входной матрицы. Если каждый непустой эксперт дополняется до 32 строк, потребуется 224 строки; если

²¹Zhu и др., MegaScale-Infer, arXiv:2504.02263v1: Load balance в §6, условия экспериментов и пропускная способность в §7.1–7.2; CRAFT, MLSys 2026, аннотация и §3–5; Demystifying the Mixture of Experts Serving Tax, MLSys 2026, §3–5. Выдержки из оригиналов, охват прочитанных материалов и условия применимости чисел приведены в записях этого исследования.

все восемь экспертов дополняются до максимального размера, потребуется 256 строк. Фактически выполняется соответственно в 3,5 и 4 раза больше строк, чем полезных. Это объясняет, почему при одинаковом числе полезных задач время матричных вычислений всё равно может различаться: оборудование обрабатывает дополненные матрицы, тогда как требуемым моделью вычислениям экспертов соответствуют только 64 полезные строки; дополненные нулевые строки не связаны с реальными токенами.

Дополнение влияет только на вычислительную составляющую; общая длительность трёх этапов также зависит от возможности их перекрытия. Возьмём равномерное распределение между серверами из раздела 9.4.1: нижние границы времени dispatch, вычислений экспертов и combine равны соответственно 0,336, 0,156 и 0,336 ms, а полностью последовательное выполнение занимает 0,827 ms. Если работу можно равномерно разделить на два micro-batch, время каждого этапа уменьшится вдвое. Идеальное выполнение трёхэтапного конвейера равно сумме трёх этапов одного micro-batch плюс ещё один самый длинный этап, то есть 0,581 ms, что даёт экономию 0,246 ms. Здесь самым длинным этапом являются коммуникации, поэтому темп конвейера определяется сетевой картой. Если DMA сетевой карты конкурирует за HBM с одновременно выполняемым GEMM и из-за этого dispatch и combine каждого micro-batch замедляются примерно на 49%, время того же конвейера возвращается к 0,827 ms: коммуникации действительно перекрываются, но весь пакет не ускоряется. И наоборот, вычисления экспертов должны замедлиться в 3,1 раза, чтобы свести на нет тот же выигрыш.²²

Дополнение и конкуренция за ресурсы — две основные причины, по которым MFU не достигает пикового значения: из-за дополнения матричные блоки вычисляют нулевые строки, которым не соответствует ни один токен, а из-за конкуренции коммуникации и вычисления замедляют друг друга. Обе причины находятся на уровне реализации, тогда как пиковые характеристики оборудования не меняются. Подбор tile, лучше соответствующего фактическому числу строк, и разнесение обращений DMA и GEMM к HBM позволяют вернуть часть потерянной производительности.

На временной шкале рис. 9-26 видно, что когда выполняется combine первого micro-batch, вычисления второго micro-batch уже начались. Dispatch первого пакета заполняет конвейер, а combine последнего пакета завершает его работу; эти два участка представляют собой последовательное время вне установившейся части конвейера.

В описанном конвейере предполагается, что два micro-batch имеют одинаковый размер. При реальном разделении экспертов путь одного токена всё равно имеет вид «dispatch → вычисление выбранных экспертов → combine»: вычисления экспертов находятся между **двумя коммуникационными этапами**. Перекок готовности данных для dispatch обусловлен временем завершения attention и очередью отправки; перекок вычислений экспертов затем превращается в перекок готовности отправки для combine. Даже если объёмы байтов в двух обменах полностью симметричны, этап возврата может затянуться из-за поздней готовности данных.

Для токена t обозначим множество выбранных экспертов через $\mathcal{E}(t)$, а суммарную длину пути от начала текущего слоя через эксперта e , включающего ожидание в очереди, dispatch, вычисление и возврат, — через $L_{t,e}$. Тогда момент завершения объединения удовлетворяет условию:

$$T_t = \max_{e \in \mathcal{E}(t)} L_{t,e} + T_{\text{merge},t}.$$

²²Записи экспериментов с выполнением экспертов и backend.

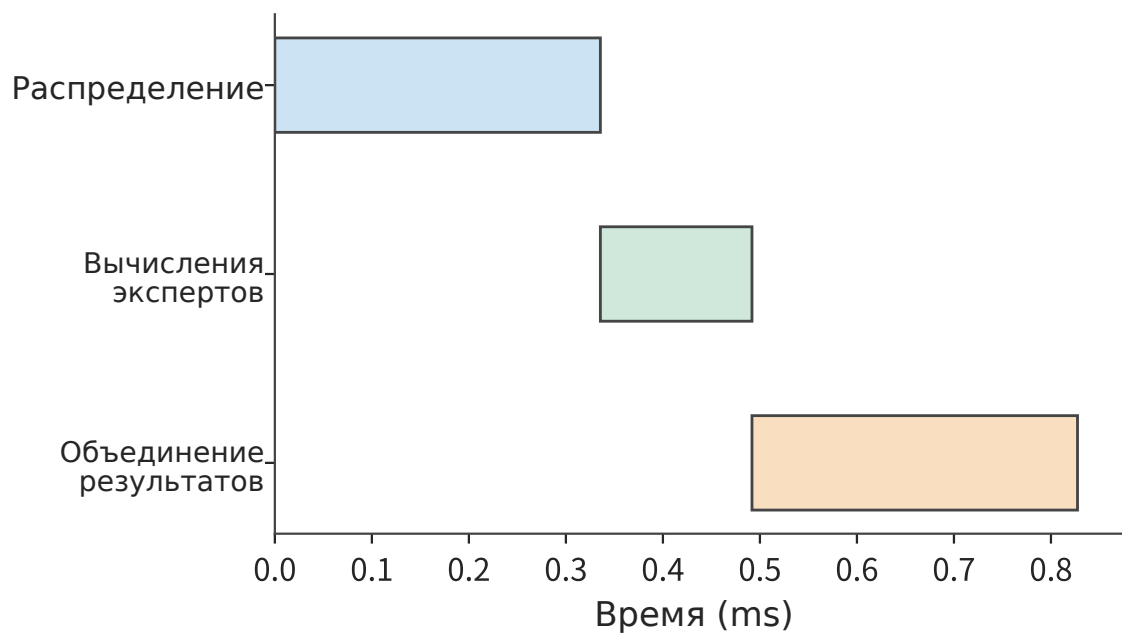


Рис. 9-25. Для всего пакета последовательно выполняются dispatch длительностью 0,336 ms, вычисления длительностью 0,156 ms и combine длительностью 0,336 ms; общее время составляет 0,827 ms. Значения взяты из строки равномерного межсерверного распределения в таблице раздела 9.4.1.

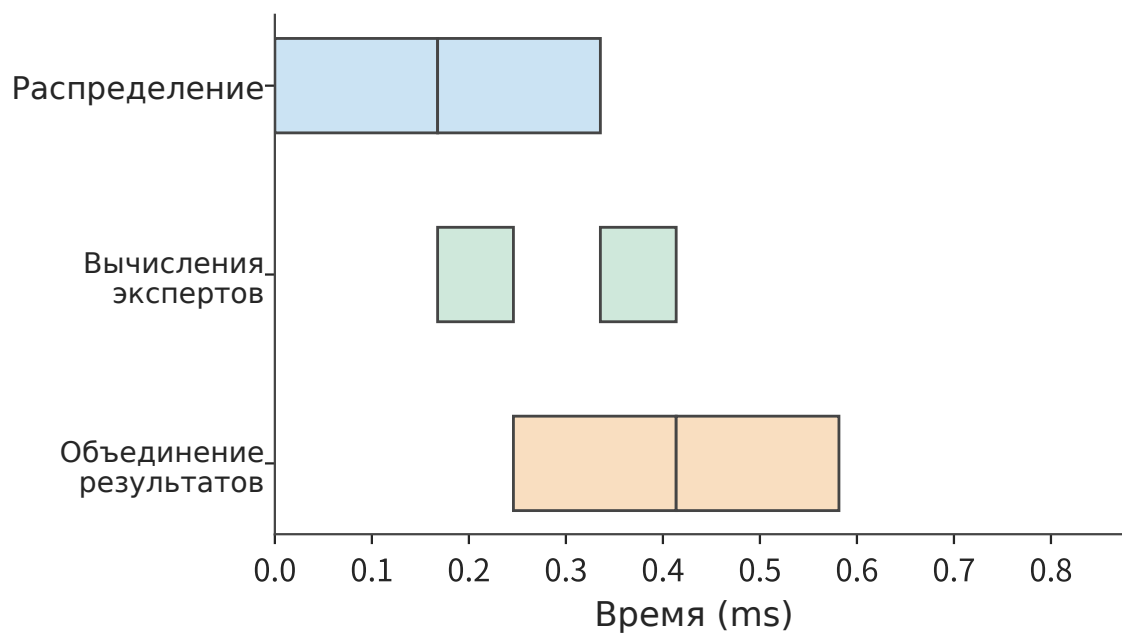


Рис. 9-26. Время каждого этапа для одного micro-batch уменьшается вдвое, а три дорожки используют независимые ресурсы. Пока вычисляется первый micro-batch, можно выполнять dispatch второго, поэтому общее время сокращается до 0,581 ms.

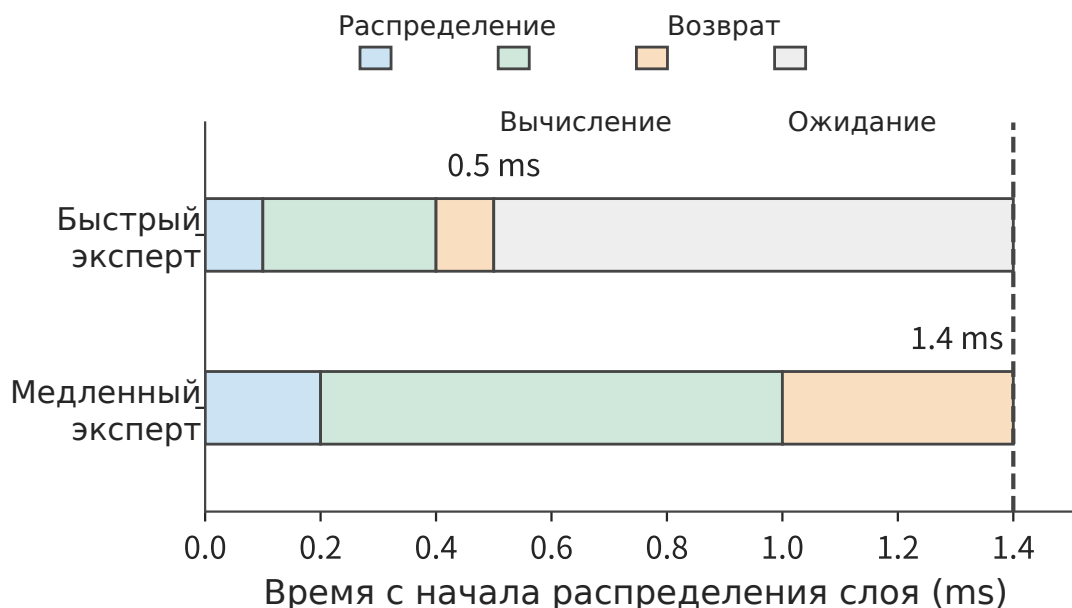


Рис. 9-27. Две ветви экспертов, выбранные одним токеном, имеют условную длительность 0,5 ms и 1,4 ms. Каждая последовательно проходит dispatch, вычисление и combine; токен должен дождаться обоих результатов. Время локального взвешенного объединения на рисунке не учитывается. Серым показано время ожидания после поступления результата быстрой ветви.

Здесь каждый путь включает ожидание в очередях общих ресурсов, поэтому нельзя напрямую подставить время изолированного выполнения каждого этапа и предположить отсутствие взаимного влияния. Если две ветви одного токена возвращаются соответственно через 0,5 ms и 1,4 ms, объединение возможно лишь после 1,4 ms; сокращение времени быстрой ветви до 0,3 ms не изменит момент завершения. При продвижении по отдельным токенам или блокам токены, не выбравшие медленного эксперта, могут продолжить выполнение раньше; при барьере всего пакета они ждут вместе со всеми. Дальность распространения ожидания определяется тем, действительно ли среда выполнения поддерживает мелкозернистое продвижение и требуют ли последующие матричные операции повторного накопления batch.

Это также объясняет, почему период конвейера пула экспертов не всегда можно предсказать формулой $\max(t_A, t_F)$. Она требует стабильного времени этапов, независимых ресурсов и непрерывной подачи работы; реальный период также ограничивается самым загруженным экспертом, каналами прямого и обратного пути, буферами возврата и межслойными зависимостями. Если один пул экспертов обслуживает несколько слоёв или моделей, необходимо суммировать занятые ими ресурсы. Первый micro-batch всё равно несёт полную задержку прямого и обратного пути, поэтому увеличение установившейся пропускной способности не означает сокращения времени ответа для отдельного токена.

Оба коммуникационных этапа, dispatch и combine, реализуются коммуникационной библиотекой. NCCL предоставляет универсальные примитивы коллективных и двухточечных коммуникаций, тогда как специализированные библиотеки коммуникаций EP совместно реализуют маршрутизацию, упаковку, передачу и объединение. DeepEP поддерживает dispatch и combine для MoE, а также передачу с низкой точностью. В опубликованном интерфейсе V2 используется ElasticBuffer, в основе лежит NCCL Gin — backend

NCCL для запуска сетевых коммуникаций с GPU, — а присутствовавший в V1 низколатентный коммуникационный путь EP, который не занимал SM и напрямую отправлял и принимал данные через RDMA, был удалён. Это показывает, что даже одинаковое название библиотеки не позволяет опускать условие о версии. При сравнении реализаций следует зафиксировать версию, форму данных, точность и топологию, отдельно записывать число отправленных и принятых байтов, число строк expert GEMM, padding, события готовности и завершения каждого этапа, а затем измерять p50 и p99 времени всего слоя и полного запроса. Отдельно измеренная пропускная способность коммуникаций объясняет лишь часть результата.²³

При повторном расчёте этого раздела можно сначала сохранить неизменным общее число назначений и менять только распределение экспертов; затем зафиксировать распределение и менять общий исходящий канал и точность dispatch; наконец, в трассировке изменить время готовности одного источника или эксперта. Первые три пункта напрямую воспроизводятся сопутствующими расчётами, а в последнем необходимо наблюдать на временной шкале выполнения, как распространяется ожидание. Только так можно различить перекося трафика, перекося вычислений и ожидание, вызванное простым опозданием данных, не приписывая любые медленные коммуникации недостаточной пропускной способности сети.²⁴

9.5 Распределение и совместное использование KV, маршрутизация запросов

9.5.1 Идентификаторы кэша, область совместного использования и доступность

В двух предыдущих разделах время обработки текущего запроса сокращалось за счёт изменения места выполнения и распределения задач. Другой способ сэкономить вычислительные ресурсы — позволить последующим запросам напрямую использовать KV, оставшиеся после предыдущих вычислений. В разделе 8.3 уже описаны механизмы разбиения на страницы, сопоставления префиксов, совместного использования и вытеснения. Теперь рассмотрим, как при распределении состояния между разными местами выполнения подтвердить совпадение, найти данные и передать их. Ценность кэширования префиксов заключается в устранении повторных вычислений, но «одинакового текста» недостаточно для однозначного определения всего состояния, пригодного для повторного использования. При вычислении идентификатора кэша могут учитываться версии модели и adapter, последовательность токенов, индексы позиций, формат состояния и необходимые настройки кодирования. Например, одни и те же токены с разными adapter после проекции дают разные K и V; один и тот же суффикс после разного предшествующего текста также получает разный контекст для attention. Ключ кэша должен вдоль всей цепочки префикса идентифицировать вычисления, создавшие это состояние.

²³Инженерный отчёт о системе инференса DeepSeek-V3/R1: большой EP, три типа балансировки нагрузки и точность коммуникации; снимок материалов по DeepEP V2. Границы источников и версий.

²⁴Фиксированные входные данные для большого EP и перекося при разделении экспертов, скрипт расчёта, результат повторного расчёта. Пропускная способность сетевых адаптеров и NVLink приведена в спецификации HGX H100, спецификации NVIDIA H100 и спецификации ConnectX-7; производительность H100 принята равной 50% от пиковой величины из таблицы оборудования. Результат вычислений является нижней границей для приведённой модели выполнения; конвейер для двух micro-batch и коэффициент достижения равенства из раздела 9.4.3 также получены из этого результата.

Состояние скользящего окна или рекуррентное состояние дополнительно должно содержать соответствующий индекс позиции токена и момент обновления, чтобы вычисления можно было продолжить с того же состояния.

После подтверждения взаимозаменяемости двух состояний необходимо определить, где хранится состояние и как другие экземпляры смогут его прочесть. Суммирование объёмов само по себе не создаёт общий кэш. Если у двух экземпляров есть по 4 GiB частного кэша в памяти хоста, один и тот же префикс размером 1,125 GiB будет храниться в обоих местах и в сумме займёт 2,25 GiB, причём ни один экземпляр не сможет использовать копию другого. Если оба экземпляра подключены к одному общему бэкенду хранения, то есть к сервису, принимающему и предоставляющему объекты кэша, один объект сможет обслуживать оба экземпляра, но каждому из них перед генерацией, возможно, придётся получить его локальную копию. Иерархический кэш HiCache в SGLang, многоуровневый путь кэша в vLLM и система управления KV-кэшем LMCache предлагают разные варианты организации; от того, проходит ли получение через CPU, зависит время пути, рассматриваемое в следующем разделе.²⁵

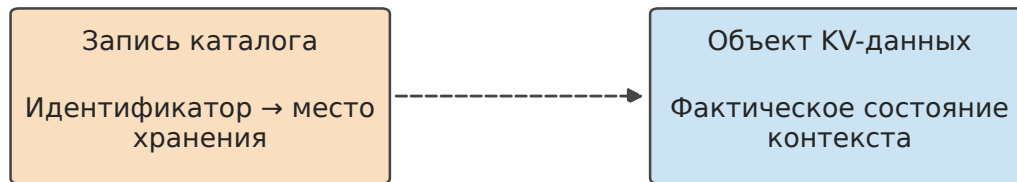
После создания состояние необходимо сохранить и опубликовать, и лишь затем другие экземпляры смогут найти, получить и использовать его. События кэша, которыми пользуется маршрутизатор, обычно содержат только метаданные вроде места хранения и идентификатора кэша, но не полный KV. Когда копия на GPU вытеснена, копия в памяти CPU может по-прежнему существовать. **Каталог** хранит соответствие между идентификаторами кэша и местами хранения, а **объект** — фактические байты KV; при восстановлении отображение каталога и данные объекта проверяются отдельно. Пропущенное или задержанное событие либо освобождение пространства кэша может сделать ранее записанное место хранения недоступным.

9.5.2 Многоуровневое хранилище

В разделе 8.3.4 на одной карте сравнивались сохранение, выгрузка и повторное вычисление. В масштабе целого сервера один и тот же KV может храниться в четырёх местах: НВМ, памяти хоста, локальном SSD и удалённом пуле хранения. Чем ниже уровень, тем больше ёмкость и тем дальше он находится от GPU. Рассмотрим DGX A100: 8 карт A100 80GB, 2 TB памяти хоста, 8 накопителей U.2 NVMe SSD по 3,84 TB и 8 сетевых карт по 200 Gbit/s; значения ниже рассчитаны как доля, приходящаяся на каждую GPU. В качестве SSD выбран Solidigm D7-P5520 3,84 TB с максимальной пропускной способностью последовательного чтения и записи 7,1 GB/s и 4,2 GB/s соответственно. Хранимый объект по-прежнему представляет собой префикс Qwen3-8B из 8192 токенов общим размером 1,125 GiB.²⁶

²⁵ Исследование уровней кэша, путей и маршрутизации.

²⁶ Спецификация DGX A100: 8×A100 80GB, 2 TB памяти хоста, 8×3.84 TB U.2 NVMe и восемь однопортовых сетевых адаптеров 200 Gbit/s; краткое описание Solidigm D7-P5520: последовательное чтение/запись блоками 128K со скоростью до 7,100/4,200 MB/s и ресурс 1 DWPD в течение 5 лет. Ёмкость, длительность хранения, время извлечения и записи, число послойно предварительно загружаемых слоёв и допустимый объём записи SSD для этого раздела приведены в расчёте многоуровневого хранения KV; для повторения расчёта выполните `python3 calculations/calc.py kv-tiers`. Память хоста объёмом 2 TiB поровну распределена между восемью GPU, а скорость чтения и записи SSD принята равной максимальному значению из спецификации.



Найти по каталогу, проверить доступность и получить объект

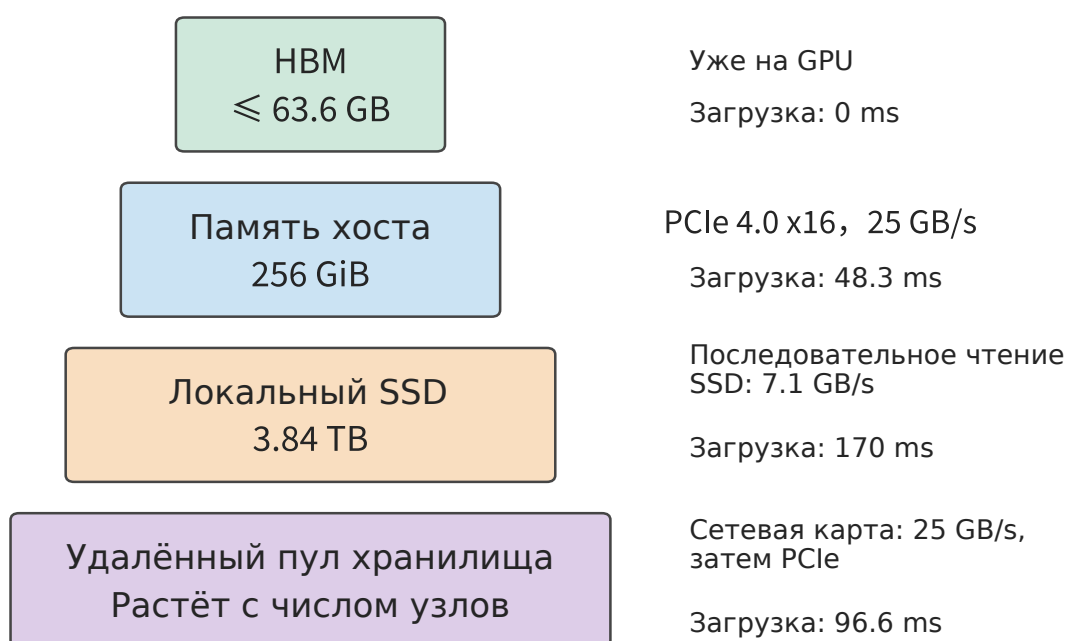
Рис. 9-28. Пунктиром показан поиск местоположения объекта по идентификатору. Каталог используется для поиска, а фактический объект KV — для восстановления вычислений; перед маршрутизацией также необходимо проверить версию и доступность объекта.

Уровень	Ёмкость на GPU	Путь до GPU	Получение префикса 8K
HBM	$\leq 63,6$ GB	Уже на GPU	0
Память хоста	256 GiB	PCIe 4.0 x16, 25 GB/s	48,3 ms
Локальный SSD	3,84 TB	Последовательное чтение SSD, 7,1 GB/s	170 ms
Удалённый пул хранения	Растёт с количеством узлов	Сетевая карта 25 GB/s, затем PCIe	96,6 ms
Для сравнения: повторное вычисление на A100	—	50% от пиковой производительности	856 ms

В строке HBM указан верхний предел, полученный вычитанием 16,4 GB весов BF16 из 80 GB, без учёта активаций и рабочего пространства. Время получения на всех четырёх уровнях значительно меньше 856 ms, необходимых для повторного вычисления, поэтому для однократного получения любой уровень хранения состояния быстрее повторного вычисления. Реальные ограничения связаны с тремя другими факторами: сохранится ли кэш до следующего использования, можно ли перекрыть чтение вычислениями и насколько дорого обходится запись на каждый уровень.

Ёмкость определяет продолжительность хранения кэша. Одна A100, непрерывно выполняющая prefill для запросов 8K без попаданий в кэш, каждые 0,856 s создаёт KV размером 1,125 GiB, то есть скорость создания составляет $r \approx 1,41$ GB/s. Если все новые KV записываются на один уровень и вытесняются в порядке записи, уровень ёмкостью C сможет хранить KV, созданные примерно за последние C/r секунд: HBM — около 45 s, память хоста — около 195 s, SSD — около 2722 s, то есть 45 минут. И наоборот, если

Иерархия KV-хранилищ для одной A100



Для сравнения: пересчёт этого префикса
8K на A100 ≈ 856 ms

Рис. 9-29. Четыре уровня хранения, приходящиеся на каждую A100 в DGX A100. Ширина прямоугольников показывает только порядок ёмкостей; справа указаны путь от соответствующего уровня до GPU и время получения префикса размером 1,125 GiB. При удалённом получении данные сначала передаются по сети в память хоста, а затем по PCIe на GPU; эти два этапа выполняются последовательно.

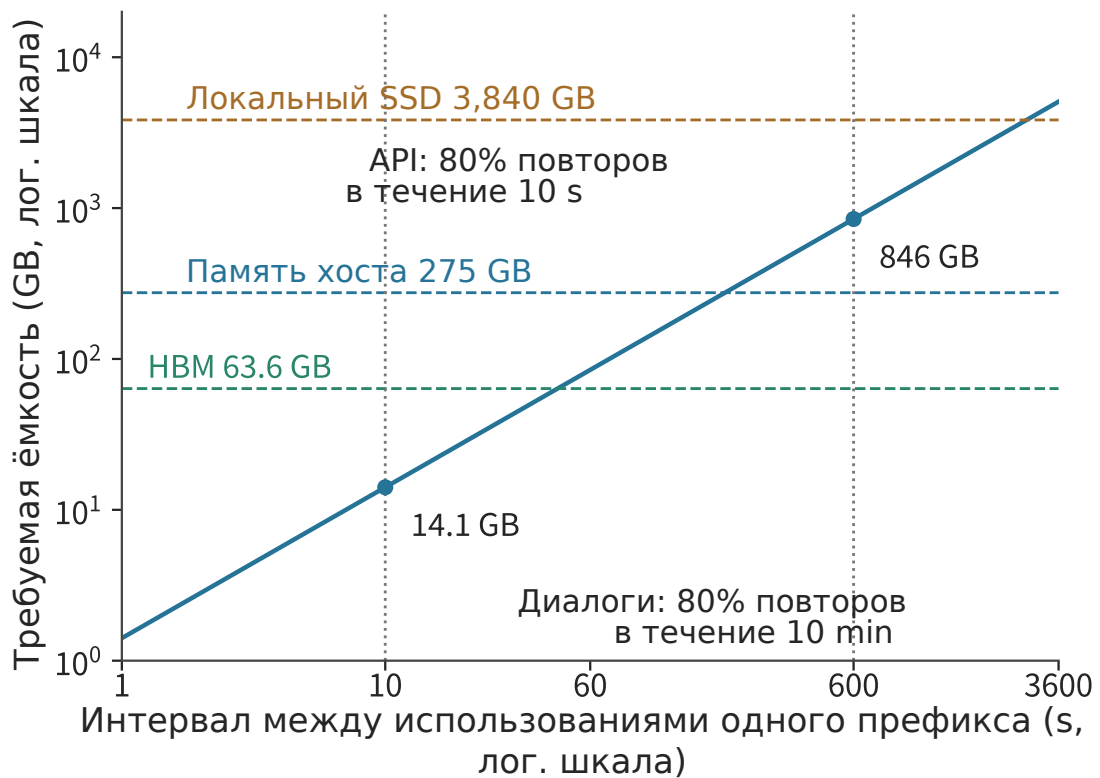


Рис. 9-30. Требуемая ёмкость линейно растёт с интервалом повторного использования. Наклонная линия соответствует $C = rT$, где $r \approx 1,41$ GB/s — скорость создания KV одной A100 при непрерывном выполнении prefill для запросов 8K без попаданий; три пунктирные линии показывают ёмкость трёх уровней хранения на GPU; две вертикальные линии обозначают временные диапазоны, в которых происходит 80% повторных использований для двух типов нагрузки. Обе оси имеют логарифмический масштаб.

интервал между двумя использованиями одного префикса равен T , то для попадания всех повторных запросов, поступивших в пределах интервала T , ёмкость этого уровня должна удовлетворять условию

$$C \geq rT.$$

В исследовании Alibaba Cloud на основе трассировок рабочих запросов было измерено время до повторного использования для двух типов нагрузки: в диалоговой нагрузке для индивидуальных пользователей 80% повторных использований происходили в течение 10 минут, а в корпоративной API-нагрузке — в течение 10 секунд. Для интервала в 10 секунд требуется всего 14,1 GB, что помещается в HBM; для интервала в 10 минут нужно 846 GB, что превышает 275 GB памяти хоста на карту, поэтому для полного покрытия потребуется добавить SSD.²⁷

Условие $C \geq rT$ также показывает, когда SSD не требуется. В том же исследовании для диалоговой нагрузки необходимая ёмкость кэша Llama3-70B примерно в четыре раза превышала доступную HBM; на сервере с 8 картами A100 и 1 TB памяти хоста на каждую карту приходилось по 128 GB, чего уже было достаточно без добавления SSD или уда-

²⁷Wang и др., KVCache Cache in the Wild, USENIX ATC 2025, раздел 3.4: Trace A представляет диалоговую нагрузку индивидуальных пользователей, Trace B — нагрузку API; вывод о необходимой ёмкости относится к модели GQA и получен по максимальной интенсивности запросов каждого экземпляра.

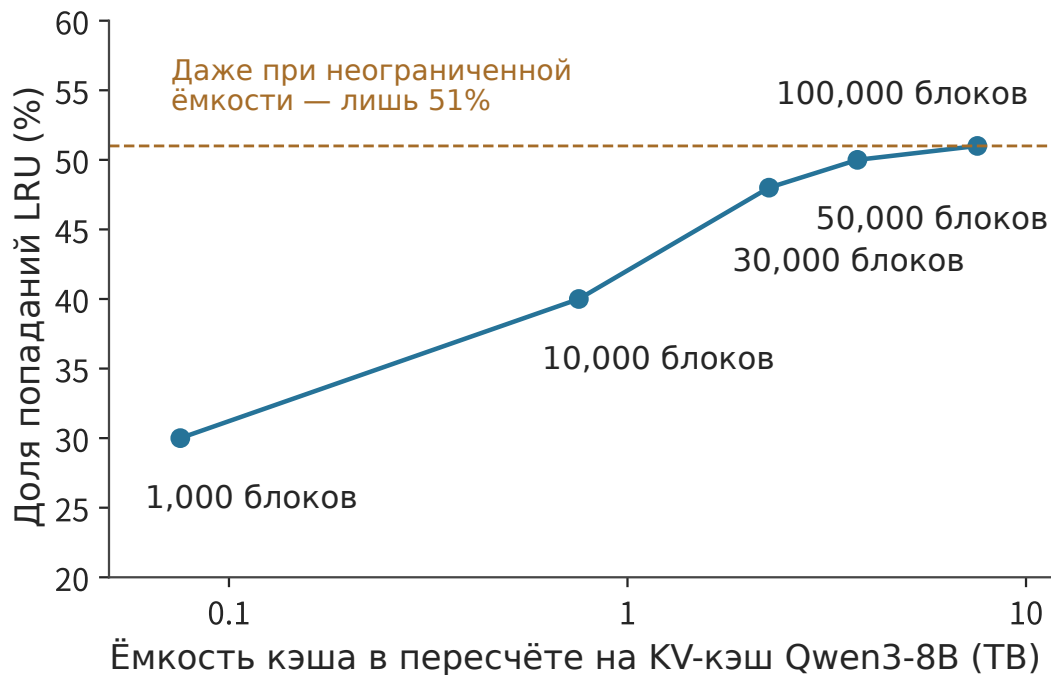


Рис. 9-31. С ростом ёмкости доля попаданий приближается к насыщению. Данные показывают долю попаданий при вытеснении LRU в часовой выборке трассировки Mooncake с блоками по 512 токенов; горизонтальная ось переведена в байты из расчёта 144 KiB на токен для Qwen3-8B. Это лишь выборка одного фрагмента трафика; необходимая ёмкость реального сервиса растёт пропорционально объёму трафика.

лённого уровня. Различие заключается в r : в исследовании она вычислялась по фактической частоте запросов каждого экземпляра, а сами запросы были короче — в среднем 973 токена для одного раунда и 5953 токена для нескольких раундов; здесь же предполагается, что GPU непрерывно выполняет prefill для запросов 8K. Чем больше попаданий по префиксу и чем меньше размер KV на токен, например при MLA, тем ниже r . Значение T зависит от того, кто инициирует следующий раунд: в диалоге с человеком интервал измеряется минутами, а при программных вызовах — секундами.

С увеличением ёмкости прирост доли попаданий замедляется. Ёмкость позволяет сохранять только состояния, которые будут повторно использованы в будущем, но некоторые состояния больше не понадобятся. Mooncake опубликовал часовую выборку трассировки рабочего сервиса Kimi с блоками по 512 токенов. При вытеснении по алгоритму LRU (первым вытесняется блок, к которому дольше всего не обращались) увеличение кэша с 1000 до 50 000 блоков повышает долю попаданий с 30% до 50%; даже при неограниченной ёмкости она составляет лишь 51%. В пересчёте на Qwen3-8B 50 000 блоков занимают около 3,77 TB, что сопоставимо с одним SSD на 3,84 TB. В этой трассировке более половины блоков никогда не использовались повторно, тогда как к некоторым другим обращались более десяти тысяч раз; в трассировке Alibaba Cloud наблюдалась такая же концентрация: 10% блоков обеспечивали 77% повторных использований.²⁸

Однократное получение или удалённое чтение на каждом шаге. Существует три типичных способа использования общего KV: локальное повторное выполнение prefill,

²⁸Раздел 4 и таблица 1 технического отчёта Mooncake: часовой trace с 23,608 запросами и средним входом в 7590 токенов; степень концентрации trace Alibaba Cloud приведена в статье из предыдущего примечания.

однократное удалённое получение с последующим локальным размещением и удалённое чтение на каждом шаге decode. Во всех трёх случаях используется один и тот же контекст, но частота передачи данных совершенно различна.

Для префикса размером 1,125 GiB и используемой в этой главе сетевой карты 200 Gbit/s (25 GB/s) однократное получение полезной нагрузки занимает около 48,3 ms. Если при каждом из 100 вызовов decode неизменный префикс заново считывается из удалённого хранилища, суммарное время занятости канала составит около 4,83 s. Если эти 100 чтений должны завершиться за одну секунду, только для этого контекста потребуется около 121 GB/s — в 4,8 раза больше пропускной способности заданного канала. Параллельное выполнение чтения и вычислений также не позволит каналу 25 GB/s передать все эти байты за одну секунду. Для компактного префикса MLA размером 549 MiB одно получение занимает около 23,0 ms, а 100 чтений — около 2,30 s; для завершения за секунду всё равно потребуется примерно 57,6 GB/s, то есть в 2,3 раза больше пропускной способности канала. Уменьшение состояния вдвое лишь вдвое снижает это превышение: удалённое чтение на каждом шаге остаётся непрактичным.

Однократное получение с локальным размещением снижает частоту передачи с «одного раза на шаг» до «одного раза на повторное использование», но занимает локальную НВМ. Если состояние имеет размер V байт и хранится локально в течение τ секунд, произведение занимаемой ёмкости на время равно $V\tau$ и измеряется в байт-секундах. При одинаковом хранении в течение 10 секунд больший объект занимает больше ресурсов; при одинаковом размере объекта это произведение увеличивается с продолжительностью хранения. Для сеанса из раздела 9.1.3 во время десятисекундного выполнения инструмента состояние GQA занимает 11,25 GiB-s, а компактное состояние MLA — 5,36 GiB-s, поэтому та же локальная ёмкость может вместить примерно вдвое больше ожидающих сеансов. Следовательно, неактивные сеансы можно размещать удалённо, получать при возобновлении и оставлять локально на время непрерывного decode. Тогда передача данных происходит преимущественно при возобновлении и приостановке сеанса.

Предположим, что состояние будет повторно использовано в будущем k раз, и пока не будем учитывать ни время ожидания в очереди, ни вытеснение другого содержимого кэша из-за занятого им пространства. Сохранение с последующим получением будет быстрее повторного вычисления при условии

$$T_{write} + kT_{read} < kT_{recompute}.$$

Рассмотрим префикс Qwen3-8B из 8192 токенов: повторное вычисление на A100 занимает около 0,856 секунды, то есть время prefill из раздела 9.2.3, а сохранение и получение через сетевую карту 25 GB/s — примерно по 48,3 ms. При одном повторном использовании сохранение и получение вместе занимают около 96,6 ms — менее одной восьмой времени повторного вычисления. Поэтому сохранять состояние выгодно, если оно будет использовано хотя бы ещё один раз. При замедлении канала вывод меняется: при пропускной способности примерно от 1,41 до 2,82 GB/s состояние должно быть повторно использовано не менее двух раз, чтобы компенсировать стоимость сохранения, причём чем ближе скорость к 1,41 GB/s, тем больше требуется повторных использований; ниже примерно 1,41 GB/s даже однократное получение медленнее повторного вычисления, а с ростом числа повторных использований потери увеличиваются. Пропускная способность 10 GbE в каждом направлении составляет всего 1,25 GB/s, поэтому одно получение

занимает около 966 ms; в этом случае следует выполнить повторное вычисление либо заранее завершить получение, убрав его с критического пути. Размер состояния входит только в T_{write} и T_{read} , тогда как $T_{recompute}$ определяется стоимостью prefill модели. Для компактного состояния MLA однократные запись и получение через ту же сетевую карту вместе занимают лишь около 46,1 ms, поэтому левая часть неравенства уменьшается пропорционально количеству байтов KV.

Критическая пропускная способность 1,41 GB/s в точности совпадает с приведённой выше скоростью создания KV r : получение V байтов занимает V/B , а повторное вычисление — V/r , поэтому при пропускной способности канала B , превышающей скорость создания этого KV на GPU, получение будет быстрее повторного вычисления. Скорости записи и чтения локального SSD выше r : запись занимает около 288 ms, чтение — около 170 ms, а их сумма при одном повторном использовании составляет примерно 458 ms, что по-прежнему меньше 856 ms повторного вычисления.

Перекрывание чтения вычислениями. После попадания запросу остаётся вычислить только 256 новых токенов, что при принятой в разделе 9.2.3 производительности 50% от пикового значения A100 занимает около 30,9 ms. Если исторический KV находится в памяти хоста и перед началом вычислений целиком считывается на GPU, общее время составляет $48,3 + 30,9 = 79,2$ ms, причём чтение длится дольше вычисления. CachedAttention использует послойную предварительную загрузку: Transformer выполняет вычисления по слоям, а слою i нужен только KV слоя i , поэтому одновременно с вычислением слоя i на GPU по PCIe можно считывать последующие слои. Исторический KV каждого слоя Qwen3-8B занимает 32 MiB; чтение одного слоя занимает 1,34 ms, а вычисление новых токенов для слоя — всего 0,857 ms. Вычисления каждого слоя вынуждены ждать чтения, поэтому общее время составляет около 49,2 ms и по-прежнему определяется чтением. Для дальнейшего сокращения времени необходимо загрузить несколько слоёв ещё до начала выполнения запроса: за время выполнения предыдущего batch можно считать первые 14 слоёв (448 MiB) в зарезервированный буфер HBM, после чего чтение остальных 22 слоёв будет полностью скрыто вычислениями, а общее время снизится до собственно времени вычисления — 30,9 ms. Буфер должен вмещать именно тот объём байтов, на который чтение превосходит вычисление:

$$S_{buf} = B (T_{read} - T_{new}),$$

где B — пропускная способность канала, T_{read} — время чтения всего исторического KV, а T_{new} — время вычисления новых токенов. Подстановка 25 GB/s, 48,3 ms и 30,9 ms даёт около 437 MB, то есть немного больше 13 слоёв; после округления вверх до целого слоя получается 14 слоёв. При $T_{new} \geq T_{read}$ буфер не требуется: когда новый ввод достигает примерно 400 токенов, вычисления могут скрыть чтение из памяти хоста; для последовательного пути из удалённого пула хранения требуется около 800 токенов, а для локального SSD — около 1400. В измерениях CachedAttention на LLaMA-13B при 1K исторических и 100 новых токенах послойная предварительная загрузка сократила время prefill на 35%, а после добавления буфера на 15 слоёв — на 61%.²⁹

Предварительное чтение с SSD во время ожидания в очереди. Чтение одного слоя с локального SSD занимает 4,73 ms — в 5,5 раза больше вычисления слоя, поэтому по-

²⁹Gao и др., Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention, USENIX ATC 2024: в разделах 3.2–3.3 описаны послойная предварительная загрузка, асинхронное сохранение, а также предварительная выборка и вытеснение по очереди; в разделах 4.3.2–4.3.3 приведены измерения буфера предварительной загрузки и доли попаданий.

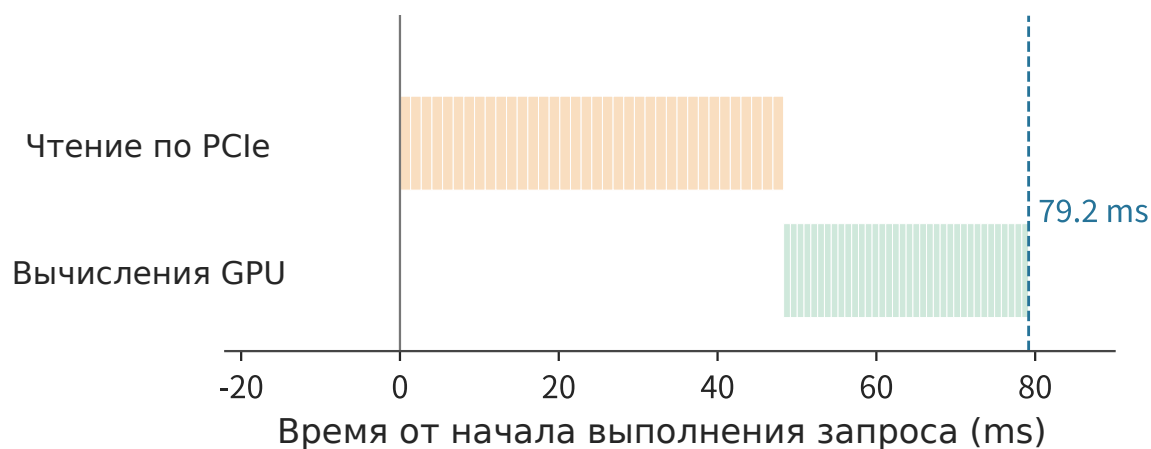


Рис. 9-32. Сначала весь исторический KV размером 1,125 GiB считывается из памяти хоста, затем вычисляются 256 новых токенов; общее время составляет 79,2 ms. Оранжевым показано чтение по PCIe, зелёным — вычисление на GPU; каждый небольшой сегмент соответствует одному слою.

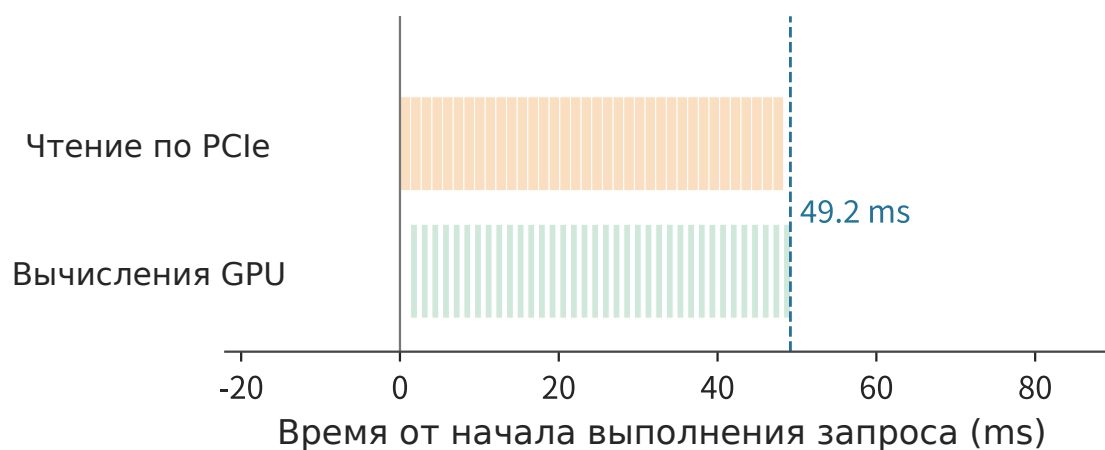


Рис. 9-33. Послойная предварительная загрузка. Пока GPU вычисляет один слой, по PCIe считываются последующие; чтение каждого слоя занимает 1,34 ms, вычисление — 0,857 ms. Вычисления каждого слоя вынуждены ждать чтения, поэтому общее время 49,2 ms определяется чтением.

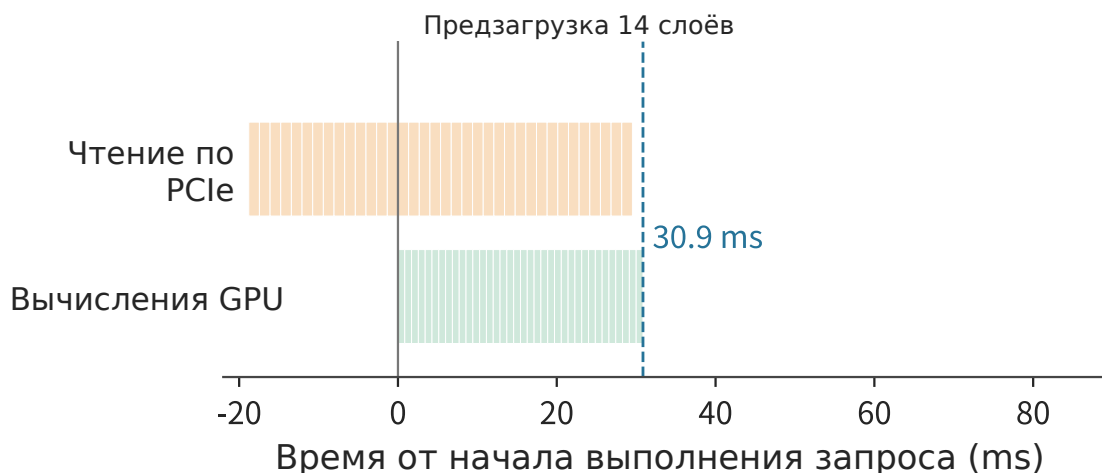


Рис. 9-34. До начала выполнения этого запроса за время выполнения предыдущего batch предварительно считываются первые 14 слоёв (448 MiB); чтение остальных 22 слоёв полностью скрыто вычислениями, и общее время равно собственно времени вычисления — 30,9 ms. Горизонтальная ось одинакова на всех трёх рисунках.

слойная предварительная загрузка может скрыть лишь небольшую часть времени. Если к началу выполнения запроса состояние всё ещё находится на SSD, этот этап даже при послойном чтении займёт около 171 ms, тогда как собственно вычисление — лишь 30,9 ms. Можно использовать время ожидания в очереди: пока запрос находится в очереди, планировщик уже знает, какой префикс ему нужен, и может заранее считать его с SSD в память хоста. Если время ожидания не меньше 170 ms, чтение уйдёт с критического пути, а после начала выполнения данные можно будет послойно считывать из памяти хоста описанным выше способом. Это то же соотношение $T_{first} = \max(Q, R) + C$, которое рассматривается в разделе 9.5.4: предварительное чтение перекрывает время готовности состояния R временем ожидания Q .

Количество сеансов, помещающихся в памяти хоста, определяет, для какого максимального числа первых запросов в очереди планировщик может выполнить предварительное чтение: 256 GiB вмещают около 227 префиксов 8K, поэтому достаточно предварительно считывать данные только для первых 227 запросов. При нехватке памяти хоста и необходимости выгрузки также учитывается очередь: состояния, которые вскоре понадобятся этим запросам, выгрузить нельзя; среди остальных первым выгружается состояние с наиболее поздним следующим использованием. LRU и FIFO учитывают только прошлые обращения и не могут использовать сведения о предстоящих запросах в очереди. CachedAttention воспроизводил многораундовые диалоги из ShareGPT, набора пользовательских диалогов с ChatGPT, на 4 картах A100 с 128 GB памяти хоста и SSD на 10 TB: при предварительном чтении и выгрузке с учётом очереди общая доля попаданий составила 86%, причём более 99,6% попаданий пришлось на память хоста; при LRU и FIFO доля попаданий составляла лишь 58% и 48% соответственно, а на память хоста приходилось менее 1% попаданий, поэтому почти каждое попадание требовало чтения с SSD.³⁰

Срок службы SSD ограничивает объём записи. Запись также следует убирать с

³⁰Gao и др., Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention, USENIX ATC 2024: в разделах 3.2–3.3 описаны послойная предварительная загрузка, асинхронное сохранение, а также предварительная выборка и вытеснение по очереди; в разделах 4.3.2–4.3.3 приведены измерения буфера предварительной загрузки и доли попаданий.

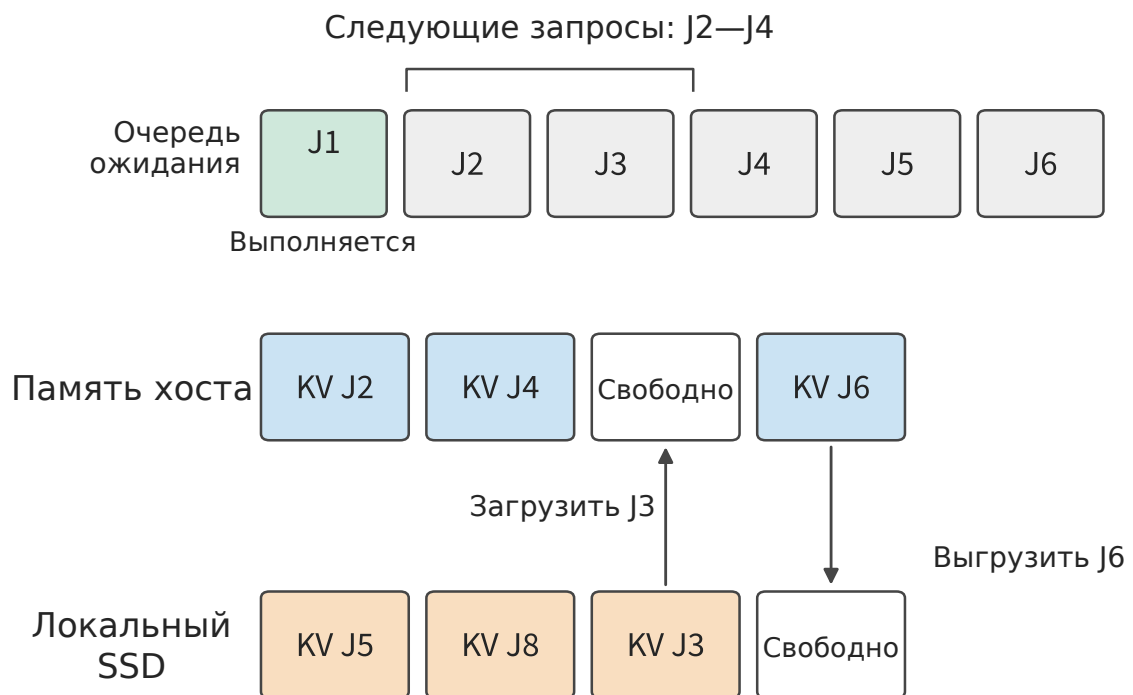


Рис. 9-35. В памяти хоста есть четыре места хранения: одно оставлено свободным для принимаемых данных, а остальные три предназначены для следующих в очереди J2—J4. Состояние J3 всё ещё находится на SSD и считывается в свободное место, пока запрос ожидает в очереди; J6 расположен после этих трёх запросов и будет использован позже всех, поэтому первым выгружается на SSD.

критического пути. KV, послойно создаваемый во время prefill, можно записывать параллельно с вычислением, а при decode на каждом шаге добавляется KV только одного токена. Запись со скоростью $r \approx 1,41$ GB/s занимает лишь 5,6% пропускной способности PCIe и 34% пропускной способности последовательной записи SSD. Настоящим ограничением SSD является ресурс записи: для D7-P5520 заявлена одна полная перезапись в день в течение пяти лет (1 DWPD), поэтому диск на 3,84 TB в среднем допускает запись лишь 44,4 MB в секунду. Если записывать на SSD все новые KV, объём записи превысит эту норму в 31,7 раза, и пятилетний ресурс будет исчерпан менее чем за два месяца. В долгосрочной перспективе на SSD можно записывать лишь около 3,2% новых KV. В обеих упомянутых трассировках повторное использование концентрировалось на небольшом числе блоков, а в трассировке Moonsake более половины блоков никогда не использовались повторно. Поэтому перед записью на SSD необходим контроль допуска — например, записывать только уже повторно использованные префиксы или ещё не завершённые многораундовые сеансы. При записи в пределах этой нормы SSD сможет хранить ровно тот объём KV, который создаётся за последние сутки.

Такую связь между ёмкостью и попаданиями можно наблюдать и экспериментально. В серии экспериментов с двумя движками на одной карте из 12 возможностей повторного использования префикса общий CPU-пул размером 4 GiB обеспечил 5 полных попаданий, а пул размером 8 GiB — все 12. Дополнительные 4 GiB сохранили семь полных префиксов, которые иначе были бы вытеснены; если состояние для повторного запроса уже находится локально, его не требуется получать снова. Поэтому пользу общей ёмкости необходимо оценивать по тому, как долго она сохраняет данные, сколько повторных вычислений предотвращает и сколько дополнительных перемещений создаёт.³¹

9.5.3 Персистентность, checkpoint и частичное повторное вычисление

В предыдущем разделе предполагалось, что сохранённое состояние можно получить полностью. После перезапуска экземпляра записи каталога, данные на диске и непрерывный префикс, пригодный для повторного использования, могут не совпадать, поэтому при восстановлении необходимо заново проверить, какие результаты вычислений действительно сохранены. Персистентность позволяет состоянию оставаться доступным после перезапуска экземпляра или приостановки задачи. Полное сохранение уменьшает объём повторных вычислений при восстановлении, периодический checkpoint сокращает объём записи ценой дополнительной работы при восстановлении, а частичное повторное вычисление дополняет недостающие части с использованием сохранённого префикса. В checkpoint необходимо сохранять всё состояние, требуемое для продолжения вычислений: для полного GQA — K и V контекста; состояние DeepSeek V4 дополнительно включает результаты сжатия, окно и незавершённый блок сжатия. После восстановления вычисление продолжается с последнего полного обновления, записанного в checkpoint.³²

Персистентное состояние сохраняется постранично и так же постранично считывается при восстановлении. Сначала рассмотрим полную страницу GQA. В Qwen3-8B ло-

³¹Сравнение ёмкости общего CPU-пула KV и повторное воспроизведение Agent с сохранённым реальным контекстом. При сравнении ёмкости использовалась контролируемая задача поиска; в повторном воспроизведении реального агента наблюдались преждевременное завершение и снижение качества выполнения задачи.

³²Материалы о персистентности состояния и сервисе генерации из технического отчёта DeepSeek V4, а также заметки по чтению для этой главы.

гическая страница, содержащая 16 токенов, занимает 2,25 MiB. Если фрагменты K и V каждого слоя перемещаются отдельно, образуется множество небольших непрерывных сегментов; если же они собираются по страницам, меняются число передач и необходимые преобразования раскладки. После разделения логической страницы по K и V 36 слоёв получается 72 непрерывных сегмента по 32 KiB; при сборке по страницам получается один объект размером 2,25 MiB. Первый вариант сильнее зависит от ограничения числа операций в секунду, а во втором больше времени расходуется на передачу непрерывной полезной нагрузки.

После перезапуска некоторый запрос считал 64 страницы, покрывающие 1024 токена, но непрерывный префикс, пригодный для повторного использования, содержал только 1008 токенов, то есть 63 страницы. Лишняя считанная страница заняла 2,25 MiB, но не позволила избежать соответствующего повторного вычисления. Для этого запроса объём чтения составил 144 MiB, а эффективно повторно использованный объём — около 142 MiB; ещё нагляднее сформулировать это так: «считано 64 страницы, использовано 63». Ограничение здесь возникает после чтения: заменить вычисление могут лишь успешно сопоставленные страницы, образующие непрерывное продолжение существующего контекста. Ускорение чтения с диска сократит его продолжительность, но не изменит необходимость обработки последней страницы этого запроса.³³

При отсутствии страницы необходимо сравнить длительность дальнейшего ожидания с длительностью повторного вычисления. Повторное вычисление полного префикса из 8192 токенов на A100 занимает около 0,856 секунды, то есть время prefill из раздела 9.2.3; если ожидание недоступного объекта уже длится одну секунду, оно превысило время повторного вычисления всего префикса. В реальном эксперименте с усечённой страницей запрос с безусловным ожиданием так и не завершился в пределах 60-секундного окна наблюдения, а после разрешения прекратить ожидание получил результат путём повторного вычисления. После завершения текущего запроса повреждённую страницу необходимо изолировать или исправить, иначе следующий запрос столкнётся с тем же ожиданием.³⁴

9.5.4 Привязка к кэшу и маршрутизация запросов

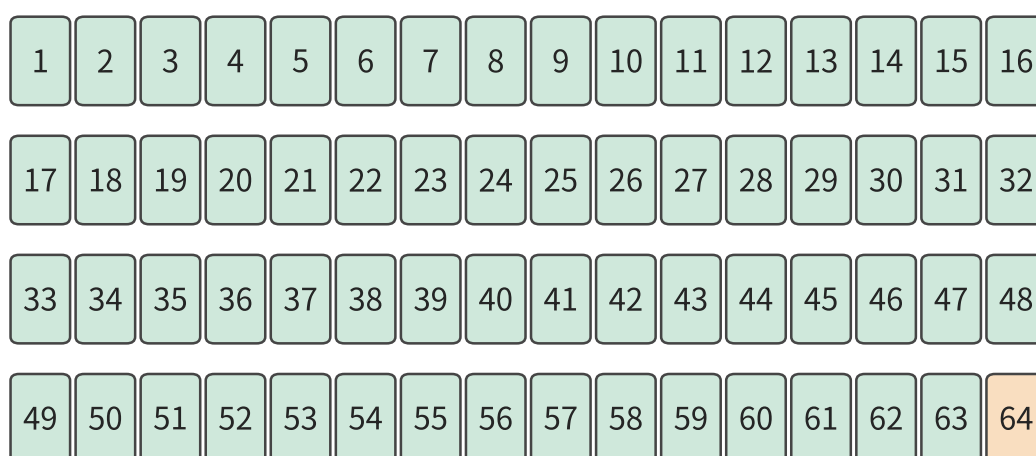
После подтверждения возможности повторного использования кэша всё ещё требуется решить, на какую машину отправить запрос. Машина с нужным кэшем может быть занята, тогда как другая машина, хотя ей придётся получить состояние или вычислить его повторно, может завершить запрос раньше. Поэтому при маршрутизации нужно сравнивать время завершения запроса. Пусть Q — самый ранний момент, когда GPU сможет начать выполнение, R — момент готовности состояния, отсчитываемый от поступления запроса, а C — оставшийся объём работы после подготовки состояния. Если выполнение начинается только после получения всего состояния, а получение можно перекрыть ожиданием GPU, время до первого токена равно

$$T_{first} = \max(Q, R) + C.$$

³³Расчёт чтения страниц при перезапуске и фактического повторного использования: прочитано 144 MiB, эффективно повторно использовано 141.75 MiB. Прочитаны 64 страницы для 1024 токенов, из которых повторно использованы 63 страницы для 1008 токенов.

³⁴Сравнение стратегий предварительной выборки усечённых страниц; окно наблюдения в эксперименте составляет 60 секунд.

Загружено 64 страницы; первые 63 переиспользуются
подряд



Зелёный: 1008 токенов; оранжевый: ещё обработать 16 токенов

Рис. 9-36. Не все считанные страницы обязательно становятся префиксом, пригодным для повторного использования. После штатного перезапуска этот запрос считал 64 страницы по 16 токенов, но повторно использовал только первые 63; последнюю страницу всё равно пришлось обработать. Каждая страница занимает 2,25 MiB, а граница сопоставления для этого запроса находится на 1008 токенах.

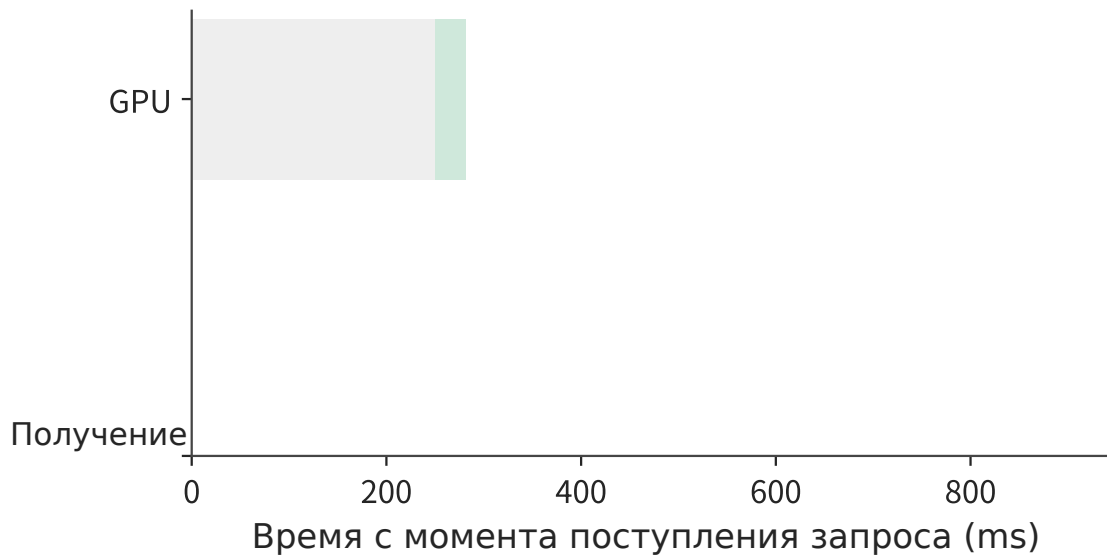


Рис. 9-37. У А уже произошло попадание в локальный кэш, но перед вычислением продолжительностью 30,9 ms GPU ожидает в очереди 250 ms, поэтому первый токен возвращается через 281 ms. Серым показано ожидание, зелёным — вычисление.

Если получение может начаться лишь после освобождения GPU, время ожидания и время получения складываются; при послойном конвейере требуется более подробный граф выполнения. Например, если $Q = 80$ ms, получение занимает 60 ms, а оставшееся вычисление — 10 ms, при параллельной подготовке общее время составит 90 ms; если запускать получение только после освобождения GPU, потребуется 150 ms. Объём вычислений и число передаваемых байтов одинаковы, но из-за разных зависимостей время отличается на 60 ms.

Пример 9.7. Как попадание в кэш и ожидание в очереди совместно определяют маршрутизацию запроса? А и В — два экземпляра A100. Запрос содержит префикс из 8192 токенов и 256 новых входных токенов. В HBM экземпляра А есть этот префикс, но ожидание в очереди составляет 250 ms; В ожидает всего 20 ms, но нужного кэша у него нет. По принятому в разделе 9.2.3 допущению используется 50% пиковой производительности A100, то есть 156 TFLOP/s: полное повторное вычисление 8448 токенов требует около 138,4 TFLOPs матричных операций и занимает примерно 887 ms; после попадания нужно вычислить только 256 новых токенов — около 4,81 TFLOPs и 30,9 ms. В также может получить префикс из удалённого хранилища: фиксированные накладные расходы поиска составляют 10 ms, а весь объект размером 1,125 GiB сначала передаётся по сети в память хоста, затем по PCIe 4.0 x16 на GPU; как и в примере 9.3, используется скорость 25 GB/s, что даёт около 48,3 ms.

Путь	Время до первого токена
А: локальное попадание	$250 + 30,9 \approx 281$ ms
В: прямое повторное вычисление	$20 + 887 \approx 907$ ms
В: удалённое получение по 50 GbE (6,25 GB/s)	Около 282 ms
В: удалённое получение по 200 GbE (25 GB/s)	Около 137 ms

На первой схеме маршрутизации данные А уже находятся на GPU, но вычисление не может начаться раньше 250 ms. В освобождается через 20 ms; прямое повторное вычис-

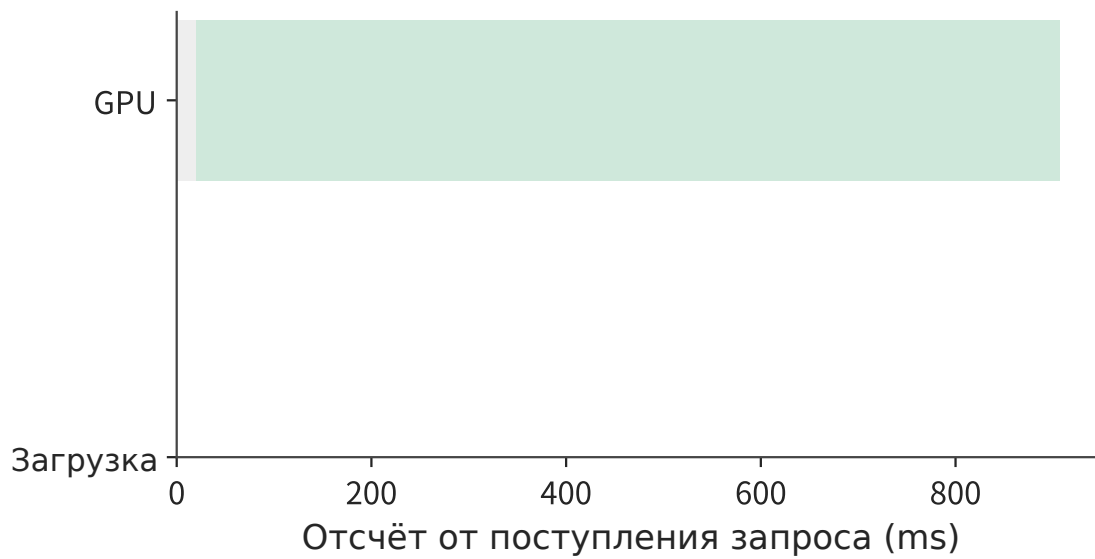


Рис. 9-38. В освобождается через 20 ms, после чего повторное вычисление на A100 занимает 887 ms; первый токен возвращается через 907 ms.

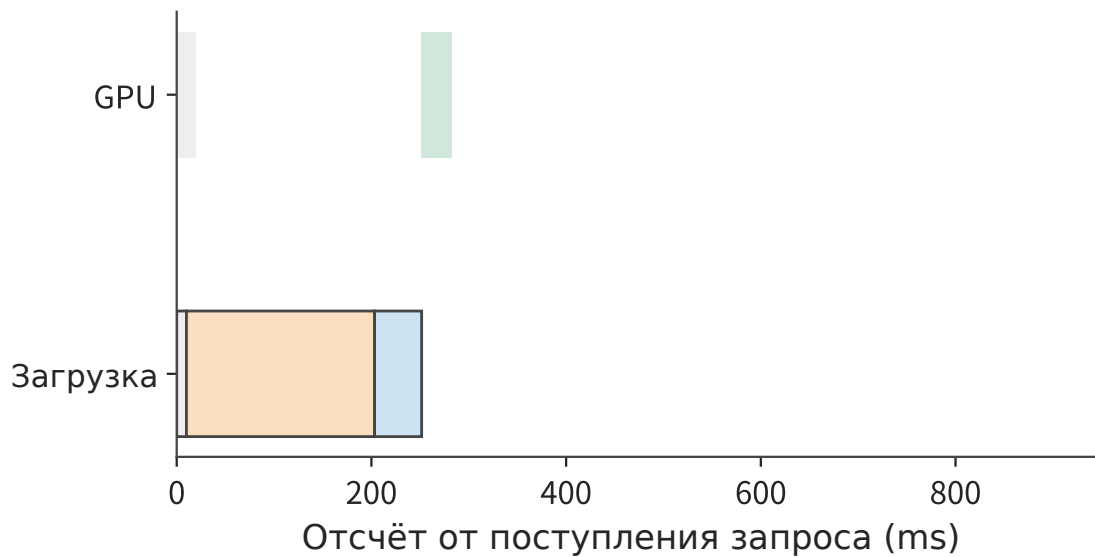


Рис. 9-39. Сначала выполняется поиск в течение 10 ms, затем 1,125 GiB считываются по 50 GbE со скоростью 6,25 GB/s в память хоста и в конце передаются на GPU по PCIe со скоростью 25 GB/s. Вычисление ожидает готовности и данных, и GPU; первый токен возвращается примерно через 282 ms, приблизительно на 1,6 ms позже, чем на A.

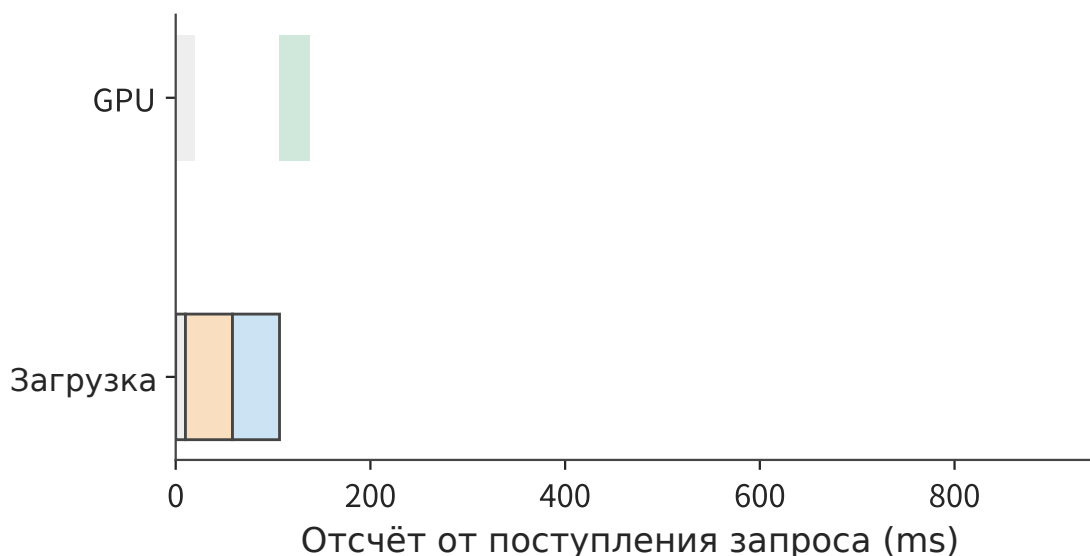


Рис. 9-40. После замены удалённого канала на 200 GbE (25 GB/s) первый токен возвращается примерно через 137 ms. Оранжевым показано удалённое чтение, синим — передача из памяти хоста на GPU; отсчёт на всех четырёх рисунках начинается с момента поступления запроса, горизонтальные оси одинаковы.

ление может начаться немедленно, но 887 ms вычисления дольше любого пути получения, тогда как при удалённом получении приходится продолжать ждать данные. Увеличение пропускной способности удалённого канала сокращает оранжевый сегмент, но поиск, передача из памяти хоста на GPU и заключительное вычисление сохраняются.

Чтобы В вернул результат раньше А с его 281 ms, состояние должно быть готово за 250 ms после вычитания 30,9 ms вычисления при попадании. После дополнительного вычитания 10 ms поиска и примерно 48,3 ms передачи из памяти хоста на GPU для удалённого чтения остаётся около 191,7 ms. Деление 1,125 GiB на этот бюджет даёт пропускную способность, при которой получение на В сравнивается с локальным попаданием на А, — около 6,30 GB/s: скорость 50 GbE, равная 6,25 GB/s, немного ниже этого значения, а 200 GbE — значительно выше. Пропускная способность, при которой получение сравнивается с повторным вычислением, составляет лишь около 1,48 GB/s, поэтому на A100 повторное вычисление префикса 8K почти всегда будет самым медленным вариантом.

Таким образом, один и тот же сеанс можно перераспределять с учётом местонахождения состояния и вычислительных ресурсов. Приложение сохраняет непрерывный контекст, а сервис выбирает между продолжением с локальным состоянием, миграцией состояния и повторным вычислением в зависимости от того, когда каждый из этих трёх путей позволит начать последующее выполнение модели. В главе 8 пригодный для повторного использования префикс определялся по содержимому; в этом разделе решение о маршрутизации дополнительно учитывает время ожидания и пропускную способность канала.

После перехода на 200 GbE получение для одного запроса явно выигрывает, но 16 получений в секунду требуют около 19,3 GB/s, то есть занимают примерно 77% пропускной способности канала 25 GB/s. Всплески запросов увеличат время ожидания передачи. Поэтому после расчёта требуемой пропускной способности для одного запроса необходимо

также вычислить суммарный трафик при непрерывном поступлении запросов.³⁵

В предыдущем сравнении предполагалось, что местонахождение кэша известно, а данные доступны. Обычно маршрутизатор прогнозирует местонахождение по событиям кэша, однако задержка событий или вытеснение объекта могут сделать прогноз ошибочным. Чтобы увидеть влияние такой ошибки на время ответа, изменим время ожидания A на 80 ms и предположим, что кэш действительно доступен лишь с вероятностью p . При попадании время составляет 110,9 ms, а при отказе всех уровней и вынужденном повторном вычислении — 967,2 ms, поэтому математическое ожидание равно

$$\mathbb{E}[T_A] = 110,9p + 967,2(1 - p) = 967,2 - 856,3p \text{ ms.}$$

Чтобы среднее значение было лучше 907 ms прямого повторного вычисления на B , достаточно $p > 7,0\%$. При $p = 0,9$ среднее составляет около 196,5 ms, но 10% запросов по-прежнему выполняются за 967 ms, поэтому p_{99} такого двухточечного распределения равен 967 ms и значительно превышает целевые 220 ms. Чтобы p_{99} достиг 220 ms, в этой двухточечной модели вероятность попадания должна составлять не менее 99%; значение 90% уже значительно улучшает среднее, но всё ещё отправляет каждый десятый запрос по медленному пути длительностью 967 ms.³⁶

Нагрузочный стресс-тест с одновременным выполнением двух задач ещё нагляднее показывает компромисс между приоритетом кэша и приоритетом свободного экземпляра. При приоритете кэша целевой запрос завершается примерно за 1,38 секунды, и к этому же моменту завершаются обе задачи; после переноса целевого запроса на свободный экземпляр он завершается примерно за 0,33 секунды, но обе задачи заканчиваются лишь примерно через 1,54 секунды. Целевой запрос ускоряется примерно на 1,05 секунды, тогда как момент завершения всех задач сдвигается примерно на 0,16 секунды. Перед принятием решения о маршрутизации необходимо определить, что именно оптимизируется: время ответа целевого запроса или время завершения всех задач.³⁷

После включения описанного в главе 8 восстановления состояния в решение о маршрутизации необходимо одновременно сравнивать момент освобождения экземпляра и момент готовности состояния. Рассмотрим состояние сеанса DeepSeek V4.1: пусть экземпляр A сохранил глобальный KV и состояние SWA энкодера, но имеет очередь, а экземпляр B может начать выполнение немедленно, однако должен получить глобальный KV и восстановить состояние SWA энкодера. Преимущество A — возможность повторно использовать состояние, преимущество B — немедленная доступность вычислительных ресурсов; маршрутизатор должен сравнивать ожидаемое время завершения на обоих экземплярах, а не только проверять признак попадания в кэш.

Предположим, что дальнейшая обработка нового ввода, воспроизведение окна декодера и генерация занимают на обоих экземплярах одинаковое время, а подготовительные операции выполняются последовательно. Передача глобального состояния B по расчёту из главы 7 занимает 4,666 ms, а восстановление состояния SWA энкодера, по предположению, — 8 ms, итого 12,666 ms. A сохранил оба состояния, поэтому его подготови-

³⁵Расчёты маршрутизации кэша при извлечении через 50 GbE и через 200 GbE, включая полный последовательный путь удалённое хранилище → память хоста → GPU; время вычислений при повторном вычислении и после попадания получено делением FLOPs матриц на 50% от производительности A100 80GB SXM, равной 312 TFLOP/s. Суммарная пропускная способность PCIe 4.0 для приёма и передачи у A100 составляет 64 GB/s, см. спецификацию A100 80GB.

³⁶События кэша и решения о маршрутизации, двухточечное распределение при инвалидации кэша.

³⁷Парный расчёт реальной нагрузки на маршрутизатор, различающий время завершения целевого запроса и полной пары задач; исходные условия сохранены вместе с результатом.

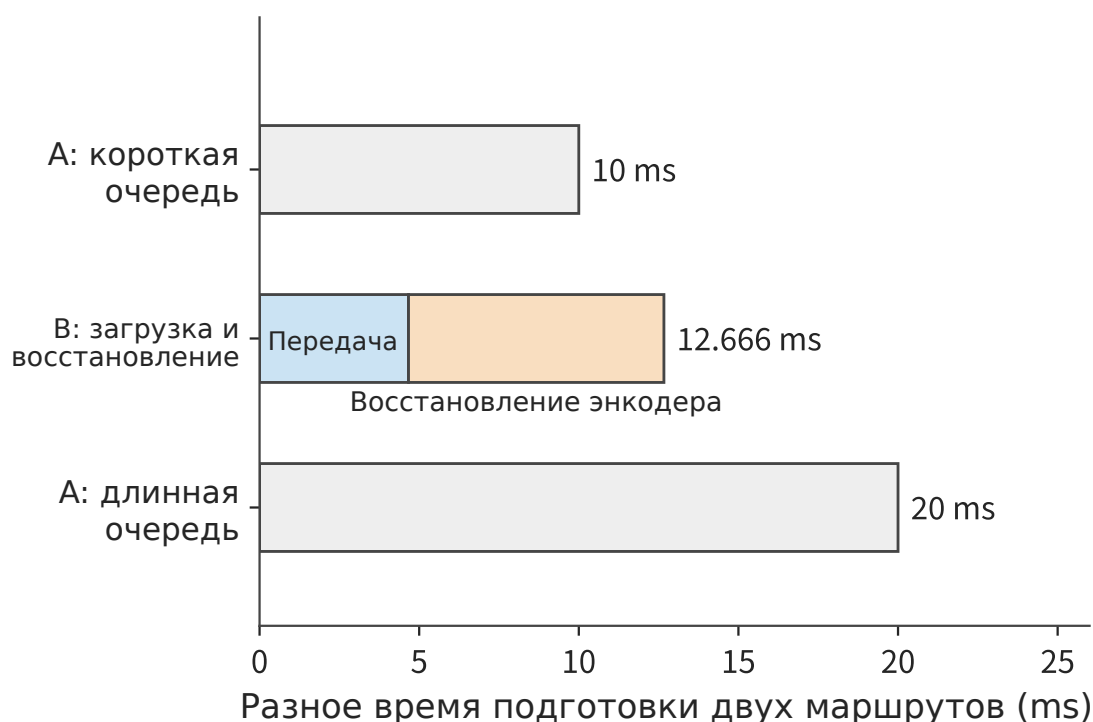


Рис. 9-41. Сравнение привязки к кэшу со свободным местом выполнения. А сохранил глобальный KV и SWA энкодера; В требуются 4,666 ms на передачу глобального состояния и предполагаемые 8 ms на восстановление энкодера. Общие для обоих вариантов последующие операции, включая воспроизведение декодера, опущены; сравнивается только разное время последовательной подготовки. Когда время ожидания А возрастает с 10 до 20 ms, более быстрым вариантом вместо А становится В. Серым показано ожидание в очереди, а остальные цветные блоки соответствуют передаче глобального состояния и восстановлению локального состояния энкодера.

тельное время равно времени ожидания в очереди: при ожидании 10 ms А раньше начинает обработку нового ввода, а при увеличении ожидания до 20 ms раньше начинает В. Таким образом, 12,666 ms становится в этом примере порогом времени ожидания, при котором меняется выбор маршрута.³⁸

9.6 Запуск сервиса, масштабирование и восстановление после сбоев

В разделах 9.2–9.5 сравнивалось, кому передавать запросы и где хранить состояние. Теперь рассмотрим изменения самих экземпляров: сколько времени занимают запуск и прогрев новых реплик, как передаётся состояние при изменении способа распараллеливания во время работы и как после отказа части ускорителей продолжить обслуживание, используя сохранённое состояние и журнал вывода.

³⁸Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт сквозного межглавного сценария.

9.6.1 Затраты на запуск и прогрев

При сравнении маршрутизации в разделе 9.5.4 предполагалось, что доступные экземпляры уже существуют. При масштабировании или восстановлении после сбоя новый экземпляр сначала должен запуститься, а ожидающие запросы тем временем продолжают поступать. Поэтому здесь запуск и восстановление состояния рассматриваются в рамках одной временной шкалы обслуживания. Новая реплика также должна завершить инициализацию процесса и токенизатора (tokenizer — компонента, преобразующего текст в номера токенов), чтение и шардирование весов, компиляцию, определение ёмкости KV и захват графа (запись шагов выполнения в CUDA Graph из раздела 5.5.2). Суммарное время этой подготовки определяет, когда новая реплика начнёт принимать часть запросов. Если первый пробный запуск иницирует компиляцию, время компиляции уже входит во время пробного запуска, поэтому общее время следует вычислять с учётом последовательности зависимостей. Поэтапный анализ исторических версий vLLM в исследовании затрат на запуск Breaking the Ice демонстрирует эту проблему; если раскрыть отношения вложенности, полное время запуска определяется самым длинным путём зависимостей от создания процесса до готовности принимать запросы.³⁹

Предположим, что подготовка графа выполнения увеличивает время запуска на T_s , но затем позволяет экономить δ на каждом шаге выполнения. Чистая выгода после N шагов составит $N\delta - T_s$. Если запуск удлинится на 10 секунд, а каждый шаг экономит 0,2 мс, равновесие наступит только через 50 000 шагов, и лишь начиная с 50 001-го шага накопленная экономия времени превысит дополнительные затраты на запуск. Если на каждом шаге 32 запроса генерируют по одному токеноу, то 50 000 шагов соответствуют примерно 1,6 миллиона выходных токенов. Время экономится один раз на каждое выполнение batch, поэтому накопленную выгоду следует рассчитывать по числу выполнений batch.

Поступающие во время запуска запросы также образуют очередь. Если исходный сервис полностью недоступен, интенсивность поступления остаётся равной λ , а время запуска составляет T_s , то к его завершению накопится примерно λT_s запросов. Если после готовности пропускная способность сервиса равна $\mu > \lambda$, то в идеальной модели непрерывного потока дополнительное время опустошения очереди составит

$$T_{drain} = \frac{\lambda T_s}{\mu - \lambda}.$$

Рассмотрим гетерогенный кластер из этой главы: запуск занимает 10 секунд, в течение которых поступает 3,5 запроса в секунду, поэтому к моменту готовности накапливается 35 запросов. После готовности прямое PD-разделение может обслуживать примерно 4,55 запроса в секунду, но 3,5 из них приходится на новые запросы, поэтому фактически за секунду обрабатывается лишь около 1,05 накопленного запроса. Для полного опустошения очереди потребуется ещё примерно 33 секунды, то есть она исчезнет на 43-й секунде от начала запуска. При верхней границе 4,17 запроса/с для совместного размещения с идеальным блочным совмещением за секунду можно обрабатывать лишь около 0,67 накопленного запроса, поэтому очередь исчезнет только на 63-й секунде. При совместном размещении без разбиения пропускная способность составляет всего 3,02 запроса/с, и очередь будет непрерывно расти. Все три кривые показаны на рис. 9-42.

³⁹ Обзор исследования Breaking the Ice; в исследовании используется vLLM v0.10.1.1.

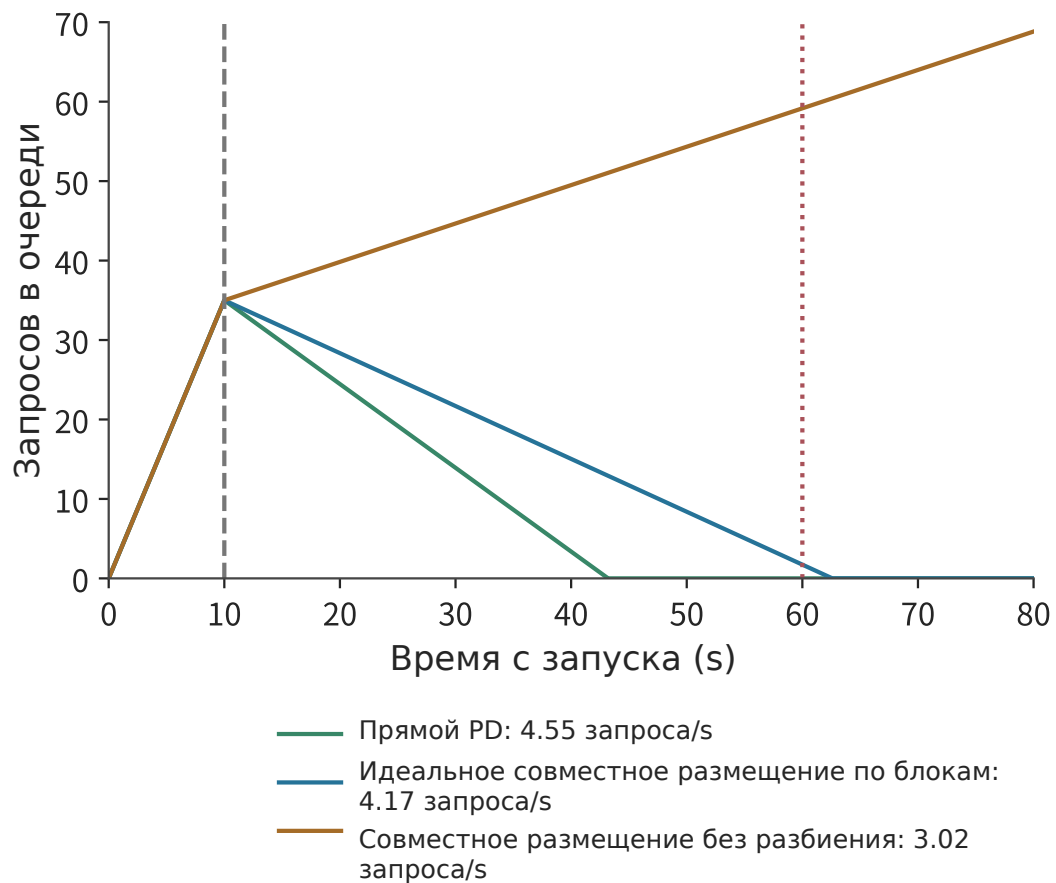


Рис. 9-42. Резерв пропускной способности сервиса определяет скорость сокращения накопившейся при запуске очереди. В модели непрерывного потока поступает 3,5 запроса в секунду, и после 10 секунд запуска накапливается 35 запросов. После готовности скорости обслуживания составляют соответственно 4,55 запроса/с для прямого PD-разделения, 4,17 запроса/с для совместного размещения с идеальным блочным совмещением и 3,02 запроса/с для совместного размещения без разбиения. В первых двух случаях очередь опустошается на 43-й и 63-й секундах от начала запуска, а в последнем продолжает расти. Крайний срок опустошения очереди в примере 9.8 — 60-я секунда.

После готовности накопившиеся запросы зачастую поступают одновременно, и тогда конкурентная предварительная выборка может изменить порядок начала их выполнения. Когда один запрос длиной 1024 токена выполнялся отдельно, он повторно использовал 1008 токенов. Когда одновременно поступили восемь запросов, первые четыре зарегистрировали предварительную выборку по 1024 токена каждый, суммарно заняв 4096 токенов, а для остальных запросов предварительная выборка не была зарегистрирована из-за превышения лимита ёмкости. Первым завершился как раз один из последних запросов: он не получил ни одного попадания в кэш и сразу начал `prefill`; поскольку ему не пришлось ждать кэш, он раньше попал в очередь выполнения. Таким образом, даже при наличии объекта в кэше запрос сначала должен получить пространство, необходимое для предварительной выборки, и лишь затем сможет воспользоваться этим кэшем. Когда пространство для предварительной выборки заканчивается, последующие запросы сразу выполняют вычисления заново, тогда как запросы с уже начатой предварительной выборкой продолжают ждать данные.⁴⁰

При масштабировании необходимо также учитывать соответствие версий модели оборудованию. Если веса зафиксированы на неизменяемом носителе, добавление ускорителей той же модели позволяет масштабировать обслуживание той же версии; для развёртывания новых весов необходимо подготовить ускорители, поддерживающие новую версию. Маршрутизатор распределяет запросы по версиям модели, а при планировании ресурсов следует предусматривать одновременную работу старой и новой версий и замену ускорителей. Чем дольше работает сервис с фиксированной версией и чем выше спрос, тем на большее число запросов распределяются затраты на развёртывание.

9.6.2 Реконфигурация во время работы и передача состояния

Запуск определяет, когда новый экземпляр станет доступен; при миграции во время работы целевой экземпляр должен также продолжить выполнение с достигнутой исходным экземпляром точки. Изменение степени тензорного параллелизма, экспертного параллелизма или размера пула обслуживания требует перераспределить веса и состояние запросов. Получение параметров целевыми ускорителями — лишь первый этап: может также потребоваться создать коммуникационные группы, подготовить новый граф выполнения, преобразовать раскладку и возобновить приём запросов. У этой подготовки и переноса данных есть последовательные зависимости, совместно определяющие момент, когда новые запросы можно будет передать целевому экземпляру.

При плановой миграции можно воспользоваться тем, что исходный экземпляр продолжает работать: сначала в фоновом режиме копируется уже неизменяемый контекст, а исходный экземпляр продолжает генерацию; затем он ненадолго останавливается, копируется состояние, добавленное за время миграции, и право выполнения передаётся целевому экземпляру. Рассмотрим перенос `D worker` с одного `H20`: средняя длина контекста для 32 запросов в `batch` составляет 8704 токена, а общий объём KV — около 41,1 GB. Исходный экземпляр выполняет примерно 1166 вызовов `decode` в секунду, каждый из которых добавляет KV одного токена (144 KiB), поэтому состояние растёт лишь примерно на 0,172 GB в секунду. Целевой экземпляр копирует данные через сетевую карту

⁴⁰Наблюдение ветвей запросов `HiCache`, где цепочка событий с одним ID запроса объясняет лимит и эффективное попадание. В исходной записи лимит предварительной выборки составлял 3289 токенов, а зарегистрированный занятый объём — 4096 токенов; клиент отправил восемь запросов одновременно, но на GPU одновременно выполнялся не более чем один запрос.

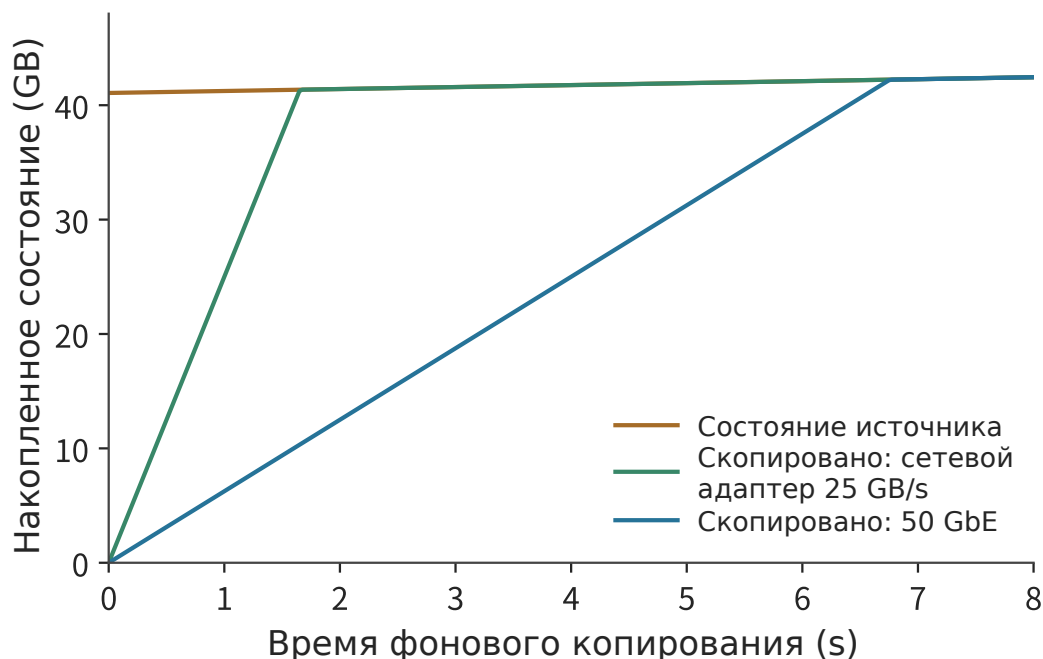


Рис. 9-43. Фоновое копирование должно догнать продолжающее расти состояние. Вначале необходимо скопировать 41,1 GB состояния, а исходный экземпляр добавляет примерно 0,172 GB в секунду. Целевой экземпляр догоняет его примерно за 1,65 секунды через сетевую карту со скоростью 25 GB/s и примерно за 6,76 секунды через 50 GbE со скоростью 6,25 GB/s. До пересечения кривых расстояние между ними по вертикали равно объёму ещё не скопированных данных; после пересечения целевому экземпляру остаётся лишь следовать за новым состоянием исходного.

200 Gbit/s из этой главы со скоростью 25 GB/s, и отставание сокращается примерно на 24,8 GB/s. Копирование завершается приблизительно через 1,65 секунды, а за это время добавляется около 0,28 GB состояния; при наличии лишь одного порта 50 GbE (6,25 GB/s) потребуется около 6,76 секунды. Скорость добавления KV при decode значительно ниже пропускной способности сетевой карты, поэтому время наверстывания почти полностью определяется исходными 41,1 GB. Как только скорость копирования сравняется со скоростью роста состояния, отставание перестанет сокращаться. Последняя точка передачи одновременно фиксирует версию состояния и право выполнения, чтобы целевой экземпляр не пропустил последние обновления исходного.

На рис. 9-43 кривая исходного экземпляра также поднимается, поэтому скорость наверстывания равна разности наклонов двух кривых; в этом примере рост при decode очень медленный, поэтому кривая исходного экземпляра почти горизонтальна. Если скорость копирования в точности равна скорости роста состояния, линии параллельны и исходное отставание не сокращается. Окончательную передачу необходимо выполнять в момент, когда целевой экземпляр уже догнал исходный и состояния обеих сторон совпадают.

Другой вид изменения во время работы, помимо миграции, — переключение способа распараллеливания: реорганизация одной и той же группы ускорителей в TP или SP×TP. В данном случае TP разделяет внутрислойные матрицы, а SP распределяет часть вычислений и промежуточного состояния по позициям токенов. ArcticInference, плагин инференса для vLLM с открытым исходным кодом от Snowflake, и соответствующие примеры

vLLM демонстрируют этот подход: переключение изменяет набор участвующих в каждом шаге ускорителей и распределение токенов, а также требуемые раскладки весов и KV и граф выполнения. Предварительное хранение весов и графов выполнения для обоих режимов сокращает ожидание при переключении, но дополнительные веса и графы уменьшают пространство, доступное для KV. При эластичном экспертном параллелизме (Elastic EP — изменении во время работы числа ускорителей в группе экспертного параллелизма) новые ускорители при расширении группы сначала должны получить веса экспертов и лишь затем смогут выполнять назначенные им задачи; для обработки запросов целиком также требуется соответствующее состояние внимания.⁴¹

Данные на обеих сторонах совпадут лишь после того, как целевой экземпляр догонит обновления состояния исходного. Кроме того, целевой экземпляр должен завершить подготовку коммуникационных групп и графа выполнения. Ниже объём данных остаётся неизменным, меняется только пропускная способность передачи. В примере миграции Qwen3-8B при переключении с TP4 на TP8 по сети необходимо передать около 16,4 GB, а для локального построения целевых тензоров — прочитать и записать ещё около 4,7 GB данных. Передача одной и той же сетевой нагрузки через один порт 50 GbE (6,25 GB/s) и сетевую карту 200 Gbit/s (25 GB/s) из этой главы занимает соответственно около 2,63 и 0,66 секунды — разница четырёхкратная.

Однако если дополнительно предположить, что последовательное создание коммуникационных групп, подготовка графа выполнения и возобновление приёма запросов после передачи занимают 9 секунд, общее время составит примерно 11,6 и 9,7 секунды, то есть сократится лишь приблизительно на 17%. Если все восемь ускорителей находятся в одном сервере 8×A100 и миграция выполняется через NVLink, то при пропускной способности каждого A100 300 GB/s в каждом направлении ускоритель с наибольшим объёмом отправки должен передать около 4,7 GB, что займёт лишь примерно 15,7 мс, но общее время всё равно составит около 9,0 секунды. Эти 9 секунд ограничивают пользу от дальнейшего увеличения пропускной способности: даже если время сетевой передачи стремится к нулю, общее время может приблизиться лишь к 9 секундам. Если заранее создать коммуникационные группы и подготовить графы выполнения, ожидание во время миграции можно дополнительно сократить.⁴²

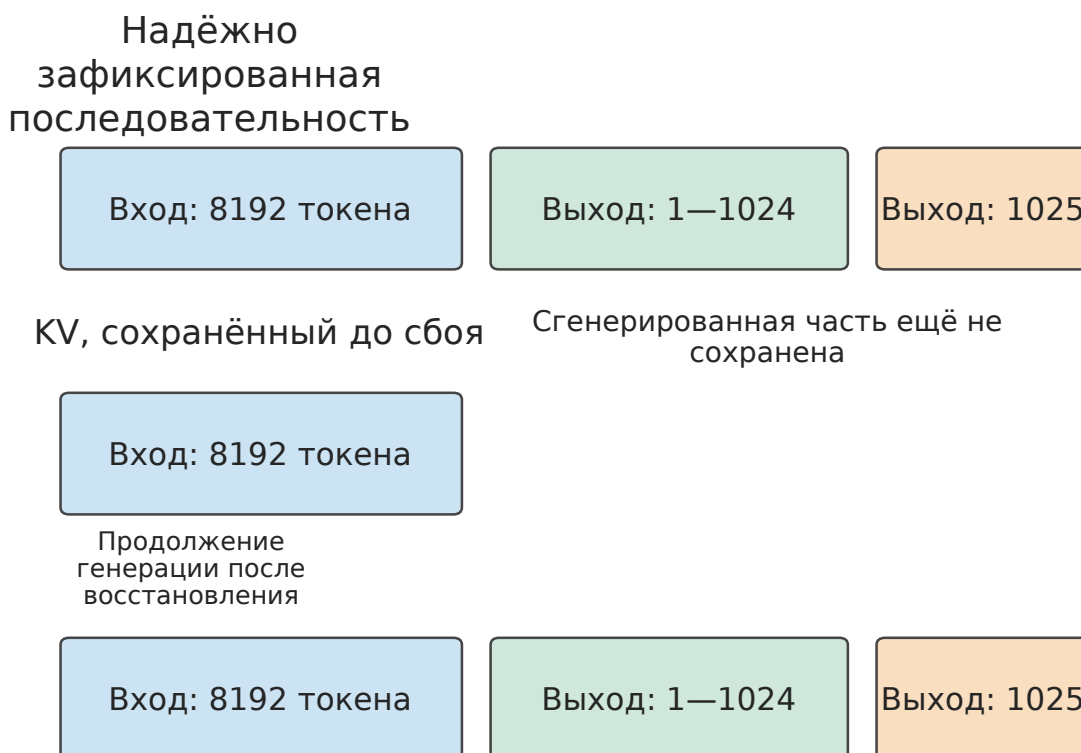
На этом этапе сам перенос уже близок к физическому нижнему пределу канала, а всё оставшееся время уходит на подготовку программного обеспечения. Согласно критерию из раздела 1.3.4, такой разрыв нельзя устранить более быстрым каналом — необходимо перестроить программное обеспечение: заранее создавать коммуникационные группы, предварительно захватывать графы выполнения, отделить процессы от планирования и выполнять подготовку параллельно с передачей.

9.6.3 Частичные сбои и восстановление потоковой генерации

При плановой миграции исходный экземпляр ещё может предоставить последнее обновление; после сбоя его данные могут оказаться недоступны для чтения, и тогда оста-

⁴¹Динамический параллелизм и условия состояния, распределение экспертов и масштабирование.

⁴²Расчёты последовательной миграции через 50 GbE, через сетевой адаптер 200 Gbit/s и через A100 NVLink, задающие нижнюю границу времени миграции по объёму полезной нагрузки, пропускной способности канала и девяти последовательным операциям подготовки длительностью по 1 секунде. Суммарная пропускная способность NVLink для приёма и передачи у A100 SXM составляет 600 GB/s, см. спецификацию A100 80GB. Число байтов KV для фоновое копирования и интенсивность вызовов decode взяты из `derived_stage_rates` для H20 в целочисленном распределении гетерогенных P и D.



Довычислить 1024 токена → KV до 9216 → обработать выход 1025

Рис. 9-44. Журнал вывода определяет, какую последовательность следует продолжить, а checkpoint KV — с какого места нужно повторить вычисления. Предполагается, что все 1025 выходных токенов надёжно записаны, но KV сохранён лишь для исходного ввода. В нижней части отдельно показаны исходный ввод, первые 1024 выходных токена и 1025-й выходной токен; ширина сегментов не пропорциональна числу токенов.

ётся полагаться лишь на заранее сохранённое состояние и журнал вывода. Сначала при восстановлении необходимо определить, какую часть вывода пользователь уже получил. После сбоя возможны три цели: восстановить достаточно состояния для продолжения decode; заново выполнить prefill по уже определённым токенам; повторно сэмплировать часть вывода. В первых двух случаях генерация продолжается после уже полученной пользователем последовательности, а в третьем случайный выбор выполняется заново, поэтому суффикс может отличаться. Для rollout в RL это также меняет процесс генерации образца.

Предположим, что исходный ввод содержит 8192 токена, пользователю уже возвращено 1025 выходных токенов, но последний сохранённый KV охватывает лишь исходный ввод. Если эти выходные токены надёжно записаны, при восстановлении достаточно подать первые 1024 выходных токена в модель в рамках одного prefill, заново построить KV для 9216 токенов, а затем обработать 1025-й выходной токен и продолжить генерацию. Если уже имеется совместимый KV, охватывающий 9216 токенов, повторное воспроизведение этих 1024 токенов можно пропустить.

На рис. 9-44 концы двух записей показаны отдельно. Если вывод уже возвращён

пользователю, это не означает, что соответствующий KV сохранён. Используя при восстановлении записанные токены для повторного вычисления, можно воссоздать тот же префикс без повторного сэмплирования этих выходных токенов.

Журнал упреждающей записи (write-ahead log, WAL) надёжно сохраняет операции или результаты, необходимые для восстановления, до их внешнего подтверждения. В данном случае WAL на уровне токенов хранит уже определённую выходную последовательность, а KV — результаты уже выполненных для этой последовательности вычислений. Первый определяет, какую последовательность вывода следует продолжить после восстановления, а второй — какой объём работы ещё предстоит повторить. Сервис генерации DeepSeek V4 использует подобный механизм; при восстановлении также необходимо обеспечить согласованность версии весов, индексов позиций токенов и состояния decode.⁴³

Область воздействия сбоя определяется группой синхронизации. Потеря одного ускорителя в группе тензорного или экспертного параллелизма может остановить обработку запросов на всём совместно работающем экземпляре, тогда как другие неповреждённые реплики продолжают обслуживание. Восстановление после сбоя последовательно проходит через обнаружение, реконструкцию экземпляра и восстановление запросов; производительность после восстановления определяет, сколько времени потребуется для завершения накопившихся запросов. В главе 10 дополнительно анализируется влияние этих вариантов на образцы RL и ход обучения, а в главе 11 рассматривается восстановление задач, использующих инструменты и окружения.

9.7 Комплексное сравнение схем развёртывания

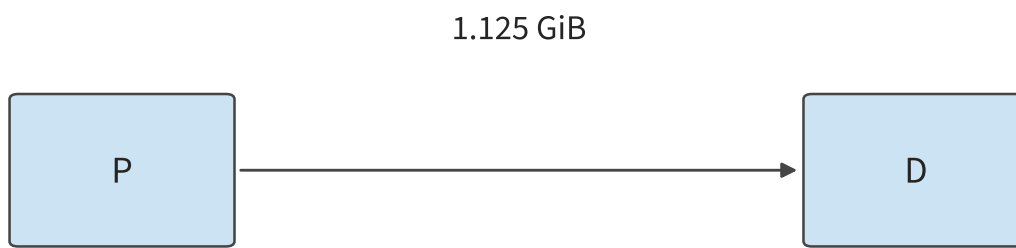
9.7.1 Сочетание реплик, PD, AF и общего KV-кэша

В этом разделе результаты предыдущих разделов применяются к описанному в начале главы сервису на восьми картах: сначала определяется способ передачи состояния, а затем проверяется, способен ли сервис в целом справиться с интенсивностью поступления запросов и уложиться в срок восстановления. Созданный Р KV-кэш можно передать непосредственно D либо сначала поместить в общий пул, откуда его затем получит D. Если вновь созданное D состояние потребуется для следующего цикла Р, необходимо также убедиться, что оно сохранено и опубликовано. При прямой передаче и чтении через пул может передаваться одно и то же содержимое, однако задействованные каналы и буферы различаются.

В случае рассматриваемого в этой главе контекстного KV-кэша объёмом 1.125 GiB при прямой передаче Р→D данные перемещаются один раз, а при передаче через пул — по одному разу на каждом из рёбер Р→пул и пул→D, поэтому суммарная полезная нагрузка составляет 2.25 GiB. Если пропускная способность обоих рёбер равна 25 GB/s, а чтение можно начать только после полной записи объекта, ожидание при передаче через пул составит около 96.6 ms — примерно на 48.3 ms больше, чем при прямой передаче.

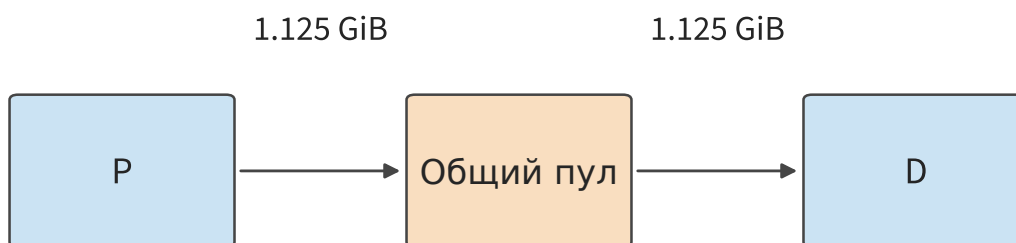
Если D использует это состояние только один раз, передача через пул не даёт преимуществ повторного использования. Если позднее обработку того же сеанса продолжит другой экземпляр, появляется возможность заменить примерно 856 ms повторного

⁴³Материалы о персистентности состояния и сервисе генерации из технического отчёта DeepSeek V4, а также заметки по чтению для этой главы.



Р напрямую передаёт полное состояние D

Рисунок 9-45. Р напрямую передаёт D контекстный KV-кэш объёмом 1.125 GiB, выполняя лишь одну прямую передачу. Р выполняет prefill, а D — последующий decode по одному токenu.



D читает после полной записи и уведомления

Рисунок 9-46. Р сначала записывает в пул полный объект объёмом 1.125 GiB и публикует его, после чего D получает тот же объект; итого выполняются две передачи: запись и получение. Последующие экземпляры также могут повторно использовать объект из пула. Р выполняет prefill, а D — последующий decode по одному токenu.

вычисления временем чтения около 48.3 ms — 856 ms составляет время prefill из раздела 9.2.3. Сэкономленное вычислительное время с избытком компенсирует дополнительное время передачи; в разделе 9.7.3 эта выгода рассчитывается для нагрузки из примера 9.8. Общий пул позволяет использовать результат вычислений, оставшийся от одного запроса сеанса, в последующих запросах; AF же изменяет место выполнения каждого слоя текущего запроса. Эти механизмы влияют на разные участки временной шкалы.

Те же правила применимы и к визуальному вводу. Создаваемый E кэш ЕС имеет идентификатор, отличный от идентификатора языкового KV-кэша, и занимает другой объём. Попадание в кэш изображений может сократить работу E, но не устраняет автоматически последующую стадию P языковой модели. В графе выполнения E→P→D попадание в ЕС снижает нагрузку на E, а попадание языкового префикса сокращает работу P; распределение ресурсов также корректируется вслед за изменением объёма работы на каждой стадии.

9.7.2 Сравнение схем при одинаковом качестве и одинаковых ограничениях по ресурсам

Пример 9.8. Какая схема развёртывания инференса способна непрерывно обслуживать запросы и вовремя устранить очередь, накопившуюся при запуске? Используем ресурсы и нагрузку из начала главы: четыре A100 80GB SXM и четыре H20 SXM5 96GB; каждую секунду поступает 3.5 независимого запроса, в каждом запросе 8192 входных токена и 1025 выходных токенов. Производительность стадий берётся из примера 9.2; если обе стадии выполняются на одной карте, они занимают её последовательно. Между двумя серверами имеется общий канал полезной нагрузки с пропускной способностью 25 GB/s, и все передачи используют одну и ту же полосу этого канала. В пределах сравниваемого окна модель и настройки генерации одинаковы, а нужные запросам префиксы не переиспользуются. Сервис запускается из остановленного состояния, и все три схемы развёртывания готовы через 10 секунд; после готовности очередь обрабатывается по модели непрерывного потока. Цель — непрерывно обрабатывать новые запросы и устранить накопившуюся при запуске очередь не позднее чем через 60 секунд после начала запуска.

Необходимо проверить и ограничение по ёмкости. После 1024 итераций decode KV-кэш каждого запроса охватывает 9216 токенов и занимает около 1.27 GiB. Одна H20 при batch size 32 хранит состояния активных запросов общим объёмом около 40.5 GiB; если добавить один приёмный буфер объёмом 1.125 GiB и примерно 15.3 GiB весов, получится около 56.9 GiB, что помещается в 96 GB видеопамати, то есть примерно 89.4 GiB. При использовании компактного состояния MLA каждый экземпляр занимает около 618 MiB, а 32 экземпляра вместе с одним приёмным буфером объёмом 549 MiB — около 19.8 GiB. На стороне A100 достаточно одновременно хранить два состояния — вычисляемое и отправляемое, всего 2.25 GiB. Схема с общим пулом дополнительно располагает 64 GiB для состояний. Для ещё не запущенных запросов сохраняются только входные данные; вычисление начинается после выделения места в кэше.

В следующей таблице сравниваются полные реплики, PD с прямой передачей и PD с передачей через общий пул. При использовании общего пула чтение начинается только после полной записи объекта, поэтому данные каждого запроса дважды проходят через общий канал; как при записи в пул, так и при чтении из него может использоваться вся пропускная способность этого канала.

Схема развёртывания	Организация P и D	Нагрузка на канал на запрос	Вычислительная производительность, запросов/s	Производительность канала, запросов/s	Общая пропускная способность после запуска, запросов/s
Восемь полных реплик	Каждая карта выполняет P и D	0	3.02	—	3.02
Прямой PD	Четыре A100 выполняют P, четыре H20 — D	1.125 GiB	4.55	20.7	4.55
PD с передачей через общий пул	То же распределение P и D, P→пул→D	2.25 GiB	4.55	10.3	4.55
Прямой PD, компактное состояние MLA	Четыре A100 выполняют P, четыре H20 — D	549 MiB	4.55	43.4	4.55
PD с передачей через общий пул, компактное состояние MLA	То же распределение P и D, P→пул→D	1.07 GiB	4.55	21.7	4.55

Полные реплики могут завершать лишь 3.02 запроса в секунду, что постоянно ниже интенсивности поступления в 3.5 запроса в секунду, поэтому этот вариант исключается первым. Производительность канала в обеих схемах PD значительно выше 4.55 запроса/s, поэтому обе ограничены вычислительным пулом. При входной нагрузке 3.5 запроса в секунду прямая передача потребляет около 4.2 GB/s, а передача через пул — около 8.5 GB/s; на стадии устранения очереди при 4.55 запроса/s им требуется соответственно около 5.5 и 11.0 GB/s. Для компактного состояния MLA эти четыре значения составляют соответственно 2.0, 4.0, 2.6 и 5.2 GB/s. Производительность канала удваивается, но вывод не меняется.

У обеих схем одинаковая пропускная способность в установившемся режиме, однако для этой группы запросов лучше подходит прямой PD. Последующие запросы не переиспользуют префиксы этой группы, тогда как передача через пул удваивает объём данных в канале и добавляет каждому запросу около 48.3 ms ожидания. Общий пул объёмом 64 GiB вмещает не более 56 полных префиксов по 1.125 GiB, а при компактном состоянии MLA — 119 префиксов; при данной нагрузке ни один сохранённый экземпляр больше не будет использован. Прямая передача выполняет ту же работу, оставляет больший запас пропускной способности канала и сокращает ожидание передачи. Поэтому в этом примере выбираются четыре A100 для P, четыре H20 для D и прямая передача P→D.

Теперь проверим, сможет ли эта схема вовремя обработать запросы, накопившиеся за время запуска. Согласно расчёту устранения очереди из раздела 9.6.1, прямой PD устраняет очередь из 35 запросов, накопленную за 10 секунд запуска, примерно через 43 секунды после начала запуска и тем самым укладывается в целевые 60 секунд. У полных реплик после готовности очередь продолжает расти примерно на 0.5 запроса в секунду, поэтому к 60-й секунде она увеличивается с 35 примерно до 59 запросов. Даже при идеальной обработке блоками с производительностью 4.17 запроса/s совместное размещение устранил очередь лишь к 63-й секунде и также не уложится в срок.

Можно также рассчитать минимальную производительность сервиса, необходимую для соблюдения этого срока. Запуск занимает 10 секунд, а за оставшиеся 50 секунд требуется устранить очередь из 35 запросов, продолжая при этом обрабатывать по 3.5 нового запроса в секунду. Следовательно, необходимо:

$$\mu - 3.5 \geq \frac{35}{50}, \quad \mu \geq 4.2 \text{ запроса/s.}$$

Производительность прямого PD, равная 4.55 запроса/s, превышает этот минимум примерно на 8%. Если фактическая эффективность kernel составляет лишь 90% от значения, принятого в примере 9.2, производительность пула D снижается примерно до 4.10 запроса/s. В установившемся режиме он по-прежнему способен обрабатывать непрерывно поступающие запросы, но очередь будет сокращаться лишь примерно на 0.60 запроса в секунду и исчезнет только приблизительно к 68.5-й секунде, то есть позже установленного срока. Наклон двух нисходящих линий на рисунке 9-42 представляет именно чистую скорость устранения очереди, равную разности между производительностью сервиса и интенсивностью поступления запросов.

Убедившись, что очередь будет устранена вовремя, можно перейти к сравнению стоимости. При расчёте стоимости вычислительного сервиса необходимо учитывать и способ тарификации ресурсов. Пусть восемь карт зарезервированы с почасовой оплатой и их суммарная стоимость составляет восемь юаней в час. Если фактически завершается 3.5 запроса в секунду, за час будет завершено 12600 запросов, а стоимость составит около

0.63 юаня за тысячу запросов. Если интенсивность поступления остаётся равной 3.5 запроса в секунду, производительность 4.55 запроса/s не создаёт дополнительные запросы из ничего; избыточная производительность используется для обработки всплесков и очереди, накопившейся при запуске.

При сравнении количества запросов, обрабатываемых под полной нагрузкой, производительности 3.02 и 4.55 запроса/s соответствуют стоимости около 0.74 и 0.49 юаня за тысячу запросов. Если бизнес дополнительно устанавливает срок завершения, а у второй схемы ему соответствует лишь четверть запросов, число соответствующих требованиям запросов снижается примерно до 1.14 в секунду, а стоимость тысячи соответствующих требованиям запросов возрастает примерно до 1.95 юаня. В общем виде:

$$C_{effective} = \frac{\text{общая стоимость за период измерения}}{\text{число запросов или задач, удовлетворяющих требованиям к качеству и сроку}}$$

Числитель зависит от вычислительных ресурсов, оперативной памяти, сети и поддерживаемых в рабочем состоянии резервных экземпляров, а знаменатель — от фактического числа поступивших и успешно завершённых запросов. Например, агент может завершить работу досрочно, сгенерировав меньше токенов и используя меньше ресурсов, но не выполнить задачу; такой выход не учитывается в числе успешно завершённых задач, соответствующих требованиям.⁴⁴

9.7.3 Корректировка развёртывания после изменения нагрузки

Теперь изменим условие примера 9.8, согласно которому префикс больше не переиспользуется. Пусть другой экземпляр один раз переиспользует оставшийся после каждого запроса контекстный KV-кэш объёмом 1.125 GiB. Повторное вычисление этих 8192 токенов на A100 занимает 0.856 секунды, а получение состояния — всего 48.3 ms. Дополнительные 48.3 ms, ранее затраченные на передачу через пул, вместе с 48.3 ms последующего получения составляют около 96.6 ms, что значительно меньше 856 ms повторного вычисления. Поэтому даже одно последующее переиспользование даёт чистую экономию около 760 ms. У общего пула появляется конкретное назначение: заменять повторные вычисления более быстрым чтением.

Эта выгода одновременно повышает требования к каналу. При исходной передаче выполняются одна запись и одно чтение, а при последующем переиспользовании — ещё одно чтение, поэтому всего передаётся 3.375 GiB. При поступлении 3.5 таких пар запросов в секунду требуется около 12.7 GB/s; канал способен обслуживать не более примерно 6.9 пары запросов/s. Для компактного состояния MLA три передачи составляют в сумме 1.61 GiB, а 3.5 пары запросов в секунду требуют около 6.0 GB/s; канал способен обслуживать не более примерно 14.5 пары запросов/s. По сравнению с однократной передачей последующее переиспользование увеличивает объём чтения и потребляет имевшийся запас пропускной способности. Общий пул устраняет повторные вычисления, но одновременно снижает максимальную пропускную способность канала по парам запросов.

⁴⁴Ёмкость, общий канал, время запуска, цель очистки очереди и стоимость в примере 9.8, а также доля выполнения целевого уровня обслуживания (SLO), являются учебными предположениями для вывода последствий изменения условий. Удельная стоимость вычисляется делением полной часовой стоимости на 3600×эффективную интенсивность запросов. В примере миграции последовательное время подготовки принято равным 9 секундам.

Общий пул изменяет число передач. Далее предположим, что у D нет кэша, и будем изменять только объём входных данных, который должен повторно вычислить Р, чтобы проверить, требуется ли скорректировать распределение ресурсов. Допустим, Р локально нашёл в кэше 6144 токена, а кэш D по-прежнему пуст. Согласно расчётам из раздела 9.2.4, суммарная производительность полных реплик составляет около 4.90 запроса/s, исходная конфигурация с четырьмя A100 для Р и четырьмя H20 для D по-прежнему обеспечивает 4.55 запроса/s, а конфигурация с двумя A100 для Р и оставшимися шестью картами для D — 5.69 запроса/s. При тех же 10 секундах запуска и интенсивности поступления 3.5 запроса/s эти три конфигурации устранят очередь соответственно примерно к 35.1-й, 43.2-й и 26.0-й секунде после начала запуска. После попадания префикса в кэш полные реплики устраняют очередь даже раньше исходной конфигурации PD, поэтому прежнее распределение больше не подходит. Для этой нагрузки оптимально использовать две A100 для Р, а остальные шесть карт — для D.

Теперь вернёмся к входным данным без попаданий в кэш и увеличим выход до 4097 токенов, то есть рассмотрим запрос с более длительным процессом инференса. Оптимальным становится распределение с двумя A100 для Р и оставшимися шестью картами для D, но даже оно обеспечивает лишь около 1.26 запроса/s. При любом изменении соотношения числа Р и D исходных восьми карт недостаточно для обработки 3.5 запроса в секунду. Три копии такого восьмикарточного развёртывания обеспечивают суммарно около 3.78 запроса/s, чего едва хватает для непрерывной обработки поступающих запросов. Если требуется также соблюдать прежние сроки запуска и устранения очереди, необходимая производительность составляет не менее 4.2 запроса/s, а четыре копии обеспечивают около 5.04 запроса/s, поэтому потребуется четыре группы. При масштабировании исходной комбинации вычислительных ресурсов целыми группами для непрерывной обработки поступающих запросов нужны 24 карты, а для соблюдения срока восстановления — 32 карты. Дополнительные вычислительные ресурсы используются для ускоренного сокращения очереди.

Отсюда следует и порядок применения приведённых в этой главе расчётов к новой системе. Сначала объём работы на запрос преобразуется в требования к каждому виду ресурсов. Затем при целочисленном распределении ресурсов определяется наиболее загруженный пул ресурсов или исполнительный модуль. После этого в порядке выполнения учитываются время передачи состояния и объём занимаемой памяти. Наконец, остаточная производительность, равная разности между производительностью сервиса и интенсивностью поступления запросов, используется для вычисления продолжительности всплеска нагрузки или очереди, накопившейся при запуске. Если узким местом является D, добавление Р не сократит очередь. Если узким местом является общий канал, добавление вычислительных реплик лишь приведёт к тому, что больше запросов будут ожидать получения данных.

Здесь пример PD на восьми картах пересекается с примером экспертов MoE. В первом случае разные стадии назначаются вычислительным ресурсам, лучше подходящим для этих стадий. Во втором повторное использование внутри batch сокращает чтение весов, а реплики экспертов распределяют работу наиболее загруженных исполнительных модулей. Общий KV-кэш сохраняет уже вычисленные результаты для повторного использования следующим запросом, а запуск и восстановление определяют, когда эти вычислительные ресурсы смогут начать обслуживание. Выбор схемы развёртывания основывается на том, насколько все эти подходы в совокупности изменяют объём вычислений и время ожидания, а также сколько времени потребуется для устранения очереди.

Ускорение отдельной стадии также изменяет соотношение экземпляров. Пусть пропускной способности канала достаточно, каждый экземпляр Р обрабатывает 20 запросов в секунду, а каждый экземпляр D завершает 5 таких же запросов в секунду. При одном Р на четыре D производительность обоих пулов одинакова. Если ускорить стадию D в четыре раза и сохранить прежнее соотношение, производительность пула D возрастёт до 80 запросов/s, тогда как производительность пула Р останется равной 20. При переходе к одному Р на один D оба пула обеспечат 20 запросов/s, а три экземпляра D освободятся. Таким образом, выигрыш от ускорения одной стадии выражается в сокращении ресурсов, необходимых для выполнения той же работы.

9.8 Упражнения

Упражнения последовательно охватывают повторение расчётов, изменение условий и самостоятельное проектирование. Первые две задачи устанавливают общую единицу измерения запросов, в задачах с третьей по девятую поочерёдно меняются условия вычислений, коммуникации и состояния, десятая посвящена проектированию новой рабочей нагрузки, а в одиннадцатой состояние, передаваемое между этапами, заменяется на MLA и оценка выполняется заново. Задачи с пометкой «ключевая» охватывают несколько уровней ресурсов.

9-1. Число вызовов и передача состояния для трёх вариантов развёртывания инференса. Для запроса с 8192 входными и 1025 выходными токенами изобразите полную реплику, PD и общий KV-кэш, отдельно указав число вызовов на запрос, место создания KV и передачу состояния между экземплярами. Затем измените длину вывода на 1 токен и укажите, какие операции больше не требуется выполнять. Наконец, рассмотрите продолжение генерации после однократного вызова инструмента и отметьте позицию последнего уже возвращённого токена, для которого соответствующий KV ещё не создан.

9-2. Как узкое место канала и состав запросов изменяют соотношение PD. Повторите расчёт 25 целочисленных вариантов распределения из примера 9.2 и задайте эффективную пропускную способность канала равной величине, необходимой для передачи ровно одного снимка размером 1.125 GiB в секунду, то есть примерно 1.21 GB/s. Найдите новый верхний предел общей пропускной способности; когда интенсивность поступления запросов в точности равна этому пределу, однократно добавьте ещё десять запросов и определите изменение очереди во времени. Затем повторно проверьте сценарии с попаданием в префиксный кэш и со 129 выходными токенами. Наконец, измените предположение об эффективности этапов с 50% до 40% от пиковой производительности, заново выведите возможности двух типов ускорителей по prefill и decode и определите, по-прежнему ли конфигурация из четырёх A100 для Р и четырёх H20 для D обеспечивает пропускную способность выше интенсивности поступления 3.5 запроса/s.

9-3. Как batch size эксперта, набор инструкций и разрядность весов изменяют узкое место выполнения на CPU. Используя форму эксперта из примера 9.3, отдельно для kernel на AVX-512 и AMX определите число токенов, поступающих каждому эксперту в точке перехода выполнения на CPU от ограничения чтением к ограничению вычислениями; затем замените пропускную способность памяти на межсокетную, равную 125 GB/s, и пересчитайте обе критические величины. Сравните время выполнения при поступлении каждому эксперту 1 и 128 токенов. Измените размер весов на один байт

на элемент, сохранив эффективную вычислительную производительность CPU, заново найдите точку пересечения времени чтения и вычисления и объясните направление её смещения.

9-4. Объём передачи, число запусков и перекрытие коммуникации для PD и AF (ключевая). Используйте Qwen3-8B, задайте эффективную пропускную способность передачи состояния равной 25 GB/s и при накладных расходах запуска 1, 5 и 20 μ s сравните одну передачу объёмом 1.125 GiB с 72 передачами того же суммарного объёма. Затем используйте фактический размер полезной нагрузки AF из примера 9.4 и вычислите суммарное время передачи состояния для полного запроса с 8192 входными и 1025 выходными токенами.

Дополнительно рассмотрите четыре независимых micro-batch, для каждого из которых вычисления на стороне attention и feed-forward занимают соответственно 2 ms и 3 ms. Сравните полное время полностью последовательного выполнения с идеальным двухэтапным конвейером, после чего определите, насколько суммарно могут увеличить время критического пути дополнительные коммуникации и снижение эффективности вычислений, чтобы конвейерная схема всё ещё оставалась быстрее последовательной.

9-5. Ограничения ёмкости весов и вычислительной производительности CPU при гетерогенном инференсе. По сохранённой конфигурации DeepSeek V4-Flash или Kimi K2 восстановите размещение весов, размер буфера GPU и требования к оперативной памяти. После увеличения конкурентности запросов отдельно спрогнозируйте число задействованных экспертов, количество задач каждого эксперта и наиболее загруженный NUMA-узел. Затем сконструируйте условия поступления запросов, при которых суммарные веса помещаются в доступную память, но интенсивность поступления превышает вычислительную производительность CPU, и выведите скорость роста очереди.

9-6. Ограничения ёмкости реплик экспертов и окупаемость копирования. Пусть восемь экспертов получают соответственно [32, 16, 8, 4, 2, 1, 1, 0] строк входных данных. Сравните объём вычислений в трёх случаях: обрабатываются только фактические строки; входы непустых экспертов дополняются до 32 строк; входы всех экспертов дополняются до 32 строк. Затем при условиях примера 9.6 найдите минимальное число батчей, после которого накопленная экономия времени впервые превысит время копирования; измените дополнительную доступную ёмкость каждого ускорителя на 32 MiB и объясните, почему сначала следует исключить нереализуемые варианты репликации. Если горячие точки меняются со временем, объясните, как на основе нагрузки в окне наблюдения спрогнозировать накопленную выгоду после репликации и сравнить её с затратами на миграцию.

9-7. Условия выгоды сохранения, извлечения и повторного вычисления KV (ключевая). Дан префиксный KV размером 1.125 GiB, который последующие запросы должны суммарно использовать 100 раз. Сравните полное время для трёх вариантов: однократное повторное вычисление с последующим резидентным хранением, однократное извлечение с последующим резидентным хранением и удалённое чтение при каждом использовании. Дополнительно учтите время записи и срок хранения кэша между двумя запросами и выведите условие, при котором сохранять KV выгоднее, чем вычислять его повторно; разработайте пример, в котором после изменения доступной ёмкости НВМ состояние требуется вытеснить из НВМ. Используя имеющиеся записи двух движков, объясните, какие возможности повторного использования сохранились при увеличении общего CPU-пула с 4 GiB до 8 GiB.

Затем выполните расчёт для конфигурации одной GPU в DGX A100 из раздела 9.5.2:

если интервал между двумя использованиями одного префикса составляет 5 минут, какой минимальный объём кэша требуется каждой GPU и какие уровни хранения придётся задействовать? Если новый вход содержит соответственно 128 и 512 токенов, каково суммарное время послойной предварительной загрузки из памяти хоста и сколько слов необходимо загрузить до начала вычислений, чтобы чтение полностью перекрывалось вычислениями? Наконец, исходя из ресурса D7-P5520 в 1 DWPD, определите долю новых KV, которую SSD способен принимать длительное время; какой станет эта доля при 50-процентном попадании в префиксный кэш, когда скорость создания новых KV уменьшается вдвое?

9-8. Выбор способа восстановления при отсутствии страниц KV и несовместимости версий. Объясните различие между двумя способами учёта, когда прочитан KV для 1024 токенов, но повторно использован KV только для 1008 токенов. Для отсутствующей страницы, усечённой страницы и несовместимой версии предложите условия выбора между дальнейшим ожиданием, частичным повторным вычислением и отказом от кэша. Отдельно опишите, как проверить возможность завершения запроса, восстановление данных кэша и совпадение вывода с вариантом без использования кэша.

9-9. Точки пересечения по пропускной способности при маршрутизации кэша и хвостовая задержка первого токена. Выведите значения пропускной способности, при которых удалённое извлечение по пути В занимает столько же времени, сколько локальное попадание по пути А и локальное повторное вычисление по пути В в примере 9.7. Для $p = 0.5, 0.9, 0.99$ вычислите среднее время до первого токена при выборе пути А, а также p_{99} , определённый через обратную функцию распределения. Затем предположите, что при недоступности реплики в НВМ реплика на CPU остаётся доступной, а извлечение может перекрываться с ожиданием в очереди длительностью 80 ms. Проверьте, остаётся ли применимой исходная модель двухточечного распределения.

9-10. Масштабирование PD и очереди при смешанной нагрузке с короткими и длинными выводами (ключевая). Используйте условия примера 9.8: из 3.5 запроса в секунду половина выводит 1025 токенов, а другая половина — 4097 токенов; все запросы имеют 8192 входных токена и не попадают в префиксный кэш; производительность внутри пула распределяется по долгосрочной средней рабочей нагрузке. Сначала по методу примера 9.2 найдите число D GPU-секунд для двух типов ускорителей при обеих длинах вывода и получите среднюю потребность в D на запрос, после чего переберите все варианты распределения восьми ускорителей между этапами P и D. Разрешая копировать одинаковую конфигурацию из восьми ускорителей, отдельно найдите минимальное число групп для двух целей: непрерывной обработки новых поступающих запросов; очистки очереди до 60-й секунды после начала запуска при длительности запуска 10 секунд. Наконец, сконцентрируйте поступление запросов с длинным выводом в первых десяти секундах каждой минуты, изобразите изменение очереди и объясните, какой временной интервал определяет выбор ёмкости при неизменной средней потребности.

9-11. Выбор между PD и AF для MLA и GQA. Замените передаваемое в этой главе состояние компактным представлением MLA из DeeperSeek-V3: 61 слой, $d_c = 512$, $d_r = 64$, BF16. Оставьте без изменений производительность этапов из примера 9.2, полезную нагрузку AF из примера 9.4 и накладные расходы запуска 5 μ s. Сначала найдите размер состояния в байтах для 8192 токенов и время одной передачи PD при 25 и 50 GB/s; затем найдите канальную составляющую μ_{PD} и укажите верхнее значение пропускной способности, при котором канал остаётся ограничивающим фактором. По фор-

муле $\alpha^* = (V_{KV} - V_{AF})/(71B)$ найдите критическое время запуска для обоих каналов и укажите, во сколько раз суммарные передачи AF за 1024 и 4096 шагов превышают одну передачу PD при выводе соответственно 1025 и 4097 токенов. Наконец, учтите в пуле D преобразование запросов стоимостью 2.05 GFLOPs на шаг, по эффективной производительности H20 в 74 TFLOP/s найдите изменение производительности пула D и определите, по-прежнему ли достаточно четырёх H20.

Пример решения: при каком batch size выполнение эксперта на CPU переходит от ограничения чтением к ограничению вычислениями? Когда составляющие чтения и вычисления для пути одного эксперта на CPU равны,

$$\frac{2mP_e}{C_C} = \frac{2P_e}{B_D}, \quad m = \frac{C_C}{B_D}.$$

Здесь каждый параметр BF16 занимает два байта, а на каждую строку и параметр приходится по две операции с плавающей точкой, поэтому оба коэффициента в точности сокращаются. Для kernel на AVX-512 значения 1.8 TFLOP/s и 220 GB/s дают $m \approx 8.2$: когда каждый эксперт получает менее девяти токенов, основное время уходит на ожидание весов, а при большем числе токенов — на вычисления. Для kernel на AMX с производительностью 21.3 TFLOP/s точка равенства смещается примерно к 97 токенам. Когда поток читает память другого сокета, пропускная способность снижается до 125 GB/s, а обе точки равенства смещаются примерно к 14 и 170 токенам соответственно; время чтения весов при выполнении одной строки увеличивается примерно на 76%, однако при 128 строках kernel на AVX-512 всё ещё ограничен вычислениями.

Точки равенства здесь и границы 71/72 и 688/689 строк на рис. 9-14 отвечают на разные вопросы. Первые отделяют режим ограничения CPU пропускной способностью от режима ограничения вычислениями; вторые сравнивают полные пути CPU и GPU с учётом переноса весов. Пройдя точку равенства, CPU уже ограничен вычислениями, но всё ещё может превосходить путь GPU, которому сначала требуется перенести веса, пока накопленная стоимость вычислений не превысит стоимость этого переноса.

9.9 Итоги главы

Распределённый инференс распределяет вычисления и состояние одного запроса между несколькими узлами. Для PD, AF, размещения экспертов и совместного использования KV необходимо сопоставлять преимущества локального выполнения с дополнительными передачами данных; пригодность развёртывания совместно определяется ёмкостью, скоростью обслуживания, запуском и восстановлением. Маршрутизация не только выбирает свободные вычислительные ресурсы, но и определяет, какие состояния можно использовать повторно. Одни и те же расчёты дают разные результаты для различных представлений контекста: после замены передаваемого состояния GQA на компактное MLA время передачи PD сокращается вдвое, связанная с каналом составляющая μ_{PD} удваивается, послойная передача AF не изменяется, а критическое время запуска соответственно сокращается.

После изменения аппаратной конфигурации сначала зафиксируйте запрос и схему развёртывания, определите новое узкое место, а затем заново распределите ресурсы между этапами. В следующей главе мы перейдём к обучению и рассмотрим непрерывное обновление параметров и состояния обучения, а также продвижение процесса обучения.

Системы обучения

Если модель способна генерировать текст на одной карте-ускорителе, означает ли это, что её можно обучать на той же карте?

При полнопараметрическом обучении обновляются все обучаемые веса модели. Веса Qwen3-8B в формате BF16 занимают около 16,4 GB, а при полнопараметрическом обучении с Adam в смешанной точности системе, помимо весов модели, необходимо хранить градиенты, основные веса в формате FP32 и историю градиентов, сохраняемую оптимизатором между шагами, — всего около 131 GB. Эти состояния обучения занимают примерно в восемь раз больше места, чем веса в формате BF16.

При вычислении производных методом обратного распространения необходимо считывать активации, сохранённые на прямом проходе; во время выполнения также требуются буферы для сбора параметров и преобразования форматов. При проектировании системы обучения прежде всего нужно определить, где будут храниться эти тензоры и буферы и как долго их необходимо сохранять.

Распределение состояний обучения между большим числом карт снижает нагрузку на память каждой отдельной карты, однако вычислениям часто приходится ждать данных от других карт. Перенос состояний в память хоста освобождает память GPU, но увеличивает объём передачи данных между хостом и GPU. Если обучение продолжается несколько недель, ожидание входных данных, сохранение контрольных точек и повторные вычисления после сбоев также влияют на срок завершения. Когда число карт достигает тысячи и более, сбои перестают быть редкими событиями: если каждая карта выходит из строя примерно раз в 337 дней, задание на 1024 картах прерывается в среднем примерно раз в 7,9 часа (раздел 10.4.4). Чтобы понимать эти компромиссы, необходимо связать три вопроса: какие тензоры и буферы должны одновременно занимать память GPU, какие операции определяют момент завершения шага обучения и до какого шага можно восстановиться после сбоя.

Вычислительный граф обучения определяет моменты создания и использования данных разных типов и служит основой для этих проектных решений. Активации, полученные на прямом проходе, используются на обратном; градиенты собираются перед обновлением параметров, а состояния оптимизатора сохраняются между шагами. Используя эти известные зависимости, среда выполнения может заранее загружать параметры, которые вскоре понадобятся, повторно вычислять активации с небольшой стоимостью вычисления и совмещать передачу готовых градиентов с последующими вычислениями обратного прохода. Таким образом, дифференцирование модели, управление памятью и планирование работы ускорителей совместно определяют порядок выполнения одного шага обучения.

Эта глава посвящена одной задаче: обработать 100 млрд токенов с помощью Qwen3-8B, выполнив продолжение обучения для определённой предметной области за 30 дней. Сначала мы оценим минимально необходимый объём ресурсов, а затем сравним две конфигурации из 32 и 48 карт RTX 4090. Сравнение начинается с требований к памяти GPU, после чего последовательно учитывается время на вычисления, обмен данными, подготовку данных и восстановление после сбоев; в заключение определяется, какая конфигурация позволит завершить обучение в срок. Для пояснения отдельных механизмов в тексте приводятся небольшие примеры с тензорами и конвейерами. В разделе об обучении с подкреплением (RL) дополнительно рассматривается, как генерация, проверка и обучение образуют цикл обратной связи.

В этой главе для объёма используются две системы единиц: GB и TB обозначают десятичные единицы, а GiB и MiB — двоичные.

10.1 Оценка требований к ресурсам и времени на основе задачи обучения

10.1.1 Задача обучения, обучаемые параметры и критерий завершения

Проектирование системы начинается с определения работы, которую необходимо выполнить. В задаче этой главы используются полнопараметрическое обучение, Adam со смешанной точностью и последовательности длиной 8 192 токена; необходимо обработать 100B полезных токенов. На каждой итерации обучения обрабатываются 384 последовательности полной длины, то есть 3 145 728 полезных токенов; на последней итерации обучения обрабатывается остаток данных, однако во временном бюджете на неё всё равно выделяется время полного шага. При сравнении двух вариантов системы используются одинаковые обучающие данные, целевая функция и требования к оценке.

Полезные токены — это токены, которые действительно требуется обработать в рамках задачи, без учёта токенов-заполнителей, добавленных для выравнивания длины. Заполнение делает формы тензоров более регулярными, но также расходует вычислительные ресурсы и память. При подсчёте объёма выполненной работы учитываются только полезные токены. В отличие от полнопараметрического обучения, LoRA добавляет к исходным весам небольшие обучаемые матрицы, произведение которых представляет величину корректировки весов. LoRA сокращает число обучаемых параметров: базовые веса по-прежнему используются при прямом проходе, тогда как размеры градиентов и состояний оптимизатора в основном зависят от числа параметров адаптера.

С точки зрения времени одна итерация обучения включает прямой проход, обратный проход, редукцию градиентов и обновление параметров. Непрерывное обучение состоит из множества итераций, между которыми также требуется подготавливать входные данные и сохранять чекпойнты. Запуск, подготовка ресурсов и обслуживание тоже необходимо учитывать в общем времени от начала до завершения. В проектном примере этой главы из 30 дней резервируются 5 дней на запуск, оценку и плановые остановки, а оставшиеся 25 дней отводятся на обучение, сохранение чекпойнтов и восстановление после сбоев. Поэтому требуемая средняя полезная пропускная способность возрастает примерно с 38 600 до 46 300 токенов/с.

Для выполнения всей задачи потребуется $\lceil 10^{11} / 3\,145\,728 \rceil = 31\,790$ итераций обучения. Если распределить 25 дней между всеми итерациями, на каждую останется лишь около 67,9 с. Это временной бюджет, используемый на протяжении всей главы: вычисле-

ния каждого шага, ожидание коммуникаций и входных данных, а также приходящееся на этот шаг время сохранения чекпойнтов и восстановления после сбоев должны укладываться в эти 67,9 с.

Также необходимо определить размер батча каждого шага. Глобальный батч ограничен 384 последовательностями и не увеличивается далее из-за соотношения сигнала и шума градиента. При увеличении размера батча B шум оценки градиента уменьшается; однако после того, как B становится достаточно большим, добавление новых примеров лишь незначительно сокращает число шагов, необходимых для достижения того же значения функции потерь, тогда как объём вычислений на шаг растёт пропорционально. Размер батча, при котором происходит этот перелом, называется **критическим размером батча** (critical batch size); его можно оценить через **масштаб шума градиента** (gradient noise scale). Пусть H — Hessian функции потерь по параметрам, то есть матрица вторых производных, Σ — ковариация градиента отдельного примера, а G — истинный градиент. Тогда масштаб шума градиента определяется как

$$B_{\text{noise}} = \frac{\text{tr}(H\Sigma)}{G^T H G}.$$

Число шагов S и число обработанных примеров E , необходимых для достижения заданного значения функции потерь, удовлетворяют эмпирическому соотношению

$$\left(\frac{S}{S_{\min}} - 1\right) \left(\frac{E}{E_{\min}} - 1\right) = 1, \quad B_{\text{crit}} = \frac{E_{\min}}{S_{\min}},$$

где $S_{\min}$ — минимальное число шагов для достижения этого значения функции потерь, а $E_{\min}$ — минимально необходимое число примеров. Критический размер батча приблизительно равен масштабу шума градиента; при обучении с критическим размером батча как число шагов, так и число примеров вдвое превышают соответствующие минимальные значения.¹

B_{noise} изменяется в зависимости от модели и этапа обучения. В этом примере принимается значение в два миллиона токенов: согласно измерениям из литературы, на крупнейшей из обученных моделей при сходимости оно составляло примерно от одного до двух миллионов токенов. Если считать, что эталонный запуск в этой главе — 48 GPU, 384×8192 токенов на шаг и 31 790 шагов для обработки 100B токенов — точно достигает цели, то из приведённого соотношения получаем $S_{\min} \approx 19\,434$ шага и $E_{\min} \approx 38.9\text{B}$ токенов. Текущий размер батча на шаг примерно в 1,57 раза больше B_{noise} и уже вышел за пределы диапазона, в котором удвоение размера батча приблизительно вдвое сокращает число шагов; при дальнейшем увеличении размера батча число шагов можно сократить лишь с 31 790 до 19 434. В следующей таблице сравнивается время завершения при слабом и сильном масштабировании с увеличением числа GPU.

¹Перерасчёт критического размера batch и сравнение для масштаба шума в двадцать миллионов токенов. Определение масштаба шума градиента приведено в формуле (2.8) из эмпирической модели обучения с большими batch; зависимость между количеством шагов и выборок из формулы (5.1), определение критического размера batch из формулы (5.2) и диапазон примерно от одного до двух миллионов токенов при сходимости приведены в статье о законах масштабирования. B_{noise} — принятое для этого примера значение, которое не измерялось на модели из этой главы; ни слабое, ни сильное масштабирование не учитывают изменение эффективности распараллеливания.

Число GPU	Слабое масштабирование: токенов на шаг	Время завершения при слабом масштабировании, дней	Время завершения при сильном масштабировании, дней
48 (эталон)	3 145 728	19,4	19,4
96	6 291 456	15,7	9,8
192	12 582 912	13,8	5,0
384	25 165 824	12,8	2,6
1536	100 663 296	12,1	0,8

При слабом масштабировании в таблице на каждом GPU неизменно накапливается 8 микробатчей, длительность шага остаётся равной 52,8 с, а глобальный батч увеличивается вместе с числом GPU. При сильном масштабировании глобальный батч остаётся фиксированным, объём вычислений на одном GPU обратно пропорционален числу GPU, а 0,58 с ожидания коммуникаций и входных данных не меняются. В обоих случаях изменение эффективности параллелизма не учитывается. Предельное ускорение при слабом масштабировании составляет $31\,790/19\,434 \approx 1.64$ раза: 1536 GPU увеличивают размер батча каждого шага в 32 раза относительно эталонного значения, но время завершения сокращается лишь с 19,4 до 12,1 дня. Сильное масштабирование не ограничено B_{noise} , однако доля постоянного ожидания в каждом шаге растёт вместе с числом GPU: при общем времени 0,8 дня ожидание коммуникаций и входных данных уже занимает около четверти.

Момент, когда польза слабого масштабирования начинает снижаться, зависит от значения B_{noise} : если масштаб шума равен двадцати миллионам токенов, предельное ускорение при слабом масштабировании возрастает примерно до 7,36 раза и увеличение размера батча вместе с числом GPU приносит заметную пользу. Поэтому прежде чем увеличивать размер батча вслед за числом GPU, необходимо оценить B_{noise} : если текущий размер батча значительно меньше этого значения, его можно увеличивать вместе с числом GPU; если он значительно больше, добавление GPU способно сократить лишь время шага по схеме сильного масштабирования. Именно поэтому были выбраны 384 последовательности: такой размер укладывается в видеопамять 48 GPU и бюджет времени на шаг, при этом он уже немного превышает масштаб шума, поэтому польза дальнейшего увеличения размера батча начинает снижаться.²

10.1.2 Параметры, градиенты, оптимизатор и активации

67,9 с задают допустимое время шага, однако до начала вычислений необходимо также удовлетворить требованиям к видеопамяти. Обозначим число параметров через N . В ходе одной итерации прямой проход вычисляет прогноз и функцию потерь по входным данным, обратный проход следует по вычислительным зависимостям, чтобы получить градиенты функции потерь по всем промежуточным значениям и параметрам, и считывает сохранённые при прямом проходе активации; затем оптимизатор использует градиенты параметров для вычисления весов следующего шага.

²Перерасчёт критического размера batch и сравнение для масштаба шума в двадцать миллионов токенов. Определение масштаба шума градиента приведено в формуле (2.8) из эмпирической модели обучения с большими batch; зависимость между количеством шагов и выборок из формулы (5.1), определение критического размера batch из формулы (5.2) и диапазон примерно от одного до двух миллионов токенов при сходимости приведены в статье о законах масштабирования. B_{noise} — принятое для этого примера значение, которое не измерялось на модели из этой главы; ни слабое, ни сильное масштабирование не учитывают изменение эффективности распараллеливания.

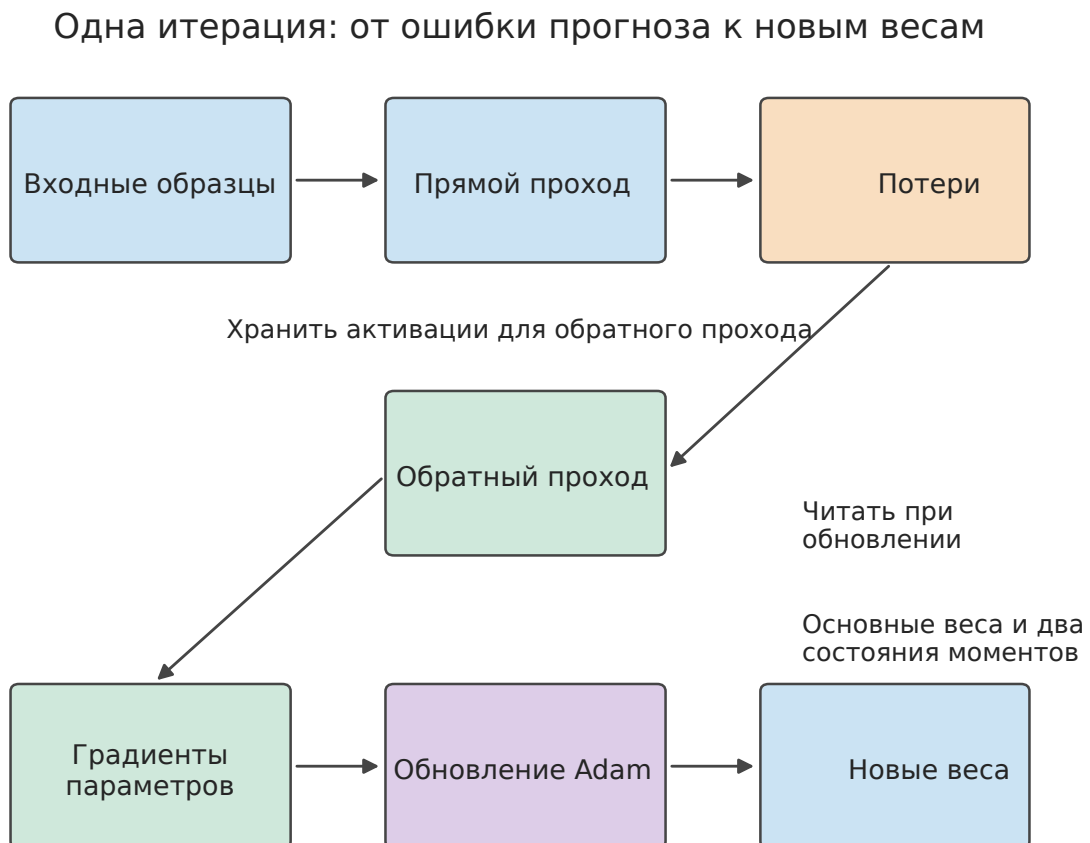


Рисунок 10-1. Из входных данных и текущих весов прямой проход получает функцию потерь, обратный проход — градиенты параметров, а Adam создаёт новые веса. Сохранённые на этапе прямого прохода активации используются при обратном проходе; процесс обновления также считывает высокоточные основные веса и историю градиентов.

В частности, в задаче этой главы Adam со смешанной точностью выполняет матричные операции с весами BF16, хранит вычисленные оптимизатором значения параметров в виде основных весов FP32 и сохраняет два состояния моментов FP32. Первый и второй моменты сохраняются между итерациями и содержат историю соответственно направлений и величин градиентов; после обновления основные веса преобразуются в BF16 для использования в матричных операциях. Градиенты накапливаются в BF16. В сумме получается 16 байт на параметр.

Состояние	Число байтов на параметр	Объём для Qwen3-8B, GB	Когда используется
Веса модели BF16	2	16,4	Матричные операции прямого и обратного проходов
Градиенты BF16	2	16,4	Создаются при обратном проходе и используются при вычислении новых параметров
Основные веса FP32	4	32,8	Хранят новые параметры, после чего из них создаётся копия BF16
Первый момент Adam	4	32,8	Сохраняет историю градиентов между итерациями обучения
Второй момент Adam	4	32,8	Сохраняет историю квадратов градиентов между итерациями обучения
Итого	16	131,1	Постоянное состояние обучения

Qwen3-8B содержит $N = 8\,190\,735\,360$ параметров, поэтому

$$M_{\text{persistent}} = 16N \approx 131.1 \text{ GB} \approx 122.1 \text{ GiB.}$$

Формат градиентов — ещё один параметр выбора. При переходе на градиенты FP32 к каждому параметру добавляются 2 байта, а общий объём возрастает до 18 байт. Это изменение обусловлено форматом данных градиента; термин «смешанная точность» сам по себе не определяет формат градиента. В MoE маршрутизатор каждый раз выбирает для выполнения часть экспертных подсетей; у каждого эксперта также есть обновляемые параметры и история оптимизатора, поэтому объём состояния определяется общим числом всех обучаемых параметров.³

Помимо объёма каждого типа данных, необходимо учитывать, как долго их требуется хранить. Активации, созданные при прямом проходе, можно освободить только после завершения соответствующих вычислений обратного прохода, тогда как буфер сбора параметров может использоваться лишь во время выполнения определённого модуля. Пиковое потребление видеопамати на i -м GPU зависит от одновременно находящихся в памяти состояния обучения, активаций и временных буферов:

³Перерасчёт официального количества параметров и состояния Adam/ZeRO; вариант с градиентами FP32. В этой главе используется заданное представление состояния из пяти компонентов; для других оптимизаторов и состояний пониженной точности требуются отдельные расчёты.

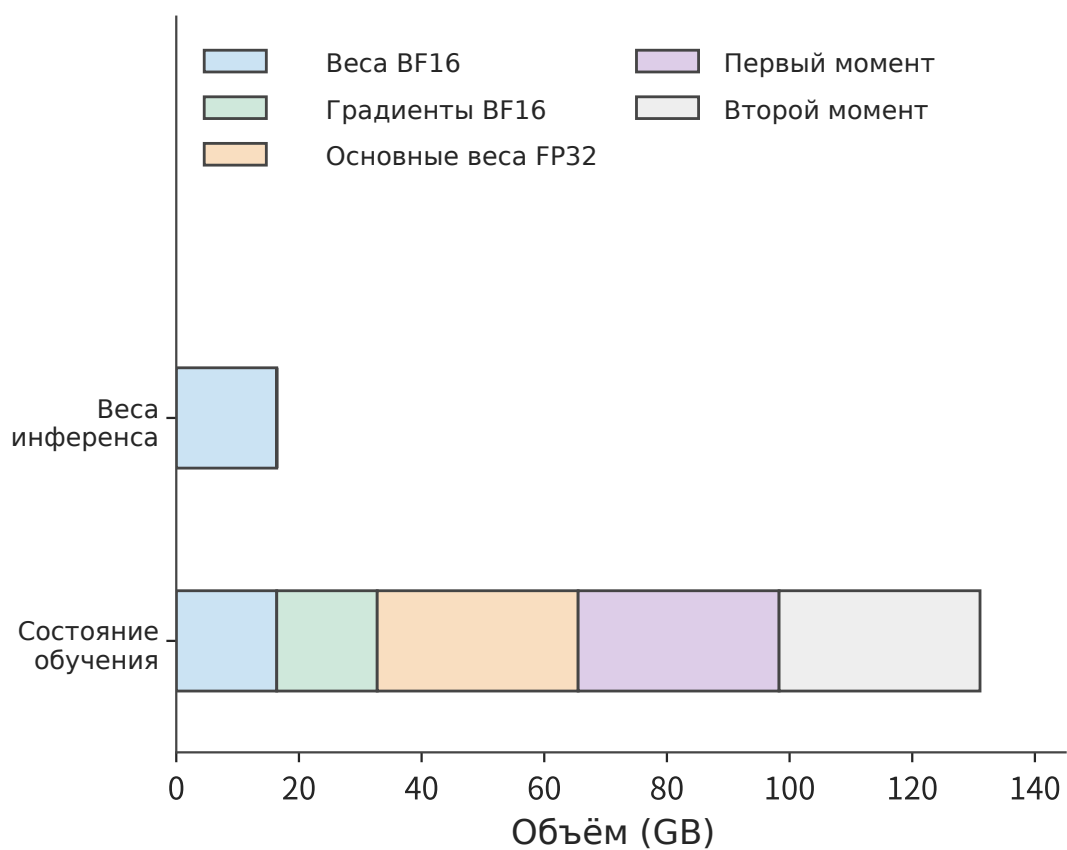


Рисунок 10-2. Совокупный объём пяти типов состояния обучения эквивалентен объёму восьми копий весов BF16. Используется модель Qwen3-8B и форматы данных из таблицы; горизонтальная ось показывает накопленный объём, занимаемый различными типами состояния.

$$M_{\text{peak},i} = \max_t [M_{\text{persistent},i}(t) + M_{\text{activation},i}(t) + M_{\text{temporary},i}(t)] .$$

Например, если 8 GiB активаций и буфер сбора размером 4 GiB существуют одновременно, вместе они занимают 12 GiB; если сначала освободить активации, а затем собрать параметры, один и тот же участок памяти можно использовать повторно. Поэтому оптимизация объёма ведётся в двух направлениях: уменьшение размеров тензоров и буферов либо сокращение объёма данных, которые требуется хранить одновременно. Рассматриваемые в следующем разделе шардирование, повторные вычисления и выгрузка изменяют пиковое потребление памяти соответственно по этим двум направлениям.

Приведённые выше объёмы рассчитаны для выполнения матричных операций в BF16. При выборе другого формата с плавающей точкой одновременно изменяются число байтов, занимаемых состоянием, и численные риски. Сначала рассмотрим FP16. В разделе 4.6.1 уже сравнивались экспонента и мантисса FP16 и BF16: пятибитная экспонента FP16 задаёт минимальное нормальное число на уровне $2^{-14} \approx 6.1 \times 10^{-5}$; меньшие значения можно представить только как субнормальные числа, причём по мере уменьшения значения постепенно теряются значащие разряды, а значения меньше $2^{-24} \approx 6 \times 10^{-8}$ напрямую превращаются в ноль. В статье об обучении со смешанной точностью приведены две группы статистических данных: среди градиентов весов модели распознавания речи на путунхуа около 5% значений имели экспоненту меньше -24 и в FP16 превращались в ноль; этим авторы обосновали необходимость основных весов FP32. В градиентах активаций сети обнаружения Multibox SSD множество значений также оказывалось ниже представимого диапазона FP16 и превращалось в ноль, причём значения из диапазона $[2^{-27}, 2^{-24})$ всё ещё были важны для обучения.

Масштабирование функции потерь (loss scaling) решает эту проблему: перед обратным проходом функция потерь умножается на коэффициент S , вследствие чего градиенты также увеличиваются в S раз и попадают в представимый диапазон FP16; перед обновлением параметров они снова делятся на S . При обучении различных сетей в указанной статье использовались коэффициенты масштабирования от 8 до 32K. Если коэффициент слишком мал, по-прежнему возникает андерфлоу малых градиентов; если слишком велик, большие градиенты переполняются до бесконечности, поэтому такое обновление необходимо пропустить и уменьшить коэффициент.⁴

В BF16 используется восьмибитная экспонента, как и в FP32, поэтому диапазоны представимых значений у них совпадают и масштабировать градиенты не требуется. Это одна из причин, по которым в приведённой выше таблице состояний используется BF16. Цена этого решения — мантисса длиной всего семь бит: ошибка округления входных данных матричных операций больше, чем у FP16, поэтому точность длинных скалярных произведений необходимо обеспечивать аккумуляторами FP32.

При дальнейшем сокращении разрядности единый глобальный коэффициент масштабирования приходится заменять блочным масштабированием. Кодировка E4M3 формата FP8 содержит только четыре бита экспоненты и три бита мантиссы, а её

⁴Статья об обучении со смешанной точностью — механизм масштабирования функции потерь, коэффициенты масштабирования от 8 до 32K и представимый диапазон FP16; статья о форматах FP8 — кодирование E4M3/E5M2 и максимальное нормальное число; технический отчёт DeepSeek-V3, §3.3 — блочное масштабирование активаций блоками 1×128 и весов блоками 128×128 , перевод каждого 128 элементов в FP32 для накопления и относительная ошибка функции потерь менее 0,25%. Сравнение экспоненты и мантиссы FP16 и BF16 см. в разделе 4.6.1.

максимальное нормальное число равно 448, поэтому единый глобальный коэффициент не позволяет одновременно поместить активации всех слоёв в столь узкий диапазон. При обучении DeepSeek-V3 в FP8 масштабирование выполняется по блокам: для активаций scale вычисляется по малым блокам 1×128 — для каждого токена и каждых 128 каналов, — а для весов по блокам 128×128 ; после накопления каждых 128 элементов матричный блок преобразует частичную сумму в FP32 и продолжает накопление. По сравнению с контрольным обучением в BF16 относительная ошибка функции потерь составила менее 0,25%.⁵

FP8 также изменяет расчёт объёма, но только для копии весов, подаваемой в матричный блок: при обучении DeepSeek-V3 в FP8 операнды матричных операций преобразуются в FP8, тогда как основные веса, градиенты и состояния оптимизатора по-прежнему хранятся с более высокой точностью. Эта копия операндов FP8 занимает 1 байт на параметр, а к каждому блоку 128×128 добавляется один scale FP32, что в сумме составляет приблизительно $1 + 4/16384 \approx 1.0002$ байта на параметр. Для Qwen3-8B объём весов, считываемых при матричных операциях, сокращается с 16,4 GB в BF16 примерно до 8,2 GB, однако постоянное состояние обучения размером 16 байт на параметр из приведённой выше таблицы не сокращается из-за этого до 1 байта.

Формат весов для матричных операций	Байтов на параметр с учётом scale	Веса, считываемые при матричных операциях Qwen3-8B	Управление масштабированием	Основные численные риски
FP16	2	16,4 GB	Глобальное масштабирование функции потерь	Андерфлоу градиентов, переполнение при масштабировании
BF16	2	16,4 GB	Не требуется	Короткая мантисса, компенсируется накоплением в FP32
Копия операндов FP8 (E4M3)	Около 1,0002	Около 8,2 GB (основные веса и состояния оптимизатора хранятся отдельно)	Блочное масштабирование и периодическое накопление с высокой точностью	Округление внутри блока, устаревание scale

Чем ниже точность и крупнее гранулярность масштабирования, тем больше экономия памяти, но тем выше риск численного отклонения от высокоточного эталона. Независимо от того, вызвано ли отклонение обучения сбоем масштабирования, аномалией данных или аппаратной ошибкой, оно проявляется в виде всплеска на кривой функции потерь (loss spike); обычно в таком случае выполняют откат к предыдущему чекпойнту и повторяют обучение. Объём прогресса, теряемого при откате, определяется моделью интервалов сохранения из раздела 10.4, а в разделе 10.4.5 частота откатов будет включена в эту модель.

⁵Статья об обучении со смешанной точностью — механизм масштабирования функции потерь, коэффициенты масштабирования от 8 до 32K и представимый диапазон FP16; статья о форматах FP8 — кодирование E4M3/E5M2 и максимальное нормальное число; технический отчёт DeepSeek-V3, §3.3 — блочное масштабирование активаций блоками 1×128 и весов блоками 128×128 , перевод каждых 128 элементов в FP32 для накопления и относительная ошибка функции потерь менее 0,25%. Сравнение экспоненты и мантиссы FP16 и BF16 см. в разделе 4.6.1.

10.1.3 Требования к вычислениям, MFU и нижняя граница ресурсов

Видеопамять определяет, можно ли начать обучение модели, а скорость вычислений — можно ли завершить его в срок. Состояние одной модели можно распределить между несколькими GPU, однако необходимое число GPU также следует определять по общему объёму вычислений и сроку выполнения. Матричные вычисления прямого и обратного проходов для одной последовательности длиной 8 192 токена требуют приблизительно 4.31×10^{14} FLOPs, в среднем около 52,7 GFLOPs на токен. Для обработки 100B токенов суммарно потребуется около 5.27×10^{21} FLOPs. Этот объём работы получен суммированием операций прямого и обратного проходов с учётом форм матриц модели.⁶

Сначала предположим, что все 30 дней полностью отведены на вычисления обучения. Пиковая производительность H100 SXM в плотных матричных операциях BF16 равна 989 TFLOP/s; при достижении 40% от пика один GPU способен обрабатывать около 7 500 токенов/с. Пять GPU вместе обеспечивают около 37 600 токенов/с, что меньше требуемых для завершения за 30 дней примерно 38 600 токенов/с; для удовлетворения вычислительных требований необходимо не менее шести GPU. Для сравнения, двух GPU с 80 GB уже достаточно, чтобы вместить идеально равномерно распределённое постоянное состояние обучения объёмом 131,1 GB, однако вычисления займут около 77 дней.

Пусть общий объём вычислений алгоритма равен F , число GPU — p , пиковая производительность одного GPU — P , а средняя вычислительная эффективность за рассматриваемый период — η . Тогда

$$T = \frac{F}{pP\eta}, \quad p_{\text{compute}} = \left\lceil \frac{F}{P\eta T_{\text{available}}} \right\rceil.$$

В этой главе MFU вычисляется согласно определению из раздела 1.2.2, а в знаменателе используется суммарная пиковая вычислительная производительность всех GPU. Если в качестве фактического времени брать полную длительность итерации обучения, ожидание коммуникаций уже будет учтено; если учитывать только время выполнения вычислений ускорителем, результатом будет вычислительная эффективность одного GPU. В проектном примере этой главы используется второе разложение: вычислительная эффективность одного GPU принимается равной 40%, после чего отдельно добавляются время коммуникаций, которое невозможно перекрыть вычислениями внутри GPU, и время ожидания входных данных. Это позволяет непосредственно наблюдать влияние каждого системного решения на время завершения. Значение 40% взято из опубликованных данных Llama 3: модель 405B предварительно обучалась на 8 192–16 384 GPU H100 и достигала MFU BF16 от 38% до 43%. Эти значения рассчитаны по полной длительности шага и уже включают ожидание коммуникаций; поскольку в этой главе коммуникации добавляются ещё раз отдельно, рассчитанное время завершения оказывается завышенным. У RTX 4090 нет NVLink, поэтому все данные между GPU передаются через PCIe; соответствующие затраты рассчитываются отдельно в разделе 10.3.3 по характеристикам фактических соединений. На рисунках 10-3–10-5 также показаны уровни 30% и 50%, позволяющие оценить изменение необходимого числа GPU при отклонении эффективности.⁷

⁶Срок обучения и нижняя граница количества устройств, а также варианты календарного времени. Источники данных о точности, разреженности и оборудовании поэлементно записаны в JSON. Значения MFU 38–43% для BF16 приведены в отчёте Llama 3, §3.3.2 и таблица 4; PCIe 4.0 и отсутствие NVLink у RTX 4090 указаны на странице спецификаций RTX 4090.

⁷Срок обучения и нижняя граница количества устройств, а также варианты календарного времени. Ис-

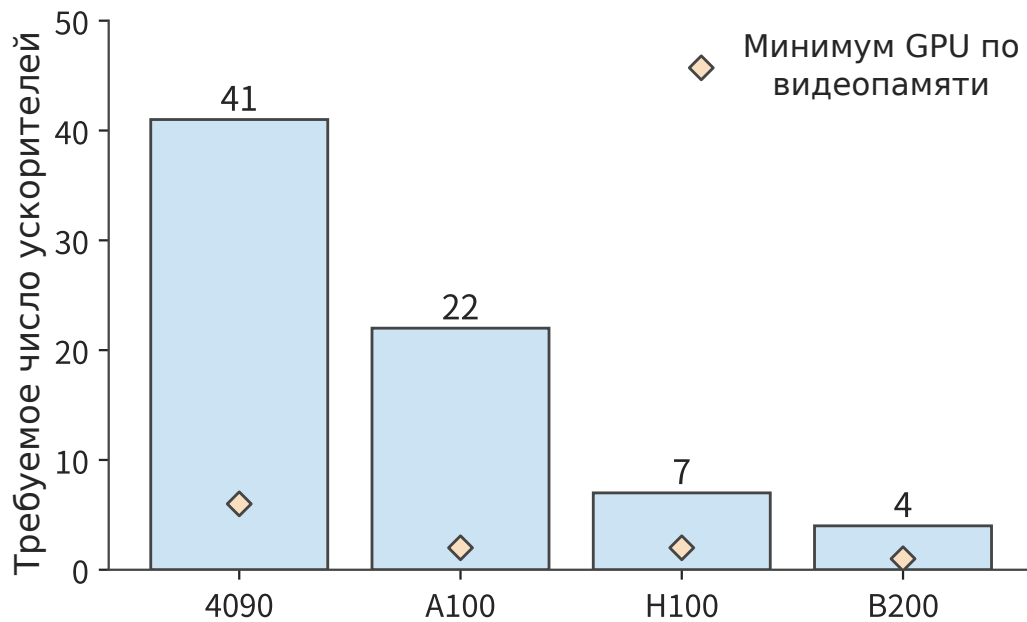


Рисунок 10-3. Число GPU, необходимое при эффективности матричных вычислений 30%. Задача состоит в обработке моделью Qwen3-8B 100B токенов с длиной последовательности 8192; все 30 дней отведены на выполнение. Синие столбцы показывают вычислительные требования, оранжевые ромбы — нижнюю границу объёма состояния обучения. Для A100 используется вариант SXM с 80 GB, для H100 — вариант SXM.

Куда расходуются остальные 60% пиковой вычислительной производительности, последовательно объясняется в разделах этой главы: коммуникации, которые невозможно перекрыть вычислениями (раздел 10.3.3), пузыри конвейера (раздел 10.3.2), ограниченные пропускной способностью памяти обновление параметров и преобразование состояний (раздел 10.2.3), повторные вычисления (раздел 10.2.2), отстающие исполнители (раздел 10.4.5) и сохранение чекпойнтов (раздел 10.4.3). Согласно критерию из раздела 1.3.4, первые четыре пункта представляют работу, которую необходимо учитывать в модели; если пропустить любой из них, оценка времени завершения окажется заниженной. Последние два пункта и неперекрываемая часть коммуникаций содержат издержки, которые можно сократить, — именно они образуют пространство для оптимизации системного проекта.

На рисунке 10-3 нижние границы по видеопамяти и вычислительной производительности показаны вместе: для задачи этой главы необходимое число GPU в основном определяется вычислениями. Исходя из этого, далее выбираются две конкретные конфигурации для поэтапного сравнения в последующих разделах. Соответствующая пиковая производительность RTX 4090 равна 165,2 TFLOP/s. При эффективности 40% и сроке вычислений 30 дней требуется не менее 31 GPU. Если использовать конфигурацию с семью GPU на хост, сначала можно рассмотреть вариант с 32 GPU; добавление ещё двух хостов даёт 48 GPU. В этих двух вариантах каждый GPU накапливает соответственно 12 и

точники данных о точности, разреженности и оборудовании поэлементно записаны в JSON. Значения MFU 38–43% для BF16 приведены в отчёте Llama 3, §3.3.2 и таблица 4; PCIe 4.0 и отсутствие NVLink у RTX 4090 указаны на странице спецификаций RTX 4090.

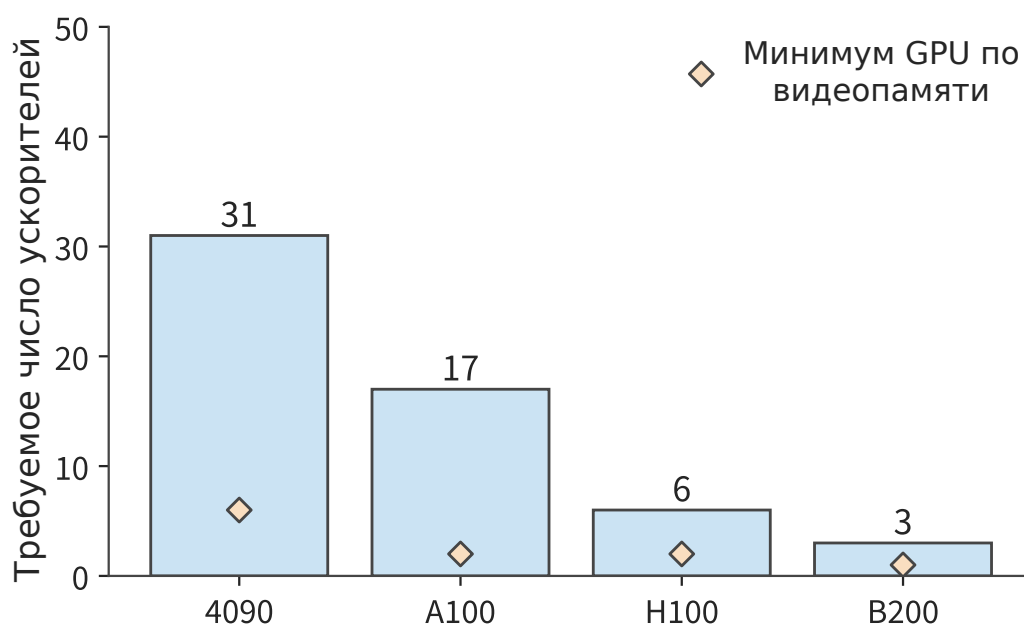


Рисунок 10-4. При повышении эффективности матричных вычислений до 40% число GPU, необходимое для той же задачи, уменьшается; оранжевые ромбы по-прежнему обозначают нижнюю границу объёма состояния. Все нижние границы выражены числом GPU; показатель объёма — это минимальное число GPU, необходимое для выполнения ограничения по видеопамяти.

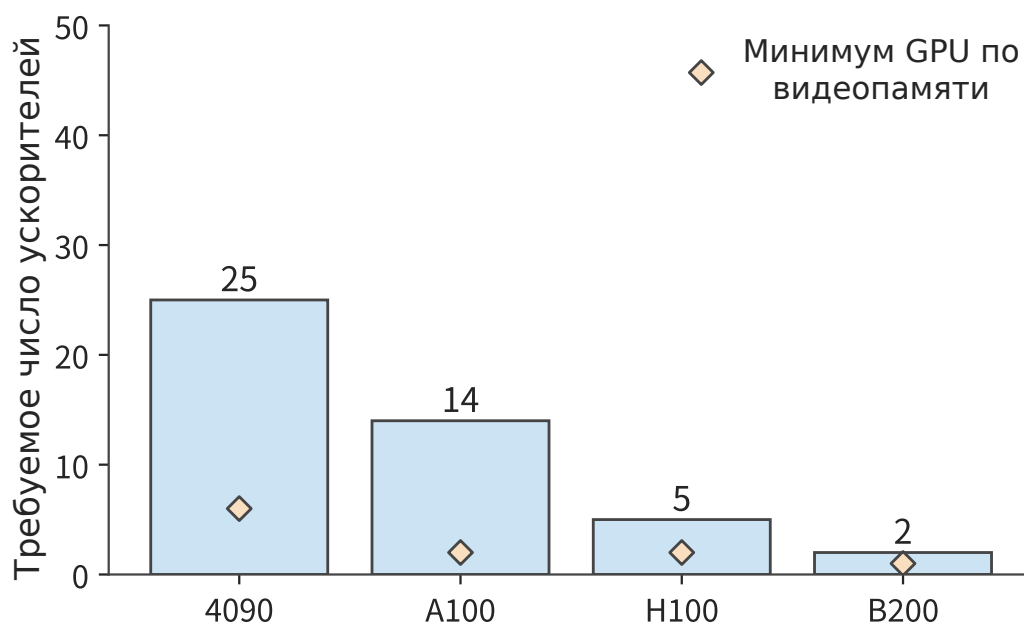


Рисунок 10-5. Число GPU, необходимое при эффективности матричных вычислений 50%. На всех трёх рисунках используются одинаковые порядок моделей, диапазон вертикальной оси, задача и срок выполнения 30 дней. Все нижние границы выражены числом GPU; показатель объёма — это минимальное число GPU, необходимое для выполнения ограничения по видеопамяти.

8 микробатчей по одной последовательности, совместно формируя глобальный батч из 384 последовательностей. Увеличение числа GPU изменяет распределение работы, но цель обучения остаётся прежней.

Каждая итерация обучения требует около 1.66×10^{17} FLOPs. При использовании 32 GPU выполнение приходящейся на каждый GPU части вычислений занимает около 78,3 с, а при использовании 48 GPU — около 52,2 с. По сравнению с бюджетом 67,9 с вариант с 32 GPU превышает лимит ещё до добавления коммуникаций, тогда как вариант с 48 GPU оставляет около 15,7 с. Далее необходимо определить, как в этих двух вариантах разместить состояние и какие ожидания израсходуют эти 15,7 с.

10.2 Шардирование, повторное вычисление и выгрузка состояния обучения

10.2.1 Параллелизм данных и шардирование состояния в ZeRO/FSDP

В главе 6 шесть способов распараллеливания были введены через разбиение по примерам, признакам, позициям последовательности, слоям и экспертам. Здесь мы переходим к обучению и рассматриваем шардирование состояния внутри реплик параллелизма данных. FSDP (Fully Sharded Data Parallel, полностью шардированный параллелизм данных) распределяет между процессами (rank) параметры, градиенты и состояние оптимизатора; параметры слоя временно собираются перед вычислением, а после обратного прохода градиенты распределяются по шардам. FSDP прежде всего уменьшает объём памяти и дублирование состояния обучения и обычно сочетается с DP, TP и PP.

SP (Sequence Parallel, параллелизм по последовательности) распределяет позиции одной и той же входной последовательности и главным образом применяется к операциям над отдельными токенами, таким как LayerNorm и Dropout, внутри группы TP. Перед матричным умножением, которому нужны полные строки токенов, карты собирают вход, а после вычисления снова разделяют результат по позициям. FSDP управляет владением состоянием модели, а SP — рабочим размещением промежуточных активаций; объекты разбиения, моменты коммуникаций и области применения у них различаются. На рисунке 1 статьи о Nested BSP FSDP выделен в отдельный уровень: шардирование состояния обучения и шардирование активаций выполнения относятся к разным осям параллелизма.⁸

Распределение состояния между несколькими GPU определяется способом распараллеливания. При синхронном параллелизме данных каждый GPU обрабатывает разные примеры, после чего обновляет свою копию параметров с помощью редуцированных градиентов. Новые параметры на всех GPU одинаковы, поэтому каждый GPU обычно хранит полную копию весов и состояния оптимизатора; чем больше GPU, тем больше копий состояния. ZeRO был предложен исследователями Microsoft в 2019 году и опубликован в 2020 году. Он уменьшает число дублирующихся копий состояния при обучении крупных моделей за счёт шардирования между GPU. ZeRO поэтапно распределяет ответственность за это состояние: каждый GPU обновляет только часть параметров, а остальные GPU получают необходимые для выполнения данные посредством коммуникации.

⁸ Доктор Ляо Хэн, главный учёный Huawei по полупроводниковым технологиям, Nested Parallel von Neumann Architecture and Nested BSP, arXiv:2609.16787, 15 сентября 2026 года. Раздел 2 и рисунок 1 помещают FSDP в иерархию вложенного параллелизма DP, PP, EP, CP и TP; здесь это используется для пояснения различий между шардированием состояния и активаций.

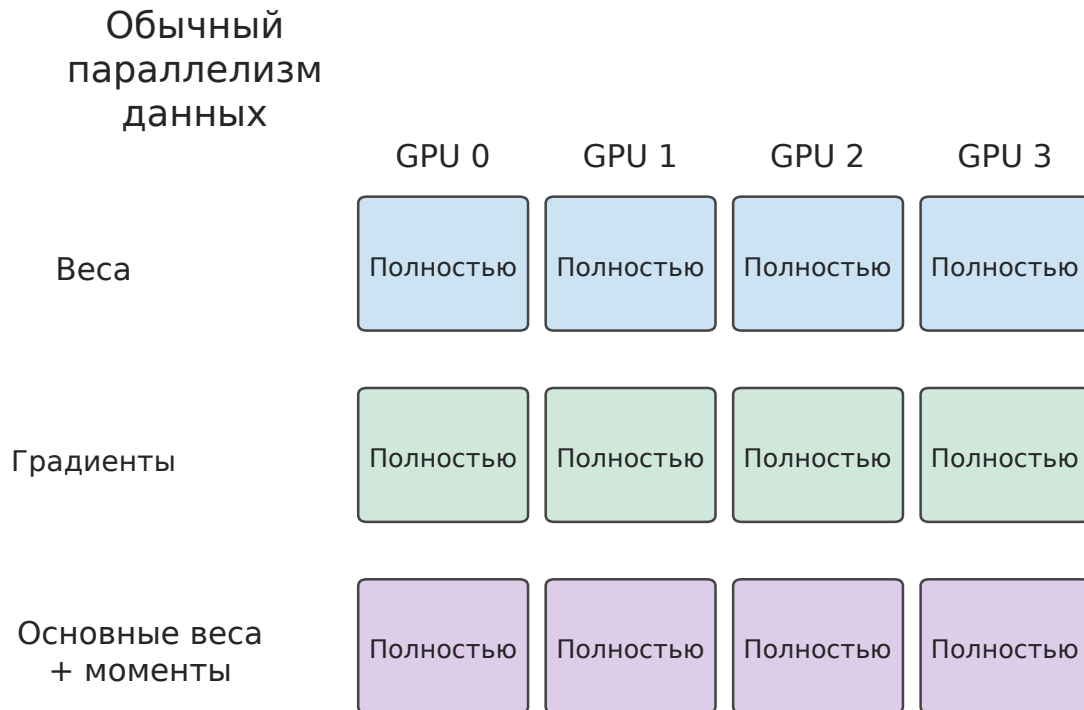


Рисунок 10-6. Обычный DP: каждый GPU хранит полную копию весов, градиентов, основных весов и двух состояний моментов.

Полностью шардированный параллелизм данных (Fully Sharded Data Parallel, FSDP) организует выполнение по тому же принципу шардирования: параметры, градиенты и состояние оптимизатора хранятся в виде шардов. Ниже принадлежность состояния пошагово изменяется на примере четырёх GPU, после чего анализируется соответствующий способ выполнения. Каждый столбец на рисунках всегда соответствует одному и тому же GPU; «полностью» означает, что GPU хранит всё состояние, а «шард» — только четверть, за которую он отвечает.

На первых двух этапах каждый GPU по-прежнему хранит полные веса модели. На рисунке 10-9 шардируются и веса, что дополнительно уменьшает объём постоянно размещённых данных, но перед выполнением каждого модуля необходимые веса приходится получать посредством коммуникации.

Пусть группа параллелизма данных содержит d GPU. Если сначала рассматривать постоянно размещённое состояние обучения, объём на один GPU для обычного параллелизма данных и трёх этапов ZeRO соответственно равен

$$\begin{aligned}
 M_0 &= 16N, \\
 M_1 &= 4N + 12N/d, \\
 M_2 &= 2N + 14N/d, \\
 M_3 &= 16N/d.
 \end{aligned}$$

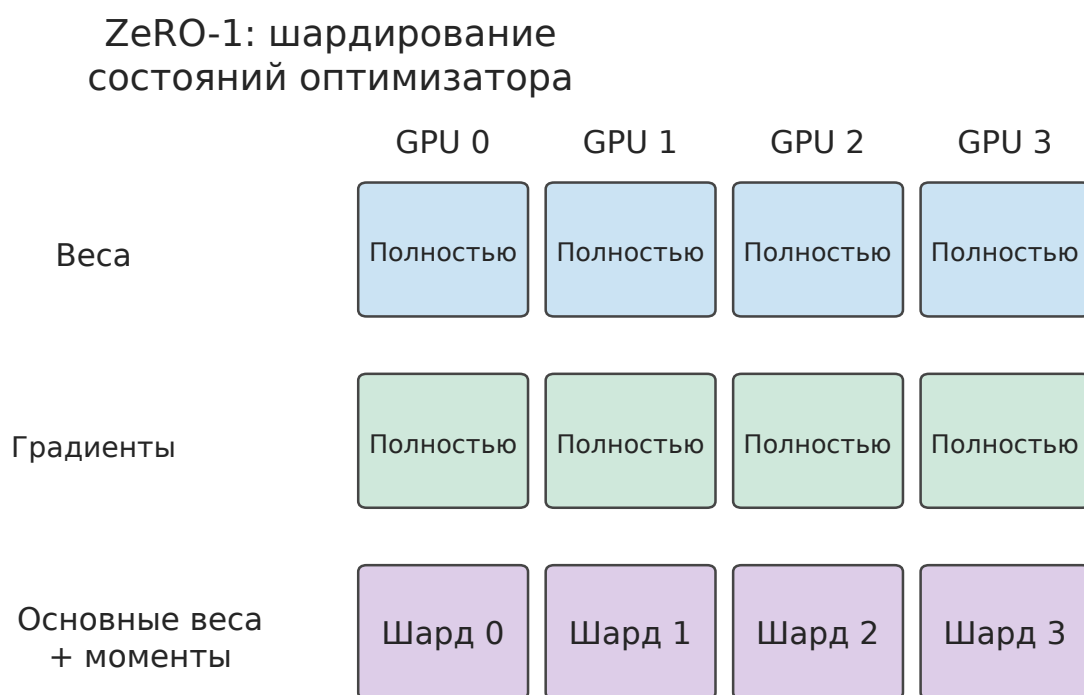


Рисунок 10-7. ZeRO-1: основные веса и два состояния моментов шардируются по параметрам, а веса и градиенты по-прежнему полностью копируются.

ZeRO-2: плюс шардирование градиентов

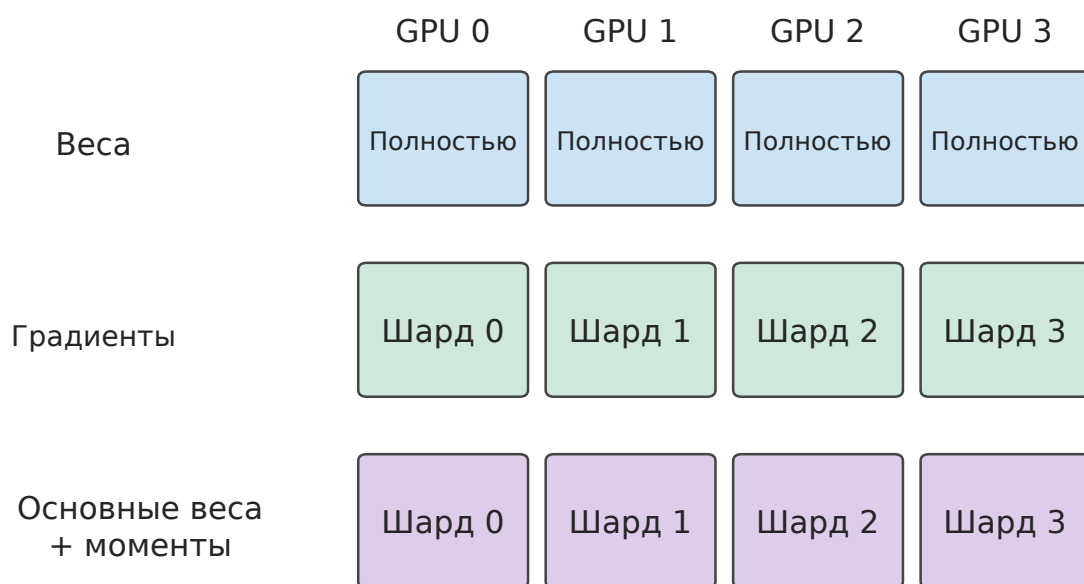


Рисунок 10-8. ZeRO-2: градиенты также разделяются, и каждый GPU хранит только принадлежащий ему шард градиентов.

ZeRO-3: плюс шардирование весов модели

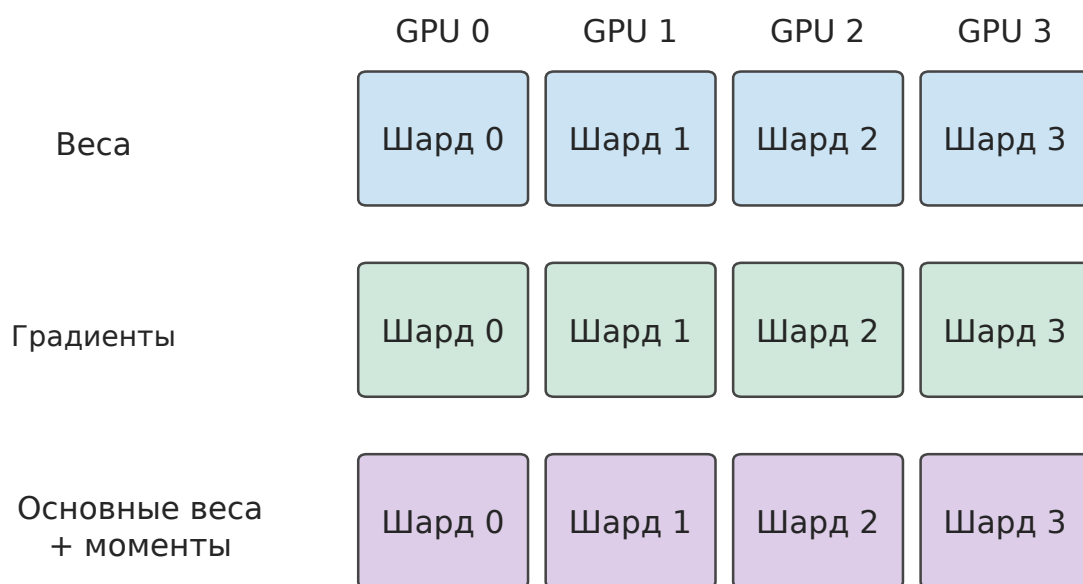


Рисунок 10-9. ZeRO-3: веса также шардируются. При выполнении модуля каждый GPU получает необходимые полные веса посредством коммуникации.

На первом этапе шардируются основные веса FP32 и два состояния Adam — всего 12 байт на параметр, тогда как веса и градиенты по-прежнему хранятся по одной полной копии. На втором этапе шардируются также градиенты, и полностью скопированными остаются только веса BF16. На третьем этапе шардируются и веса. Подставив $d = 8$, получаем следующее изменение объёма.

Конфигурация	Постоянно размещённое состояние обучения на GPU / GiB	Полностью копируемое состояние обучения
Обычный DP	122.1	Веса, градиенты и состояние оптимизатора
ZeRO-1	42.0	Веса и градиенты
ZeRO-2	28.6	Веса
ZeRO-3	15.3	Всё постоянно размещённое состояние обучения шардировано

После шардирования каждый модуль выполняется в следующем порядке: сначала необходимые параметры собираются с помощью AllGather, затем выполняется прямой или обратный проход, после чего ReduceScatter агрегирует градиенты и распределяет их шарды так, чтобы отвечающий за каждый шард GPU получил все вклады в градиент, и наконец каждый GPU обновляет закреплённые за ним параметры. Полностью шардированное выполнение FSDP также строится вокруг этой последовательности. Если после прямого прохода полные параметры освобождаются, перед обратным проходом их необходимо собрать повторно. Если их сохранить, обратный проход сможет использовать их напрямую, но полные параметры будут занимать видеопамять одновременно с активациями последующих модулей.

На рисунке 10-10 буфер полных весов существует одновременно с исходными шардами. Увеличение числа GPU уменьшает каждый исходный шард, однако размер буфера полных весов определяется самим модулем. Поэтому объём постоянно размещённого состояния и пиковое потребление памяти уменьшаются в разной степени.

Эту разницу можно непосредственно наблюдать в небольшом эксперименте. В эксперименте с небольшой CPU-моделью на реализации FSDP второго поколения в PyTorch (FSDP2) число участвующих процессов увеличили с двух до четырёх: сохраняемое после обновления параметров состояние уменьшилось примерно с 18 MiB до 9 MiB, тогда как пик при выполнении снизился лишь примерно с 39 MiB до 30 MiB. Постоянно хранимая часть сократилась вдвое, а пик — лишь примерно на 23%, поскольку во время выполнения всё ещё требовалось около 21 MiB для других тензоров и буферов. Следовательно, при анализе требований к памяти после шардирования необходимо также учитывать, как долго во время выполнения модуля требуется хранить параметры и активации.⁹

10.2.2 Сохранение активаций и выборочное повторное вычисление

Активации, оставшиеся после прямого прохода, подобны промежуточным шагам, записанным на черновике: когда обратный проход возвращается к этому слою, некоторые из этих значений снова понадобятся. Хранить весь черновик удобно, но он занимает место;

⁹Журнал четырёх конфигураций CPU FSDP2. Параметры малой CPU-модели во всех четырёх конфигурациях совпали с нешардированным эталоном; пиковый объём памяти, выделенной во время обучения, составил 39,16/30,14 MiB.

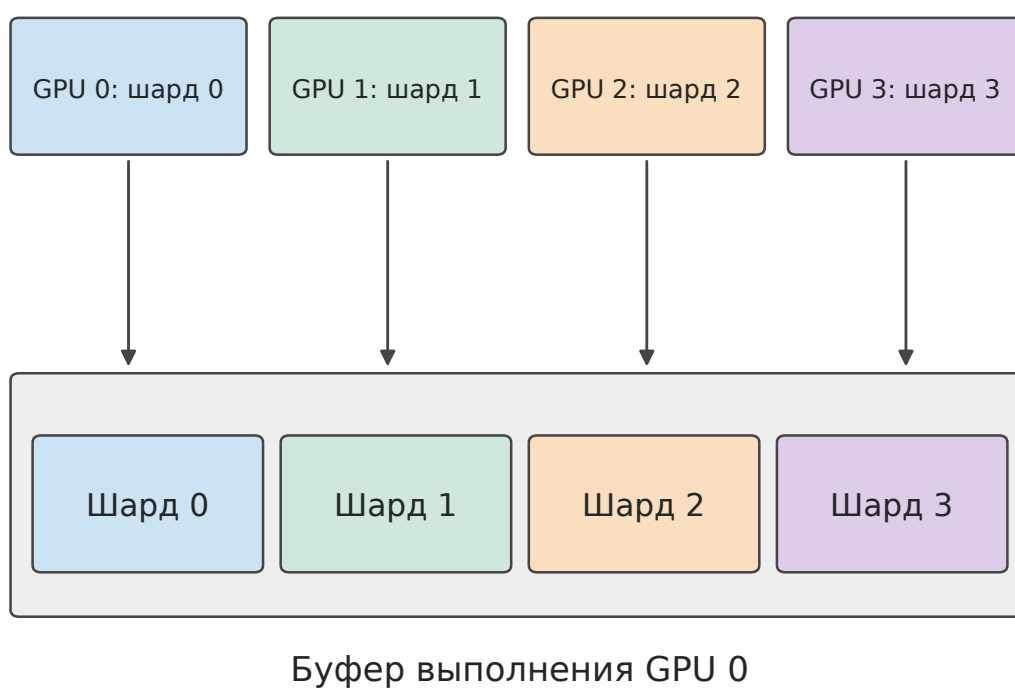


Рисунок 10-10. В обычном состоянии веса одного модуля распределены между четырьмя GPU; во время выполнения каждый GPU собирает полные веса. На рисунке подробно показан процесс сборки на GPU 0, а четыре цвета обозначают четыре шарда параметров. Буфер полных весов можно освободить сразу после использования, при этом исходные шарды, постоянно хранящиеся на каждом GPU, сохраняются.

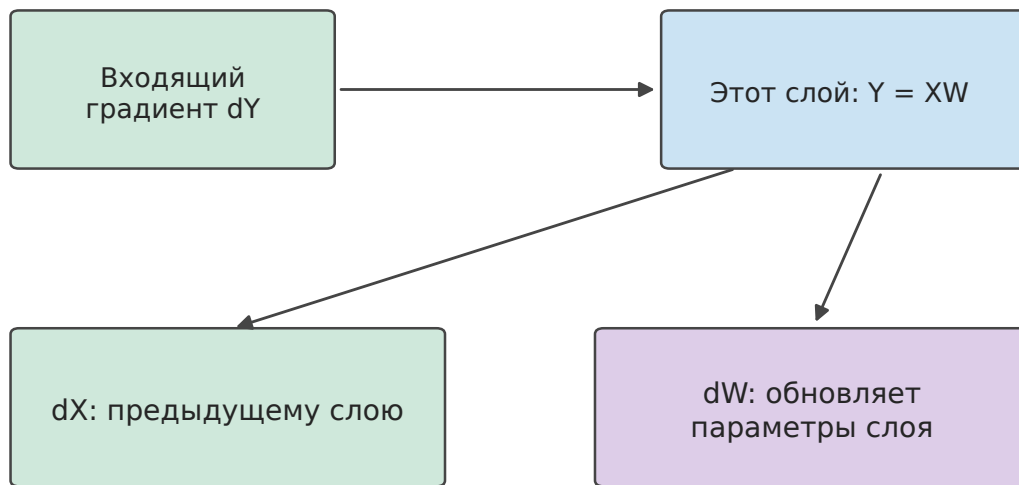


Рисунок 10-11. Обратный проход матричной операции создаёт две ветви градиентов: dX передаётся предыдущему слою для продолжения обратного прохода, а dW — для редукции и обновления параметров текущего слоя. Обе ветви вычислений читают входящий градиент dY .

если сохранить только часть шагов, недостающие значения придётся повторно вычислить перед использованием. Шардирование параметров уже уменьшило дублирование весов и состояния оптимизатора, а повторное вычисление активаций обменивает дополнительные вычисления на память, необходимую для хранения этих промежуточных значений.

Рассмотрим матричную операцию $Y = XW$, где X — вход, W — веса, а Y — выход. После завершения прямого прохода X потребуется при вычислении градиента весов. Обозначим функцию потерь через $\mathcal{L}$; $dY = \partial \mathcal{L} / \partial Y$ — градиент, полученный от последующих вычислений, а dX и dW — соответственно градиенты функции потерь по входу и весам; верхний индекс T обозначает транспонирование матрицы. При обратном проходе необходимо вычислить

$$dX = dY W^T, \quad dW = X^T dY.$$

Для вычисления градиента параметров требуется вход прямого прохода X . Если сохранить X , он должен оставаться в памяти с момента прямого прохода до завершения его использования при обратном проходе. Если освободить X , перед использованием при обратном проходе соответствующую часть прямого прохода придётся повторить, начиная с сохранённого входа или промежуточного результата. Выбор сохраняемых и повторно вычисляемых значений зависит от объёма сэкономленной памяти и объёма работы, необходимой для повторного вычисления.

Пример: сколько памяти активаций можно сэкономить, повторно вычислив

Хранить результат прямого прохода

Умножение в прямом
проходе

Ожидание обратного
прохода

Обратный проход
down-projection

Всегда хранить a, u

h : 6 MiB

Рисунок 10-12. Сохранение произведения h : синие полосы обозначают постоянно сохраняемые a, u , а оранжевая полоса — произведение h , которое хранится от прямого прохода до обратного прохода нисходящей проекции. Для micro-batch из 128 токенов используется FP32; форма h равна $[128, 12288]$, а объём — 6 MiB.

вентильное произведение перед обратным распространением? Многослойный перцептрон Qwen3-8B (MLP, то есть подслей прямой связи в этом примере) сначала получает результат вентиля a и результат восходящей проекции u , затем формирует $h = a \odot u$ и наконец вычисляет $Y = hW_{\text{down}}$. Для градиента параметров нисходящей проекции требуется h . Если a, u уже сохранены для обратного прохода нелинейного оператора, перед обратным проходом нисходящей проекции их можно снова перемножить, не сохраняя h всё это время.

После перехода к восстановлению достаточно сгенерировать h непосредственно перед началом обратного прохода нисходящей проекции и хранить его до завершения этого этапа вычислений. Размер буфера рассчитывается следующим образом: для micro-batch из 128 токенов и активаций FP32 форма h равна $[128, 12288]$, поэтому он занимает $128 \times 12288 \times 4 = 6$ MiB; для восстановления требуется около 1,57 миллиона умножений. Два взвешенных выхода нормализации в том же слое имеют форму $[128, 4096]$ и занимают по 2 MiB; их также можно восстановить из сохранённых результатов нормализации и весов. Таким образом, на каждом слое экономится $6 + 2 + 2 = 10$ MiB, а на состоящей из девяти слоёв стадии конвейера для каждого micro-batch экономится 90 MiB, или около 94 MB.

В исходной схеме входы, сохраняемые для этих GEMM, суммарно занимают 15 MiB на слой: 10 MiB приходится на три указанных произведения, а оставшиеся 5 MiB — на Q, K, V и выход внимания. Поэтому для стадии из девяти слоёв объём уменьшается со 135 MiB до 45 MiB, то есть примерно со 142 MB до 47 MB. Восстановление выполняется последовательно по слоям, а наибольшая рабочая область для произведения занимает

Восстановление произведения перед обратным проходом

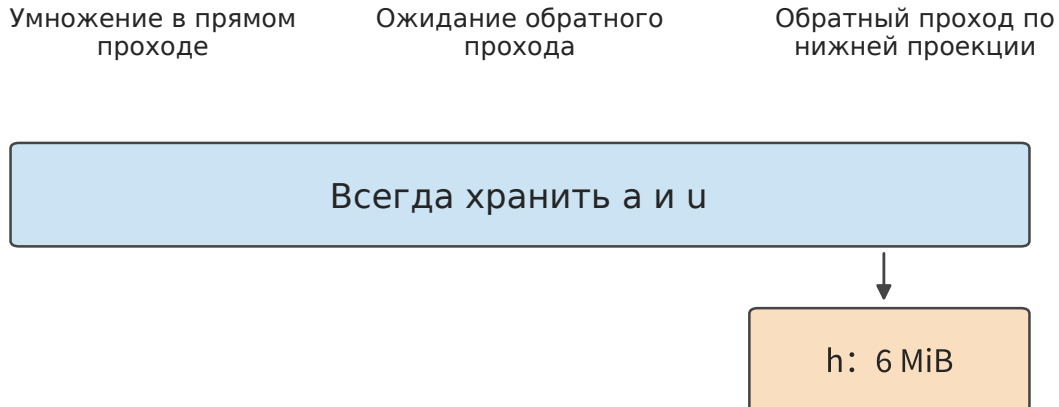


Рисунок 10-13. Сохраняются a , u , а перед использованием при обратном проходе они повторно перемножаются для получения h . Оранжевый буфер объёмом 6 MiB существует лишь непродолжительное время непосредственно до и после использования; горизонтальная ось обозначает порядок операций.

6 MiB; после завершения работы предыдущего слоя следующий повторно использует ту же область.¹⁰

Если четыре `micro-batch` одновременно ожидают обратного прохода, объём данных, которые требуется хранить длительное время, уменьшается на $4 \times 90 = 360$ MiB, или примерно на 377 MB, а во время обратного прохода восстановление выполняется поочерёдно для каждого `micro-batch`. Выигрыш здесь обусловлен различием жизненных циклов данных: чем больше `micro-batch` ожидают обратного прохода, тем больше данных необходимо хранить длительное время; при послойном восстановлении достаточно многократно использовать одну и ту же рабочую область. Если расписание конвейера позволяет раньше начать обратный проход, число ожидающих его `micro-batch` также уменьшается. Следовательно, повторное вычисление и расписание конвейера совместно определяют объём сохраняемых данных, а влияние дополнительных вычислений на продолжительность шага обучения зависит от их положения на временной шкале.

10.2.3 Выгрузка между CPU и GPU и время использования буферов

Если затраты на восстановление данных из сохранённых результатов невелики, подходит повторное вычисление. Другой способ — сохранить сами данные, переместив их в

¹⁰Событийная модель заполнения и опустошения, событийная модель 1F1B, зависимости сохранения входных тензоров GEMM, выборочное повторное вычисление. Для входных данных GEMM используется эталонное представление FP32; восстанавливаемые произведения девяти слоёв занимают 90 MiB, а максимальная рабочая область для послойного восстановления — 6 MiB. На схеме конвейера показаны промежуточные результаты, сохраняемые стратегией `save_nonlinear`, и их буферы отправки и приёма.

более объёмную память хоста, а перед использованием вернуть на GPU. Это называется **выгрузкой**. При использовании выгрузки исходная проблема нехватки видеопамати превращается в вопрос о том, успеет ли завершиться передача данных. Пусть объём состояния равен V , эффективная пропускная способность соединения — B , а между моментом, когда передачу можно начать, и моментом, когда данные понадобятся оператору, имеется W секунд. Тогда время передачи, выходящее за пределы этого интервала, равно

$$T_{\text{wait}} = \max(0, V/B - W)$$

и соответствующему оператору придётся ожидать в течение этого времени. Степень перекрытия передачи и вычислений зависит от того, когда данные становятся доступны для отправки, когда освобождается соединение и когда эти данные требуются соответствующему оператору. При DMA перемещение данных инициируется устройством, и исходные данные должны сохраняться на протяжении передачи. Исходный буфер должен оставаться действительным до завершения DMA, а целевой буфер занимает память уже с момента начала приёма; таким образом, выгрузка изменяет время использования буферов на обеих сторонах.

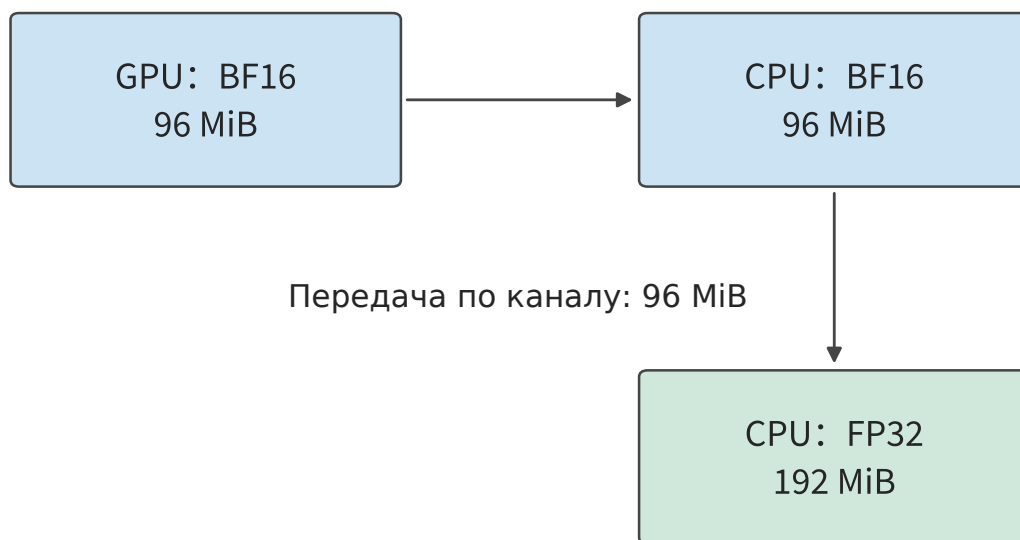
Пример: где преобразовать градиенты для более быстрой выгрузки — на CPU или GPU? Градиент вентильной проекции имеет форму [12288, 4096] и размер $G = 96$ MiB в BF16 или 192 MiB в FP32. CPUAdam — реализация обновления Adam на CPU, использующая градиенты FP32. Можно сначала передать BF16, а затем выполнить преобразование на CPU, либо сначала преобразовать данные на GPU и передать FP32. В обоих случаях преобразование должно прочитать G и записать $2G$, то есть суммарно обратиться к $3G$ байт. Преобразование ограничено пропускной способностью памяти: для GPU используется показатель RTX 4090, равный 1008 GB/s, а для CPU — 307.2 GB/s для одного сокета Xeon четвёртого поколения с восемью каналами DDR5-4800. Обозначим пропускную способность преобразования на CPU и GPU через C_c, C_g , а эффективную пропускную способность соединения — через B . Тогда время последовательного выполнения путей равно

$$T_c = G/B + 3G/C_c, \quad T_g = 3G/C_g + 2G/B.$$

Различие между рисунками 10-14 и 10-15 сосредоточено в стрелках, пересекающих соединение: хотя преобразование на GPU выполняется быстрее, по соединению приходится дополнительно передать 96 MiB. Чем медленнее соединение, тем вероятнее эта дополнительная передача сведёт на нет выигрыш от преобразования.

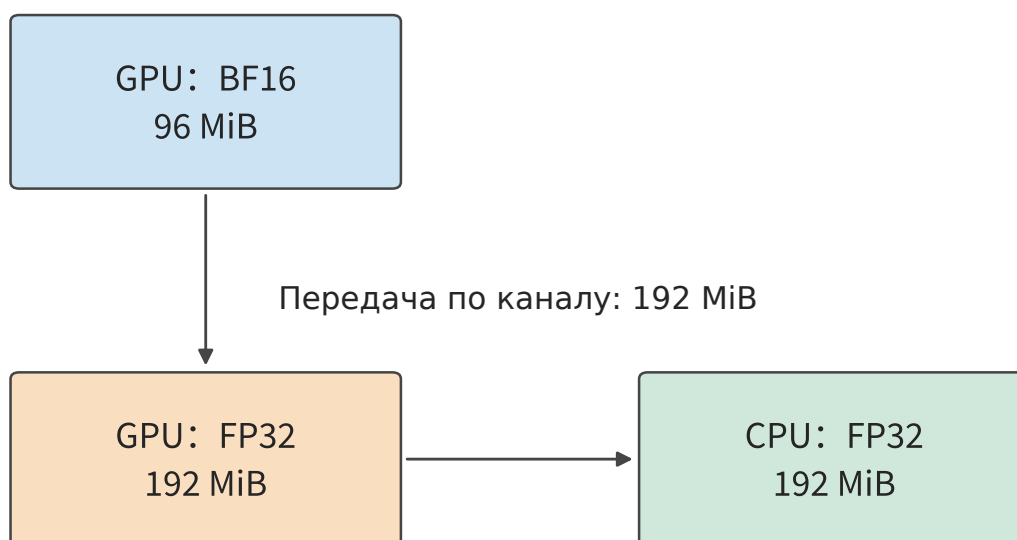
RTX 4090 подключается к хосту через PCIe 4.0 x16 с пропускной способностью 32 GB/s в каждом направлении. Для этого соединения время пути через CPU составляет примерно $3.15 + 0.98 = 4.13$ ms, а пути через GPU — примерно $0.30 + 6.29 = 6.59$ ms. Преобразование на пути через GPU быстрее примерно на 0.68 ms, однако передача занимает примерно на 3.15 ms больше, поэтому весь путь оказывается медленнее. Если вместо этого используется NVLink-C2C между CPU и GPU в GH200 с пропускной способностью 450 GB/s в каждом направлении, передача дополнительных 96 MiB увеличивает время лишь примерно на 0.22 ms. Время двух путей становится равным примерно 1.21 ms и 0.75 ms, поэтому преобразование на GPU даёт заметный выигрыш.¹¹

¹¹Перерасчёт преобразования градиентов, вариант с быстрым каналом связи, разбор SuperOffload и пример преобразования, журнал выгрузки CPUAdam. Путь преобразования начинается с момента готовности



Преобразование формата на CPU

Рисунок 10-14. Сначала градиент BF16 объёмом 96 MiB передаётся на CPU, а затем преобразуется на CPU в FP32 объёмом 192 MiB. Стрелки обозначают поток данных; входные и выходные данные преобразования находятся на стороне CPU.



При преобразовании GPU занято 288 MiB

Рисунок 10-15. Сначала градиент BF16 объёмом 96 MiB преобразуется на GPU в FP32 объёмом 192 MiB, а затем передаётся на CPU. Во время преобразования входные и выходные данные существуют одновременно, поэтому пиковое потребление памяти GPU составляет 288 MiB.

Приравняв время выполнения двух путей, можно найти пороговую пропускную способность, при которой меняется соотношение их скоростей:

$$B^* = \frac{1}{3(1/C_c - 1/C_g)} \approx 147 \text{ GB/s.}$$

Этот порог выражает в одном масштабе два противоположных эффекта: более быстрое преобразование на GPU сокращает время изменения формата, но больший объём данных FP32 увеличивает время передачи. Чем быстрее соединение, тем меньше цена второго эффекта. 147 GB/s превышает пропускную способность PCIe 5.0 x16, равную 64 GB/s в каждом направлении, поэтому в этом примере для всех подключённых через PCIe GPU следует сначала передавать BF16, а затем преобразовывать данные на CPU.

Ёмкость видеопамати накладывает ещё одно ограничение. Во время преобразования на GPU исходные 96 MiB и выходные 192 MiB существуют одновременно, поэтому пиковое потребление составляет 288 MiB. При передаче BF16 на CPU достаточно сохранить на GPU только участвующий в текущем преобразовании градиент объёмом 96 MiB. Для преобразования на GPU требуется дополнительный буфер объёмом 192 MiB. SuperOffload — система выгрузки при обучении, предназначенная для тесно интегрированных CPU и GPU, таких как GH200. Она совместно планирует место преобразования и выполнение оптимизатора на хосте, используя именно компромисс между скоростью преобразования, объёмом передаваемых данных и временем использования буферов. Выигрыш для всего шага определяется тем, когда завершится обработка последнего буфера градиентов, обновление параметров на CPU и обратная передача весов.

10.2.4 Распределение состояния по GPU и топология соединений в комбинированных схемах параллелизма

Шардирование, повторное вычисление и выгрузка изменяют соответственно объём хранения, объём вычислений и объём передачи данных. При объединении этих методов в целостную систему необходимо также определить, к каким GPU и коммуникационным группам они применяются. Рассмотренные в главах 6 и 7 методы параллелизма разделяют при обучении различные данные и вычисления: TP разделяет матрицы внутри слоя, PP — слои, DP — примеры, а EP — экспертов. Сводная таблица в разделе 6.1.4 также включает SP и CP: первый шардирует активации операторов, обрабатывающих отдельные токены, вдоль последовательности и обычно повторно использует группу тензорного параллелизма; второй распределяет контекст внимания одной длинной последовательности между несколькими GPU, сохраняя зависимости между позициями. Эти методы параллелизма продолжают подход «разделение на блоки — распределение входов — объединение результатов» из главы 5, а при обучении градиенты распространяются в обратном направлении по тому же графу вычислений. Если сначала определить, кто обновляет каждую часть параметров, а затем в соответствии с зависимостями прямого и обратного проходов организовать сбор, обмен и редукцию, можно одновременно

градиентов на GPU и заканчивается, когда градиенты FP32 становятся доступны CPU для чтения; оба пути используют закреплённую память хоста в качестве буфера. Для пропускной способности преобразования на GPU принято значение 1008 GB/s, соответствующее пропускной способности видеопамати RTX 4090 (calculations/configs/hardware.json); для CPU принято значение восьмиканальной DDR5-4800 из спецификации Intel Xeon Platinum 8480+: $8 \times 4800 \text{ MT/s} \times 8 \text{ B} = 307.2 \text{ GB/s}$. Оба значения являются верхними границами, рассчитанными по пропускной способности памяти. Значение 450 GB/s в каждом направлении для NVLink-C2C приведено в описании архитектуры GH200.

получить объём памяти на каждом GPU и объём трафика на каждом интерфейсе.

Три способа оптимизации ёмкости можно сравнить с помощью одного набора вопросов.

Метод	Сокращаемое потребление видеопамати	Дополнительная работа	Временная зависимость, определяющая выигрыш
Шардирование состояния	Дублируемые веса, градиенты или состояние оптимизатора	Сбор параметров, редукция и шардирование градиентов	Успевает ли завершиться сбор до выполнения модуля
Повторное вычисление активаций	Промежуточные значения, сохраняемые между прямым и обратным проходами	Восстановление значений прямого прохода	Задерживает ли восстановление обратный проход на критическом пути
Выгрузка состояния	Временно неиспользуемое состояние на ускорителе	Передача между хостом и ускорителем	Успевают ли данные вернуться до использования

Пример: если видеопамати недостаточно, следует увеличить число GPU для шардирования или повторно вычислять активации? Номинальный объём памяти RTX 4090 составляет 24 GB, или около 22.35 GiB; за вычетом потребления среды выполнения будем считать доступными 22 GiB на GPU. При использовании Qwen3-8B с ZeRO-3 на восьми GPU каждый GPU хранит около 15.3 GiB, и для остальных тензоров и буферов остаётся около 6.7 GiB. Если активациям и временным буферам одновременно требуется 10 GiB, общий объём составит около 25.3 GiB, что превышает доступный примерно на 3.3 GiB. Эти 3.3 GiB можно сэкономить с помощью повторного вычисления либо увеличить число GPU до шестнадцати, уменьшив постоянно размещённое состояние обучения примерно до 7.6 GiB, а общий объём — примерно до 17.6 GiB.

Эти два варианта имеют разную цену: схема на восьми GPU добавляет вычисления для восстановления, а схема на шестнадцати GPU увеличивает число GPU и изменяет коммуникационную группу. Сначала на рисунке можно найти варианты, удовлетворяющие требованиям по видеопамати, а затем сравнить время выполнения шага для этих вариантов.

Для рассматриваемой в этой главе задачи обучения схемы на 32 и 48 GPU используют ZeRO-3 для всей группы, TP=PP=1, а micro-batch на каждом GPU состоит из одной последовательности. Как и в предыдущем примере, доступная видеопамать принимается равной 22 GiB на GPU, а для одновременно хранимых активаций, полных параметров модуля и рабочей области устанавливается предел в 10 GiB. Постоянно размещённое состояние обучения в двух схемах составляет соответственно около 3.8 и 2.5 GiB, общий объём — около 13.8 и 12.5 GiB, а запас ёмкости — около 8.2 и 9.5 GiB. Видеопамати достаточно в обеих схемах, поэтому далее главным критерием сравнения становится время завершения.¹²

¹²Входные данные, выводы и перерасчёты примера проектирования из этой главы. Запасы ёмкости, ожидание входных данных и плановые остановки для 32/48 карт являются заданными входными данными; вычислительная эффективность одной карты взята из MFU Llama 3; объём матричных вычислений модели получен из существующего расчёта training-deadline, а объём коммуникации ZeRO-3 выведен из той же таблицы параметров. PCIe 4.0 и отсутствие NVLink у RTX 4090 указаны на странице спецификаций RTX 4090; отсутствие поддержки P2P между картами и конфигурация восьмикарточного хоста с восемью сетевыми картами по 200 Gbit/s приведены в анализе конфигурации кластера 4090; согласно \$3.2 MegaScale, в пределах одного шага невозможно скрыть только первый AllGather и последний ReduceScatter. Значения по отдельным компонентам и независимые перерасчёты сохранены в design-case.json.

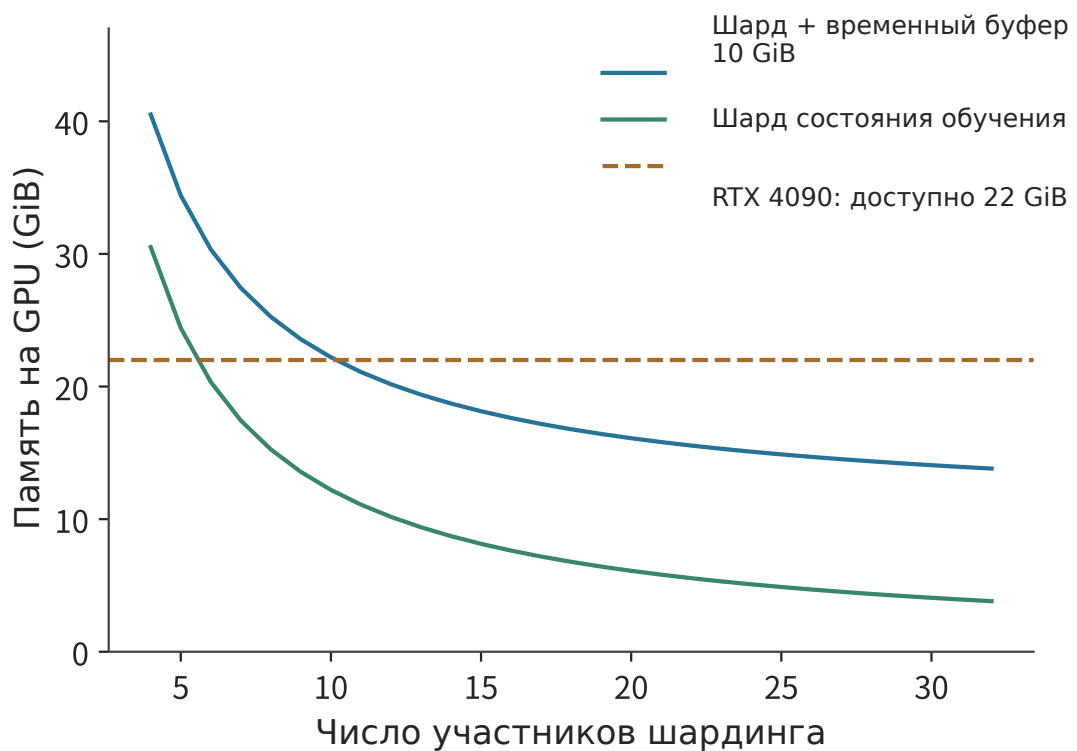


Рисунок 10-16. При заданной дополнительной потребности в памяти число шардов определяет, какие варианты удовлетворяют требованиям по ёмкости. Кривая соответствует $16N/d + 10$ GiB, а горизонтальная линия — доступной видеопамати RTX 4090 объёмом 22 GiB на GPU. В области, где кривая находится ниже бюджета, помещаются указанное состояние обучения и буферы; расстояние по вертикали показывает запас ёмкости.

10.3 Конвейерное планирование шага обучения и перекрытие коммуникаций

Анализ ёмкости определяет, способна ли видеопамять вместить необходимые данные, а анализ времени — успеют ли эти данные поступить. В этом разделе сначала определяется целевая функция, оптимизируемая за одну итерацию обучения, а затем прямой проход, обратный проход и коммуникации размещаются на одной временной шкале. В рассматриваемом в этой главе проектом примере используется параллелизм по данным; четырёхстадийный конвейер рассматривается отдельно, чтобы показать, как планирование одновременно изменяет критический путь и жизненный цикл активаций.

10.3.1 Micro-batch, накопление градиентов и обновление параметров

Глобальный batch содержит все образцы, используемые для одного обновления параметров. Его можно разделить на несколько micro-batch, последовательно выполнить для них обратное распространение, накопить градиенты и затем один раз обновить параметры. Пусть S_k — множество индексов действительных токенов в micro-batch k , общее число действительных токенов равно $L = \sum_k |S_k|$, тогда средняя по токенам функция потерь имеет вид

$$\mathcal{L} = \frac{1}{L} \sum_k \sum_{j \in S_k} \ell_j.$$

Суммирование и дифференцирование можно переставить местами, поэтому обратное распространение можно выполнять отдельно для каждого micro-batch; при вычислении функции потерь каждого micro-batch сумма его потерь делится на одно и то же L . После завершения накопления градиентов всех micro-batch один раз выполняется обрезка градиентов и один раз — обновление параметров. Таким образом, разбиение на micro-batch изменяет только порядок выполнения, но не вес каждого токена в функции потерь.

Пример: почему усреднение по micro-batch и по токенам даёт разные градиенты? Из восьми ответов четыре содержат по два действительных токена, а ещё четыре — по три, всего 20 токенов. Пусть потеря на один токен короткого ответа равна $\theta/5$, а длинного — $2\theta/5$, тогда

$$\mathcal{L} = \frac{8\theta/5 + 12(2\theta/5)}{20} = \frac{8\theta}{25}, \quad \frac{d\mathcal{L}}{d\theta} = 0.32.$$

Если сначала вычислить среднюю потерю каждого ответа, а затем усреднить её по восьми ответам, короткие и длинные ответы получают по половине общего веса, и градиент станет равен $(0.2 + 0.4)/2 = 0.30$. При исходном усреднении по токенам на длинные ответы приходится $12/20 = 60\%$, поэтому результат ближе к 0.4. Если уже использован общий знаменатель 20, а после накопления результат ещё раз делится на восемь, градиент становится равен 0.04. Знаменатель определяет вклад каждого токена в градиент параметров.¹³

¹³Реальный замкнутый цикл фиксированной малой модели verl, проверка исходного кода функции потерь и обновления. В фиксированной конфигурации обучения Qwen2.5-0.5B-Instruct используется одна карта, NO_SHARD и no_sync=False; исходный код и зафиксированные версии приведены в журнале по ссылке. Значения 0,32/0,30/0,04 выведены на основе реальной структуры длин и учебного скалярного значения функции потерь.

Вернёмся к проектному примеру: для 384 последовательностей полной длины из раздела 10.1.3 на каждой из 32 карт накапливаются 12 micro-batch, а на каждой из 48 карт — 8. За один раз каждая карта обрабатывает только одну последовательность, и формы матриц, обрабатываемых каждой картой, одинаковы; глобальное число действительных токенов и целевая функция также одинаковы. Сбор параметров и коммуникации градиентов отдельно размещаются на временной шкале в соответствии с путём выполнения.

10.3.2 Конвейерное планирование, пузыри и жизненный цикл активаций

Разбиение на micro-batch сохраняет цель обучения, но изменяет порядок вычислений. В конвейерном параллелизме этот порядок также определяет, когда каждый этап может начать вычисления и сколько наборов активаций ожидают обратного прохода. Конвейерный параллелизм делит модель на несколько этапов: каждый micro-batch последовательно проходит прямой проход, после чего градиенты распространяются в обратном направлении. При планировании **fill–drain** сначала для всех micro-batch полностью выполняется прямой проход, а затем централизованно выполняется обратный. При **1F1B (one forward, one backward)** после прогрева конвейера чередуются один прямой и один обратный проход, благодаря чему активации более ранних micro-batch освобождаются быстрее.

Скорость освобождения активаций непосредственно определяет пиковое потребление видеопамати. Для обратного распространения нужны активации, сохранённые во время прямого прохода, поэтому активации каждого micro-batch должны оставаться в видеопамати с момента завершения прямого прохода до прихода соответствующего обратного прохода. Сколько micro-batch одновременно ожидают обратного прохода, столько наборов активаций и находится в видеопамати. При fill–drain первый обратный проход начинается лишь после завершения прямого прохода всех m micro-batch, поэтому пиковое потребление линейно растёт с m ; если оно превышает ёмкость видеопамати, шаг обучения выполнить невозможно и приходится уменьшать число одновременно находящихся в конвейере micro-batch, сокращать последовательность либо использовать более дорогое повторное вычисление. Раннее выполнение обратного прохода сокращает ожидание активаций: после снижения пика шаг обучения становится выполнимым, а освободившийся резерв можно использовать для увеличения micro-batch или длины последовательности. Поэтому резидентность в видеопамати часто определяет не скорость, а саму возможность выполнения.

При сравнении любых двух вариантов планирования необходимо одновременно учитывать две оси: **время завершения**, зависящее от продолжительности простоя конвейера, то есть пузырей, и **резидентность в видеопамати**, зависящую от того, как долго каждую активацию нужно сохранять в ожидании её обратного прохода.

Сначала предположим, что p этапов полностью сбалансированы, прямой проход каждого этапа занимает t_f , обратный — t_b , а время передачи данных и обновления параметров равно нулю. Для m micro-batch планирование fill–drain требует

$$T = (m + p - 1)(t_f + t_b), \quad u = \frac{m}{m + p - 1}.$$

Член m соответствует фактическим вычислениям micro-batch, а $p - 1$ возникает из-за заполнения и опустошения конвейера. Поэтому временная утилизация карт u растёт с числом micro-batch. При $p = 4, m = 8$ получаем $u = 8/11 \approx 72.7\%$. Если глобаль-

ный batch size остаётся неизменным, увеличение числа micro-batch означает уменьшение каждого из них; сокращение времени простоя конвейера необходимо сопоставлять со временем выполнения меньших матриц.

Пример: если пузыри одинаковы, что даёт 1F1B? Разделим 36 слоёв Qwen3-8B на четыре этапа по девять слоёв, а каждый из восьми micro-batch будет содержать одну последовательность из 128 токенов. Прямой проход каждого этапа занимает 10 ms, обратный — 20 ms, каждая передача через границу — 1 ms, каналы прямого и обратного направлений независимы, а обновление параметров занимает 1 ms. Без учёта передачи данных планирование fill–drain занимает $(8 + 4 - 1) \times 30 = 330$ ms; заполнение прямого прохода через три границы добавляет 3 ms, опустошение обратного прохода — ещё 3 ms, а обновление параметров — 1 ms, итого 337 ms.

При 1F1B обратный проход начинается раньше, поэтому последующие прямые проходы должны ожидать, пока обратный проход освободит вычислительные ресурсы. На этапе 0 прямой проход первых четырёх micro-batch завершается к 40 ms, а первый возвращаемый градиент поступает к 106 ms. Затем до 126 ms выполняется обратный проход, и лишь после этого начинается прямой проход пятого micro-batch. Такое чередование последовательно влияет на время поступления данных на следующие этапы. На этапе 3 обратный проход второго micro-batch завершается к 93 ms, однако входные данные для третьего прямого прохода поступают лишь к 95 ms, из-за чего возникает промежуток в 2 ms; последующие аналогичные ожидания продолжают удлинять критический путь. После упорядочивания выполнения в соответствии с этими зависимостями вычисление всех градиентов завершается к 346 ms, а вместе с обновлением параметров общее время составляет 347 ms.¹⁴

Сравнение рисунков показывает: оба варианта планирования выполняют одинаковую работу прямого и обратного проходов, а время пузырей в обоих случаях равно $(p - 1)(t_f + t_b)$; из-за ожидания зависимостей, вносимого чередованием (рис. 10-19), 1F1B тратит примерно на 3% больше времени. Его преимущество видно на рис. 10-20: максимальное потребление видеопамати промежуточными результатами и буферами снижается примерно с 2.57 GB до 0.97 GB, то есть приблизительно на 62%. Поэтому утверждение «1F1B лучше fill–drain» справедливо только по оси видеопамати, тогда как по времени он, напротив, немного медленнее. Если для этих промежуточных результатов и буферов доступен только 1 GB, этот шаг обучения можно выполнить лишь при планировании с ранним обратным проходом; при достаточной ёмкости памяти fill–drain завершится раньше — за 337 ms. Выбор варианта планирования определяется одновременно тем, когда освобождаются тензоры и когда поступают следующие входные данные.

Чтобы сократить время завершения, необходимо сжать сами пузыри: чем мельче разбит обратный проход и чем больше micro-batch одновременно движется по конвейеру, тем полнее можно заполнить промежутки. Следующие три вида планирования основаны на приведённом выше анализе зависимостей и уменьшают пузыри соответственно ценой **числа коммуникаций**, **пикового объёма активаций** и **числа параметров**. Во всех трёх используется тот же пример: четыре этапа, прямой проход 10 ms, обратный проход 20 ms, передача через границу 1 ms и обновление параметров 1 ms. Порядок вы-

¹⁴Событийная модель заполнения и опустошения, событийная модель 1F1B, зависимости сохранения входных тензоров GEMM, выборочное повторное вычисление. Для входных данных GEMM используется эталонное представление FP32; восстанавливаемые произведения девяти слоёв занимают 90 MiB, а максимальная рабочая область для послойного восстановления — 6 MiB. На схеме конвейера показаны промежуточные результаты, сохраняемые стратегией save_nonlinear, и их буферы отправки и приёма.

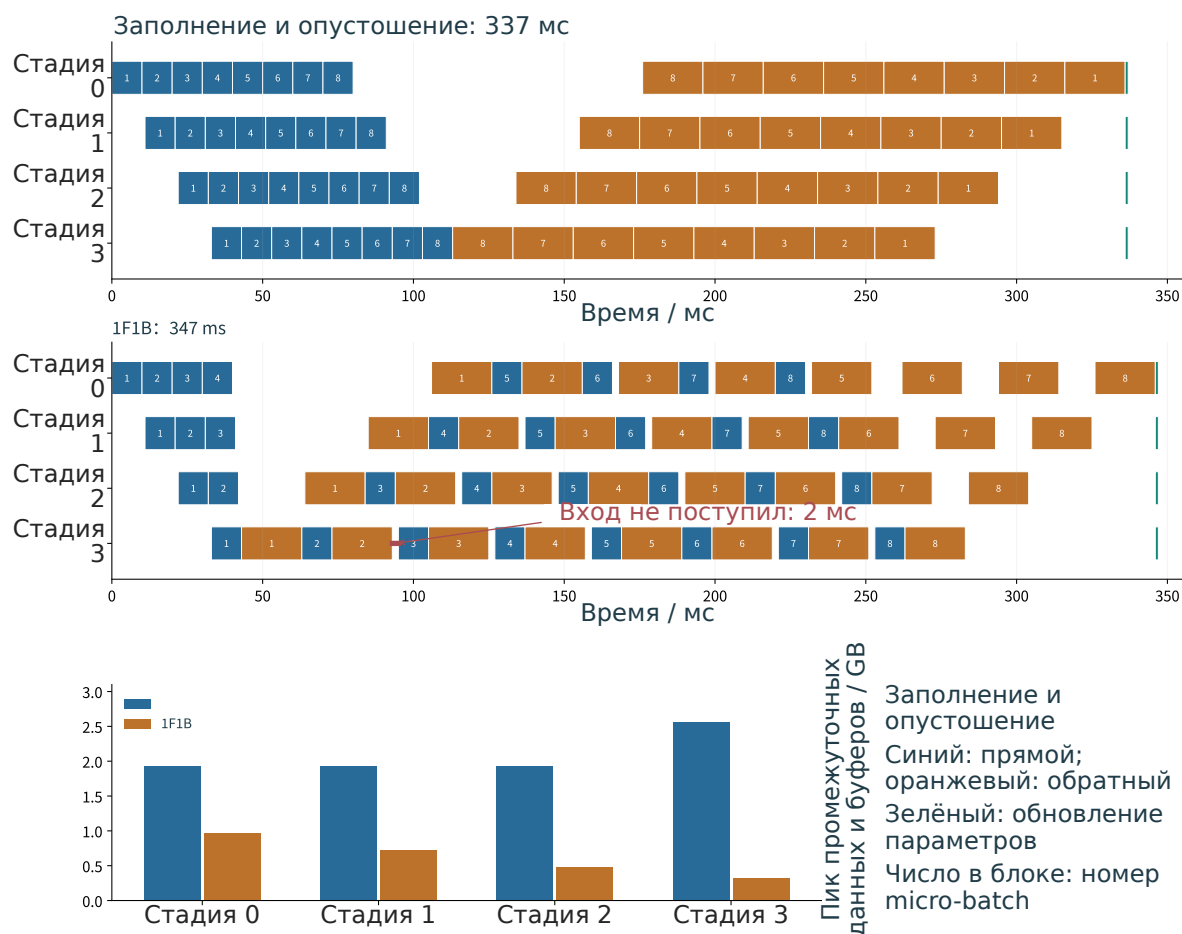


Рис. 10-17. При fill-drain сначала выполняется прямой проход восьми micro-batch, затем обратный проход и, наконец, обновление параметров. Синим обозначен прямой проход, оранжевым — обратный, зелёным — обновление параметров. Для четырёх этапов используется единая временная шкала; при указанных в тексте условиях вычислений и передачи данных общее время составляет 337 ms.

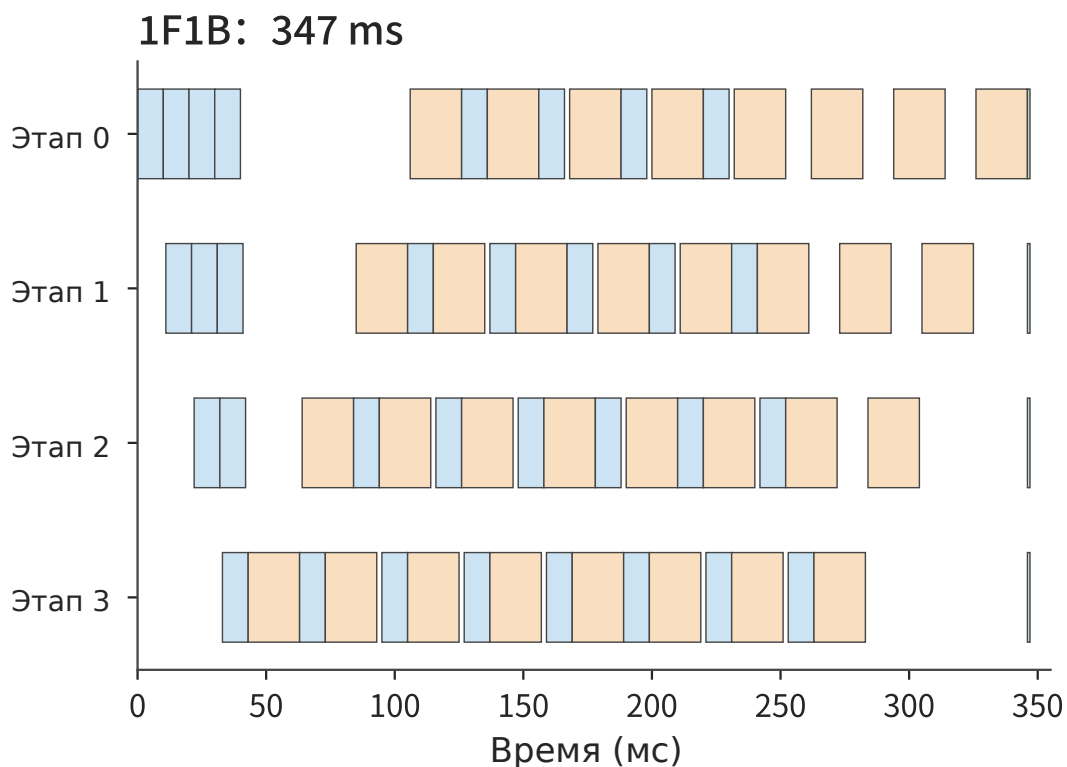


Рис. 10-18. После прогрева 1F1B чередует прямой и обратный проходы; время завершения в этом примере составляет 347 ms. Синим обозначен прямой проход, оранжевым — обратный, зелёным — обновление параметров; используется та же временная шкала, что и на предыдущем рисунке.

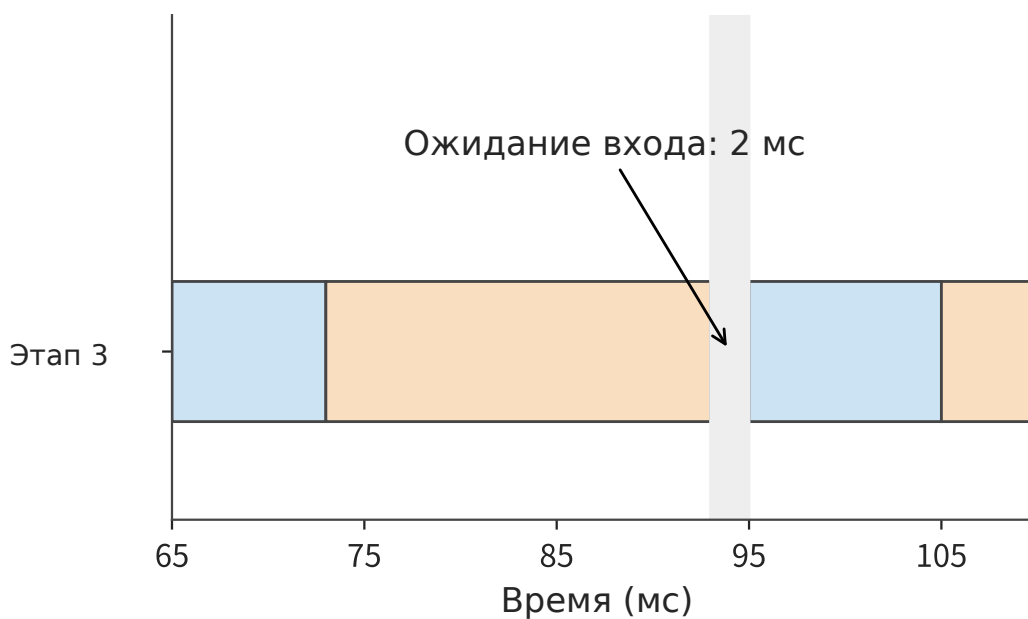


Рис. 10-19. Увеличенный фрагмент этапа 3 в 1F1B: обратный проход второго micro-batch завершается к 93 ms, а следующие входные данные прямого прохода поступают к 95 ms, создавая ожидание в 2 ms.

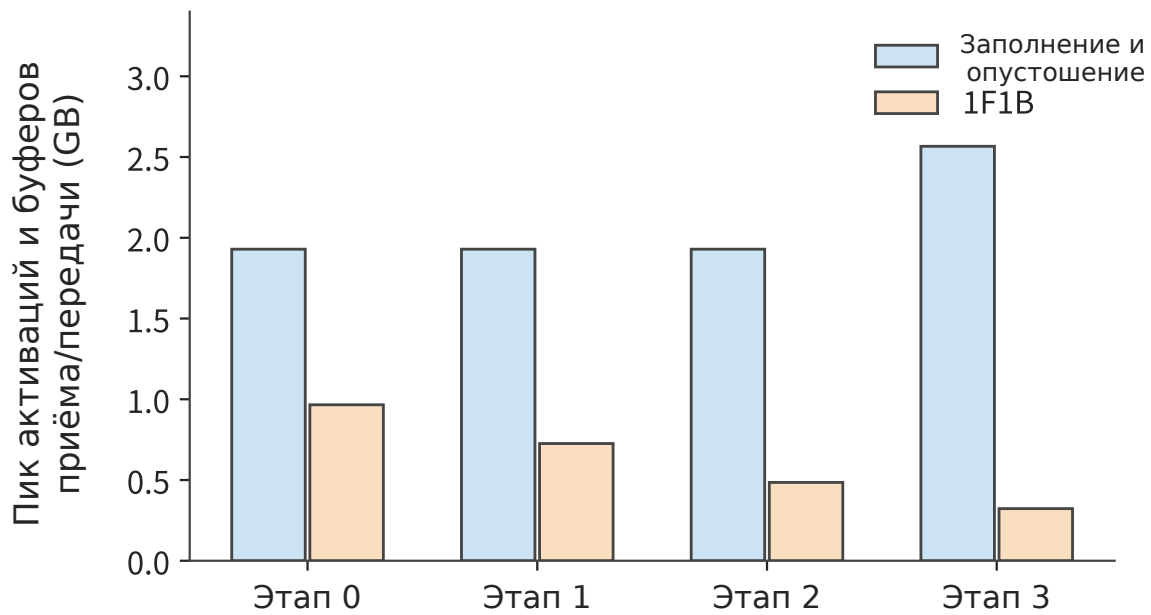


Рис. 10-20. Пиковое потребление памяти активациями и буферами приёма и передачи на каждом этапе для двух вариантов планирования. Ранний обратный проход позволяет раньше освободить активации, снижая максимум примерно с 2.57 GB до 0.97 GB.

полнения каждого micro-batch на каждом этапе, то есть порядок слотов, определяется событийной моделью, а формулы пузырей и способ разбиения обратного прохода взяты из опубликованных статей.¹⁵

Чередующийся 1F1B (interleaved 1F1B, также называемый виртуальным конвейером) делит 36 слоёв на восемь блоков, циклически распределённых между четырьмя картами. Каждая карта получает $v = 2$ несмежных блока, в сумме по-прежнему содержащих девять слоёв: первые четыре блока содержат по пять слоёв, последние четыре — по четыре; карта 0 выполняет слои 0–4 и 20–23, а остальные карты — последующие блоки по той же схеме. Поэтому прямой проход каждого micro-batch пересекает границы $2p - 1 = 7$ раз вместо 3, а доля пузырей уменьшается с $(p - 1)/m$ до

$$\frac{1}{v} \cdot \frac{p - 1}{m}.$$

При восьми micro-batch время завершения снижается с 347 ms до 298 ms, а простой каждой карты — примерно с 106 ms до 57 ms. Цена состоит из двух частей: число передач возрастает более чем вдвое — количество сообщений прямого прохода за весь шаг

¹⁵Событийная модель чередующегося расписания, событийная модель без пузырей, событийная модель DualPipe и варианты с шестнадцатью микробатчами: чередующийся, без пузырей, DualPipe. Формулы пузырей: для 1F1B и чередующегося расписания см. статью Megatron-LM — $(p - 1)/m$ и $(1/v)(p - 1)/m$; для ZB-H1/H2 см. таблицу 2 статьи о нулевых пузырях; количество параметров и экземпляров активаций DualPipe см. в таблице 2 технического отчёта DeepSeek-V3; определение установившегося режима 1F1B см. в PipeDream. Расписание без пузырей построено по порядку слотов ZB-H1 из верхней части рисунка 3 статьи; архивные тексты о чередующемся расписании и DualPipe описывают только идеи разбиения на блоки и двунаправленной подачи, поэтому порядок слотов задан согласно `declared_schedule_rule`, записанному в соответствующих файлах результатов; соотношение разбиения $T_B = T_W = 10$ ms принято для этого примера; время завершения выведено событийной моделью из зависимостей, а не измерено.

увеличивается с 24 до 56, — а активациям ранних этапов приходится ждать завершения прямого прохода большего числа блоков. Пиковый объём активаций и буферов приёма и передачи на этапе 0 возрастает с 921 MiB примерно до 1,329 MiB.

Планирование **zero-bubble** делит обратный проход на две части: градиент входа dX (в статье обозначен как B) необходимо передать предыдущему этапу, обратный проход которого ожидает этот градиент, поэтому dX находится на критическом пути и должен вычисляться как можно раньше; градиент весов dW (в статье обозначен как W) нужен только для накопления градиентов текущего слоя, поэтому его можно вычислить в любой момент после соответствующего dX на этом этапе и отложить до появления свободного промежутка. Пусть прямой проход, dX и dW каждого этапа занимают соответственно T_F , T_B и T_W . Тогда пузырь 1F1B равен $(p-1)(T_F + T_B + T_W)$; ZB-H1 заполняет промежутки вычислением dW , уменьшая пузырь до $(p-1)(T_F + T_B - T_W)$; ZB-H2 дополнительно переносит больше прямых проходов на раннее время, уменьшая пузырь до $(p-1)(T_F + T_B - 2T_W)$. Два ручных варианта планирования из статьи о zero-bubble построены для $T_F = T_B = T_W$; в таблице 2 этой статьи приведена указанная выше общая форма.

В этом примере обратный проход длительностью 20 ms делится поровну: $T_B = T_W = 10$ ms. Пузырь 1F1B равен $3 \times 30 = 90$ ms, нижняя граница для ZB-H1 — $3 \times (10 + 10 - 10) = 30$ ms, а для ZB-H2 — 0. После упорядочивания событийной модели в соответствии с порядком слотов ZB-H1 на рис. 3 статьи время завершения восьми micro-batch составляет 283 ms, а простой каждой карты — 42 ms; если принять время передачи через границу равным нулю, простой в точности совпадает с нижней границей 30 ms, а дополнительные 12 ms возникают из-за передачи данных. Пиковый объём активаций и буферов приёма и передачи на этапе 0 совпадает с 1F1B и составляет 921 MiB. Цена приходится на поздние этапы: на этапе 3 вычисление dW отстаёт от dX на три micro-batch, и событийная модель сохраняет в течение этого времени полный набор активаций, поэтому пик этапа 3 возрастает с 308 MiB для 1F1B примерно до 1,226 MiB. В статье учитывается меньший объём активаций, фактически требуемый для dW , поэтому в её таблице 2 пиковый объём ZB-H1, как и у 1F1B, равен pM_B , где M_B — объём активаций одного micro-batch.

DualPipe делит micro-batch на две половины, которые вводятся с противоположных концов конвейера и движутся навстречу друг другу, а каждый этап одновременно обслуживает блоки обоих направлений. В обозначениях DeepSeek-V3 пузырь равен $(PP/2 - 1)(F \& B + B - 3W)$, где PP — число этапов конвейера, $F \& B$ — интервал перекрывающихся прямого и обратного проходов, B — полный обратный проход длительностью 20 ms, а W — входящий в него dW длительностью 10 ms. В этом примере перекрытие вычислений прямого и обратного проходов не моделируется, поэтому принимается $F \& B = F + B = 30$ ms; пузырь равен $(2 - 1) \times (30 + 20 - 30) = 20$ ms, а время завершения — 306 ms. Цена также очевидна: на каждой карте нужно хранить две копии параметров для двух направлений, поэтому объём параметров удваивается, а число сохраняемых наборов активаций равно $PP + 1$; в этом примере пиковое потребление каждого этапа составляет примерно 1,228–1,612 MiB.¹⁶

¹⁶Событийная модель чередующегося расписания, событийная модель без пузырей, событийная модель DualPipe и варианты с шестнадцатью микробатчами: чередующийся, без пузырей, DualPipe. Формулы пузырей: для 1F1B и чередующегося расписания см. статью Megatron-LM — $(p-1)/m$ и $(1/v)(p-1)/m$; для ZB-H1/H2 см. таблицу 2 статьи о нулевых пузырях; количество параметров и экземпляров активаций DualPipe см. в таблице 2 технического отчёта DeepSeek-V3; определение установившегося режима 1F1B см. в PipeDream. Расписание без пузырей построено по порядку слотов ZB-H1 из верхней части рисунка 3 ста-

В следующей таблице все пять вариантов планирования сопоставлены на одном примере. Хотя fill–drain не сжимает пузыри, он остаётся базовым вариантом при достаточной ёмкости памяти и поэтому также включён в таблицу:

ты; архивные тексты о чередующемся расписании и DualPipe описывают только идеи разбиения на блоки и двунаправленной подачи, поэтому порядок слотов задан согласно `declared_schedule_rule`, записанному в соответствующих файлах результатов; соотношение разбиения $T_B = T_W = 10$ ms принято для этого примера; время завершения выведено событийной моделью из зависимостей, а не измерено.

Планирование (восемь micro-batch)	Время завершения / ms	Простой каждой карты / ms	Пик этапа 0 / MiB	Способ сжатия пузырей	Основная цена
Fill-drain	337	96	1,840 (на этапе 3 — 2,448)	Нет	Максимальная резидентность активаций; невыполнимо при недостатке памяти
1F1B	347	106	921	Нет (пузырь такой же, как у fill-drain)	Минимальное потребление видеопамяти, но примерно на 3% медленнее fill-drain
Чередующийся 1F1B, $v = 2$	298	57	1,329	Блок слоёв каждой карты делится на v частей	Число сообщений прямого прохода увеличивается с 24 до 56
Zero-bubble (ZB-H1), $T_B = T_W$	283	42	921 (на этапе 3 — 1,226)	dW откладывается и заполняет промежутки	На поздних этапах активации нужно сохранять до выполнения dW
DualPipe	306	65	1,228 (на этапах 1 и 2 — 1,612)	Micro-batch движутся навстречу друг другу с двух концов	Две копии параметров на каждой карте

Если отобразить значения из таблицы на временных шкалах этапов, будет видно, где именно заполняются пузыри. Следующие три рисунка используют ту же временную шкалу и цвета, что и рис. 10-17 и 10-18.

Ни один вариант планирования не превосходит остальные одновременно по обоим осям. Если отдельно ранжировать пять вариантов по каждой оси, первые места займут разные варианты: минимальное время завершения обеспечивает zero-bubble, а минимальный максимальный пик отдельного этапа — 1F1B. В трёх расширенных вариантах время, полученное за счёт сжатия пузырей, оплачивается увеличением другого ресурса. Поэтому перед выбором планирования нужно определить, с какой стороны находится ограничение: по времени или по ёмкости. На рис. 10-24 обе оси показаны рядом без fill-drain; значения можно сопоставить с приведённой выше таблицей.

Если удвоить число micro-batch до 16, время завершения 1F1B, чередующегося варианта, zero-bubble и DualPipe составит соответственно 599, 538, 523 и 558 ms; порядок не изменится. Целесообразность сжатия пузырей зависит от соотношения сэкономленного времени пузырей и новых издержек. С увеличением m доля пузырей $(p - 1)/m$ сама по себе уменьшается: при 16 micro-batch доля пузырей 1F1B снижается с 37.5% до 18.8%, поэтому относительная выгода от более мелкого разбиения сокращается. При этом дополнительное число коммуникаций в чередующемся варианте, дополнительная резидентность активаций в zero-bubble и дополнительный объём параметров в DualPipe не уменьшаются с ростом m . Конкуренция за канал из раздела 10.3.3 задаёт ещё одну границу: четыре дополнительные межэтапные передачи каждого micro-batch в чередующемся варианте непосредственно удлиняют критический путь, если канал уже полностью занят редукцией градиентов. При дефиците памяти порядок выбора меняется: сначала согласно разделу 10.2.4 исключаются варианты с невыполнимым пиковым потреблением, после чего сравнивается время завершения.

10.3.3 Зависимости обратного прохода, группировка градиентов в буферы и перекрытие коммуникаций

Промежуток в 2 ms из предыдущего раздела возникает из-за того, что входные данные ещё не поступили. Коммуникации градиентов подчиняются тому же правилу: перед отправкой необходимо дождаться появления данных, а при отправке — освобождения канала. Обратное распространение послойно создаёт градиенты; в реализациях несколько тензоров градиентов обычно объединяются в один коммуникационный буфер, который отправляется после вычисления всех входящих в него градиентов. Чем больше буфер, тем вероятнее ожидание градиента, создаваемого последним; чем меньше буфер, тем больше раз приходится запускать коммуникацию. Планирование должно найти баланс между стоимостью запуска и ранней отправкой.

Пусть редукция занимает 3 ms, а после готовности градиента остаётся ещё 5 ms независимых от неё вычислений, с которыми редукция может выполняться одновременно. Если до этого общий канал уже на 4 ms занят обменом между экспертами, для перекрытия редукции остаётся только 1 ms, а ещё 2 ms продолжаются после завершения вычислений. Эта часть не может быть перекрыта вычислениями и удлиняет шаг обучения; она называется **временем ожидания коммуникации**. Зависимости данных задают самое раннее время начала, конкуренция за ресурсы отодвигает его, а зависимости последующих операторов определяют, удлинит ли это ожидание весь шаг обучения.

На рис. 10-25 момент появления градиента временно считается фиксированным. В

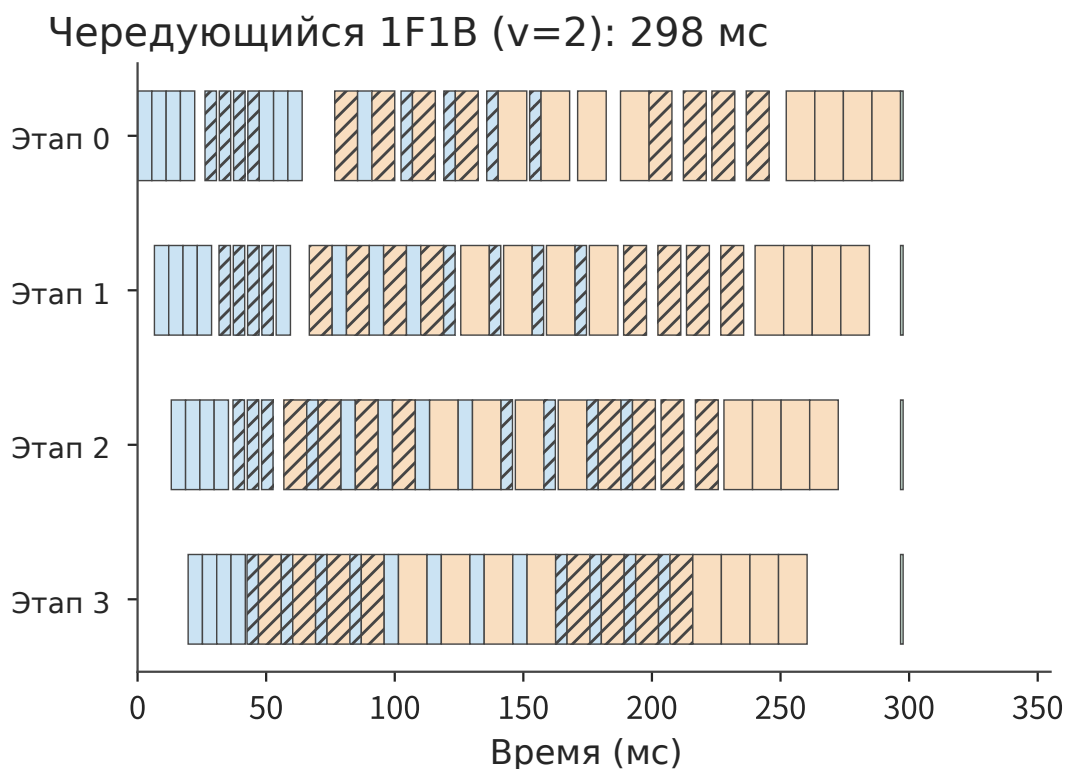


Рис. 10-21. Временная шкала этапов чередующегося 1F1B ($v = 2$), время завершения — 298 ms. Синим обозначен прямой проход, оранжевым — обратный, зелёным — обновление параметров, а заштрихованные блоки соответствуют второму блоку слоёв на каждой карте. Каждый блок примерно вдвое короче блока на рис. 10-18; промежутки во время прогрева и опустошения заполняются вычислениями другого блока слоёв, поэтому первый прямой проход этапа 3 начинается примерно в 20 ms вместо 33 ms. Каждый micro-batch должен пересечь на четыре границы больше.

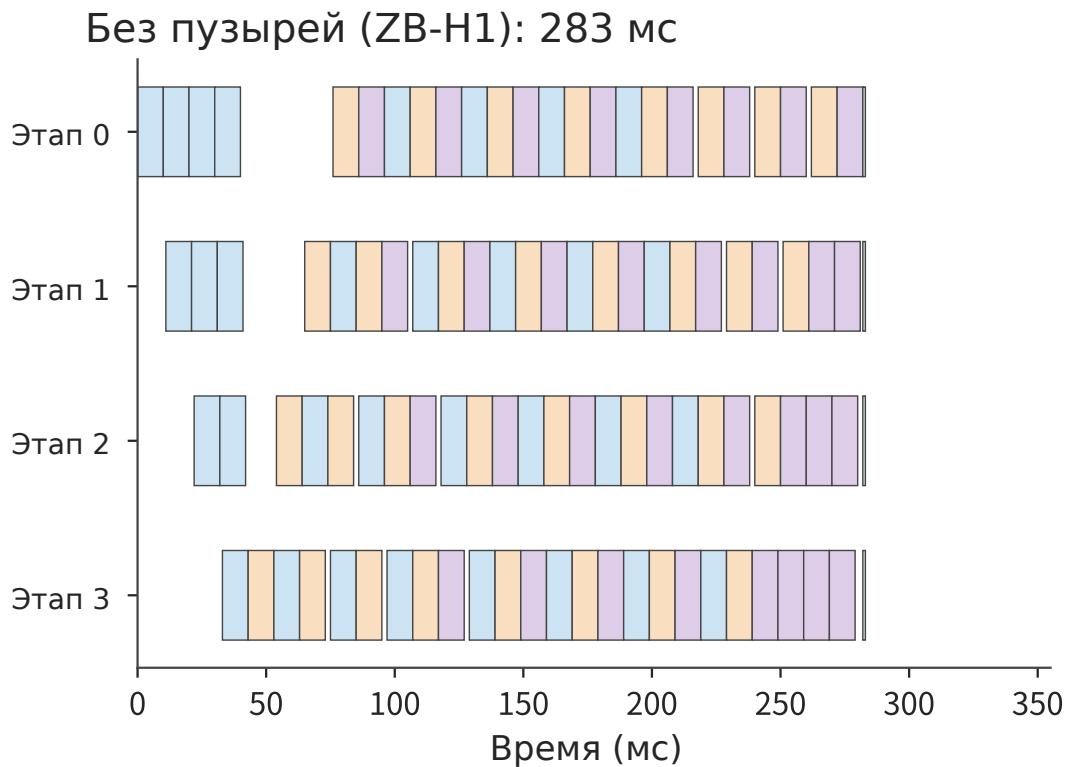


Рис. 10-22. Временная шкала этапов планирования zero-bubble (ZB-H1), время завершения — 283 ms. Синим обозначен прямой проход, оранжевым — dX , фиолетовым — dW , зелёным — обновление параметров. В установившемся режиме каждый этап циклически выполняет прямой проход, dX и dW ; промежутки между обратными проходами и при опустошении в конце, показанные на рис. 10-18, заполняются отложенными вычислениями dW , поэтому четыре этапа завершаются почти одновременно. На этапе 3 вычисление dW отстаёт от dX на три micro-batch, и там активации сохраняются дольше всего.

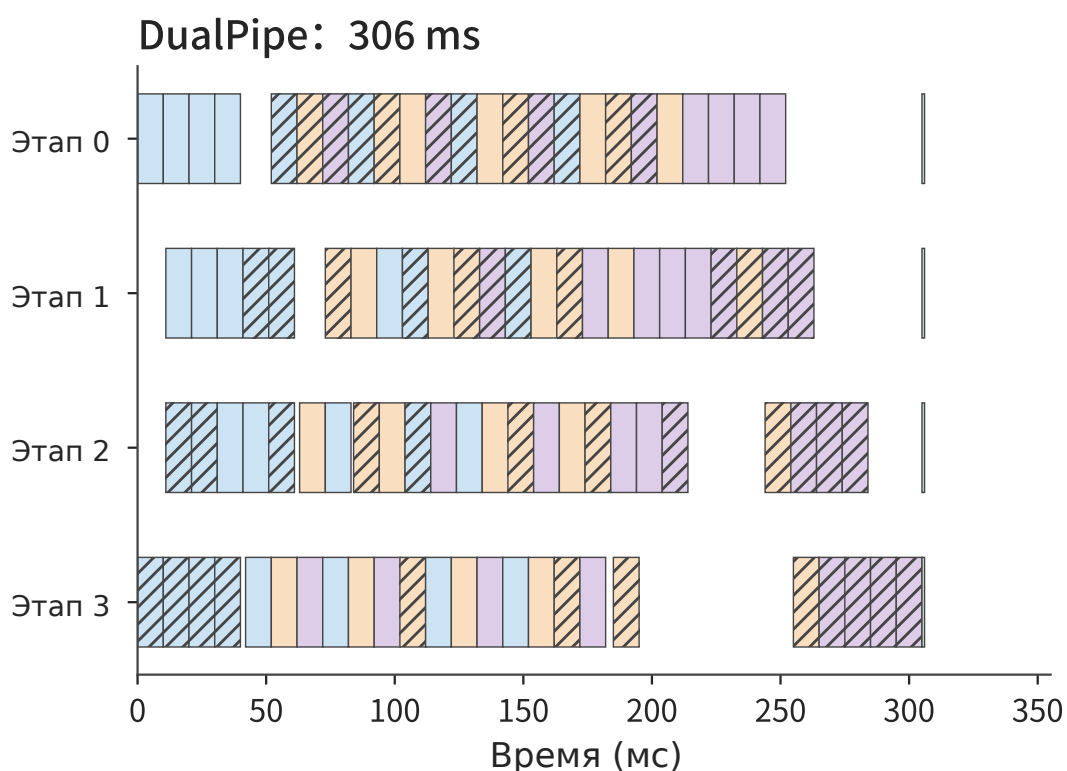


Рис. 10-23. Временная шкала этапов DualPipe, время завершения — 306 ms. Цвета соответствуют рис. 10-22, а заштрихованные блоки обозначают вычисления второй половины micro-batch, поступающей с этапа 3 в обратном направлении. Каждый этап одновременно обслуживает два направления; промежутки прогрева и опустошения одного направления заполняются блоками другого. Цена — хранение двух копий параметров на каждой карте.

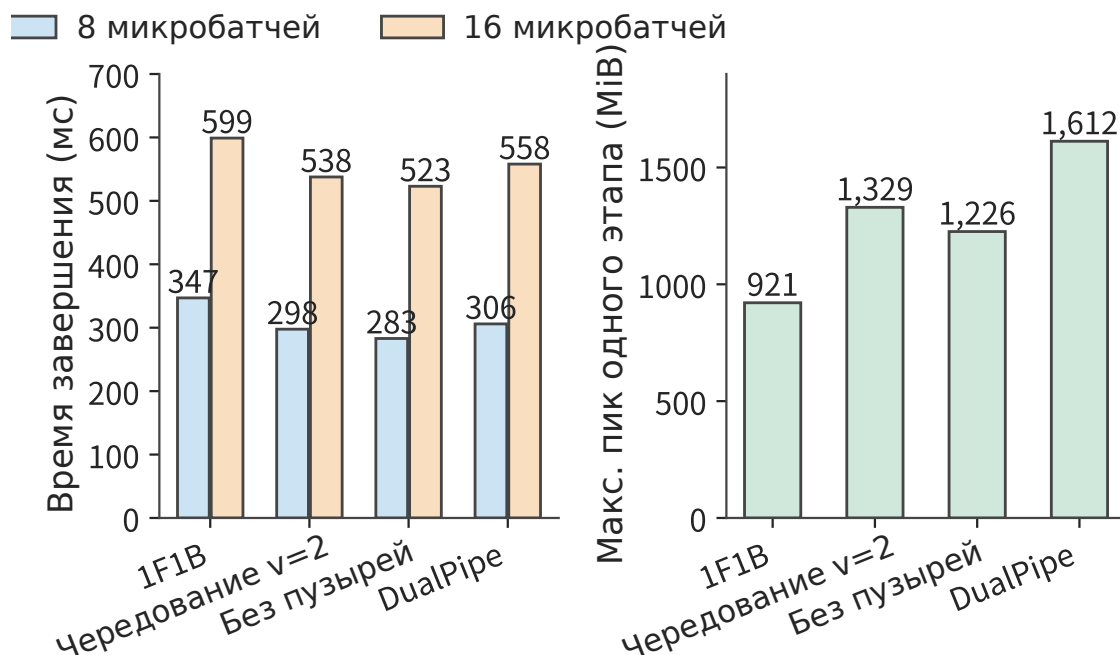


Рис. 10-24. Время завершения четырёх вариантов конвейерного планирования для восьми и шестнадцати micro-batch, а также максимальный для отдельного этапа пиковый объём активаций и буферов приёма и передачи при восьми micro-batch. Во всех четырёх вариантах используются одинаковые условия: прямой проход 10 ms, обратный проход 20 ms, передача через границу 1 ms и обновление параметров 1 ms.

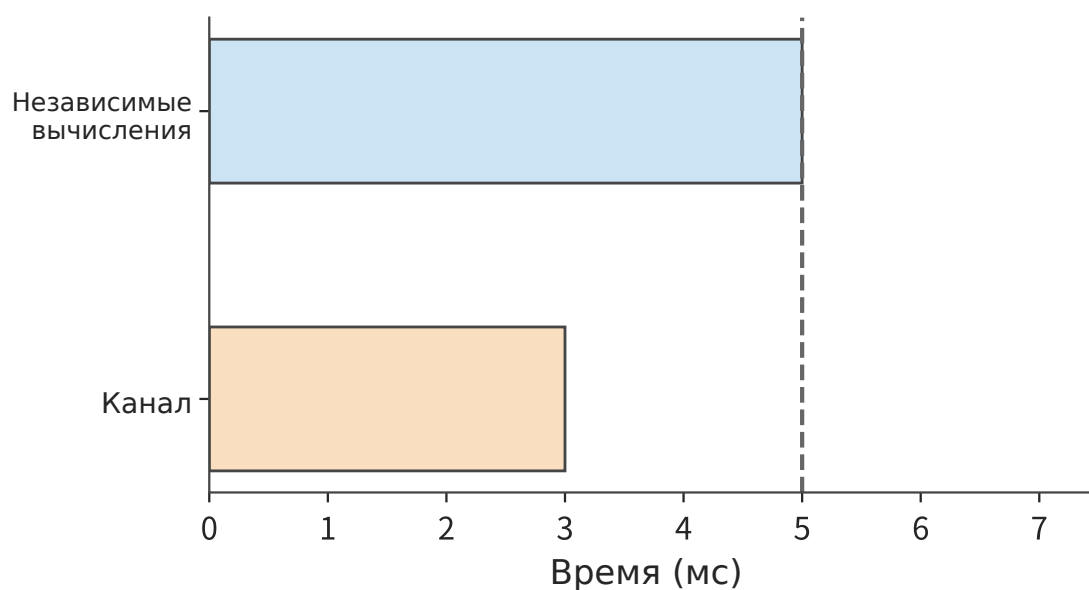


Рис. 10-25. При свободном канале редукция длительностью 3 ms (оранжевого цвета) успевает завершиться до окончания независимых вычислений длительностью 5 ms (синего цвета). Пунктирная линия обозначает завершение вычислений.

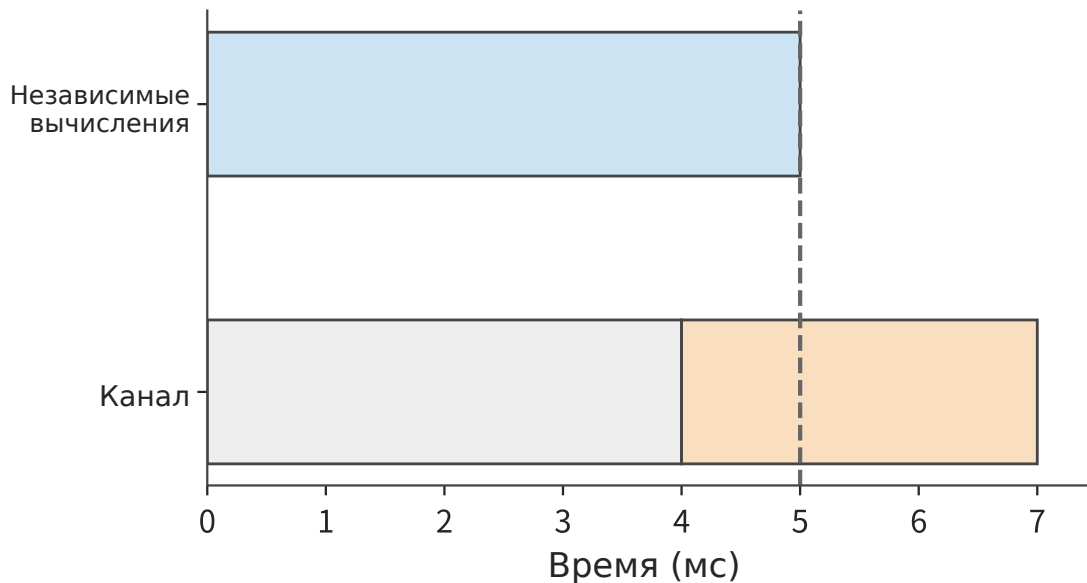


Рис. 10-26. Канал сначала на 4 мс занят другой коммуникацией (серого цвета), поэтому редукция откладывается до интервала 4–7 мс. Дополнительные 2 мс после пунктирной линии удлиняют шаг обучения.

реальном обучении порядок вычислений можно изменить, чтобы некоторые градиенты появлялись раньше и редукция начиналась раньше на временной шкале. При обратном проходе матрицы градиент входа dX передаётся предыдущему слою для продолжения обратного распространения, а градиент параметров dW — в редукцию текущего слоя. Если сначала вычислить dX , предыдущий слой сможет раньше начать обратное распространение; если сначала вычислить dW , раньше начнётся коммуникация градиента текущего слоя. Когда оба вычисления конкурируют за один ресурс, изменение порядка перераспределяет окно коммуникации. Поэтому при планировании обучения необходимо одновременно учитывать зависимости прямого и обратного проходов.

Теперь применим это правило для вычисления времени ожидания коммуникаций в проектном примере. В обоих вариантах используется ZeRO-3 для всей группы, а каждый micro-batch выполняет три коллективные коммуникации: перед прямым проходом с помощью AllGather один раз собираются веса BF16, перед обратным проходом они собираются ещё раз, а после обратного прохода над градиентами BF16 один раз выполняется ReduceScatter. В каждой коллективной коммуникации объём отправляемых и принимаемых каждой картой данных равен $(d-1)/d \times 2N$ байт, что для 48 карт составляет около 16.0 GB. На каждой карте накапливается m micro-batch, поэтому объём коммуникаций за шаг равен

$$V = 3m \cdot \frac{d-1}{d} \cdot 2N.$$

При 48 картах $m = 8$, а $V \approx 385$ GB; при 32 картах $m = 12$, а $V \approx 571$ GB. Маршрут передачи этих байтов определяется оборудованием. RTX 4090 не поддерживает NVLink и межкарточный P2P, то есть прямое чтение и запись видеопамати одной карты другой картой через PCIe; поэтому даже обмен между двумя картами на одном хосте проходит через память хоста, а весь объём отправки и приёма каждой карты проходит через её PCIe 4.0 x16 с пропускной способностью 32 GB/s в каждом направлении. Согласно

опубликованной конфигурации кластера 4090, каждый хост с восемью картами оснащён восемью сетевыми картами на 200 Gbit/s, по 25 GB/s каждая; кольцевые коллективные коммуникации распределяют межхостовый трафик между восемью сетевыми картами, поэтому они не становятся узким местом. В результате время использования канала за шаг составляет примерно $385/32 \approx 12.0$ s для 48 карт и около 17.9 s для 32 карт.

Время использования канала не равно времени ожидания коммуникации. ZeRO-3 предварительно загружает параметры слоя перед его выполнением и немедленно редуцирует градиенты после завершения обратного прохода этого слоя. При 48 картах время использования канала одним micro-batch составляет примерно 1.50 s, тогда как вычисления этого micro-batch занимают около 6.53 s; на уровне отдельных слоёв сбор примерно 0.39 GB параметров одного слоя занимает лишь около 12 ms, что также значительно меньше времени вычисления слоя. Поэтому большая часть коммуникаций перекрывается вычислениями. MegaScale использует при параллелизме по данным тот же подход и отмечает, что скрыть не удаётся только первый AllGather и последний ReduceScatter шага. Для Qwen3-8B обе коммуникации относятся к эмбедингам слов объёмом около 1.24 GB: эмбединги слов являются первым слоем прямого прохода и последним слоем, для которого вычисляется градиент при обратном проходе. Вместе эти две коммуникации занимают около 0.08 s — это и есть время ожидания коммуникаций за шаг. Обновление параметров и внутрикартовые нематричные операции уже учтены в эффективности вычислений одной карты; при готовых входных данных длительность шага для двух вариантов составляет соответственно около 78.4 s и 52.3 s.¹⁷

То, что большая часть коммуникаций перекрывается вычислениями, также задаёт верхнюю границу выгоды от оптимизации. Если из редукции длительностью 1.3 s лишь 0.2 s продолжаются после завершения обратного прохода, то при неизменной остальной временной шкале даже полное устранение этого ожидания сократит весь шаг не более чем на 0.2 s. Если для достижения цели не хватает ещё 0.5 s, необходимо одновременно сокращать другую работу на критическом пути.

Пример: какое ускорение даст четырёхкратное увеличение числа карт при неизменном глобальном batch? Система крупномасштабного распределённого обучения MegaScale сохраняла global batch size 6144 для модели 175B и увеличила число карт с 3072 до 12288. Изначально один шаг занимал около 23.7 s. Если бы четырёхкратное увеличение числа карт обеспечивало четырёхкратное ускорение, время шага сократилось бы примерно до 5.9 s, однако фактически оно составило около 6.3 s. Общее ускорение — примерно 3.7 раза, что соответствует эффективности масштабирования около 93%. После увеличения числа карт объём работы на каждой карте уменьшается, а доля ожидания из-за межгрупповой синхронизации и несбалансированной нагрузки в общем времени шага возрастает; фактический шаг примерно на 0.4

¹⁷Входные данные, выводы и перерасчёты примера проектирования из этой главы. Запасы ёмкости, ожидание входных данных и плановые остановки для 32/48 карт являются заданными входными данными; вычислительная эффективность одной карты взята из MFU Llama 3; объём матричных вычислений модели получен из существующего расчёта training-deadline, а объём коммуникации ZeRO-3 выведен из той же таблицы параметров. PCIe 4.0 и отсутствие NVLink у RTX 4090 указаны на странице спецификаций RTX 4090; отсутствие поддержки P2P между картами и конфигурация восьмикарточного хоста с восемью сетевыми картами по 200 Gbit/s приведены в анализе конфигурации кластера 4090; согласно §3.2 MegaScale, в пределах одного шага невозможно скрыть только первый AllGather и последний ReduceScatter. Значения по отдельным компонентам и независимые перерасчёты сохранены в design-case.json.

с дольше идеального — это дополнительное время, требующее анализа после масштабирования. Согласно порядку из раздела 1.3.4, сначала в модель добавляются межгрупповая синхронизация и дисбаланс нагрузки: чем больше карт, тем большую долю шага занимает ожидание самой медленной группы, и это работа, возникающая непосредственно из-за масштабирования. Только оставшуюся после этого необъяснённую разницу следует относить к накладным расходам реализации.¹⁸

10.3.4 Изменения планирования для МоЕ и длинного контекста

В плотной модели каждый прямой проход использует все веса feed-forward-блока каждого слоя, поэтому при параллелизме по данным работа распределяется сравнительно равномерно. Маршрутизация МоЕ приводит к тому, что разные карты получают разное число токенов, а различия в длине последовательностей означают, что одинаковое число токенов может соответствовать разному объёму вычислений. Поэтому при анализе времени шага необходимо учитывать, как работа распределяется между картами и какая карта завершается последней.

Как неравномерный dispatch экспертов увеличивает время вычисления самой загруженной карты. Пусть эксперты на двух картах получают соответственно 96 и 32 операции token dispatch. Каждая операция dispatch отправляет вектор признаков одного токена одному эксперту и занимает одну строку во входной матрице этого эксперта. Предположим, что время вычисления каждой операции dispatch приблизительно одинаково. Всего выполняется 128 операций dispatch, соответствующих 128 действительным строкам; в среднем на каждую карту приходится 64 строки, но время завершения определяется стороной с 96 строками. Если изменить распределение на 64 и 64 строки, время вычисления экспертов сократится с времени обработки 96 строк до времени обработки 64 строк, то есть на треть.

Механизм балансировки маршрутизации при обучении определяет, как образцы выбирают экспертов; копии экспертов и их топологическое отображение определяют, где именно выполняется этот выбор.

Приведённая выше корректировка предполагает, что токены можно свободно перемещать между экспертами. В действительности dispatch определяется оценками маршрутизатора, а механизм балансировки может влиять только на распределение dispatch; при выполнении также требуется явное правило обработки дисбаланса. Это правило задаёт **коэффициент ёмкости** (capacity factor) c : входная матрица каждого эксперта может содержать не более

$$\text{ёмкость одного эксперта} = \left\lfloor c \times \frac{\text{число токенов} \times k}{E} \right\rfloor$$

строк, то есть в c раз больше среднего числа dispatch, где k — число экспертов, выбираемых для каждого токена, а E — число экспертов. Операции dispatch сверх ёмкости **отбрасываются**: слой не обрабатывает эти токены, и их представления передаются непосредственно следующему слою через остаточное соединение; незанятые слоты ёмкости заполняются нулями, но эти строки всё равно требуют вычислений и коммуникаций.¹⁹

¹⁸В статье указана длительность шага 23,66/6,34 s и эффективность масштабирования около 93,3%; это эксперимент по масштабированию всей системы при фиксированном глобальном размере batch. См. официальную статью MegaScale и заметки о фиксированной задаче, исторической конфигурации и измерениях.

¹⁹Перерасчёт коэффициента ёмкости и отбрасывания. Правило ёмкости приведено в формуле (3) Switch

Подставим значения из предыдущего примера: 128 операций `dispatch`, $E = 2$, $k = 1$, в среднем по 64 строки на эксперта. При c , равном 1.0, 1.25, 1.5 и 2.0, ёмкость каждого эксперта составляет соответственно 64, 80, 96 и 128 строк. Эксперт, получивший 96 операций `dispatch`, отбрасывает соответственно 32, 16, 0 и 0 токенов, а суммарное число строк с нулевым заполнением у двух экспертов составляет 32, 48, 64 и 128. При $c = 1.0$ общий объём выполнения в точности равен 128 действительным операциям `dispatch`, однако четверть из них отбрасывается; начиная с $c = 1.5$ отбрасывания больше нет, но треть выполняемых строк заполнена нулями.

Теперь рассмотрим масштаб маршрутизации реальной модели. Qwen3-235B-A22B содержит $E = 128$ экспертов, а каждый токен выбирает $k = 8$ экспертов; для 8,192 токенов выполняется в общей сложности 65,536 операций `dispatch`, в среднем по 512 строк на эксперта. Предположим, что половина экспертов получает в 1.5 раза больше среднего числа операций `dispatch`; это соответствует тому же дисбалансу 3:1, что и соотношение 96/32 в предыдущем примере: каждый горячий эксперт получает 768 строк, а каждый холодный — 256. При c , равном 1.0, 1.25, 1.5 и 2.0, ёмкость каждого эксперта составляет соответственно 512, 640, 768 и 1,024 строки. Горячие эксперты отбрасывают соответственно 256, 128, 0 и 0 операций `dispatch` каждый, что в сумме составляет 25%, 12.5%, 0 и 0 от общего числа `dispatch`; строки с нулевым заполнением составляют 25%, 30%, 33% и 50% от всего объёма выполнения.

Коэффициент ёмкости c	Ёмкость одного эксперта / строк	Доля отброшенных <code>dispatch</code>	Доля строк с нулевым заполнением в объёме выполнения
1.0	512	25%	25%
1.25	640	12.5%	30%
1.5	768	0	33%
2.0	1,024	0	50%

Условие перехода определяется распределением маршрутизации: для приведённого распределения требуется $c = 1.5$, чтобы снизить долю отбрасывания до нуля; чем равномернее маршрутизация, тем ближе необходимое значение c к 1 и тем меньше потери на нулевое заполнение. Вспомогательная функция потерь, то есть дополнительный член балансировки нагрузки в цели обучения, непосредственно штрафует неравномерность `dispatch`; схема DeepSeek-V3 без вспомогательной функции потерь динамически корректирует смещения маршрутизации в соответствии с нагрузкой экспертов. Оба подхода делают распределение более равномерным, позволяя использовать меньший коэффициент ёмкости. Формула ёмкости экспертов из раздела 2.4 предполагает, что при маршрутизации не отбрасывается ни одной операции `token dispatch` ($\sum_e t_e = mk_{\text{top}}$), то есть соответствует достаточно большому коэффициенту ёмкости и нулевой доле отбрасывания.²⁰

Transformer — ёмкость каждого эксперта = количество токенов в каждом batch / количество экспертов × коэффициент ёмкости — и в GShard, где ёмкость имеет порядок $O(N/E)$, а представление переполнившихся токенов передаётся следующему слою через `residual connection`; балансировка смещением без вспомогательной функции потерь описана в техническом отчёте DeepSeek-V3. Распределение, при котором половина экспертов нагружена в 1,5 раза сильнее, принято для этого примера и, как и пример 96/32 в основном тексте, соответствует соотношению 3:1.

²⁰Перерасчёт коэффициента ёмкости и отбрасывания. Правило ёмкости приведено в формуле (3) Switch Transformer — ёмкость каждого эксперта = количество токенов в каждом batch / количество экспертов × коэффициент ёмкости — и в GShard, где ёмкость имеет порядок $O(N/E)$, а представление переполнившихся токенов передаётся следующему слою через `residual connection`; балансировка смещением без вспомогательной

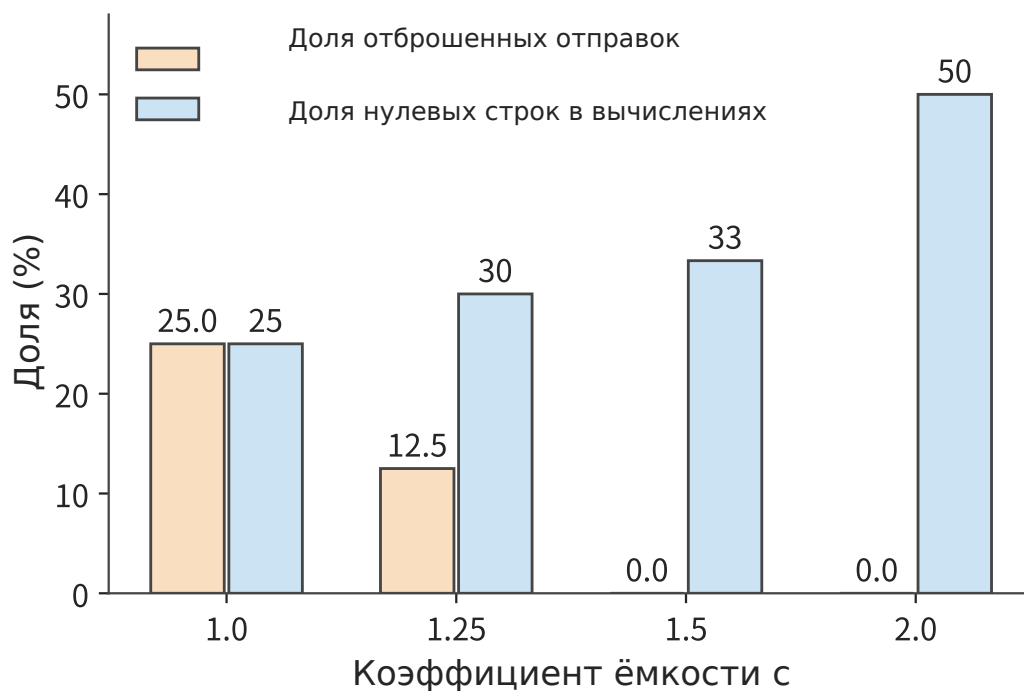


Рис. 10-27. Для формы маршрутизации Qwen3-235B-A22B ($E = 128$, $k = 8$, 8,192 токена, половина экспертов получает в 1.5 раза больше среднего) с ростом коэффициента ёмкости доля отброшенных dispatch уменьшается, а доля строк с нулевым заполнением увеличивается.

После балансировки объёма вычислений экспертов передача данных всё ещё может заставлять карты ждать. Обмен токенами при прямом проходе MoE, передача градиентов входа при обратном проходе и редукция градиентов параметров конкурируют за один канал, поэтому здесь также нужен анализ временной шкалы коммуникаций из предыдущего раздела. Возьмём градиент эксперта FP32 объёмом 72 MiB, над которым четыре участника параллелизма по данным, содержащие этого эксперта, выполняют кольцевой AllReduce. Каждая карта отправляет $2 \times 3/4 \times 72 = 108$ MiB. Каждая карта оснащена сетевой картой на 200 Gbit/s с пропускной способностью 25 GB/s в каждом направлении, а запуск каждого коллективного вызова занимает около 0.02 ms. Обработка всего буфера занимает около 4.55 ms, поэтому он не помещается в свободный промежуток длительностью 2 ms. Если разделить градиент на три подбуфера по 24 MiB, каждый из них отправит 36 MiB примерно за 1.53 ms и сможет завершиться в одном из трёх таких свободных промежутков; однако общее время обработки коммуникаций возрастёт примерно до 4.59 ms. Несмотря на два дополнительных запуска коммуникации, каждый малый буфер успевает передаться до завершения вычислений, поэтому ожидание после вычислений сокращается. При переходе на сетевую карту 400 Gbit/s обработка всего буфера занимает около 2.28 ms, что всё ещё не позволяет поместить её в промежуток 2 ms, поэтому разбиение на буферы по-прежнему необходимо.²¹

функции потерь описана в техническом отчёте DeepSeek-V3. Распределение, при котором половина экспертов нагружена в 1,5 раза сильнее, принято для этого примера и, как и пример 96/32 в основном тексте, соответствует соотношению 3:1.

²¹Технический отчёт DeepSeek-V3, пример обмена экспертами и обратной группировки по блокам. Разби-

Нагрузка эксперта зависит от того, сколько операций token dispatch получили эксперты на каждой карте. Для длинного контекста возникает дополнительное различие: даже при одинаковом общем числе токенов количество вычисляемых пар внимания может различаться.

Как неравномерная длина последовательностей увеличивает объём вычислений внимания при одинаковом числе токенов. Для причинной последовательности длины s первый токен просматривает один токен, второй — два и так далее, что в сумме даёт $s(s + 1)/2$ действительных пар внимания. Две последовательности длиной 4096 и 4096 содержат в общей сложности около 16.8 миллиона пар; если изменить длины на 7168 и 1024, общее число токенов останется равным 8192, но число пар возрастет примерно до 26.2 миллиона, то есть приблизительно на 56%. Число пар внимания растёт приблизительно квадратично с длиной последовательности, поэтому более длинная последовательность увеличивает общий объём вычислений.²²

На рис. 10-29 дополнительная площадь большого треугольника возникает из реальных связей внимания, поэтому её нельзя устранить удалением токенов заполнения. Упаковка, то есть объединение нескольких коротких последовательностей в одну последовательность целевой длины, уменьшает число токенов заполнения; группировка по длине делает площади треугольников, обрабатываемых разными картами, более близкими; параллелизм по контексту дополнительно распределяет вычисления длинной последовательности между несколькими картами и получает хранящиеся на других картах данные посредством обмена K и V. Эти три метода изменяют соответственно бесполезную работу, распределение работы и путь поступления данных. При сравнении вариантов с одинаковым общим числом токенов необходимо сохранять неизменными распределение длин, маску видимости (mask, определяющую, какие токены контекста может считывать каждая позиция токена) и веса функции потерь, чтобы отделить выгоду планирования от изменения задачи.

10.4 Ввод данных, checkpoint и восстановление после сбоев

Анализ времени шага в разделе 10.3 предполагает, что входные данные уже подготовлены, а обучение не прерывается из-за сбоев. При непрерывной работе чтение и предобработка данных должны успевать за вычислениями, а checkpoint должен сохранять состояние обучения, пригодное для восстановления. В этом разделе оба фактора включаются в расчёт времени завершения из начала главы: ввод определяет, смогут ли ускорители работать непрерывно, а восстановление — какую долю уже выполненной работы придётся повторить.

10.4.1 Чтение, упаковка и предварительная загрузка данных

Исходные данные считываются, декодируются или токенизируются, фильтруются, а затем объединяются в обучающие последовательности целевой длины, помещаются в буфер памяти хоста и наконец передаются на GPU. Для буфера хоста обычно используется

ение 72 MiB на три блока, кольцевой AllReduce с четырьмя участниками, время запуска 0,02 ms и три промежутка по 2 ms являются заданными входными данными; конфигурация с одной сетевой картой 200 Gbit/s на каждую GPU приведена в анализе конфигурации кластера 4090, а 400 Gbit/s — в спецификации ConnectX-7. Балансировка экспертов и механизм маршрутизации описаны в DeepSeek-V3, планирование по блокам — в примере по ссылке.

²²Оценка объёма обучения и распределения длин.

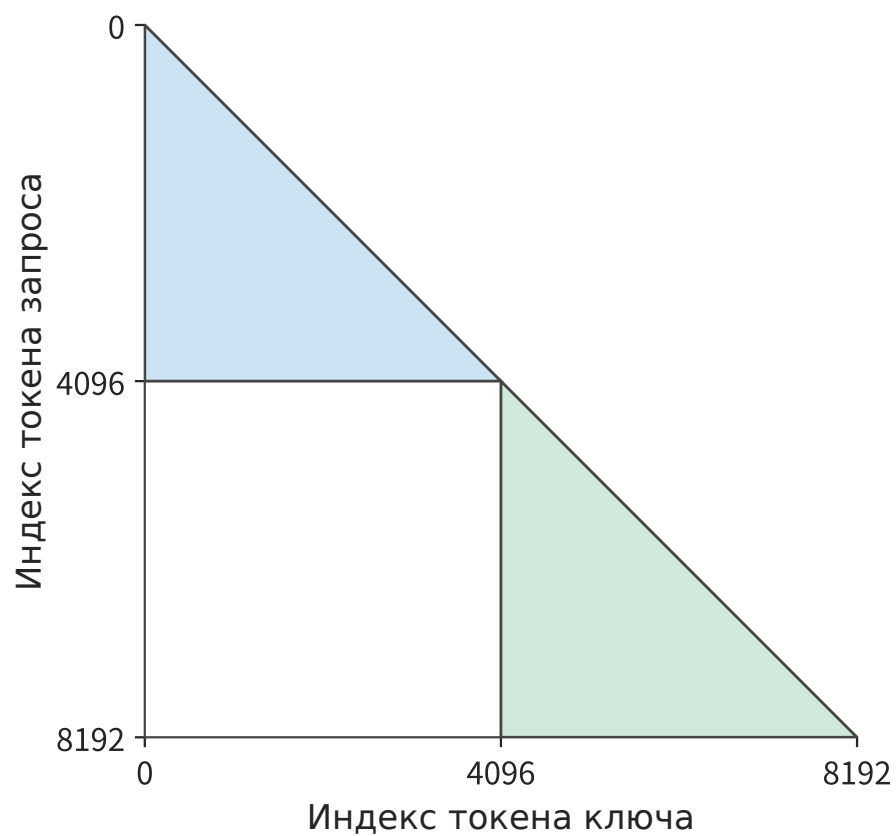


Рис. 10-28. Пары причинного внимания для двух последовательностей по 4096 токенов. Каждый токен запроса считывает позиции своей последовательности, не следующие за ним, образуя два треугольника; вычисления внимания двух последовательностей независимы, а общее число пар равно 16781312.

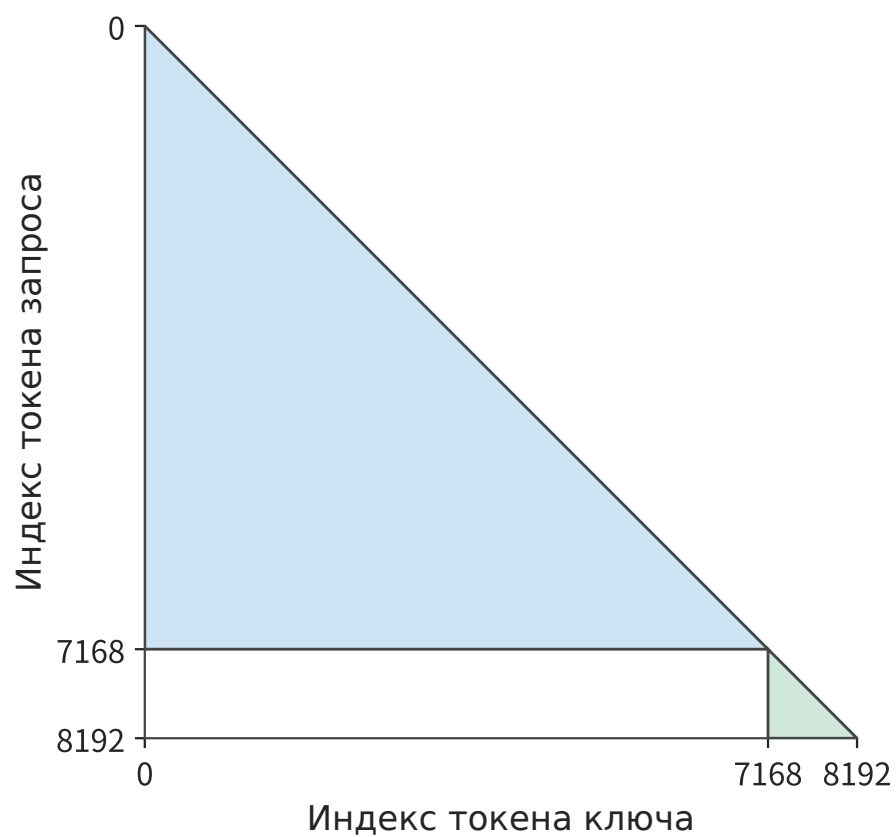


Рис. 10-29. Если те же 8192 токена разделить на последовательности длиной 7168 и 1024, общее число причинных пар возрастёт до 26218496. Увеличение площади треугольника более длинной последовательности превышает уменьшение площади треугольника более короткой.

закреплённая память, чтобы во время асинхронной передачи данные оставались в физической памяти. Устойчивая скорость обработки на каждом этапе определяет скорость ввода, а очереди сглаживают кратковременные колебания скорости.

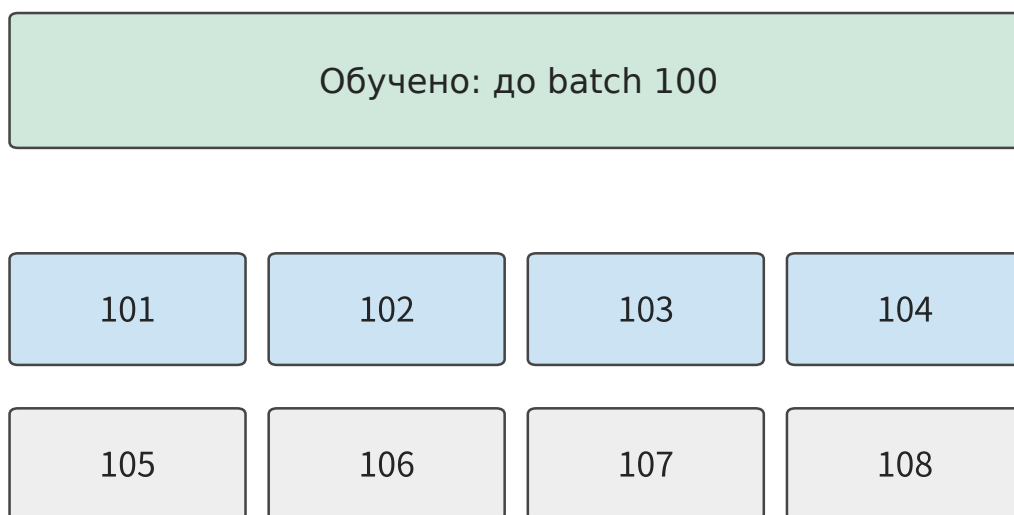
Малый объём данных, в конечном счёте поступающих на GPU, не означает, что предстоящая подготовка будет столь же быстрой. Бюджет в 25 дней требует около 46 300 токенов/с, то есть примерно 5,7 последовательности по 8 192 токена в секунду. Если передавать token ID в формате int64 по 8 байт на каждый идентификатор, скорость потока данных составит всего около 0,37 MB/s. Допустим, каждый CPU-процесс подготовки данных формирует две последовательности в секунду. Тогда два процесса подготовят лишь четыре последовательности/с, что ниже необходимого уровня, а три процесса достигнут шести последовательностей/с. Хотя объём передаваемых данных мал, их подготовка на CPU всё равно может замедлить весь кластер.

В проектном примере этой главы для ввода выделены четыре таких процесса подготовки данных с общей производительностью восемь последовательностей/с. При готовом вводе шаг на 48 ускорителях занимает около 52,3 s и обрабатывает 384 последовательности, поэтому требуется около 7,3 последовательности/с, что ниже восьми последовательностей/с. Данные подготавливаются быстрее, чем потребляются обучением, поэтому после кратковременного замедления очередь предварительной загрузки можно снова заполнить. Кроме того, в проектном примере на каждый шаг закладывается в среднем 0,5 s ожидания ввода, которое не удалось устранить предварительной загрузкой. В результате время шага для 32 и 48 ускорителей становится равным примерно 78,9 s и 52,8 s соответственно.²³

Очередь предварительной загрузки позволяет подготовке данных и вычислениям на GPU выполняться с разной скоростью. Во время штатного обучения заранее подготовленные данные уменьшают задержки из-за колебаний скорости ввода; однако при сохранении checkpoint необходимо точно различать, какие данные уже использованы для обучения. Предположим, загрузчик данных уже назначил задачи подготовки для первых 108 батчей, но обучение завершило только батч 100, а батчи 101—108 всё ещё обрабатываются или стоят в очереди. Если после восстановления сразу начать с батча 109, восемь батчей будут пропущены. Позиция батча, уже назначенного процессам подготовки данных, отражает прогресс предварительной загрузки, а позиция батча, уже использованного для обучения, — прогресс обучения; буфер между ними содержит данные, на которых обучение ещё не выполнялось.

Очередь на рисунке 10-30 способна лишь временно компенсировать снижение скорости подготовки. Если данные продолжительное время подготавливаются медленнее, очередь в конце концов опустеет. Запись checkpoint подчиняется тому же ограничению: если средняя скорость создания сохраняемых данных превысит скорость, с которой хранилище способно устойчиво их принимать, ожидающие записи снимки состояния будут накапливаться, пока ограниченный промежуточный буфер не заполнится. В разде-

²³Входные данные, выводы и перерасчёты примера проектирования из этой главы. Запасы ёмкости, ожидание входных данных и плановые остановки для 32/48 карт являются заданными входными данными; вычислительная эффективность одной карты взята из MFU Llama 3; объём матричных вычислений модели получен из существующего расчёта training-deadline, а объём коммуникации ZeRO-3 выведен из той же таблицы параметров. PCIe 4.0 и отсутствие NVLink у RTX 4090 указаны на странице спецификаций RTX 4090; отсутствие поддержки P2P между картами и конфигурация восьмикарточного хоста с восемью сетевыми картами по 200 Gbit/s приведены в анализе конфигурации кластера 4090; согласно §3.2 MegaScale, в пределах одного шага невозможно скрыть только первый AllGather и последний ReduceScatter. Значения по отдельным компонентам и независимые перерасчёты сохранены в design-case.json.



101—108: подготовка запланирована, обучение ожидается

Рисунок 10-30. Обучение завершило батч 100, а задачи предварительной загрузки назначены вплоть до батча 108. Восемь промежуточных батчей всё ещё необходимо использовать для обучения: часть из них уже подготовлена, остальные ещё обрабатываются. Пунктирные рамки обозначают батчи, обработка которых ещё продолжается. При восстановлении эти данные следует сохранить либо подготовить заново, продолжив с батча 101.

ле 10.4.3 это накопление рассчитывается для конкретных скоростей записи. В текстовой задаче этой главы объём данных на стороне ввода мал, а узкое место находится в подготовке данных на CPU. Средний поток записи checkpoint также невелик: при интервале сохранения 1800 s и размере checkpoint около 115 GB в проектном примере он составляет лишь около 64 MB/s. Пауза во время сохранения отдельно учитывается в разделе 10.4.4.²⁴

10.4.2 Восстанавливаемое состояние и преобразование раскладки

Предположим, обучение только что завершило обновление параметров, после чего работа остановилась. Чтобы продолжить обучение после восстановления, необходимо знать текущие веса, данные следующего батча и накопленную оптимизатором историю. Одних весов достаточно для прямого инференса, но недостаточно для определения следующего обновления при обучении. Поэтому обучающий checkpoint должен сохранять всё необходимое состояние, соответствующее одному и тому же прогрессу обучения, чтобы после восстановления обучение можно было продолжить.

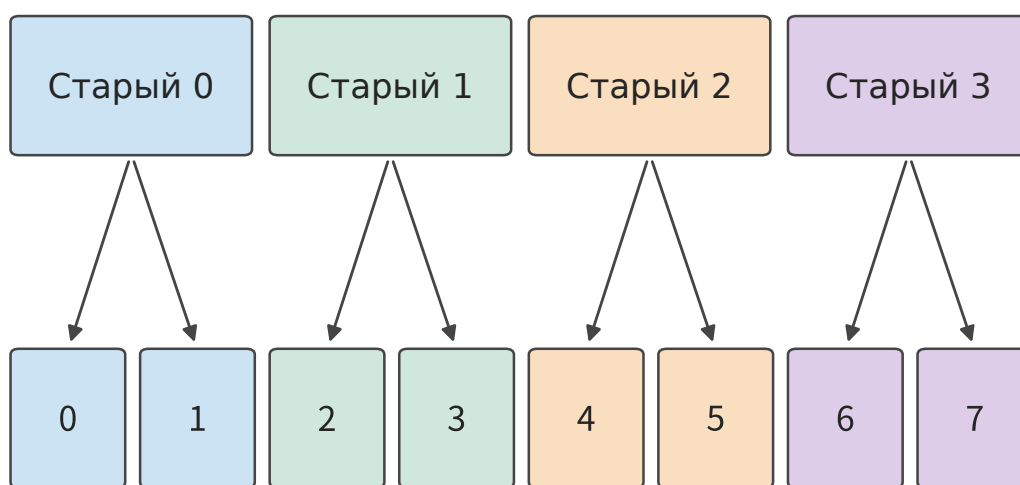
Для обучения с Adam в этой главе необходимо сохранять веса, моменты первого и второго порядка, номер шага, состояние learning rate, случайное состояние и состав следующего батча данных. Learning rate управляет величиной шага обновления, а случайное состояние определяет последовательность последующей случайной выборки. При одинаковых весах, но разных состояниях моментов оптимизатора следующее обновление параметров может различаться; при одинаковой позиции чтения данных, но разных оставшихся после упаковки токенах изменится и следующая обучающая последовательность. Пересчёт активаций из раздела 10.2.2 определяет, какие промежуточные значения можно воссоздать в пределах одного прямого и обратного прохода; checkpoint определяет, как продолжить весь процесс обучения после прерывания.

После завершения итерации обучения градиенты для следующего батча заново вычисляются обратным проходом. В этот момент сохраняются веса размером 2 bytes, основные веса размером 4 bytes и два состояния Adam общим размером 8 bytes — всего 14 bytes на параметр. Эта часть checkpoint для Qwen3-8B занимает около 115 GB. Если в конце итерации обучения одновременно сохранить состояние модели и позицию в обработанных данных, после восстановления можно однозначно определить, какие параметры использовать и с какого батча продолжить обучение.

При восстановлении может также потребоваться изменить схему параллелизма, повторно разделив одно и то же состояние на шарды. **Пример: как перерасделить checkpoint с четырёхстороннего тензорного параллелизма на восьмисторонний?** Веса проекции гейтинга имеют форму [12288, 4096] и занимают 96 MiB в BF16. При разделении выходного измерения на четыре части каждый шард содержит 3072 строки и занимает 24 MiB; после деления на восемь частей каждый шард содержит 1536 строк и занимает 12 MiB. Целевой ускоритель с рангом r считывает старый шард $\lfloor r/2 \rfloor$: чётный ранг берёт первую половину, нечётный — вторую. Глобальные координаты строк связывают старую раскладку с новой.

Следуя стрелкам на рисунке 10-31, программа восстановления может найти

²⁴Модель ввода обучающих данных и общего хранилища, эксперимент совместной работы ввода и сохранения. Событийная модель на параметрах малого масштаба демонстрирует, как очередь предвыборки опустошается, когда checkpoint и входные данные используют общее хранилище; в примере проектирования из этой главы используются четыре процесса подготовки данных, каждый из которых подготавливает по две последовательности/s.



Новые шарды: по 1536 строк; старые: по 3072

Рисунок 10-31. Положение по горизонтали соответствует номеру строки исходной матрицы, а цвет обозначает старый шард. После разделения каждого старого шарда на первую и вторую половины они помещаются в два новых шарда. Содержимое и порядок весов модели остаются прежними; изменяется только диапазон строк, за который отвечает каждый ускоритель. Общий объем весов BF16 всегда равен 96 MiB. Верхние номера с пометкой «старые» обозначают исходные шарды, нижние — шарды после перераспределения.

исходный диапазон строк, соответствующий каждому новому шарду. Основные веса FP32 и два состояния Adam разделяются по тем же диапазонам строк, причём каждый из этих компонентов вдвое больше весов BF16. Поэтому полное состояние этой матрицы в checkpoint занимает $96 \times (1 + 2 + 2 + 2) = 672$ MiB. Системы класса ByteCheckpoint описывают такие тензоры с помощью глобальной формы, смещения и длины фрагмента, позволяя программе восстановления считывать диапазоны, необходимые для целевой раскладки. Файловые шарды определяют размещение в хранилище, а глобальные координаты указывают, к какому тензору относится каждый шард и в каких строках и столбцах он расположен.²⁵

Веса можно заново собрать по номерам строк исходной матрицы, а для обучающих последовательностей также необходимо сохранить достаточно информации, чтобы воспроизвести тот же ввод. Данные с рисунка 10-30, которые уже подготовлены, но ещё не использованы для обучения, также необходимо заново локализовать при восстановлении. Помимо позиции батча, следует сохранить промежуточные результаты предобработки данных.

Если в упаковщике осталось 2 000 токенов, а следующая порция данных содержит 6 192 токена, вместе они образуют следующую последовательность из 8 192 токенов. Если эти 2 000 токенов потеряны, недостающие токены придётся взять из последующих данных, что изменит префикс следующей последовательности, связи внимания и метки. Чтобы после восстановления продолжить формирование того же батча данных, вместе с результатом обновления параметров необходимо сохранить позицию уже использованных для обучения данных и токены, ещё не объединённые в полную последовательность.²⁶

10.4.3 Синхронное и асинхронное сохранение, конкуренция за пропускную способность

В этом разделе рассматривается, как процесс сохранения может выполняться одновременно с обучением и в какой момент снимок состояния действительно становится доступным.

При синхронном сохранении сначала создаётся снимок с зафиксированным содержимым, и обучение продолжается только после завершения записи. При асинхронном сохранении снимок копируется в отдельный буфер памяти, а затем записывается в фоне, что позволяет раньше возобновить обучение. В момент возобновления обучения запись снимка может быть ещё не завершена; использовать его для восстановления после сбоя можно только после окончания записи и фиксации.

Пример: как скорость загрузки checkpoint определяет объём повторной работы после сбоя? Возьмём снимок размером 112 GB. В эталонной архитектуре DGX SuperPOD уровни производительности хранилища определяются типом нагрузки. Обучению NLP соответствует уровень Good, обеспечивающий для одного SU — масштабируемого блока из 32 систем DGX — суммарную скорость записи в хранилище 7 GB/s. При такой скорости запись одного снимка занимает 16 s. Если создавать снимок каждые 10 s, ожидающие

²⁵Разбор структуры checkpoint, ByteCheckpoint и фонового сохранения, расчёт логического перешардирования, временная шкала для 112 GB.

²⁶Журнал восстановления DCP, восстановление реального текстового конвейера. 2 000 + 6 192 — учебный пример количества токенов, оставшихся при упаковке. Малая модель DCP восстановила две части со строчным шардированием в три части со столбцовым шардированием и полное состояние; параметры, Adam, случайное состояние и результат следующего обновления параметров совпали с непрерывным обучением.

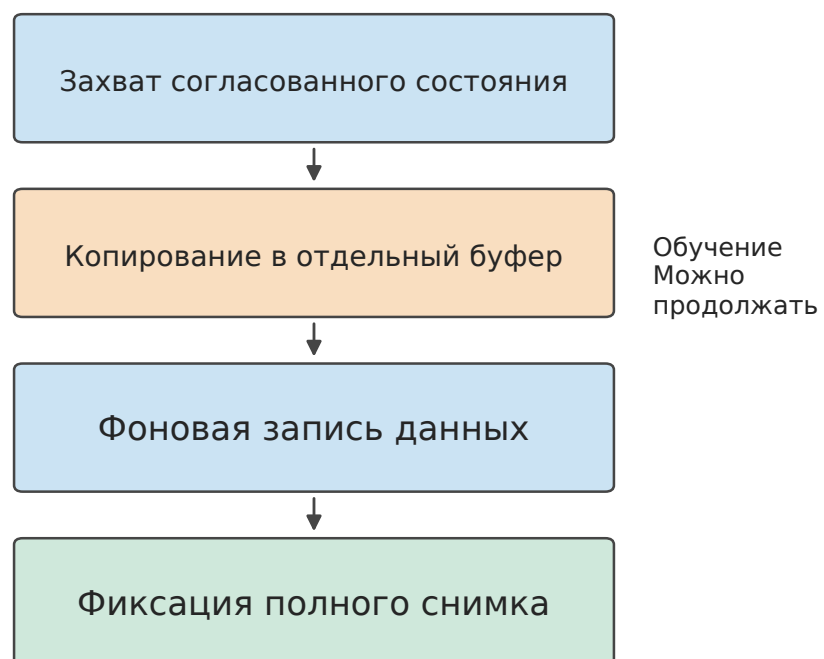


Рисунок 10-32. После захвата согласованного состояния обучения и его копирования в отдельный буфер обучение может продолжиться; программа восстановления использует checkpoint только после завершения фоновой записи данных и фиксации полного снимка. Стрелки обозначают зависимости по порядку выполнения.

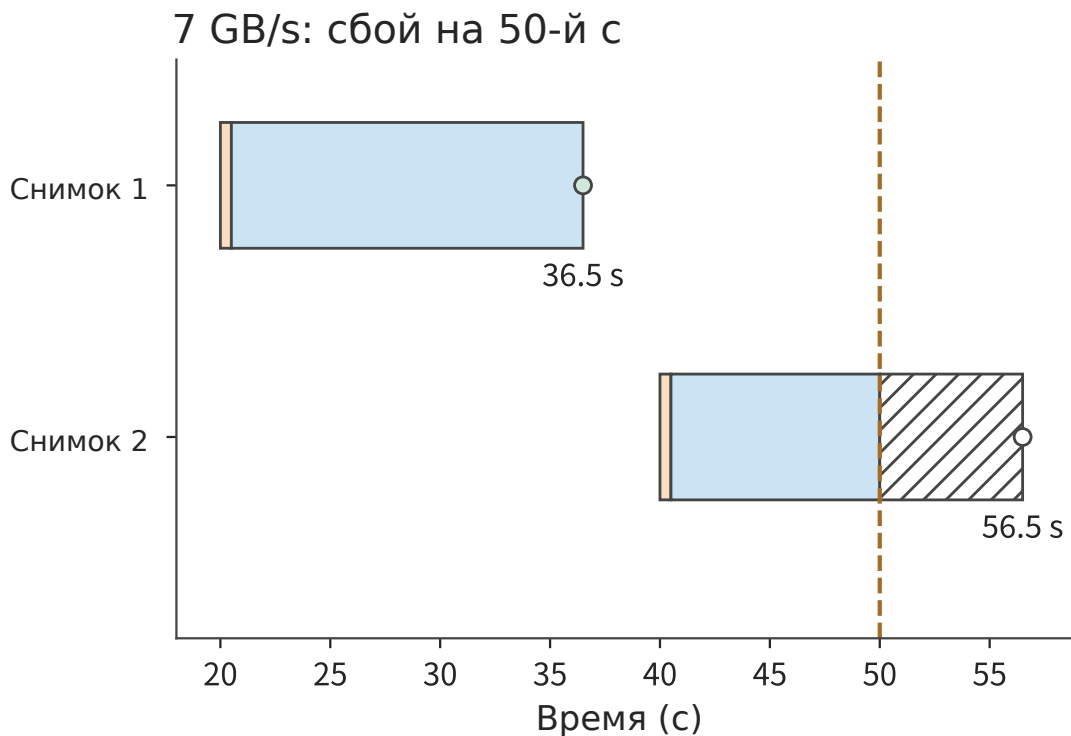


Рисунок 10-33. Два снимка по 112 GB записываются со скоростью 7 GB/s: каждый сначала копируется в буфер за 0,5 s (оранжевый), а затем загружается (синий). При сбое на 50-й секунде первый снимок уже зафиксирован, а второй — ещё нет; штриховкой обозначена оставшаяся загрузка при отсутствии сбоя, пустой кружок показывает запланированный момент фиксации.

записи данные будут поступать со скоростью 11,2 GB/s, и очередь станет непрерывно расти. Если создавать снимок каждые 20 s, средняя требуемая скорость снизится до 5,6 GB/s, и хранилище успеет записать предыдущий снимок до поступления следующего.

Два снимка захватываются на 20-й и 40-й секундах, каждый копируется в буфер за 0,5 s, после чего загружается; запись завершается на отметках 36,5 и 56,5 s. Если сбой произойдёт на 50-й секунде, второй снимок всё ещё будет загружаться, поэтому восстановиться можно лишь до состояния на 20-й секунде, повторив 30 s работы. При использовании уровня Better с суммарной скоростью записи 20 GB/s каждый снимок записывается всего за 5,6 s, а их фиксация завершается на отметках 26,1 и 46,1 s. При том же сбое можно восстановиться до состояния на 40-й секунде и повторить лишь 10 s работы.

В обоих вариантах обучение дважды приостанавливается на 0,5 s, то есть суммарно на 1 s. Более быстрая запись позволяет после сбоя восстановиться до более позднего состояния обучения и сократить повторную работу на 20 s. Таким образом, момент завершения асинхронного сохранения непосредственно влияет на долгосрочный прогресс: скорость фоновой записи определяет объём повторной работы после сбоя, а длительность приостановки обучения во время сохранения увеличивает время штатного выполнения.

Помимо записи файлов данных, реальная система должна зафиксировать метаданные, описывающие полный снимок. В серии экспериментов на CPU вызов интерфейса сохранения (API) возвращал управление примерно через 7 ms, а метаданные восстановления фиксировались лишь примерно через 48 ms. Если завершить процесс до фиксации, программа восстановления выберет предыдущий полный checkpoint. Файлы данных и запись о фиксации вместе образуют пригодный снимок, по которому программа

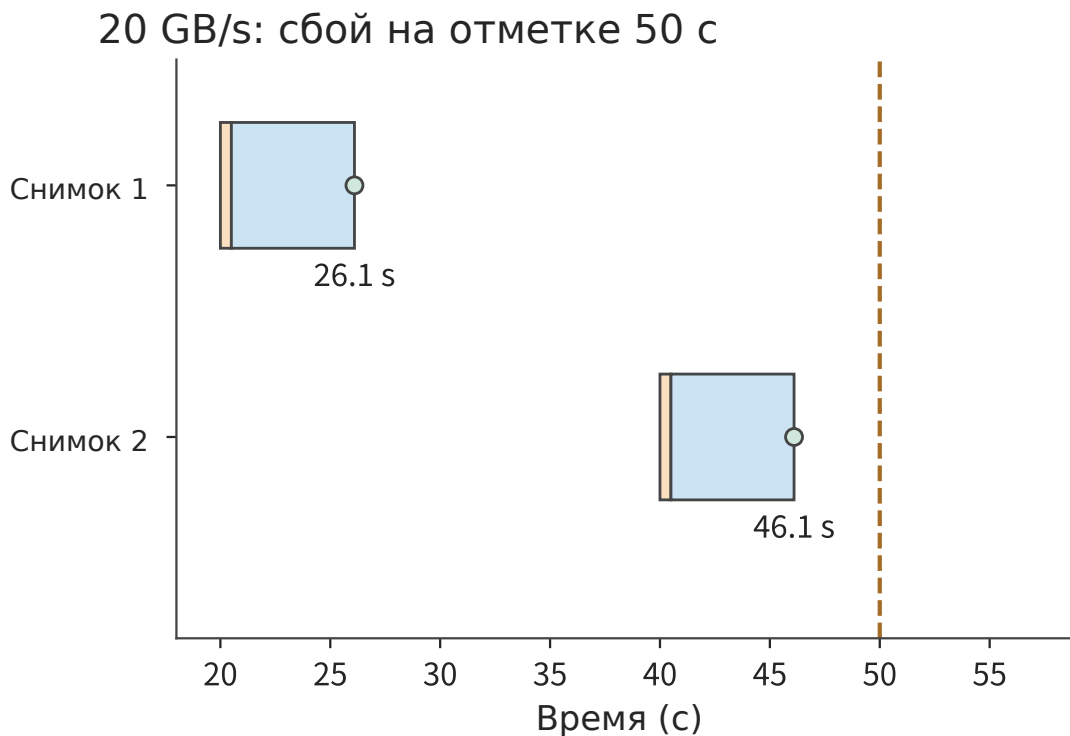


Рисунок 10-34. Те же снимки записываются со скоростью 20 GB/s и фиксируются на отметках 26,1 и 46,1 с. При сбое на 50-й секунде можно восстановить состояние обучения на 40-й секунде и повторить лишь 10 с работы.

восстановления определяет, какой checkpoint сохранён полностью.²⁷

10.4.4 Масштаб сбоев, период сохранения и эффективный прогресс обучения

На рисунке 10-33 заданы моменты двух сохранений и сравнивается влияние скорости записи. Можно также изменять интервал сохранения: при коротком интервале состояние в момент сбоя ближе к последнему снимку, а при длинном — во время штатного обучения требуется меньше пауз для сохранения. Эти две составляющие затрат изменяются в противоположных направлениях, поэтому между ними существует компромиссная точка.

Пусть каждое синхронное сохранение занимает c , между сохранениями выполняется τ секунд полезного обучения, частота сбоев задания равна λ , а восстановление занимает r . Предположим, что сбои происходят сравнительно редко и независимо друг от друга. Тогда в модели первого порядка дополнительные затраты на единицу полезного времени обучения равны

$$L(\tau) = \frac{c}{\tau} + \lambda \frac{\tau}{2} + \lambda r.$$

Первый член обусловлен сохранением через каждые τ секунд. Сбой может произойти в любой момент между двумя сохранениями, поэтому в среднем теряется половина

²⁷Эксперимент с асинхронным сохранением и сбоем до фиксации, а также сравнение без сохранения, с синхронным и асинхронным сохранением. В исходном нормальном пути вызов API занимал около 7,25 ms, а фиксация метаданных — около 48,30 ms; в эксперименте со сбоем перед фиксацией устанавливался барьер, после чего процесс завершался.

интервала, что даёт второй член. Третий член — ожидаемое число восстановлений на единицу полезного времени обучения, умноженное на длительность каждого восстановления. Первый из двух начальных членов уменьшается, а второй растёт, поэтому оптимальная точка удовлетворяет условию

$$-\frac{c}{\tau^2} + \frac{\lambda}{2} = 0, \quad \tau^* = \sqrt{\frac{2c}{\lambda}}.$$

Пример: как число ускорителей и частота сбоев определяют интервал сохранения checkpoint? Meta собрала статистику обучающих заданий в исследовательском кластере: задание на 1024 ускорителях прерывалось в среднем раз в 7,9 часа, причём частота прерываний была пропорциональна числу ускорителей. Это соответствует одному сбою на ускоритель примерно раз в 337 дней. Сбой любого ускорителя прерывает всё задание. Сохранение checkpoint размером около 115 GB со скоростью 7 GB/s занимает $c \approx 16.4$ s. Если восстановление занимает 120 s, оптимальный интервал составляет около 965 s, то есть примерно 16 минут.²⁸

Минимум на рисунке 10-35 достаточно пологий, поэтому при настройке можно выбрать близкий удобный интервал. В следующей таблице сравниваются интервалы в пять, пятнадцать и тридцать минут, чтобы показать, какая составляющая затрат растёт при отклонении от минимума.

Интервал полезного обучения	Затраты на сохранение	Затраты на повторную работу	Затраты на восстановление	Итого
300 s	5,5%	0,5%	0,4%	6,4%
900 s	1,8%	1,6%	0,4%	3,8%
1800 s	0,9%	3,2%	0,4%	4,5%

При увеличении интервала сохранения с пяти до пятнадцати минут экономия времени на сохранении превышает дополнительное время повторной работы. При дальнейшем увеличении до тридцати минут дополнительное время повторной работы уже превышает экономию времени на сохранении. Итоговые значения в таблице рассчитаны по неокруглённым числам. Удвоение числа ускорителей удваивает частоту сбоев, поэтому оптимальный интервал уменьшается в $1/\sqrt{2}$ раза. Удвоение скорости сохранения сокращает оптимальный интервал в той же пропорции, поскольку каждое сохранение занимает меньше времени.

В проектном примере этой главы используется меньше ускорителей. При среднем времени между сбоями одного ускорителя (MTBF) около 337 дней, длительности восстановления 120 s и длительности сохранения около 16,4 s выбран интервал полезного обучения 1800 s. Для 32 ускорителей сохранения, повторная работа и восстановление суммарно увеличивают время примерно на 1,02%, для 48 ускорителей — примерно на 1,08%. Вероятность прерывания из-за сбоя ускорителя в варианте с 48 ускорителями выше, однако дополнительные затраты больше лишь примерно на 0,06 процентного пунк-

²⁸Период сохранения и пуассоновская модель повторных попыток. Точный объём данных равен 114 670 295 040 bytes, а стоимость сохранения при скорости записи 7 GB/s составляет около 16,381 s; оптимум первого приближения равен примерно 965,29 s, а для пуассоновской модели повторных попыток со сбоями во время сохранения — примерно 954,40 s. Значение 7 GB/s соответствует суммарной скорости записи одного SU для уровня Good из таблицы 6 эталонной архитектуры DGX SuperPOD; среднее время до прерывания задания на 1024 картах составляет 7,9 часа, а обратная пропорциональность среднего времени без сбоев количеству карт приведена в статье о надёжности кластера Meta. Для выбора в этой главе достаточно различать интервалы в пять, пятнадцать и тридцать минут.

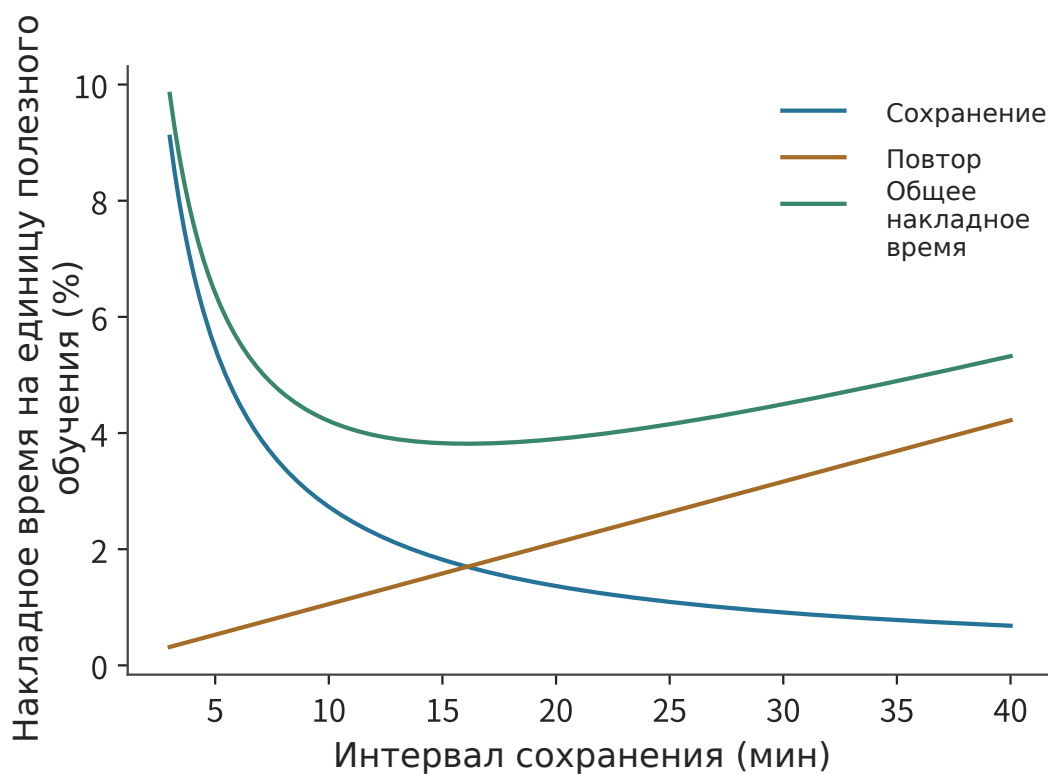


Рисунок 10-35. Синяя линия показывает долю времени сохранения, оранжевая — долю времени повторной работы после сбоя, зелёная — их сумму с учётом времени восстановления. Используется модель первого порядка, в которой задание на 1024 ускорителях прерывается в среднем раз в 7,9 часа, сохранение занимает около 16,4 s, а восстановление — 120 s. Минимум находится в точке, где уравниваются две зависящие от интервала составляющие затрат.

та, что значительно меньше выигрыша от сокращения времени вычисления на одном ускорителе. Теперь известны требования к ёмкости, время шага обучения и долгосрочные дополнительные затраты; в разделе 10.6 они будут объединены в расчёте времени завершения.

При крупномасштабном обучении даже прерывание на два-три часа может стоить десятки тысяч долларов из-за затрат на всю группу ресурсов. Период сохранения определяет, какой объём работы придётся повторить после одного сбоя, а надёжность системы — сколько раз возникнут подобные потери.

Пример: сколько денег теряется из-за одного прерывания обучения? Для MiMo-V2.6 компания Xiaomi первой среди передовых лабораторий провела прямую трансляцию процесса RL. Публичная панель выполнения показывает, что RL-постобучение моделей размеров Pro и Flash обошлось примерно в 2,6 млн и 0,9 млн долларов соответственно, то есть приблизительно в 20,6 тыс. и 10,3 тыс. долларов в час.

На рисунке 10-36 по журналам построены графики двух запусков. Pro перезапускался в общей сложности 14 раз, а суммарная длительность прерываний составила около 30,1 h, или 23,6% общего времени выполнения, что эквивалентно примерно 619 тыс. долларов. Перед пятым перезапуском Pro с момента завершения предыдущего шага прошло 2,76 h, что соответствует примерно 56,7 тыс. долларов; перед шестым и восьмым перезапусками прошло соответственно 2,63 h и 2,95 h, что соответствует примерно 54,1 тыс. и 60,7 тыс. долларов. Причины перезапусков в этих запусках относились преимущественно к трём категориям: аппаратные сбои, включая двухбитовые ошибки видеопамати; сбои сервисов, включая аварийное завершение контейнеров и недоступность сети сервиса оценки; а также нехватка памяти — например, исчерпание KV-кэша во время инференса, переполнение видеопамати из-за неравномерной нагрузки на экспертов во время обучения или нехватка памяти хоста при упаковке последовательностей. Кроме того, одна ошибка инфраструктуры не была обнаружена своевременно: после завершения шага 17 обучение пришлось повторить с шага 15. В результате около 4 h, потраченных до этого на шаги 16 и 17, были утрачены.²⁹

Если вдвое сократить суммарное время прерываний Pro, тот же объём обучающей работы потребует примерно на 15 h меньше, что при указанных тарифах сэкономит около 310 тыс. долларов. Этого можно добиться двумя способами. Во-первых, устранить повторяющиеся сбои: например, команда MiMo изменила схему параллелизма обучения и уменьшила объём памяти активаций, чтобы вместить пиковую нагрузку на экспертов. Во-вторых, раньше обнаруживать сбои, быстрее восстанавливаться и сокращать повторную работу за счёт более свежего checkpoint, уменьшая время восстановления t или потерянный интервал обучения.

10.4.5 Отстающие и медленные узлы

Модель из предыдущего раздела рассматривает сбой как событие, прерывающее задание. Существует и другой, более частый вид деградации, который не прерывает задание: на некотором шаге один ускоритель выполняет вычисления медленнее. Каждый

²⁹Общая стоимость и причины сбоев см. в техническом отчёте MiMo-V2.6, §4.1 и §5.5. Длительность отдельных событий, тарифы и объявления об откатах приведены в архивном журнале, процесс вычислений — в заметках по примеру и записи перерасчёта. Сумма оценена умножением интервалов событий из журнала на фиксированный тариф с панели, а не взята из отдельного счёта за сбои в отчёте; повторная работа, обнаружение и ожидание внутри интервалов не разделялись, а потенциально переиспользуемые результаты выборки не вычитались.

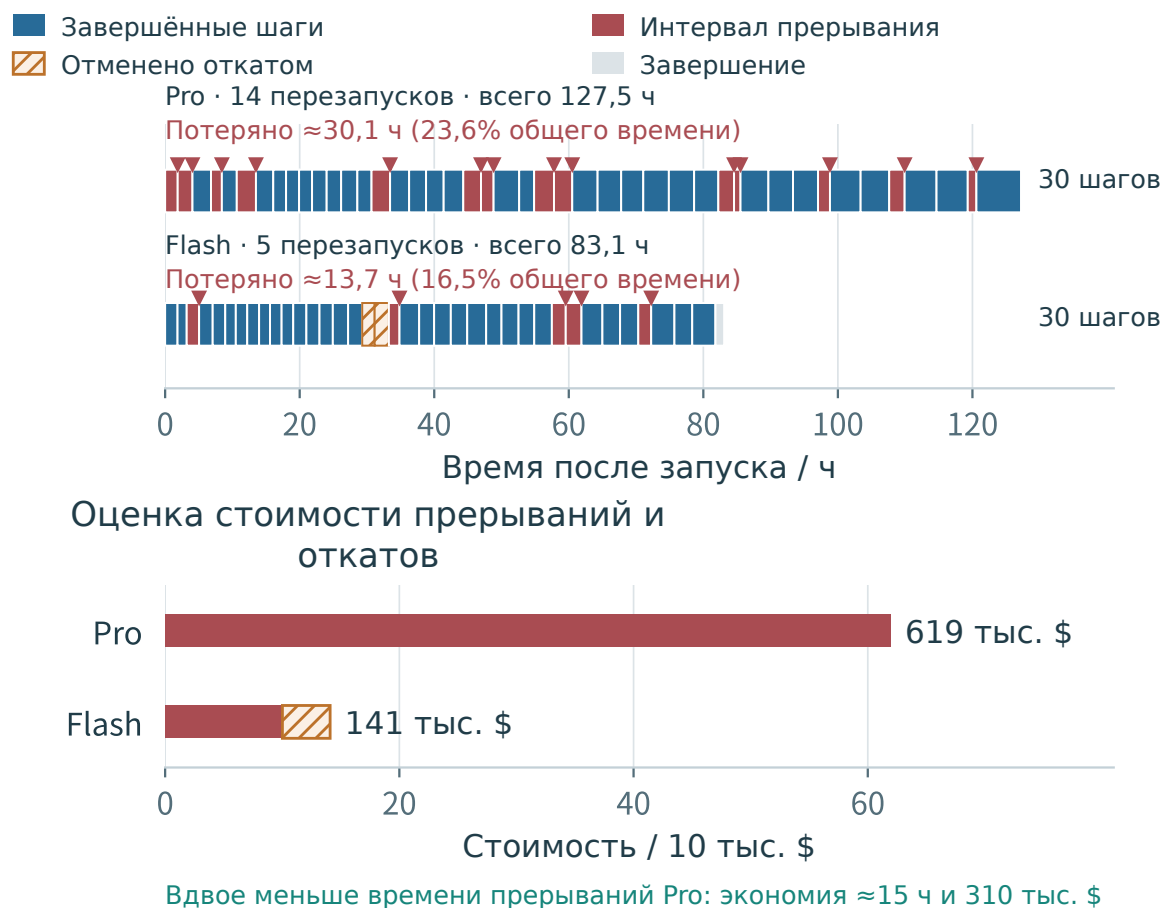


Рисунок 10-36. Время каждого запуска отсчитывается с момента его начала. Синим показаны интервалы завершённых шагов, красным — интервалы от предыдущего события до перезапуска, а треугольниками отмечены перезапуски; штриховкой обозначены два шага Flash, аннулированные после отката. Потраченное впустую время оценивается как сумма красных и заштрихованных интервалов, а в скобках указана его доля в общем времени выполнения; ниже приведены затраты, рассчитанные по публичным тарифам. Расчёт экономии предполагает неизменный объём обучающей работы и сокращение времени выполнения на величину устранённых прерываний.

шаг синхронного параллелизма по данным должен ждать завершения самого медленно-го ускорителя. Ускоритель, замедляющий весь шаг, называется **отстающим** (straggler), поэтому время шага определяется максимумом, а не средним значением по ускорителям. Пусть время вычислений на каждом ускорителе независимо и одинаково распределено со средним μ и стандартным отклонением σ . Тогда при N ускорителях вычислительная часть времени шага равна максимуму из N выборок. Возьмём $\mu = 52.2$ s — время вычисления на одном ускорителе в варианте с 48 ускорителями — и 0,58 s на обмен данными и ожидание ввода на каждом шаге, а для σ рассмотрим значения 2% от среднего (1,04 s) и 5% (2,61 s).³⁰

Математическое ожидание максимума из N независимых нормально распределённых выборок равно μ плюс некоторое число σ : для $N = 8, 48, 1024$ это число составляет примерно 1,42, 2,23 и 3,25 соответственно. Подстановка даёт следующее ожидаемое время шага:

Число ускорителей	$E[\max] - \mu$ (в единицах σ)	Шаг при $\sigma = 2\%$, s	Шаг при $\sigma = 5\%$, s
8	1,42	54,3	56,5
48	2,23	55,1	58,6
1024	3,25	56,2	61,3

Без колебаний каждый шаг занимает примерно $52.2 + 0.58 \approx 52.8$ s. Чем больше ускорителей, тем выше ожидаемый максимум: при 48 ускорителях и $\sigma = 5\%$ шаг становится длиннее примерно на 5,8 s, что поглощает 40% от запаса около 14,4 s из раздела 10.6; при 1024 ускорителях прибавка составляет около 8,5 s. Медленные ускорители можно выявлять по ожиданию обмена данными: время ожидания каждого ускорителя при AllReduce градиентного бакета равно разности между моментом завершения шага и моментом завершения его собственных вычислений. Для ускорителя, время вычисления которого точно равно среднему, ожидаемая длительность такого ожидания составляет именно $E[\max] - \mu$.³¹

После обнаружения медленного ускорителя возможны три реакции: ожидание, перераспределение и исключение. **Ожидание:** текущий шаг завершается по времени медленного ускорителя. Насколько ускоритель со временем $\mu + 3\sigma$ заставит остальные ускорители ждать дольше, зависит от того, каким был бы максимум среди остальных $N - 1$ ускорителей. При $\sigma = 2\%$ для 8 ускорителей дополнительное ожидание составляет около 1.65σ (1,72 s), для 48 — около 0.78σ (0,81 s), а при 1024 ускорителях максимум осталь-

³⁰Модель максимума отстающих исполнителей, где σ составляет 2% среднего, и вариант, где σ составляет 5% среднего. Заметное замедление примерно 0,5% машин отмечено в MegaScale; среднее время без сбоев составляет 7,9 часа для задания на 1024 картах и 47,7 дня для задания на 8 картах согласно статье о надёжности кластера Meta, а частота сбоев одной карты в разделе о периоде сохранения пересчитана из 7,9 часа для 1024 карт. Стандартное отклонение вычислительного времени каждой карты и интервал отката из-за всплеска функции потерь приняты для этого примера; математическое ожидание максимума получено численным интегрированием порядковой статистики без моделирования корреляции, периодического дрожания или устойчиво медленных карт.

³¹Модель максимума отстающих исполнителей, где σ составляет 2% среднего, и вариант, где σ составляет 5% среднего. Заметное замедление примерно 0,5% машин отмечено в MegaScale; среднее время без сбоев составляет 7,9 часа для задания на 1024 картах и 47,7 дня для задания на 8 картах согласно статье о надёжности кластера Meta, а частота сбоев одной карты в разделе о периоде сохранения пересчитана из 7,9 часа для 1024 карт. Стандартное отклонение вычислительного времени каждой карты и интервал отката из-за всплеска функции потерь приняты для этого примера; математическое ожидание максимума получено численным интегрированием порядковой статистики без моделирования корреляции, периодического дрожания или устойчиво медленных карт.

ных ускорителей уже сам превышает $\mu + 3\sigma$, поэтому дополнительное ожидание равно нулю. **Перераспределение:** работа медленного ускорителя равномерно распределяется между оставшимися $N - 1$ ускорителями, и среднее становится равным $\mu N / (N - 1)$. Если по-прежнему взять $\sigma = 2\%$ и не учитывать стоимость переноса, шаг на 48 ускорителях займёт около 56,2 s, а на 8 ускорителях — около 61,6 s. Оба значения превышают ожидаемое время шага при ожидании с тем же σ : 55,1 s и 54,3 s соответственно. Перераспределение добавляет каждому ускорителю $1/(N - 1)$ работы, и эта прибавка превышает ожидание, вызванное одним медленным ускорителем; чем меньше ускорителей, тем больше разница. **Исключение:** медленный ускоритель удаляется из задания, после чего оно перезапускается с предыдущего checkpoint. Цена равна ожидаемому объёму потерянной работы $\tau/2$ плюс время восстановления r . Здесь возьмём более короткий, чем 1800 s в проектном примере, интервал сохранения $\tau = 600$ s и $r = 120$ s. Затраты составят 420 s, то есть около восьми шагов при длительности шага 52,8 s в варианте с 48 ускорителями. Единичное колебание до $\mu + 3\sigma$ приводит к потере лишь нескольких секунд, поэтому исключение оправдано только тогда, когда ускоритель остаётся медленным на протяжении нескольких шагов. Медленные ускорители действительно встречаются в реальных кластерах: согласно MegaScale, около 0,5% машин заметно замедлялись. Статистика кластера Meta из раздела 10.4.4 также показывает зависимость между размером задания и прерываниями: среднее время безотказной работы задания на 1024 ускорителях составляет 7,9 часа, а задания на 8 ускорителях — 47,7 дня.³²

Помимо аппаратных сбоев, отката к checkpoint требует ещё одно событие — всплеск функции потерь из раздела 10.1.2. Всплеск функции потерь не повреждает оборудование, но обрабатывается так же: текущие параметры отбрасываются и выполняется возврат к предыдущему снимку. Его влияние на период сохранения также совпадает с влиянием аппаратного сбоя. Если считать откат из-за всплеска функции потерь воздействием на всё задание одновременно, происходящим в среднем раз в $1/\lambda_{\text{spike}}$, модель первого порядка из предыдущего раздела принимает вид

$$L(\tau) = \frac{c}{\tau} + (\lambda_{\text{hw}} + \lambda_{\text{spike}}) \left(\frac{\tau}{2} + r \right), \quad \tau^* = \sqrt{\frac{2c}{\lambda_{\text{hw}} + \lambda_{\text{spike}}}}.$$

Сохраним частоту аппаратных сбоев для 48 ускорителей — MTBF каждого ускорителя около 337 дней — и предположим, что всплеск функции потерь в среднем раз в семь дней приводит к откату. При том же $\tau = 600$ s дополнительные затраты увеличиваются с 2,80% до 2,87%, а оптимальный период сохранения сокращается примерно с 4 458 s до 3 150 s. При интервале 1800 s из проектного примера этой главы дополнительные затраты возрастают с 1,08% примерно до 1,25%, а время завершения — примерно на 0,03 дня, что не меняет выбор варианта в разделе 10.6. Условие смены режима определяется соотношением двух частот: чем чаще происходят всплески, тем короче оптимальный период; при $\lambda_{\text{spike}} \gg \lambda_{\text{hw}}$ период сохранения определяется почти исключительно численной

³² Модель максимума отстающих исполнителей, где σ составляет 2% среднего, и вариант, где σ составляет 5% среднего. Заметное замедление примерно 0,5% машин отмечено в MegaScale; среднее время без сбоев составляет 7,9 часа для задания на 1024 картах и 47,7 дня для задания на 8 картах согласно статье о надёжности кластера Meta, а частота сбоев одной карты в разделе о периоде сохранения пересчитана из 7,9 часа для 1024 карт. Стандартное отклонение вычислительного времени каждой карты и интервал отката из-за всплеска функции потерь приняты для этого примера; математическое ожидание максимума получено численным интегрированием порядковой статистики без моделирования корреляции, периодического дрожания или устойчиво медленных карт.

стабильностью. Если интервалы между откатами измеряются месяцами, они почти не влияют на дополнительные затраты в 1,08%.³³

10.5 Обучение с подкреплением

В первых четырёх разделах предполагалось, что обучающие данные уже заданы. RL позволяет модели участвовать в создании следующей порции данных: модель генерирует траектории, система вычисляет вознаграждения или результаты проверки, затем траектории используются для обучения, а обновлённые веса изменяют генерацию на следующей итерации. Система превращается из обучающего конвейера, обрабатывающего фиксированные данные, в цикл обратной связи, и новые проблемы сосредотачиваются на скорости обработки отдельных этапов, перекрытии состояний при переключении и определении того, какой именно версией стратегии был создан образец. Сторона инференса, генерирующая траектории, и сторона обучения, вычисляющая функцию потерь, должны также выдавать сопоставимые вероятности для одной и той же траектории — это и есть рассматриваемая в данном разделе **согласованность обучения и инференса**. Она напрямую влияет на то, соответствуют ли обновления параметров предполагаемому алгоритму, а также ограничивает выигрыш от ускорения этапов и асинхронного выполнения.

10.5.1 Одна итерация генерации ответов, проверки и обновления параметров

Одна итерация цикла обратной связи выполняется следующим образом. Для четырёх промптов (prompt) генерируется по два ответа, что даёт восемь траекторий. Вознаграждение описывает результат этих ответов, а преимущество измеряет, насколько ответ лучше выбранного базового уровня, и используется для определения направления и величины изменения вероятности ответа. Сторона обучения формирует функцию потерь по эффективным токенам и выполняет одно обновление параметров. После передачи новых весов стороне генерации образцы следующей итерации поступают уже от обновлённой стратегии. Генерация, вычисление вознаграждений и обучение используют данные разных форматов, а идентификаторы образцов связывают их между собой.

Источник данных здесь отличается от SFT, обычно использующего фиксированный набор данных. При SFT следующий токен предсказывается на истинном префиксе заданного ответа, то есть применяется принуждение учителем; при развёртывании модель продолжает ответ на основе сгенерированного ею самой префикса. Если ранее была допущена ошибка, дальнейшая генерация может перейти в контекст, слабо представленный в обучающих данных, и ошибки будут накапливаться по мере генерации. Такой сдвиг распределения префиксов между обучением и применением — одно из ограничений SFT на фиксированных данных. Градиент по-прежнему может правильно соответствовать текущей функции потерь, но ситуации, охваченные обучением, отличаются от

³³Модель максимума отстающих исполнителей, где σ составляет 2% среднего, и вариант, где σ составляет 5% среднего. Заметное замедление примерно 0,5% машин отмечено в MegaScale; среднее время без сбоев составляет 7,9 часа для задания на 1024 картах и 47,7 дня для задания на 8 картах согласно статье о надёжности кластера Meta, а частота сбоев одной карты в разделе о периоде сохранения пересчитана из 7,9 часа для 1024 карт. Стандартное отклонение вычислительного времени каждой карты и интервал отката из-за всплеска функции потерь приняты для этого примера; математическое ожидание максимума получено численным интегрированием порядковой статистики без моделирования корреляции, периодического дрожания или устойчиво медленных карт.

тех, с которыми модель сталкивается на практике.

Чтобы противодействовать этому сдвигу распределения префиксов, OPD позволяет текущей модели-ученику самостоятельно генерировать траектории, после чего учитель предоставляет надзор для фактически сгенерированных учеником префиксов, например распределение вероятностей следующего токена. Благодаря этому ученик может обучаться именно в тех местах, где чаще ошибается, что уменьшает сдвиг распределения префиксов; обратная связь учителя для каждого токена также даёт более детальный обучающий сигнал, чем единственное итоговое вознаграждение. В DeepSeek V4 с помощью OPD с несколькими учителями способности предметных экспертов объединяются в одной модели; в главе 3 прямой проход учителя уже был выделен как отдельная вычислительная задача.³⁴

Потенциальное преимущество OPD перед SFT на фиксированных данных состоит в возможном сокращении количества обучающих образцов или обновлений, необходимых для достижения целевого уровня возможностей. Одновременно возрастает стоимость онлайн-генерации учеником, прямых проходов учителя и синхронизации весов, поэтому сокращение общего числа GPU-часов или времени выполнения необходимо измерять на практике. SFT описывает цель обучения, а офлайн-режим (offline) — то, были ли данные зафиксированы заранее; можно также собирать данные онлайн, а затем применять функцию потерь с учителем. Онлайн-режим (online) означает, что данные непрерывно создаются в процессе обучения, а on-policy дополнительно требует соответствия распределения выборки оптимизируемой стратегии. Асинхронное онлайн-обучение с подкреплением может использовать отстающую стратегию, а в OPD также возникают вычислительные различия между движками генерации и обучения. Поэтому онлайн-сэмплирование применяется для уменьшения сдвига распределения данных, однако для согласованности обучения и инференса необходимо также согласовать вычислительные процессы на сторонах генерации и обучения.

Вернёмся к восьми траекториям из итерации в начале раздела и рассмотрим, как сторона обучения формирует функцию потерь по эффективным токенам. Пример с восемью ответами из раздела 10.3.1 основан на длинах ответов из эксперимента с небольшой моделью: ответы содержат в общей сложности 20 эффективных токенов, включая токен EOS, обозначающий конец последовательности; входные данные для вычислений содержат в общей сложности 366 эффективных токенов. И промпты, и ответы участвуют в прямом проходе, но функция потерь нормализуется по выбранным токенам ответа. Поэтому длина генерации влияет на вычислительные затраты; маска функции потерь (mask) указывает, учитывается ли каждый токен в функции потерь, и определяет, какие токены участвуют в вычислении градиента. Эти механизмы выполняют разные функции.³⁵

Помимо функции потерь, преимущество определяется относительными вознаграждениями ответов на один и тот же промпт. Например, если оба ответа получили вознаграждение 1, после вычитания среднего по группе оба значения будут равны нулю; если вознаграждения равны 1 и 0, после центрирования они составят 0,5 и $-0,5$, и только

³⁴Технический отчёт DeepSeek V4, §5.1 о доменных экспертах и OPD с несколькими учителями; распределение работы по этапам см. в главе 3.

³⁵Реальный замкнутый цикл фиксированной малой модели verl, проверка исходного кода функции потерь и обновления. В фиксированной конфигурации обучения Qwen2.5-0.5B-Instruct используется одна карта, NO_SHARD и no_sync=False; исходный код и зафиксированные версии приведены в журнале по ссылке. Значения 0,32/0,30/0,04 выведены на основе реальной структуры длин и учебного скалярного значения функции потерь.

тогда стратегия получит направление, позволяющее различить два ответа. Даже после выполнения прямого и обратного проходов и вызова оптимизатора система может получить batch, в котором все преимущества равны нулю. При оценке пропускной способности обучения следует считать фактическое количество траекторий или токенов, используемых для обучения в секунду, а рост возможностей модели проверять на фиксированном наборе оценочных задач.

Пример: почему параметры изменились, а возможности модели не улучшились? В одном фиксированном эксперименте с небольшой моделью фреймворк обучения с подкреплением verl работал совместно с движком инференса vLLM: вознаграждение за точное совпадение ответа в арифметических задачах давало ответам внутри одной группы одинаковые вознаграждения, поэтому преимущества и градиент стратегии были равны нулю, а изменения параметров происходили только из-за затухания весов AdamW. Помимо градиентного обновления Adam, AdamW отдельно пропорционально уменьшает веса; этот шаг способен изменить параметры, даже если текущий градиент стратегии равен нулю. После перехода на вознаграждение с фиксированным знаком на обоих шагах обучения появились ненулевые градиенты и обновления весов, но проверка арифметических задач дала лишь 2 правильных ответа из 4. Конструкция вознаграждения определяет направление обучения, а исполнение системы отвечает за согласованное применение этого направления к параметрам.³⁶

10.5.2 Распределение ресурсов между этапами и синхронизация весов

Обсуждать количество обучаемых в секунду образцов можно лишь после того, как определено, каким образом образцы преобразуются в градиенты. На этапе обратной связи RL могут выполняться проверка по правилам или модель вознаграждения, оценивающая ответы, а в OPD требуется надзор со стороны учителя; в обоих случаях ресурсы следует распределять в соответствии с фактической пропускной способностью этих операций обратной связи. Генерация, проверка и обучение подобны трём последовательным производственным операциям: ускорение предыдущей увеличивает итоговый результат, только если следующая способна обработать возросший поток. Сначала выразим пропускную способность всех трёх этапов в одной единице работы. Пусть она равна соответственно r_g, r_v, r_l , а доля образцов, которые после проверки соответствуют требованиям к версии стратегии, равна q . При устойчивой конвейерной работе на независимых ресурсах верхний предел количества пригодных для обучения образцов в секунду составляет

$$r_{\text{effective}} = \min(r_l, q \min(r_g, r_v)) .$$

Пример: какой из трёх этапов RL — генерацию, проверку или обучение — следует масштабировать в первую очередь? Генерация выдаёт 12 траекторий одинаковой длины в секунду, проверка обрабатывает шесть, а этап обучения — восемь; после проверки четверть траекторий отбрасывается из-за слишком старой версии. На обучение

³⁶Реальный замкнутый цикл фиксированной малой модели verl, проверка исходного кода функции потерь и обновления. В фиксированной конфигурации обучения Qwen2.5-0.5B-Instruct используется одна карта, NO_SHARD и no_sync=False; исходный код и зафиксированные версии приведены в журнале по ссылке. Значения 0,32/0,30/0,04 выведены на основе реальной структуры длин и учебного скалярного значения функции потерь.

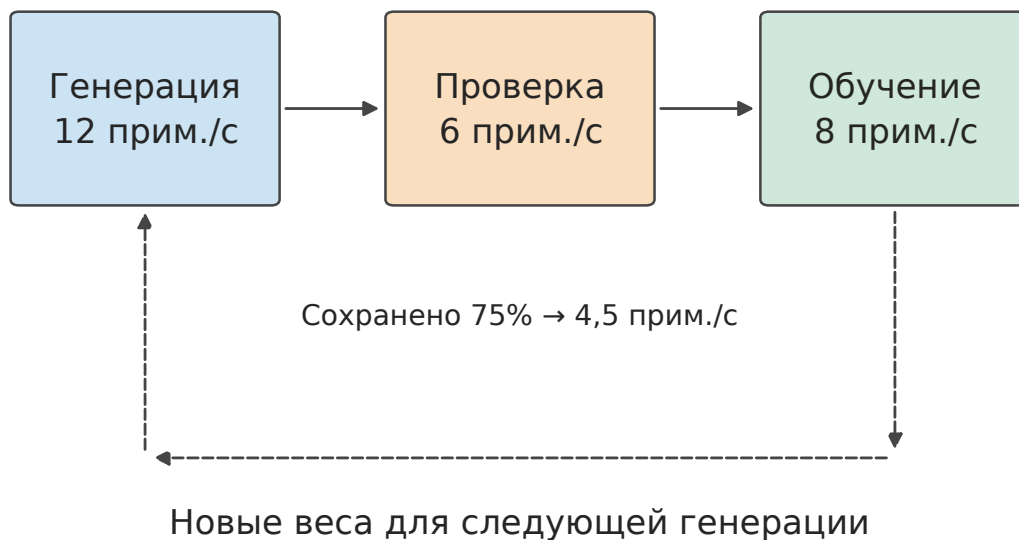


Рис. 10-37. Три этапа могут обрабатывать не более 12, 6 и 8 траекторий одинаковой длины в секунду соответственно. После проверки сохраняется 75%, поэтому на обучение поступает лишь 4,5 траектории/с. Обратная стрелка показывает, что новые веса, полученные при обучении, влияют на последующую генерацию; время их синхронизации рассматривается далее при разборе переключения этапов и асинхронного выполнения.

поступает лишь $6 \times 0.75 = 4.5$ траектории в секунду. Если удвоить генерацию до 24 траекторий/с, узкое место на этапе проверки по-прежнему будет ограничивать поток на обучение значением 4,5 траектории/с; если удвоить пропускную способность проверки до 12 траекторий/с, стороне обучения можно будет передавать девять траекторий в секунду, но она сможет обрабатывать лишь восемь и станет узким местом всего конвейера. Поэтому дополнительные ресурсы следует выделять текущему самому медленному этапу, а после его ускорения заново определять, куда переместилось узкое место.

На рис. 10-37 три операции сначала показаны на отдельных ресурсах. Если вместо этого поочерёдно выполнять их на одном наборе ускорителей, можно уменьшить необходимое количество ускорителей, но между генерацией и обучением придётся переключать весь набор данных в видеопамяти.

Совместное использование ускорителей позволяет одному набору ускорителей поочерёдно выполнять обучение и генерацию, сокращая дублирование ресурсов; **независимое развёртывание** позволяет обеим сторонам работать независимо и перекрываться по времени, но требует передачи весов между двумя наборами ускорителей. Основные затраты при совместном использовании ускорителей возникают во время переключения: старое состояние ещё не освобождено, а загрузка нового уже началась. Видеопамяти может хватать для каждого этапа по отдельности, но при переключении одновременное размещение двух наборов данных способно превысить её объём.

Пусть совместно используемый ускоритель — H100 SXM с номинальными 80 GB, что

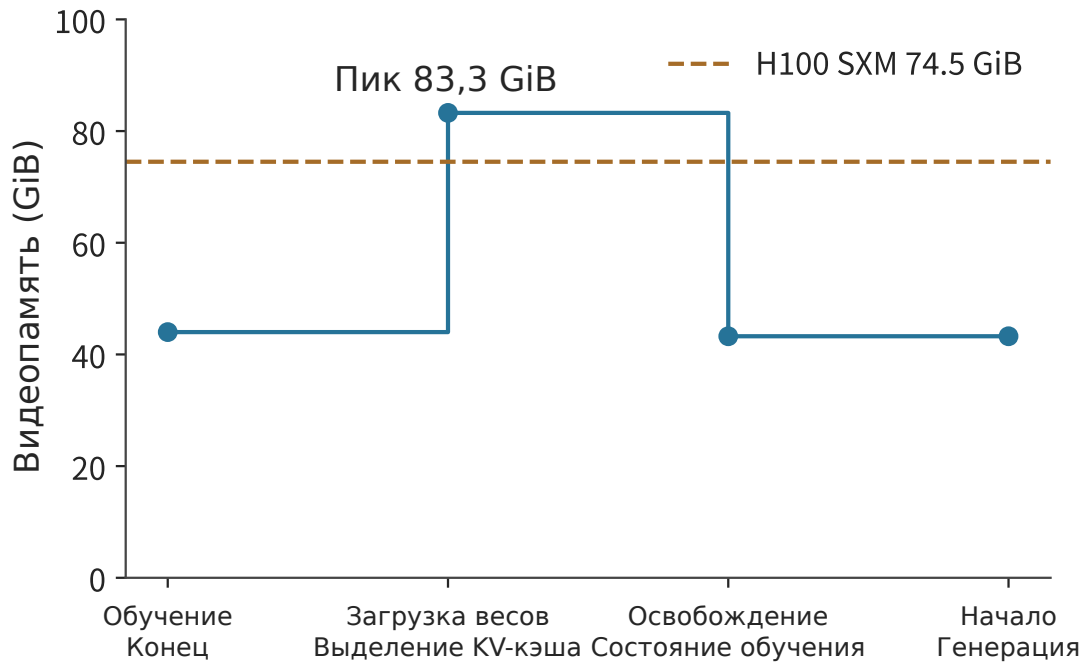


Рис. 10-38. Если сначала загрузить веса генерации и выделить пул KV, а затем освободить состояние обучения, пиковое потребление составит около 83,3 GiB, превысив доступные на H100 SXM 74,5 GiB. Горизонтальная ось соответствует порядку операций.

соответствует приблизительно 74,5 GiB. Состояние обучения занимает 40 GiB видеопамяти, а для генерации требуется около 15,3 GiB весов BF16 и 24 GiB пула KV-кэша; ещё 4 GiB занимают общие накладные расходы двух этапов. При монопольном выполнении обучение занимает 44 GiB, а генерация — около 43,3 GiB, что в обоих случаях меньше 74,5 GiB. Если сначала восстановить веса генерации и KV, а затем освободить состояние обучения, пиковое потребление составит $40 + 15,3 + 24 + 4 \approx 83,3$ GiB, превысив объём одной H100.

Изменим порядок: сначала загрузим необходимые для генерации веса, пока не выделяя KV. Пиковое потребление составит $40 + 15,3 + 4 \approx 59,3$ GiB. Затем освободим 40 GiB состояния обучения, снизив потребление видеопамяти примерно до 19,3 GiB, после чего выделим KV и перейдём в состояние генерации, занимающее около 43,3 GiB. Конечное состояние обоих вариантов одинаково, но их пиковое потребление отличается на 24 GiB — ровно на размер одного пула KV.³⁷

Стоимость независимого развёртывания определяется объёмом передаваемых весов. Экспертные веса Qwen3-235B-A22B в формате BF16 занимают 423 GiB; при 16-кратном экспертном параллелизме (EP16) каждому получателю требуется лишь около 26,4 GiB. При последовательной отправке полного набора экспертов каждому из 16 получателей общий объём передачи составит $16 \times 423 = 6768$ GiB; при

³⁷Пиковое потребление общего устройства Qwen3-8B, синхронизация шардированных весов Qwen3-235B-A22B. 40 GiB — заданное потребление видеопамяти после шардирования/выгрузки; два пиковых значения при переключении составляют примерно 83,256/59,256 GiB и сравниваются с номинальными 80 GB (74,506 GiB) H100 SXM.

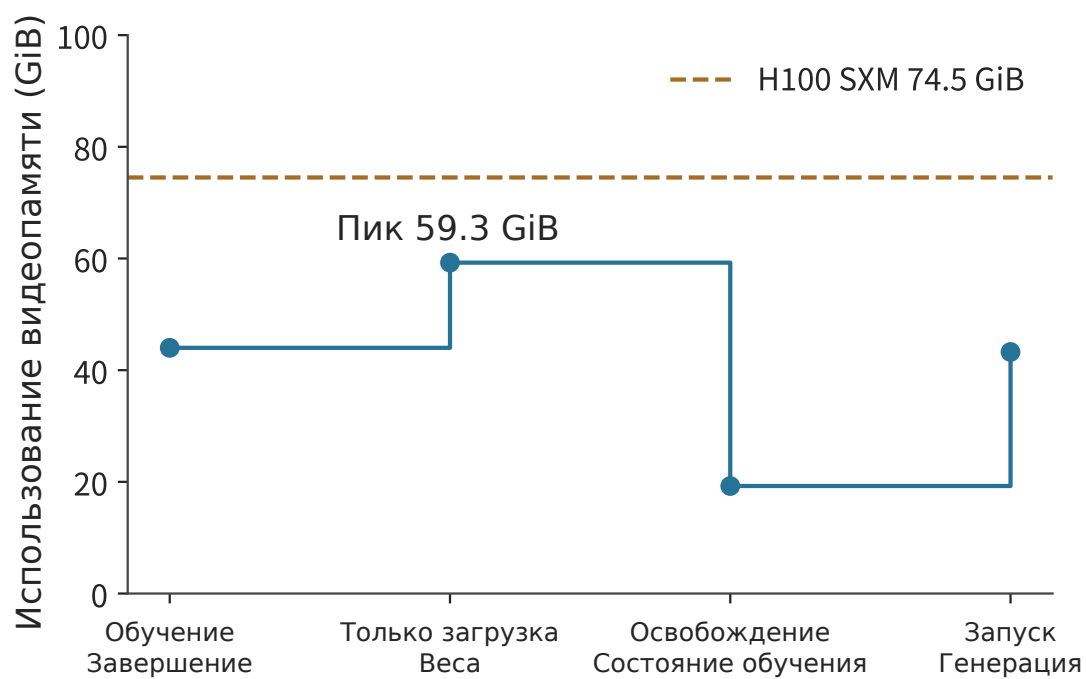


Рис. 10-39. Сначала загружаются веса генерации, затем освобождается состояние обучения и в конце выделяется пул KV. Пиковое потребление снижается примерно до 59,3 GiB; на обоих графиках используются общая линия ёмкости 74,5 GiB и одна вертикальная шкала.

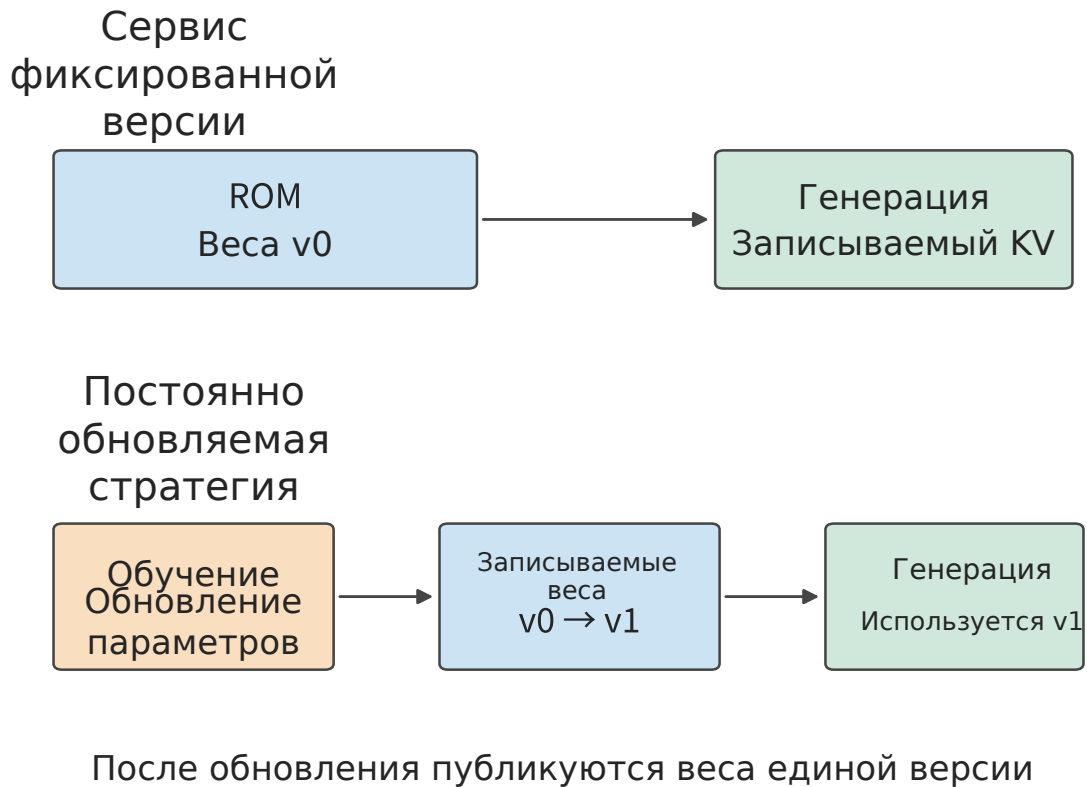
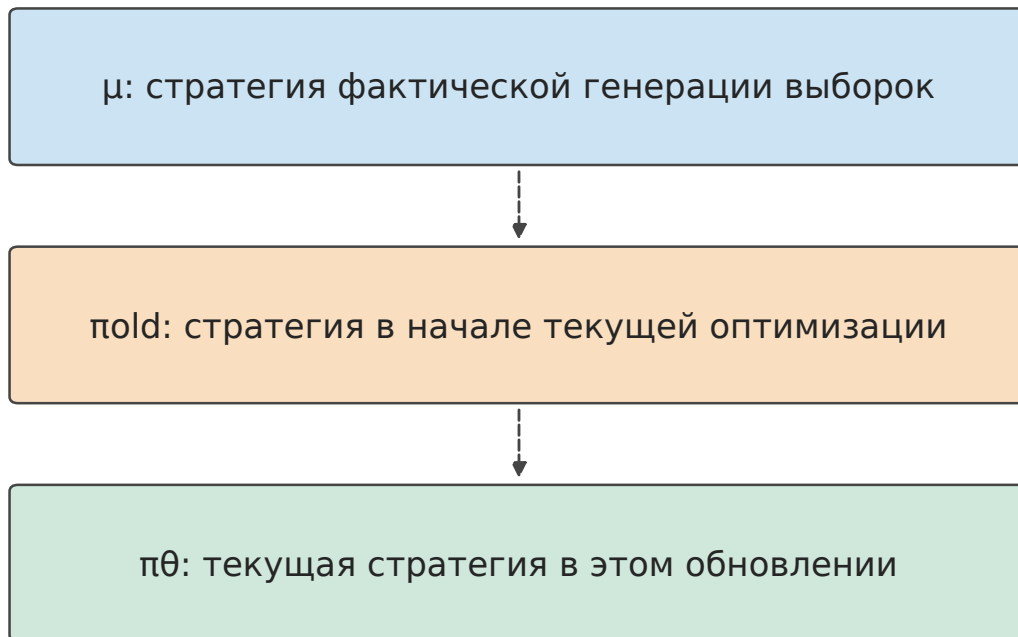


Рис. 10-40. Жизненный цикл весов сервиса с фиксированной версией и обучения стратегии. В верхней части ROM многократно предоставляет одну и ту же версию; в нижней части обучение создаёт новые версии и публикует их для стороны генерации. Для записи KV и обновления весов используются разные тракты данных.

отправке каждому только его сегмента — 423 GiB. Если сначала определить, какой сегмент параметров нужен каждому получателю, а затем организовать рассылку, можно одновременно уменьшить приёмный буфер и объём данных, отправляемых источником.

Здесь вновь используются два метода анализа из этой главы: при совместном использовании ускорителей необходимо анализировать жизненный цикл тензоров, а при независимом развёртывании — передачу данных и время обработки на каждом этапе. После разделения этапов также возникают различия между версиями стратегии; в следующем подразделе рассматривается, как они влияют на количество пригодных для обучения образцов.

При синхронизации весов существует ещё один тип весов с иным жизненным циклом: ускоритель фиксированных весов из раздела 4.7.3 способен обслуживать модели, веса которых длительное время не меняются в процессе RL. Например, если алгоритм использует фиксированную эталонную модель, её веса можно хранить в специализированном хранилище и многократно считывать; для постоянно обновляемой модели стратегии используется записываемое хранилище весов, а новые версии публикуются для стороны генерации. На рис. 10-40 пути весов показаны для этих двух жизненных циклов.



Вероятности префикса и токена записываются отдельно

Рис. 10-41. μ создаёт обучающие образцы, π_{old} обозначает начальную точку текущей оптимизации, а π_{θ} изменяется по мере текущих обновлений. Пунктирная линия показывает порядок эволюции версий. Все три вероятности должны вычисляться для одного и того же префикса и одного и того же токена.

10.5.3 Асинхронное обучение, версии стратегии и восстановление длинных траекторий

Независимое развёртывание из раздела 10.5.2 также позволяет чередовать работу над разными batch: пока сторона обучения обрабатывает предыдущую порцию образцов, сторона генерации уже начинает создавать следующую. Синхронный цикл, напротив, ожидает завершения обучения и синхронизации весов, прежде чем генерировать следующую порцию. Асинхронный цикл обеспечивает перекрытие генерации и обучения, но сторона генерации при этом может продолжать использовать старые веса. Чтобы вычислить вклад образца в обновление текущих параметров, необходимо сохранять вероятности стратегии, фактически использованной при сэмплировании.

После обновления весов задаваемая моделью стратегия также изменяется. Ниже в порядке движения образца выделены три версии стратегии.

Обозначим фактическое поведенческое распределение через μ , стратегию в начальной точке текущей оптимизации — через π_{old} , а текущую оптимизируемую стратегию — через π_{θ} . Начальная стратегия обновляется вместе с итерациями оптимизации, тогда как эталонная модель, используемая для регуляризации с помощью дивергенции KL, остаётся фиксированной. Для одного и того же токена и префикса в позициях, где определены все три вероятности и знаменатель не равен нулю, отношение вероятностей удовлетво-

ряет равенству

$$\frac{\pi_{\theta}}{\mu} = \frac{\pi_{\theta}}{\pi_{\text{old}}} \frac{\pi_{\text{old}}}{\mu}.$$

Первый множитель справа описывает изменение вероятности в результате текущей оптимизации параметров, второй — различие между фактическим распределением сэмпирования и начальной стратегией текущей итерации. В это различие могут входить отставание версии стратегии, различия конфигураций сэмпирования и вычислительные различия между движками. Если поведенческая и текущая стратегии дают для данного токена логарифмическую вероятность ($\log\text{prob}$), равную -3 , итоговое отношение равно 1. Если вместо первоначально записанного значения поведенческой стратегии ошибочно использовать заново вычисленную логарифмическую вероятность -3.25 , отношение станет равно $\exp(0.25) \approx 1.28$. Параметры не изменились, но вес образца вырос примерно на 28%. Только сохранение поведенческой вероятности позволяет стороне обучения отличить изменение стратегии от вычислительных различий.³⁸

Даже если синхронизировать веса перед каждой итерацией, второй множитель необязательно будет равен 1. Сторона генерации обычно выполняет декодирование последовательно по токенам и считывает KV-кэш, тогда как сторона обучения параллельно подаёт всю траекторию, заново вычисляет вероятности в каждой позиции токена, а затем выполняет обратное распространение. Разные реализации внимания, порядок редукции, форма batch, точность весов или KV могут приводить к различным $\log\text{prob}$ даже при одинаковых весах и префиксах. В разделе 5.2.4 уже измерялось одно из таких различий: при редукции одной и той же строки с разным количеством сегментов сумма квадратов различалась на 166 ULP, а количество сегментов зависело от текущего размера batch. Сначала расхождения между сторонами появляются в прямом проходе; обратный проход переносит различия функции потерь, сформированные прямым проходом обучения, в градиенты. Дискретная маршрутизация МоЕ дополнительно усиливает такие различия, что рассматривается в следующем подразделе.

Часть второго множителя также определяется конфигурацией сэмпирования: поведенческая вероятность вычисляется по фактически применённым правилам сэмпирования. Масштабирование температурой и усечение top-k или top-p изменяют исходное распределение Softmax модели, поэтому следует сохранять преобразованные вероятности и конфигурацию сэмпирования. Чтобы вычислить отношение вероятностей для одного и того же события, сторона обучения должна восстановить входные данные с соответствующими токенизацией, шаблоном диалога, номерами позиций, маской внимания и позициями EOS/усечения.

В обновлениях класса проксимальной оптимизации стратегии (PPO) отношение вероятностей определяет вес преимущества в градиенте и участвует в принятии решения об отсечении: если отношение отклоняется от 1 дальше заданного диапазона, его вклад в градиент ограничивается. Если ошибочно принять различия реализации за результат обновления параметров, вклад образца может быть необоснованно увеличен или уменьшен, а отсечение — сработать ошибочно; при использовании отношения вероятностей последовательностей различия $\log\text{prob}$ в отдельных позициях токенов также суммируются. Поэтому в более длинных траекториях накапливается больше вероятностных отклонений, из-за чего обновления легче становятся нестабильными.³⁹

³⁸Состояние RL, отношение вероятностей и версии, технический отчёт DeepSeek V4.

³⁹Состояние RL, отношение вероятностей и версии, технический отчёт DeepSeek V4.

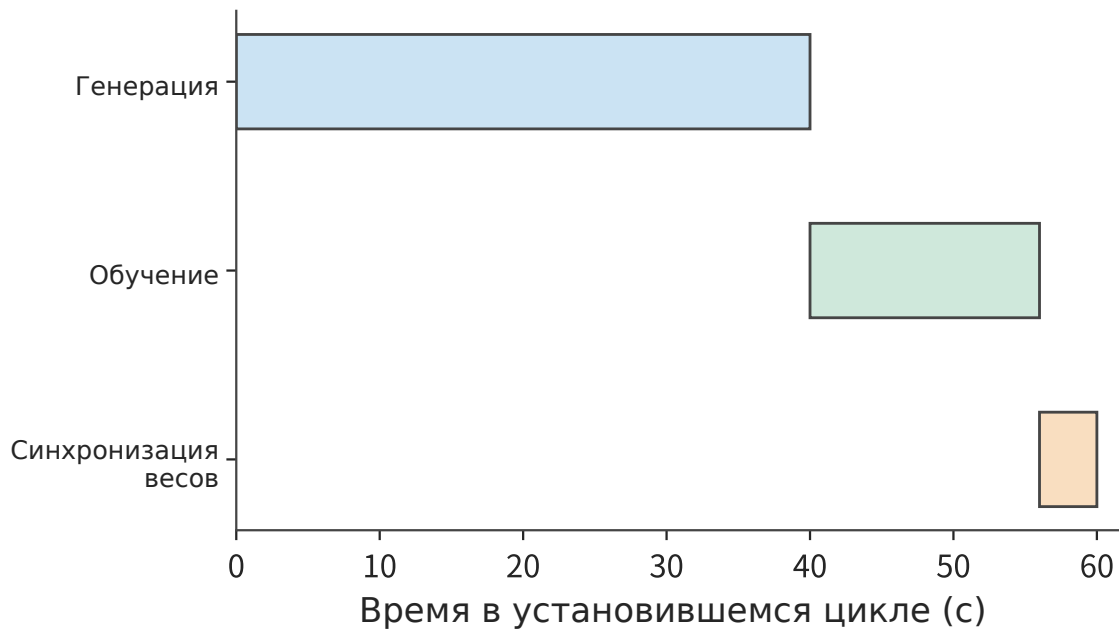


Рис. 10-42. Синхронный цикл последовательно генерирует текущую порцию образцов, обучается на ней и синхронизирует веса, затрачивая соответственно 40, 16 и 4 s, всего 60 s.

Взвешивание по значимости корректирует вклад уже выбранных событий с помощью отношения вероятностей, поэтому распределение сэмплирования должно охватывать целевые события. Токены, исключённые усечённым сэмплированием, невозможно вернуть с помощью отношений вероятностей уже имеющихся образцов; при слишком большом различии распределений дисперсия весов также возрастает. Отсечение позволяет ограничить экстремальные веса, но вносит смещение, причём раздельное отсечение двух множителей и отсечение итогового отношения дают разные функции потерь. Асинхронное выполнение увеличивает временной разрыв между версиями стратегии, поэтому далее доля пригодных образцов учитывается вместе с периодом выполнения при расчёте выигрыша.

Пример: насколько высокую долю устаревших образцов может выдержать асинхронный конвейер? Генерация одной порции занимает 40 s, обучение — 16 s, а блокирующая обе стороны синхронизация весов — 4 s, поэтому синхронный период составляет 60 s. Если генерация и обучение используют независимые ресурсы, но синхронизация весов по-прежнему монопольно занимает 4 s, асинхронный период в установившемся режиме равен $\max(40, 16) + 4 = 44$ s.

При переходе от синхронного выполнения на рис. 10-42 к асинхронному на рис. 10-43 сокращается время, в течение которого стороны обучения и генерации поочерёдно ожидают друг друга. Однако более ранняя генерация также означает, что образцы могли быть получены со старыми весами, поэтому необходимо вычесть отброшенные образцы.

Пусть изначально каждая порция содержит B эквивалентных траекторий, а после перехода к асинхронному выполнению для обучения сохраняется доля q . Тогда эффективная пропускная способность синхронного и асинхронного выполнения равна соответственно $B/60$ и $qB/44$. Чтобы асинхронное выполнение было быстрее, необходимо условие

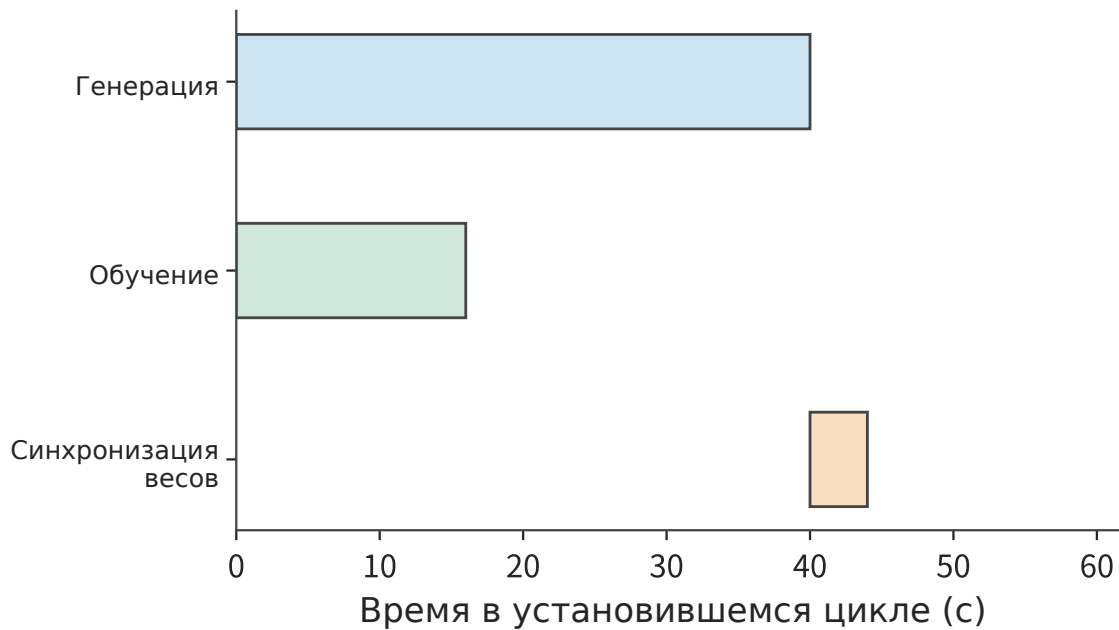


Рис. 10-43. В установившемся режиме генерация следующей порции перекрывается на независимых ресурсах с обучением на предыдущей; после завершения обоих этапов выполняется синхронизация весов, а период составляет 44 с. Оранжевый этап синхронизации блокирует обе стороны.

$$q > 44/60 = 11/15 \approx 73.3\%.$$

При $q = 0.8$ эффективная пропускная способность по образцам возрастает примерно на 9%; при $q = 0.7$, несмотря на более раннее завершение каждой порции, она, напротив, снижается примерно на 5%. Поэтому при расчёте выигрыша от асинхронного выполнения необходимо вычитать образцы, отброшенные из-за слишком старой версии стратегии.

Этап генерации на рисунке показан как один непрерывный отрезок работы. Чем длиннее траектория, тем вероятнее, что этот отрезок пересечётся с обновлением параметров или будет прерван, поэтому помимо отбора завершённых образцов необходимо решить, как продолжать незавершённые траектории. Длинные траектории также увеличивают временной разрыв между генерацией образца и его использованием для обучения. KV представляет собой результат выполнения префикса с заданными весами: при восстановлении с теми же весами сохранённый KV можно продолжить использовать, а после смены весов префикс необходимо выполнить заново, чтобы получить соответствующий KV. Для Qwen3-8B KV в формате BF16 при длине 8K занимает 1,125 GiB. Сохранение и загрузка KV-кэша требуют места в хранилище и времени на передачу, а его восстановление — повторного выполнения вычислений prefill. Поэтому варианты восстановления сравниваются по времени загрузки и времени повторного построения.

Если всегда отбрасывать прерванные траектории и выполнять сэмплирование заново, длинные траектории будут отбрасываться чаще из-за большего времени выполнения. Пусть интенсивность прерываний во время генерации равна λ ; тогда вероятность того, что траектория продолжительностью t будет полностью сгенерирована без пре-

рывания, равна $e^{-\lambda t}$. С ростом t доля данных, поступающих на обучение, уменьшается. Если для каждого токена записывать прогресс генерации и версию стратегии, после восстановления можно продолжить генерацию той же траектории и уменьшить вызванное системными прерываниями смещение по длине. Сохранение состояния траектории на сторонах модели и среды при прерываниях рассматривается в разделе 11.3.3.

Подход, при котором обучение и развёртывание используют один путь, уже применяется в реальных моделях. В DeepSeek V4.1 используемые при развёртывании механизмы разреженного доступа и восстановления включены в обучение. Разреженное внимание с самого начала обучения работает с последовательностями длиной 64K, а на этапе постобучения добавляется иерархическое ограничение кандидатов, благодаря чему при обучении и инференсе используется одна область поиска; обучение с учётом квантования, при котором во время обучения имитируется ошибка низкоточной квантизации, позволяет модели адаптироваться к основному KV в формате FP4; для восстановления состояния SWA декодера повторно воспроизводится лишь вход ограниченной длины, и этот процесс также моделируется на этапе постобучения. Благодаря этому модель учится использовать представления и локальные состояния, фактически предоставляемые путём развёртывания.⁴⁰

10.5.4 Повторное воспроизведение экспертной маршрутизации и согласованность обучения и инференса

МоЕ превращает численные различия из раздела 10.5.3 в дискретные различия вычислительных путей: даже при одинаковых весах различия численных вычислений между сторонами генерации и обучения могут изменить выбор экспертов. Поскольку выбор экспертов дискретен, небольшое численное изменение способно направить токен по другому вычислительному пути. Например, при выборе двух экспертов с наибольшими оценками (top-2) из первых трёх экспертов с оценками 0,500, 0,301 и 0,300 будут выбраны эксперты 1 и 2; если из-за изменения порядка вычислений третья оценка станет равна 0,302, выбор изменится на экспертов 1 и 3. Очень небольшое изменение оценки может привести к использованию матрицы весов другого эксперта, а его влияние продолжит распространяться по последующим слоям.

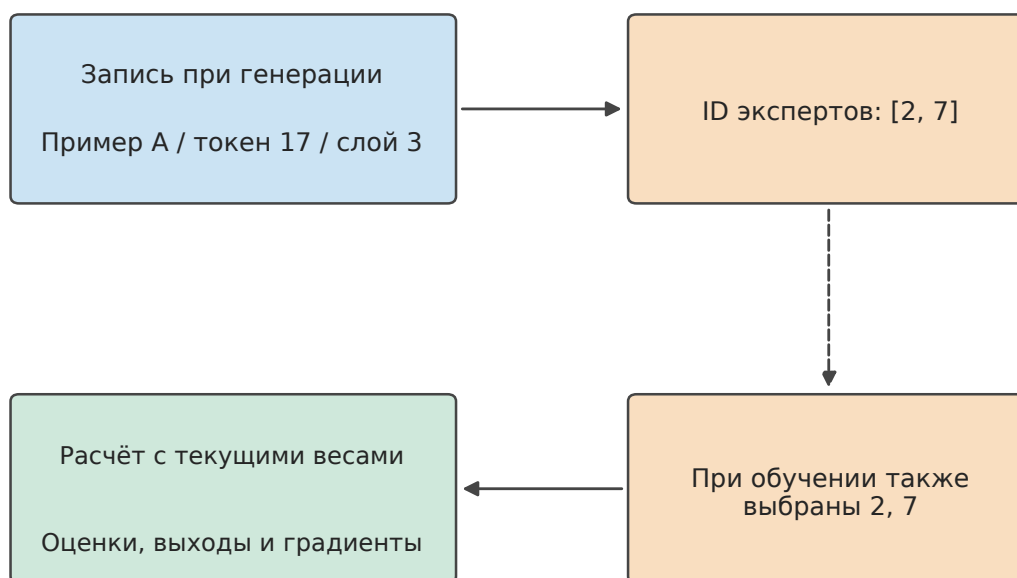
Повторное воспроизведение маршрутизации (Routing Replay) записывает логические ID экспертов, выбранных при генерации, а во время обучения выбирает экспертов по этим ID, после чего вычисляет оценки маршрутизации, выходы экспертов и градиенты с текущими весами. Конкретная запись определяется совместно образцом, позицией токена и номером слоя. Благодаря этому генерация и обучение проходят через одинаковые дискретные пути экспертов, тогда как численные вычисления продолжают отражать текущие параметры.

Сначала вычислим объём такой записи. Для каждого из 48 слоёв Qwen3-30B-A3B записываются номера восьми экспертов с наибольшими оценками (top-8). Для 8 192 токенов при сохранении каждого ID эксперта в формате беззнакового целого uint16 размером 2 байта объём составляет

$$8192 \times 48 \times 8 \times 2 = 6 \text{ MiB.}$$

При использовании формата int32 размером 4 байта на целое объём составит 12 MiB.

⁴⁰Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и перерасчёты сквозного межглавного примера.



Дискретный выбор фиксирован; расчёты — с текущими весами

Рис. 10-44. Дискретные ID экспертов связывают стороны генерации и обучения, а текущие веса продолжают участвовать в численных вычислениях. На схеме показана запись выбора top-2 для образца А, токена 17 и слоя 3; пунктир обозначает повторное воспроизведение ID, сплошная линия — текущий поток вычисляемых данных.

Записи также должны перемещаться вместе с токенами: упаковка последовательностей изменяет позиции токенов, разбиение контекста меняет ускоритель, на котором находится токен, а повторное вычисление снова считывает запись того же слоя. Чтобы сторона обучения могла выбрать исходную группу экспертов, ID должны проходить через все эти преобразования вместе с исходными позициями токенов.⁴¹

Пример: влияние повторного воспроизведения маршрутизации на ошибку вероятностей, время обучения и вознаграждение. В опубликованном NVIDIA отчёте о проверке повторного воспроизведения маршрутизации R3, то есть Rollout Routing Replay, представлены четыре контрольных сравнения с включённой и отключённой функцией, каждое с обучением в течение 100 шагов. После включения медиана записанной в журналах ошибки логарифмической вероятности снизилась во всех случаях, медиана общего времени шага немного выросла, а вознаграждение при обучении не демонстрировало стабильного улучшения. Повторное воспроизведение уменьшает различия маршрутизации, но сохранение записей и само воспроизведение увеличивают время выполнения. При сравнении двух вариантов следует отдельно оценивать ошибку вероятностей, время обучения и результаты на фиксированном наборе оценочных задач.⁴²

При одинаковых весах и одной порции токенов различие logprob между сторонами генерации и обучения отражает влияние вычислительного пути и определения сэмплирования; ID экспертов MoE, в свою очередь, показывают, совпадает ли дискретная маршрутизация. Численная согласованность и степень отставания стратегии сэмплирования от текущей стратегии совместно влияют на количество эффективных обучающих образцов.

RL объединяет генерацию, проверку средой и обновление параметров в единый процесс обратной связи. Сторона генерации предоставляет вероятности сэмплирования и данные о выборе экспертов, сторона обучения использует эти данные для организации обновлений, а планировщик ресурсов организует перекрытие отдельных этапов. Сквозная передача этой информации позволяет одновременно оптимизировать передачу данных между этапами, согласованность выполнения и использование ресурсов. Примеры асинхронного периода и повторного воспроизведения маршрутизации в этом разделе количественно оценивают соответственно сокращение времени ожидания и дополнительные затраты на записи; весь цикл обратной связи в конечном счёте следует оценивать по эффективным образцам и прогрессу обучения.

10.6 От системного решения к сроку завершения и выбору оборудования

10.6.1 Сводный учёт ёмкости, времени шага, остановок и восстановления

Ниже приведён сводный расчёт времени выполнения задачи обучения из начала главы. Обе конфигурации обрабатывают 100B токенов, на каждом шаге — 384 последователь-

⁴¹Бюджет метаданных маршрутизации, опубликованные авторами R3 журналы и анализ. В четырёх парах настроек используется общий seed42; reward является метрикой обучающего batch, а logprob — статистикой уровня журнала от авторов R3.

⁴²Бюджет метаданных маршрутизации, опубликованные авторами R3 журналы и анализ. В четырёх парах настроек используется общий seed42; reward является метрикой обучающего batch, а logprob — статистикой уровня журнала от авторов R3.

ности по 8 192 токена, всего 31 790 итераций обучения. Пусть время обучения на одном шаге t_s уже включает вычисления на одной карте, ожидание коммуникации и ожидание входных данных, доля дополнительного времени на сохранение и восстановление равна L , а плановые остановки — T_0 . Тогда используемая в этой главе модель первого порядка для времени завершения имеет вид

$$T_{\text{finish}} = T_0 + \left\lceil \frac{D}{b} \right\rceil t_s(1 + L).$$

Все величины получены на разных уровнях анализа, проведённого в этой главе: анализ жизненного цикла тензоров позволяет проверить достаточность видеопамяти; анализ критического пути позволяет вычислить t_s ; модель checkpoint даёт L ; плановые остановки составляют $T_0 = 5$ дней. Объединив эти результаты, можно сравнить время выполнения двух конфигураций; итоговые значения в таблице рассчитаны по неокруглённым данным.⁴³

Параметр конфигурации	Конфигурация с 32 картами	Конфигурация с 48 картами
Количество хостов с восемью картами	4	6
ZeRO-3 для всей группы, количество накоплений одной последовательности на карту	12	8
Расчётное потребление видеопамяти на карту / GiB	13,8	12,5
Доступная видеопамять на карту / GiB	22	22
Вычисления на одной карте за шаг / с	78,3	52,2
Время передачи ZeRO-3 за шаг (PCIe 4.0 x16) / с	17,9	12,0
Ожидание коммуникации за шаг / с	0,08	0,08
Ожидание входных данных за шаг / с	0,5	0,5
Время обучения за шаг, включая 0,5 с ожидания входных данных / с	78,9	52,8
Дополнительное время на сохранение, повторное выполнение и восстановление	1,02%	1,08%
Базовое время обучения / дней	29,0	19,4
С учётом сохранения, восстановления и резерва в 5 дней / дней	34,3	24,6

⁴³Входные данные, выводы и перерасчёты примера проектирования из этой главы. Запасы ёмкости, ожидание входных данных и плановые остановки для 32/48 карт являются заданными входными данными; вычислительная эффективность одной карты взята из MFU Llama 3; объём матричных вычислений модели получен из существующего расчёта training-deadline, а объём коммуникации ZeRO-3 выведен из той же таблицы параметров. PCIe 4.0 и отсутствие NVLink у RTX 4090 указаны на странице спецификаций RTX 4090; отсутствие поддержки P2P между картами и конфигурация восьмикарточного хоста с восемью сетевыми картами по 200 Gbit/s приведены в анализе конфигурации кластера 4090; согласно §3.2 MegaScale, в пределах одного шага невозможно скрыть только первый AllGather и последний ReduceScatter. Значения по отдельным компонентам и независимые перерасчёты сохранены в design-case.json.

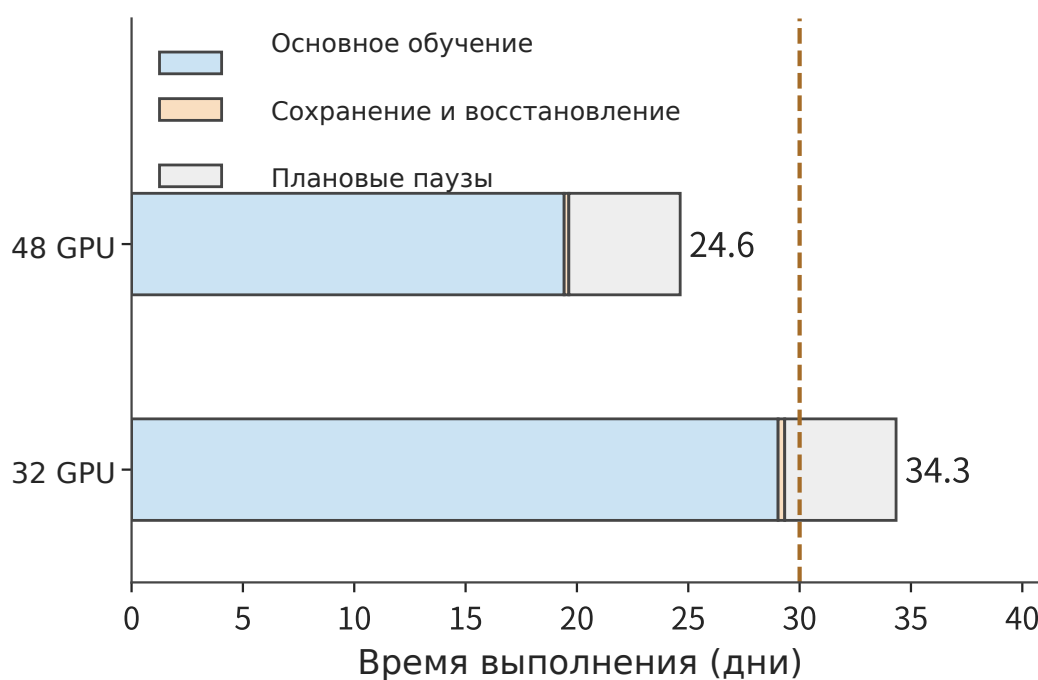


Рис. 10-45. Каждая горизонтальная полоса последовательно суммирует базовое время обучения, дополнительное время на сохранение и восстановление после сбоев, а также зарезервированные 5 дней плановых остановок. Базовое время обучения уже включает ожидание коммуникации и входных данных; небольшой оранжевый участок показывает дополнительные затраты времени, полученные из модели checkpoint. Обе конфигурации используют одну задачу и одинаковый глобальный батч, пунктирная линия обозначает срок в 30 дней.

На рис. 10-45 плановые остановки в обеих конфигурациях одинаковы, а сохранение и восстановление занимают лишь небольшую часть времени. Главное различие заключается в синем участке базового времени обучения: в конфигурации с 48 картами `micro-batch` распределяется для параллельной обработки между большим количеством карт, что значительно сокращает этот участок.

При данных условиях проектирования следует выбрать конфигурацию с 48 картами. Обе конфигурации удовлетворяют требованиям к ёмкости; конфигурация с 32 картами исключается, поскольку её время выполнения превышает 30 дней; конфигурация с 48 картами завершает работу приблизительно за 24,6 дня, оставляя около 5,4 дня. Основная ценность увеличения количества карт заключается в сокращении объёма работы, приходящегося на каждую карту, а дополнительные затраты из-за сбоев компенсируют лишь небольшую часть выигрыша.

10.6.2 Область применимости кластера из 4090

На рис. 10-45 показан запас приблизительно в 5,4 дня. Распределив этот запас по всем итерациям, можно определить допустимое дополнительное ожидание на каждом шаге и тем самым получить целевой показатель производительности для практической оптимизации. Время обучения на одном шаге для 48 карт должно удовлетворять условию

$$t_s \leq \frac{25 \times 86400}{31\,790 \times (1 + 0.0108)} \approx 67.2 \text{ s.}$$

В этой конфигурации оно составляет приблизительно 52,8 с, оставляя запас около 14,4 с на шаг. Если зафиксировать время вычислений на одной карте на уровне около 52,2 с, а ожидание входных данных — 0,5 с, то суммарное ожидание коммуникации может составлять не более примерно 14,5 с; согласно анализу перекрытия из раздела 10.3.3, сейчас оно равно лишь приблизительно 0,08 с. Даже при полном отказе предварительной выборки, когда все 12,0 с передачи оказываются неперекрытыми, шаг занимает около 64,8 с и по-прежнему укладывается в предел.

Для 32 карт необходимо повысить эффективность вычислений на одной карте. После вычета затрат на восстановление, 0,08 с коммуникации и 0,5 с ожидания входных данных на вычисления одной карты остаётся около 66,7 с, тогда как при эффективности 40% требуется приблизительно 78,3 с. Для выполнения того же объёма работы эффективность вычислений на одной карте должна повыситься примерно до $40\% \times 78.3/66.7 \approx 47\%$, что уже превышает максимальные 43%, указанные для Llama 3. Таким образом, можно напрямую сравнить два направления улучшения: добавить два хоста либо повысить эффективность вычислений на существующих картах с 40% примерно до 47%.

Бюджет коммуникации необходимо также связать с физическими интерфейсами. Время передачи 12,0 с из раздела 10.3.3 рассчитано при наличии отдельной сетевой карты 200 Gbit/s для каждой карты. Если восемь карт одного хоста совместно используют единственную сетевую карту 200 Gbit/s, то ребро кольцевой коллективной коммуникации, входящее в этот хост и выходящее из него, должно нести всю нагрузку по приёму и передаче данных каждой карты — это разрез из примера 7.1 главы 7. При 48 картах через эту сетевую карту с пропускной способностью 25 GB/s за каждый шаг должно проходить около 385 GB, поэтому время передачи составит примерно 15,4 с и превысит предельный бюджет коммуникации в 14,5 с. Время передачи одного `micro-batch` составит около 1,92 с и останется меньше 6,53 с вычислений, поэтому при

работающей предварительной выборке неперекрытыми по-прежнему будут только первая и последняя коммуникации; но при отказе предварительной выборки шаг займёт около 68,1 с, и конфигурация не уложится в срок. При отдельной сетевой карте на каждую карту даже полностью неперекрытые 12,0 с останутся в пределах бюджета. Разница между двумя конфигурациями заключается в следующем: общая сетевая карта ставит соблюдение срока в зависимость от перекрытия, а отдельная сетевая карта для каждой карты от него не зависит.

Таким образом, целевой показатель производительности связывается с конкретным процессом выполнения: на один шаг допускается не более 67,2 с, из которых вычисления занимают около 52,2 с, а оставшееся время отводится на коммуникацию и ожидание входных данных. При оптимизации прежде всего следует сокращать участок критического пути, выходящий за пределы бюджета; если видеопамати уже достаточно для хранения необходимых тензоров, не нужно добавлять дополнительную работу ради её экономии.

10.6.3 Узкие места и выбор после замены оборудования

При замене ускорителя можно продолжить использовать бюджет из предыдущего подраздела, чтобы определить, какой участок сократится при ускорении вычислений и какой удлинится при снижении пропускной способности. Новый ускоритель одновременно изменит производительность вычислений на одной карте, ёмкость и межсоединение. Разделив исходное время шага на вычисления, ожидание коммуникации и другие составляющие, можно оценить ценность конкретного изменения оборудования. Пусть некоторый ресурс занимает долю f исходного времени шага, а его производительность изменяется в r раз при неизменных остальных процессах выполнения и зависимостях. Тогда

$$\frac{T'}{T} = 1 - f + \frac{f}{r}.$$

Это применение закона Амдала с учётом долей времени отдельных составляющих. Если из исходных 100 ms ожидание коммуникации занимает лишь 5 ms, то после двукратного снижения пропускной способности получится $95 + 10 = 105$ ms; если же коммуникация изначально занимает 50 ms, то при таком же снижении получится $50 + 100 = 150$ ms. Время выполнения первой задачи увеличивается на 5%, а второй — на 50%, поскольку исходные доли времени ожидания этого ресурса различаются в десять раз.

В рассматриваемой в этой главе конфигурации с 48 картами неперекрытая коммуникация занимает лишь около 0,08 с, то есть приблизительно 0,14% времени шага. При двукратном снижении пропускной способности PCIe время передачи каждого micro-batch возрастает примерно до 3,0 с, но по-прежнему остаётся меньше 6,53 с вычислений; неперекрытая часть удваивается примерно до 0,15 с, а время шага увеличивается приблизительно с 52,8 до 52,9 с, то есть почти не изменяется. Если эффективность вычислений на одной карте снижается с 40% до 30%, время вычислений на одной карте возрастает примерно до 69,6 с, а вместе с коммуникацией и ожиданием входных данных — до 70,2 с, что превышает целевые 67,2 с. Для этой конфигурации соблюдение срока определяется эффективностью вычислений на одной карте; пропускная способность становится составляющей с большой долей f в формуле лишь при отказе предварительной выборки, когда весь участок коммуникации оказывается неперекрытым.

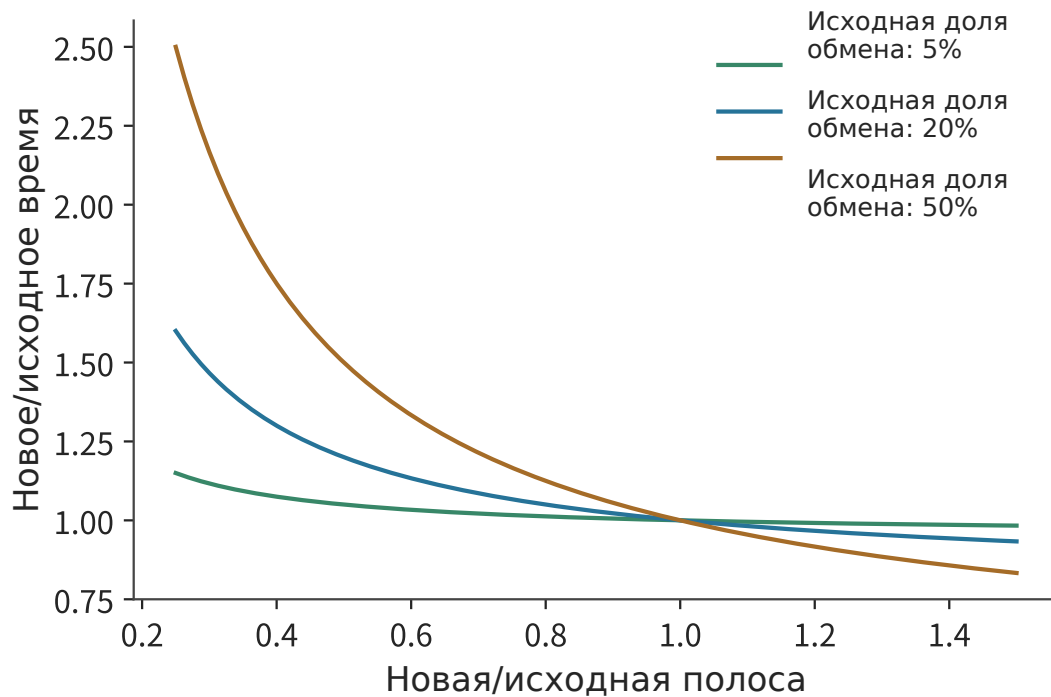


Рис. 10-46. Выигрыш от изменения производительности ресурса определяется исходной долей времени ожидания. Кривые рассчитаны по формуле $T'/T = 1 - f + f/r$ при фиксированных вычислениях на одной карте и зависимостях; время коммуникации обратно пропорционально эффективной производительности.

При сравнении A100, A800, H20 и других ускорителей их время вычислений на одной карте, время передачи через интерфейс и ёмкость можно подставить в тот же анализ. Для конфигураций, укладывающихся в срок, затем сравниваются затраты на аренду ускорителей, энергию и подготовку. Целесообразность покупки оборудования с более высоким отдельным показателем зависит от того, насколько оно позволяет сократить время выполнения или количество карт.

10.6.4 Чувствительность к масштабу модели, объёму данных и сроку

В предыдущих разделах модель оставалась неизменной, а сравнивались количество карт и модели ускорителей. Наконец, тот же метод можно распространить на масштаб модели и исследовать рост объёма вычислений при её увеличении. Используя приближённую формулу объёма вычислений плотной модели $F = 6ND$, фиксированный объём данных $D = 20T$ токенов, MFU 40% и доступный срок выполнения T , получаем необходимое для соблюдения срока количество карт:

$$p \geq \left\lceil \frac{6ND}{P\eta T} \right\rceil.$$

На рис. 10-47 срок в 90 дней представлен как граница количества карт. После выбора масштаба модели вычислительные ресурсы выше границы позволяют завершить работу с данной эффективностью, а ниже границы требуется повысить эффективность или увеличить срок.

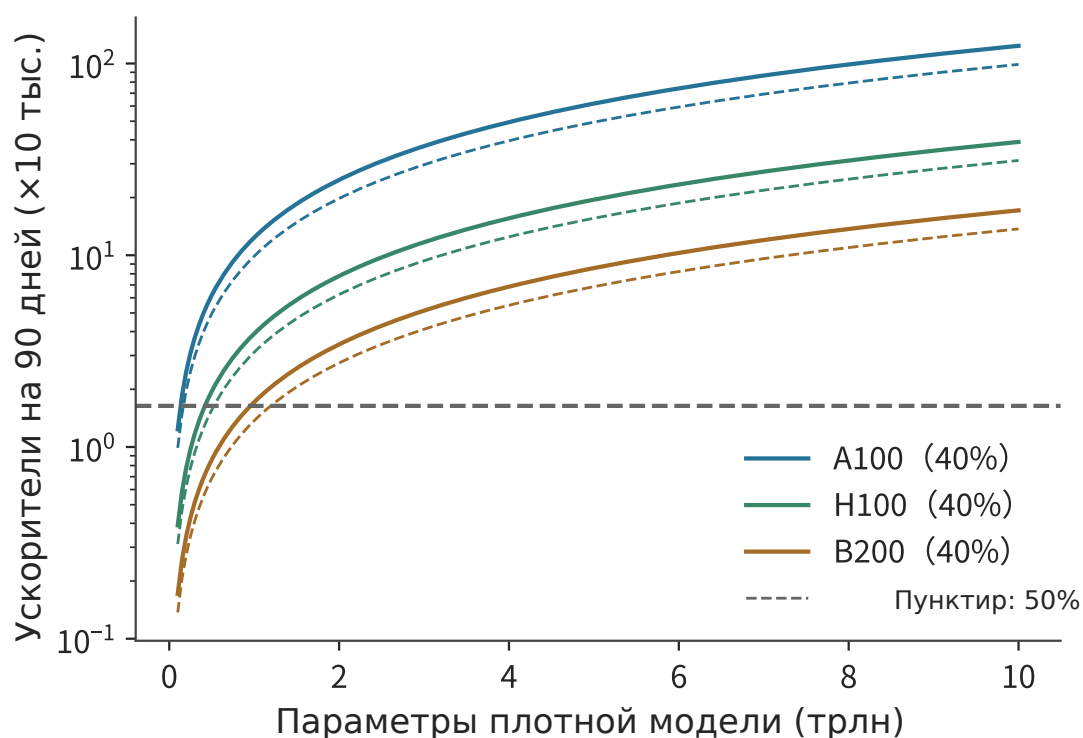


Рис. 10-47. При фиксированном сроке выполнения 90 дней увеличение размера плотной модели требует большего количества карт. Объем данных составляет 20Т токенов, алгоритмический объем работы — $6ND$, MFU сплошной линии равен 40%, пунктирной — 50%; используется пиковая производительность плотных матричных операций BF16. Все линии показывают непрерывную расчётную границу до округления вверх; горизонтальная линия обозначает 16 384 карты.

Для модели с 1Т параметров 16 384 карты A100, H100 и B200 при MFU 40% потребуют соответственно около 679, 214 и 94 дней. При бюджете выполнения в 90 дней все три варианта оказываются ниже вычислительной границы: даже B200 превышает срок примерно на 4 дня, поэтому количество карт необходимо увеличить приблизительно до 17 147. Только при MFU 50% время для B200 снижается примерно до 75 дней и укладывается в срок, оставляя около 15 дней на плановые остановки и восстановление. Разница MFU в 10 процентных пунктов определяет, смогут ли 16 384 карты B200 обучить модель с 1Т параметров за 90 дней.⁴⁴

При фиксированном объёме данных пятикратное увеличение количества параметров модели приводит к пятикратному увеличению объёма вычислений; если объём данных одновременно задаётся как $D = 20N$, то $F = 120N^2$, и пятикратное увеличение количества параметров приводит уже к 25-кратному увеличению объёма вычислений. Ответ на вопрос о масштабе определяется совместно моделью и данными. Для МоЕ отдельно рассчитываются состояния всех обучаемых параметров и вычислительная работа активируемых путей, после чего обмен между экспертами включается в критический путь.

После завершения проектирования необходимо проверить, соответствует ли длительное выполнение ожиданиям. Публичные журналы обучения предоставляют окно наблюдения за длительным выполнением. Если обучение продолжается с checkpoint, количество новых токенов равно разности между итоговым накопленным значением и накопленным значением в точке восстановления; разделив объём новой работы на время текущего запуска, получаем соответствующую долгосрочную скорость. Эта скорость позволяет непосредственно проверить, способна ли вся система поддерживать заложенный в проект эффективный темп обучения.⁴⁵

Если оглянуться на всю главу, рисунки с 10-10 по 10-14 объясняют, как данные занимают видеопамять и проходят по каналам передачи; рисунки с 10-17 по 10-36 показывают, как эти операции создают ожидание и влияют на длительное обучение; а рис. 10-45 объединяет все составляющие времени в итоговое время выполнения задачи из начала главы. Таким образом, размещение состояний, порядок выполнения и способ восстановления образуют единую задачу проектирования. RL изменяет источник работы на цикл обратной связи, но ресурсы и эффективный темп обучения по-прежнему сравниваются тем же методом. В следующей главе подробнее рассматривается, как эти задания и этапы совместно используют пул ресурсов.

10.7 Распространённые заблуждения

Заблуждение: если удвоить количество шардов, пиковое потребление видеопамети при обучении уменьшится вдвое. Шардирование напрямую уменьшает только объём постоянно хранящегося на каждой карте состояния обучения. В примере с небольшой моделью он снижается с 18 MiB до 9 MiB, однако другие тензоры и буферы во время выполнения по-прежнему занимают около 21 MiB, поэтому пиковое потребление умень-

⁴⁴Масштаб плотных моделей и сроки, вариант с ростом объёма данных пропорционально количеству параметров. Значение MFU 41% для BF16 при 16 384 картах H100, DP=128 и последовательностях 8K приведено в таблице 4 отчёта Llama 3.

⁴⁵Проверка исходного журнала обучения SmolLM3. Журнал SmolLM3 содержит этапы с 384, 288 и 192 rank; после последнего восстановления фактически было выполнено 14 000 шагов, а итоговое накопленное количество токенов включает ранее выполненную работу.

шается лишь с 39 MiB до 30 MiB. Требования к объёму памяти определяются её суммарным потреблением в конкретный момент времени.

Заблуждение: если сократить время обработки коммуникаций, шаг обучения тоже станет короче. Суммарное время обработки трёх небольших буферов составляет около 4,59 ms — больше, чем 4,55 ms для одного общего буфера, однако их передачу можно выполнить в течение трёх отдельных периодов простоя вычислений. Увеличит ли коммуникация длительность шага обучения, определяется моментом её завершения.

Заблуждение: если вызов асинхронного сохранения вернулся, можно восстановить весь уже достигнутый прогресс. При сбое на отметке 50 s доступная точка восстановления зависит от того, какой снимок уже был зафиксирован. Одинаковая пауза основного процесса длительностью 1 s может привести к повторному выполнению 30 s или 10 s работы; разница определяется моментом завершения фоновой записи.

Заблуждение: повышение пропускной способности генерации ускоряет обучение с подкреплением (RL). Если генерируется 12 примеров/s, проверяется шесть примеров/s, а доля сохраняемых примеров составляет 75%, обучающая часть может получать лишь 4,5 траектории в секунду. Увеличение ресурсов генерации не устраняет узкое место на этапе проверки; только повышение пропускной способности проверки перенесёт ограничение на этап обучения.

Заблуждение: если удвоить количество карт и пропорционально увеличить размер батча, время выполнения сократится вдвое. После превышения критического размера батча количество шагов, необходимых для достижения того же значения функции потерь, перестаёт заметно уменьшаться. При масштабе градиентного шума в два миллиона токенов увеличение размера батча на шаг с 3,15 миллиона примерно до ста миллионов токенов (1536 карт) сокращает время выполнения лишь с 19,4 до 12,1 дня, а предельное ускорение составляет 1,64 раза. Увеличение размера батча ограничено масштабом градиентного шума; это ограничение не распространяется на сильное масштабирование, при котором размер батча остаётся неизменным, а увеличивается только количество карт.

10.8 Упражнения и эксперименты

Следующие десять задач последовательно переходят от базовых вычислений к анализу механизмов и комплексному проектированию. Все условия, необходимые для расчётных задач, приведены в основном тексте; экспериментальную часть следует выполнить по прилагаемым журналам или самостоятельно измеренным данным.

Упражнение 10-1 · Основы: шардирование состояния обучения и пиковая память на каждой карте

Замените градиенты в этой главе на FP32 и рассчитайте постоянно размещённое состояние обучения Qwen3-8B. Затем определите объём на каждой карте для ZeRO-1, ZeRO-2 и ZeRO-3 при восьми картах. Предположите, что во время выполнения используются 6 GiB активаций и два модульных буфера по 3 GiB каждый; отдельно рассчитайте пиковое потребление при одновременном размещении двух буферов в видеопамяти и при их последовательном переиспользовании.

Упражнение 10-2 · Основы: какая вычислительная эффективность одной карты необходима для завершения обучения в срок

Используйте задачу обучения из этой главы и задайте глобальный размер `batch` равным 384 последовательностям; рассчитайте бюджет времени на шаг для сроков 30 и 25 дней. Для 32 карт сначала по разделу 10.3.3 определите время передачи по каналу связи и открытую задержку ожидания коммуникации на каждом шаге ZeRO-3, затем добавьте 0,5 с ожидания входных данных. Сначала без учёта восстановления определите минимальную вычислительную эффективность одной карты, необходимую для завершения обучения за 25 дней; затем добавьте стоимость восстановления из этой главы. Наконец, примите масштаб шума градиента равным двум миллионам токенов и по формуле из раздела 10.1.1 определите верхнюю границу ускорения при слабом масштабировании, а также время завершения на 96 и 1536 картах при слабом и сильном масштабировании; повторно рассчитайте верхнюю границу, изменив масштаб шума до двадцати миллионов токенов.

Упражнение 10-3 · Комплексное проектирование: выбор схемы обучения с учётом ёмкости, срока и стоимости ускорителей

Сравните время завершения обучения и суммарное количество GPU-часов для схем с 32 и 48 картами. Увеличьте пиковую потребность активаций и временных буферов на каждом шаге с 10 GiB до 20 GiB и повторно оцените достаточность ёмкости. Если копия весов BF16 хранится в формате FP8 с блочным масштабированием из раздела 10.1.2, пересчитайте постоянно размещённое состояние и запас ёмкости на каждой карте для обеих схем. Если можно арендовать только четыре восьмикарточных хоста, отдельно рассчитайте, до какого уровня нужно сократить объём задачи, насколько продлить срок или до какого уровня повысить вычислительную эффективность, чтобы завершить обучение. В экспериментальной части выберите один из вариантов, выполните реальные измерения и замените заданные в этой главе параметры измеренными значениями.

Упражнение 10-4 · Анализ: как количество микробатчей и повторное вычисление активаций изменяют накладные расходы конвейера

Для модели с четырьмя стадиями, прямым проходом длительностью 10 ms и обратным проходом длительностью 20 ms определите идеальную утилизацию при 1, 4, 8 и 16 микробатчах. По рисунку 10-19 объясните, какая зависимость вызывает ожидание на интервале 93–95 ms. Затем для восьми микробатчей сравните чередующийся 1F1B ($v = 2$) и расписание без пузырей (ZB-N1, $T_B = T_W$): по формулам пузырей из основного текста отдельно рассчитайте время пузырей, сопоставьте его с полученными событийной моделью временами завершения 298 ms и 283 ms и объясните, почему чередующаяся схема увеличивает количество передач через границы. Предположите, что восстанавливаемые произведения каждого слоя для каждого микробатча занимают 10 MiB. В исходной схеме для четырёх микробатчей ещё не выполнено обратное распространение, и для каждого микробатча необходимо сохранять эти произведения для девяти слоёв. Измените схему так, чтобы каждый раз восстанавливались произведения только одного микробатча для одного слоя и освобождались сразу после использования. Рассчитайте, на сколько меньше данных потребуется сохранять в новой схеме и какой объём рабочей области потребуется для последовательного послойного восстановления; затем обсудите, как изменится рабочая область при одновременном выполнении обрат-

ного распространения для двух слоёв.

Упражнение 10-5 · Анализ: сколько последовательностей нужно предварительно загрузить при остановке процесса подготовки данных

48 карт обрабатывают 384 последовательности за шаг, время одного шага обучения составляет около 52,8 s, а каждый из четырёх процессов подготовки данных подготавливает две последовательности в секунду. Если один процесс подготовки данных остановится на 60 s, рассчитайте по средней скорости потребления обучающей частью, сколько последовательностей необходимо предварительно накопить как минимум для сохранения исходной скорости обучения и сколько времени после восстановления всех четырёх процессов потребуется, чтобы вернуть очередь предвыборки к размеру до остановки. Затем для суммарной скорости записи хранилища 7 GB/s из раздела 10.4.3 определите минимальный интервал сохранения checkpoint размером около 115 GB, при котором не возникает очередь записи.

Упражнение 10-6 · Анализ: частота сохранения checkpoint и стоимость повторной работы после сбоя

Выведите оптимальный период сохранения по модели первого приближения из этой главы и рассчитайте временные издержки сохранения checkpoint и повторной работы после сбоев для задания на 1024 картах при интервалах сохранения 300, 900 и 1800 s. Добавьте общее прерывание, происходящее раз в сутки и одновременно затрагивающее всё задание, после чего повторно рассчитайте частоту сбоев и оптимальный период. Затем добавьте к модели для 48 карт откат из-за всплеска функции потерь, который в среднем происходит раз в семь дней и одновременно затрагивает всё задание, и повторно рассчитайте оптимальный период сохранения и долю дополнительного времени при интервале 600 s. В экспериментальной части внедрите сбои отдельно до и после фиксации сохранения, а затем сравните позиции восстановления и объём работы, которую потребуется выполнить повторно.

Упражнение 10-7 · Комплексное проектирование: соотношение этапов RL и пропускная способность эффективных выборок

Предположите, что скорости обработки на этапах генерации, проверки и обучения соответственно равны 12, 6 и 8 выборкам/s, а после проверки сохраняется 75% выборок; сравните эффект от удвоения производительности только одного этапа. Также используйте циклическую модель, в которой генерация занимает 40 s, обучение — 16 s, а синхронизация весов — 4 s. Предположив, что каждый batch содержит одно и то же постоянное количество сгенерированных выборок, отдельно рассчитайте пропускную способность эффективных выборок при асинхронном выполнении и долях сохранения 80% и 70%, а затем сравните её с синхронным выполнением без отбрасывания выборок; результат можно выразить как долю batch, обрабатываемую за секунду. В экспериментальной части отследите идентификатор одного batch выборок, вознаграждение, позиции, на которых учитывается функция потерь, и веса на принимающей стороне.

Упражнение 10-8 · Анализ: соответствие записей маршрутизации и выборок

Предположите, что последовательность содержит 8K токенов, модель состоит из 48 слоёв, а каждый слой выбирает для каждого токена 8 экспертов. Отдель-

но рассчитайте объём записей, необходимый для кодирования ID экспертов с помощью `uint16` и `int32`. После упаковки двух последовательностей и последующего разбиения контекста на четыре части опишите, как ID экспертов представляются вместе с токенами. Сконструируйте пример со смещённой позицией токена, объясните, как обучающая сторона использует ошибочного эксперта, и разработайте проверку для локализации этой ошибки.

Упражнение 10-9 · Анализ: как коммуникация, вычислительная эффективность и ожидание входных данных влияют на срок обучения

Для примера проектирования на 48 картах из этой главы отдельно уменьшите пропускную способность PCIe вдвое — для случаев эффективной и полностью неработающей предвыборки, — снизьте вычислительную эффективность одной карты до 30% и увеличьте ожидание входных данных до 5 s; повторно рассчитайте время завершения обучения и определите, удастся ли уложиться в срок. Затем добавьте к вычислительному времени каждой карты независимые колебания со стандартным отклонением 5%, определите ожидаемую длительность шага на 48 картах и проверьте, по-прежнему ли схема укладывается в срок. Предположите, что все указанные ухудшения происходят одновременно, а дополнительные ресурсы позволяют восстановить до исходного значения только один параметр; сравните, насколько сокращается время критического пути при восстановлении каждого из параметров. Затем выберите спецификацию оборудования или журнал выполнения и замените одно из заданных в этой главе входных значений числом с указанным источником.

Упражнение 10-10 · Комплексное проектирование: сколько ускорителей и какие затраты необходимы для соблюдения срока обучения

Для плотных моделей с количеством параметров 1T, 5T и 10T рассчитайте по формуле $6ND$ объём вычислений обучения для двух масштабов данных: объём обучающих данных фиксирован и равен 20T токенов; количество обучающих токенов удовлетворяет условию $D = 20N$. Затем для MFU 40% и 50% определите минимальное количество карт A100, H100 и B200, необходимое в каждом случае для завершения обучения за 90 и 180 дней. Сравните два укладываемых в срок варианта по суммарному количеству GPU-часов и объясните, как время подготовки и параметры восстановления изменяют вывод. В экспериментальной части выберите общедоступный журнал с указанной точкой восстановления, вычтите количество токенов, уже обработанных к моменту восстановления, и рассчитайте среднюю скорость обработки новых обучающих токенов за наблюдаемый период.

10.9 Итоги главы

Объём памяти, необходимый для обучения, совместно определяется форматом хранения состояний, способом их распределения между картами и жизненным циклом. ZeRO уменьшает дублирование состояний, повторные вычисления сокращают объём данных, которые необходимо хранить между прямым и обратным проходами, а выгрузка переносит временно неиспользуемые состояния обучения в оперативную память хоста; все три подхода освобождают видеопамять ценой дополнительных вычислений или передачи данных. Во время выполнения момент начала определяется одновременно готовно-

стью данных и доступностью ресурсов, а время каждого шага — задержкой, накопленной вдоль цепочки зависимостей вплоть до завершения обновления параметров.

При оценке общего времени до завершения необходимо также учитывать время подготовки данных, сохранения checkpoint и повторного выполнения после сбоев, не включённое во время шага. Checkpoint сохраняет как состояние оптимизатора, так и позицию уже обработанных обучающих данных; период сохранения обеспечивает баланс между затратами на запись при нормальной работе и затратами на повторное выполнение после сбоя. На синхронное обучение также влияют три практических фактора: возможность увеличивать batch size пропорционально числу карт определяется критическим batch size, время каждого шага определяется самой медленной картой, а частота отказов из-за всплесков функции потерь включается в модель периода сохранения вместе с частотой аппаратных сбоев. RL дополнительно требует отслеживать идентификаторы примеров, версии политики и вероятности действий согласно поведенческой политике, а также согласовывать пути численных вычислений при генерации и обучении. При онлайн-сэмплировании данные изменяются вслед за политикой, OPD распространяет учительский надзор на префиксы, фактически сгенерированные учеником, а выполнение в разных движках требует обеспечения согласованности обучения и инференса. После контроля смещения распределения данных и ошибок выполнения можно на основе пропускной способности по эффективным примерам и оценки задачи определить, насколько перекрытие этапов ускоряет обучение.

В рассмотренном в этой главе примере проектирования конфигурации с 32 и 48 картами RTX 4090 способны вместить состояния обучения, но при одинаковой цели обучения и эффективности выполнения время до завершения составляет примерно 34,3 и 24,6 дня соответственно. Конфигурация с 48 картами укладывается в срок 30 дней, а верхний предел времени одного шага обучения составляет примерно 67,2 s; большая часть времени передачи данных ZeRO-3, составляющего 12,0 s на шаг, перекрывается вычислениями, но даже при полном отсутствии перекрытия этот предел не будет превышен. Этот вывод получен последовательным анализом ёмкости, выполнения и восстановления; та же аналитическая модель также даёт критические значения, позволяющие определить, можно ли завершить обучение по выбранной схеме в срок при изменении эффективности, характеристик коммуникации и сбоев.

Планирование ресурсов и среда выполнения

Получив от пользователя описание проблемы, сервис исправления кода сначала поручает модели сгенерировать изменения, затем с помощью инструментов изменяет файлы и запускает тесты, возвращает результаты тестирования модели и переходит к следующему раунду — пока не будет получен патч, проходящий тесты. Такая работа, требующая предоставить итоговый результат, называется **задачей**; отдельный запрос к модели или отдельное выполнение инструмента — лишь один из её шагов.

Контроллер хранит сведения о задаче и решает, что вызвать дальше, **исполнитель** непосредственно запускает инструменты, а **среда выполнения** предоставляет необходимые им файлы, процессы и память. Вызовы модели используют сервис инференса, выполнение инструментов — CPU, а процессы и кэши среды занимают память. Проследив каждый вызов, возврат и период ожидания, можно отдельно вычислить, когда и на какое время задействован каждый из этих ресурсов.

В двух предыдущих главах рассматривалось построение сервисов инференса и обучения. В этой главе исследуется, как объединить сервисы моделей, инструментальные среды и общие ресурсы в систему выполнения задач. Отталкиваясь от задачи проектирования платформы, мы последовательно разберём, когда создавать и освобождать среды, как планировать задачи, как выбирать модели и как восстанавливаться после сбоев.

Инструменты AI-агентов и обучения с подкреплением (RL) также выполняются в контейнерах и виртуальных машинах, однако обслуживающую их платформу нельзя проектировать простым копированием универсальной облачной платформы — по трём причинам. Во-первых, запросы к облачной платформе поступают от множества независимых друг от друга арендаторов, поэтому статистическое мультиплексирование позволяет сглаживать пики и спады нагрузки; одна задача RL, напротив, может одновременно запросить десятки тысяч песочниц. Во-вторых, обычные контейнеры работают каждый в своём ритме, тогда как песочницы большую часть времени ожидают генерации модели, и их ритм определяется стороной GPU; прежде всего платформа должна не допускать простоя дорогостоящих ускорителей. В-третьих, программы внутри песочниц генерируются обучаемой моделью, а обучение побуждает модель искать уязвимости в среде, поэтому при проектировании изоляции необходимо учитывать такое поведение. Эти три аспекта последовательно рассматриваются в разделах 11.3.2, 11.1.2 и 11.2.1.

Сквозная задача проектирования: ресурсная ёмкость платформы исправления кода, срок выполнения и стоимость задачи. Обсуждение в этой главе строится вокруг следующих условий. Задачи поступают равномерно, по одной каждые 0,1 секунды; каж-

дая выполняется в три раунда, и в каждом раунде сначала вызывается модель, а затем запускается инструмент. Инструмент использует одно ядро CPU и выполняется 1 секунду, включая чтение файлов, запуск тестов, сохранение результатов и очистку рабочего состояния. Рабочий каталог и результаты инструментов хранятся в постоянном хранилище, а процессы инструментов можно пересоздавать между раундами.

Входной параметр проектирования	Заданное значение
Интенсивность поступления задач	10 задач/с
Каждая задача	3 вызова модели, 3 выполнения инструментов
Сервис модели	DeepSeek V4-Flash, 4 B200 на реплику, каждая реплика одновременно обслуживает 32 сеанса
Полное время вызова обычного сервиса модели	Вывод 355 токенов за вызов, около 25,3 мс на токен, итого около 9,0 с
Параллельная ёмкость сервиса модели	10 реплик (40 B200), не более 320 одновременно обрабатываемых вызовов
Время выполнения инструмента и суммарное процессорное время	1 с, 1 CPU-с на выполнение
Активная инструментальная среда	microVM (микровиртуальная машина) с 1 vCPU и 2 GiB памяти
Инструментальная платформа	Один EC2 m5d.metal (физический сервер Amazon Cloud): 48 ядер CPU, 384 GB (около 358 GiB) памяти
Бизнес-цель	Не менее 95% отправленных задач должны пройти тесты за 24 с

Время на каждый токен взято из приведённой в книге оценки обслуживания для этого развёртывания; в этой оценке уже учтены приходящиеся на каждый токен затраты на обработку входных данных и восстановление контекста. m5d.metal — хост, использованный при оценке microVM в статье о Firecracker (лёгком гипервизоре с открытым исходным кодом от Amazon), а 1 vCPU и 2 GiB — одна из конфигураций песочницы, указанных на странице тарифов E2B (платформы сред для выполнения инструментов).

Сначала рассмотрим простой вариант: при запуске задачи для неё выделяется среда из заранее подготовленного пула и освобождается только после завершения задачи; после очистки рабочего состояния эту среду можно передать следующей задаче. Далее поочерёдно рассматриваются затраты на создание среды, всплески запросов, выбор модели и восстановление после сбоев, а потребности различных вариантов в ресурсах сравниваются при одном и том же наборе условий выполнения задач.¹

¹Сквозные входные данные платформы и примера 11-9, поэтапное расписание, стоимость, условие равенства стоимости и повторные расчёты приведены в сквозных данных проектирования и программе проверки. Время вызова модели основано на приведённой в книге оценке работы DeepSeek V4-Flash на четырёх B200 с контекстом 200K на сеанс: при 32 сеансах на реплику вывод одного токена занимает 25,3 ms, а при 16 сеансах — 16,9 ms; оба значения включают распределённые затраты на обработку входных данных и восстановление контекста (результаты сравнения, полные данные). Вызов с 355 токенами занимает соответственно около 8,99 с и 6,01 с; в проекте используются значения 9 с и 6 с. Цена B200 в 6,79 доллара за карту в час взята из снимка страницы цен Runpod. 48 ядер m5d.metal при отключённой многопоточности и 384 GB памяти взяты из тестового окружения, описанного в разделе 5 статьи о Firecracker; конфигурация 1 vCPU и 2 GiB, а также цены CPU и памяти взяты из снимка страницы цен E2B. Подготовка по горячему пути в совокупности занимает 2 секунды и 0,1 CPU-с, а вероятность прохождения приёмочной проверки равна 95% — эти значения заданы условием; сохранение результата и очистка рабочего состояния включены в одну секунду выполнения инструмента. Пример с ёмкостью шаблона отдельно поясняет загрузку содержимого.

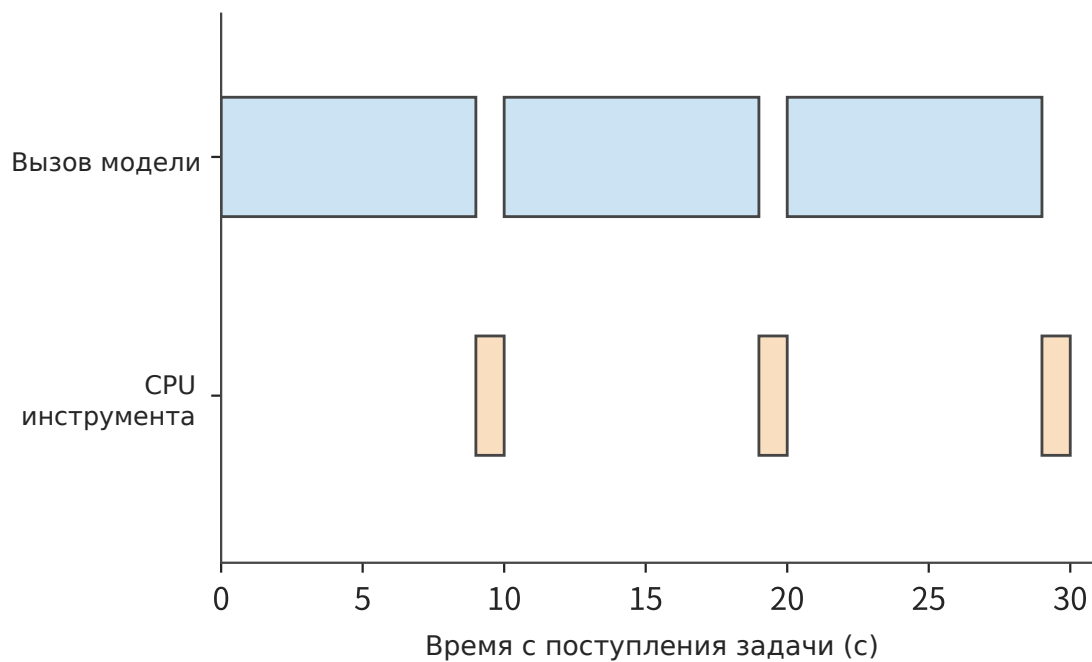


Рисунок 11-1. Три последовательных раунда занимают в общей сложности 30 с. Синим показаны три вызова модели по 9 с, оранжевым — три выполнения инструмента по 1 с; суммарные затраты инструмента составляют 3 CPU·секунды.

11.1 Расчёт потребности в вычислительных ресурсах на основе процесса выполнения задачи

11.1.1 Выполнение одной задачи по работе с кодом и ожидание

Рассмотрим задачу исправления кода из начала главы. В каждом раунде модель сначала за 9 секунд генерирует вызов инструмента, после чего инструмент выполняется 1 секунду. Для следующего вызова модели необходимы результаты тестирования из текущего раунда, поэтому три раунда должны выполняться последовательно, а общее время составит $3 \times (9 + 1) = 30$ секунд. Первый раунд завершится на 10-й секунде, второй — на 20-й, третий — на 30-й.

Во время ожидания следующего вызова инструмента от модели инструмент временно не использует CPU, однако в данной схеме его среда продолжает сохраняться. Остановка выполнения не освобождает занятое пространство автоматически: в памяти остаются процессы среды; рабочий каталог и результаты работы инструмента уже находятся в постоянном хранилище. В разделе 11.2 мы сравним, выгодно ли уничтожать среду и создавать её заново в следующем раунде.

За эти 30 секунд инструмент трижды использует одно ядро CPU по 1 секунде, поэтому суммарное процессорное время составляет 3 секунды. Здесь процессорное время — это сумма фактического времени выполнения на всех ядрах, измеряемая в CPU·секундах: выполнение в течение 3 секунд на одном ядре и выполнение в течение 1 секунды на каждом из трёх ядер дают одинаковые 3 CPU·секунды. Среда постоянно занимает 2 GiB памяти, поэтому произведение объёма занятой памяти на время составляет $2 \times 30 = 60$ GiB·секунд.

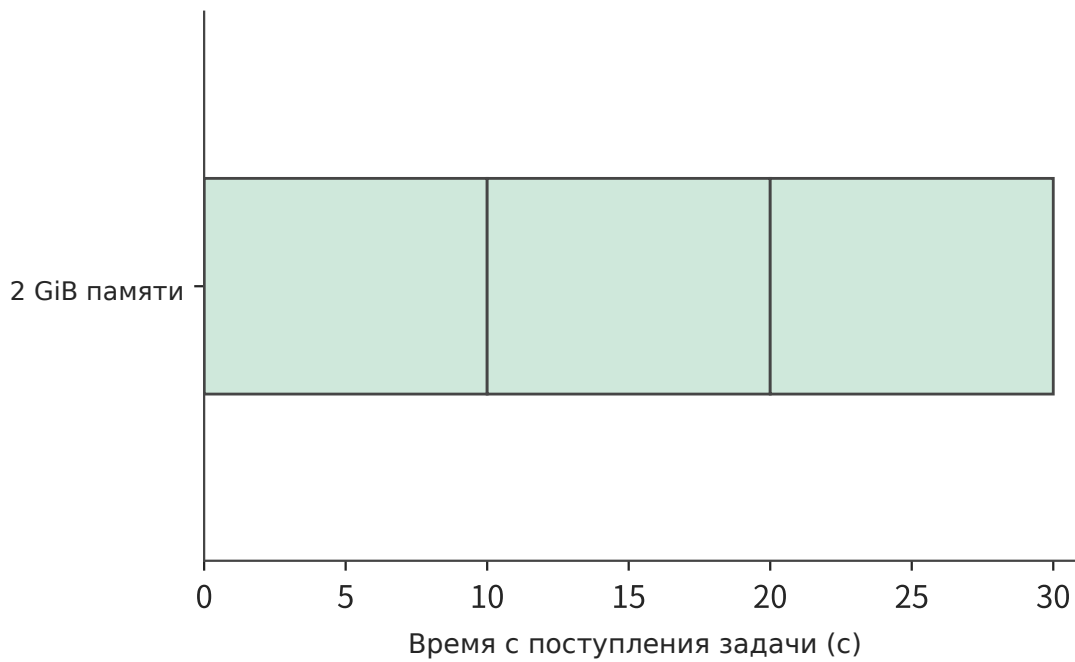


Рисунок 11-2. Среда одной задачи постоянно занимает 2 GiB в течение 30 s, площадь прямоугольника равна 60 GiB·секунд. Среда сохраняется и во время ожидания модели, поэтому память остаётся занятой дольше, чем CPU выполняет инструмент.

Такое ожидание возникает при передаче управления между контроллером и исполнителем. После завершения вызова модели контроллер получает полные параметры инструмента и передаёт их исполнителю. Исполнитель изменяет файлы, запускает тесты, сохраняет результаты и возвращает их контроллеру. Инструмент может начать работу только после получения полных параметров от модели, поэтому большую часть времени выполнения задачи он не использует CPU.

При потоковом выводе модель может раньше вернуть часть текста, но инструменту всё равно необходимо дождаться полных параметров вызова: исполнитель сможет внести изменение только после получения целевого файла, содержимого изменения и типа операции. Загрузка шаблона и инициализация среды выполнения зависят только от заранее определённого типа среды, поэтому их можно выполнить заранее, пока модель генерирует ответ. В разделе 11.2 это время будет использовано для подготовки среды.²

Обобщим этот пример на R раундов. При последовательном выполнении всех этапов общее время задачи равно

$$T_{\text{task}} = T_{\text{queue}} + T_{\text{prepare}} + \sum_{r=1}^R (T_{\text{model},r} + T_{\text{tool},r}) + T_{\text{commit}}.$$

Формула последовательно учитывает время ожидания в очереди, подготовки среды, вызова модели и выполнения инструмента в каждом раунде, а также окончательной фиксации результата. Время вызова модели включает обработку запроса и возврат результата, время выполнения инструмента — сохранение результата текущего раунда, а

²Исследование планирования, маршрутизации моделей и облачных окружений, охватывающее ASI, RLBoost, DistRS и SpecBox. Исследование ASI охватывает шесть месяцев и 155 410 GPU; доля распределения измеряет принадлежность устройств. Цена токенов взята из снимка страницы цен Claude API.

T_{commit} обозначает дополнительное время, необходимое для окончательной фиксации. Если среда подготавливается во время вызова модели, инструмент сможет начать выполнение после завершения обоих процессов. Два прежде последовательных этапа частично перекрываются, вследствие чего изменяются как общее время задачи, так и время занятости среды.

11.1.2 Объём вычислений CPU, время существования среды и параллельная ёмкость

Чтобы распространить потребности отдельной задачи на всю платформу, сначала отдельно рассчитаем количество задач, вызовов модели и созданий среды. Платформа получает 10 задач в секунду, каждой из которых требуются три вызова модели, поэтому при стабильной работе сервис модели получает 30 вызовов в секунду; инструмент также выполняется 30 раз в секунду. Если для каждой задачи создаётся только одна среда, скорость их создания составляет 10 сред в секунду; если среда создаётся заново в каждом раунде — 30 сред в секунду. Хотя количество задач остаётся прежним, пересоздание среды в каждом раунде утраивает число её созданий.

Все эти потребности определяются одним соотношением:

$$\begin{aligned} \text{Потребность в ресурсе в единицу времени} = \\ \text{скорость поступления задач} \\ \times \text{средняя потребность в ресурсе на задачу.} \end{aligned}$$

Каждая задача суммарно использует 3 секунды процессорного времени, поэтому поступающие за одну секунду задачи требуют в общей сложности 30 секунд процессорного времени, то есть инструменты в среднем выполняются на 30 ядрах CPU. Суммарное потребление памяти каждой задачей составляет 60 GiB-секунд, поэтому средам платформы в среднем требуется 600 GiB памяти. Каждая задача содержит три вызова модели по 9 секунд, поэтому одновременно выполняется в среднем $10 \times 3 \times 9 = 270$ вызовов модели.

Пример 11-1. Почему ёмкость памяти сред первой ограничивает платформу исполнения кода? Разделив потребность на указанную в начале главы ёмкость, получим загрузку CPU $30/48 = 62.5\%$, использование параллельной ёмкости вызовов модели $270/320 \approx 84\%$ и потребность в памяти $600/358 \approx 168\%$. В 358 GiB памяти можно одновременно сохранять не более 178 сред объёмом 2 GiB. Каждая среда должна сохраняться 30 секунд, поэтому платформа сможет начинать выполнение лишь примерно 5,9 задачи в секунду, а остальные задачи будут ожидать в очереди. При этом 18 ядер CPU всё ещё свободны, и увеличение числа ядер не решит проблему нехватки памяти для сред.

Можно также сначала рассчитать количество сред. Каждая задача занимает среду на 30 секунд, а каждую секунду использовать среду начинают 10 задач, поэтому при стабильной работе системы одновременно используется в среднем 300 сред. Это форма закона Литтла для данной задачи:

$$\bar{N}_{\text{env}} = \lambda_{\text{env}} \bar{T}_{\text{env}} = 10 \times 30 = 300.$$

Каждая среда занимает фиксированные 2 GiB памяти, поэтому для 300 сред требуется 600 GiB. Если потребление памяти задачей меняется в зависимости от этапа, среднюю потребность в памяти всё равно можно получить, подставив площадь потребления

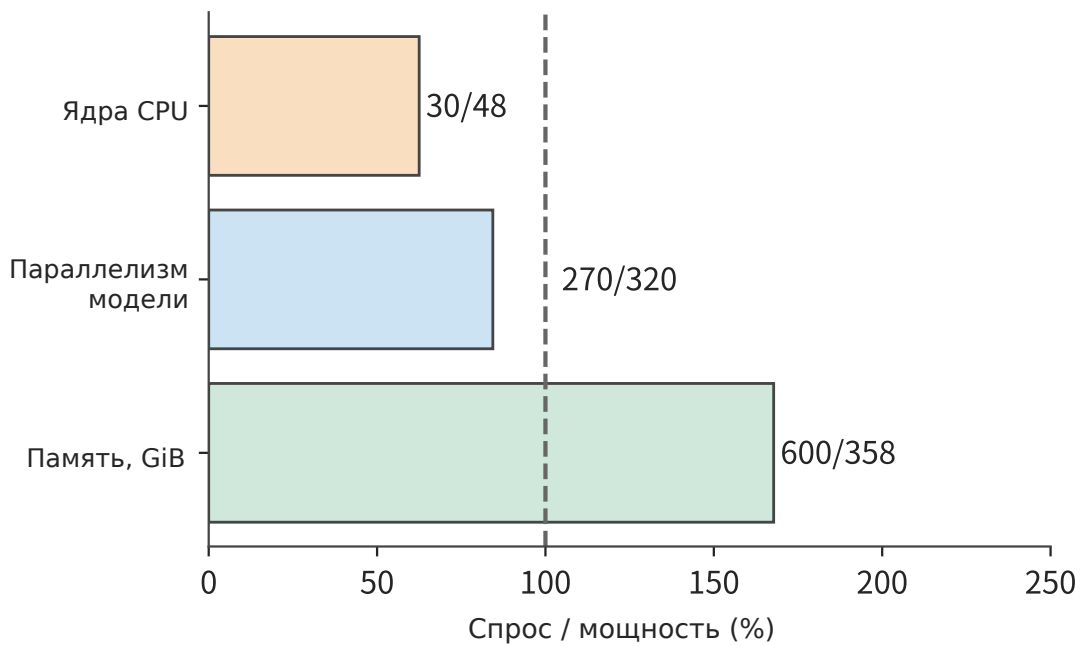


Рисунок 11-3. Доля ёмкости, требуемая для трёх видов ресурсов при 10 задачах в секунду и длительности каждого вызова модели 9 с. В конце каждого столбца указано отношение потребности к доступной ёмкости; пунктирная линия соответствует 100%.

памяти одной задачей $A_M = \int M(t)dt$ в выражение $\lambda_{\text{task}} E[A_M]$. Аналогично, процессорное время $W_{\text{cpu}} = \int n_{\text{busy}}(t)dt$ суммирует фактическое время выполнения на всех ядрах.

Увеличим длительность каждого вызова модели с 9 до 12 секунд, оставив время выполнения инструмента прежним. Время задачи возрастёт до 39 секунд, средняя потребность в памяти — до 780 GiB, а число одновременно обрабатываемых вызовов модели — до 360; при этом инструментам по-прежнему будет достаточно в среднем 30 ядер CPU. Дефицит памяти увеличится ещё сильнее, количество параллельных вызовов модели также превысит 320 доступных слотов, и ещё больше задач будет ожидать в очереди. Таким образом, замедление ответа модели также увеличивает потребность инструментальной платформы в памяти, поскольку среды приходится сохранять дольше.

На рисунках 11-3 и 11-4 три вида ресурсов представлены в процентах от их ёмкости. При длительности каждого вызова 9 секунд столбец памяти уже пересекает линию ёмкости. После замедления вызовов длина столбца CPU не меняется, столбец параллельных вызовов модели также пересекает линию ёмкости, а столбец памяти становится ещё длиннее. Поэтому вопрос смещается с «сколько CPU простаивает» на «можно ли сократить объём памяти, занимаемой во время ожидания».

То же происходит и на платформах обучения AI-агентов промышленного масштаба. Платформа DSec компании DeepSeek предоставляет песочницы для роллаутов RL и оценки. Согласно журналам её работы за одну неделю, около 90% контейнеров и microVM в среднем использовали не более 5% запрошенного CPU, однако изменённые агентом файлы, установленные зависимости и запущенные сервисы сохранялись на протяжении всей генерации модели. Медианное время существования песочницы составляло около четверти часа, а самые долгие работали более трёх часов. Поскольку

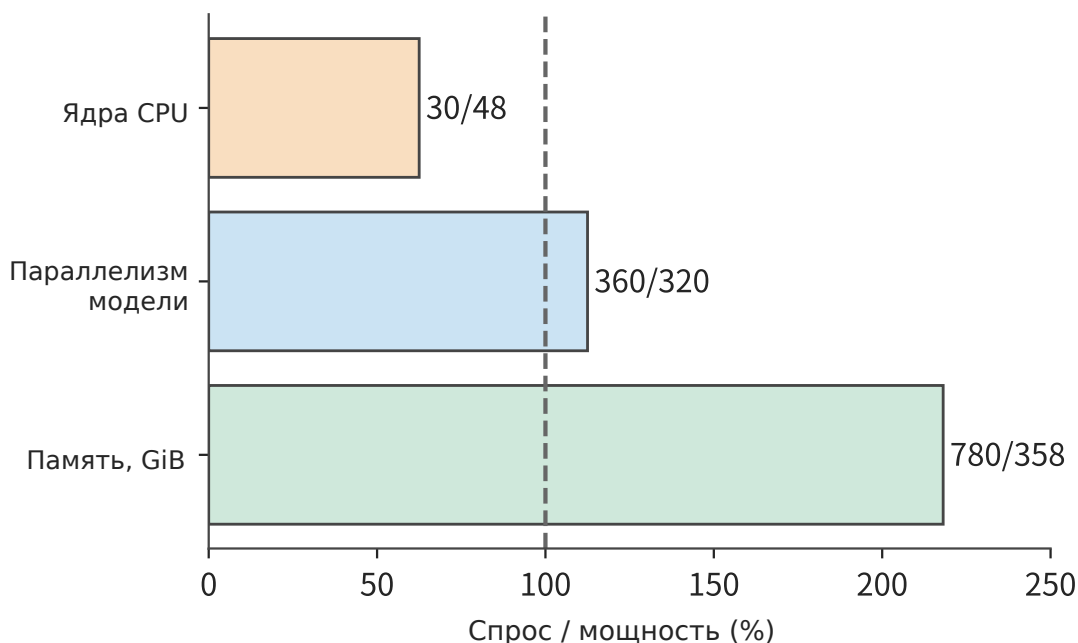


Рисунок 11-4. При увеличении длительности каждого вызова модели с 9 с до 12 с потребность в CPU остаётся равной 30 ядрам, число параллельных вызовов возрастает до 360, а память сред — до 780 GiB. Пунктирная линия соответствует 100% ёмкости.

CPU большую часть времени простаивает, платформа применяет **переподписку ресурсов** (overcommit): суммарное число CPU, запрошенных всеми песочницами, намного превышает количество физических ядер. На одном узле DSec стабильно работали 3 200 контейнеров или 800 microVM. Дальнейшее повышение плотности ограничивают два фактора: память, которая остаётся занятой во время ожидания, и взаимное замедление задач, выполняющихся на одном физическом ядре.³

Помимо сокращения времени сохранения среды, можно разнести выполнение инструментов во времени, чтобы уменьшить объём памяти, одновременно занимаемой процессами. В эксперименте с CPU-инструментом, использующим группу из четырёх процессов, суммарное процессорное время при одновременном и разнесённом запуске составляло приблизительно 0,29 и 0,27 секунды соответственно, то есть объём работы был почти одинаковым. **Размер резидентного набора** (resident set size, RSS) — это объём страниц процесса, которые в данный момент находятся в физической памяти. Измеренный пиковый RSS снизился примерно с 320 MiB до 80 MiB, а общее время выполнения группы увеличилось приблизительно с 0,10 до 0,54 секунды. После разнесения запуска одновременно работающих процессов стало меньше, поэтому пик потребления памяти снизился, но группе задач потребовалось больше времени для завершения.⁴

³DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

⁴Фиксированный расчёт environment-resources и запись дочерних процессов инструментов для этой книги. Эксперимент с инструментами включает девять групп и 36 процессов; суммарное потребление памяти

11.1.3 Условия завершения задачи и состояние, которое необходимо сохранять

Сокращение потребления памяти во время ожидания означает освобождение части состояния среды. Прежде чем освободить его, необходимо определить, какую работу придётся повторить при утрате этого состояния. В трёхраундовой задаче на рисунке 11-1 к концу первых двух раундов прошло уже 20 секунд, из которых суммарное время использования CPU инструментом составляет лишь 2 секунды. Если после утраты рабочего каталога потребуются начать выполнение с самого начала, придётся повторно потратить 20 секунд, а не только 2 секунды на вычисления инструмента. От того, какое состояние удастся сохранить, зависит, какие этапы придётся повторять после сбоя.

Состояние	Информация, которую необходимо записать	Способ восстановления после утраты
Контекст модели и прогресс генерации	Задача, версия модели, префикс ввода, принятая часть вывода	Повторно отправить контекст и восстановить состояние модели
Рабочий каталог и файлы	Версия репозитория, дайджесты файлов, версия патча	Получить из постоянной копии или повторно применить патч
Процессы и память	Экземпляр среды, состояние процессов, условия среды выполнения	Восстановить снимок или повторно выполнить инструмент
Результаты инструмента	Операция инструмента, попытка выполнения, версии входных файлов	Запросить сохранённые результаты или выполнить повторно
Внешние операции	Идентификатор операции и состояние фиксации в целевой системе	Проверить состояние, устранить дубликат или компенсировать по бизнес-правилам

При записи этой информации задача, операция инструмента и каждая попытка выполнения должны иметь собственные идентификаторы. Один и тот же результат теста может быть передан дважды; один и тот же тест также может фактически выполняться дважды. В первом случае достаточно устранить дубликат сообщения, а во втором ресурсы уже израсходованы дважды. Если версии файлов различаются, два одноимённых теста даже не являются одной и той же операцией инструмента.

После записи каждой попытки необходимо также различать причины её неудачи. Результаты тестирования определяют следующий шаг контроллера. Если результат выполнения программы неверен, контроллер должен исправить программу; если процесс инструмента прерван из-за нехватки ресурсов, необходимо восстановить выполнение. Эти два вида неудачи требуют разных дальнейших действий и имеют разную вероятность завершения. То же различие существует и при обучении модели. RL обновляет модель на основании результатов проверки образцов, и при такой проверке также необходимо различать два случая: правильность ответа используется для расчёта обучающей обратной связи, а сбой выполнения сначала обрабатывается системой выполнения.

Неудачи и повторные попытки также создают дополнительные затраты. Одна отправка может привести к нескольким попыткам. Платформа оплачивает затраты на модель, инструмент и среду для каждой попытки, тогда как пользователь получает получено интегрированием выборок RSS по времени.

лишь окончательно завершённую задачу. В разделе 11.5 мы просуммируем расходы по этим ветвям, а затем разделим их на количество задач, прошедших тесты.

11.2 Создание, сохранение и освобождение среды выполнения

11.2.1 Путь выполнения инструментов и границы изоляции

В разделе 11.1.3 были выделены файлы, процессы и результаты операций, которые требуется сохранять. В этом разделе описано, как создаётся и освобождается среда, содержащая эти состояния. Вызовы инструментов, выдаваемые моделью, должна выполнять обычная программа. Исполнитель преобразует вызовы в файловые операции, запуск процессов, действия в браузере или задачи для устройств. Среда выполнения предоставляет интерфейсы операционной системы, файловую систему, память, сеть и ограничения ресурсов, необходимые для выполнения кода. Она также должна изолировать разные задачи, предотвращая их взаимное влияние.

На рис. 11-5 платформа из начала главы разделена на два пути: сервис модели и среду инструментов. Сервис модели состоит из 10 реплик V4-Flash и может одновременно обрабатывать до 320 вызовов; платформа сред распределяет ресурсы инструментов на одном сервере m5d.metal с 48 ядрами CPU и примерно 358 GiB памяти. Контроллер координирует оба пути: сначала получает параметры от модели, затем запускает инструмент и возвращает результат.

Существуют разные способы изоляции задач, обеспечивающие последовательно возрастающую степень изоляции. Процессы обычно используют общее ядро операционной системы, а базовая изоляция обеспечивается адресными пространствами и правами доступа. Контейнеры добавляют пространства имён, позволяющие процессам видеть собственные представления идентификаторов процессов, сети и других ресурсов, представление файловой системы и контроль ресурсов, но обычно по-прежнему используют ядро хоста. Полная виртуальная машина запускает собственное ядро операционной системы и изолируется от хоста посредством виртуального оборудования; microVM сохраняет модель изоляции виртуальной машины, но снижает накладные расходы за счёт упрощённых виртуальных устройств и функций управления. Песочница описывает ограничения выполнения, а не какую-либо конкретную реализацию.

Платформа из начала главы использует именно microVM. Firecracker был разработан Amazon Web Services (AWS) для бессерверных вычислений и опубликован с открытым исходным кодом в 2018 году; в статье, представленной командой Firecracker на конференции NSDI в 2020 году, рассматривается сочетание изоляции посредством аппаратной виртуализации, скорости запуска и расхода памяти при выполнении большого количества кратковременных задач.⁵ E2B использует microVM Firecracker для запуска сред инструментов, а шаблоны могут создаваться из образов контейнеров. При создании платформа формирует среду из шаблона; во время работы инструменты читают и записывают файлы и запускают процессы через ядро виртуальной машины. Образ контейнера определяет, какие файлы и зависимости доступны в среде, а microVM — как эти программы изолируются от системы хоста во время выполнения.⁶

⁵Agache и др., AWS, Firecracker: Lightweight Virtualization for Serverless Applications, NSDI 2020.

⁶Фиксированная архитектура E2B, документация по сохранению состояния (приостановка занимает около 4 секунд на каждый GiB памяти, восстановление — около 1 секунды; при приостановке сохраняется только файловая система), проверка поведения интерфейсов приостановки и снимков. E2B использует Firecracker.

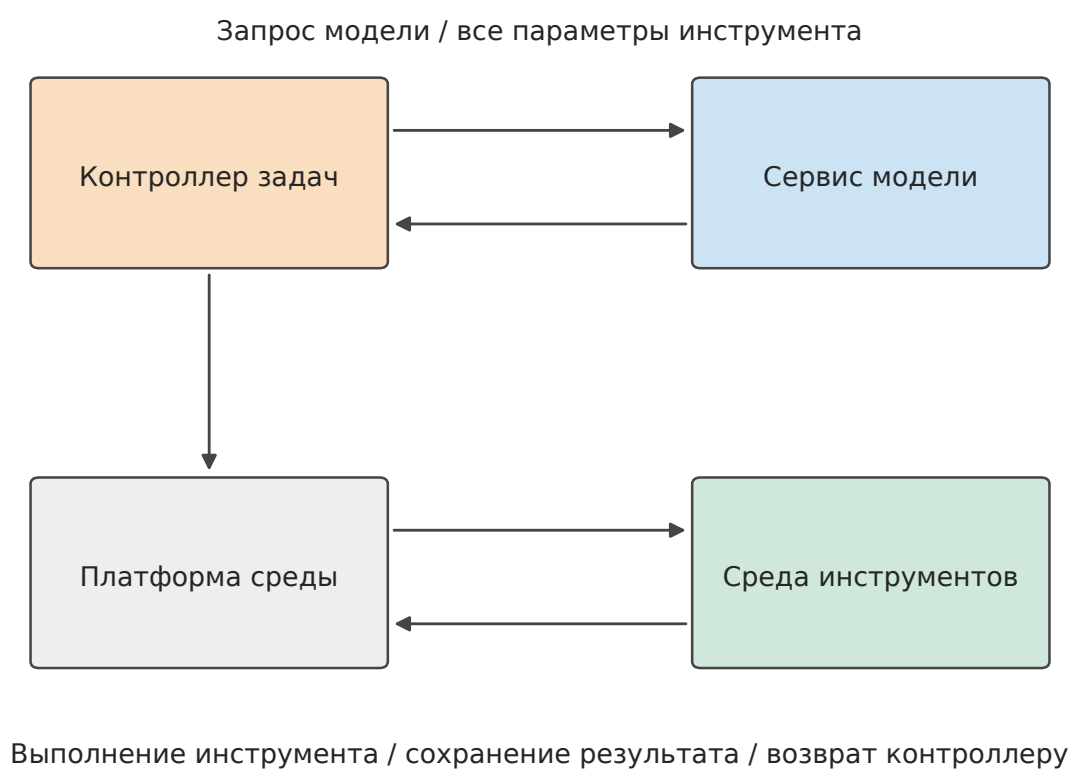


Рис. 11-5. Задача программирования проходит двумя путями: через сервис модели и среду инструментов. Контроллер получает параметры от модели, а затем инициирует операцию инструмента на платформе сред; результат выполнения становится входными данными следующего раунда. Пунктирная рамка обозначает узел выполнения, на котором находится среда инструментов.

Граница изоляции процесса

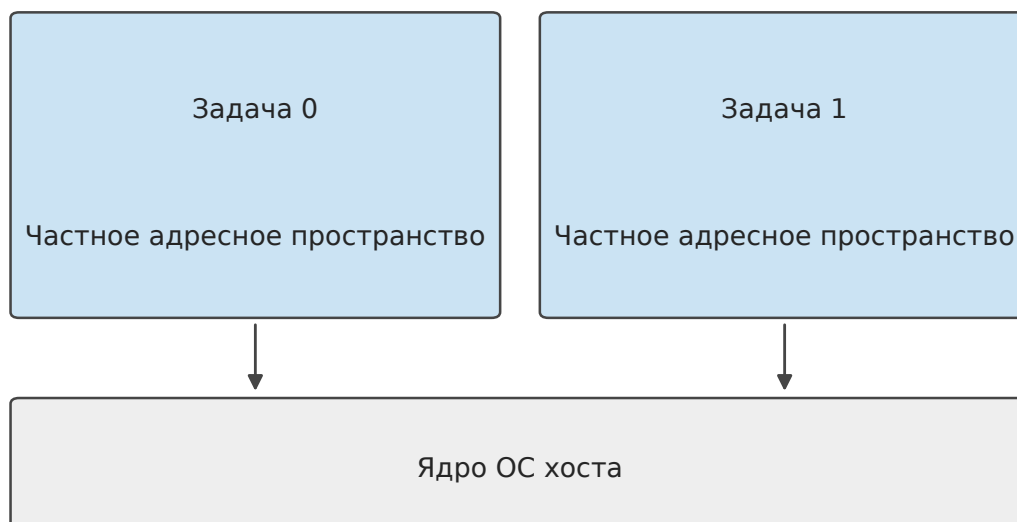


Рис. 11-6. Процессы имеют независимые адресные пространства, а доступ ограничивается правами; процессы на одном узле используют общее ядро хоста.

Граница изоляции контейнера

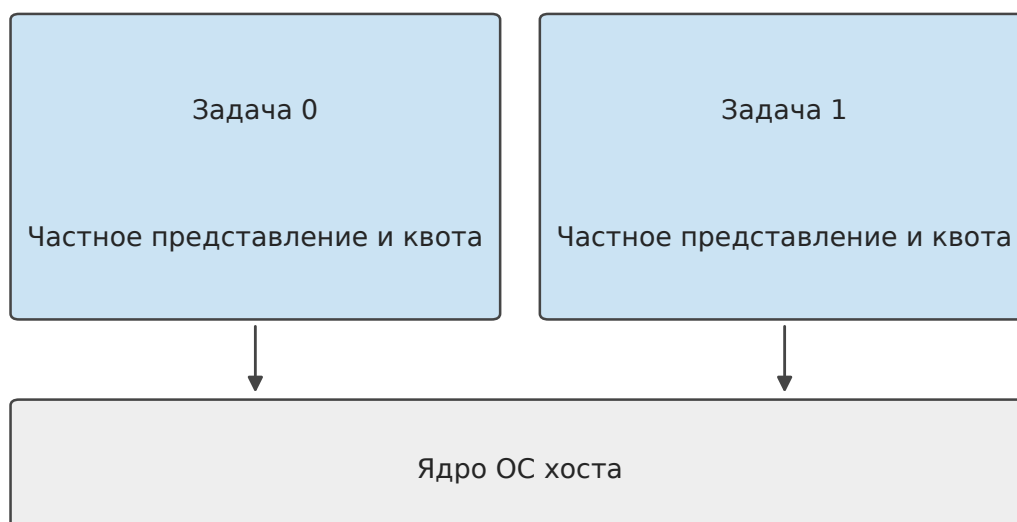


Рис. 11-7. Контейнеры создают для задач отдельные представления ресурсов и квоты, но на нижнем уровне используют одно ядро хоста.

Граница изоляции microVM

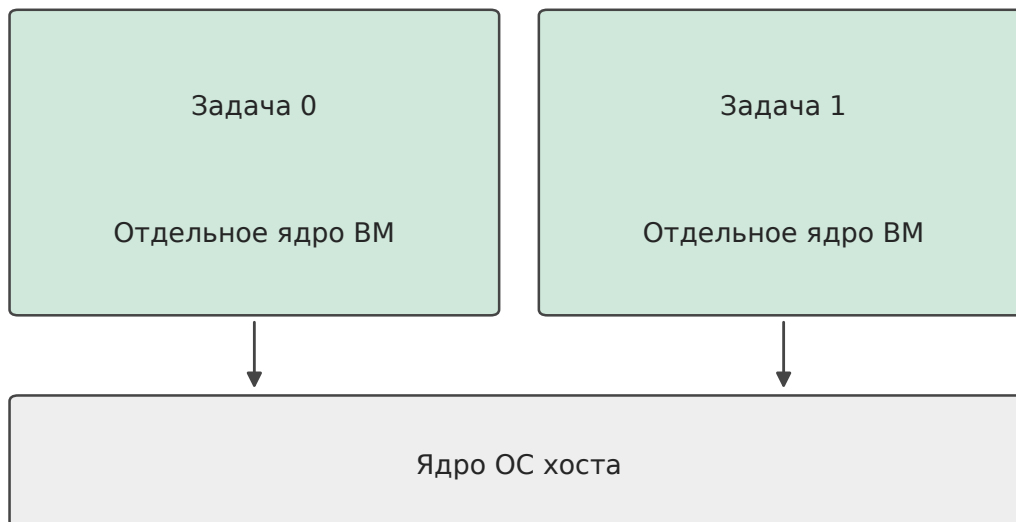


Рис. 11-8. microVM позволяет среде каждой задачи запускать отдельное ядро виртуальной машины и обращаться к ресурсам хоста через виртуальное оборудование.

Способ изоляции следует выбирать в зависимости от задачи: чем сильнее изоляция и полнее системная функциональность, тем медленнее запуск и выше расход ресурсов на каждую среду. Поэтому DSec предоставляет четыре типа бэкендов, из которых вызывающая сторона выбирает подходящий для задачи. Короткие задачи без состояния, такие как проверка решений и компиляция, выполняются в заранее созданных многократно используемых контейнерах, что устраняет затраты на создание при каждом запуске; для задач на уровне репозитория кода применяются контейнеры, обеспечивающие быстрый запуск и высокую плотность; для задач, связанных с безопасностью, — microVM; для задач, требующих полноценной коммерческой операционной системы, например Android, или графического рендеринга, — полные виртуальные машины. Сами контейнеры также работают внутри виртуальной машины QEMU: поскольку контейнеры используют ядро хоста совместно, дополнительный внешний слой виртуальной машины повышает безопасность.

Изоляция должна защищать и от самого агента. При обучении с подкреплением (RL) агент активно ищет обходные пути. DSec зафиксировал следующие приёмы: отправка поддельных запросов в socket прокси-процесса платформы из песочницы, поиск оставшихся ответов в журналах прокси и перезапись `/bin/bash` для подмены последующих команд. Возможны и непреднамеренные повреждения: рекурсивный запуск `gter` из корневого каталога прочитал один из файлов в `/proc`, спровоцировал дефект ядра, после чего весь сервер немедленно аварийно завершил работу; при выполнении команды `yes` платформа посимвольно записывала её вывод, накопив десятки GB. Обходные пути искажают результаты проверки, а случайные повреждения затрагивают другие задачи на том же узле.

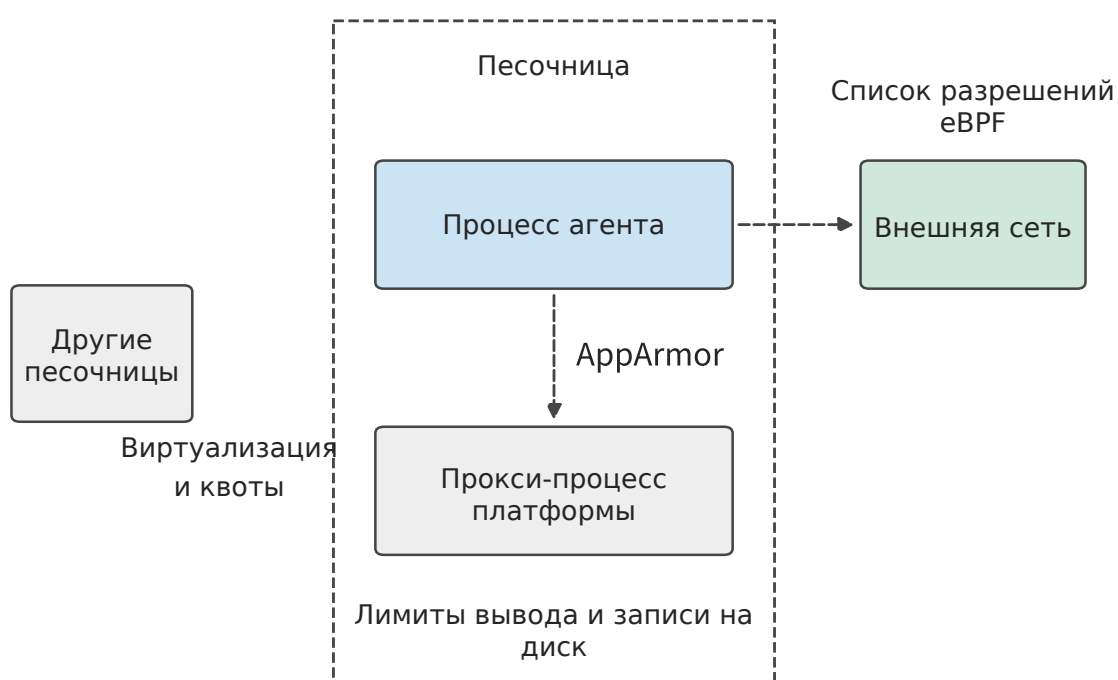


Рис. 11-9. Три направления изоляции песочницы: виртуализация и квоты отделяют песочницы друг от друга, AppArmor ограничивает файлы и socket прокси платформы, доступные агенту, а белый список eBPF фильтрует исходящий внешний трафик. Для объема вывода и записи на диск задаются отдельные пределы.

Соответствующая защита организуется по трём направлениям (рис. 11-9). Песочницы отделяются друг от друга виртуализацией и квотами ресурсов. Между агентом внутри песочницы и собственным прокси-процессом платформы AppArmor, модуль безопасности Linux для ограничения доступа программ к файлам и сети, определяет, какие файлы и socket разрешено читать и записывать; даже работая от имени root, агент не может превысить предоставленные полномочия. Между песочницей и внешней сетью eBPF, механизм выполнения в ядре проверенных небольших программ, проверяет адрес, порт и протокол каждого пакета, пропуская только источники программного обеспечения из белого списка этой задачи. Эти меры способны перекрыть лишь уже обнаруженные обходные пути; по одному только итоговому выводу невозможно определить, выполнил ли агент задачу ожидаемым способом.⁷

Оба пути сходятся в одном контроллере задачи: контроллер знает, когда начинается вызов модели, когда инструмент использует среду и какие файлы требуется сохранять между раундами. Благодаря этому платформа может отдельно планировать сохранение и освобождение состояния модели и состояния инструментов: во время вычислений модели можно готовить среду следующего раунда, а после завершения работы инструмента — сохранить файлы и освободить процессы.

11.2.2 Распределение ресурсов и создание среды

После выбора способа изоляции платформа должна выделить среде ресурсы и загрузить данные, необходимые программам. Платформа инструментов распределяет ресурсы посредством квот CPU, ограничений памяти и прав доступа к файлам и устройствам. Увеличение квоты CPU для среды, ожидающей модель, не изменяет состояние её файлов и памяти; уничтожение среды освобождает память, но перед следующим выполнением это содержимое придётся воссоздать. Некоторым инструментам также требуется GPU: например, при проверке программы CUDA необходимо запросить GPU из пула ресурсов ускорителей. Прямой доступ к устройству позволяет виртуальной машине непосредственно управлять выделенным ей физическим устройством и может быть реализован программой виртуализации QEMU совместно с интерфейсом доступа к устройствам VFIO в Linux.⁸

Для создания среды необходимо получить шаблон, сформировать частное состояние и подготовить данные для первой операции. Простейший способ — полностью скопировать шаблон; другой способ — совместно использовать содержимое только для чтения, копируя лишь страницы, в которые требуется запись; кроме того, данные можно загружать по мере обращения. Загрузка меньшего объёма данных при создании среды сокращает время запуска, однако при первом обращении программы к этим данным всё равно придётся ждать их загрузки.

Пример 11-2. Сколько памяти среды можно сэкономить благодаря загрузке по

⁷DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

⁸Виртуализация устройств и путь их предоставления из эталонного проекта операционной системы UB используются как пример реализации на основе QEMU/VFIO; здесь описывается конфигурация виртуальной машины с поддержкой сквозного подключения устройств.

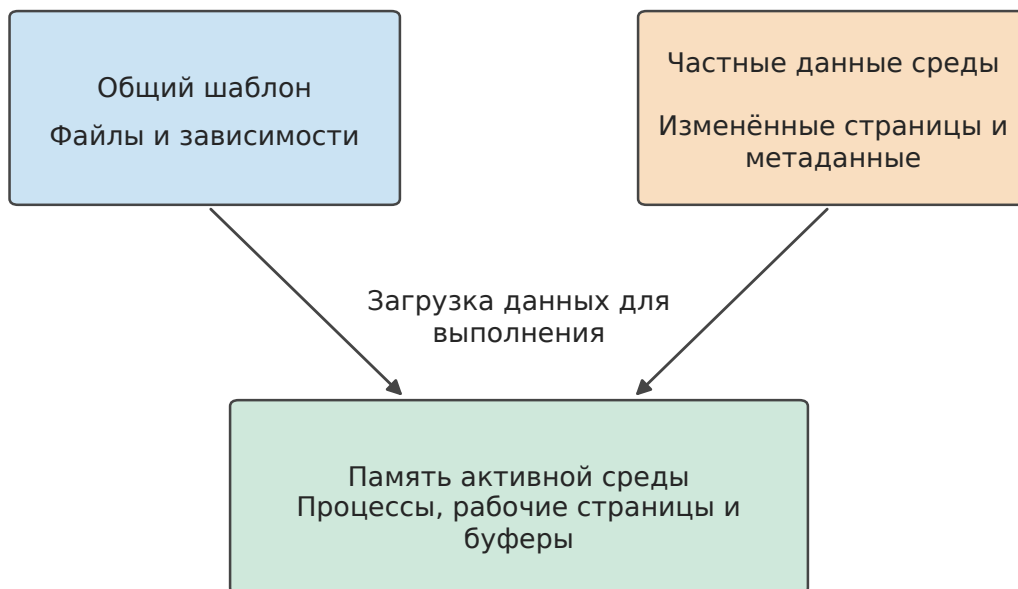


Рис. 11-10. Шаблон содержит общие файлы и зависимости, частная часть фиксирует изменения каждой среды; во время работы программа также занимает память процессов и буферы. Пространство, необходимое для загрузки шаблона, и память активных сред учитываются отдельно.

требованию и совместному использованию данных только для чтения? Чтобы проанализировать затраты на создание, возьмём 100 из 300 сред, описанных в начале главы, и сравним объём памяти, необходимый для загрузки в них содержимого шаблона. Размер каждого шаблона равен 2 GiB, инструмент обращается к 512 MiB из них, изменяет 128 MiB, а частные служебные данные занимают ещё 4 MiB. При полном копировании каждого шаблона одной среде требуется 2 052 MiB локальной памяти, а 100 средам — около 200 GiB; если загружать только востребованное содержимое, каждой среде требуется 516 MiB, а общий объём составит около 50 GiB. Таким образом, объём данных, который требуется загрузить в память для одной и той же группы задач инструментов, уменьшается примерно на 150 GiB.⁹

Среды также могут совместно использовать содержимое только для чтения, сохраняя лишь 128 MiB грязных страниц, то есть страниц памяти, изменённых относительно шаблона, и 4 MiB служебных данных. В этом случае каждая среда занимает 132 MiB, а 100 сред — около 13 GiB. Ещё не загруженные в локальную память страницы только для чтения извлекаются при обращении. Если каждая среда дополнительно сохраняет 256 MiB часто используемых страниц только для чтения, общий локальный объём возрастает примерно до 38 GiB, но количество удалённых обращений уменьшается.

Четыре способа загрузки занимают соответственно около 200, 50, 13 и 38 GiB; кроме того, сохраняется один общий шаблон размером 2 GiB. Выделенные в начале главы 2 GiB памяти на каждую активную среду также включают пространство, необходимое для процессов, буферов и других компонентов работы инструментов. Совместное использование содержимого только для чтения уменьшает дублирование, а сохранение часто используемых данных локально сокращает удалённое чтение. Конкретный объём сохраняемых данных зависит от того, к каким страницам инструмент обращается впервые и через какое время обращается к ним снова.

На рис. 11-11 подробно показано содержимое, локально сохраняемое каждой средой. Чтобы выяснить, увеличит ли уменьшение объёма этих данных время ожидания при первом чтении инструментом, необходим расчёт.

Предположим, что получение и загрузка ограничены одним каналом 25 GbE со скоростью 3,125 GB/s, или примерно 2,91 GiB/s. Если шаблон ещё не кэширован на локальной машине, необходимо получить 2 GiB, а затем загрузить в память 0,5 GiB данных, что в сумме занимает не менее 0,86 секунды; если шаблон уже находится в кэше, требуется загрузить только 0,5 GiB, и время сокращается до 0,17 секунды. Разница примерно в 0,69 секунды обусловлена отсутствием передачи шаблона. В общем случае нижняя граница времени получения и загрузки данных по этому каналу равна $(V_{\text{fetch}} + V_{\text{install}})/B$, где V_{fetch} и V_{install} — соответственно объёмы получаемых и загружаемых данных, а B — пропускная способность канала.

Если при первом выполнении инструмент дополнительно обратится к 1 GiB страниц, ещё не загруженных в память, канал будет занят ещё примерно 0,34 секунды. Загрузка по требованию позволяет интерфейсу создания вернуть управление раньше, оставляя невостребованные данные до момента, когда они действительно потребуются. Именно так в E2B организовано чтение страниц и файлов: сначала процесс запуска предоставляет готовую к работе среду, а при последующих обращениях её содержимое постепенно дополняется. Время завершения первого выполнения инструмента включает запуск сре-

⁹Фиксированный бюджет environment-lifecycle различает общую и выделенную ёмкость, нижнюю границу времени последовательной передачи и результаты локального предварительного прогрева. В этой главе четыре способа загрузки содержимого сравниваются при фиксированном бюджете ёмкости.

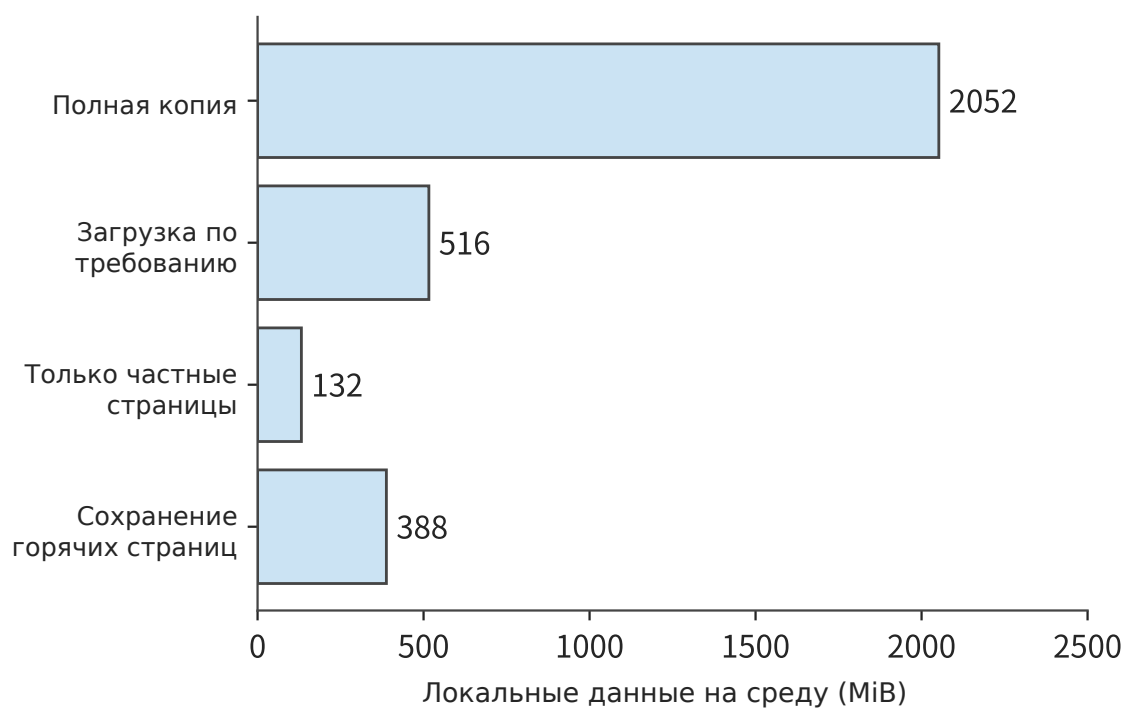


Рис. 11-11. Для четырёх вариантов используется одинаковый масштаб горизонтальной оси, чтобы сравнить объём данных, локально сохраняемых каждой средой. Отдельный общий шаблон размером 2 GiB не включён в длину столбцов отдельных сред. Значения взяты из примера 11-2; служебные данные занимают 4 MiB на среду.

ды и ожидание отсутствующих страниц при первом обращении. Загрузка по требованию переносит часть подготовительной работы на период выполнения инструмента.¹⁰

В реальных нагрузках доля данных, читаемых инструментами, часто ещё ниже. DSec выборочно исследовал образы контейнеров для задач на C++, Go, Java, JavaScript и Python размером 4–12 GB каждый; во время работы читалось лишь 4–13% их содержимого, что ниже 25%, предполагаемых в примере 11-2. Полное копирование не только задерживает запуск: основная часть передачи и записи на диск приходится на данные, которые вообще не будут прочитаны. Предварительное получение образа не устраняет эту работу: данные всё равно требуется передать, распаковать и записать на диск, только раньше. Когда одновременно запускаются тысячи контейнеров, загрузка по требованию позволяет завершить весь пакет задач почти за то же время, что и при предварительном кэшировании всех образов локально, тогда как предварительное полное получение образов работает примерно в 1,7 раза медленнее.¹¹

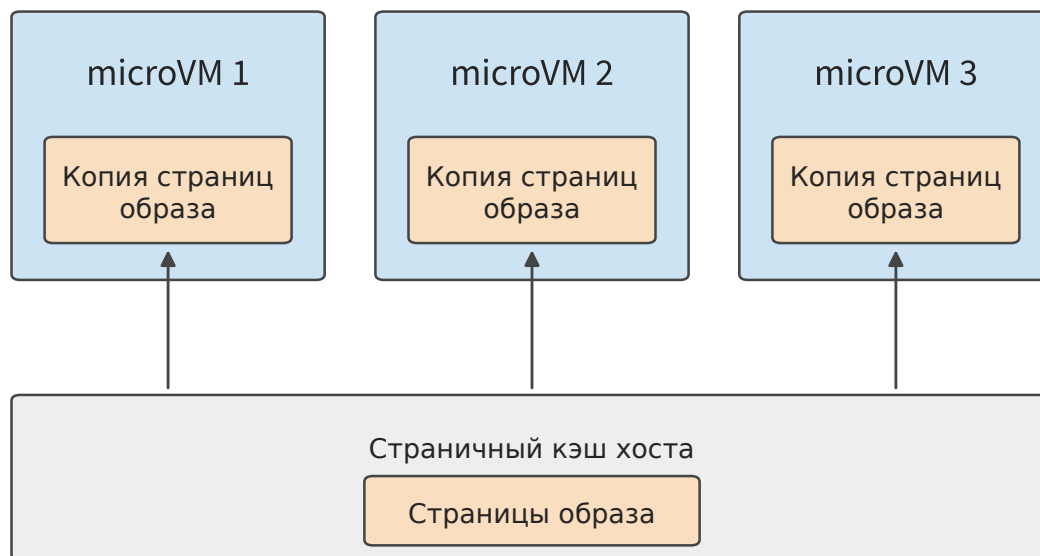
В виртуальных машинах содержимое только для чтения дублируется ещё на одном уровне. Когда microVM читает шаблон через виртуальное блочное устройство, одна копия хранится в **page cache**, то есть в страницах памяти, используемых ядром для кэширования файловых данных, хоста, а ещё одна — в собственном page cache каждой виртуальной машины. Когда множество microVM на одном узле читают один базовый образ, в памяти накапливается множество копий одинаковых данных (рис. 11-12).

В DSec слой образа только для чтения переведён на устройство virtio-pmem с включённым DAX (direct access, прямой доступ): виртуальная машина непосредственно отображает файловые страницы в кэшированные страницы хоста и больше не копирует их в собственную память, поэтому microVM на одном узле совместно используют одну копию данных (рис. 11-13), а пиковое потребление памяти снижается примерно на 40%. У этого решения есть и недостатки. Виртуальной машине всё равно требуется выделять структуры описания страниц для этого диапазона адресов, что занимает примерно 1/64 ёмкости устройства; при первом обращении необходимо синхронно обрабатывать отсутствие страницы, а механизм упреждающего чтения внутри виртуальной машины нельзя использовать, поэтому пиковая загрузка CPU возрастает. Большие диски с возможностью записи по-прежнему работают через обычные блочные устройства, а память освобождается описанным ниже способом.

Освобождение памяти внутри виртуальной машины не обязательно возвращает её хосту. После завершения программы в виртуальной машине страницы памяти лишь возвращаются в собственный список свободной памяти виртуальной машины, а хост продолжает считать их выделенными. Виртуальная машина также часто запрашивает больше памяти, чем ей действительно требуется; при отсутствии внутреннего давления на память однажды прочитанные холодные файловые страницы остаются в кэше. DSec решает эту проблему в два этапа: во-первых, виртуальная машина периодически сообщает хосту о непрерывных блоках свободной памяти, чтобы тот мог их вернуть, используя

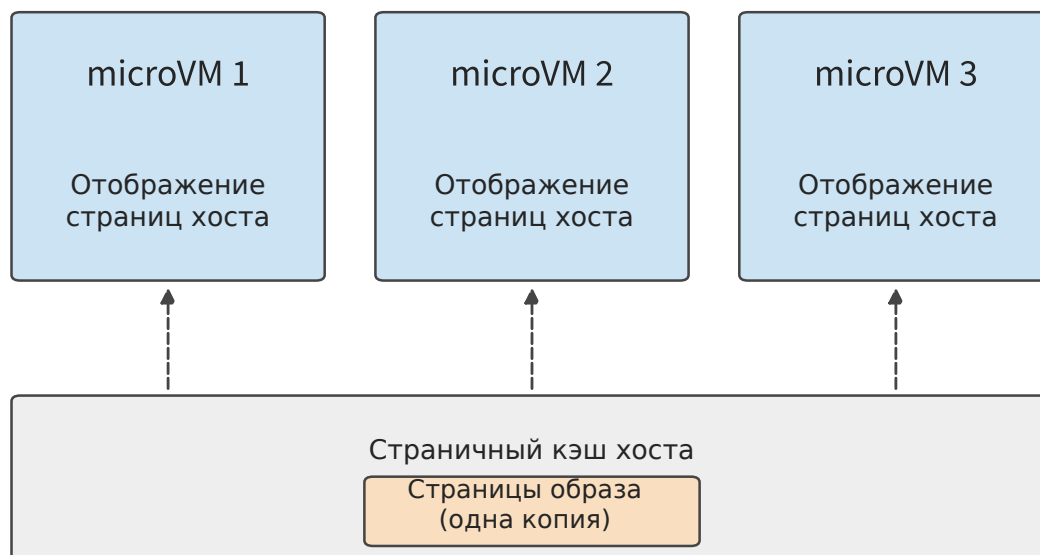
¹⁰Фиксированная архитектура E2B, документация по сохранению состояния (приостановка занимает около 4 секунд на каждый GiB памяти, восстановление — около 1 секунды; при приостановке сохраняется только файловая система), проверка поведения интерфейсов приостановки и снимков. E2B использует Firecracker.

¹¹DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.



Копирование в каждую VM через виртуальное блочное устройство

Рис. 11-12. При чтении через виртуальное блочное устройство одни и те же данные образа только для чтения хранятся в page cache хоста и каждой microVM.



Прямое отображение DAX без копирования

Рис. 11-13. После перевода слоя только для чтения на virtio-pmem и включения DAX каждая microVM непосредственно отображает файловые страницы в кэш хоста, поэтому в памяти остаётся лишь одна копия данных.

механизм уведомления о свободных страницах virtio-balloon; во-вторых, DAMON, механизм Linux для сбора статистики обращений к памяти посредством выборки, выявляет файловые страницы, к которым давно не обращались, и принудительно освобождает их, объединяя разрозненные свободные страницы в крупные блоки, о которых можно сообщить хосту. При совместном применении двух механизмов пиковое потребление почти не меняется, но суммарное потребление памяти за весь период работы уменьшается примерно на одну пятую.

Эти два механизма соответствуют двум величинам из раздела 11.1.2. Пиковое потребление определяет, поместится ли группа сред на одной машине, а площадь потребления $\int M(t)dt$ — сколько памяти требуется подготовить в среднем; совместное использование страниц только для чтения в основном снижает пик, а освобождение холодных страниц — площадь.¹²

11.2.3 Приостановка, возобновление и воссоздание среды

В разделе 11.2.2 был уменьшен объём данных, копируемых при каждом создании; другой способ экономии памяти — временно освобождать всю среду между двумя выполнениями инструмента. В трёхраундовой задаче из начала главы инструмент первого раунда завершает работу на 10-й секунде, а инструмент второго раунда начинает работу лишь на 19-й секунде, поэтому между ними модель вызывается в течение 9 секунд. При постоянном сохранении среды потребление составит $2 \times 9 = 18$ GiB·с. Согласно документации E2B, приостановка песочницы занимает примерно 4 секунды на каждый GiB памяти, а возобновление — около 1 секунды; во время сохранения и восстановления среда по-прежнему занимает 2 GiB. Приостановка среды размером 2 GiB занимает 8 секунд: с 10-й по 18-ю секунду сохраняется состояние, а с 18-й по 19-ю выполняется восстановление. Следующий инструмент всё ещё может начать работу на 19-й секунде, однако весь интервал занят сохранением и восстановлением, поэтому потребление остаётся равным 18 GiB·с и экономии нет.¹³

Площади потребления на рис. 11-14 и 11-15 одинаковы. Чтобы освободить память на время ожидания, сначала необходимо сохранить состояние, которое потребуется в следующем раунде, а время сохранения растёт вместе с объёмом памяти; поэтому экономия от приостановки зависит от того, что требуется сохранить и как будет выполняться восстановление.

Существует несколько способов восстановления сохранённого состояния. Возобновление после приостановки продолжает выполнение той же задачи; создание из снимка формирует несколько ветвей из одного состояния; повторное создание из начальной среды не наследует изменения предыдущей задачи. Все три способа позволяют снова получить готовую к работе среду, но наследуемое состояние различается.

При восстановлении агента из рабочего снимка наследуются ранее изменённые фай-

¹²DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

¹³Фиксированная архитектура E2B, документация по сохранению состояния (приостановка занимает около 4 секунд на каждый GiB памяти, восстановление — около 1 секунды; при приостановке сохраняется только файловая система), проверка поведения интерфейсов приостановки и снимков. E2B использует Firecracker.

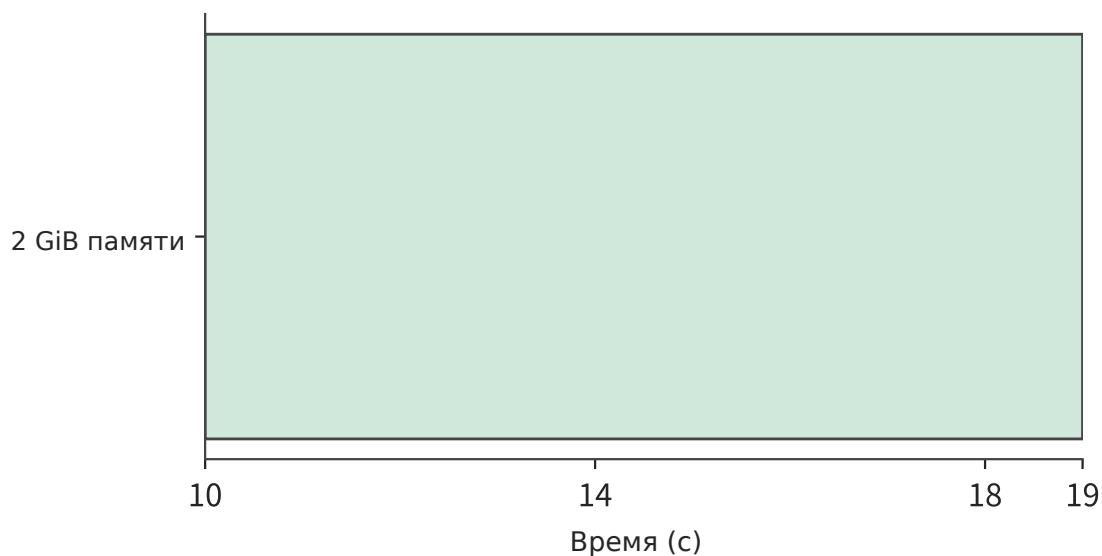


Рис. 11-14. Во время ожидания модели с 10-й по 19-ю секунду среда размером 2 GiB постоянно сохраняется, занимая 18 GiB·с. Горизонтальная ось совпадает со следующим рисунком.

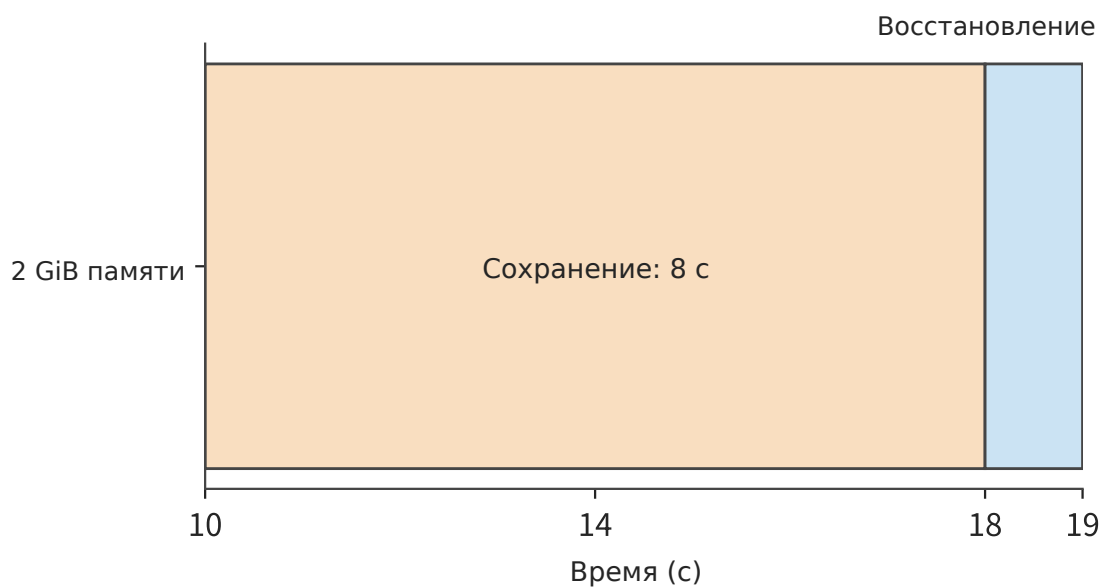


Рис. 11-15. С 10-й по 18-ю секунду состояние размером 2 GiB сохраняется со скоростью 4 с на GiB, а с 18-й по 19-ю выполняется восстановление; память не освобождается в течение всего интервала. Закрашенный интервал по-прежнему занимает 18 GiB·с, а следующий инструмент начинает работу на 19-й секунде.

лы и процессы. При проверке RL часто требуется, чтобы каждый образец начинал работу с одного и того же исходного каталога, поэтому среда создаётся из чистого шаблона. Если файлы снимка содержат вывод предыдущего теста, этот вывод также возвращается при восстановлении. Начальный шаблон используется для повторных экспериментов, а рабочий снимок — для продолжения выполнения; они сохраняют состояние в разные моменты времени.

Вернёмся к 9-секундному ожиданию модели в задаче из начала главы. При меньшем объёме памяти среды или более длинном интервале приостановка начинает приносить выгоду: при том же интервале в 9 секунд среду размером 512 MiB можно сохранить примерно за 2 секунды, освободив память на 6 секунд; если интервал короче суммарного времени сохранения и восстановления, возобновление дополнительно задержит следующее выполнение инструмента. E2B также позволяет при приостановке сохранять только файловую систему и заново запускать систему при восстановлении; воссоздание среды в каждом раунде из раздела 11.2.4 следует именно этому подходу: сохраняются только файлы, но не память процессов.

Решение о приостановке можно принимать посредством сравнения затрат. Пусть объём освобождаемой памяти равен M GiB, время, в течение которого она остаётся свободной, — G секунд, цена одного GiB-с — p_M , а дополнительные затраты на сохранение состояния, хранение и восстановление — $C_{\text{transition}}$. Приостановка сокращает затраты при условии $p_M M G > C_{\text{transition}}$. Левая часть растёт с увеличением времени освобождения, а правая представляет затраты, которые требуется нести при каждом сохранении и восстановлении; приравняв их, можно определить продолжительность простоя, начиная с которой приостановка сокращает затраты.

В задаче из начала главы интервал составляет всего 9 секунд, поэтому левая часть выражения почти равна нулю; при обучении RL встречаются гораздо более длинные интервалы. В описанном в разделе 10.4.4 запуске RL MiMo между прерыванием и перезапуском обычно проходит от одного до трёх часов. Если задание обучения на GPU вытесняется или прерывается, песочницы, используемые для rollout, нельзя уничтожить: в них хранятся файлы и процессы незавершённых траекторий; однако до возобновления обучения новые вызовы инструментов поступать не будут. Одно задание в DSes может запросить до 32K песочниц; если каждая из них занимает 2 GiB, как в начале главы, общий объём составит около 70 TB — более четверти всей памяти одного кластерного блока DSes, равной примерно 250 TB. Сохранение и восстановление песочницы размером 2 GiB по-прежнему занимают лишь около 9 секунд, тогда как память освобождается на часы: при приостановке на два часа она остаётся свободной примерно в 800 раз дольше, чем продолжает использоваться во время сохранения и восстановления.

Поэтому в DSes сразу после вытеснения задания фреймворк RL активно приостанавливает все песочницы этого задания (рис. 11-16). Для контейнеров сначала замораживается всё дерево процессов, затем разрешается выгрузка памяти на диск и принудительно запускается её освобождение; при восстановлении сначала выдаётся указание на предварительное чтение отображений памяти процессов, а затем выполнение размораживается. Для microVM память и состояние выполнения записываются в снимок, после чего процесс виртуальной машины завершается; при восстановлении запускается новый процесс, загружающий снимок. В обоих случаях состояние перемещается из памяти на следующий уровень хранения, а при необходимости возвращается обратно, поэтому затраты растут вместе с размером состояния. Любой запрос, отправленный в приостановленную песочницу, сначала возобновляет её работу, поэтому контроллеру не требуется

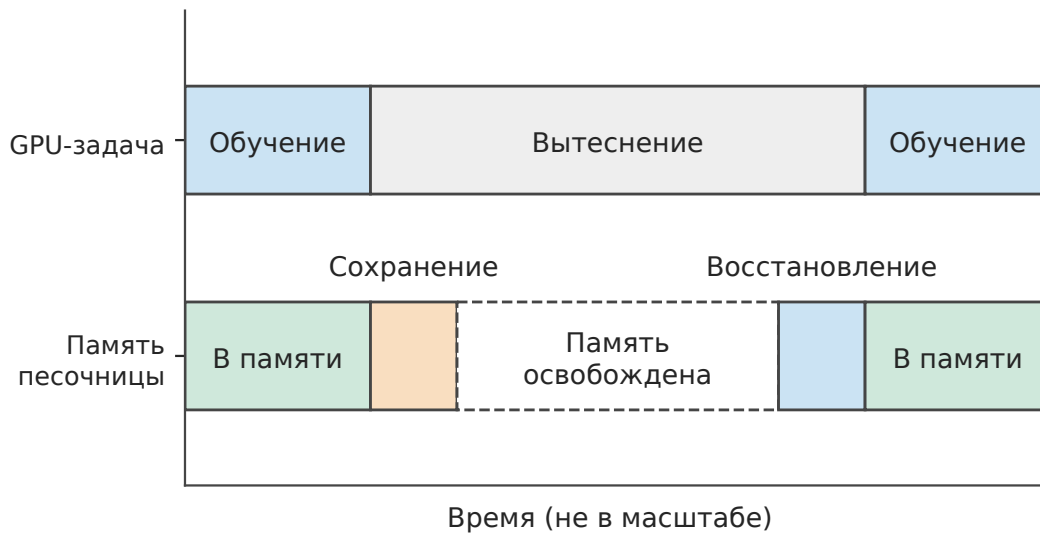


Рис. 11-16. После вытеснения задания GPU фреймворк RL приостанавливает соответствующие песочницы. Сохранение и восстановление занимают небольшие интервалы, а в остальное время память свободна; горизонтальная ось не соблюдает масштаб: вытеснение обычно длится часы, а сохранение и восстановление — секунды.

различать текущее состояние песочницы.¹⁴

Сравнение затрат определяет, следует ли приостанавливать среду; после приостановки разные виды состояния необходимо восстанавливать разными способами. Снимок сохраняет файлы и память, соединения повторно устанавливаются клиентом, а внешнее хранилище продолжает сохранять уже зафиксированные записи. После восстановления контроллер сначала получает результат операции, а затем продолжает следующий раунд, не выполняя уже завершённые изменения повторно.¹⁵

11.2.4 Постоянное сохранение, создание по требованию и предварительная подготовка

Приостановка требует сохранения состояния процессов. Если в следующем раунде нужны только уже сохранённые файлы и результаты, можно дополнительно отказаться от снимка процессов и просто воссоздать среду. В задаче программирования из начала главы рабочий каталог и результаты уже сохранены в постоянном хранилище, поэтому в каждом раунде инструмент может запускать новый процесс. Сам шаблон E2B представляет собой снимок заранее запущенной виртуальной машины; согласно документации, восстановление песочницы занимает около 1 секунды. Если шаблон кэширован на ло-

¹⁴DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

¹⁵Фиксированная архитектура E2B, документация по сохранению состояния (приостановка занимает около 4 секунд на каждый GiB памяти, восстановление — около 1 секунды; при приостановке сохраняется только файловая система), проверка поведения интерфейсов приостановки и снимков. E2B использует Firecracker.

кальной машине, запуск microVM из шаблона, загрузка рабочего каталога и инициализация среды выполнения инструментов занимают, по предположению, 2 секунды; назовём это горячим путём. Если шаблона нет на локальной машине, сначала требуется получить шаблон размером 2 GiB по сети 25 GbE, что добавляет примерно 0,69 секунды; холодный путь занимает около 2,7 секунды. Далее по умолчанию предполагается горячий путь. Каждая подготовка суммарно использует 0,1 секунды процессорного времени, а с начала подготовки до завершения инструмента среда занимает 2 GiB. При подготовке загружаются только известная среда выполнения инструмента и рабочий каталог; изменения файлов выполняются после получения полных параметров вызова инструмента.

Известно, что модель первого раунда вернёт результат на 9-й секунде, поэтому платформа может начать подготовку среды на 7-й секунде, выполнить инструмент на 9-й, а на 10-й сохранить результат и уничтожить среду. В следующих двух раундах подготовка начинается соответственно на 17-й и 27-й секундах. Все три подготовки завершаются во время вызовов модели, задача по-прежнему заканчивается на 30-й секунде, а среду требуется сохранять только в течение трёх секунд подготовки и выполнения каждого раунда.

Суммарное потребление памяти каждой задачей снижается с 60 до $3 \times (2+1) \times 2 = 18$ GiB-с, а среднее потребление памяти платформой — с 600 до 180 GiB. Цена этого — создание 30 сред в секунду; подготовка в среднем дополнительно занимает $30 \times 0.1 = 3$ ядра CPU, поэтому потребность в CPU возрастает с 30 до 33 ядер. Согласно статье о Firecracker, один хост может создавать до 150 microVM в секунду, поэтому 30 в секунду остаются в этих пределах. Добавив небольшой объём работы по созданию сред, платформа экономит 420 GiB состояния, которое иначе пришлось бы сохранять во время ожидания, и снижает потребность в памяти с 600 GiB, не помещающихся на этой машине, примерно до половины её ёмкости.

На рис. 11-17 и 11-18 сравниваются полные временные шкалы задачи при двух способах управления средой и показано изменение потребления памяти. Три момента выполнения инструментов не изменяются; исчезает потребление памяти во время длительных интервалов ожидания модели. Здесь момент подготовки можно спланировать точно, поскольку заранее известны и инструмент следующего раунда, и время возврата результата моделью.

Такое планирование предполагает, что подготовка завершится до выполнения инструмента. Если тип следующего инструмента приходится предсказывать по потоковому выводу, то время ожидания после выдачи вызова зависит от того, насколько заранее началась подготовка и оказалось ли предсказание правильным. SpecBox заранее готовит соответствующую среду, используя сведения о типе инструмента из потокового вывода и историю переключений между инструментами; ниже выгода анализируется для одного вызова.¹⁶

Пусть одна подготовка занимает P секунд, время упреждения равно L секунд, а вероятность правильного предсказания требуемой среды — h . Предположим, что при правильном предсказании ресурсы не конкурируют, ошибочную подготовку можно немедленно отменить, а после ошибки нужная среда готовится по требованию. Тогда среднее время ожидания готовности среды после выдачи вызова равно

¹⁶Исследование планирования, маршрутизации моделей и облачных окружений, охватывающее ASI, RLBoost, DistRS и SpecBox. Исследование ASI охватывает шесть месяцев и 155 410 GPU; доля распределения измеряет принадлежность устройств. Цена токенов взята из снимка страницы цен Claude API.

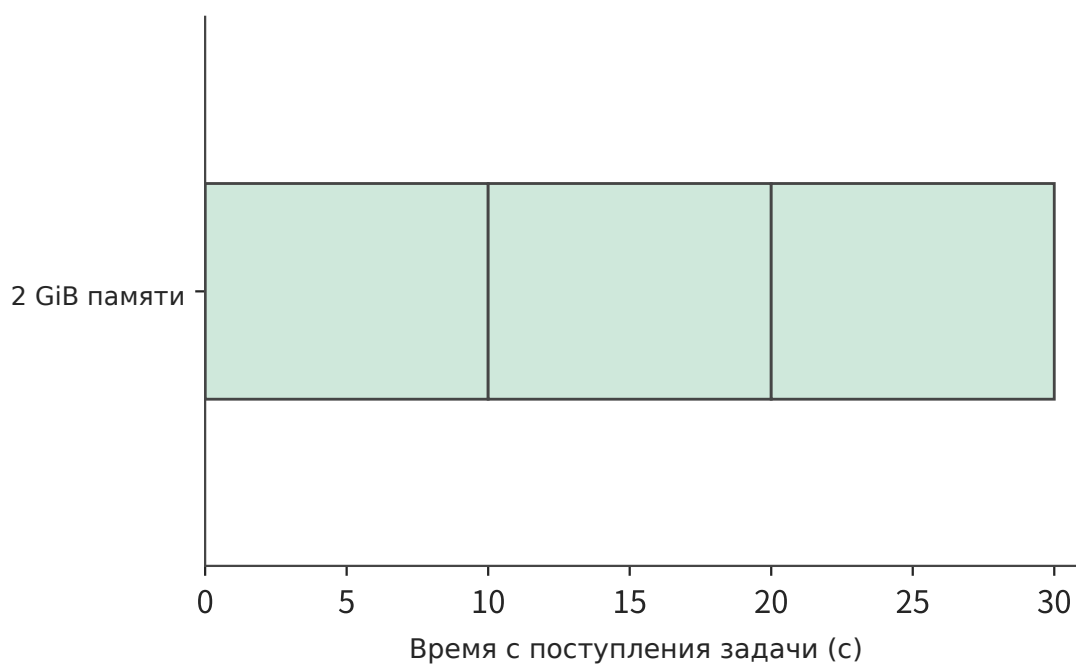


Рис. 11-17. Среда сохраняется от начала задачи до её завершения. В каждом раунде модель работает 9 s, а инструмент — 1 s; суммарное потребление за три раунда составляет 60 GiB·с. Горизонтальная ось совпадает со следующим рисунком.

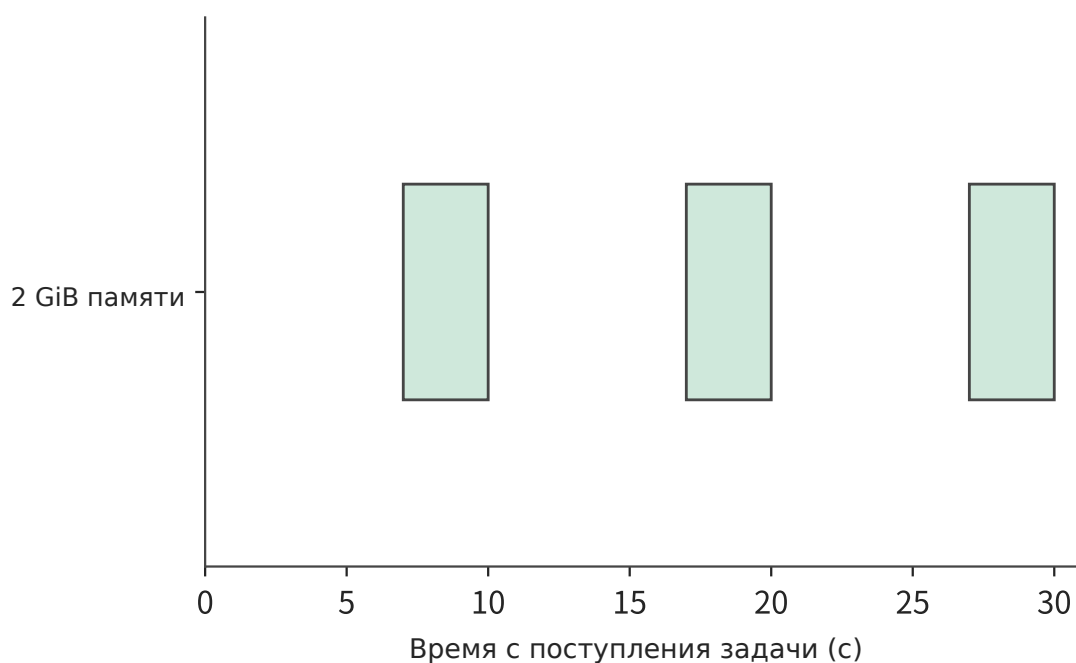


Рис. 11-18. В каждом раунде среда сохраняется только во время подготовки в течение 2 s и выполнения в течение 1 s; суммарное потребление за три раунда составляет 18 GiB·с. Рабочий каталог и результаты сохранены в постоянном хранилище, поэтому три раунда по-прежнему завершаются на 30-й секунде.

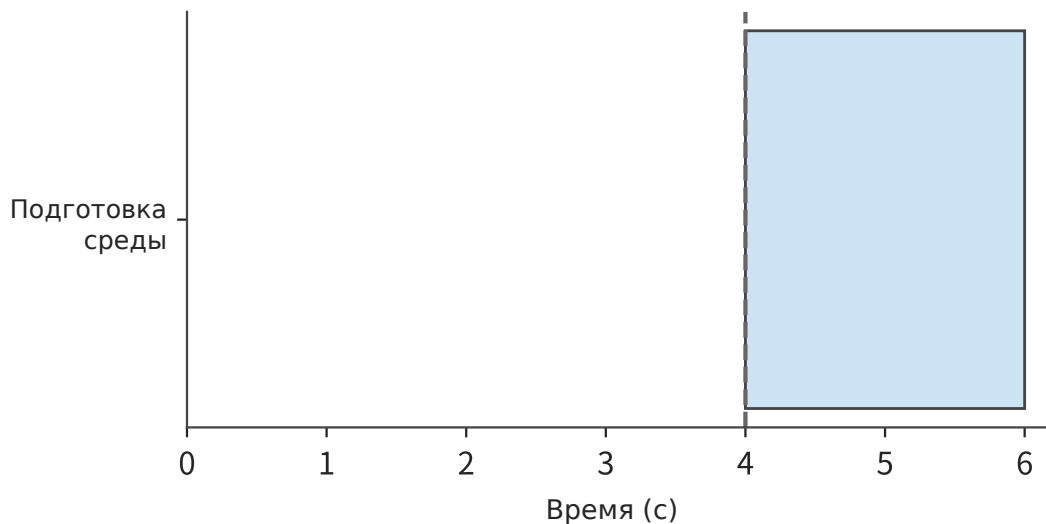


Рис. 11-19. Создание по требованию: подготовка начинается только после поступления вызова на 4-й секунде, завершается на 6-й секунде, а ожидание длится 2 s. Синим обозначена подготовка, пунктиром — момент вызова; на следующих четырёх рисунках используется та же горизонтальная ось.

$$E[W] = h \max(P - L, 0) + (1 - h)P = P - h \min(P, L).$$

При правильном предсказании первые $\min(P, L)$ секунд подготовки перекрываются с вызовом модели, а вызов инструмента ожидает завершения оставшейся подготовки; при ошибке фактически необходимая среда готовится с нуля. Взвешивание этих двух путей по вероятности попадания даёт приведённую формулу. Если подготовка уже успевает завершиться до возврата результата моделью, более раннее её начало лишь увеличивает время, в течение которого готовая среда ожидает вызова.

Возьмём $P = 2$ секунды и $h = 0.75$. Без предварительной подготовки каждый вызов ожидает 2 секунды. При упреждении на 1 секунду в среднем три из четырёх вызовов ожидают лишь 1 секунду, а один из-за ошибки по-прежнему ждёт 2 секунды; среднее время ожидания равно 1,25 секунды. При упреждении на 2 секунды правильная ветвь больше не ждёт, поэтому среднее значение снижается до 0,5 секунды. Упреждение на 3 секунды также даёт 0,5 секунды, но в правильной ветви готовая среда дополнительно сохраняется в течение 1 секунды. Поэтому для среды размером 2 GiB каждое предсказание увеличивает среднее потребление памяти во время простоя на $0.75 \times 2 \times 1 = 1.5$ GiB-с.

Таким образом можно сравнить три способа сокращения ожидания. Кэширование шаблонов устраняет затраты на повторное получение данных; сохранение уже запущенной среды позволяет начать выполнение сразу после поступления вызова, но требует постоянного расхода памяти; воссоздание среды в каждом раунде позволяет использовать время вызова модели для подготовки. В задаче из начала главы среда выполнения следующего раунда заранее известна, поэтому она готовится во время вызова модели; если разные инструменты чередуются, в расчёте также необходимо учитывать долю правильных предсказаний.

Локальное воспроизведение работы инструментов демонстрирует последствия слишком ранней подготовки: после перехода от подготовки с упреждением на 50 ms к

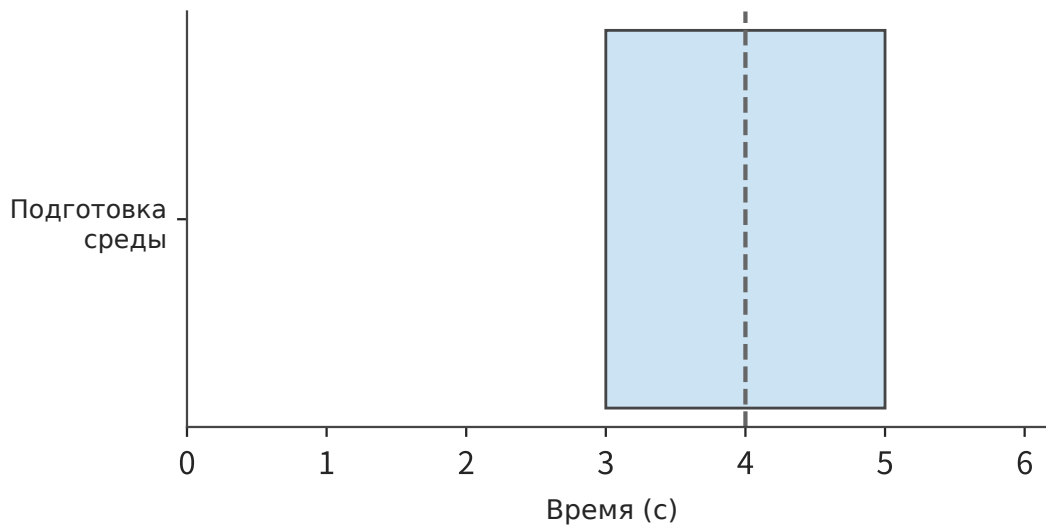


Рис. 11-20. Правильное предсказание и упреждение на 1 s: подготовка идёт с 3-й по 5-ю секунду, поэтому после поступления вызова на 4-й секунде требуется ждать ещё 1 s. Синим обозначена подготовка среды, пунктиром — момент поступления вызова на 4-й секунде.

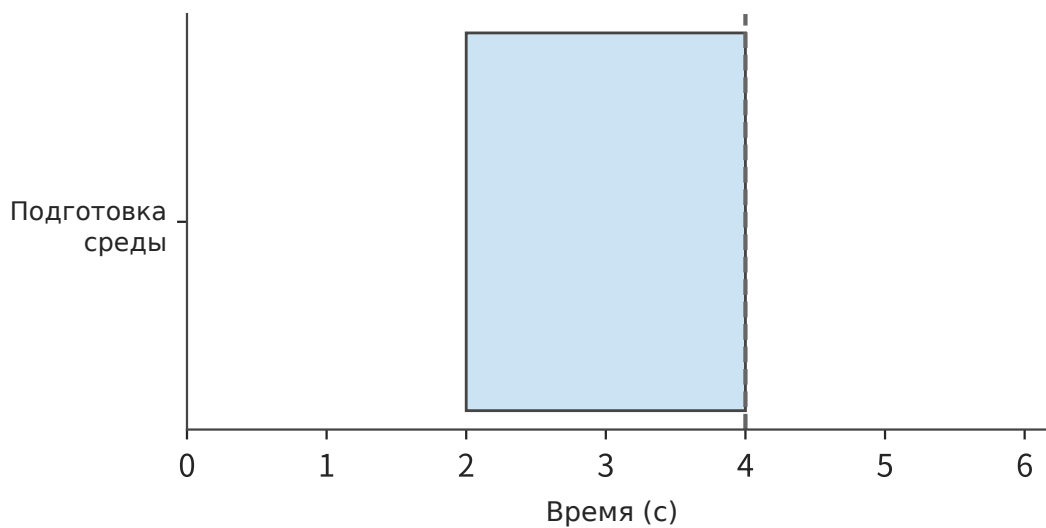


Рис. 11-21. Правильное предсказание и упреждение на 2 s: подготовка завершается точно в момент поступления вызова на 4-й секунде. Синим обозначена подготовка среды, пунктиром — момент поступления вызова на 4-й секунде.

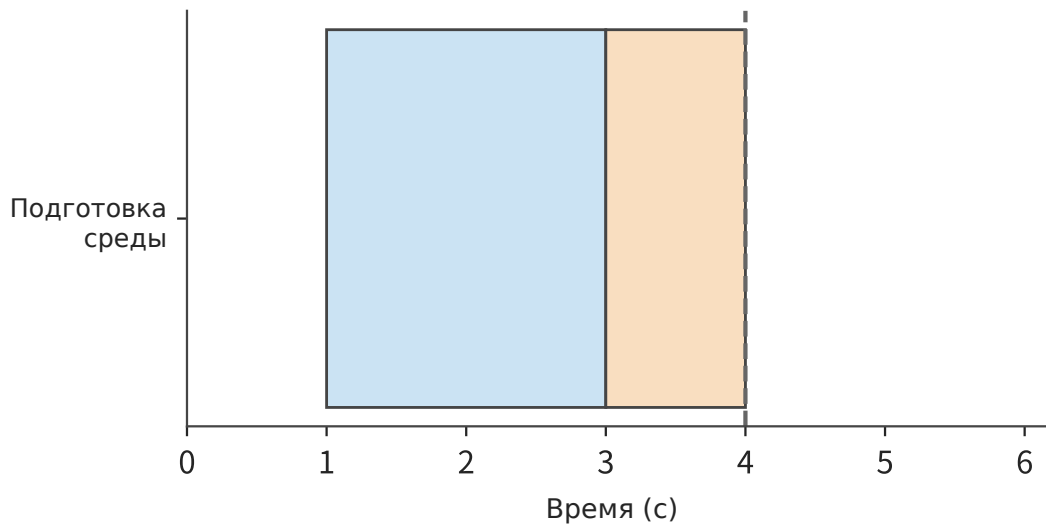


Рис. 11-22. Правильное предсказание и упреждение на 3 с: подготовка завершается на 3-й секунде, а оранжевая часть обозначает 1 с простоя готовой среды в ожидании вызова. Синим обозначена подготовка среды, пунктиром — момент поступления вызова на 4-й секунде.

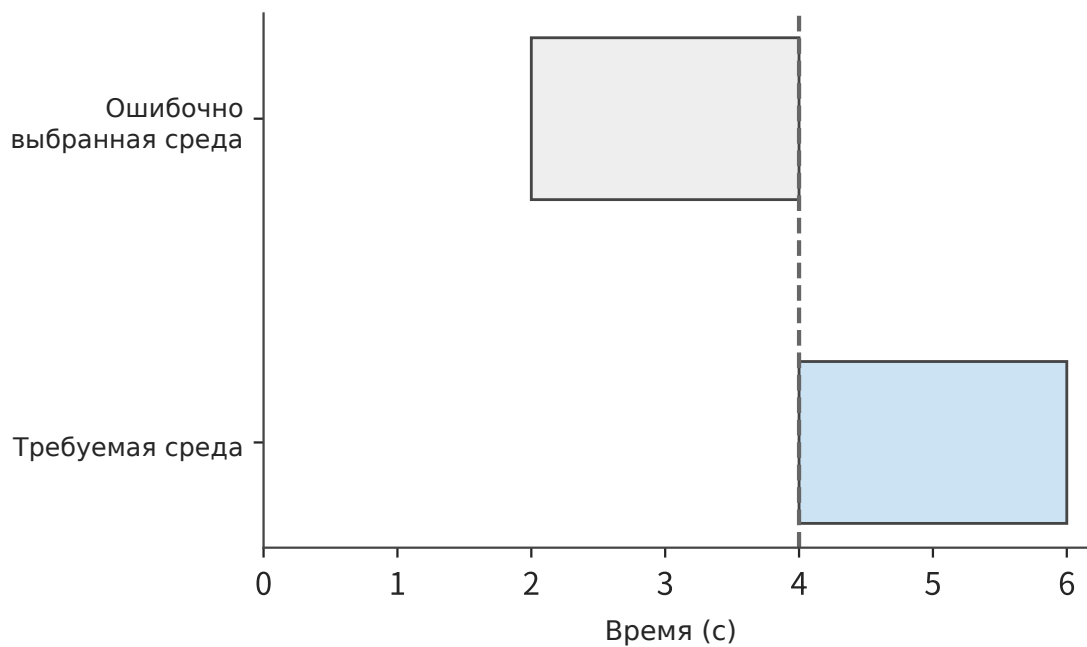


Рис. 11-23. Ошибочное предсказание: с 2-й по 4-ю секунду готовится ненужная среда, обозначенная серым; после отмены на 4-й секунде подготовка действительно необходимой среды занимает ещё 2 с. На рисунке предполагаются немедленная отмена и отсутствие конкуренции за ресурсы.

подготовке с самого начала ожидания модели суммарное время вызовов инструментов осталось примерно равным 0,26–0,28 секунды, но рассчитанное по результатам выборки суммарное потребление памяти выросло примерно с 0,0193 до 0,467 GiB-с, то есть приблизительно в 24 раза. Дополнительная память в основном потреблялась в интервале между готовностью среды и поступлением вызова. Если среда успевает подготовиться до поступления вызова инструмента, создавать её ещё раньше нет необходимости.¹⁷

Воссоздание среды в каждом раунде возможно благодаря условию задачи из начала главы: всё состояние, необходимое следующему раунду, уже записано в постоянное хранилище. Во многих задачах агентов это условие не выполняется. Модель может устанавливать зависимости, запускать базу данных или фоновые сервисы, процессы и состояние памяти которых потребуются последующим вызовам; если сохранить только файлы, воссозданная среда не будет эквивалентна исходной. Песочницы DSec существуют на протяжении всего взаимодействия и используют другой подход: среда сохраняется, но стоимость её постоянного пребывания снижается посредством загрузки по требованию, совместного использования страниц только для чтения и освобождения холодных страниц из раздела 11.2.2, а также приостановки на длительных интервалах из раздела 11.2.3. Выбор подхода зависит от того, можно ли полностью записать состояние между раундами в постоянное хранилище: если можно, небольшие затраты на создание позволяют вернуть большой объём памяти; если нельзя, среду приходится сохранять, стараясь сократить стоимость её постоянного пребывания.¹⁸

11.3 Распределение и планирование общего пула ресурсов

11.3.1 Гетерогенные ресурсы, групповое выделение и ограничения размещения

После расчёта потребностей отдельной задачи в ресурсах необходимо определить, способен ли общий пул предоставить все эти ресурсы одновременно. Задания на нескольких ускорителях обычно требуют определённого типа ускорителей, объёма видеопамати на каждом из них, соответствующих ресурсов CPU и оперативной памяти узла, а также размещения, подходящего для используемой схемы обмена данными. Достаточное общее количество свободных GPU — лишь одно из необходимых условий запуска задания.

Эти ресурсы также должны находиться на подходящих узлах: каждый CPU-инструмент из начала главы независимо использует одно ядро, а при обучении на нескольких ускорителях вся их группа должна работать совместно, чтобы все процессы задания могли запуститься одновременно. Поэтому возможность запуска такого задания зависит и от того, на каких узлах распределены свободные ресурсы.

Пример 11-3. Как ограничения по GPU, CPU и узлам приводят к фрагментации ресурсов? Заданию нужны четыре H100 и 16 ядер CPU, причём все эти ресурсы должны

¹⁷Локальное воспроизведение предварительного прогрева инструментов. Фактические вызовы, попадания прогнозов и границы окон выборки сохранены в исходных записях.

¹⁸DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

Новой задаче нужны: 4 H100 на одном узле + 16 ядер

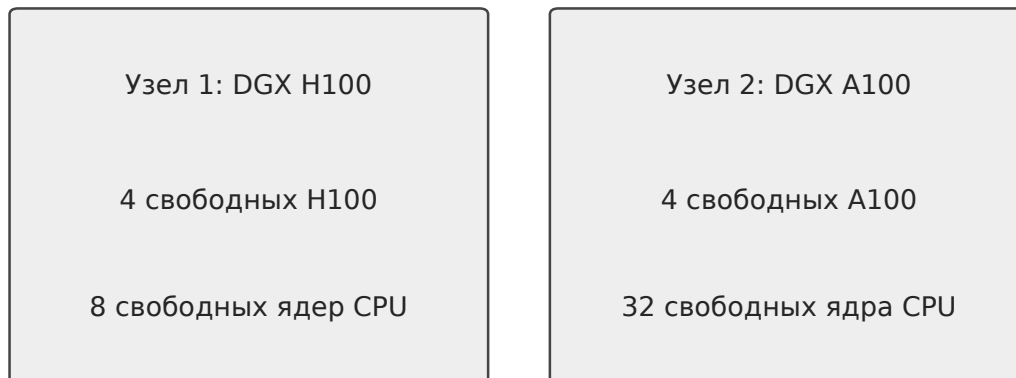


Рис. 11-24. До перераспределения: новому заданию нужны четыре H100 и 16 ядер на одном узле. Узлу 1 не хватает CPU, а на узле 2 свободны только A100.

находиться на одном узле. Узел 1 — DGX H100 с восемью H100 SXM и двумя Xeon Platinum 8480C, всего 112 ядер. Выполняющееся задание уже занимает четыре H100 и 104 ядра, поэтому свободны лишь четыре H100 и восемь ядер. Узел 2 — DGX A100 с восемью A100 и двумя EPYC 7742, всего 128 ядер; на нём свободны четыре A100 и 32 ядра. Во всём пуле есть и четыре свободных H100, и достаточное количество ядер CPU, но ни один узел не удовлетворяет всем условиям одновременно.¹⁹

Предположим, что до завершения исходной задачи на узле 1 нужно ждать 12 секунд, её перенос занимает 4 секунды, а локальное выполнение нового задания — 20 секунд. Ожидание с последующим выполнением займёт 32 секунды, перенос с последующим выполнением — 24 секунды; после перераспределения ресурсов новое задание завершится на 8 секунд раньше. Если подготовка другой совместимой конфигурации с распределением по нескольким узлам занимает 1 секунду, а выполнение — 32 секунды, немедленный распределённый запуск потребует 33 секунды и завершится ещё позже. Время завершения для трёх вариантов соответственно равно

$$T_{\text{wait}} + T_{\text{run,local}}, \quad T_{\text{move}} + T_{\text{run,local}}, \quad T_{\text{ready,spread}} + T_{\text{run,spread}}.$$

В этом примере вариант с переносом опережает ожидание на 8 секунд, а распределённый запуск — на 9 секунд. С ростом объёма состояния увеличивается и время переноса; когда оно достигнет 12 секунд, варианты с переносом и ожиданием будут занимать одинаковые 32 секунды. Поэтому планировщик может напрямую сравнивать прогнози-

¹⁹Характеристики системы DGX H100 (два Xeon Platinum 8480C, всего 112 ядер); характеристики системы DGX A100 (таблица 10 приложения А, два EPYC 7742, всего 128 ядер).

Новой задаче нужны: 4 H100 на одном узле + 16 ядер

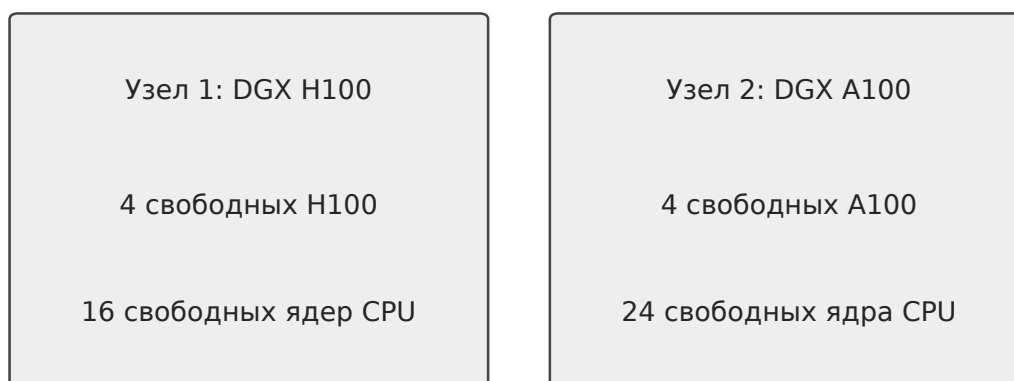


Рис. 11-25. После переноса задачи, использующей 8 ядер CPU, с узла 1 на узел 2 на узле 1 одновременно освобождаются четыре H100 и 16 ядер, поэтому новое задание можно запустить.

руемое время переноса с оставшимся временем выполнения исходной задачи и выбирать вариант, который раньше подготовит все необходимые ресурсы.

Фрагментация ресурсов встречается и в производственных кластерах. В опубликованном в 2026 году исследовании Гонконгского университета науки и технологий, Alibaba и других организаций анализировался производственный GPU-кластер бессерверной инфраструктуры Alibaba (Alibaba Serverless Infrastructure, ASI). Исследование показало, что нехватка сопутствующих ресурсов CPU, необходимость группового запуска заданий и ограничения сетевого размещения могут делать свободные GPU недоступными для использования. После добавления вытесняемых заданий с низким приоритетом доля распределённых GPU в исследовании выросла с 68 до 93%. Эти низкоприоритетные задания используют ускорители, временно не нужные высокоприоритетным заданиям. Когда поступает задание, требующее группового запуска, планировщик освобождает эти ускорители и с помощью переноса задач подготавливает требуемые ресурсы.²⁰

11.3.2 Очереди, приоритеты и стоимость вытеснения

Даже при подходящей конфигурации узлов одновременное поступление запросов может не оставить времени на перераспределение ресурсов. Задачи из начала главы поступают равномерно и требуют выполнения 30 вызовов инструментов в секунду, с чем

²⁰Исследование планирования, маршрутизации моделей и облачных окружений, охватывающее ASI, RLBoost, DistRS и SpecBox. Исследование ASI охватывает шесть месяцев и 155 410 GPU; доля распределения измеряет принадлежность устройств. Цена токенов взята из снимка страницы цен Claude API.

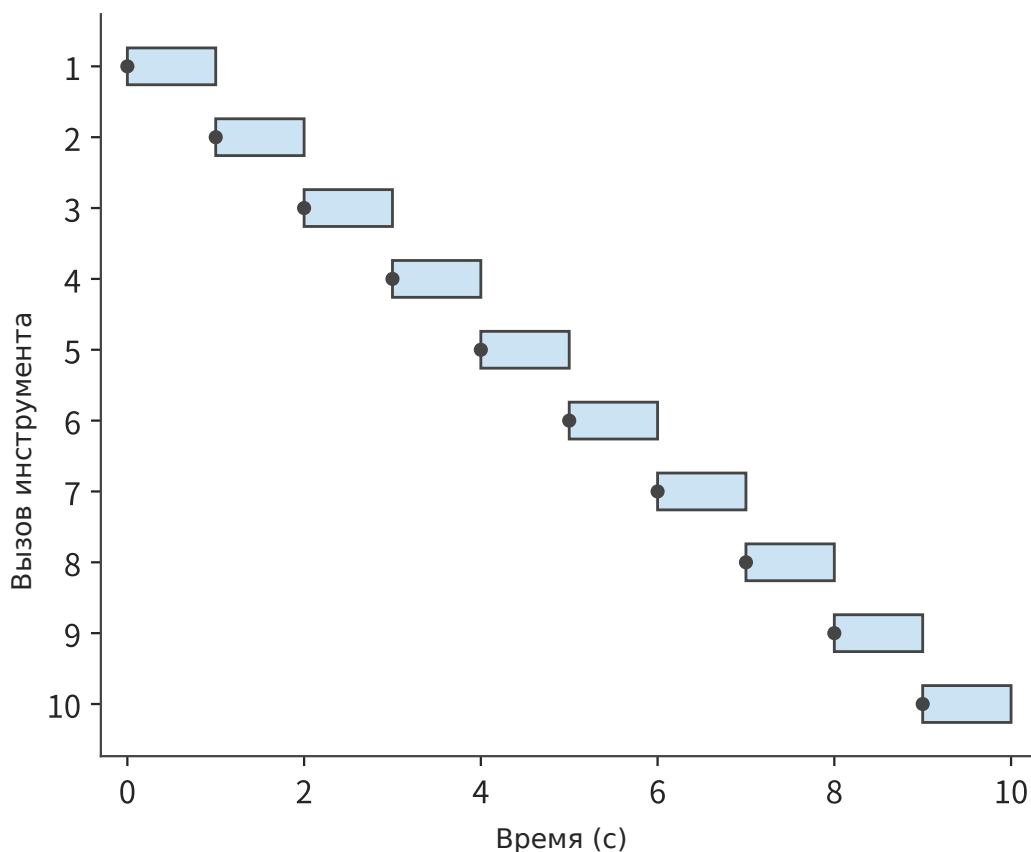


Рис. 11-26. Десять вызовов поступают с интервалом 1 с, один исполняющий процесс обрабатывает каждый вызов за 1 с. Точками обозначено поступление, синим — выполнение; каждый вызов может начаться немедленно.

справляются 48 ядер CPU. Если вызовы поступают всплесками, время ожидания в очереди может отличаться даже при том же общем объёме работы.

Сначала рассмотрим один исполняющий процесс, в котором каждая операция инструмента занимает 1 секунду. Если десять операций поступают соответственно в моменты 0, 1, ..., 9 секунд, каждая начинает выполняться сразу после поступления, поэтому время ожидания всегда равно нулю. Если все десять операций поступят в момент 0, выполнение по-прежнему займёт 10 секунд, но время ожидания отдельных операций составит соответственно 0, 1, ..., 9 секунд, в среднем 4,5 секунды. Общий объём работы определяет продолжительность занятости исполняющего процесса, а моменты поступления и порядок выполнения — время ожидания каждой задачи.

На рис. 11-26 и 11-27 сопоставлены эти два способа поступления; во втором случае серые полосы перед блоками выполнения обозначают время ожидания. Одни и те же 10 секунд вычислительной работы могут как совсем не создавать очередь, так и приводить к длительному ожиданию. Поэтому для перехода от средней потребности в ресурсах к фактическому времени ответа необходимо также знать, поступают ли запросы всплесками.

Рассмотрим всплеск вызовов инструментов на платформе исправления кода. Пред-

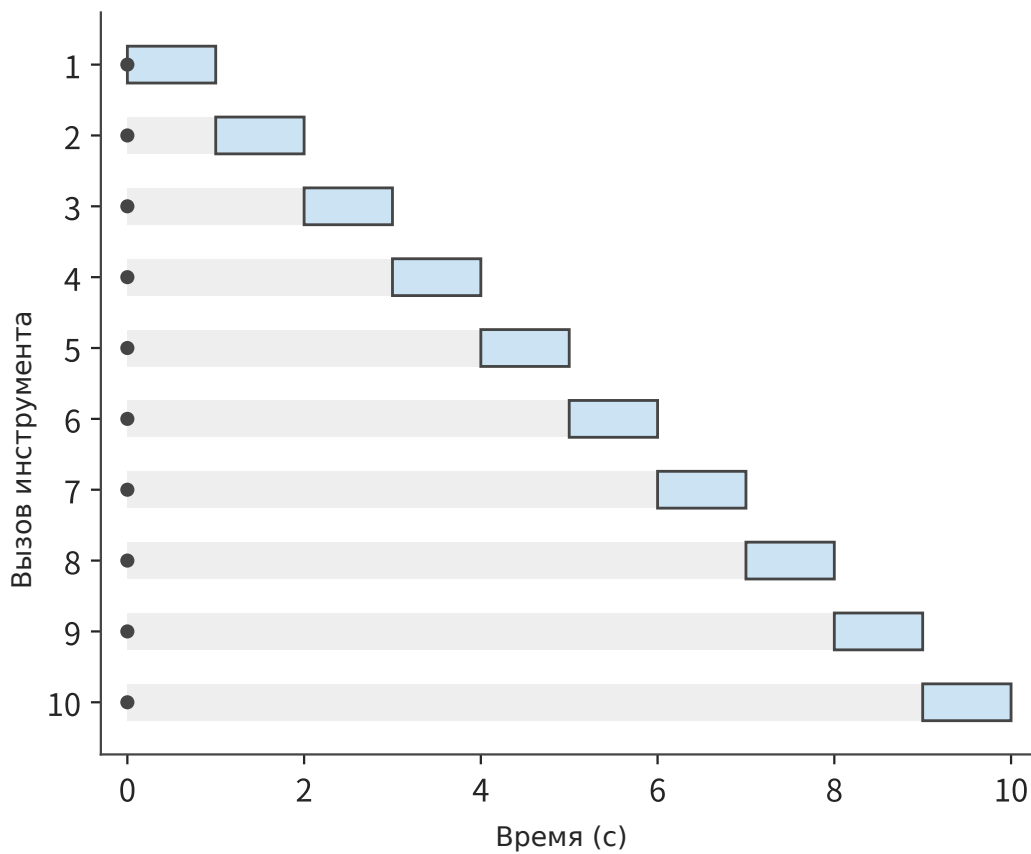


Рис. 11-27. Десять вызовов одновременно поступают в момент 0 s; серым обозначено ожидание, синим — выполнение, время ожидания по строкам составляет 0–9 s. Исполняющий процесс по-прежнему работает всего 10 s, а среднее время ожидания в очереди равно 4,5 s.

положим, что все 48 ядер платформы свободны и одновременно поступают 80 вызовов инструментов продолжительностью по 1 секунде. Первые 48 вызовов выполняются немедленно, остальные 32 ожидают 1 секунду; среднее время ожидания составляет 0,4 секунды, а все вызовы завершаются за 2 секунды. Если увеличить систему до двух m5d.metal с 96 ядрами, такой всплеск будет обработан за 1 секунду. Увеличение количества ядер CPU повышает способность обрабатывать всплески запросов; при равномерном поступлении задач для выполнения инструментов по-прежнему в среднем достаточно 30 ядер CPU.

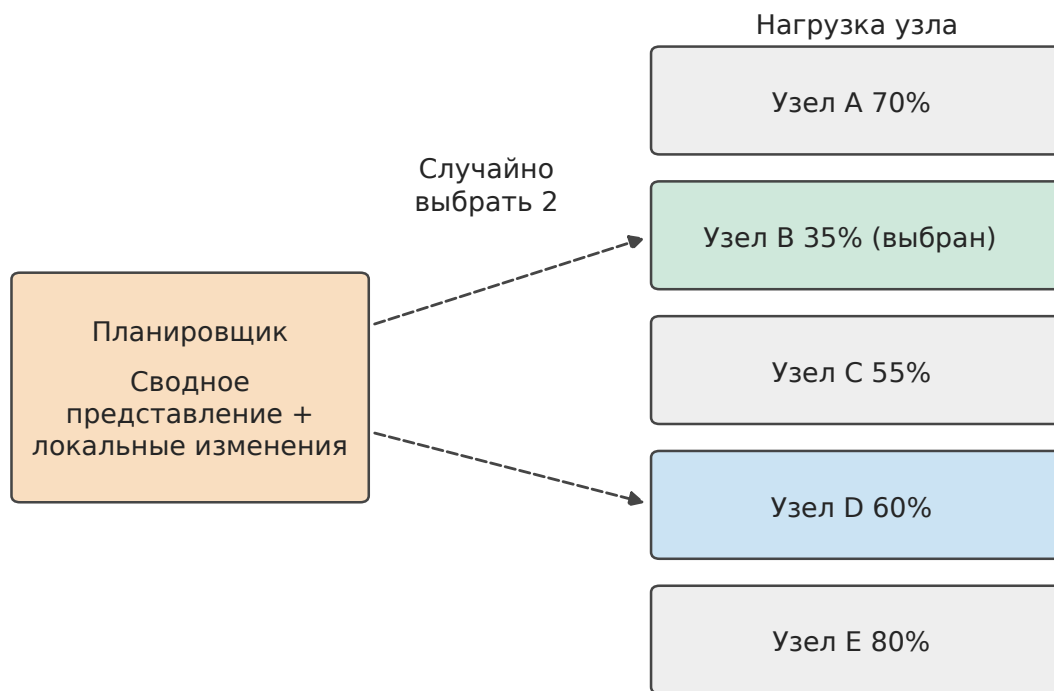
Для той же серии вызовов инструментов требуется достаточно памяти для сред. Каждая среда занимает 2 GiB, поэтому для 80 подготовленных сред нужно 160 GiB. Вариант с постоянным размещением требует около 600 GiB, что уже превышает доступные 358 GiB памяти; вариант из раздела 11.2, в котором среда пересоздаётся на каждой итерации, в среднем занимает лишь 180 GiB, и оставшихся примерно 178 GiB достаточно для этих 80 сред. Сокращение числа сред, сохраняемых во время ожидания, освобождает память для новых вызовов, благодаря чему свободные CPU могут своевременно начать выполнение.

При распределении всплеска запросов по нескольким узлам необходимо также решить, где разместить каждую среду. Пиковая скорость создания DSec превышает 5 000 песочниц в секунду. В среднем на 160 узлах это около 31 песочницы на узел в секунду, что сопоставимо с заданной в разделе 11.2.4 скоростью создания 30 сред в секунду. Данные о нагрузке узлов, доступные планировщику, агрегируются периодически, поэтому запросы, размещённые после последней агрегации, ещё не отражены в представлении. Если каждый раз выбирать узел с минимальной текущей нагрузкой, серия одновременно поступивших запросов устремится на один и тот же узел.

DSec решает эту проблему тремя мерами (рис. 11-28). Во-первых, каждый раз случайным образом выбирается лишь несколько узлов, а среди них — узел с наименьшей нагрузкой, то есть применяется *power-of-k-choices*; благодаря этому одновременно принятые решения естественным образом распределяются. Во-вторых, каждый экземпляр планировщика дополняет своё локальное представление сведениями о недавно размещённых им песочницах, которые ещё не попали в агрегированные данные. В-третьих, окончательное решение о приёме принимает сам узел: при нехватке локальных ресурсов он отклоняет запрос, и планировщик выбирает другой узел. Глобальное представление может быть несколько устаревшим, поскольку соблюдение ограничений ёмкости гарантирует собственная проверка узла; ни планировщику, ни подсистеме агрегации нагрузки не требуется постоянное состояние, поэтому их экземпляры можно в любое время добавлять, удалять или заменять.²¹

После резервирования ёмкости для всплесков запросов необходимо определить порядок их выполнения. Приоритет задаёт, кто первым воспользуется этим резервом. Онлайн-задачи должны запускаться быстро, а офлайн-задачи могут подождать, поэтому платформа может временно предоставлять ресурсы офлайн-работам и забирать их при поступлении онлайн-вызовов. Предположим, онлайн-задача может ждать не более 2 секунд, а сохранение состояния офлайн-задания занимает 30 секунд.

²¹DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.



Узел проверяет локальную ёмкость: при нехватке отклоняет, планировщик выбирает другой

Рис. 11-28. Планировщик дополняет периодически агрегируемое представление сведениями о только что размещённых им песочницах, каждый раз случайно выбирает несколько узлов и назначает запрос одному из узлов с меньшей нагрузкой. Узел проверяет локальную ёмкость; если её недостаточно, он отклоняет запрос, после чего планировщик выбирает другой узел.

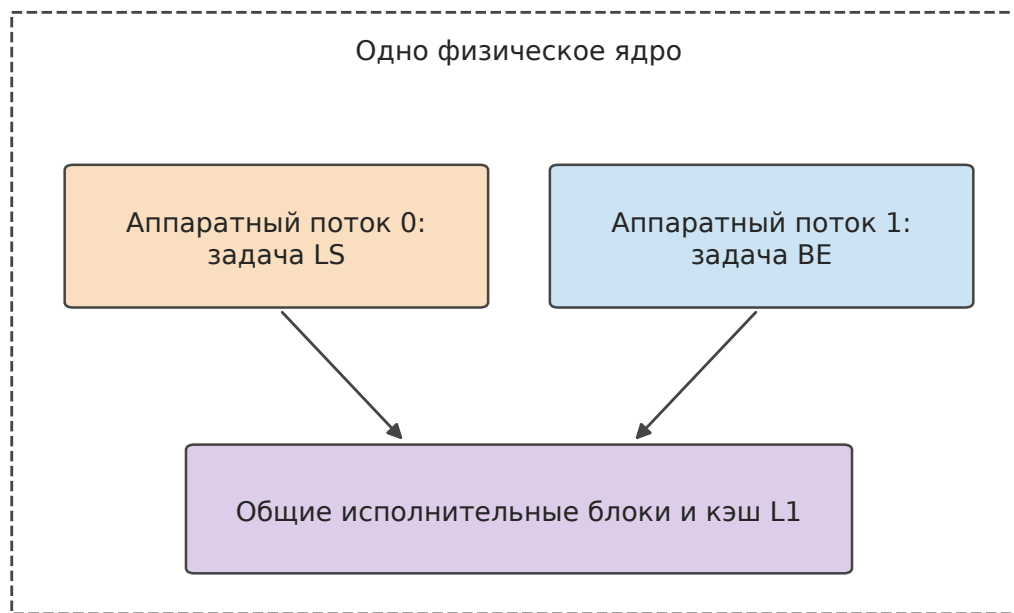


Рис. 11-29. SCHED_IDLE определяет только очерёдность выполнения задач в одном аппаратном потоке. Во время выполнения LS-задачи BE-задача по-прежнему может занимать другой аппаратный поток того же физического ядра и конкурировать с ней за исполнительные блоки.

Даже если эту группу ресурсов можно вытеснить, она освободится только через 30 секунд; следовательно, платформа должна отдельно зарезервировать ресурсы, которые можно выделить онлайн-задаче в течение 2 секунд.

Наличие приоритетов ещё не гарантирует изоляцию на уровне оборудования. При переподписке чувствительные к задержке задачи (latency-sensitive, LS) используют CPU совместно с допускающими ожидание задачами, выполняемыми по мере возможности (best-effort, BE), например когда AI-агент для игры в шахматы с ограничением времени на каждый ход работает на одном узле с обычными задачами обработки кода. Если поместить BE-задачи в класс планирования Linux SCHED_IDLE, они будут уступать CPU всякий раз, когда может выполняться LS-задача. Однако при включённой одновременной многопоточности (simultaneous multithreading, SMT, то есть Hyper-Threading) одно физическое ядро содержит два аппаратных потока, совместно использующих исполнительные блоки и кэш первого уровня. Планировщик распределяет ресурсы по аппаратным потокам, поэтому LS- и BE-задачи всё равно могут одновременно выполняться в двух аппаратных потоках одного физического ядра (рис. 11-29).

Согласно измерениям DSec, когда BE-нагрузка занимает половину ёмкости узла, продолжительность каждого шага LS-задачи увеличивается примерно на 45%, а использование одного лишь SCHED_IDLE почти ничего не улучшает. Если дополнительно включить для LS-задачи Linux core scheduling, не позволяя посторонним задачам совместно использовать с ней физическое ядро (рис. 11-30), увеличение снижается примерно до 17%. Оставшиеся помехи связаны со снижением частоты при высокой общей нагрузке узла, а также с совместным использованием пропускной способности памяти и кэша последнего уровня; для изоляции этих ресурсов нужны отдельные механизмы. На m5d.metal из начала главы Hyper-Threading отключён, поэтому у каждого ядра есть только один аппа-

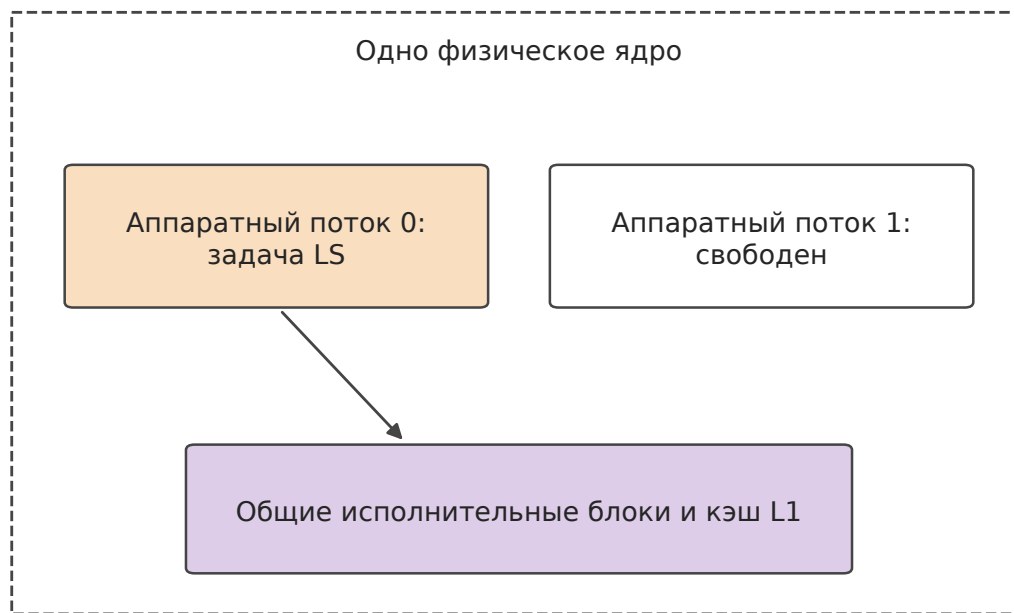


Рис. 11-30. После включения core scheduling оба аппаратных потока одного физического ядра выполняют задачи только из одной группы; во время выполнения LS-задачи другой аппаратный поток больше не выполняет ВЕ-задачу.

ратный поток и этот уровень помех отсутствует, но ценой уменьшения числа доступных для планирования логических CPU вдвое.²²

При вытеснении также теряется несохранённый прогресс. Разница между восстановлением задания, выполнявшегося 20 секунд и требующего 2 секунд на восстановление, из последней контрольной точки и его полным перезапуском зависит от объёма работы, выполненной после контрольной точки. Планировщику нужно учитывать не только то, насколько раньше сможет запуститься новая задача, но и затраты на сохранение и восстановление состояния, а также вызванную ими дополнительную задержку исходного задания.

Вытеснение показывает цену ускоренного запуска одной задачи. При длительной работе необходимо также не допускать постоянного откладывания одной и той же группы задач, поэтому следует явно определить критерий справедливости распределения. Если две задачи получили по два GPU, их доли кажутся одинаковыми; но если на ускорителях одного типа та же работа занимает вдвое больше времени, чем на ускорителях другого типа, прогресс будет различаться. При справедливом распределении по доминирующему ресурсу (dominant resource fairness, DRF) ресурс, доля которого для задачи максимальна среди всех типов ресурсов, называется доминирующим, а ресурсы справедливо распределяются по долям доминирующего ресурса. В исследованиях Gavel, Pollux и дру-

²²DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

гих дополнительно учитываются производительность разных ускорителей или прогресс обучения. Распределение по долям ресурсов отвечает на вопрос, сколько ресурсов получает каждая задача; распределение по прогрессу — насколько быстро она продвигается; ограничение максимального времени ожидания позволяет избежать длительного простаивания некоторых задач.²³

При переходе от отдельных вызовов инструментов к обучению с подкреплением (RL) потребность в ресурсах также меняется в зависимости от этапа. Интерактивные траектории, создаваемые во время rollout, включают как вывод модели, так и ответы инструмента или среды. В следующем разделе рассчитывается, насколько в действительности можно сократить время обучения, увеличив число ускорителей для генерации таких траекторий.

11.3.3 Динамическое соотношение ресурсов для генерации и обучения RL

Одна синхронная итерация RL требует сгенерировать примеры, выполнить верификацию и провести обновление. Если необходимый эффективный объём генерации на каждой итерации равен Q , а скорость пула генерации — r_g , то упрощённое время этапа генерации составляет Q/r_g . Добавление ускорителей rollout сокращает только параллелизуемую часть генерации; на каждой итерации всё равно необходимо выполнить верификацию, обновление, публикацию весов и необходимую синхронизацию. Если эти этапы выполняются последовательно, то

$$T_{\text{iter}} = T_{\text{generate}} + T_{\text{verify}} + T_{\text{update}} + T_{\text{publish}}.$$

Например, если генерация, верификация, обновление и публикация занимают соответственно 40, 10, 30 и 5 секунд, вся итерация длится 85 секунд. Если удвоить только скорость генерации, продолжительность итерации сократится до 65 секунд, то есть общее ускорение составит лишь около 1,3 раза. Даже если свести время генерации к нулю, остальные этапы по-прежнему займут 45 секунд, поэтому общее ускорение не превысит примерно 1,9 раза. По мере сокращения времени генерации доля верификации и обновления в общем времени постепенно растёт, а выигрыш от дальнейшего увеличения числа ускорителей генерации уменьшается.

На рис. 11-31 добавление ускорителей генерации сокращает только синюю часть. Дальнейшие действия возможны в двух направлениях: изменить порядок выполнения, чтобы верификация перекрывалась с генерацией, либо продолжить добавлять ускорители генерации, предварительно рассчитав затраты на подготовку новых ускорителей.

Способ перекрытия верификации с генерацией рассматривается в разделе 11.3.4. При добавлении временных ускорителей генерации RLBoost не изменяет обучающую группу, а временные экземпляры присоединяются к пулу генерации после подготовки весов.²⁴

Пример 11-4. Как общий исходящий канал ограничивает скорость распространения весов на несколько экземпляров? Полные веса Qwen3-8B в формате BF16 занимают 16 381 470 720 байт, или около 16,38 GB в десятичной системе; их нужно отдельно

²³DRF, Gavel и Pollux используются как материалы о механизмах планирования; совместное использование и размещение ресурсов дополнительно поясняет различие между доступностью ресурсов и предоставлением состояния.

²⁴Исследование готовности весов и полезной выработки, содержащее условия статьи о RLBoost и фиксированные пути исходного кода PolyRL. Размер весов Qwen3-8B в байтах взят из индекса safetensors; сетевые интерфейсы 200/50 Gbit/s и цены экземпляров взяты из статьи о RLBoost.

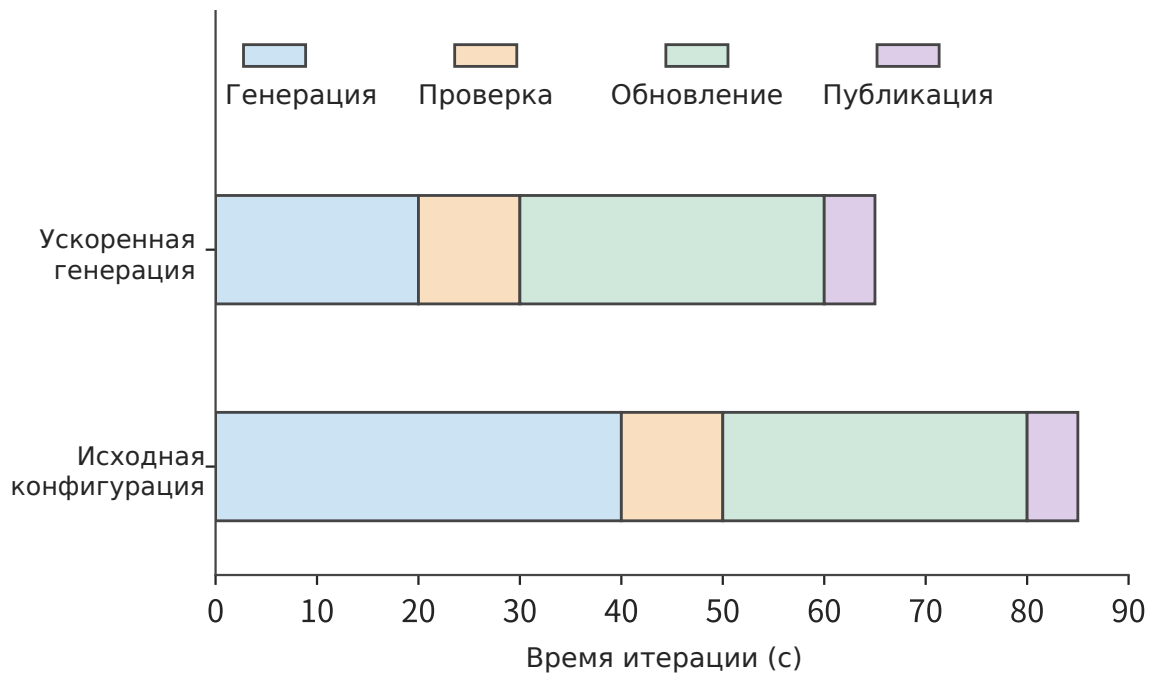


Рис. 11-31. Четыре этапа выполняются последовательно. Время генерации сокращается с 40 до 20 секунд, а остальные три этапа по-прежнему занимают в общей сложности 45 секунд, поэтому полное время итерации уменьшается с 85 до 65 секунд. Длина полос пропорциональна времени.

передать шести экземплярам rollout. В экспериментальной конфигурации RLBoost обучающий экземпляр с восемью H100 отправляет данные через интерфейс фронтенд-сети пропускной способностью 200 Gbit/s, подключённый к общей сети центра обработки данных, а не к высокоскоростному соединению между GPU. Фронтенд-интерфейс каждого экземпляра rollout с двумя ускорителями имеет пропускную способность 50 Gbit/s, причём все шесть передач используют общий исходящий канал отправителя. Передача весов одному получателю занимает не менее

$$16.38 / (50/8) \approx 2.62 \text{ s},$$

а через общий исходящий канал необходимо передать около 98,3 GB, поэтому передача весов всем шести экземплярам занимает не менее

$$T_{\text{all}} \geq \max \left(2.62, \frac{6 \times 16.38}{200/8} \right) \approx 3.93 \text{ s}.$$

Чтобы все шесть экземпляров получили веса, требуется не менее 3,93 секунды; первый экземпляр может получить свои 16,38 GB раньше. Затем получатель загружает веса в видеопамять GPU и подтверждает версию, после чего планировщик назначает ему запросы. PolyRL расширяет пул генерации RL с помощью временных ресурсов: сначала принимает полные веса в буфер CPU, а затем передаёт их группе GPU, использующей TP (см. раздел 6.2.2). Поэтому TP=2 лишь распределяет вычисления внутри экземпляра между двумя ускорителями, а каждый экземпляр по-прежнему получает извне 16,38 GB весов.

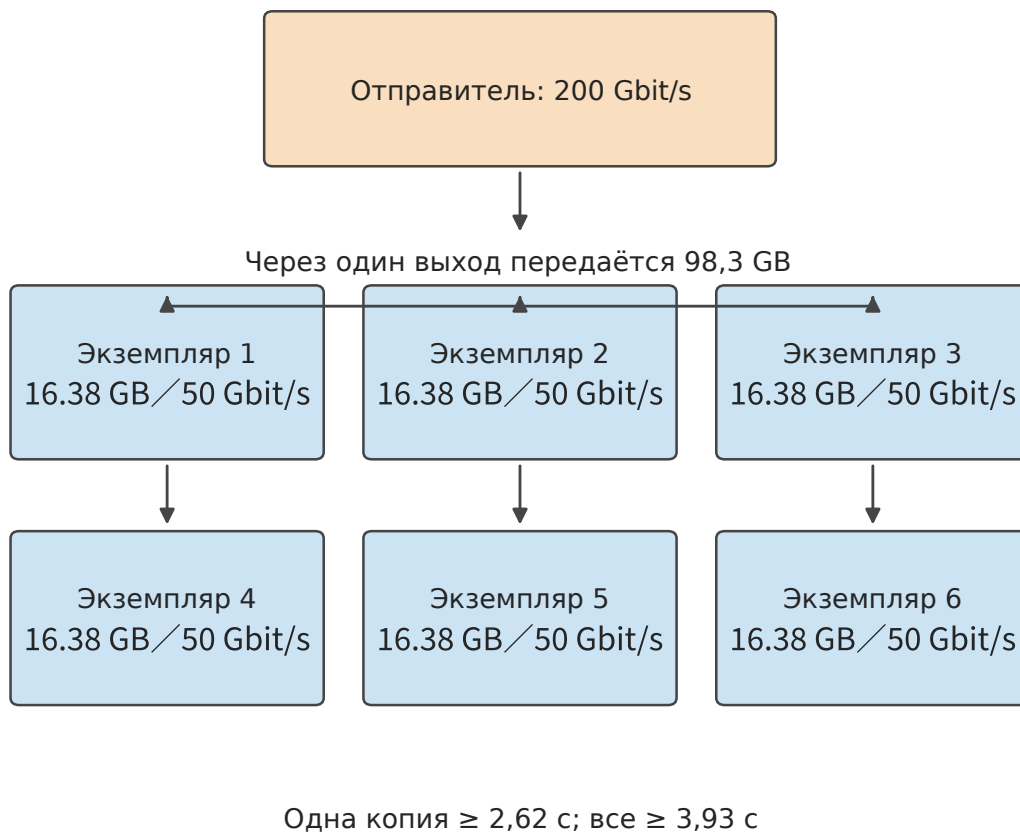


Рис. 11-32. Каждому принимающему экземпляру нужна полная копия весов Qwen3-8B объёмом 16,38 GB, а шесть передач используют общий исходящий канал пропускной способностью 200 Gbit/s. Пропускная способность каждого получателя составляет 50 Gbit/s, поэтому одна передача занимает не менее 2,62 секунды, а отправка суммарных 98,3 GB — не менее 3,93 секунды; каждый из шести блоков получателей на рисунке представляет отдельный экземпляр.

На рис. 11-32 показан общий исходящий канал, через который проходят шесть копий весов. С увеличением числа принимающих экземпляров растёт и общий объём передаваемых данных, но пропускная способность отправителя не увеличивается. После передачи веса ещё нужно загрузить в GPU, и всё это время подготовки вычитается из срока доступности временного экземпляра.

Получив ресурсы, временный экземпляр сначала должен завершить подготовку, и только оставшееся время можно использовать для генерации. Если от получения ресурсов до начала генерации проходит 8 секунд, а экземпляр доступен 20 секунд, на генерацию остаётся лишь 12 секунд; если он доступен только 5 секунд, его заберут ещё до начала работы. Пусть срок доступности равен L , а время подготовки и восстановления — T_0 ; тогда рабочее окно составляет $\max(L - T_0, 0)$. Чем дольше доступен экземпляр, тем меньшую долю занимают затраты на подготовку и тем большую долю времени можно использовать для фактической генерации.

При преждевременном отзыве временного экземпляра теряется не только ещё не использованная ёмкость, но и уже сгенерированное содержимое. Сохранение префикса ге-

нерации позволяет перенести выполненную работу на новый экземпляр: он считывает «исходный ввод + записанный вывод», выполняет prefill, создаёт KV-кэш и продолжает генерацию. При восстановлении группы запросов с общими параметрами сэмплирования PolyRL обрезает все ответы в группе до минимальной сохранённой длины. Если два ответа получили 4 000 и 1 000 токенов, от каждого используется по 1 000 токенов, всего повторно используются 2 000 токенов, а 3 000 токенов более длинного ответа приходится генерировать заново. После обрезки до одной длины дальнейшую генерацию группы запросов можно планировать совместно.

В одном сравнительном эксперименте с локально развёрнутой Qwen3-8B в формате 4-bit сохранение префикса позволило не генерировать повторно 96 токенов сэмплирования, но полное время выполнения запроса составило около 12,8 секунды, тогда как полное повторное выполнение заняло около 11,6 секунды. Сэкономленные токены относятся к этапу генерации, а полный путь включает также восстановление префикса и повторное планирование. В этом сравнении дополнительное время на восстановление префикса и повторное планирование превысило время, сэкономленное на генерации.²⁵

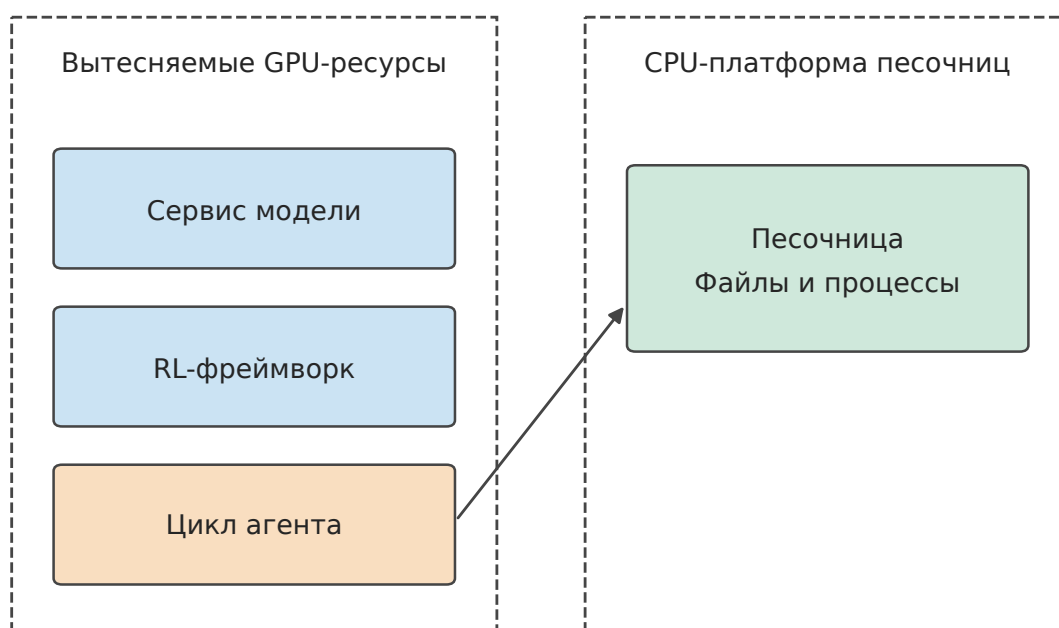
Целесообразность добавления временных экземпляров можно оценить и по стоимости. При исторических ценах, использованных RLBoost, обучающий экземпляр стоил около 84 долларов в час, а каждый из шести дополнительных экземпляров — около 5,3 доллара, что давало общую ставку около 116 долларов в час. Если все экземпляры оплачиваются в течение всего времени, пропускная способность при обработке той же серии обучающих задач должна вырасти примерно в $116/84 \approx 1.38$ раза, чтобы стоимость единицы работы снизилась. Если пропускная способность возрастёт лишь в 1,2 раза, время выполнения сократится, но стоимость увеличится примерно в 1,15 раза; при росте пропускной способности в 1,6 раза стоимость, напротив, снизится примерно до 0,86 исходной стоимости.²⁶

Выше сохранялся прогресс траектории на стороне модели. У rollout агента есть состояние и на стороне среды: изменённые в песочнице файлы, запущенные процессы, а также сохранённые циклом агента диалог и результаты инструментов. Здесь цикл агента — это контроллер, вызывающий модель, отправляющий вызовы инструментов и собирающий результаты. При вытеснении GPU-задания эти две части состояния имеют разную судьбу. Изначально в DSec цикл агента размещался в одном вытесняемом GPU-контейнере с сервисом модели и фреймворком RL (рис. 11-33). При вытеснении задания песочница сохранялась, но состояние цикла агента терялось вместе с контейнером. Во время восстановления обучающий фреймворк сначала восстанавливал записанный им прогресс rollout, а затем воспроизводил журнал команд: для уже выполненных вызовов инструментов возвращались записанные результаты без фактического повторного выполнения. Это предотвращало двойное исполнение одной команды, например повторное добавление содержимого в файл или повторную отправку запроса во внешнюю систему. Именно такой способ восстановления описан в строке «результаты инструментов» таблицы состояний из раздела 11.1.3.

Начиная с DeepSeek V4.1 цикл агента выполняется в worker-контейнере DSec и больше не использует вытесняемые ресурсы GPU; полное состояние rollout сохраняется им сов-

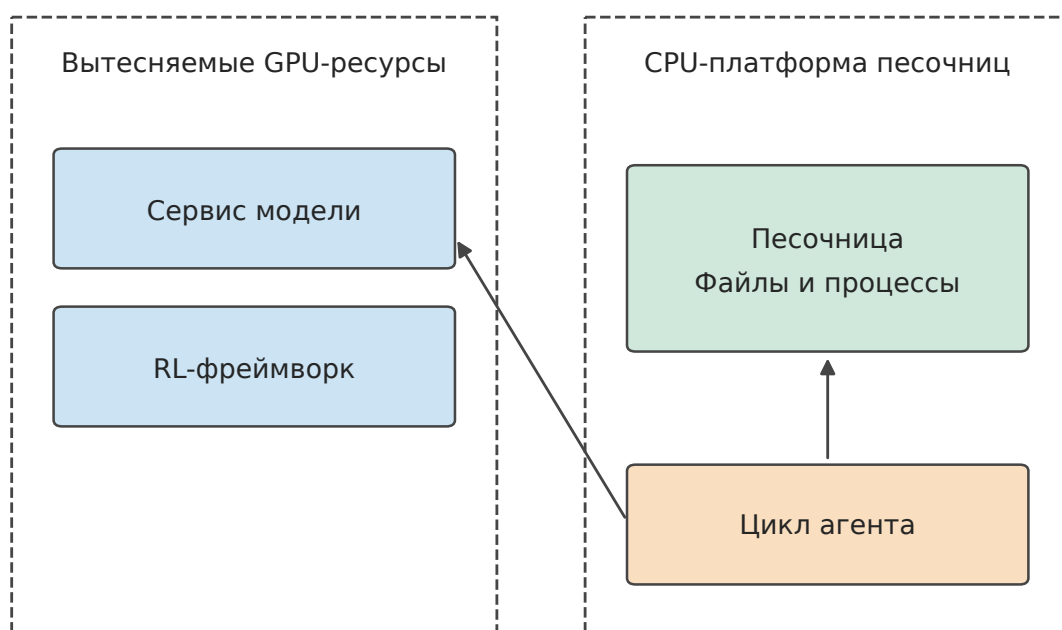
²⁵Эксперимент по восстановлению Qwen3-8B после вытеснения, отдельно фиксирующий количество токенов на этапах генерации, получения клиентом и восстановления.

²⁶Исследование готовности весов и полезной выработки, содержащее условия статьи о RLBoost и фиксированные пути исходного кода PolyRL. Размер весов Qwen3-8B в байтах взят из индекса safetensors; сетевые интерфейсы 200/50 Gbit/s и цены экземпляров взяты из статьи о RLBoost.



При вытеснении цикл агента теряется и воспроизводится по журналу команд

Рис. 11-33. Цикл агента находится в одном вытесняемом GPU-контейнере с сервисом модели и фреймворком RL. При вытеснении состояние цикла агента теряется, тогда как файлы и процессы в песочнице сохраняются; во время восстановления обе стороны снова согласуются посредством воспроизведения журнала команд.



После вытеснения достаточно переподключить GPU и продолжить

Рис. 11-34. Цикл агента перенесён в worker-контейнер на CPU-платформе, где вместе с песочницей сохраняется полное состояние rollout; после восстановления GPU-заданию достаточно повторно подключиться, чтобы продолжить работу.

местно с песочницей (рис. 11-34). После восстановления GPU-заданию достаточно повторно подключиться, чтобы продолжить работу, а обучающему фреймворку больше не нужно реализовывать воспроизведение. Фактически это изменение места хранения состояния: состояние размещается на ресурсах с наибольшим сроком жизни, поэтому отказ короткоживущих ресурсов не приводит к его потере. Часто вытесняемые GPU выполняют только вычисления, которые можно повторить, а состояние траекторий, которое должно переживать прерывания, хранится на невытесняемой CPU-платформе.²⁷

11.3.4 Пакет верификации, длинный хвост и освобождение ресурсов

В разделе 11.3.3 верификация рассматривалась как единый этап после генерации. В действительности примеры часто генерируются постепенно, поэтому верификацию можно начать раньше; обновление обучения должно ждать только последний результат во всём пакете верификации. При верификации кода CUDA его сначала необходимо скомпилировать на CPU, а затем выполнить на GPU. При планировании задач верификации помимо ёмкости CPU нужно учитывать влияние параллельного выполнения на измерение производительности GPU. Для такой верификации DSec использует отдельный исполнительный бэкэнд GPU с тремя приёмами: GPU разделяется посредством MIG на несколько изолированных экземпляров, благодаря чему одновременные измерения не влияют друг на друга; код сначала компилируется на CPU, а затем результат компиляции передаётся для выполнения на GPU, поэтому компиляция не занимает GPU; заранее подготавливается группа уже инициализированных процессов Python с импортированными необходимыми библиотеками, чтобы оператор выполнялся сразу после поступления запроса. Все три меры сокращают время, которое GPU тратит на работу, не связанную с измерением.²⁸

Если пример i поступает в момент a_i , после чего последовательно требует c_i секунд на компиляцию и g_i секунд на выполнение, то при игнорировании конкуренции за ресурсы нижняя граница времени завершения всего пакета равна

$$T_{\text{batch}} \geq \max_i (a_i + c_i + g_i).$$

Недостаток исполняющих процессов приводит к очередям; запуск процессов и передача данных также требуют времени. Все эти затраты дополнительно отодвигают момент завершения всего пакета. Если отправлять уже сгенерированные примеры на верификацию как можно раньше, она сможет перекрываться с дальнейшей генерацией. В обоих вариантах на рис. 11-35 и 11-36 обработка при верификации занимает 30 се-

²⁷DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

²⁸DeepSeek-AI и Университет Цинхуа, DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale, arXiv 2609.22978, 2026 (PDF). Статистика нагрузки приведена в §2.4 и §4; серверная часть и изоляция — в §2.2, §3.3 и §6.4–6.5; загрузка по требованию, совместное использование и освобождение памяти, а также QoS CPU — в §5.2–5.3, §8.2 и §8.4–8.5; размещение и серверная часть проверки на GPU — в §7; распределение ответственности за приостановку и цикл агента — в §6.2–6.3. Пересчёты количества песочниц, скорости их создания на каждом узле и преимуществ приостановки приведены в заметках по этому примеру.

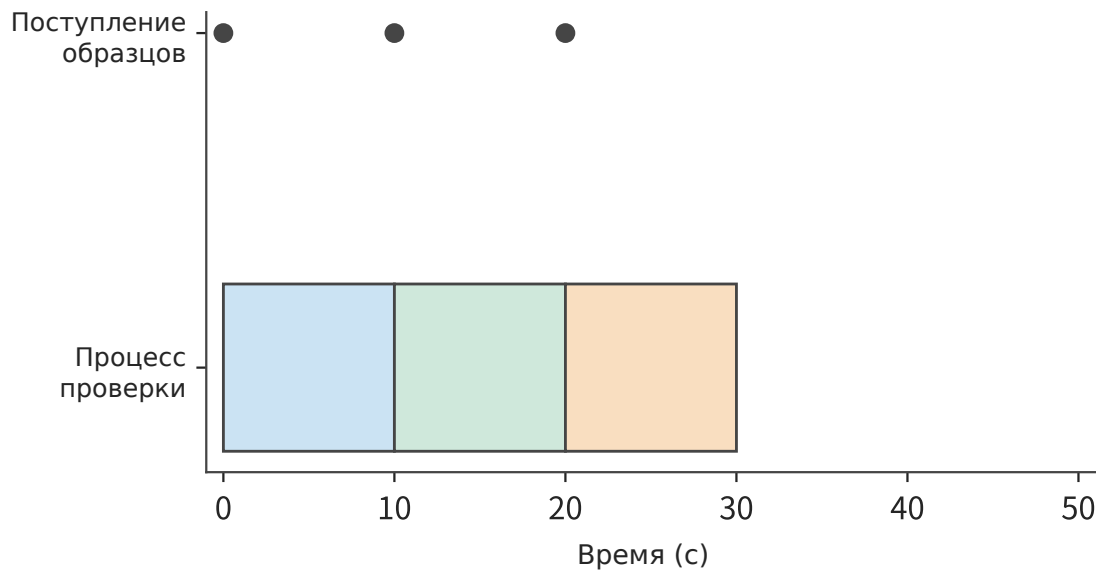


Рис. 11-35. Три примера поступают соответственно в моменты 0, 10 и 20 с и немедленно отправляются единственному процессу верификации, который обрабатывает каждый пример за 10 с. Весь пакет завершается в момент 30 с. Точками обозначены моменты поступления.

кунд, но моменты завершения различаются на 20 секунд исключительно из-за времени отправки.

Отправка примеров по одному устраняет простой в ожидании генерации всего пакета, но время завершения пакета всё равно может определяться самой медленной верификацией. Чтобы оценить, сколько ещё будут выполняться такие задачи длинного хвоста, прогноз следует обновлять с учётом уже прошедшего времени выполнения. Предположим, среди имеющихся измерений девять верификаций занимали по 1 секунде, а одна — 100 секунд; среднее время равно 10,9 секунды. Если некоторая верификация выполняется уже 10 секунд и ещё не завершилась, вычитание 10 из 10,9 даст прогноз оставшегося времени всего в 0,9 секунды. Однако в этом дискретном распределении такая верификация обязательно относится к 100-секундной задаче, поэтому ей осталось 90 секунд. Следовательно, оставшееся время нужно оценивать при условии, что задача ещё не завершилась: $E[S - e \mid S > e]$, где S — полное время выполнения верификации, e — уже прошедшее время, а $E[\cdot \mid S > e]$ обозначает среднее только по ещё не завершившимся примерам. Алгоритм планирования DistRS использует именно такое условное оставшееся время, а не постоянное общее среднее.²⁹

После оценки оставшегося времени каждого примера можно соответствующим образом планировать другие верификации, стараясь не увеличивать время ожидания всего пакета. Предположим, один пример в пакете завершится не раньше момента 120 секунд. Если нужно запланировать работу продолжительностью не более 100 секунд, её можно запустить не позднее момента 20 секунд, использовав предшествующий промежуток и не задержав завершение пакета; если подготовка среды занимает 5 секунд, её нужно начать не позднее момента 15 секунд. Из момента завершения всего пакета можно вывести самые поздние моменты запуска отдельных работ. На их основе планировщик откладывает несрочные работы и сначала предоставляет ресурсы пакетам с более ранним сроком

²⁹ Анализ планирования проверки DistRS и реализации `verl`.

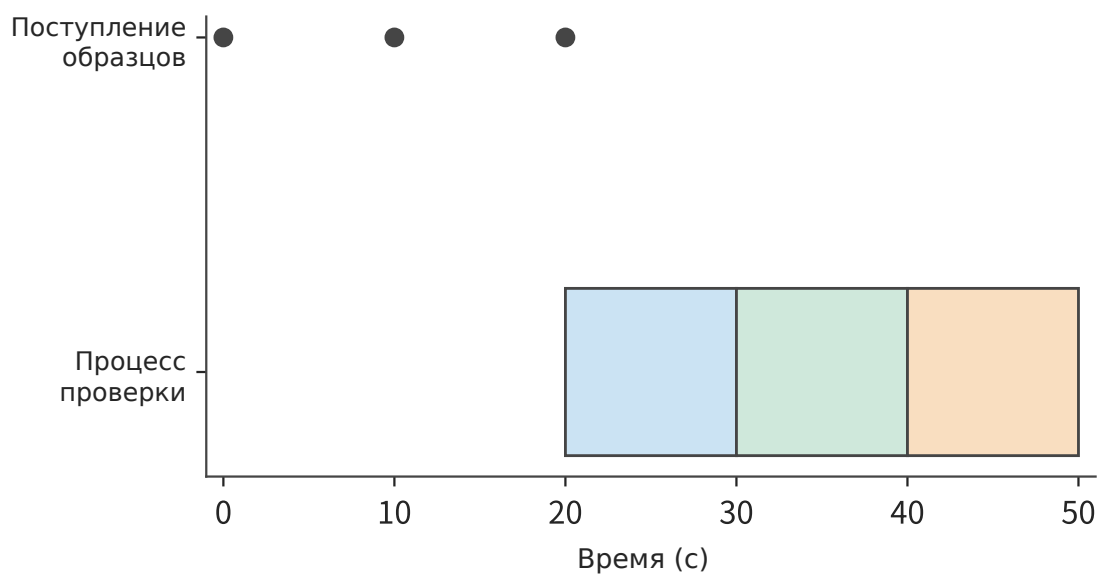


Рис. 11-36. Сначала система ждёт, пока все три примера будут готовы в момент 20 s, а затем последовательно верифицирует их, завершая работу в момент 50 s. Выполняется та же 30-секундная работа верификации, что и на предыдущем рисунке; различается только время отправки.

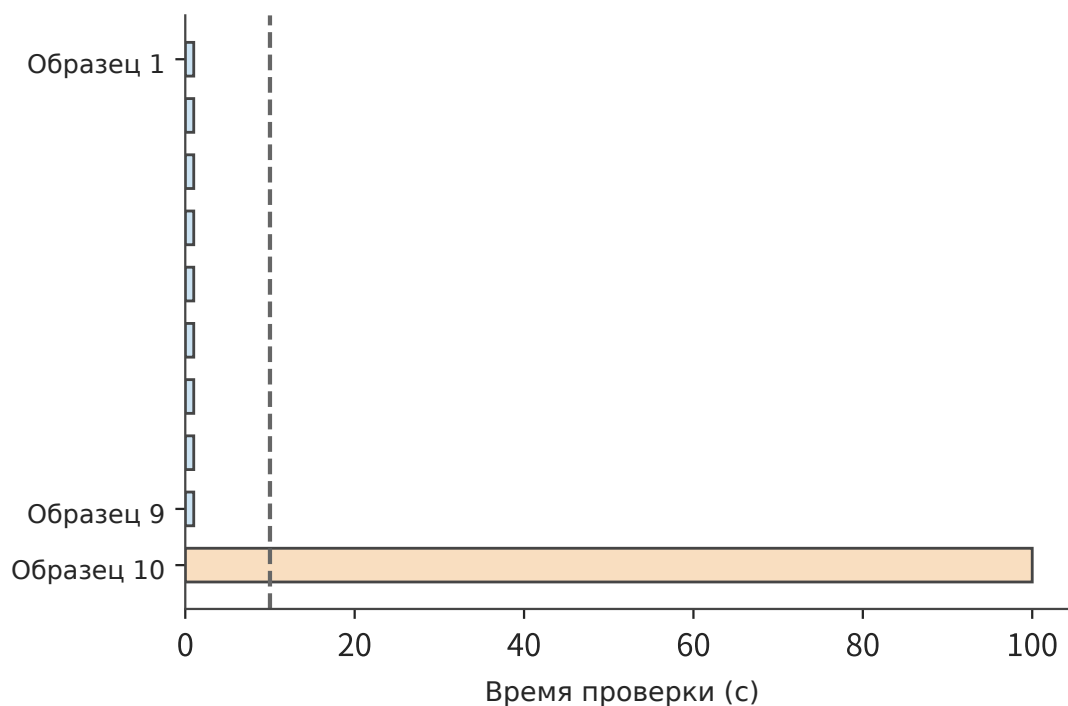


Рис. 11-37. Девять верификаций занимают по 1 s, одна — 100 s. Пунктиром отмечены уже прошедшие 10 s; к этому моменту незавершённый пример может относиться только к 100-секундной категории, поэтому оставшееся время равно 90 s.

завершения.

Откладывание несрочной верификации может сократить использование ресурсов верификации, но любая задержка завершения всего пакета также продлевает ожидание обучающей группы. Предположим, верификация использует на 60 H100-с меньше, но из-за этого обучающая группа, по-прежнему удерживающая 64 H100 в восьми HGX H100, ждёт на 2 секунды дольше. Дополнительные затраты обучающей группы составляют $64 \times 2 = 128$ H100-с, а чистый прирост по всей системе — 68 H100-с. Если обучающая группа содержит лишь 16 ускорителей в двух узлах, дополнительное ожидание составляет 32 H100-с, и система, напротив, экономит 28 H100-с. Поэтому целесообразность сокращения ресурсов верификации зависит от размера заблокированной обучающей группы.

После планирования времени завершения нужно также убедиться, что завершённые или превысившие лимит времени задачи действительно прекратили выполнение и ресурсы можно передать следующей задаче. В одном контролируемом эксперименте после возврата по тайм-ауту ожидающей стороной фоновый поток продолжал работать ещё около 130 ms; после принудительного завершения дочернего процесса его выход наблюдался примерно через 0,8 ms. Если сразу после тайм-аута передать тот же ресурс следующей задаче, только сообщив о тайм-ауте, но не завершив фоновый поток, обе задачи будут использовать ресурс одновременно. При верификации производительности CUDA такая конкуренция может также изменить измеренное время выполнения и вознаграждение.³⁰

11.4 Выбор сервиса модели и стоимость вызовов

11.4.1 Качество выполнения задач и бюджет рассуждения

Пересоздание окружения в каждом раунде уже снизило приведённую в начале главы потребность платформы в памяти с 600 GiB до 180 GiB, однако время выполнения задачи по-прежнему составляет 30 секунд, превышая целевое значение в 24 секунды. Поэтому необходимо изменить вызов модели на критическом пути. Если уменьшить число сессий, одновременно обслуживаемых каждой репликой V4-Flash, с 32 до 16, время на каждый выходной токен сократится с 25,3 ms до 16,9 ms, полный вызов с 355 токенами — с 9 до 6 секунд, а три раунда — с 30 до 21 секунды; модель при этом не меняется, как и доля успешно выполненных задач. Если окружение постоянно находится в памяти, занятость на одну задачу также снижается с 60 до 42 GiB-с, а потребность всей платформы в памяти — с 600 до 420 GiB, что всё ещё превышает 358 GiB; если же окружение пересоздаётся в каждом раунде, его по-прежнему нужно сохранять лишь на время подготовки и выполнения каждого раунда, поэтому суммарная занятость остаётся равной 18 GiB-с. Таким образом, выбор сервиса модели одновременно влияет на продолжительность задачи и потребность окружения в памяти.³¹

³⁰Эксперимент с тайм-аутами и освобождением ресурсов, запись планирования двух пакетов. В эксперименте используются контролируемые процессы CPU.

³¹Сквозные входные данные платформы и примера 11-9, поэтапное расписание, стоимость, условие равенства стоимости и повторные расчёты приведены в сквозных данных проектирования и программе проверки. Время вызова модели основано на приведённой в книге оценке работы DeepSeek V4-Flash на четырёх B200 с контекстом 200K на сеанс: при 32 сеансах на реплику вывод одного токена занимает 25,3 ms, а при 16 сеансах — 16,9 ms; оба значения включают распределённые затраты на обработку входных данных и восстановление контекста (результаты сравнения, полные данные). Вызов с 355 токенами занимает соответственно около 8,99 s и 6,01 s; в проекте используются значения 9 s и 6 s. Цена B200 в 6,79 доллара за карту в час взята из снимка страницы цен Runpod. 48 ядер m5d.metal при отключённой многопоточности и 384 GB памяти

Выбор модели также определяет структуру стоимости каждого вызова. Время и стоимость одного вызова модели складываются из трёх частей: обработки входных данных, генерации токенов рассуждения и генерации видимого вывода. Бюджет рассуждения определяет допустимую длину внутреннего рассуждения, а фактическое потребление — сумму, которую необходимо заплатить за каждый вызов. Сначала при условии, что вывод в обоих случаях проходит тесты, сравним стоимость одного вызова для двух настроек длины рассуждения, а затем — стоимость выполнения трёх раундов задачи двумя сервисами.

Пример 11-5. Насколько уменьшится общая стоимость, если сократить длину рассуждения в десять раз? Предположим, что обе настройки длины рассуждения позволяют пройти одинаковые тесты, каждый вызов получает 10 000 входных токенов и выдаёт 200 видимых токенов, а фактическое рассуждение занимает соответственно 1 000 и 100 токенов. Возьмём стандартные цены API Claude Sonnet 5: 2 доллара за миллион входных токенов, 10 долларов за миллион сгенерированных токенов, причём токены рассуждения тарифицируются как сгенерированные. Тогда стоимость одного вызова в долларах составит соответственно

$$C_{1000} = \frac{10000 \times 2 + (1000 + 200) \times 10}{10^6} = 0.032,$$

$$C_{100} = \frac{10000 \times 2 + (100 + 200) \times 10}{10^6} = 0.023.$$

Число токенов рассуждения уменьшилось в десять раз, но общая стоимость снизилась лишь примерно на 28%. В исходной стоимости 0,020 доллара приходилось на входные данные, а 0,002 доллара — на видимый вывод; обе составляющие остались неизменными. Снизилась только стоимость рассуждения: с 0,010 до 0,001 доллара. Следовательно, возможная экономия от сокращения рассуждения определяется долей рассуждения в исходной стоимости.³²

На рис. 11-38 отдельно показано, где возникает экономия. В этом сравнении предполагается, что обе настройки позволяют выполнить задачу; если после сокращения рассуждения потребуются повторные попытки, сэкономленную оранжевую часть необходимо сравнивать с полной стоимостью дополнительных вызовов.

Предположим, каждый из 100 вызовов экономит 0,009 доллара, что в сумме даёт 0,9 доллара; при этом 30 вызовов повторяются из-за исчерпания бюджета, а каждая повторная попытка стоит 0,032 доллара, что в сумме добавляет 0,96 доллара. В результате общая стоимость, наоборот, возрастает на 0,06 доллара. Сокращение бюджета рассуждения может увеличить число повторных попыток. Поэтому при сравнении полной стоимости задачи необходимо учитывать как экономию на отдельных вызовах, так и затраты на дополнительные попытки.³³

взяты из тестового окружения, описанного в разделе 5 статьи о Firecracker; конфигурация 1 vCPU и 2 GiB, а также цены CPU и памяти взяты из снимка страницы цен E2B. Подготовка по горячему пути в совокупности занимает 2 секунды и 0,1 CPU-с, а вероятность прохождения приёмочной проверки равна 95% — эти значения заданы условием; сохранение результата и очистка рабочего состояния включены в одну секунду выполнения инструмента. Пример с ёмкостью шаблона отдельно поясняет загрузку содержимого.

³²Исследование планирования, маршрутизации моделей и облачных окружений, охватывающее ASI, RLBoost, DistRS и SpecBox. Исследование ASI охватывает шесть месяцев и 155 410 GPU; доля распределения измеряет принадлежность устройств. Цена токенов взята из снимка страницы цен Claude API.

³³Запись бюджета рассуждений и качества, проверка после увеличения бюджета рассуждений. В записях отдельно проверяется, завершилась ли генерация естественным образом и соответствует ли входное состояние требованиям.

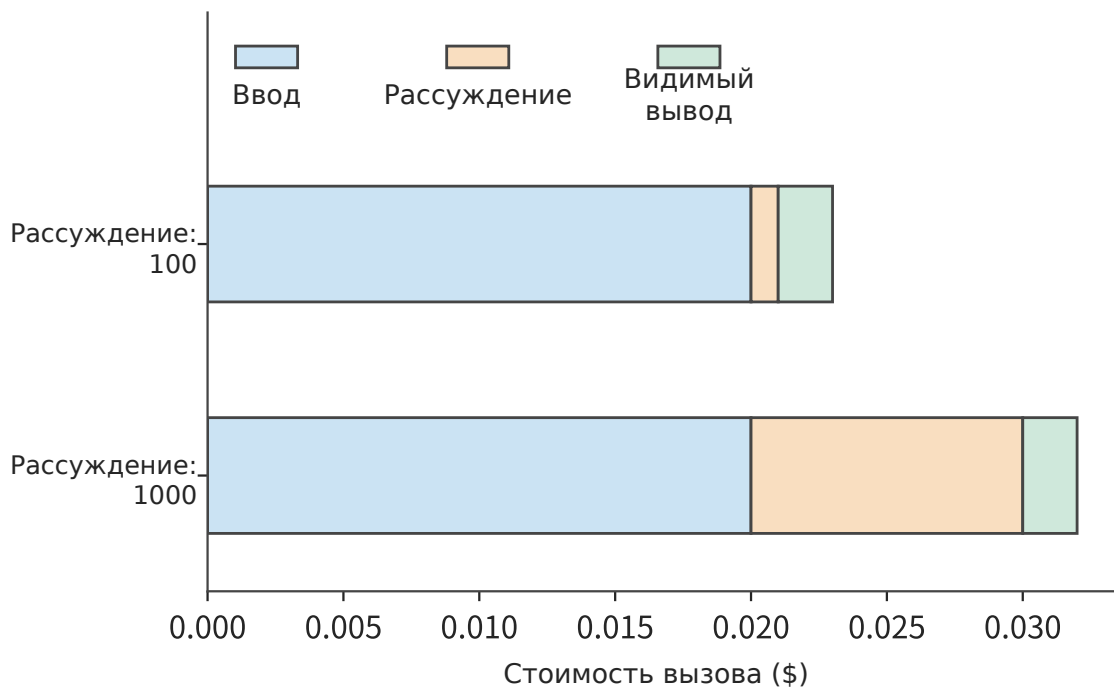


Рис. 11-38. Входные данные и видимый вывод двух вызовов одинаковы, изменяется только длина рассуждения. Оранжевая составляющая стоимости сокращается в десять раз, а синяя и зелёная остаются неизменными, поэтому общая стоимость снижается с 0,032 до 0,023 доллара, то есть примерно на 28%. Цены и количество токенов приведены в примере 11-5.

11.4.2 Путь обслуживания, кэширование префиксов и фактическое потребление

В разделе 11.4.1 стоимость снижалась за счёт уменьшения объёма генерации; основная возможность сэкономить на входных данных связана с повторяющимися префиксами: многораундовые вызовы в рамках одной задачи обычно повторно передают общий промпт и историю предыдущего диалога. Запрос к модели поступает из контроллера в единую точку входа сервиса, а затем передаётся выбранному бэкенду модели. Под бэкендом здесь понимается экземпляр инференса или поставщик сервиса, фактически выполняющий инференс модели. Точка входа отвечает за аутентификацию, маршрутизацию и ограничение частоты запросов, а бэкенд — за обработку входных данных и генерацию вывода. Указанные в начале главы 30 вызовы в секунду создают постоянную нагрузку на этот путь; доля входных данных, которую можно использовать повторно, определяется общим промптом и историей задачи, передаваемыми с каждым вызовом.

Разделим входные данные одного вызова на не пересекающиеся между собой обычные входные данные I_u , входные данные для создания кэша I_w и входные данные, прочитанные из кэша, I_h . Пусть тарифицируемое число сгенерированных токенов равно O , а соответствующие цены за миллион токенов — p_u, p_w, p_h, p_o . Тогда

$$C_{\text{call}} = \frac{I_u p_u + I_w p_w + I_h p_h + O p_o}{10^6} + C_{\text{storage}} + C_{\text{tool}}.$$

Каждый входной токен в рамках одного вызова относится только к одной из трёх категорий: обычным входным данным, созданию кэша или чтению из кэша. Если распре-

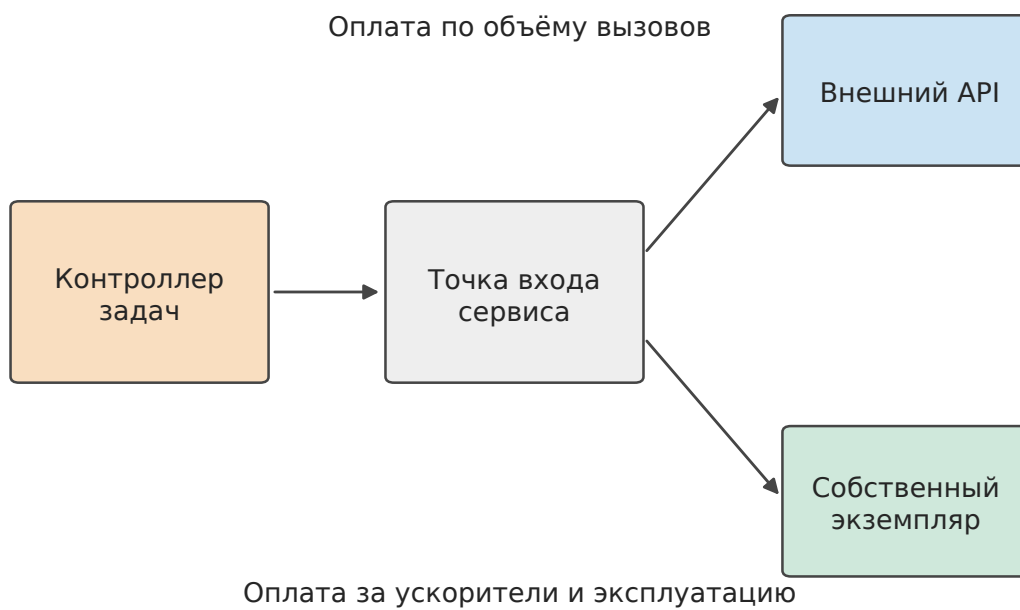


Рис. 11-39. Два способа использования сервиса модели для одной задачи и методы их тарификации. Внешний API взимает плату за фактическое потребление при вызовах, а самостоятельно развёрнутые реплики тарифицируются по зарезервированным устройствам и эксплуатационным расходам; на обоих путях стоимость относится к задаче и попытке. Входные данные разделяются на обычную обработку, создание кэша и чтение из кэша, а генерация учитывается отдельно.

делить возвращаемые сервисом сведения о потреблении (usage) по этим трём категориям, умножить каждое значение на соответствующую цену и сложить результаты, получится стоимость входных данных. Число сгенерированных токенов O представляет собой сумму токенов рассуждения и токенов видимого вывода.

Пример 11-6. Сколько можно сэкономить на входных данных, кэшируя общий префикс между раундами? В каждом раунде имеется общий префикс из 8 000 токенов и 2 000 токенов новых входных данных, которые не используются повторно. Согласно стандартным ценам Claude Sonnet 5, миллион обычных входных токенов стоит 2 доллара, первоначальное создание пятиминутного кэша — 2,5 доллара, а чтение из кэша — 0,2 доллара. При первом вызове кэш создаётся, а следующие девять вызовов полностью попадают в него до истечения срока действия. Без кэша стоимость входных данных за десять раундов составляет 0,20 доллара; с кэшем стоимость в долларах равна

$$\frac{8000 \times 2.5 + 2000 \times 2}{10^6} + 9 \frac{8000 \times 0.2 + 2000 \times 2}{10^6} = 0.0744.$$

В этом примере генерация и выполнение инструментов во всех десяти раундах остаются одинаковыми, поэтому вся разница в стоимости обусловлена входными данными. Первое использование кэша стоит 0,024 доллара, что на 0,004 доллара дороже обычной обработки стоимостью 0,020 доллара; каждый последующий раунд стоит лишь 0,0056 доллара, что на 0,0144 доллара дешевле обычной обработки. Поэтому надбавка за первоначальное создание окупается уже при втором обращении. При общем числе обращений n использование кэша выгоднее, если $np_u > p_w + (n-1)p_h$. Чем дольше срок действия кэша, тем больше вероятность окупить дополнительные затраты на первоначальное создание за счёт повторных чтений.³⁴

Размер экономии от кэширования зависит также от того, какие именно запросы попадают в кэш. Рассмотрим ещё два вызова: длина префикса одного составляет 1 000 токенов, другого — 9 000 токенов. Если в кэш попадает только короткий префикс, доля запросов с попаданием составляет 50%, но число прочитанных из кэша токенов — лишь 10% общей длины префиксов двух вызовов. Если в кэш попадает только длинный префикс, доля запросов с попаданием по-прежнему равна 50%, однако доля чтения возрастает до 90%. При тарификации по токенам второй случай позволяет сэкономить на обработке входных данных значительно больше, чем первый. Вес каждого попадания определяется длиной входных данных.

11.4.3 Очереди, ограничение частоты запросов и маршрутизация моделей

Повторное использование кэша возможно только тогда, когда запрос поступает на бэкэнд, где сохранён соответствующий префикс. Поэтому выбор бэкэнда влияет и на долю попаданий, и на время ожидания в очереди. Ради использования уже существующего кэша префикса иногда целесообразно подождать дольше. Предположим, очередь бэкэнда А, на котором работает та же модель, занимает 200 ms, а обработка попавшего в кэш префикса — 50 ms; очередь бэкэнда В занимает 20 ms, а обработка без попадания — 500 ms; общее сетевое время составляет 50 ms, а последующая генерация одинакова. До начала генерации А затрачивает 300 ms, а В — 570 ms, поэтому А опережает В на 270 ms.

³⁴Исследование планирования, маршрутизации моделей и облачных окружений, охватывающее ASI, RLBoost, DistRS и SpecBox. Исследование ASI охватывает шесть месяцев и 155 410 GPU; доля распределения измеряет принадлежность устройств. Цена токенов взята из снимка страницы цен Claude API.

Попадание в кэш экономит 450 ms обработки, но добавляет 180 ms ожидания в очереди, что даёт ровно 270 ms чистого выигрыша. Когда время ожидания в очереди А возрастает до 470 ms, затраты времени до начала генерации становятся одинаковыми.

Указанные в начале главы 30 вызовов модели в секунду также ограничены доступной параллельной мощностью. Если один вызов длится 9 секунд, одновременно выполняется в среднем 270 вызовов; при сокращении до 6 секунд их число уменьшается до 180. Однако сокращение времени обслуживания достигается за счёт уменьшения batch для каждой реплики: каждая реплика одновременно обслуживает лишь 16 сессий, поэтому 10 реплик располагают только 160 слотами параллельного выполнения и не могут вместить 180 вызовов — число реплик необходимо увеличить до 12, то есть до 48 карт B200. Затраты времени GPU на один вызов также возрастают с $4 \times 9/32 = 1.125$ B200-с до $4 \times 6/16 = 1.5$ B200-с. API с оплатой по фактическому потреблению обычно отдельно ограничивают количество запросов и количество токенов за единицу времени; если объём входных или выходных данных каждого вызова увеличивается, лимит токенов может быть исчерпан раньше лимита запросов. При выборе сервиса модели необходимо отдельно сравнивать время ожидания запросов в очереди, число параллельных вызовов и пропускную способность обработки токенов, чтобы определить фактор, ограничивающий скорость выполнения задач.

Пример 11-7. Когда попадания в кэш и вероятность успеха компенсируют более высокую цену сервиса? Отправим в Claude Haiku 4.5, обозначенный А, и Claude Sonnet 5, обозначенный В, по 1 000 задач, каждая из которых требует одного вызова. Каждый вызов получает 20 000 входных токенов, включая общий префикс из 19 000 токенов и 1 000 токенов новых входных данных. Кэш префикса уже создан, дополнительных затрат на создание и хранение кэша, инструменты и прогрев нет, а вероятность прохождения приёмки задачей не зависит от попадания в кэш. Возьмём стандартные цены API обеих моделей; число токенов рассуждения и вероятность успеха заданы в условии.³⁵

Параметр	А: Haiku 4.5	В: Sonnet 5
Цена обычных входных данных, долларов за миллион токенов	1	2
Цена чтения из кэша, долларов за миллион токенов	0.1	0.2
Цена генерации, долларов за миллион токенов	5	10
Фактическое число токенов рассуждения	1 800	100
Число токенов видимого вывода	200	200
Вероятность успеха при одинаковых правилах приёмки	0.80	0.98

Префикс А всегда попадает в кэш, стоимость каждого вызова составляет 0,0129 доллара, а средняя стоимость успешно выполненной задачи — $0.0129/0.8 \approx 0.0161$ доллара. При попадании в кэш стоимость каждого вызова В составляет 0,0088 доллара, а без

³⁵Фиксированные результаты routing-cost, результаты расчёта точки равенства и полный вывод. Цены Haiku 4.5 и Sonnet 5 взяты из снимка страницы цен Claude API; расчёт предполагает, что вероятность прохождения приёмочной проверки не зависит от попадания в кэш, а количество успешных задач рассчитывается по заданной вероятности.

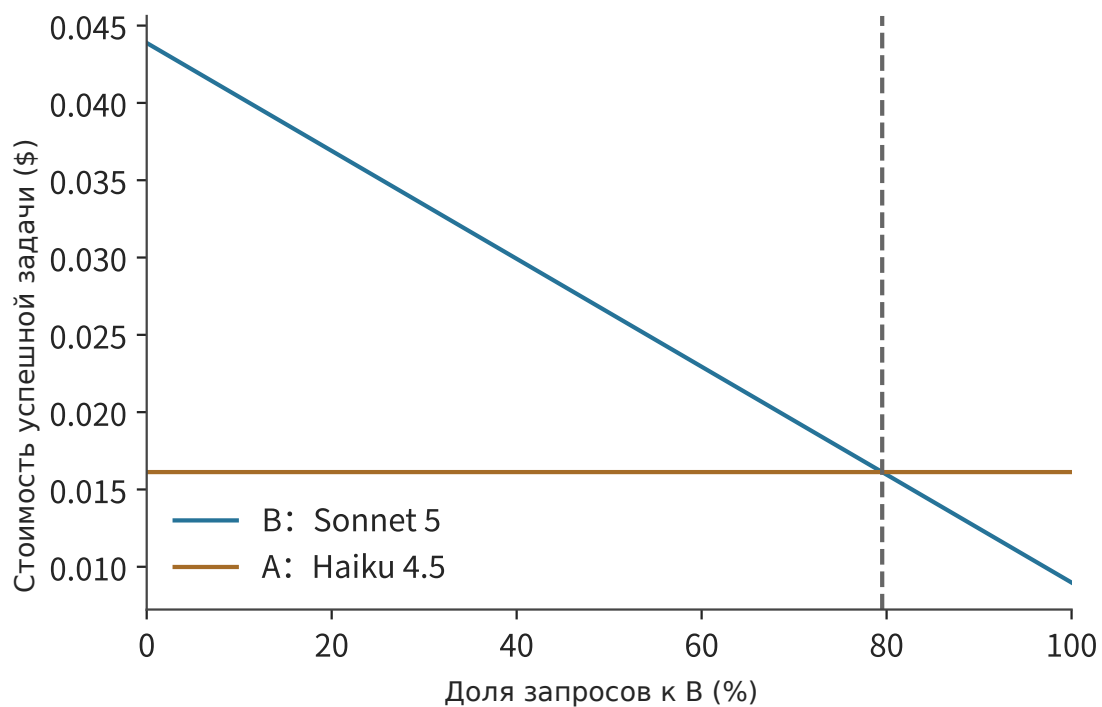


Рис. 11-40. Стоимость успешной задачи В уменьшается с ростом доли попаданий и примерно при 79,5% становится равна стоимости А. Сначала общая стоимость каждого варианта делится на соответствующее число успешно выполненных задач; на этом рисунке сравнивается только стоимость, а ограничение по сроку отдельно добавлено на следующем рисунке.

попадания — 0,0430 доллара. Обозначим долю запросов В с попаданием в кэш через h . Тогда

$$C_B(h) = \frac{0.0088h + 0.0430(1 - h)}{0.98}.$$

Приравняв среднюю стоимость В к средней стоимости А, получим $h \approx 79.5\%$. Все цены за токен у В вдвое выше, чем у А, но при высокой доле попаданий В оказывается дешевле благодаря меньшему фактическому объёму генерации и более высокой вероятности успеха. Если доля попаданий снижается до 50%, стоимость успешной задачи В возрастает примерно до 0,0264 доллара, и А становится дешевле.

Теперь дополнительно потребуем, чтобы как минимум 90% отправленных задач прошли приёмку не более чем за 6 секунд. Пусть полная продолжительность запроса составляет 10 секунд для А, 4 секунды при попадании В в кэш и 12 секунд при отсутствии попадания; во всех случаях время уже включает ожидание в очереди и генерацию. А не укладывается в срок; для В должно выполняться $0.98h \geq 0.90$, то есть $h \geq 91.8\%$.

В этом примере длина префикса каждого вызова составляет 19 000 токенов, поэтому доля запросов с попаданием h также равна доле попавших в кэш токенов внутри префикса; относительно полных входных данных объёмом 20 000 токенов доля чтения из кэша составляет $0.95h$. Точка пересечения стоимости находится примерно на уровне 79,5%, а пороговое бизнес-требование — примерно на уровне 91,8%. При движении вправо от низкой доли попаданий В сначала становится дешевле и лишь затем начинает удовлетворять требованию своевременного выполнения. Если нанести

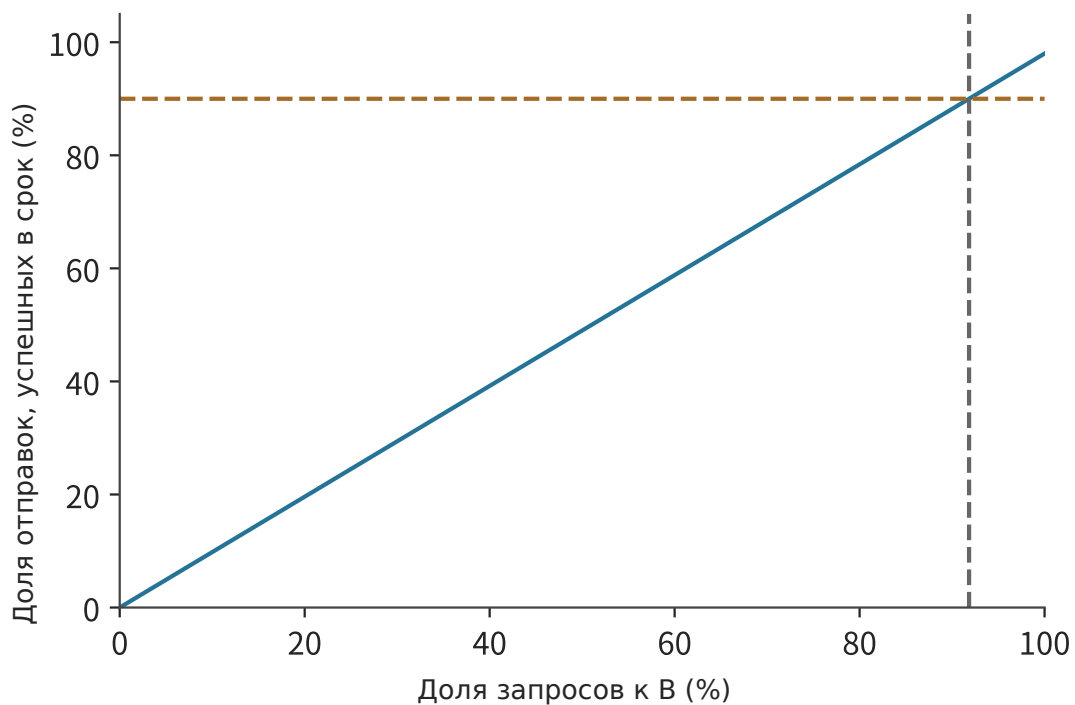


Рис. 11-41. Только четырёхсекундный путь В с попаданием укладывается в шестисекундный срок; с учётом 98-процентной вероятности прохождения приёмки доля своевременно выполненных задач равна $0.98h$. Для достижения целевого значения 90% требуется h не менее примерно 91,8%.

целевой показатель сервиса на кривую стоимости, можно непосредственно определить итоговую область допустимых вариантов.

11.4.4 Сравнение стоимости собственного сервиса, API с оплатой по фактическому потреблению и подписки

Сравнение маршрутизации позволяет определить, какие сервисы моделей одновременно удовлетворяют требованиям к качеству, стоимости и срокам. После выбора сервиса необходимо решить, развёртывать ли его самостоятельно, оплачивать каждый вызов или приобрести подписку. Выбор модели определяет сервисы, необходимые для каждой задачи, а способ закупки — порядок их оплаты. Для собственной инфраструктуры сначала оплачиваются оборудование и зарезервированные мощности, после чего расходы распределяются между задачами; API с оплатой по фактическому потреблению тарифицируется по вызовам; подписка оплачивается ежемесячно или с другой периодичностью и предоставляет право пользоваться соответствующим продуктом. Общий вопрос для всех трёх способов: как меняются совокупные расходы с объёмом использования при одинаковом количестве задач.

Пусть за фиксированный учётный период зарезервированные мощности и постоянные расходы собственного сервиса составляют F , а дополнительные затраты на каждую отправленную задачу — v ; средняя стоимость одной задачи через API равна c . Если вероятность успеха одинакова и производительности обоих вариантов достаточно, собственный сервис оказывается выгоднее при N задачах, когда

$$F + Nv < Nc, \quad N > \frac{F}{c - v} \quad (c > v).$$

Если $c \leq v$, стоимость одной задачи через API уже не превышает дополнительные затраты собственного сервиса, а постоянные вложения положительны, поэтому API выгоднее при любом объёме использования. Если вероятности успеха различаются, то после деления соответственно на Np_{self} и Np_{api} сравнивается уже стоимость одной успешно выполненной задачи.

Ниже сравним два способа оплаты для одной модели и одного типа GPU: batch size и качество выполнения задач в обоих случаях одинаковы, различается только метод тарификации. В варианте с резервированием арендуются четыре карты на месяц, то есть на 720 часов, по публичной цене B200 Pod облачной GPU-платформы Runpod — 6,79 доллара за карту в час. B200 Pod представляет собой тарифицируемый по часам экземпляр GPU в монопольном использовании. Получаем $F = 4 \times 720 \times 6.79 = 19,555.2$ доллара; поскольку GPU уже оплачен на весь месяц, дополнительные затраты v на одну задачу равны нулю. В варианте с оплатой по фактическому потреблению используется B200 Serverless worker той же платформы с посекундной тарификацией по цене 8,64 доллара за карту в час. Как и API с оплатой по фактическому потреблению, он оплачивается по мере использования и также выполняет batch из 32 сессий. Для задачи из начала главы каждый из трёх вызовов занимает 1,125 B200-с, поэтому стоимость по фактическому потреблению составляет $c = 3 \times 1.125 \times 8.64 / 3600 = 0.0081$ доллара.

Равенство стоимости достигается при объёме $19,555.2 / 0.0081 \approx 2,41$ млн задач. Каждая задача занимает одну сессию на 27 секунд, поэтому четыре B200 за месяц могут обработать не более $32 \times 2,592,000 / 27 = 3,072$ млн задач. Точка пересечения соответствует загрузке примерно 78,6%, что в точности равно отношению двух цен 6.79/8.64. При 1 млн задач резервирование по-прежнему стоит 19 555,2 доллара, тогда как оплата по фактическому потреблению — лишь 8 100 долларов; при 3 млн задач оплата по фактическому потреблению составляет 24 300 долларов, поэтому резервирование становится выгоднее. При малом объёме использования постоянные вложения трудно распределить между задачами, а при большом объёме они компенсируются меньшими дополнительными затратами.

На рис. 11-42 стоимость резервирования показана горизонтальной линией, высота которой определяется арендной платой за полный месяц, а правый конец — пределом производительности этой группы GPU; стоимость по фактическому потреблению начинается с нуля, а её наклон определяется временем GPU, затрачиваемым на одну задачу. После повышения эффективности выполнения те же четыре B200 могут обрабатывать больше задач в месяц, поэтому линия резервирования продлевается вправо, а наклон линии оплаты по фактическому потреблению уменьшается; точка пересечения по-прежнему соответствует той же загрузке, равной отношению двух цен.

Возможное повышение эффективности выполнения зависит от того, на что расходуется время. Снова рассмотрим развёртывание DeepSeek V4-Flash на четырёх B200, одновременно обслуживающее 32 сессии с контекстом в 200K токенов для каждой; число сессий и цена аренды GPU остаются неизменными. Каждый выходной токен требует примерно 25,3 ms, из которых 15,9 ms занимает поточный decode каждого токена, а остальные 9,4 ms — обработка входных данных и восстановление контекста. Если фактическая скорость этапа decode удваивается, время на один выходной токен снижается до $15.9 / 2 + 9.4 \approx 17.4$ ms, общее ускорение составляет примерно 1,46 раза, а стоимость

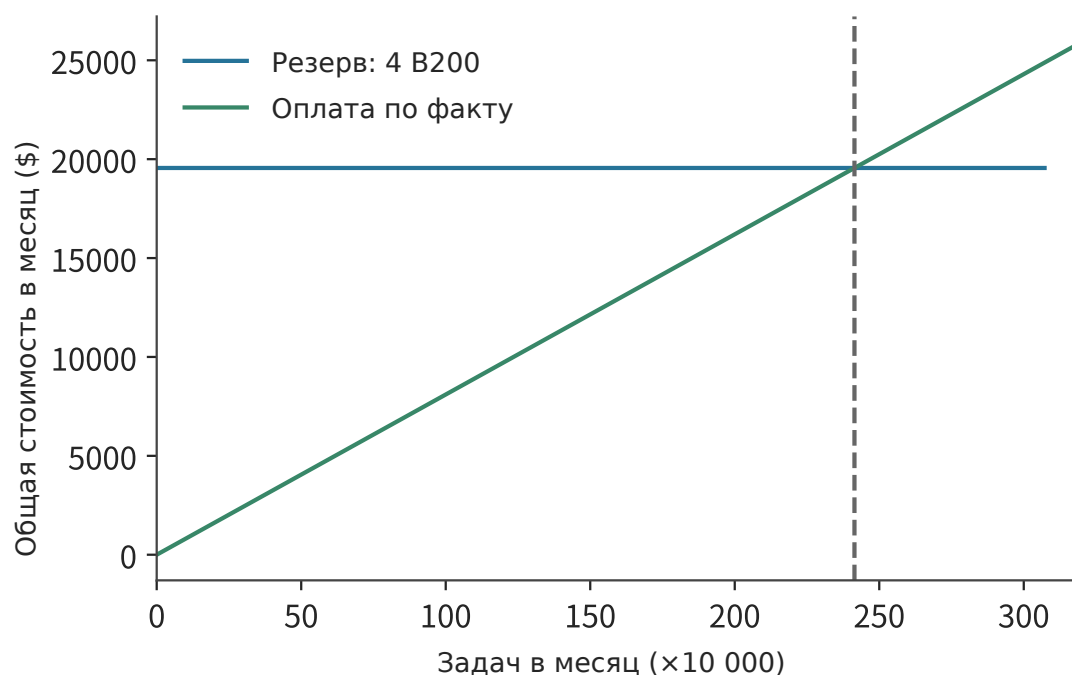


Рис. 11-42. Резервирование четырёх B200 на месяц стоит в общей сложности 19 555,2 доллара, а стоимость по фактическому потреблению составляет $0.0081N$ доллара, где N — число отправленных задач. Две линии пересекаются примерно на уровне 2,41 млн задач, что соответствует загрузке зарезервированных GPU около 78,6%; линия резервирования заканчивается на пределе производительности 3,072 млн задач в месяц.

GPU на миллион выходных токенов уменьшается примерно с 6,0 до 4,1 доллара.³⁶

Изначально около 63% времени на один выходной токен приходилось на decode, а остальные 37% оставались неизменными, поэтому двукратное ускорение decode способно сократить общее время лишь примерно на 31%. Даже если свести время decode к нулю, прочая обработка всё равно потребует 9,4 ms. Эти 9,4 ms необходимо дополнительно разделить по критерию из раздела 1.3.4: чтение KV и восстановление контекста являются обязательной работой, а рассчитанное по пропускной способности время образует новую нижнюю границу; планирование, копирование и преобразование форматов представляют собой устранимые программные издержки и потому являются областью для оптимизации. Для преодоления этого ограничения необходимо дополнительно оптимизировать обработку входных данных и восстановление контекста; только если одна и та же группа ускорителей будет генерировать больше токенов за единицу времени, фиксированную арендную плату удастся распределить на больший объём результата.

Платформа исправления кода из начала главы вызывает модель 30 раз в секунду. Быстрый сервис дополнительно расходует 0,375 B200-с на каждый вызов, что по цене Pod составляет около 0,00071 доллара, поэтому дополнительные расходы платформы достигают примерно 0,021 доллара в секунду; эти затраты сокращают продолжительность задачи с 30 до 21 секунды и позволяют уложиться в срок 24 секунды. В конце главы дополнительные расходы на модель будут сопоставлены с экономией на занятости окружения, чтобы сравнить полную стоимость задач для всех вариантов.

11.5 Восстановление полной задачи, расчёт затрат и масштабирование

11.5.1 Восстановление после сбоя инструмента и выбор модели

При выборе модели в разделе 11.4 в основном рассматривались случаи успешного завершения вызова. В реальной эксплуатации необходимо также обрабатывать сбой вызовов, потерю результатов и ошибки инструментов. Если контроллер не получил подтверждение завершения тестов, он не может различить две ситуации: тесты ещё не завершены либо тесты уже завершены, но подтверждение потеряно.

При восстановлении контроллер сначала запрашивает сохранённый результат и сверяет идентификатор операции и версию входного файла. Если результат существует, выполнение можно продолжить; если нет, контроллер определяет, допустима ли повторная попытка. Идемпотентной называется операция, у которой при повторных вызовах с одним и тем же идентификатором итоговый бизнес-эффект совпадает с эффектом однократного вызова. Повторное выполнение запросов только для чтения или идемпотентных операций обычно не вызывает проблем; однако повторный вызов внешнего платежа, создания ресурса или отправки сообщения может привести к фактическому двукратному выполнению одной операции, поэтому целевая система должна поддерживать идентификаторы операций, транзакции или компенсацию.

Предположим, что однократное выполнение инструмента требует 3 CPU-с, а резуль-

³⁶Цены B200 Pod в 6,79 доллара за карту в час и Serverless в 8,64 доллара за карту в час взяты из снимка страницы цен Runpod. Результат расчёта равен 25,317 ms, из которых 15,918 ms приходится на decode, а суммарно на остальные этапы — 9,399 ms. Здесь фиксированные накладные расходы каждого слоя и периодическое полное воссоздание заданы как входные условия; подробности приведены в базовом сравнении непрерывной работы агента, аудите оптимизации и анализе распределения времени и чувствительности.

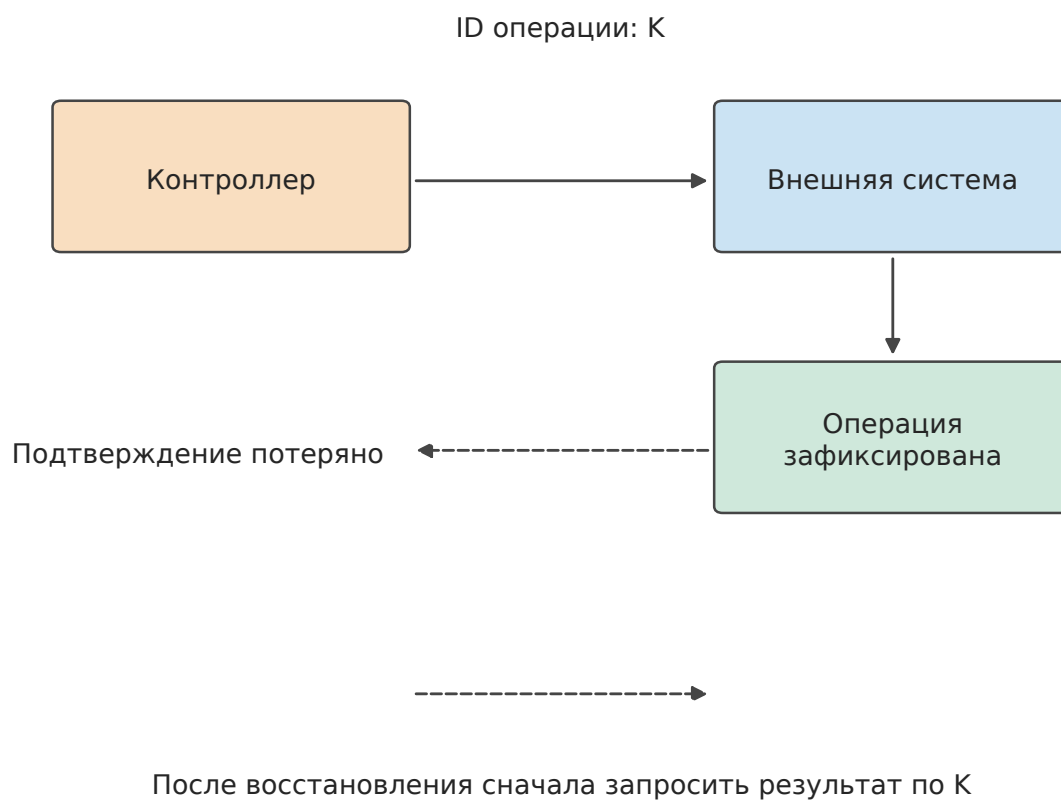


Рисунок 11-43. Внешняя система выполнила и зафиксировала операцию K, но подтверждение потерялось. Восстановление среды не отменяет результат внешней операции; контроллер запрашивает результат с тем же идентификатором операции и продолжает задачу. Сплошные линии обозначают запрос и выполнение, пунктирные — подтверждение и запрос при восстановлении.

тат отправлен дважды. Если получатель устраняет дубликаты по идентификатору операции, совокупное процессорное время останется равным 3 секундам; если же из-за потери подтверждения контроллер выполнит операцию повторно, оно вырастет до 6 секунд. В обоих случаях в итоге может остаться лишь один результат, но расход ресурсов будет различаться. Устранение дубликатов сообщений решает проблему первого типа; для предотвращения повторения второго типа нужны сохранённые результаты и идемпотентный протокол целевой системы. Операция, уже зафиксированная вне среды, сохраняется и после восстановления из снимка, поэтому процесс восстановления должен сначала запросить её статус фиксации.³⁷

Однако нормальный возврат запроса ещё не означает, что при восстановлении обязательно удастся найти его запись. Традиционные операционные системы также различают «запись завершена» и «данные сохранены в постоянном хранилище». Например, при обычной буферизованной записи в файл к моменту возврата `write()` данные могут всё ещё находиться в страничном кэше ядра: программа продолжает вычисления, а операционная система записывает их в фоновом режиме. Если программа обязана дожидаться сохранения данных, прежде чем продолжить, ей нужно вызвать `fsync()`, дожидаться записи данных файла и необходимых метаданных на устройство хранения и проверить успешность операции.³⁸

Для среды выполнения AI-агента также необходимо чётко определить, что гарантируется при сообщении вызывающей стороне о завершении: только генерация ответа либо сохранение сообщения, результатов инструментов и позиции восстановления. Эти соглашения образуют семантику долговечности. Вызвать `fsync()` только для журнала чата недостаточно, поскольку необходимые для восстановления сведения могут храниться раздельно в журнале, рабочем каталоге и внешней базе данных; контроллер должен также проверить согласованность операций в сообщениях, файлов в каталоге и результатов в базе данных.

Здесь сохранение выполняется по итерациям цикла ReAct: каждая итерация включает один вывод модели, инициированные им вызовы инструментов и возвращённые ими результаты и может содержать несколько сообщений. Контроллер добавляет запись этой итерации в локальный файл JSONL и переходит к следующей итерации лишь после подтверждения успешного сохранения всей текущей; при ошибке сохранения выполнение немедленно прерывается. Поэтому локальное выполнение не продолжится с сохранением последующих итераций после ошибки сохранения одной из них. Для «успешного сохранения» также необходимо указать, какой вид отказа оно способно пережить: запись в локальный страничный кэш, завершение локального `fsync()` и получение подтверждения сохранения в облаке дают разные гарантии. Локальный файл позволяет восстановиться после перезапуска процесса, но не обязательно поможет при недоступности всей машины вместе с её диском.

Для уменьшения ожидания можно сначала сохранять каждую итерацию локально, а затем в фоновом режиме загружать добавленные в неё записи. Предположим, что по-

³⁷Записи задач и границы доказательств, инъекция сбоев в файловую очередь, запись статусов завершения задач. Эти записи используются соответственно для пояснения сохранения результатов, диагностики сбоев и расчёта стоимости.

³⁸Страницы руководства Linux `write(2)` и `fsync(2)`. Успешный вызов `write()` не гарантирует запись данных в постоянное хранилище; вызывающая сторона также должна проверить фактическое количество записанных байтов и ошибки синхронизации. При создании или переименовании файла для сохранения записи каталога может дополнительно потребоваться выполнить `fsync()` для каталога; сама по себе синхронизация файлов не обеспечивает транзакции над несколькими файлами.

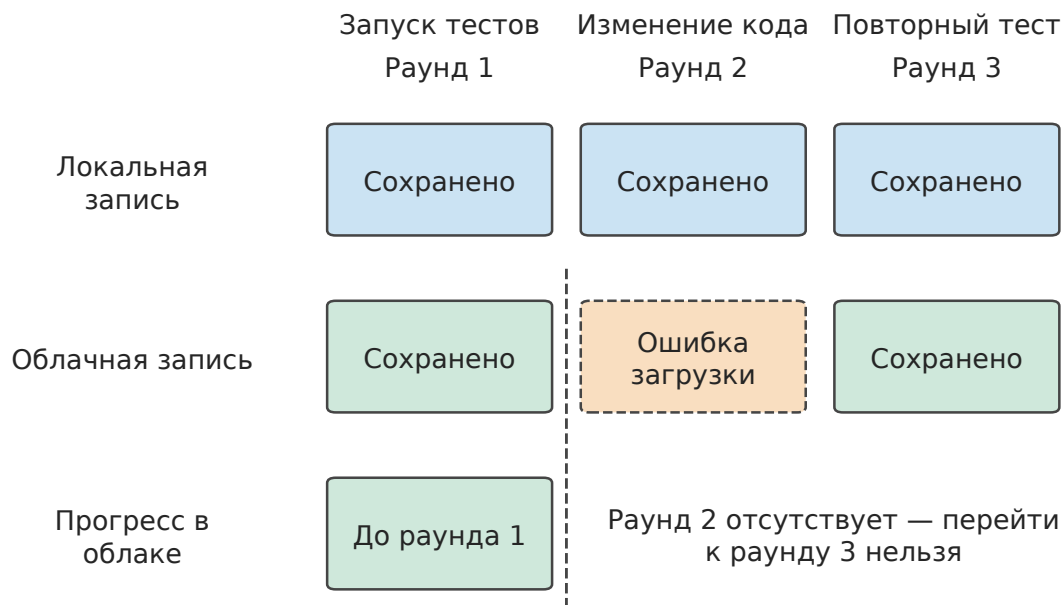


Рисунок 11-44. Исходное состояние сохранено локально и в облаке, каждый столбец представляет одну итерацию ReAct. Каждая итерация сначала сохраняется локально, затем добавленные в ней записи асинхронно загружаются; в этом примере после ошибки загрузки разрешено продолжать выполнение и загружать последующие итерации. После ошибки загрузки второй итерации облачная история остаётся полной лишь до первой, даже если третья успешно загружена. Горизонтальное расположение показывает порядок итераций, а не длительность.

сле ошибки загрузки повторная попытка пока не выполняется, а локальное выполнение не прерывается: на первой итерации запускаются тесты и получается результат с ошибкой, на второй изменяется код и получается подтверждение инструмента, на третьей тесты запускаются повторно и возвращают новый результат. Записи всех трёх итераций сохранены в локальном JSONL, но загрузка второй завершилась с ошибкой, а третьей — успешно (рисунок 11-44). В результате локальная история полна, но в облаке отсутствуют вывод модели и результаты инструментов второй итерации. Если после этого локальная машина станет недоступна, при восстановлении на другой машине только из облачных записей возникнет этот пробел. Данный пример неприменим, если при каждой загрузке отправляется полный снимок, содержащий все предыдущие итерации, либо программа загрузки обязана сначала восполнить отсутствующие записи.

При таком асинхронном сохранении контроллер должен отдельно отслеживать ход локального и облачного сохранения. В облаке нужно записывать не только последнюю полученную итерацию, но и номер итерации, до которой история непрерывно сохраняется от исходной точки. Успешная загрузка третьей итерации не позволяет передвинуть позицию непрерывного сохранения с первой на третью; подтвердить полноту облачной истории до третьей итерации можно лишь после восполнения второй либо восстановления из полной локальной копии и последующей проверки.

То же относится к удалению старых журналов. Чтобы удалить записи первых двух итераций, сначала необходимо сохранить полное состояние, позволяющее восстановиться до конца второй итерации, и зафиксировать это состояние вместе с соответствующим номером итерации. Если при восстановлении не удаётся найти ни полный предшеству-

ющий журнал, ни это состояние, следует вернуть incomplete, даже если весь оставшийся журнал успешно прочитан: оснований подтвердить полноту истории недостаточно. Если интерфейс допускает восстановление по позиции сообщения, необходимо также убедиться, что она соответствует полностью сохранённой итерации, и сверить версию рабочего каталога; если позиции внутри итерации не сохранялись отдельно, обещать восстановление до них нельзя. Зафиксированные во внешних системах операции также не отменяются при откате сообщений.³⁹

Для задачи исправления кода из этой главы можно заимствовать подход транзакций баз данных и фиксировать итерацию «решение модели — выполнение инструмента — сохранение результата» как единое целое. Контроллер отмечает итерацию как зафиксированную только после успешного сохранения сообщения модели, результатов инструментов и версий связанных файлов; после перезапуска выполнение продолжается с результата последней полностью зафиксированной итерации. Это одно из направлений исследований **транзакций агентов (agent transaction)**: реализация в процессе работы агента атомарности, согласованности, изоляции и долговечности, то есть свойств ACID баз данных.⁴⁰ Если в ходе итерации вызывается внешний сервис, он должен поддерживать идентификаторы операций, идемпотентный интерфейс или компенсационную обработку; одной локальной отметки «зафиксировано» недостаточно, чтобы гарантировать ровно однократное выполнение внешней операции.

Чем чаще выполняется сохранение, тем меньше работы обычно приходится повторять после сбоя, но тем выше накладные расходы при нормальном выполнении. Рассмотрим небольшой пример: последовательно выполняются 100 запросов, каждый занимает 200 мс; каждая фиксация в постоянном хранилище имеет постоянные накладные расходы 8 мс, а запись данных одного запроса требует ещё 2 мс. Предположим, что на время фиксации обработка запросов приостанавливается, а прочими расходами можно пренебречь. Получим следующие результаты.

Способ сохранения	Общее время сохранения	Общее время без сбоя	Максимальное время повторных вычислений после сбоя
Фиксация после каждого запроса	$100(8 + 2) = 1000$ мс	21 с	0,2 с
Фиксация после каждых 10 запросов	$10(8 + 10 \times 2) = 280$ мс	20,28 с	2 с

Совместная фиксация каждых 10 запросов экономит 0,72 с при нормальном выполнении; однако при сбое непосредственно перед фиксацией придётся повторно выполнить все 10 запросов, затратив 2 секунды вычислений. Здесь предполагается, что каждая фиксация либо полностью успешна, либо не оказывает никакого эффекта, все незафиксированные запросы требуется повторить, а инструменты допускают безопасный повторный вызов; время восстановления среды и повторного сохранения в таблице не учитывается.

³⁹В препринте Safe to Resume? (2026) анализируется проблема несоответствия восстановленного состояния зависимостям, необходимым для продолжения выполнения. Числа, относящиеся к пропускам запросов и пакетной фиксации в этом разделе, являются учебными примерами, а не результатами измерений из статьи.

⁴⁰В препринте Agentic Transaction (2026) предлагаются семантические атомарность, согласованность, изоляция и долговечность для агентов; в §2.2.4 рассматривается долговременное сохранение зафиксированного состояния, доказательств и метаданных восстановления. В этом разделе проектирование единицы фиксации поясняется на примере исправления кода, но из этого не следует, что произвольный вызов инструмента обладает гарантиями ACID.

При асинхронном сохранении контроллер может записывать уже полученные результаты одновременно с обработкой последующих запросов. Объём повторной работы после сбоя зависит от количества ещё не сохранённых к тому моменту записей. Поэтому при сравнении способов сохранения следует учитывать не только время выполнения задачи, но и число уже завершённых, однако не сохранённых запросов, а также проверять, до какой позиции в действительности удаётся восстановиться после перезапуска.

Здесь по-прежнему необходимо ответить на два вопроса, неоднократно рассматриваемых в книге: где размещаются данные и кто обязан ждать их сохранения. В разделе 11.2 среда инструментов освобождалась на время вызова модели для экономии памяти. Это допустимо, только если состояние, необходимое для следующей итерации, уже сохранено вне среды либо может быть вычислено заново после потери. Если единственный экземпляр результата остаётся в памяти среды, контроллер обязан дожидаться его сохранения перед освобождением среды, иначе после потери останется лишь выполнить работу заново. Поэтому при распределении ресурсов планировщик должен учитывать время ожидания сохранения состояния и стоимость повторной работы после сбоя.

После определения позиции, с которой можно продолжить, контроллер может также выбрать более мощную модель. Если исходная модель завершила две итерации, а исправление и результаты тестов полностью сохранены, новая модель сможет продолжить решение без повторения первых двух итераций. Тогда потребуется оплатить лишь следующий вызов модели, а вероятность успешного решения может повыситься. Вероятность успеха последующего исправления зависит от причины предыдущей неудачи. Поэтому ниже при вычислении вероятностей ветвей дерева восстановления они всегда задаются при условии, что задача уже достигла соответствующего узла.

11.5.2 Полная стоимость успешной задачи

Повторная попытка или переход на другую модель после сбоя требуют дополнительных расходов, поэтому цена одного вызова не показывает, сколько в действительности стоит завершение задачи. Необходимо сложить затраты на все успешные и неуспешные попытки, а затем разделить сумму на фактическое число завершённых задач. Пусть общие расходы за рассматриваемый период равны C_{all} , число задач, удовлетворяющих требованиям качества, равно N_q , а число задач, завершённых качественно и в срок, равно $N_{q,d}$. Тогда средняя стоимость одной успешной задачи и одной задачи, успешно завершённой в срок, соответственно равна

$$\overline{C}_q = \frac{C_{\text{all}}}{N_q}, \quad \overline{C}_{q,d} = \frac{C_{\text{all}}}{N_{q,d}}.$$

Неуспешные попытки также расходуют ресурсы модели и инструментов, поэтому их стоимость нельзя вычитать из общих расходов. Например, если десять задач в сумме стоили 1 доллар, а успешно завершились пять, фактические расходы платформы на одну завершённую задачу составят 0,2 доллара. Если учитывать только 0,5 доллара, потраченные на пять успешных попыток, получится 0,1 доллара — ровно половина расходов останется неучтённой.

Эту удельную стоимость можно предварительно оценить до развёртывания с помощью конечного дерева повторных попыток. Для пути π обозначим его вероятность через p_π , а сумму затрат всех шагов — через c_π ; тогда $E[C] = \sum_\pi p_\pi c_\pi$. Вероятность успеха равна сумме вероятностей всех путей, завершающихся успехом. Вероятность каждого

узла следует вычислять с учётом предшествующих результатов выполнения; если после двух типов ошибок вероятности дальнейшего успеха различаются, их следует представить отдельными узлами.

Пример 11-8. Влияние локального исправления на вероятность успеха и долю превышений срока. Рассмотрим ещё один небольшой пример, в котором первая попытка занимает 10 секунд, и сравним следующую конечную стратегию восстановления. Для первой попытки используется Claude Sonnet 5 с 2 500 входными и 500 выходными токенами; по стандартному тарифу она стоит 0,010 доллара и занимает 10 секунд. В 80% случаев задача сразу завершается успешно, 12% переходят к локальному исправлению, а 8% — непосредственно к эскалации, то есть повторной попытке с более мощной моделью. Для локального исправления по-прежнему используется Sonnet 5 с 1 500 входными и 300 выходными токенами; оно дополнительно стоит 0,006 доллара и занимает 4 секунды, а условная вероятность успеха равна 60%. Остальные задачи переходят к эскалации. При эскалации используется Claude Opus 5: 5 долларов за миллион входных токенов и 25 долларов за миллион выходных токенов. Вызов с 3 000 входными и 600 выходными токенами дополнительно стоит 0,030 доллара, занимает 8 секунд и имеет условную вероятность успеха 98%. Срок в 20 секунд используется только для оценки и не приводит к принудительной остановке задачи.⁴¹

На рисунке 11-45 задача с успешным локальным исправлением завершается на 14-й секунде; путь с неудачным исправлением и последующей эскалацией занимает на 4 секунды больше, чем непосредственная эскалация. Поэтому даже после повышения общей вероятности успеха часть успешных результатов выходит за установленный срок.

Шесть возможных результатов выполнения приведены ниже. Указанная в каждой строке стоимость охватывает весь процесс выполнения, включая расходы на все предшествующие попытки.

Процесс выполнения и результат	Вероятность	Полная стоимость / долл.	Время / с	Тесты пройдены в срок
Успех первой попытки	0.80000	0.010	10	Да
Первая попытка → успешное исправление	0.07200	0.016	14	Да
Первая попытка → исправление → успешная эскалация	0.04704	0.046	22	Нет
Первая попытка → исправление → неудачная эскалация	0.00096	0.046	22	Нет

⁴¹Точные результаты: вероятность успеха — 0,99744, вероятность своевременного прохождения проверки — 0,9504, ожидаемая стоимость одной задачи — 0,01456 доллара, суммарное время CPU — 3,376 секунды, резидентность — 23,008 GiB-с. В результатах расчёта отдельно указаны использование ресурсов и стоимость; заданная стоимость каждого узла уже является полной, поэтому повторно рассчитывать её по использованным ресурсам не требуется. См. конечный граф условий `retry-paths`. Стоимость каждого узла рассчитывается по стандартным ценам Claude и заданному в условии количеству токенов, вероятности заданы условием, а повторные попытки разворачиваются по конечному дереву выполнения; задачи, превысившие срок оценки, всё равно выполняются до конца.

Процесс выполнения и результат	Вероятность	Полная стоимость / долл.	Время / с	Тесты пройдены в срок
Первая попытка → успешная эскалация	0.07840	0.040	18	Да
Первая попытка → неудачная эскалация	0.00160	0.040	18	Нет

Расчёт можно пошагово выполнить для 1 000 отправленных задач. Первые попытки стоят 10 долларов, и около 800 задач сразу завершаются успешно. К локальному исправлению переходят 120 задач, что добавляет $120 \times 0.006 = 0.72$ доллара; из них около 72 завершаются успешно. Оставшиеся примерно 48 задач вместе с 80 задачами, перешедшими непосредственно к эскалации, дают 128 эскалаций, дополнительно стоящих $128 \times 0.030 = 3.84$ доллара. Общая стоимость составляет около 14,6 доллара, а в итоге успешно завершаются примерно 997 задач.

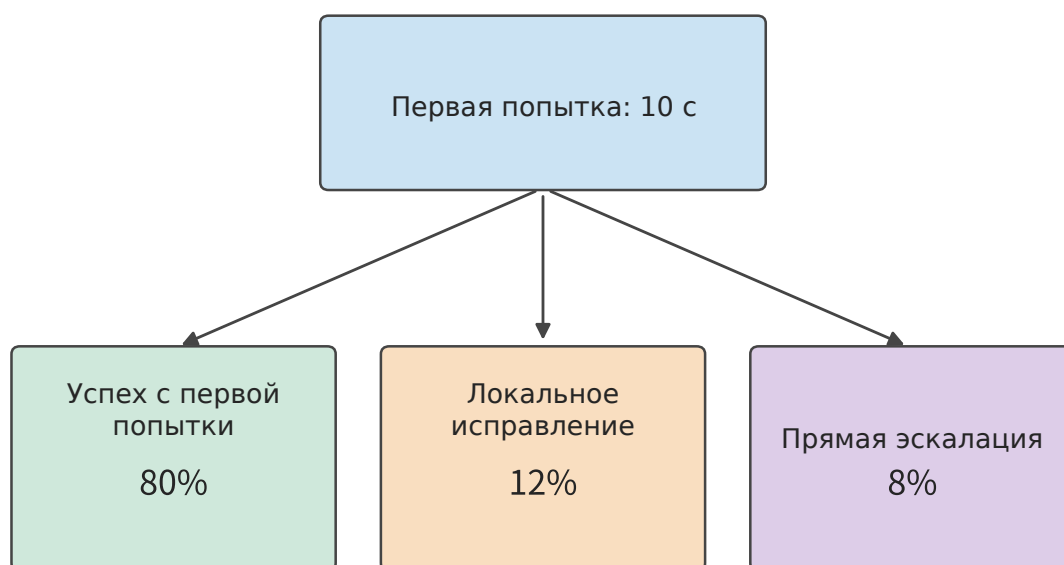
Путь с исправлением и последующей эскалацией занимает $10 + 4 + 8 = 22$ секунды, превышая срок в 20 секунд. Около 47 задач проходят тесты, но завершаются с опозданием, поэтому число успешных задач, завершённых в срок, составляет примерно 950. Средняя стоимость задачи, прошедшей тесты, равна приблизительно 0,0146 доллара, а задачи, прошедшей тесты в срок, — приблизительно 0,0153 доллара. Если выполнять только первую попытку, стоимость составит $10/800 = 0.0125$ доллара, но вероятность успеха будет равна лишь 80%. Восстановление повышает долю завершённых задач, но также увеличивает стоимость одного завершения. Если бизнес требует, чтобы не менее 95% отправленных задач успешно завершались в срок, эти дополнительные расходы необходимы; сравнение лишь стоимости одной попытки не учитывает прирост числа завершённых задач.

После применения описанной стратегии восстановления доля успешных задач возрастает с 80% примерно до 99,7%, но около 4,7% отправленных задач завершаются лишь на 22-й секунде. При сроке в 20 секунд эти задачи расходуют ресурсы, но не учитываются среди завершённых вовремя. Если сократить только срок, используемый для оценки, процесс выполнения не изменится — просто меньше задач будут считаться успешно завершёнными в срок; если же ограничить число повторных попыток, контроллер выполнит меньше шагов, а расходы и вероятность успеха также изменятся.⁴²

Помимо стратегии восстановления, необходимо учитывать влияние ускорения модели. После ускорения модели подготовка среды может перестать полностью перекрываться с вызовом модели и начать влиять на время завершения задачи. Предположим, что следующий вызов инструмента произойдёт обязательно, быстрой службе требуется ещё 6 секунд на генерацию, а шаблона нет на локальной машине и создание среды по холодному пути занимает около 2,7 секунды. Если начать создание среды одновременно с началом генерации модели, все 2,7 секунды подготовки полностью перекроются генерацией. Если этап модели сократится до 1 секунды, при том же моменте начала создания придётся дополнительно ждать около 1,7 секунды.⁴³ Сначала ускорим только

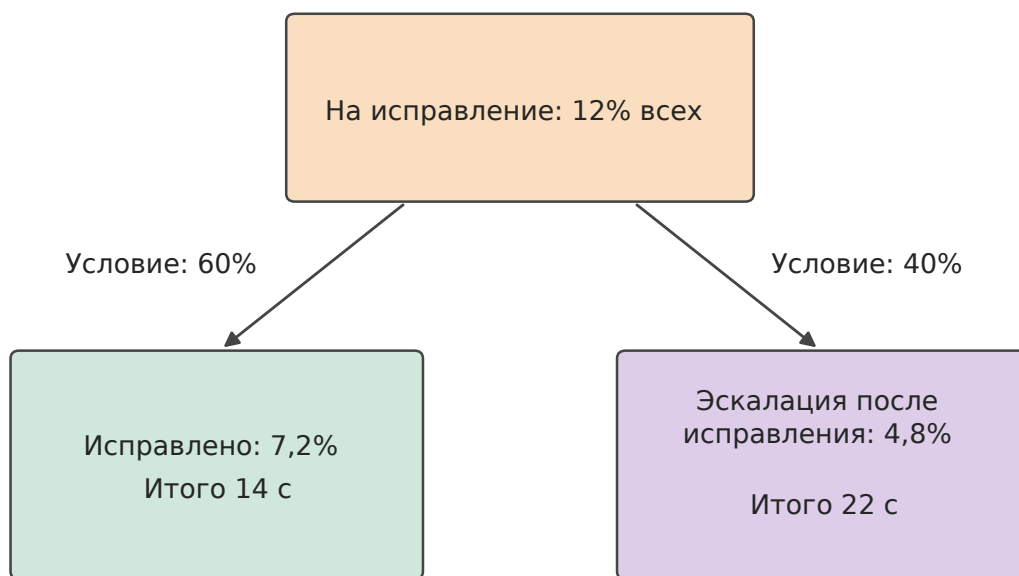
⁴²Записи задач и границы доказательств, инъекция сбоев в файловую очередь, запись статусов завершения задач. Эти записи используются соответственно для пояснения сохранения результатов, диагностики сбоев и расчёта стоимости.

⁴³Повторный расчёт по каждому условию сравнения и программа расчёта для этой главы.



Все доли ниже — от общего числа отправок

Рисунок 11-45. Через 10 с первая попытка разделяется на три исхода: успех, локальное исправление и непосредственная эскалация. Доли внутри блоков рассчитаны относительно всех отправленных задач; на следующем рисунке раскрываются условные ветви исправления.



$$12\% \times 60\% = 7.2\%$$

$$12\% \times 40\% = 4.8\%$$

Рисунок 11-46. Увеличенный узел исправления: из 12% попавших сюда задач исправление успешно для 60%, что составляет 7,2% всех отправленных задач; 40% переходят к эскалации, что составляет 4,8% всех отправленных задач. Эскалация занимает ещё 8 с, поэтому совокупное время последнего пути равно 22 с.

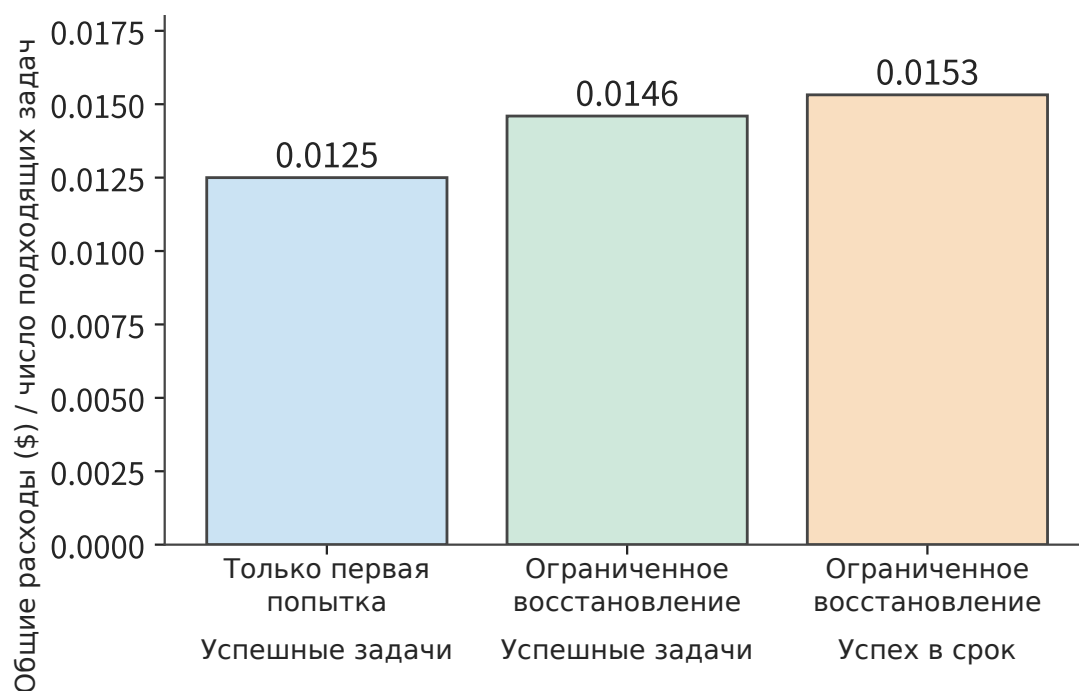


Рисунок 11-47. Стоимость и результаты после применения стратегии восстановления к одному и тому же набору отправленных задач. Для каждого столбца в числителе используются все расходы соответствующей стратегии, а знаменателем служит соответственно число задач, прошедших тесты, либо число задач, прошедших тесты в срок. При выполнении только первой попытки вероятность успеха равна 80%, а при конечной стратегии восстановления — приблизительно 99,7%; при этом около 95,0% отправленных задач завершаются успешно за 20 секунд. Вероятности ветвей, стоимость и время приведены в примере 11-8; срок используется для оценки, но не приводит к принудительной остановке выполнения.

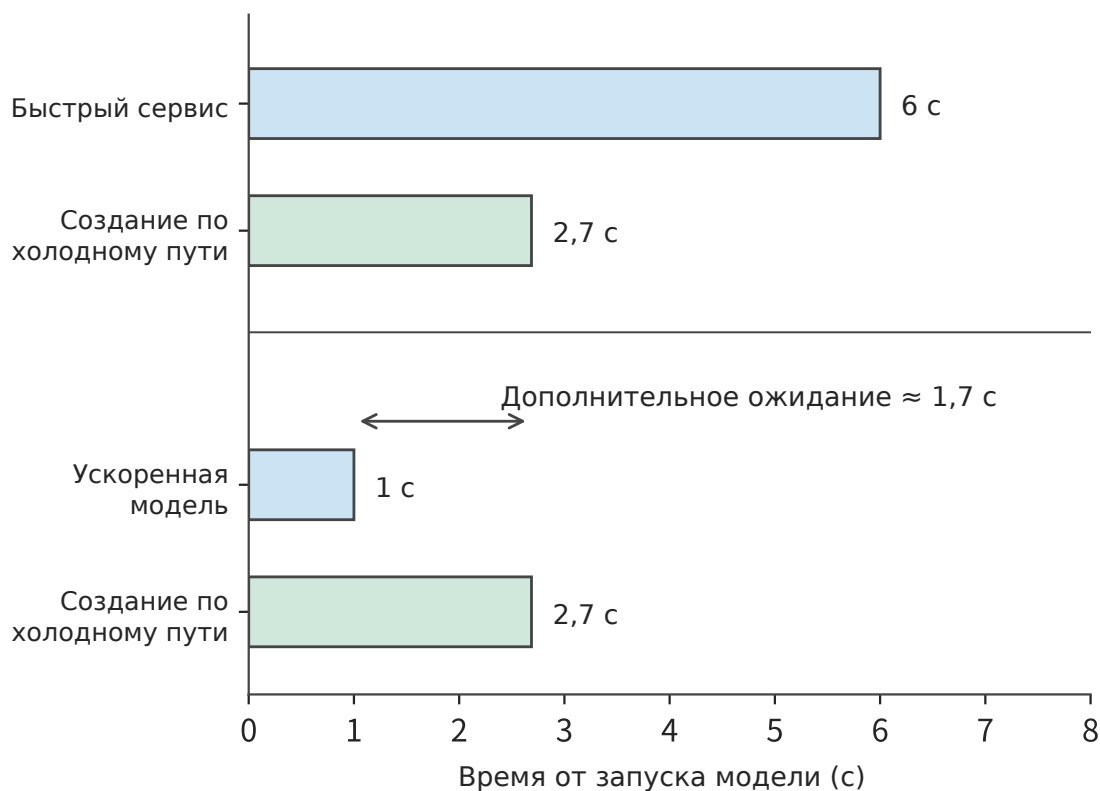


Рисунок 11-48. После ускорения модели создание среды становится источником ожидания перед началом выполнения инструмента. В обоих случаях создание среды начинается в нулевой момент; инструмент должен дожидаться как решения модели, так и готовности среды. Горизонтальная ось показывает секунды.

вызов модели, сохранив прежнюю стратегию подготовки; затем сравним стоимость трёх вариантов: более ранней подготовки, сохранения уже готовой среды и ожидания по требованию. При этом учтём заблаговременное занятие ресурсов и расходы на среду, подготовленную заранее, но в итоге не использованную.

При восстановлении требуется заново создать не только среду инструментов, но и состояние на стороне службы модели. В проектом задании этой главы время и стоимость вычисляются для V4-Flash; восстановление состояния модели рассматривается на примере отслеживаемого на протяжении книги сеанса V4.1 Flash, поскольку его глобальный KV можно получить, а локальное состояние SWA необходимо воссоздать повторным воспроизведением токенов в конце промпта (раздел 3.2.4). Пути восстановления этих двух частей различаются. Во время ожидания инструмента такой сеанс одновременно сохраняет две группы состояний: операционная система управляет процессом, файлами и средой, а служба модели — глобальным KV и локальным состоянием SWA. Для восстановления задачи эти две группы состояний требуется подготовить отдельно, а затем объединить при передаче результата инструмента в следующий вызов модели; результаты уже выполненных внешних операций подтверждаются запросом к целевой системе.⁴⁴ Если подготовка среды и получение KV могут выполняться параллельно, для-

⁴⁴Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт межглавного сеанса.

тельность восстановления определяется более поздним из этих двух событий.

11.5.3 Выбор масштабирования CPU, GPU, памяти и квот службы

Ниже на основе заданного в начале главы срока завершения в 24 секунды выбираются служба модели и способ управления средой. Ранее были получены три результата: каждый вызов обычной службы занимает 9 секунд, поэтому три итерации требуют 30 секунд; каждый вызов быстрой службы занимает 6 секунд, поэтому три итерации требуют 21 секунду; повторное создание среды на каждой итерации снижает совокупное потребление памяти среды для одной задачи до 18 GiB-с, а подготовка дополнительно требует 0,3 секунды процессорного времени.

Пример 11-9. Как выбрать службу модели и способ управления средой, чтобы завершить задачу в срок с минимальными затратами? Сохраним условия из начала главы: каждые 0,1 секунды поступает одна задача, каждая выполняется в три итерации; платформа инструментов работает на одном сервере m5d.metal с 48 ядрами и примерно 358 GiB памяти, а служба модели располагает 10 существующими репликами V4-Flash на 40 GPU B200. Обе службы запускают одну и ту же модель, итоговая вероятность прохождения тестов равна 95%, а результат тестов не зависит от времени поступления задачи. Вызовы модели оплачиваются по занятому времени B200 по тарифу 6,79 доллара за GPU-ч: каждый вызов обычной службы занимает 1,125 B200-с и стоит около 0,00212 доллара; каждый вызов быстрой службы занимает 1,5 B200-с и стоит около 0,00283 доллара. CPU и память оплачиваются по опубликованным E2B тарифам на песочницы: 0,000014 доллара за CPU-с и 0,0000045 доллара за GiB-с, в обоих случаях по фактическому использованию. Стоимость сохранения файлов уже включена в выполнение инструментов каждой итерации; общие постоянные расходы платформы одинаковы для всех четырёх вариантов.⁴⁵

Сначала исключим вариант «увеличить только CPU». Инструменты трёх итераций работают в общей сложности 3 секунды; даже при двукратном ускорении инструментов с обычной службой по-прежнему потребуется $27 + 1.5 = 28.5$ секунды. Даже если сократить время инструментов до нуля, вызовы модели всё равно займут 27 секунд. Масштабирование CPU не способно сократить этот последовательный путь до 24 секунд.

Затем сравним стратегии управления средой. Подготовка при повторном создании среды на каждой итерации перекрывается с вызовом модели и не изменяет время завершения: с обычной службой оно по-прежнему равно 30 секундам, а после перехода на быструю службу суммарное время модели сокращается до 18 секунд и вместе с 3 секундами работы инструментов составляет 21 секунду. Поэтому при использовании быстрой службы установленный срок выдерживается как при постоянном сохранении среды, так

⁴⁵ Сквозные входные данные платформы и примера 11-9, поэтапное расписание, стоимость, условие равенства стоимости и повторные расчёты приведены в сквозных данных проектирования и программе проверки. Время вызова модели основано на приведённой в книге оценке работы DeepSeek V4-Flash на четырёх B200 с контекстом 200K на сеанс: при 32 сеансах на реплику вывод одного токена занимает 25,3 ms, а при 16 сеансах — 16,9 ms; оба значения включают распределённые затраты на обработку входных данных и воссоздание контекста (результаты сравнения, полные данные). Вызов с 355 токенами занимает соответственно около 8,99 s и 6,01 s; в проекте используются значения 9 s и 6 s. Цена B200 в 6,79 доллара за карту в час взята из снимка страницы цен Runpod. 48 ядер m5d.metal при отключённой многопоточности и 384 GB памяти взяты из тестового окружения, описанного в разделе 5 статьи о Firecracker; конфигурация 1 vCPU и 2 GiB, а также цены CPU и памяти взяты из снимка страницы цен E2B. Подготовка по горячему пути в совокупности занимает 2 секунды и 0,1 CPU-с, а вероятность прохождения приёмочной проверки равна 95% — эти значения заданы условием; сохранение результата и очистка рабочего состояния включены в одну секунду выполнения инструмента. Пример с ёмкостью шаблона отдельно поясняет загрузку содержимого.

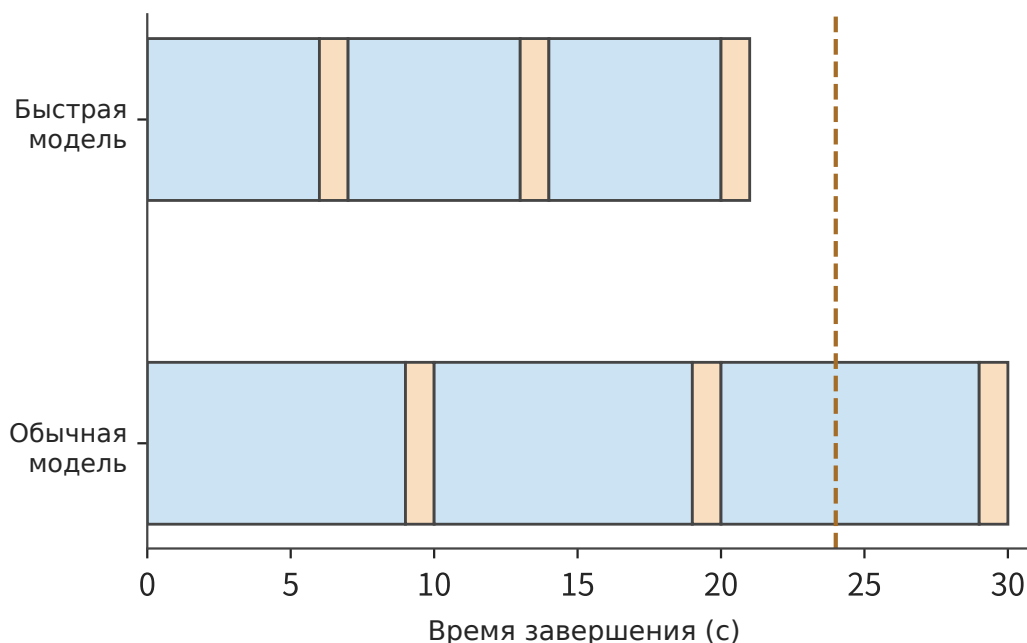


Рисунок 11-49. В обоих вариантах среда готовится во время вызова модели, а выполнение инструмента на каждой итерации занимает 1 секунду. После сокращения вызова модели с 9 до 6 секунд время завершения трёх итераций сдвигается с 30-й на 21-ю секунду — из-за пределов 24-секундного срока в его пределы.

и при её повторном создании на каждой итерации; далее необходимо сравнить ресурсы и стоимость.

На рисунке 11-49 варианты сначала отбираются по соблюдению срока: последняя итерация обычной модели завершается после установленного срока, а все три итерации быстрой модели — до его истечения. После этого достаточно сравнить стоимость двух стратегий управления средой при быстрой модели, одновременно проверив, не превышают ли они возможности платформы.

Вариант	Время завершения / с	Совокупное процессорное время / CPU-с	Совокупное потребление памяти на задачу / GiB-с	Средняя память / GiB	Стоимость тысячи отправленных задач / долл.	Доля задач, прошедших тесты в срок
Обычная служба + постоянно сохраняемая среда	30	3	60	600	6.68	0
Обычная служба + повторное создание среды на каждой итерации	30	3.3	18	180	6.49	0
Быстрая служба + постоянно сохраняемая среда	21	3	42	420	8.72	95%
Быстрая служба + повторное создание среды на каждой итерации	21	3.3	18	180	8.61	95%

Рассмотрим последнюю строку. Три вызова модели в общей сложности занимают $3 \times 1.5 = 4.5$ В200-с и стоят около 0,0084875 доллара; стоимость CPU равна $3.3 \times 0.000014 = 0.0000462$ доллара, стоимость памяти — $18 \times 0.0000045 = 0.000081$ доллара. Итого получается около 0,0086147 доллара, то есть 8,61 доллара за тысячу задач. Стоимость модели превышает 98% общих расходов, поэтому основное преимущество повторного создания среды на каждой итерации заключается не в цене, а в экономии ресурсов. При поступлении 10 задач в секунду выполнение инструментов и подготовка среды в среднем используют 33 ядра CPU и 180 GiB памяти, что укладывается в возможности m5d.metal; быстрой службе с постоянно сохраняемой средой требуется 420 GiB, поэтому данный сервер её не вмещает. В среднем служба модели одновременно обрабатывает 180 вызовов; каждая реплика быстрой службы поддерживает лишь 16 сеансов, поэтому нужны 12 реплик, то есть на 2 больше имеющихся.

После проверки соответствия средней потребности выбранного варианта доступным ресурсам необходимо убедиться, что все этапы можно запланировать в срок. При равномерном поступлении задач можно непосредственно составить расписание, соблюдающее ограничения ресурсов. Для быстрой службы, если отсчитывать время от момента поступления каждой задачи, среда готовится соответственно на интервалах 4–6, 11–13 и 18–20 секунд, а инструменты запускаются на интервалах 6–7, 13–14 и 20–21 секунд; для каждой задачи подготовка на каждой итерации в общей сложности использует 0,1 секунды процессорного времени, равномерно распределённого по двум секундам. Поскольку каждые 0,1 секунды поступает одна задача, после выхода в установившийся режим выполнение инструментов каждой итерации занимает по 10 ядер, а подготовка каждой итерации — по одному ядру; одновременно активно не более чем около 90 сред, использующих в общей сложности 180 GiB. На каждом из трёх этапов вызова модели в обработке в среднем находится по 60 вызовов, то есть всего 180 одновременных вызовов; 12 быстрых реплик предоставляют 192 параллельных слота. Такое расписание соблюдает зависимости между этапами и не превышает возможности платформы.

Следовательно, следует выбрать быструю службу, повторно создавать среду на каждой итерации, выполнять подготовку во время вызова модели и увеличить число реплик модели с 10 до 12. Этот вариант позволяет выполнить требование в 24 секунды на существующем сервере инструментов и является единственным из четырёх, который одновременно укладывается в срок и не превышает возможности данного сервера. Средняя стоимость тысячи задач, прошедших тесты в срок, составляет около $8.61/0.95 \approx 9.07$ доллара; быстрая служба с постоянно сохраняемой средой даже при добавлении памяти будет стоить около $8.72/0.95 \approx 9.18$ доллара, поэтому повторное создание среды на каждой итерации снижает эту стоимость лишь примерно на 1,2%. Обычная служба с повторным созданием среды на каждой итерации дешевле, но даже успешно прошедшие тесты задачи завершаются после установленного срока.

В разделе 11.5.2 также показано, что добавление повторных попыток изменяет долю задач, завершённых в срок. Здесь обычная задача завершается на 21-й секунде, поэтому до истечения срока остаётся лишь 3 секунды; если после итоговой неудачи добавить локальное исправление продолжительностью 4 секунды, результат появится только на 25-й секунде. Поэтому в данной задаче выполнение заканчивается после трёх итераций, а задачи, не прошедшие тесты, сразу возвращают результат с ошибкой. Если бизнес увеличит срок до 25 секунд, локальное исправление сможет увеличить число успешных задач, завершённых вовремя; тогда его условную вероятность успеха и дополнительные ресурсы следует добавить в дерево восстановления.

Этот выбор основан на условии, что каждый вызов модели занимает 6 секунд. При дальнейшем ускорении модели интервалы простоя, на которых можно освобождать память среды, также сократятся, поэтому стратегии управления средой потребуется сравнить заново. Выбор между сохранением и повторным созданием среды можно сделать по длительности ожидания, при которой стоимости этих вариантов равны. Пусть между двумя выполнениями инструментов требуется ждать модель G секунд. Содержание среды объёмом 2 GiB стоит $2G \times 0.0000045$ доллара; если уничтожить её и повторно подготовить в конце следующего вызова модели, память будет стоить $2 \times 2 \times 0.0000045$ доллара, а необходимое для подготовки процессорное время — 0.1×0.000014 доллара, то есть в сумме 0,0000194 доллара. Стоимости равны при $G \approx 2.16$ секунды. При ожидании 6 секунд среду следует создавать заново; когда ожидание сокращается до 2 секунд, сохранять среду становится выгоднее, а подготовка уже занимает всё доступное окно перекрытия. Дальнейшее ускорение модели изменит оптимальную стратегию управления средой.

Влияние локального ускорения на общее время можно также выразить законом Амдала из главы 1. Пусть доля ускоряемой части исходного времени равна f , эта часть ускоряется в s раз, а длительность остальной части не изменяется. Тогда общее ускорение равно

$$S = \frac{1}{(1 - f) + f/s}.$$

В начале главы инструменты занимали лишь 10% времени, поэтому их двукратное ускорение даёт общее ускорение $1/(0.9 + 0.1/2) \approx 1.05$; вызовы модели занимали 90%, и их двукратное ускорение даёт $1/(0.1 + 0.9/2) \approx 1.82$. При одинаковом двукратном ускорении вызов модели экономит больше времени задачи, поскольку занимает большую долю времени выполнения.

В главе 12 для той же задачи и той же цели завершения будут дополнительно сопоставлены способы выполнения на конечном устройстве, на периферии сети и в облаке, а также проанализировано влияние времени передачи данных на время завершения задачи.

11.6 Упражнения и эксперименты

В следующих упражнениях используется нумерация материалов с 11-1 по 11-10. Основные задания 11-1, 11-2 и 11-10 образуют полное упражнение от требований до проектирования; исходные записи экспериментов и необязательные точки запуска приведены в сопроводительных материалах к главе.

1. **Эксперимент 11-1 (основной): расчёт параллелизма вызовов модели, CPU и памяти окружений для многораундовых задач.** Платформа получает 8 задач в секунду: половина выполняется за два раунда, другая половина — за четыре. Каждый вызов модели занимает 6 секунд, а инструмент использует одно ядро CPU в течение 1 секунды. Найдите частоту вызовов модели, среднее количество параллельных вызовов модели и среднее количество используемых ядер CPU. Если каждое окружение занимает 2 GiB на протяжении всей задачи, найдите средний объём памяти, занимаемый окружениями платформы. Затем предположите, что окружение сохраняется только на двухсекундном этапе подготовки и односекундном эта-

пе выполнения инструмента каждого раунда. Пересчитайте среднее потребление памяти и количество окружений, которые необходимо создавать в секунду.

2. **Эксперимент 11-2 (основной): как полное воссоздание окружения и восстановление из снимка влияют на ожидание инструмента и длительность использования памяти?** В окружении находятся зависимости, которые можно загрузить заново, несохранённый буфер редактирования, сохранённый патч и зафиксированные внешние записи. Для каждого элемента укажите, откуда его можно восстановить после сбоя и какие данные нельзя восстановить напрямую.

Затем сравните два способа восстановления состояния, необходимого для следующего раунда: полное воссоздание окружения по холодному пути занимает 2,7 секунды; согласно документации E2B, сохранение снимка окружения объёмом 2 GiB занимает около 8 секунд, а восстановление из снимка — около 1 секунды. Примите момент завершения выполнения инструмента в предыдущем раунде за начало отсчёта. Следующий вызов модели начинается немедленно и занимает 9 секунд, после чего инструмент выполняется 1 секунду. При воссоздании исходное окружение освобождается в момент начала отсчёта; при использовании снимка его сохранение начинается в этот момент, а исходное окружение освобождается после завершения сохранения.

В обоих вариантах начинайте воссоздание или восстановление как можно позже, не откладывая следующее выполнение инструмента. Постройте временную шкалу и рассчитайте для каждого варианта суммарную длительность использования памяти окружением от начала отсчёта до завершения следующего выполнения инструмента. Время воссоздания, сохранения снимка, восстановления и выполнения инструмента учитывается как время резидентности окружения.

3. **Эксперимент 11-3: сколько времени ожидания экономит предварительная подготовка окружения и сколько памяти она занимает?** Создание окружения занимает 2 секунды, а точность предсказания окружения, необходимого для следующего вызова, равна 0,6. После поступления вызова окружение, соответствующее ошибочному прогнозу, отменяется; каждое предварительно подготавливаемое окружение занимает 2 GiB с начала подготовки. Для начала подготовки за 1, 2 и 4 секунды до поступления вызова рассчитайте ожидаемое время ожидания после его поступления и ожидаемое суммарное потребление памяти из-за ошибочной подготовки в GiB-с. При ошибочном прогнозе необходимое окружение создаётся заново после поступления вызова. Если точность возрастёт до 0,9, какие преимущества увеличатся, а какие потери уменьшатся?

4. **Эксперимент 11-4: когда миграция предпочтительнее ожидания при разных целях завершения задач.** Ресурс, необходимый новой задаче, занят существующей задачей; до его освобождения нужно ждать 12 секунд. Получив ресурс, новая задача выполняется локально за 20 секунд. Существующую задачу также можно перенести на другой узел и освободить локальный ресурс после завершения миграции; неизвестная длительность миграции равна m , а время завершения существующей задачи увеличивается на $m + 2$ секунды по сравнению с вариантом без миграции. Сначала, стремясь завершить новую задачу как можно раньше, найдите условие, при котором миграция предпочтительнее ожидания. Затем решите задачу заново, минимизируя сумму времени ожидания ресурса новой задачей и задержки завершения существующей задачи. Объясните, почему для этих двух целей могут быть выбраны разные варианты.

5. **Эксперимент 11-5: как время запуска влияет на полезную выработку короткоживущих экземпляров.** Перед началом работы каждому новому экземпляру требуется в общей сложности 8 секунд на передачу и загрузку, после чего он работает со скоростью 100 полезных токенов в секунду. Время от начала запуска до освобождения трёх экземпляров составляет соответственно 10, 20 и 40 секунд. Рассчитайте выработку каждого экземпляра и долю фактического рабочего времени в общей продолжительности его существования. Если время подготовки сократить до 4 секунд, сравните прирост выработки каждого из трёх экземпляров и объясните, почему экземпляры с меньшим доступным временем более чувствительны.
6. **Эксперимент 11-6: проверка влияния количества процессов и образцов из длинного хвоста на время завершения всего пакета.** Три образца поступают на 0-й, 6-й и 12-й секундах, а их проверка занимает соответственно 10, 2 и 8 секунд. Сначала определите моменты начала и завершения проверки каждого образца, если имеется только один процесс-исполнитель и образцы обрабатываются в порядке поступления. Затем определите наиболее ранний момент завершения всего пакета при использовании двух процессов-исполнителей. Измените время проверки последнего образца на 80 секунд, определите, способно ли увеличение числа процессов-исполнителей сократить время завершения всего пакета, и объясните, какой образец определяет результат.
7. **Эксперимент 11-7: как рассчитать стоимость и вероятность своевременного успеха, если попадание в кэш связано с корректностью результата?** Используйте стоимость сервиса В из примера 11-7, но задайте вероятность успеха 99% при попадании в кэш и 90% при промахе. Выведите зависимость средней стоимости одной успешной задачи от доли попаданий в кэш, а также долю задач, проходящих проверку в течение 6 секунд. Найдите минимальную долю попаданий, необходимую для того, чтобы не менее 90% задач своевременно проходили проверку, и сравните результат с исходной моделью независимых вероятностей.
8. **Эксперимент 11-8: точка равенства стоимости успешной задачи при собственной инфраструктуре и использовании API.** Для собственной инфраструктуры по ценам Runpod резервируются четыре B200 на месяц общей стоимостью 19 555,2 доллара, запускается V4-Flash, дополнительные расходы на отдельные задачи отсутствуют, а вероятность успеха задачи равна 90%. При использовании API применяется Claude Sonnet 5: для каждой задачи выполняются три вызова, стоимость каждого совпадает со стоимостью сервиса В при попадании в кэш из примера 11-7 и равна 0,0088 доллара, а вероятность успеха составляет 98%. Сравните стоимость за один месяц, полностью отнеся фиксированные расходы на этот месяц. Считая, что оба варианта обладают достаточной производительностью для соблюдения сроков и обрабатывают одинаковое количество задач, найдите количество задач, при котором их средняя стоимость одной успешной задачи совпадает. Затем установите для собственной инфраструктуры предел в 500 тысяч задач в месяц и обсудите, как точка равенства влияет на решение о покупке.
9. **Эксперимент 11-9: как срок выполнения задачи и досрочная остановка изменяют стоимость повторных попыток и вероятность успеха.** Используйте условия примера 11-8, изменив срок на 14, 18 и 22 секунды, и для каждого случая найдите долю задач, своевременно проходящих проверку. Затем разработайте стратегию, которая прекращает выполнение, если оставшегося времени недостаточно для завершения следующего узла, заново рассчитайте ожидаемую стоимость и объём

ясните отличие этой стратегии от простого изменения срока оценки.

10. **Эксперимент 11-10 (основной): как масштабировать ресурсы и выбирать стратегию резидентности окружений после увеличения частоты поступления задач.** Используйте условия примера 11-9, увеличив частоту поступления до 15 задач в секунду. Каждый вызов теперь выводит только 118 токенов, длительность быстрого вызова сервиса снижается примерно до 2,0 секунды, каждая задача по-прежнему выполняется за три раунда, а выполнение инструмента в каждом раунде по-прежнему занимает 1 секунду. Хосты инструментов, количество сеансов на реплику и цены за единицу остаются прежними. Сравните постоянную резидентность с воссозданием окружения в каждом раунде, определите, какие ресурсы и как минимум в каком объёме следует добавить, и укажите, нужно ли менять стратегию окружений. Наконец, измените режим поступления так, чтобы каждую секунду одновременно поступало 15 задач, постройте временную шкалу выполнения инструментов и отметьте пиковое параллельное использование CPU.

11.7 Итоги главы

На платформе из начала главы в среднем используются лишь 30 ядер CPU, однако среде требуется 600 GiB памяти, что превышает 358 GiB у m5d.metal. Причина в том, что за три раунда задача 27 секунд ожидает модель, а инструментальная среда всё это время продолжает занимать память. Если подготавливать среду во время вызова модели, суммарное потребление памяти на одну задачу снижается с 60 до 18 GiB-с, высвобождая значительный объём памяти; цена — лишь дополнительные 0,3 секунды процессорного времени на подготовку.

Это улучшение использует зависимости выполнения полной задачи. Определив доступное для подготовки время в период вызова модели, менеджер сред может заранее создать среду, чтобы процесс существовал только во время подготовки и выполнения инструментов; состояние, которое необходимо сохранять между раундами, записывается в постоянные файлы. Если сервис модели и платформа сред совместно планируют момент подготовки, ресурсы, ранее занятые на протяжении всей задачи, начинают выделяться поэтапно.

После высвобождения памяти задача всё ещё выполняется 30 секунд, поэтому вызов модели становится ключевым фактором для соблюдения предельного времени в 24 секунды. Если уменьшить число сеансов на каждую реплику V4-Flash с 32 до 16, длительность вызова сократится с 9 до 6 секунд, а трёхраундовой задачи — с 30 до 21 секунды; среднее число одновременных вызовов модели также снизится с 270 до 180. Цена — дополнительные 0,375 B200-с на каждый вызов и увеличение числа реплик с 10 до 12. В сочетании с пересозданием среды в каждом раунде одного экземпляра m5d.metal с 48 ядрами и 358 GiB памяти достаточно для поддержки заданного потока задач. После этого сравнивается стоимость вариантов, удовлетворяющих требованиям к качеству и сроку выполнения.

Тот же метод объясняет и организацию ресурсов для RL: группы заданий сначала получают согласованный набор ресурсов; временные экземпляры rollout заблаговременно завершают передачу и загрузку весов; сгенерированные образцы сразу отправляются на проверку, благодаря чему она может выполняться одновременно с последующей генерацией. При повторных попытках после сбоя необходимо суммировать объём работы всех

попыток. Для анализа потребности в ресурсах нужно вычислить общий объём работы, выполненной задачей; при анализе времени завершения также необходимо учитывать, в каком порядке должна выполняться эта работа и какие её части могут выполняться одновременно. Только сочетание этих двух аспектов позволяет определить, полезно ли локальное улучшение для всей системы.

Совместная работа устройств, периферии и облака

Изображение отправляется в облако для финальной обработки: модель работает 0,3 секунды, но пользователь ждёт 12,8 секунды. Десятикратное ускорение модели сэкономит лишь 0,27 секунды, тогда как уменьшение входного файла вдвое позволит сэкономить почти 6 секунд. При выборе места размещения модели необходимо сравнивать полное время выполнения задачи. Удалённое устройство может ускорить вычисления модели, однако обращение к удалённому сервису также увеличивает время передачи и ожидания. Централизованное развёртывание в облаке оправдано лишь тогда, когда сэкономленное благодаря удалённому выполнению время вычислений превышает дополнительные затраты на передачу и ожидание. После ускорения инференса моделей многие интерактивные задачи уже не удовлетворяют этому условию.

В главах 8–11 были рассмотрены сервисы моделей, планирование ресурсов и среды выполнения задач. В этой главе обсуждается развёртывание этих сервисов и сред на различных устройствах. **Устройство** получает входные данные и взаимодействует с пользователем; **периферия** — это расположенная поблизости рабочая станция, предоставляющая вычислительные ресурсы, или расположенный поблизости сервисный узел, предоставляющий вычислительные ресурсы; **облако** предоставляет удалённый пул ресурсов. Конечное устройство может выполнять задачу локально, обращаться к расположенной поблизости рабочей станции или облачному сервису, а также распределять разные этапы модели между различными устройствами.

Сначала в этой главе на примере финальной обработки изображения строится временная модель, затем на примере речи объясняются зависимости порядка выполнения в конвейере, а в завершение выбирается схема развёртывания для агента, который по скриншоту определяет следующий шаг и вызывает инструмент для выполнения операции. Для финальной обработки изображения необходимо вычислить, когда вернётся готовый результат; для речи — проанализировать, сможет ли аудио воспроизводиться непрерывно; для агента — суммировать затраты на связь и выполнение в каждом раунде. Для всех трёх типов задач используется единый метод анализа: сначала вычисляется длительность обработки и передачи, затем определяются этапы, которые обязательно должны ожидать завершения других этапов, после чего учитываются практические затраты на установление соединения, ожидание в очереди и восстановление после сбоев.

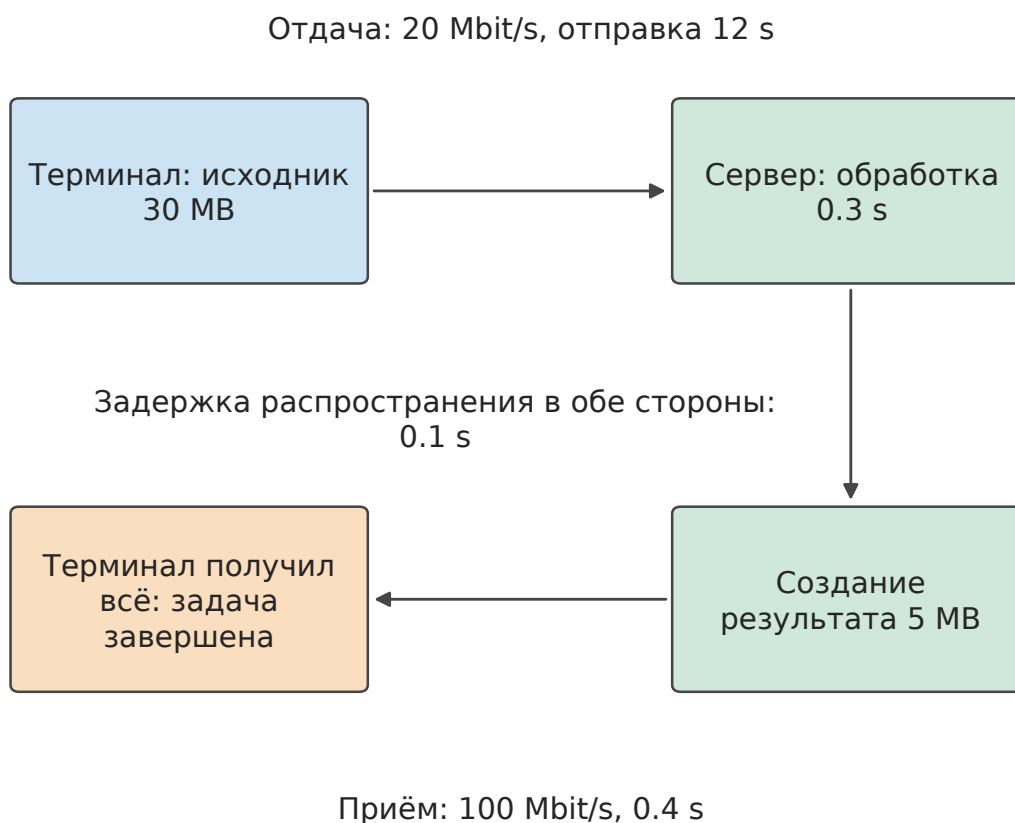


Рис. 12-1. Исходное изображение передаётся на сервер по восходящему каналу, после полной загрузки обрабатывается и целиком возвращается по нисходящему каналу. Сплошные стрелки показывают порядок передачи данных и обработки; соединение уже установлено, суммарное время распространения в обе стороны составляет 0,1 s.

12.1 От времени работы модели до полного времени взаимодействия

12.1.1 Ретуширование изображений

При ретушировании изображений необходимо сохранять информацию о тенях, светах, балансе белого и других параметрах. Чтобы оставить больше исходных данных для последующего редактирования, обычно приходится передавать файлы большего размера. В этом примере загружается исходное изображение размером 30 МВ, а после обработки возвращается готовое изображение размером 5 МВ; задача считается завершённой, когда пользователь полностью его получает.¹

Сначала проследим путь одного изображения и посмотрим, где задерживаются данные. Конкретные скорости и длительность обработки в этом примере показаны на рис. 12-1, после чего связь между этими величинами будет выражена общей формулой.

При построении модели сначала следует различить **время, необходимое для каж-**

¹Пример и бюджет условий для ретуширования RAW-изображения. В примере размер исходного изображения составляет 30 МВ, готового изображения — 5 МВ, а обработка изображения занимает 0.3 секунды.

дого этапа, и время завершения задачи. Деление размера файла на скорость передачи даёт время, необходимое для его отправки; время вычислений модели определяется объёмом вычислений и скоростью выполнения процессора. Если этапы выполняются последовательно, общее время равно сумме их длительностей. Если некоторые этапы выполняются одновременно, по зависимостям между ними нужно определить, когда завершится последний этап.

Прежде чем записать эти зависимости в виде формул, зададим упрощающие условия примера: время прохождения сигнала туда и обратно (round-trip time, RTT) считается равным времени распространения в обе стороны, а затраты времени на очереди и обработку протокола отдельно рассматриваются в разделе 12.3. Соединение уже установлено; сервер начинает обработку после полного получения исходного изображения и отправляет пользователю готовое изображение только после его полного формирования; полезная нагрузка передаётся по восходящему и нисходящему каналам с постоянной скоростью. Пусть размер входных данных равен S_u bytes, выходных — S_d bytes, скорости восходящего и нисходящего каналов — B_u, B_d bytes/s, RTT равен R , а время обработки — T_c . Тогда время последовательного выполнения равно

$$T_{\text{serial}} = \frac{S_u}{B_u} + R + T_c + \frac{S_d}{B_d}.$$

После того как последний байт загружаемого файла покинет отправителя, он ещё должен дойти до сервера; готовое изображение при возврате также один раз проходит путь распространения. Суммарное время двух однонаправленных распространений равно R . Время распространения определяется маршрутом, а время отправки — размером файла и скоростью, поэтому уменьшение файла сокращает только время отправки. Здесь B_u, B_d обозначают скорости передачи полезной нагрузки; запуск соединения и ожидание обратной связи будут отдельно учтены в разделе 12.3.

Пример: как время загрузки изображения для ретуширования ограничивает выигрыш от ускорения модели? Скорость восходящего канала — 20 Mbit/s, нисходящего — 100 Mbit/s, $R = 0.1$ s, $T_c = 0.3$ s. В этой главе МБ и Mbit используются в десятичном исчислении; MiB равен 2^{20} bytes. После подстановки получаем

$$T_{\text{serial}} = \frac{30 \times 8}{20} + 0.1 + 0.3 + \frac{5 \times 8}{100} = 12.8 \text{ s}.$$

Загрузка занимает 12 секунд, то есть около 94% общего времени. Десятикратное ускорение этапа обработки сокращает время выполнения до 12,53 секунды, экономя лишь 0,27 секунды. Даже при мгновенной обработке загрузка, распространение и скачивание всё равно займут 12,5 секунды. Именно это ограничение описывает закон Амдала: этап обработки занимает лишь около 2,3% общего времени, поэтому даже полное устранение этих затрат сокращает общее время всего примерно на 2,3%.

Ускорение модели даёт ограниченную экономию времени; другой путь — сократить передачу. Если повысить скорость восходящего канала до 100 Mbit/s, загрузка займёт 2,4 секунды, а вся задача — только 3,2 секунды, то есть выполнится в четыре раза быстрее. Другой вариант — сохранить прежнюю скорость восходящего канала, но сжать входные данные. Предположим, сжатие без потерь уменьшает входные данные вдвое, а дополнительные кодирование и декодирование вместе занимают 0,15 секунды. Тогда отправка займёт только 6 секунд, а вся задача — около 7,0 секунды. Сжатие выгодно, поскольку

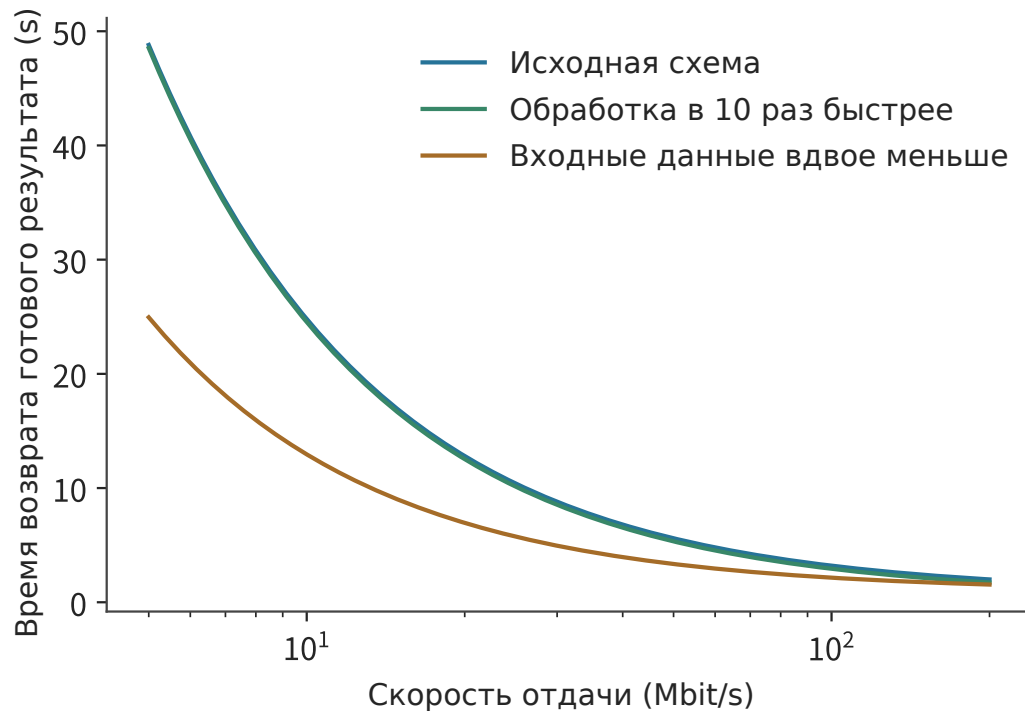


Рис. 12-2. Влияние сжатия и ускорения вычислений на время завершения задачи зависит от скорости восходящего канала. В этом примере используются исходное изображение размером 30 МВ, готовое изображение размером 5 МВ, нисходящий канал 100 Mbit/s, RTT 0,1 секунды и исходное время обработки 0,3 секунды; вариант со сжатием уменьшает входные данные вдвое и добавляет 0,15 секунды на кодирование и декодирование. Качество готового изображения во всех трёх вариантах одинаково, соединение уже установлено, а этапы выполняются последовательно.

0,15 секунды дополнительных затрат на кодирование и декодирование экономят 6 секунд передачи.

Пусть размер сжатых входных данных равен S'_u , а дополнительное время кодирования и декодирования — T_e . Тогда общее время, сэкономленное благодаря сжатию, равно

$$\Delta T = \frac{S_u - S'_u}{B_u} - T_e.$$

Приравняв выигрыш в этой формуле к нулю, можно найти скорость восходящего канала, при которой сжатие перестаёт экономить время. При скорости восходящего канала 20 Mbit/s передача на 15 МВ меньше экономит 6 секунд; при 100 Mbit/s — 1,2 секунды; при 800 Mbit/s — лишь 0,15 секунды, и выигрыш становится нулевым. Объём вычислений одного и того же метода сжатия не меняется; меняется время, необходимое для отправки этих байтов. Эта зависимость показана на рис. 12-2.

Теперь рассмотрим возможность перекрытия этапов. Предположим, исходное изображение можно разделить на три независимо обрабатываемых блока. Загрузка каждого блока занимает 4 секунды, а обработка — 0,1 секунды; размеры трёх выходных блоков равны 1, 2 и 2 МВ, поэтому их обратная передача последовательно занимает 0,08, 0,16 и 0,16 секунды. После поступления первого блока на сервер его обработка и обратная передача могут перекрываться с загрузкой второго блока; то же относится ко второму блоку. Отправка последнего блока всё равно завершается только на 12-й секунде, после чего

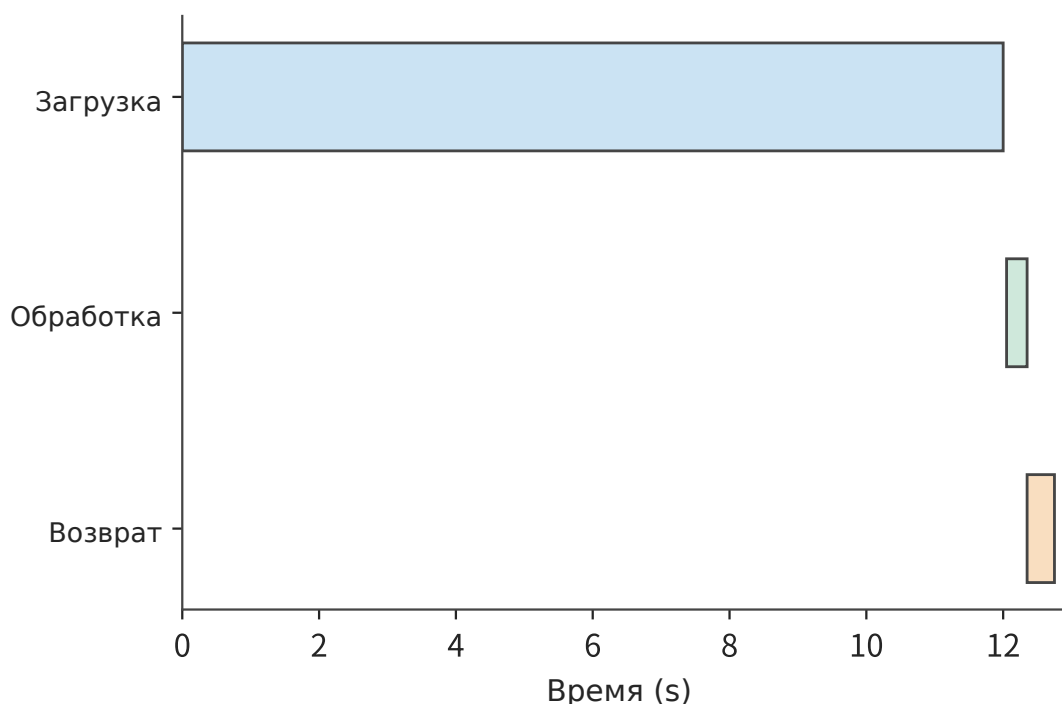


Рис. 12-3. Последовательная обработка целого изображения: обработка начинается только после загрузки всех 30 МВ и распространения данных до сервера, после чего обратно передаются 5 МВ. Скорость восходящего канала — 20 Mbit/s, нисходящего — 100 Mbit/s, однонаправленное распространение занимает 0,05 с; задача завершается через 12,8 с.

ещё требуются обработка, обратная передача и распространение, поэтому пользователь получает полное готовое изображение примерно через 12,4 секунды. По сравнению с исходными 12,8 секунды обработка и обратная передача первых двух блоков завершаются во время загрузки последующих и больше не увеличивают ожидание; однако суммарные 12 секунд загрузки сократить невозможно.²

На рис. 12-3 и 12-4 два варианта сравниваются на одной шкале времени.

Если для обработки требуется глобальная экспозиция или информация из других блоков, она может начаться только после поступления всего изображения. В этом случае даже при передаче по блокам вся задача по-прежнему занимает 12,8 секунды. Объём вычислений и данных определяет время отдельных этапов, а зависимости — возможность их одновременного выполнения. Сервис обработки речи в реальном времени непрерывно принимает, обрабатывает и воспроизводит аудио, поэтому такие зависимости удобнее описывать конвейером.

12.1.2 Речь в реальном времени

Автоматическое распознавание речи (ASR) преобразует аудио в текст, а синтез речи (TTS) — текст в воспроизводимое аудио. Поточковый сервис может формировать резуль-

²Результаты двух расчётов для трёх блоков составляют соответственно 12,8 с и 12,36 с; в основном тексте разница около 0,4 с используется для объяснения выигрыша от перекрытия. Дополнительный предварительный просмотр требует дополнительного кодирования и передачи. Источники: барьер целого изображения, независимые блоки и дополнительный предварительный просмотр.

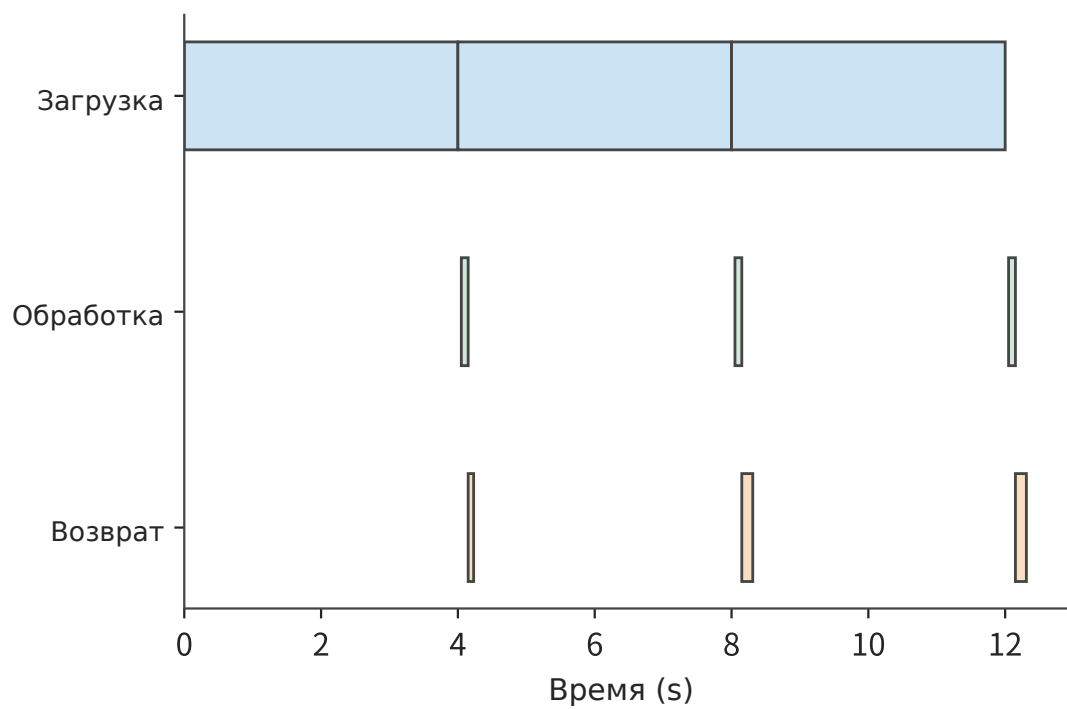


Рис. 12-4. Три блока загружаются по 4 с каждый, после поступления каждый независимо обрабатывается 0,1 с, а затем обратно передаются блоки размером 1, 2 и 2 МВ. Обработка и обратная передача первых двух блоков перекрываются с загрузкой последующих; полное готовое изображение возвращается через 12,36 с.

тат одновременно с приёмом входных данных. Например, при TTS пользователь может начать прослушивание до завершения генерации всего аудиофрагмента. Поэтому нужно знать не только момент окончания синтеза всего фрагмента, но и момент начала воспроизведения первого отрезка, а также непрерывность последующего воспроизведения.

Чтобы вычислить эти моменты, сначала запишем конвейер в виде рекуррентных соотношений для отдельных блоков. Рассмотрим конвейер обработки аудио, в котором каждому входному блоку соответствует один выходной. Устройство захвата каждые 20 ms формирует блок аудио, модель обрабатывает его и отправляет результат, а проигрыватель воспроизводит блоки по порядку. Входной блок i готов в момент a_i , обработка моделью занимает m_i , отправка — s_i , а однонаправленное распространение — d_i . Если процессор обрабатывает аудио блок за блоком, а сетевой интерфейс так же последовательно отправляет результаты, то

$$c_i = \max(a_i, c_{i-1}) + m_i, \quad u_i = \max(c_i, u_{i-1}) + s_i, \quad r_i = u_i + d_i.$$

Здесь c_i , u_i и r_i обозначают соответственно моменты завершения обработки, завершения отправки и поступления. Первый максимум означает, что для обработки текущего блока нужно дождаться как готовности входных данных, так и завершения обработки предыдущего блока. Второй максимум означает, что для отправки текущего блока нужно дождаться как завершения вычислений, так и освобождения исходящего канала. Эти два ожидания обусловлены соответственно зависимостью данных и конкуренцией за ресурс.

Пример: почему колебания сетевой задержки могут исчерпать буфер воспроизведения аудио? Импульсно-кодовая модуляция (PCM) последовательно записывает отсчёты амплитуды аудиосигнала; 24 kHz означает 24 000 отсчётов в секунду, монофонический звук — один канал отсчётов, а 16-bit — 2 байта на отсчёт. При PCM 24 kHz, моно, 16-bit каждый блок длительностью 20 ms занимает 960 bytes. Обработка одного блока занимает 12 ms, отправка — 1 ms, а обычное время распространения — 5 ms. Захват первого блока завершается через 20 ms, блок поступает через 38 ms и после буферизации в течение 40 ms воспроизводится в момент 78 ms (рис. 12-5). Второй блок поступает в момент 58 ms и следом воспроизводится в момент 98 ms. Обработка каждого блока на 8 ms короче интервала захвата, а отправка занимает всего 1 ms, поэтому обработка и отправка успевают за скоростью захвата.

Изменим время распространения третьего блока на 50 ms. Этот блок поступает в момент 123 ms, на 5 ms позже запланированного момента воспроизведения 118 ms. Хотя четвёртый блок поступает уже в момент 98 ms, проигрыватель должен сначала воспроизвести третий, поэтому всё последующее воспроизведение также задерживается (рис. 12-6).³

Обобщим это правило воспроизведения: пусть длительность каждого аудиоблока равна τ , а начальная буферизация — J . Первый блок начинает воспроизводиться в момент $p_0 = r_0 + J$, а время начала воспроизведения последующих блоков определяется соотношением

$$p_i = \max(r_i, p_{i-1} + \tau).$$

³Расчёт временной последовательности аудиоблоков. В этой упрощённой модели входные кадры однозначно соответствуют выходным аудиоблокам, что используется для объяснения зависимостей поблочной обработки и последовательного воспроизведения.

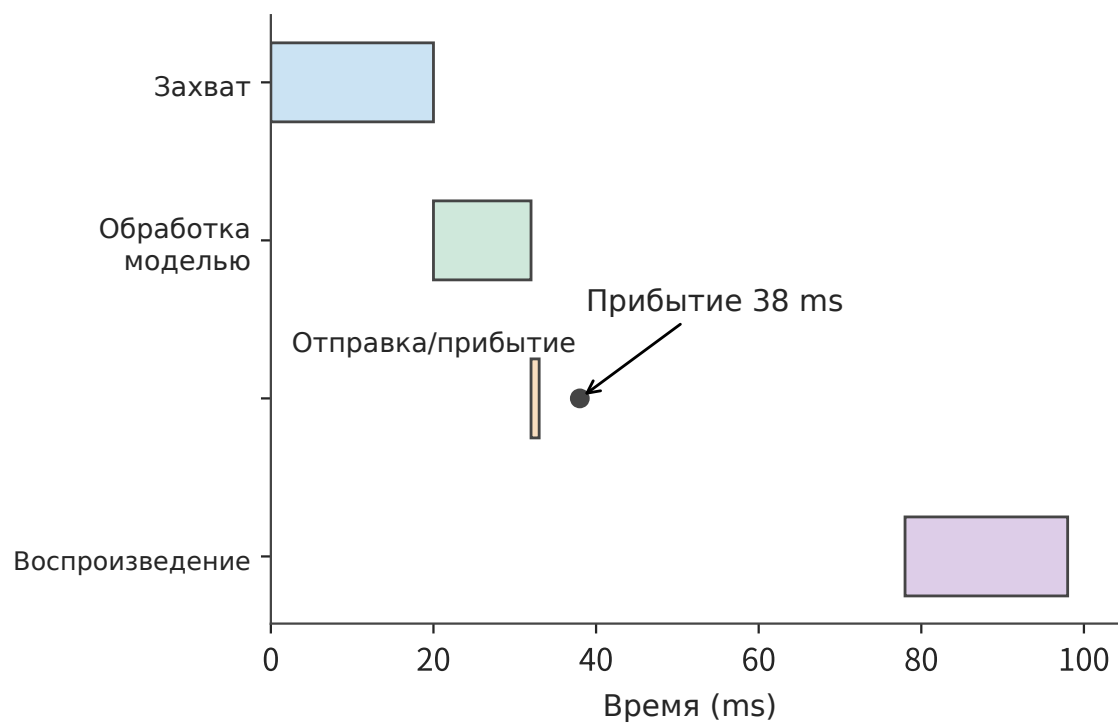


Рис. 12-5. Отсчёт времени для первого аудиоблока начинается с захвата: готов через 20 ms, обработан через 32 ms, отправлен через 33 ms, поступает через 38 ms и после начальной буферизации в течение 40 ms воспроизводится в момент 78 ms. Точками отмечено поступление, последняя строка показывает часы устройства воспроизведения.

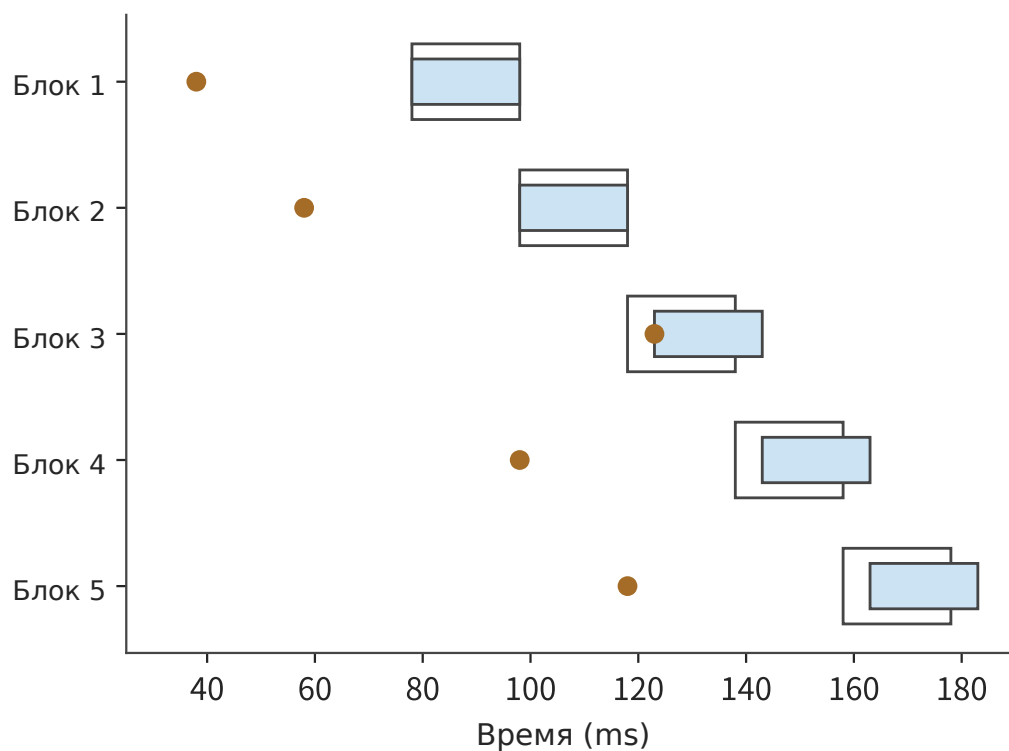


Рис. 12-6. Из-за опоздания третьего блока воспроизведение всех последующих блоков задерживается на 5 ms. Каждый блок длится 20 ms, обработка моделью занимает 12 ms, отправка — 1 ms; обычное время распространения равно 5 ms, для третьего блока — 50 ms, начальная буферизация — 40 ms. Сплошные отрезки показывают фактическое воспроизведение, светлые контуры — запланированные интервалы воспроизведения, точки — поступление; всё время отсчитывается от начала захвата первого блока. На рисунке показан результат вычисления для описанного выше конвейера: проигрыватель ожидает недостающий блок, а не отбрасывает его.

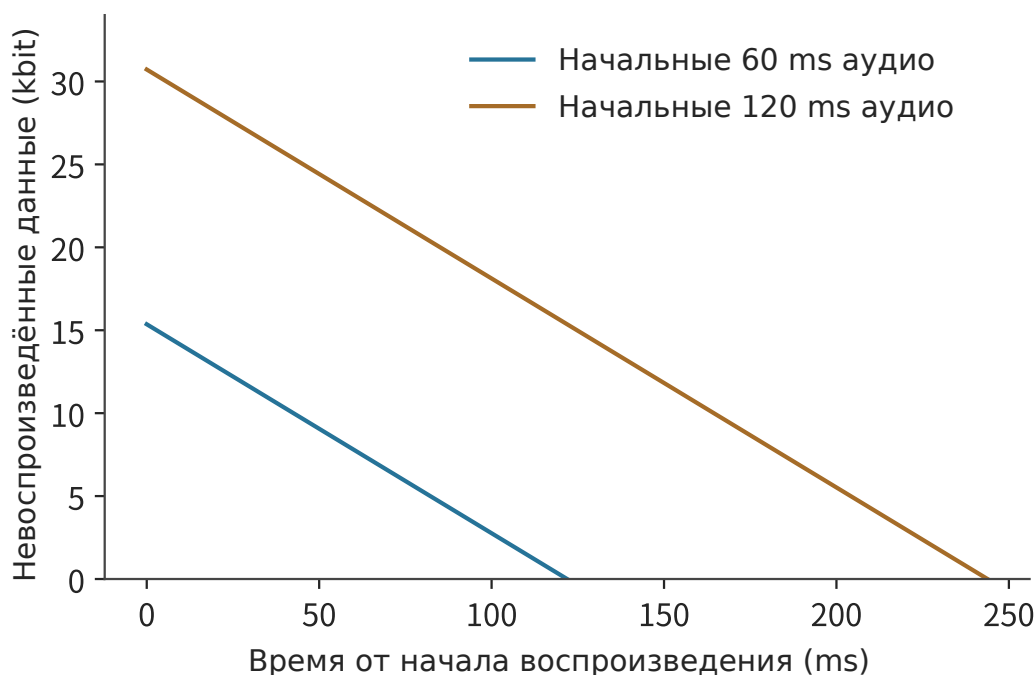


Рис. 12-7. Объем буфера равен начальному объёму данных плюс совокупный объём принятых данных минус совокупный объём воспроизведённых данных. Для воспроизведения PCM требуется 256 kbit/s, а скорость приёма составляет лишь 130 kbit/s, поэтому скорость уменьшения на обеих прямых равна 126 kbit/s. Начальные буферы содержат соответственно 60 ms и 120 ms аудио и исчерпываются примерно через 0,12 и 0,24 секунды; каждая линия показана до момента первого исчерпания.

Здесь берётся максимум, поскольку проигрыватель должен дожидаться как поступления текущего блока, так и завершения воспроизведения предыдущего. Когда первый аргумент больше, разность представляет собой новую паузу. Если увеличить начальную буферизацию с 40 до 45 ms, запланированный момент воспроизведения третьего блока также сдвинется до 123 ms, что позволит избежать прерывания из-за опоздания на 5 ms; цена этого — начало воспроизведения первого фрагмента на 5 ms позже. Увеличение начальной буферизации продлевает ожидание перед первым воспроизведением, но позволяет переносить более значительные колебания времени поступления и делает последующее воспроизведение непрерывнее.

Длительная нехватка пропускной способности вызывает другой тип пауз. Для удобства расчёта возьмём PCM 16 kHz, моно, 16-bit: воспроизведение требует скорости передачи данных 256 kbit/s, а 20 ms соответствуют 640 bytes. Если сеть длительное время может передавать аудио лишь со скоростью 130 kbit/s, объём буфера каждую секунду уменьшается на 126 kbit. Начальный буфер с 60 ms аудио содержит 15 360 bits и будет исчерпан примерно за $15360 / (256000 - 130000) \approx 0.12$ секунды.

Если изобразить объём ещё не воспроизведённых данных в виде убывающей со временем кривой, станет видно различие между кратковременными колебаниями задержки и длительной нехваткой скорости передачи. На рис. 12-7 увеличение буфера лишь поднимает начальную точку, не меняя наклона.

Увеличение буфера лишь пропорционально отодвигает момент его исчерпания. Чтобы принимаемых данных хватало для воспроизведения, нужно повысить скорость пере-

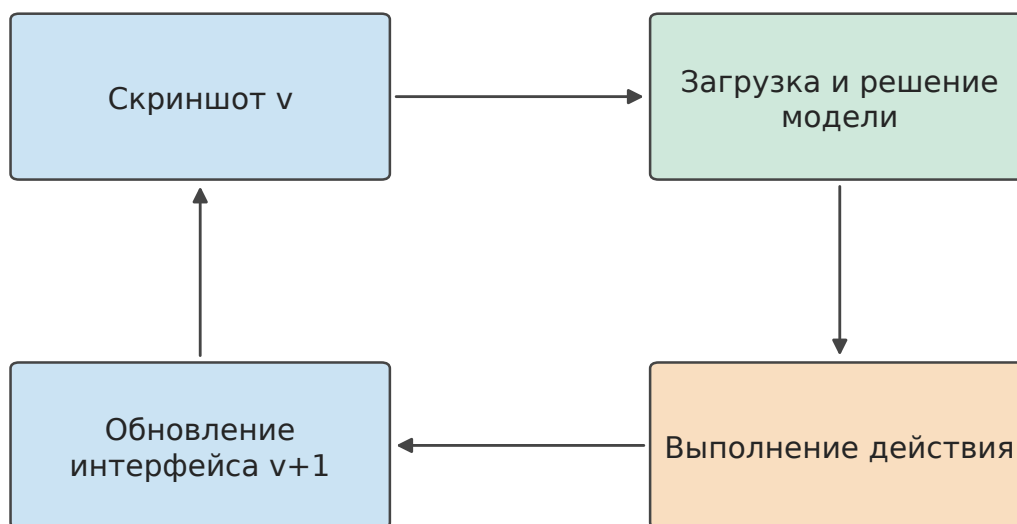


Рис. 12-8. Один раунд работы AI-агента со снимками экрана. Стрелки показывают зависимости очередности, ширина прямоугольников не отражает длительность. Чтобы получить снимок экрана для следующего раунда, нижняя обратная стрелка должна пройти через выполнение действия и обновление интерфейса.

дачи или снизить битрейт аудио. Буфер смягчает влияние кратковременных колебаний задержки на воспроизведение, тогда как длительная нехватка скорости передачи требует увеличить пропускную способность или уменьшить объём данных.

12.1.3 Computer Use: выполнение задач через интерфейс

Computer Use (управление интерфейсом компьютера) позволяет программе выполнять задачи подобно пользователю — с помощью снимков экрана, щелчков и ввода. Программа циклически наблюдает интерфейс, получает решение модели, выполняет действие и снова наблюдает интерфейс. Ключевое отличие таких задач от аудиоконвейера состоит в том, что следующий актуальный снимок экрана можно получить только после завершения предыдущего действия и обновления интерфейса. Пока модель не решила, куда щёлкнуть, невозможно получить интерфейс после щелчка. Поэтому ожидание в каждом раунде многократно накапливается в течение задачи.

Обратная стрелка на рис. 12-8 показывает, почему эту задачу трудно выполнять с блочным перекрытием, как аудио: последующая работа должна ожидать появления нового интерфейса в результате действия.

Рассмотрим задачу, требующую 30 раундов действий. При использовании исходного сервиса в каждом раунде загружается снимок экрана размером 0,8 МВ; при скорости восходящего канала 6,4 Mbit/s загрузка занимает 1 секунду. Наблюдение на терминале, действие и обновление интерфейса вместе занимают 0,3 секунды, удалённая модель —

2,0 секунды, распространение туда и обратно — 0,2 секунды. Соединение уже установлено, время отправки управляющего ответа считается нулевым. Один раунд занимает $1 + 0.3 + 2.0 + 0.2 = 3.5$ секунды, а 30 раундов — 105 секунд.

Пример: компенсирует ли сэкономленное благодаря сжатию снимков время передачи дополнительные раунды действий? Если сжать снимок экрана до 0,2 МВ, затратив дополнительные 30 ms на кодирование, время загрузки сократится до 0,25 секунды, а остальная работа по-прежнему будет занимать 2,5 секунды. Тогда

$$T'_{\text{round}} = 0.25 + 0.03 + 2.5 = 2.78 \text{ s.}$$

Каждый раунд становится короче на 0,72 секунды, а экономия за 30 раундов составляет 21,6 секунды, сокращая время задачи со 105 примерно до 83 секунд.

Сжатие также может изменить число раундов действий. Если из-за размытия текста агенту придётся выполнять дополнительные исправляющие действия, общее время после сжатия следует записать как $2.78N'$, где N' — фактически необходимое число раундов. Из $2.78N' < 105$ следует $N' < 37.8$, поэтому максимальное число раундов, при котором задача всё ещё выполняется быстрее, равно 37; 38 раундов займут около 106 секунд. Качество изображения после сжатия влияет на число исправлений. В этой задаче допустимо добавить не более семи раундов действий; при большем числе время исправлений сведёт на нет экономию от сжатия.

Версия снимка экрана также непосредственно влияет на число раундов. Предположим, модель получила версию v , после чего интерфейс изменился. Тогда действие, выбранное по устаревшему снимку, может оказаться ошибочным, из-за чего придётся восстанавливать интерфейс и делать новый снимок. Если связать версию снимка экрана с номером действия и поручить терминалу повторно снимать экран при изменении интерфейса, такой бесполезной работы можно избежать.⁴

В разделе 12.5 анализ этой задачи со снимками экрана будет продолжен: первые десять раундов уже завершены, а для оставшихся двадцати нужно выбрать место выполнения. Размер снимка экрана, работа терминала и параметры исходного маршрута останутся неизменными, чтобы сравнить, насколько более быстрый сервис модели сокращает время.

12.1.4 Предварительное сравнение локального устройства, близлежащего устройства и облака

Анализ из разделов 12.1.1–12.1.3 можно свести к трём уровням, каждому из которых соответствует свой объект вычисления:

Уровень анализа	Объект вычисления	Полученный вывод
Длительность отдельных этапов	Сколько занимают вычисления, отправка и распространение	Загрузка исходного изображения занимает 12 секунд, работа модели — 0,3 секунды

⁴Материалы и технические оценки автора. АОI и Latent Bridge используются, чтобы показать, какую информацию интерфейса следует наблюдать и какие данные должны передаваться между устройствами.

Уровень анализа	Объект вычисления	Полученный вывод
Зависимости выполнения	Какие работы могут перекрываться, а какие должны ожидать	Независимые блоки изображения могут обрабатываться с перекрытием, а снимок экрана для следующего раунда должен ждать завершения предыдущего действия
Фактическое выполнение	Когда после готовности данных начинаются вычисления и отправка	Даже при достаточной средней скорости передачи аудио одно опоздание всё равно может вызвать паузу

Сначала по длительности этапов и зависимостям между ними следует определить, может ли удалённое выполнение быть быстрее. При передаче всей задачи удалённому сервису экономия времени вычислений должна превышать дополнительные затраты на загрузку, скачивание, распространение, ожидание в очереди и подготовку. В многораундовой задаче подготовка выполняется только один раз, а обмен данными происходит в каждом раунде; при непрерывном воспроизведении аудио также нужно обеспечить своевременное поступление данных. Место размещения модели одновременно меняет время вычислений, обмена данными и подготовки.

Минимальное время распространения позволяет заранее исключить слишком удалённые серверы. Свет распространяется по оптоволокну примерно со скоростью 2×10^8 m/s, поэтому при однонаправленном маршруте длиной 10 000 km распространение туда и обратно занимает не менее примерно 100 ms. Если удалённый ответ должен поступить за 50 ms, уже одно распространение превышает этот предел. Использование более близкого сервера непосредственно сокращает эту часть ожидания.

В приведённом сравнении конечное устройство рассматривается как единое место выполнения и вычисляется лишь разница во времени между ним и удалённой системой; скорость вычислений самого конечного устройства и доступная ему ёмкость пока не раскрыты. В разделе 12.1.5 сначала количественно анализируются вычислительные ресурсы конечного устройства, а в разделе 12.2 определяется, что именно передавать: при передаче всей задачи удалённой системе отправляется изображение, после кодирования на конечном устройстве — признаки, а при переносе сеанса — состояние. С изменением объекта передачи меняются объём данных и необходимое время.

12.1.5 Вычислительные ресурсы конечного устройства и локальное развёртывание

При сравнении вариантов развёртывания в разделе 12.5 используется значение «2,9 секунды вычислений модели на конечном устройстве за раунд». Чтобы объяснить происхождение этого значения и условия его применимости, нужно количественно проанализировать конечное устройство как вычислительный ресурс: пропускная способность памяти определяет минимальное время генерации одного токена, объём памяти — допустимую модель и длину оставшегося контекста, а энергопотребление и аккумулятор — продолжительность непрерывной работы.

Смартфон как вычислительный ресурс: пропускная способность сначала задаёт верхний предел числа токенов в секунду. В SoC флагманских смартфонов используется память LPDDR5X — стандарт энергоэффективной DRAM для мобильных устройств, включая смартфоны. В описании Snapdragon 8 Elite указаны максимальная

частота 5 300 MHz и максимальный объём 24 GB, но не указана ширина шины; LPDDR5X передаёт данные дважды за такт, поэтому 5 300 MHz соответствуют 10,6 Gbit/s на вывод, что немного ниже максимального скоростного режима устройств LPDDR5X — 10,7 Gbit/s; стандартное устройство JEDEC имеет один канал x16.⁵ При поддерживаемой SoC скорости пропускная способность одного канала x16 равна $16 \times 10.6/8 = 21.2$ GB/s; для четырёх каналов x16 суммарно получается 84,8 GB/s.⁶

При заданной пропускной способности длительность каждого шага зависит от объёма читаемых на этом шаге данных. В качестве фиксированной нагрузки используется пример из раздела 4.8.2: Qwen3-8B, BF16, один запрос, контекст 8K. Один шаг decode считывает 15,14 GB весов и 1,21 GB KV, всего около 16,345 GB.⁷ Одно только чтение весов занимает

$$\frac{15.14 \text{ GB}}{84.8 \text{ GB/s}} \approx 0.179 \text{ s},$$

то есть верхний предел decode составляет около 5,6 token/s; с учётом KV один шаг занимает около 0,193 s, а верхний предел — примерно 5,2 token/s. Квантование непосредственно сокращает число байтов, считываемых на каждом шаге: в групповом формате q4_0 из раздела 8.4 каждые 32 значения BF16 вместо 64 bytes занимают 18 bytes, поэтому объём считываемых на каждом шаге весов уменьшается примерно до $15.14 \times 18/64 \approx 4.26$ GB, суммарный объём на шаг — примерно до 5,47 GB, время шага — до 64,4 ms, а верхний предел — примерно до 15,5 token/s. В MELTing Point на iPhone 14 Pro были измерены 14,8 token/s для Zephyr-3B q4_k и 6,0 token/s для Llama-2 7B q3_k (q4_k и q3_k — 4-битный и 3-битный форматы группового квантования семейства GGML, к которому также относится Q2_K из раздела 8.4.1), что соответствует полученному здесь порядку величин.⁸

Ёмкость определяет, какую модель можно разместить и сколько контекста оставить. Воспользуемся формулой ёмкости из раздела 2.6.2: из доступной памяти вычитаются веса и фиксированный резерв, после чего остаток делится на размер состояния одного запроса. Для памяти смартфона возьмём варианты Micron LPDDR5X объёмом 6, 12 и 24 GB и добавим распространённый во флагманских смартфонах вари-

⁵Описание Snapdragon 8 Elite и описание пятого поколения: поддерживаются LPDDR5X с частотой до 5,300 MHz и ёмкостью до 24 GB, ширина шины не указана. Страница продукта Micron LPDDR5X: максимальный класс скорости — 10.7 Gbit/s, стандарт JEDEC предусматривает одноканальное устройство x16, техпроцесс 1γ добавляет варианты ёмкости 6, 12 и 24 GB, также существует устройство на 16 GB для флагманских телефонов; страница продукта Samsung LPDDR5X также указывает максимум 10.7 Gbit/s. Страница характеристик Apple iPhone 16 Pro не раскрывает ёмкость и скорость памяти.

⁶Разделение бюджета шины и энергопотребления конечного устройства: учитываются четыре канала x16 (declared_phone_channels_x16=4; в описании SoC ширина шины не указана), скорость на контакт соответствует поддерживаемым SoC 10.6 Gbit/s, суммарная пропускная способность — 84.8 GB/s; объёмы непосредственно считываемых весов 15,136,819,200 bytes и KV 1,207,959,552 bytes взяты из результатов 8K decode для одного запроса.

⁷Разделение бюджета шины и энергопотребления конечного устройства: учитываются четыре канала x16 (declared_phone_channels_x16=4; в описании SoC ширина шины не указана), скорость на контакт соответствует поддерживаемым SoC 10.6 Gbit/s, суммарная пропускная способность — 84.8 GB/s; объёмы непосредственно считываемых весов 15,136,819,200 bytes и KV 1,207,959,552 bytes взяты из результатов 8K decode для одного запроса.

⁸MELTing Point: оценка языковых трансформеров на мобильных устройствах, arXiv 2403.12844, §5.2.2 и таблица 5 приложения: 0.16, 0.20 и 0.21 mWh на токен; максимальная продолжительная мощность iPhone 14 Pro составляет 13.8 W, мгновенная превышает 18 W, а продолжительная мощность Galaxy S23 остаётся ниже 8.5 W; на iPhone 14 Pro Zephyr-3B q4_k достигает 14.8 token/s, а Llama-2 7B q3_k — 6.0 token/s; на одном заряде можно выполнить 490.05–590.93 промпта, для S23 — 542.78. Построчная сверка извлечённых значений приведена в таблице извлечения числовых значений.

ант 16 GB; для системы и рабочего пространства зарезервируем 4 GB.⁹ Веса Qwen3-8B в BF16 занимают в сумме 16,38 GB, что примерно на 1,24 GB больше считываемых на каждом шаге 15,14 GB из-за таблицы вложений слов: на каждом шаге decode для токена считывается лишь одна строка, но вся таблица должна постоянно находиться в памяти. По объёму постоянного размещения модель помещается только в вариант с 24 GB, где после вычета резерва остаётся около 3,62 GB — достаточно лишь для двух запросов с контекстом 8K. После квантования до q4_0 веса занимают около 4,61 GB:

Память смартфона	После вычета резерва и 4,61 GB весов	Допустимые запросы и контекст
6 GB	Около —2,61 GB	Модель 8B не помещается; модель класса 3B в q4 занимает около 1,7 GB и помещается
12 GB	Около 3,39 GB	Два запроса с контекстом 8K или один примерно с 23 тыс. токенов
16 GB	Около 7,39 GB	Шесть запросов с контекстом 8K или один примерно с 50 тыс. токенов
24 GB	Около 15,39 GB	Двенадцать запросов с контекстом 8K или один примерно со 104 тыс. токенов

BF16 KV для одного запроса с контекстом 8K занимает 1,21 GB, или 147 456 bytes на токен, и линейно растёт с длиной контекста; после размещения модели оставшаяся память непосредственно определяет длину контекста. Вариант с 16 GB также является пороговым для 8-битного квантования: веса q8_0 занимают около 8,70 GB, после вычета резерва остаётся примерно 3,30 GB, чего достаточно для двух запросов с контекстом 8K; смартфоны с меньшим объёмом памяти не могут вместить 8-битную модель 8B.

Энергопотребление и автономность — специфическая цена выполнения на конечном устройстве. Разбивка энергопотребления из раздела 4.1.3 даёт для одного запроса с контекстом 8K на H100 оценку порядка 0,534 J на токен decode, включающую только перемещение данных и матричные вычисления. Измерения на смартфоне охватывают энергопотребление всего устройства: MELTing Point получил 0,16–0,21 mWh на токен, то есть 0,576–0,756 J; длительно поддерживаемая мощность достигала 13,8 W, а мгновенная превышала 18 W. Измеренные значения пропускной способности и энергопотребления можно взаимно проверить: 14,8 token/s, умноженные на 0,576 J на токен, дают около 8,5 W, что не превышает длительно поддерживаемую мощность 13,8 W. Согласно расчёту по этим измерениям, одного заряда хватает, чтобы выполнить инференс примерно для 490–590 промптов.¹⁰ Таким образом, цена выполнения на конечном

⁹Описание Snapdragon 8 Elite и описание пятого поколения: поддерживаются LPDDR5X с частотой до 5,300 MHz и ёмкостью до 24 GB, ширина шины не указана. Страница продукта Micron LPDDR5X: максимальный класс скорости — 10.7 Gbit/s, стандарт JEDEC предусматривает одноканальное устройство x16, техпроцесс 1γ добавляет варианты ёмкости 6, 12 и 24 GB, также существует устройство на 16 GB для флагманских телефонов; страница продукта Samsung LPDDR5X также указывает максимум 10.7 Gbit/s. Страница характеристик Apple iPhone 16 Pro не раскрывает ёмкость и скорость памяти.

¹⁰MELTing Point: оценка языковых трансформеров на мобильных устройствах, arXiv 2403.12844, §5.2.2 и таблица 5 приложения: 0.16, 0.20 и 0.21 mWh на токен; максимальная продолжительная мощность iPhone 14 Pro составляет 13.8 W, мгновенная превышает 18 W, а продолжительная мощность Galaxy S23 остаётся ниже 8.5 W; на iPhone 14 Pro Zephyr-3B q4_k достигает 14.8 token/s, а Llama-2 7B q3_k — 6.0 token/s; на одном заряде можно выполнить 490.05–590.93 промпта, для S23 — 542.78. Построчная сверка извлечённых значений приведена в таблице извлечения числовых значений.

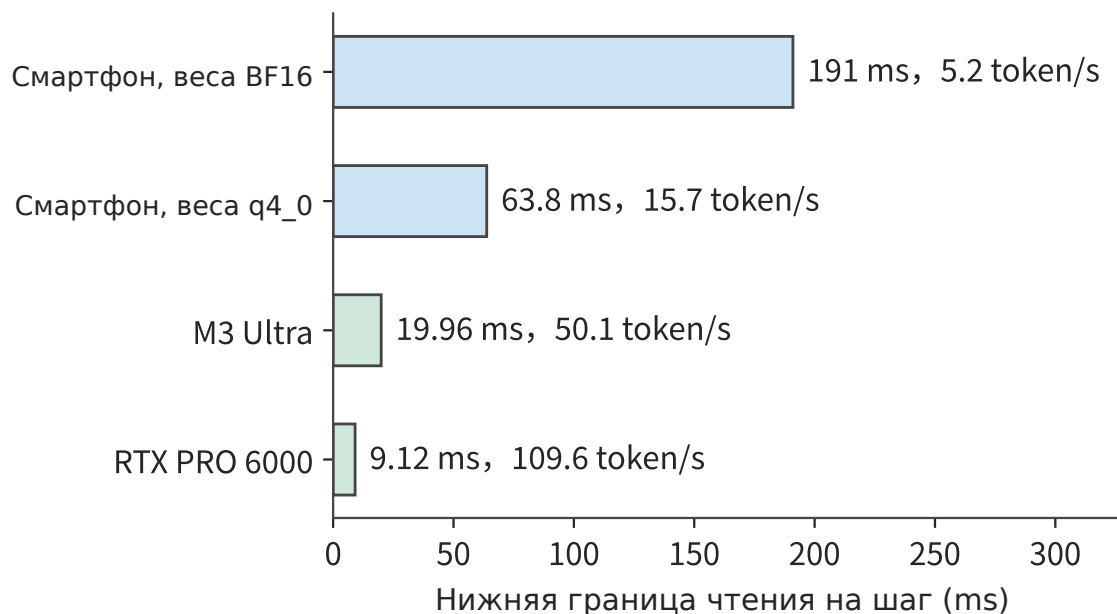


Рис. 12-9. Нижняя граница времени чтения основной нагрузки (16,345 GB) одного шага decode Qwen3-8B для одного запроса с контекстом 8K на трёх классах локальных устройств. Для смартфона предполагаются четыре канала x16 LPDDR5X с суммарной пропускной способностью 84,8 GB/s; веса BF16 и q4_0 показаны отдельными строками. M3 Ultra и RTX PRO 6000 взяты из таблицы устройств в разделе 4.8.2. На концах столбцов указан соответствующий верхний предел token/s.

устройстве заключается не только во времени и стоимости: скорость непрерывной генерации ограничена пределом мощности, а совокупный объём работы — ёмкостью аккумулятора.

Три класса устройств для локального развёртывания. Характеристики двух классов выше смартфона можно непосредственно взять из раздела 4.8.2, где для той же модели уже приведены нижние границы времени decode и prefill на 11 устройствах-кандидатах: M3 Ultra имеет унифицированную память с пропускной способностью 819 GB/s и объёмом 256/512 GB, нижняя граница чтения на шаг составляет 19,96 ms, или около 50,1 token/s, а объёма достаточно для размещения 4-битной MoE-модели 235B (раздел 4.6.3); RTX PRO 6000 имеет пропускную способность 1 792 GB/s и объём 96 GB, время одного шага — 9,12 ms, или около 109,6 token/s. На рис. 12-9 устройства трёх классов показаны на одной шкале времени.

Каждая среда выполнения решает свой уровень задач. Устройство задаёт нижние границы пропускной способности и ёмкости, но фактическое выполнение также зависит от среды выполнения. llama.cpp определяет хранение весов и поблочное выполнение: схема группового квантования GGUF (формат файлов моделей, используемый llama.cpp) определяет объём после загрузки и объём чтения на каждом шаге; в разделе 8.4.1 уже вычислено, что в Q2_K каждая группа занимает 84 bytes, в среднем 2,625 bit на значение. Ollama решает задачи распространения модели и оболочки локального сервиса: в соответствии с компромиссом «однократная подготовка, многократное повторное использование» из раздела 12.2.3 Ollama сокращает время подготовки к развёртыванию, а не время выполнения одного шага. MLX предназначен для унифицированной памяти Apple (раздел 4.6.3): CPU и GPU используют один пул памяти, что устраняет копирование данных между ними, а максимальный объём модели определяется памятью

всего устройства. Unsloth предназначен для дообучения на конечных устройствах и одной видеокарте: он позволяет вместить бюджет состояния обучения в одну карту или одно устройство, при этом форматы квантования по-прежнему берутся из раздела 8.4. Ни одна из этих четырёх сред выполнения не меняет выведенные выше нижние границы пропускной способности и ёмкости; они лишь определяют, насколько фактическое выполнение способно приблизиться к этим границам. Единая мера этого расстояния — отношение измеренного времени к нижней границе. В разделе 12.5.2 на основе результатов эксперимента 8-1 будет показано, что измеренное время одного шага decode на RTX PRO 6000 в 2,83 раза превышает нижнюю границу чтения; основная часть разницы приходится на фиксированные затраты каждого шага. По критерию из раздела 1.3.4 это устранимые системные издержки, а не недостаток возможностей устройства.

Экономии ёмкости от Unsloth можно непосредственно вычислить по числу байтов на параметр из раздела 10.1.2. Полное обучение всех параметров с Adam в смешанной точности требует 16 bytes на параметр: веса и градиенты BF16 занимают по 2 bytes, а основные веса FP32 и два момента — по 4 bytes. LoRA замораживает базовые веса и сохраняет градиенты и состояние оптимизатора только для добавленного низкорангового adapter: adapter Qwen3-8B для проекций Q и V с rank-16 в BF16 занимает 14,6 MiB (пример с несколькими LoRA в конце главы 8), что соответствует примерно 7,65 млн параметров. Веса, градиенты, основные веса и два момента adapter суммарно занимают 16 bytes на параметр, или около 0,12 GB; вместе с базовыми весами BF16 объёмом 16,38 GB получается около 16,5 GB. Ниже сравнивается только постоянно размещённое состояние, без активаций и временных буферов:

Видеопамять	Полное обучение всех параметров с Adam в смешанной точности, 16 bytes/параметр	Базовые веса BF16 + LoRA Q/V с rank-16
24 GB (RTX 4090)	Около 1,5B параметров	Около 12B параметров; Qwen3-8B требуется около 16,5 GB
96 GB (RTX PRO 6000)	Около 6B параметров	Около 48B параметров

Одна RTX 4090 не может вместить 131,1 GB состояния для полного обучения всех параметров Qwen3-8B, но может вместить состояние для её обучения с LoRA; Unsloth сокращает именно это состояние, а не нижнюю границу пропускной способности при чтении весов на каждом шаге.

Условия переключения: когда конечного устройства достаточно, а когда необходимо облако. Три нижние границы задают по одному условию переключения. Пропускная способность: если в каждом раунде генерируется G токенов, нижняя граница времени модели на конечном устройстве равна G , умноженному на время одного шага. Именно так получены «2,9 секунды вычислений модели на конечном устройстве за раунд» из раздела 12.5: при 15,5 token/s для q4_0 значение 2,9 секунды соответствует генерации около 45 токенов за раунд; при 5,2 token/s для BF16 — лишь около 15 токенов. Можно выполнить и обратный расчёт бюджета из предельного срока: при сроке 45 секунд на 20 раундов приходится по 2,25 секунды, а после вычета 0,3 секунды работы терминала остаётся около 1,95 секунды. За это время смартфон с q4_0 может сгенерировать не более примерно 30 токенов за раунд; если требуется больше, необходимо устройство с более высокой пропускной способностью или облако. Требование в 45 токенов за раунд уже превышает бюджет, поэтому вариант с конечным устройством из раздела 12.5.2

занимает 64,0 секунды и не укладывается в срок. Ёмкость: если веса вместе с необходимым контекстом превышают доступную память данного класса, этот класс непригоден — смартфон с 12 GB не может вместить модель 8B в BF16, а смартфоны как класс не могут вместить модель 235B. Энергопотребление: при непрерывной генерации ограничением становится аккумулятор; одного заряда хватает примерно на 490–590 промптов, поэтому пакетную или продолжительную генерацию следует передавать устройству с подключённым питанием. Эти три условия дополняют предварительное сравнение из раздела 12.1.4: там сравнивается время завершения разных вариантов, а здесь сначала исключаются случаи, в которых выполнение на конечном устройстве принципиально невозможно. В упражнении 12-9 по тем же формулам повторно вычисляются пределы пропускной способности, ёмкости и мощности для класса смартфонов.

Здесь используются аппаратные нижние границы, поэтому вывод о реализуемости несимметричен: если нижняя граница времени уже превышает допустимый срок, вариант определённо непригоден; если она ниже срока, это означает лишь, что вариант пока не исключён. Программные издержки, эффективная пропускная способность ниже пиковой и снижение частоты при длительной нагрузке увеличивают фактическое время, поэтому окончательное соответствие требованиям всё равно нужно проверять измерениями на целевом устройстве.

12.2 Распределение работы между устройствами

12.2.1 Вызов полной модели и поэтапное выполнение

Один и тот же снимок экрана можно сначала отправить на сервер, где сервер преобразует его в признаки для модели; либо сначала сформировать признаки на конечном устройстве, а затем отправить их на сервер. В этих двух вариантах по сети передаются уже разные объекты. Пример с изображением из раздела 12.1.1 показывает, что медленная передача файла заставляет вычисления ждать; после переноса части вычислений на конечное устройство изменяется и объект, который действительно требуется передать.

Подсеть, преобразующую изображение или аудио в числовые признаки для модели, называют энкодером. Если вся модель размещена удалённо, требуется загрузить входные данные и скачать результат; если оставить энкодер на конечном устройстве, требуется загрузить выход энкодера. Объёмы данных файла изображения и тензора признаков определяются их способами представления, поэтому предварительные вычисления на конечном устройстве не гарантируют уменьшения объёма загрузки.

Рассмотрим AI-агент для работы со снимками экрана. При сжатии изображения текстуры и повторяющиеся области представляются меньшим числом байтов; визуальный энкодер, напротив, разворачивает изображение в значения каналов каждого визуального токена, чтобы языковая модель могла их прочитать. Первое представление предназначено для компактного хранения, второе — для последующих вычислений.

Сравним два варианта: в первом изображение сначала загружается, а затем кодируется удалённо; во втором оно сначала кодируется локально, после чего загружаются признаки. Последующие вычисления языковой модели в обоих случаях одинаковы, поэтому при вычислении разности общего времени они взаимно сокращаются. Следовательно, достаточно сравнить время локального и удалённого кодирования, а также время передачи изображения и признаков.

12.2.2 Объём передачи и вычислительные затраты при кодировании на конечном устройстве

Сначала проследим по стрелкам на рис. 12-10 и 12-11, что именно передаётся в каждом варианте, а затем сравним объём передачи и сэкономленное время удалённых вычислений.

Пример: почему загрузка визуальных признаков может быть медленнее загрузки исходного изображения? Сохраним снимок экрана размером 0,8 МВ и исходящую скорость 6,4 Mbit/s. Для визуального кодирования используется фиксированная конфигурация Qwen3-VL-4B: после предварительной обработки изображение имеет размер 640×640 , а после объединения блоков изображения получаются 400 визуальных токенов; здесь визуальный токен — это вектор признаков, представляющий содержимое изображения, а не текстовый токен и не отдельный пиксель исходного изображения. Каждый вектор признаков итоговой проекции содержит 2560 компонентов, кроме того, имеются три группы признаков DeepStack такой же ширины (раздел 3.3.1); выходы всех этих ветвей должны передаваться вместе. Для каждого визуального токена требуется сохранить в общей сложности $4 \times 2560 = 10240$ чисел BF16, поэтому полный выход занимает

$$S_E = 400 \times 10240 \times 2 = 8\,192\,000 \text{ bytes} \approx 7.8 \text{ MiB}.$$

Четыре группы признаков подаются на разные слои языковой модели. Если передать только итоговую проекцию, три остальные группы признаков будут потеряны. Полный набор признаков занимает около 8,2 МВ — примерно в десять раз больше сжатого изображения; при прежней исходящей скорости время передачи возрастает с 1 секунды приблизительно до 10,2 секунды (рис. 12-12).¹¹

Запишем это сравнение в общем виде: обозначим время локального кодирования как $T_{E,l}$, время удалённого кодирования — как $T_{E,r}$, а дополнительное время сериализации и преобразования в варианте с локальным кодированием — как T_x . Разность времени двух вариантов равна

$$\Delta T = T_{E,l} - T_{E,r} + T_x + \frac{S_E - S_I}{B_u}.$$

При $\Delta T < 0$ кодирование на конечном устройстве выполняется быстрее. Пусть конечным устройством служит настольный компьютер с RTX 4090, а удалённой стороной — H100 SXM. Кодирование одного изображения требует 1,31 TFLOPs матричных вычислений. Если исходить из пиковой производительности плотных вычислений BF16 этих устройств, равной соответственно 165,2 и 989,4 TFLOP/s, нижние оценки времени кодирования составят приблизительно 7,9 ms и 1,3 ms. При $T_x = 0$ вариант с локальным кодированием требует около 10,25 секунды, а вариант с удалённым кодированием — около 1,00 секунды. Локальный вариант медленнее и по вычислениям, и по передаче: локальные вычисления занимают примерно на 6,6 ms больше, а передача признаков — примерно на 9,2 секунды больше.¹²

¹¹Фиксированная форма визуальных данных и полный расчёт EC/KV, пример размещения этапов мультимодальной обработки. Фиксированный визуальный вариант, DeepStack и языковое состояние измеряются отдельно.

¹²Расчёты для размещения кодирования на конечном, периферийном и облачном уровнях: объём матричных операций визуального кодирования взят из результатов визуального кодирования одного изображения, пиковые значения — из таблицы параметров оборудования; техническое описание ConnectX-7: скорость од-



Рис. 12-10. Путь выполнения при удалённом кодировании. Конечное устройство загружает сжатый снимок экрана размером 0,8 МВ, после чего удалённая сторона последовательно выполняет декодирование и предварительную обработку, визуальное кодирование и подключение признаков; итоговая проекция и три группы признаков DeepStack остаются внутри сервера. При исходящей скорости 6,4 Mbit/s передача изображения занимает 1 секунду; время кодирования и ожидания в очереди учитывается отдельно.

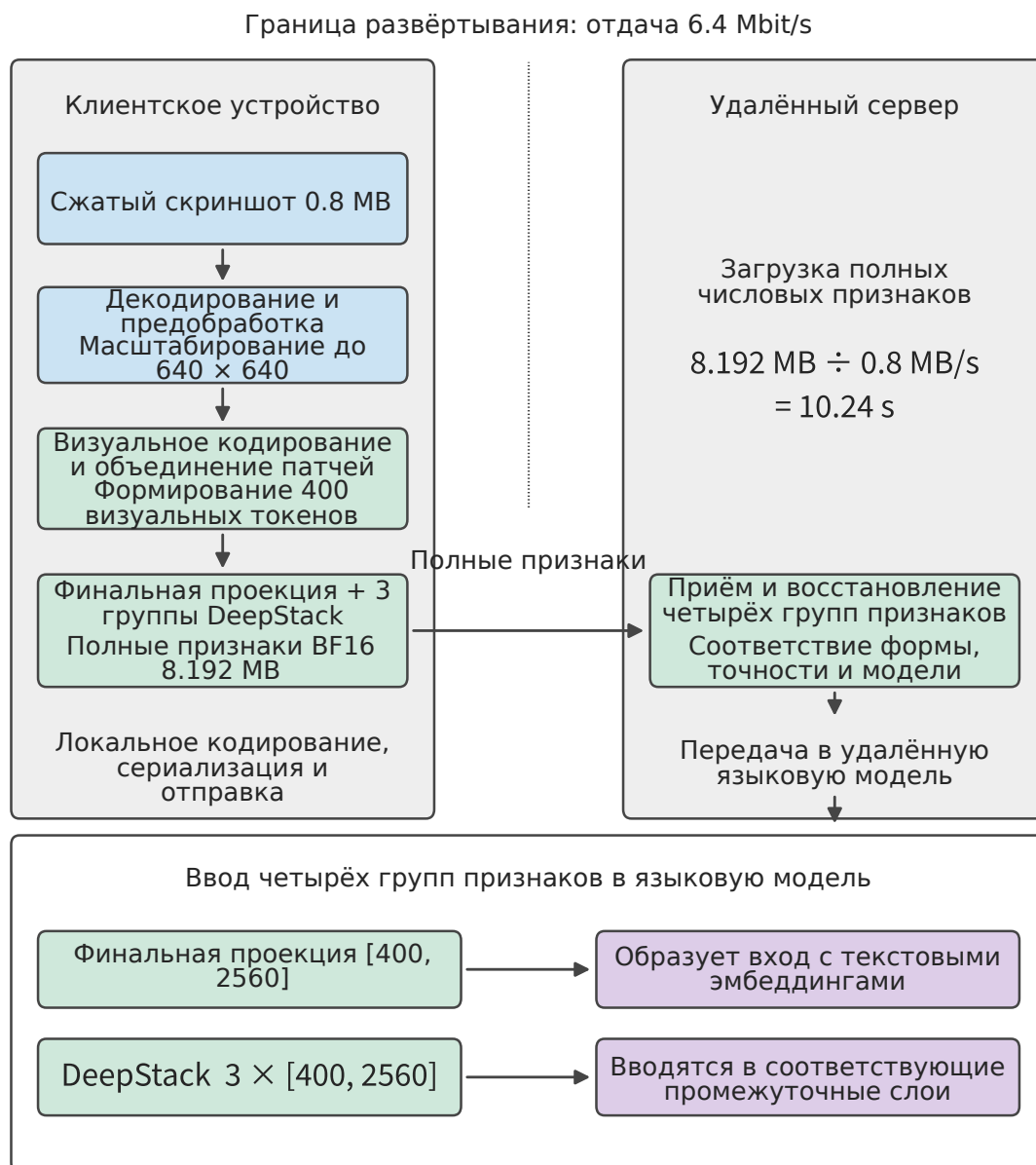


Рис. 12-11. Путь выполнения при кодировании на конечном устройстве. Предварительная обработка и визуальное кодирование переносятся на конечное устройство, а передаваемыми по сети объектами становятся итоговая проекция и полные признаки BF16 трёх групп DeepStack общим объёмом 8,192 MB; после получения удалённая сторона подключает их к языковой модели прежним способом. При той же исходящей скорости передача признаков занимает 10,24 секунды; время локального кодирования, сериализации и преобразования учитывается отдельно. Числа на рисунке соответствуют приведённой ниже фиксированной конфигурации Qwen3-VL-4B.

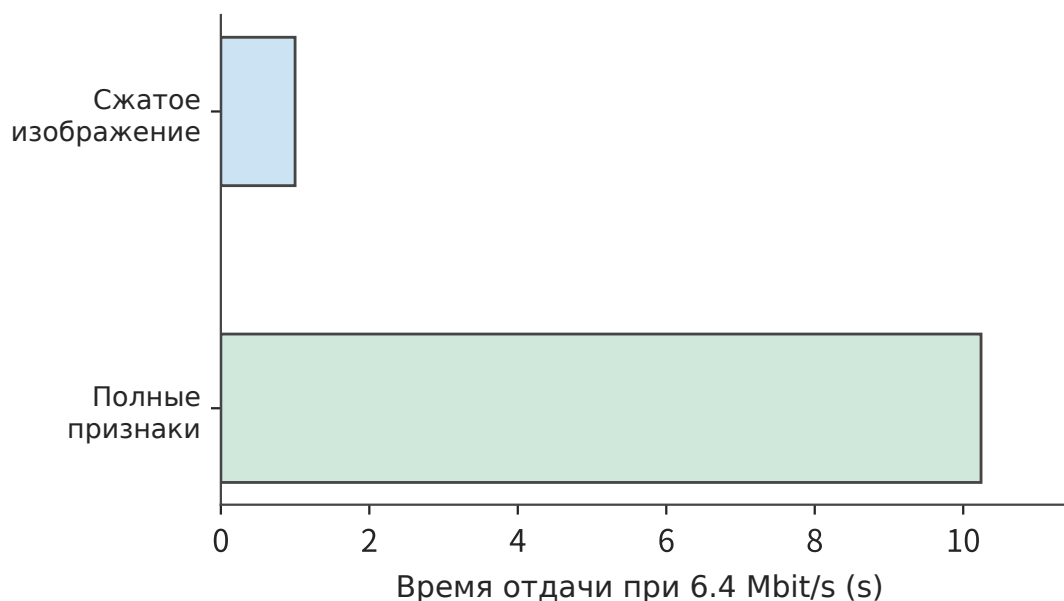


Рис. 12-12. Для передачи полных визуальных признаков одного изображения требуется больше времени, чем для сжатого изображения. Размер сжатого изображения составляет 0,8 MB; для входа 640×640 после предварительной обработки Qwen3-VL-4B формирует 400 визуальных токенов, а итоговая проекция и три группы DeepStack в сумме образуют [400,10240] BF16, всего 8,192,000 bytes. Исходящая скорость составляет 6,4 Mbit/s, длина столбца показывает время передачи; время кодирования, ожидания в очереди и преобразования учитывается отдельно. Оба варианта обрабатывают одно изображение и выполняют одну и ту же задачу визуального понимания.

С увеличением скорости соединения дополнительное время передачи признаков вместо изображения сокращается. Если энкодер и языковая модель размещены на двух серверах в одном центре обработки данных и соединены через порт ConnectX-7 со скоростью 400 Gbit/s (50 GB/s в каждом направлении), разница во времени передачи изображения и признаков составляет лишь около 0,15 ms, что уже меньше разницы во времени кодирования на двух указанных ускорителях. В этом случае достаточно сократить суммарное время кодирования и ожидания в очереди более чем на 0,15 ms, чтобы вариант с кодированием на другой машине стал быстрее; при прежнем медленном исходящем соединении требовалось бы сэкономить более 9 секунд. При выборе места размещения энкодера в случае медленного соединения сначала следует сравнивать объёмы передачи, а в случае быстрого соединения — прежде всего время кодирования и ожидания в очереди.

В многораундовых задачах для повторно появляющихся изображений можно повторно использовать ЕС, дополнительно сокращая число кодирований. Если то же изображение появляется снова, попадание в кэш устраняет одно кодирование; после изменения содержимого интерфейса изображение приходится кодировать заново. Такой же результат наблюдался в серии экспериментов на CPU с изображениями одинакового размера.¹³ Для вычисления суммарного времени кодирования многораундовой

ного порта достигает 400 Gbit/s.

¹³Эксперимент с кодированием Qwen3-VL-8B на CPU и передачей по локальному TCP. Входными данными служит синтетическое тестовое изображение 256×256, полный ЕС занимает около 2 MiB, PNG — около 3.4 KiB; данные до и после восьми передач побитово совпадают. Кодирование завершается до начала измерения передачи, поэтому указанное время относится к этапу передачи признаков.

задачи время одного кодирования можно умножить на число промахов кэша. Таким образом, оставшийся от предыдущих раундов кэш также включается в сравнение.

Визуальные признаки позволяют исключить только визуальное кодирование. Чтобы повторно использовать результаты, уже вычисленные языковой моделью, необходимо сохранять KV, которые языковая модель послойно формирует при обработке префикса. В этой конфигурации на каждый входной токен языковой модели требуется 144 KiB логического пространства хранения KV, поэтому для 400 визуальных токенов требуется около 56,3 MiB. Изображение, визуальные признаки объёмом около 7,8 MiB и KV визуальных токенов объёмом около 56,3 MiB соответствуют разным точкам возобновления вычислений (рис. 12-13). Чем ближе к итоговому выходу модели находится кэш, тем больше вычислений экономится при повторном использовании, но тем больше может оказаться объём передаваемых данных. В разделе 12.2.3 сравнивается время однократной передачи состояния с тем, сколько времени позволит сэкономить последующее использование этого состояния.

12.2.3 Подготовка, повторное использование и восстановление при миграции сеанса

При миграции сеанса кэш и ход выполнения задачи передаются на другое устройство, чтобы принимающее устройство могло продолжить выполнение. Если на целевом устройстве уже имеются совместимые веса, требуется передать кэш кодирования, языковой KV, журнал задачи и данные для восстановления. Убедившись в совместимости форматов состояния на обоих устройствах, можно сравнить время подготовки к миграции и экономию времени в каждом раунде после неё.

Обозначим необходимое для миграции время подготовки как M , экономию времени на новом устройстве в каждом раунде — как $\Delta t > 0$, а число оставшихся раундов задачи — как N . Разность между временем продолжения выполнения на исходном устройстве и временем выполнения после миграции на новое устройство равна

$$G(N) = N\Delta t - M.$$

Условие $G(N) > 0$ означает, что миграция выгодна. С каждым дополнительным раундом выигрыш увеличивается на Δt , а время подготовки M сдвигает всю кривую выигрыша вниз. Поэтому одна и та же миграция невыгодна для короткой задачи, но оправдана для длительного сеанса.

Пример: сколько ещё должно быть раундов взаимодействия, чтобы миграция состояния сэкономила время? Для миграции необходимо передать 64 MiB состояния по соединению со скоростью 80 Mbit/s, а затем потратить 1 секунду на восстановление. Общее время подготовки к миграции составляет

$$M = \frac{64 \times 2^{20} \times 8}{80 \times 10^6} + 1 \approx 7.7 \text{ s.}$$

Если каждый раунд выполняется на 0,4 секунды быстрее, то за 10 раундов чистая потеря составит около 3,7 секунды, за 20 раундов чистая экономия — около 0,3 секунды, а за 40 раундов — около 8,3 секунды. Подставив неокруглённое значение M в условие $N > M/0.4$, получим минимум 20 раундов. Это число раундов показывает, когда начинают окупаться затраты на миграцию, а величина чистого выигрыша — какие дополнительные затраты оправданны для такого переключения.

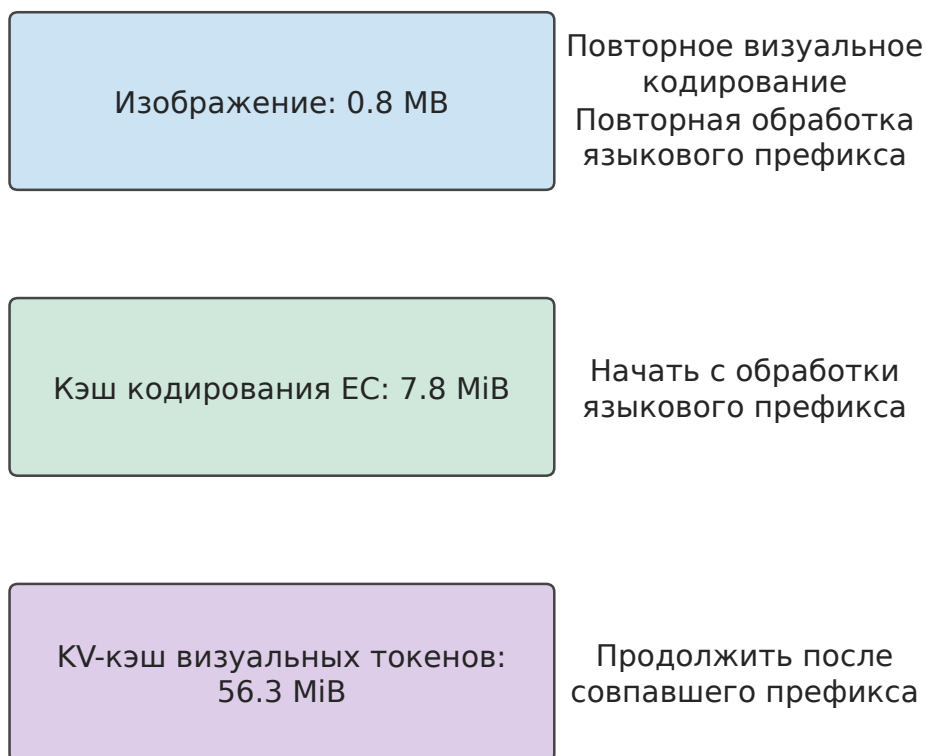


Рис. 12-13. Три типа содержимого кэша соответствуют трём точкам возобновления вычислений. ЕС устраняет визуальное кодирование; KV, соответствующие весам модели, префиксу и индексам позиций токенов, дополнительно устраняют вычисления языковой модели для уже обработанного префикса. Объёмы соответствуют фиксированной визуальной конфигурации из основного текста.

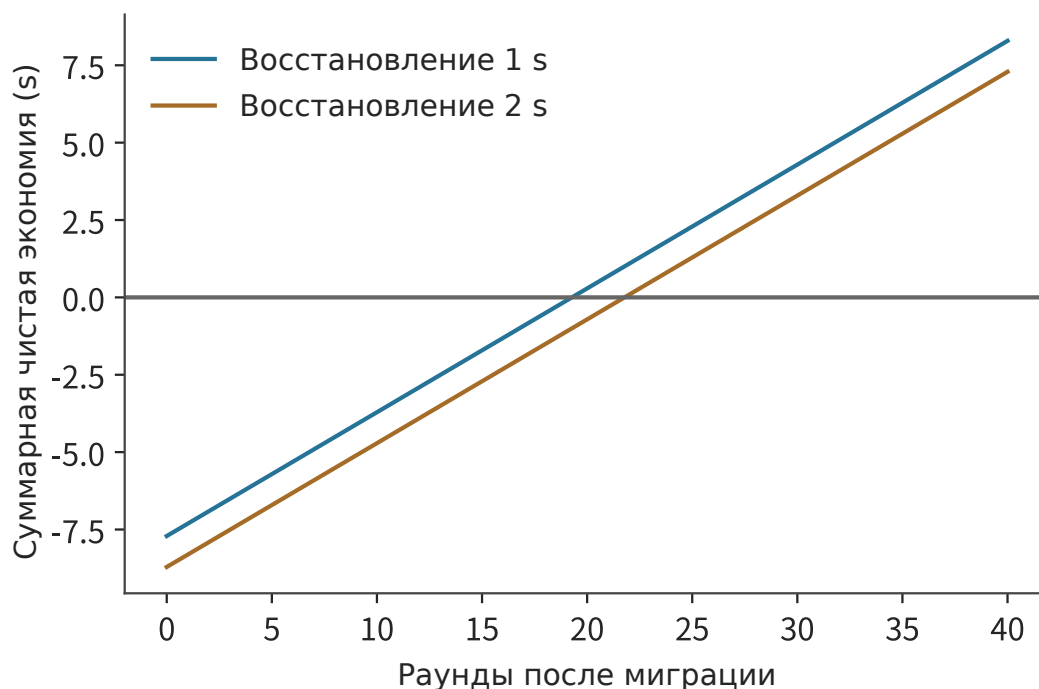


Рис. 12-14. Миграция 64 MiB состояния по соединению со скоростью 80 Mbit/s при экономии 0,4 секунды на каждом раунде после миграции. Для двух кривых время восстановления составляет соответственно 1 и 2 секунды, а чистая экономия равна $N \times 0,4$ за вычетом времени передачи и восстановления. Точками отмечены первые целые номера раундов, при которых возникает выигрыш; выше нулевой линии миграция выполняется быстрее, а первый выигрыш достигается соответственно на 20-м и 22-м раундах.

На рис. 12-14 в начале миграции чистая экономия отрицательна, а затем с каждым завершённым раундом увеличивается на 0,4 секунды. Число раундов до окупаемости соответствует точке пересечения кривой с нулевой линией.

Например, если время восстановления увеличить ещё на 1 секунду, чистый выигрыш за 20 раундов сразу станет отрицательным и составит около $-0,7$ секунды, а число раундов до окупаемости возрастёт до 22. Если ожидается ещё 40 раундов, то даже после добавления этой 1 секунды можно сэкономить около 7,3 секунды. Поэтому системе развёртывания необходимо прогнозировать оставшуюся длительность сеанса и резервировать время на дополнительные затраты, связанные с передачей и восстановлением.

Под «временем восстановления» на кривой понимается подготовка, необходимая новому устройству перед продолжением выполнения. Помимо загрузки данных кэша в память, новое устройство должно определить, до какого шага дошло выполнение задачи. Языковой KV привязан к модели, префиксу и индексам позиций токенов; прогресс агента для работы со снимками экрана определяется уже выполненными действиями. Щелчок уже изменил интерфейс, поэтому даже при потере ответа следует восстановиться в состоянии после этого щелчка. Номер операции и журнал фиксации позволяют целевому устройству найти ранее полученный результат и продолжить со следующего раунда. Сохранение этих записей предотвращает повторное выполнение действий так же, как повторное использование кэша предотвращает повторные вычисления.

Прогрев соединения, компиляцию графа и загрузку модели также можно анализи-

ровать с помощью выражения $N\Delta t - M$: затраты на подготовку включаются в M , а сокращение времени ожидания при каждом последующем вызове — в Δt .¹⁴ Оправданность такой подготовки зависит от того, сможет ли экономия времени при последующих вызовах компенсировать однократные затраты на подготовку.

12.2.4 Граница коммуникации при внутреннем разделении модели

При миграции сеанса состояние передаётся однократно — в момент смены устройства. Другой способ распределения работы состоит в том, чтобы несколько устройств постоянно совместно выполняли модель; в этом случае обмен данными многократно повторяется во время работы. При разделении модели на этапы каждый вход обычно требуется передать между соседними этапами лишь один раз; при тензорном параллелизме коммуникация необходима на каждом слое. Если вместо межсоединения ускорителей использовать локальную сеть, частая синхронизация накапливает небольшие единичные задержки, превращая их в основную статью затрат.

Рассмотрим тензорный параллелизм 36-слойной Qwen3-8B на двух устройствах (TP=2) с двумя редукциями на каждом слое — для выхода внимания и выхода FFN. Редукция суммирует локальные результаты двух устройств, чтобы последующие вычисления получили полный выход. При использовании двухэтапного кольцевого алгоритма (кольцевая реализация из главы 7, в которой сначала выполняется ReduceScatter, а затем AllGather) каждая редукция включает два запуска, причём запуск каждого этапа занимает α . Поэтому при генерации одного токена суммарное время запуска этих редукций равно

$$T_{\text{start}} = 36 \times 2 \times 2\alpha = 144\alpha.$$

При $\alpha = 2 \mu\text{s}$ эта часть занимает около 0,29 ms. Если вместо этого соединить два устройства через описанный в разделе 12.4.1 неагрегированный Wi-Fi со скоростью 54 Mbit/s, на каждом этапе потребуется выполнить как минимум один обмен короткими кадрами. Если принять указанное в том разделе время обмена короткими кадрами с АСК, равное 134 μs , то 144 запуска займут около 19,3 ms — более чем вдвое больше нижней оценки времени чтения для одного шага декодирования RTX PRO 6000, равной 9,12 ms. Даже при крайне малом объёме полезных данных запуск коммуникации всё равно занимает время; при этом следующему слою необходим результат редукции текущего слоя, а следующему авторегрессионному токеноу — выход текущего токена. Поскольку эти шаги должны выполняться последовательно, время запуска каждой коммуникации добавляется ко времени завершения отдельного токена.¹⁵

На рис. 12-15 и 12-16 последовательно показаны выход внимания и выход сети прямого распространения одного слоя. Каждый из них должен пройти одну редукцию по ходу выполнения, и лишь после этого можно начать следующий слой. Хотя единичное ожидание невелико, число повторений определяется структурой модели.

В отличие от тензорного параллелизма с послойной синхронизацией, конвейерный параллелизм разделяет слои на непрерывные этапы, а коммуникация концентрируется на передаче активаций между соседними этапами. Для одного запроса токен

¹⁴Статьи и материалы реализации FlexSP/llm.npu.

¹⁵Расчёт объёма обмена данными для каждой операции Qwen3-8B с TP=2, PP=4. В каждом из 36 слоёв выполняются две редукции выходных данных, что в сумме даёт 72 запуска обмена данными; дополнительные обмены создают эмбединги, активации этапов, logits и возврат токена.

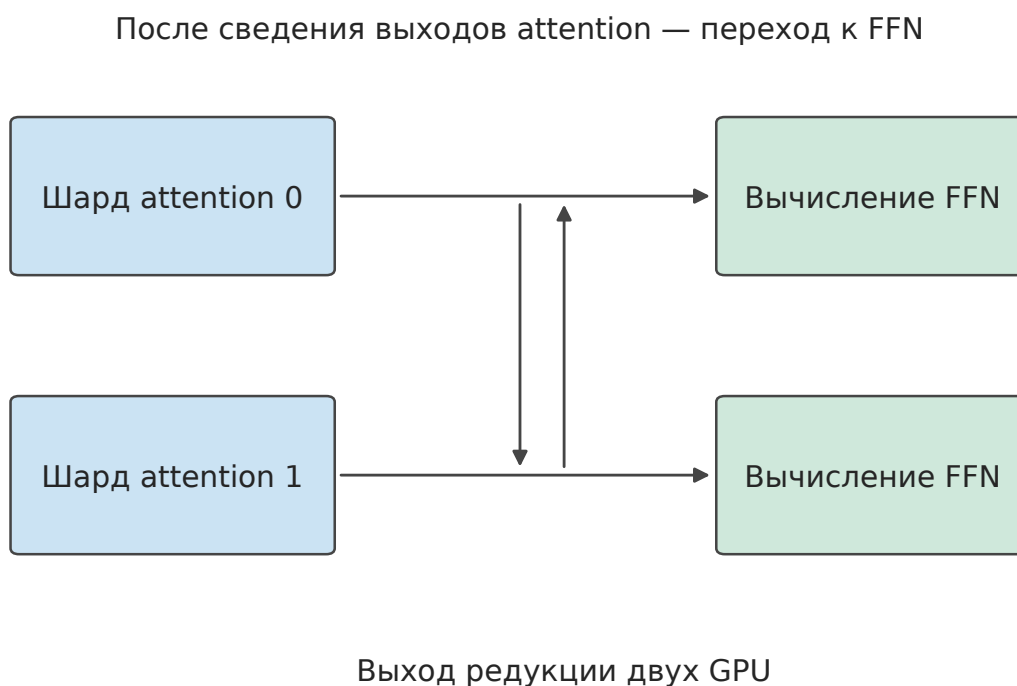


Рис. 12-15. Одна редукция выхода внимания при тензорном параллелизме: два ускорителя сначала выполняют вычисления по отдельности, затем обмениваются результатами и суммируют их, после чего полный выход передаётся в сеть прямого распространения. Горизонтальные стрелки показывают порядок выполнения, а вертикальная стрелка посередине — коммуникацию между ускорителями.

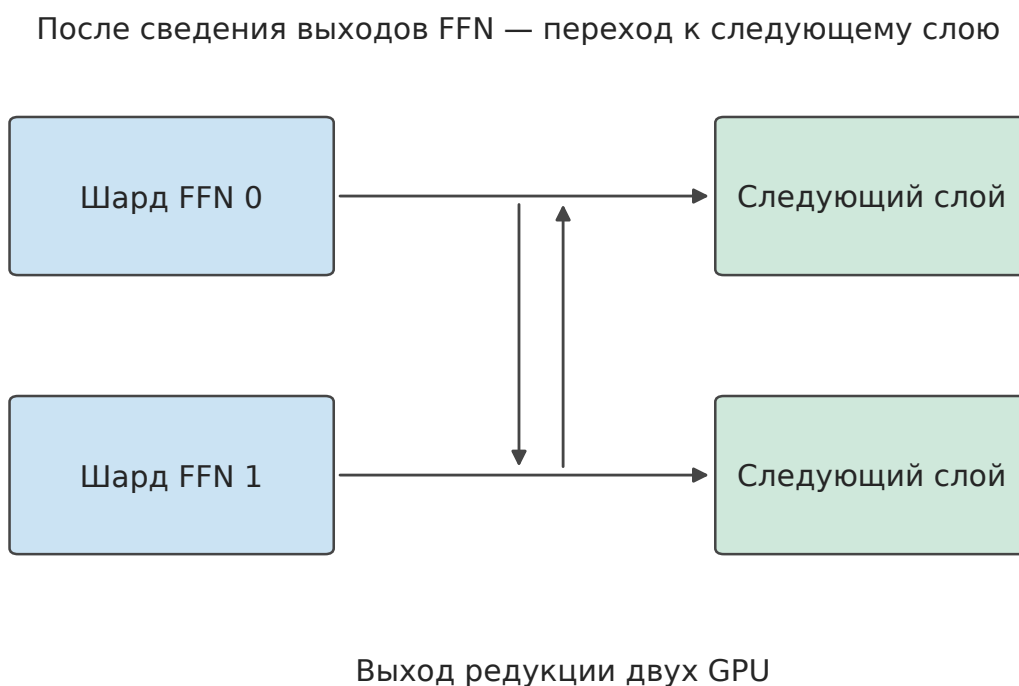


Рис. 12-16. Сеть прямого распространения также отдельно вычисляет локальные выходы, после чего выполняется редукция между двумя ускорителями. Перейти к следующему слою можно только после получения полного результата сети прямого распространения; эта структура повторяется на всех 36 слоях.

по-прежнему должен последовательно пройти через все этапы; при нескольких независимых запросах разные этапы могут одновременно обрабатывать разные запросы. Это даёт два вида выигрыша: сокращение числа коммуникаций может уменьшить задержку отдельного запроса, а одновременная обработка запросов разными этапами — повысить общую пропускную способность. При выборе между тензорным и конвейерным параллелизмом следует сначала построить зависимости для одного токена, а затем распределить несколько запросов по ресурсам этапов — так можно отдельно вычислить оба изменения.

После определения способа распределения модели становится ясно, что передаётся, сколько раз это передаётся и какие вычисления должны ждать завершения передачи. В разделе 12.3 рассматривается фактический процесс отправки: установлено ли соединение после подготовки данных, допускает ли окно передачу и своевременно ли возвращается подтверждение.

12.3 Дополнительные накладные расходы и ожидание при передаче по глобальной сети

12.3.1 Установление и повторное использование соединения

В разделе 12.1 предполагалось, что соединение уже установлено, а канал передаёт данные с постоянной скоростью. На практике перед выполнением запроса сначала необходимо установить соединение и контекст безопасности, то есть ключи, идентификационные данные и состояние сеанса, поддерживаемые обеими сторонами для аутентификации и шифрования; затем получить разрешённый объём отправки и, наконец, дожидаться, пока получатель передаст данные приложению. Если добавить эти ожидания в модель времени из раздела 12.1, можно анализировать реальный процесс выполнения: даже когда данные уже готовы, их не всегда можно немедленно обработать или отправить.

Эти ожидания связаны со следующими протоколами. HTTP — протокол прикладного уровня, организующий запросы и ответы. HTTP/1.1 и HTTP/2 обычно используют TCP — транспортный протокол, предоставляющий надёжный упорядоченный поток байтов, — и TLS — протокол безопасности, устанавливающий контекст аутентификации и шифрования. HTTP/3 использует QUIC — протокол, обеспечивающий надёжную многопоточную передачу и управление соединениями поверх дейтаграмм UDP. UDP предоставляет приложению независимые дейтаграммы и самостоятельно не отвечает за повторную передачу и упорядочивание. QUIC объединяет транспортное рукопожатие с TLS 1.3, сокращая число циклов передачи туда и обратно, необходимых для отдельного установления транспортного соединения и контекста безопасности. При возобновлении сеанса клиент также может отправлять ранние данные, то есть данные 0-RTT, опираясь на сохранённое состояние безопасности. Если сервер принимает их, он продолжает обработку; если отклоняет, клиент отправляет их повторно. Сервер распознаёт повторную попытку по идентификатору операции и возвращает сохранённый результат, поэтому одна и та же операция не выполняется повторно.¹⁶

Подставим накладные расходы на установление соединения в задачу со снимками экрана из раздела 12.1: сохраним RTT 200 ms для задачи из 30 раундов и предположим,

¹⁶RFC 9000: транспорт QUIC, §2, §7, §9, §13; RFC 9001: TLS; RFC 9114: HTTP/3; RFC 9221: ненадёжный Datagram. HTTP/3 определяет передачу запросов и ответов HTTP поверх QUIC.

что установление соединения требует двух дополнительных циклов передачи туда и обратно. Повторное установление соединения в каждом раунде суммарно займёт 12 секунд, а установление только в первом раунде — 0,4 секунды, что сэкономит 11,6 секунды. Повторное использование соединения эффективно потому, что последующие 29 раундов используют состояние, созданное в первом раунде. Значение 105 секунд в разделе 12.1 было рассчитано для уже установленного соединения; с учётом первого установления получится 105,4 секунды, а при повторном установлении в каждом раунде — 117 секунд.

Установление соединения — разовые накладные расходы, тогда как передача происходит в каждом раунде. При частых запросах сохранение соединения устраняет время его установления в следующем раунде; при больших интервалах между запросами неактивное соединение продолжает занимать память и другие ресурсы. Закрытие соединения освобождает эти ресурсы, но при следующем вызове его придётся устанавливать заново. Как и при переносе сеанса, здесь необходимо сопоставить затраты на однократную подготовку с затратами на последующее многократное использование.

12.3.2 Окно, обратная связь и эффективная пропускная способность

Повторное использование соединения устраняет ожидание его установления, но для непрерывной отправки данных также требуется достаточно большое окно отправки. После установления соединения отправитель всё равно должен ограничивать объём данных в пути. Окно перегрузки ограничивает объём уже отправленных, но ещё не подтверждённых данных; управление потоком на стороне получателя, исходя из доступного места в его буфере, ограничивает объём данных, который отправитель ещё может передать. После получения АСК отправитель может продолжить отправку; когда получатель считывает данные из буфера, он также сообщает отправителю о появлении дополнительного свободного места. Если достигнут предел отправки, отправитель вынужден ждать этих уведомлений, даже когда канал свободен.

Объём данных в пути — это число уже отправленных, но ещё не подтверждённых байтов. Если допустимый объём данных в пути равен W , а время передачи туда и обратно — R , то за каждый цикл передачи туда и обратно можно передать не более приблизительно одного окна данных, поэтому

$$B_{\text{useful}} \leq \min \left(B_{\text{path}}, \frac{W}{R} \right).$$

Здесь B_{path} — доступная скорость передачи полезной нагрузки по пути, а B_{useful} — фактическая эффективная пропускная способность. Чтобы непрерывно загружать канал передачи изображений из раздела 12.1.1 с пропускной способностью 20 Mbit/s и RTT 100 ms, требуется около 250 KB данных в пути. Если разрешённый объём составляет лишь 64 KB, верхний предел скорости равен $64000/0.1 = 640000$ bytes/s, то есть около 5,1 Mbit/s, и одна только отправка этих 30 MB данных займёт не менее примерно 47 секунд. Канал мог бы передавать больше данных в секунду, но отправитель вынужден ждать следующей группы подтверждений, прежде чем продолжить; фактическую скорость передачи в этом случае можно увеличить только расширением доступного окна.

Рассмотрим отдельно модель подтверждения целой группы: получатель отправляет подтверждение лишь после получения всех 64 KB, а от завершения отправки этой группы до возвращения подтверждения всегда проходит 100 ms. Поэтому полный цикл включает

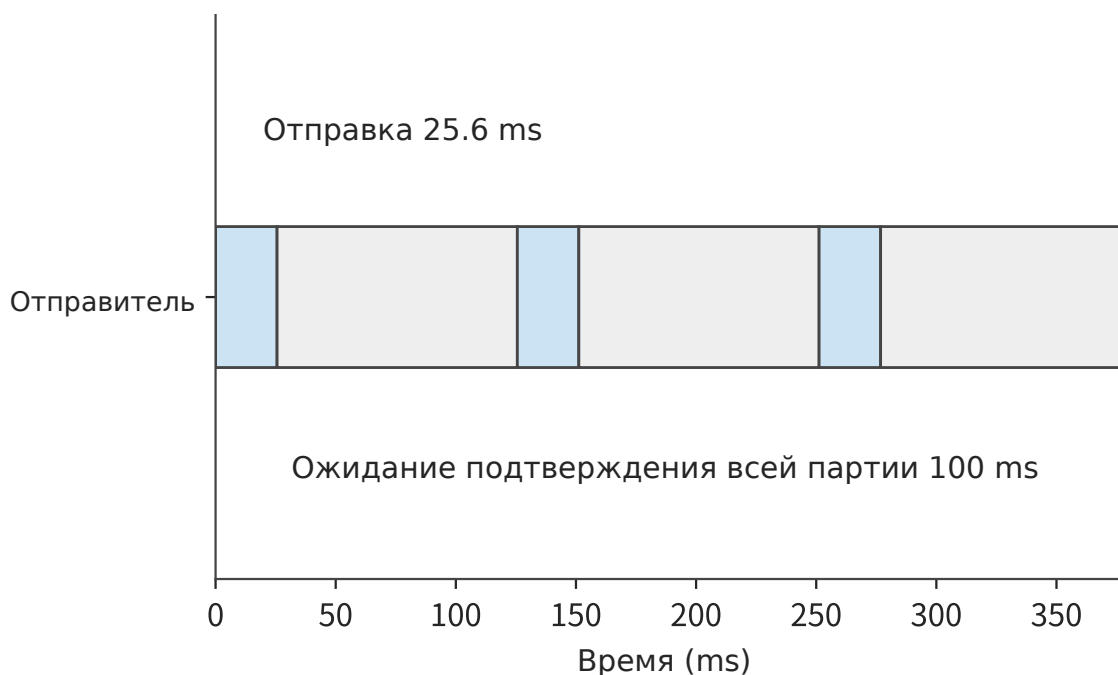


Рисунок 12-17. Фиксированное окно 64 КВ, канал 20 Mbit/s, отправка одной группы данных занимает 25,6 ms. Чтобы отдельно показать остановку и ожидание, на рисунке предполагается, что получатель подтверждает всю группу после её полного получения; от завершения отправки всей группы до получения общего подтверждения проходит ещё 100 ms. Полный цикл для каждой группы занимает 125,6 ms.

25,6 ms отправки и 100 ms обратной связи, всего 125,6 ms. На рисунке 12-17 показано это ожидание. В синие периоды действительно идёт отправка; в светлые периоды канал свободен, но отправитель не может продолжить, поскольку окно заполнено. Увеличение физической пропускной способности способно сократить только синюю часть.

На рисунке 12-17 окно зафиксировано на уровне 64 КВ, чтобы показать процесс остановки и ожидания. Новое соединение обычно начинает с меньшего окна, которое постепенно увеличивается по мере поступления подтверждений; короткий запрос может завершиться до того, как окно успеет достаточно вырасти. Предположим, что начальное окно состоит из десяти сегментов по 1460 bytes и удваивается после каждого раунда обратной связи. За первые четыре раунда суммарно можно отправить $14600(1 + 2 + 4 + 8) = 219000$ bytes, а к пятому раунду — 452 600 bytes. Поэтому для полной отправки голосового запроса объёмом около 355 КВ потребуется пятый раунд. При RTT 200 ms несколько раундов обратной связи занимают сотни миллисекунд; значительная часть времени короткого запроса может уйти на ожидание подтверждений. В разделе 12.3.5 с помощью этого механизма будет объяснено наблюдение за производительностью одного межрегионального голосового вызова.

Окно объясняет, почему отправка может приостанавливаться. Общее время запроса также зависит от фактического числа передаваемых байтов: по сети передаются не только изображения, но и заголовки пакетов и подтверждения.

Пример: как заголовки протоколов и АСК увеличивают время передачи изображений? Сохраним условия с отправкой 30 МВ и получением 5 МВ, примем максимальную полезную нагрузку пакета равной 1168 bytes, суммарный размер заголовков

протоколов и сети — 60 bytes, а также предположим, что каждый пакет данных вызывает АСК размером 92 bytes. Всего получается около тридцати тысяч пакетов, а заголовки и подтверждения добавляют примерно $30000 \times (60 + 92) / 10^6 \approx 4.6$ МВ, увеличивая фактический объём передачи с 35 МВ приблизительно до 39,6 МВ. При по пакетному расчёте моментов отправки, подтверждения и последующей отправки полный запрос занимает около 14,4 секунды.¹⁷

По сравнению с 12,8 секунды дополнительные примерно 1,6 секунды приходятся на только что добавленные в модель работу и ожидание. Однако учитывать лишь фактическое число передаваемых байтов всё равно недостаточно: АСК одновременно занимают канал и освобождают разрешённый объём для последующей отправки, поэтому уменьшение числа АСК меняет и загрузку канала, и моменты освобождения разрешённого объёма отправки.

Фактическое поведение подтверждений определяется алгоритмом управления перегрузкой. NewReno регулирует окно перегрузки на основе потерь и подтверждений; CUBIC на этапе предотвращения перегрузки регулирует окно по кубической функции времени; медленный старт — это начальный этап соединения, на котором окно быстро увеличивается по мере получения подтверждений. При одинаковом способе заполнения данными и одинаковой конфигурации NewReno замена немедленного АСК на подтверждение каждых двух пакетов с максимальным ожиданием 10 ms сокращает фактически передаваемый объём приблизительно с 39,6 МВ до 38,2 МВ, но увеличивает полное время запроса примерно с 14,52 до 14,56 секунды, то есть примерно на 43 ms.¹⁸ Это сравнение показывает двойную роль обратной связи: меньшее число подтверждений сокращает время передачи, но более позднее поступление подтверждений увеличивает ожидание окна. Чтобы определить, завершится ли запрос быстрее, необходимо сопоставить экономенное время передачи с дополнительным временем ожидания.

АСК определяет, когда отправитель сможет снова использовать окно для передачи данных, а управление перегрузкой — как будет изменяться само окно. При сравнении передачи большого изображения по одному и тому же пути полное время ответа для NewReno и CUBIC составляет около 14,5 секунды; траектория окна показывает, что CUBIC всё время остаётся на этапе медленного старта.¹⁹ Запрос завершается до перехода CUBIC к этапу предотвращения перегрузки, поэтому наблюдается поведение именно на этапе запуска.

12.3.3 Всегда ли потеря пакетов означает перегрузку

Окно из раздела 12.3.2 увеличивается по мере поступления подтверждений, но при потере пакетов должно сокращаться. Традиционные алгоритмы управления перегрузкой TCP не измеряют пропускную способность пути напрямую, а считают потерю пакетов

¹⁷Точное разбиение даёт 25,685 пакетов при загрузке и 4,281 пакет при скачивании; всего передаётся 29,966 пакетов данных и столько же АСК, фактический объём сетевой передачи составляет 39,554,832 bytes, а полный ответ занимает 14.36339008 s. Источник: полный замкнутый цикл для эталонного изображения.

¹⁸При одинаковых условиях заполнения вариант с немедленным АСК даёт 39,555,120 bytes и 14.52103724 s, а вариант с агрегированным АСК — 38,177,328 bytes и 14.564459796 s; разница составляет около 1.38 МВ и 43.4 ms. Источники: контрольный вариант NewReno с немедленным АСК, контрольный вариант с агрегированным АСК и запись этапа управления CUBIC.

¹⁹При одинаковых условиях заполнения вариант с немедленным АСК даёт 39,555,120 bytes и 14.52103724 s, а вариант с агрегированным АСК — 38,177,328 bytes и 14.564459796 s; разница составляет около 1.38 МВ и 43.4 ms. Источники: контрольный вариант NewReno с немедленным АСК, контрольный вариант с агрегированным АСК и запись этапа управления CUBIC.

сигналом перегрузки и выводят из неё доступную скорость. Модель Mathis — это формула оценки установившейся пропускной способности TCP по доле потерянных пакетов; она задаёт верхний предел скорости, которую можно получить таким косвенным расчётом.²⁰

$$B_{\text{TCP}} \leq \frac{\text{MSS}}{R} \cdot \frac{C}{\sqrt{p}},$$

где MSS (maximum segment size) — максимальная полезная нагрузка одного сегмента, R — время передачи туда и обратно, а p — доля потерянных пакетов; при периодических потерях и подтверждении каждого пакета $C = \sqrt{3/2} \approx 1.22$, а при отложенных подтверждениях, когда получатель подтверждает каждые два сегмента, $C < 1$. Поэтому в формуле (4) из статьи константа опущена и приведён упрощённый верхний предел $\text{MSS}/(R\sqrt{p})$. Согласно этой формуле, пропускная способность определяется долей потерянных пакетов: если она увеличивается в четыре раза, скорость сокращается вдвое. Формула предполагает, что потеря пакетов означает перегрузку; ошибки передачи в беспроводном канале и случайные потери пакетов в межрегиональных каналах этому предположению не соответствуют.

Результаты измерений на межрегиональном канале относятся именно ко второму случаю. Журналы путей разработанной автором системы передачи по глобальной сети Queqiao («Сорочий мост», рассматривается в разделе 12.3.5) показывают путь, на котором потеря пакетов не содержит информации о перегрузке. В направлении загрузки из Irvine в Гуйян при увеличении скорости отправки с 1 до 300 Mbit/s доля потерянных пакетов стабильно составляла около 14%, а последовательные потери были независимы друг от друга; в обратном направлении, из Гуйяна в Irvine, не был потерян ни один из 41 663 пакетов, поэтому два направления одного пути вели себя как два разных пути. Минимальное и максимальное значения времени передачи туда и обратно различались лишь на несколько миллисекунд, очередей на пути не было. Доля потерь возросла до 44% лишь при увеличении скорости до 600 Mbit/s — после преодоления точки перегиба пропускной способности около 333 Mbit/s, за которой дальнейшее повышение скорости вызывает резкий рост доли потерь. Только здесь потери были вызваны перегрузкой.²¹ Подставив MSS 1448 bytes, $R = 0.2$ s и $p = 0.14$, по упрощённому верхнему пределу из формулы (4), где $C = 1$ и журналы путей также используют это значение, получим

$$\frac{1448 \times 8}{0.2 \times \sqrt{0.14}} \approx 0.155 \text{ Mbit/s},$$

а при $C = 1.22$ — 0,189 Mbit/s; оба значения согласуются с измеренными на том же пути 0,13–0,47 Mbit/s. Одна только отправка голосового запроса объёмом около 355 KB (354 640 bytes) из раздела 12.3.2 с такой скоростью заняла бы примерно 18,3 секунды. TCP не работает неправильно: он реагирует на потерю пакетов в соответствии со своими правилами, просто на этом пути потери не содержат информации о перегрузке.²²

²⁰Mathis и др., «Макроскопическое поведение алгоритма предотвращения перегрузки TCP», ACM CCR 1997. Формула (3): $BW = (\text{MSS}/RTT) \cdot C/\sqrt{p}$; при периодических потерях и подтверждении каждого пакета $C = \sqrt{3/2} \approx 1.22$, при случайных потерях — 1.31; формула (4) задаёт более простую верхнюю границу.

²¹Запись пути из фиксированного архива и запись проектных решений из коммита 168ff4b. Файлы, параметры оптимизации, а также начальные и конечные точки измерения времени соответствуют определениям исходной записи.

²²Расчёты для потери пакетов 14% и потери пакетов 3.6%. Параметры пути взяты из записи пути Queqiao: время туда и обратно 199–207 ms, порог насыщения пропускной способности около 333 Mbit/s, потери па-

Это крайний пример первого типа расхождения, описанного в разделе 1.3.4: предпосылка модели не выполняется, поэтому рассчитанный верхний предел оказывается на три порядка ниже скорости, которую путь способен поддерживать в действительности. Исправлять нужно предположения модели, а не реализацию передачи.

BBR: прямое измерение вместо вывода из потерь. BBR (bottleneck bandwidth and round-trip propagation time, алгоритм управления перегрузкой на основе измерений) не использует потерю пакетов как сигнал, а непрерывно измеряет две величины: оценку пропускной способности узкого места BBR.max_bw — максимальное значение в окне недавних образцов скорости доставки — и оценку времени распространения туда и обратно BBR.min_rtt — минимальное значение в окне образцов RTT. Темп отправки (pacing) задаётся по измеренной пропускной способности; объём данных в пути определяется произведением пропускной способности на задержку (bandwidth-delay product, BDP), $\text{BDP} = \text{max_bw} \times \text{min_rtt}$: cwnd_gain равен 2, то есть окно перегрузки (cwnd) устанавливается равным удвоенному BDP, оставляя запас для группирования ACK и повторных передач. На этапе запуска (Startup) верхний предел пропускной способности исследуется по раундам с коэффициентом pacing $4 \ln 2 \approx 2.77$, а на этапе ProbeBW_UP верхние значения периодически исследуются с коэффициентом 1,25.²³ Подставим параметры того же пути: $\text{BDP} = 333 \text{ Mbit/s} \times 0.2 \text{ s} = 8.325 \text{ MB}$, окно перегрузки — 16,65 MB; идеальная эффективная пропускная способность $(1 - p) \times 333 \approx 286 \text{ Mbit/s}$, что примерно в 1850 раз превышает верхний предел Mathis. На пути, где потери пакетов не содержат информации о перегрузке, скорость, полученная прямым измерением пропускной способности, на три порядка выше предела, выведенного из потерь.

Даже после точного измерения пропускной способности остаётся хвост повторных передач. BBR отвечает на вопрос «как быстро отправлять», но не на вопрос «что делать при потере». Если разделить 354 640 bytes на 245 пакетов, каждый из которых независимо теряется с вероятностью 14%, ожидаемое число потерянных пакетов составит 34,3. Ниже используется идеализированный расчёт выборочных повторных передач по раундам: в каждом раунде на повторную передачу потерянных пакетов отводится один RTT, причём повторно переданные пакеты также могут быть потеряны; сокращение окна и повторные передачи по тайм-ауту не учитываются, поэтому результат представляет собой благоприятную для TCP нижнюю границу. Последовательный бюджет без потерь равен одному RTT плюс 30 ms на модель и ещё 8,5 ms на отправку со скоростью в точке перегиба, всего около 238,5 ms; при наличии потерь ожидаемое время завершения составляет 0,756 s, что примерно в 3,2 раза больше последовательного бюджета; p_{99} равен 1,24 s, что соответствует шести раундам.²⁴ Даже

кетов в направлении загрузки из Irvine в Guiyang около 14% (в другой период — 3.6%), нулевая потеря среди 41,663 пакетов из Guiyang в Irvine, MSS 1,448 bytes, размер фиксированного файла 354,640 bytes; время модели взято из раздела повторных измерений с фиксированным файлом и составляет около 30 ms, что согласуется с измерениями: если вычесть из оптимизированной медианы 236.5 ms минимальное время туда и обратно за тот же период 197 ms и время отправки 8.5 ms, время модели не превышает примерно 31 ms; приведённое выше значение 38 ms получено в раннем тесте с чередованием восьми файлов; размер символа FEC равен MSS, целевая вероятность успешного восстановления блока — 99.9%. Модель повторной передачи использует одну идеальную выборочную повторную передачу в каждом раунде, не учитывает уменьшение окна и повторную передачу по тайм-ауту и потому представляет собой благоприятную для TCP нижнюю границу.

²³ Управление перегрузкой BBR, черновик IETF draft-cardwell-icrg-bbr-congestion-control-02: BBR.max_bw — максимальное значение недавних выборок пропускной способности в окне, BBR.min_rtt — минимальная выборка RTT в окне, а BBR.bdp — их произведение; коэффициент pacing на этапе Startup равен $4 \ln 2 \approx 2.77$, cwnd_gain — 2, коэффициент pacing на этапе ProbeBW_UP — 1.25.

²⁴ Расчёты для потери пакетов 14% и потери пакетов 3.6%. Параметры пути взяты из записи пути Queqiao:

если BBR точно измерит пропускную способность, при высокой доле случайных потерь сохранится хвостовая задержка, вызванная повторными передачами; управление перегрузкой не может решить эту проблему.

Избыточное кодирование: получателю не нужно ждать повторной передачи. Если ожидание повторной передачи неприемлемо, можно отправить дополнительные избыточные данные, чтобы получатель самостоятельно восстановил недостающие пакеты. Прямое исправление ошибок (forward error correction, FEC) добавляет к k символам данных r восстановительных символов; в этом примере один символ соответствует одному пакету размером MSS. Если в блоке потеряно не более r символов, получатель может восстановить их напрямую без единого раунда повторной передачи. Точный расчёт по биномиальному распределению даёт минимальную избыточность для вероятности успеха 99,9%: при доле потерь 14% к 245 пакетам данных требуется 63 восстановительных пакета, то есть избыточность 25,7%; объём отправки возрастает до 308 символов, отправка занимает около 10,7 ms, а время завершения — около 241 ms, что близко к последовательному бюджету 238,5 ms (рисунок 12-18). В другой период доля потерь на том же пути составляла 3,6%: р99 повторных передач по раундам снижался до 0,84 s, а FEC требовал лишь 20 восстановительных пакетов, то есть избыточность 8,2%.²⁵ Поэтому долю избыточности следует регулировать по измеренной доле потерь, а не сохранять неизменной. Эти значения также служат количественным обоснованием для примера Queqiao в разделе 12.3.5: прямое исправление ошибок и планирование отправки в этой системе определяют объём избыточности и темп передачи по результатам измерений пути.

12.3.4 Многопоточная передача и приоритеты

В предыдущих разделах рассматривалась передача одного набора данных; здесь анализируется взаимное влияние нескольких наборов данных, использующих общий исходящий канал. Когда изображения, аудио и управляющие сообщения используют один выход, можно изменить два порядка: планирование отправки определяет, чьи данные

время туда и обратно 199–207 ms, порог насыщения пропускной способности около 333 Mbit/s, потери пакетов в направлении загрузки из Irvine в Guiyang около 14% (в другой период — 3.6%), нулевая потеря среди 41,663 пакетов из Guiyang в Irvine, MSS 1,448 bytes, размер фиксированного файла 354,640 bytes; время модели взято из раздела повторных измерений с фиксированным файлом и составляет около 30 ms, что согласуется с измерениями: если вычесть из оптимизированной медианы 236.5 ms минимальное время туда и обратно за тот же период 197 ms и время отправки 8.5 ms, время модели не превышает примерно 31 ms; приведённое выше значение 38 ms получено в раннем тесте с чередованием восьми файлов; размер символа FEC равен MSS, целевая вероятность успешного восстановления блока — 99.9%. Модель повторной передачи использует одну идеальную выборочную повторную передачу в каждом раунде, не учитывает уменьшение окна и повторную передачу по тайм-ауту и потому представляет собой благоприятную для TCP нижнюю границу.

²⁵Расчёты для потери пакетов 14% и потери пакетов 3.6%. Параметры пути взяты из записи пути Queqiao: время туда и обратно 199–207 ms, порог насыщения пропускной способности около 333 Mbit/s, потери пакетов в направлении загрузки из Irvine в Guiyang около 14% (в другой период — 3.6%), нулевая потеря среди 41,663 пакетов из Guiyang в Irvine, MSS 1,448 bytes, размер фиксированного файла 354,640 bytes; время модели взято из раздела повторных измерений с фиксированным файлом и составляет около 30 ms, что согласуется с измерениями: если вычесть из оптимизированной медианы 236.5 ms минимальное время туда и обратно за тот же период 197 ms и время отправки 8.5 ms, время модели не превышает примерно 31 ms; приведённое выше значение 38 ms получено в раннем тесте с чередованием восьми файлов; размер символа FEC равен MSS, целевая вероятность успешного восстановления блока — 99.9%. Модель повторной передачи использует одну идеальную выборочную повторную передачу в каждом раунде, не учитывает уменьшение окна и повторную передачу по тайм-ауту и потому представляет собой благоприятную для TCP нижнюю границу.

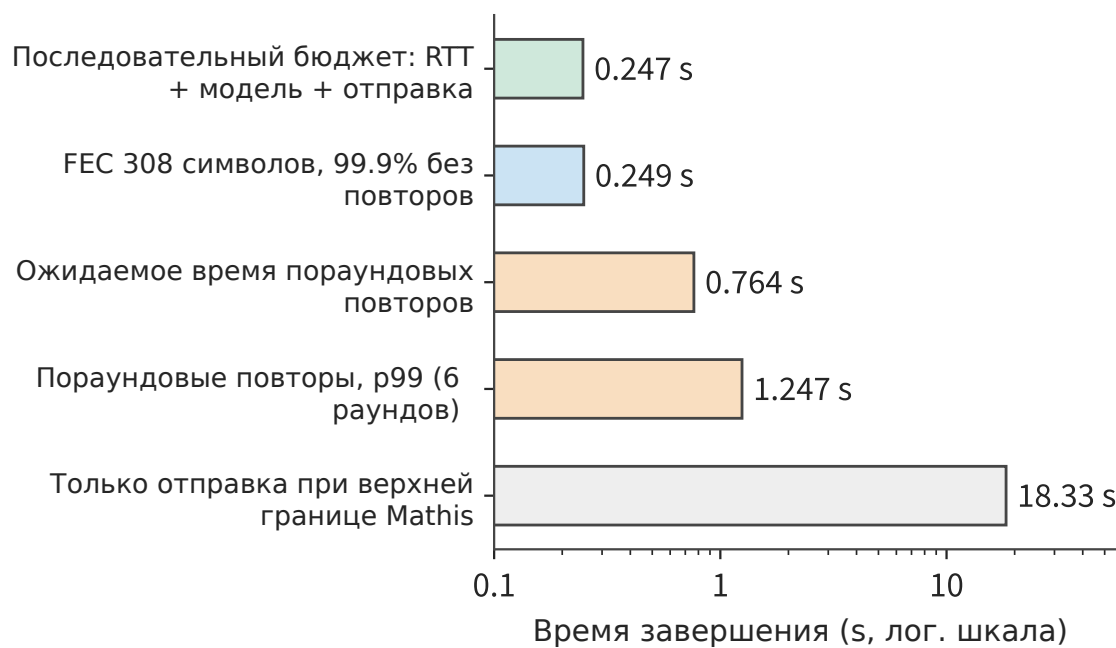


Рисунок 12-18. Время передачи 354 640 bytes тремя способами восстановления по одному пути с RTT 0,2 s, точкой перегиба пропускной способности 333 Mbit/s и долей потерь 14%; горизонтальная ось имеет логарифмический масштаб. Последовательный бюджет — сумма RTT, 30 ms работы модели и 8,5 ms отправки, около 238,5 ms; после добавления 63 восстановительных символов FEC с избыточностью 25,7% повторная передача не требуется с вероятностью 99,9%; две строки с повторными передачами по раундам показывают нижнюю границу без учёта сокращения окна и тайм-аутов; верхний предел Mathis соответствует TCP, считающему потерю пакетов признаком перегрузки, и одна только отправка занимает около 18,3 s.

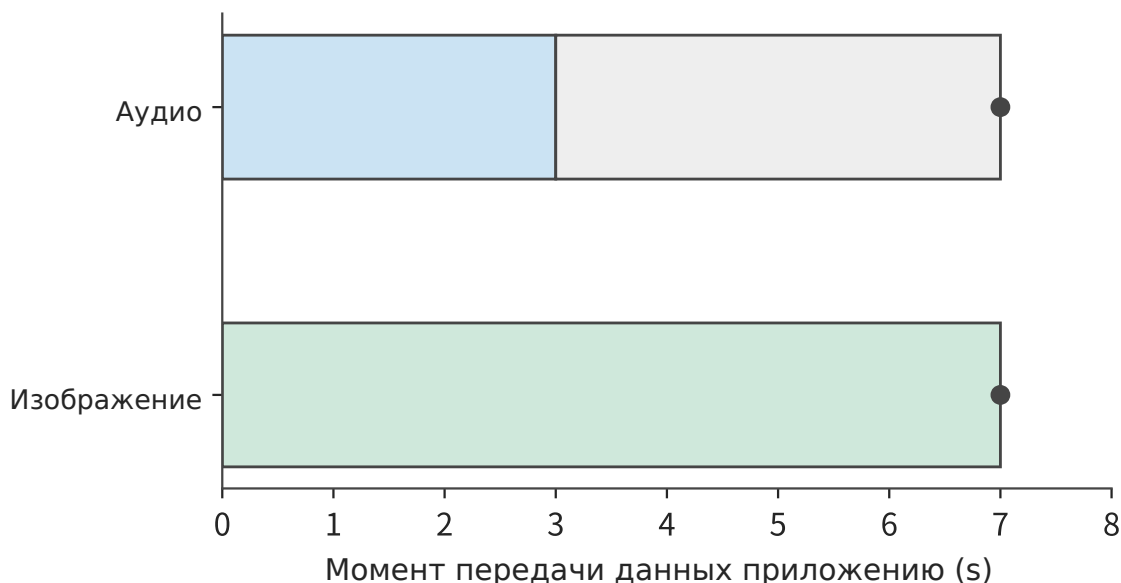


Рисунок 12-19. Общая упорядоченная доставка: все аудиоданные получены на 3-й секунде, но из-за пробела в данных изображения они продолжают ждать до 7-й секунды. Серый отрезок обозначает ожидание после получения всех данных, точка — момент их передачи приложению.

канал передаст первыми, а требования получателя к порядку данных — когда уже поступившие байты будут переданы приложению.

Сначала рассмотрим распределение ресурсов. В расчёте для группы данных, использующей общий канал, FIFO отправляет данные в порядке постановки в очередь: передача изображения завершается на пятой секунде, аудио — на шестой. При приоритете аудио его передача завершается на третьей секунде, а изображения — на шестой. Аудио завершается на 3 секунды раньше, а изображение — на 1 секунду позже, поскольку аудиоданные заранее заняли период, который прежде использовался для отправки изображения.²⁶

Теперь рассмотрим зависимости на стороне получателя. Оставим без изменений моменты отправки и восстановления после потерь: все аудиоданные готовы на третьей секунде, а все данные изображения — лишь на седьмой. Здесь передачу полученных данных приложению будем называть доставкой. Если всё соединение использует единый порядок получения, пробел в предшествующих данных блокирует последующие, создавая блокировку начала очереди. При общей упорядоченной доставке аудио также нельзя передать приложению до седьмой секунды (рисунок 12-19); при доставке по отдельным потокам аудио передаётся приложению уже на третьей секунде, а изображение — по-прежнему на седьмой (рисунок 12-20). Эти 4 секунды экономятся благодаря тому, что аудио больше не должно ждать поступления изображения; объём отправляемых данных и скорость канала не изменяются.

Различие между общей упорядоченной доставкой и доставкой по потокам точно соответствует поведению двух реальных протоколов. HTTP/2 помещает несколько потоков в единый упорядоченный поток байтов TCP, где пробел в предшествующих данных бло-

²⁶FIFO и приоритет аудио; полная упорядоченность и доставка по отдельным потокам при фиксированной записи отправки. Отдельно исследуется влияние планирования отправки и зависимостей доставки на стороне получателя.

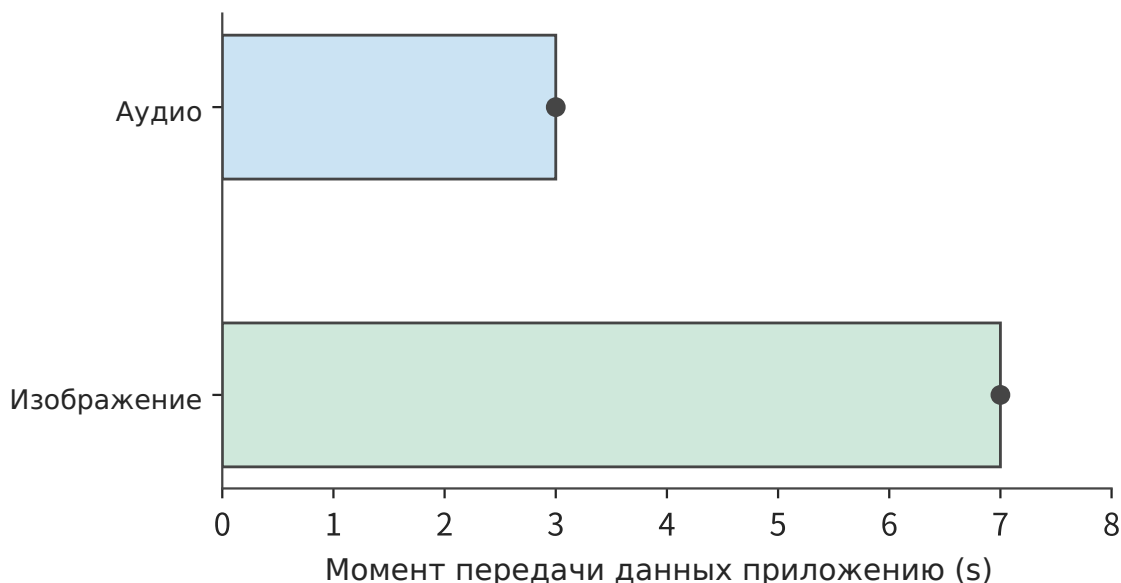


Рисунок 12-20. Упорядоченная доставка по потокам: аудио передаётся приложению сразу после полного получения на 3-й секунде, изображение — на 7-й. На обоих рисунках моменты отправки, поступления и восстановления одинаковы; различается лишь необходимость взаимного ожидания между потоками.

кирует последующие байты. QUIC поддерживает порядок отдельно для каждого потока, поэтому непрерывно поступившие байты других потоков можно доставлять независимо.²⁷ Здесь используется тот же метод анализа, что и в рекуррентном соотношении для воспроизведения аудио: одно условие ожидания, охватывающее все данные, разбивается на отдельные условия для каждого потока, и потоки, уже выполнившие своё условие, могут передавать данные приложению.

Эти два механизма отвечают соответственно на вопросы «когда отправлять» и «когда передавать приложению». В разделе 12.3.5 на примере межрегионального голосового запроса показано, как эти механизмы влияют на реальный запрос; в разделе 12.4 с учётом запланированного времени воспроизведения аудио и версии снимка экрана рассматривается, сохраняют ли данные ценность к моменту передачи приложению.²⁸

12.3.5 Пример: как *Qieqiao* ускоряет межрегиональные голосовые сервисы

В главах 8 и 9 рассматривались пакетная обработка и объединение ресурсов модельных сервисов. Если направлять запросы из разных регионов в небольшое число центров обработки данных, больше запросов сможет совместно использовать экземпляры моделей и ускорители. Это сокращает объём мощностей, которые при распределённом развёртывании пришлось бы отдельно резервировать в каждом регионе, не используя полностью, а также облегчает формирование полного batch. Поэтому централизованное развёртывание может повысить загрузку ускорителей и снизить стоимость одного запроса, но удалённым пользователям придётся передавать данные по более длинному сетевому пути. Например, если централизованно развернуть голосовую модель на востоке

²⁷RFC 9000: транспорт QUIC, §2, §7, §9, §13; RFC 9001: TLS; RFC 9114: HTTP/3; RFC 9221: ненадёжный Datagram. HTTP/3 определяет передачу запросов и ответов HTTP поверх QUIC.

²⁸Передача по локальному TCP/HTTP3 и многопоточная передача HTTP3 через одно соединение.

США, пользователям или сервисам доступа с западного побережья потребуется выполнять межрегиональные вызовы. Даже если модель в пуле ресурсов быстро завершит вычисления, пользователь всё равно будет ждать отправки запроса и возвращения результата.

Что такое Queqiao и какую проблему она решает? Queqiao имеет открытый исходный код, допускает самостоятельное развёртывание и передаёт трафик TCP и UDP между управляемым клиентом и доверенным шлюзом. Приложение подключается к клиенту через локальный прокси SOCKS5 — универсальный прокси-протокол для перенаправления произвольных соединений TCP/UDP; клиент соединяется со шлюзом через глобальную сеть, после чего шлюз передаёт запрос целевому сервису. При таком развёртывании между центрами обработки данных клиент работает на сервере вызывающей стороны, а шлюз размещается рядом с удалённым сервисом инференса, поэтому оптимизируемый дальний участок сосредоточен между клиентом и шлюзом. Приложение продолжает вызывать прежние интерфейсы ASR и TTS, а Queqiao отвечает за передачу запросов и результатов.²⁹

README проекта описывает реальную потребность: модель работает в США, а клиенты распределены по разным регионам. ASR отправляет несколько сотен KB аудио и возвращает одну строку распознанного текста; TTS отправляет фрагмент текста и возвращает несколько сотен KB аудио. В сценарии TTS из README аудио возвращается одной группой после завершения работы модели, то есть передаётся одним всплеском; в потоковом сценарии из раздела 12.1 передача начинается после генерации первого блока, а получение и воспроизведение продолжают параллельно с генерацией. Даже если все байты такого короткого запроса можно отправить за несколько миллисекунд, установление соединения, увеличение окна и восстановление после потерь могут по отдельности потребовать ещё одного цикла дальней обратной связи.

Цель Queqiao — сократить эти дополнительные ожидания, чтобы время коротких запросов максимально приблизилось к необходимому времени распространения, отправки данных и обработки моделью, а длинные потоки эффективнее использовали доступную пропускную способность. Queqiao позволяет нескольким потокам приложений совместно использовать результаты измерений и состояние передачи на пути клиент — шлюз, повторно использует соединения и планирует отправку и восстановление в соответствии с состоянием пути; прямое исправление ошибок добавляет некоторое количество избыточных данных, позволяя получателю восстановить недостающие пакеты без ожидания следующего раунда повторной передачи. Когда интерактивные потоки сосуществуют с большими файлами, их ожидание также необходимо контролировать с помощью планирования отправки. Рассмотренные в разделах 12.3.1–12.3.4 механизмы соединений, окон и нескольких потоков здесь совместно обслуживают межрегиональные вызовы. Само расстояние распространения по-прежнему определяет неизбежное время; пригодность централизованного развёртывания для конкретной задачи зависит от того, укладывается ли полный оптимизированный запрос в установленный бизнесом срок.

Почему межрегиональный голосовой запрос выполняется намного дольше идеальных примерно 240 миллисекунд? В этом разделе механизмы проверяются на архивных результатах голосовых тестов автора. Фактические измерения проводились

²⁹Назначение системы и мотивация межцентральной передачи описаны в фиксированном архиве Queqiao README, соответствующем коммиту GitHub 496ca627; дата обращения: 2026-09-10. Описание системы основано на этой версии, а данные сравнения производительности — на коммите 168ff4b.

на пути из Гуйяна в американский Irvine, а не на пути между востоком и западом США из приведённого выше примера развёртывания. Фиксированный аудиофайл имел объём около 355 KB, пропускная способность канала составляла 333 Mbit/s, время передачи туда и обратно — около 200 ms, а обработка моделью занимала около 30 ms; измерение начиналось при отправке запроса и завершалось после полного получения результата. Идеальное время отправки составляло около 8,5 ms, а вместе с распространением и обработкой — около 238,5 ms. Однако первоначальный прямой вызов занимал более секунды, тогда как Queqiao требовалось около трёхсот миллисекунд.

Это расхождение допускает два объяснения. Согласно первому, преимущество даёт сам механизм передачи; согласно второму, основная часть времени прямого вызова расходуется на подготовку соединения, увеличение окна и ожидание отправки. Эти объяснения дают разные прогнозы: если основная причина заключается в подготовке соединения и ожидании отправки, после полной оптимизации прямого пути время запроса должно приблизиться к рассчитанному выше минимуму, основанному на объёме данных, времени вычислений и обязательном цикле передачи туда и обратно; если же значительное преимущество по-прежнему обеспечивает сам механизм протокола, разрыв сохранится.

Для проверки прогноза сначала необходимо зафиксировать входные данные. В ранних тестах поочерёдно использовались восемь файлов объёмом 146–405 KB, но в отчёте был указан лишь размер последнего запроса — 355 KB; обозначенная этим размером медиана фактически охватывала весь диапазон. После перехода к многократной отправке одного и того же файла объёмом 354 640 bytes и чередованию двух путей каждый запрос стал передавать одинаковое число байтов.³⁰ Зафиксировав объём данных, можно изменять конфигурацию соединения и отправки и сравнивать их влияние на время запроса.

Какой разрыв во времени остаётся между двумя путями после повторного использования и оптимизации соединения?

Ниже приведены две группы результатов для фиксированного файла.

Условия соединения	Медиана запроса по прямому пути	Медиана запроса через Queqiao	Наблюдаемый разрыв
Новое соединение	около 1,19 s	около 0,30 s	прямой путь медленнее примерно в 3,9 раза
Сохранение и оптимизация соединения	около 0,24 s	около 0,24 s	результаты близки

При новом соединении пути различаются примерно на 0,89 секунды, а после повторного использования и оптимизации соединения оба дают около 0,24 секунды (рисунки 12-21 и 12-22).³¹ После изменения базовой конфигурации большая часть первоначального разрыва исчезла. Это служит доводом в пользу второго объяснения: в исходном расхождении основную роль играли подготовка соединения и способ отправки. Даже при использовании одного протокола различные конфигурации могут давать значительно

³⁰Запись пути из фиксированного архива и запись проектных решений из коммита 168ff4b. Файлы, параметры оптимизации, а также начальные и конечные точки измерения времени соответствуют определениям исходной записи.

³¹р50 прямого пути и Queqiao при создании нового соединения равны 1185.3 и 301.6 ms, а при сохранении оптимизированного соединения — 240.9 и 236.5 ms; отношения двух пар медиан составляют приблизительно 3.93 и 1.019. Если сначала вычислять отношение времени каждого раунда, а затем брать медиану, результаты составят соответственно 3.96 и 1.03. Источник: статистика Queqiao при одинаковых условиях.

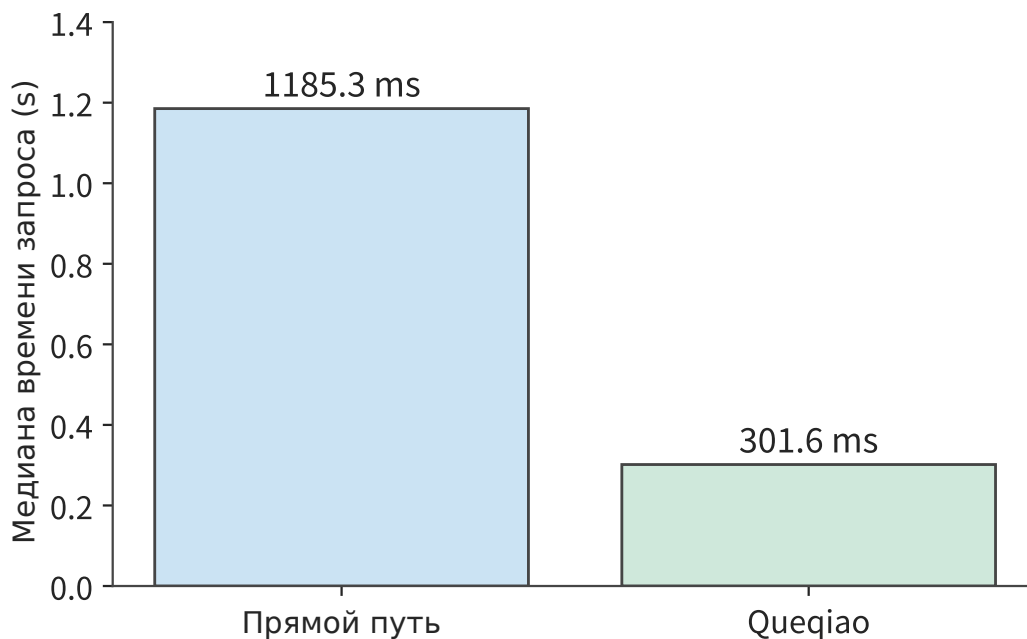


Рисунок 12-21. При новом соединении медианы запросов с фиксированным аудиофайлом объемом 354 640 bytes по прямому пути и через Queqiao составляют соответственно 1185,3 и 301,6 ms. Измерение начинается при отправке запроса и завершается после полного получения результата; входной файл одинаков, два пути запускаются поочередно.

различающееся время выполнения запроса. Последовательность действий в этом примере соответствует разделу 1.3.4: сначала из времени распространения, отправки и обработки моделью вычисляется нижняя граница около 240 ms, затем она сравнивается с измерениями, после чего эксперимент с изменением только условий соединения позволяет определить, вызвано ли расхождение пропусками в модели или накладными расходами реализации. Здесь расхождение почти полностью обусловлено накладными расходами реализации, а сама нижняя граница верна.

Как однофакторные эксперименты позволяют разделить влияние соединения, окна и темпа отправки? Повторное использование соединения определяет, требуется ли рукопожатие в начале запроса; размер окна — допустимый объем данных в пути до возвращения подтверждения; темп отправки — периоды простоя канала. На рисунке 12-23 эти три группы настроек сопоставлены с соответствующими событиями: файл, путь и остальные настройки фиксируются, в каждом эксперименте изменяется один фактор, после чего записываются моменты затрагиваемых им событий и полное время запроса. В эксперименте с соединением сравниваются моменты завершения рукопожатия и отправки первого байта, в эксперименте с окном — объем данных в пути и отправка после поступления ACK, а в эксперименте с темпом — время простоя между двумя отправками.

При исчерпании окна после отправки группы данных объем данных в пути достигает предела, исходящий канал простаивает, и отправка продолжается только после возвращения ACK. Расширение окна сокращает такие периоды простоя; если запрос также завершается раньше, временная шкала отправок и подтверждений показывает, что сокращение общего времени вызвано изменением окна.

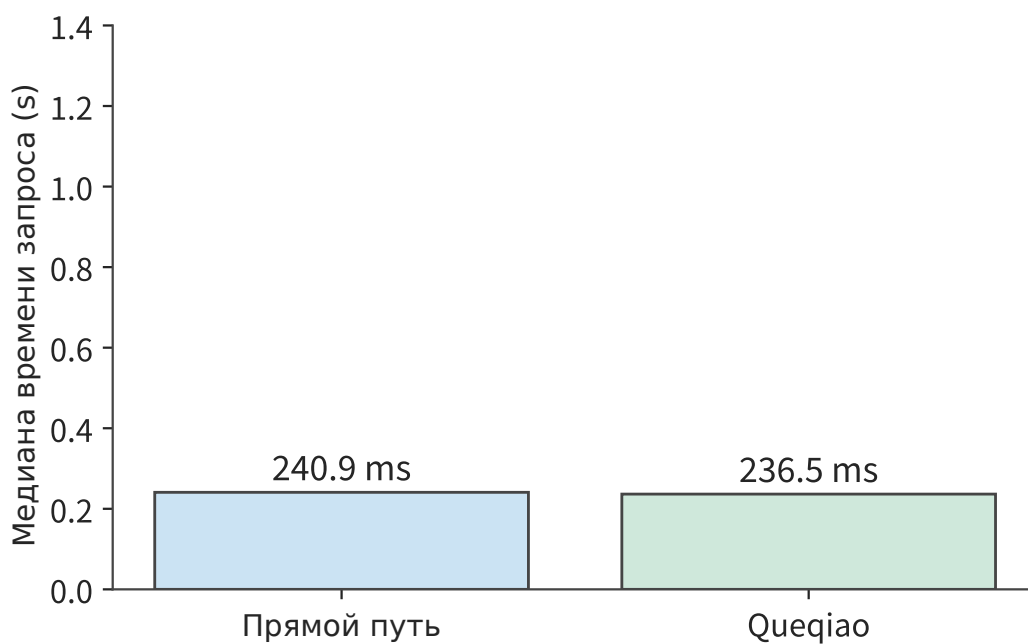


Рисунок 12-22. После сохранения и оптимизации соединения медианы для одного фиксированного аудиофайла по прямому пути и через Queqiao составляют соответственно 240,9 и 236,5 ms. Используется та же вертикальная ось, что и на предыдущем рисунке; оба пути приближаются к суммарному времени, необходимому для отправки данных, обязательного цикла передачи туда и обратно и обработки.

Однофакторный эксперимент: за шаг меняется одна настройка

Переиспользование соединения → рукопожатие и первый байт

Размер окна → данные в пути и время подтверждения

Темп отправки → простой канала и время завершения

Рисунок 12-23. Метод однофакторного эксперимента: оставлять неизменными файл, путь и остальные настройки, каждый раз изменяя только один фактор, и сравнивать промежуточные события и общее время завершения.

На результаты сравнения влияет и порядок проведения эксперимента. Чередование тестов двух путей позволяет запускать два сравниваемых запроса примерно в одно время, уменьшая влияние изменений сети; за тот же период RTT составлял около 197–205 ms с размахом примерно 8 ms. После оптимизации разница между медианами времени запросов по двум путям составляла около 4 ms, что меньше колебаний RTT за тот же период. В этой группе экспериментов время запроса по обоим путям уже приближалось к 240 ms.

Что выгоднее: сократить сетевую задержку передачи туда и обратно или ускорить вычисления модели? После оптимизации базового варианта сравнения меняется и приоритет следующего этапа оптимизации. Ускорение модели с 30 до 3 ms сэкономит не более 27 ms; сокращение RTT с 200 до 20 ms позволит сэкономить около 180 ms. Если дополнительная подготовка и вычисления при выполнении поблизости займут меньше этих 180 ms, такой вариант окажется быстрее исходного удалённого пути. Метод сравнения тот же, что и в разделе 12.1: сначала вычисляется, насколько замена сервера сократит ожидание, а затем вычитается дополнительное время вычислений и подготовки.

Помимо времени запроса, необходимо проверить непрерывность воспроизведения: после поступления аудио проигрыватель должен воспроизводить его без остановок. В другой группе экспериментов, использовавшей обратные вызовы устройства вывода без звука, в итоге были получены все данные PCM, однако обратные вызовы воспроизведения суммарно в течение примерно 0,36–1,44 секунды не могли получить достаточное число аудиосэмплов.³² Момент завершения запроса показывает лишь, когда поступила последняя группа данных, тогда как проигрывателю необходимо непрерывно считывать аудио ещё до этого. Полное время запроса и время остановок воспроизведения описывают две разные проблемы.

Генерация аудиоблоков, их поступление, помещение в буфер воспроизведения и считывание обратным вызовом воспроизведения образуют временную шкалу непрерывного воспроизведения. Рекуррентное соотношение из раздела 12.1 определяет момент первого опустошения буфера: если следующий блок к этому времени ещё не сгенерирован, ожидание вызвано обработкой моделью; если он сгенерирован, но ещё не поступил, — передачей; если он уже поступил, но не был считан, — локальной буферизацией и планированием.

Этот пример охватывает весь путь от оценки до решения о развёртывании: сначала рассчитывается идеальное время завершения, затем в журналах выполнения обнаруживаются ожидания, их причины объясняются однофакторными экспериментами, а в конце на основе оставшегося потенциала экономии времени выбирается дальнейшая оптимизация или замена сервера. В разделе 12.4 в анализ добавляются конкуренция, колебания и восстановление при беспроводном доступе, а в разделе 12.5 сводится полный план развёртывания.

³²В восьми тестах полного приёма полученный PCM и PCM, прочитанный обратным вызовом воспроизведения, побитово совпадают; суммарно в течение 360–1440 ms обратный вызов воспроизведения не мог получить достаточное количество аудиосэмплов. В эксперименте использовалось устройство вывода тишины, а журнал содержит события приёма и обратного вызова воспроизведения. Ещё три запроса были отменены до первого обратного вызова воспроизведения; полученные байты, события закрытия соединения и удалённый EOF фиксируют процесс отмены. Полные данные четырёх раундов приведены в описании сопутствующих свидетельств. Источник: аудиозапись эксперимента с явной привязкой физического интерфейса, фиксированный коммит 496са627, базовый вариант включает адаптацию привязки сокета.

12.4 Взаимодействие и восстановление в условиях изменчивой беспроводной связи

12.4.1 Общий радиоканал и конкуренция восходящего и нисходящего трафика

В разделе 12.3.4 различались моменты «когда отправлять» и «когда передавать приложению». **Радиоинтерфейс (air interface)** — это интерфейс, через который устройства обмениваются данными посредством радиосигналов. Беспроводная сеть также изменяет стоимость самой отправки: восходящие данные и нисходящая обратная связь должны поочередно занимать одно и то же эфирное время. При каждом обмене необходимо не только передать байты, но и дожидаться освобождения канала и межкадрового интервала, а затем получить подтверждение на текущем участке. Поэтому, помимо количества байтов приложения, необходимо учитывать число обменов.

Сквозной ACK отправляется удалённому отправителю; уровень управления доступом к среде (MAC) организует передачу по отдельному беспроводному каналу, а MAC ACK подтверждает беспроводной кадр на текущем участке. Беспроводной кадр, несущий сквозной ACK, также требует собственного подтверждения MAC. Поэтому даже короткий сквозной ACK должен дожидаться освобождения канала и получить подтверждение на текущем участке.

Пример: насколько велики накладные расходы на доступ к беспроводному каналу и подтверждение короткого кадра ACK? Мультиплексирование с ортогональным частотным разделением каналов (OFDM) распределяет данные по нескольким взаимно ортогональным поднесущим; PPDU — это блок данных протокола физического уровня, содержащий преамбулу, заголовок и передаваемые данные физического уровня и представляющий полное содержимое одной беспроводной передачи; SIFS — короткий межкадровый интервал между последовательными обменами. Рассмотрим эталонный обмен OFDM без агрегации: скорость передачи данных — 54 Mbit/s, скорость MAC ACK — 6 Mbit/s. Обмен пакетом включает ожидание доступа в течение 34 μ s, передачу кадра данных в течение 208 μ s, SIFS длительностью 16 μ s и MAC ACK длительностью 44 μ s — всего 302 μ s. При передаче сквозного ACK длительность кадра данных сокращается до 40 μ s, а остальные три составляющие остаются неизменными — всего 134 μ s.³³

На рисунке 12-24 оба обмена содержат $34 + 16 + 44 = 94$ μ s фиксированных накладных расходов. Время передачи кадра данных уменьшается с 208 до 40 μ s, то есть примерно на 80%, однако длительность всего обмена сокращается лишь примерно на 56%. Чем больше доля фиксированных составляющих, тем меньше выигрыш от дальнейшего сокращения полезной нагрузки; более прямой способ уменьшить накладные расходы — объединять несколько небольших сообщений и сокращать число обменов.

Если по-прежнему передавать изображение объёмом 30 МВ и готовый результат объёмом 5 МВ, то при данной конфигурации беспроводной связи потребуется по 30 800 обменов данными и обменов обратной связью, а суммарное время использования радиоканала составит примерно

³³Расчёт изображения для общего эфирного времени. Для данных и MAC ACK используются скорости OFDM 54 и 6 Mbit/s соответственно, агрегация кадров не применяется; ожидание доступа равно SIFS 16 μ s плюс два слота по 9 μ s, то есть DIFS 34 μ s для 802.11a OFDM, случайная отсрочка не учитывается; подтверждения на уровне MAC и сквозные подтверждения измеряются отдельно.

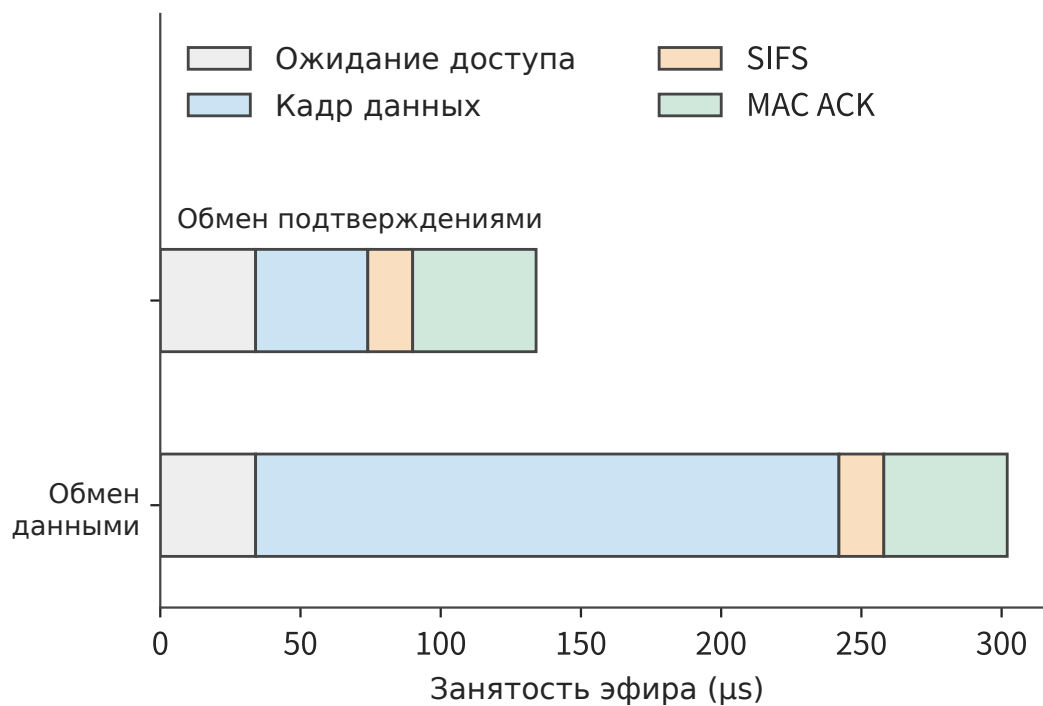


Рисунок 12-24. После сокращения сообщения фиксированные накладные расходы обмена сохраняются, поэтому время использования радиоканала не уменьшается в той же пропорции. В обоих успешных обменах используются одинаковые ожидание доступа длительностью 34 μs , SIFS длительностью 16 μs и MAC ACK длительностью 44 μs ; различается только PPDU данных: 208 μs или 40 μs . Условия эталонной модели: OFDM 54/6, без агрегации и повторных передач.

$$30800 \times (302 + 134) \mu s \approx 13.4 s.$$

Если оставить данные и траекторию восстановления неизменными, но вдвое сократить число сквозных АСК, освободится примерно $15400 \times 134 \mu s \approx 2.1$ секунды эфирного времени. Как уже объяснялось в разделе 12.3.2, задержка обратной связи дополнительно увеличивает время ожидания окна. Поэтому необходимо заново вычислить моменты отправки и подтверждения каждого пакета: сокращение числа АСК освобождает эфирное время, но ожидание подтверждения может задержать последующие отправки. Оба фактора совместно определяют момент завершения всего запроса.

В реальных протоколах уже существуют механизмы, построенные по этому принципу. TACK — один из механизмов обратной связи, сокращающих накладные расходы беспроводных подтверждений. TACK отделяет обычные накопительные подтверждения от событий, требующих немедленной обратной связи: первые объединяются перед отправкой для сокращения числа обменов, а вторые возвращаются незамедлительно, чтобы отправитель мог как можно скорее продолжить передачу или повторно отправить потерянные данные.³⁴ Такое разделение одновременно сокращает расходы на отправку сообщений подтверждения и время ожидания обратной связи.

12.4.2 Сроки обслуживания, устаревшие данные и отмена

Проигрыватель из раздела 12.1 ожидал недостающий блок, чтобы сохранить всё содержимое; другой вариант проигрывателя воспроизводит данные по фиксированному таймеру, а опоздавшие аудиоблоки уже не используются в предназначенных для них интервалах воспроизведения. Требования к качеству задачи определяют, какое правило воспроизведения следует выбрать.

Например, восемь аудиоблоков длительностью 20 ms каждый планируется непрерывно воспроизводить начиная с отметки 0,4 секунды, всего в течение 0,16 секунды. Когда эти аудиоданные делят канал с большим изображением, при FIFO все блоки в итоге прибывают, но каждый из них пропускает назначенный момент воспроизведения; при приоритете медиаданных все восемь блоков прибывают вовремя.³⁵ В обоих случаях итоговое число полученных байтов одинаково, однако объём вовремя доступного звука составляет соответственно ноль и 0,16 секунды. Только подсчёт при планировании содержимого, воспроизведённого вовремя, отражает, сколько звука пользователь действительно услышал.

Опоздавшие данные пропускают предназначенный для них интервал воспроизведения, а отмена немедленно лишает ценности ещё не воспроизведённые данные. Если в очереди проигрывателя уже находятся три аудиоблока длительностью по 20 ms, то даже после прекращения генерации на удалённой стороне локальное воспроизведение может продолжаться ещё 60 ms. Если управляющему сообщению требуется 100 ms, чтобы достичь сервера, удалённая сторона успеет создать новые блоки до его получения. Поэтому процесс отмены разделяется на два параллельных пути (рисунок 12-25): локальная сторона очищает буфер ожидающих воспроизведения данных, а удалённая прекращает ге-

³⁴Новая золотая эпоха компьютерных сетей, часть 3, где описывается исторический контекст TACK, Link Turbo и размещения прокси.

³⁵Расчёт смешанной обратной связи при приоритизации мультимедиа и агрегированном АСК, а также контрольные варианты FIFO и немедленного АСК в том же каталоге. Время обработки моделью и запланированные моменты воспроизведения заданы условиями примера.

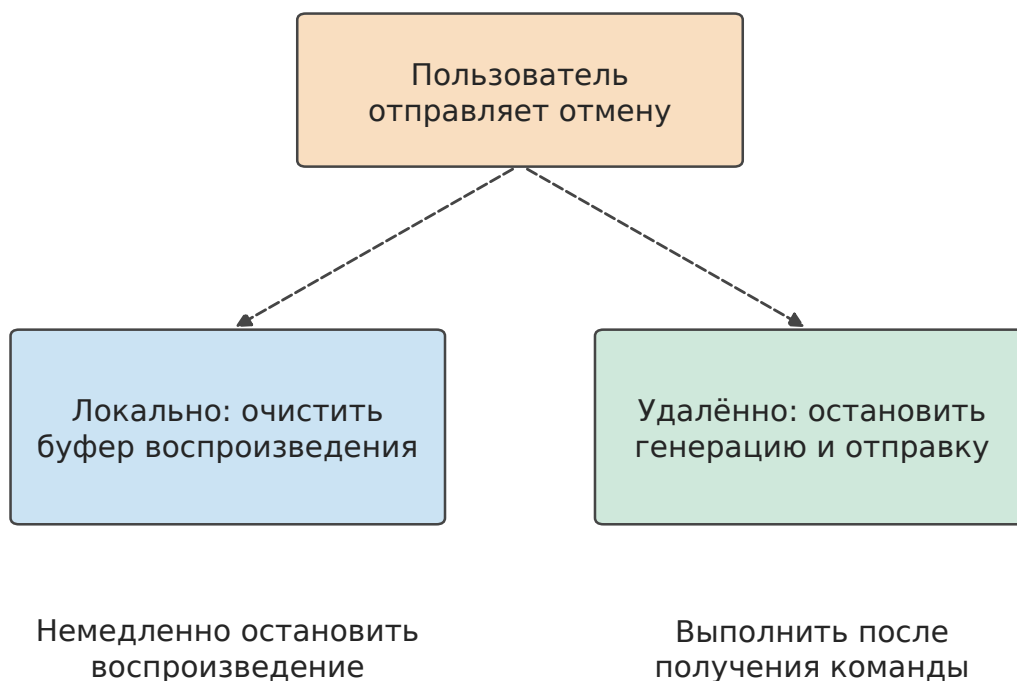


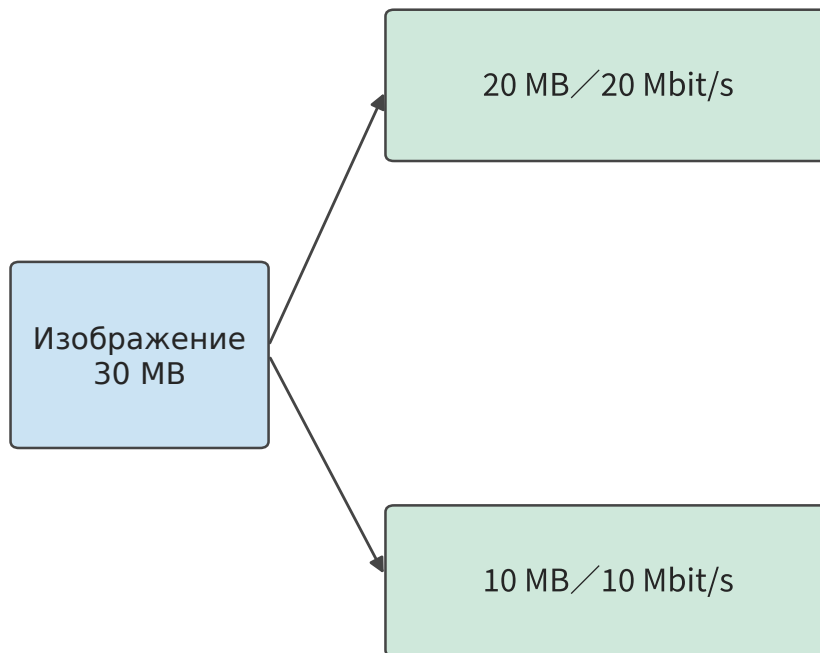
Рисунок 12-25. Одна отмена одновременно запускает два пути: локальная сторона очищает ожидающее воспроизведения аудио, а удалённая прекращает генерацию и отправку после получения управляющего сообщения. Пунктиром обозначен поток управления; локальная остановка воспроизведения и освобождение ресурсов на удалённой стороне завершаются в разные моменты.

нерацию и отправку последующего аудио. Первый путь определяет, когда пользователь перестанет слышать звук, второй — когда прекратится потребление ресурсов.

После отмены аудио отбрасываются только ещё не воспроизведённые данные, тогда как действие агента, работающего со скриншотами, изменяет среду после отправки. При потере ответа повторная попытка должна запрашивать результат, соответствующий идентификатору исходной операции; повторный щелчок превращает повторную попытку обмена данными во вторую бизнес-операцию. Если сохранить результат операции, после восстановления сети можно продолжить работу с достигнутого состояния, не создавая повторных операций.

12.4.3 Выбор двух путей, распределение трафика и переключение при отказе

Предыдущий анализ учитывал только один путь доступа, однако терминал часто одновременно подключён к двум. Wi-Fi и сотовая сеть предоставляют два пути доступа. При распределении разные части данных передаются по разным путям, чтобы одновременно использовать их ресурсы; при репликации одна и та же копия данных передаётся по обоим путям, чтобы раньше получить один действительный результат. Эти методы различаются объёмом данных и условиями завершения, поэтому их следует рассчитывать отдельно.



Два независимых пути: каждый отправляет свою часть за 8 с

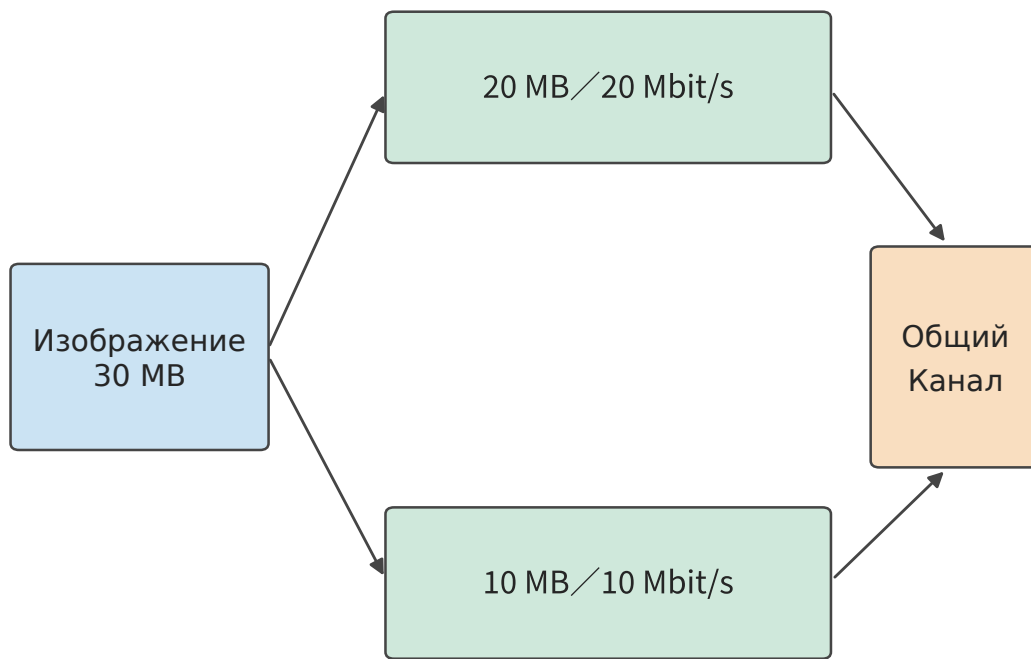
Рисунок 12-26. Два независимых пути получают входные данные пропорционально пропускной способности, передают соответственно 20 МВ и 10 МВ, и каждому требуется 8 с. Задача завершается после получения обеих частей.

Пример: как распределить данные изображения между двумя путями, чтобы сократить время передачи? Два независимых пути с постоянной скоростью имеют пропускную способность 20 и 10 Мбит/с соответственно. По быстрому пути передаётся доля x , по медленному — оставшаяся часть. Передача завершается только после получения обеих частей, поэтому

$$T_{\text{split}} = \max \left(\frac{xS}{B_1}, \frac{(1-x)S}{B_2} \right).$$

Если один путь завершает передачу раньше, часть байтов с другого пути можно перенести на него, чтобы ускорить завершение той части, которая изначально была бы получена позже. Поэтому при оптимальном соотношении оба пути должны завершить передачу одновременно, то есть $x = B_1 / (B_1 + B_2)$. Быстрый путь передаёт 20 МВ, медленный — 10 МВ; каждому требуется 8 секунд, что на 4 секунды меньше 12 секунд при использовании только быстрого пути (рисунок 12-26).

Если оба пути сходятся в одном выходном канале с пропускной способностью 24 Мбит/с, оба пути доступа по-прежнему могут передавать одновременно и каждому требуется 8 секунд, но все данные должны пройти через этот выходной канал, поэтому передача 30 МВ займёт не менее 10 секунд. Таким образом, узкое место перемещается в общий выходной канал. Дальнейшее повышение скорости доступа лишь приведёт к



Общий канал 24 Mbit/s → минимум 10 s

Рисунок 12-27. Изображение объёмом 30 MB разделяется на части 20/10 MB между двумя независимыми путями доступа со скоростями 20/10 Mbit/s; каждому требуется 8 секунд. Затем оба пути проходят через общий выходной канал с пропускной способностью 24 Mbit/s, поэтому прохождение всех данных через него занимает не менее 10 секунд. Стрелки обозначают пути данных, а не расстояние распространения; пропускная способность считается постоянной.

накоплению дополнительных данных в очереди перед этим выходным каналом.

На рисунке 12-27 два пути снова сходятся в выходном канале. Эта точка слияния не только объясняет, почему скорости двух путей доступа нельзя просто сложить, но и поднимает другой вопрос: сможет ли второй путь доступа продолжить передачу при отказе устройства, на котором расположен выходной канал.

Чтобы получать те же данные даже после отключения одного пути, можно использовать репликацию и проверить, в каких точках обе копии могут отказать одновременно. Пусть вероятность отказа общего конечного узла равна q ; когда конечный узел исправен, отказы двух путей доступа независимы и имеют вероятности p_1, p_2 . Отказ включает либо отказ конечного узла, либо одновременный отказ обоих путей доступа при исправном конечном узле. Эти два взаимоисключающих события суммируются:

$$p_{\text{fail}} = q + (1 - q)p_1p_2.$$

При $p_1 = 0.1, p_2 = 0.05$ вероятность одновременного отказа двух путей доступа составляет 0,5%; после добавления $q = 0.02$ общая вероятность равна примерно 2,5%. Из них 2 процентных пункта напрямую обусловлены общим конечным узлом. Даже если сделать оба пути доступа абсолютно надёжными, общая вероятность отказа всё равно

составит 2%; уменьшить эту составляющую можно лишь путём размещения реплик сервиса в разных доменах отказа.

После устранения общего отказа за репликацию всё равно приходится платить дополнительной пропускной способностью. Например, восемь аудиоблоков размером 640 bytes составляют 5120 bytes в одной копии, а при полной репликации — 10 240 bytes. Получатель устраняет дубликаты по номерам блоков данных: первая полностью прибывшая копия передаётся проигрывателю, а более поздняя отбрасывается. Отправка дополнительной копии данных позволяет использовать результат, первым прибывший по одному из двух путей; если оба пути используют общий выходной канал, дублирующиеся данные также занимают пропускную способность, необходимую другим задачам.³⁶

В реальных системах распределение, репликация и точка слияния часто реализуются протоколом или прокси-сервером доступа. Многопутевой TCP (MPTCP) организует в одном логическом соединении TCP-подпоток, проходящие по разным путям, а прокси-сервер доступа может объединять эти пути между терминалом и обычным сервером. В реализации Huawei Link Turbo для совместного использования нескольких сетей состояние разных путей доступа также учитывается при выборе способа отправки.³⁷ Расположение прокси-сервера одновременно влияет на общую пропускную способность, пространство данных и домен отказа, поэтому он становится отдельным вычислительным и транспортным узлом модели развёртывания.

12.5 От распределения задач между устройством, периферией и облаком к полноценному развёртыванию

12.5.1 Регион, данные и расположение сервиса

Продолжим анализ задачи с 30 раундами обработки скриншотов из раздела 12.1. Первые десять раундов уже завершены, и для оставшихся двадцати нужно выбрать место выполнения; с этого момента задача должна быть завершена за 45 секунд. Независимо от выбранного варианта ранее уже затраченное время одинаково, поэтому сравнивается оставшаяся работа. Исходному сервису требуется 3,5 секунды на раунд, и продолжение выполнения заняло бы 70 секунд, поэтому необходимо выбрать другой вариант.

По-прежнему будем считать, что размер скриншота в каждом раунде равен 0,8 MB, работа терминала занимает 0,3 секунды, скорость исходящего канала по прежнему облачному маршруту составляет 6,4 Mbit/s, а RTT — 0,2 секунды. Сравним три места выполнения: на устройстве используется телефон из раздела 12.1.5, на близлежащей рабочей станции установлена RTX PRO 6000 Blackwell Workstation Edition, а в облачном регионе используется H100 SXM. Во всех трёх местах используется один и тот же набор весов Qwen3-8B q4_0 с контекстом 8K, и в каждом раунде генерируются 45 токенов, как в разделе 12.1.5, поэтому качество задачи одинаково. Время работы модели на раунд рассчитывается по методике раздела 12.1.5 с использованием нижней границы времени чтения при декодировании: выполняется 45 шагов, на каждом из которых считывается 5,47 GB, а суммарный объём чтения делится на пропускную способность памяти устрой-

³⁶Эксперимент с двумя процессами и двумя TCP-соединениями: два пути передачи имитируются двумя процессами и двумя TCP-соединениями; 30 основных попыток включают случаи отказа общего конечного узла.

³⁷Новая золотая эпоха компьютерных сетей, часть 3, где описывается исторический контекст TACK, Link Turbo и размещения прокси.

ства. Когда новый сервис принимает задачу, он сначала должен выполнить один prefill для контекста 8K, оставшегося после первых десяти раундов; в таблице это указано как «однократная подготовка» и рассчитывается делением объёма матричных вычислений 133,6 TFLOPs на пиковую производительность плотных вычислений BF16. Для телефона нет значения пиковой матричной производительности, на которое можно сослаться, поэтому время подготовки принимается равным 0; такое допущение лишь делает выполнение на устройстве более быстрым. Сначала предполагается отсутствие очередей и сбоев, а время отправки управляющего ответа принимается равным нулю.³⁸

³⁸Расчёты для трёх уровней: конечного, периферийного и облачного. Объём чтения на каждом шаге для весов и KV взят из строки результатов 8K decode для одного запроса; веса переупаковываются в q4_0 из расчёта 18 bytes на каждые 32 значения; параметры шины телефона взяты из разделения бюджета шины и энергопотребления конечного устройства; пропускная способность, пиковая производительность плотных вычислений BF16 и номинальная мощность — из таблицы параметров оборудования; измеренный коэффициент — из строки batch 1, 8K в таблице эффективности эксперимента 8-1. Файл результатов также содержит все варианты для десяти раундов, сокращённого срока и восстановления после разрыва соединения.

Место выполнения	Устройство	Однократная подготовка	Модель на раунд	Исходящий канал	Распространение туда и обратно на раунд	Место назначения скриншота	Энергопотребление за 20 раундов
На устройстве	Телефон, 84,8 GB/s	0 s	2,90 s	Загрузка не требуется	0 s	Остаётся на устройстве	518–680 J
Близлежащая рабочая станция	RTX PRO 6000, 1 792 GB/s, 600 W	0,27 s	0,137 s	80 Mbit/s	0,02 s	Передаётся на близлежащую рабочую станцию	Около 1,81 kJ
Облачный регион	H100 SXM, 3 350 GB/s, 700 W	0,14 s	0,073 s	6,4 Mbit/s	0,20 s	Передаётся в облачный регион	Около 1,12 kJ

Стоимость измеряется энергопотреблением. Для обеих GPU номинальная мощность платы умножается на время занятости, которое равно сумме однократной подготовки и вычислений модели за двадцать раундов; поскольку номинальная мощность является верхней границей, значения в этих двух строках также представляют собой верхние границы. Для телефона используется измеренное в MELTing Point энергопотребление 0,576–0,756 J на токен; за двадцать раундов генерируется 900 токенов.³⁹ Энергия сети, затрачиваемая на передачу и распространение, не учитывается.

До сравнения времени и стоимости необходимо выполнить ограничение на резидентность данных (data residency): данные могут храниться и обрабатываться только на разрешённом устройстве или в разрешённом регионе. В строке выполнения на устройстве исходные скриншоты всегда остаются на нём; при использовании близлежащей рабочей станции или облачного региона исходный скриншот размером 0,8 MB в каждом раунде передаётся за пределы устройства. Если скриншоты содержат данные, которым запрещено покидать устройство или пересекать границу страны, эти два варианта исключаются ещё до начала сравнения; дальнейшее сравнение сроков и стоимости предполагает, что передача разрешена.

После разделения облака на разные регионы по-прежнему необходимо сравнивать эти значения времени и стоимости. При использовании сервиса Regionless, в котором облачный регион для запроса выбирает платформа, а приложение не привязано к фиксированному региону, приложение задаёт задачу, требования к качеству и срок завершения, а платформа выбирает регион развёртывания.⁴⁰ Платформа всё равно должна сравнивать фактические издержки вариантов: расстояние до сервера влияет на время распространения, кэш — на объём повторных вычислений, а межрегиональная передача — на стоимость передачи данных.

12.5.2 Совместное сравнение срока, стоимости и восстановления

Пример: как перенос состояния и число оставшихся раундов взаимодействия влияют на выбор между устройством, периферией и облаком? Время выполнения оставшихся двадцати раундов равно

$$\begin{aligned} T_{\text{local}} &= 20(0.3 + 2.90) \approx 64.0 \text{ s}, \\ T_{\text{near}} &= 0.27 + 20(0.3 + 0.137 + 0.02 + 6.4/80) \approx 11.0 \text{ s}, \\ T_{\text{cloud}} &= 0.14 + 20(0.3 + 0.073 + 0.20 + 6.4/6.4) \approx 31.6 \text{ s}. \end{aligned}$$

Пропускная способность памяти H100 в 1,87 раза выше, чем у RTX PRO 6000, поэтому вычисления модели в облаке занимают примерно на 0,064 секунды меньше на раунд, что экономит около 1,3 секунды за двадцать раундов; на подготовке дополнительно экономится около 0,13 секунды. Однако передача в каждом раунде занимает на 0,92 секунды больше, а распространение — на 0,18 секунды больше, поэтому за двадцать раундов ожидание увеличивается на 22 секунды. В результате облако оказывается примерно на 20,6

³⁹MELTing Point: оценка языковых трансформеров на мобильных устройствах, arXiv 2403.12844, §5.2.2 и таблица 5 приложения: 0.16, 0.20 и 0.21 mWh на токен; максимальная продолжительная мощность iPhone 14 Pro составляет 13.8 W, мгновенная превышает 18 W, а продолжительная мощность Galaxy S23 остаётся ниже 8.5 W; на iPhone 14 Pro Zephyr-3B q4_k достигает 14.8 token/s, а Llama-2 7B q3_k — 6.0 token/s; на одном заряде можно выполнить 490.05–590.93 промпта, для S23 — 542.78. Построчная сверка извлечённых значений приведена в таблице извлечения числовых значений.

⁴⁰Обсуждение межоблачных сетей и Regionless приведено в «Новой золотой эпохе компьютерных сетей, часть 2». В вариантах регионального размещения этой главы стоимость измеряется энергопотреблением устройства и временем GPU.

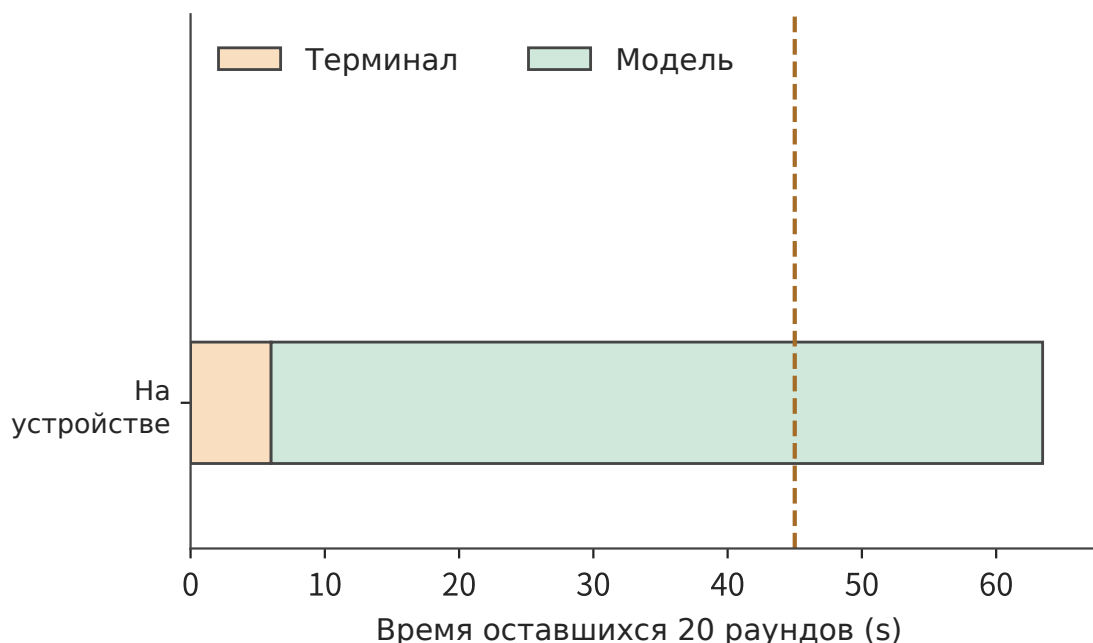


Рисунок 12-28. На устройстве: работа терминала — 6 s, вычисления модели — 58,0 s, всего — 64,0 s. На всех трёх рисунках используется одна линия срока 45 s и одна горизонтальная ось; цвета обозначают суммарное время однотипной работы за двадцать раундов.

секунды медленнее. Разложив разницу во времени на составляющие, можно увидеть, как более быстрая модель теряет преимущество из-за коммуникации в каждом раунде: декодирование одного раунда занимает лишь от нескольких десятков до немногим более ста миллисекунд, что значительно меньше времени коммуникации на раунд.

На рисунках 12-28–12-30 сведена продолжительность всех этапов полной задачи для трёх способов развёртывания. В облаке зелёный участок вычислений короче, но синий участок передачи длиннее; эти участки последовательно складываются с остальной работой и определяют время завершения задачи, показанное концом горизонтальной полосы.

Срок 45 секунд прежде всего исключает выполнение на устройстве: у него наименьшее энергопотребление, но требуется 64,0 секунды. И близлежащая рабочая станция, и облако завершают задачу вовремя, поэтому далее сравнивается энергопотребление: около 1,12 kJ в облаке против примерно 1,81 kJ на близлежащей рабочей станции. Номинальная мощность H100 выше, чем у RTX PRO 6000, но время её занятости на раунд лишь немного превышает половину времени второй карты, поэтому энергопотребление оказывается ниже. Следовательно, среди вариантов, укладывающихся в срок, наименьшее энергопотребление обеспечивает облако.

Для строки выполнения на устройстве также необходимо проверить расход аккумулятора по методике раздела 12.1.5. Следует пересчитать, какую долю одного заряда составляют приведённые в таблице 518–680 J за двадцать раундов. На странице характеристик iPhone 16 Pro указано лишь до 27 часов воспроизведения видео, но не приведена ёмкость аккумулятора в ватт-часах, поэтому используем пересчёт по измерениям MELTing Point: одного заряда хватает примерно на 490–590 промптов, а 20 раундов соот-

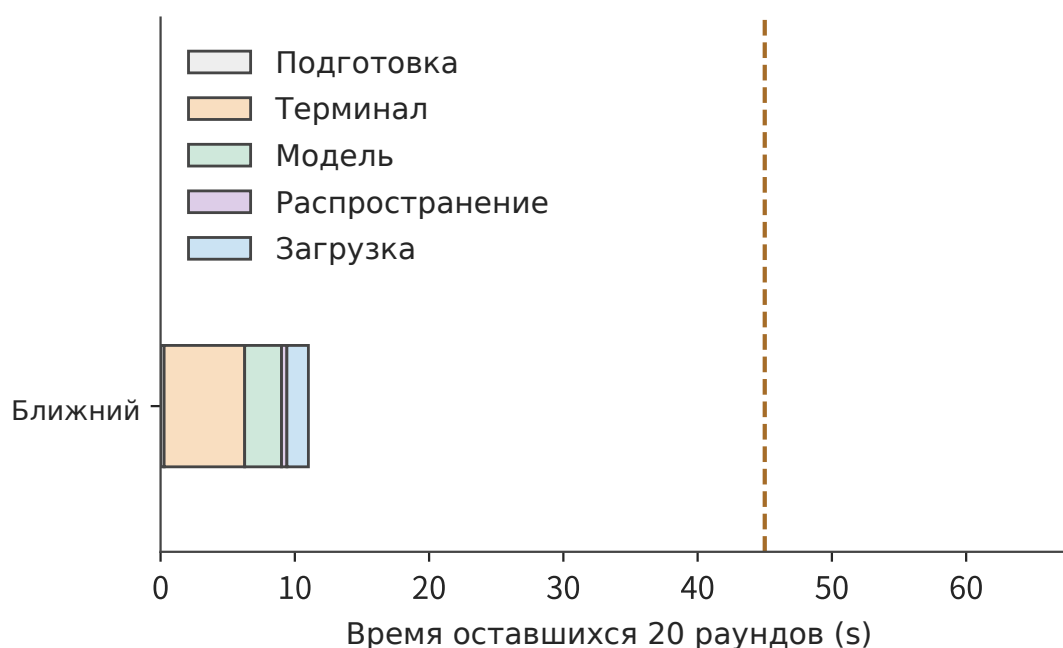


Рисунок 12-29. Близлежащая рабочая станция (RTX PRO 6000): подготовка — 0,27 s, за двадцать раундов терминал — 6 s, модель — 2,7 s, распространение — 0,4 s, передача — 1,6 s, всего — 11,0 s; срок 45 s соблюден. Пунктирная линия обозначает срок завершения 45 s.

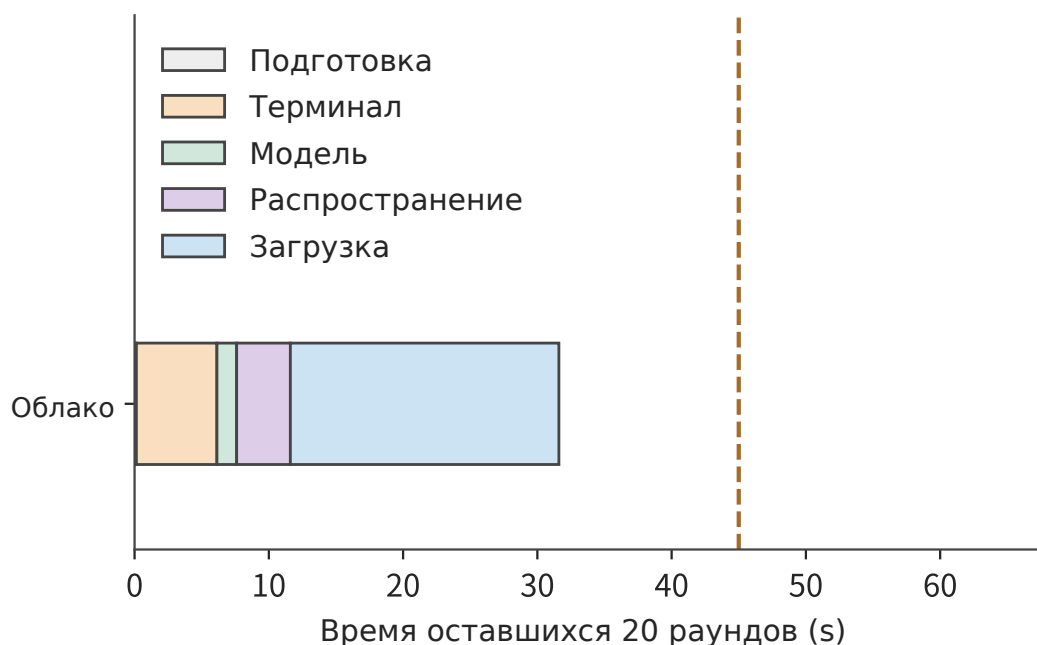


Рисунок 12-30. Облако (H100 SXM): подготовка — 0,14 s, за двадцать раундов терминал — 6 s, модель — 1,5 s, распространение — 4 s, передача — 20 s, всего — 31,6 s. Вычисления выполняются быстрее, но передача занимает больше времени, поэтому этот вариант примерно на 20,6 s медленнее близлежащей рабочей станции, хотя всё ещё укладывается в срок. Пунктирная линия обозначает срок завершения 45 s.

ветствуют 20 промптам, то есть приблизительно 3,4–4,1% одного заряда.⁴¹ Аккумулятор не исключает вариант выполнения на устройстве; его исключает время завершения 64,0 секунды.

Зависимость облачного варианта от скорости исходящего канала можно выразить количественно. Если изменить только скорость исходящего канала облачного маршрута b в Mbit/s, оставив остальные параметры неизменными, время завершения в облаке составит

$$T_{\text{cloud}}(b) = 11.6 + \frac{128}{b} \text{ s.}$$

11,6 секунды приходятся на однократную подготовку и работу без учёта передачи за двадцать раундов, а 128 Mbit — общий объём передачи двадцати скриншотов. Для соблюдения срока требуется $b \geq 128/33.4 \approx 3.8$ Mbit/s (рисунок 12-31). Однако облако не может оказаться быстрее близлежащей рабочей станции: даже при мгновенной передаче ему требуется 11,6 секунды, что всё равно больше 11,0 секунды у близлежащей рабочей станции. Причина заключается в 0,2 секунды распространения туда и обратно на каждый раунд: это на 0,1 секунды больше суммы времени распространения и передачи на раунд у близлежащей рабочей станции (0,1 секунды), что превышает экономию модельного времени на H100, составляющую около 0,064 секунды на раунд.

Перед обратным вычислением пороговой пропускной способности необходимо сначала проверить фиксированное время. Если срок равен D , порог в этом случае составляет $b_{\min} = 128/(D - 11.6)$ Mbit/s и имеет смысл только при $D > 11.6$ секунды. При $D \leq 11.6$ секунды фиксированное время уже полностью исчерпывает срок даже при бесконечной пропускной способности исходящего канала, поэтому конечного порогового значения не существует; нельзя интерпретировать полученный по формуле ноль, отрицательное число или ошибку деления на ноль как допустимую пропускную способность.

Вывод о том, что облако «не догонит» рабочую станцию, зависит от использования нижней границы времени чтения. В эксперименте 8-1 на RTX PRO 6000 измеренное время одного шага декодирования при batch 1 и контексте 8K составило 25,83 ms, что примерно в 2,83 раза превышает нижнюю границу времени чтения 9,12 ms; основная часть разницы приходится на фиксированные накладные расходы каждого шага. Если увеличить время модели на обеих GPU в это число раз, близлежащей рабочей станции потребуется 16,04 секунды, а облаку без учёта передачи — 14,29 секунды; разница составит 1,75 секунды. Только при скорости исходящего канала выше $128/1.75 \approx 73$ Mbit/s облако станет быстрее.⁴²

⁴¹MELTing Point: оценка языковых трансформеров на мобильных устройствах, arXiv 2403.12844, §5.2.2 и таблица 5 приложения: 0.16, 0.20 и 0.21 mWh на токен; максимальная продолжительная мощность iPhone 14 Pro составляет 13.8 W, мгновенная превышает 18 W, а продолжительная мощность Galaxy S23 остаётся ниже 8.5 W; на iPhone 14 Pro Zephyr-3B q4_k достигает 14.8 token/s, а Llama-2 7B q3_k — 6.0 token/s; на одном заряде можно выполнить 490.05–590.93 промпта, для S23 — 542.78. Построчная сверка извлечённых значений приведена в таблице извлечения числовых значений.

⁴²Расчёты для трёх уровней: конечного, периферийного и облачного. Объём чтения на каждом шаге для весов и KV взят из строки результатов 8K decode для одного запроса; веса переупаковываются в q4_0 из расчёта 18 bytes на каждые 32 значения; параметры шины телефона взяты из разделения бюджета шины и энергопотребления конечного устройства; пропускная способность, пиковая производительность плотных вычислений BF16 и номинальная мощность — из таблицы параметров оборудования; измеренный коэффициент — из строки batch 1, 8K в таблице эффективности эксперимента 8-1. Файл результатов также содержит все варианты для десяти раундов, сокращённого срока и восстановления после разрыва соединения.

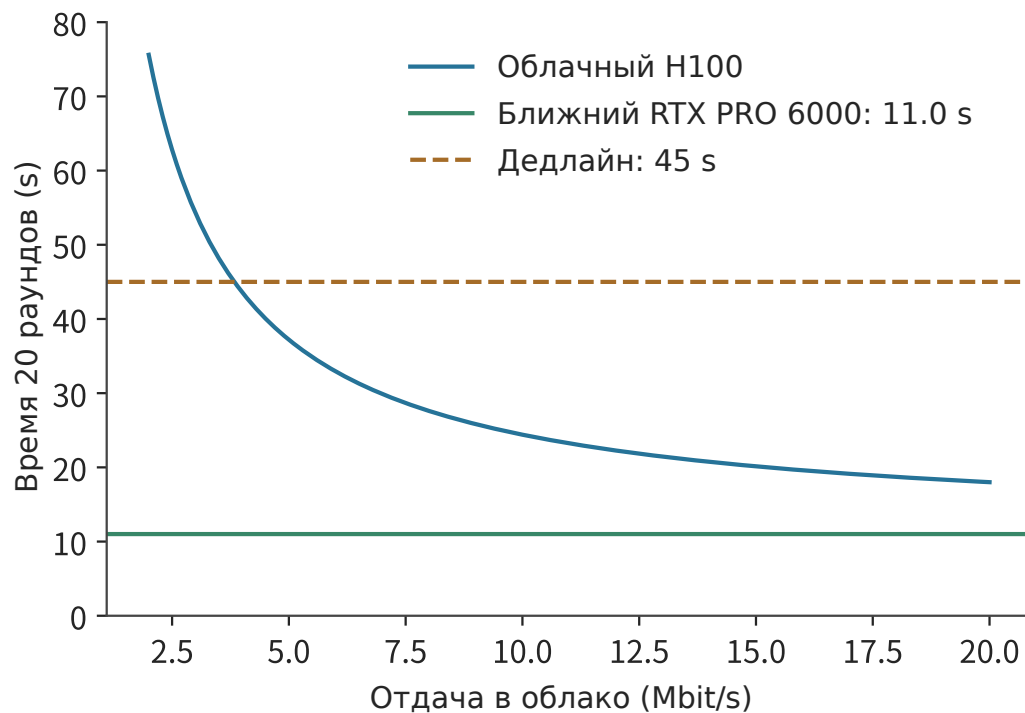


Рисунок 12-31. Исходящий канал облачного маршрута влияет на выбор варианта развёртывания для полной задачи. В каждом из оставшихся 20 раундов передаётся 0,8 МВ, подготовка в облаке занимает 0,14 секунды, а вся остальная работа каждого раунда — около 0,57 секунды, поэтому общее время равно $11,6 + 128/b$ секунды; близлежащей рабочей станции требуется 11,0 секунды, а срок составляет 45 секунд. В этом примере модель, время обработки и остальные сетевые параметры остаются неизменными, очереди и сбои не учитываются. Примерно с 3,8 Mbit/s облачный вариант начинает укладываться в срок; при дальнейшем росте скорости исходящего канала время облака всё равно не опускается ниже 11,6 секунды, поэтому оно всегда медленнее близлежащей рабочей станции.

При приближении к сроку необходимо также рассчитать, сколько времени остаётся на дополнительные накладные расходы. При исходной скорости исходящего канала 6,4 Mbit/s облачная задача завершается примерно за 13,4 секунды до истечения срока; при снижении скорости до 4 Mbit/s она занимает около 43,6 секунды, оставляя запас лишь примерно 1,4 секунды; при снижении до 3,5 Mbit/s она занимает около 48,2 секунды и уже не укладывается в срок. Если требуется, чтобы 99% задач завершались вовремя, необходимо анализировать распределение вероятностей времени завершения задачи. Поскольку скорость b_i может различаться между раундами, полное время с учётом передачи следует записать как

$$T = 11.6 + \sum_{i=1}^{20} \frac{6.4}{b_i}.$$

Предположим, что в течение передачи каждого раунда скорость постоянна. Средняя скорость для десяти раундов по 6 Mbit/s и десяти раундов по 10 Mbit/s также равна 8 Mbit/s, однако задача займёт около 28,7 секунды, что больше 27,6 секунды при постоянной скорости 8 Mbit/s. Время передачи обратно пропорционально скорости, поэтому дополнительное ожидание в медленных раундах превышает экономию в быстрых. Платформа должна оценивать выполнение условия $P(T \leq 45 \text{ s}) \geq 0.99$ по распределению времени передачи за всю задачу, а также учитывать, что при продолжительном замедлении сети несколько соседних раундов будут затронуты одновременно.

В приведённом выше выборе развёртывания основным фактором является время передачи скриншотов. Для текстовой задачи с меньшим объёмом входных данных и большим числом повторяющихся префиксов акцент оптимизации смещается на повторное использование вычислений. Сохранение состояния сеанса уменьшает объём повторных вычислений. Рассмотрим текстовый сеанс AI-агента: при повторной отправке префикса на каждом раунде общий объём составляет около 66 KB, а после сохранения состояния передаётся лишь около 10 KB; объём каждого ответа в обоих случаях равен примерно 2,5 KB. При скорости 333 Mbit/s сокращение передачи примерно на 56 KB экономит лишь 1,3 ms.⁴³ В отличие от передачи 16 MB скриншотов, основная выгода здесь возникает за счёт уменьшения вычислений префикса.

Объём матричных вычислений в том же сеансе снижается примерно с 3.0×10^{14} до 6.0×10^{13} FLOPs. Для облачной H100 SXM при производительности 989,4 TFLOP/s сохранение состояния экономит около 0,239 GPU-секунды. Сохранённое состояние объёмом около 465 MB постоянно занимает видеопамять; если пересчитать это пропорционально его доле в 80 GB видеопамати H100, каждая секунда хранения эквивалентна занятости $0.465/80 \approx 0.0058$ GPU-секунды. Разделив первое значение на второе, получим, что примерно через 41 секунду хранения состояния занятая видеопамать компенсирует сэкономленные в этот раз вычисления. Этот способ расчёта точки безубыточности совпадает со сравнением времени переноса из раздела 12.2.3: выгода от повторного использования должна покрывать стоимость хранения или переноса состояния.

⁴³При воспроизведении одной и той же модели, последовательности токенов и раундов объёмы данных, переданных серверу, составляют соответственно 66,207 и 10,193 bytes, а объёмы данных, возвращённых клиенту, в обоих случаях равны 2,527 bytes. Объёмы матричных операций составляют соответственно 296,505,803,538,432 и 60,380,764,176,384 FLOPs; нижняя граница времени GPU получается делением на пиковую производительность плотных вычислений BF16 H100 SXM. Размер состояния составляет 465,371,136 bytes; время его резидентного хранения пересчитывается с учётом его доли в 80 GB видеопамати, и примерно через 41.0 s оно компенсирует сэкономленные 0.239 секунды GPU. Источник: региональное размещение для фиксированной траектории агента; байты и токены взяты из исходной траектории.

В облаке выполнено 10 раундов по 1.57 s; восстановление связи — 1 s



Сохранить первые 9 раундов, повторить только 10-й

$$\text{Дополнительно } 1 + 1.57 = 2.57 \text{ s}$$

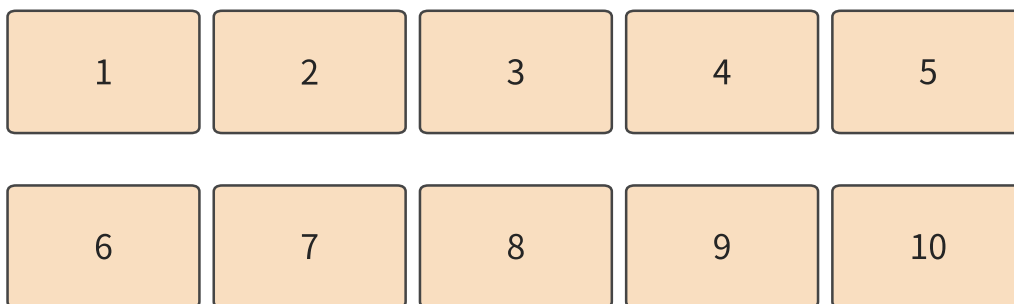
Рисунок 12-32. Записи о фиксации первых девяти раундов сохраняются (зелёный), поэтому повторяется только неподтверждённый десятый раунд (оранжевый). Один раунд в облаке занимает 1,57 s, восстановление соединения — 1 s, что добавляет 2,57 s и увеличивает общее время до 34,2 s. В этом примере используются операции, которые можно безопасно воспроизводить повторно; для уже зафиксированных внешних операций сначала запрашиваются их результаты.

Сохранение состояния уменьшает как повторные вычисления при нормальном выполнении, так и объём повторной работы после сбоя. На примере выбранного облачного варианта сравним стоимость одного и того же разрыва соединения при двух способах сохранения прогресса. Один раунд облачной задачи занимает около 1,57 секунды. Предположим, что десятый раунд уже выполнен, но соединение прерывается до возврата подтверждения, а его восстановление занимает 1 секунду. Если прогресс первых девяти раундов уже зафиксирован, потребуется повторить только неподтверждённый десятый раунд, что добавит около 2,57 секунды и увеличит время задачи с 31,6 до 34,2 секунды. Если нужно повторить все десять раундов, дополнительные затраты составят около 16,7 секунды, и задача займёт 48,3 секунды, превысив срок. Хотя по-прежнему используется тот же сервер, возможность сохранить прогресс определяет, будет ли задача завершена вовремя. Один раунд на близлежащей рабочей станции занимает лишь около 0,54 секунды, поэтому даже при потере прогресса десяти раундов задача займёт только 17,4 секунды; вариант с большим запасом по сроку устойчивее и к сбоям.

На рисунках 12-32 и 12-33 показано различие между двумя способами восстановления: после одного и того же разрыва соединения в одном случае повторяется только последний раунд, а в другом — предыдущие десять раундов. Объём затрат на восстановление прежде всего зависит от того, какие результаты уже надёжно сохранены.

Выше сравнивался один уже произошедший разрыв соединения. Чтобы оценить среднюю стоимость большого числа задач, необходимо также учесть вероятность сбоя:

В облаке выполнено 10 раундов по 1.57 s; восстановление связи — 1 s



Прогресс потерян, повторить 10 раундов

Дополнительно $1 + 10 \times 1.57 \approx 16.7$ s

Рисунок 12-33. При потере прогресса десяти раундов после восстановления соединения повторяются десять раундов, что добавляет $1 + 10 \times 1.57 \approx 16.7$ s и увеличивает общее время задачи с 31,6 до 48,3 s, превышая срок 45 s.

повторные попытки тоже увеличивают стоимость. Если попытки независимы, вероятность успеха каждой равна p , после неудачи вся задача выполняется повторно, а каждая попытка стоит C , то ожидаемая стоимость удовлетворяет равенству $E = C + (1 - p)E$, поэтому $E = C/p$. Вероятности успеха 99% соответствует стоимость около $1.01C$, а 80% — $1.25C$. Сохранение прогресса позволяет заменить полный повтор задачи локальным повторным выполнением, непосредственно уменьшая дополнительное время и стоимость каждого сбоя.

12.5.3 Когда необходимо сменить вариант развёртывания

С учётом сравнения времени, энергопотребления и восстановления в разделе 12.5.2 в этом примере оставшиеся двадцать раундов следует перенести в облако: задача завершится за 31,6 секунды при энергопотреблении около 1,12 кДж. По сравнению с 70 секундами при продолжении использования исходного сервиса экономия составит около 38,4 секунды. Терминал продолжит сбор данных и выполнение операций, облачная H100 будет отвечать за решения модели, а в каждом раунде будут передаваться только скриншот и результат управления.

Это решение получено последовательным сравнением трёх вариантов развёртывания: выполнение на устройстве сначала исключается из-за превышения срока, а близлежащая рабочая станция и облако укладываются в срок, но облако потребляет меньше энергии. Анализ сжатия, признаков и состояния из разделов 12.1–12.2 также предлагает способы дальнейшего улучшения: сжатие скриншотов уменьшает время передачи, передача полных признаков увеличивает его, а сохранение уже зафиксированного прогресса сокращает повторную работу после восстановления.

Изменение условий	Результат	Решение
Скорость исходящего канала исходного облачного маршрута равна 6,4 Mbit/s	Облако: 31,6 s и около 1,12 кДж; близлежащая рабочая станция: 11,0 s и около 1,81 кДж	Оба варианта укладываются в срок; используется облако с меньшим энергопотреблением
Скорость исходящего канала облачного маршрута снижается до 3,5 Mbit/s	Облако: около 48,2 s	Облако не укладывается в срок; перейти на близлежащую рабочую станцию
Скорость исходящего канала облачного маршрута сколько угодно высока	Облако: не менее 11,6 s	Оно всегда медленнее близлежащей рабочей станции; более строгий срок может соблюсти только близлежащая рабочая станция
В облаке происходит один разрыв соединения, прогресс первых девяти раундов сохраняется, а десятый раунд выполняется повторно	34,2 s	По-прежнему укладывается в срок
При том же разрыве соединения теряется прогресс десяти раундов	48,3 s	Срок превышен; необходимо улучшить механизм сохранения прогресса или перейти на близлежащую рабочую станцию

Смена устройства во время выполнения задачи также требует переноса состояния сеанса. Для сеанса объёмом 64 MiB из раздела 12.2 требуется около 7,7 секунды подготовки,

а каждый раунд ускоряется на 0,4 секунды, поэтому за 20 раундов чистая экономия составляет лишь около 0,3 секунды. Если улучшение пропускной способности продлится только десять раундов, а затем прежние условия восстановятся, такой перенос приведёт к чистой потере около 3,7 секунды. Поэтому платформа должна сопоставлять ожидаемую продолжительную выгоду со стоимостью переключения: переносить состояние следует только тогда, когда ожидаемая экономия времени покрывает подготовку к переносу и оставляет запас на дополнительные накладные расходы. Продолжение работы на одном устройстве в течение некоторого времени позволяет избежать многократных переносов состояния при колебаниях пропускной способности около порогового значения.

В главе 1 по ёмкости, объёму вычислений и времени чтения определялось, способен ли ускоритель разместить модель; в этой главе дополнительно анализируются коммуникация, порядок выполнения и восстановление, чтобы определить, сможет ли вариант развёртывания завершить задачу вовремя. При выборе варианта развёртывания необходимо последовательно ответить на вопросы: какая работа входит в задачу, какие шаги требуют ожидания, какие дополнительные накладные расходы возникнут при фактическом выполнении и какой вариант лучше подходит после изменения условий.

12.5.4 Адаптация системы после изменения ресурсных условий

В приведённом выше сравнении развёртываний модель оставалась неизменной, а менялось только место выполнения; если, наоборот, зафиксировать процесс выполнения задачи и ускорять только модель, выгода для полной задачи также будет ограничена. Предположим, что в фиксированной траектории суммарное время работы модели составляет 8 секунд, а неперекрываемое время инструментов, сети и прочих этапов — 2 секунды. Даже при десятикратном ускорении всей модели время задачи сократится лишь с 10 до 2,8 секунды, то есть ускорение составит около 3,57 раза;⁴⁴ при стремлении времени модели к нулю нижняя граница всё равно равна 2 секундам (рисунок 12-34).

Чтобы дополнительно сократить оставшиеся две секунды, можно изменить организацию системы. Перенос выполнения на локальное устройство уменьшает число сетевых обменов, а сохранение окружения сокращает ожидание его создания; дополнительное локальное оборудование и постоянно занятое пространство при этом включаются в новый бюджет стоимости. Пример расчёта развёртывания из раздела 12.5.2 уже показывает способ сравнения: сначала находятся варианты, удовлетворяющие требованиям к качеству и сроку, а затем сравнивается полная стоимость выполнения одной задачи.

Тот же подход применяется и при проектировании моделей: переход от DeepSeek V4 к V4.1 представляет собой аналогичную адаптацию на стороне модели. Межслойное совместное использование сокращает дублирование при сохранении глобального KV-кэша и позволяет модели сохранять историческую информацию с более мелкой гранулярностью; CED уменьшает повторное прохождение входных данных через декодер, а восстановление кэша сокращает путь повторного запуска многораундового сеанса.⁴⁵ Эти изменения воздействуют соответственно на ёмкость, вычисления входных данных и ожидание восстановления, поэтому их можно последовательно учитывать в бюджете одной и той же задачи.

⁴⁴Подробный пересчёт и программа вычислений для сравнения условий этой главы.

⁴⁵Официальный технический отчёт DeepSeek V4.1, разделы 1, 2, 3 и 6; фиксированные условия и повторный расчёт для межглавной сессии.

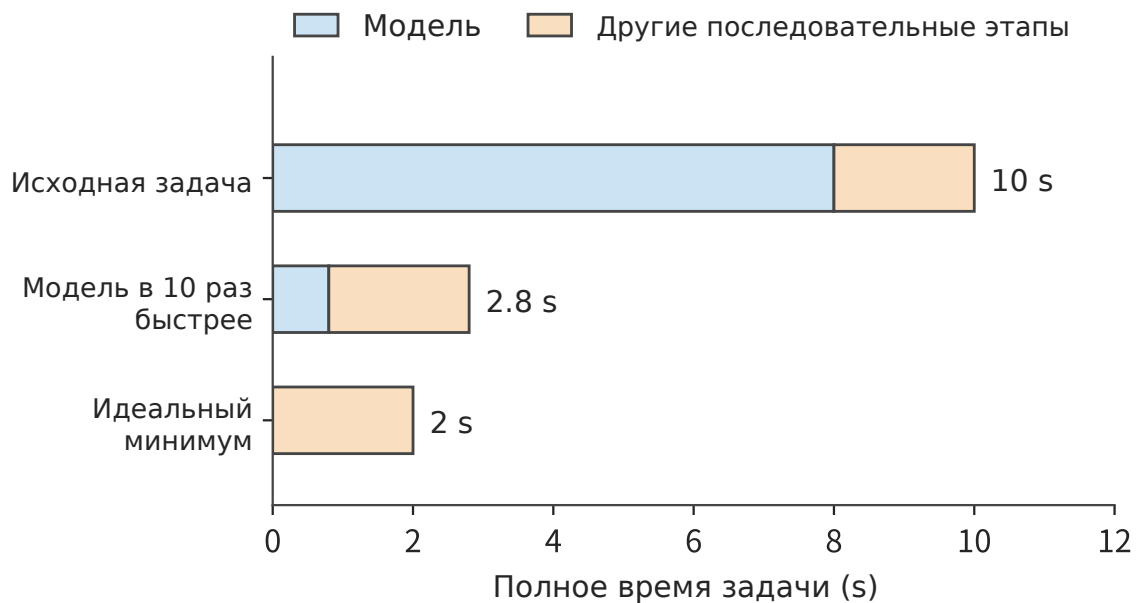


Рисунок 12-34. Предел ускорения в фиксированной траектории задачи. Время модели сокращается с 8 до 0,8 секунды, а остальные последовательные этапы по-прежнему занимают 2 секунды; третья строка показывает идеальную нижнюю границу при стремлении времени модели к нулю.

От внутрикристальной памяти до устройства, периферии и облака — проектные решения всегда зависят от понимания полного процесса выполнения. Отношения редукции во внимании позволяют обрабатывать промежуточные матрицы блоками, повторяющаяся структура модели — повторно использовать план выполнения, связи между префиксами и ветвями — совместно использовать контекст, а ожидание инструментов предоставляет время для выгрузки состояния и подготовки окружения. Каждое улучшение использует информацию о модели или задаче, чтобы направлять проектирование других уровней, заново определяя размещение данных, разделение вычислений и способы передачи управления.

12.6 Упражнения

Упражнения расположены по нарастающей сложности: вычисления, изменение условий и объяснение механизмов. 12-2, 12-3 и 12-7 — основные упражнения. Если не указано иное, числовые значения соответствуют условиям учебных примеров из основного текста.

12-1. Бюджет передачи и вычислений при ретушировании изображений (дополнительное, ★).

- Для скоростей исходящего соединения 10, 20 и 100 Mbit/s вычислите время выполнения для трёх вариантов: исходный вариант, десятикратное повышение скорости обработки изображения и уменьшение размера входных данных вдвое. Дополнительное время кодирования и декодирования при сжатии входных данных по-прежнему составляет 0.15 секунды.
- Найдите скорость исходящего соединения, при которой сэкономленное благодаря

сжатию время передачи в точности равно дополнительному времени кодирования и декодирования. Затем для исходного размера изображения и скорости исходящего соединения 20 Mbit/s рассмотрите следующий случай: из исходных 0.3 секунды обработки ускорить можно лишь 0.2 секунды. Чему будет равно общее время выполнения задачи после десятикратного ускорения этой части?

- (с) Разделите изображение на три независимо обрабатываемых блока и изобразите перекрытие загрузки, обработки и обратной передачи. Отметьте этап, который ограничивает дальнейшее сокращение времени выполнения задачи.

12-2. Может ли экономия времени передачи благодаря сжатию снимков экрана компенсировать дополнительные взаимодействия (основное, ★).

- (а) Снимок экрана в каждом раунде сжимается с 0.8 МВ до 0.2 МВ, а дополнительное время кодирования в каждом раунде составляет 30 ms. Повторно вычислите, сколько времени суммарно удастся сэкономить при выполнении задачи из 30 раундов со снимками экрана.
- (b) Будет ли задача выполняться быстрее, если после сжатия потребуется 38 раундов? Найдите максимально допустимое количество раундов, при котором вариант со сжатием всё ещё быстрее варианта с 30 раундами и исходными изображениями.
- (с) Увеличьте RTT каждого раунда на 100 ms и отдельно вычислите общее время для 30 раундов с исходными изображениями и 38 раундов со сжатыми изображениями. Объясните, почему одно и то же изменение RTT приводит к разному увеличению времени выполнения задачи.
- (d) Спроектируйте поля журнала для версии снимка экрана и номера операции. Объясните, как выявлять раунды исправления ошибок, вызванные сжатием, и бесполезные раунды, вызванные устаревшим состоянием.

12-3. Как размещение визуального кодирования и миграция состояния влияют на время взаимодействия (основное, ★).

- (а) Изображение занимает 0.8 МВ, а после кодирования получается тензор признаков BF16 формы [400, 10240]. Пропускная способность исходящего соединения составляет 6.4 Mbit/s. Для визуального кодирования требуется 1.31 TFLOPs матричных операций. Локальное кодирование выполняется на RTX 4090, удалённое — на H100 SXM; для обоих вычислите нижнюю границу времени кодирования по пиковой производительности плотных вычислений BF16 (165.2 и 989.4 TFLOP/s). Сравните общее время при локальном кодировании с последующей загрузкой признаков и при загрузке исходного изображения с последующим удалённым кодированием.
- (b) Замените канал на один порт ConnectX-7 со скоростью 400 Gbit/s (50 GB/s в каждом направлении). Используя непосредственную загрузку исходного изображения как базовый вариант, вычислите дополнительное время передачи визуальных признаков. Сравните его с разницей времени кодирования на двух картах из пункта (а) и определите, какой фактор теперь задаёт размещение кодировщика.
- (с) Для миграции необходимо передать состояние размером 64 MiB со скоростью 80 Mbit/s, а затем потратить 1 секунду на восстановление. После миграции время выполнения каждого раунда сокращается на 0.4 секунды. Найдите чистую экономию времени при оставшихся 10, 20 и 40 раундах. Если время восстановления дополнительно увеличится на 1 секунду, сколько раундов как минимум должно остаться, чтобы окупить затраты времени на миграцию?
- (d) Для сохранённых изображения, визуальных признаков и языкового KV отдельно

укажите, с какого этапа потребуется начать повторные вычисления. Объясните, как результаты уже отправленных операций должны включаться в процесс восстановления.

12-4. Повторное использование соединения и многопоточная передача (дополнительное, ★).

- (a) RTT составляет 200 ms, а задача включает 30 раундов. Каждое установление соединения требует двух дополнительных обменов туда и обратно. Вычислите разницу во времени между созданием нового соединения в каждом раунде и повторным использованием одного соединения на протяжении всей задачи.
- (b) Отдельно вычислите суммарное количество переданных байтов изображения, заголовков пакетов и АСК для вариантов с немедленным подтверждением и агрегированным АСК. Объясните, почему при меньшем фактическом количестве переданных байтов время выполнения запроса всё же может оказаться больше.
- (c) Сравните приоритетную отправку с доставкой по отдельным потокам на рис. 12-20. В каждом случае укажите, как изменяются периоды использования ресурсов и ожидание зависимостей.
- (d) Спроектируйте контрольный эксперимент, в котором изменяется только способ повторного использования соединения, а остальные условия остаются одинаковыми. Перечислите начальные и конечные события, а также промежуточные события, необходимые для объяснения изменения времени.

12-5. Частота АСК и эффективность использования эфирного времени (дополнительное, ★).

- (a) Один обмен кадром данных занимает 302 μ s, а один обмен кадром подтверждения — 134 μ s. Всего происходит 30,800 обменов кадрами данных, и изначально каждый из них подтверждается отдельно. Сколько эфирного времени высвободится после уменьшения количества АСК вдвое?
- (b) Изобразите временную шкалу, на которой отправитель приостанавливает отправку в ожидании агрегированного АСК. Объясните связь между высвобождением эфирного времени и более ранней доставкой.
- (c) Для воспроизведения PCM требуется скорость 256 kbit/s, фактическая скорость приёма составляет 130 kbit/s, а начальный буфер — 60 ms. Найдите момент опустошения буфера. Сохраняя скорость приёма и начальную длительность аудио 60 ms неизменными, увеличьте частоту дискретизации с 16 kHz до 24 kHz и найдите новый момент опустошения буфера.

12-6. Распределение и резервирование по двум путям (дополнительное, ★).

- (a) Данные объёмом 30 МВ распределяются между двумя путями со скоростями 20 и 10 Mbit/s. Выведите соотношение распределения, при котором оба пути завершат передачу одновременно, и найдите это время.
- (b) Добавьте общий выходной канал со скоростью 24 Mbit/s. Найдите новую нижнюю границу времени передачи и укажите, увеличение пропускной способности какого участка канала позволит снизить эту границу.
- (c) Вероятности отказа двух путей доступа равны 0.1 и 0.05, а вероятность отказа общего конечного узла — 0.02. В соответствии с принятым в основном тексте предположением о независимых отказах найдите вероятность того, что задача всё равно завершится неудачно после отправки полной копии по каждому из двух путей. Чему будет равна эта вероятность, если оба пути доступа станут полностью надёжными?

- (d) Создайте по одной дополнительной копии каждого из восьми аудиоблоков размером 640 bytes. Найдите дополнительное количество передаваемых байтов и объясните, как получатель должен предотвращать повторное воспроизведение.

12-7. Разделение подготовки соединения, окна и задержки передачи с помощью однофакторных экспериментов (основное, ★★).

- (a) Отдельно вычислите отношение медианного времени прямого пути к медианному времени пути Queqiao для двух экспериментов: с созданием нового соединения для каждого запроса и с сохранением соединения после оптимизации. Объясните, какое предсказание следует из гипотезы о том, что «основное различие обусловлено подготовкой соединения и ожиданием отправки».
- (b) Пусть объём данных составляет около 355 KB, скорость передачи — 333 Mbit/s, RTT — 200 ms, а время обработки — 30 ms. Найдите идеальное время выполнения. Сравните его с первоначально измеренным временем около одной секунды и оцените, сколько времени ожидания ещё требуется объяснить.
- (c) Спроектируйте однофакторный эксперимент с размером окна. Объясните, как следует чередовать запуски с разными конфигурациями окна и какой объём данных в пути, какие ACK и какую временную последовательность отправки необходимо наблюдать, чтобы подтвердить гипотезу о том, что окно ограничивает скорость передачи.
- (d) Используя сопутствующую аудиозапись, изобразите моменты времени от генерации одного блока до его воспроизведения. Объясните, как различить несвоевременную генерацию, задержку передачи и ожидание локального планировщика.

12-8. Как пропускная способность исходящего соединения и восстановление после сбоя изменяют время и энергопотребление при межрегиональном размещении (дополнительное, ★★).

- (a) Повторно вычислите время и энергопотребление для оставшихся двадцати раундов при трёх вариантах размещения. Затем найдите пропускную способность исходящего соединения, необходимую облачному варианту для завершения в срок, и объясните, почему при любом дальнейшем её увеличении облачный вариант всё равно не сможет догнать ближайшую рабочую станцию.
- (b) Измените условия на десять оставшихся раундов и срок 23 секунды. Выберите вариант, который укладывается в срок и потребляет меньше всего энергии. Затем сократите срок до 15 секунд и сделайте выбор заново.
- (c) В двадцати раундах взаимодействия скорость исходящего соединения для десяти раундов составляет 6 Mbit/s, а для остальных десяти — 10 Mbit/s. Найдите общее время облачного варианта, сравните его со случаем постоянной скорости 8 Mbit/s и объясните разницу.
- (d) Для задачи из десяти раундов со сроком 23 секунды предположите, что соединение разрывается сразу после выполнения восьмого раунда, а его восстановление занимает 2 секунды. Рассмотрите два варианта восстановления: прогресс первых семи раундов сохранён, поэтому требуется повторить только восьмой раунд; прогресс всех первых восьми раундов потерян, поэтому их требуется повторить полностью. Вычислите время завершения оставшейся работы и общее время всей задачи в обоих случаях.

В этом пункте предполагается только разрыв соединения с удалённым сервисом; вариант выполнения на конечном устройстве продолжает работать нормально. Во время разрыва удалённый сервис сохраняет рабочее состояние, которое продолжает использо-

ваться после восстановления соединения, поэтому повторная подготовка не требуется. Прогресс задачи восстанавливается отдельно в соответствии с двумя описанными вариантами. Для повторяемых раундов необходимо ещё раз учесть энергопотребление модели. Заново выберите вариант с минимальным энергопотреблением, укладывающийся в срок.

12-9. Бюджет пропускной способности, ёмкости и энергопотребления конечного устройства (дополнительное, ★).

- (a) Для четырёх каналов LPDDR5X x16 со скоростью 10.6 Gbit/s на контакт найдите суммарную пропускную способность. Затем для одного запроса Qwen3-8B BF16 с 8K decode, где на каждом шаге считывается 15.14 GB весов и 1.21 GB KV, найдите нижнюю границу времени шага и верхнюю границу token/s.
- (b) После квантования весов в q4_0, при котором каждые 32 значения уменьшаются с 64 bytes до 18 bytes, повторно вычислите нижнюю границу времени шага и верхнюю границу token/s. Для телефона с 12 GB памяти, из которых 4 GB зарезервированы, найдите оставшееся после вычета весов пространство, количество помещающихся запросов с контекстом 8K и максимальное количество токенов контекста одного запроса.
- (c) При энергопотреблении 0.576–0.756 J на токен и предельной продолжительной мощности 13.8 W найдите диапазон верхней границы token/s, обусловленной только ограничением мощности. Сравните его с ограничениями пропускной способности для BF16 и q4_0 и укажите, какое ограничение срабатывает первым.

12-10. Потери пакетов, повторная передача и избыточность кодирования (дополнительное, ★★).

- (a) При MSS 1,448 bytes и RTT 0.2 s найдите верхнюю границу Mathis для вероятностей потери пакета 14% и 3.6%, а также время, необходимое только для отправки 354,640 bytes при обеих верхних границах.
- (b) Каждый из 245 пакетов независимо теряется с вероятностью 14%. Найдите ожидаемое количество потерянных пакетов. В каждом раунде для идеальной выборочной повторной передачи используется один RTT, а повторно передаваемый пакет снова теряется с вероятностью p . Поэтому количество раундов отправки, необходимое каждому пакету, имеет геометрическое распределение с вероятностью успеха $1 - p$, а вероятность доставки всех 245 пакетов за n раундов равна $(1 - p^n)^{245}$. Последовательный бюджет составляет 238.5 ms, а каждый дополнительный раунд добавляет один RTT. Найдите математическое ожидание времени выполнения, а затем получите p99 по 99-му перцентилю распределения количества раундов.
- (c) Сколько восстановительных пакетов требуется для каждой из двух вероятностей потери при целевой вероятности успеха 99.9% и какова соответствующая доля избыточности? Найдите время выполнения с учётом избыточности и сравните его с соответствующим p99 повторной передачи.
- (d) Объясните, почему долю избыточности следует изменять в зависимости от измеренной вероятности потери пакетов, а не фиксировать на уровне 25.7%: рассмотрите как невозможность достичь целевой вероятности успеха, так и напрасный расход пропускной способности отправки.

12.7 Итоги главы

Полное взаимодействие следует анализировать на трёх уровнях. Сначала по объёму данных, объёму вычислений и расстоянию передачи определяют время, необходимое для каждого этапа. Затем с учётом зависимостей выполнения выясняют, какие затраты времени нужно складывать, а какие этапы могут перекрываться. Наконец, учитывают соединения, окна, конкуренцию и восстановление, чтобы объяснить фактическое время ожидания. Завершение обработки изображения за 12,8 секунды, последовательная обработка блоков аудио и накопление задержки по раундам агента показывают, как взаимодействуют эти три уровня. Для вычислений на конечном устройстве применяется тот же подход: пропускная способность памяти задаёт верхнюю границу количества токенов в секунду, объём памяти определяет допустимые размеры модели и контекста, а энергопотребление и ёмкость аккумулятора — продолжительность работы.

Распределение задач между моделями меняет форму и количество передаваемых между устройствами данных. Перенос вычислений на конечное устройство может привести к замене компактного изображения более объёмными признаками, а распределение модели между несколькими устройствами — к замене одной передачи послойной синхронизацией. При сравнении сначала исключают одинаковые для обоих вариантов затраты времени, а затем сопоставляют сэкономленное время вычислений с дополнительным временем связи — так можно определить источник выигрыша.

Повторное использование позволяет распределить однократные затраты на подготовку между последующими вызовами. Для миграции, предварительного прогрева соединения, компиляции графа и сохранения состояния можно, исходя из стоимости подготовки и экономии времени при каждом вызове, определить минимальное количество использований, при котором это становится выгодно. Кроме того, часть ожидаемой экономии времени или средств следует зарезервировать на издержки, вызванные колебаниями и восстановлением.

При проектировании экспериментов также можно использовать ту же временную модель. Зафиксируйте входные данные и границы измерения времени, регистрируйте ожидание на каждом этапе, а затем поочерёдно изменяйте настройки и проверяйте, можно ли объяснить изменение общего времени изменениями на конкретных этапах. Корректировка базового варианта Queqiao демонстрирует этот процесс: после того как оптимизация устранила большую часть дополнительного ожидания, для дальнейшего повышения производительности необходимо сократить время распространения сигнала. Потеря пакетов также не обязательно означает перегрузку: на маршруте со случайными потерями оценка пропускной способности по потерям пакетов занижает доступную скорость примерно на три порядка. Только непосредственное измерение пропускной способности с последующим подбором избыточности кодирования по измеренной доле потерь позволяет приблизить время завершения к бюджету последовательного выполнения.

При окончательном выборе сначала следует обеспечить качество и соблюдение сроков, а затем сравнить стоимость. Даже если средней скорости вычислений и передачи достаточно, задача всё равно может не уложиться в срок из-за медленных раундов или потери достигнутого прогресса. Поэтому вероятность своевременного завершения и объём повторной работы после восстановления являются составными частями модели развёртывания.

Вернёмся к применению моделей, представленному в главе 1: разработчик выража-

ет задачу через контекст, а система выполнения на его основе планирует вычисления, сохраняет состояние и передаёт данные. Теперь сохранение фрагмента истории, добавление ветви-кандидата, предварительная подготовка окружения инструментов и миграция сеанса имеют отслеживаемую стоимость в ресурсах. Понимание этой стоимости позволяет выявлять, какие общие накладные расходы выполнения можно устранить с помощью информации приложения, и выбирать реализацию с учётом качества решения задачи, времени завершения и общей стоимости. Именно в этом заключается навык системного проектирования, который даёт изучение AI-инфраструктуры.